DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

Convolutional Neural Networks with TensorFlow in Python

Convolutions, pooling, regularization, hyperparameter tuning and TensorBoard

Volume 1 of 2 · Sections 1–38

38	537	411
SECTIONS	FIGURES	PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Neural Networks 4
LECTURE 2 Computer Vision and Self-driving Cars 6
LECTURE 3 Image Transformation 8
LECTURE 4 Original Sharpened 13
LECTURE 5 Blur: Matrix Form 41
LECTURE 6 Edge Cases 47
LECTURE 7 The Problem 57
LECTURE 8 Train an NN to Recognize the Digits 67
LECTURE 9 Feature Map 97
LECTURE 10 Smears Out 137
LECTURE 11 Input Convolutional Layer 162
LECTURE 12 Name Formula Derivative Graph Range 177
LECTURE 13 Overfitting and Validation Set 187
LECTURE 14 Optimization Algorithms 198
LECTURE 15 Relevant Packages and Anaconda Python 210
LECTURE 16 Julia File 211
LECTURE 17 Python and Anaconda 225
LECTURE 18 Python Notebook 226
LECTURE 19 Cell 227
LECTURE 20 Install TensorFlow and PIP Install 239
LECTURE 21 Problem to Solve Methodology 243
LECTURE 22 The CNN Architecture 247
LECTURE 23 Data Set 285
LECTURE 24 Early Stopping and Layer 290
LECTURE 25 Validation Accuracy and Data Set 294
LECTURE 26 Date Time and TensorBoard 296
LECTURE 27 Confusion Matrix 302
LECTURE 28 Filter Size and Hyper Parameters 319

LECTURE 29 Data Set and MNIST Data 322
LECTURE 30 Inhibit the Capacity 326
LECTURE 31 L2 Regularization Weight Decay 341
LECTURE 32 Neurons 353
LECTURE 33 Data Augmentation 367
LECTURE 34 Products and Online Platform 377
LECTURE 35 Data Set and Shoes 383
LECTURE 36 Image Module 398
LECTURE 37 Confusion Matrix 402
LECTURE 38 Primary Classification 405

LECTURE 1

Neural Networks

Hi, my name is Nicola, and I will be taking you through this convolutional neural networks course. I am a natural sciences graduate from the University of Cambridge in the UK with a passion for mathematics, physics and programming. In fact, I enjoy the subject matter so much. I've taken part in multiple national and international competitions where I've won numerous awards. I also hold a silver medal from the International Physics Olympiad and while I have this background, I am not doing much physics nowadays. I am a firm believer that you don't need high school or university to become proficient in a particular area. All you need is passion and a little guidance.

That's why we devised this course for you to help you master convolutional neural networks. And I say we because I didn't work on this project alone. It's a collaborative effort with Eskin is Grant is a very capable software developer and computer scientist with a bachelor's degree in computer science and physics from the University of Edinburgh and a master's degree in computer science from the University of Oxford. This grant has been engaged in deep learning programming for more than five years, with a focus on recurrent neural networks. His help and input were invaluable to the creation of this course, and it was a complete pleasure to work together on this project.

So what are we going to learn in the lessons to come? We'll start by taking a look at kernels in the context of image processing as they are an essential tool for working with and understanding convolutional neural networks. We'll explore how to achieve different image transformations and help you grasp the role of a mathematical operation of convolution. In this process, this will be the basis for our next topic, Convolutional Layer's. Armed with all that knowledge, we will introduce the main subject of the course. Convolutional neural networks here will discuss intriguing concepts such as feature maps and pooling. What's more, we'll inspect how such a network transforms the dimensions of the tensors.

Then what follows is a short and optional neural networks revision. CNN's are simply a subtype of deep neural networks, so a general understanding of ANNs is required. That's why we'll explain the basics. Activation functions early stopping and optimizers. Once we've covered all that, you will have the minimum required knowledge to start putting all this theory to practice by building your first convolutional neural network. Working on the MNIST data set will help you grasp the general workflow of creating a CNN architecture and build one from scratch. You are going to train it to recognize handwritten digits, a very useful tool in the real world. At this point, you will get the hands on opportunity to tinker and change the network and see the results for yourself.

And we won't stop at creating the scenes. We will also spend a good amount of time exploring them through the proper visualizations, through TensorBoard, the go to visualization and logging tool when working with TensorFlow. This will make your journey and experimentation in the field more straightforward and definitely more memorable. Neural networks are notorious

for their difficult interpretation, so we will examine the confusion matrix as a tool to help you understand and interpret the results of your networks. Finally, we'll show you how to easily tune the hyper parameters of your networks. But there's more. We will also show you how to master three common techniques to improve the performance of your models.

In fact, you will have the opportunity to apply those techniques to the networks we create for the next practical section. You heard that right. The idea of this course is to give you the real CNN experience. We will have an enormous practical exercise so you can work on a real world project to do that. We've created our very own custom data set that comes from the fashion industry. It consists of more than 16000 images of trousers, jeans, shoes, glasses and sunglasses. And we will be using these for numerous practical examples and problems. We've devised a task to classify the different items with a corresponding label. Not only that, but we will also determine other characteristics such as the item, subtype and gender.

Given the nature of these, we will be able to try out different techniques to achieve our goal and compare how these approaches fare against each other. You'll get a taste of the real world challenges of solving such a task and gain experience with a real project that you can later add to your portfolio. Finally, to cap it all off, we end this course with a review of the timeline of convolutional neural networks professional research. Given my background, this section is a complete must. We will dive into the workings of some popular CNN architectures and all stars like AlexNet, GoogLeNet, as well as Resnet all make an appearance. By the end of this course, you will be completely equipped with all the tools you need to confidently work on CNN projects.

Sounds great, doesn't it? This will be an awesome adventure to get the most out of this course. Please don't skip any lectures as we will gradually build on your knowledge. Now, the lessons in this course contain plenty of downloadable resources that will help you reinforce what you have learnt. Course notes, exercise files, PDF materials, handy notebook files, everything is included and can be downloaded easily. I strongly encourage you to complete all the exercises. They are designed not only for practice but also as an additional source of information that deepens your understanding of convolutional neural networks. And please don't hesitate to post in Q&A if you experience some difficulties, have a question to ask or simply want to share something with the community.

Are you excited? Awesome. Let's dive straight in and begin this journey together.

LECTURE 2

Computer Vision and Self-driving Cars

Ever since the dawn of computer technology and electronics, many scientists and engineers have been experimenting with ways to make the computers see. This refers to the ability of a computer to process visual information in ways comparable to humans and is called, unsurprisingly, computer vision. Computer vision can have a huge impact on numerous areas because there are many tasks that require awareness of the surroundings or just the ability to understand what is illustrated in the photo to advance the field, researchers took inspiration from the source itself, the human visual system. They looked at how individual neurons operate and found some of them only search for specific patterns, such as circles.

Inspired by this, they devised a special type of neural network, the convolutional neural network. This reinvigorated computer vision and this field started rapidly expanding with the beginning of the 21st century. So in the following lectures will dive precisely into the world of convolutional neural networks. But what does this term mean? Well, convolutional neural networks or CNN, in short, are a subtype of deep neural networks that have at least one convolutional layer. The details of that will be discussed in the following sections. Now, all you need to know is that they are extensively used in the field of computer vision. Computer vision is an A.I. feel that explores how computers can gain a high level understanding from digital images or videos that can be as simple the task as classifying an image to be a dog or a cat.

But it can also explode in complexity, as is the case with self-driving cars, for example. Speaking of the recent innovations in self-driving cars made computer vision a really hot topic. After all, a crucial aspect of self-driving cars is for the machines to see their surroundings in real time with the help of cameras and make decisions based on that. This, however, is no easy task because a live video can contain an enormous amount of detail. Besides this input, video data needs to be filtered in real time. It wouldn't do you any good to know there is a fence ahead of you if you learn about it 10 seconds after you've crashed into it. Right. And this is where CNN has come into play.

They are specialized in inferring information from data that has some spatial structure in correlation since images are structured in a 2D plane. This is the perfect avenue for this type of neural networks other than self-driving cars. There are many more avenues in which convolutional neural networks are extensively employed. For instance, you may have noticed that when you upload an image to Facebook or similar social networks, the website automatically detects the faces of all people on the photo and sometimes even recognizes the person in question. This is all done through a machine learning algorithm with a CNN at the forefront. If you think that this is simply a gadget, you should know it's far from where the contributions of CNN's.

And they can also help detect illegal items at airport security's visualize radio signals and astronomy, provide faster passport control through face recognition, help understand and predict the climate and weather, and even localize wildfires and illegal fishing through satellite imaging. In short, seasons are essential for modern technologies and our continued advancement. Moreover,

they are finding new applications in various fields every day. This means that you can be the person to come up with the next breakthrough in this area. Once you complete this course, you will have a deeper understanding of their workings and given enough passion and hard work, I'm confident you'll have great success.

Without further ado, let's get down to business. Thanks for watching.

LECTURE 3

Image Transformation

I know you must be eager to dive straight into the advanced stuff, but before proceeding to the convolutional neural network SPETZ, there was one subject matter we have to discuss first, kernels. And that is precisely the topic of this section. So what are image kernels? Well, an image, kernel, has been a well-known tool in image processing since at least the 70s. It is the basic building block of filters such as the ones employed by Instagram and is also the linking bridge between simple neural networks and convolutional neural networks. However, since this is a very visual topic, it is best to explain it through visuals. Let's take a look at this image.

FIGURE 3.1

It is a simple photo of a house in Greyscale. Nothing special about it whatsoever. Now, here's a question. Can you think of some popular transformations we can apply to this image to change it up a bit? I assume that some of you thought of blurring. Indeed, that is a very prominent image transformation. So let's see how this image looks after blurring it.

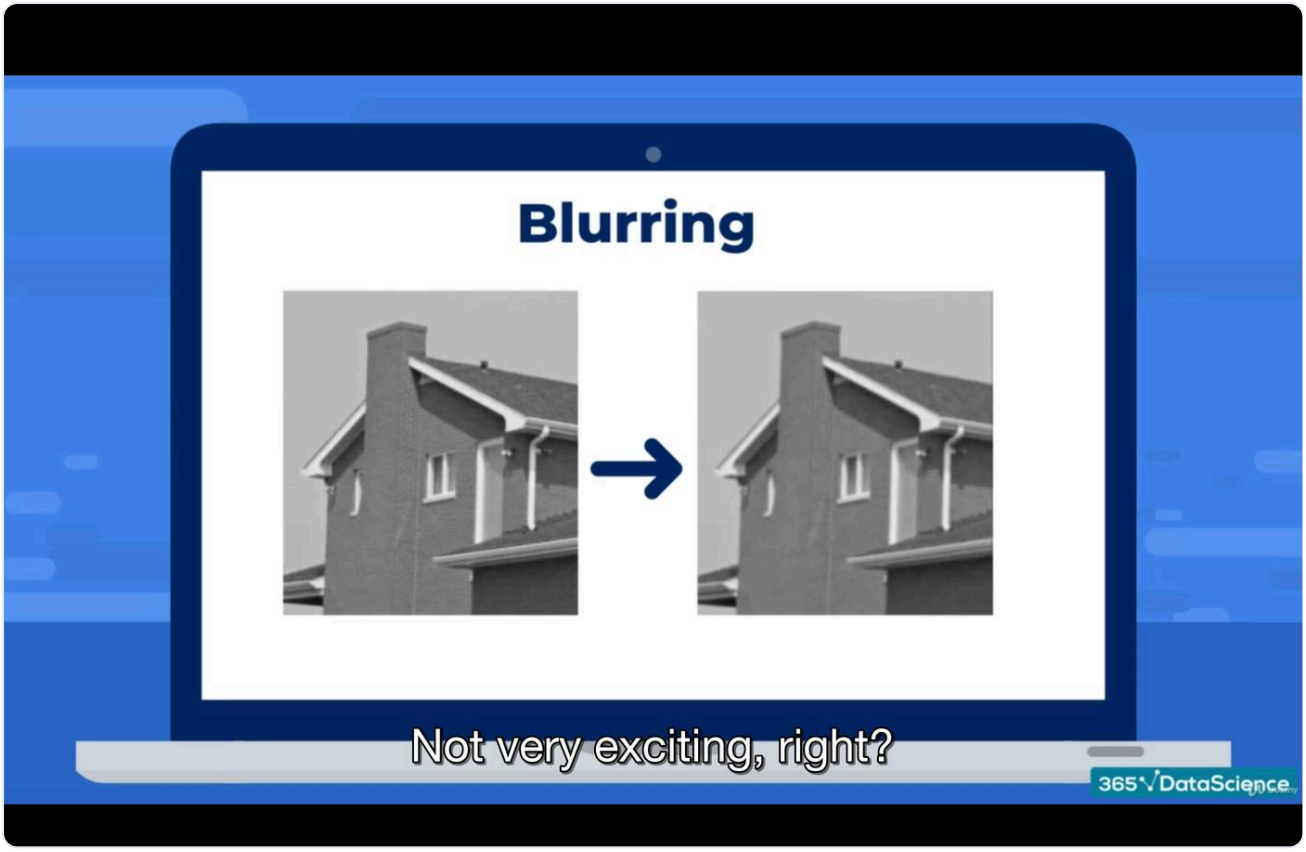

FIGURE 3.2

Not very exciting, right? Well, the reason is that blurring is a pretty natural image transformation. For instance, when a person is tired, sick, intoxicated or simply has forgotten their glasses. They see the world in a blurry way. However, when talking about digital images, blurring is nothing but a mathematical transformation. In fact, what blurring does is to smear out the differences in the neighboring pixel values. But what if we want to do the opposite? Highlight the differences further.

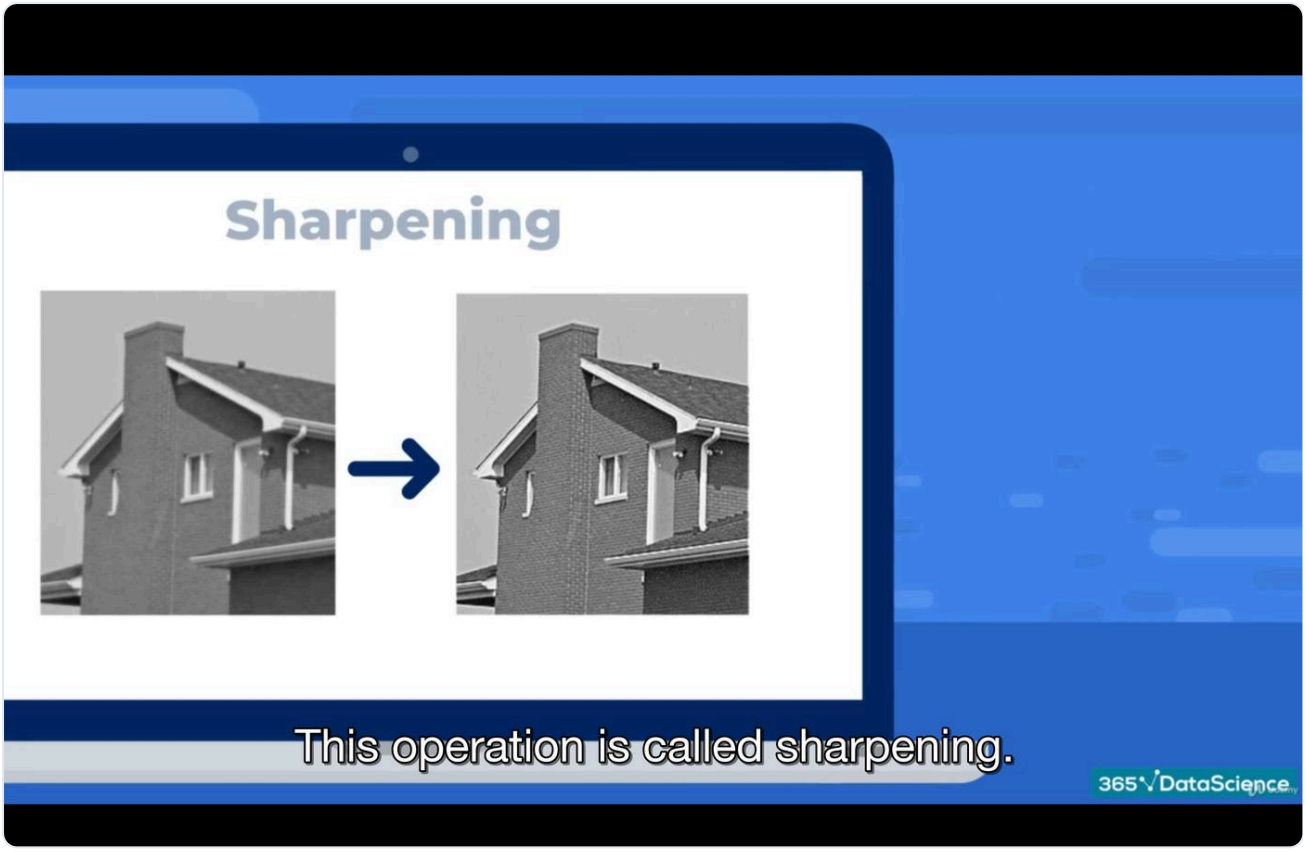

FIGURE 3.3

This operation is called sharpening. Here is how it looks in our original image. Notice the more pronounced lines between the bricks. It almost looks like we increase the resolution and somehow we have more information than before. However, we should not be fooled by this visual illusion. As whatever transformations we apply, we cannot create new information, but only trick our brains into thinking there is more information. Zoom and Enhance is only possible in TV shows such as CSI Miami. All right, let's show one final transformation. What if we want to detect only the edges in our original image? Here's an example of such an operation.

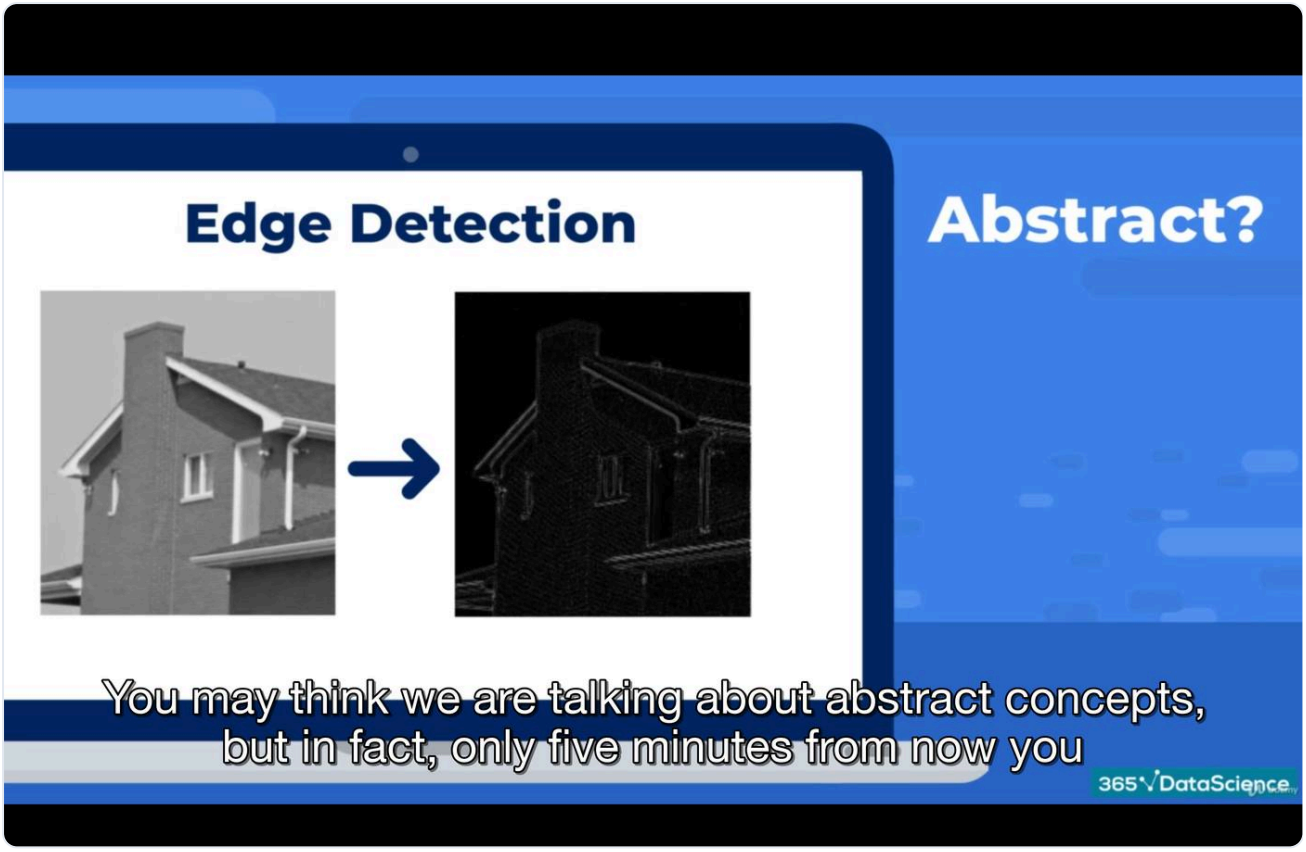

FIGURE 3.4

You may think we are talking about abstract concepts, but in fact, only five minutes from now you will be able to do it on your own. Great.

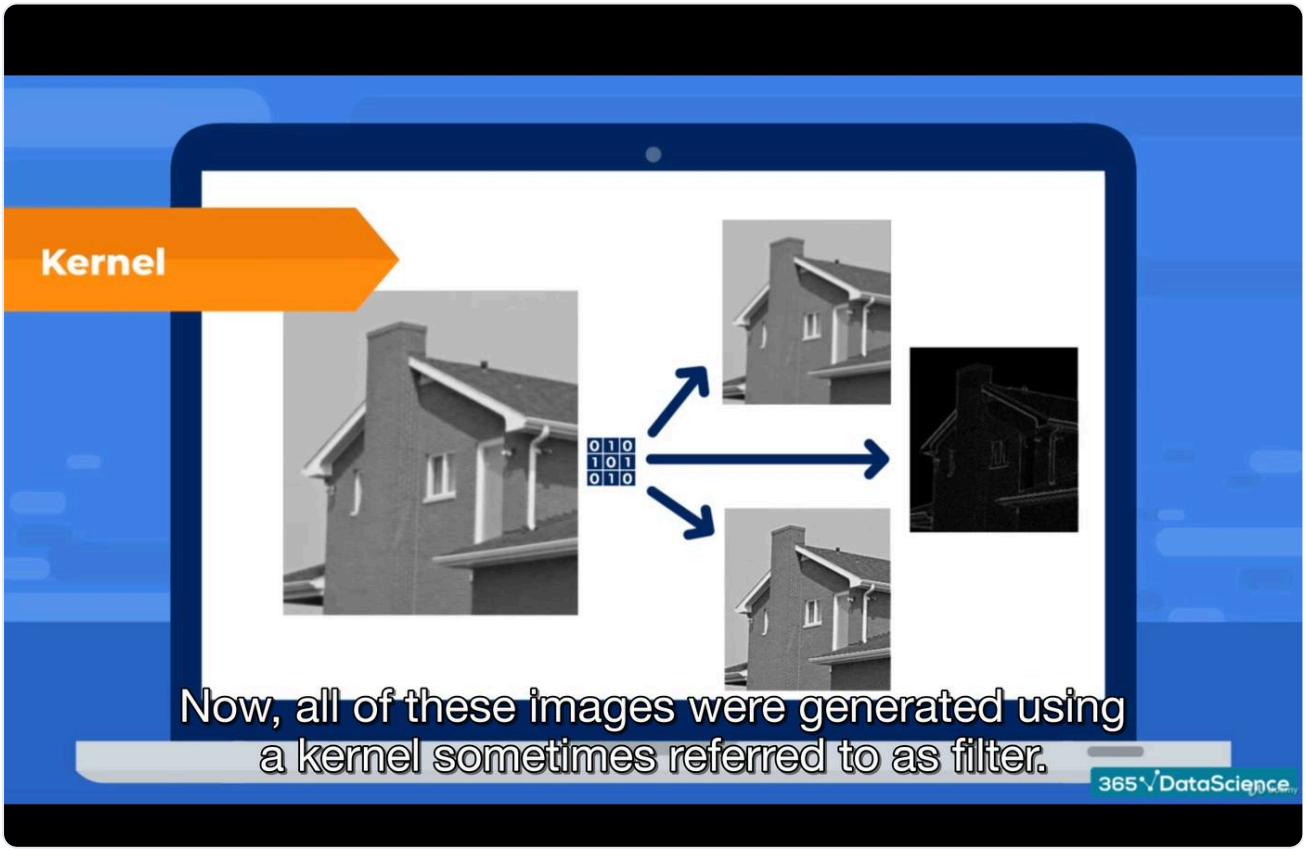

FIGURE 3.5

Now, all of these images were generated using a kernel sometimes referred to as filter.

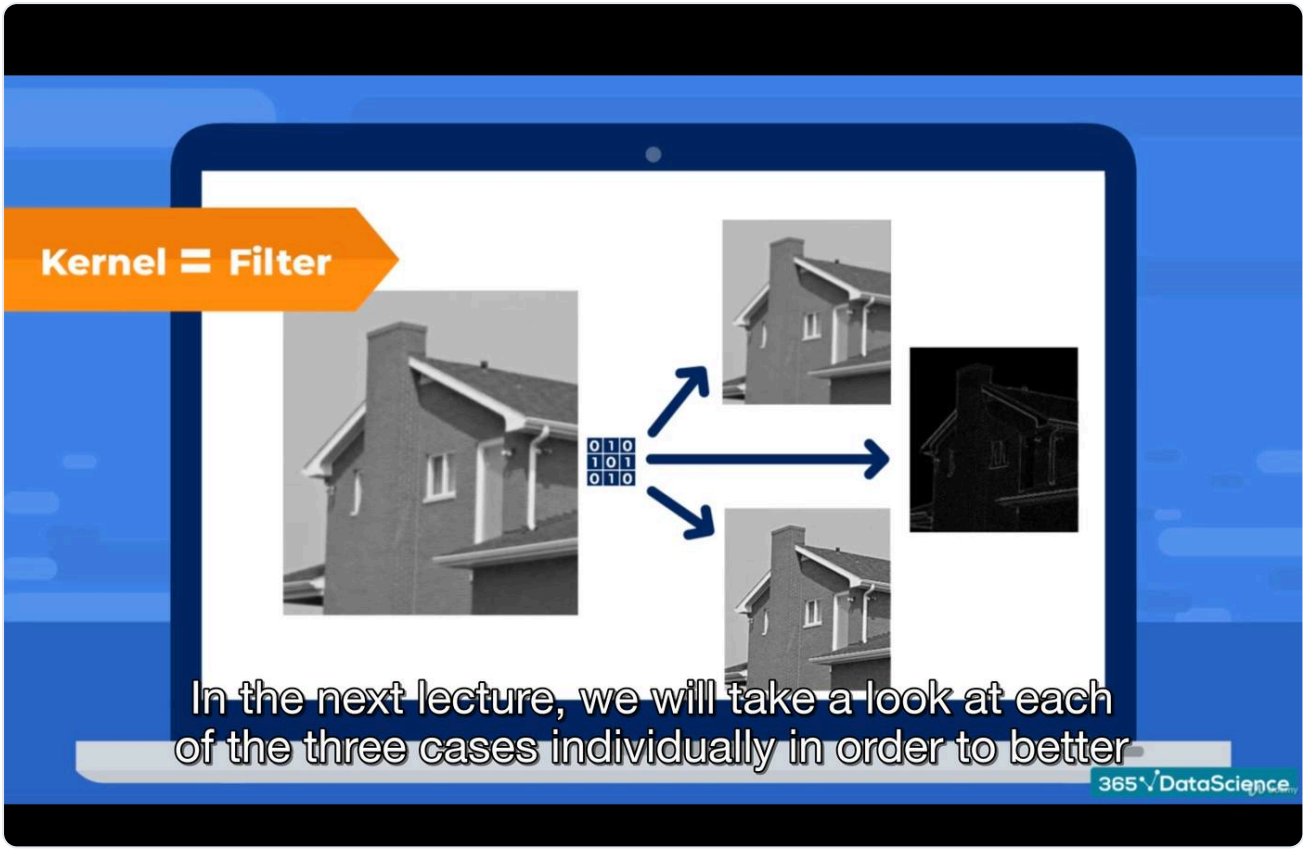

FIGURE 3.6

In the next lecture, we will take a look at each of the three cases individually in order to better understand what's happening. See you there. And thanks for watching.

LECTURE 4

Original Sharpened

Hey, welcome back. Last time we showed you three different transformations of a single image using kernels in this video, we will dive deeper into the mechanics of how they were generated. This may sound a bit disconnected from CNNs, but it is an important step in this direction. We promise. OK, let's start with blurring. Blurring or smoothing is one of the simplest ways we can change our original image. And you probably intuitively know how it works. Basically, its purpose is to smear out the differences, in other words, to average them out.

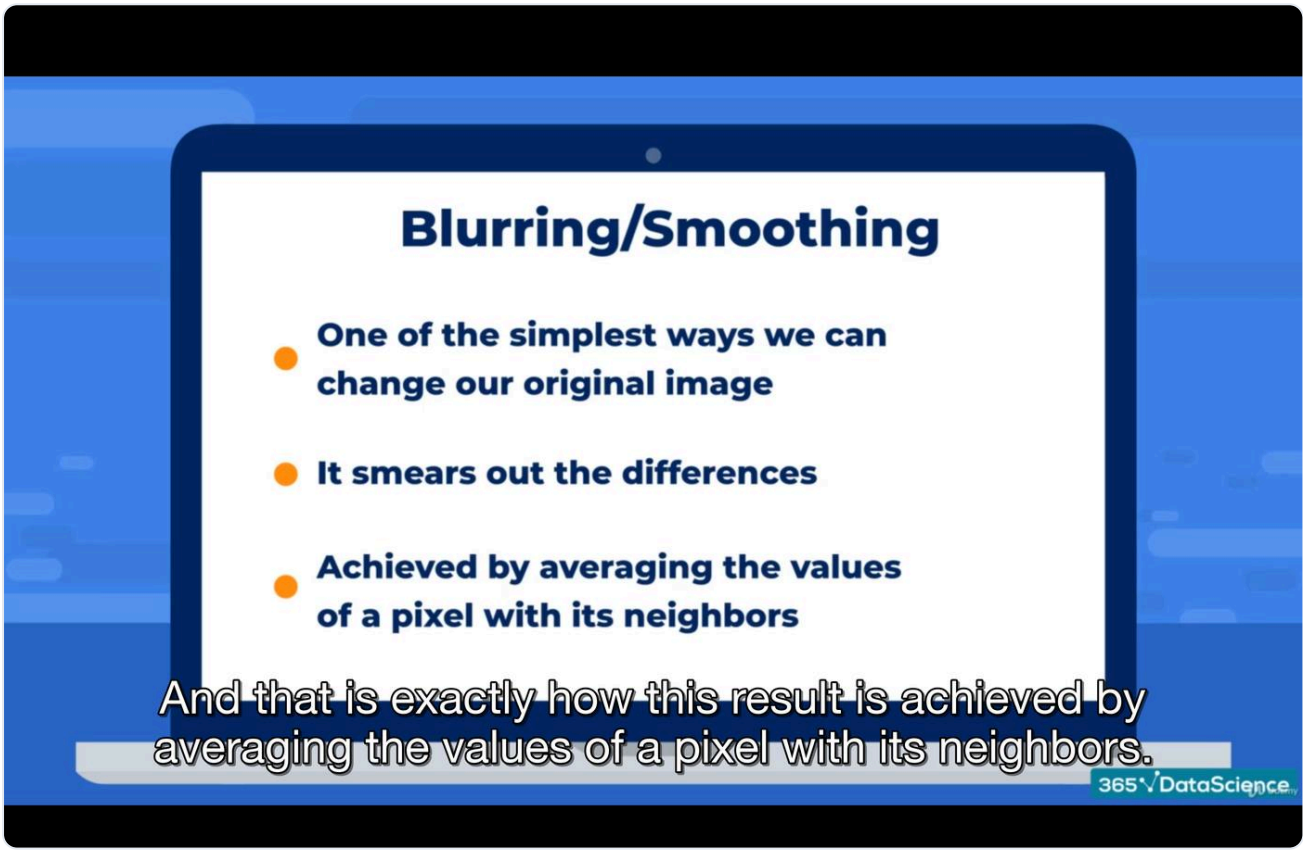

FIGURE 4.1 | Blurring/Smoothing

And that is exactly how this result is achieved by averaging the values of a pixel with its neighbors.

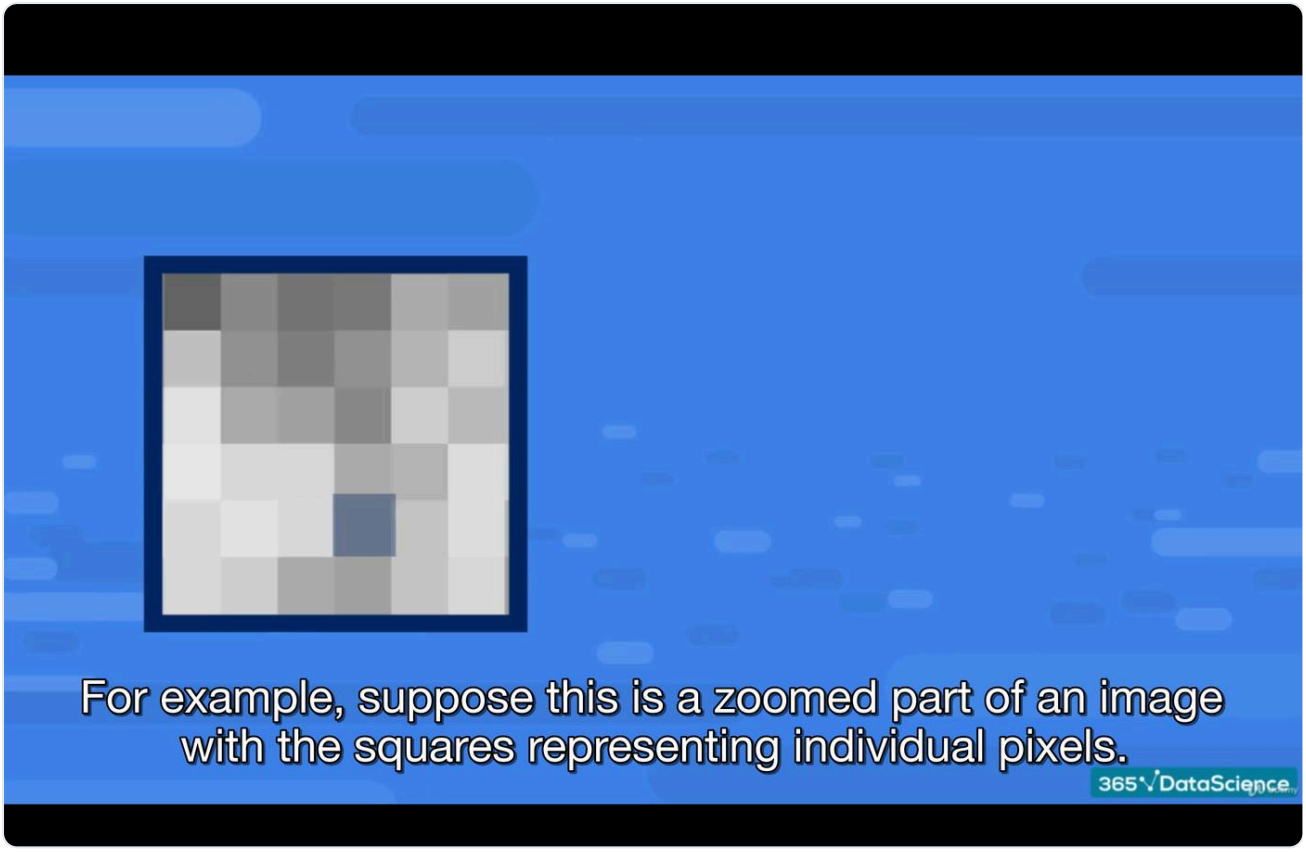

FIGURE 4.2

For example, suppose this is a zoomed part of an image with the squares representing individual pixels.

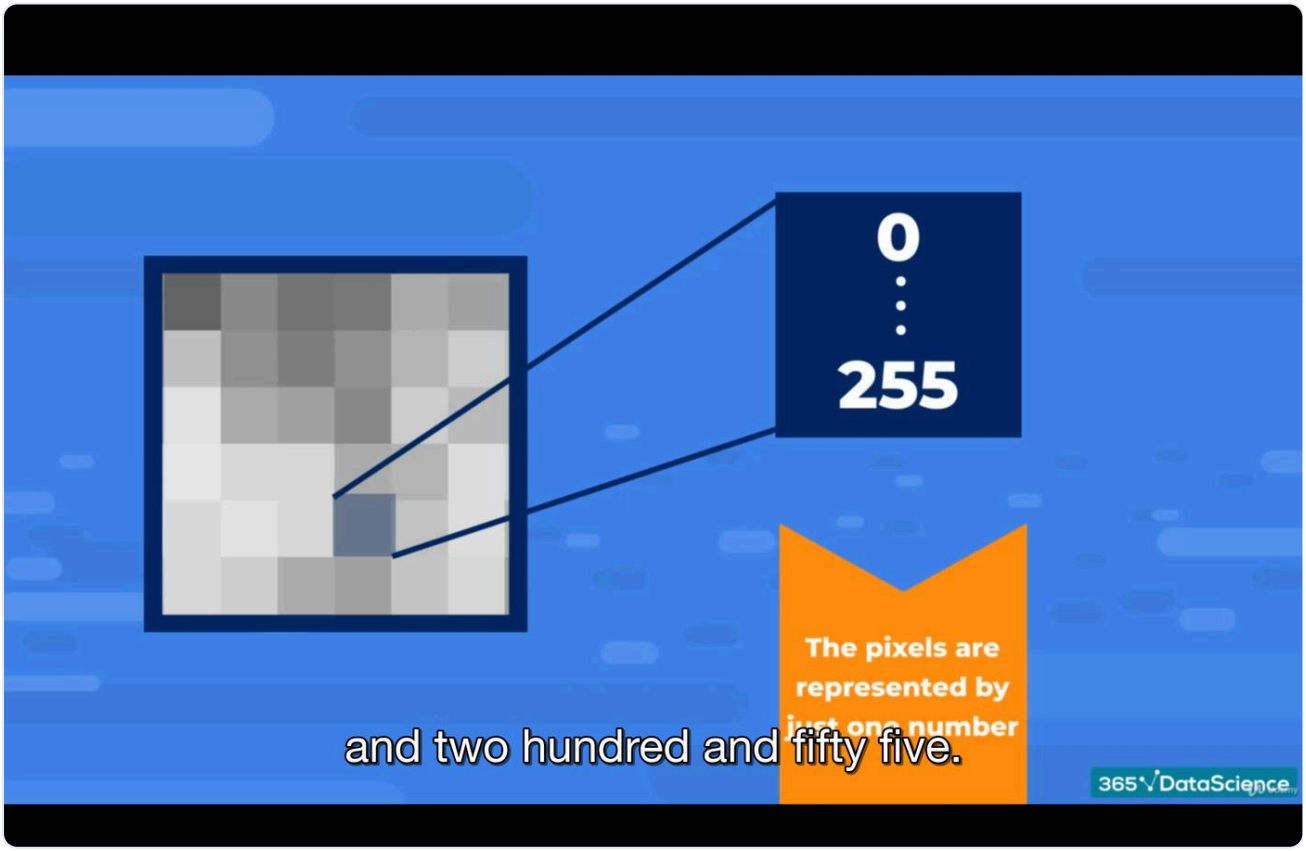

FIGURE 4.3

Since we are working with grayscale images, the pixels are represented by just one number between zero and two hundred and fifty five. The principles we are going to show can be applied to color images as well, in which every pixel has three values a red or a green and a blue one.

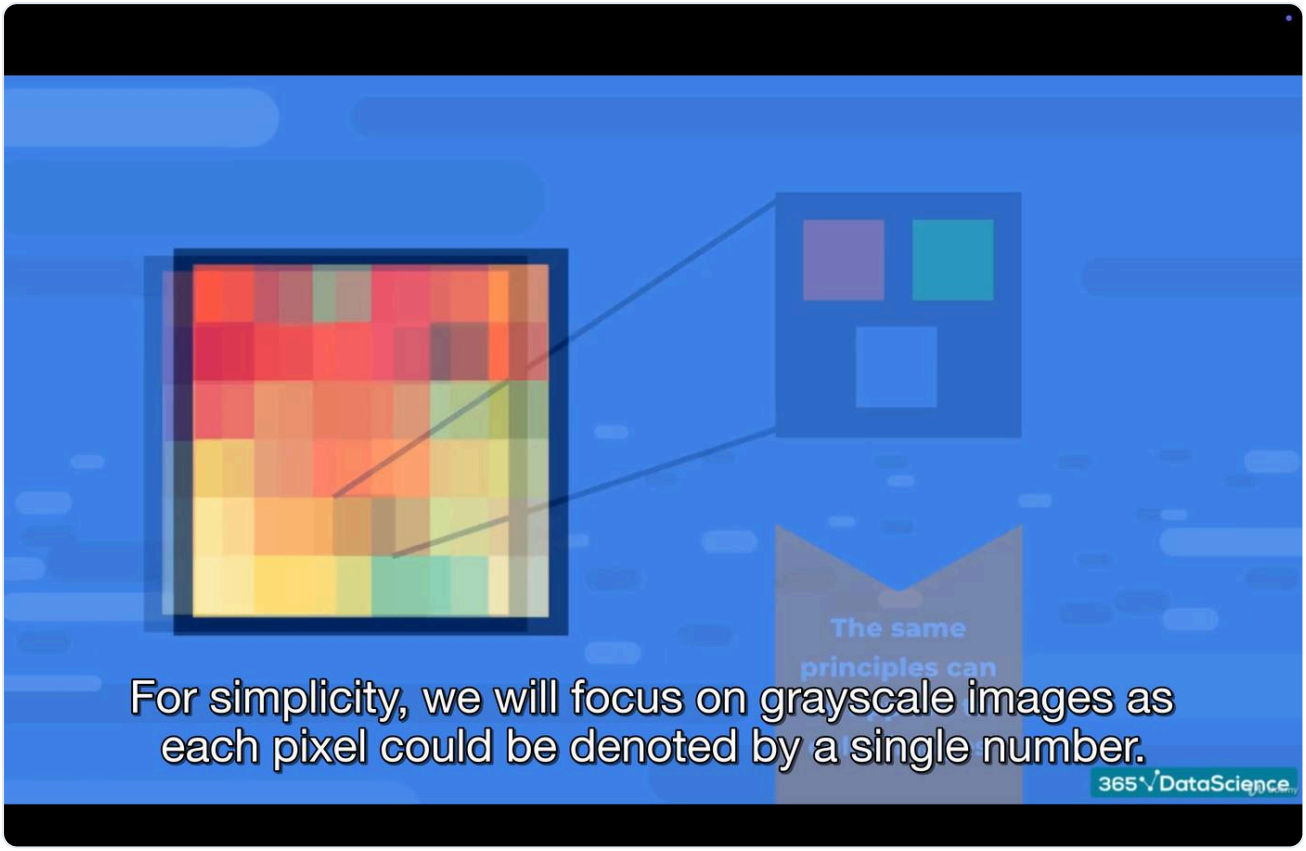

FIGURE 4.4

For simplicity, we will focus on grayscale images as each pixel could be denoted by a single number. All right, let's take this three by three portion of the image, it has a total of nine pixels, therefore nine values.

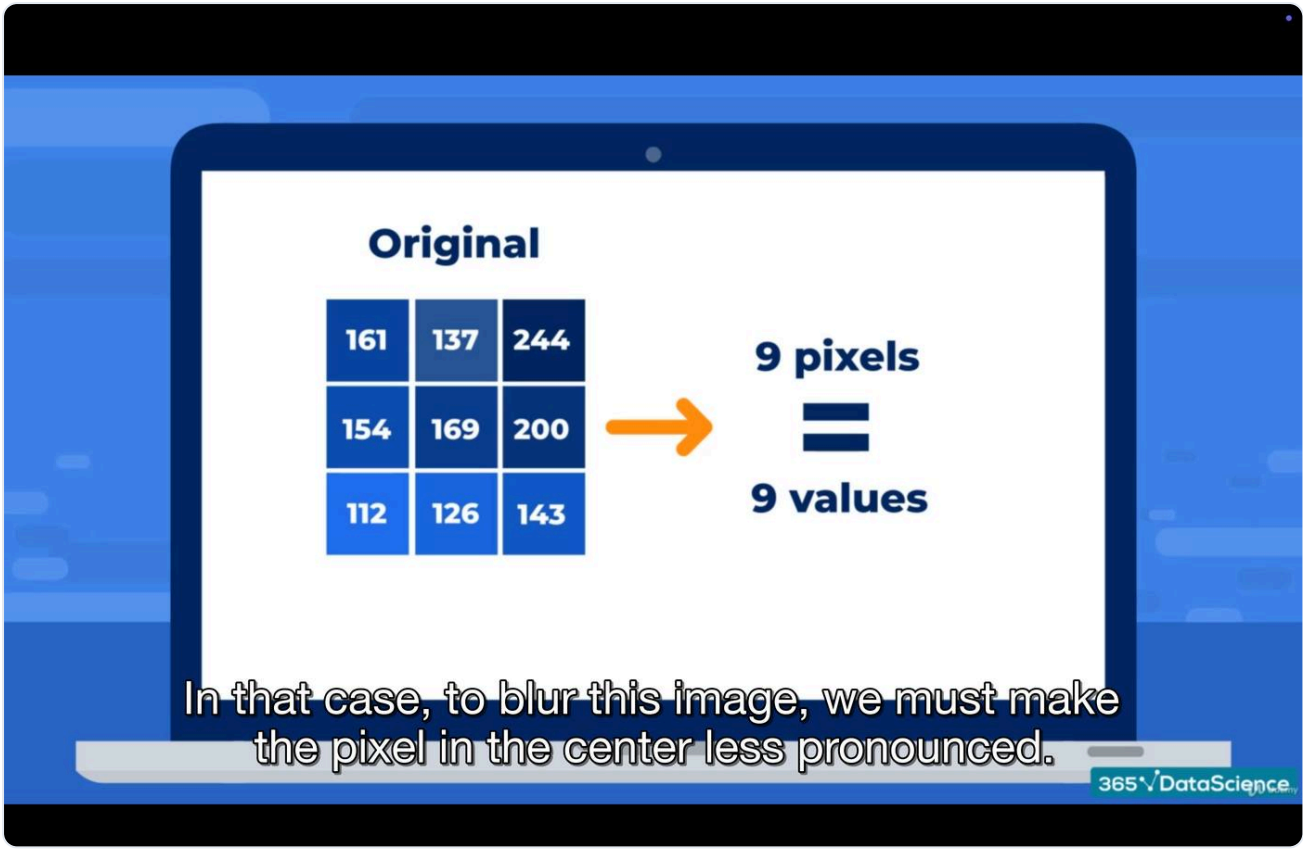

FIGURE 4.5

In that case, to blur this image, we must make the pixel in the center less pronounced. Therefore, we need to calculate a new value for it. So it's time to see how many neighbors it as well. There are different definitions of neighbors, depending on whether you count only direct neighbors, diagonals and so on. For this example, we will count the diagonals as well, which means that the direct neighbors of the center pixel are these. There are eight in total.

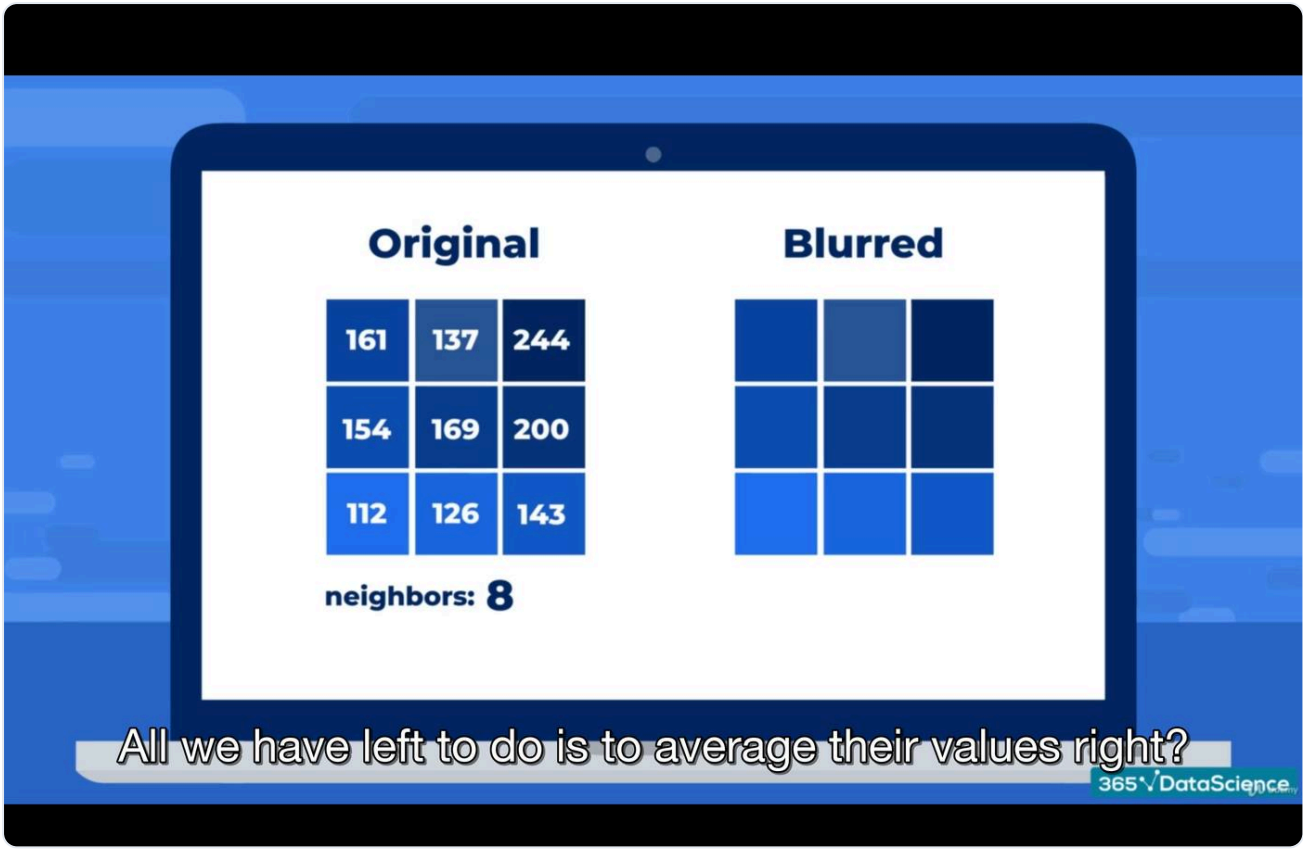

FIGURE 4.6 | Original Blurred

All we have left to do is to average their values right?

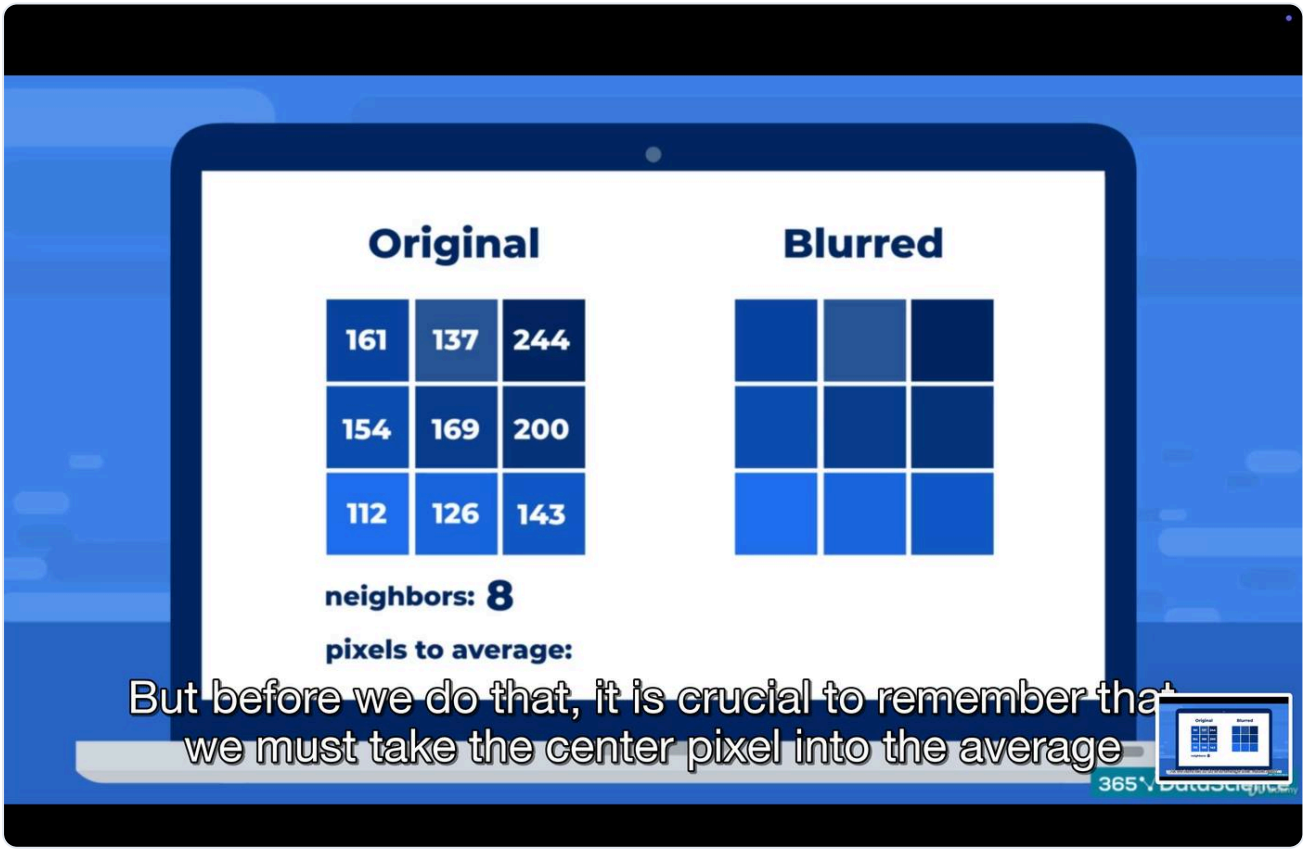

FIGURE 4.7 | Original Blurred

But before we do that, it is crucial to remember that we must take the center pixel into the average as well. After all, it is the fundament of the blurring. Therefore, we take all nine pixels.

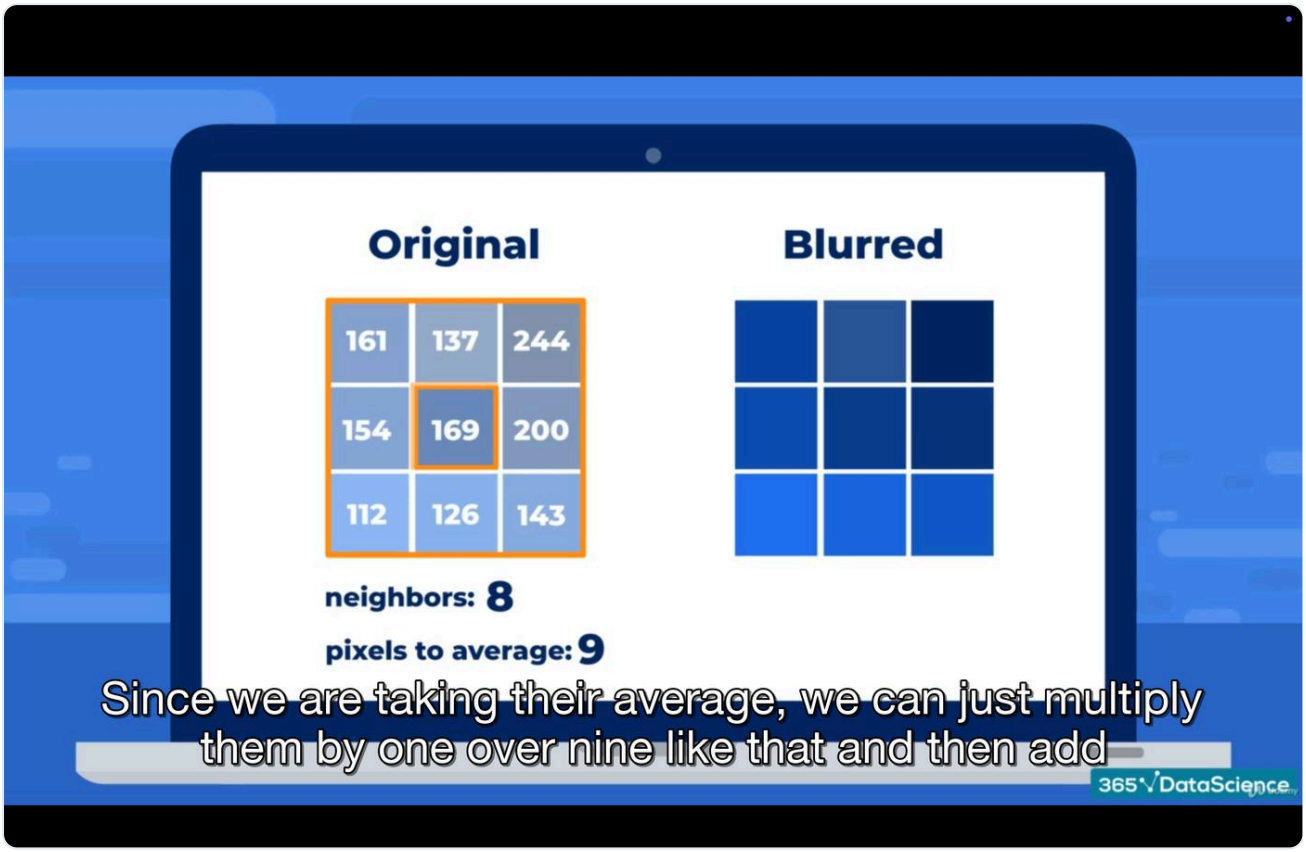

FIGURE 4.8 | Original Blurred

Since we are taking their average, we can just multiply them by one over nine like that and then add them together.

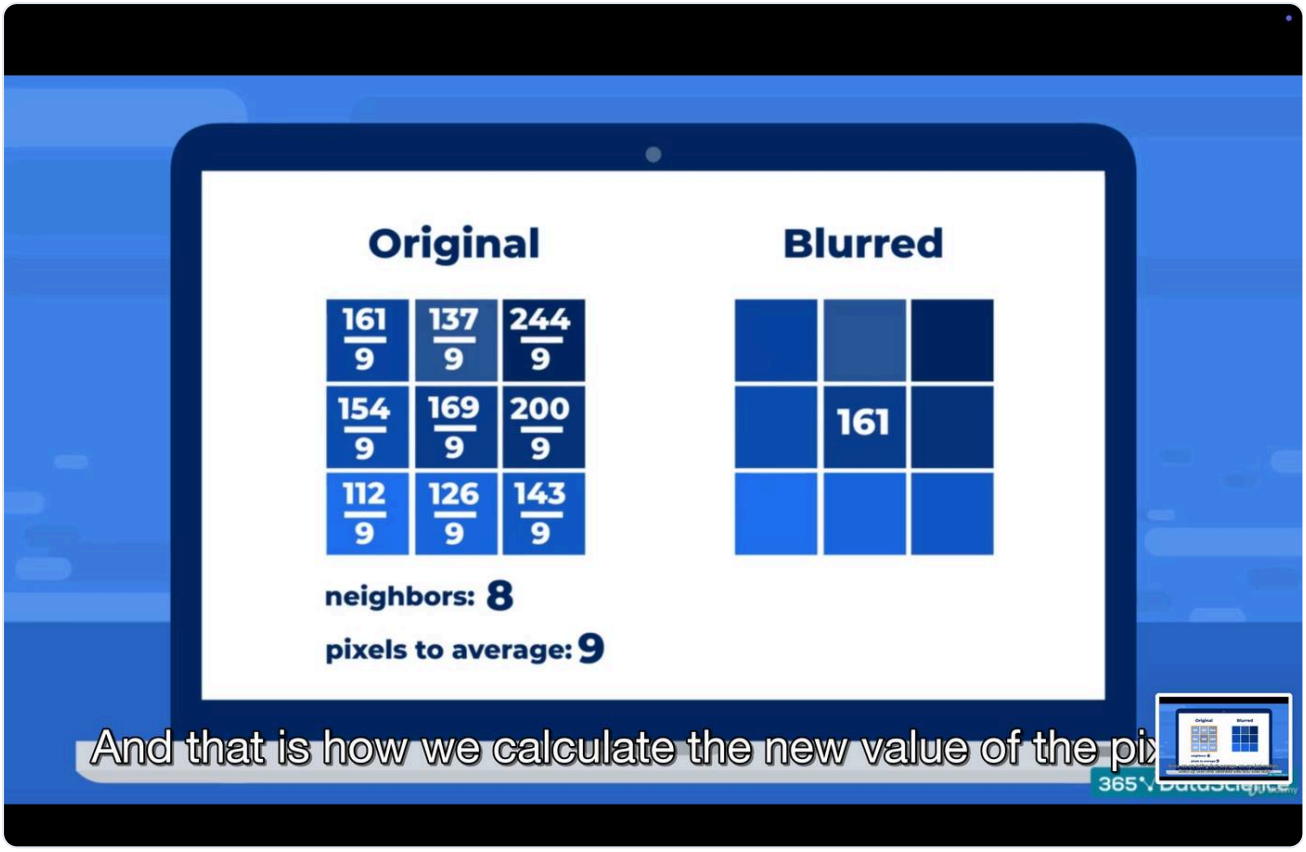

FIGURE 4.9 | Original Blurred

And that is how we calculate the new value of the pixel.

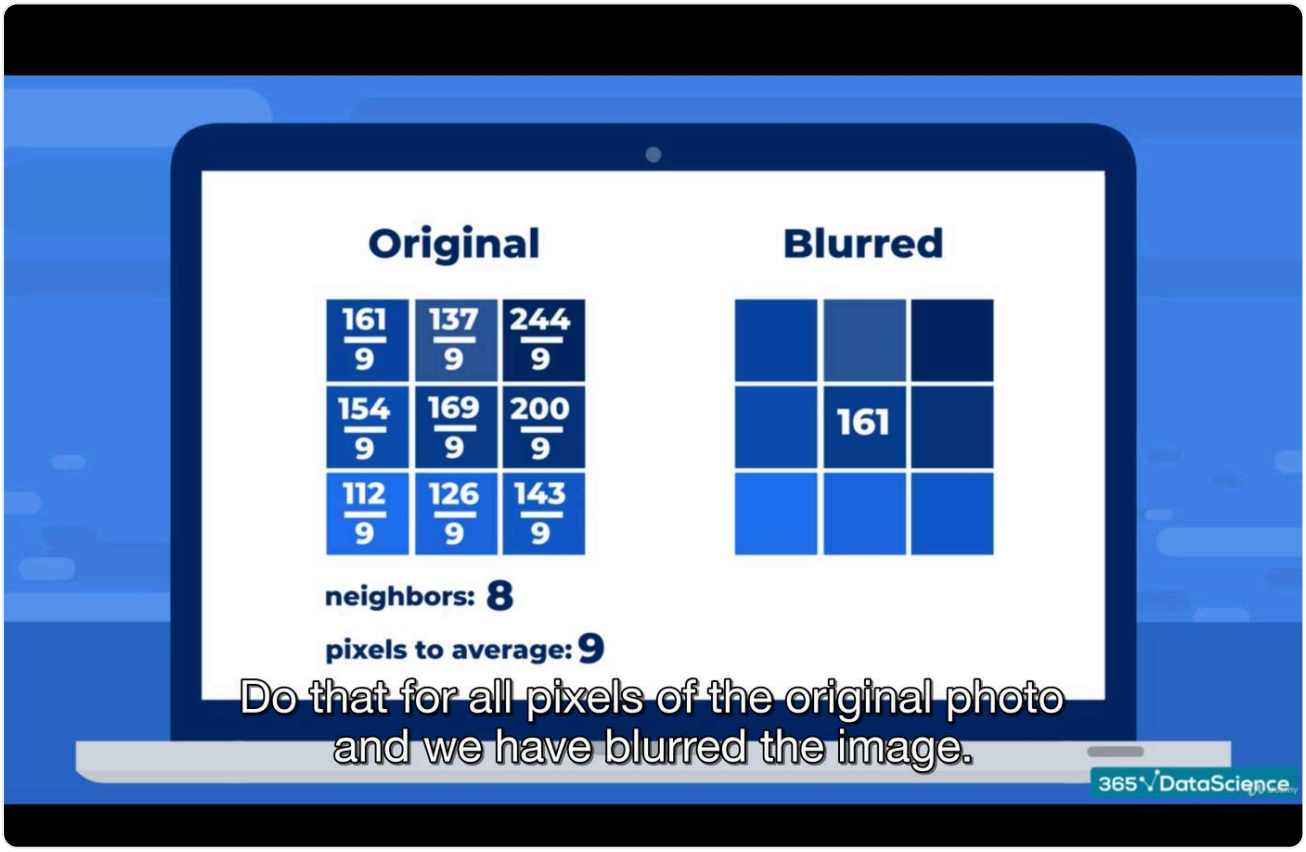

FIGURE 4.10 | Original Blurred

Do that for all pixels of the original photo and we have blurred the image. All right.

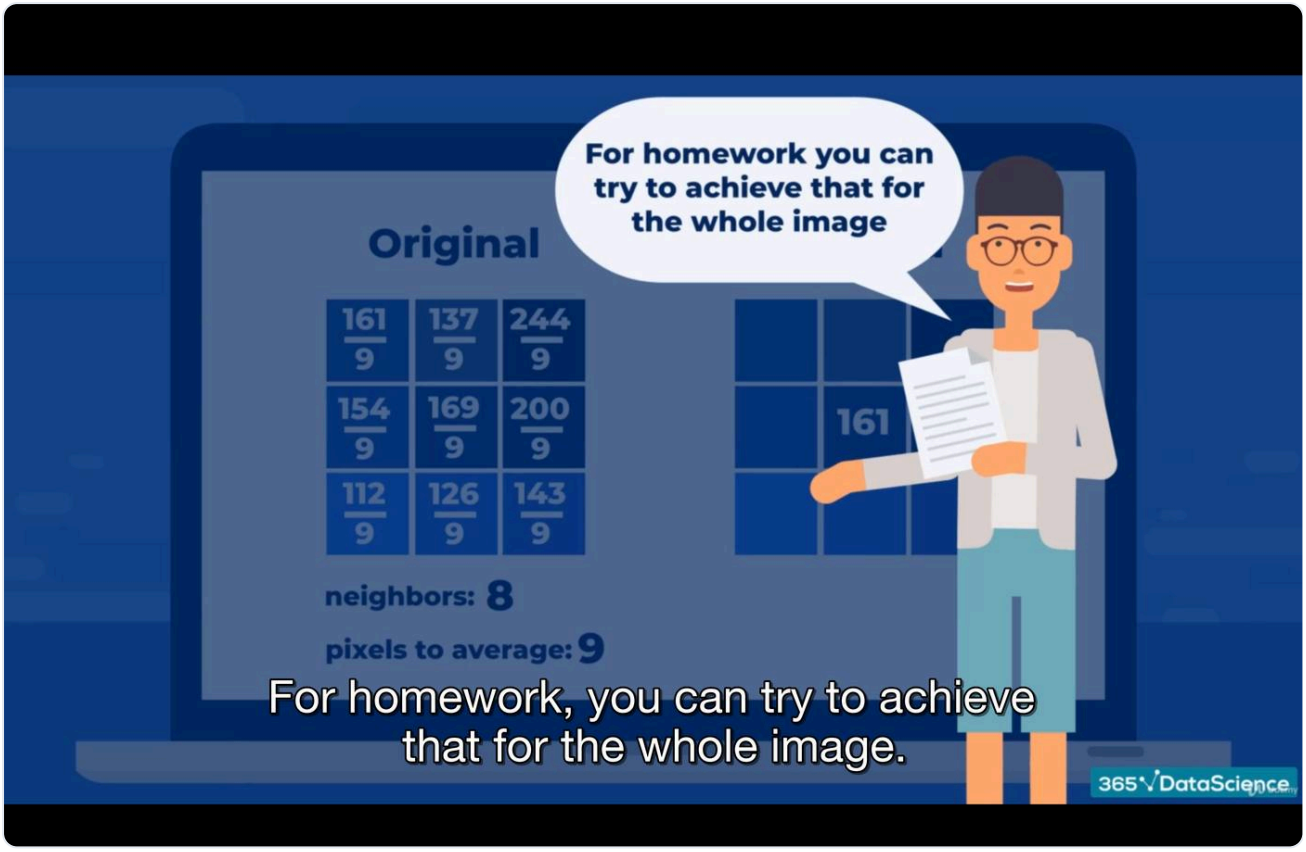

FIGURE 4.11

For homework, you can try to achieve that for the whole image. Now, this was fairly easy. How about sharpening? Well, this one is a little bit trickier, first, we've got to ask ourselves, what is it we are trying to accomplish? In contrast to the previous example, we are trying to highlight the difference in pixel values. One way to think about it is as the opposite of blurring.

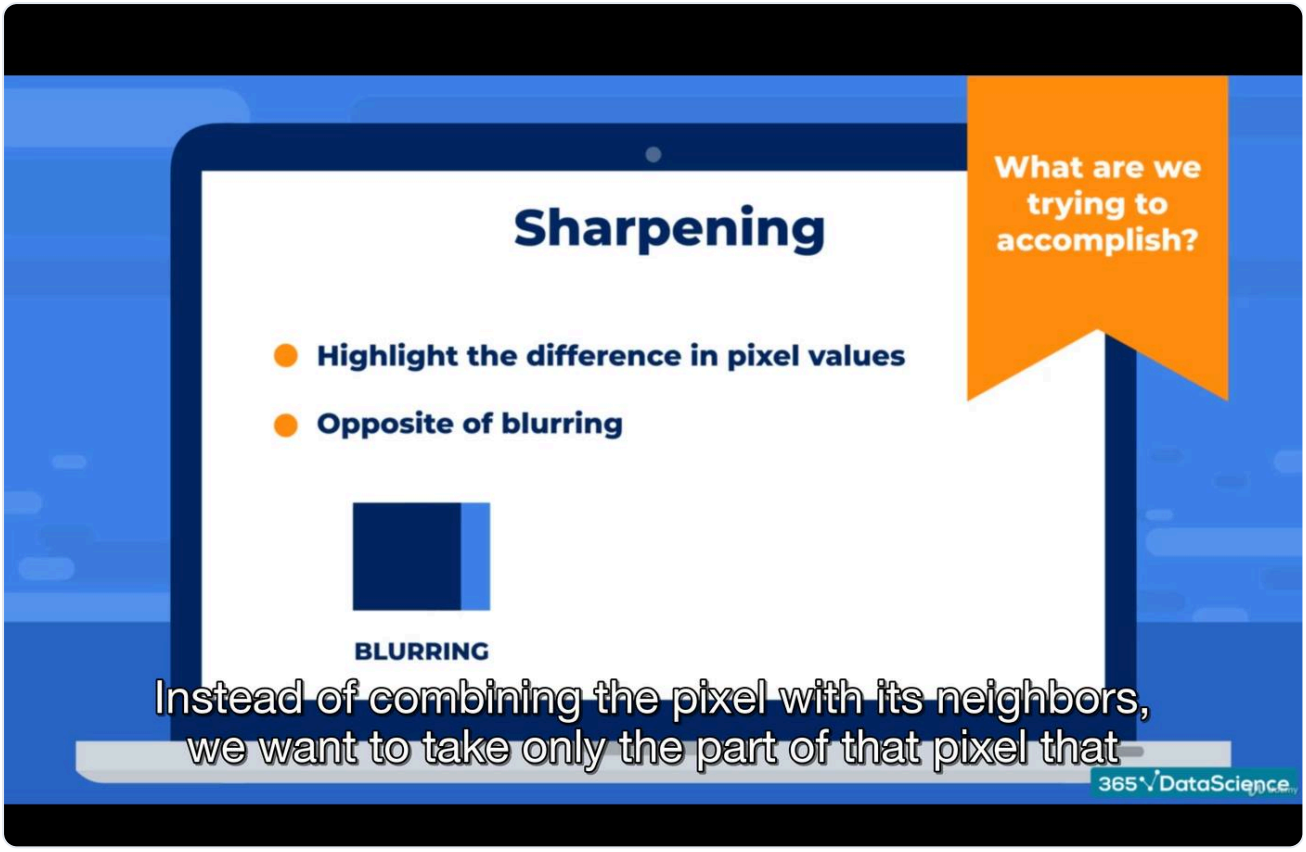

FIGURE 4.12 | Highlight the Difference in Pixel Values

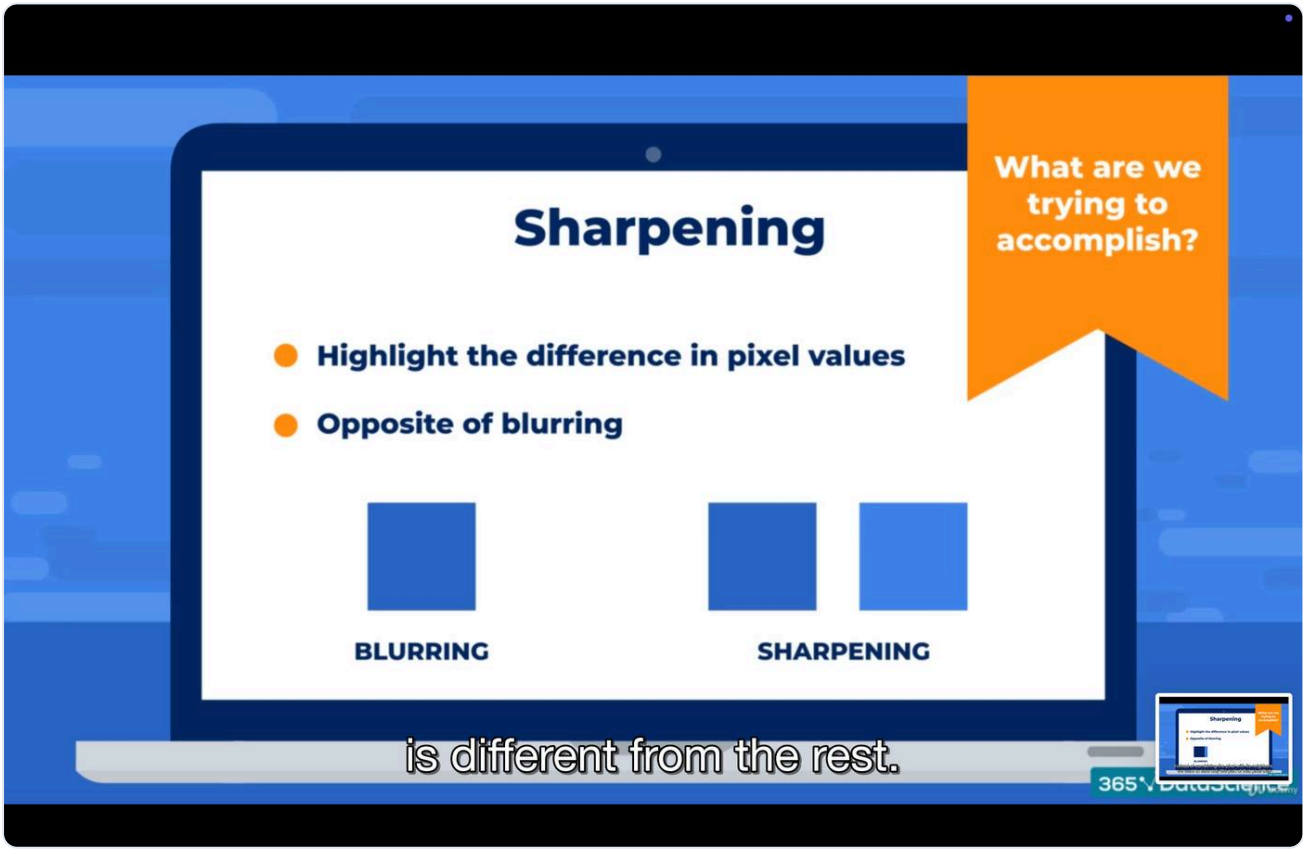

FIGURE 4.13 | Highlight the Difference in Pixel Values

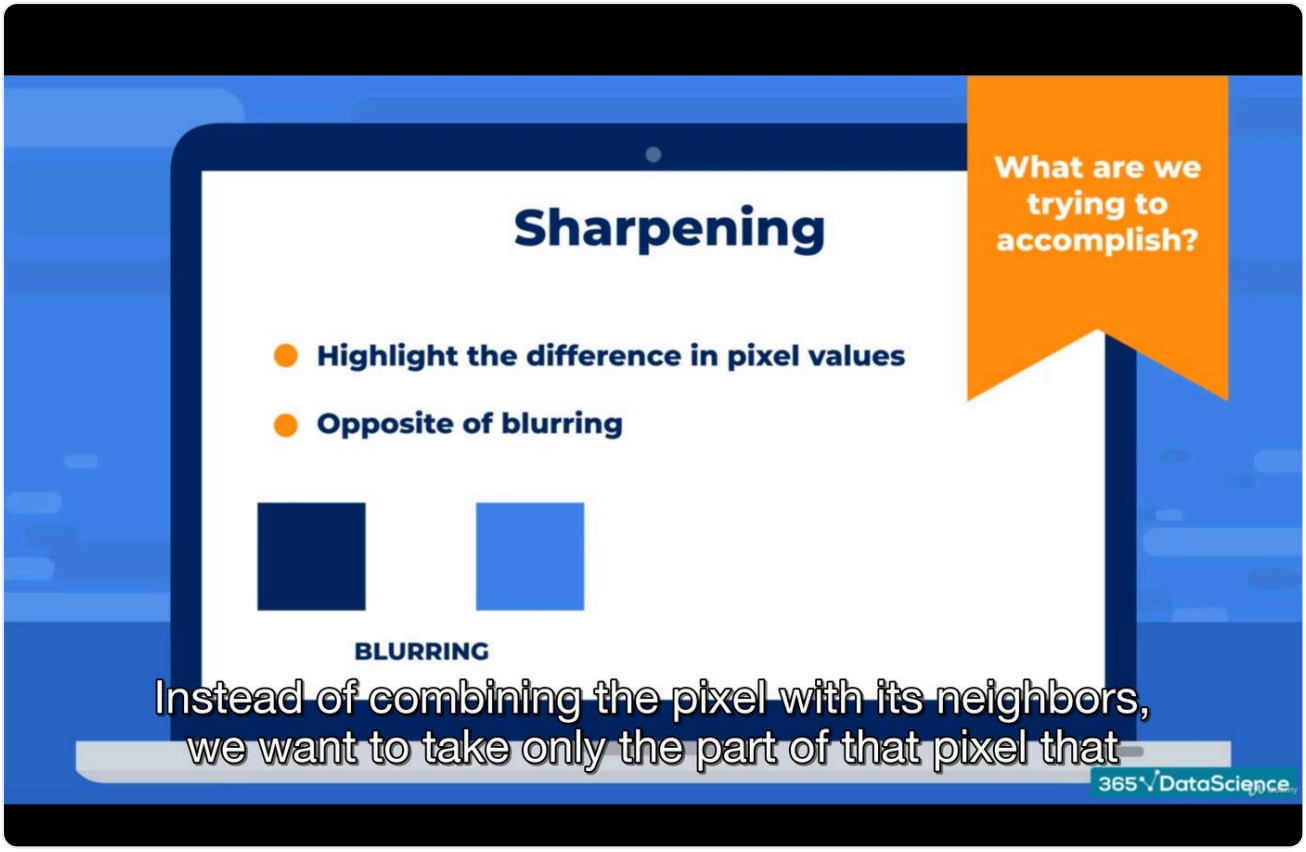

FIGURE 4.14 | Highlight the Difference in Pixel Values

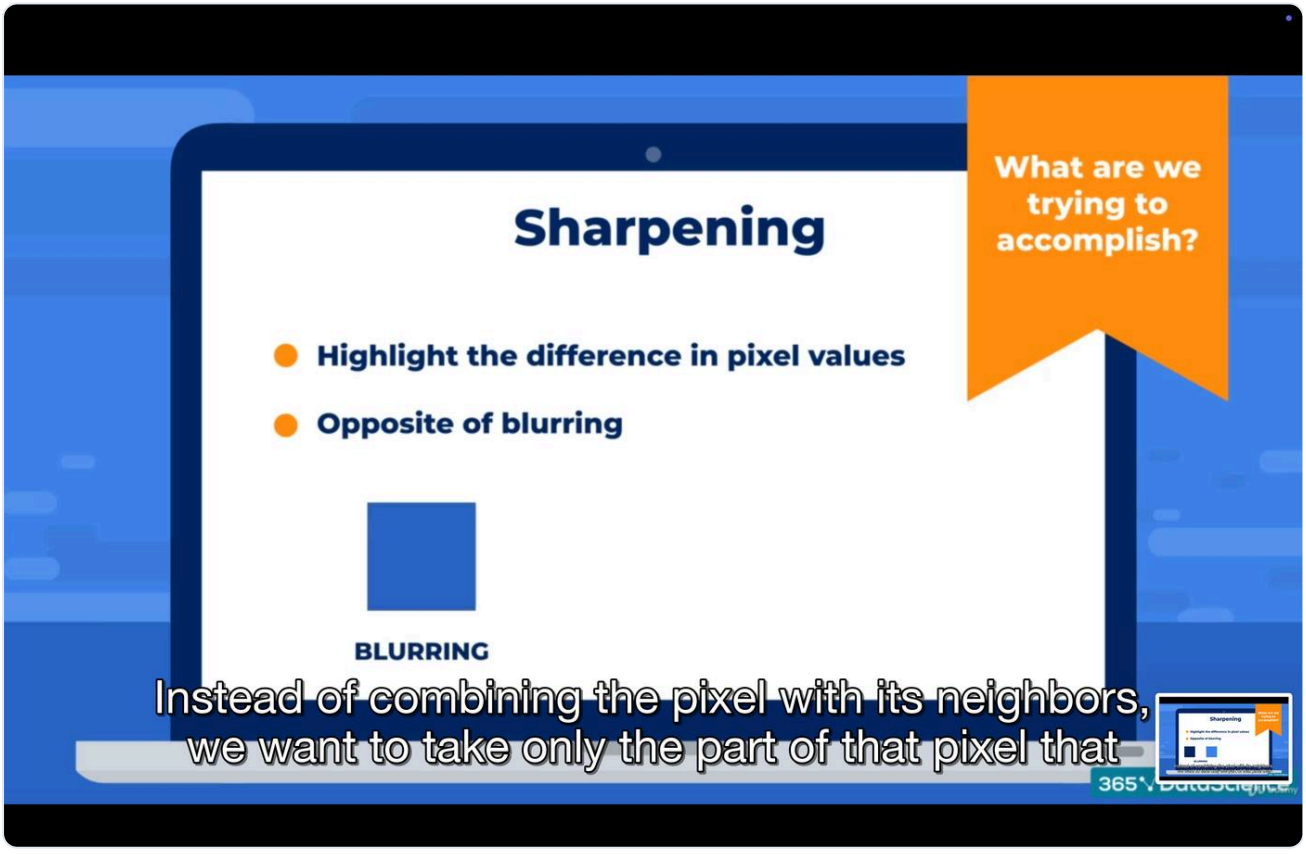

FIGURE 4.15 | Highlight the Difference in Pixel Values

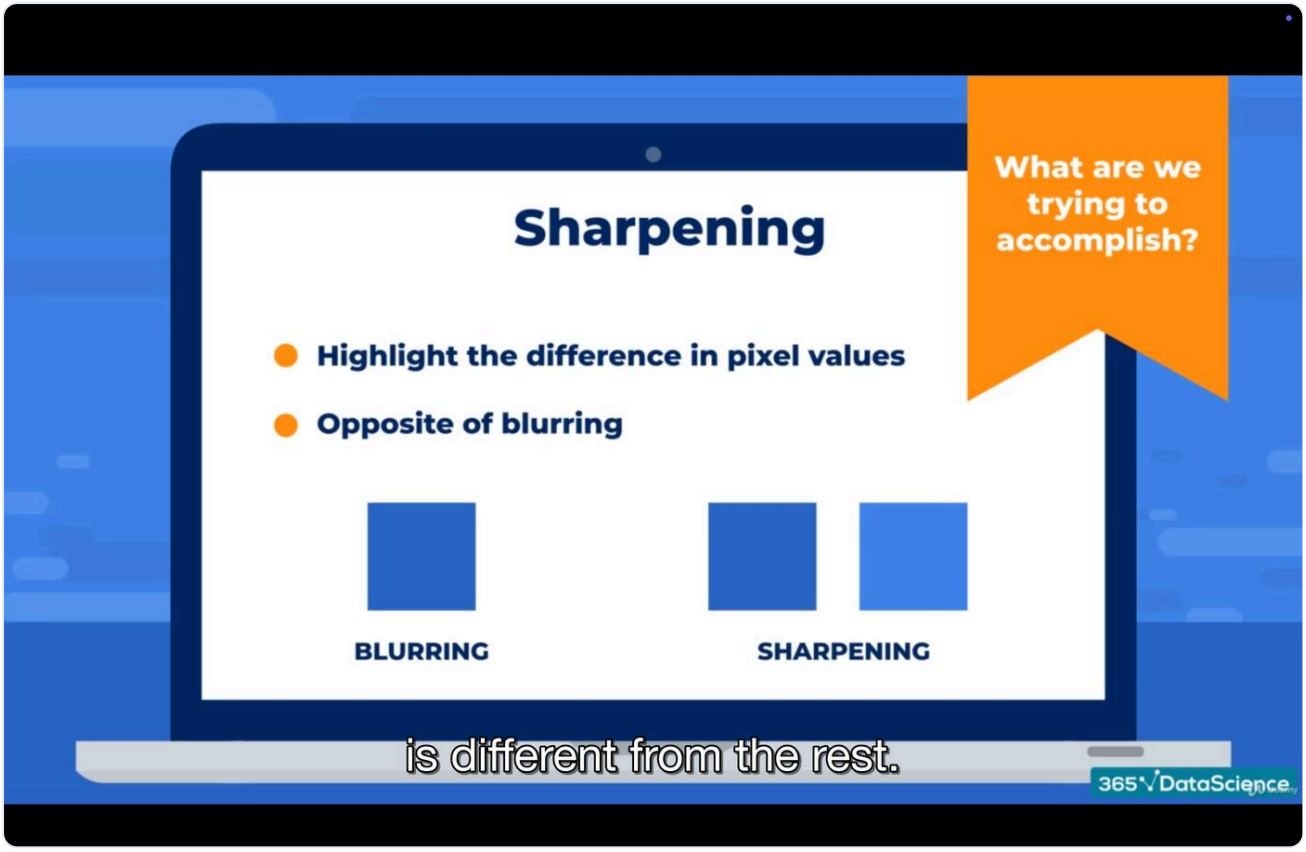

FIGURE 4.16 | Highlight the Difference in Pixel Values

Instead of combining the pixel with its neighbors, we want to take only the part of that pixel that is different from the rest. That would be the change from the previous value.

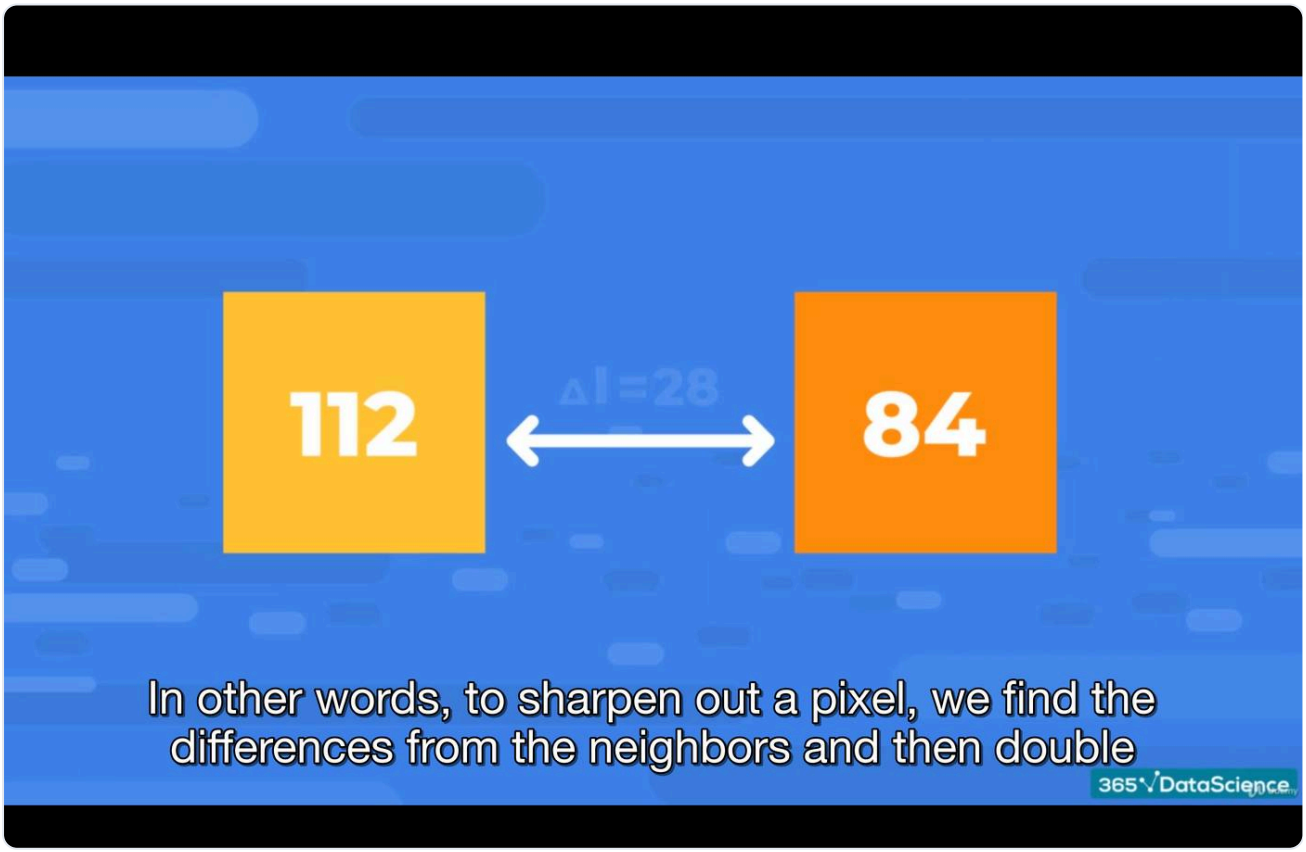

FIGURE 4.17

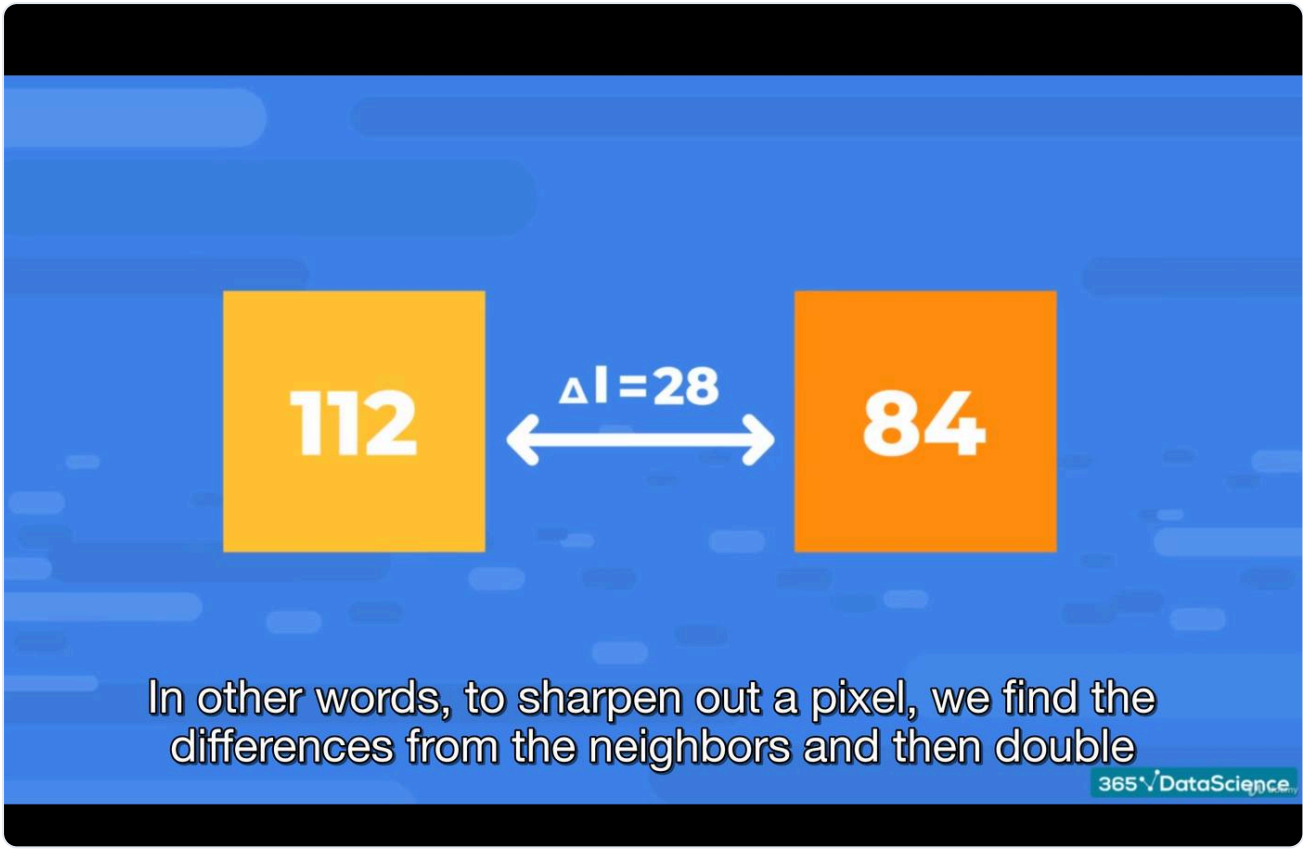

FIGURE 4.18

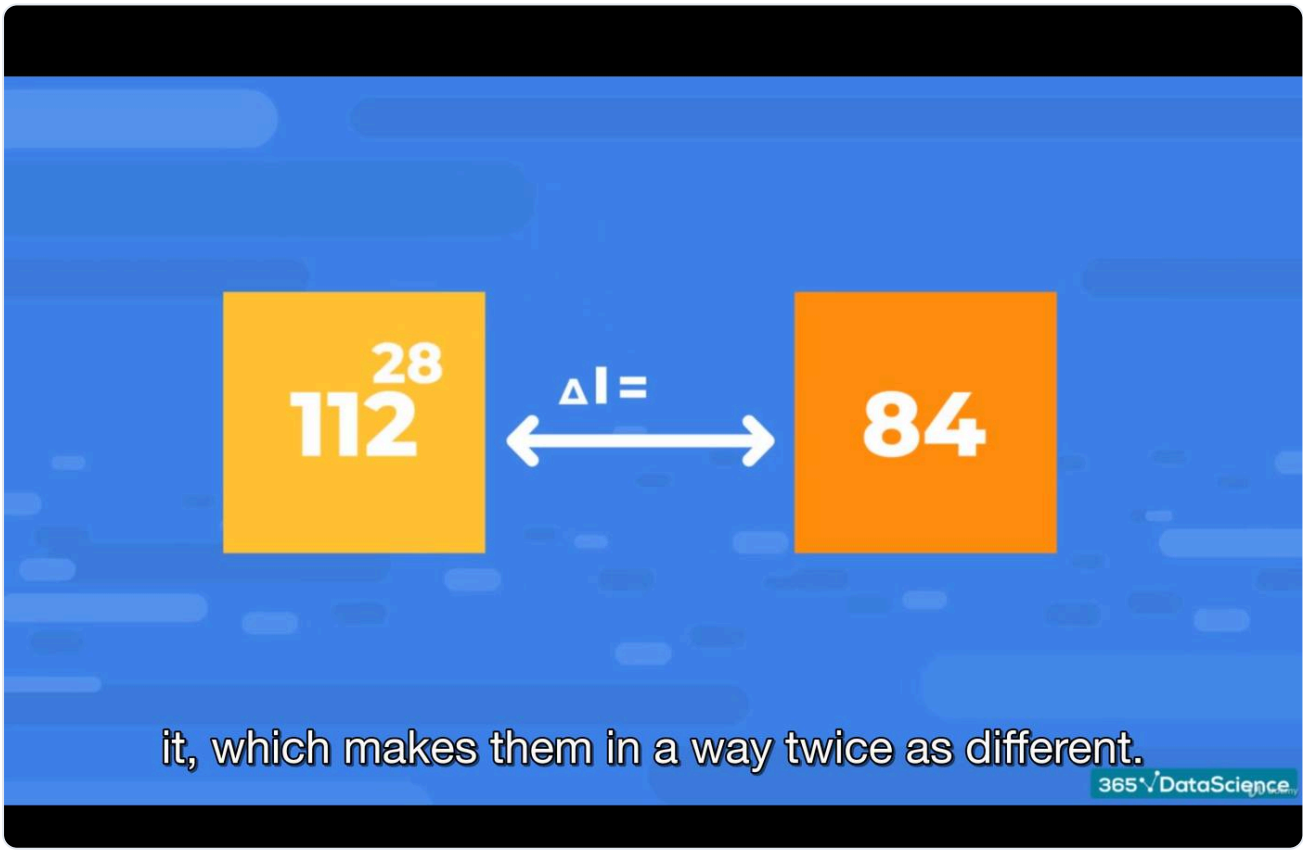

FIGURE 4.19

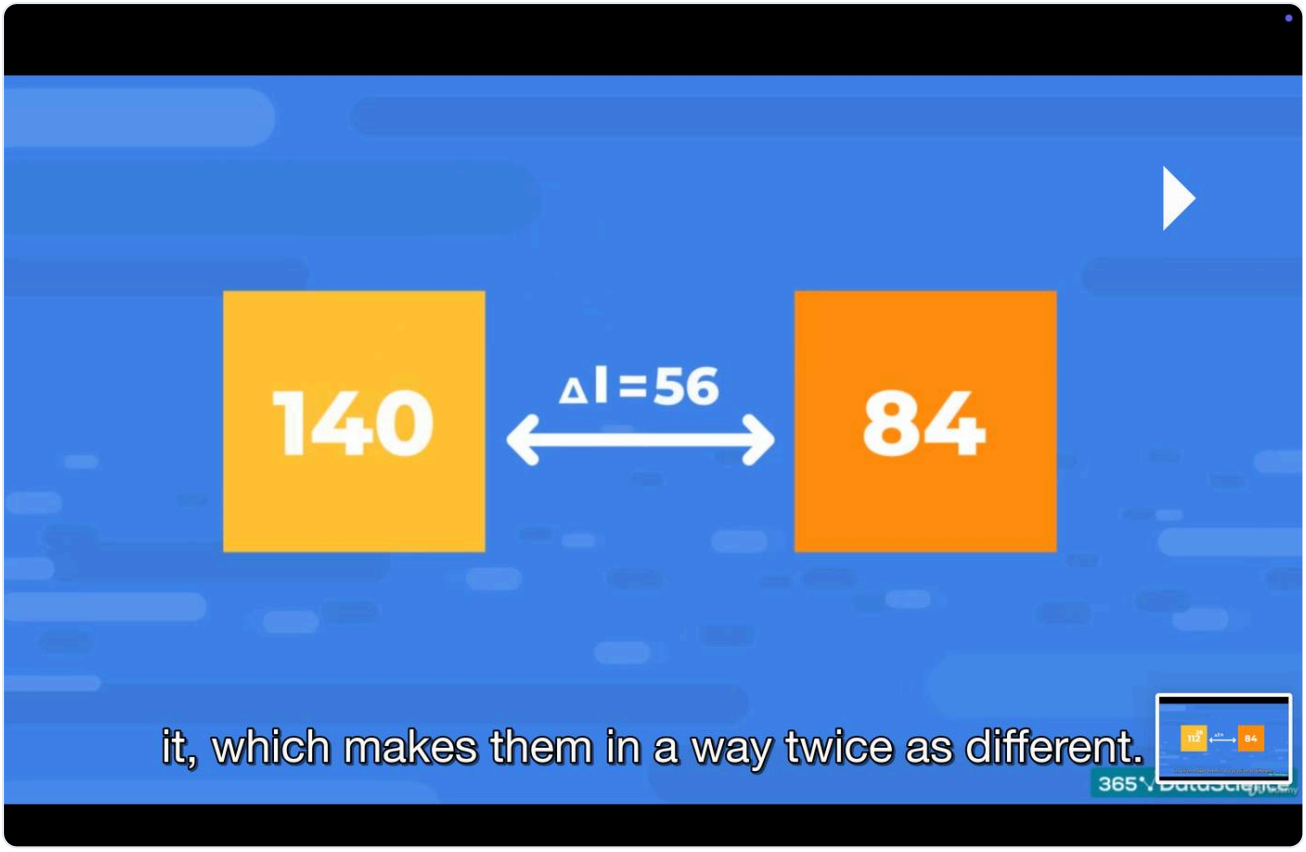

FIGURE 4.20

In other words, to sharpen out a pixel, we find the differences from the neighbors and then double it, which makes them in a way twice as different. So let's take a look at the pixilated image once again.

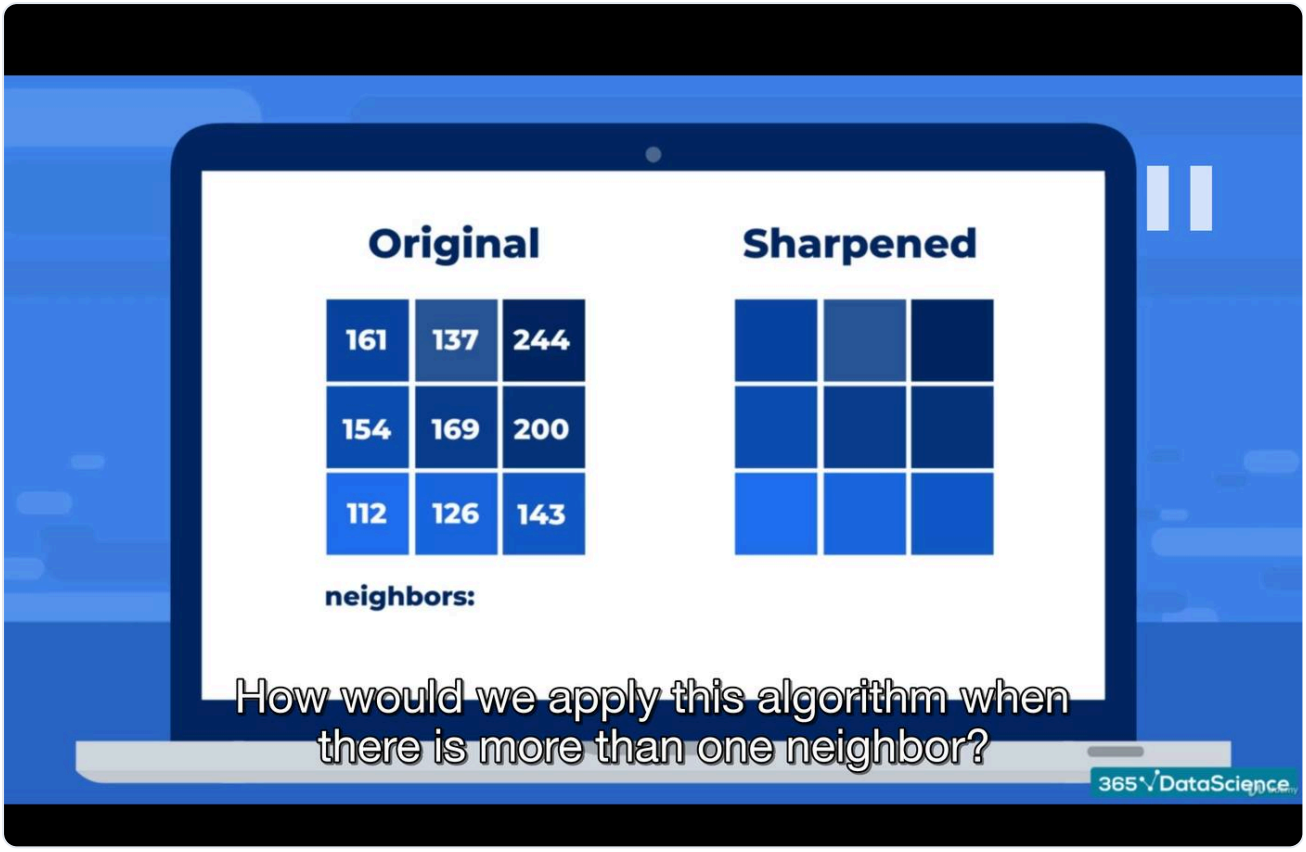

FIGURE 4.21 | Original Sharpened

How would we apply this algorithm when there is more than one neighbor? First, we start by identifying which neighbors to use for my experiments.

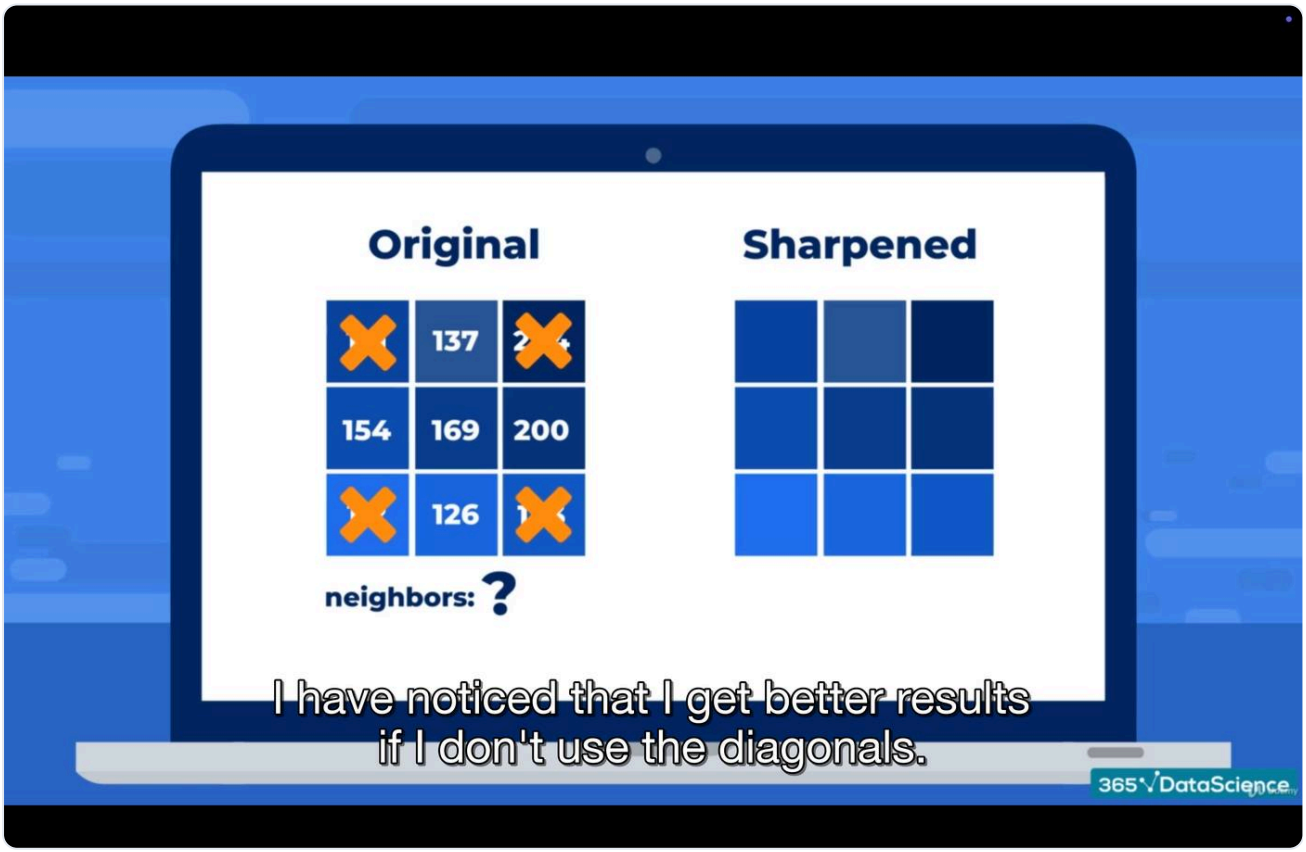

FIGURE 4.22 | Original Sharpened

I have noticed that I get better results if I don't use the diagonals. So we are going to use only those four neighbors. Now, as we discussed, we need to subtract those values from the center one.

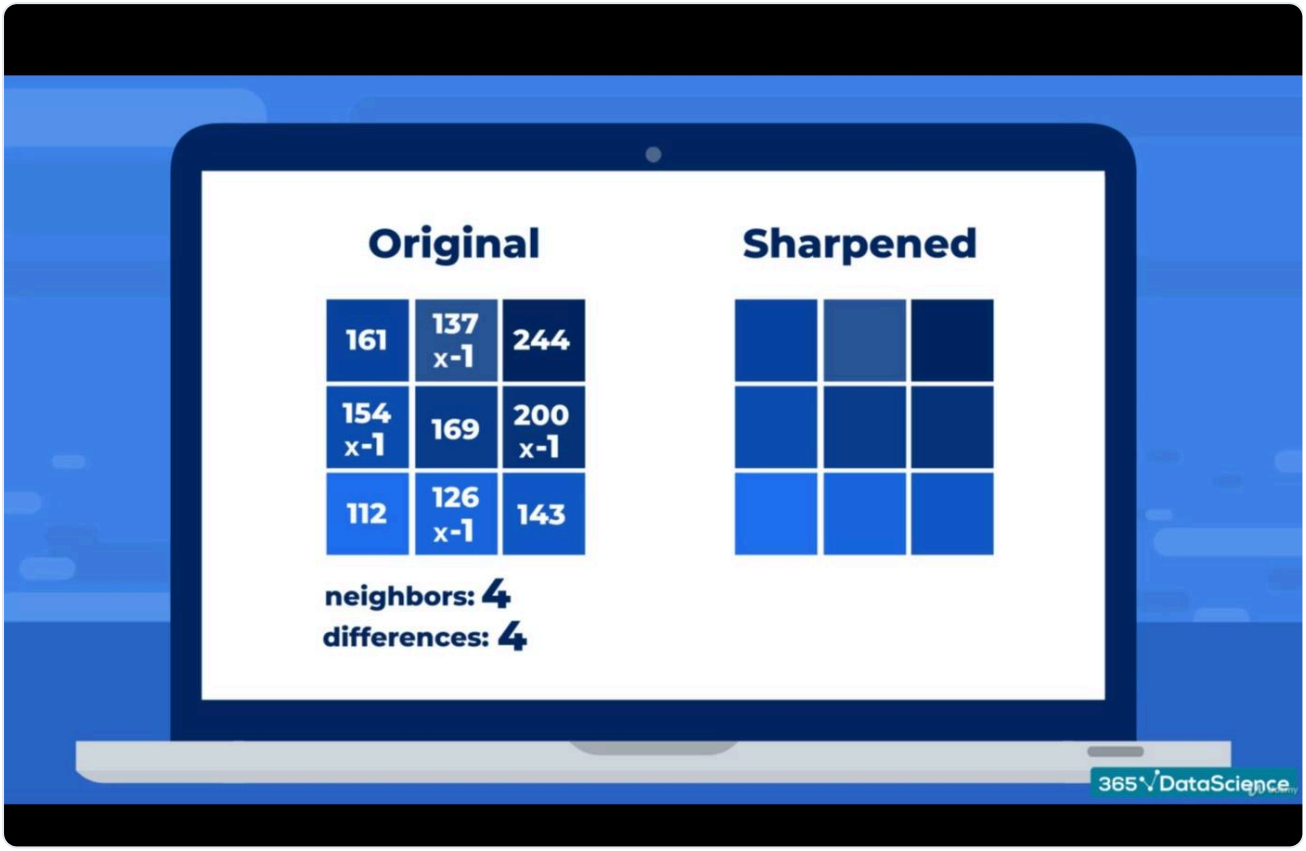

FIGURE 4.23 | Original Sharpened

So what we are going to do is multiply them by minus one to signify this. Since we have four neighbors, then we have four differences. So we need four copies of the original pixel.

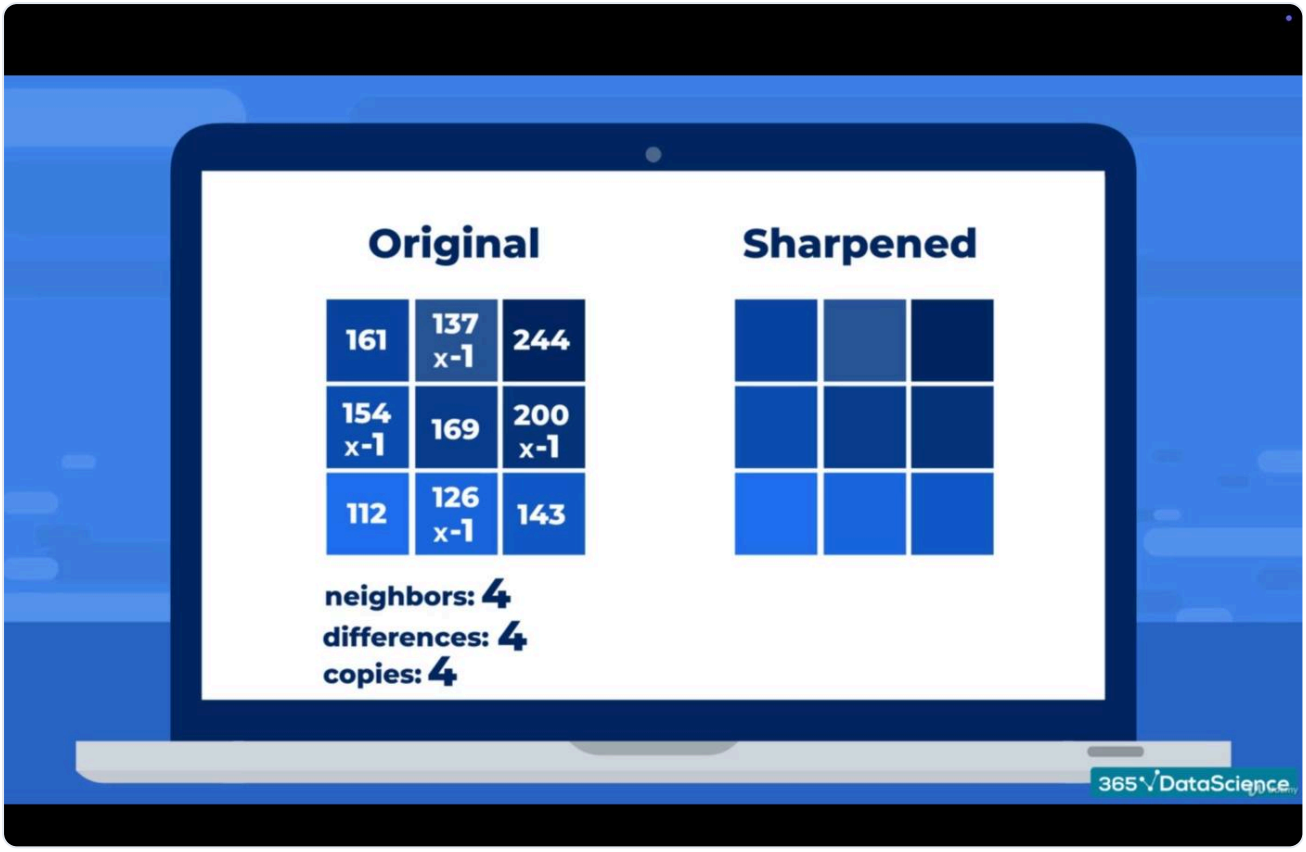

FIGURE 4.24 | Original Sharpened

So far we've accounted for the differences. Apart from that, we also need to add the original pixel itself. This results in multiplying the center value by a total of five.

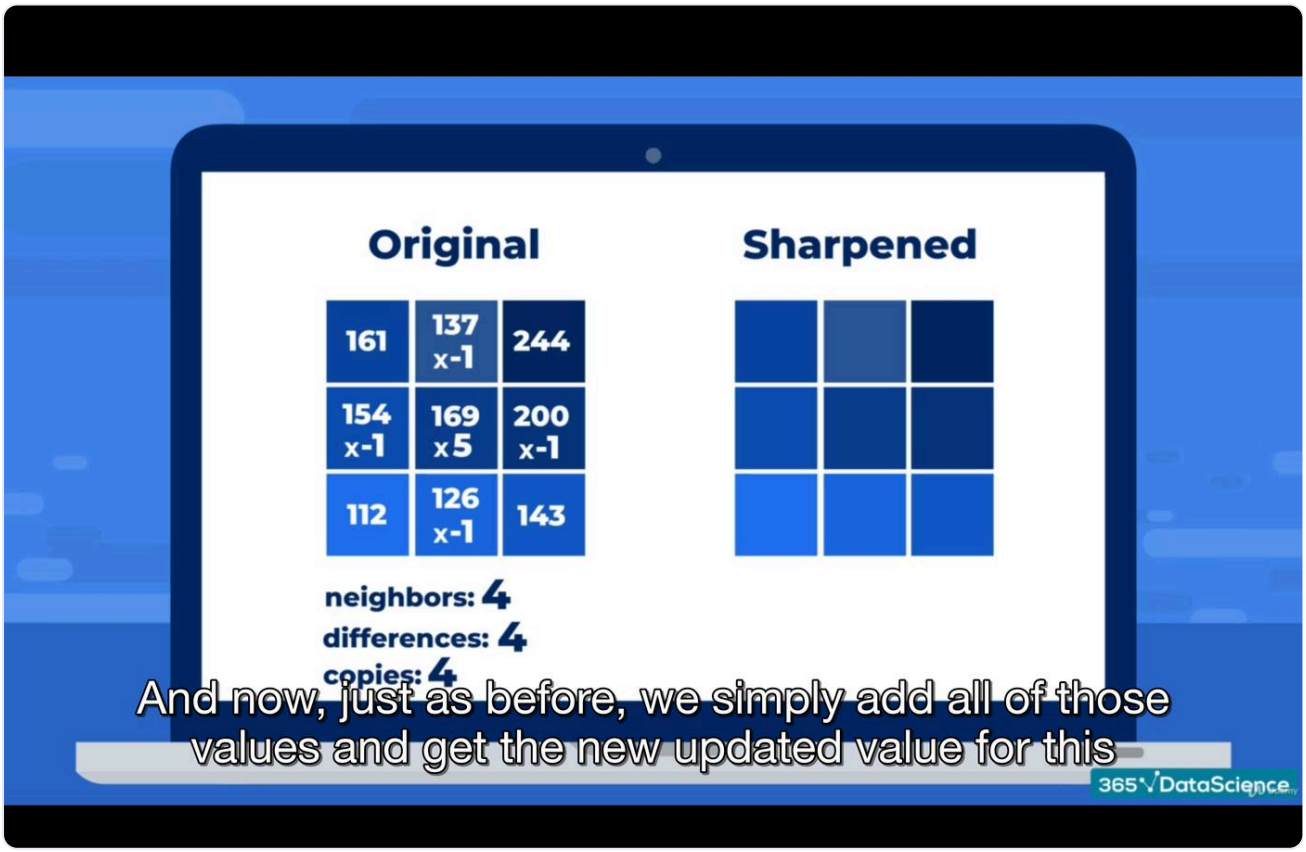

FIGURE 4.25 | Original Sharpened

And now, just as before, we simply add all of those values and get the new updated value for this pixel. I know this may seem a bit daunting, but don't worry about it. Our goal is not to teach you how these specific filters work, but to give you an insight into how filters in general are constructed. It will all become a bit clearer in a couple of lectures. All right.

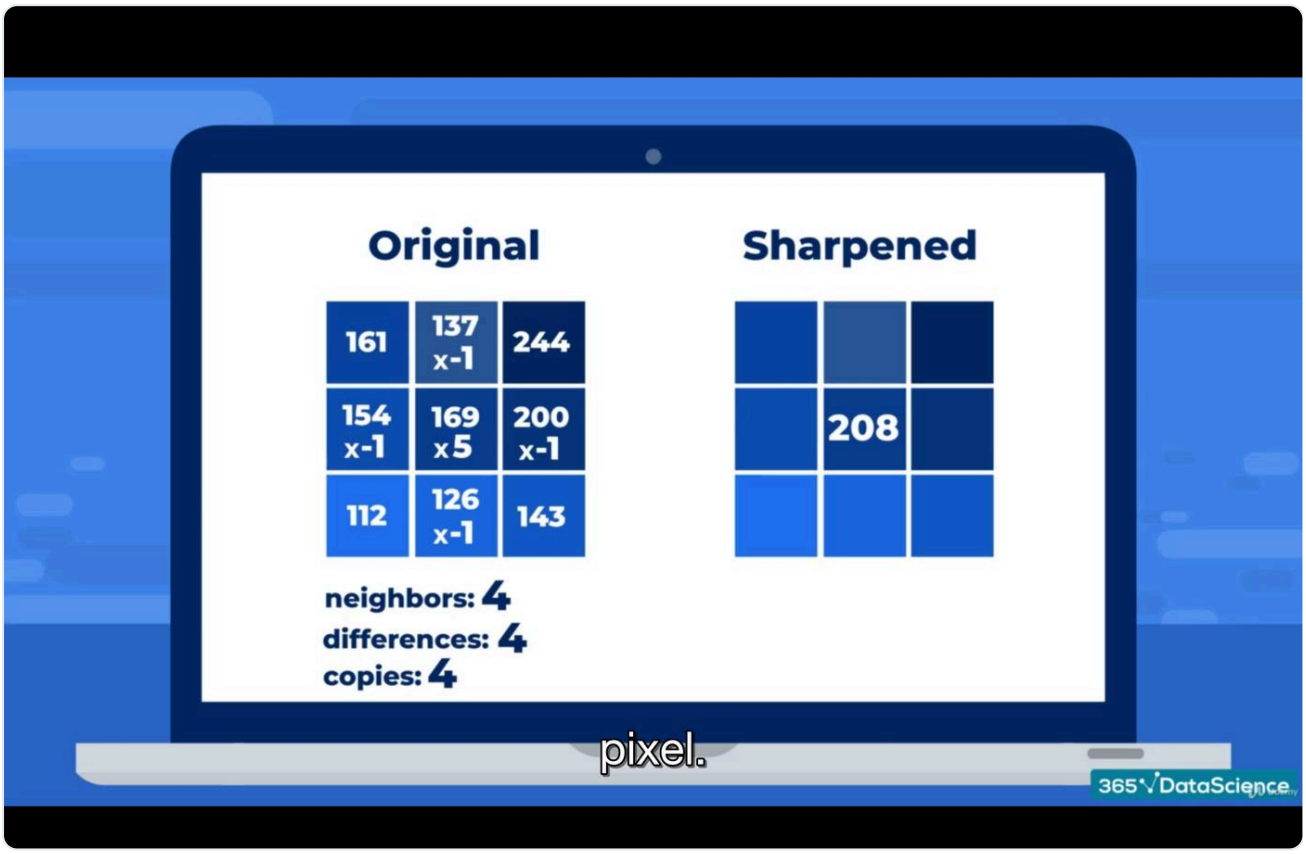

FIGURE 4.26 | Original Sharpened

Finally, let's talk about edge detection. Now, this may give the impression of magic, but actually it is not that difficult to understand this time. It will be easier to work backwards, though. I'm going to show you the solution and then discuss why it works. So let's go back to the pixels image one more time in order to obtain the new value for the center pixel.

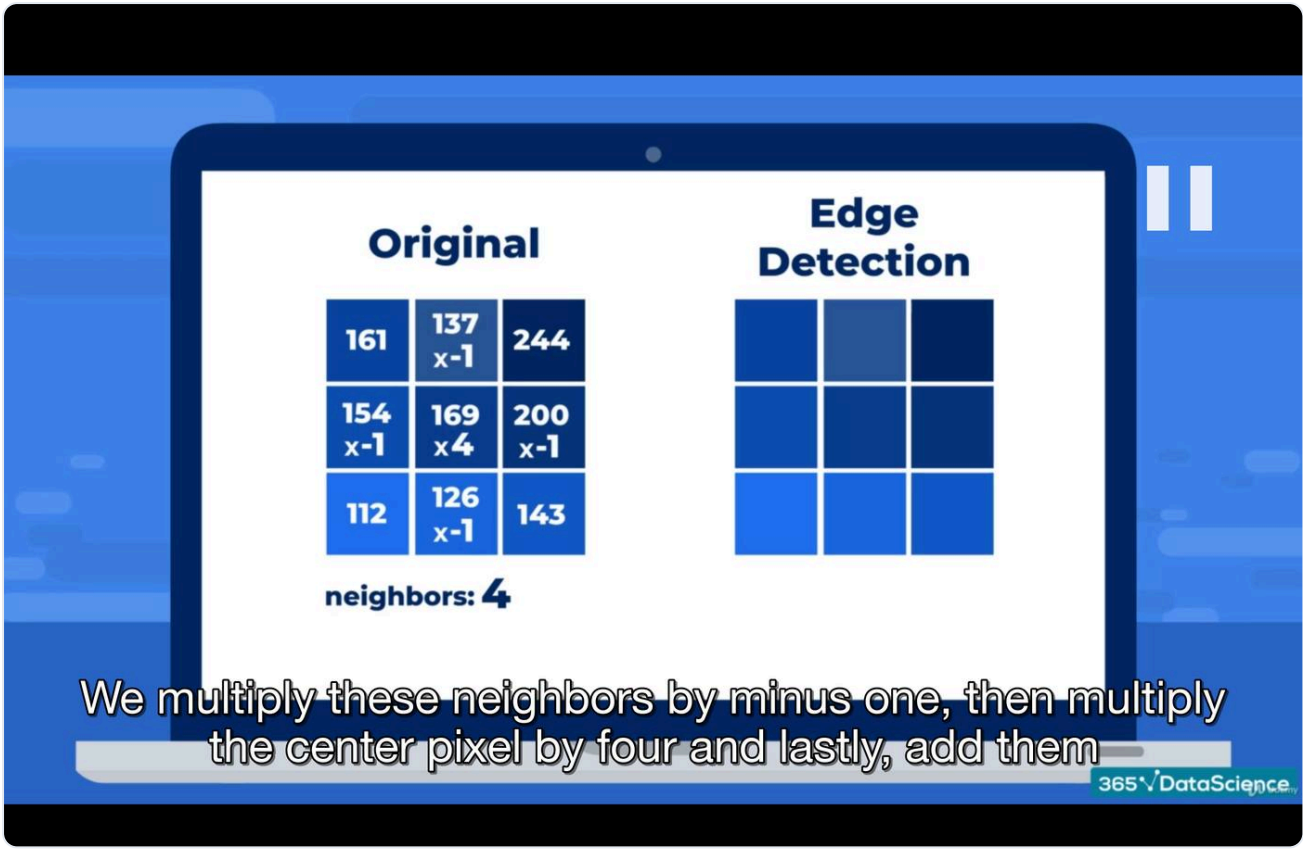

FIGURE 4.27 | Original Detection

We multiply these neighbors by minus one, then multiply the center pixel by four and lastly, add them together.

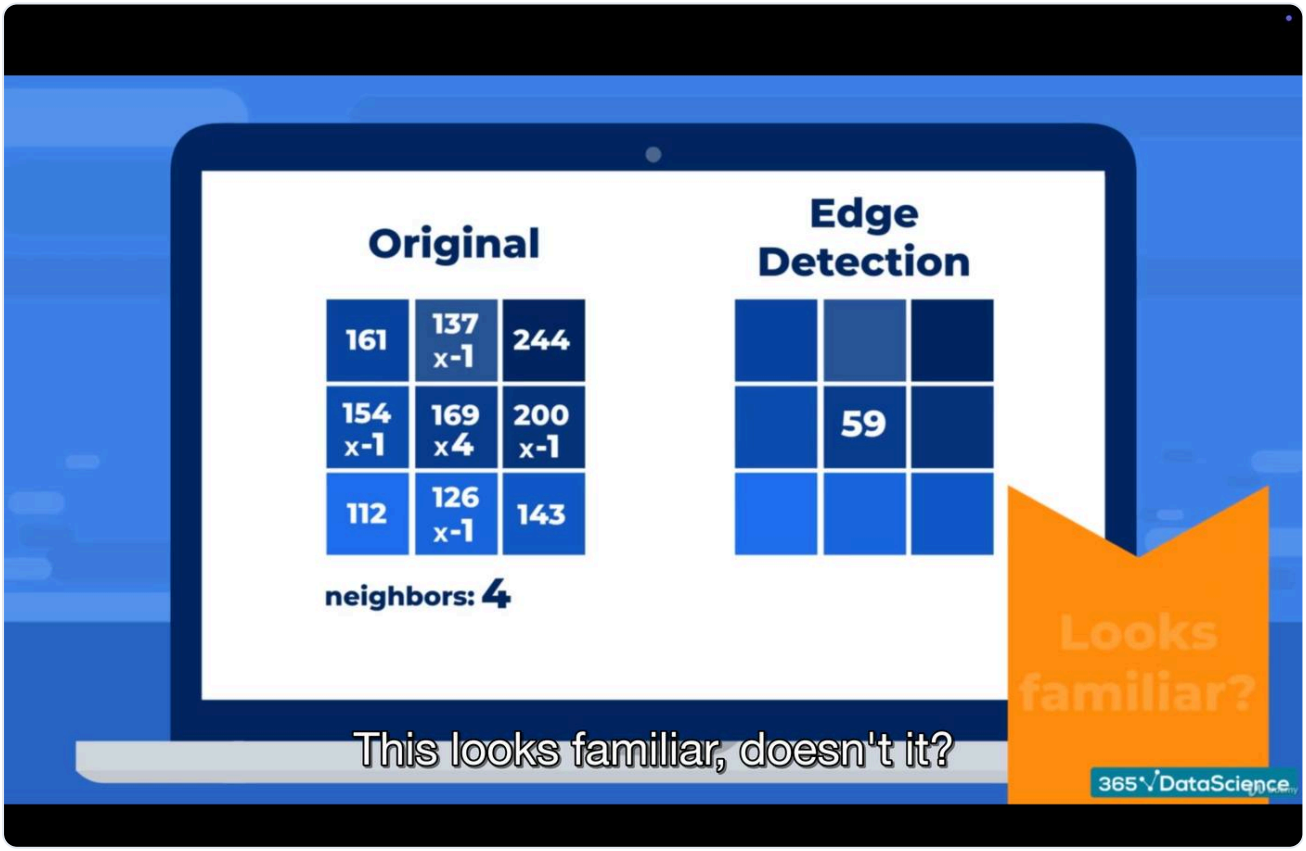

FIGURE 4.28

This looks familiar, doesn't it? That's because it is almost identical to the sharpened filter, with the only difference being that we multiply the center by four instead of five. So why is that? Well, at the end of the day, the purpose of edge detection is to highlight the edges. That is to highlight the portions of the image that have high contrast. In other words, here we want to see only the difference, hence the factor of four. By the way, this is exactly what we were trying to do with sharpening.

FIGURE 4.29 | Edge Detection

What distinguishes the two is that with sharpening, we were adding these edges on top of the original image.

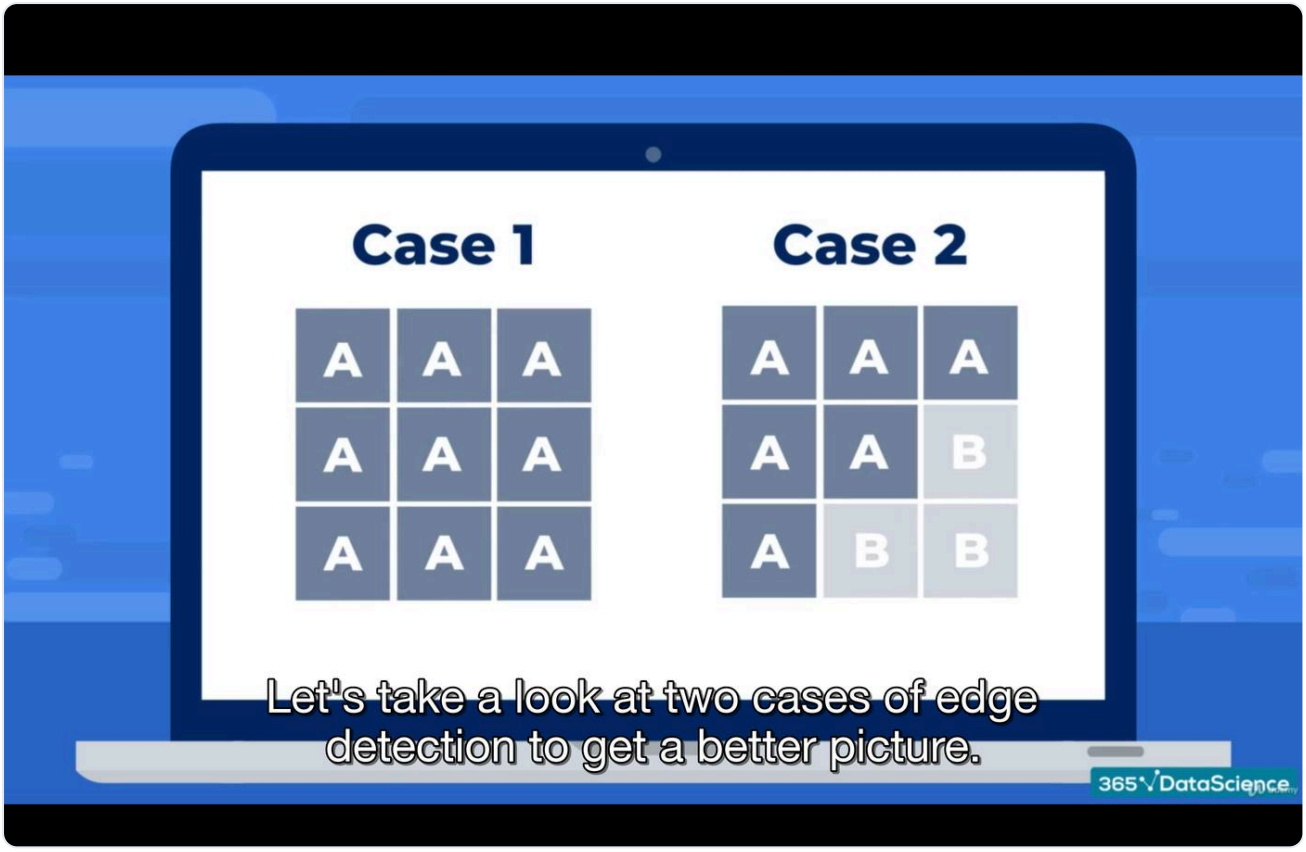

FIGURE 4.30

Let's take a look at two cases of edge detection to get a better picture.

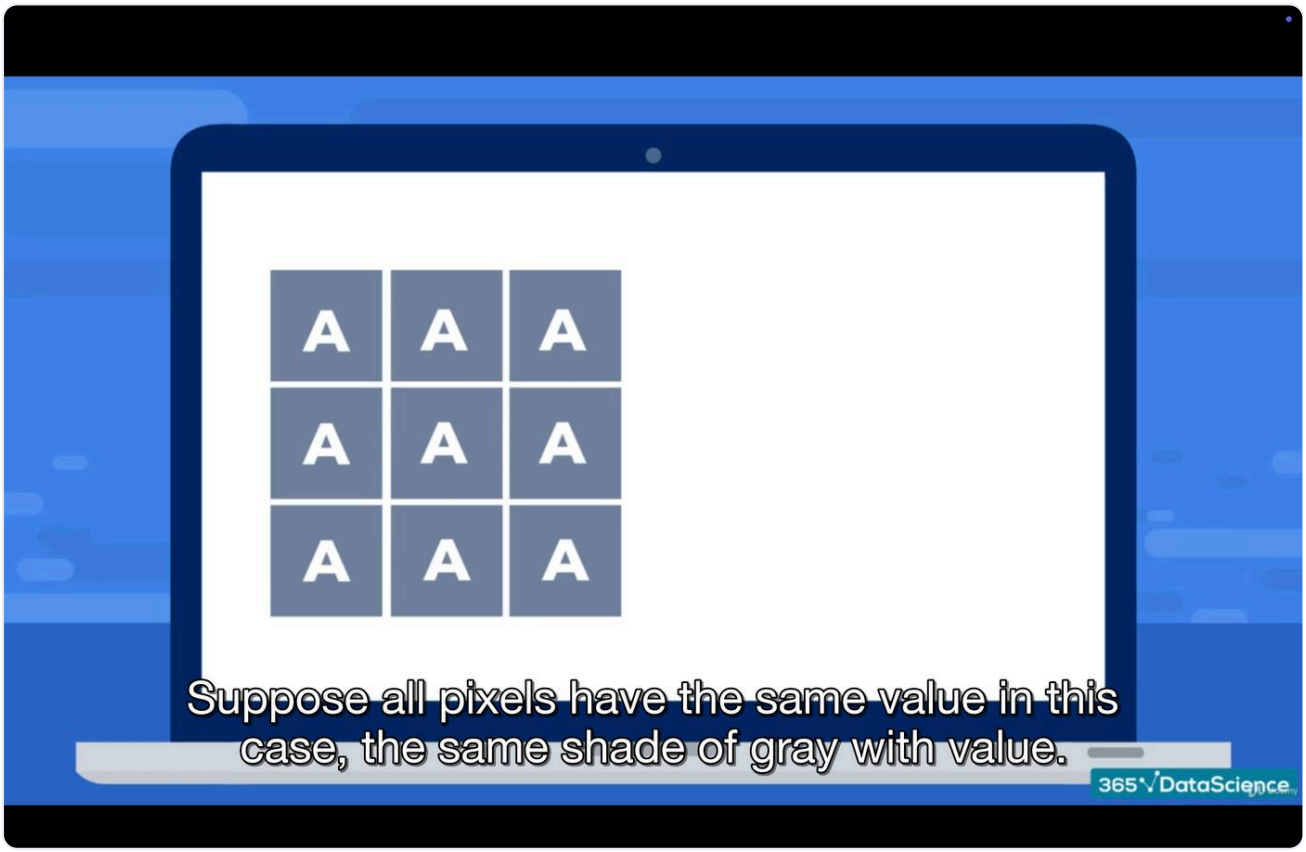

FIGURE 4.31

Suppose all pixels have the same value in this case, the same shade of gray with value. A let's apply the algorithm we discussed.

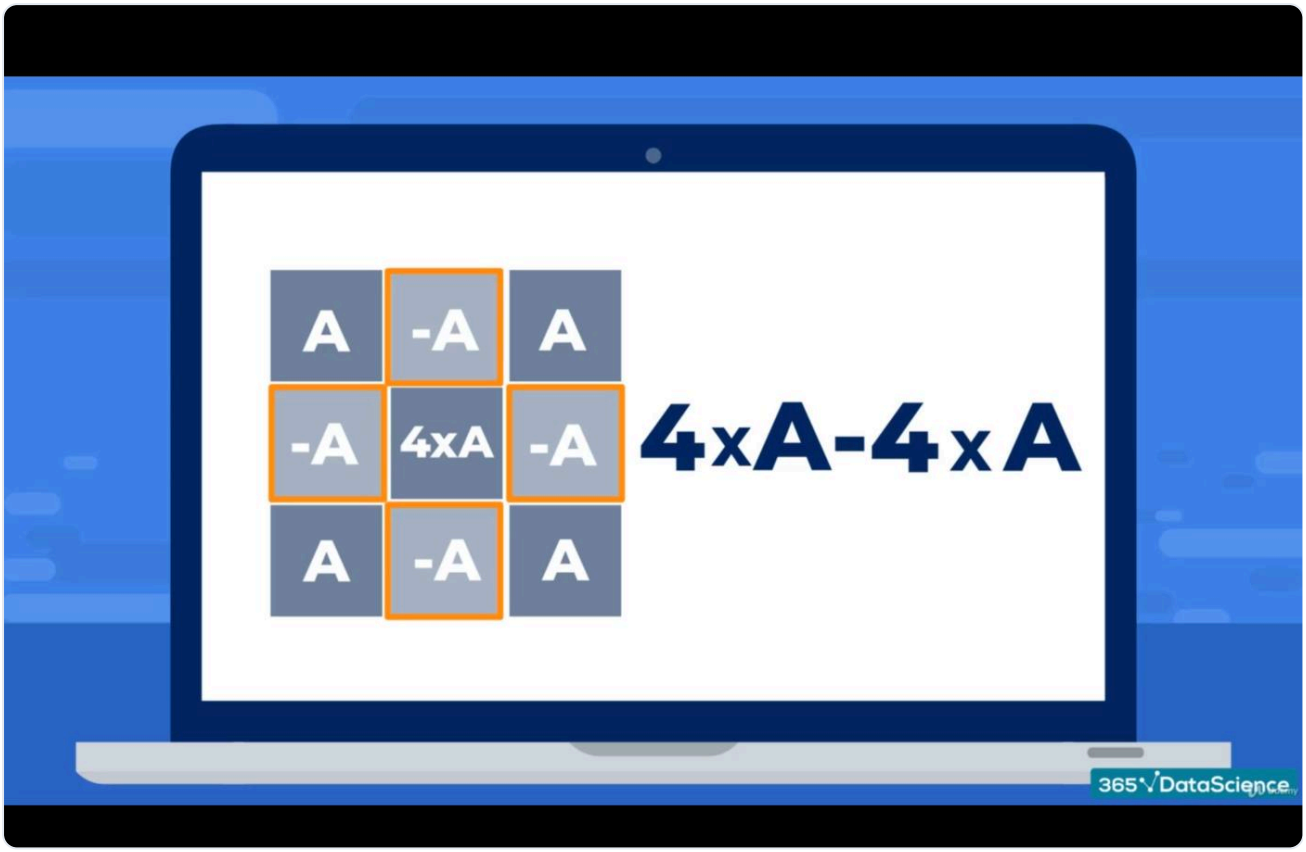

FIGURE 4.32

We take the center value and multiply it by four. That's four times a then we subtract the values of the neighbors.

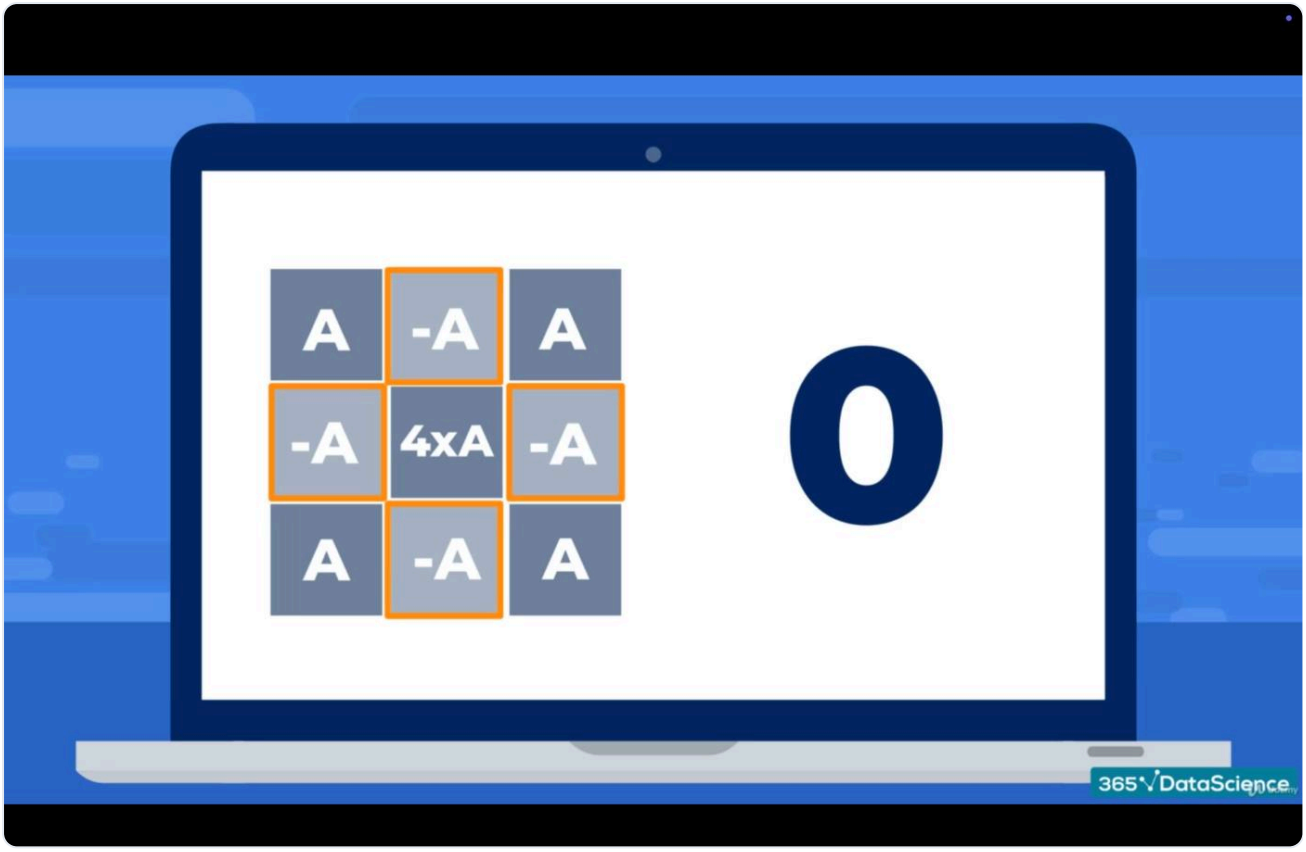

FIGURE 4.33

In this case, that is again four times. So in the end we are left with zero.

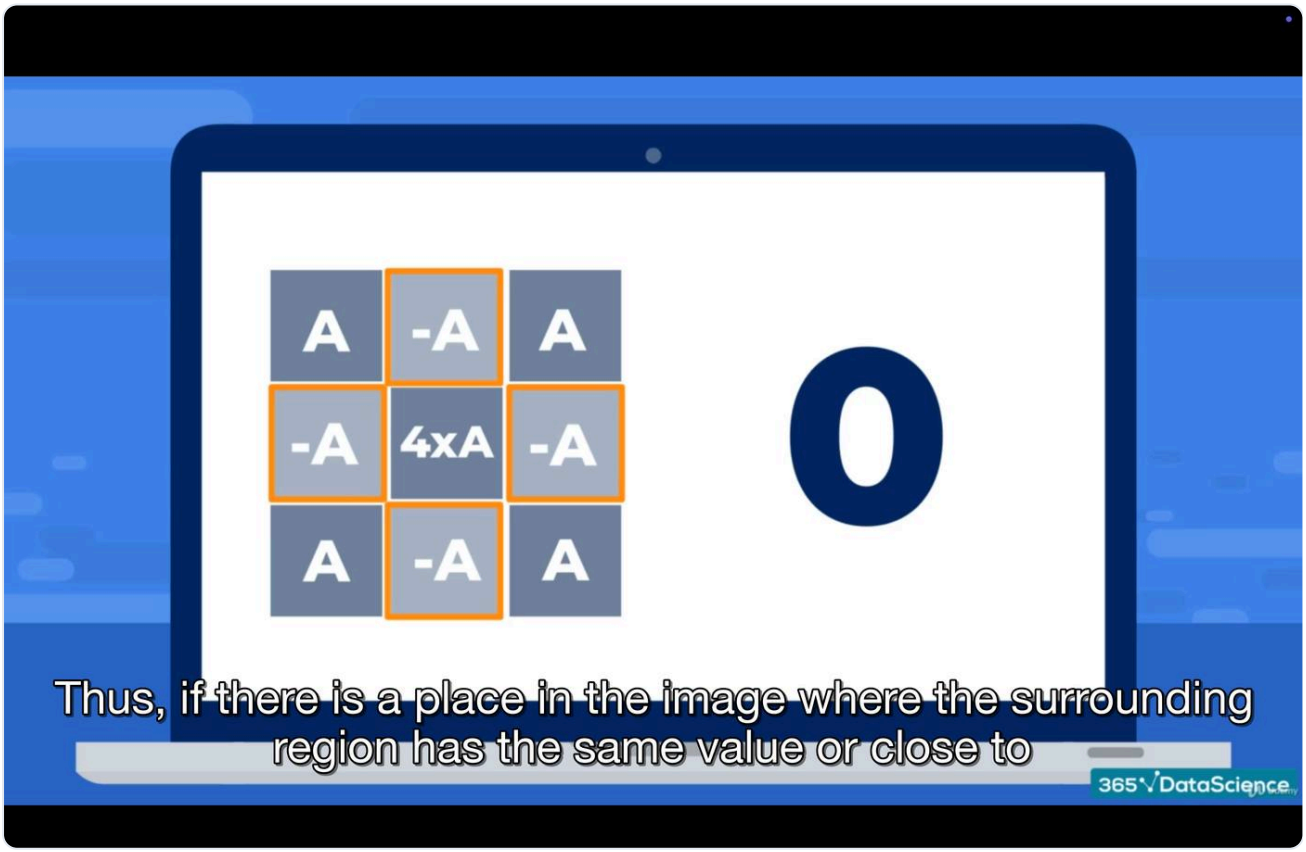

FIGURE 4.34

Thus, if there is a place in the image where the surrounding region has the same value or close to it, that is there is no edge.

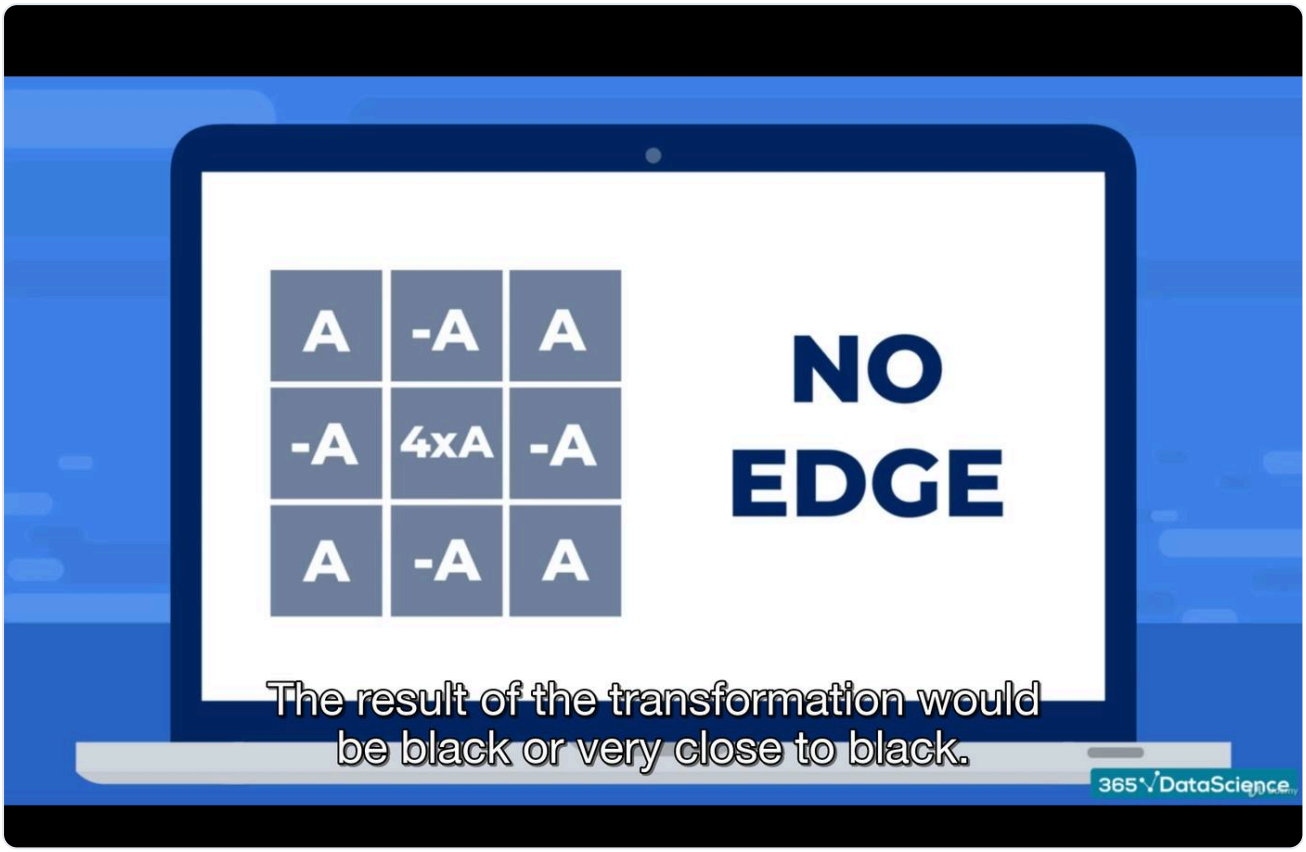

FIGURE 4.35

The result of the transformation would be black or very close to black.

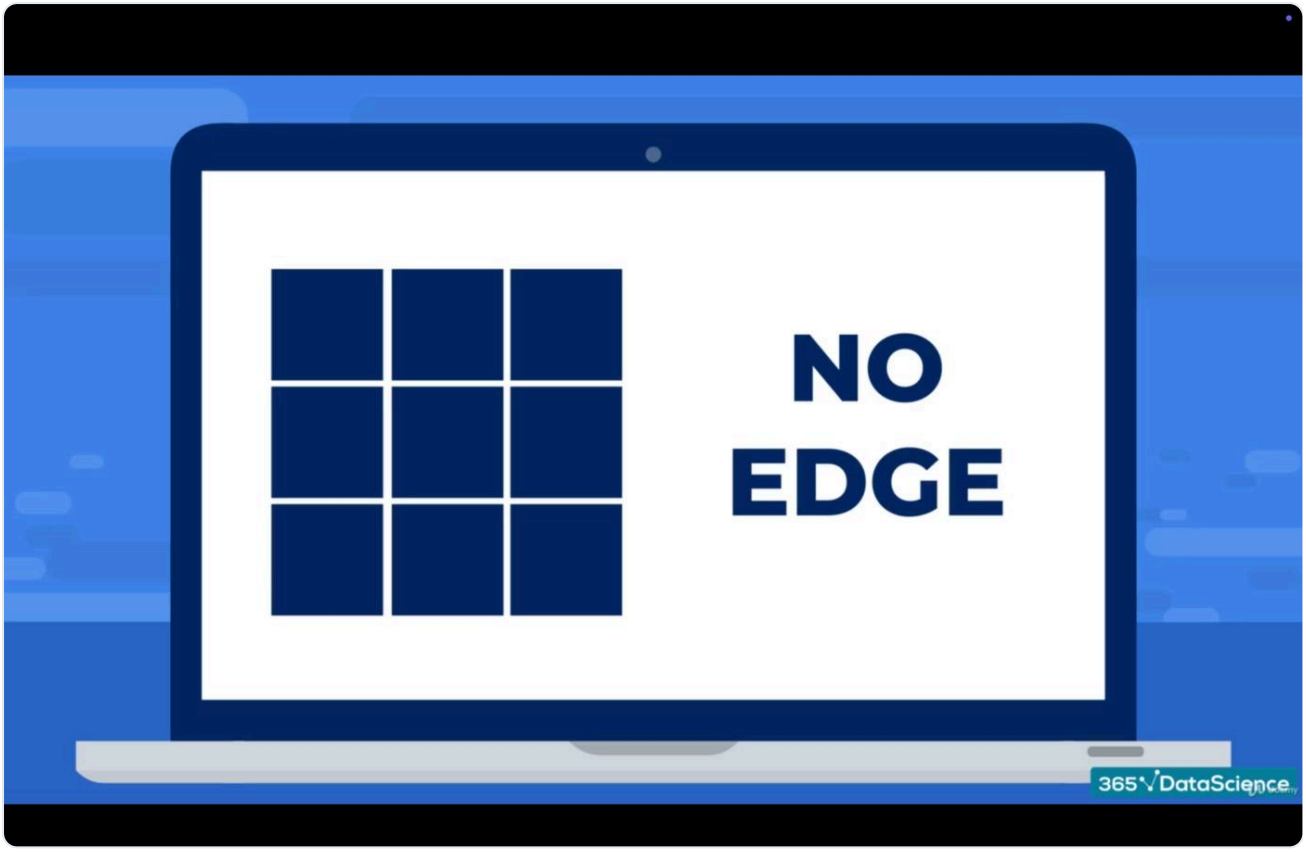

FIGURE 4.36

Now let's look at another situation.

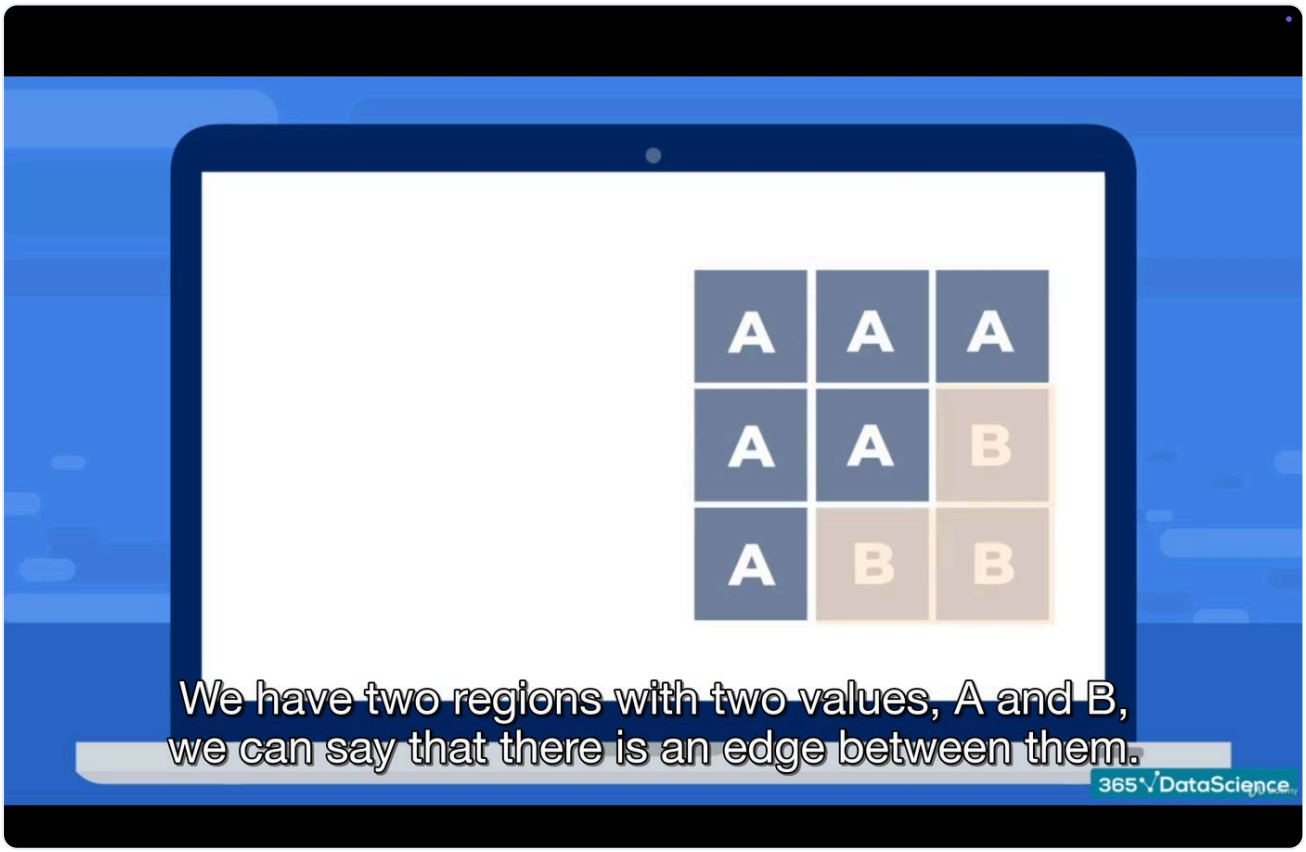

FIGURE 4.37

We have two regions with two values, A and B, we can say that there is an edge between them.

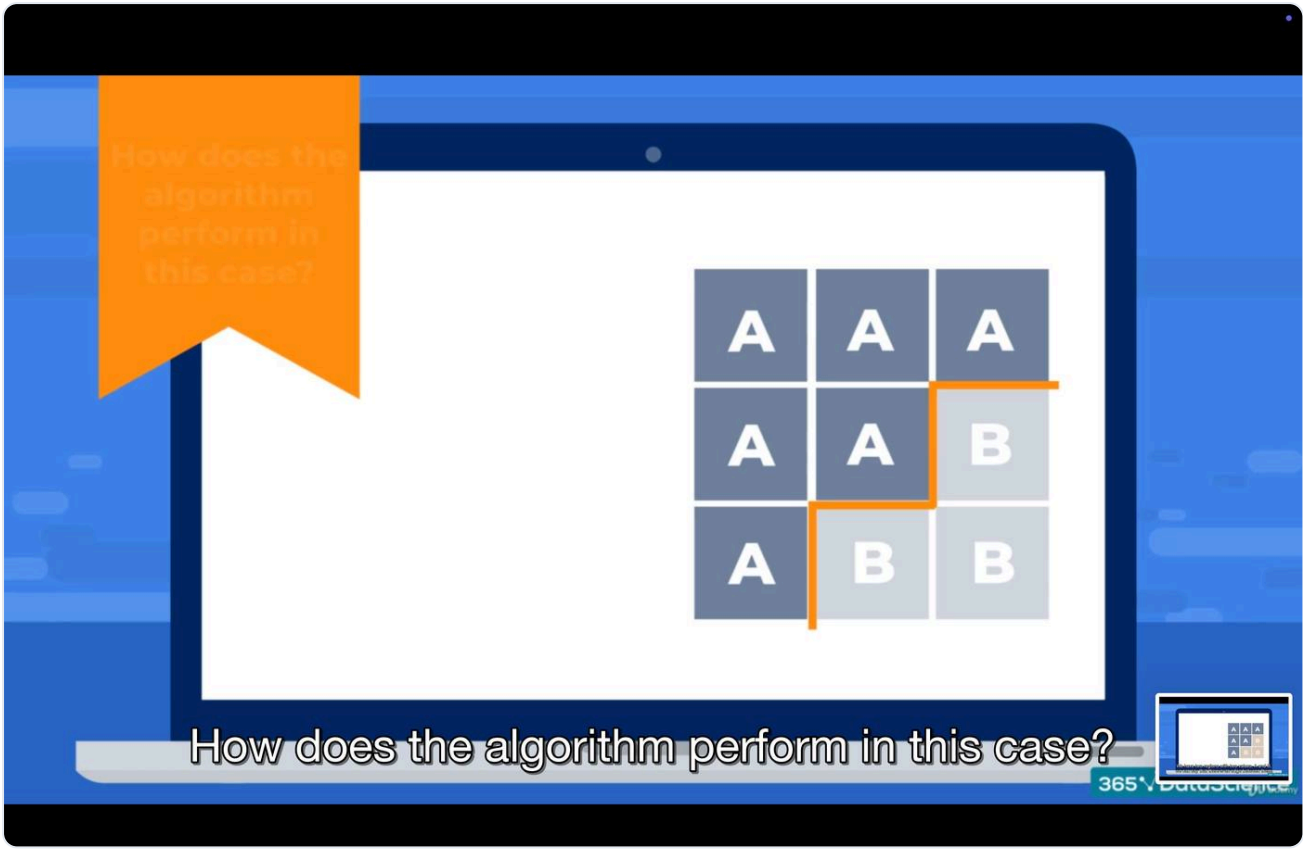

FIGURE 4.38

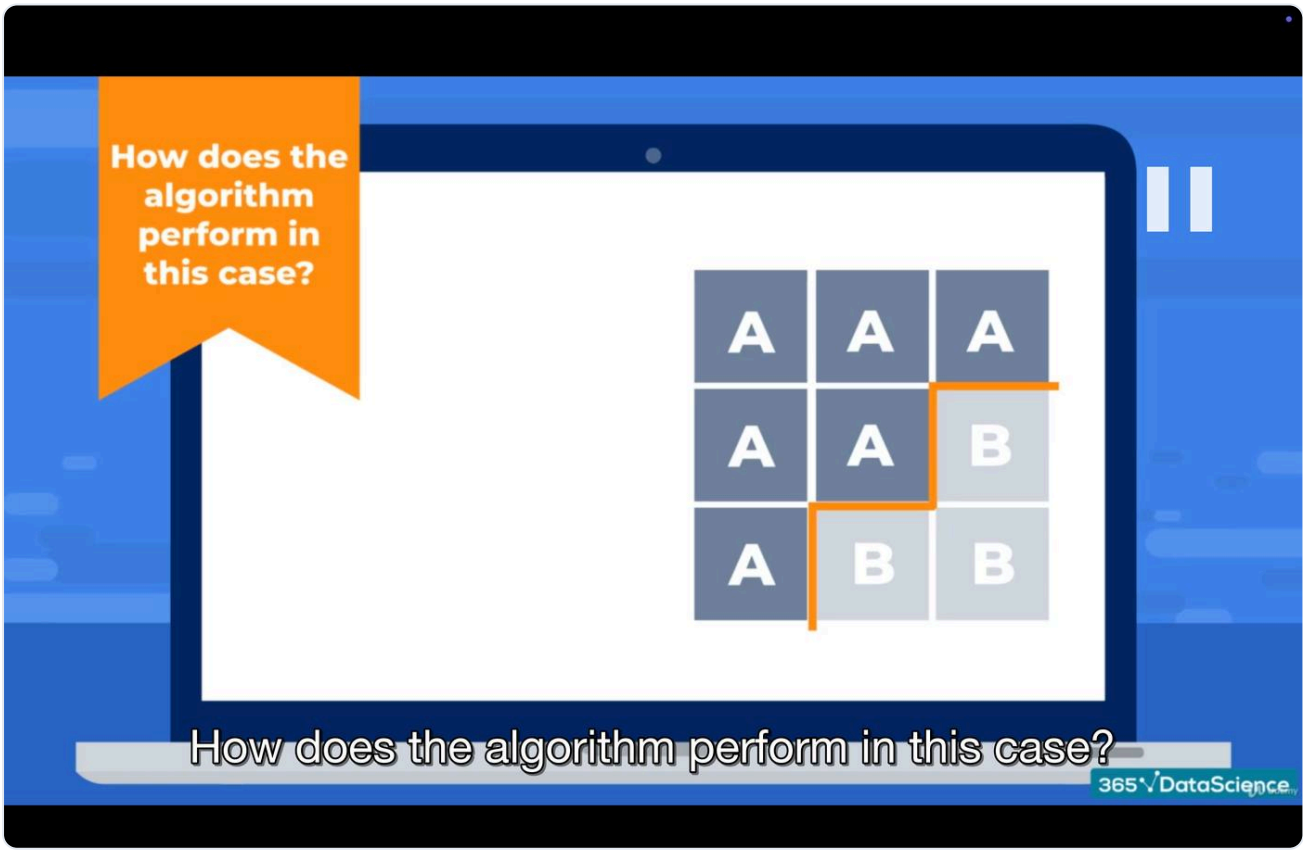

FIGURE 4.39

How does the algorithm perform in this case?

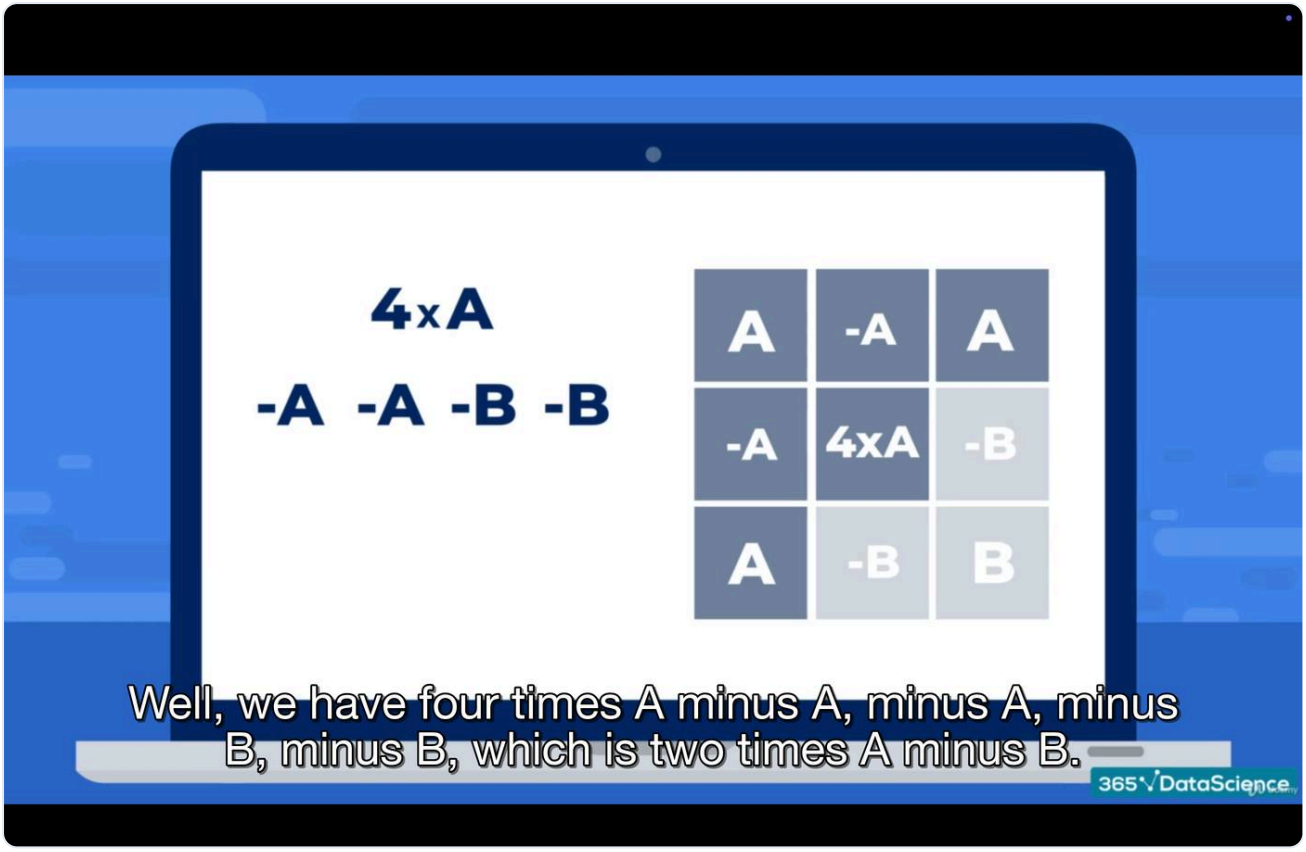

FIGURE 4.40

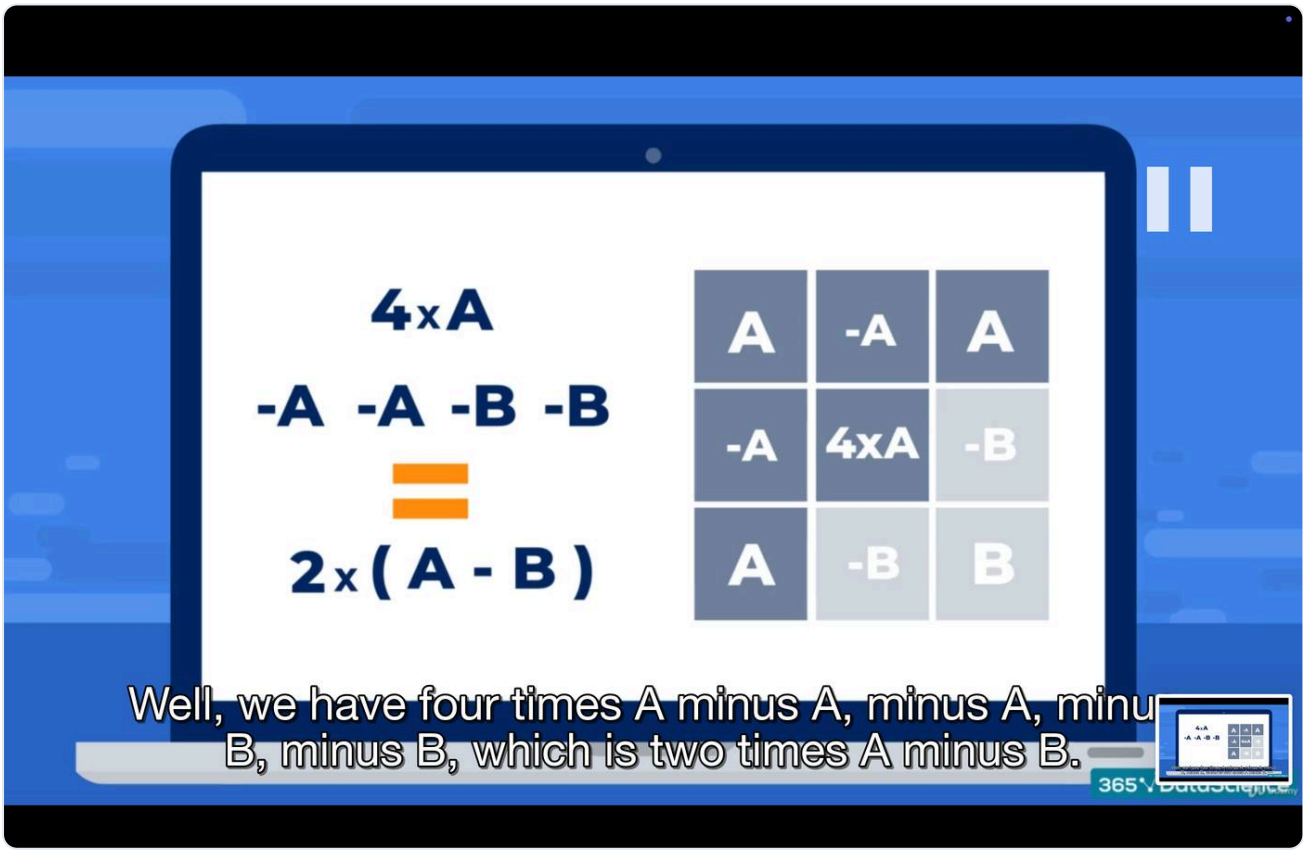

FIGURE 4.41

Well, we have four times A minus A, minus A, minus B, minus B, which is two times A minus B.

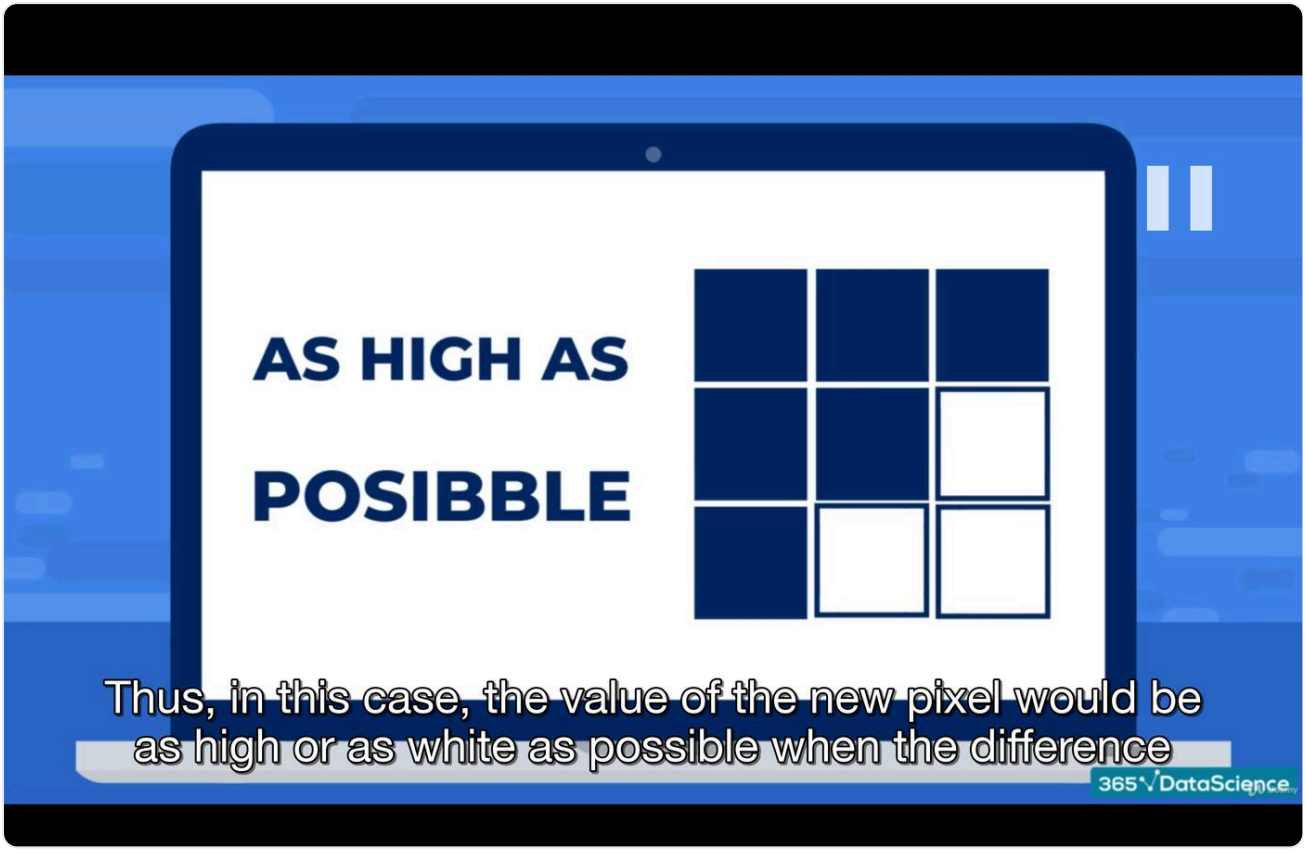

FIGURE 4.42

Thus, in this case, the value of the new pixel would be as high or as white as possible when the difference between A and B is bigger.

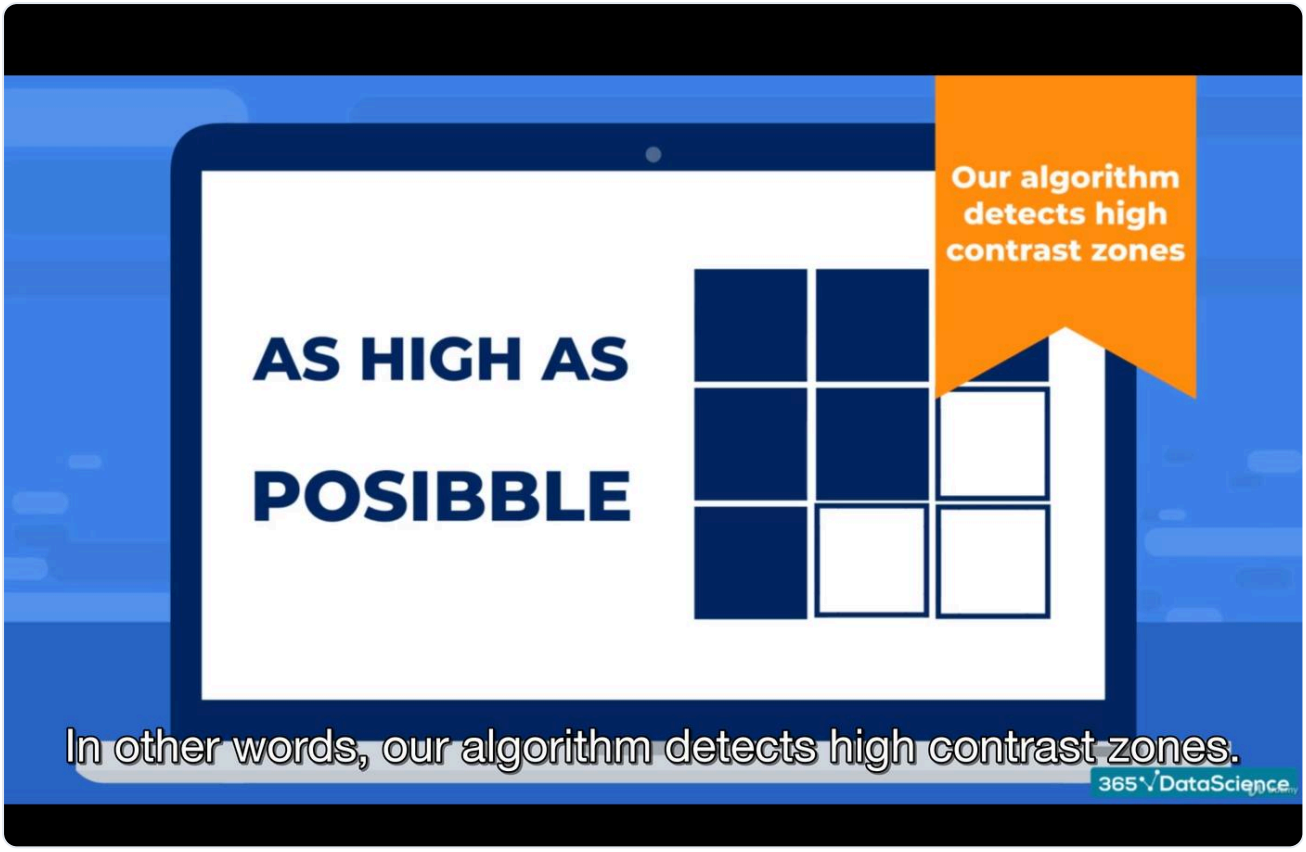

FIGURE 4.43

In other words, our algorithm detects high contrast zones. All right, that's enough about those filters we used in the next lecture. We will connect those to the concept of kernels. Stay tuned. And thanks for watching.

LECTURE 5

Blur: Matrix Form

Hello there and welcome to this lecture. Last time we discussed in detail how the three different transformation algorithms work, now we will see how that connects to kernels.

FIGURE 5.1

So far we've seen how filters work by substituting each pixel value with an appropriately weighted average of its value and its neighbors values. We visualize that by having the factors go on top of each pixel. This can be seamlessly expressed in a matrix form. For instance, recall that the blur consisted of factors of only ninths.

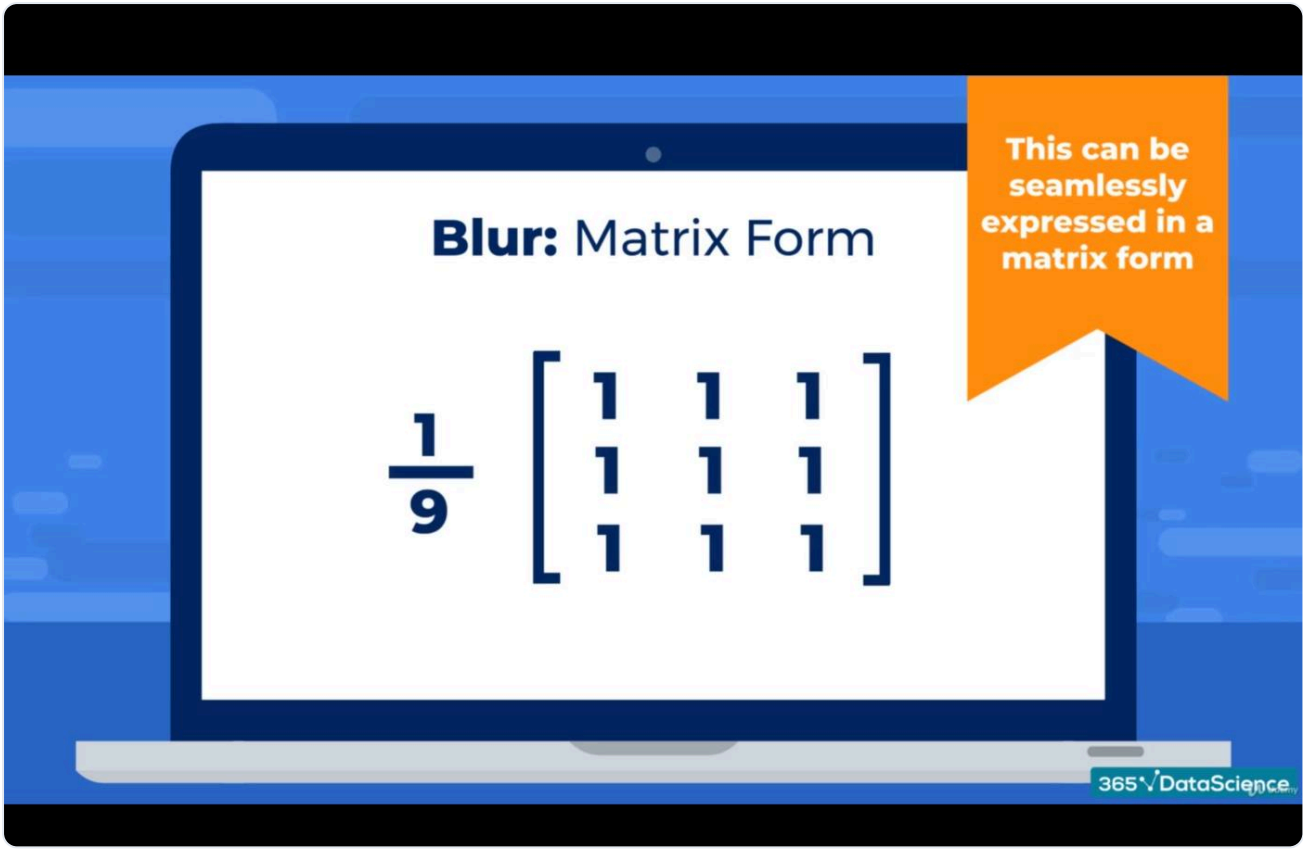

FIGURE 5.2 | Blur: Matrix Form

We can express that with this matrix. Notice that in the Matrix itself, we included only ones while we have the one 9th at the front. This more clearly shows that we take all neighboring pixels with equal importance.

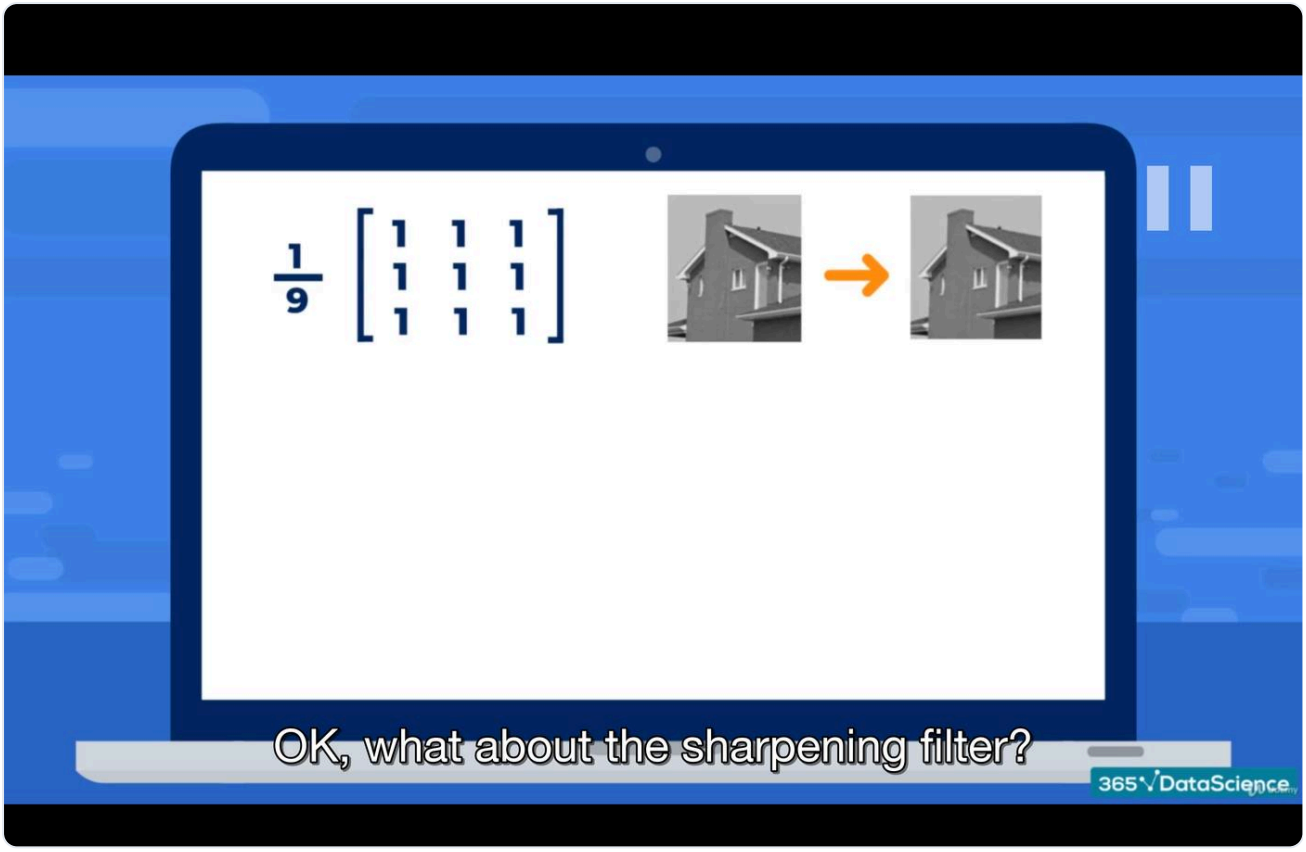

FIGURE 5.3

OK, what about the sharpening filter? Well, it is straightforward. Here is the matrix for it.

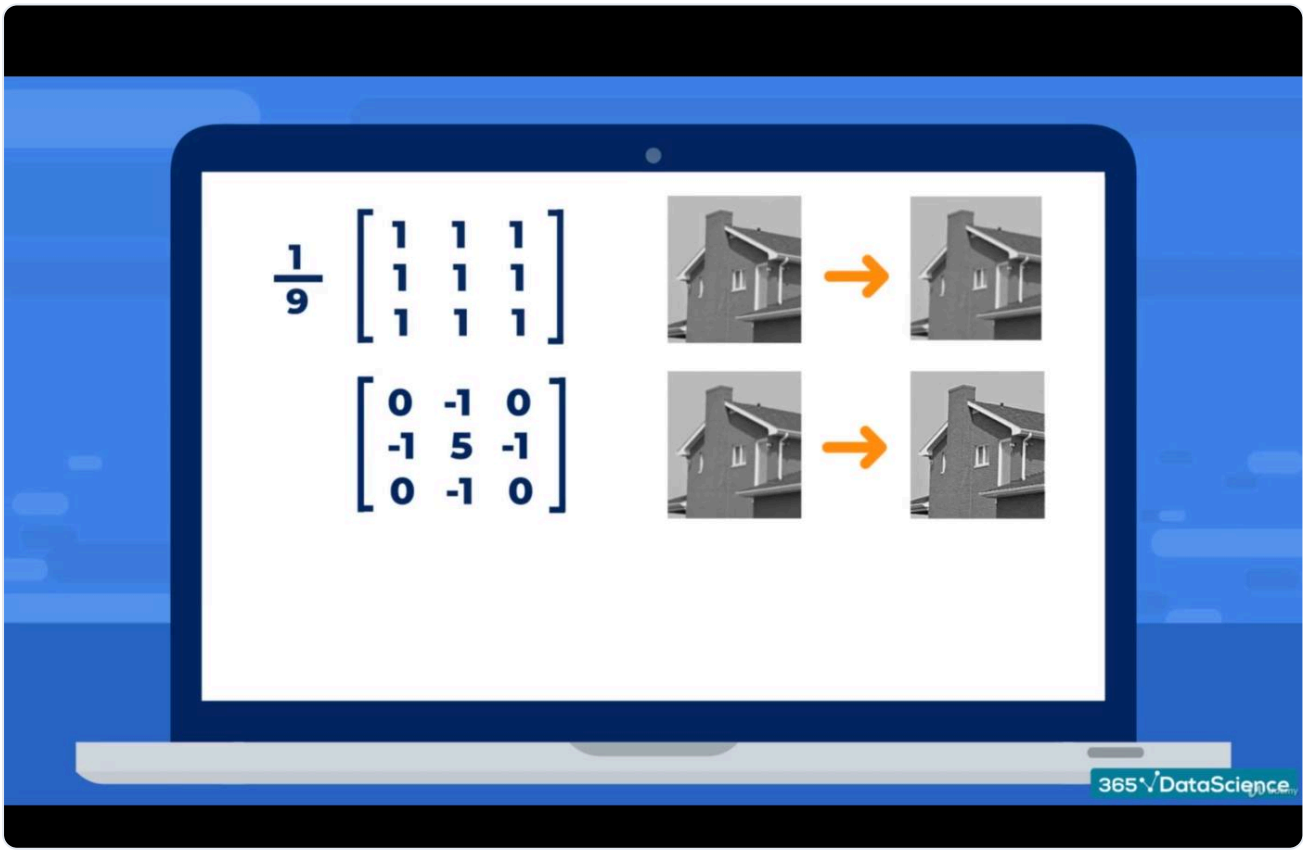

FIGURE 5.4

Just as we discussed, there are four factors of minus one and one factor of five at the center. The rest of the pixels that we don't use are zeros. Finally, here is the matrix for edge detection.

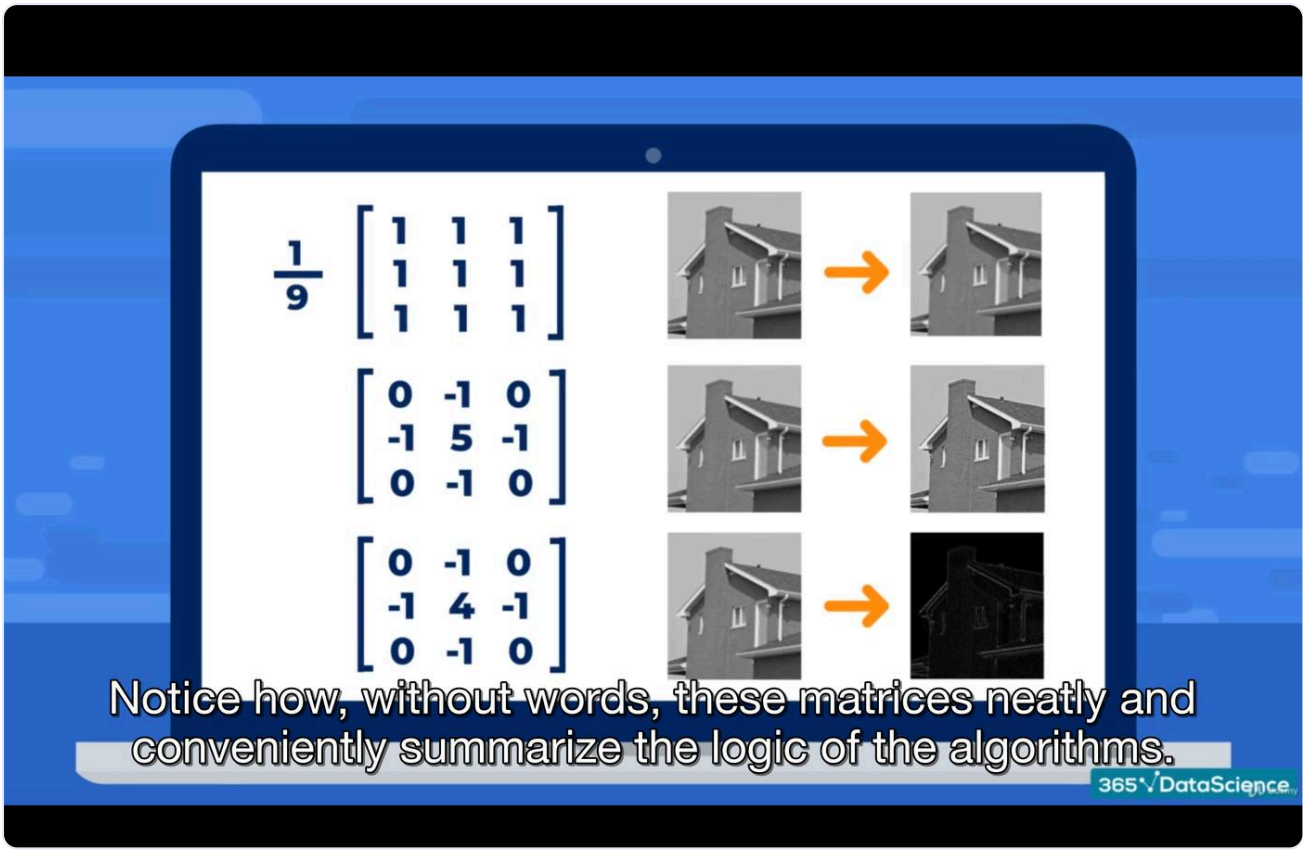

FIGURE 5.5

Notice how, without words, these matrices neatly and conveniently summarize the logic of the algorithms. And that is what we call kernels kernels are the matrices that describe how a certain transformation is to be applied to an image. Kernels are central to image processing because of how well they represent the information it is provided both in a visual way for humans to understand at a glance, as well as in a form optimized for the computer to operate with.

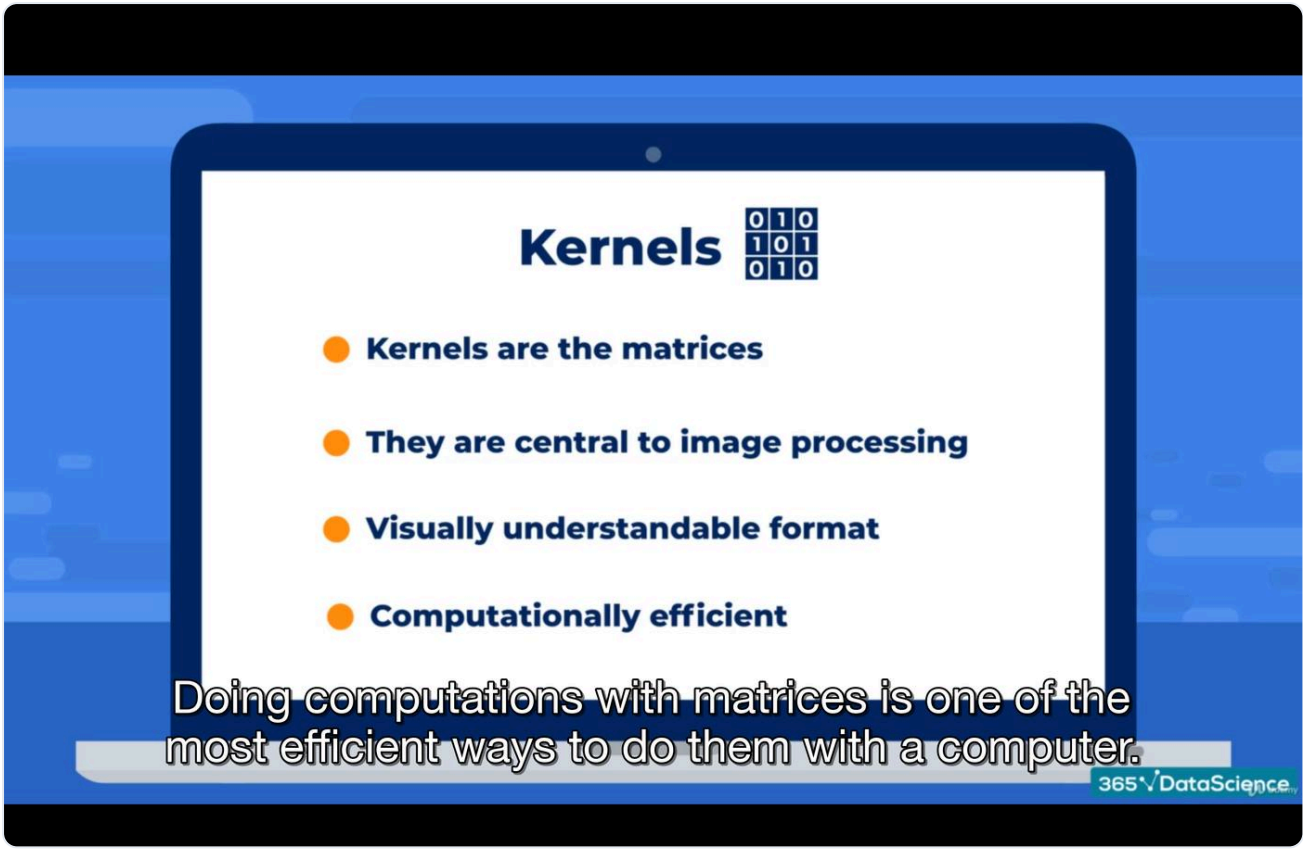

FIGURE 5.6 | Kernels Are the Matrices

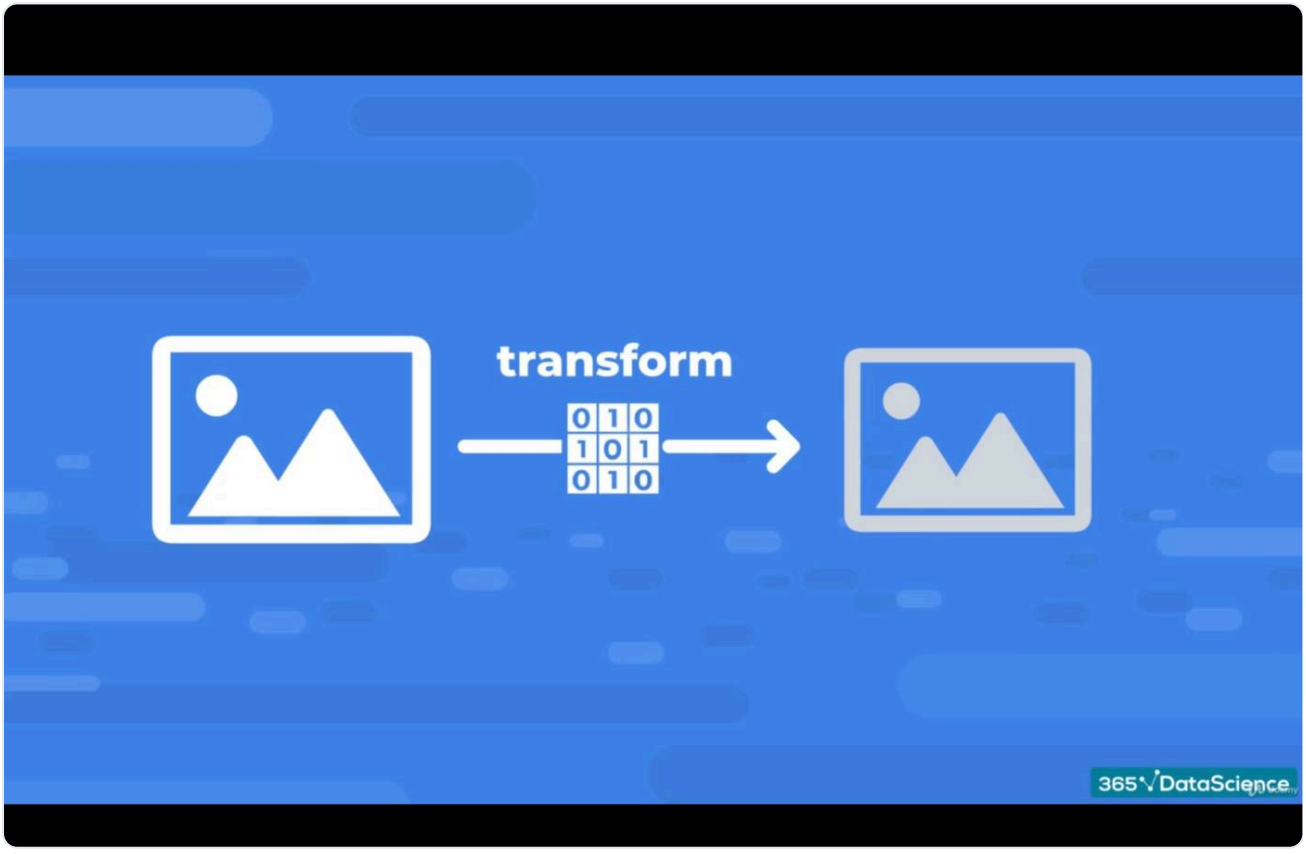

FIGURE 5.7

Doing computations with matrices is one of the most efficient ways to do them with a computer. Great, but what is the relation of kernels to convolutional neural networks? Well, CNN's are usually employed when the data has some structure, such as images. In that case, we need a way to transform the data without losing the underlying structure. As you probably guessed, that is done through the use of kernels. That is why it is important to understand how they work. All right. Tune in the next video where we explain how kernels are applied and why those types of networks are called convolutional. Thanks for watching.

LECTURE 6

Edge Cases

Now we know how to construct the new value of a pixel by combining the old image and placing the kernel over the desired pixel. But how do we create the new image?

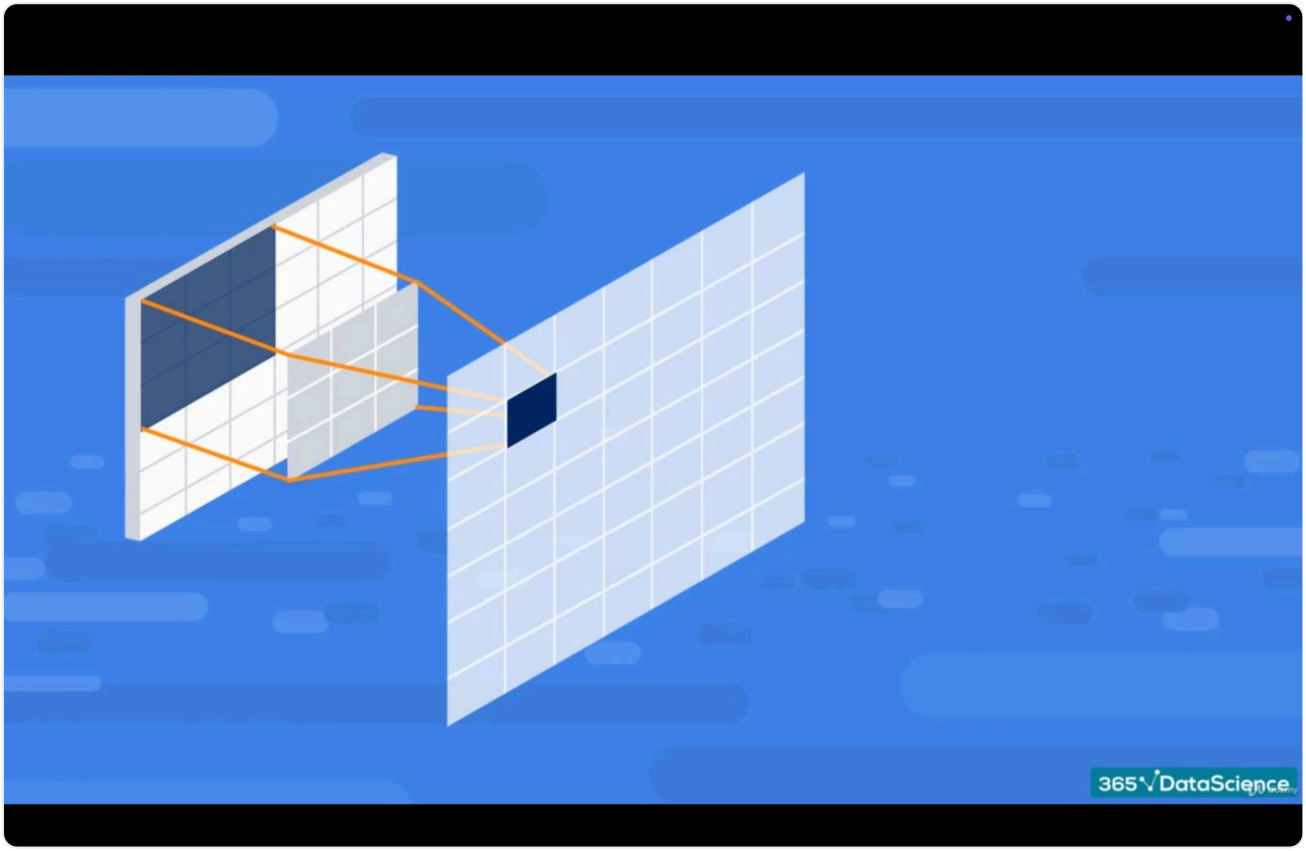

FIGURE 6.1

Well, the answer comes quite naturally.

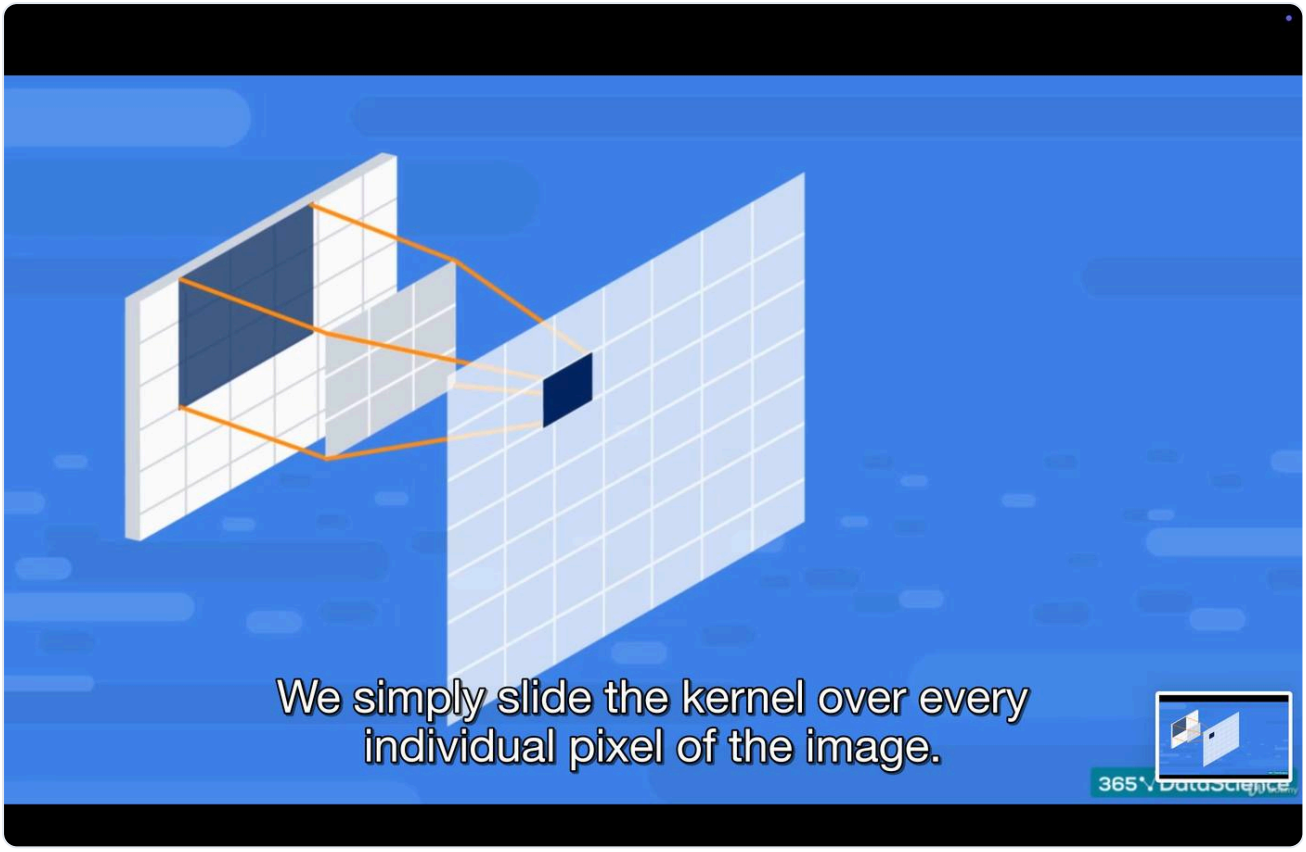

FIGURE 6.2

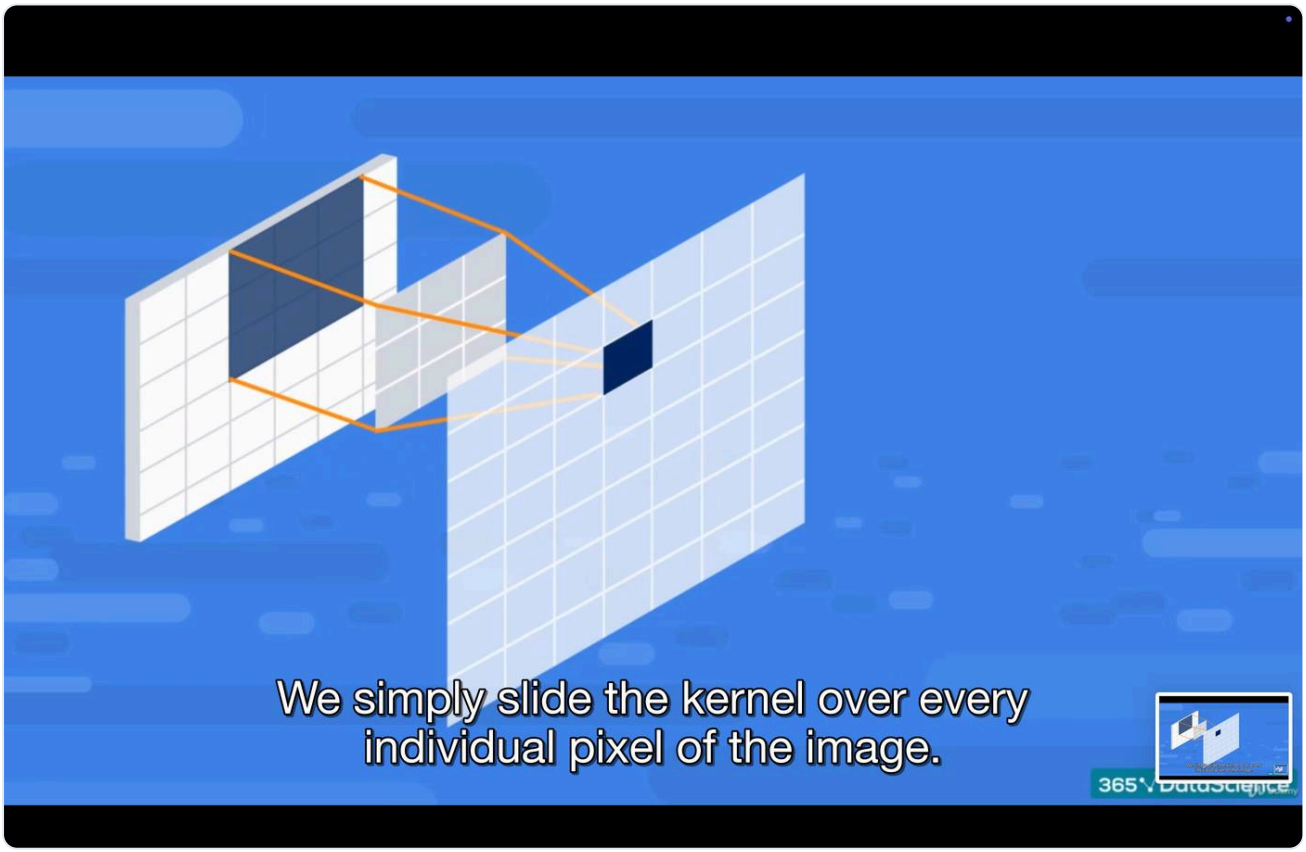

FIGURE 6.3

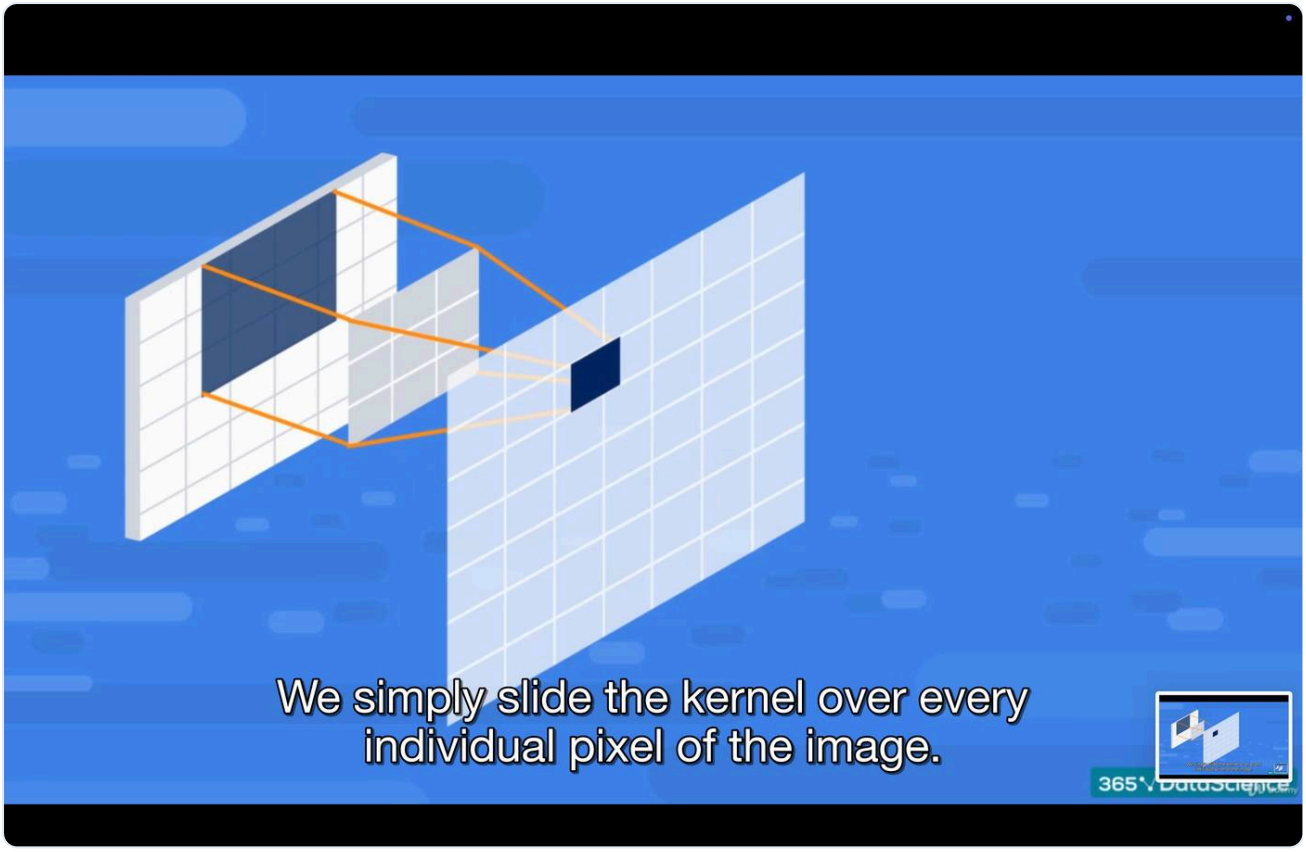

FIGURE 6.4

We simply slide the kernel over every individual pixel of the image. This operation of sliding the kernel over an image and calculating the value for every position is called convolution in computer science.

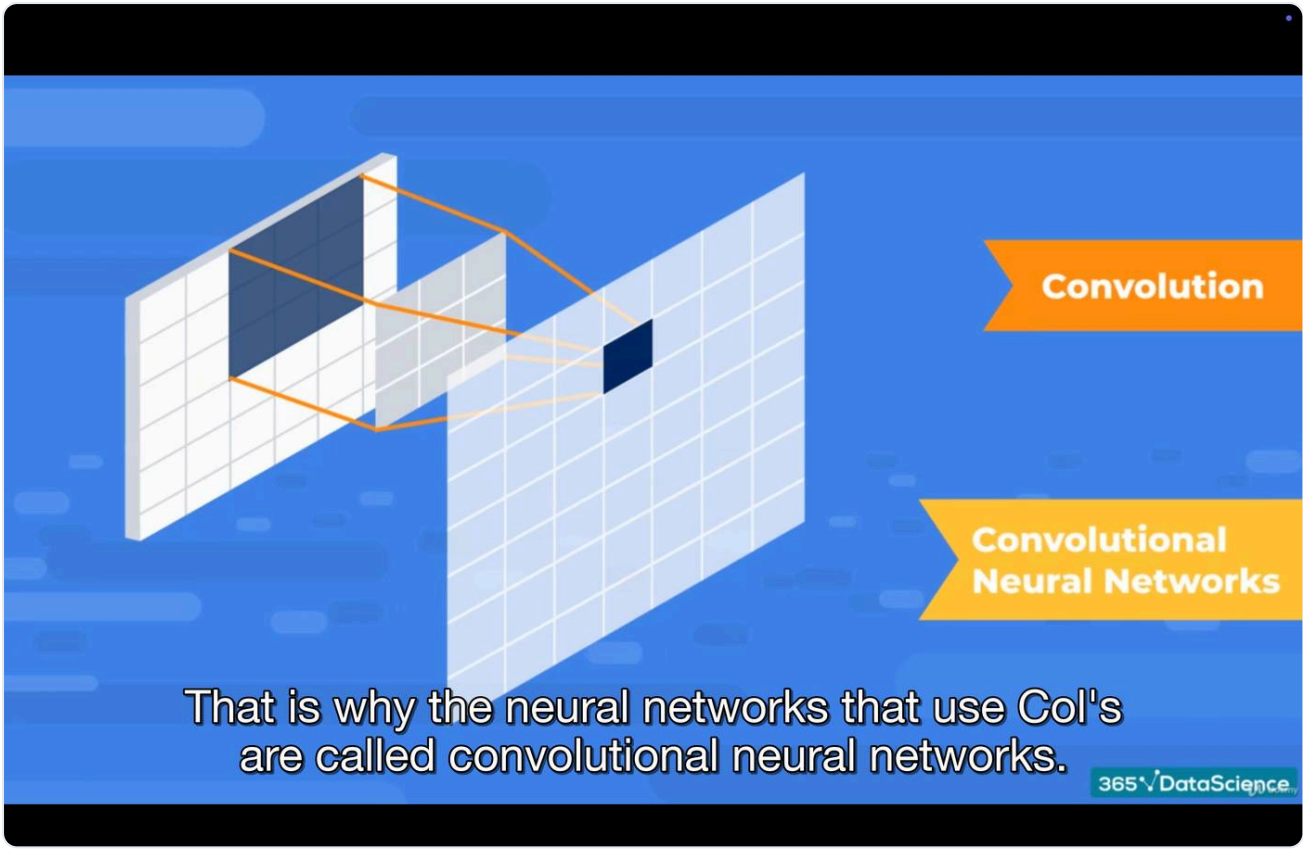

FIGURE 6.5

That is why the neural networks that use kernels are called convolutional neural networks.

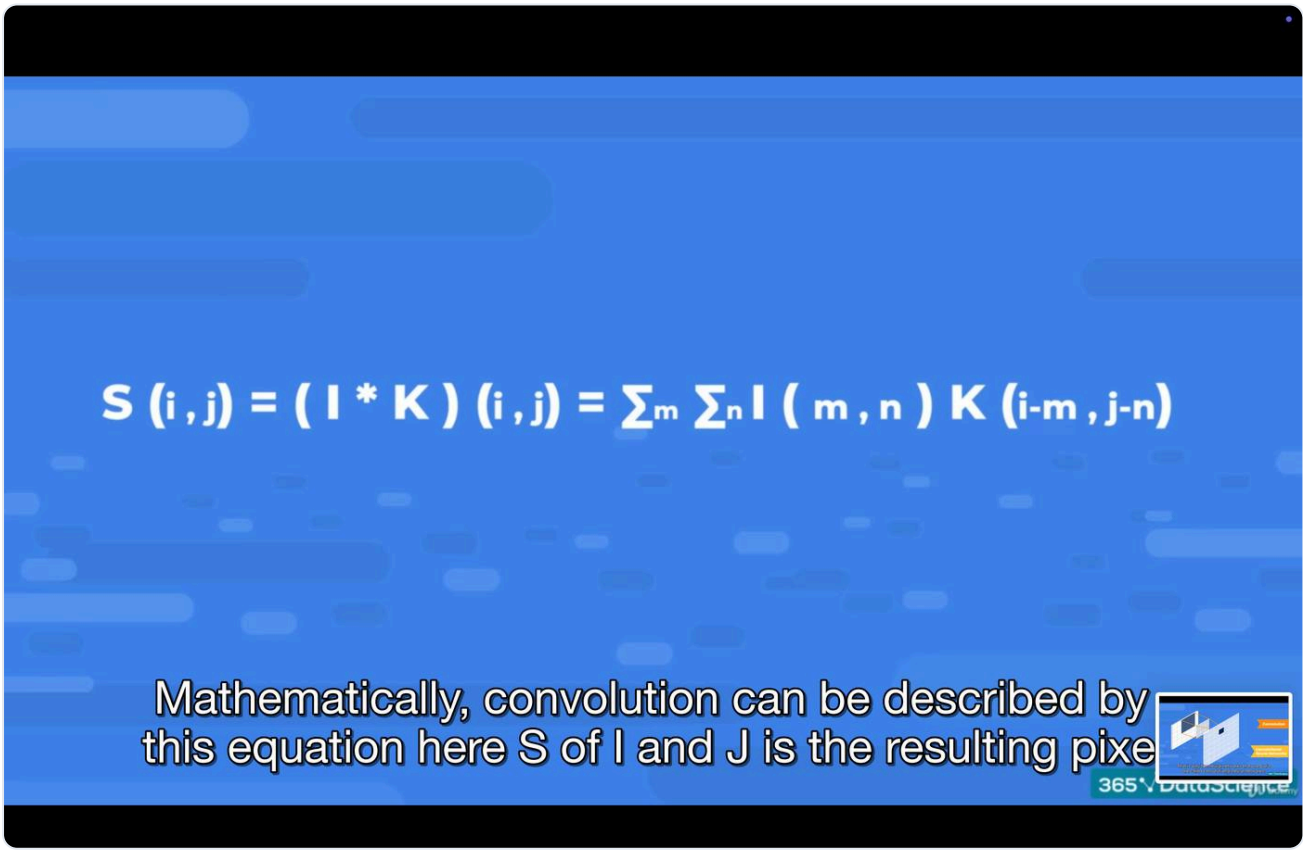

FIGURE 6.6

Mathematically, convolution can be described by this equation here S of I and J is the resulting pixel at position I. GE Capital II is the original image and Capital K is the kernel or filter convolution is described with the star symbol. So that's why we have AI Star K here. Finally, the convolution itself is this double some here.

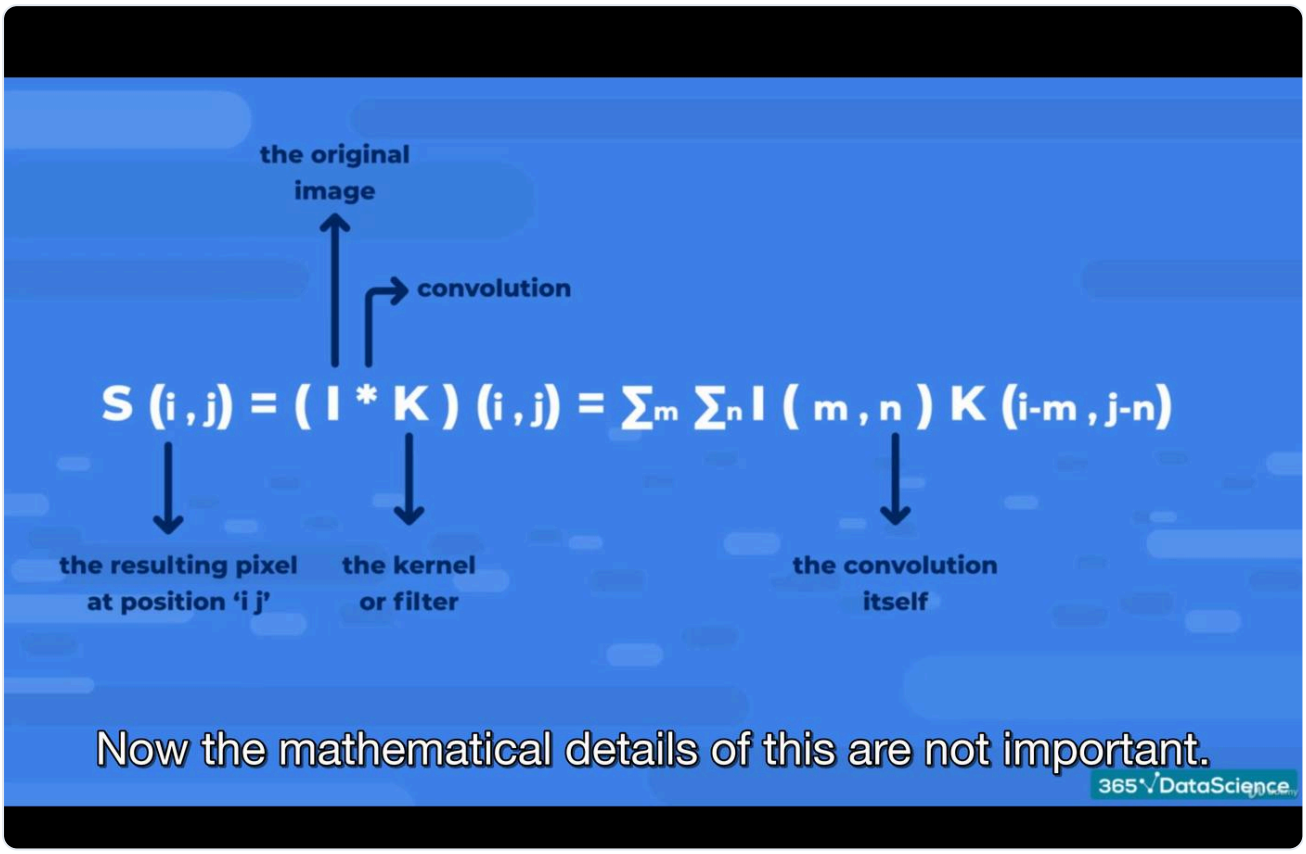

FIGURE 6.7

Now the mathematical details of this are not important. However, some of you probably noticed that the kernel is actually inverted in this equation. That is to say that if we follow this equation closely, it is equivalent to rotating the kernel and only then multiplying and adding the result. And that is true. Convolution flips the kernel around.

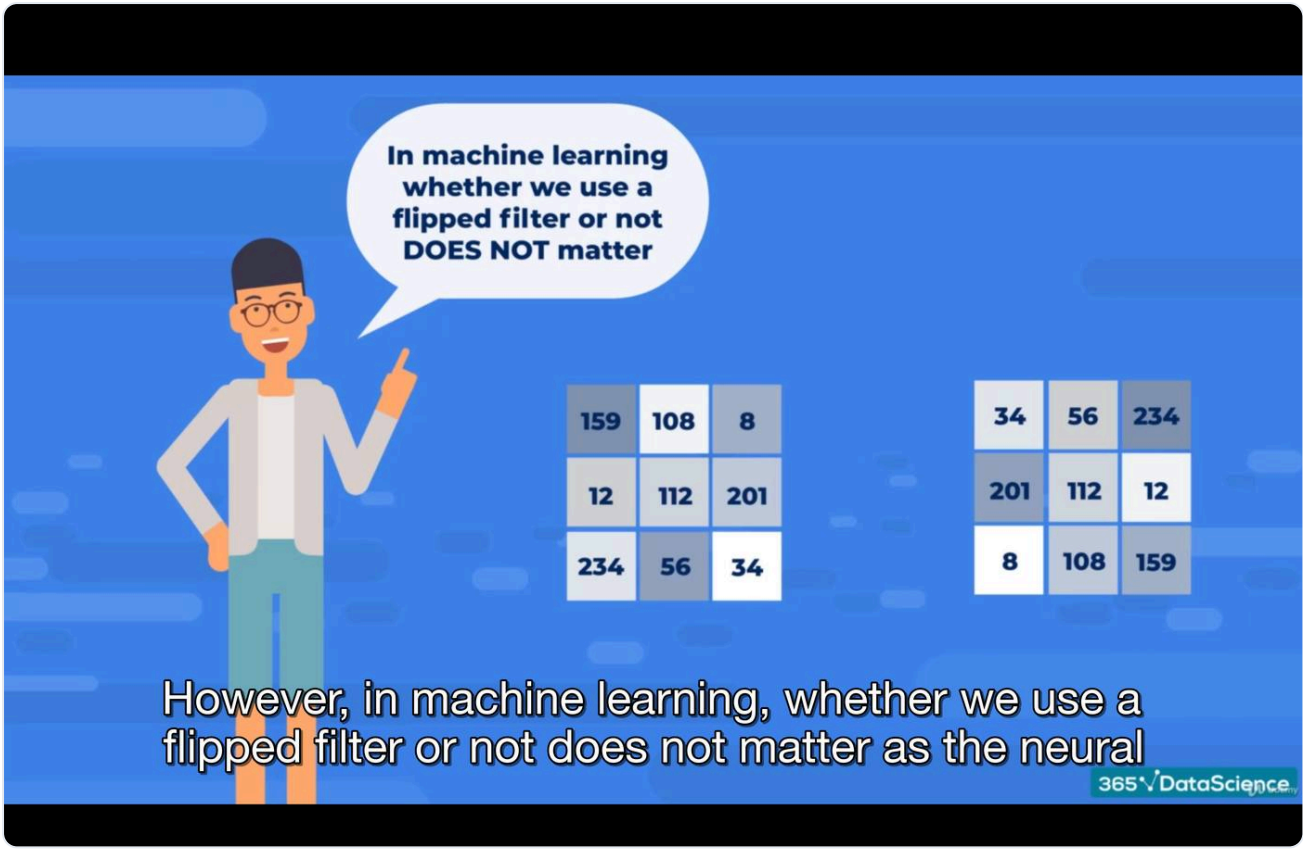

FIGURE 6.8

However, in machine learning, whether we use a flipped filter or not does not matter as the neural network learns the different parameters anyway. So if we flip the kernel, the network will just learn the parameters in flipped order.

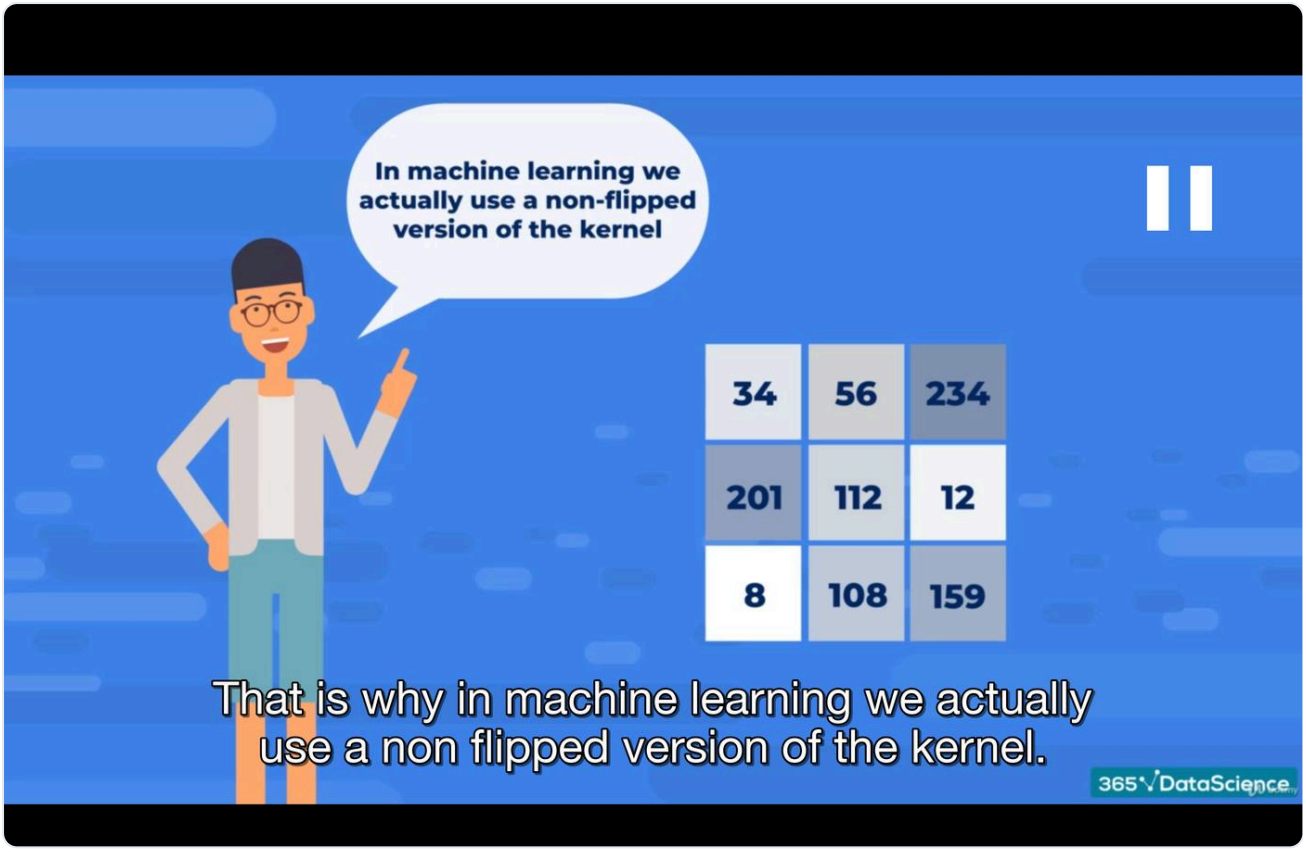

FIGURE 6.9

That is why in machine learning we actually use a non flipped version of the kernel. Finally, for those of you who are curious, we'd like to take a quick sidestep.

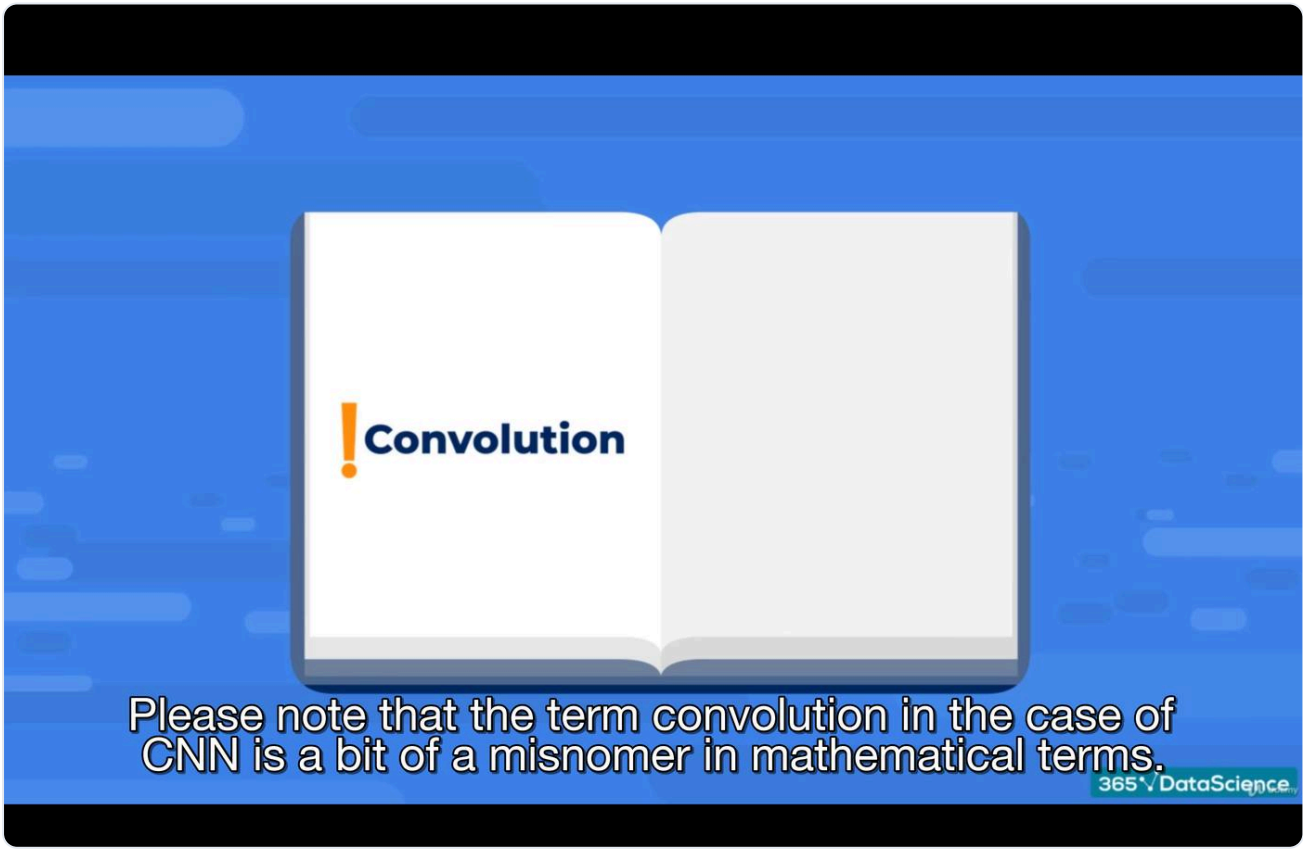

FIGURE 6.10 | ENN Is'a Bitof a Misnomer in Mathematical Terms:

Please note that the term convolution in the case of CNN is a bit of a misnomer in mathematical terms.

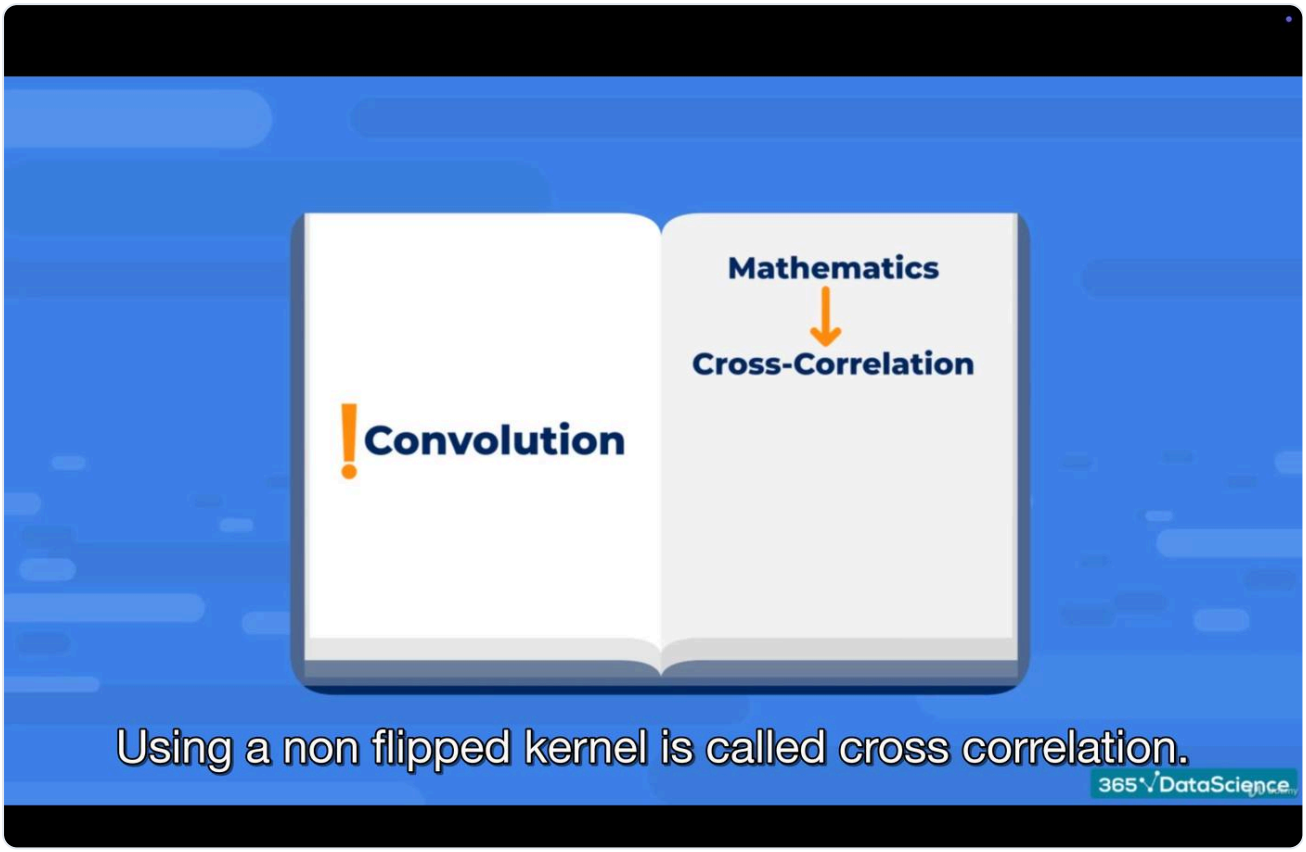

FIGURE 6.11

Using a non flipped kernel is called cross correlation.

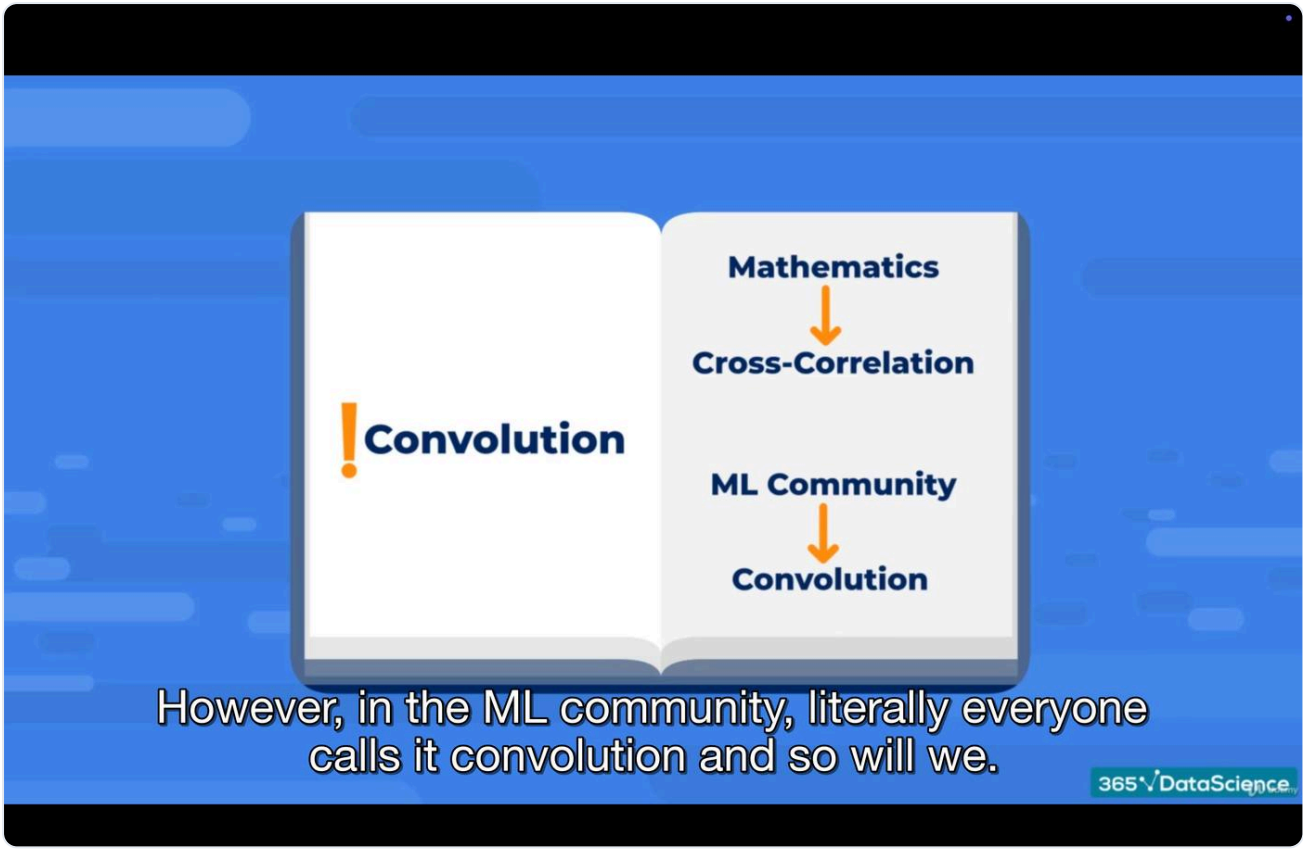

FIGURE 6.12

However, in the ML community, literally everyone calls it convolution and so will we. Great.

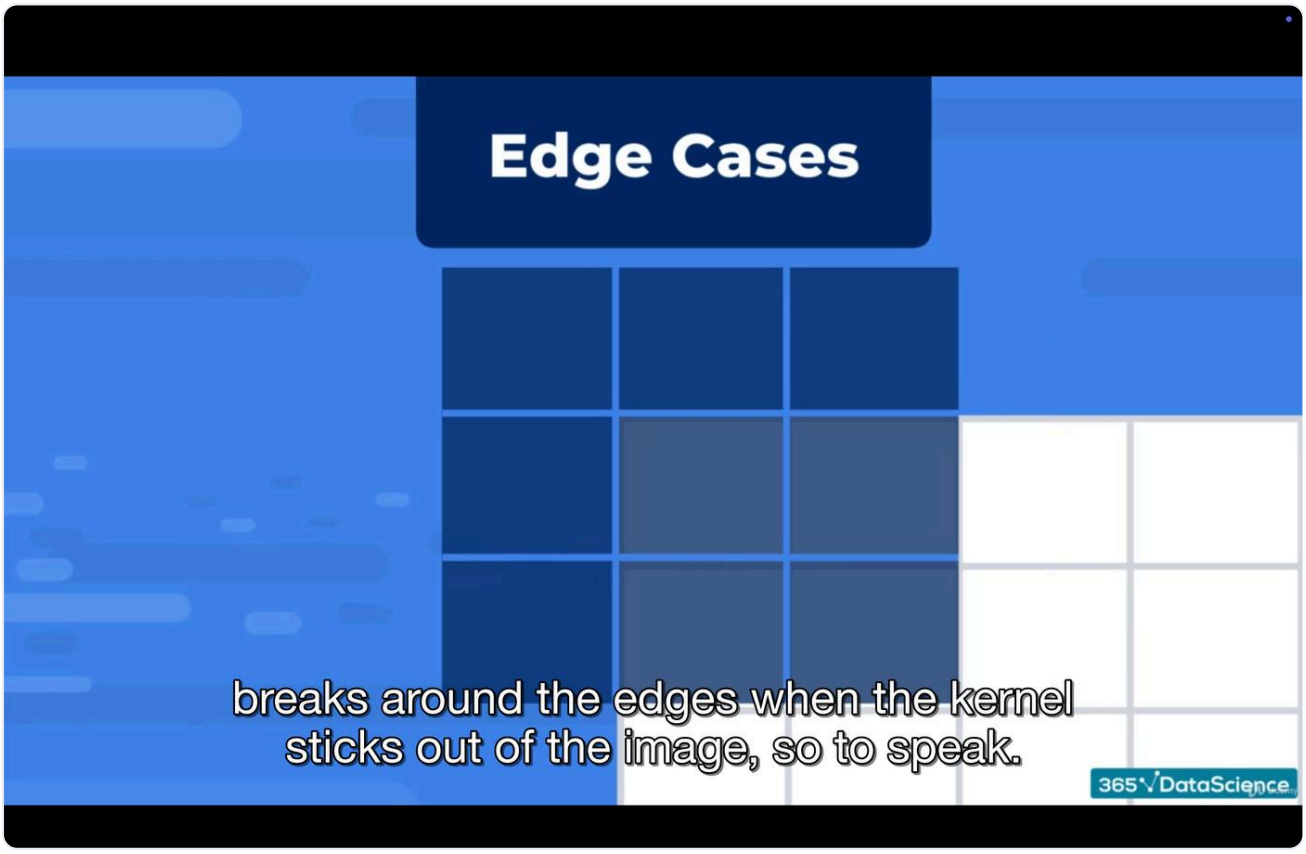

FIGURE 6.13 | Edge Cases

While the convolution formula looks straightforward now, there are some edge cases and in fact it literally breaks around the edges when the kernel sticks out of the image, so to speak. So how do we proceed in this situation? Well, that is what we are going to discuss in the next lecture. Thanks for watching.

LECTURE 7

The Problem

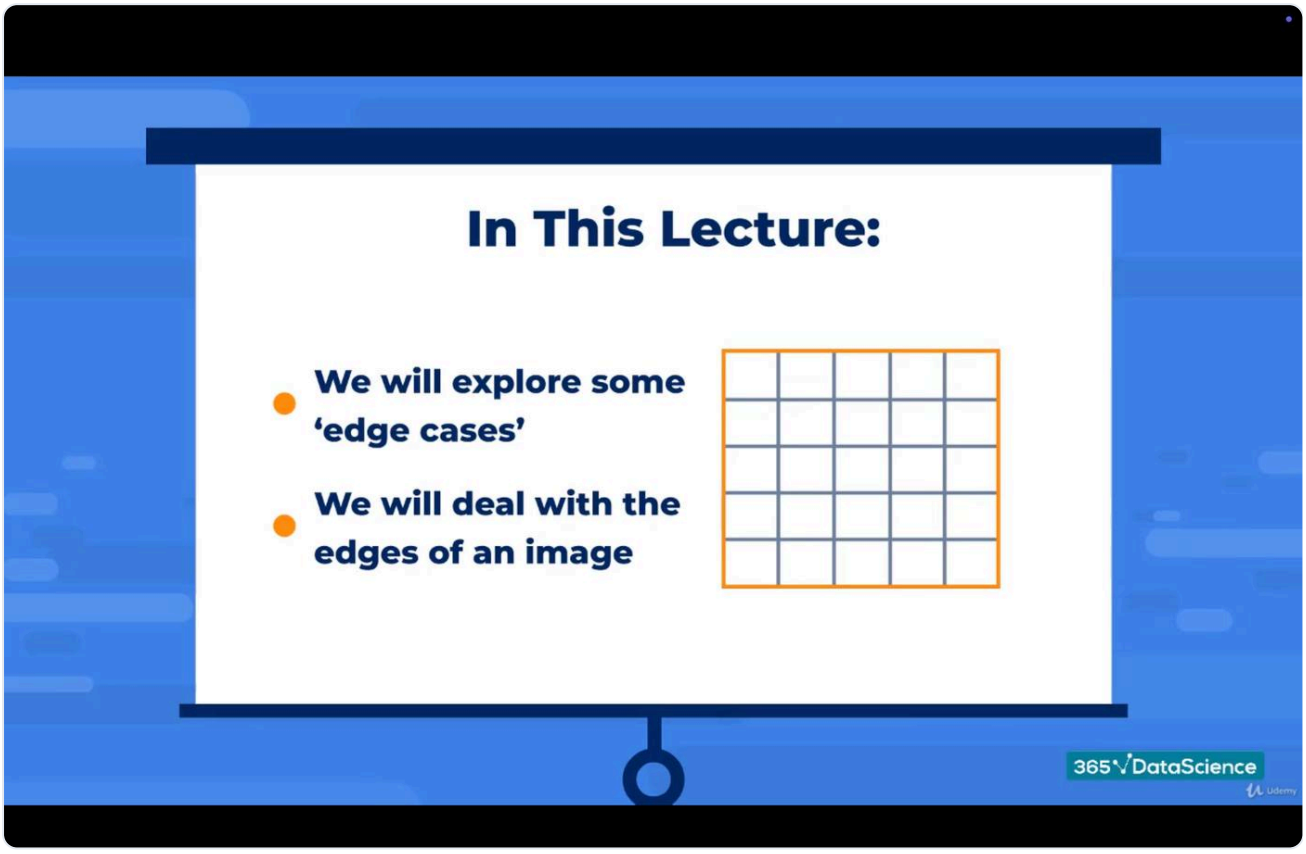

FIGURE 7.1 | We Will Explore Some

In this video, we will explore some edge cases, we will literally deal with the edges of an image. Here's the problem.

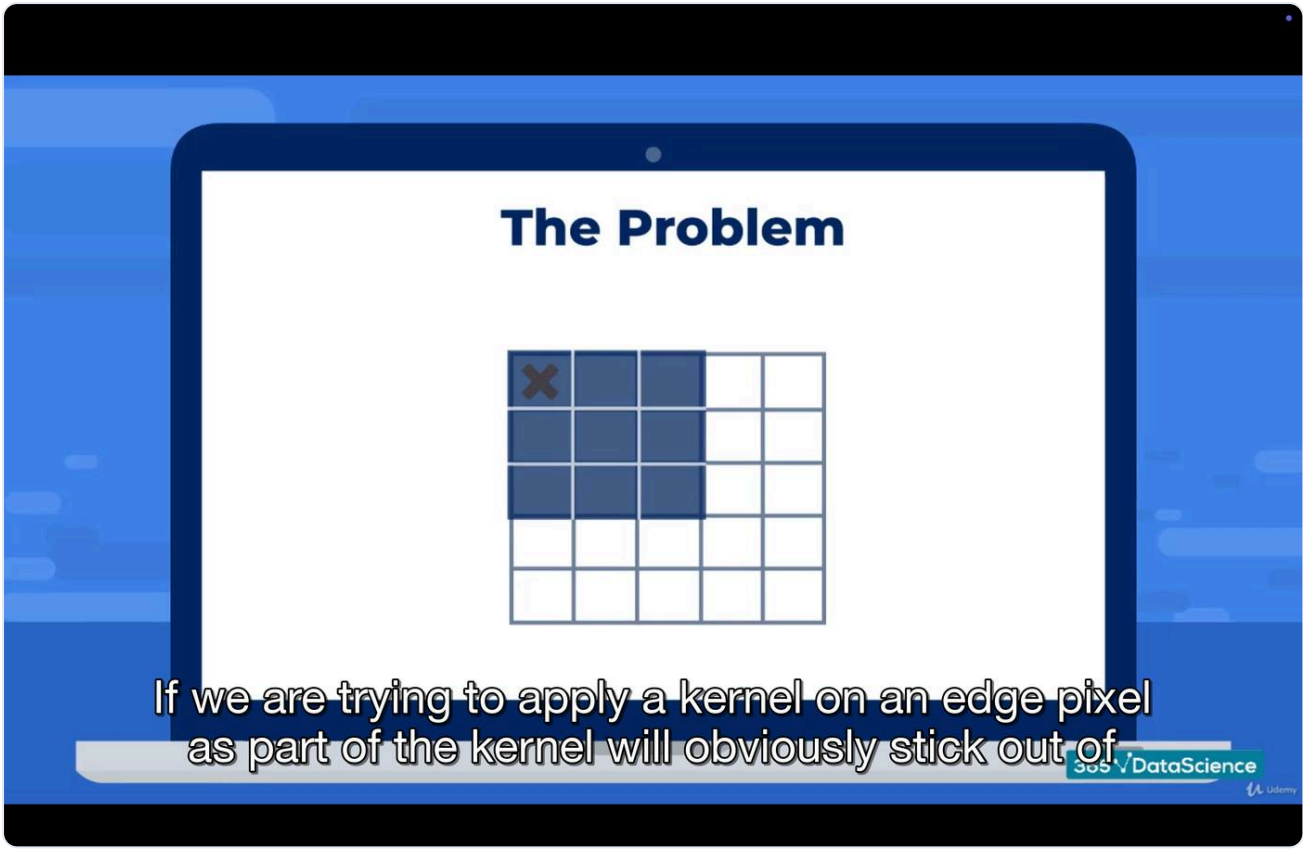

FIGURE 7.2 | The Problem

If we are trying to apply a kernel on an edge pixel as part of the kernel will obviously stick out of the image.

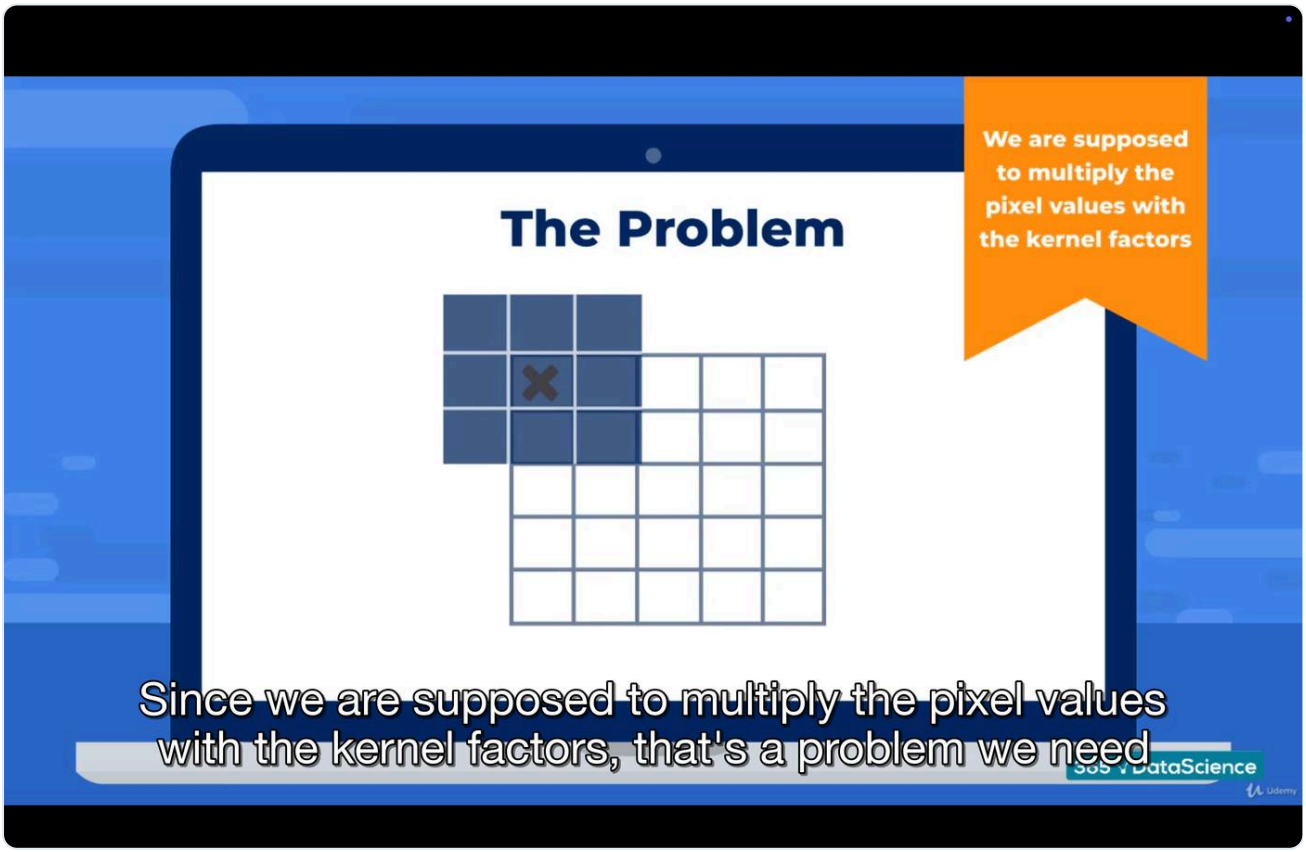

FIGURE 7.3

Since we are supposed to multiply the pixel values with the kernel factors, that's a problem we need to resolve.

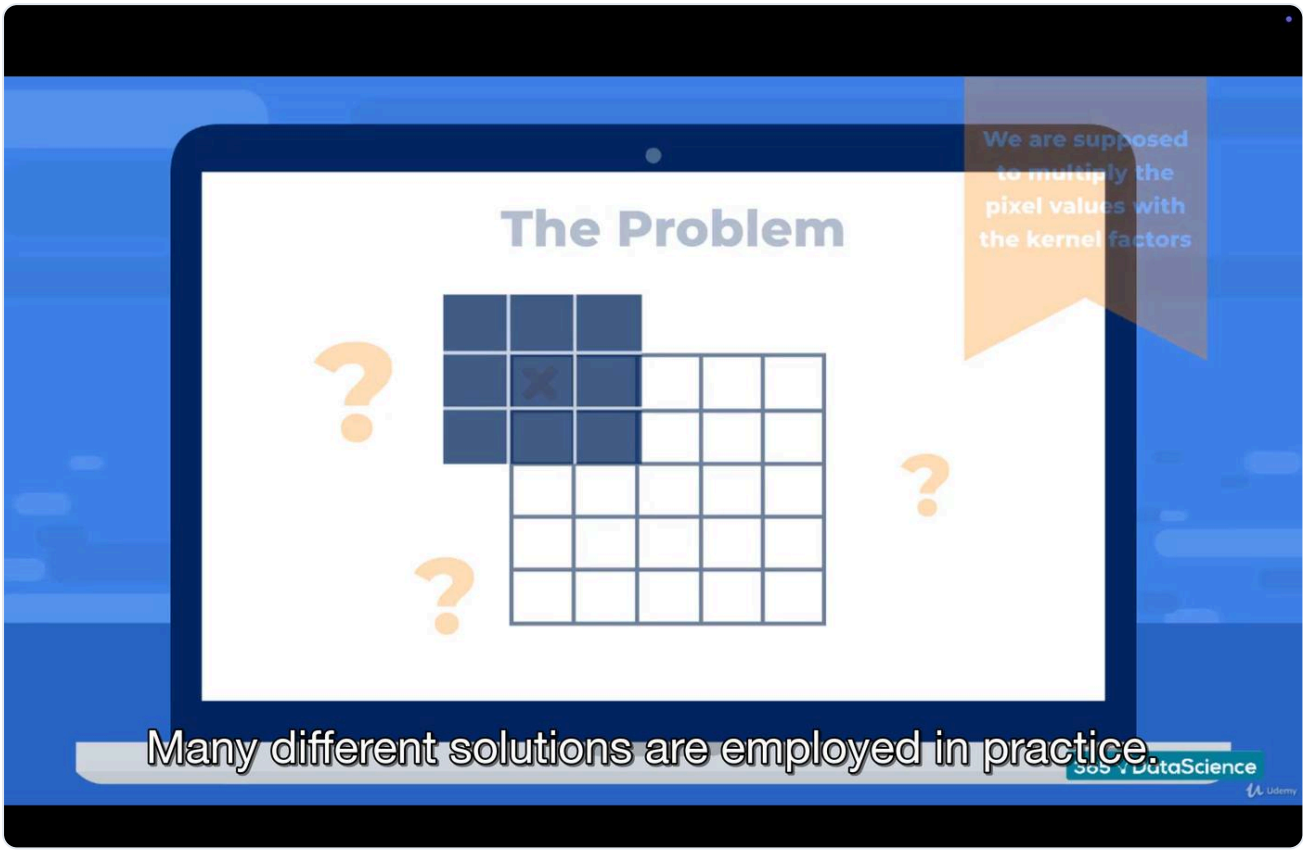

FIGURE 7.4

Many different solutions are employed in practice. One of the easiest ones is to extend the image outwards with pixel values set to zero.

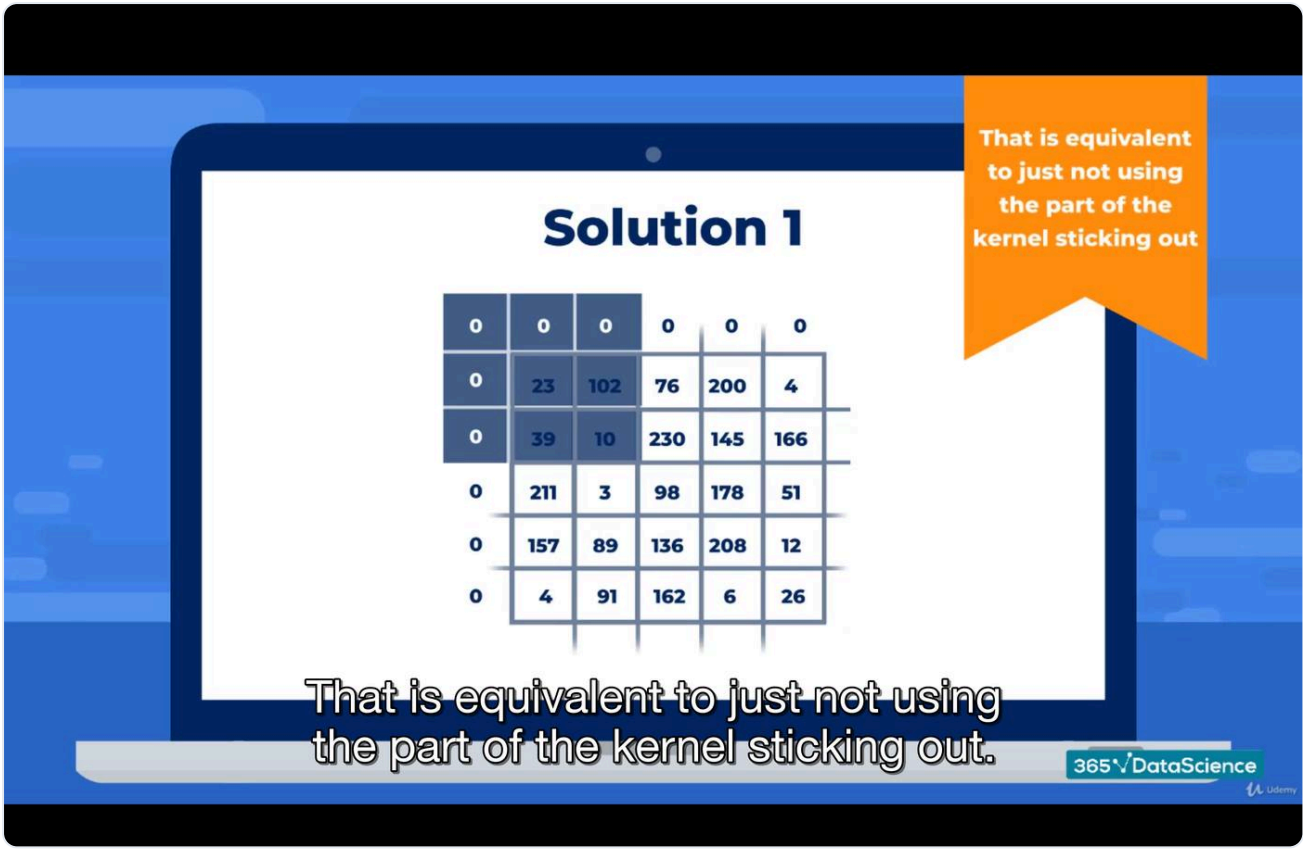

FIGURE 7.5

That is equivalent to just not using the part of the kernel sticking out. Since it is easier to simply overwrite the sticking parts with zero, we prefer to use that when coding. What about the performance of this remedy? Well, those simplistic that works well for a lot of situations and is one of the most popular methods to deal with the issue.

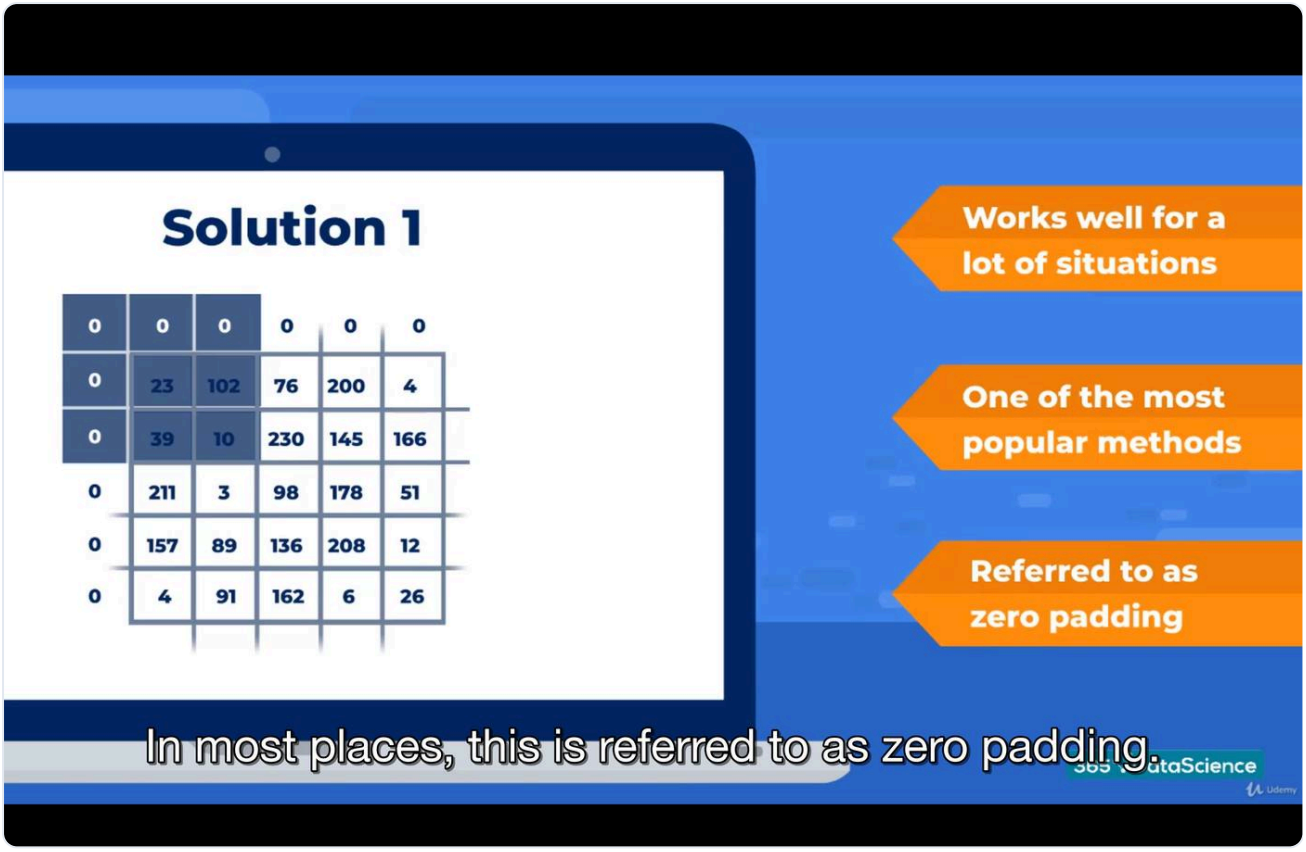

FIGURE 7.6 | Solution 1 Works Well for a

In most places, this is referred to as zero padding. OK, another common solution is to expand the image, however, instead of setting all the values to zero, they may be taken from the actual image. This is like we are placing a mirror at the edge that reflects all values. Sometimes these values are taken with a lower weight.

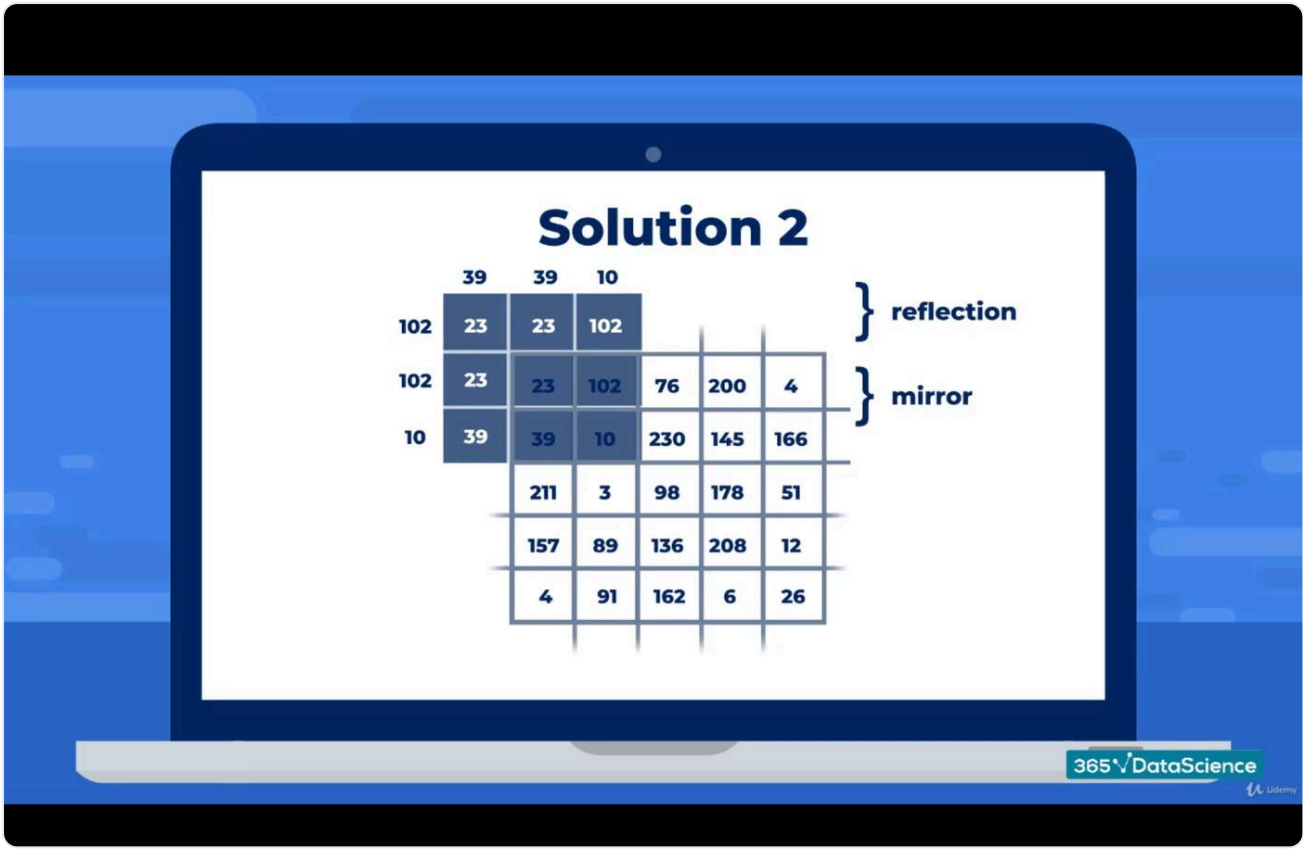

FIGURE 7.7

All right. But if you want to really out of the box solution, here's one. Sometimes the best way to resolve the problem may be to just not deal with that problem at all in practice. That would mean to simply omit all pixels for which the kernel sticks out of the image.

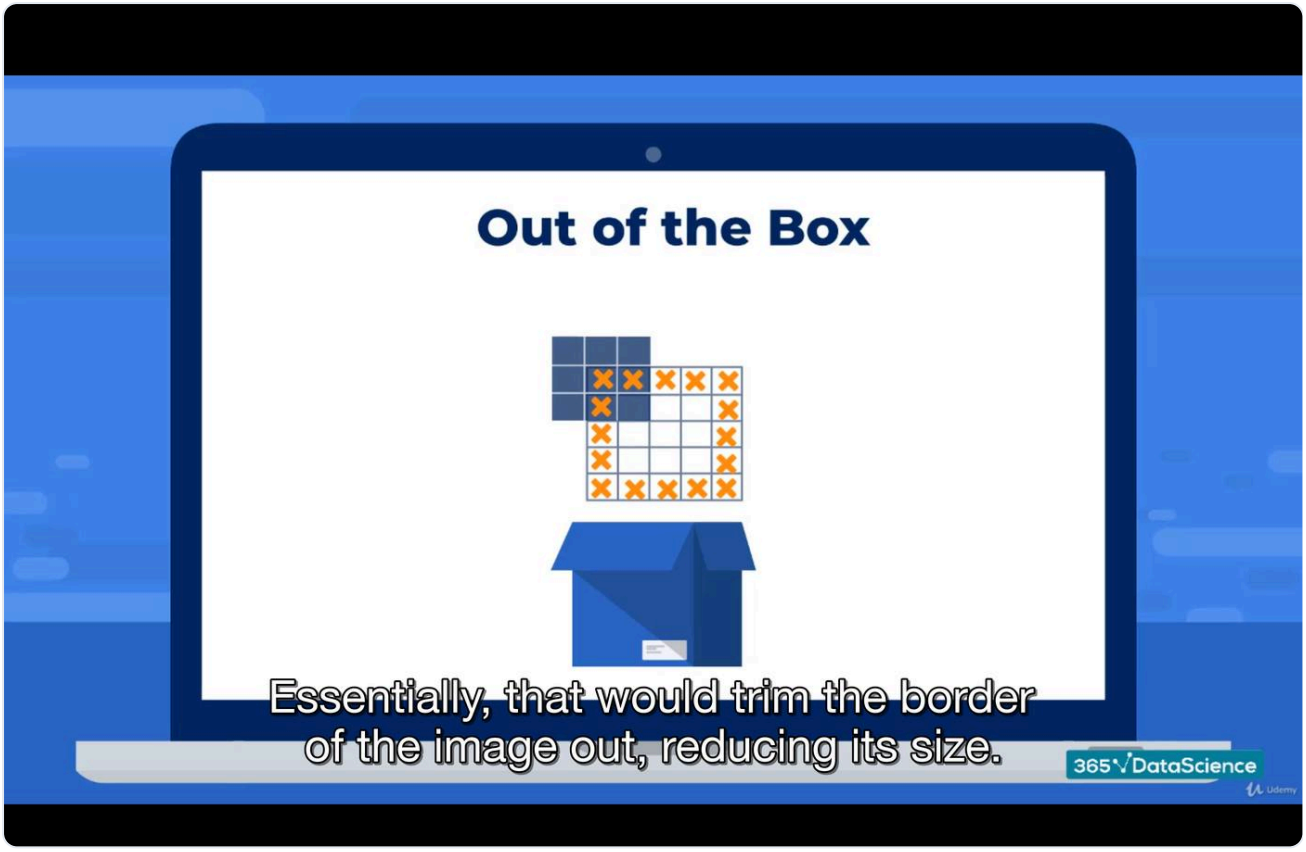

FIGURE 7.8

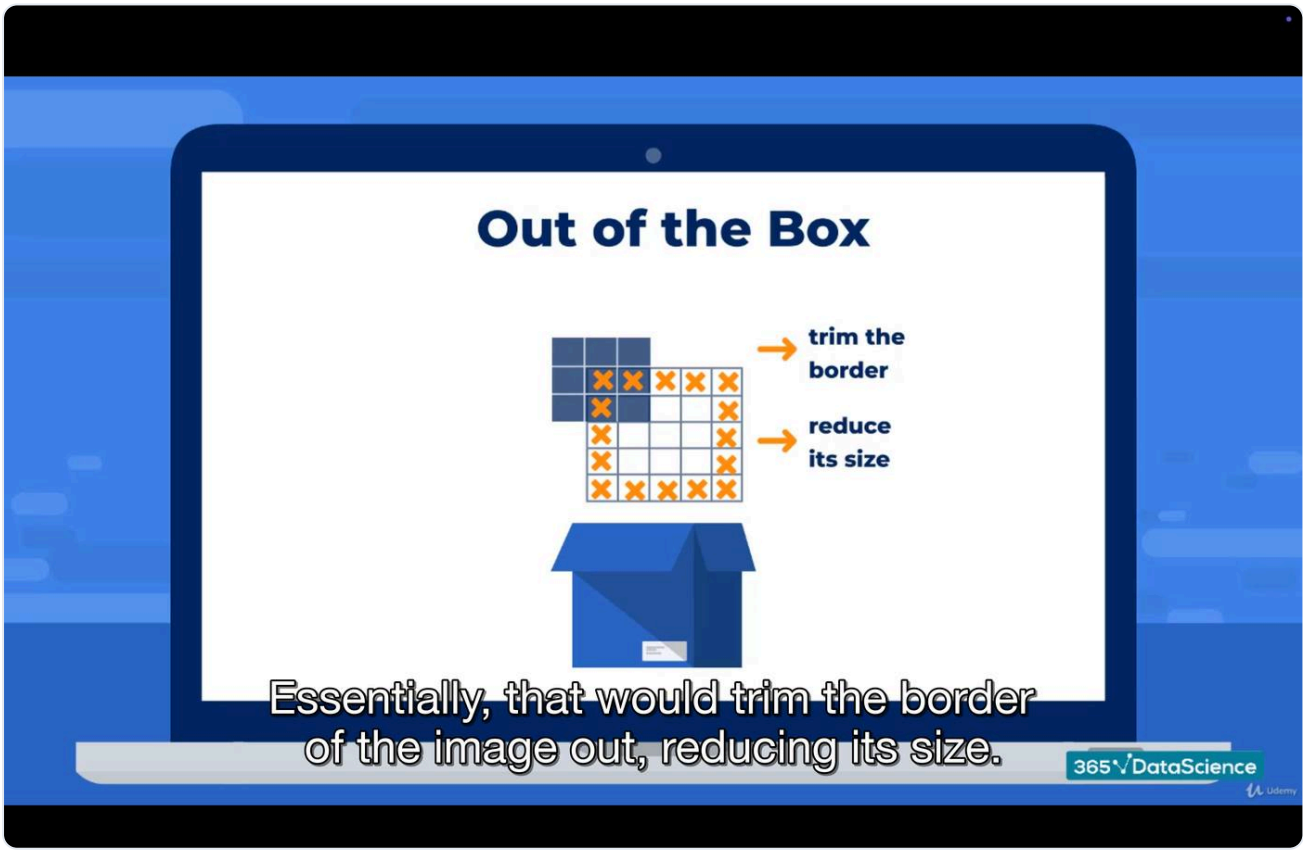

FIGURE 7.9

Essentially, that would trim the border of the image out, reducing its size. For instance, suppose we have a 256 by 256 pixels image and our kernel has a dimensionality of five by five. If we don't deal with the edges, we will obtain a 252 by 252 image. This is not a huge loss of information, especially for images bigger than 256 pixels. So there are situations in which that is an OK practice.

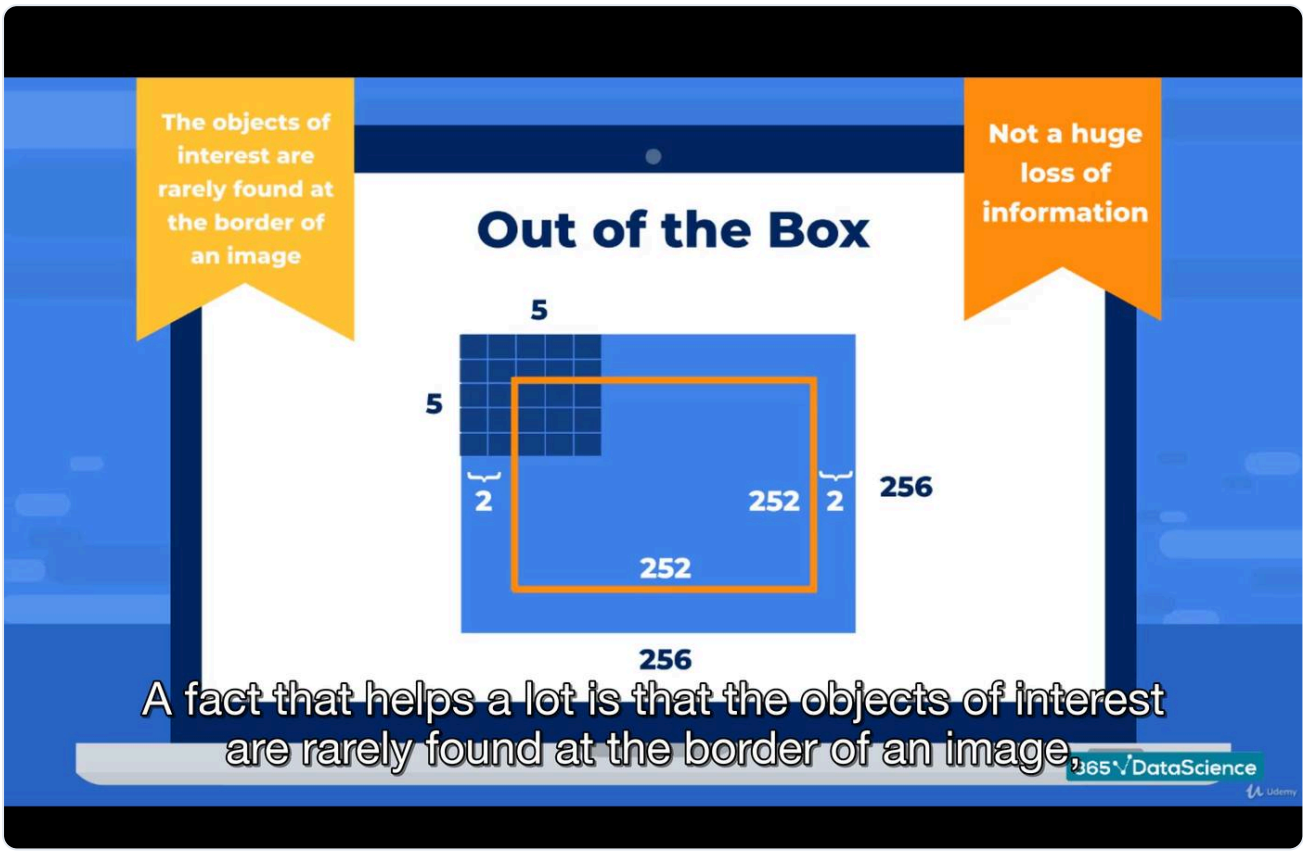

FIGURE 7.10 | Out of the Box

A fact that helps a lot is that the objects of interest are rarely found at the border of an image, as you will see later on in CNN's. Oftentimes we don't care that much about a few pixels at the border.

FIGURE 7.11

Moreover, CNN's involve operations such as pooling, where we basically throw away half of the information in an image anyway.

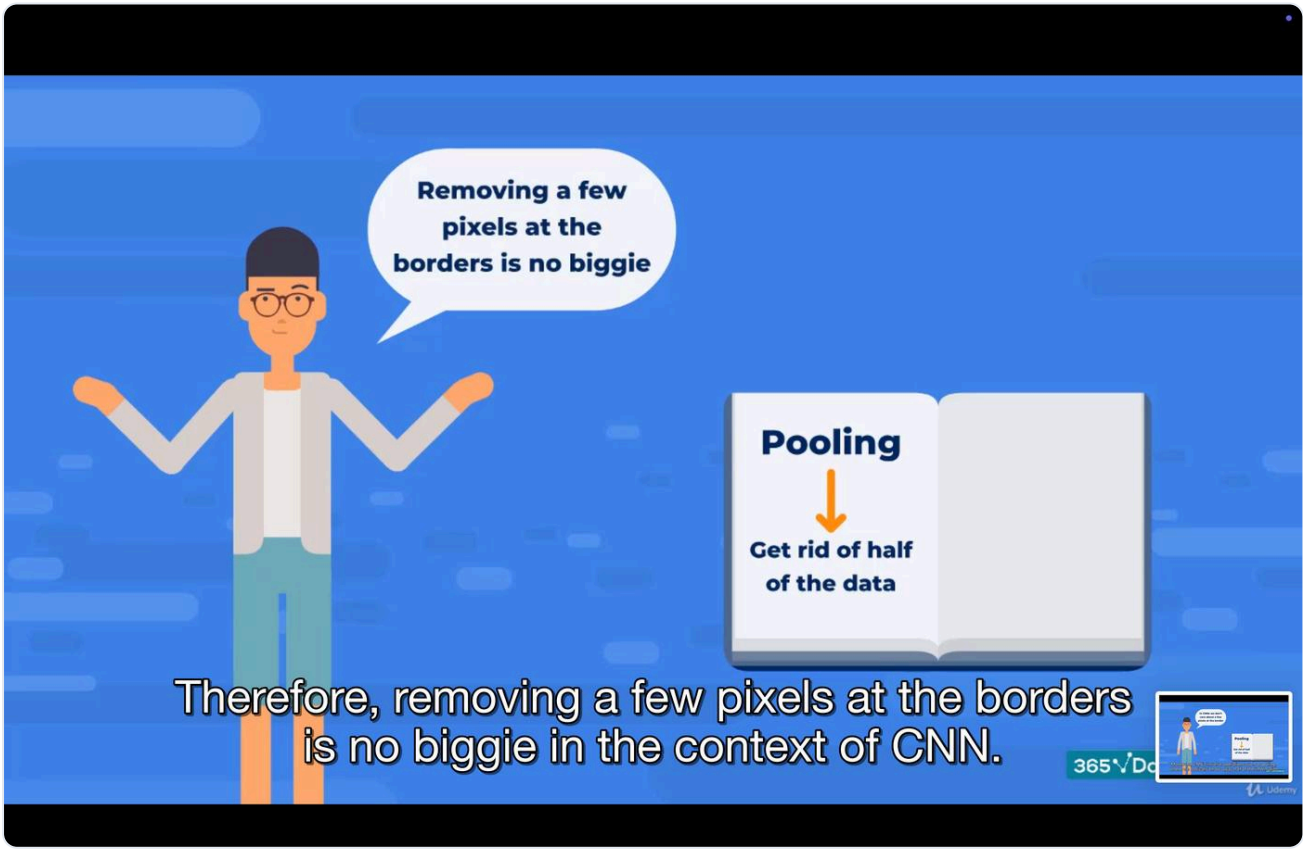

FIGURE 7.12

Therefore, removing a few pixels at the borders is no biggie in the context of CNN. Great. Now we know what kernels are and how they are applied. In the next section, we will explore the role in convolutional neural networks and how everything is constructed. Stay tuned and thanks for watching.

LECTURE 8

Train an NN to Recognize the Digits

Hello and welcome back.

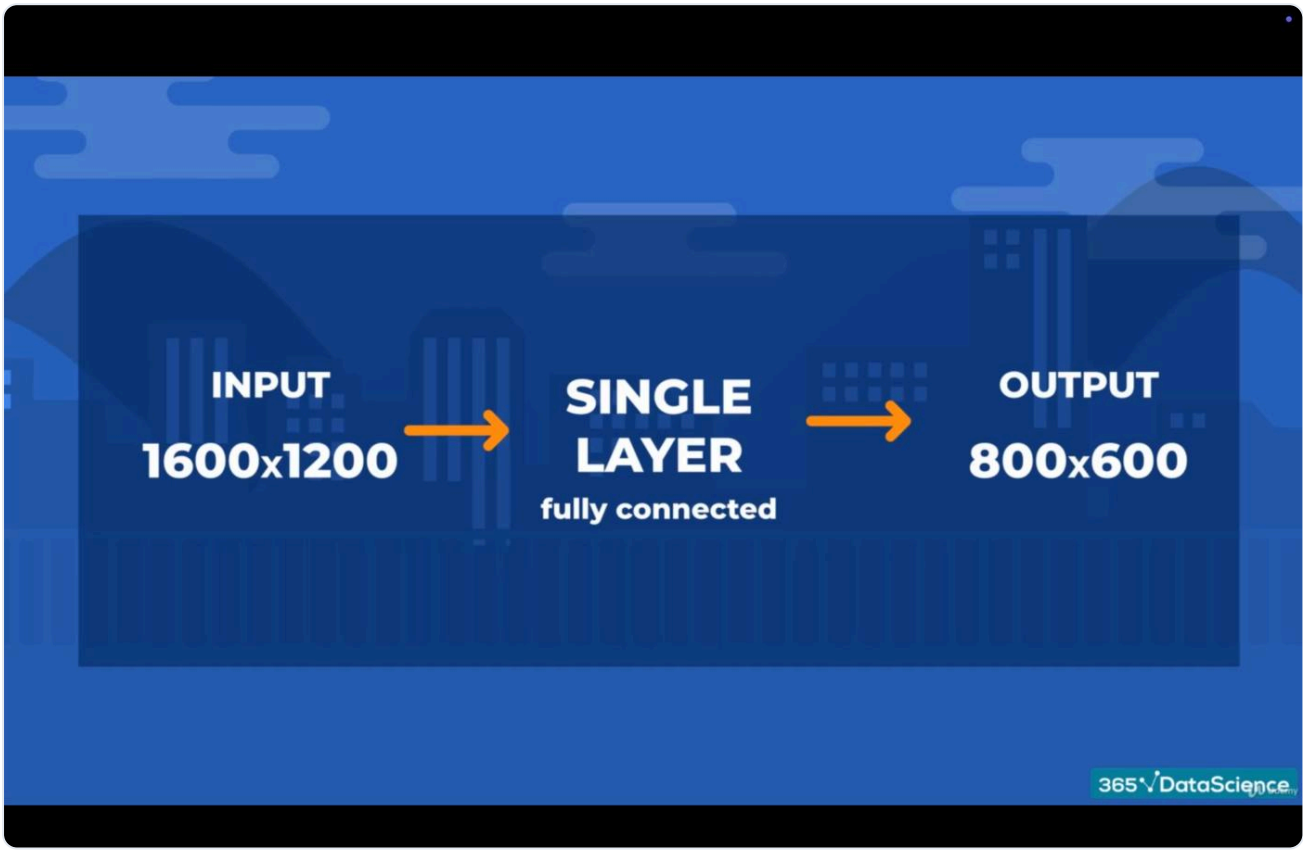

FIGURE 8.1

In the previous section, we talked about kernels and how to transform images in a certain way.

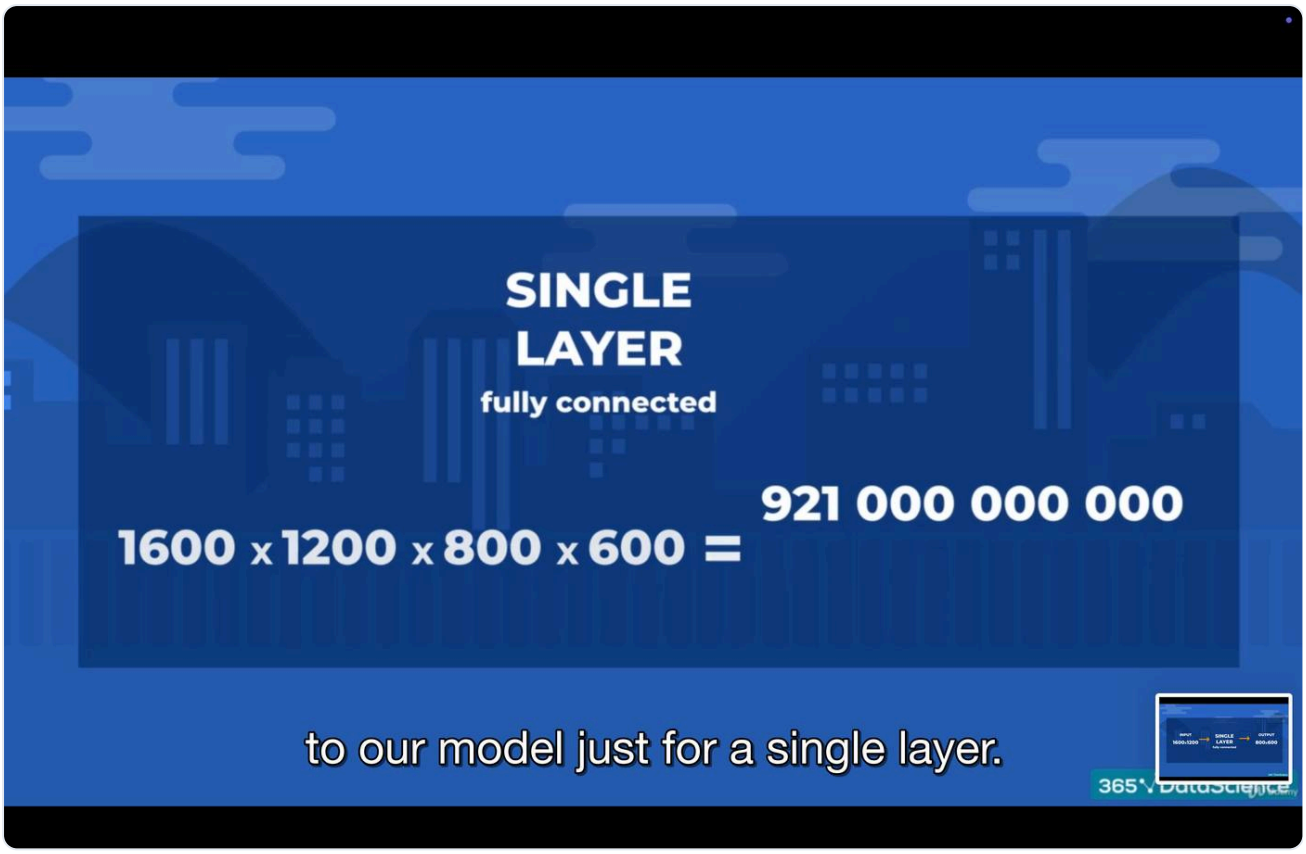

FIGURE 8.2

We understood how blurring, sharpening and edge detection are achieved and what their connection to kernels is.

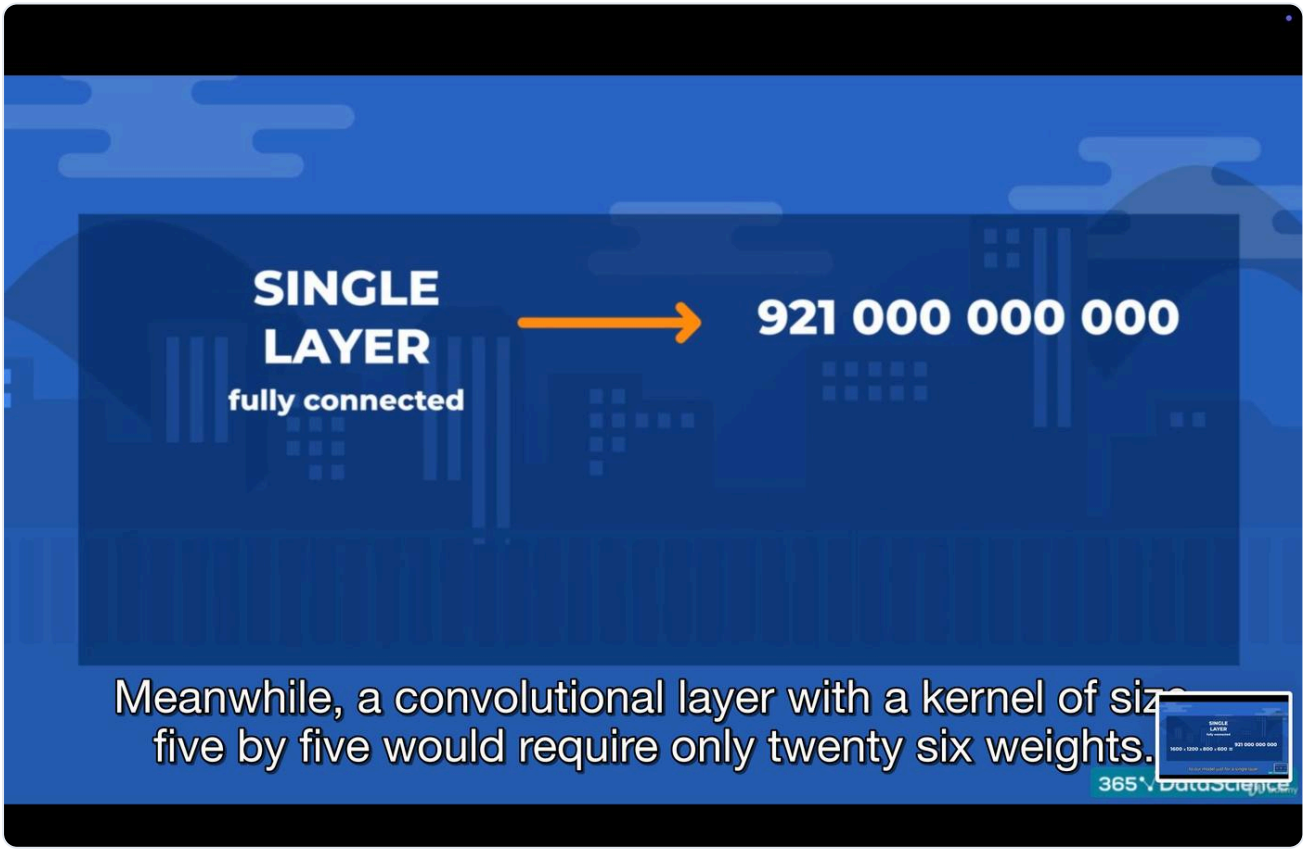

FIGURE 8.3

In this section, we will leverage this knowledge to learn more about convolutional neural networks.

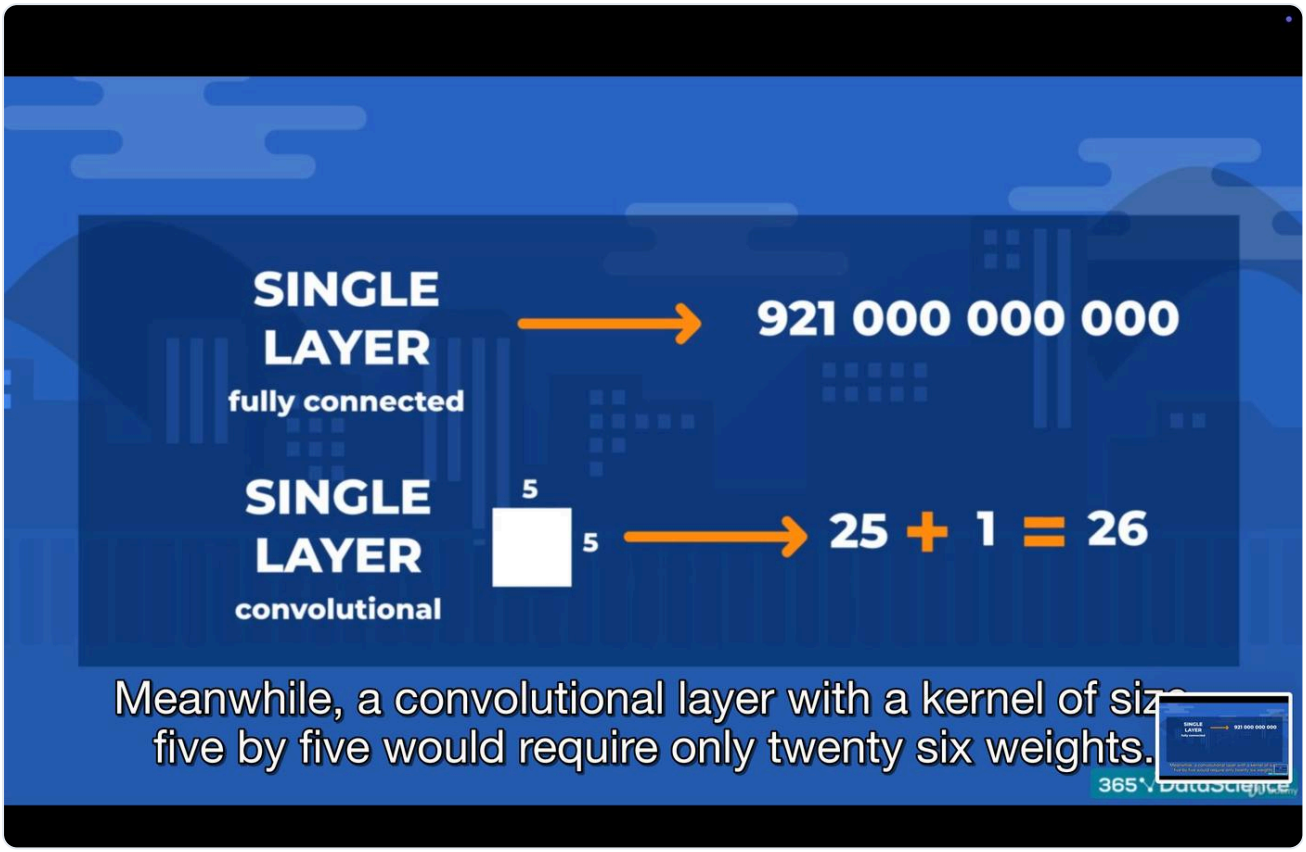

FIGURE 8.4

We will discuss why we need them and how they work.

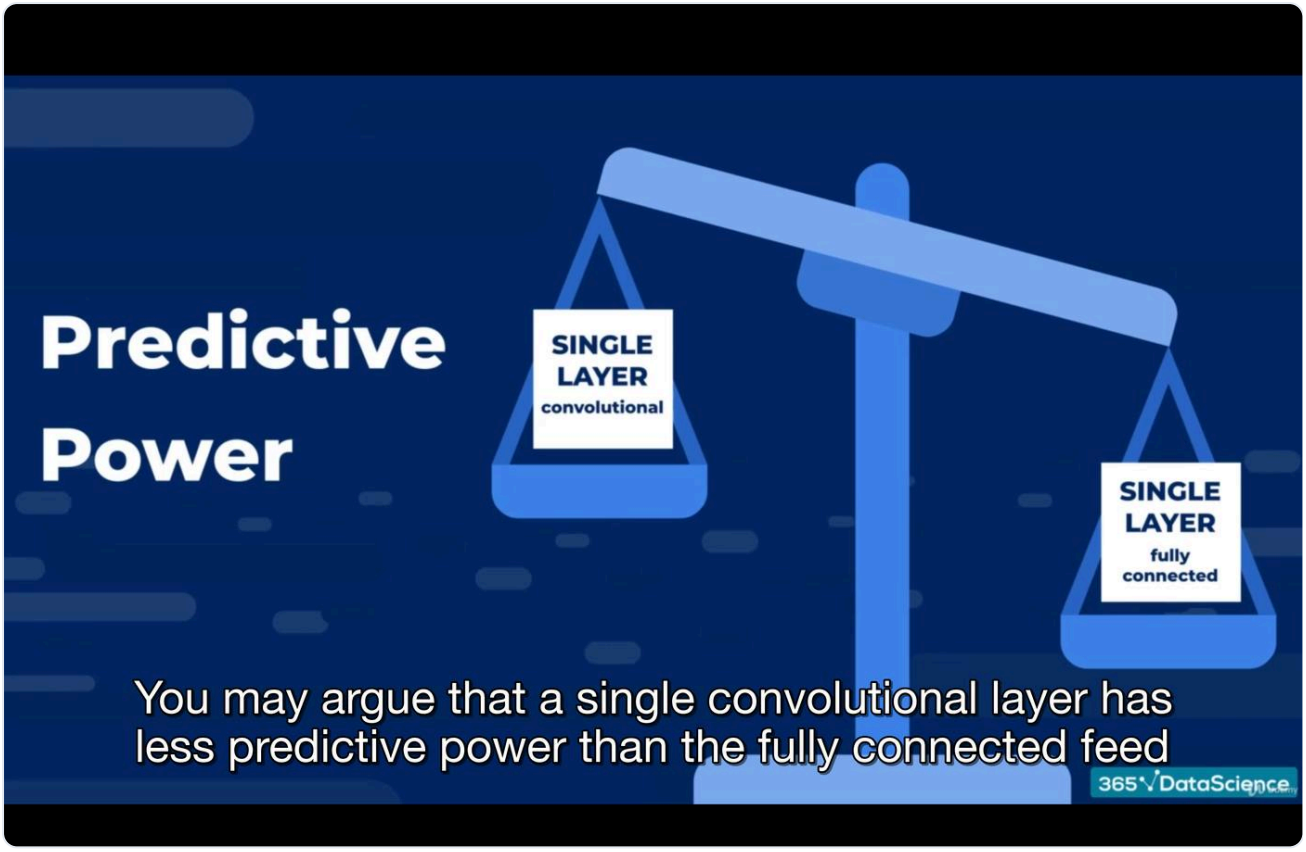

FIGURE 8.5

Then we will construct our first CNN on the MNIST data set.

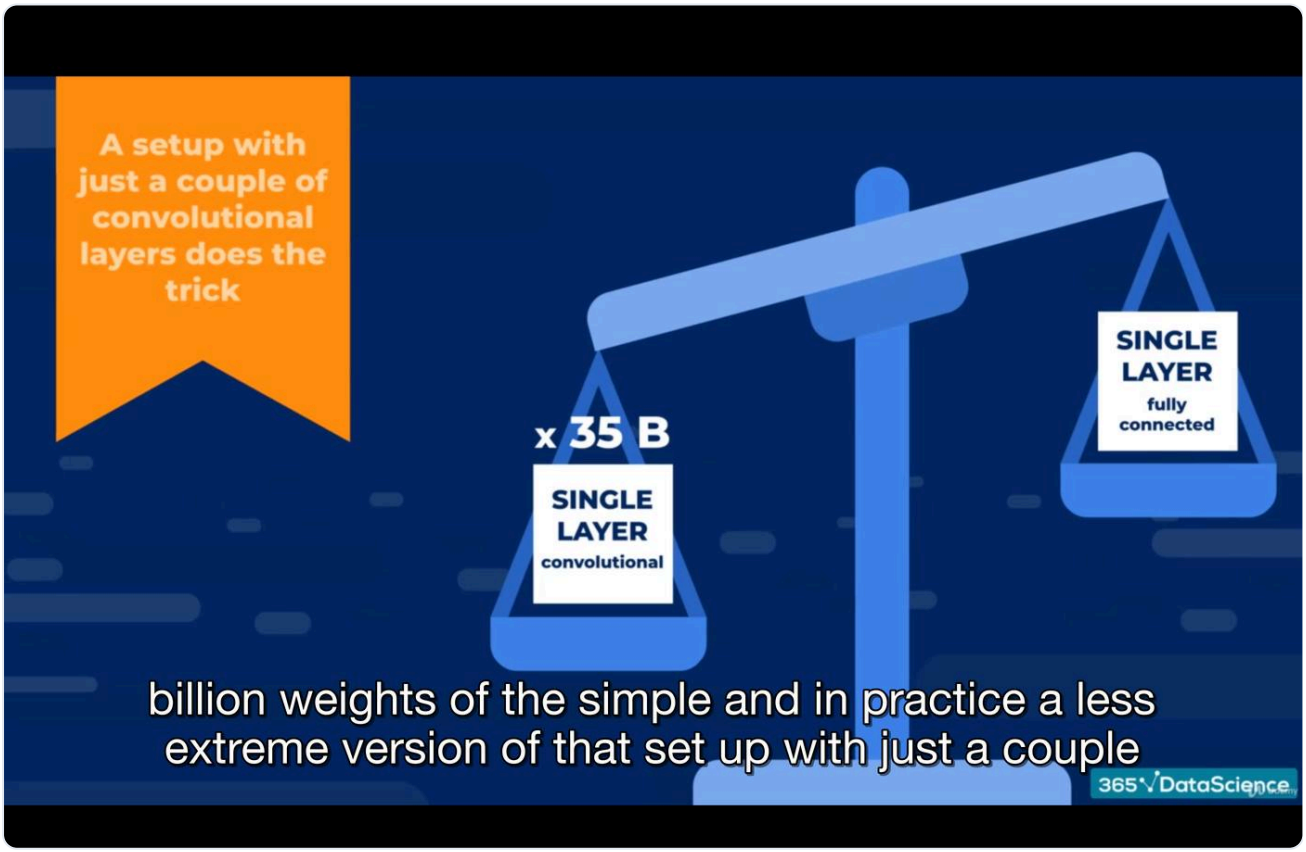

FIGURE 8.6

All right, let's get into it.

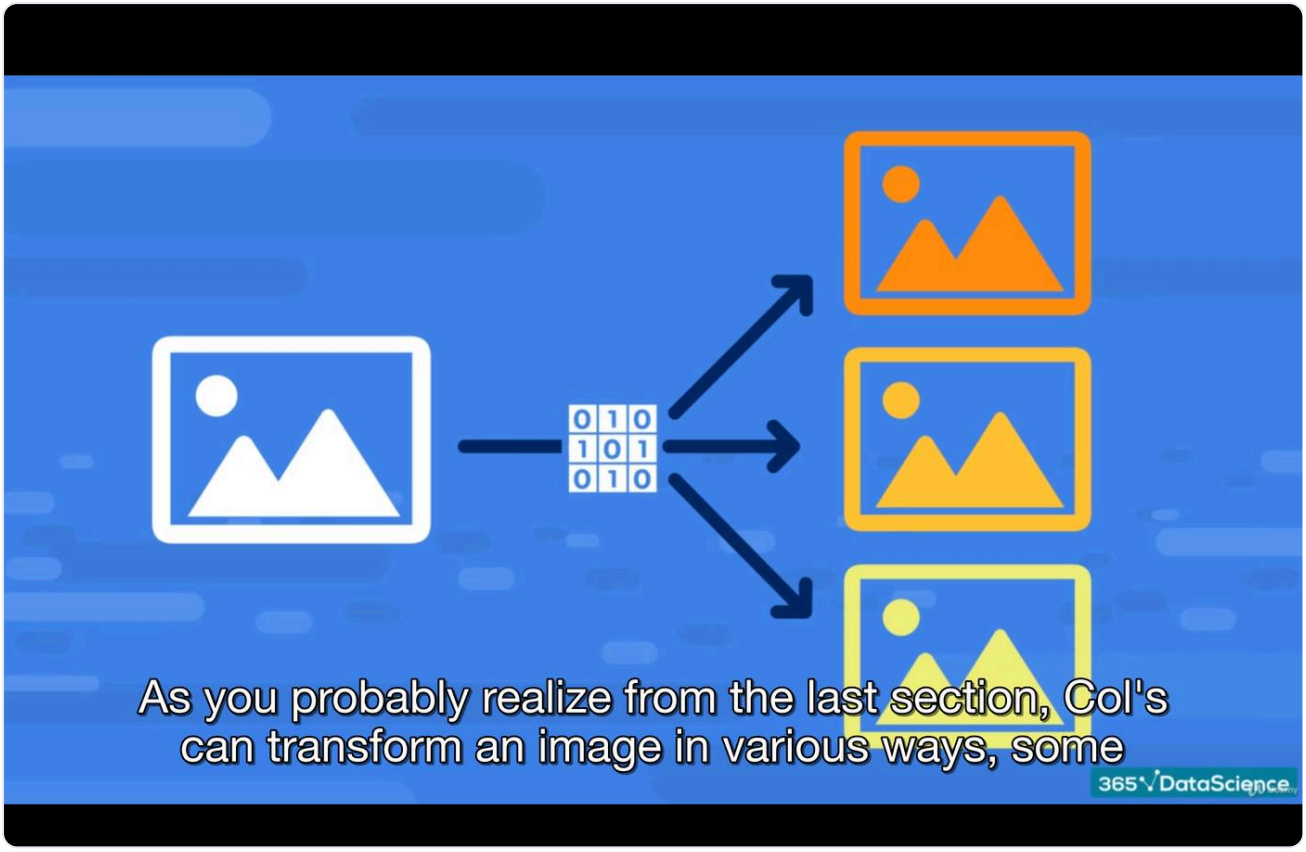

FIGURE 8.7

As you probably realize from the last section, kernels can transform an image in various ways, some more useful than others.

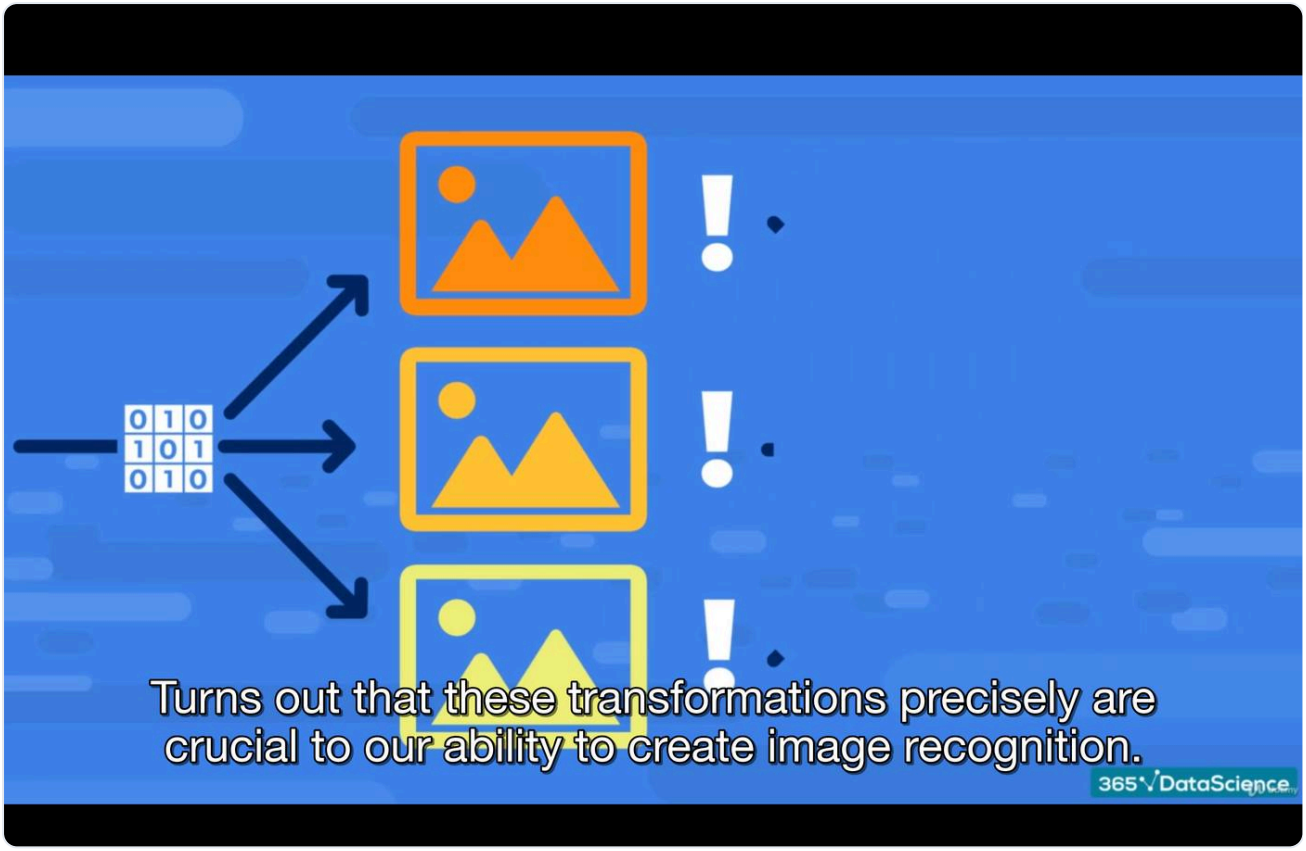

FIGURE 8.8

Turns out that these transformations precisely are crucial to our ability to create image recognition. How would that work?

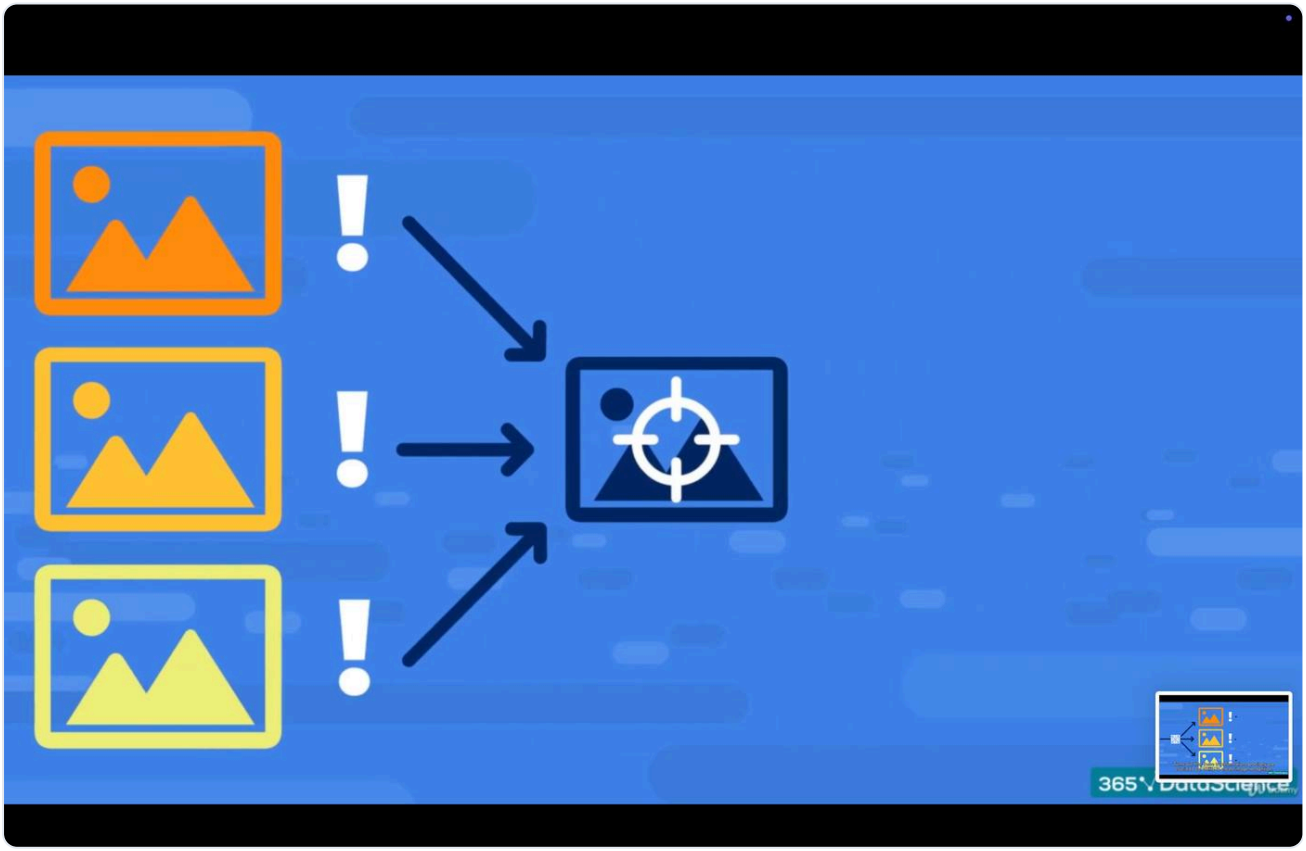

FIGURE 8.9

Theoretically, let's say we want to classify an image as either a dog or a cat similar to our edge detection kernel.

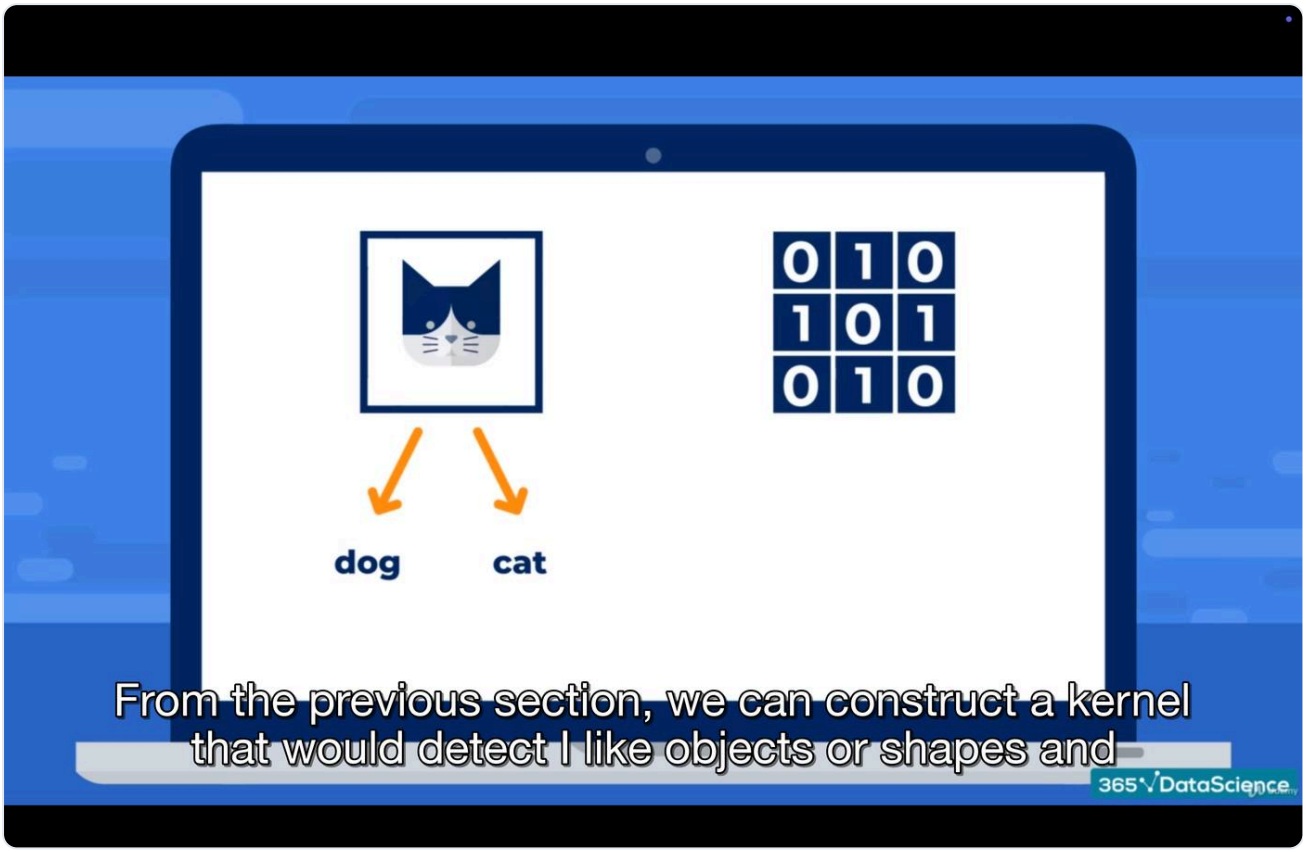

FIGURE 8.10

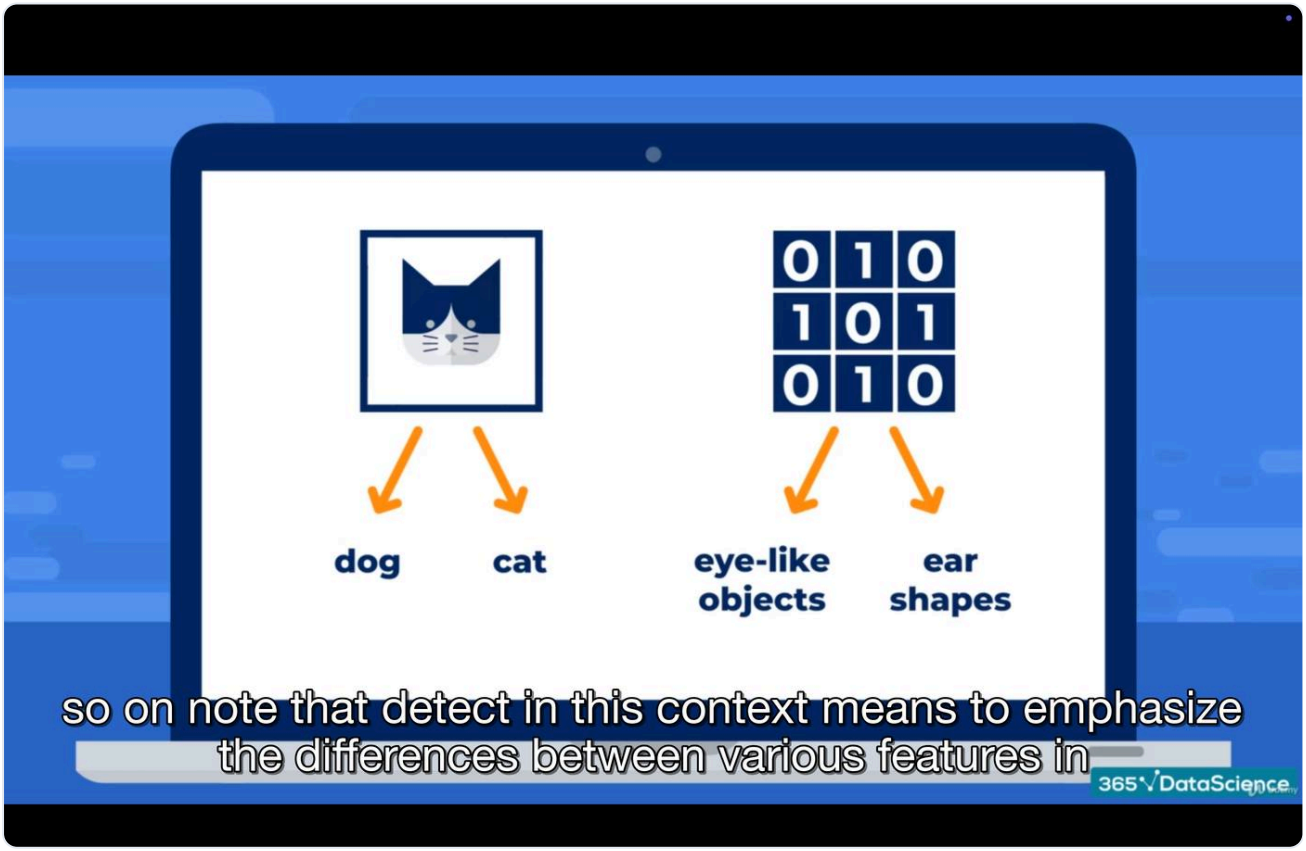

FIGURE 8.11

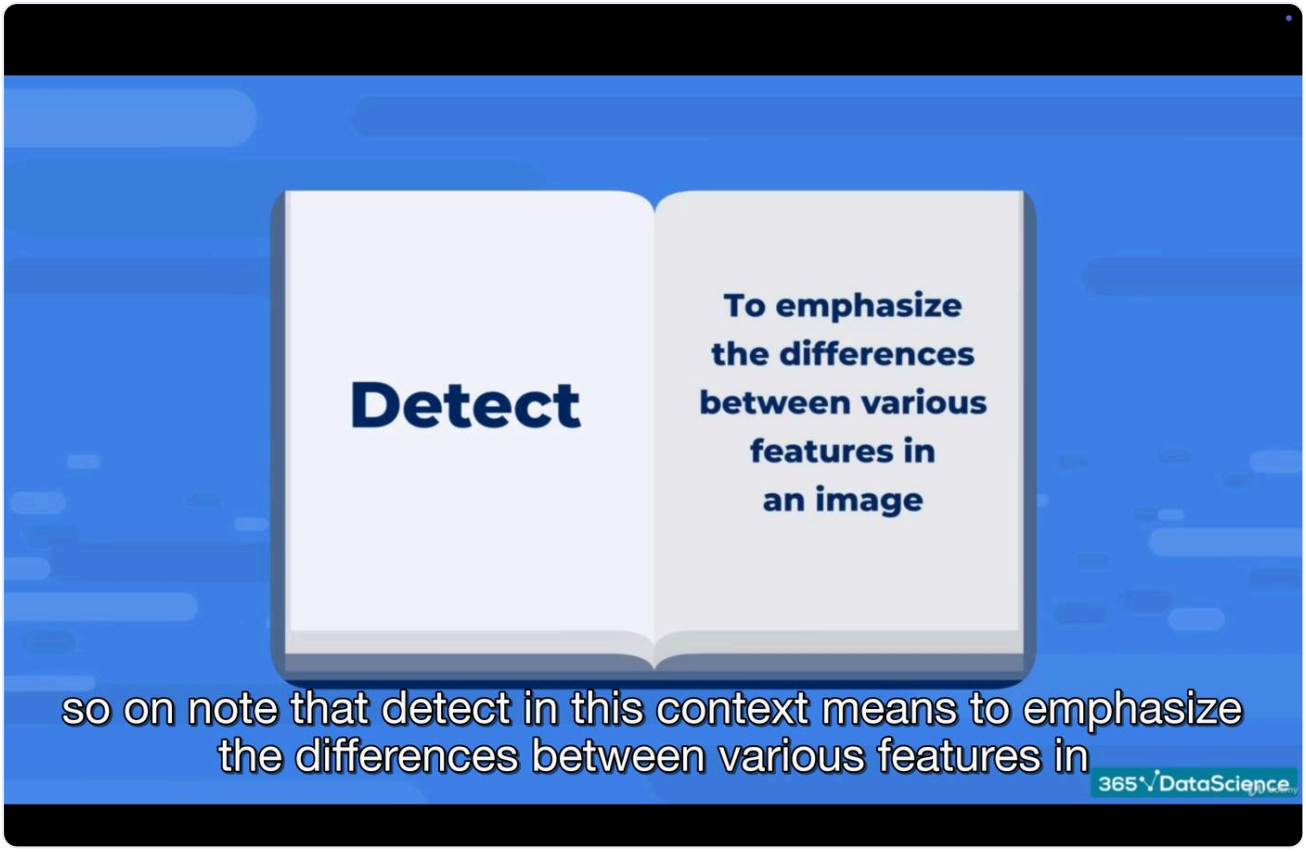

FIGURE 8.12

From the previous section, we can construct a kernel that would detect I like objects or shapes and so on note that detect in this context means to emphasize the differences between various features in an image.

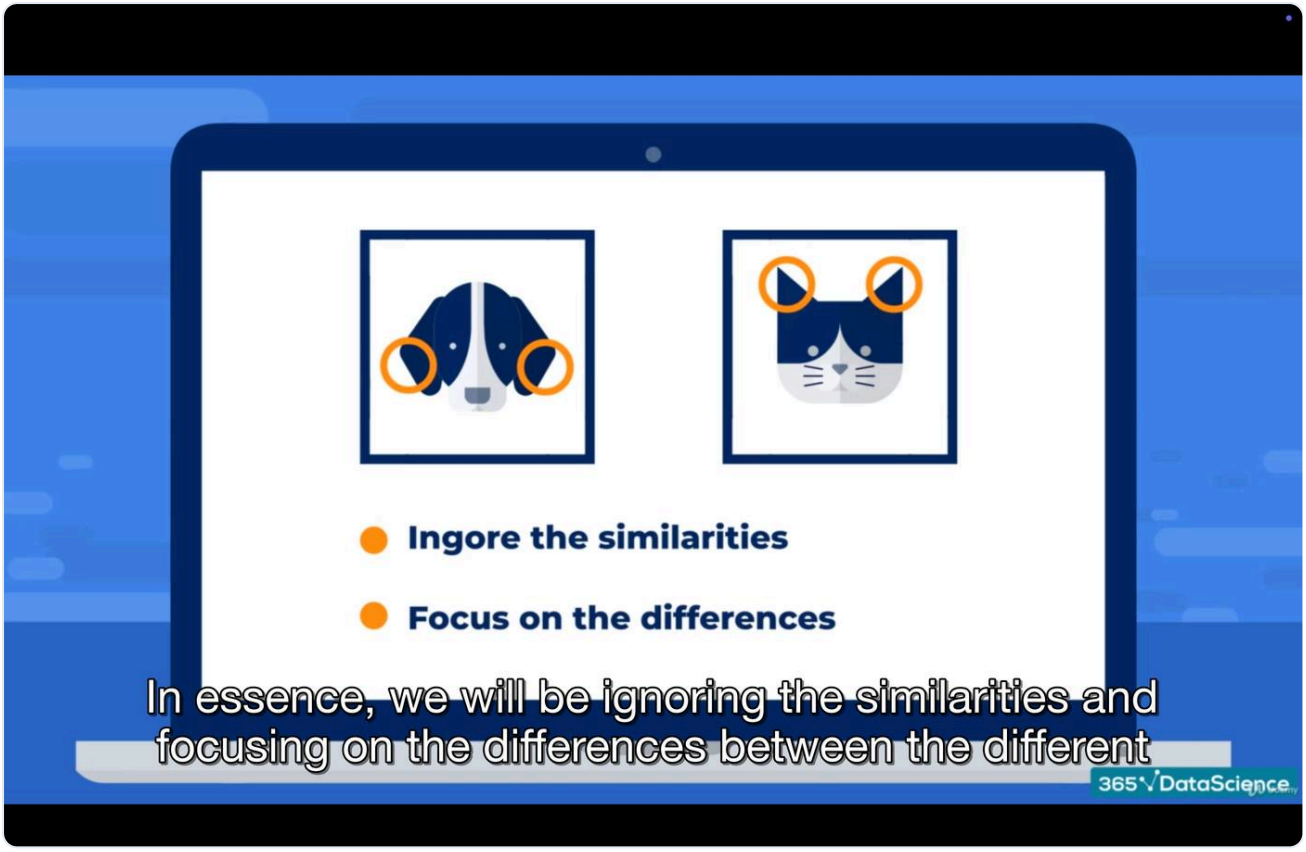

FIGURE 8.13

In essence, we will be ignoring the similarities and focusing on the differences between the different photos. Based on these differences, we could classify the image as a cat or a dog.

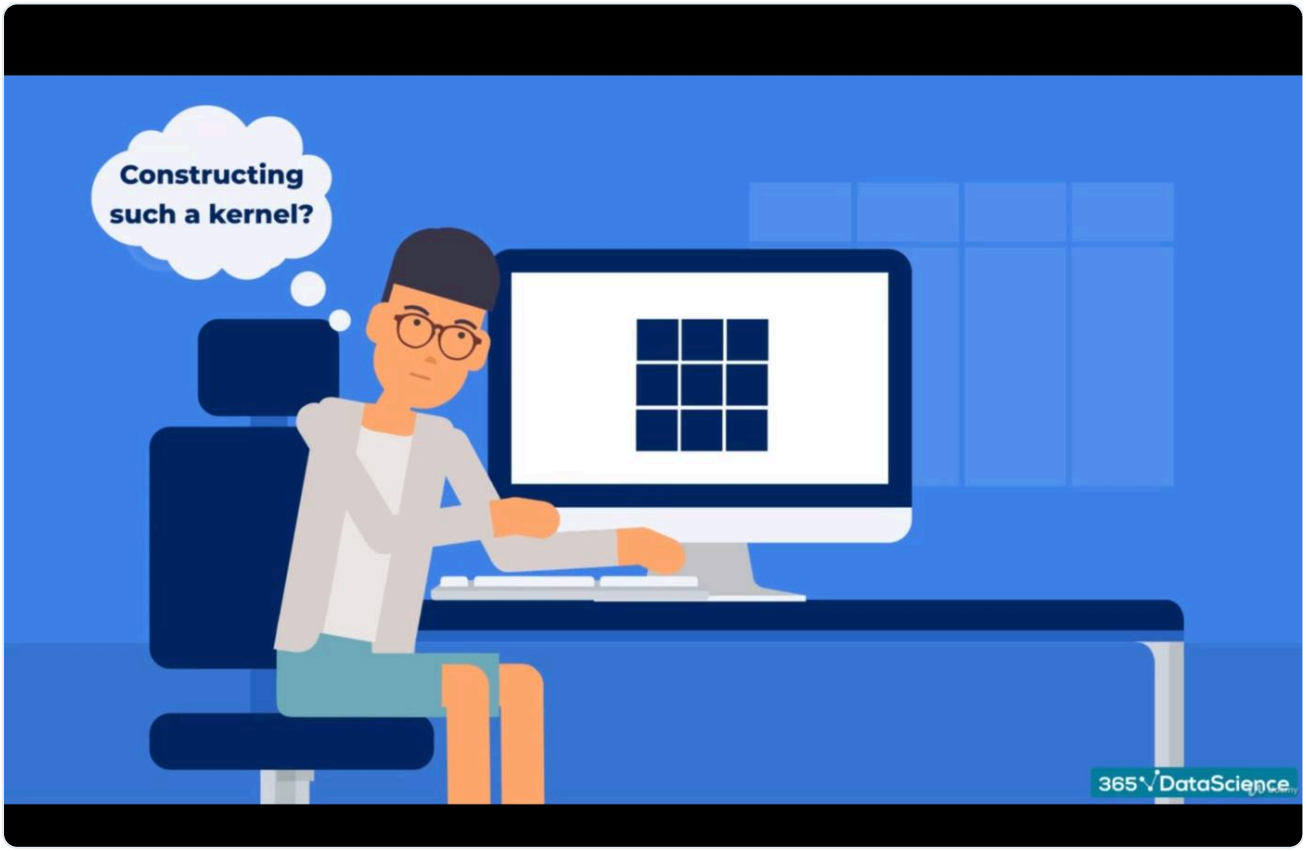

FIGURE 8.14

With this conceptual idea in mind, we may try to actually construct such a kernel.

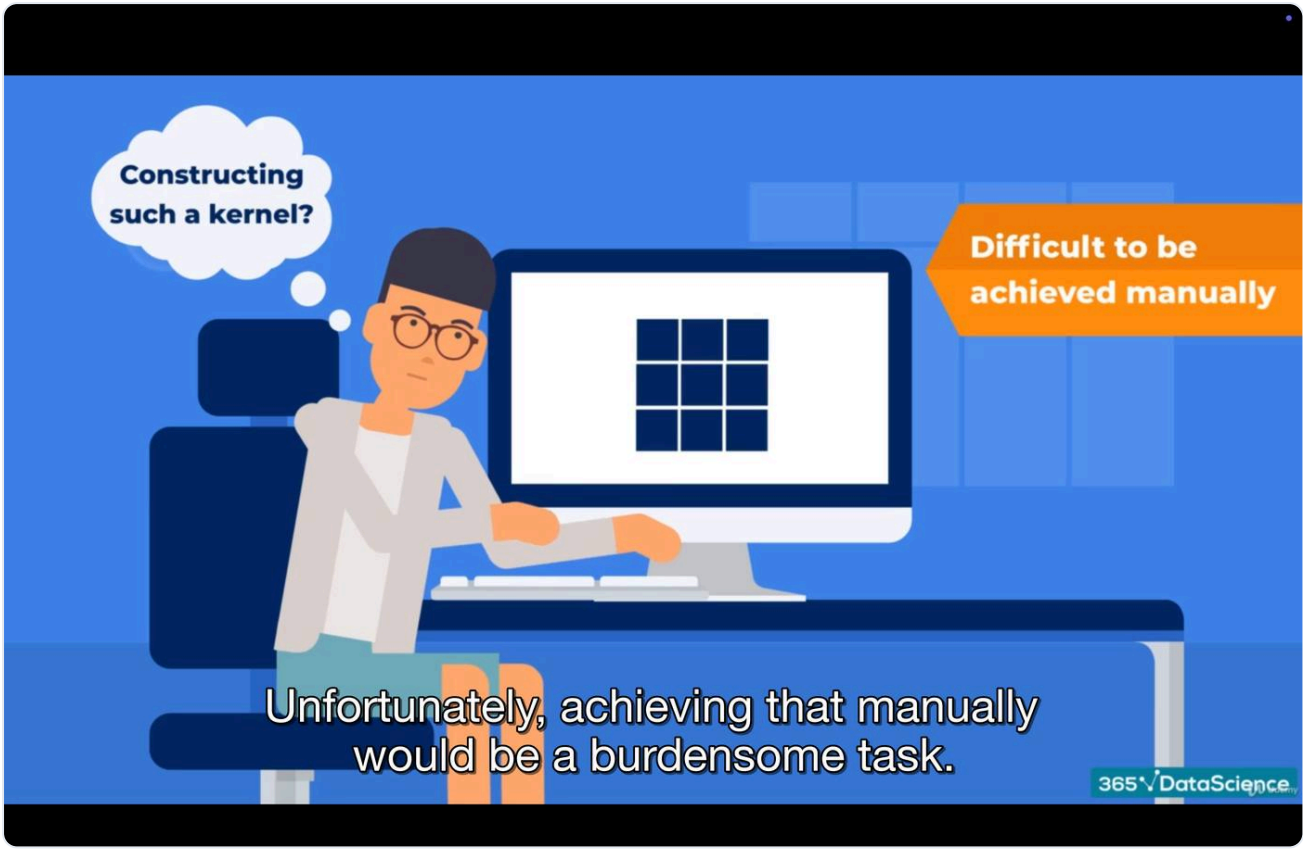

FIGURE 8.15

Unfortunately, achieving that manually would be a burdensome task.

FIGURE 8.16

Also, it will probably fail when being faced with different dog and cat breeds. What if we could find a way in which to create such a kernel automatically and reliably? What if we didn't even know which features of the image would be worth detecting?

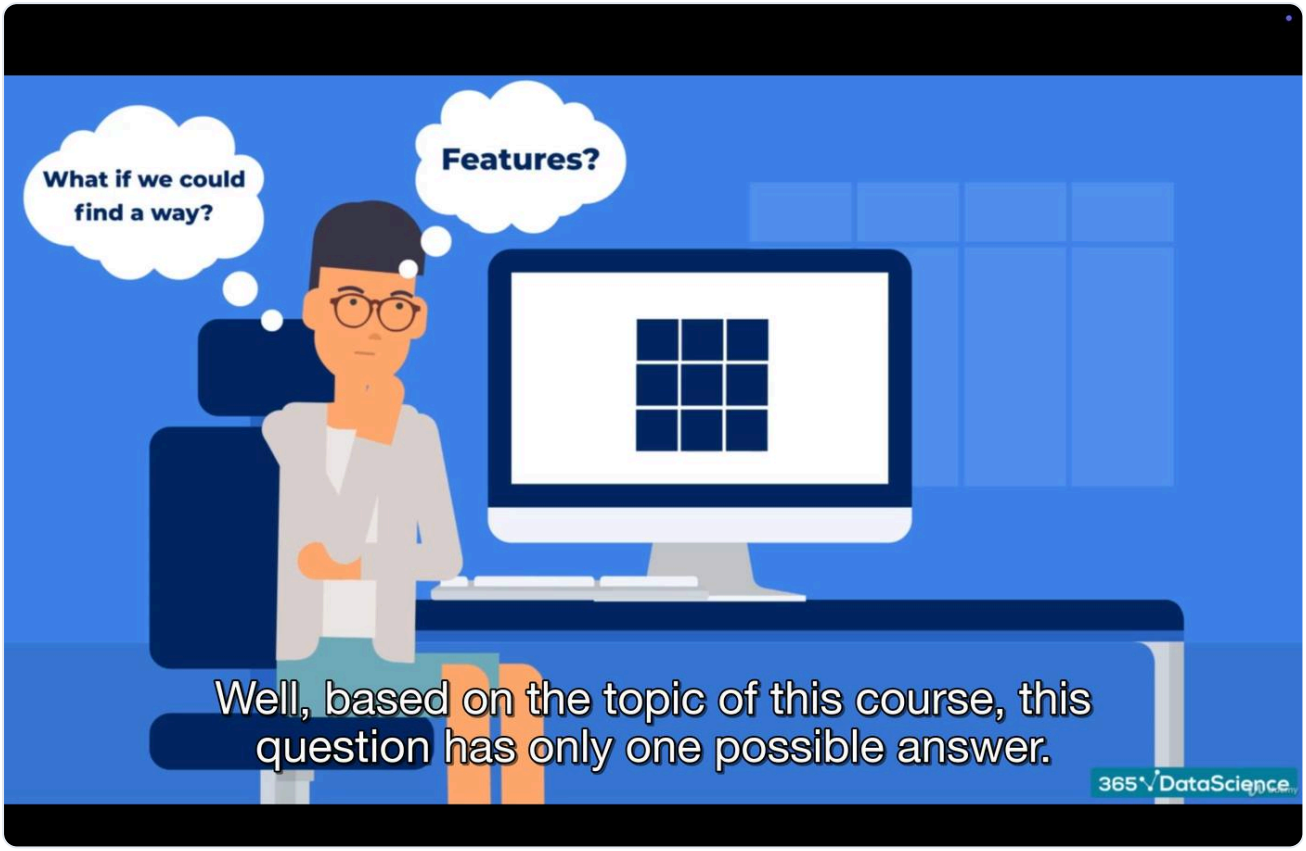

FIGURE 8.17 | What If We Could

Well, based on the topic of this course, this question has only one possible answer. We can do that and we will. We will let the computer learn what the best possible kernel to use is. On top of that, instead of having just one kernel, we can have hundreds of them since the computer does all the work. And that's it. That is the crucial revelation that leads to the concept of convolutional neural networks.

FIGURE 8.18

Those are neural networks that use kernels to transform the input in a more useful form and discover the best kernels to use on their own.

FIGURE 8.19

There are many more technical aspects and subtleties, but this is the general idea. As a side note, in the context of neural networks, kernels are often referred to as filters.

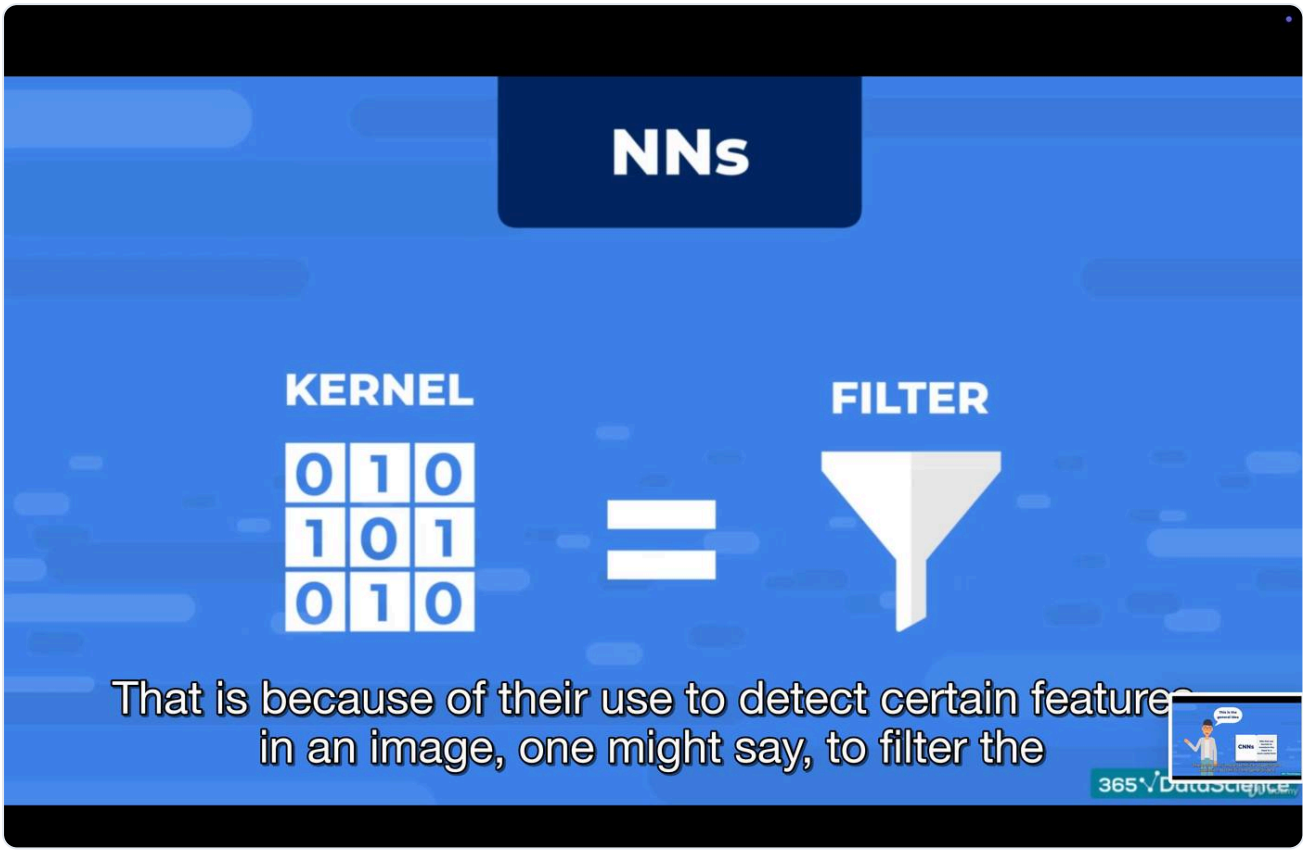

FIGURE 8.20

That is because of their use to detect certain features in an image, one might say, to filter the interesting features. In this course, we will use the two terms interchangeably.

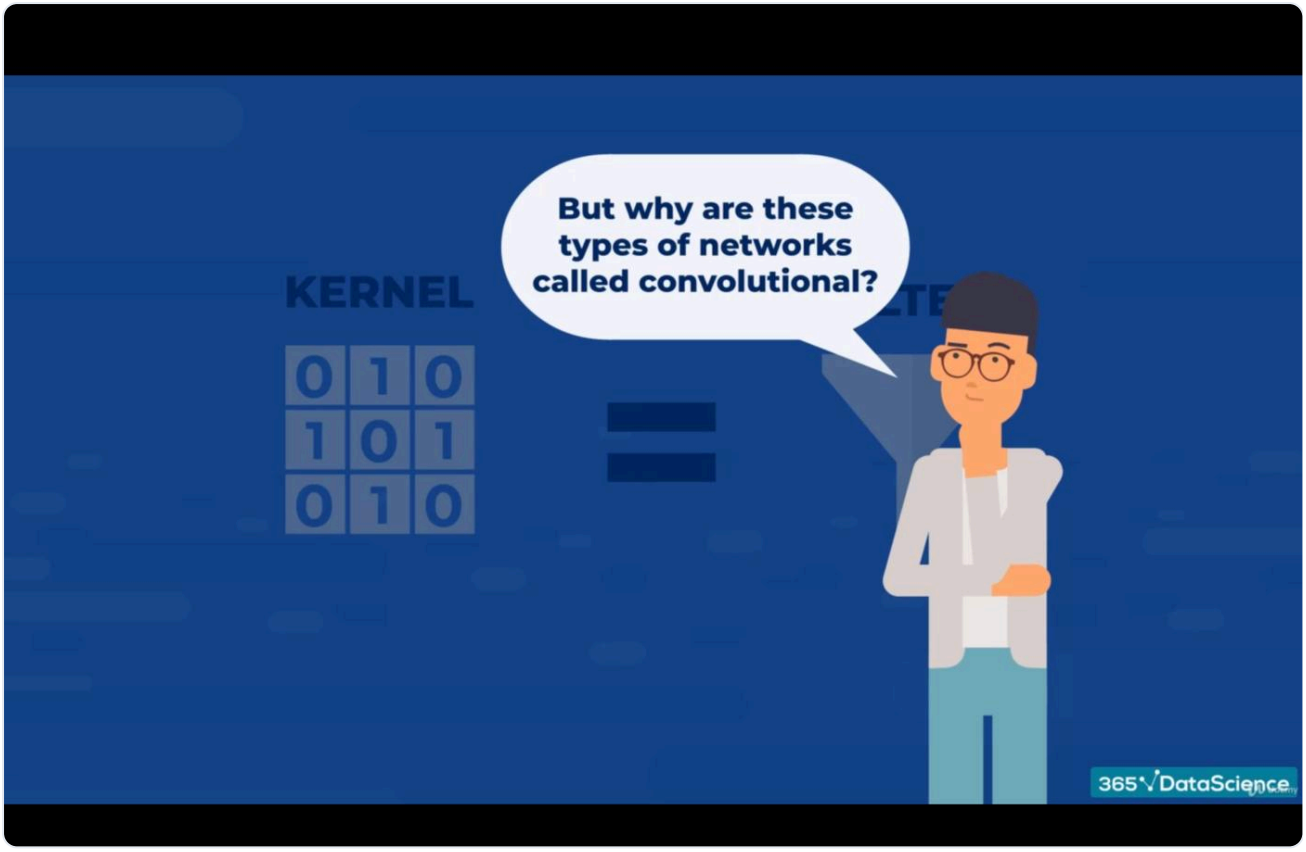

FIGURE 8.21 | But Why Are These

OK, but why are these types of networks called convolutional? Well, the name is related to how kernels are applied.

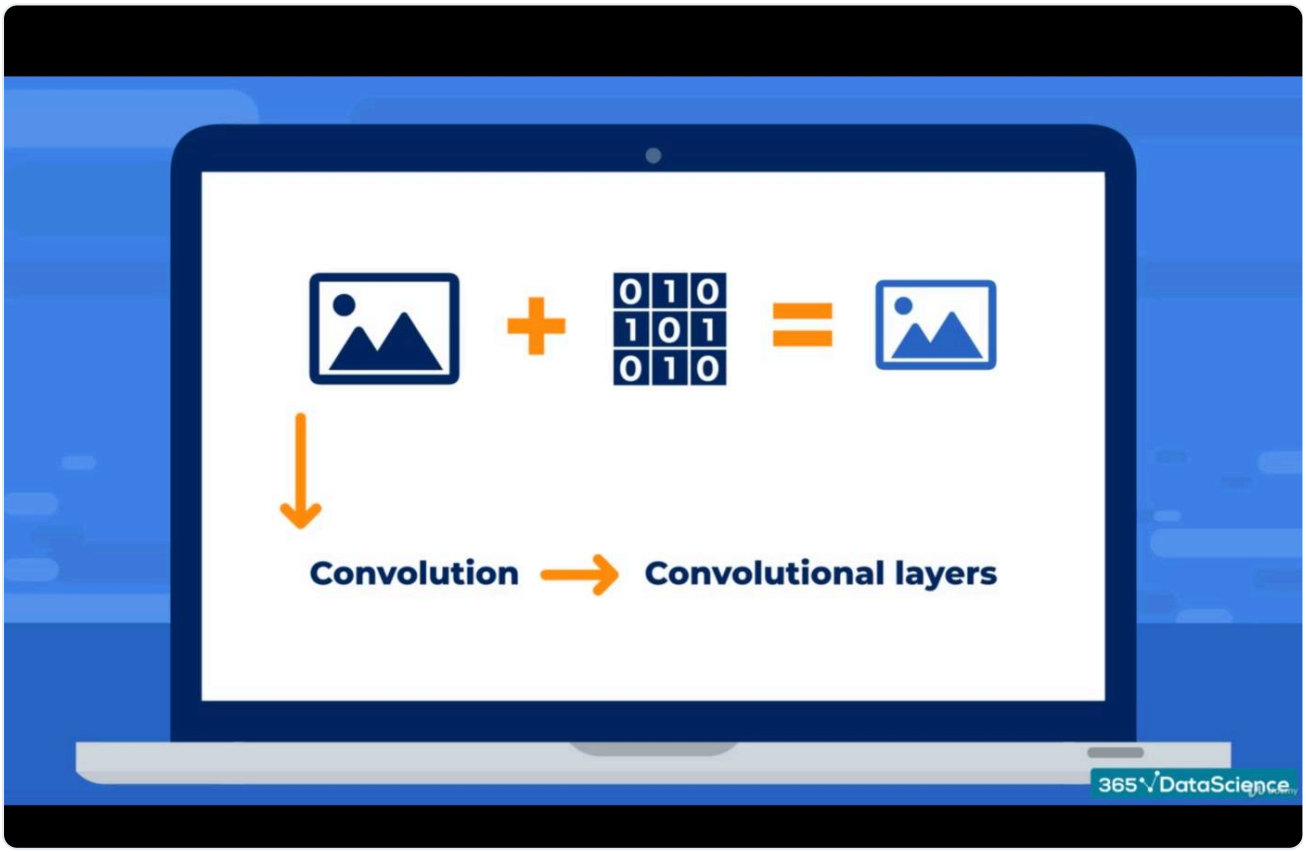

FIGURE 8.22 | Convolution Convolutional Layers

If you remember from our last section, in order to produce the transformed image, we convolve the original one and the kernel that is, we use the mathematical operation convolution subsequently in a neural network. The layers that use kernels are called convolutional layers.

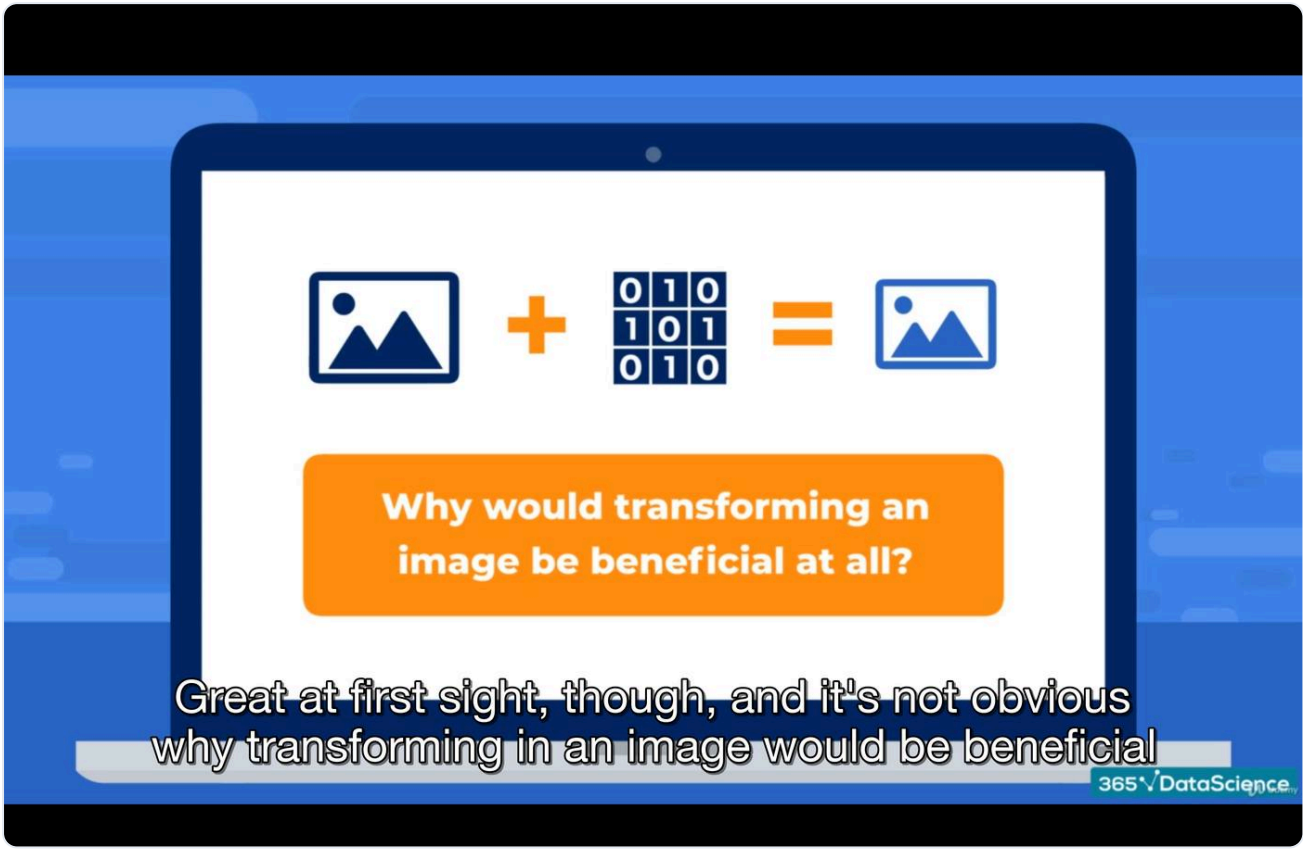

FIGURE 8.23

Great at first sight, though, and it's not obvious why transforming in an image would be beneficial at all.

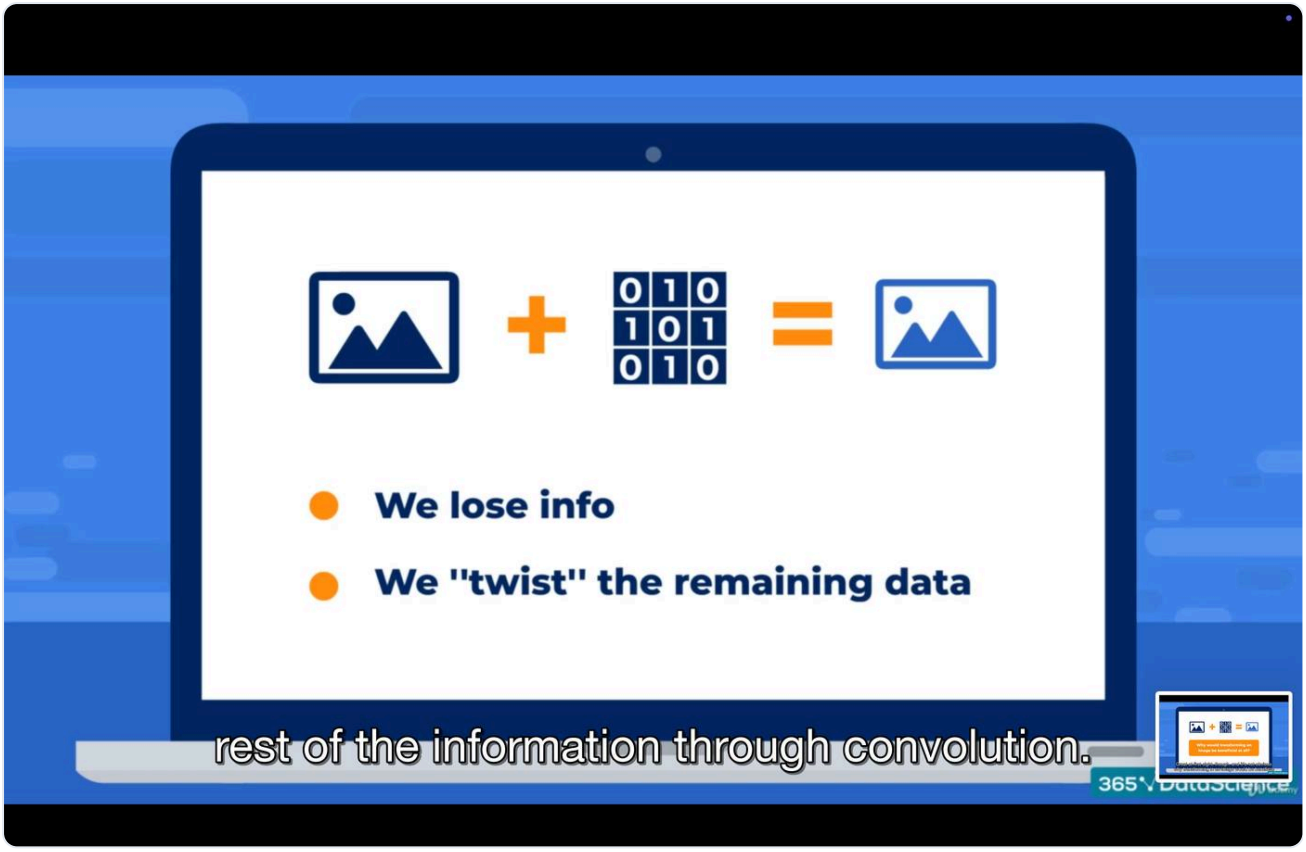

FIGURE 8.24

At the end of the day, when we transform it, we lose some information and even worse, we twist the rest of the information through convolution. So why don't we just use the whole image with the maximum information? Well, to answer this, let's take a step back and look at the next example we solved with deep networks in one of our case studies.

FIGURE 8.25 | Handwritten Digits 0-9

For those that don't know or don't remember MNIST, a data set of handwritten digits, zero to nine, consisting of seventy thousand images of size twenty eight by twenty eight pixels. In that exercise, the goal was to train a feed for neural network to recognize those digits.

FIGURE 8.26 | Train an NN to Recognize the Digits

FIGURE 8.27 | Train an NN to Recognize the Digits

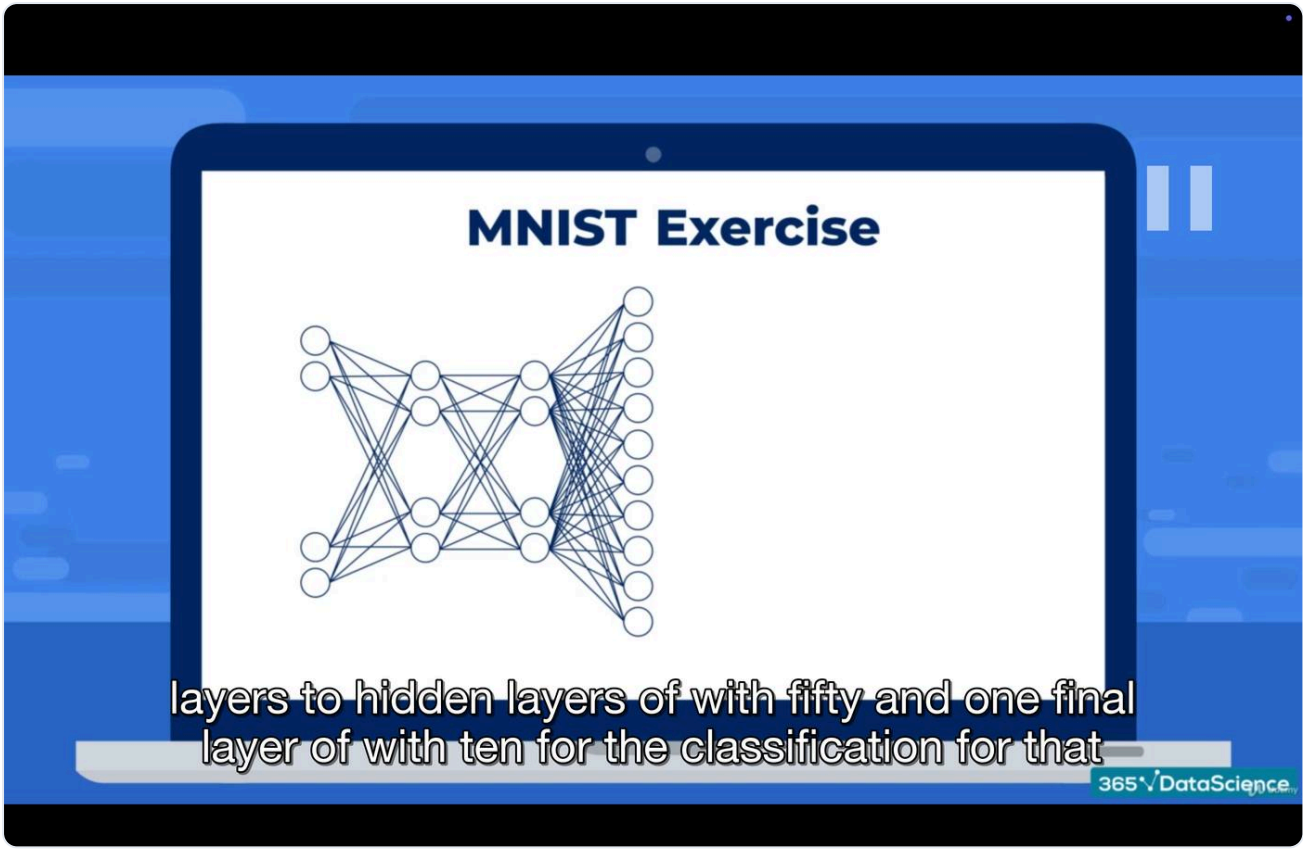

FIGURE 8.28

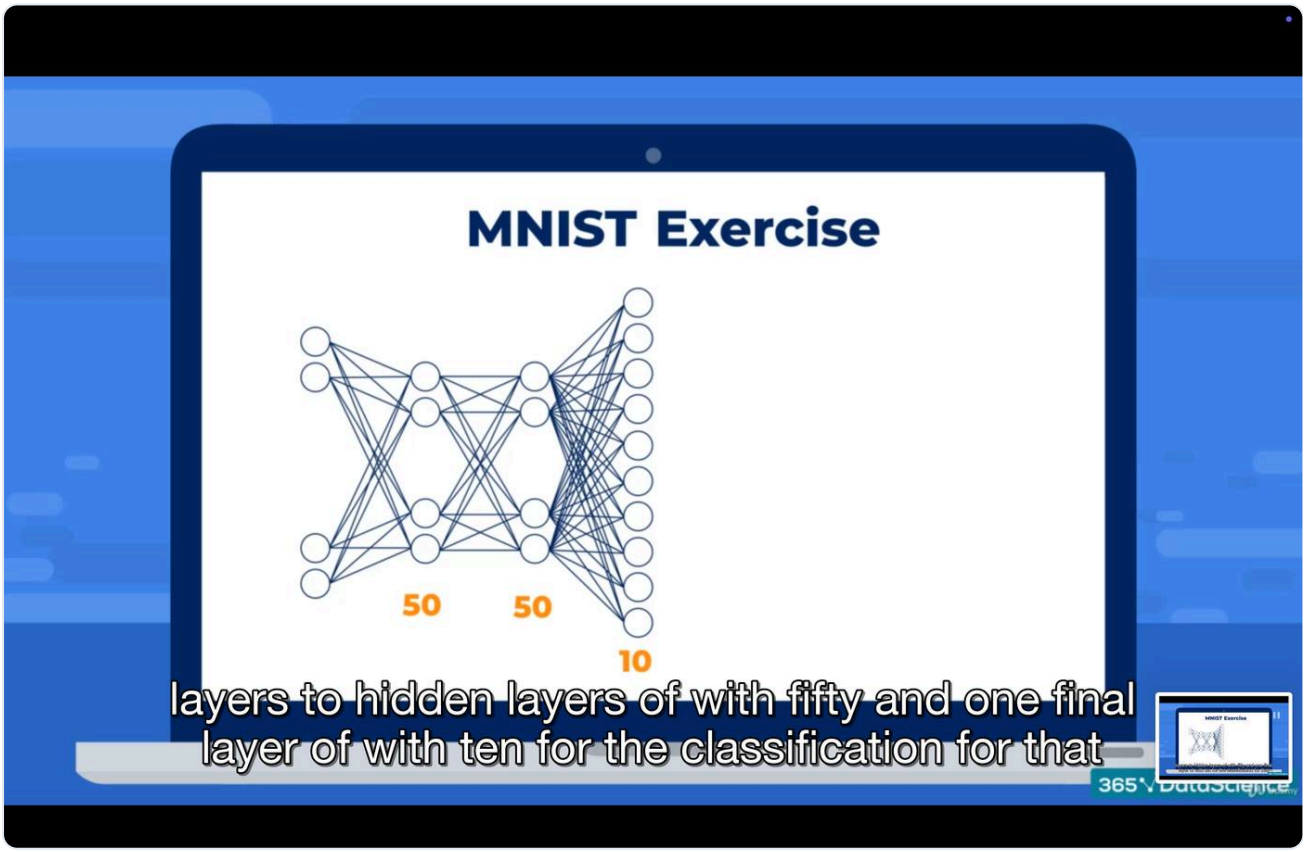

FIGURE 8.29

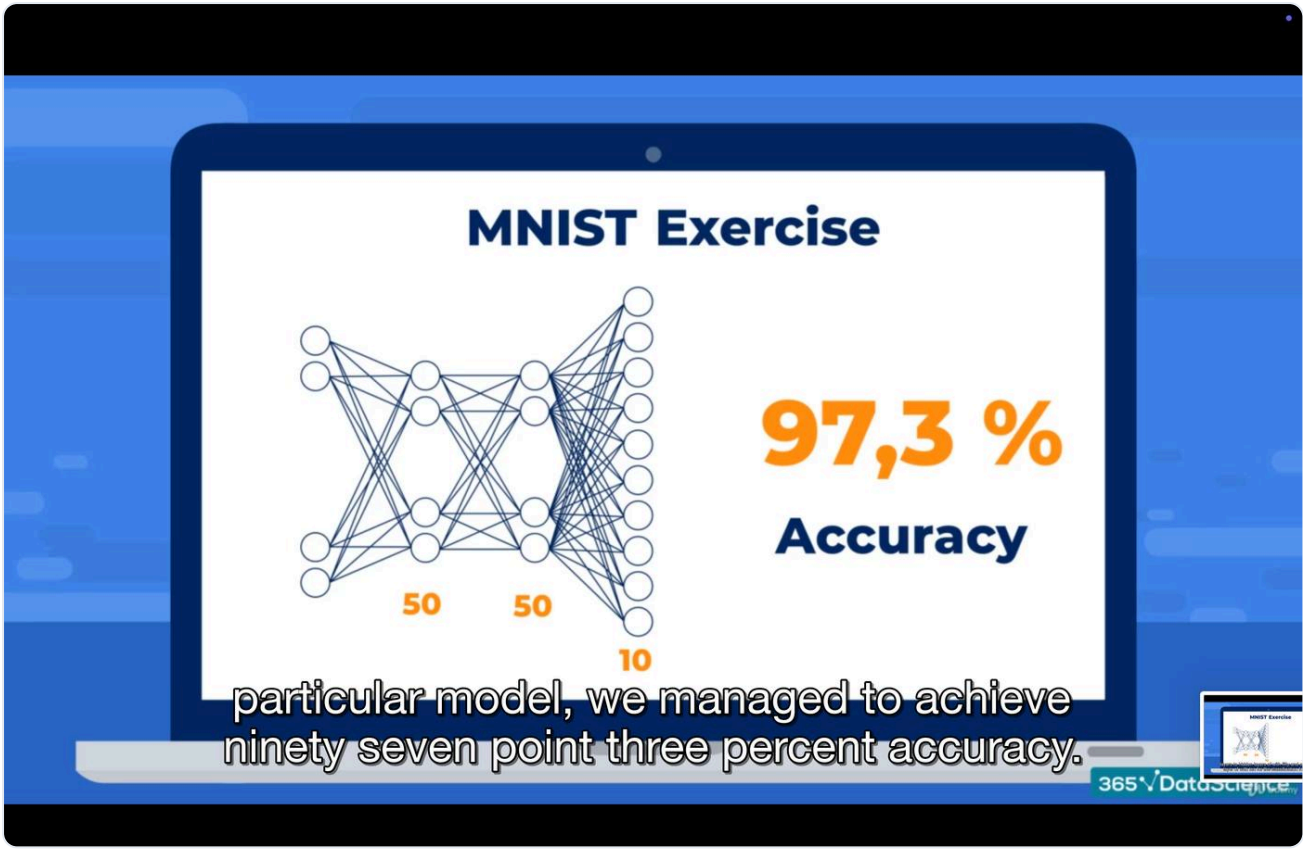

FIGURE 8.30

We achieved that by flattening the images into vectors and used a deep network with three fully connected layers to hidden layers of with fifty and one final layer of with ten for the classification for that particular model, we managed to achieve ninety seven point three percent accuracy. That was pretty amazing. So if we can achieve such great results with a deep model, why don't we just stick with it? Well, first, that is not that great of an accuracy. I don't get me wrong, it's good, but we can do much better.

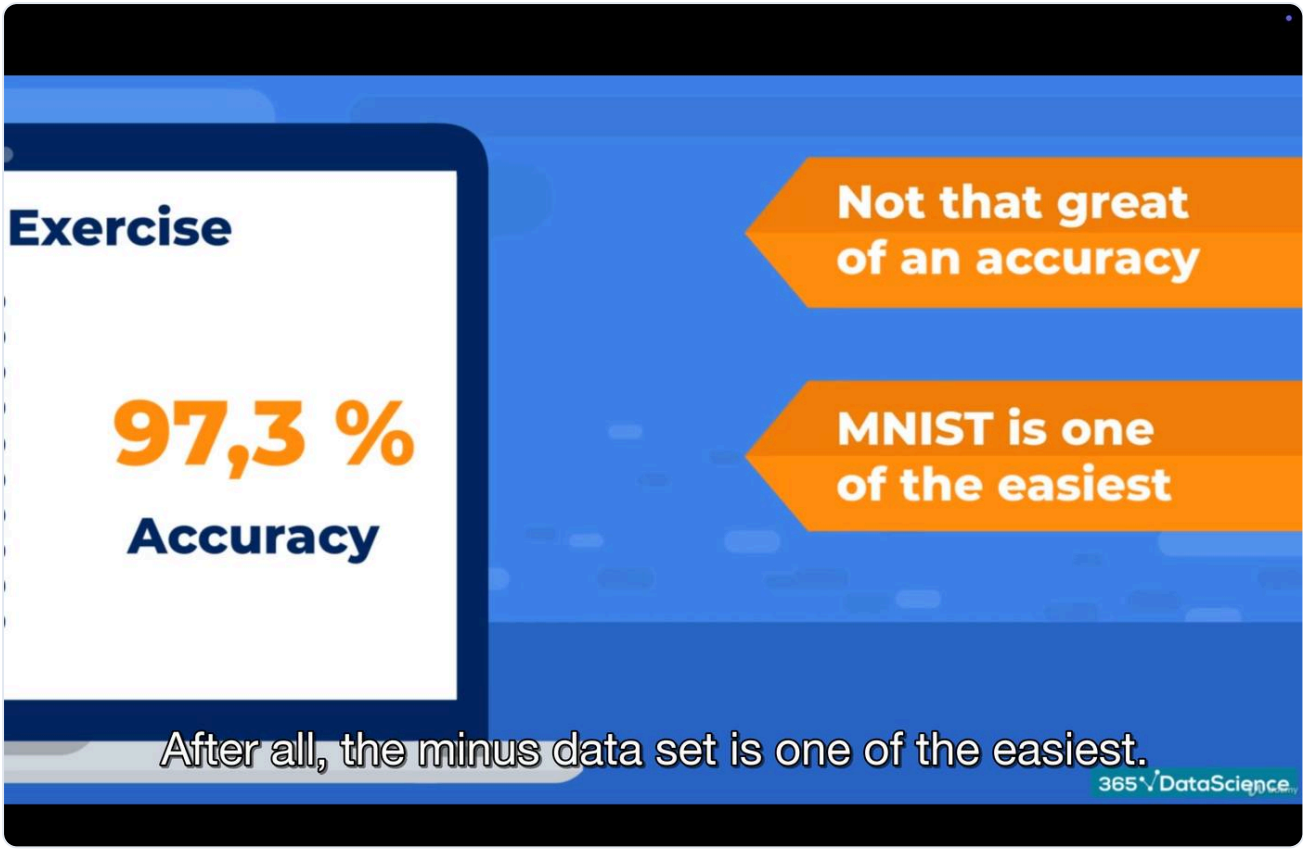

FIGURE 8.31 | Not That Great

After all, the minus data set is one of the easiest.

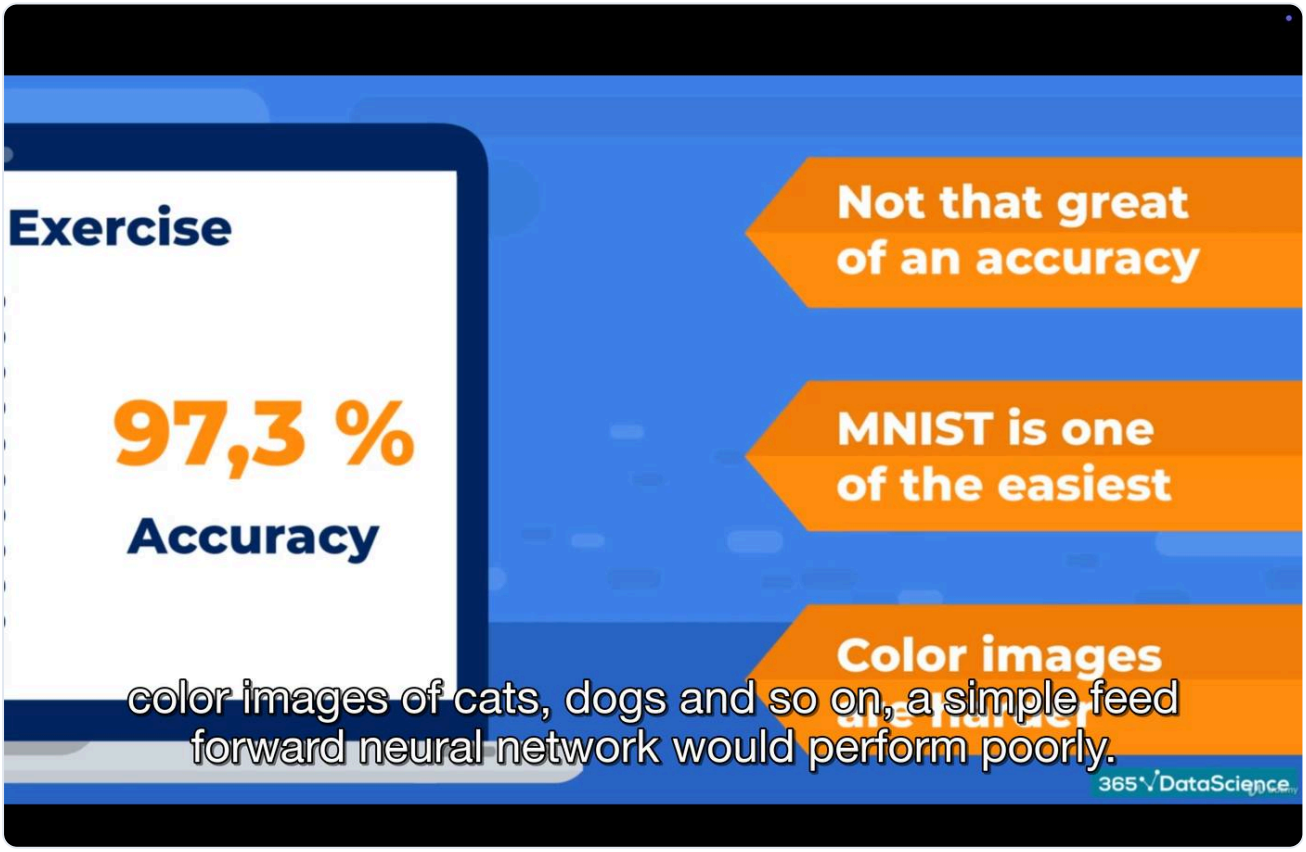

FIGURE 8.32 | Not That Great

Also, we've achieved this accuracy on a particularly simple grayscale data set when we get to real color images of cats, dogs and so on, a simple feed forward neural network would perform poorly.

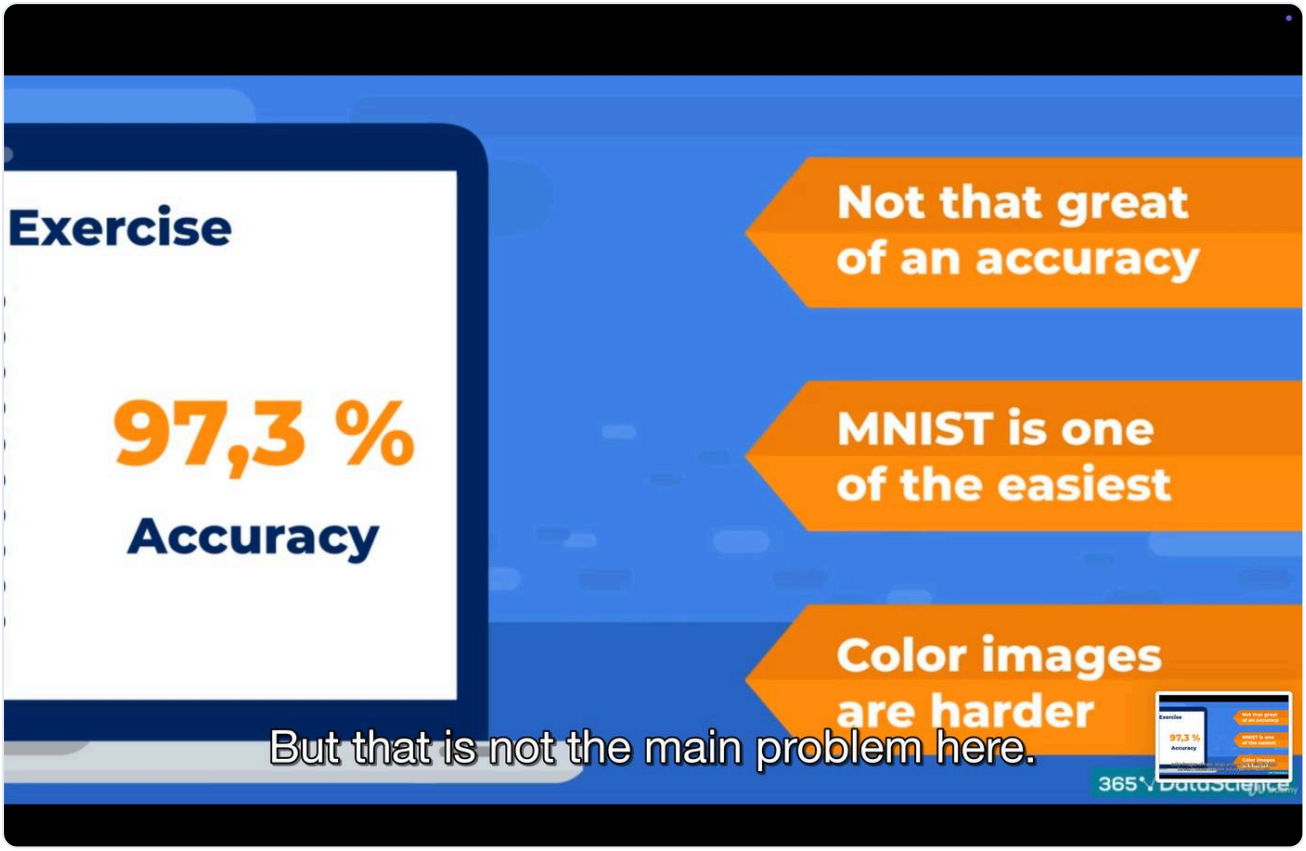

FIGURE 8.33 | MNIST Is One

But that is not the main problem here. The main issue comes from how the model treats the image, as you remember, to feed the twenty eight by twenty eight TensorFlow of pixels, we first unwrap it to a vector.

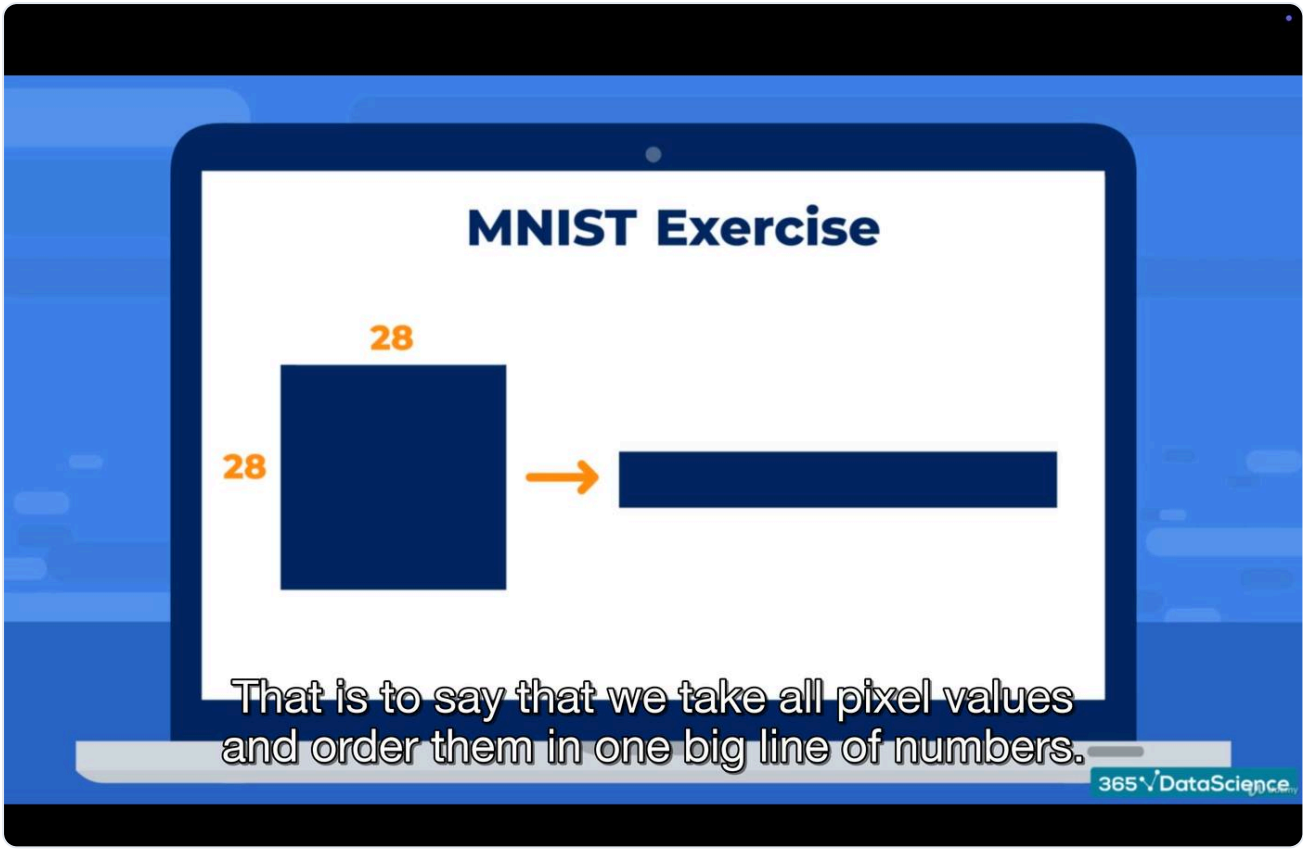

FIGURE 8.34

That is to say that we take all pixel values and order them in one big line of numbers.

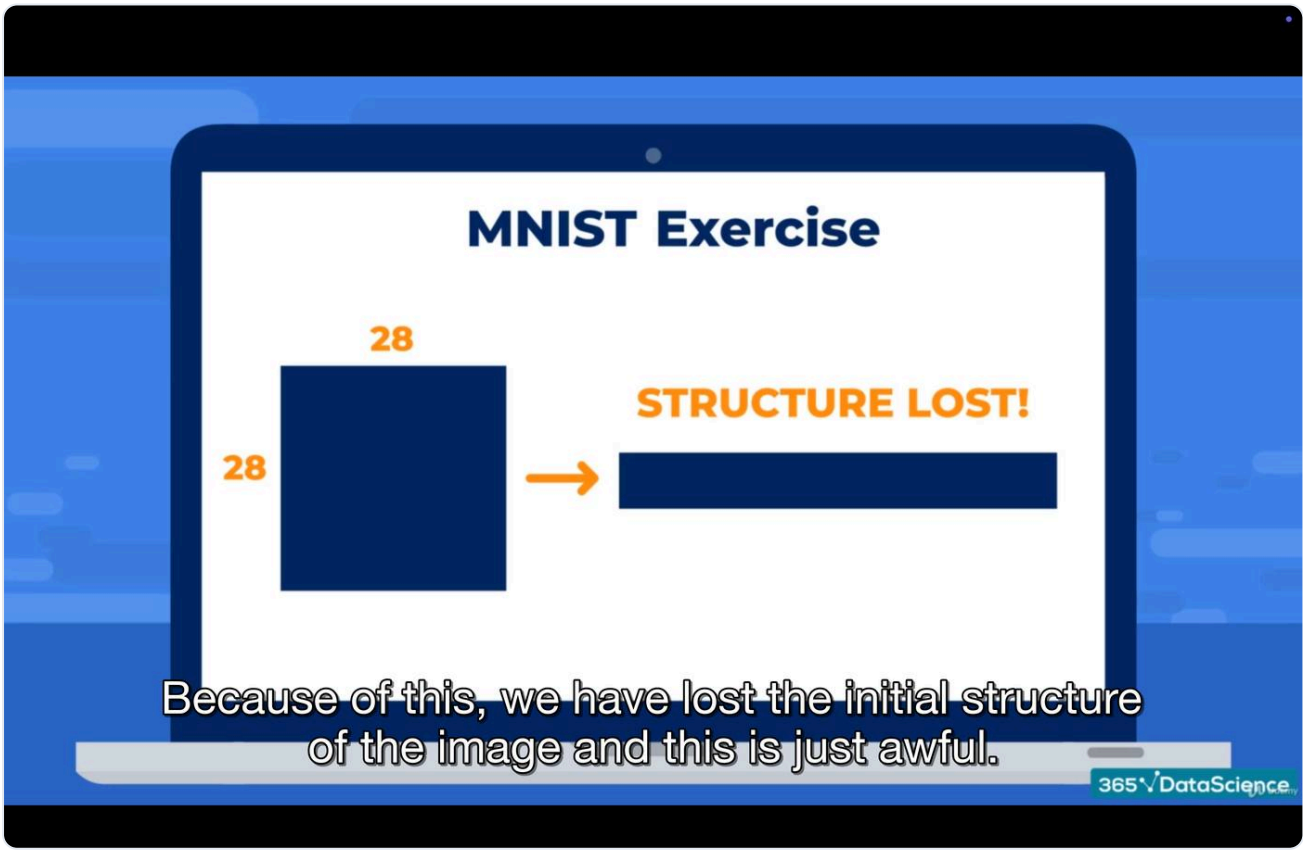

FIGURE 8.35

Because of this, we have lost the initial structure of the image and this is just awful. We have lost the spatial information.

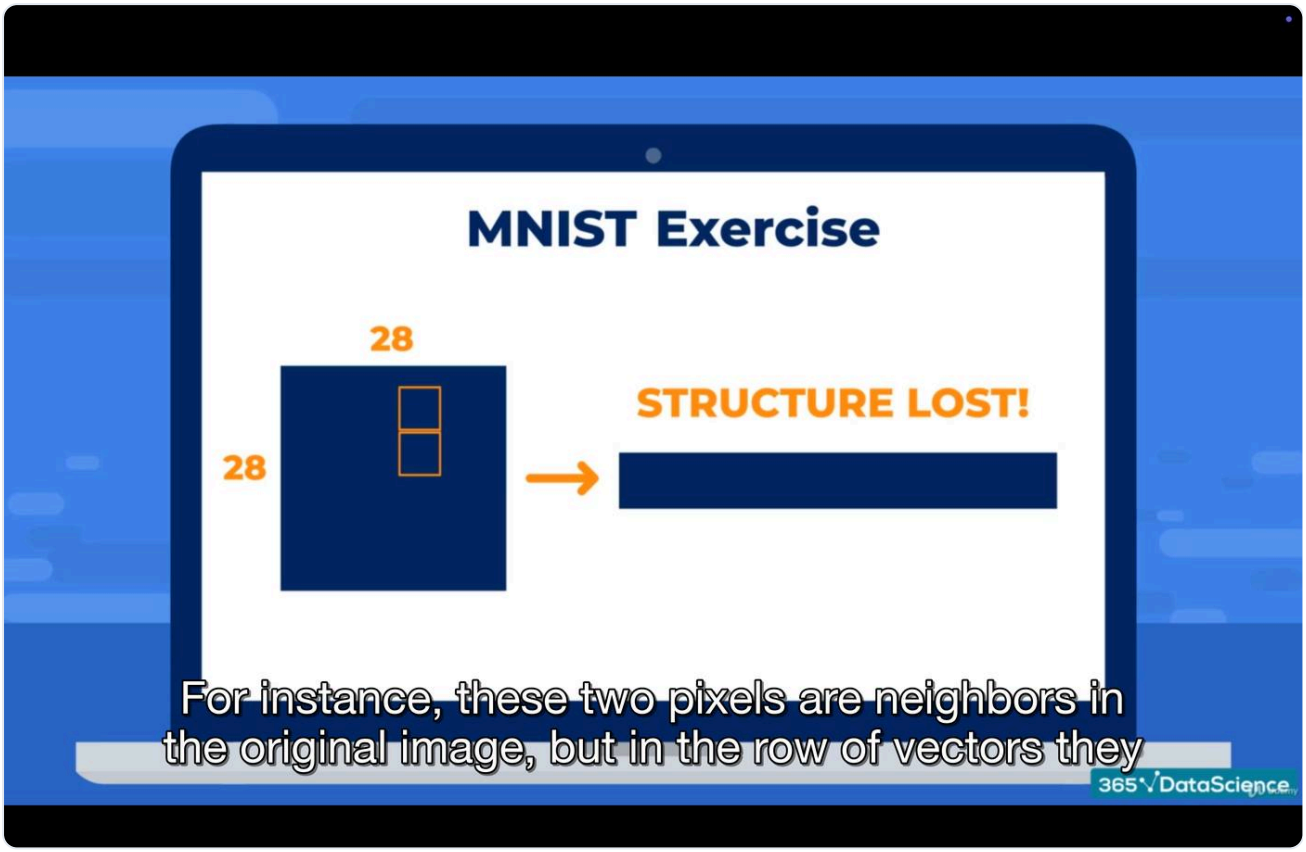

FIGURE 8.36

For instance, these two pixels are neighbors in the original image, but in the row of vectors they are pretty far away.

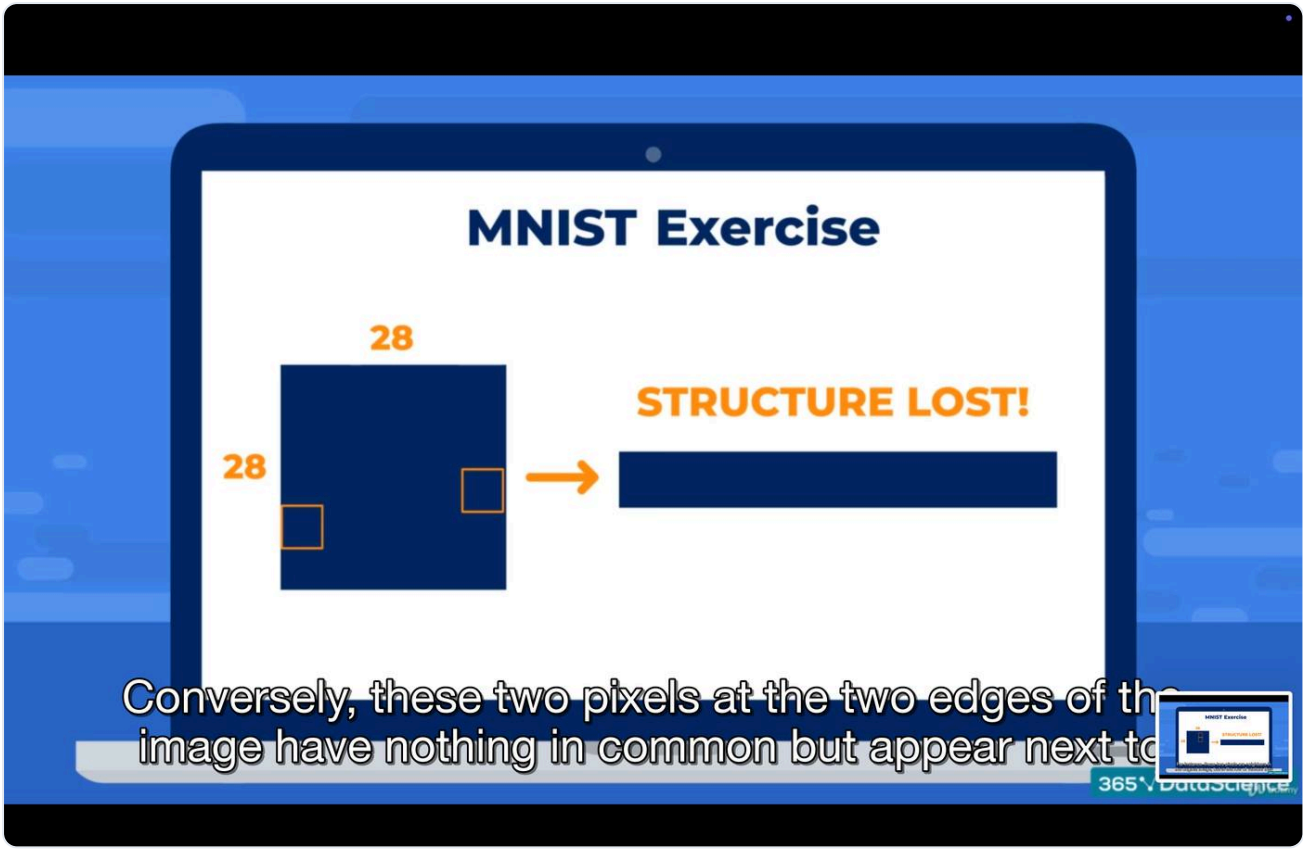

FIGURE 8.37

Conversely, these two pixels at the two edges of the image have nothing in common but appear next to each other in the flattened vector.

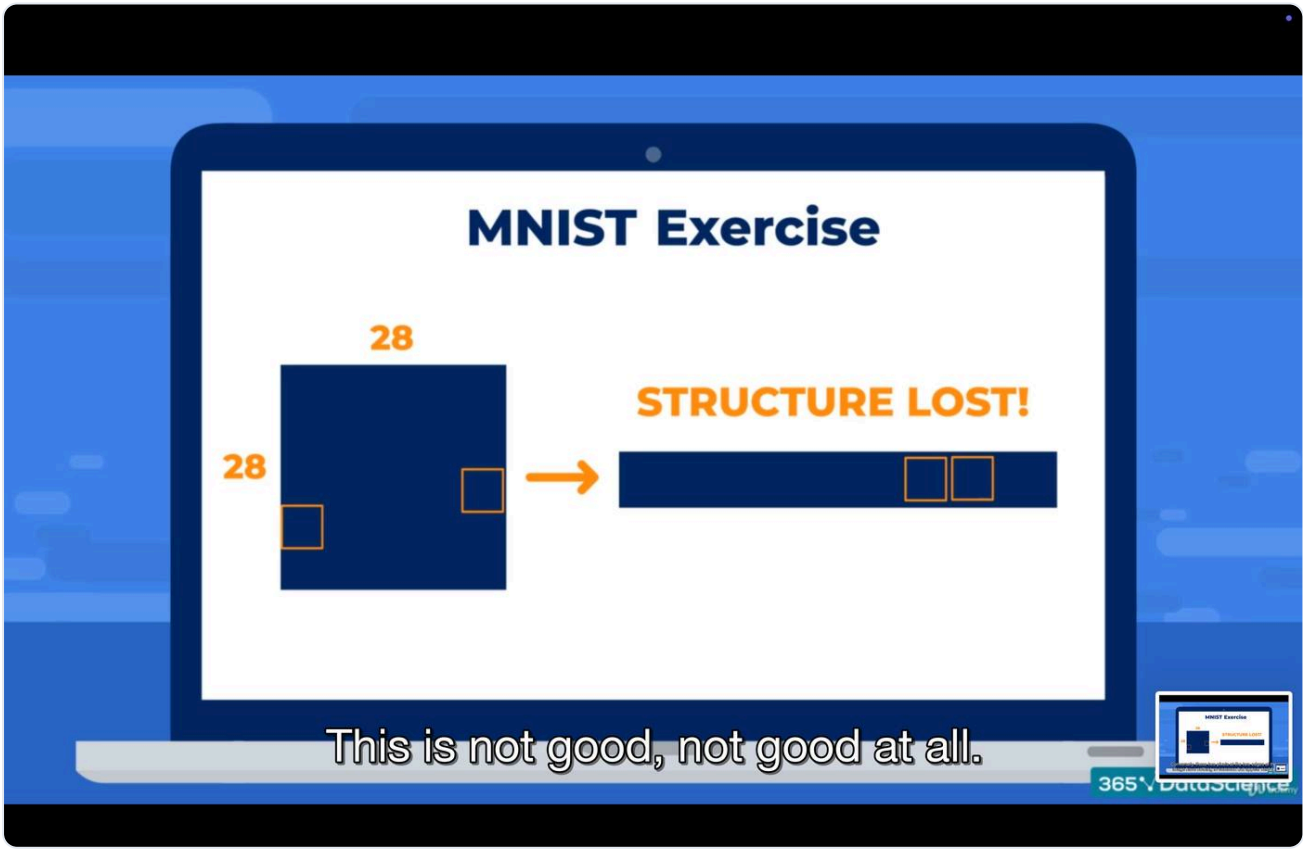

FIGURE 8.38

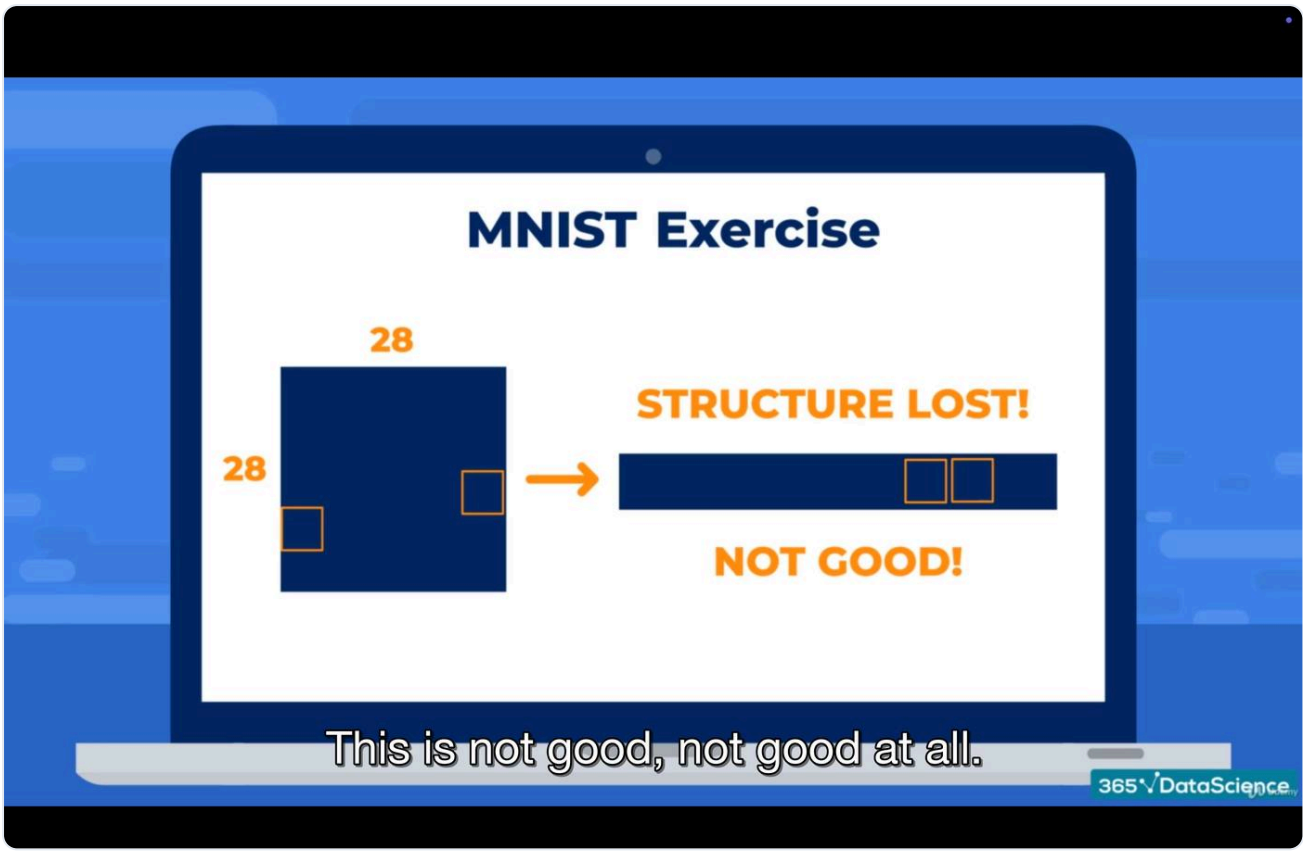

FIGURE 8.39

This is not good, not good at all. Our network doesn't actually see the image. It just sees a string of numbers.

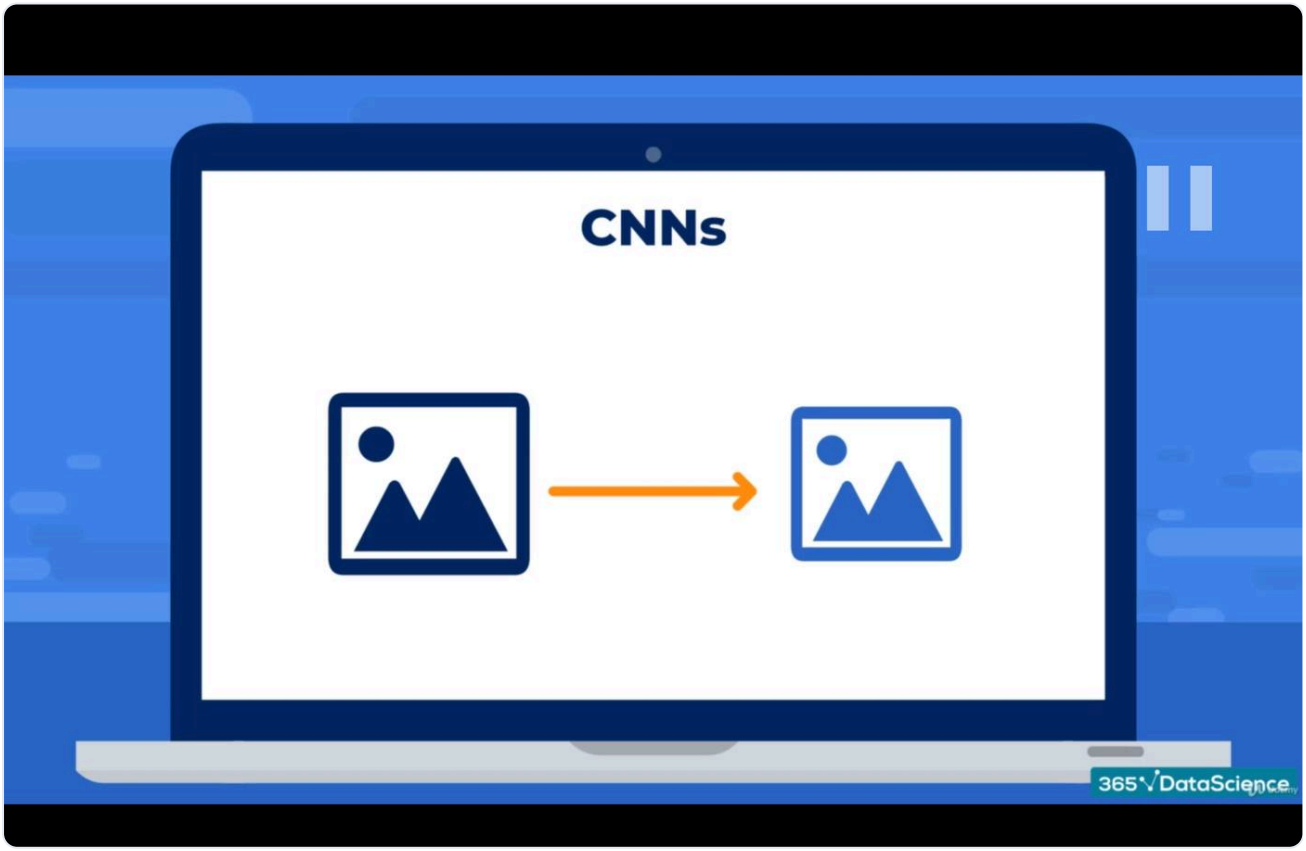

FIGURE 8.40

And that means that if we take an image that our model learn to classify and just move it a bit to the edge, the algorithm may be confused and no longer recognize it. You see, this method is just not good for inputs with an underlying structure like images enter scenes in scenes. We transform the image while preserving its structure.

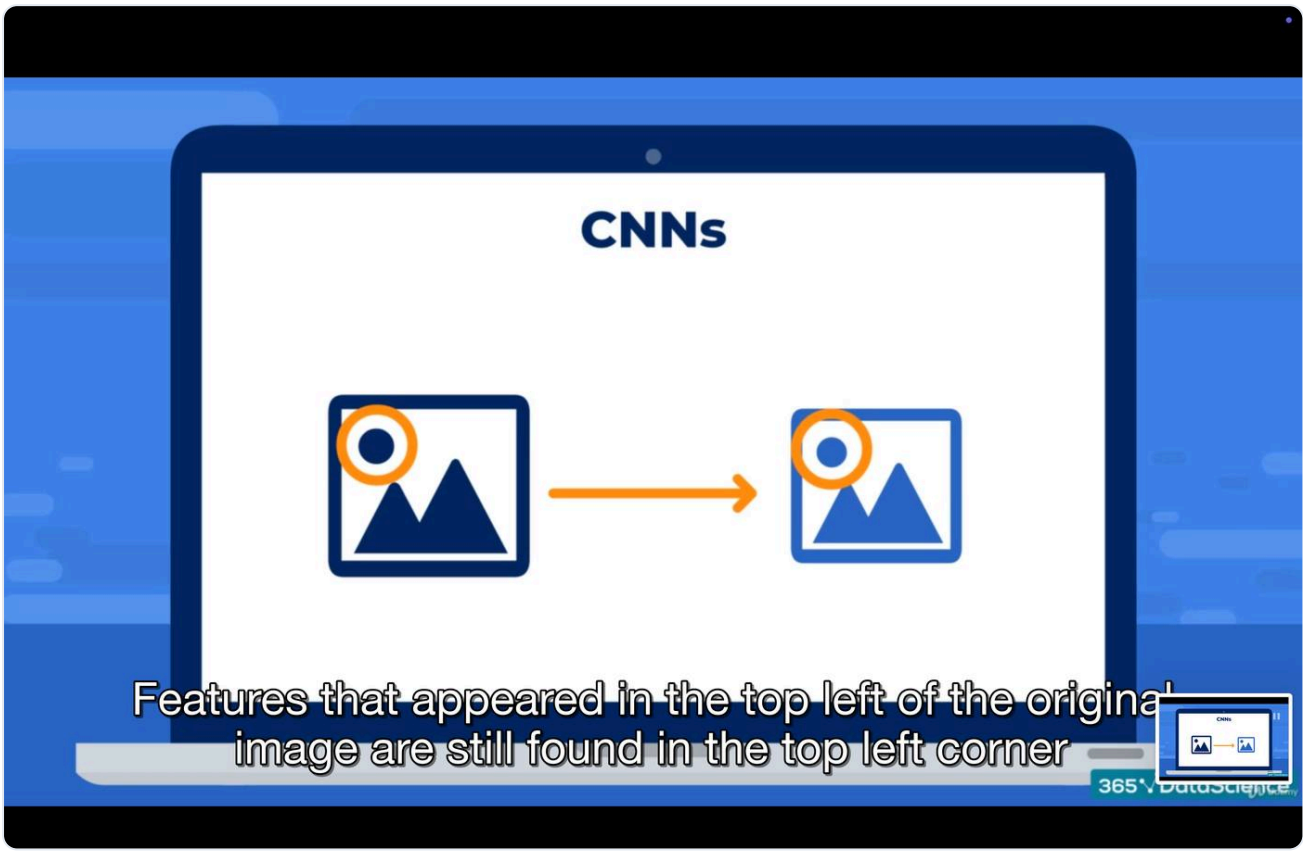

FIGURE 8.41

Features that appeared in the top left of the original image are still found in the top left corner of the transformed image. We don't unfold the image to a vector until the very last layer as an input.

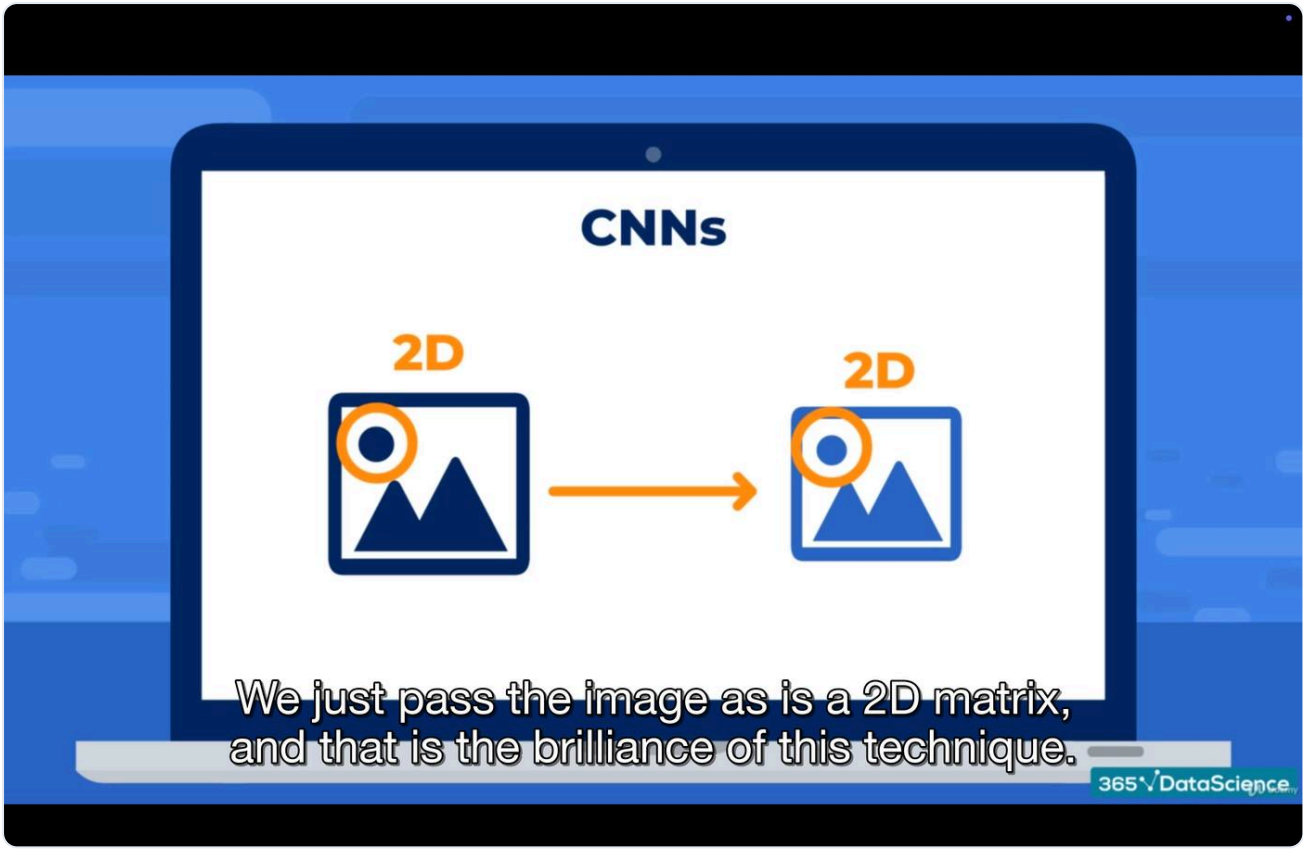

FIGURE 8.42

We just pass the image as is a 2D matrix, and that is the brilliance of this technique.

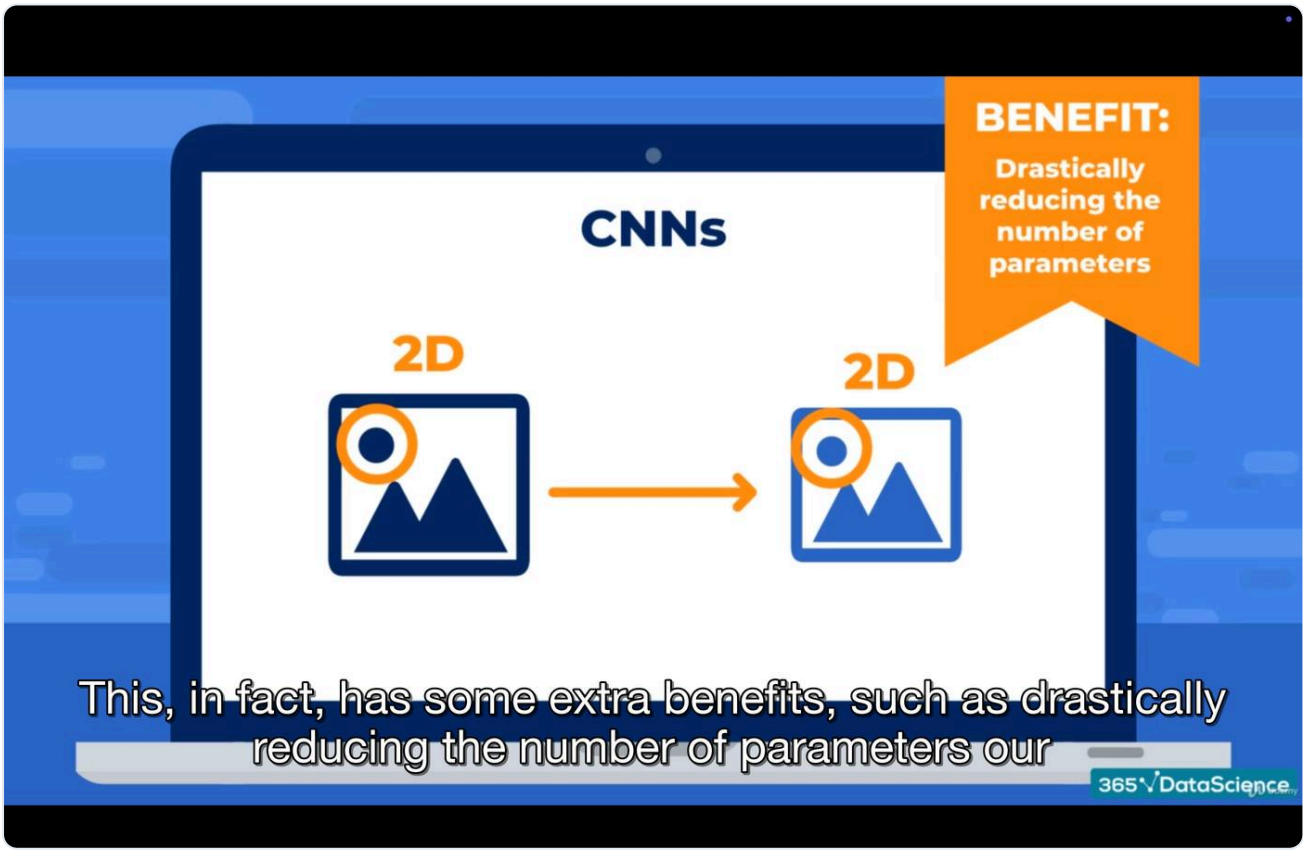

FIGURE 8.43

This, in fact, has some extra benefits, such as drastically reducing the number of parameters our network needs to optimize. Imagine you want to build a self-driving car for this purpose.

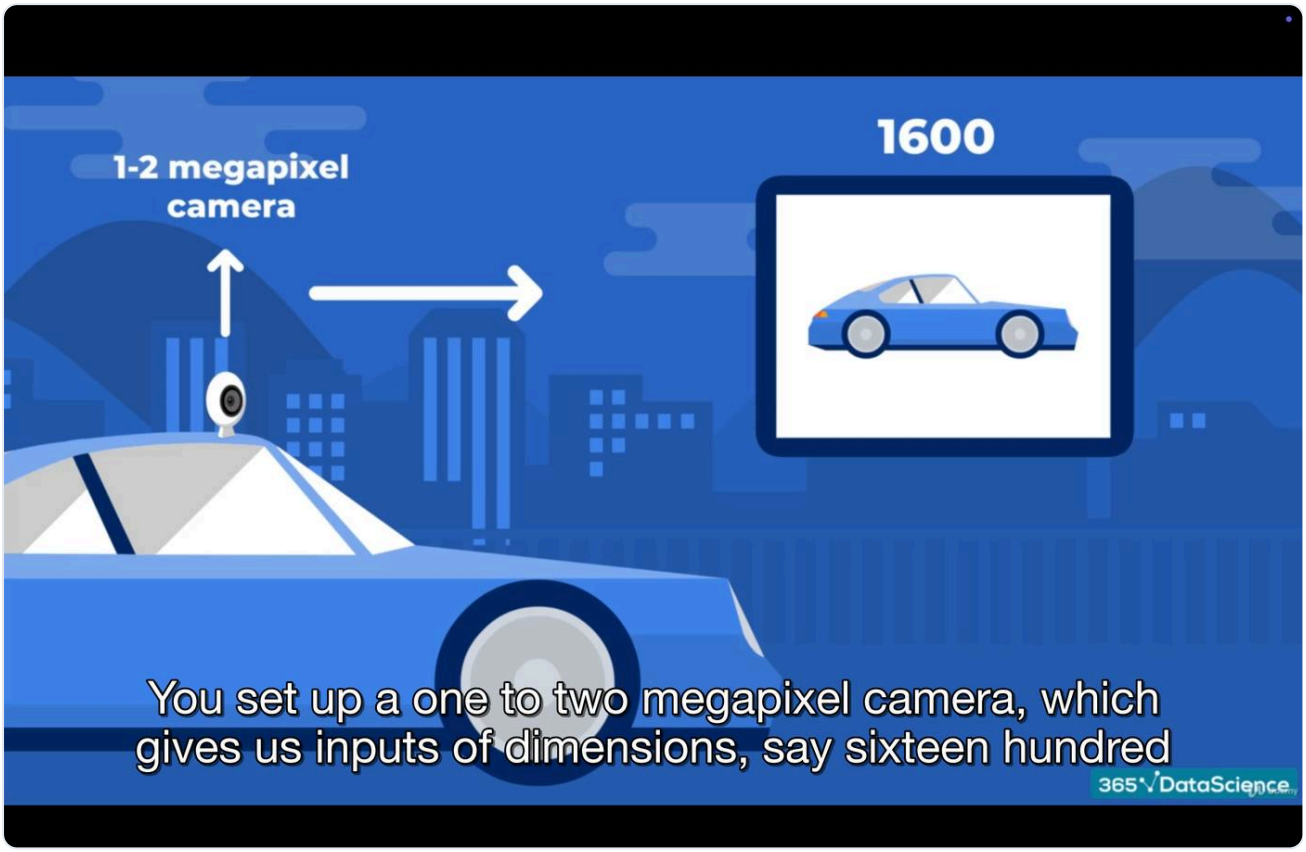

FIGURE 8.44

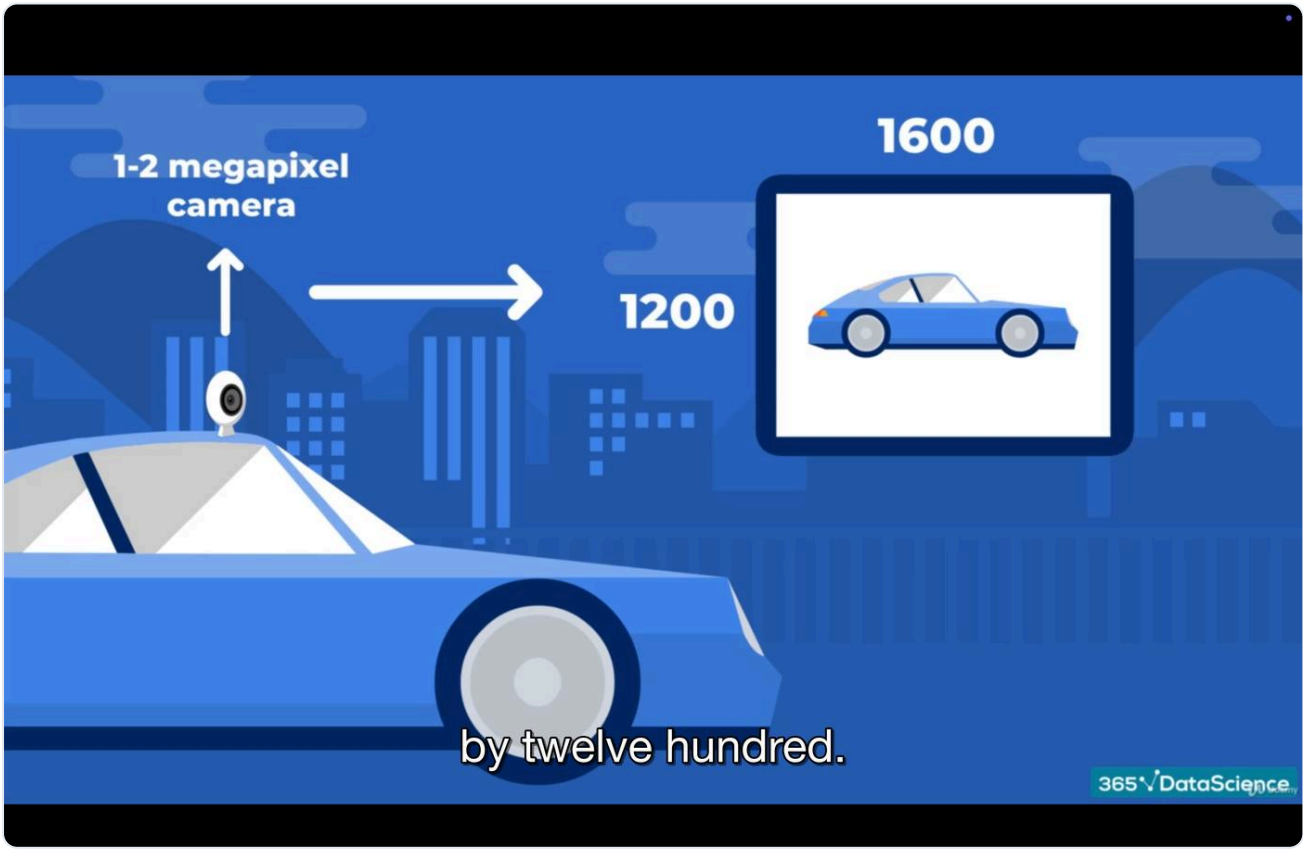

FIGURE 8.45

You set up a one to two megapixel camera, which gives us inputs of dimensions, say sixteen hundred by twelve hundred. Pretty standard. Now consider a single fully connected feed forward layer acting on this input that reduces its dimensionality by half. That would require us to introduce the staggering amount of nine hundred and twenty one billion weights to our model just for a single layer. Meanwhile, a convolutional layer with a kernel of size five by five would require only twenty six weights. Twenty five for the kernel matrix and one additional weight for the bias. There is no need to elaborate further on the computational advantages of scenes.

You may argue that a single convolutional layer has less predictive power than the fully connected feed forward. And however, if we were to match the number of parameters, we could in principle stack thirty five billion convolutional layers with twenty five parameters each to reach the nine hundred and twenty one billion weights of the simple and in practice a less extreme version of that set up with just a couple of convolutional layers. Does the trick right now you understand what the motivation behind the concept of convolutional neural networks is. So it is time to get a little bit more practical. Join us in the next lecture where we are going to talk about local receptive field and feature maps.

Thanks for watching.

LECTURE 9

Feature Map

So far, we discussed why we need CNN when dealing with visual data. Now we are going to talk about how they are implemented in practice. As already stated, the crucial part of a CNN are its convolutional layers in that layer.

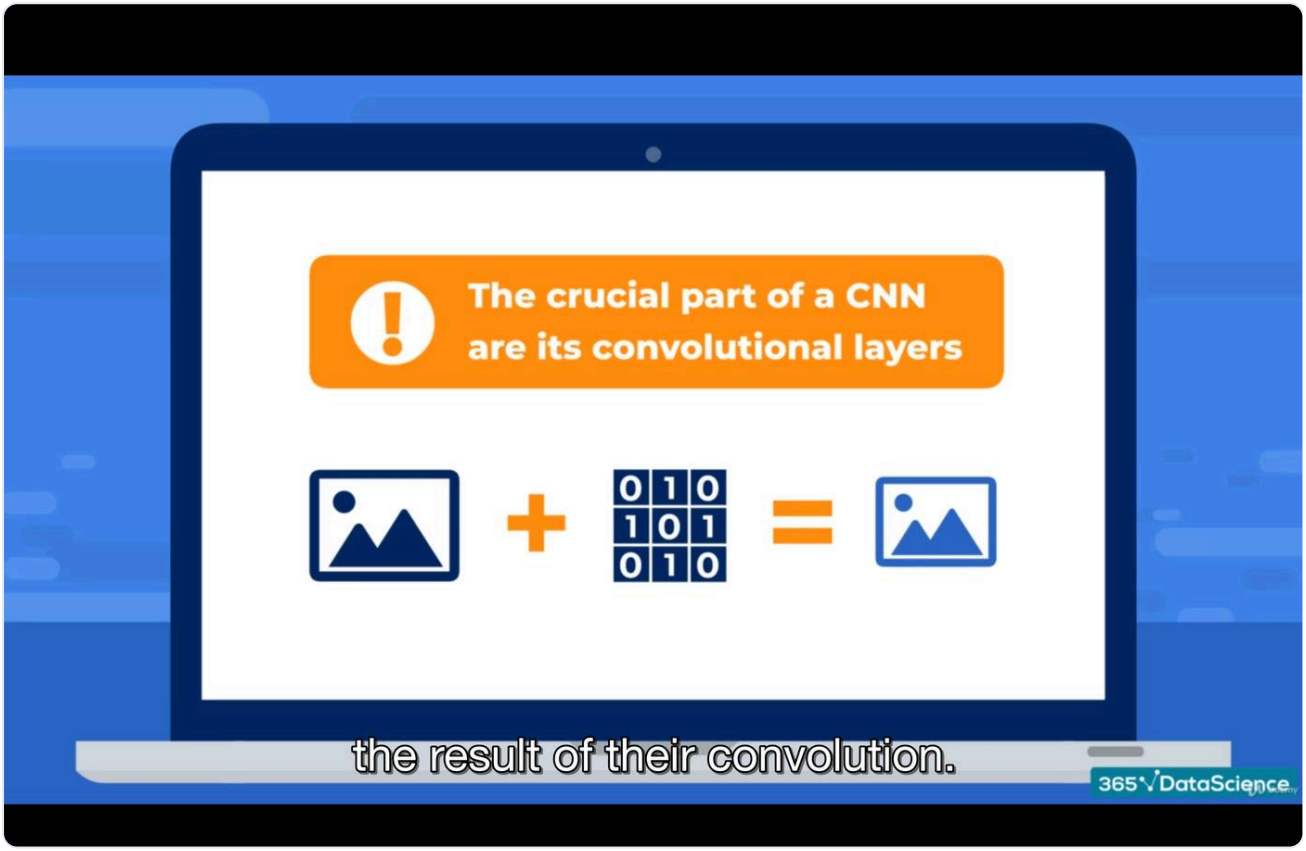

FIGURE 9.1

We have our image as an input, the kernel matrix with weights sometimes referred to as a filter and the result of their convolution. Let's visualize how that works one more time to make it clearer. First, we get the image of interest.

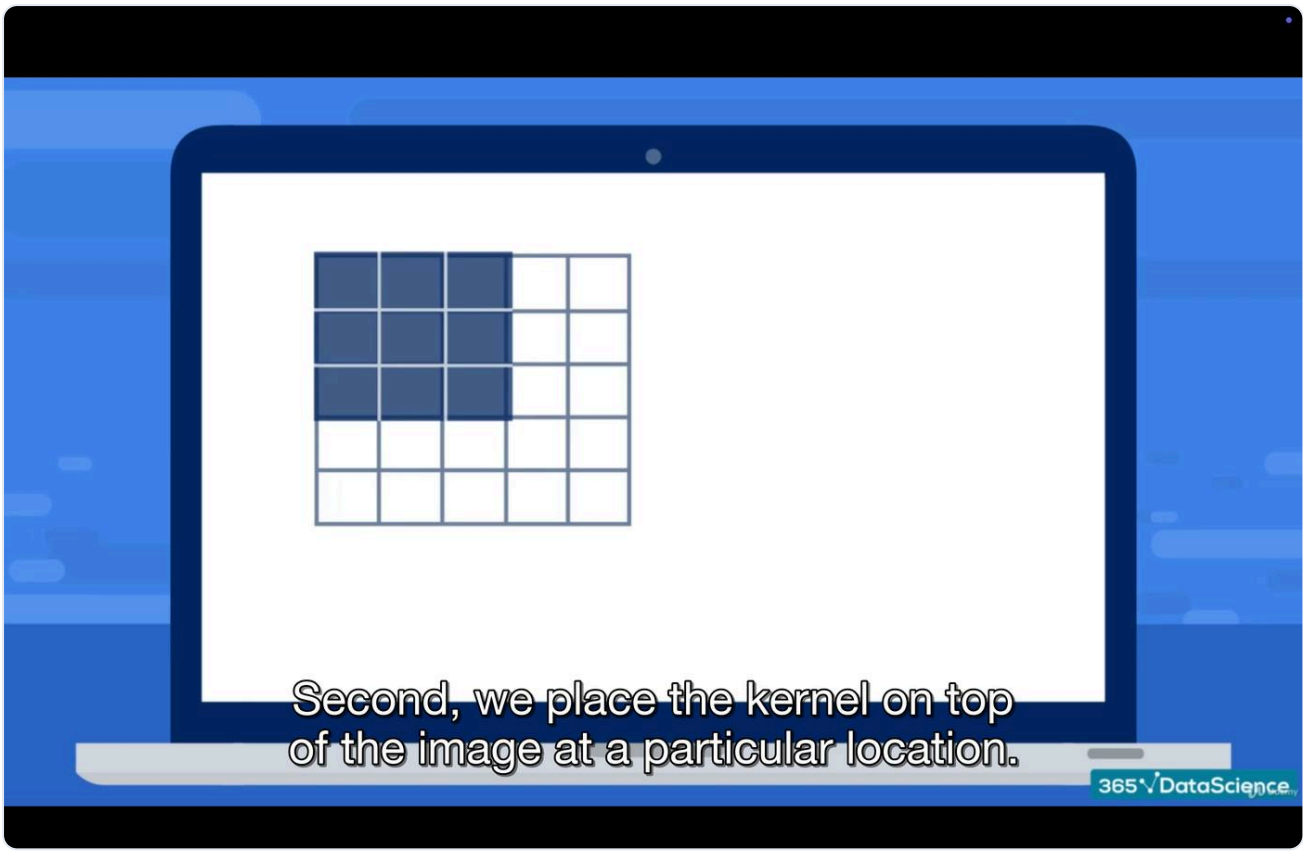

FIGURE 9.2

Second, we place the kernel on top of the image at a particular location. Then all pixel values are multiplied by the parameters of the kernel and some together to produce a single number.

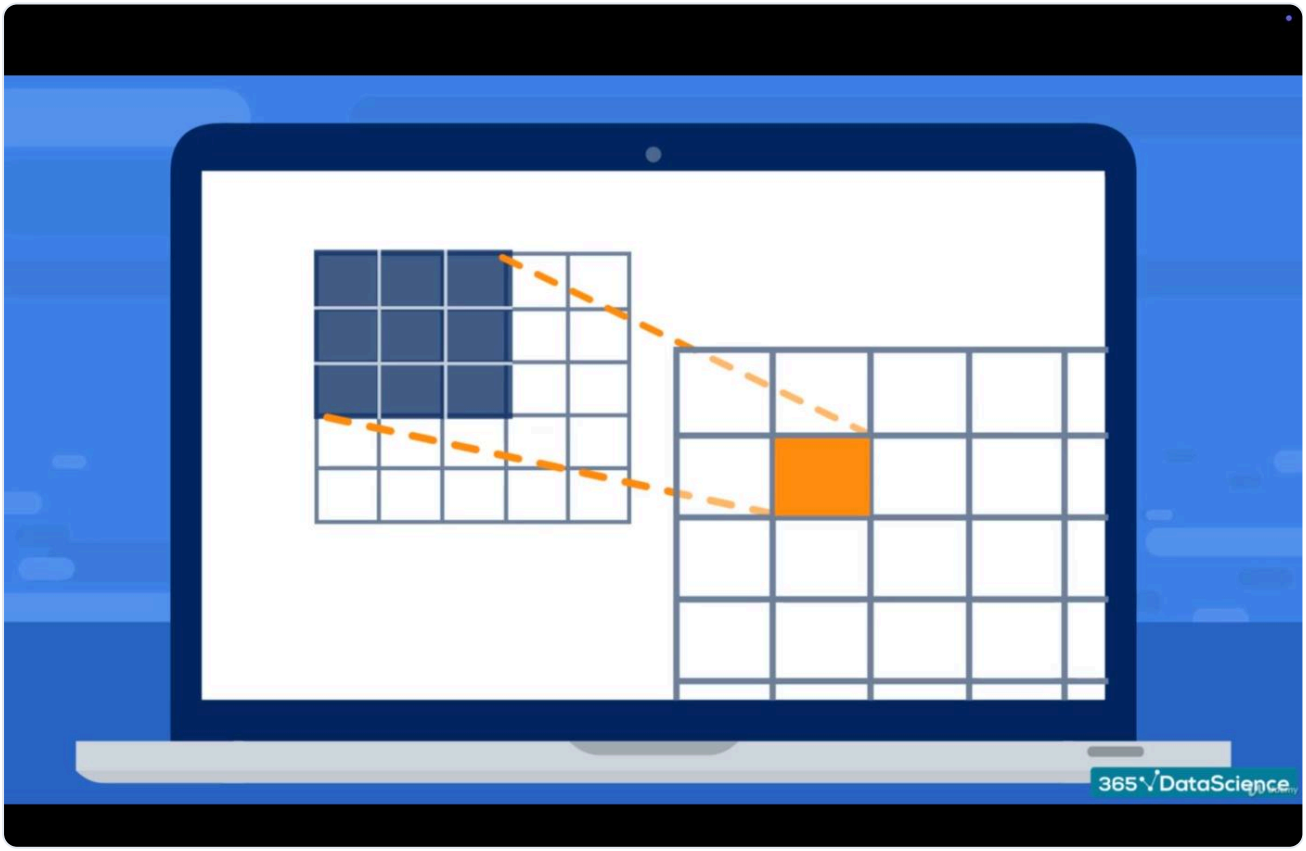

FIGURE 9.3

That number is the new pixel value at the position we place the kernel.

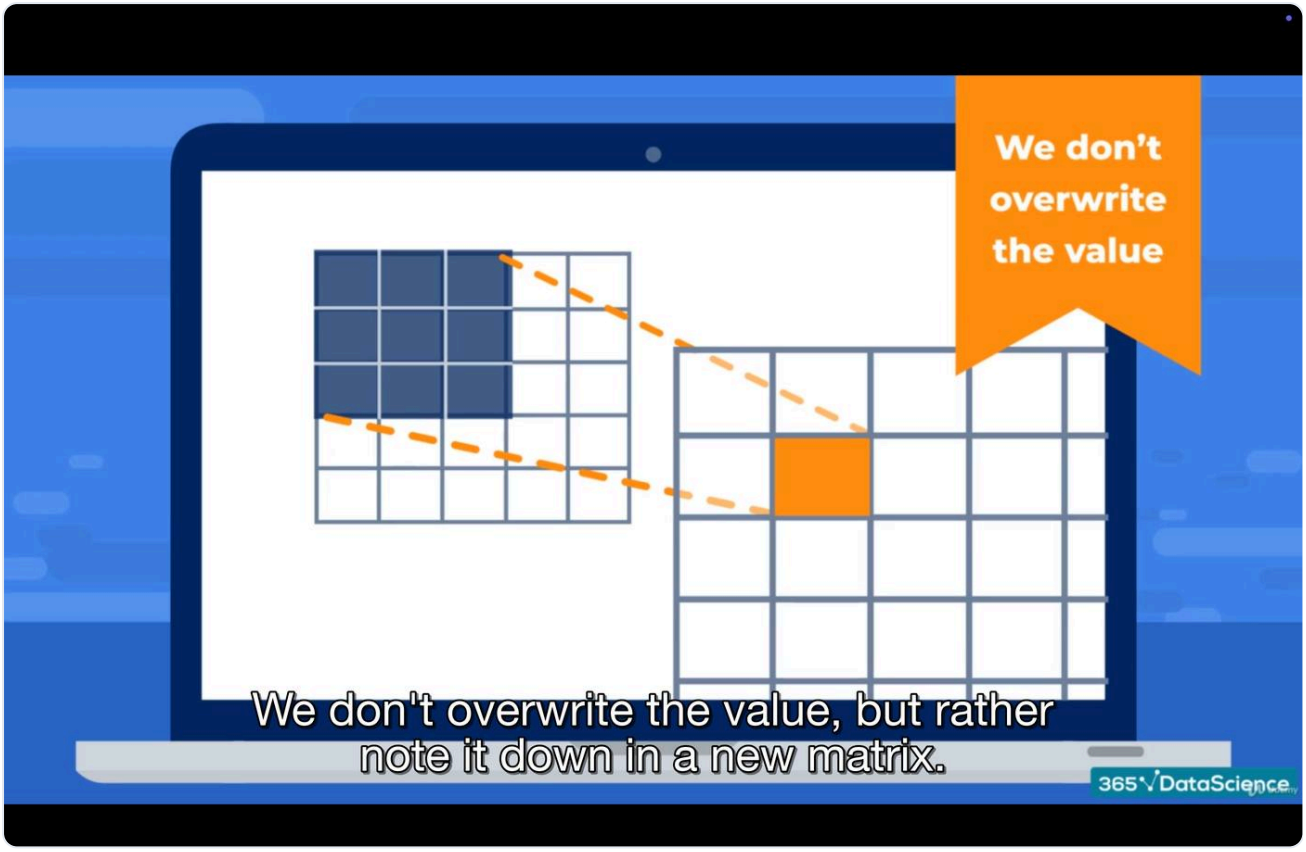

FIGURE 9.4

We don't overwrite the value, but rather note it down in a new matrix.

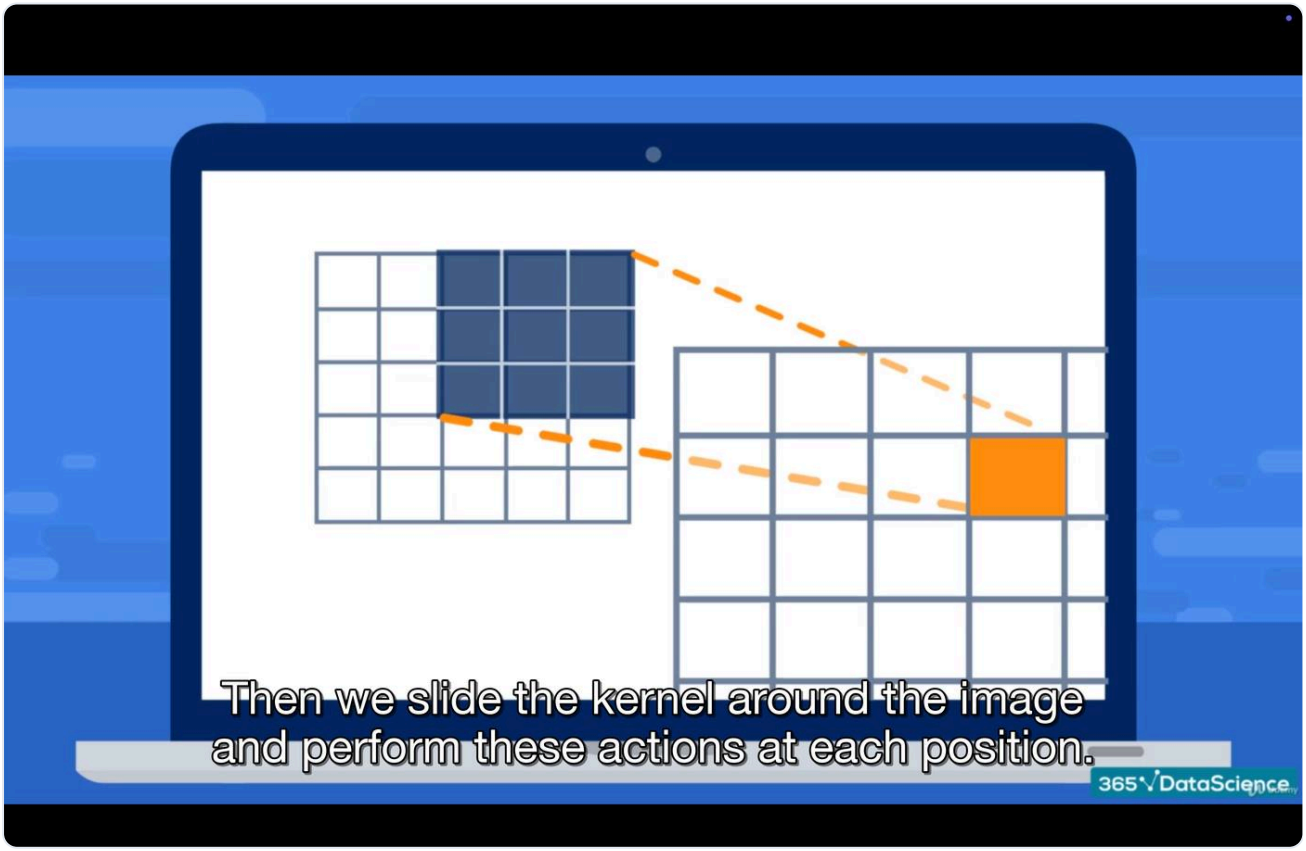

FIGURE 9.5

Then we slide the kernel around the image and perform these actions at each position. Notice how since we placed the kernel at every position possible, there is a lot of overlap.

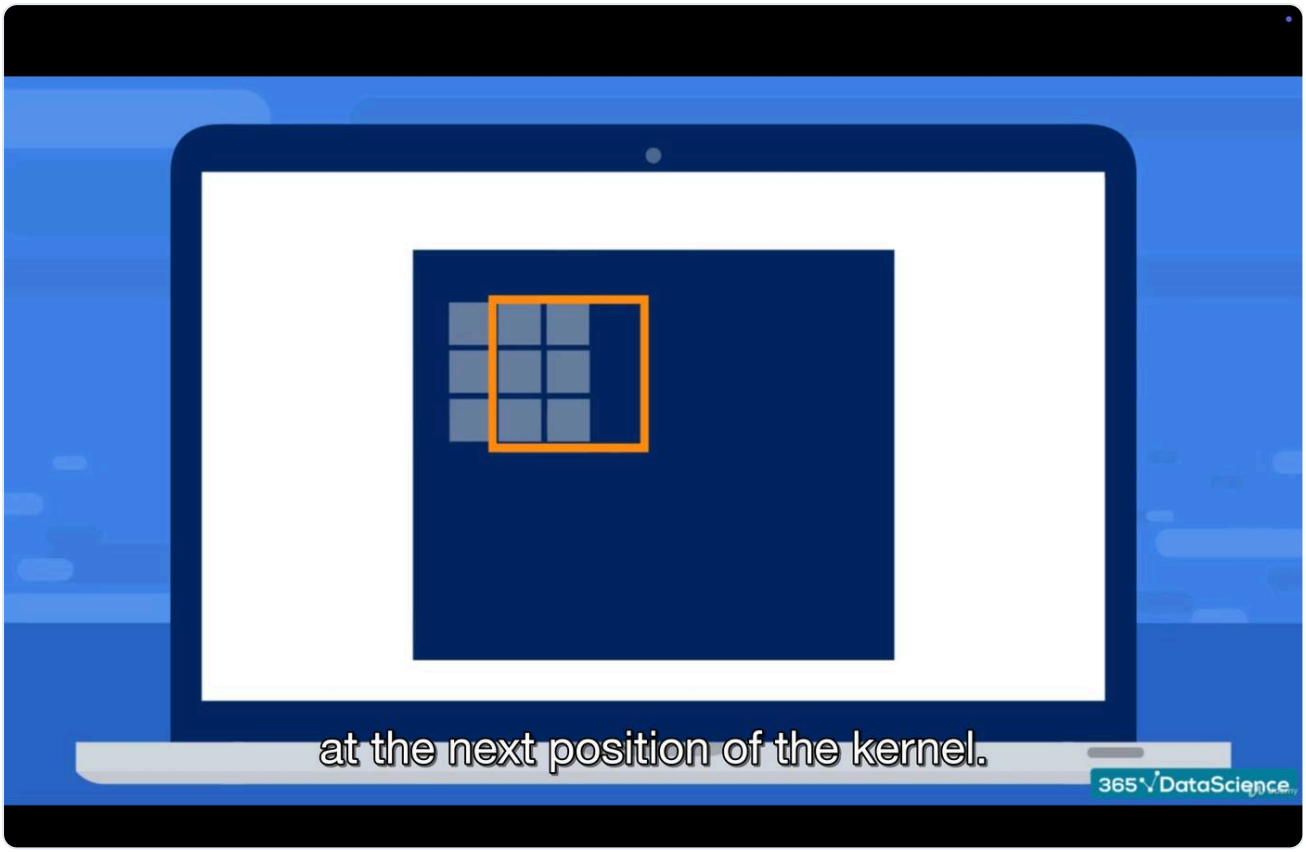

FIGURE 9.6

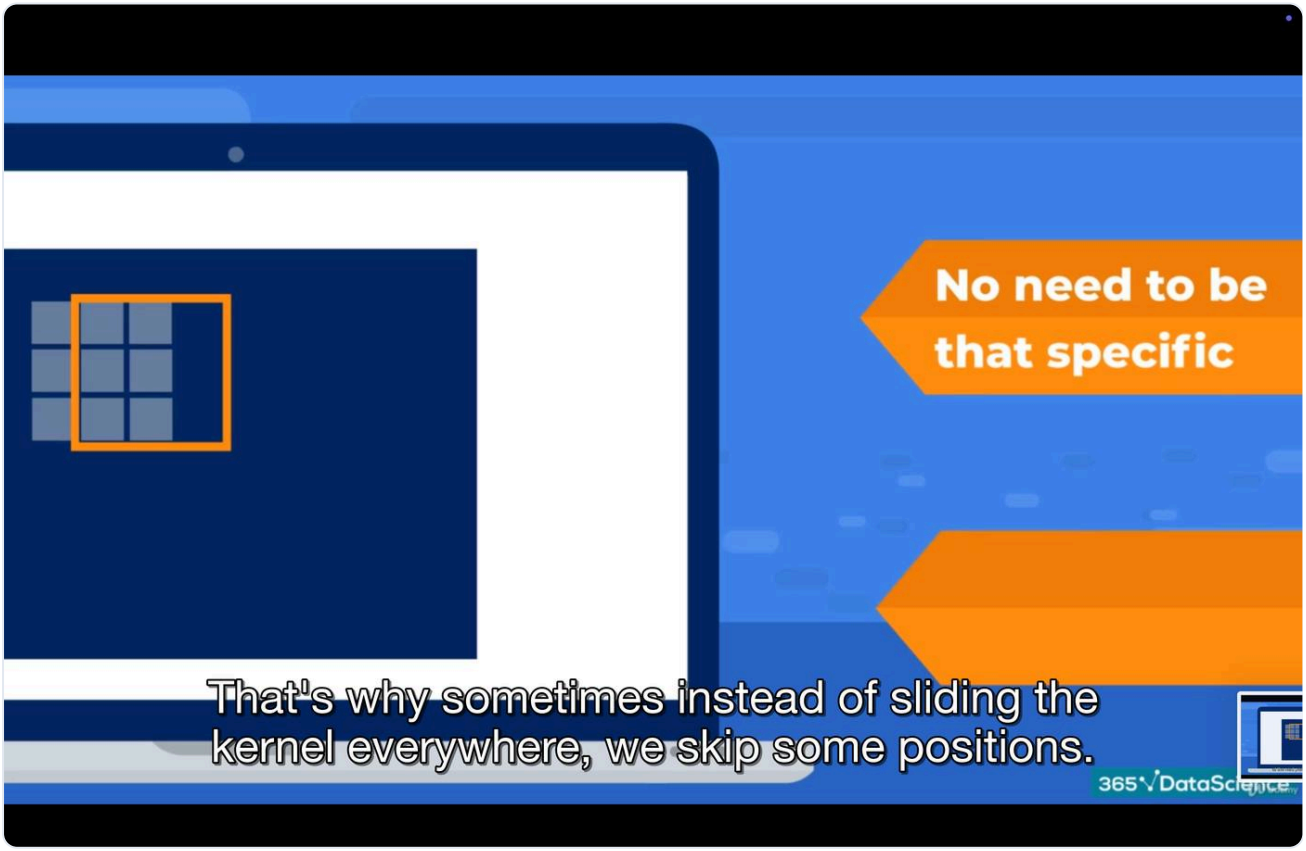

FIGURE 9.7 | That Sswhyssometimes- Instead of Sliding the

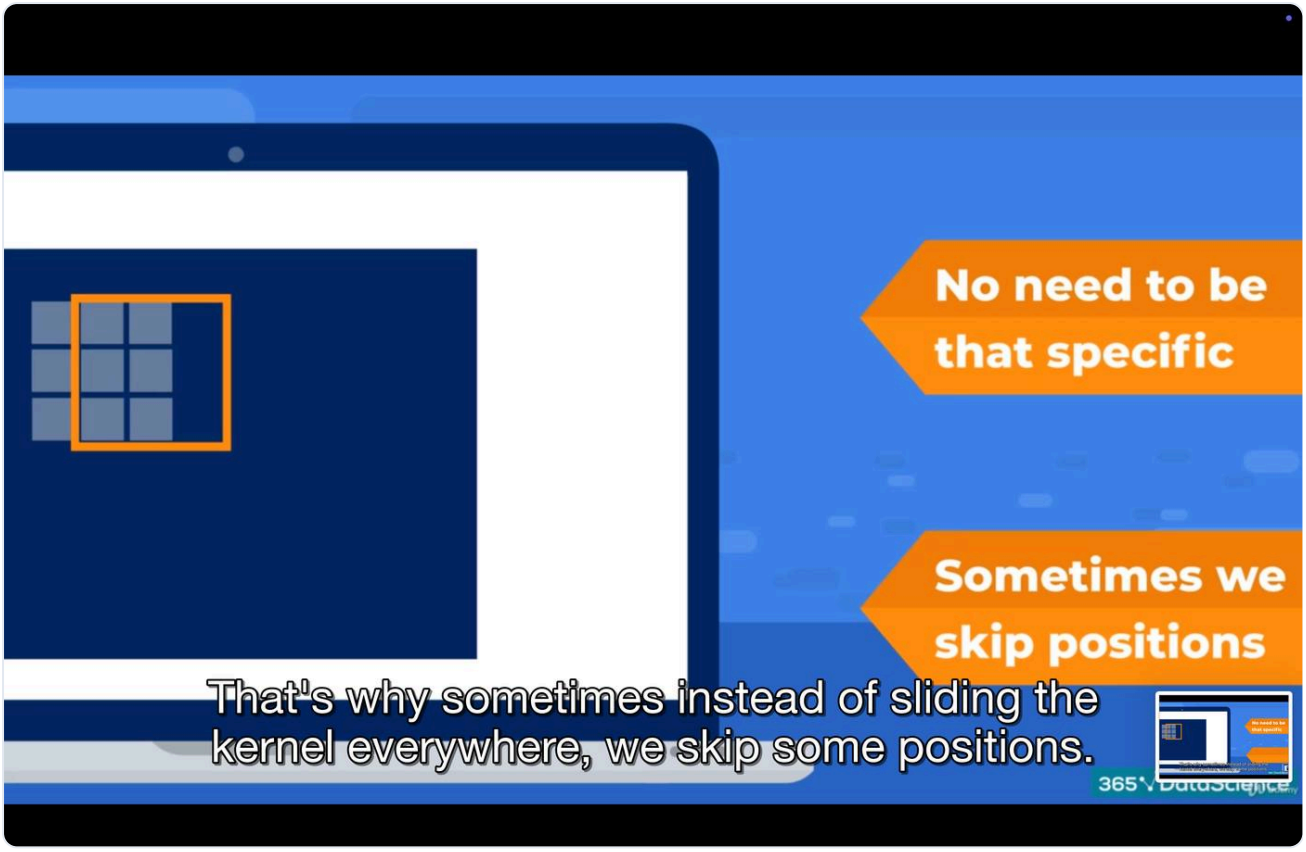

FIGURE 9.8

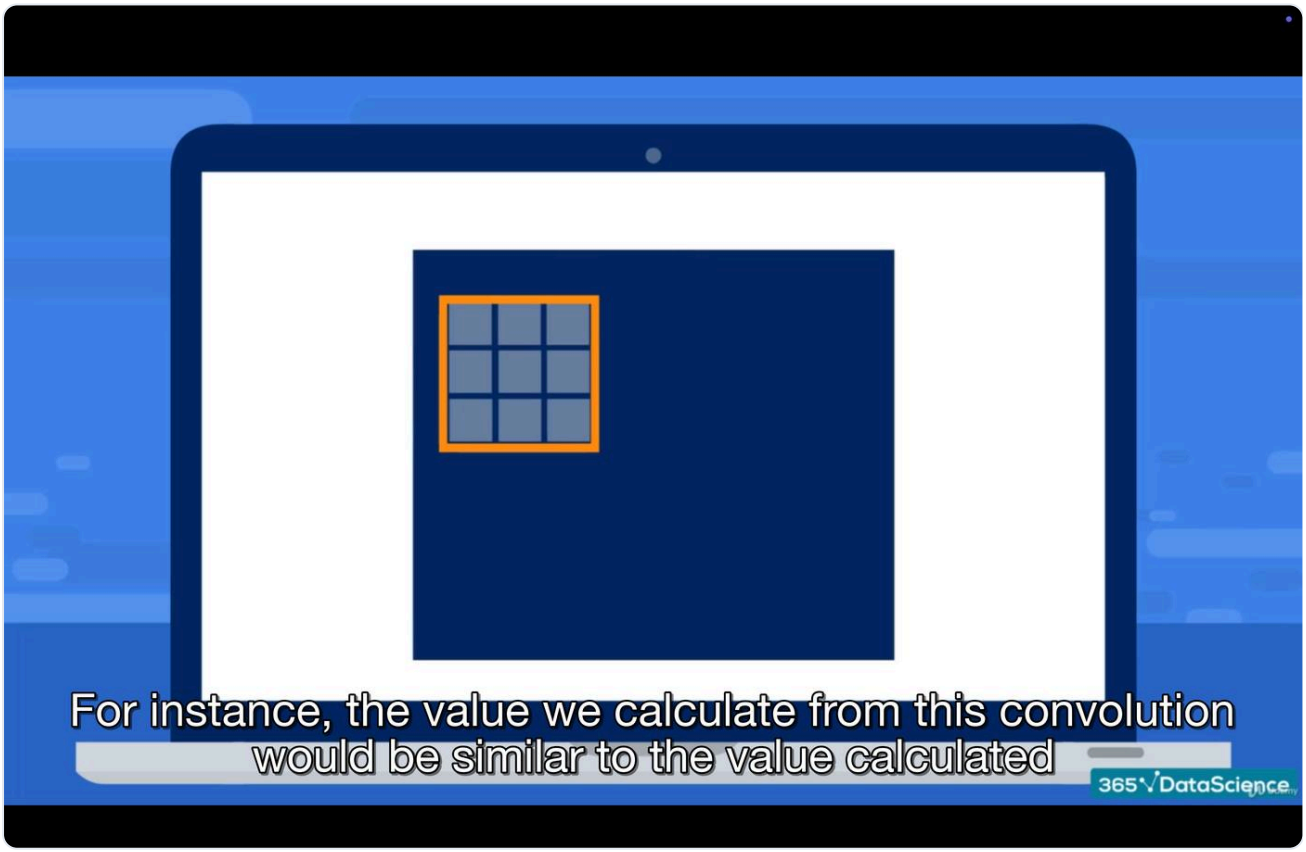

FIGURE 9.9

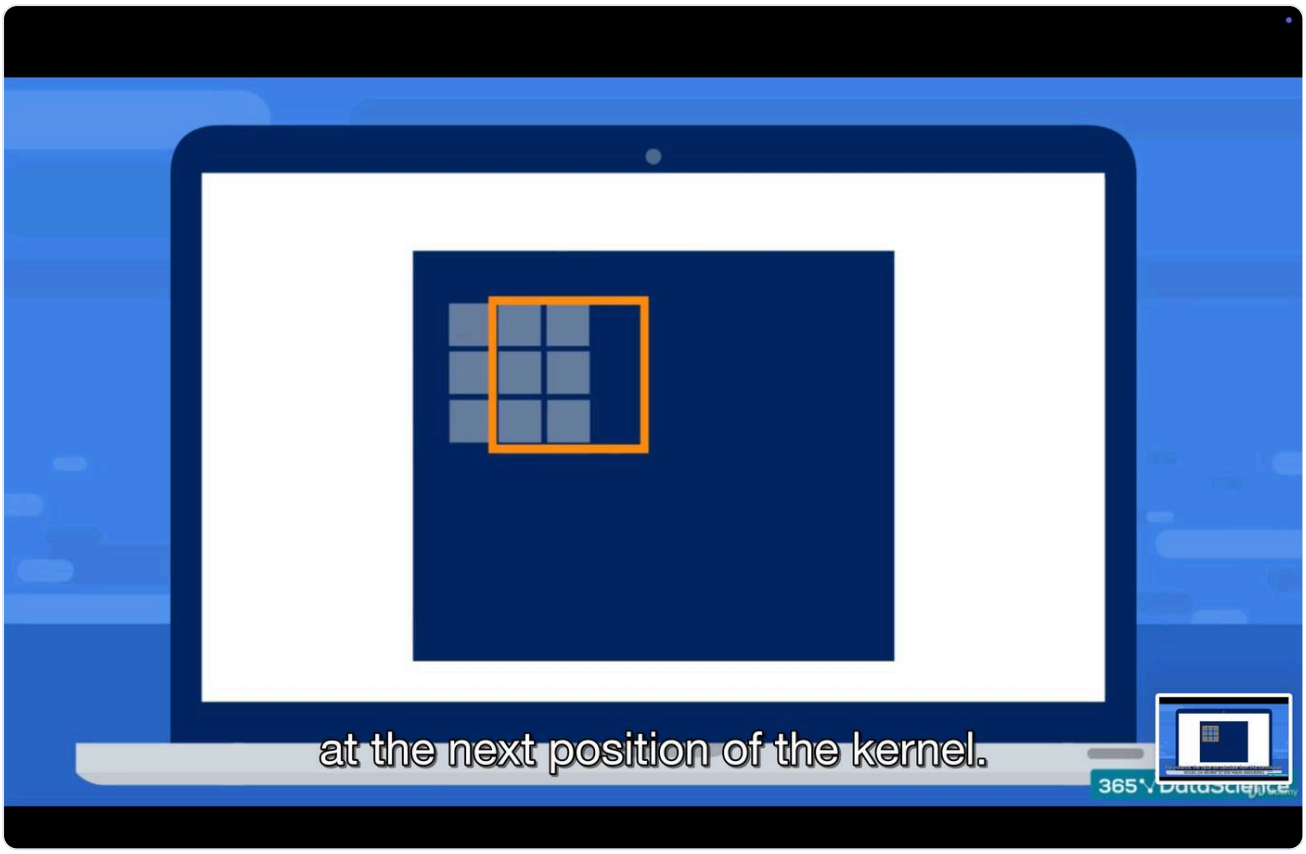

FIGURE 9.10

For instance, the value we calculate from this convolution would be similar to the value calculated at the next position of the kernel. This level of specificity is not always necessary.

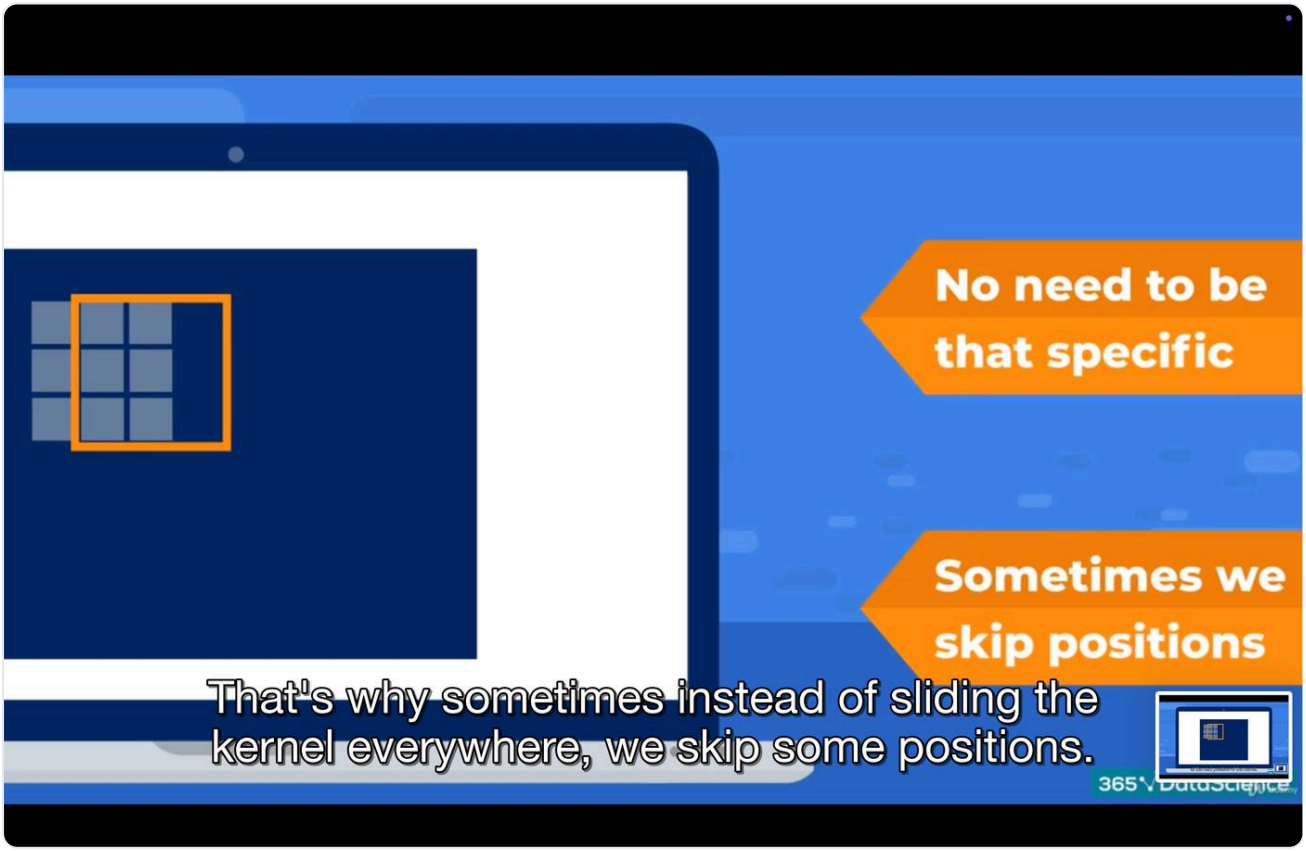

FIGURE 9.11

That's why sometimes instead of sliding the kernel everywhere, we skip some positions.

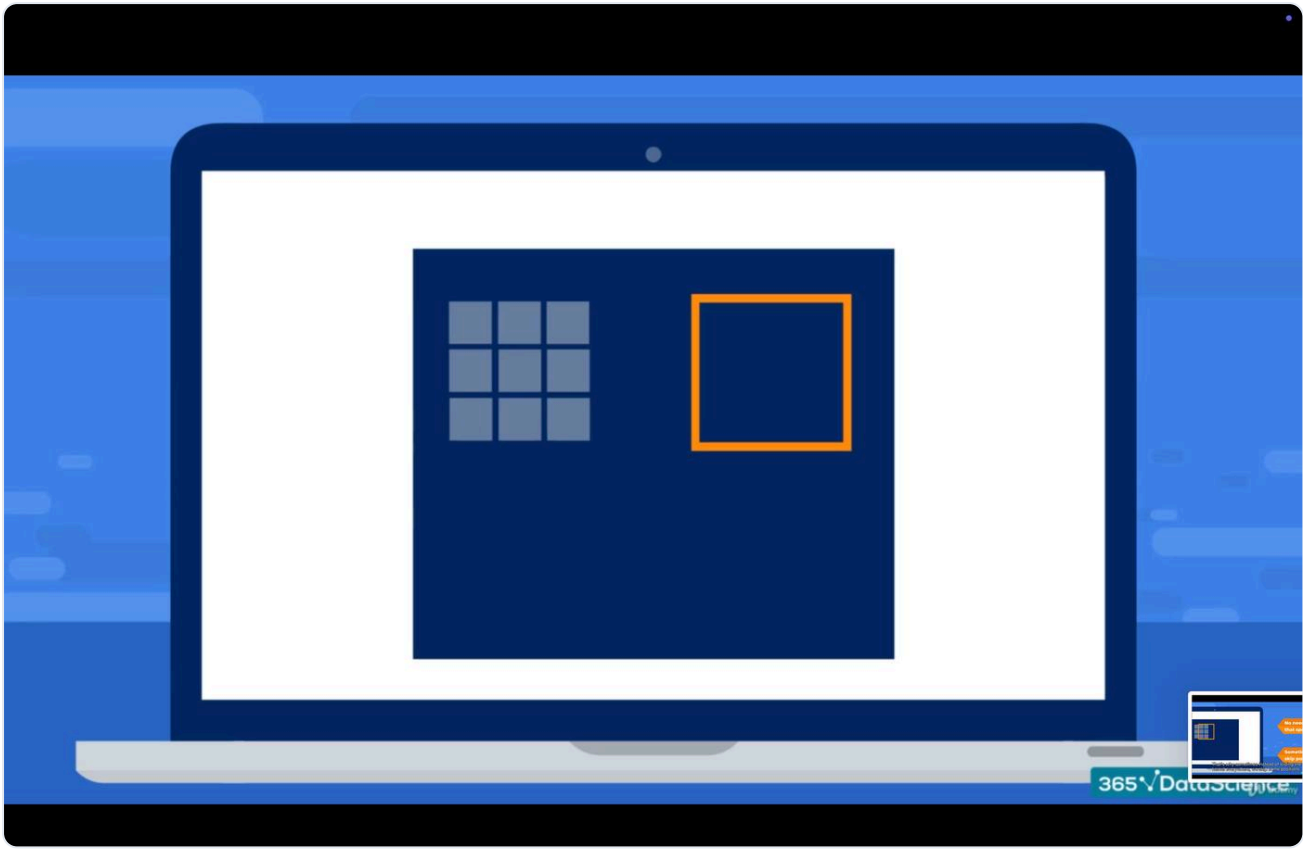

FIGURE 9.12

Thus we may apply the kernel once every two pixels or so.

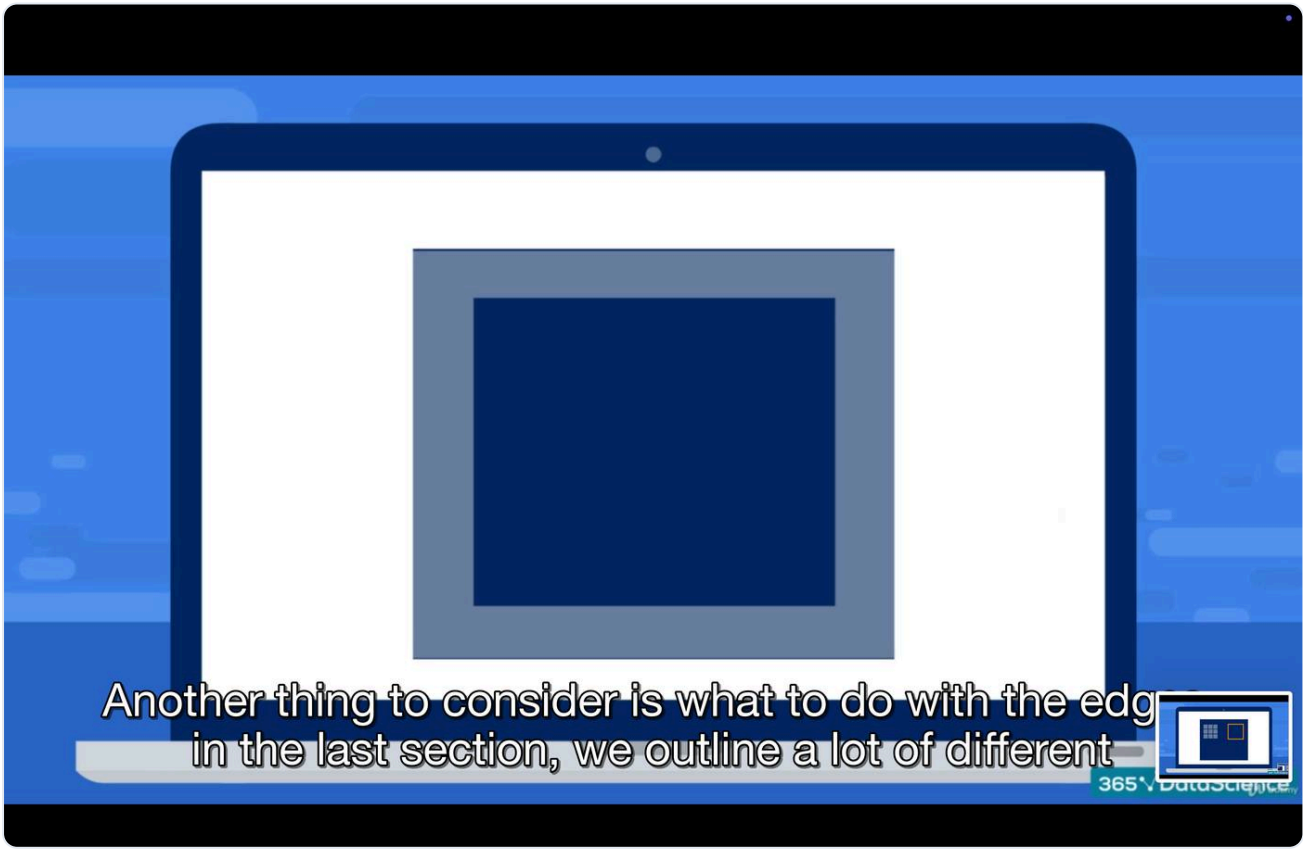

FIGURE 9.13

Another thing to consider is what to do with the edges in the last section, we outline a lot of different strategies to deal with that.

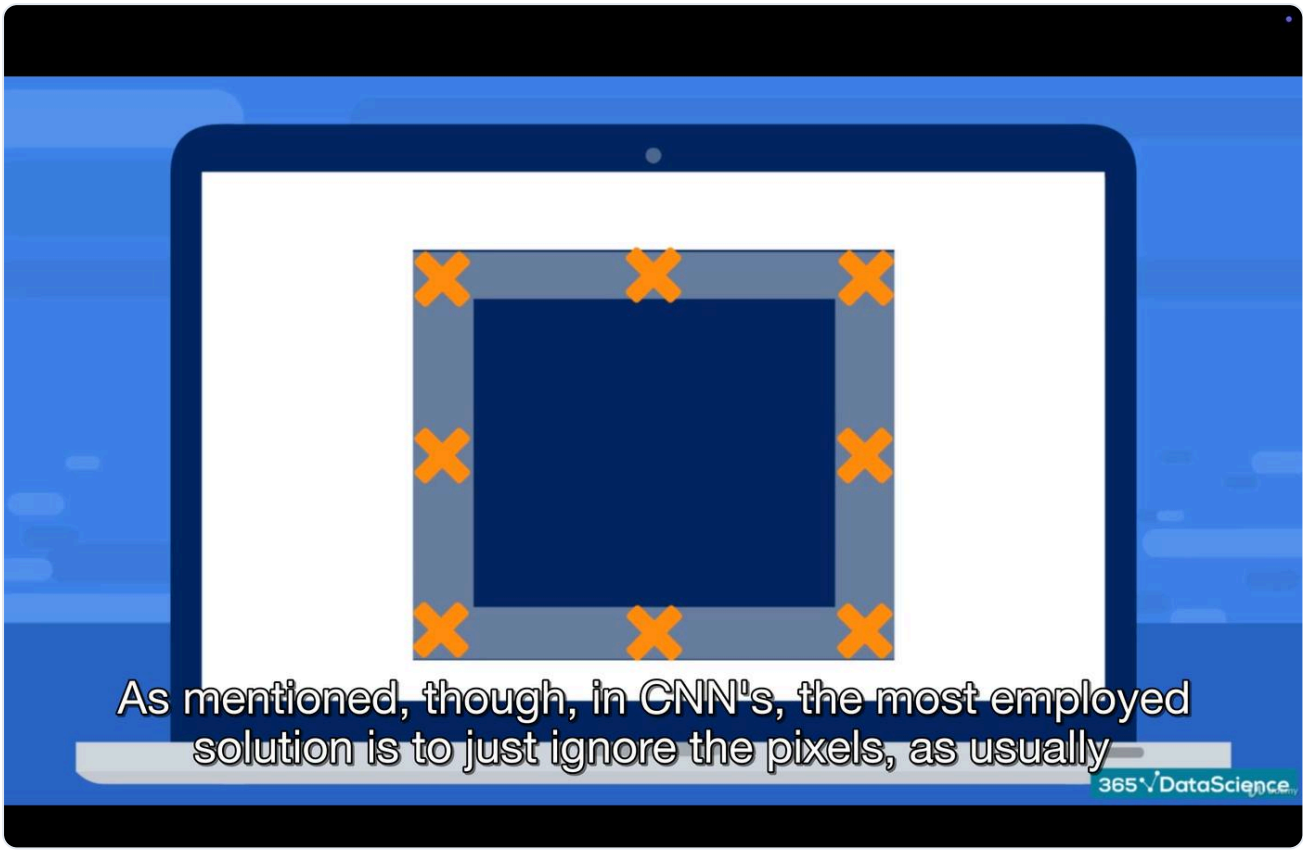

FIGURE 9.14

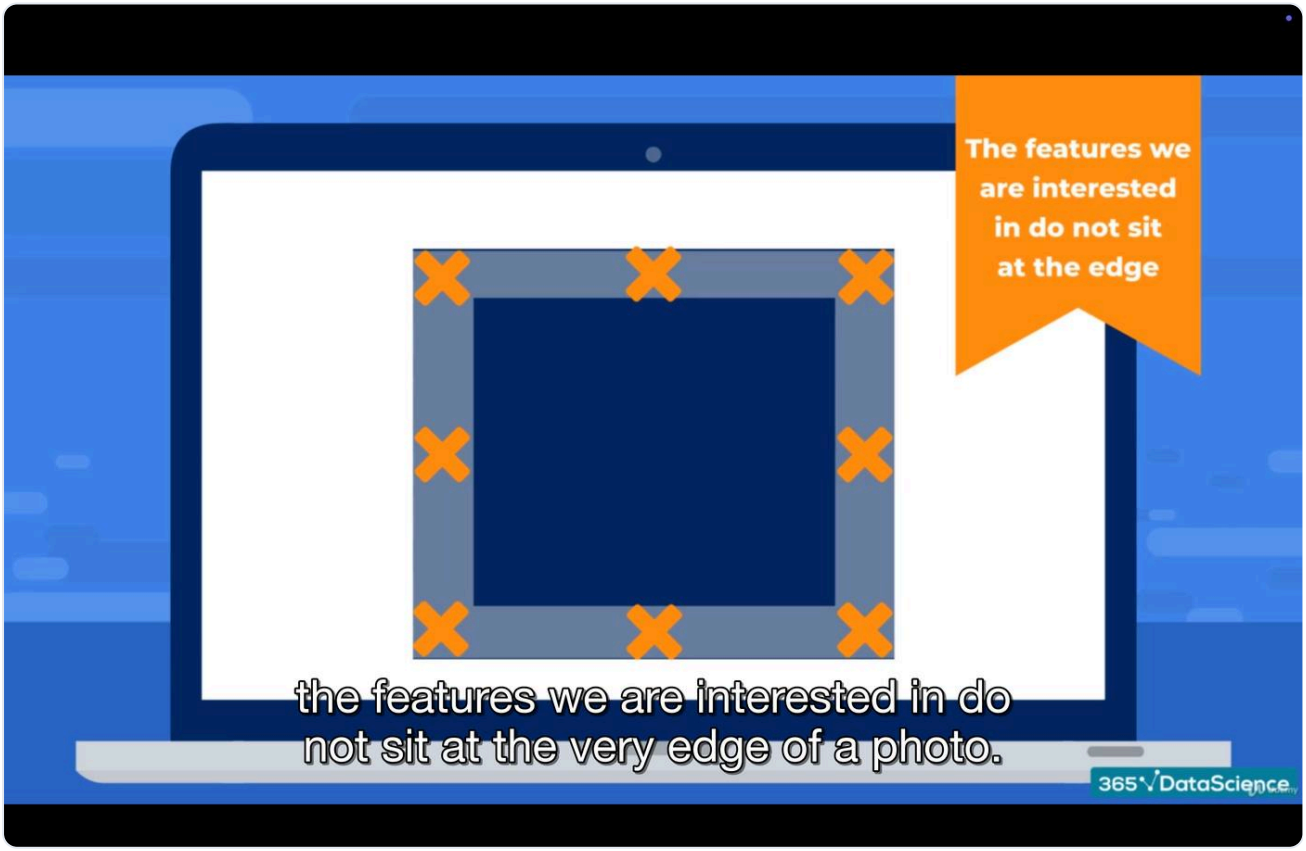

FIGURE 9.15

As mentioned, though, in CNN's, the most employed solution is to just ignore the pixels, as usually the features we are interested in do not sit at the very edge of a photo. Now, after exhausting all positions of the filter, we have compiled a new matrix of numbers. If we have skipped some positions or ignored the edges, its size may be smaller than the original image.

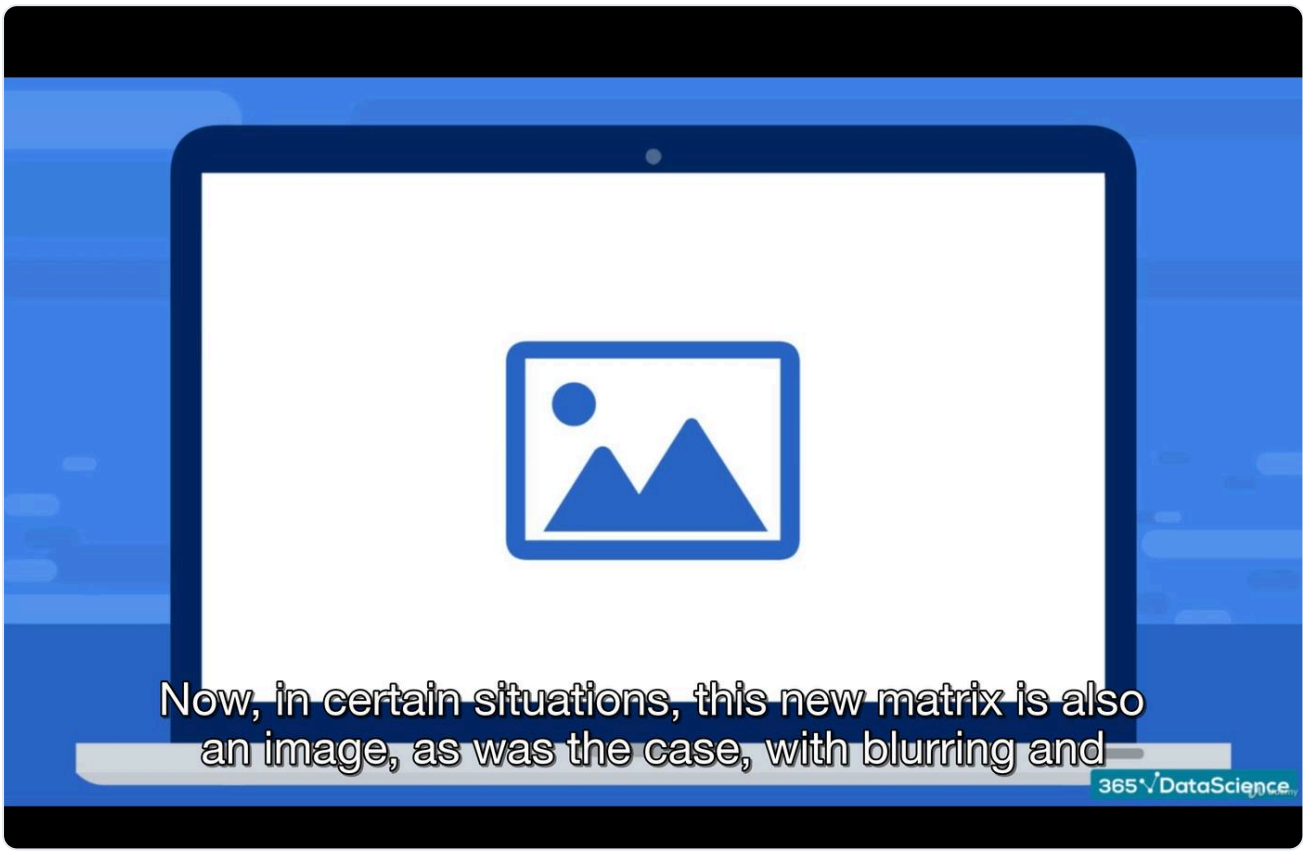

FIGURE 9.16

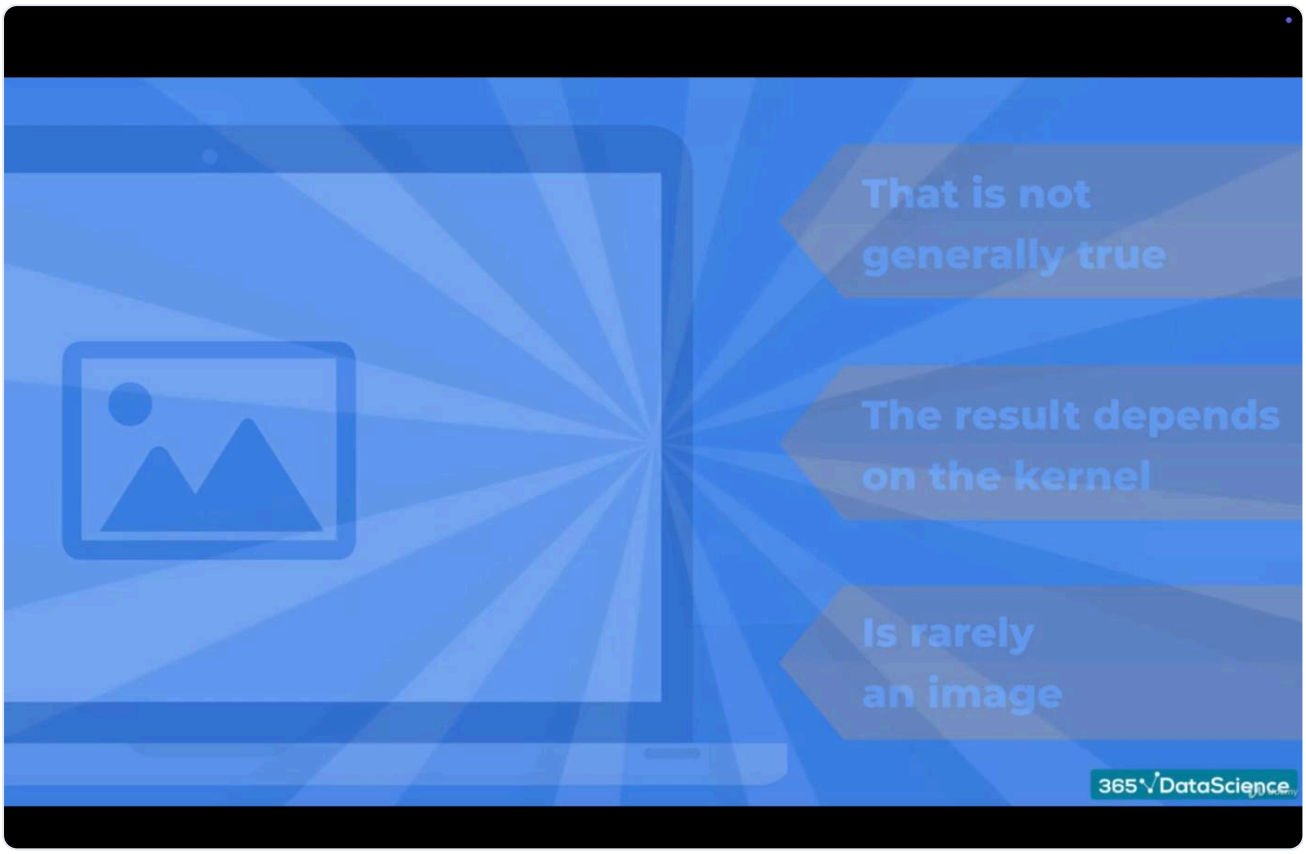

FIGURE 9.17

Now, in certain situations, this new matrix is also an image, as was the case, with blurring and sharpening.

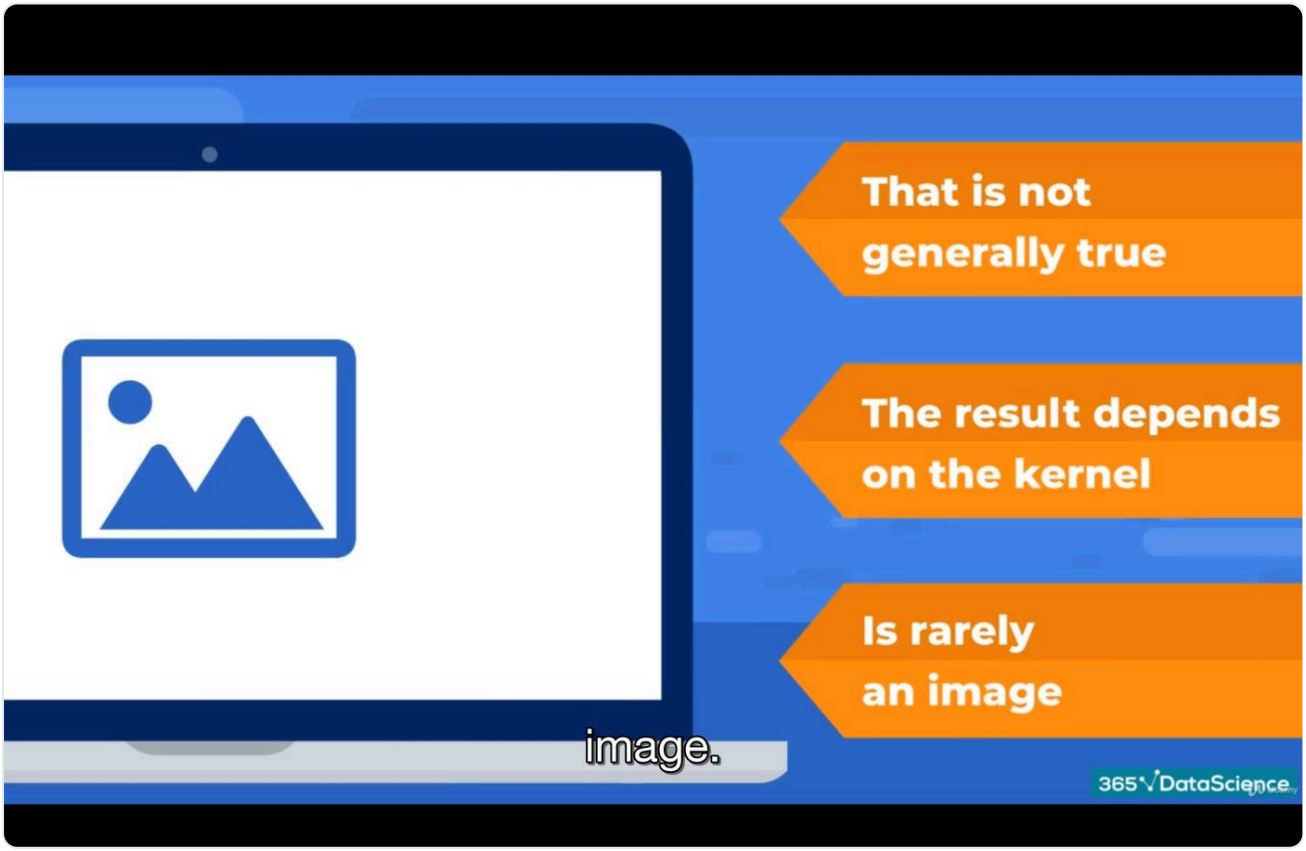

FIGURE 9.18 | That Is Not

Nevertheless, that is not generally true. The visualization and interpretation of the result depend on the kernel we use and is rarely a legible image. This is one of the most fascinating aspects of convolution. So let's investigate that further. We'll take the next example once again.

FIGURE 9.19

One recurring pattern in handwritten digits is an arc, a part of a circle zero two three, five, six, eight and nine.

FIGURE 9.20

All have at least one arc, while one four and seven don't. So if I give you this arc, could you say which digit it is from?

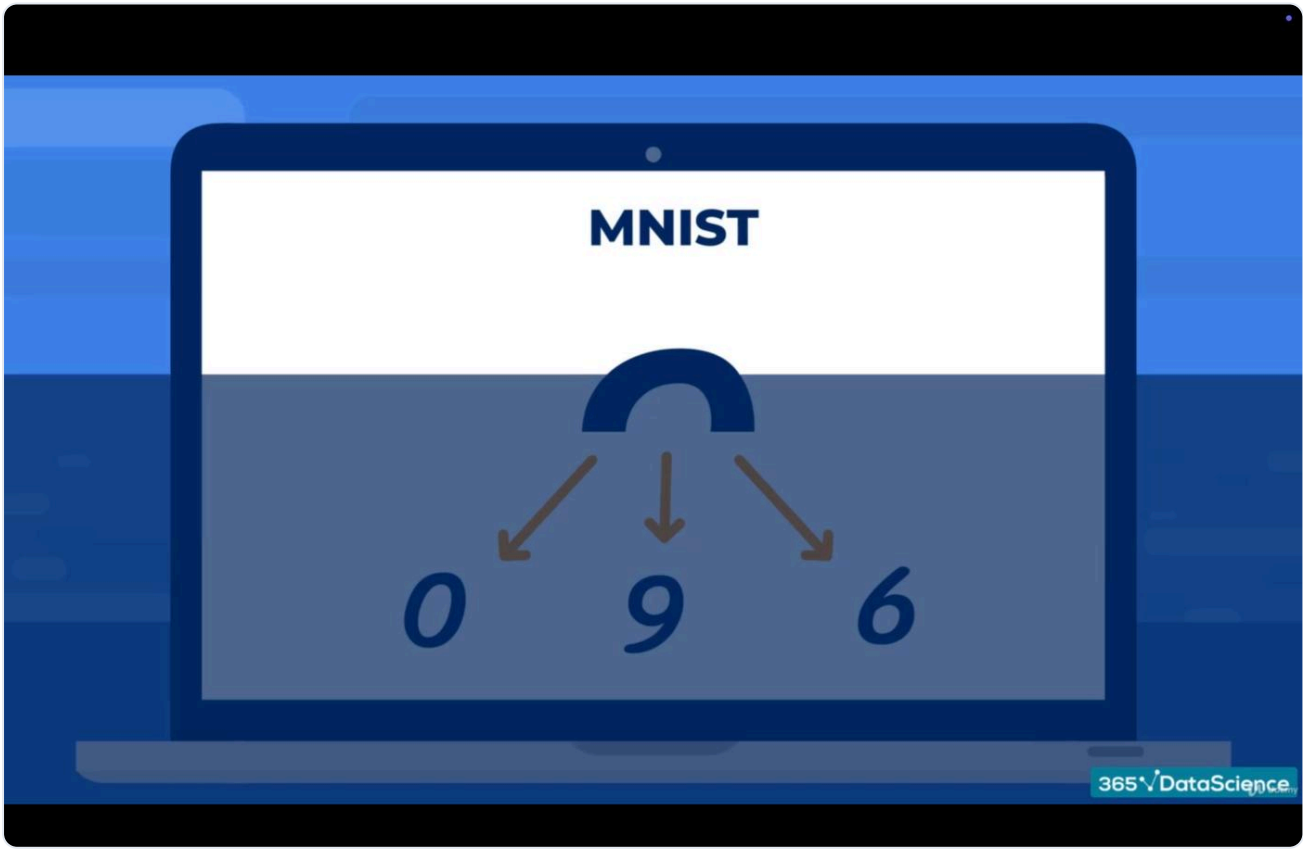

FIGURE 9.21

Is it a piece from a zero or a nine? Maybe a six. You can't tell, can you?

FIGURE 9.22

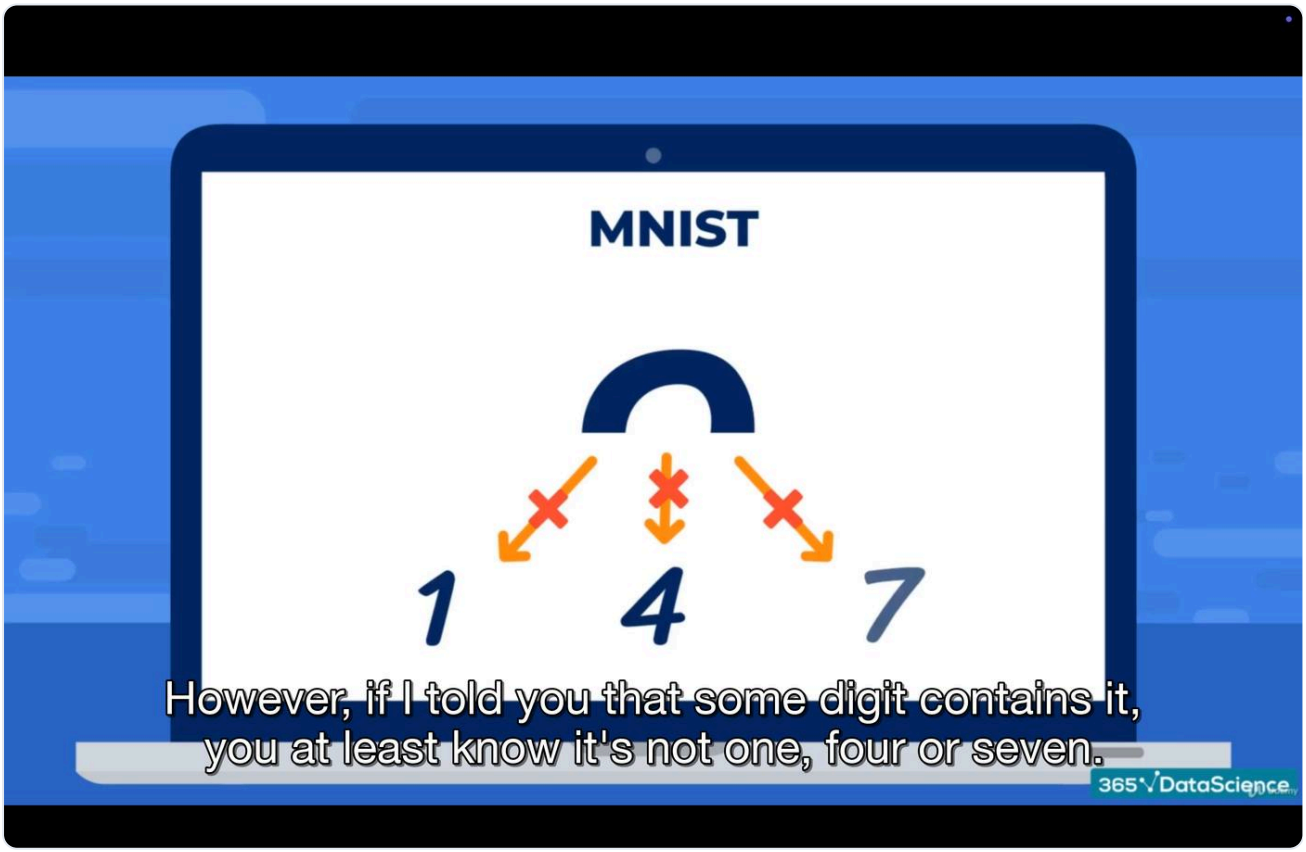

FIGURE 9.23

However, if I told you that some digit contains it, you at least know it's not one, four or seven.

FIGURE 9.24

Thus, if the model detects arcs, this will increase the probability of associated numbers to be classified with the corresponding image.

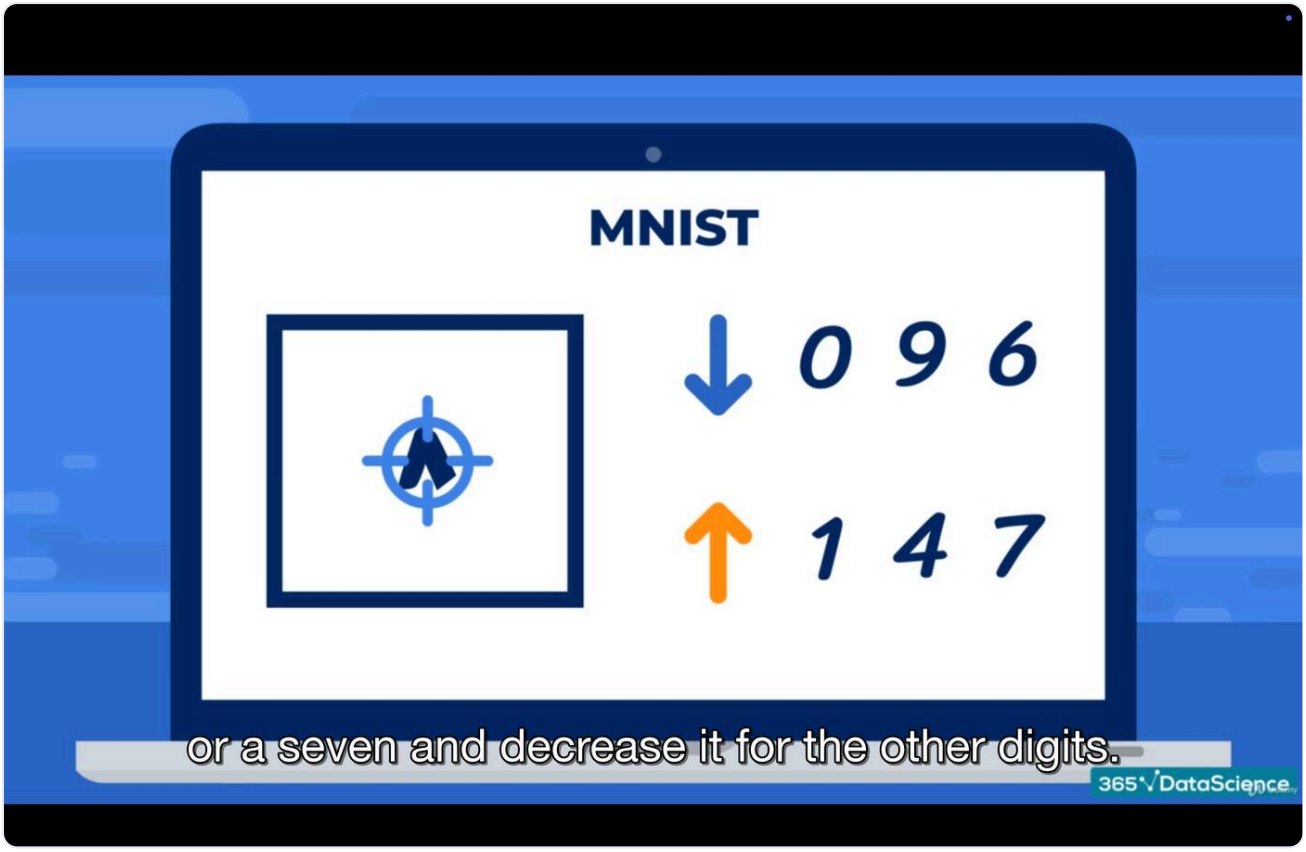

FIGURE 9.25

Similarly, detecting an acute angle would increase the probabilities of the image being a one, a four or a seven and decrease it for the other digits. So far, so good.

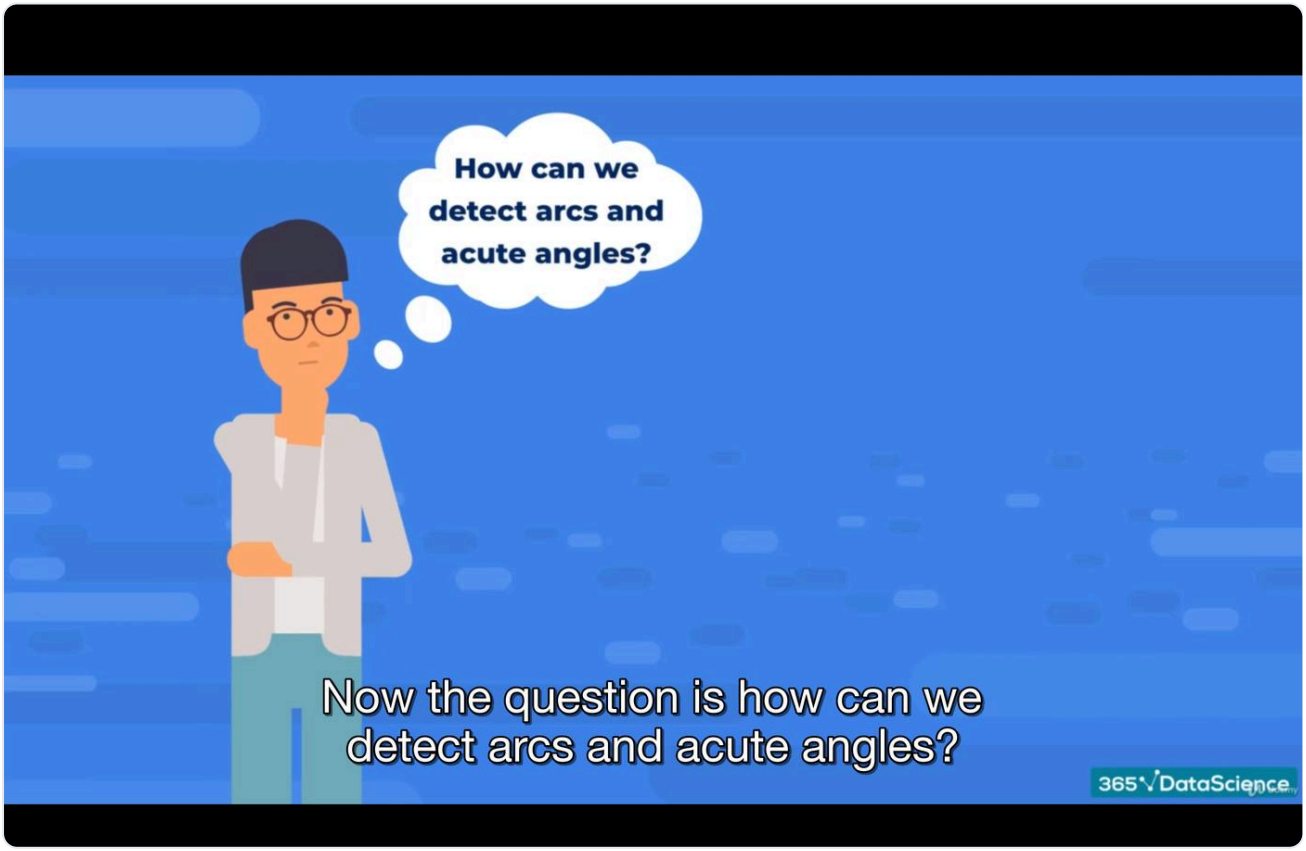

FIGURE 9.26

Now the question is how can we detect arcs and acute angles?

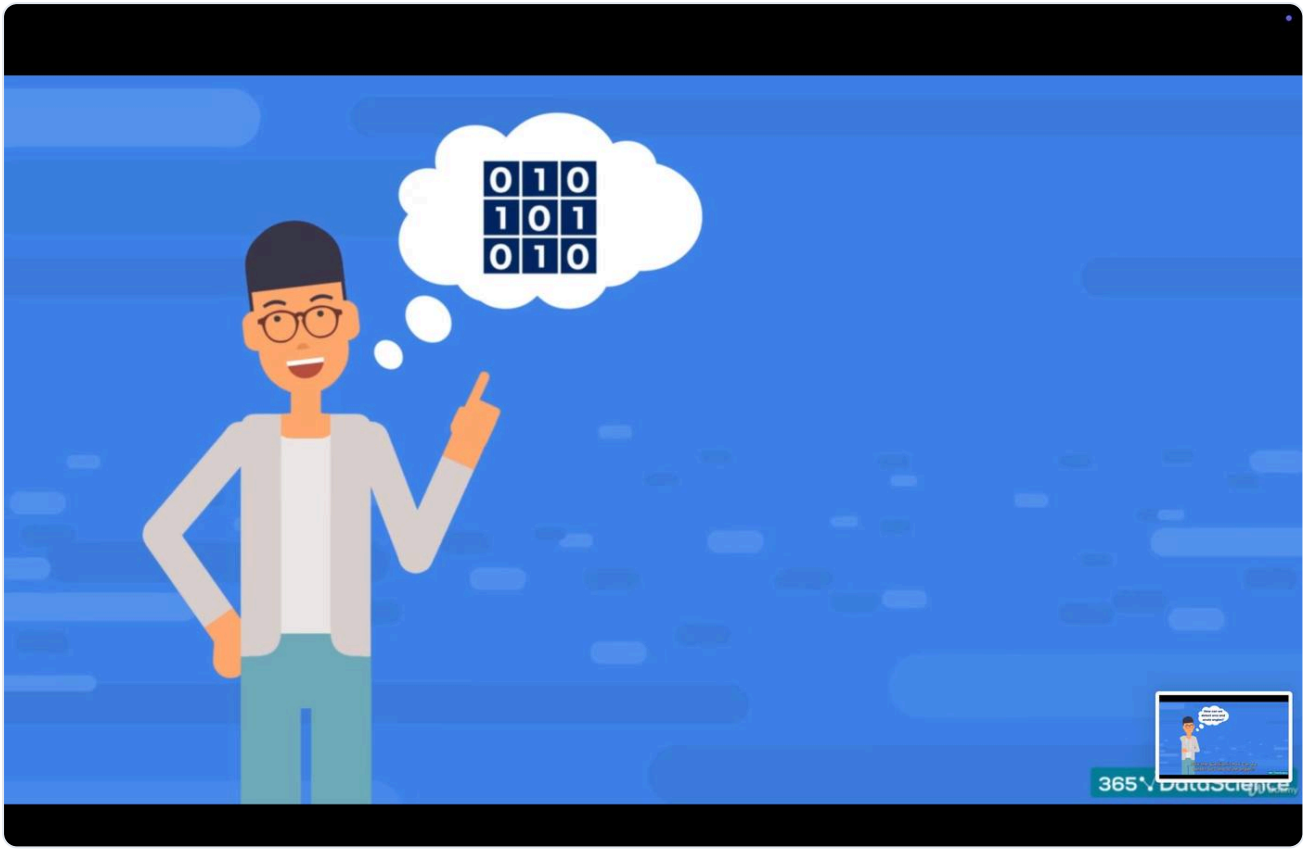

FIGURE 9.27

The answer pops immediately kernels.

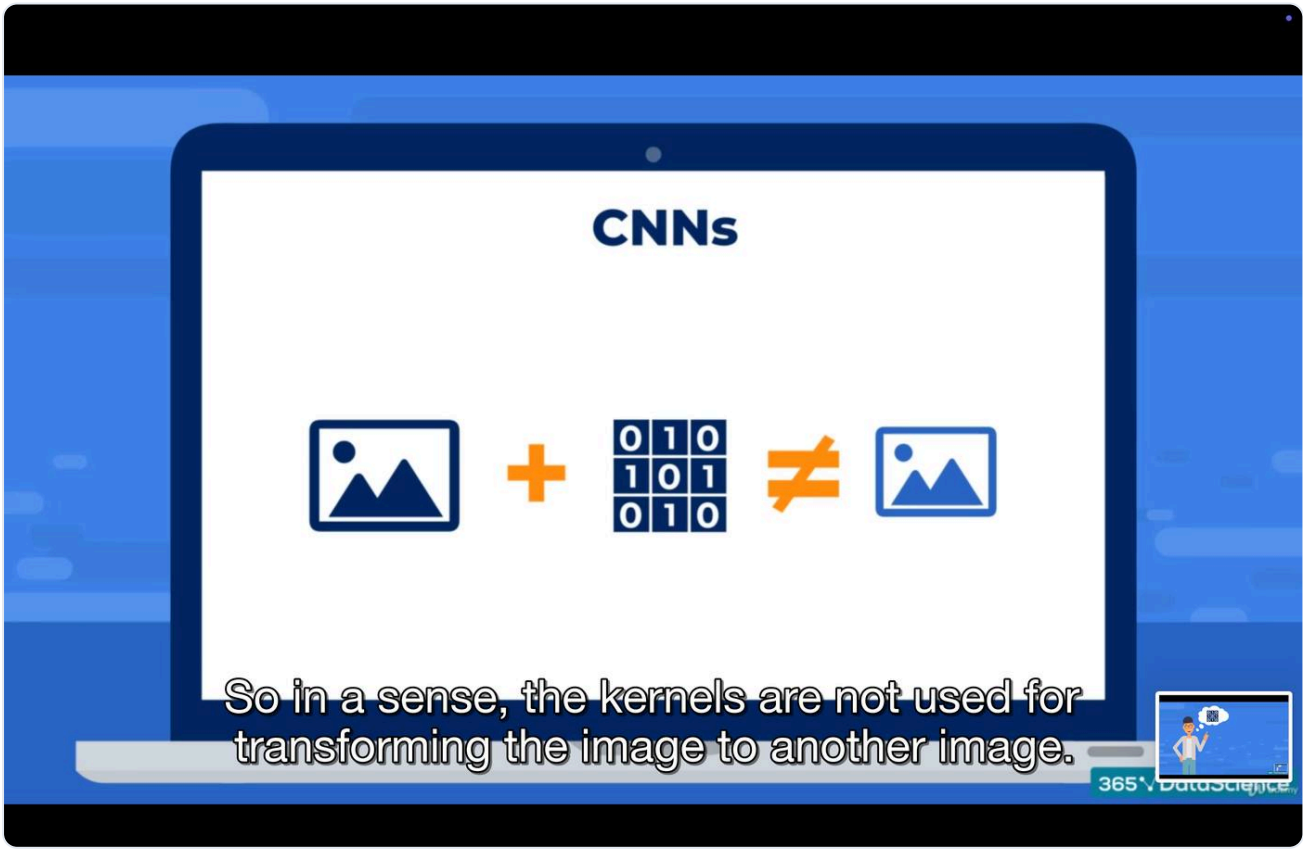

FIGURE 9.28

So in a sense, the kernels are not used for transforming the image to another image.

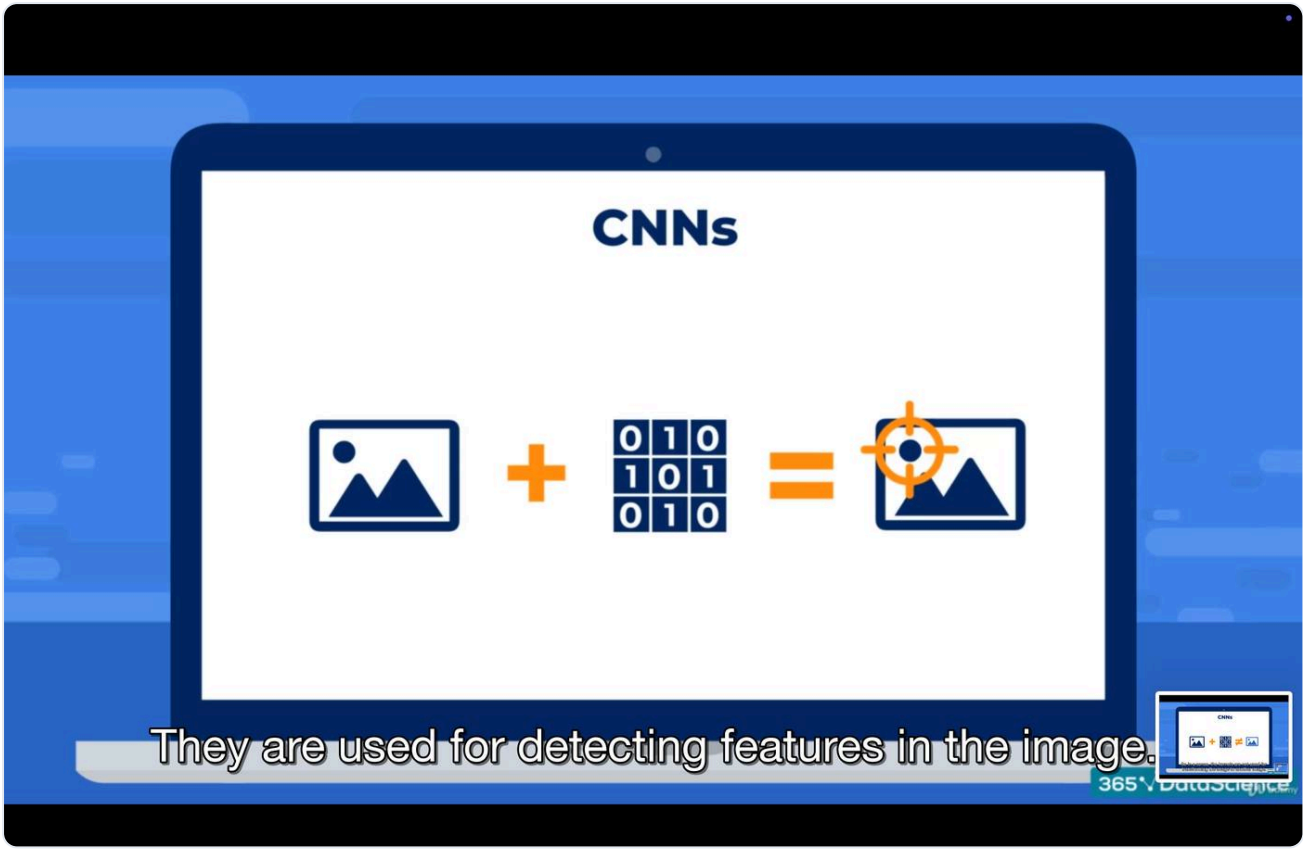

FIGURE 9.29

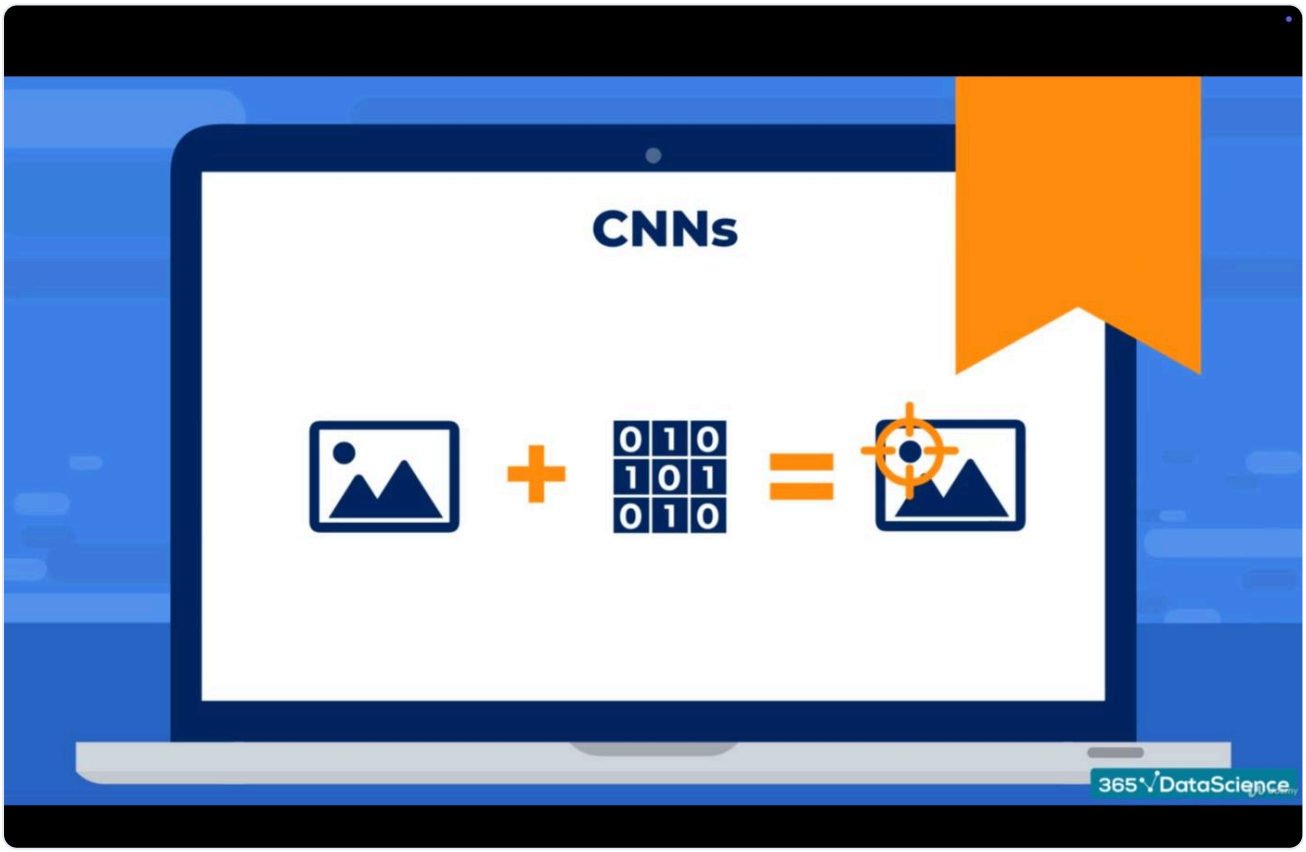

FIGURE 9.30

They are used for detecting features in the image.

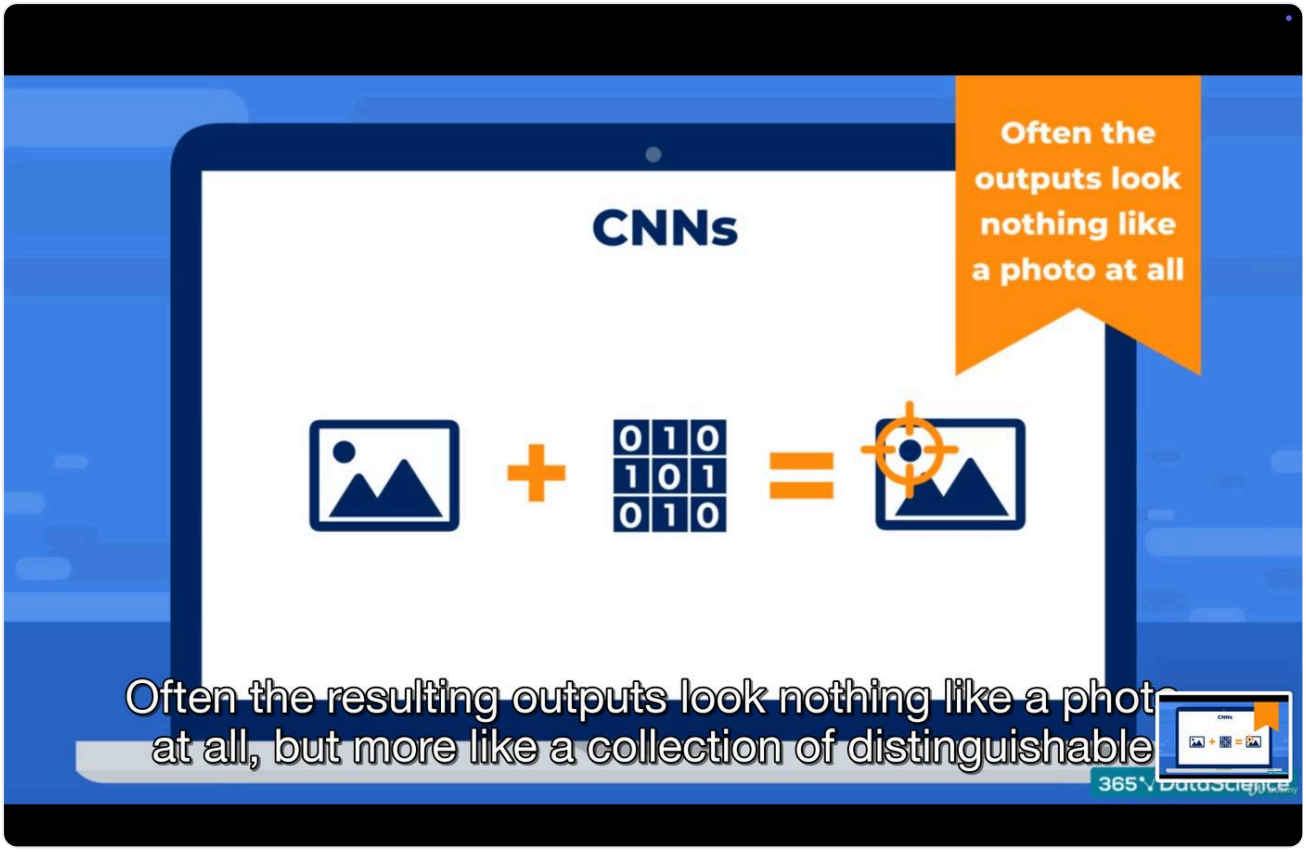

FIGURE 9.31

Often the resulting outputs look nothing like a photo at all, but more like a collection of distinguishable features of the image.

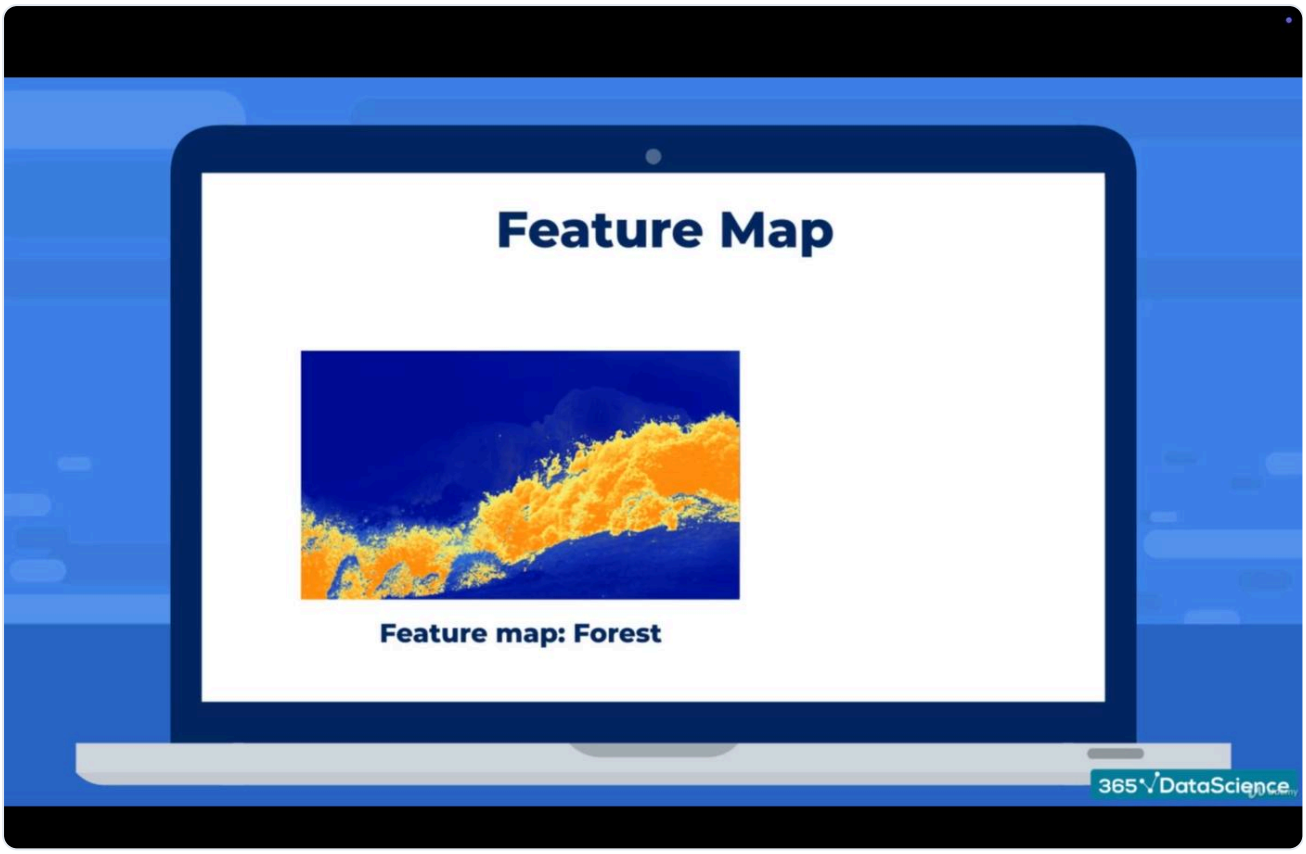

FIGURE 9.32 | Feature Map

Because of this, the resulting matrix is called a feature map.

FIGURE 9.33 | Feature Map

You can think of it as a map showing how strong the presence of a certain feature in that part of the image is. We have already seen one feature map, the edge detection.

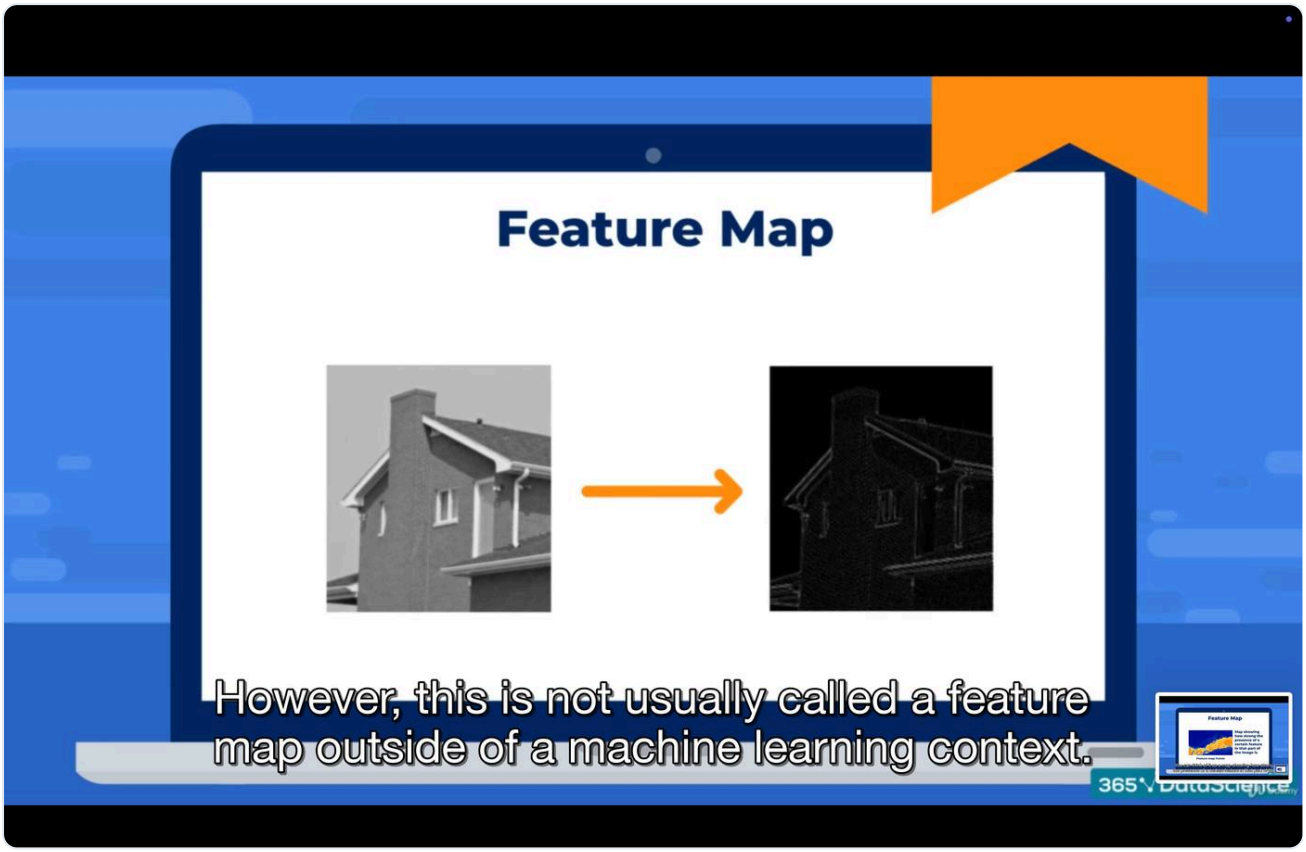

FIGURE 9.34 | Feature Map

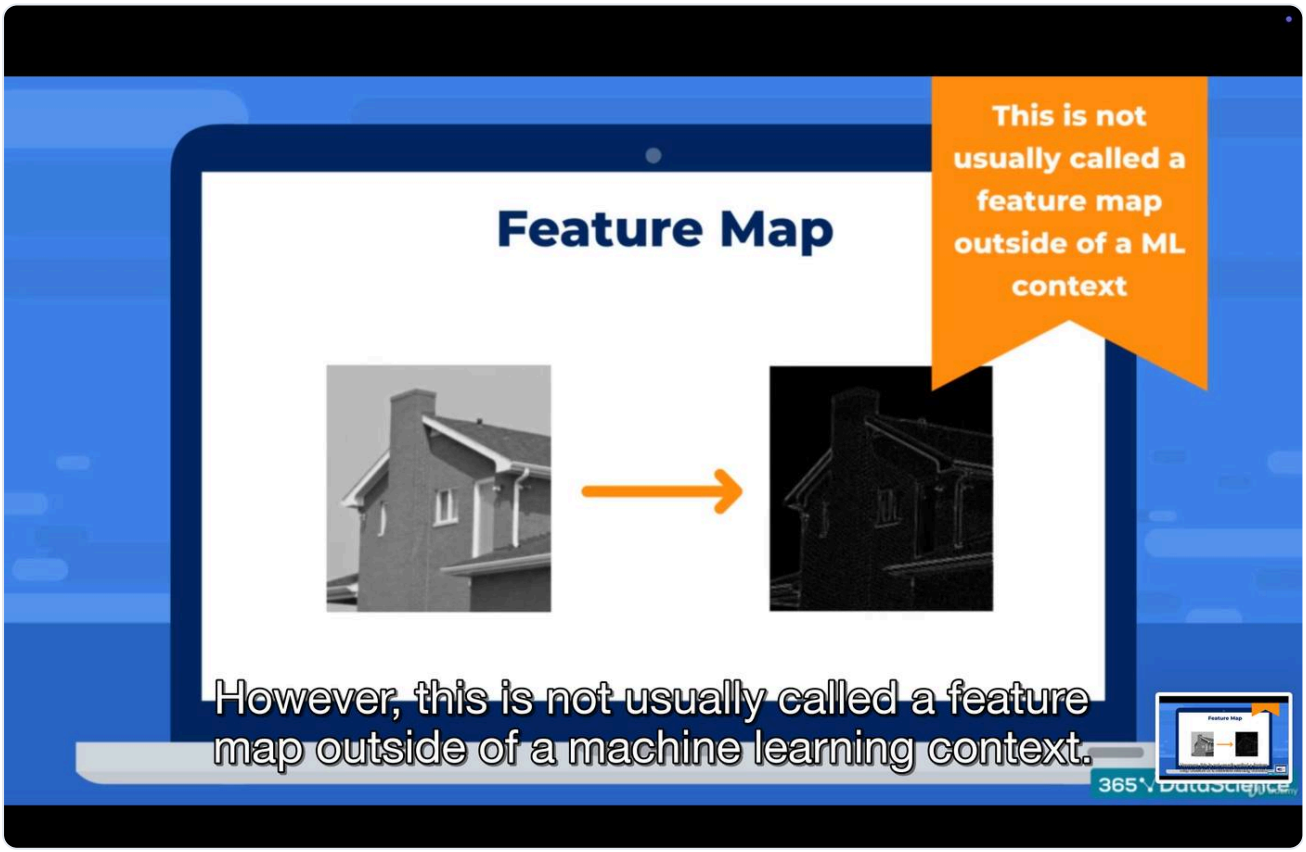

FIGURE 9.35 | Feature Map

However, this is not usually called a feature map outside of a machine learning context. The main advantage of convolutional neural networks is that they can learn by themselves what features to look for in the image and how to construct the kernel for them.

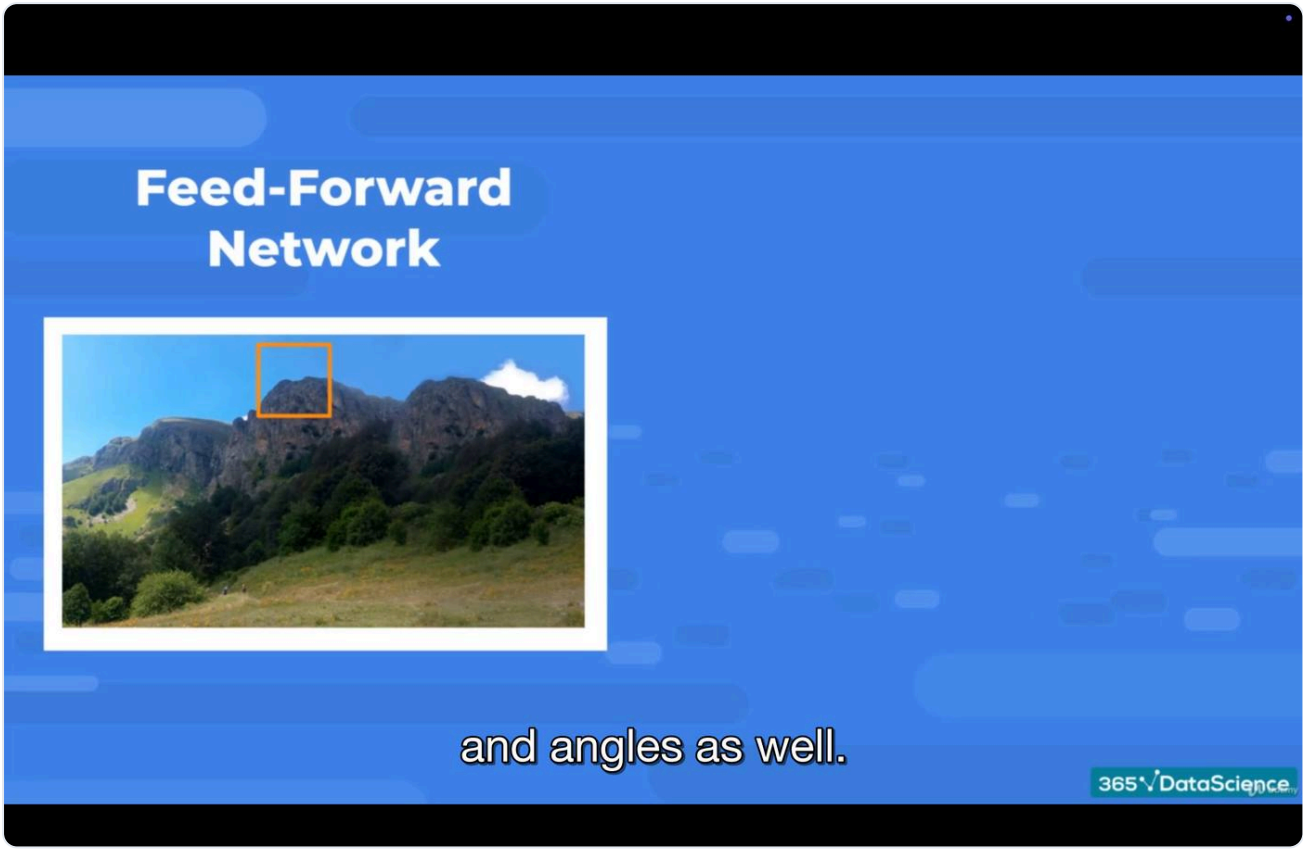

FIGURE 9.36

As a side note, a feed forward networks can in theory learn to detect individual features like arcs and angles as well.

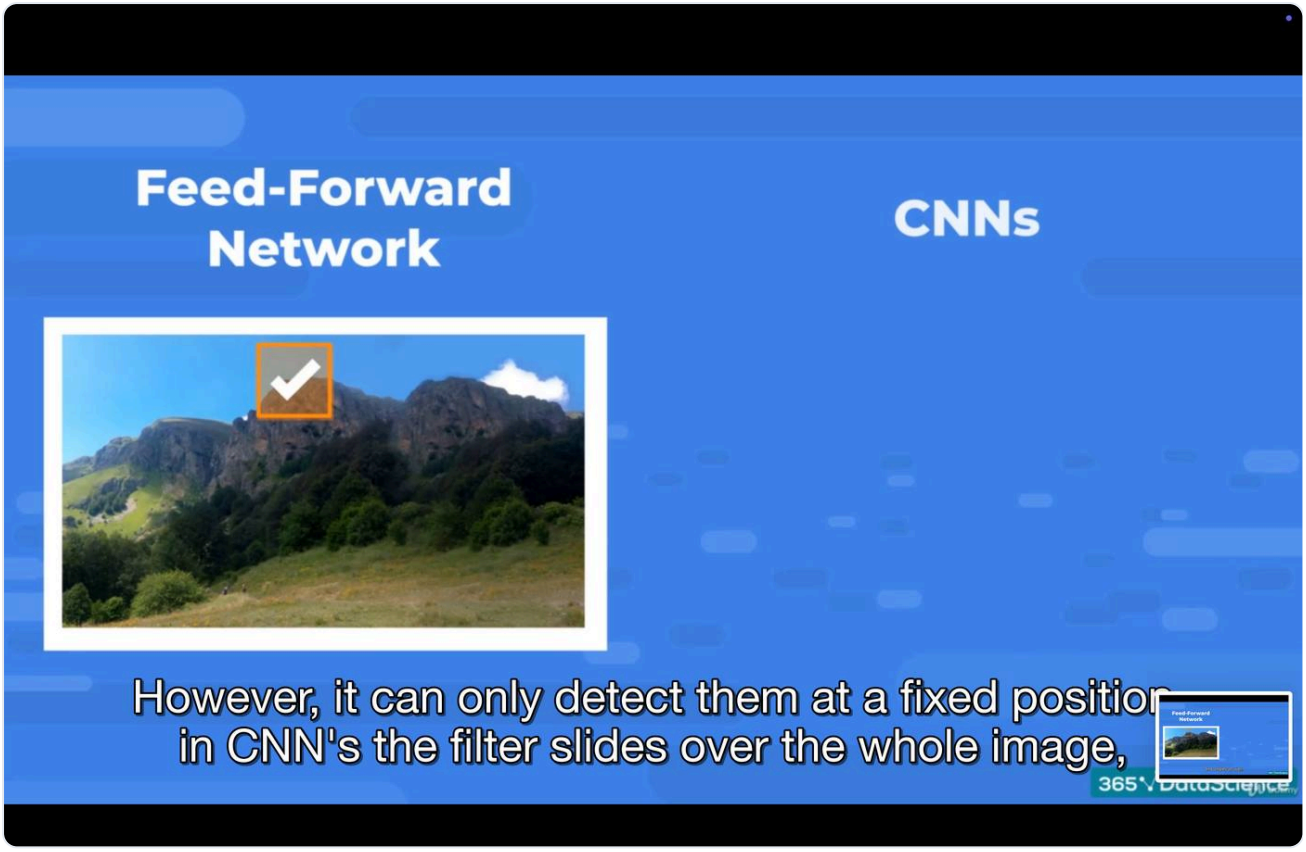

FIGURE 9.37

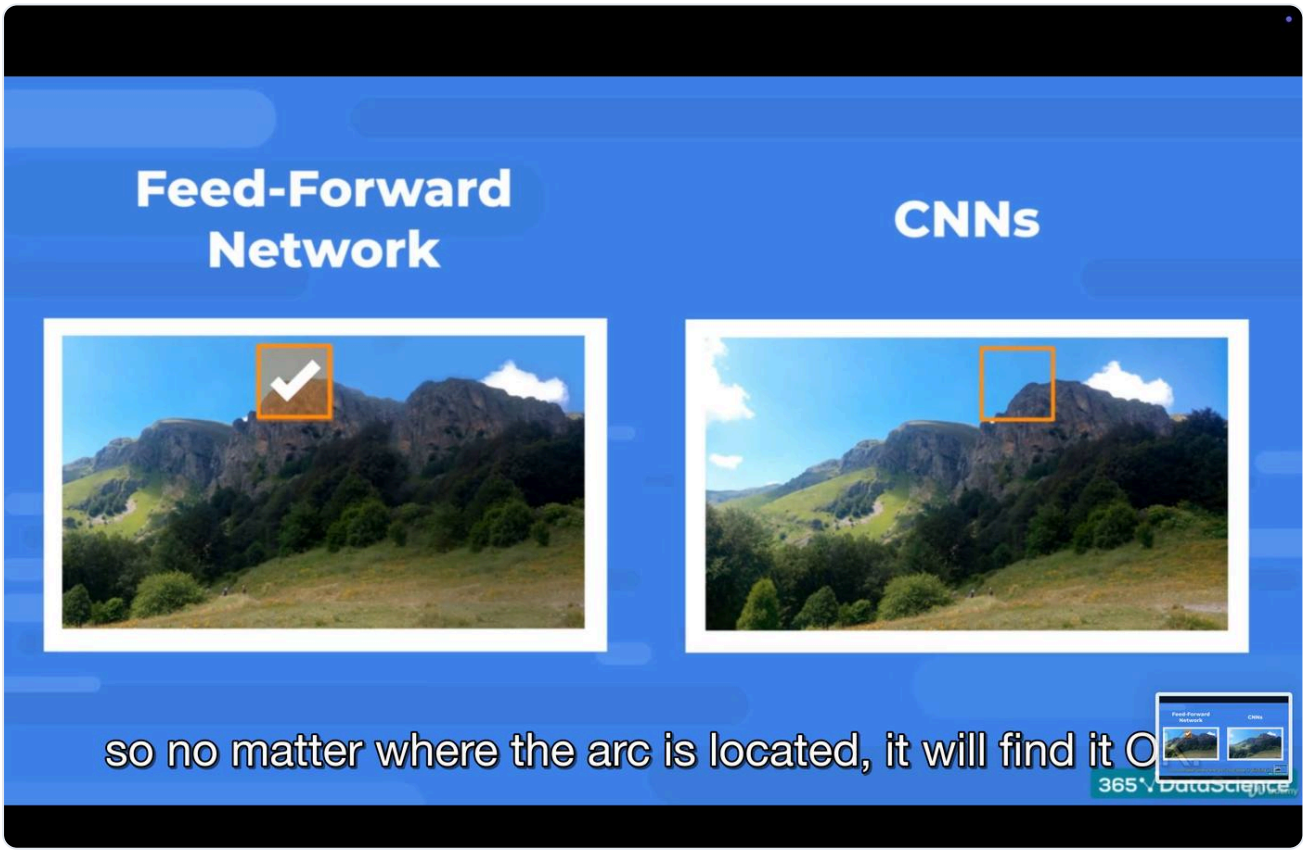

FIGURE 9.38

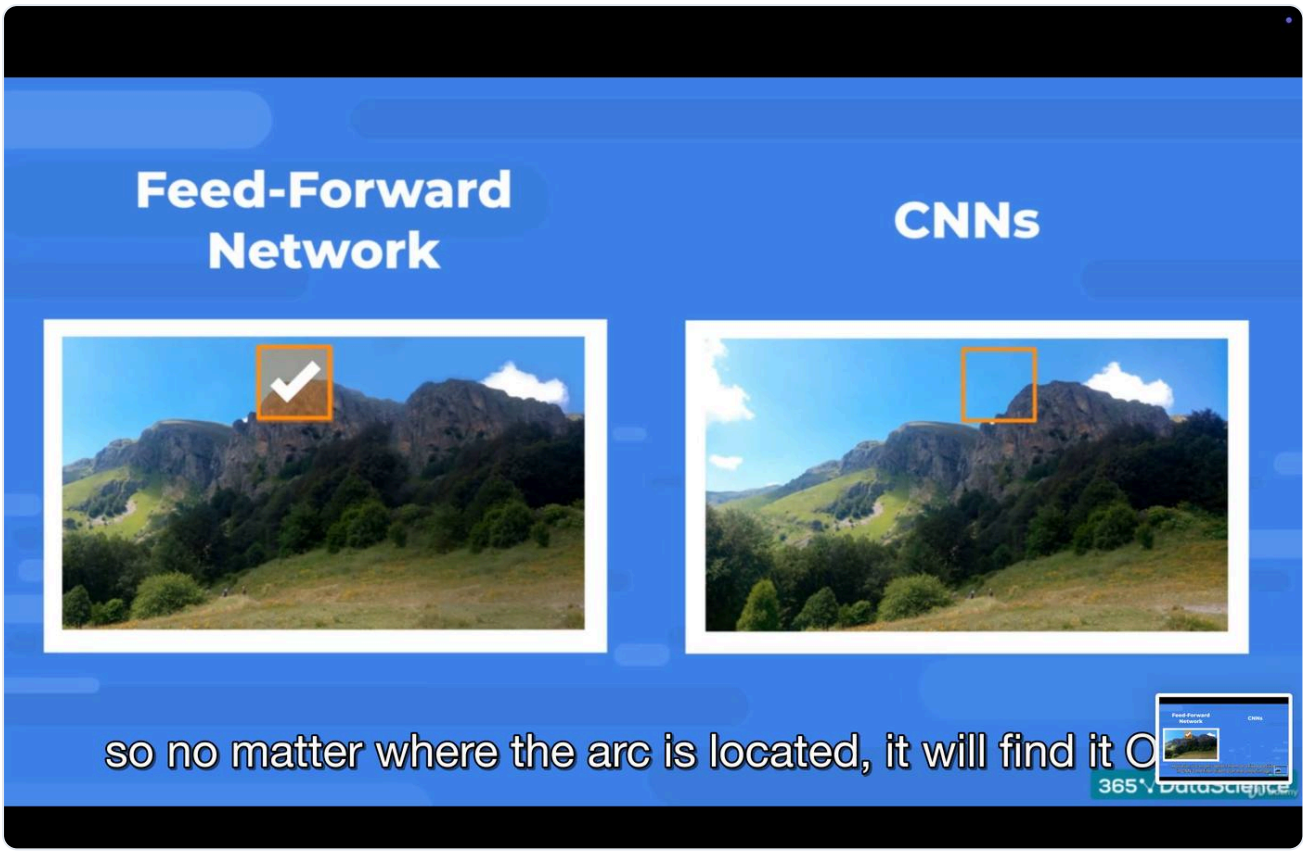

FIGURE 9.39

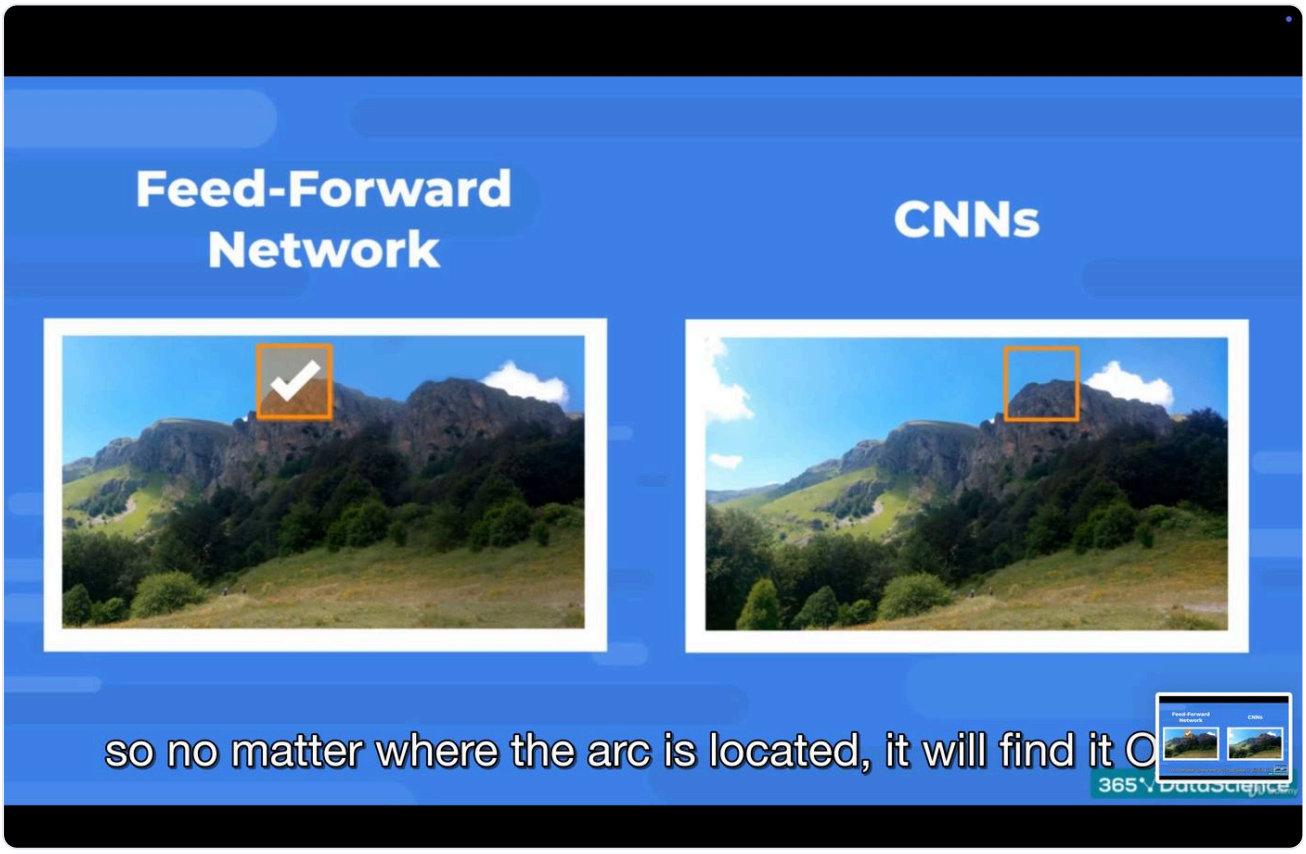

FIGURE 9.40

FIGURE 9.41

However, it can only detect them at a fixed position in CNN's the filter slides over the whole image, so no matter where the arc is located, it will find it OK. So the convolutional layer is simply a layer in which we search for a certain pattern. The kernel may be searching for arcs or for acute angles, or it may be searching for faces.

FIGURE 9.42

FIGURE 9.43

We generally don't really care what the algorithm optimizes for as long as there is a useful pattern to find it then outputs a map of where that pattern has been found.

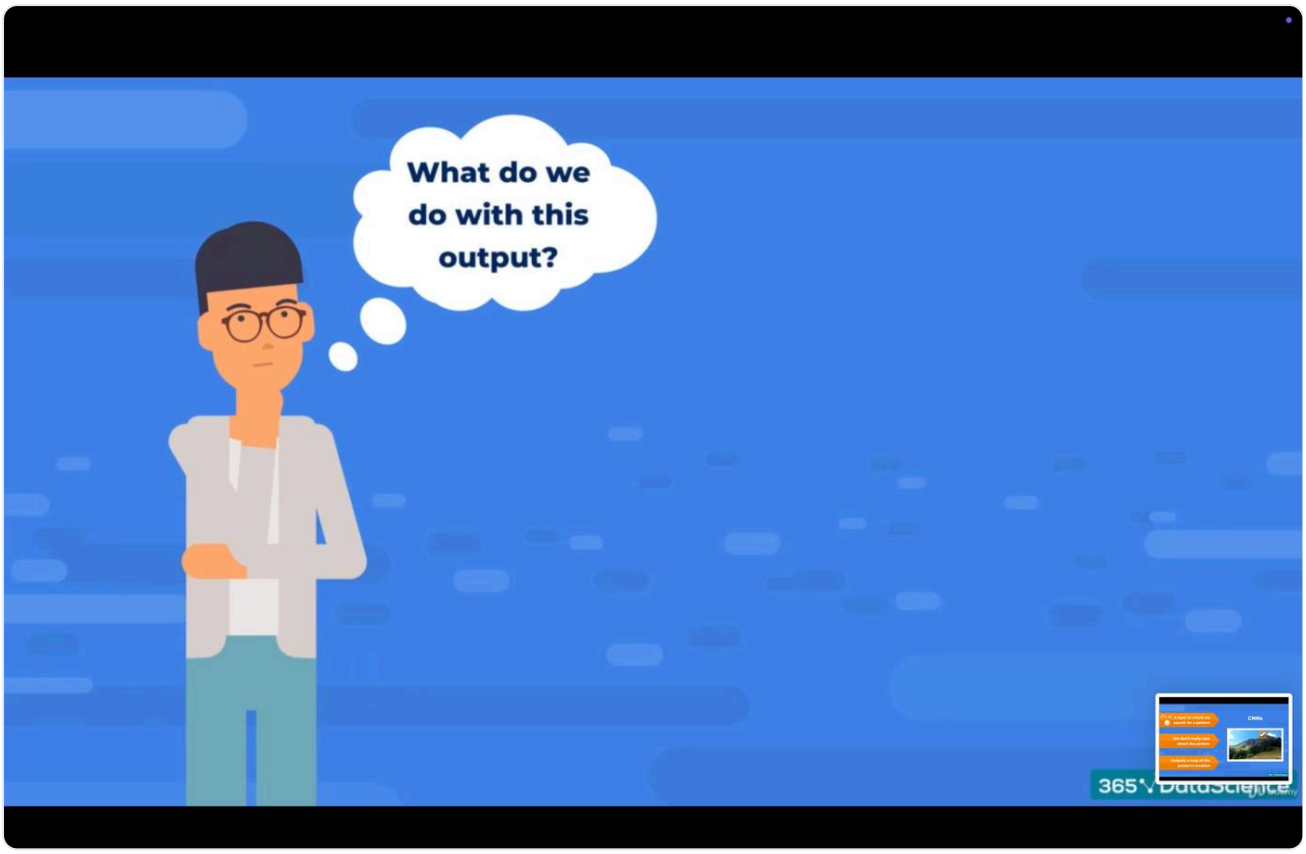

FIGURE 9.44 | What Do We

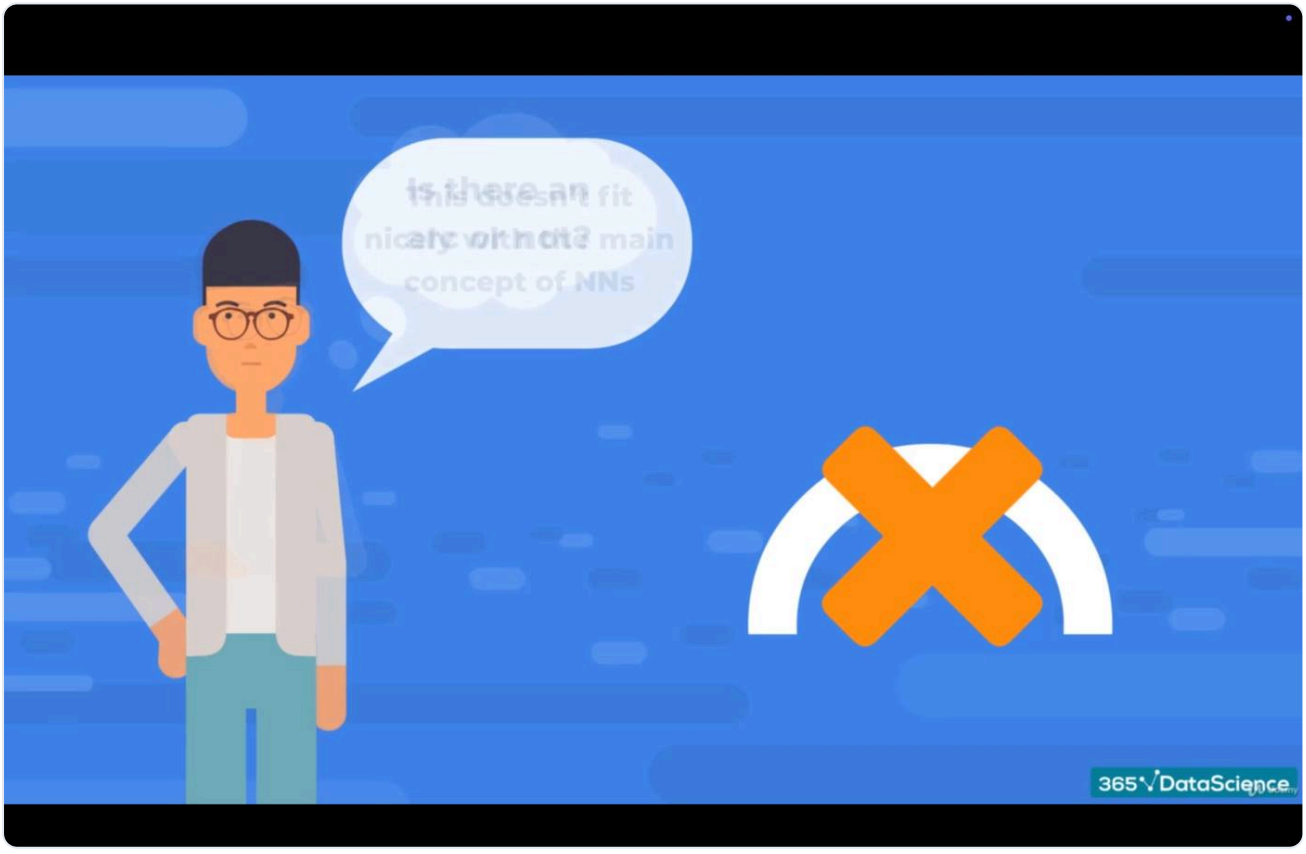

FIGURE 9.45

Great, but what do we do with this output?

FIGURE 9.46 | What Do We

FIGURE 9.47

It doesn't seem useful to stack more layers on top of that because we have erased most of the information and all that is left is an is there an arc here or not?

FIGURE 9.48

This doesn't fit nicely with the main concept of neural networks has layers that can be stacked on top of each other. So what we actually do in convolutional layers is just a bit different. Instead of having one kernel that we convolve, we may have hundreds of kernels.

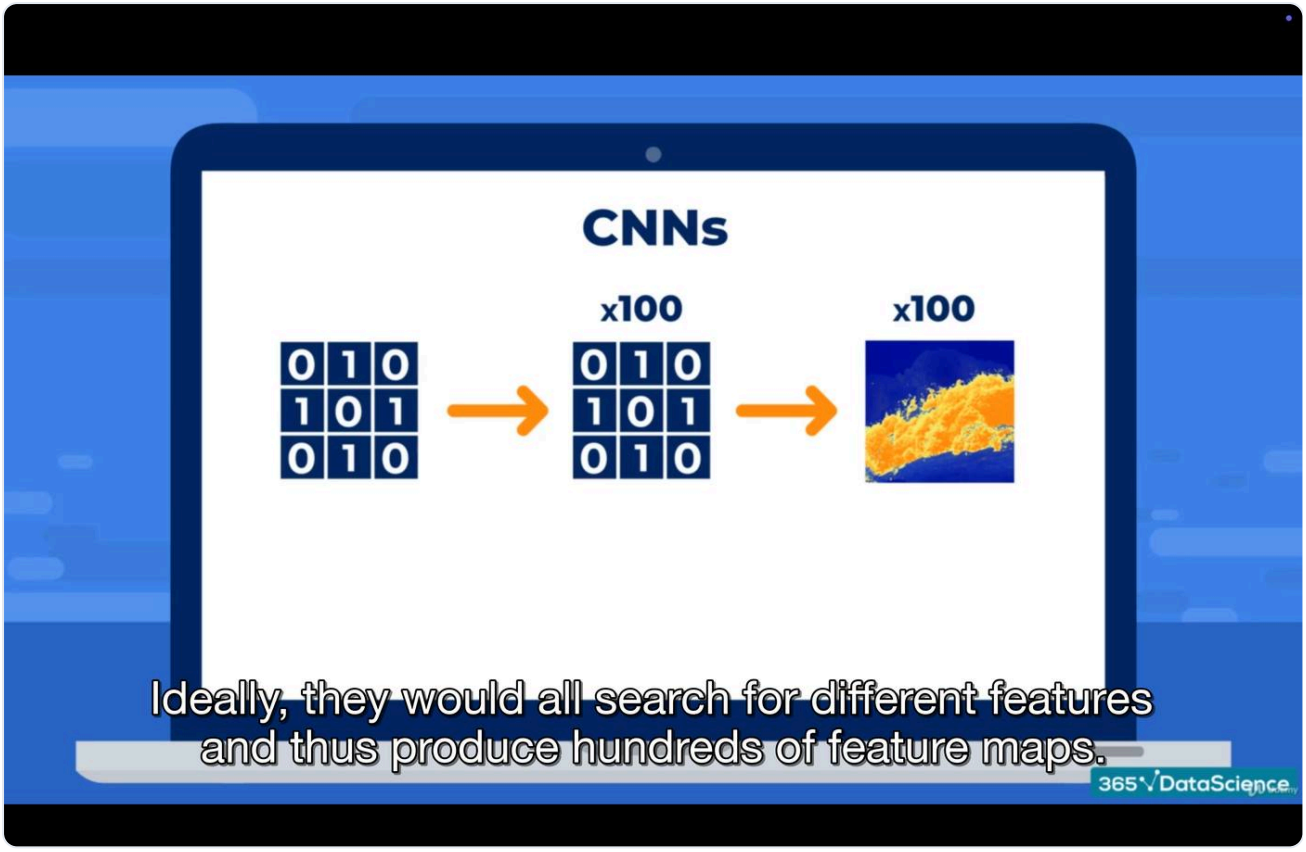

FIGURE 9.49

Ideally, they would all search for different features and thus produce hundreds of feature maps.

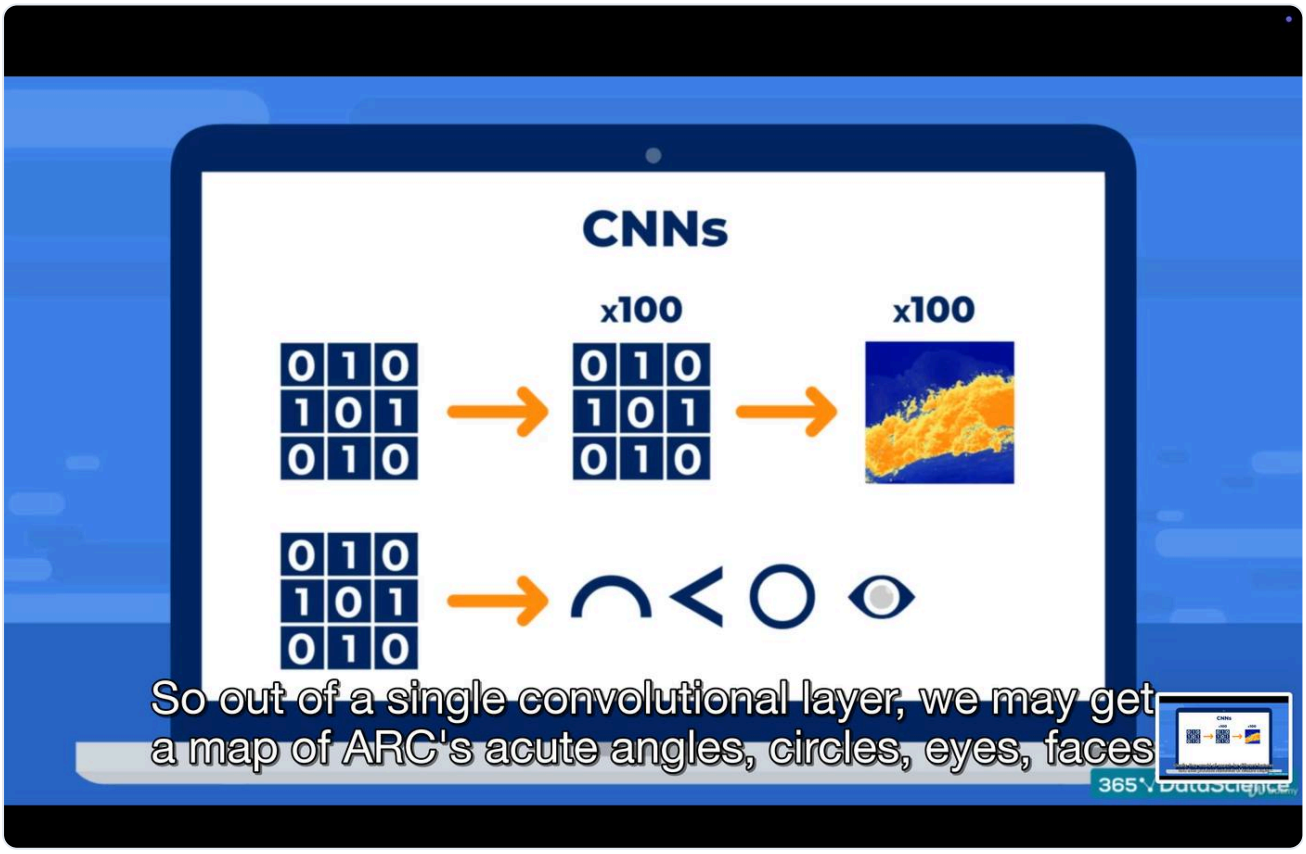

FIGURE 9.50

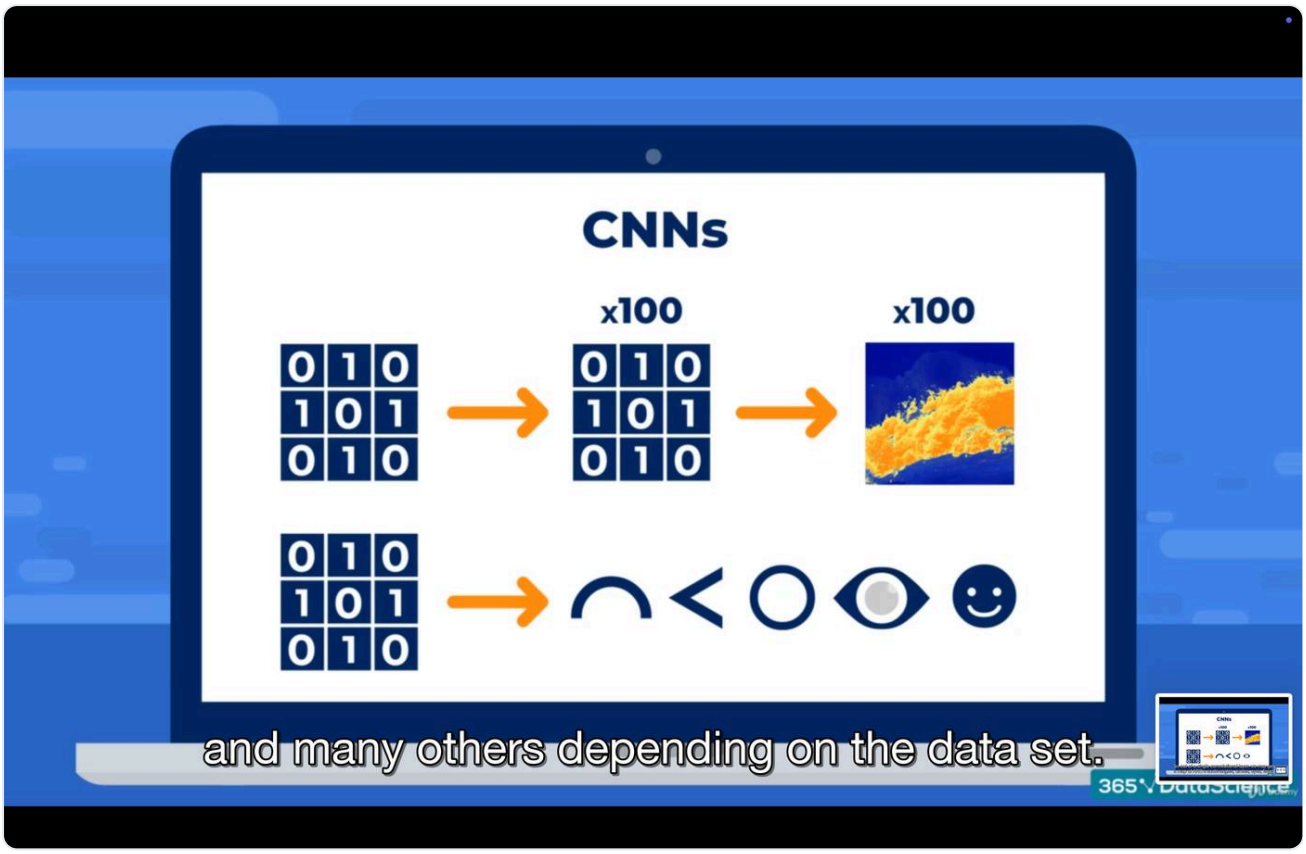

FIGURE 9.51

So out of a single convolutional layer, we may get a map of arcs acute angles, circles, eyes, faces and many others depending on the data set. Cool.

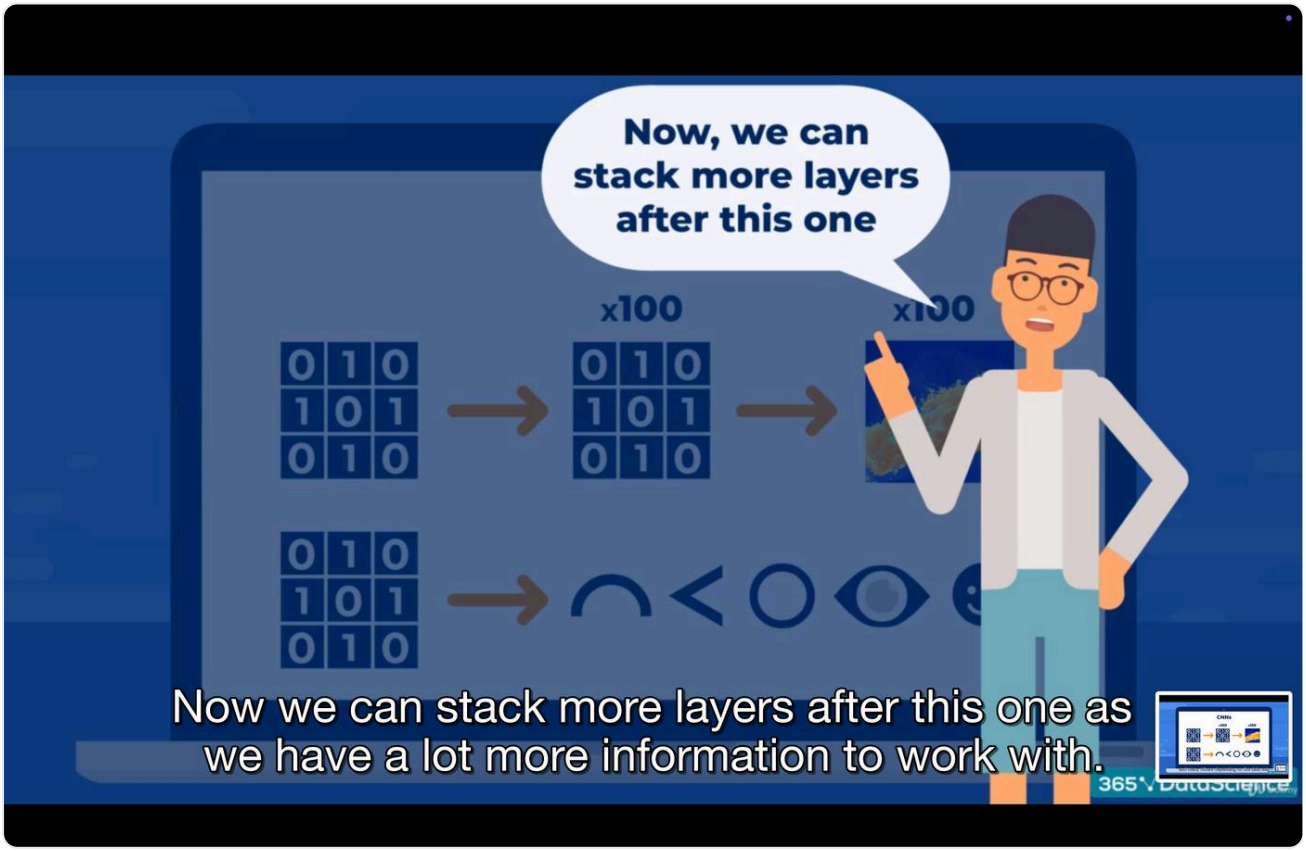

FIGURE 9.52

Now we can stack more layers after this one as we have a lot more information to work with.

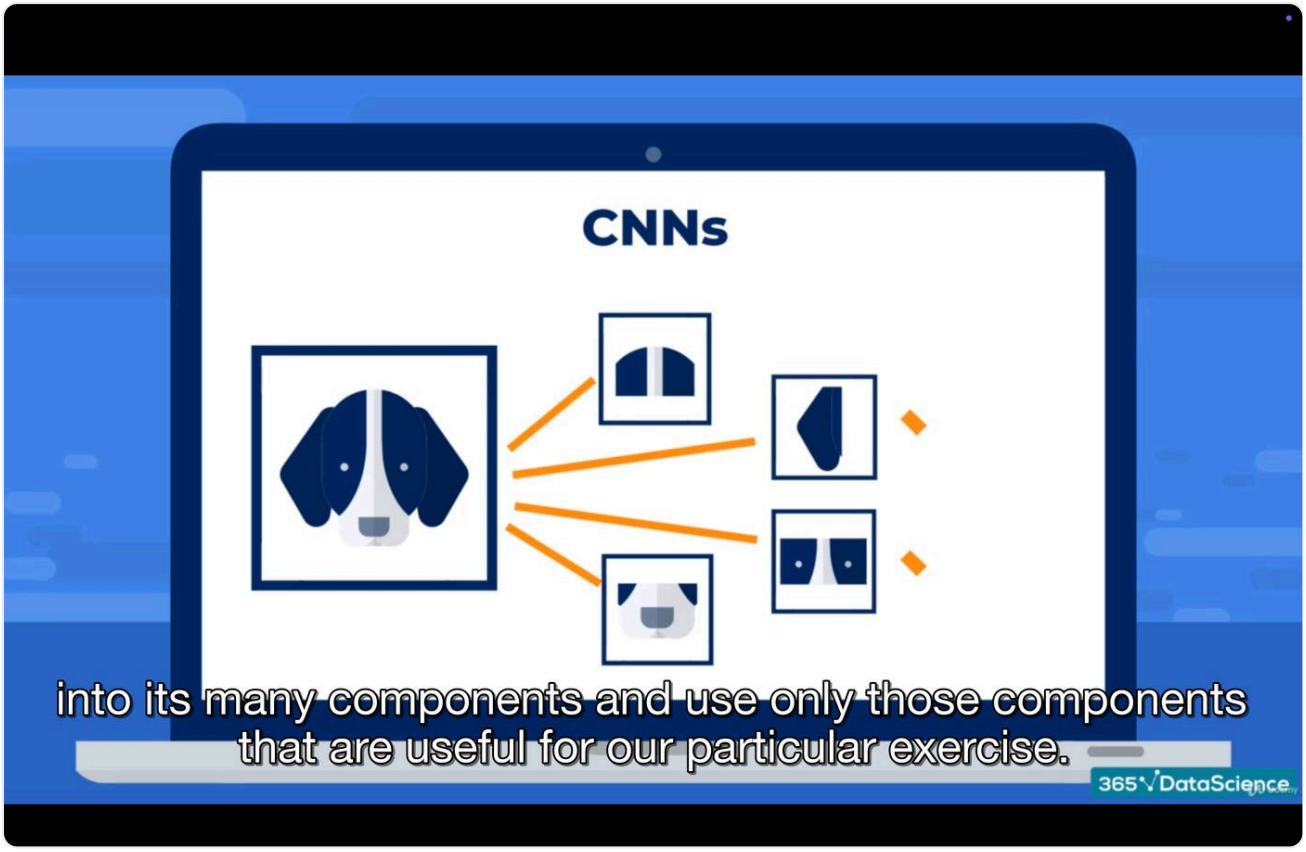

FIGURE 9.53

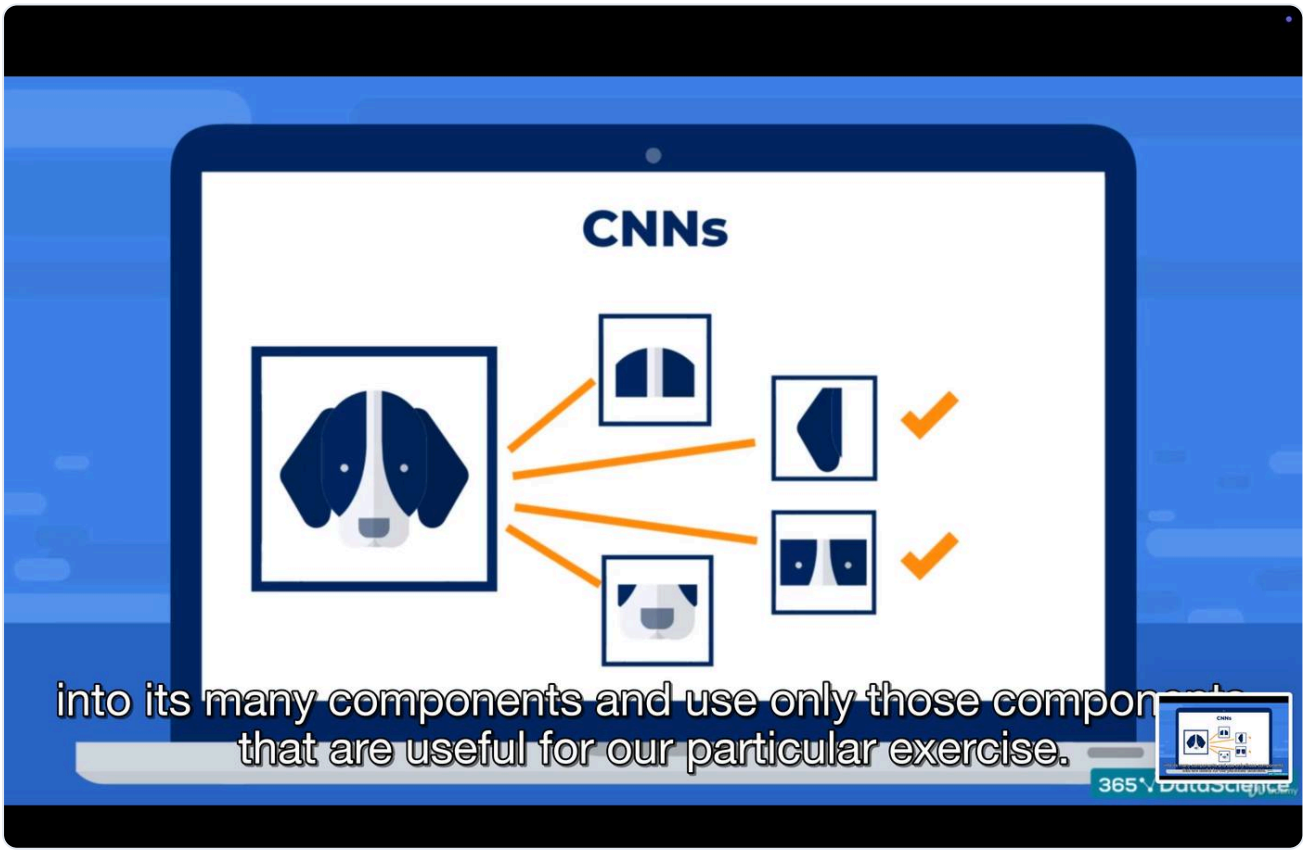

FIGURE 9.54

Now, one might say that all we do with a convolutional layer is that we collapse the original image into its many components and use only those components that are useful for our particular exercise. That is pretty much all there is to the convolutional layer. With one exception.

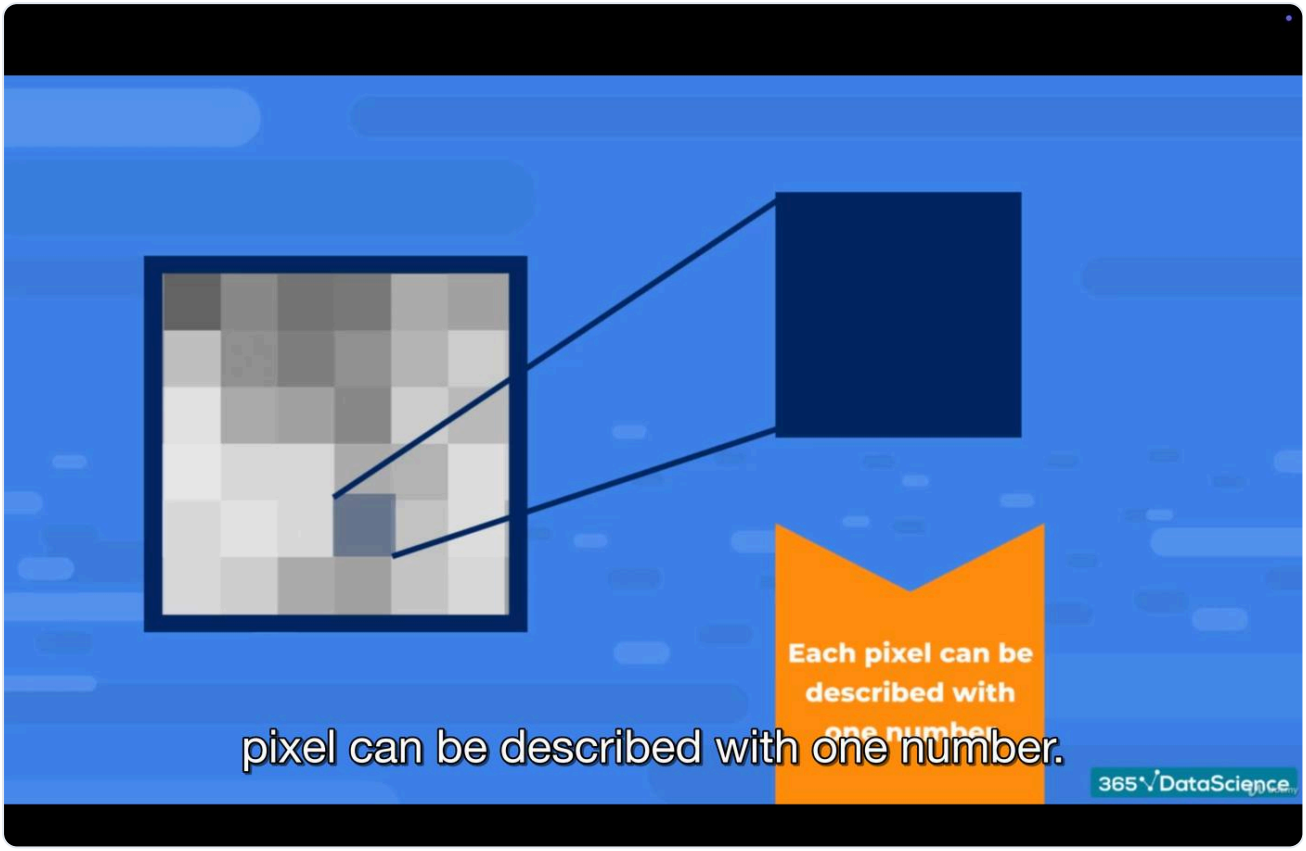

FIGURE 9.55

So far in the course, we have always assumed that the input image is in grayscale, meaning that each pixel can be described with one number.

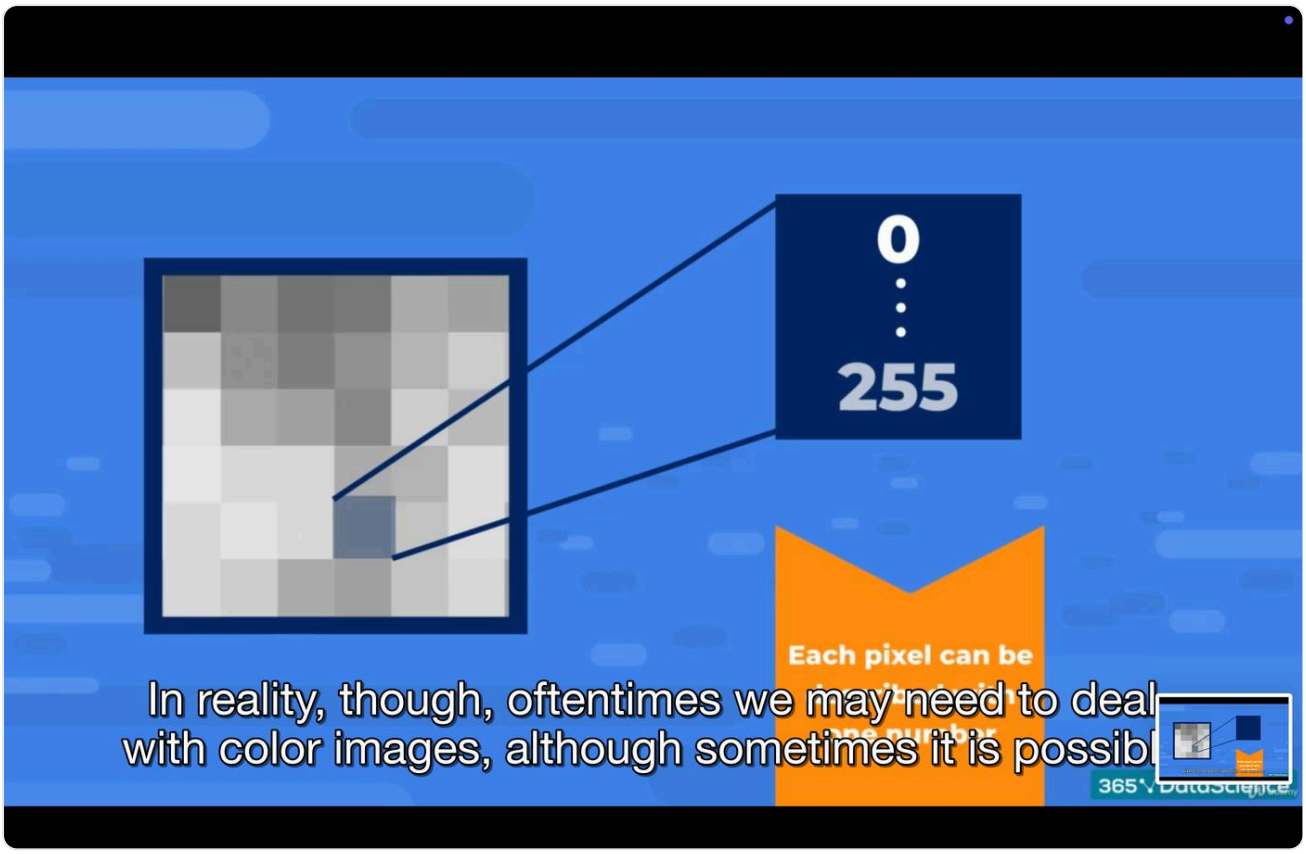

FIGURE 9.56

FIGURE 9.57

In reality, though, oftentimes we may need to deal with color images, although sometimes it is possible to get good results.

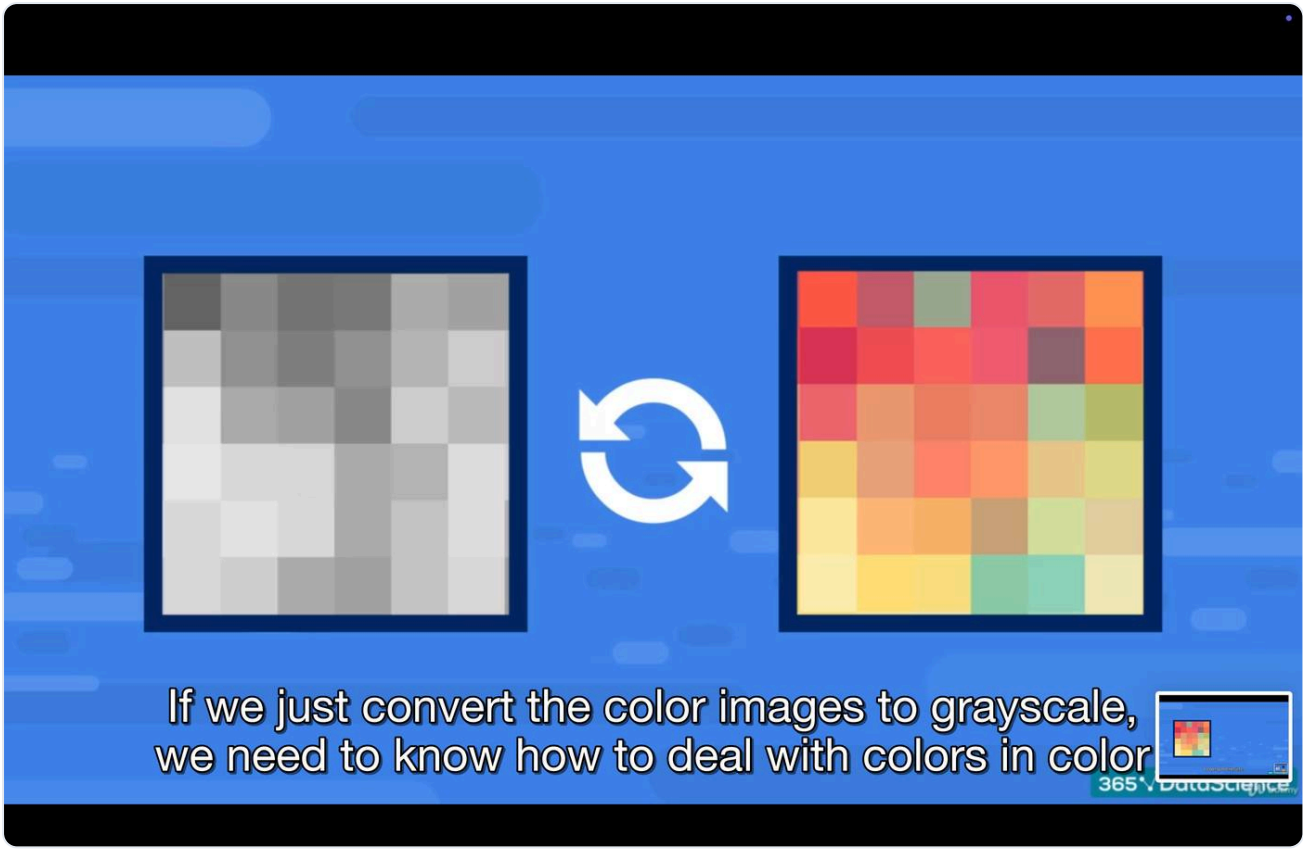

FIGURE 9.58

If we just convert the color images to grayscale, we need to know how to deal with colors in color images. Each pixel is represented by three numbers red, green and blue intensity.

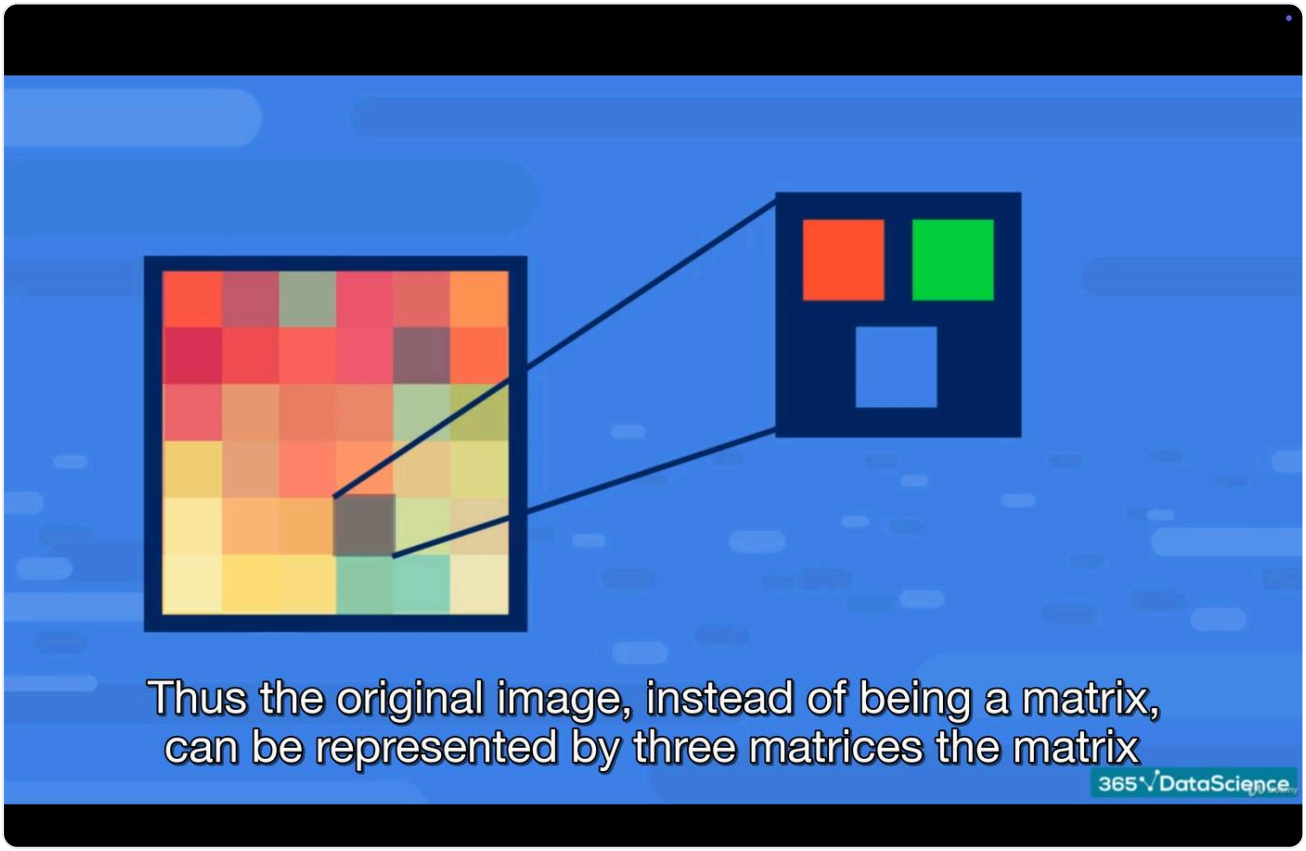

FIGURE 9.59

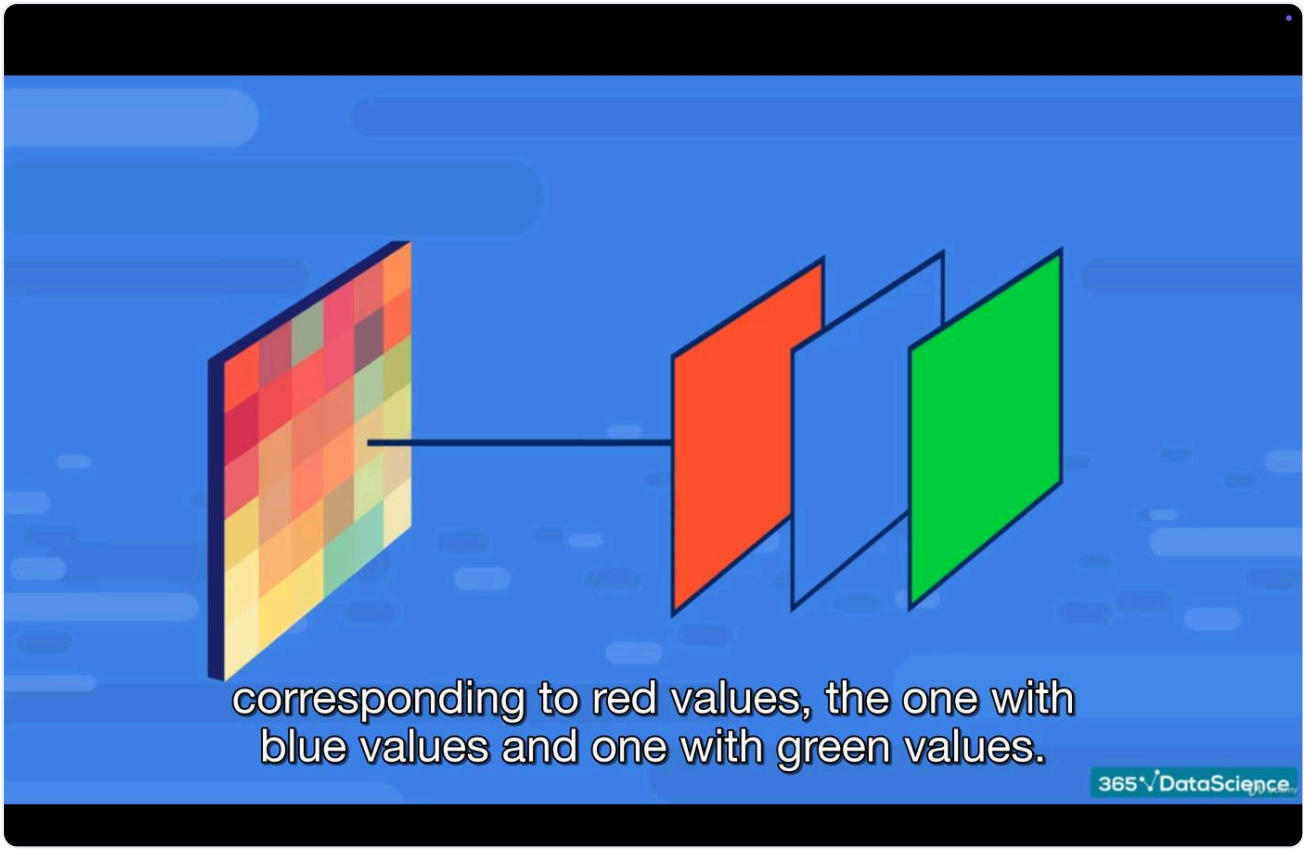

FIGURE 9.60

Thus the original image, instead of being a matrix, can be represented by three matrices the matrix corresponding to red values, the one with blue values and one with green values. So the image transforms from a 2D tensor to a.

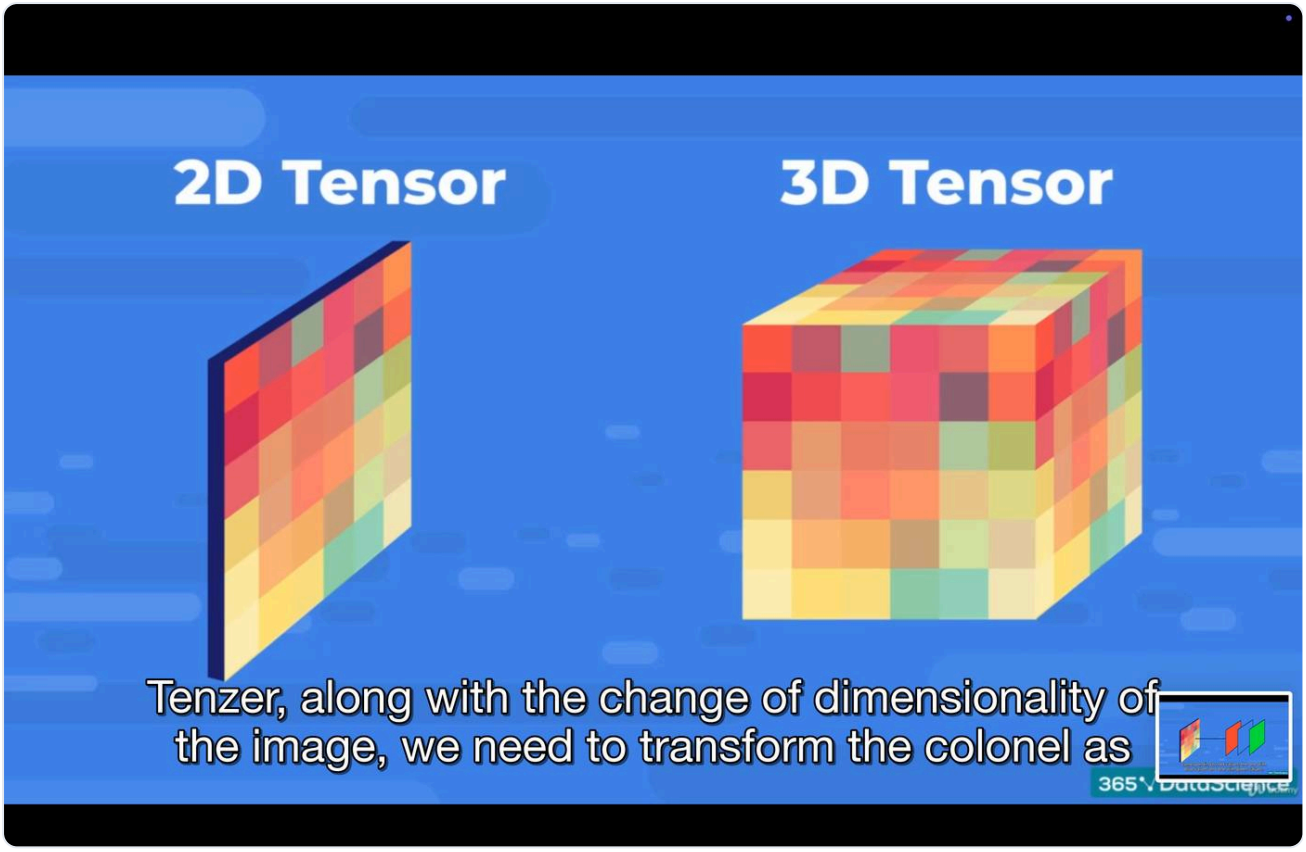

FIGURE 9.61

TensorFlow, along with the change of dimensionality of the image, we need to transform the kernel as well. Previously, a Matrix kernel worked well, but now we need to introduce depth to the kernel as well.

FIGURE 9.62

Luckily, this isn't that hard to achieve. The rule is that the kernel always needs to have the same depth as the image.

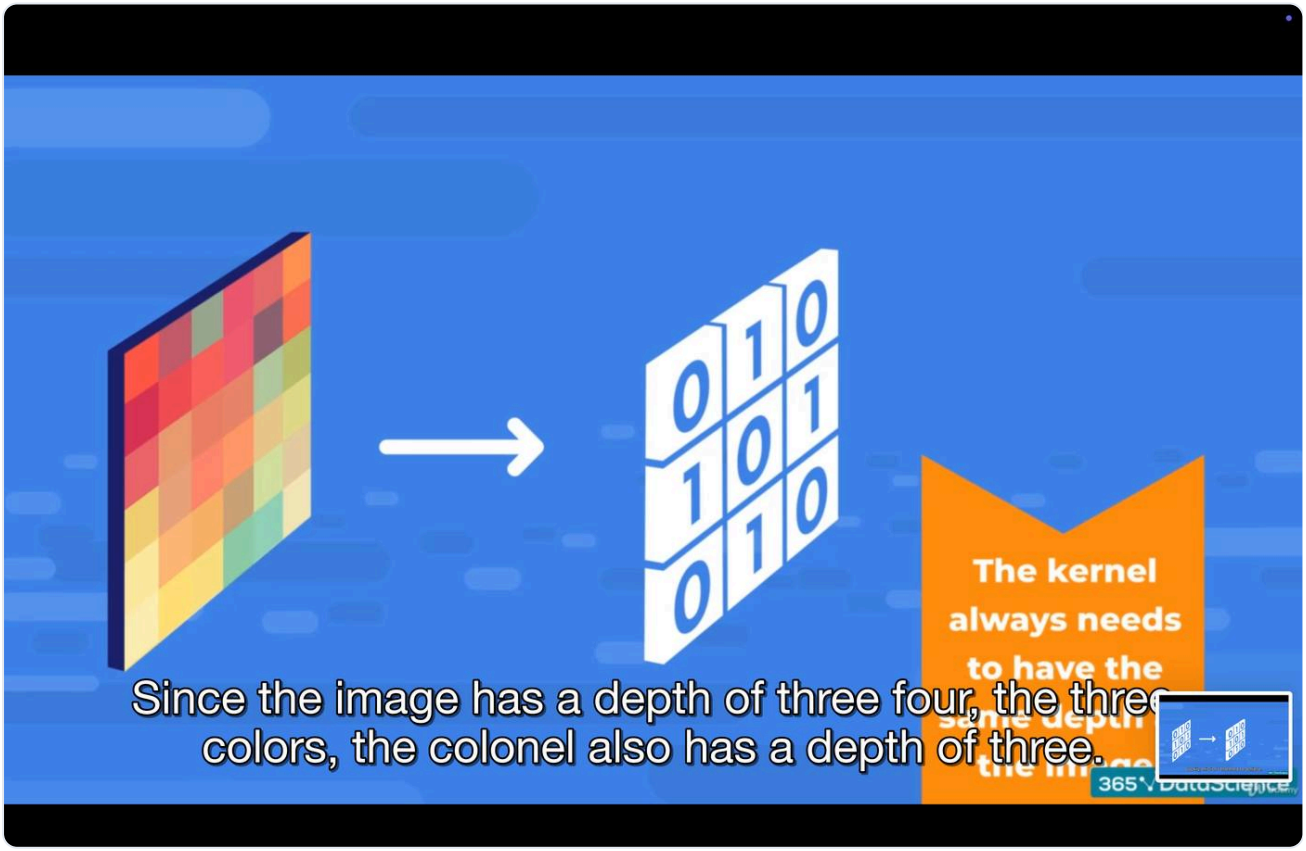

FIGURE 9.63

Since the image has a depth of three four, the three colors, the kernel also has a depth of three.

FIGURE 9.64

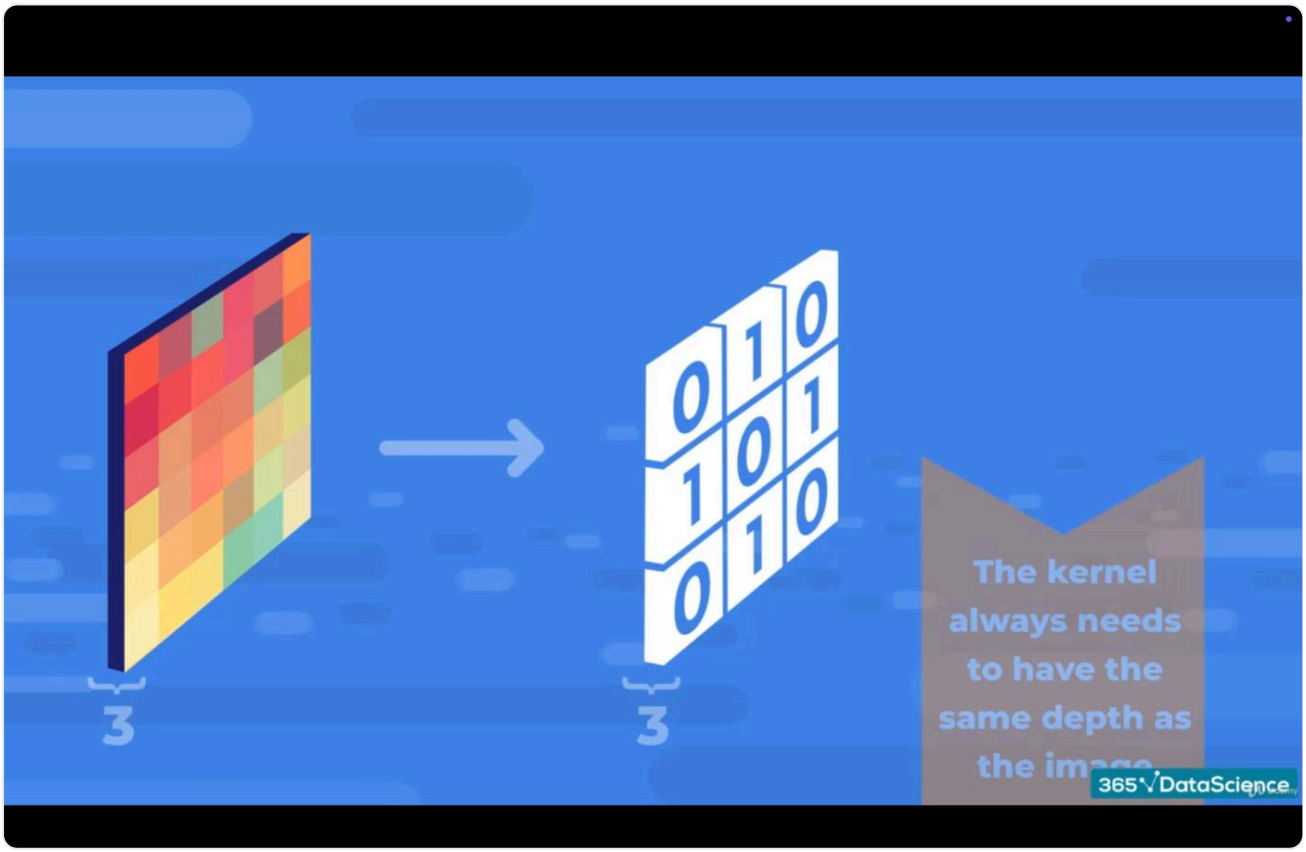

FIGURE 9.65

Most importantly, the output doesn't change.

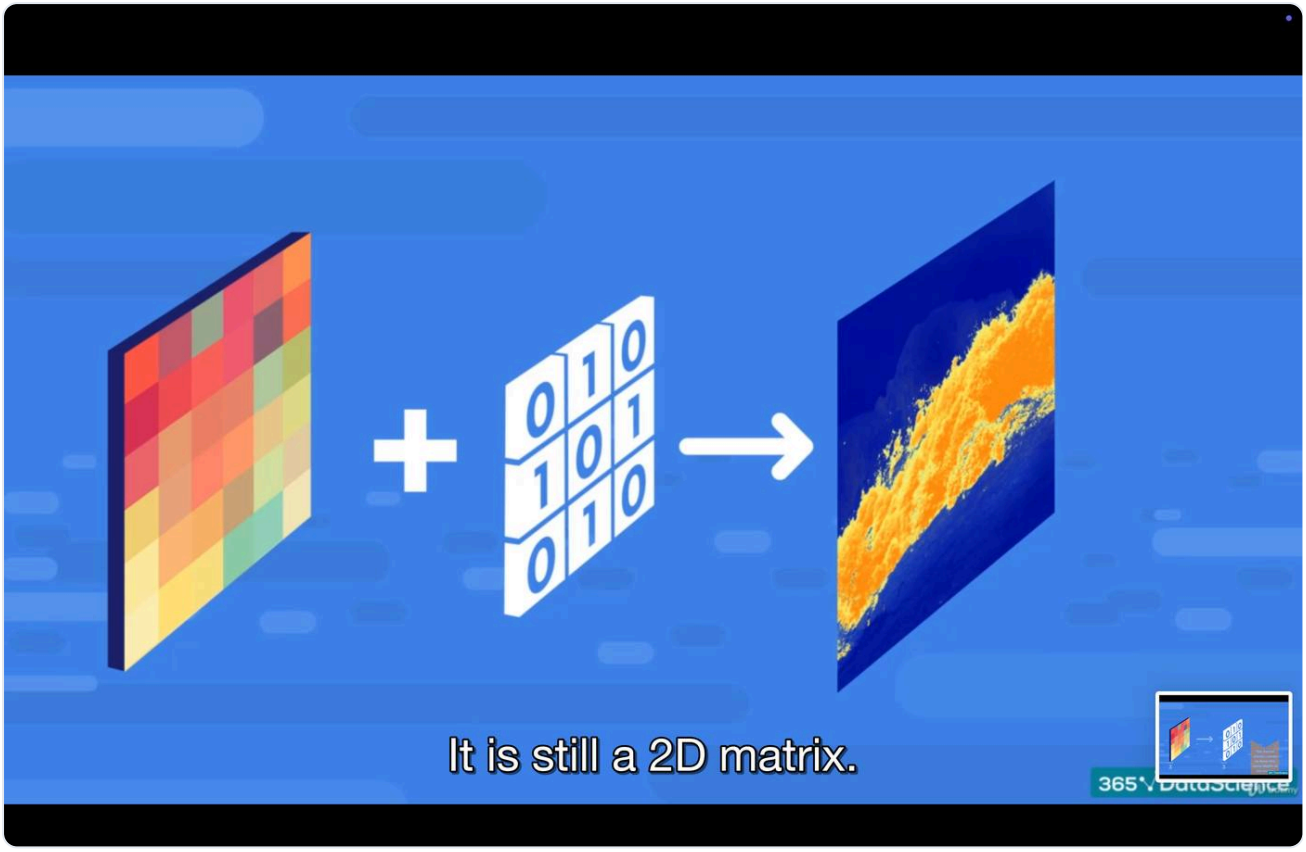

FIGURE 9.66

It is still a 2D matrix.

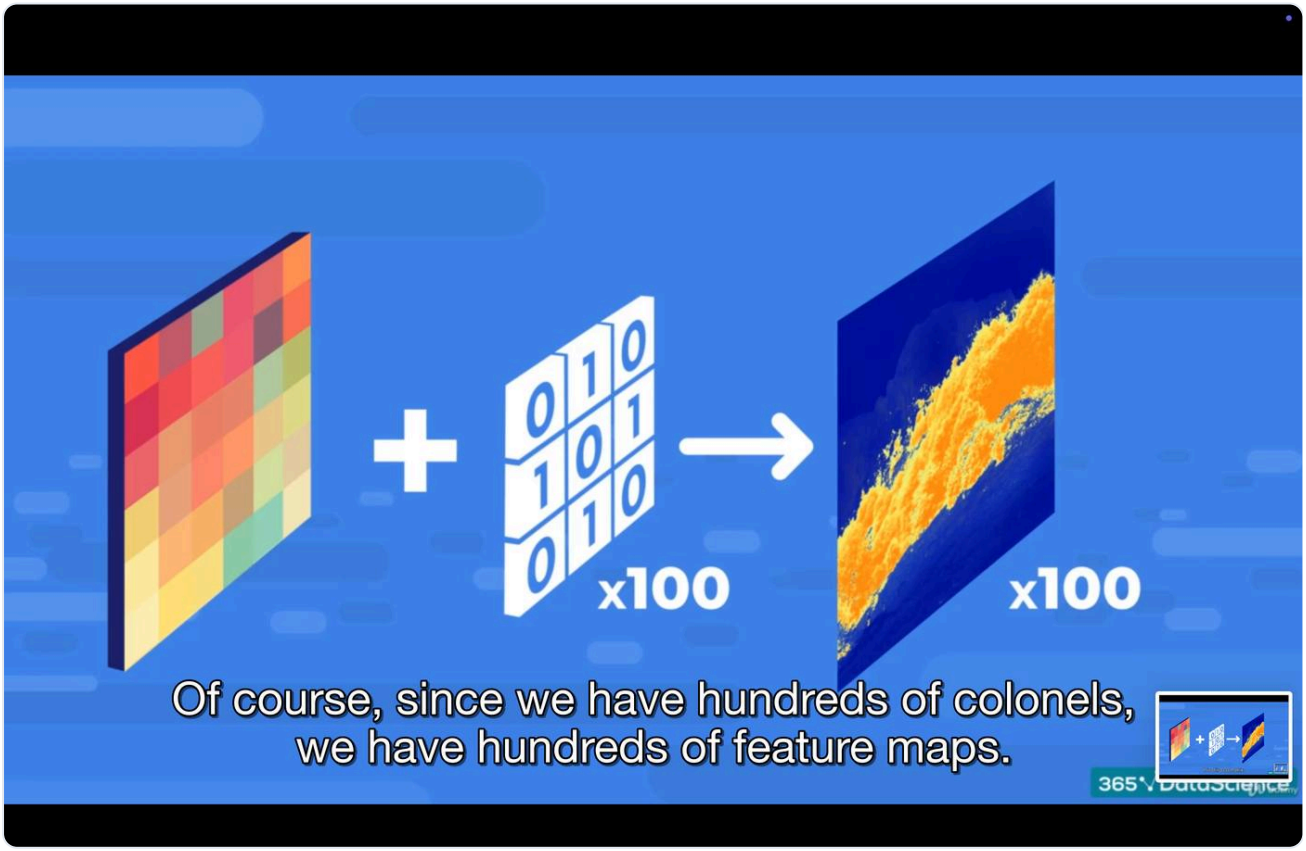

FIGURE 9.67

Of course, since we have hundreds of colonels, we have hundreds of feature maps. OK, so far in this section we saw what a convolutional layer is and how it is constructed. But that's only part of the story of a CNN tune in the next lecture where we will talk about the other ingredients pooling layers. Thanks for watching.

LECTURE 10

Smears Out

Welcome back to this section on CNN's in this lecture, we will discuss the second ingredient in a CNN pooling layers. We will first explain how they work and then motivate their usage. OK. If we have an image that is one thousand nine hundred and twenty pixels by one thousand and eighty pixels, we can safely assume that making it much smaller, say nine sixty by five forty, what an effect.

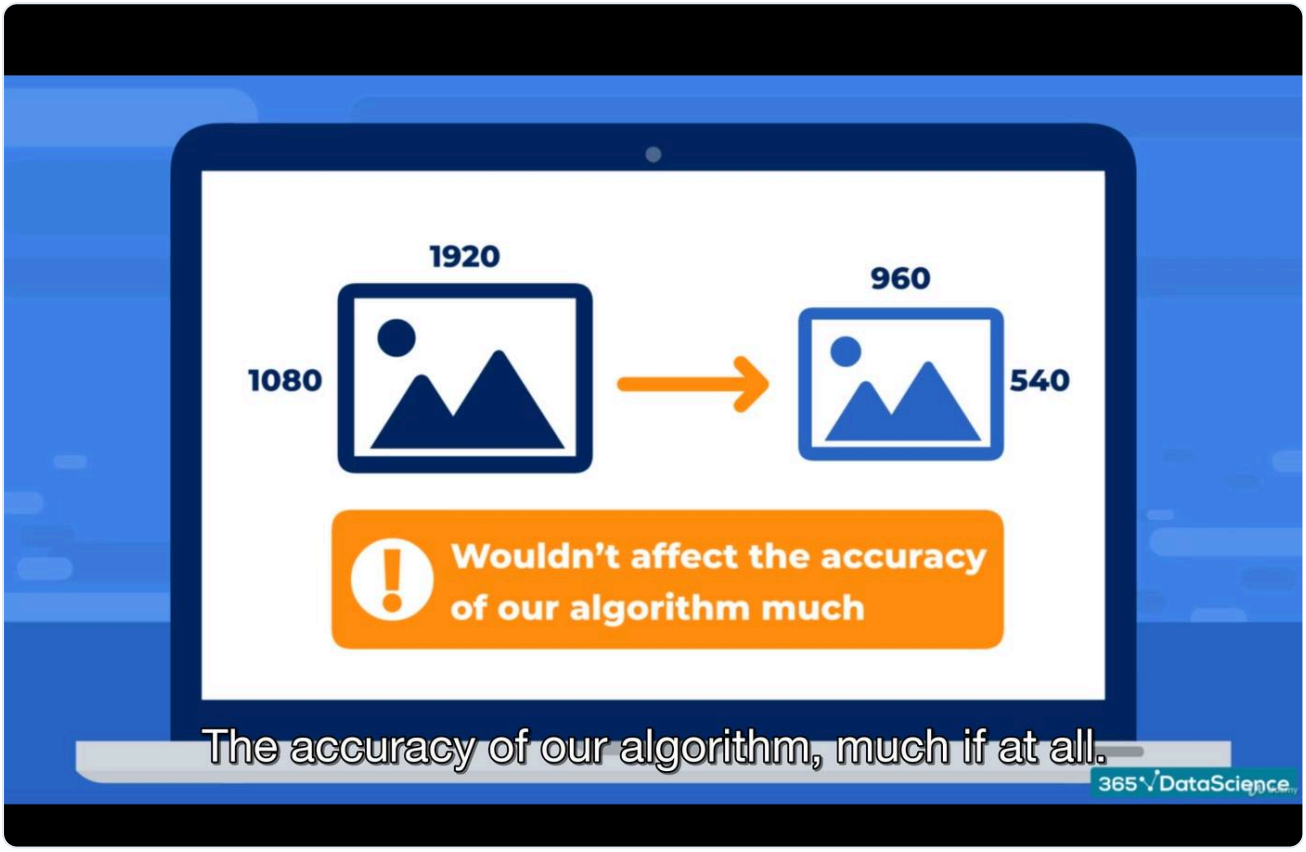

FIGURE 10.1

The accuracy of our algorithm, much if at all.

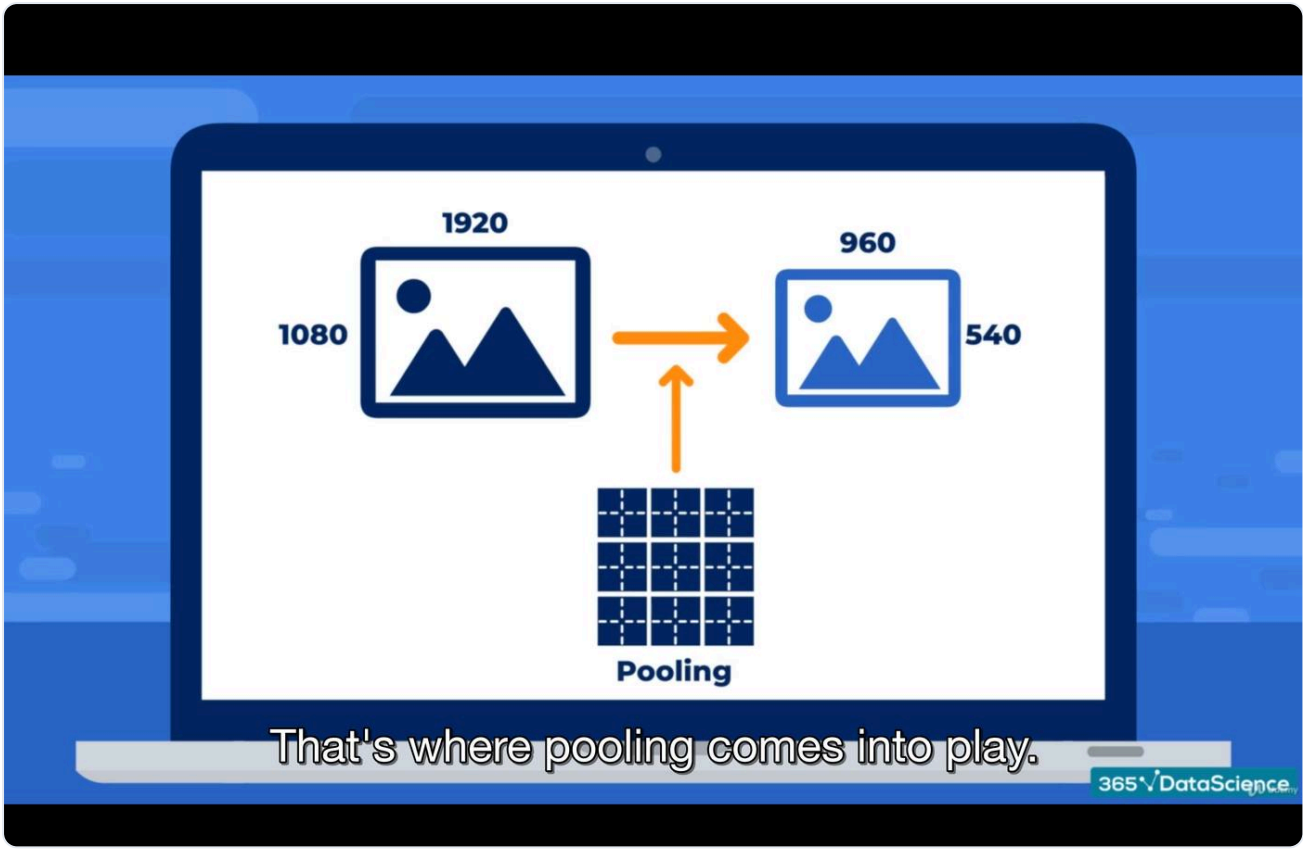

FIGURE 10.2

That's where pooling comes into play. Pooling essentially reduces the dimensionality of the problem we are facing.

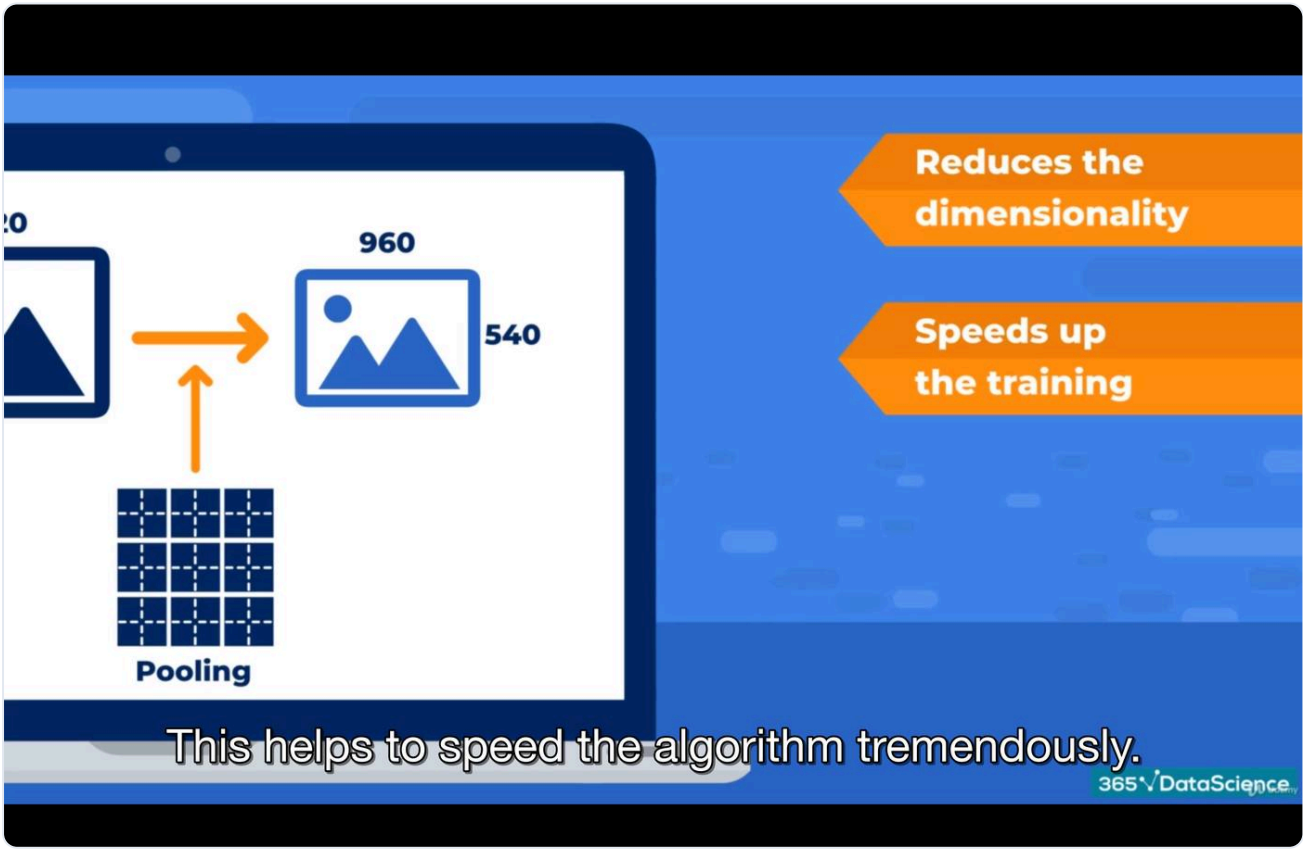

FIGURE 10.3 | Reduces the

This helps to speed the algorithm tremendously. Why does pooling have such a special name if it is nothing but a way to shrink an image?

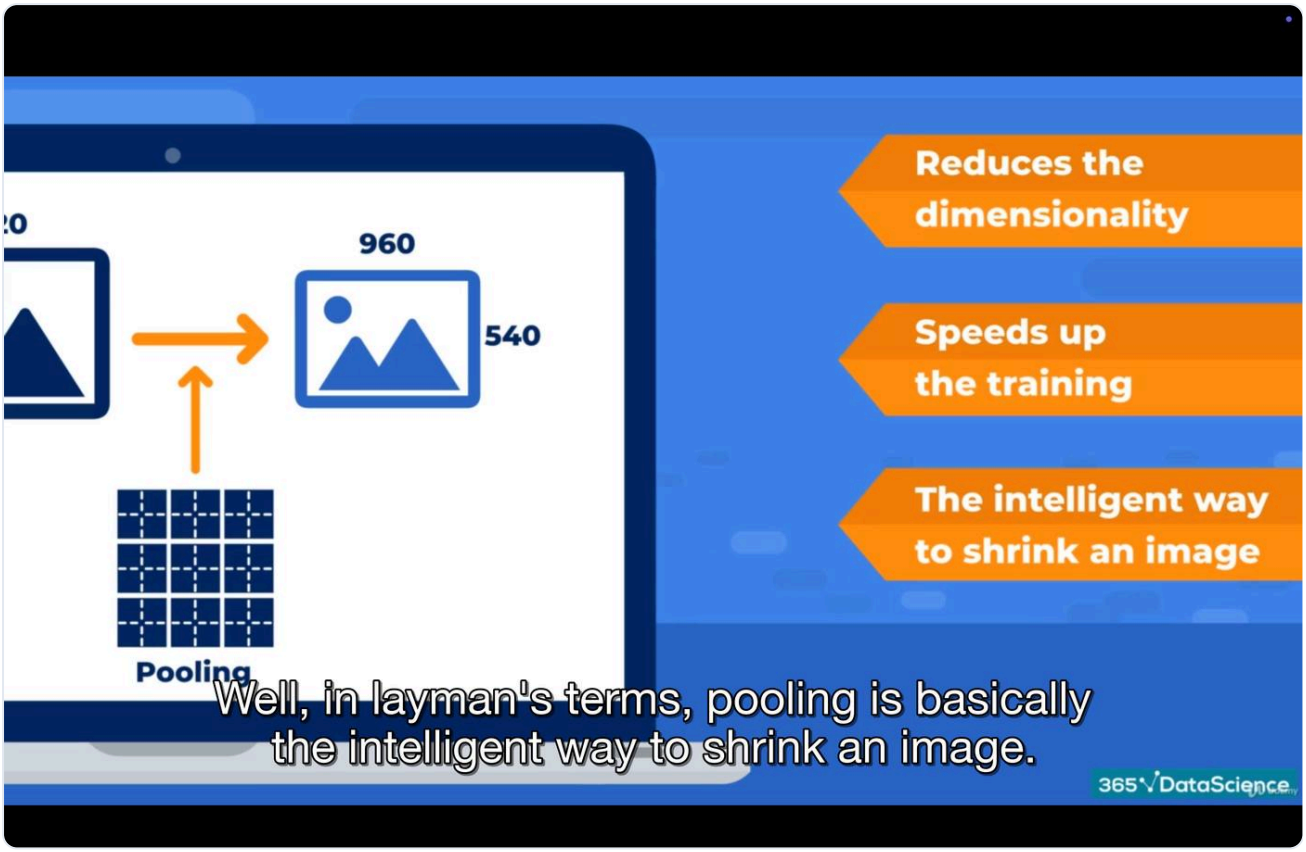

FIGURE 10.4 | Reduces the

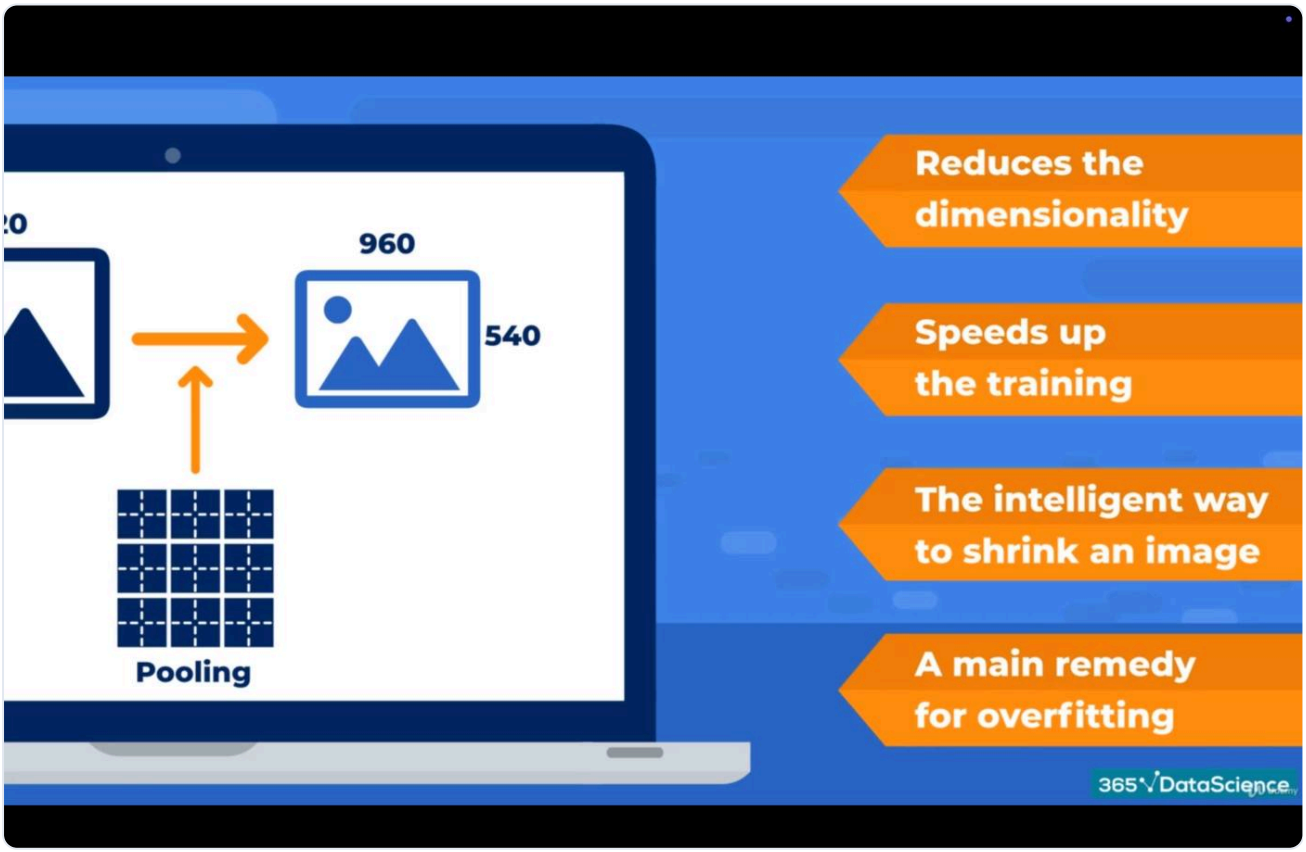

FIGURE 10.5 | Reduces the

Well, in layman's terms, pooling is basically the intelligent way to shrink an image. Moreover, as we will see in just a bit, pooling is also one of the main remedies for overfitting. Let's jump straight into an example. So there are a few types of pooling, but let's start with the most commonly used one, two by two max pooling layer with stride to the idea behind this layer is quite simple. We take a feature map from the previous layer, then partition it into two by two regions.

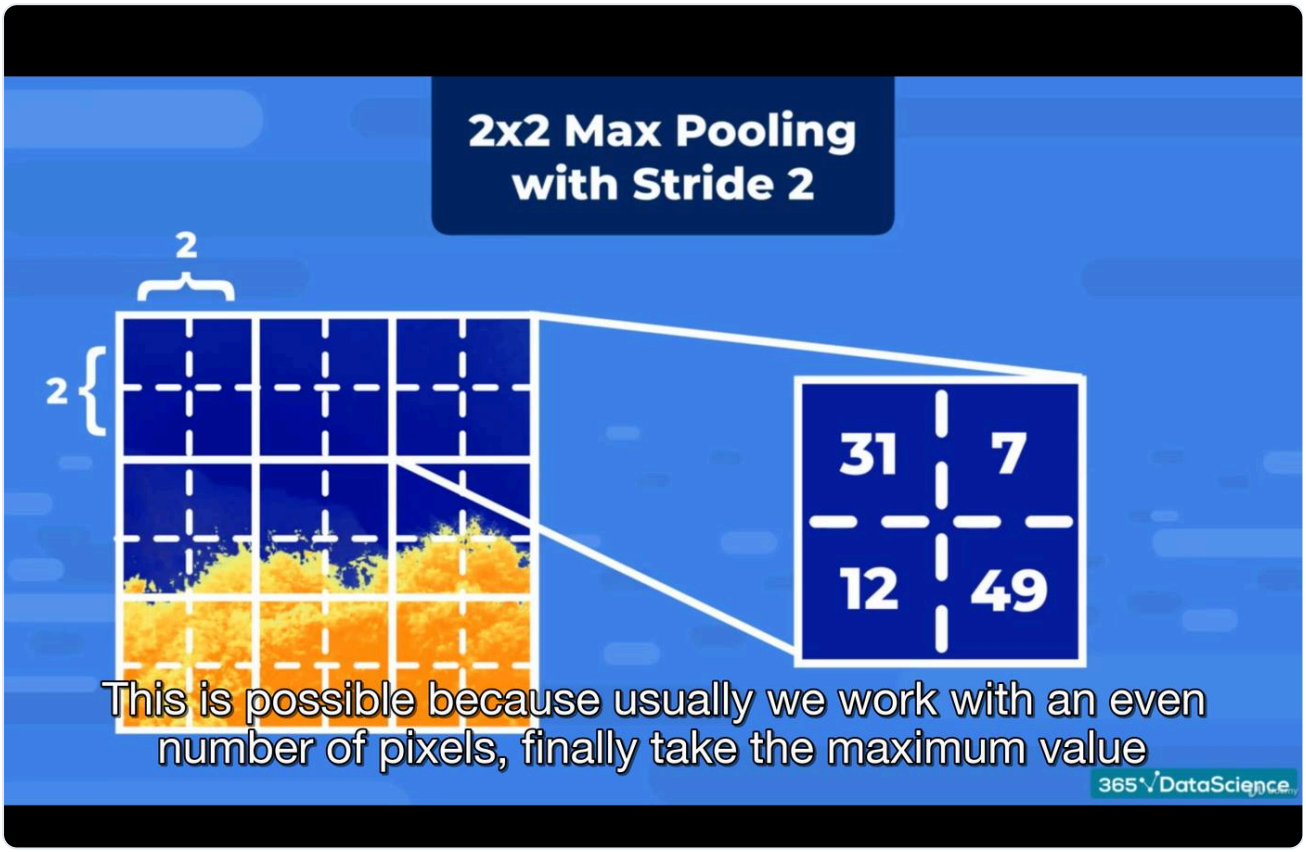

FIGURE 10.6

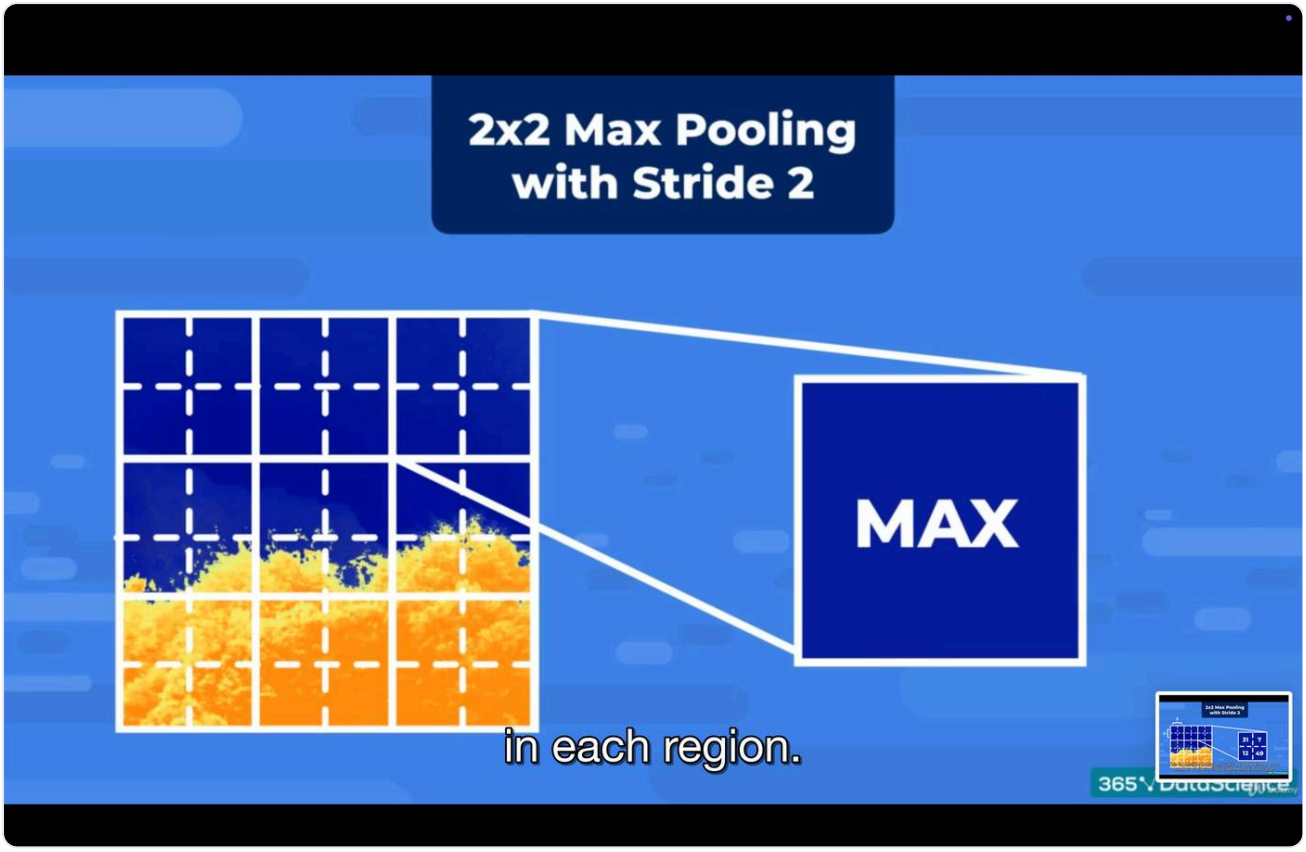

FIGURE 10.7

This is possible because usually we work with an even number of pixels, finally take the maximum value in each region. That's it.

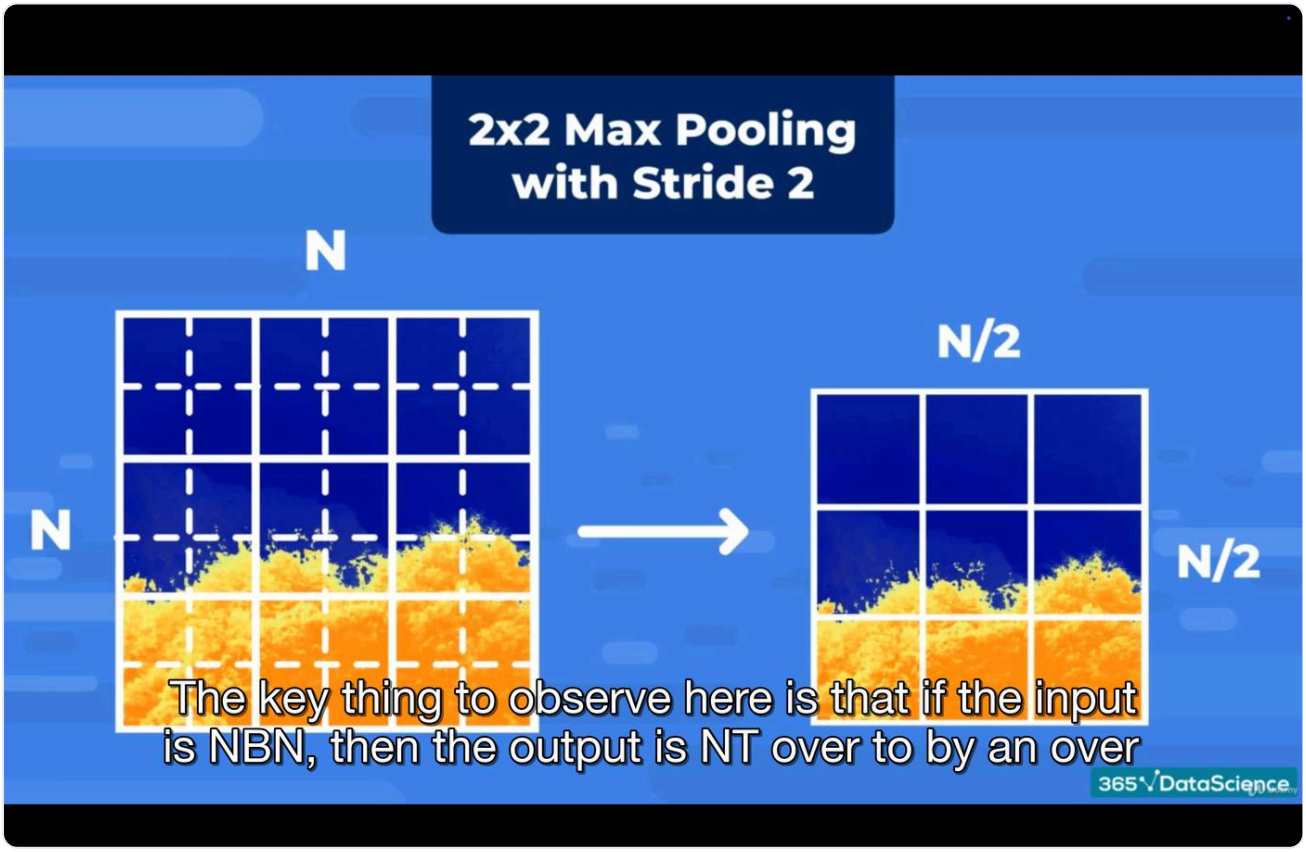

FIGURE 10.8

The key thing to observe here is that if the input is NBN, then the output is NT over to by an over two, we replace a region of two by two four numbers with a single number, their maximum.

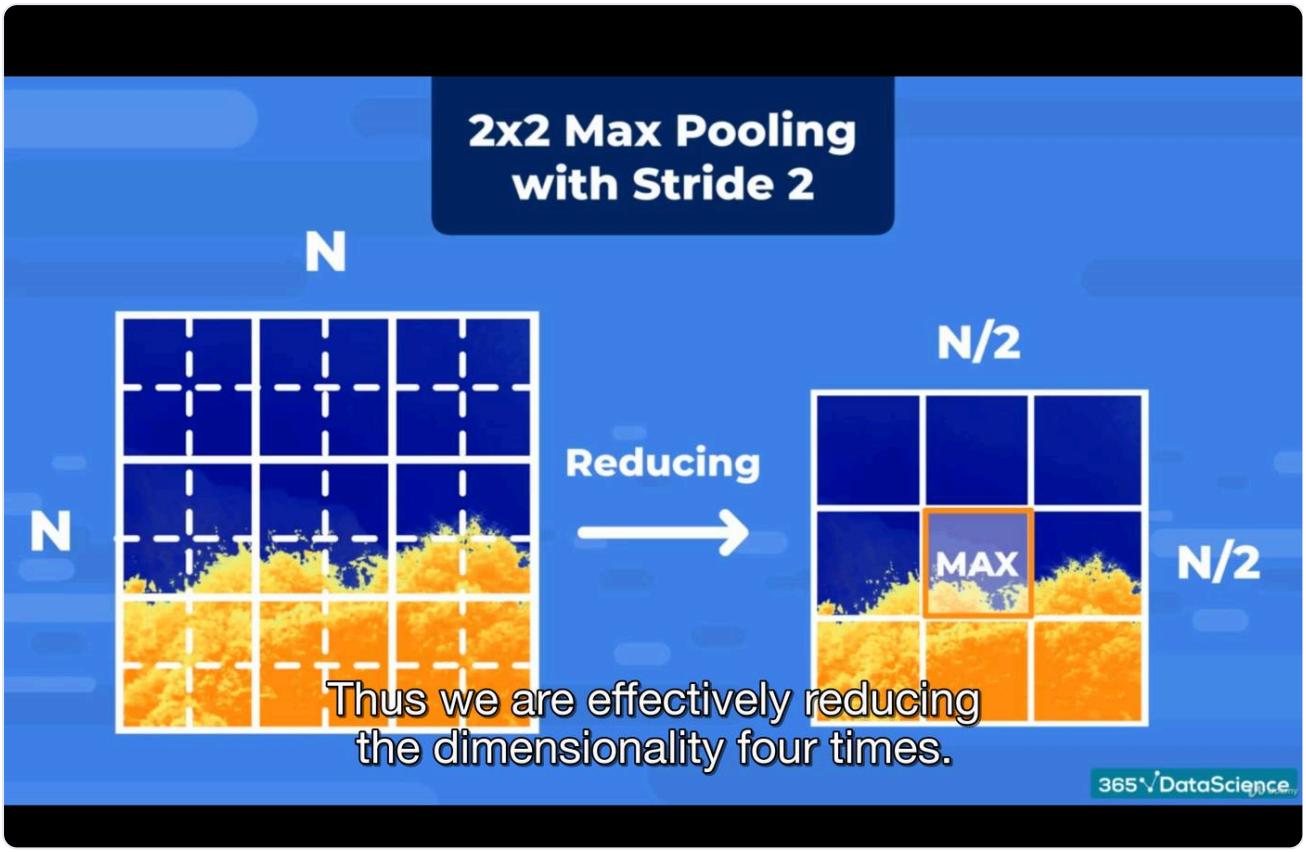

FIGURE 10.9

Thus we are effectively reducing the dimensionality four times. All right.

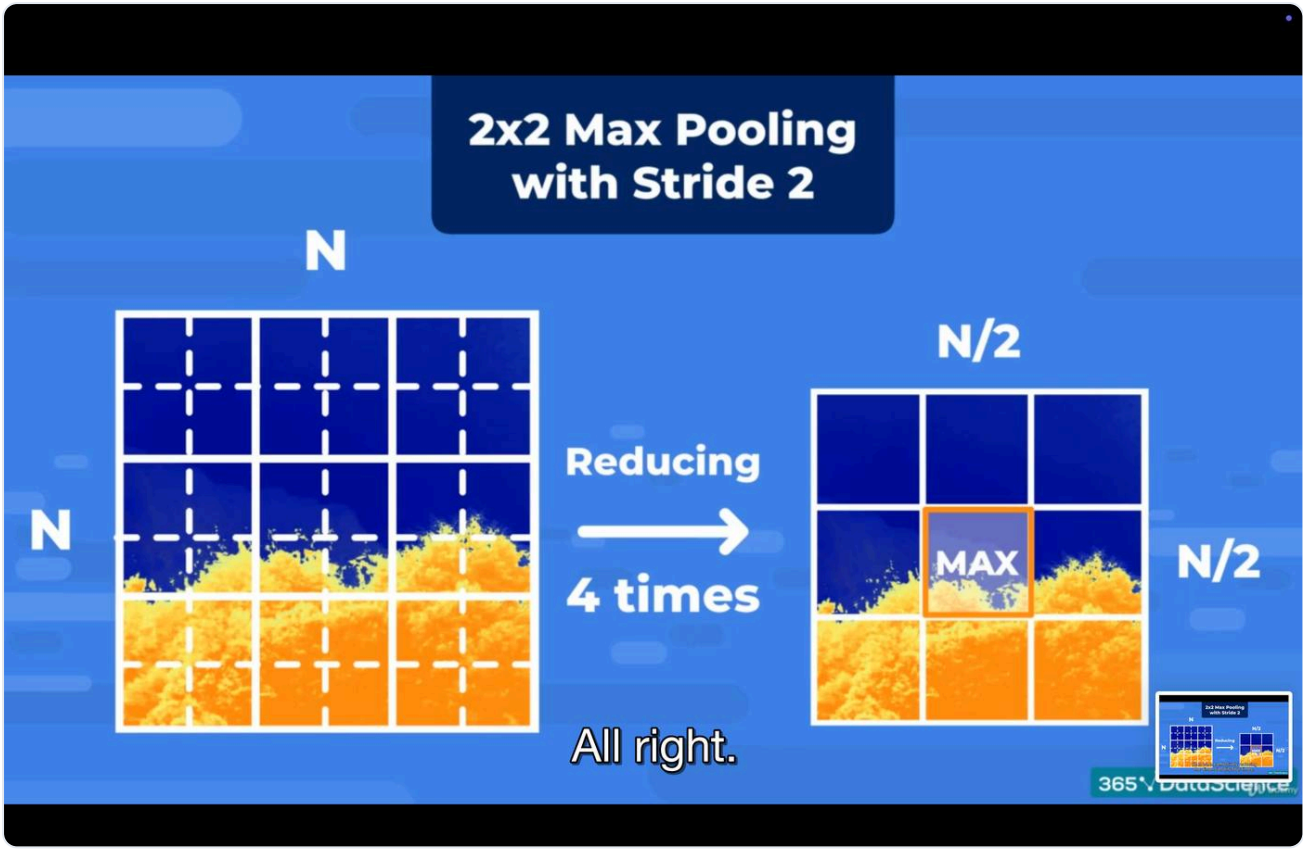

FIGURE 10.10

So why would that be useful at all? Well, there are two main reasons.

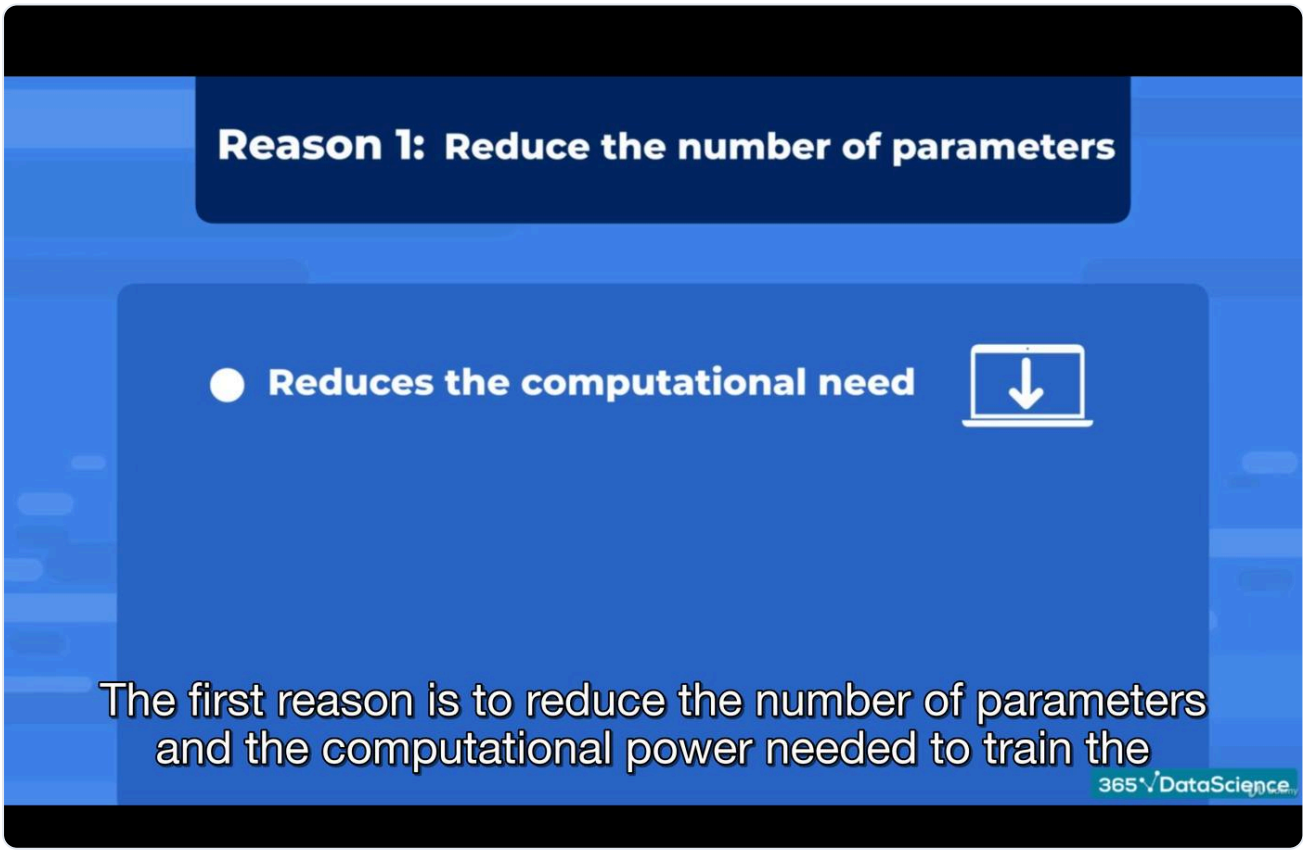

FIGURE 10.11

The first reason is to reduce the number of parameters and the computational power needed to train the model while preserving the most important information. In general, efficient computation is a big topic in machine learning, as oftentimes models may be limited by the parameters of the machines on which we are training. Even if they aren't achieving the same result 100 times faster is always a plus as it helps us try more variations or even focus on other problems.

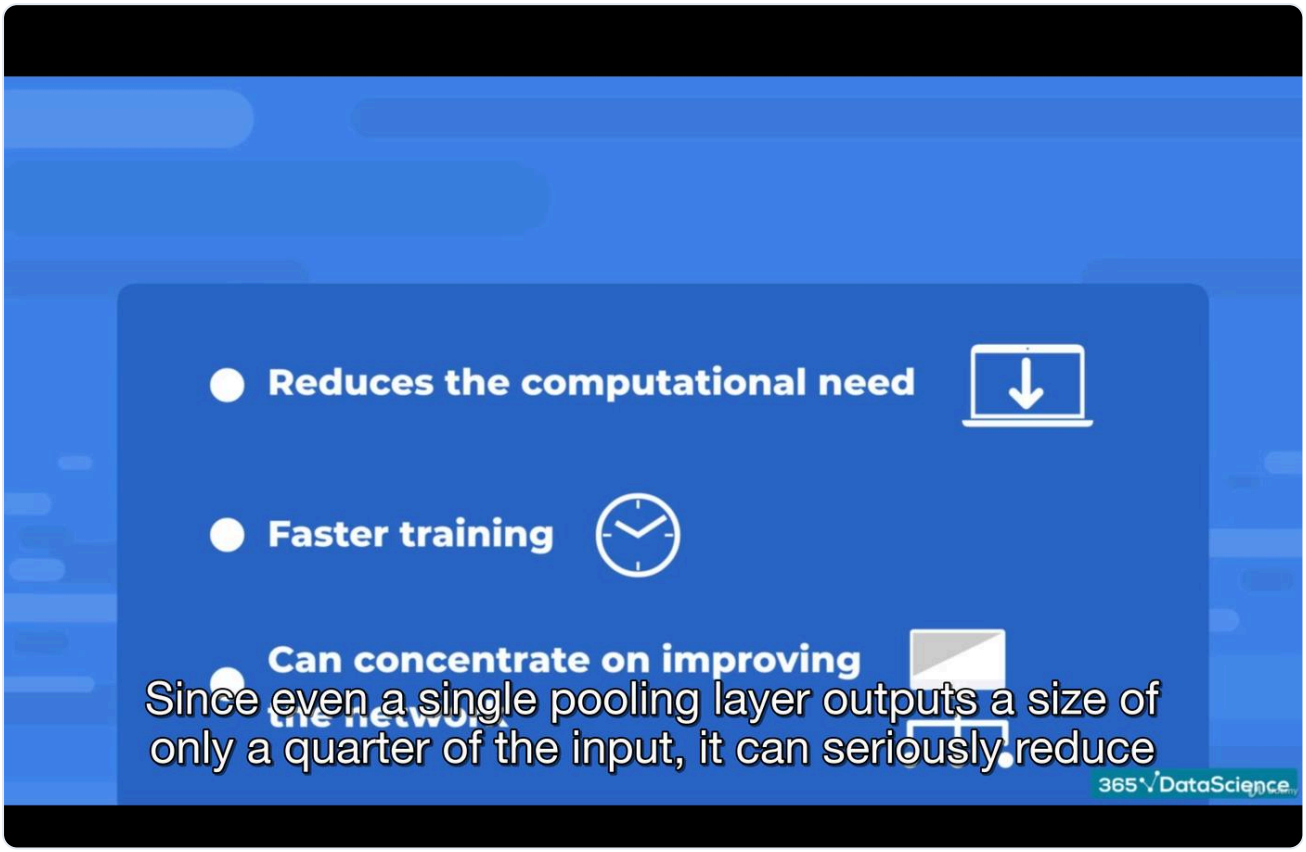

FIGURE 10.12

Since even a single pooling layer outputs a size of only a quarter of the input, it can seriously reduce the computational strain on the system. Great training the network faster is amazing, but that is not the only reason we may wish to include pooling layers.

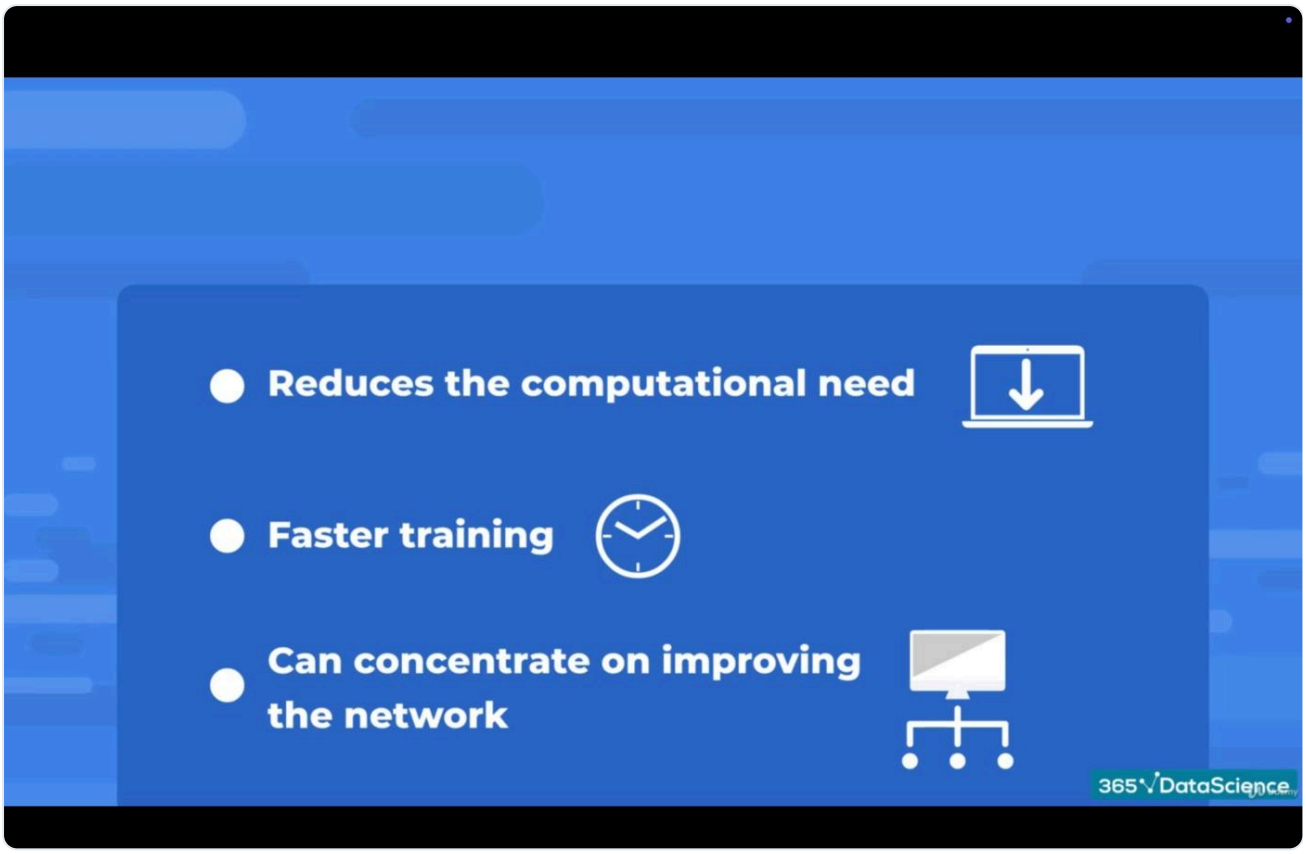

FIGURE 10.13

The second reason, arguably even more important than the first, is to prevent Overfitting. You've probably heard it a hundred times by now, but it's worth repeating.

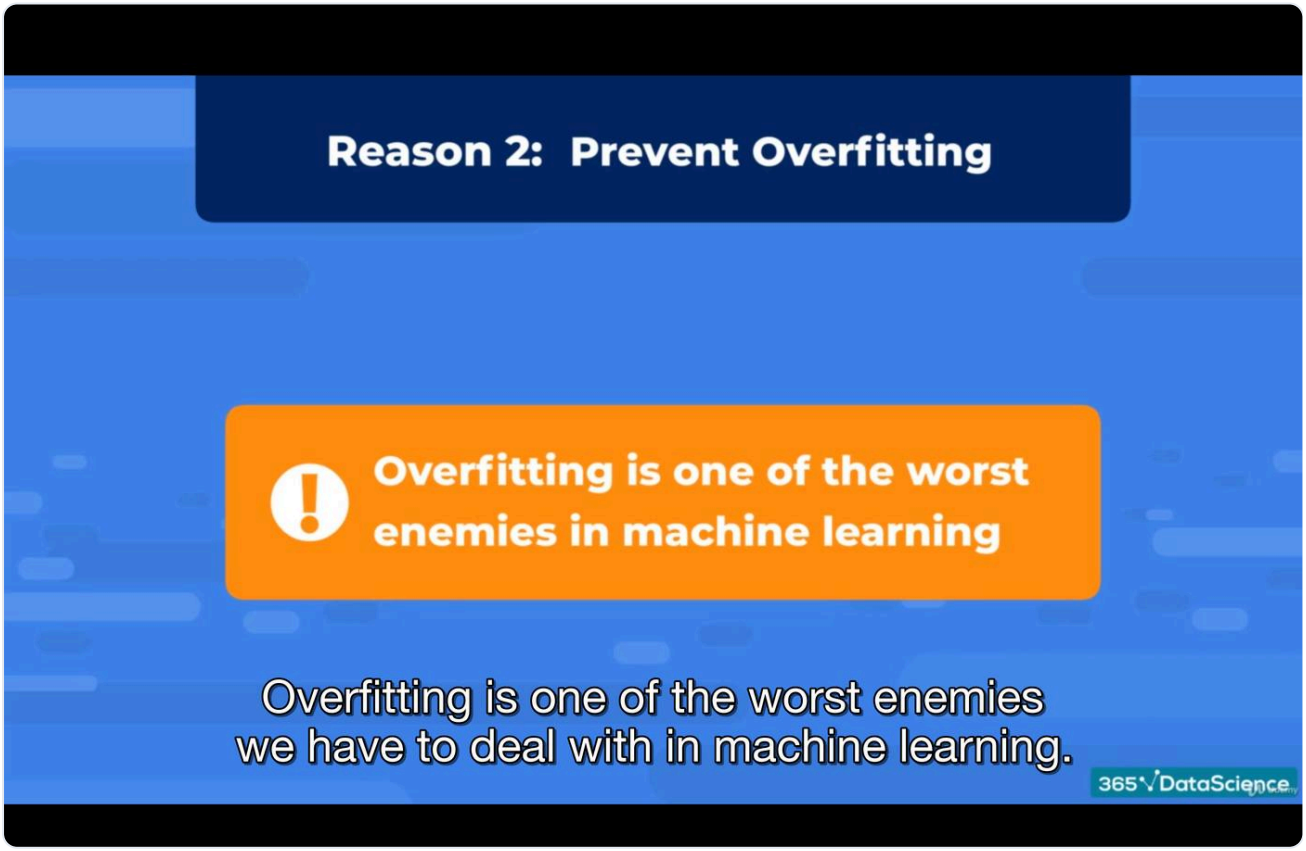

FIGURE 10.14 | Reason 2: Prevent Overfitting

Overfitting is one of the worst enemies we have to deal with in machine learning.

FIGURE 10.15

So how can pooling help us overcome it?

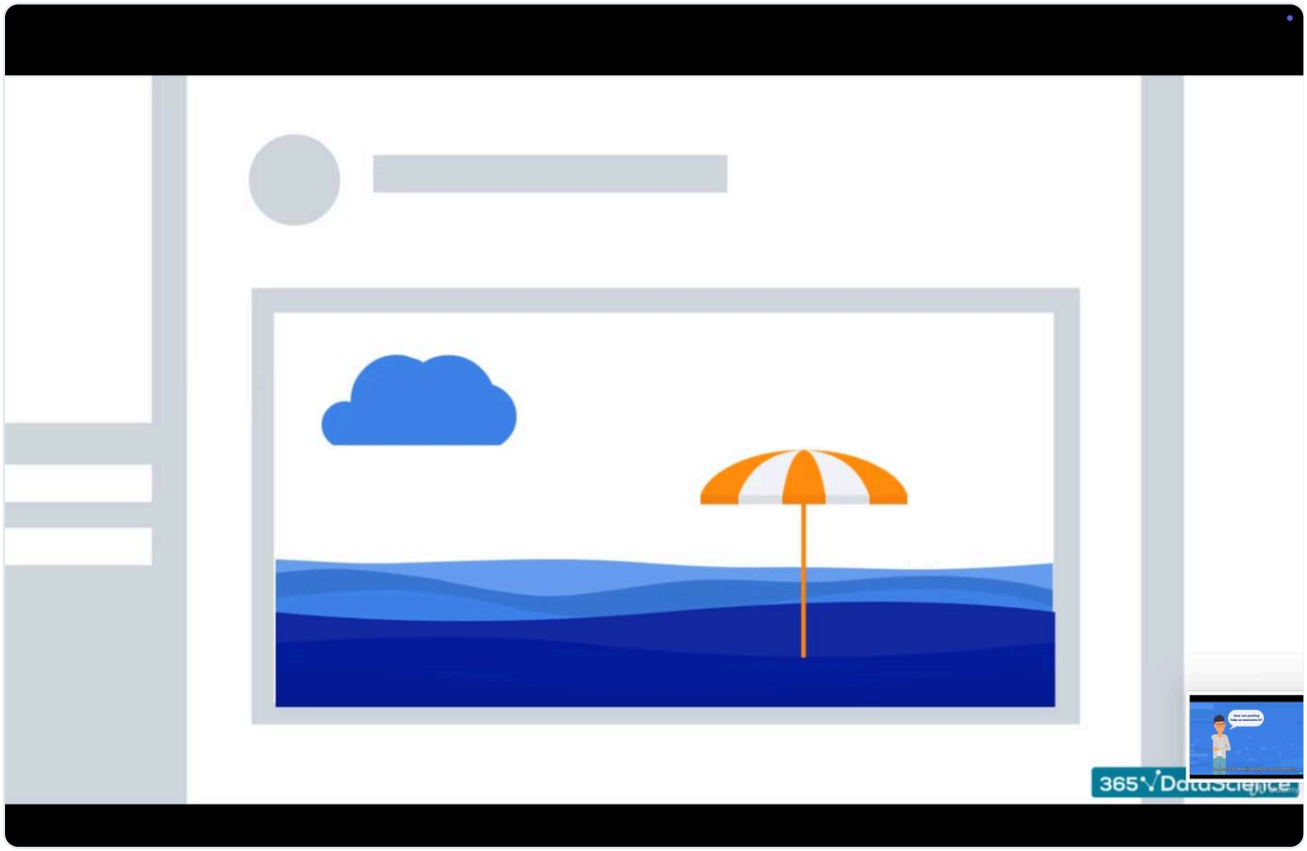

FIGURE 10.16

Well, imagine you're looking at a friend's photo from their vacation and see a cloud in the sky in the top left corner.

FIGURE 10.17

It doesn't matter to you if the cloud is that coordinates one seventy four by one fifty one or at one seventy three by 152.

FIGURE 10.18

It's just a simple cloud at the top left.

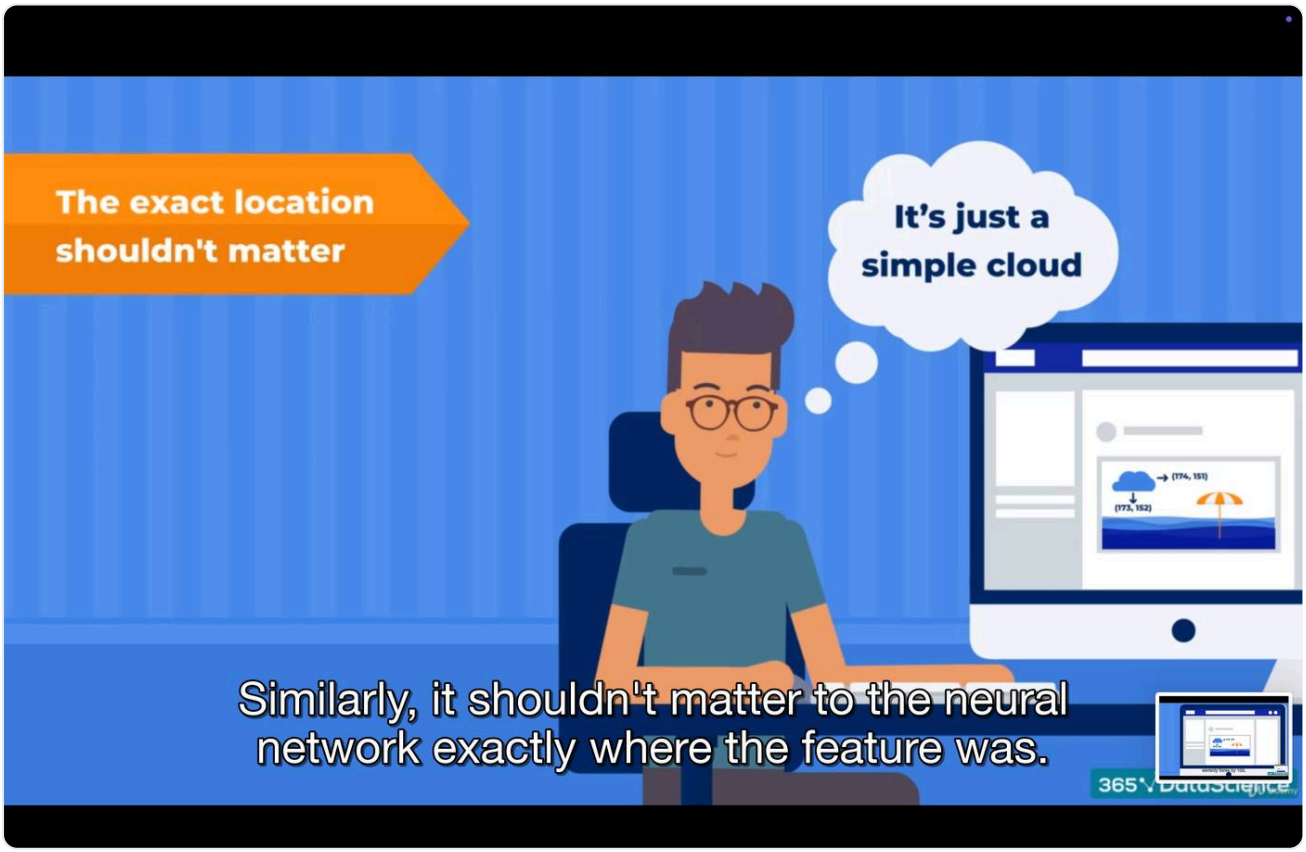

FIGURE 10.19

Similarly, it shouldn't matter to the neural network exactly where the feature was. It just needs to know the rough area. However, when performing the convolution, we generally get a feature map with the same precision as the original image.

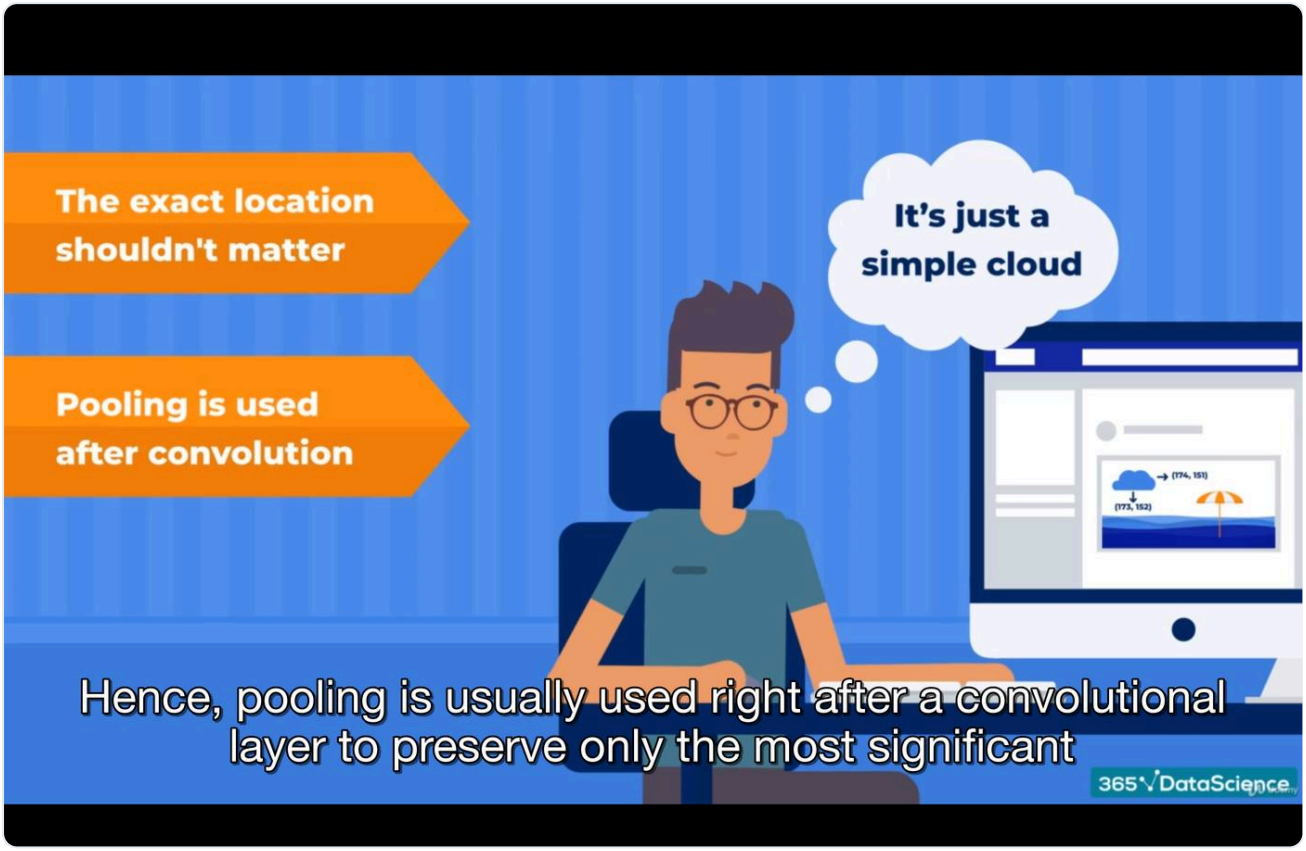

FIGURE 10.20

FIGURE 10.21

Hence, pooling is usually used right after a convolutional layer to preserve only the most significant features and to make them less dependent on their precise positioning. A pooling layer effectively erases small differences in positioning. In layman's terms, this whole procedure smears out the noise.

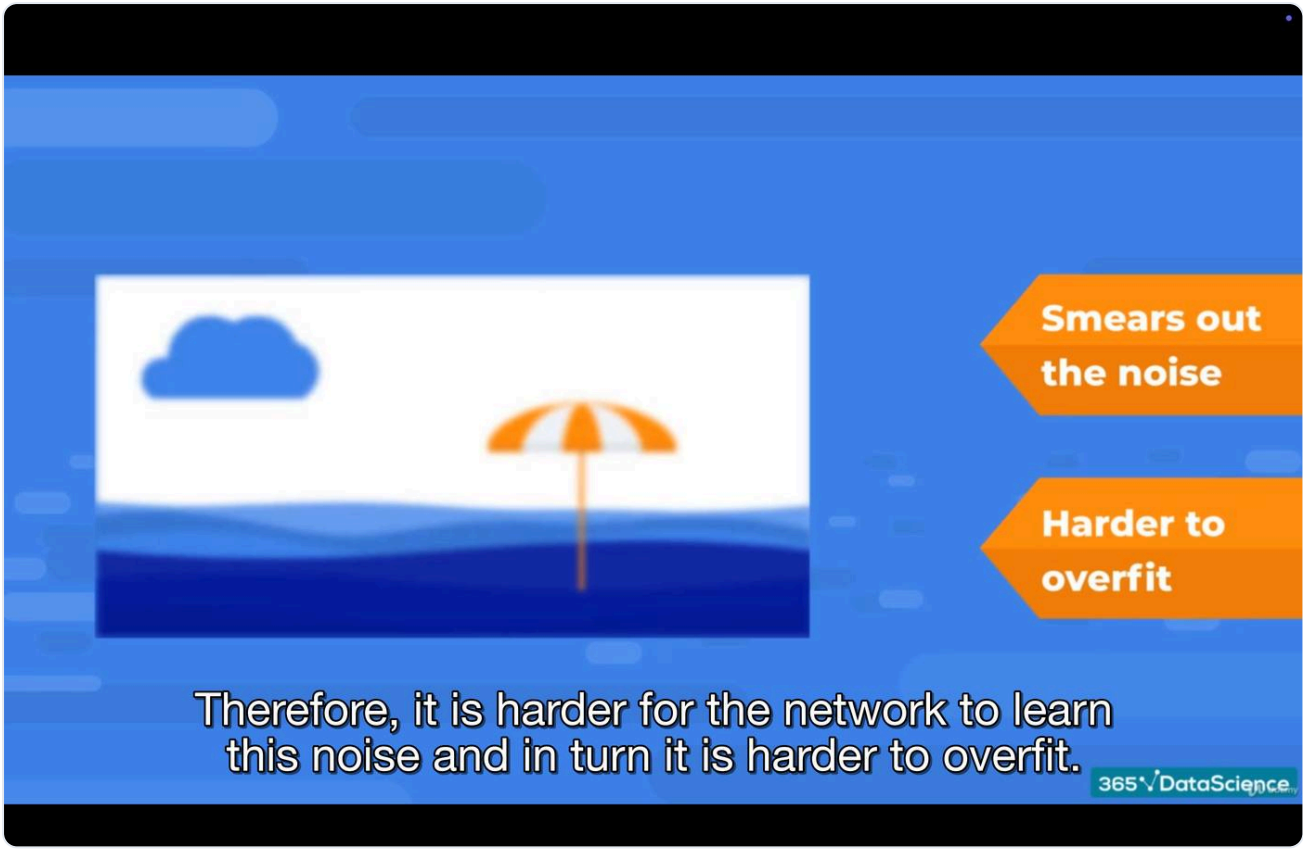

FIGURE 10.22 | Smears Out

Therefore, it is harder for the network to learn this noise and in turn it is harder to overfit. So far, so good. Now let's talk about the types of pooling in stride. In general, a pooling layer can use any kernel and function that produces a single number.

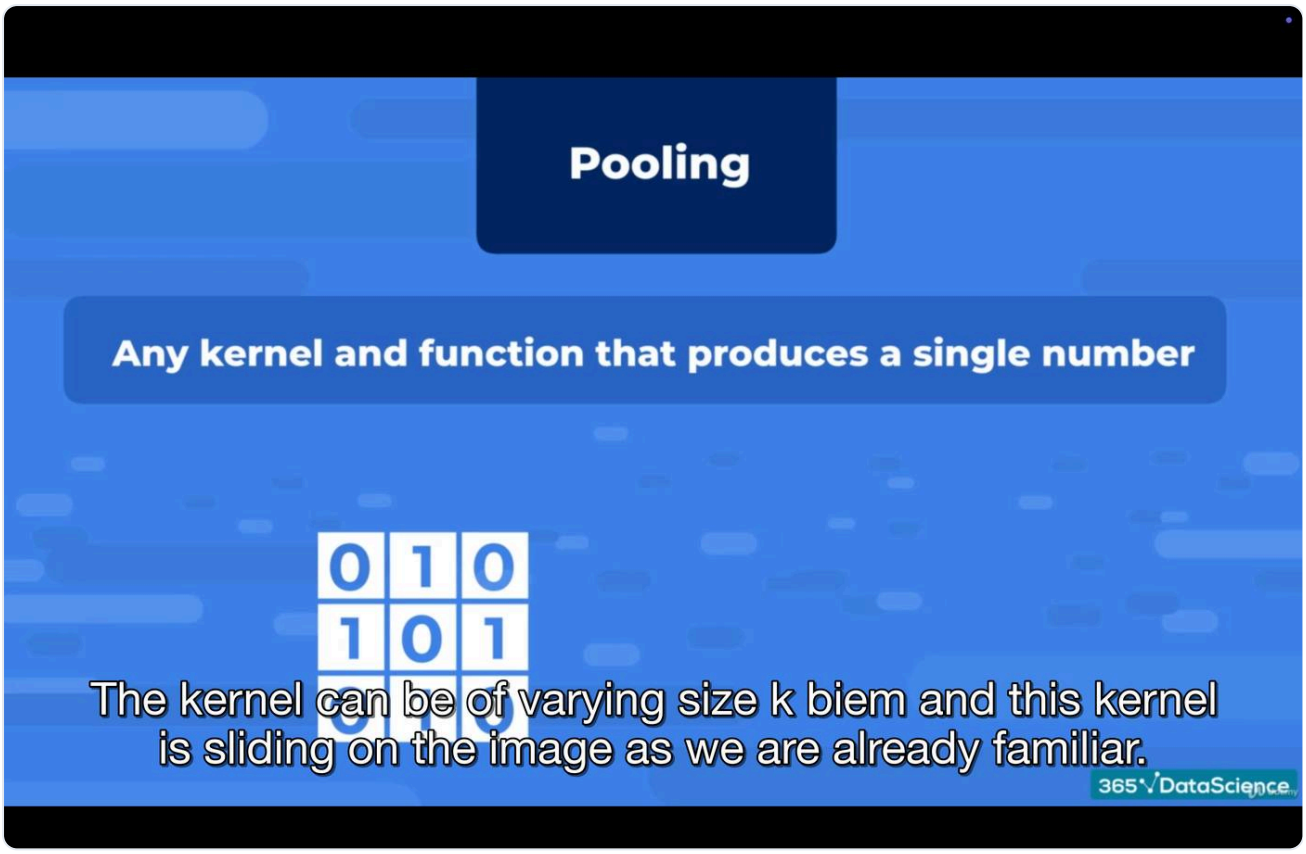

FIGURE 10.23

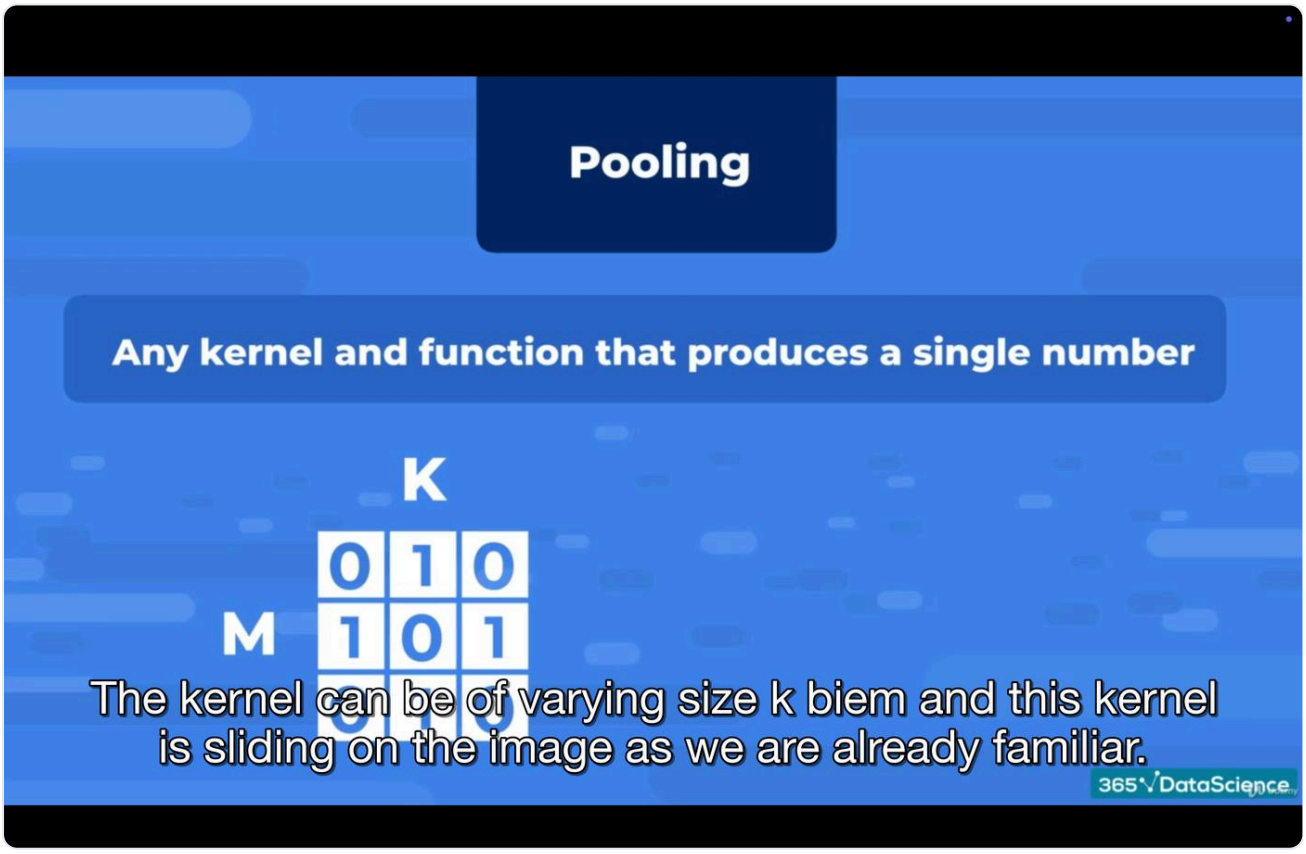

FIGURE 10.24

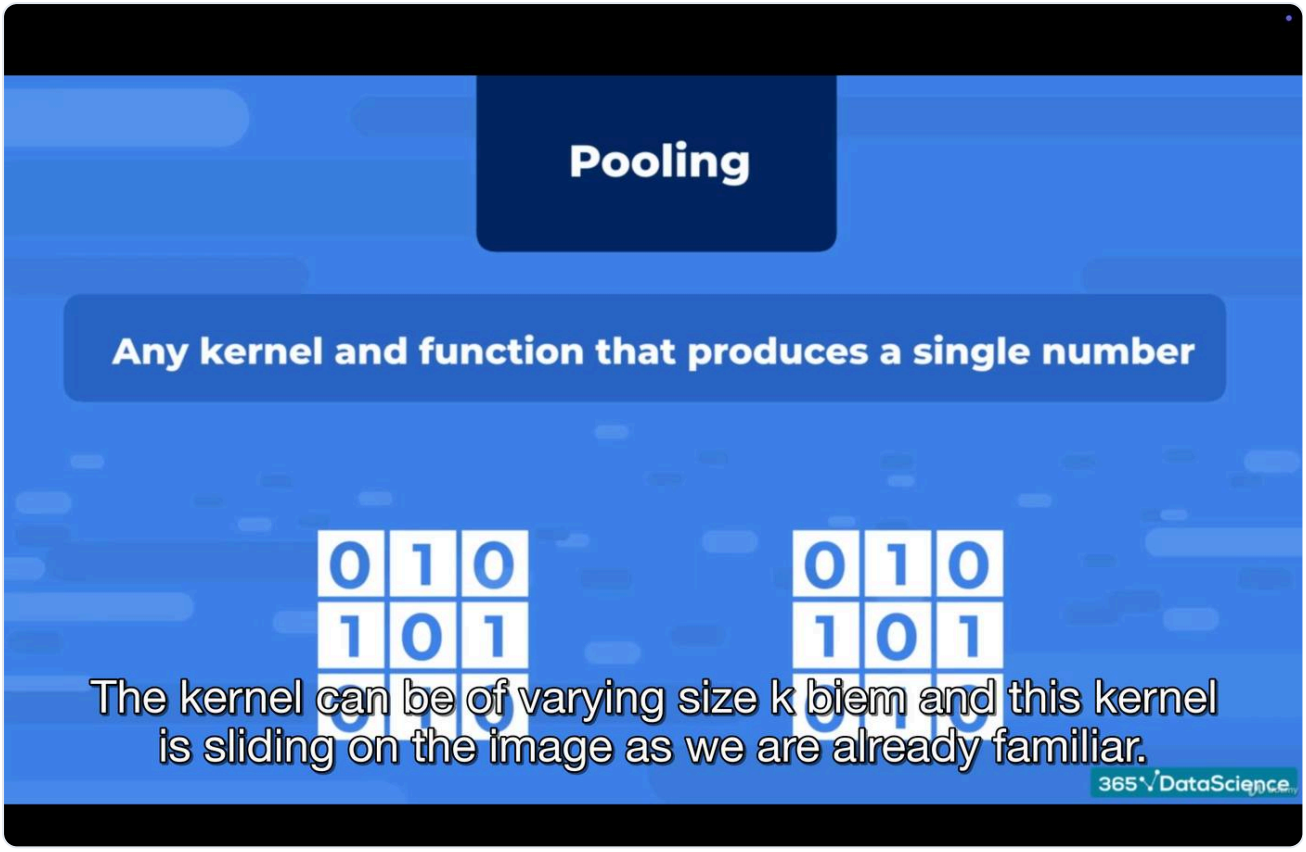

FIGURE 10.25

The kernel can be of varying size k by m and this kernel is sliding on the image as we are already familiar. However, usually.

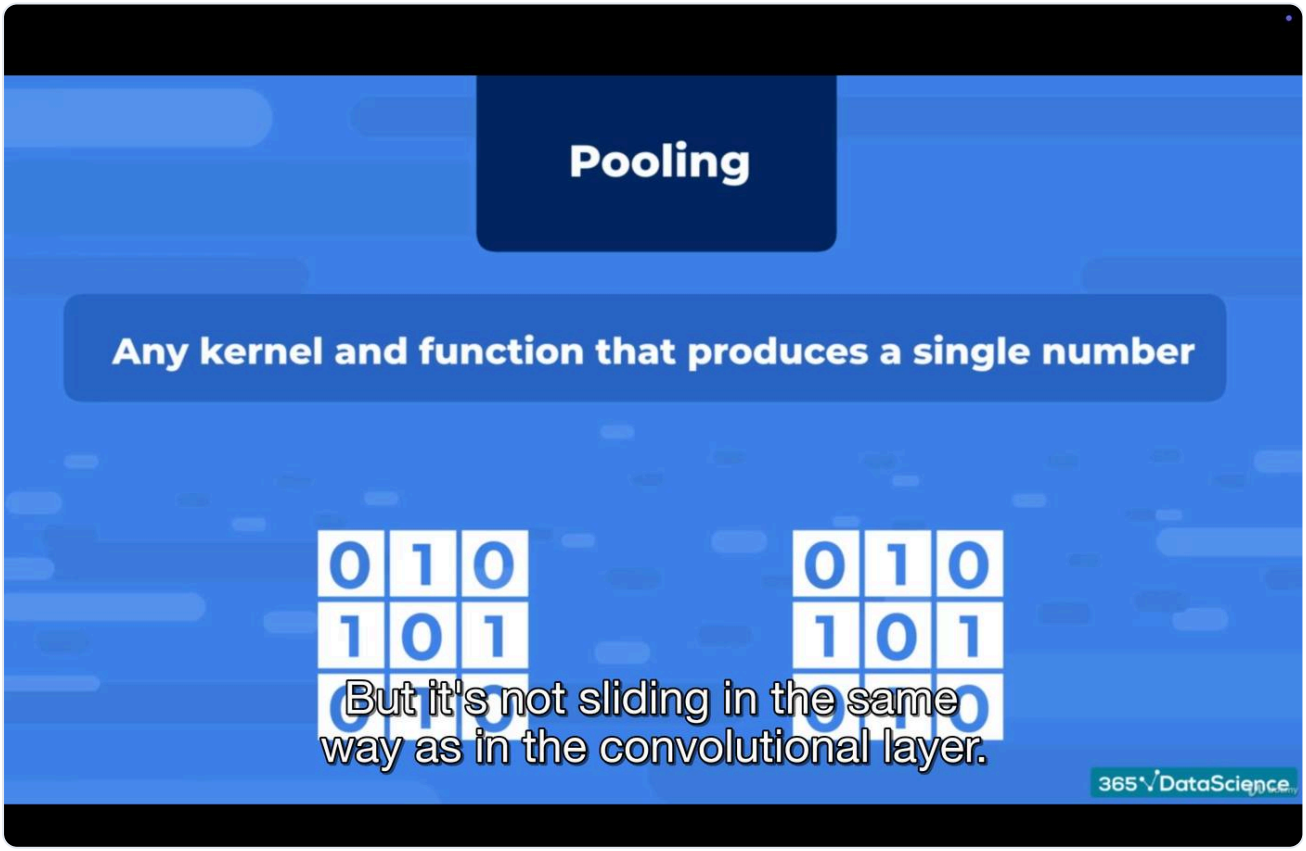

FIGURE 10.26

But it's not sliding in the same way as in the convolutional layer.

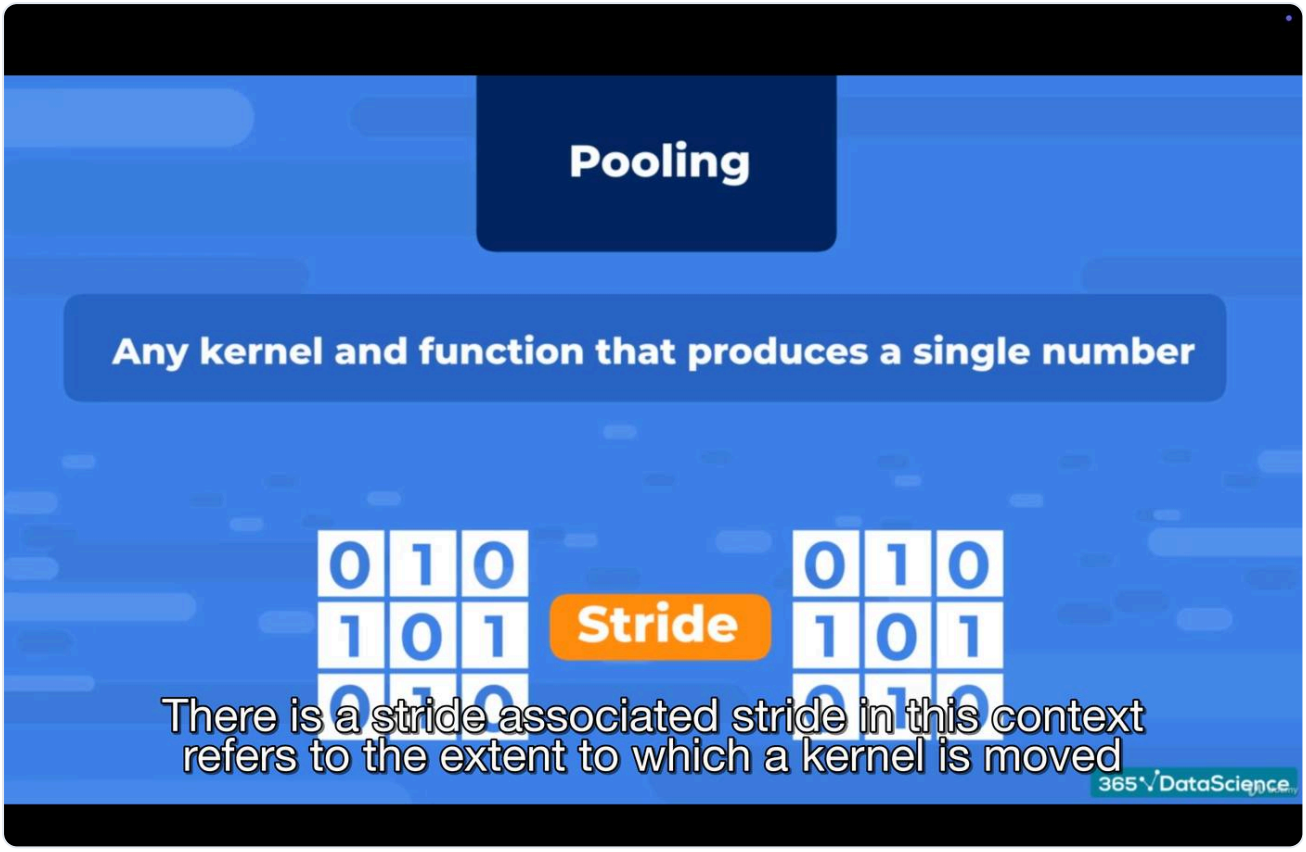

FIGURE 10.27

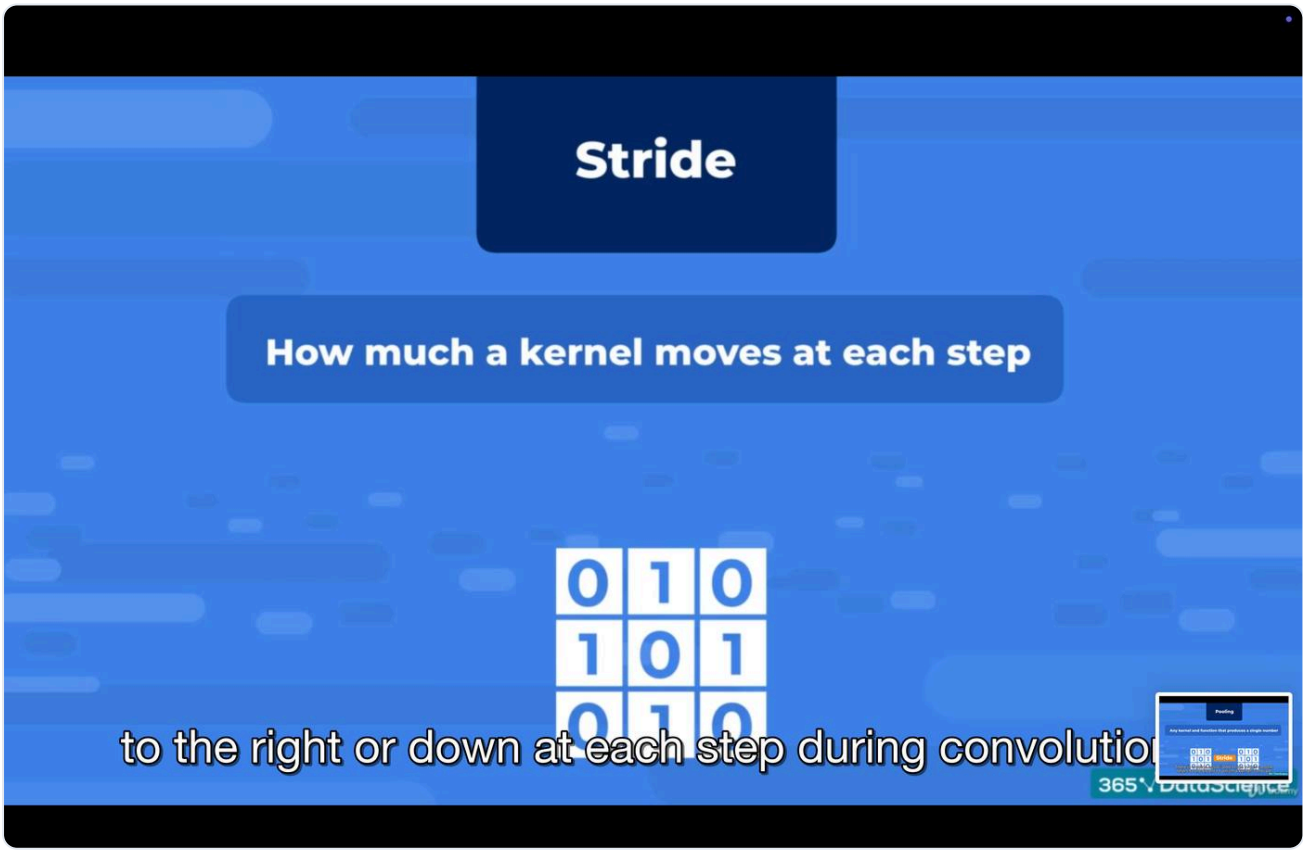

FIGURE 10.28

There is a stride associated stride in this context refers to the extent to which a kernel is moved to the right or down at each step during convolution. We have only dealt with stride.

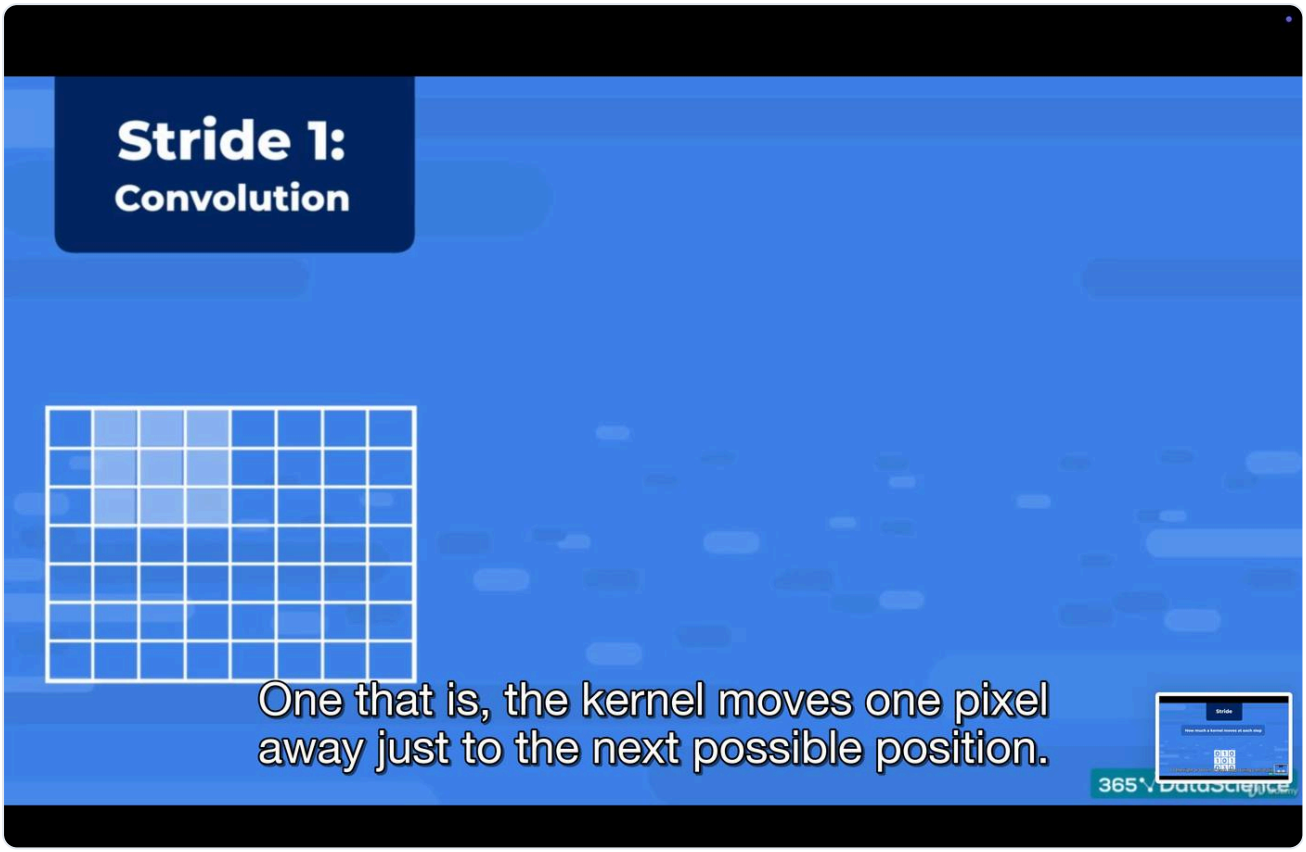

FIGURE 10.29

One that is, the kernel moves one pixel away just to the next possible position. However, other values for the stride are possible.

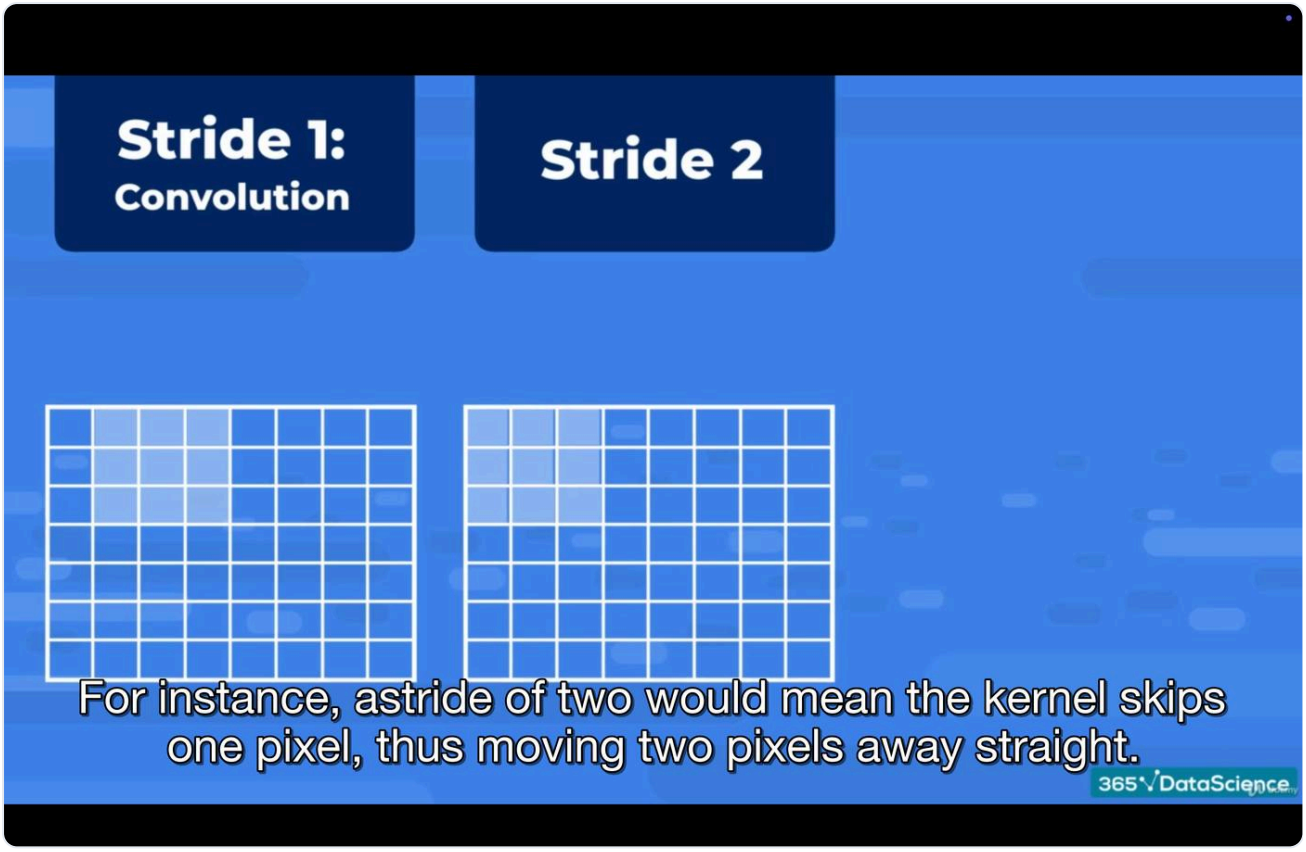

FIGURE 10.30

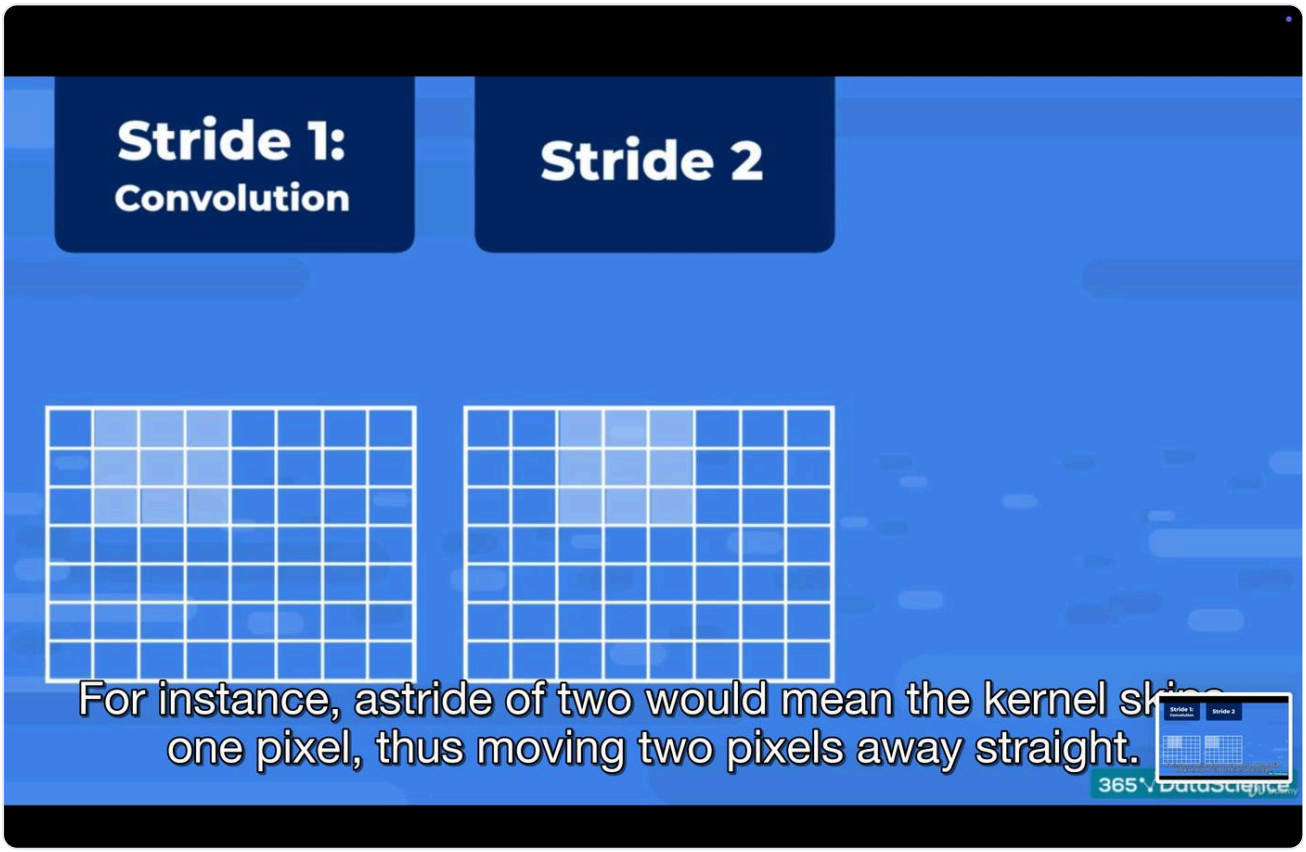

FIGURE 10.31

For instance, astride of two would mean the kernel skips one pixel, thus moving two pixels away straight.

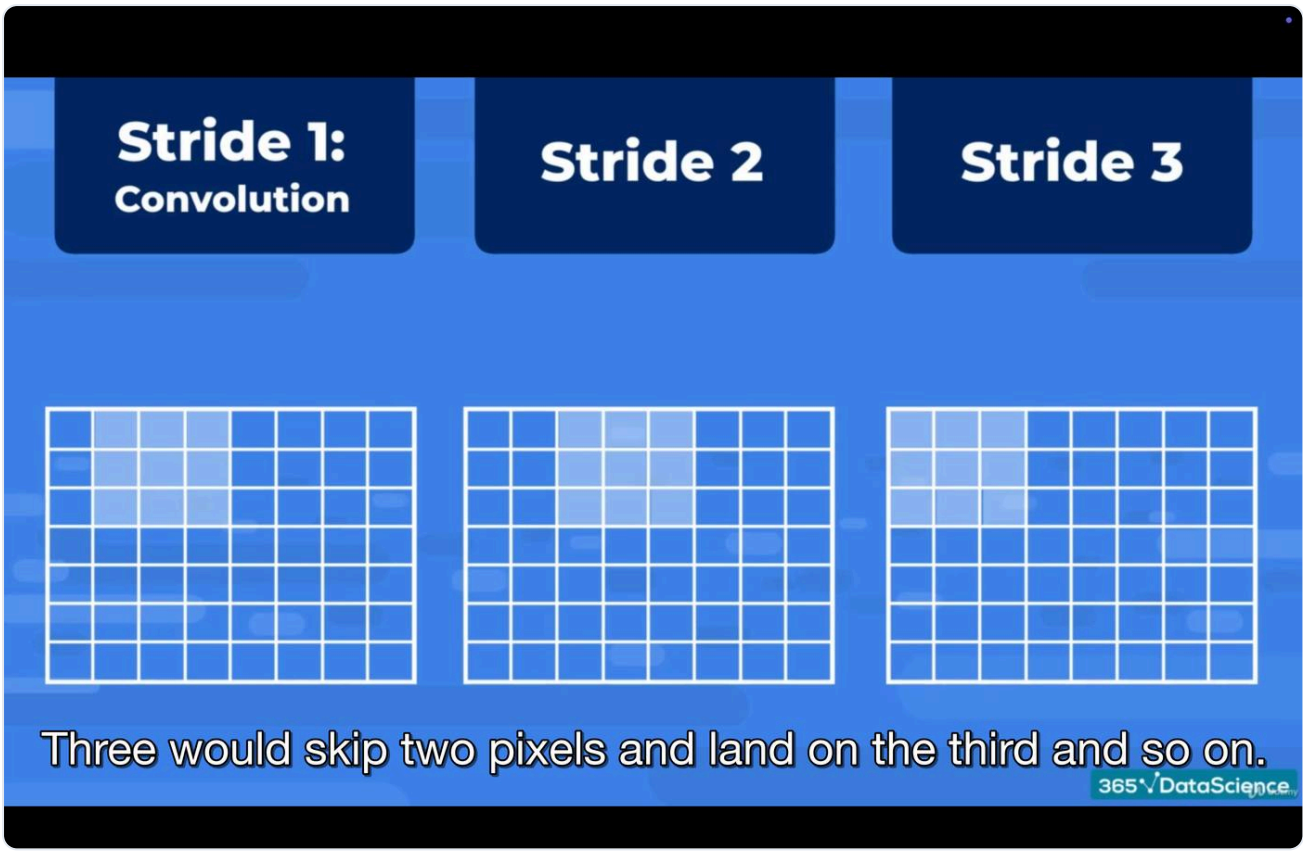

FIGURE 10.32

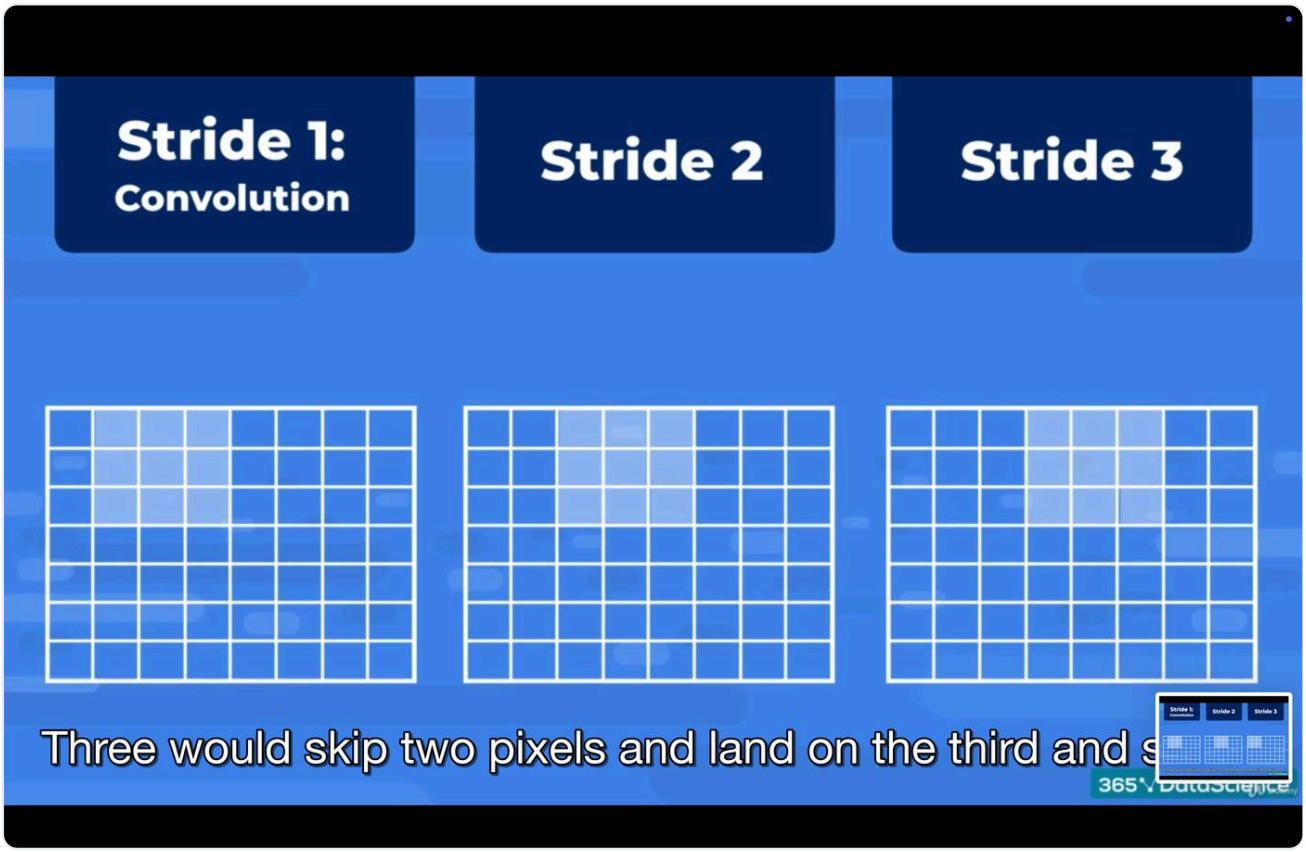

FIGURE 10.33

Three would skip two pixels and land on the third and so on. By changing the stride, you can make your pooling kernels overlap, just touch or have gaps in between.

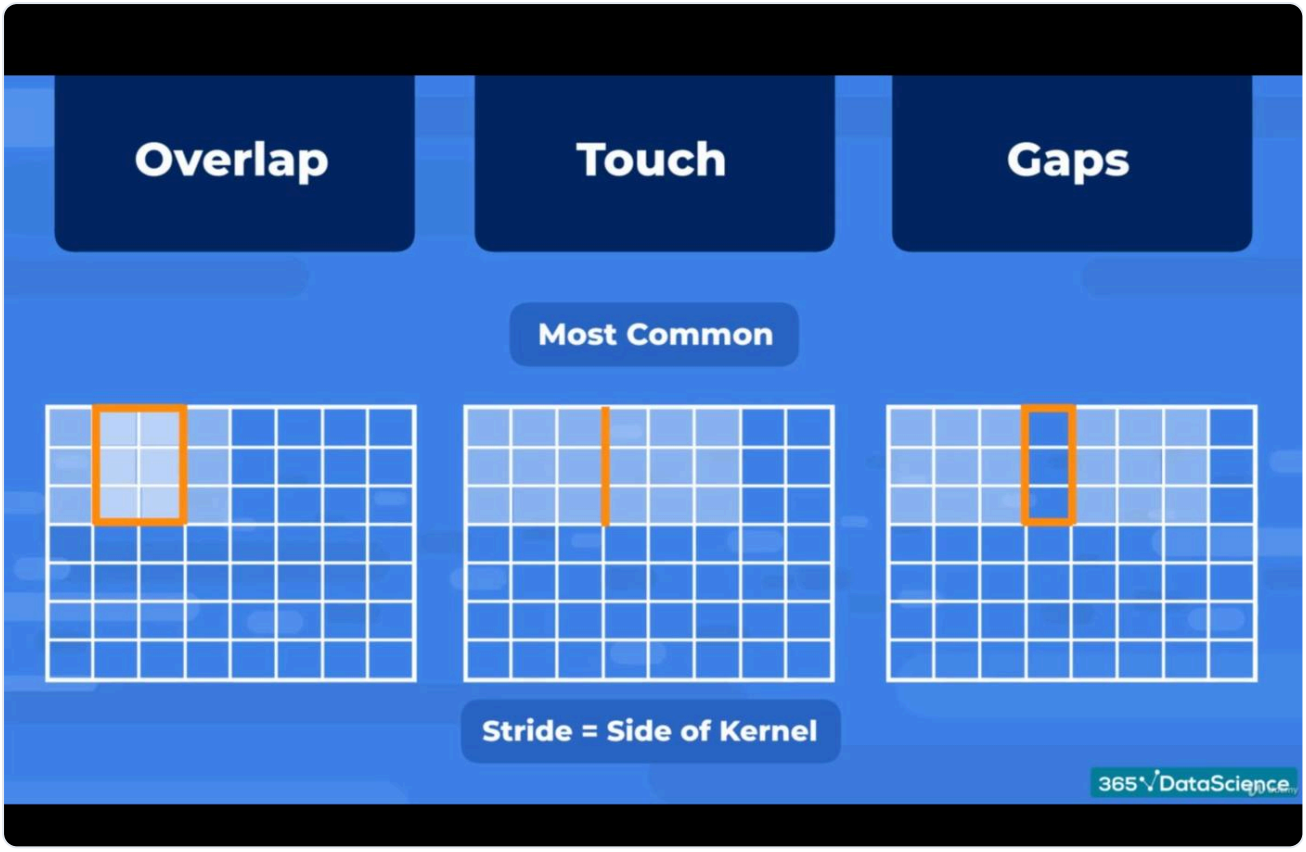

FIGURE 10.34

The most common practice is to have the stride equal the side of the kernel. In that way, you get full coverage of the inputs without overlap. That is exactly what, two by two max pool with stride two means.

FIGURE 10.35

First, the kernel size is two by two. Then we have the stride, which is again two. That means that the next kernel position will just touch the previous.

FIGURE 10.36

This is certainly the most logical way to pull pixels together, but it's worth mentioning that it is not the only one. Finally, the word max in the name indicates the function we use on the kernel.

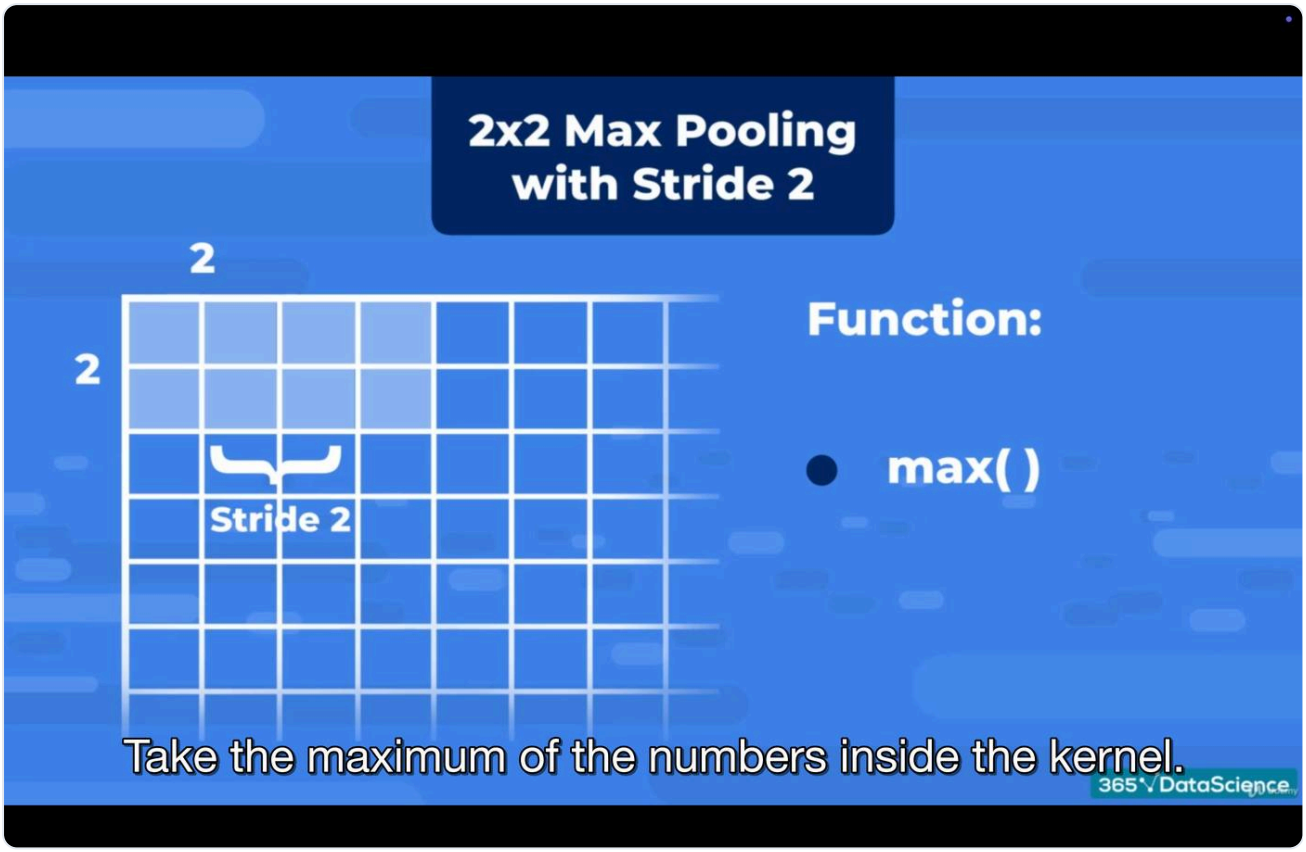

FIGURE 10.37

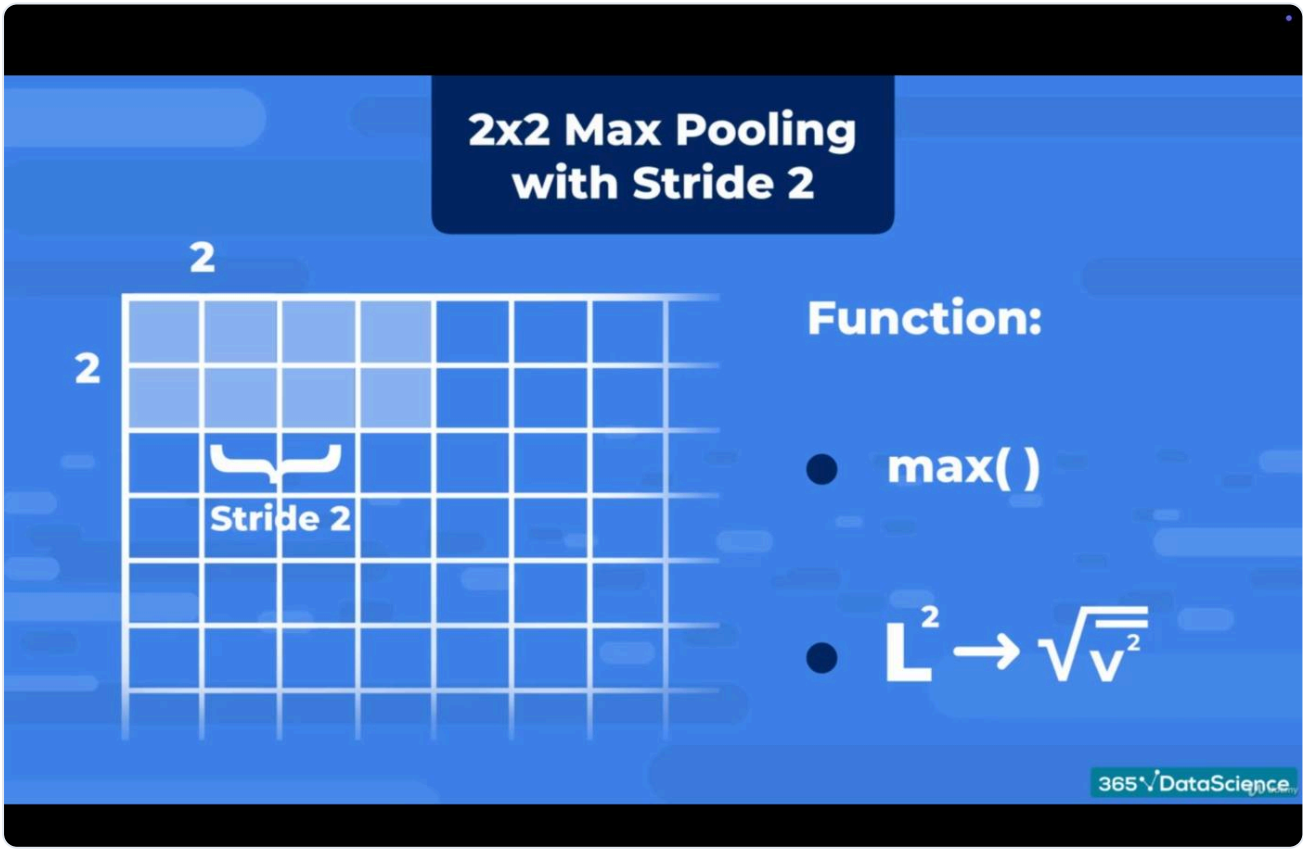

FIGURE 10.38

Take the maximum of the numbers inside the kernel. Of course, we can change the function as well. L2 norm pooling is another possibility. Where you take the square root of the average square values in the region, still, Max pooling remains the most popular choice and probably the only one you would need to know. Great. That is all of the basics of convolutional neural networks, in the next lecture, we will spend some time on the dimensionality of all inputs and outputs. Thanks for watching.

LECTURE 11

Input Convolutional Layer

Welcome back. While discussing kernels convolution pooling stride and colours, we cannot help but talk about dimensions, in fact, the dimensions of the various layers in a network have the biggest impact on training, time and memory.

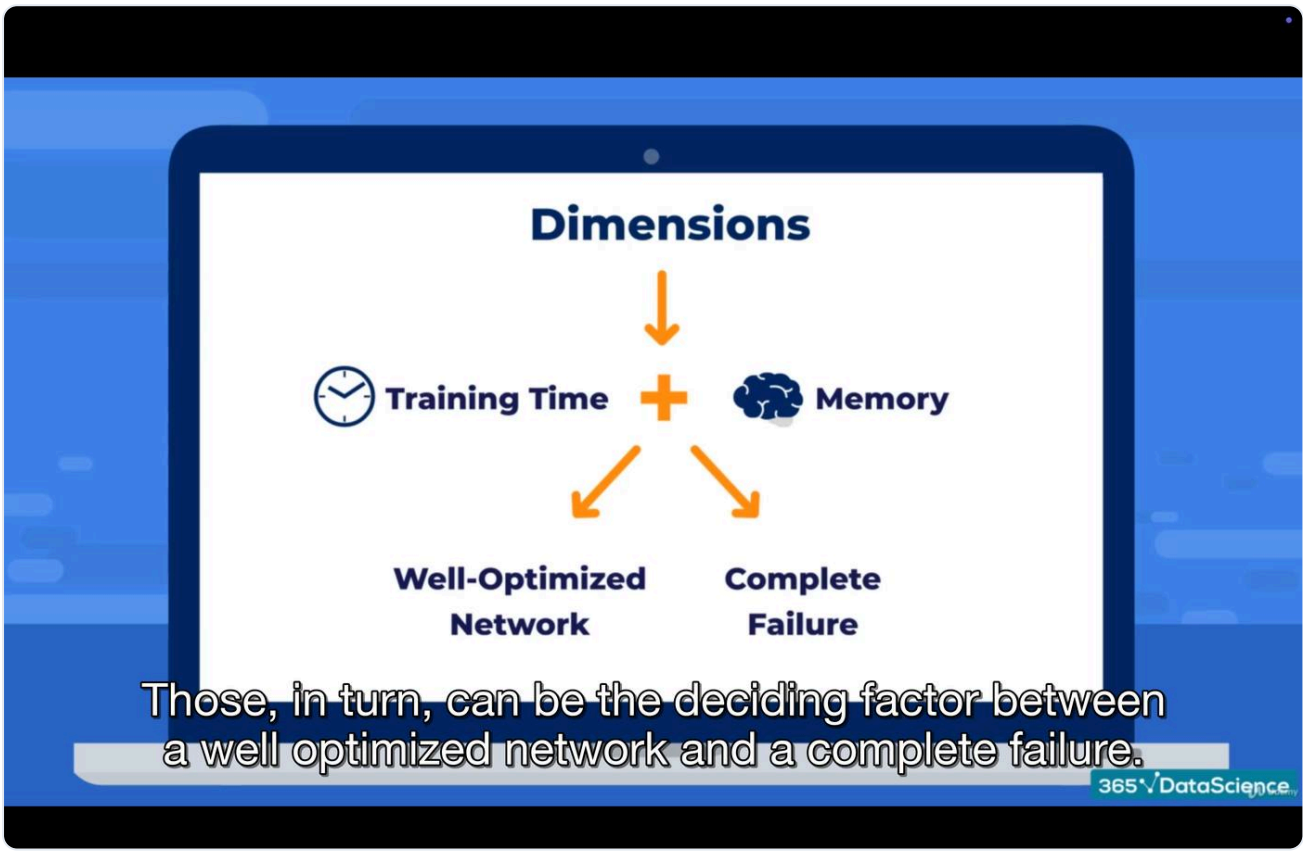

FIGURE 11.1

Those, in turn, can be the deciding factor between a well optimized network and a complete failure.

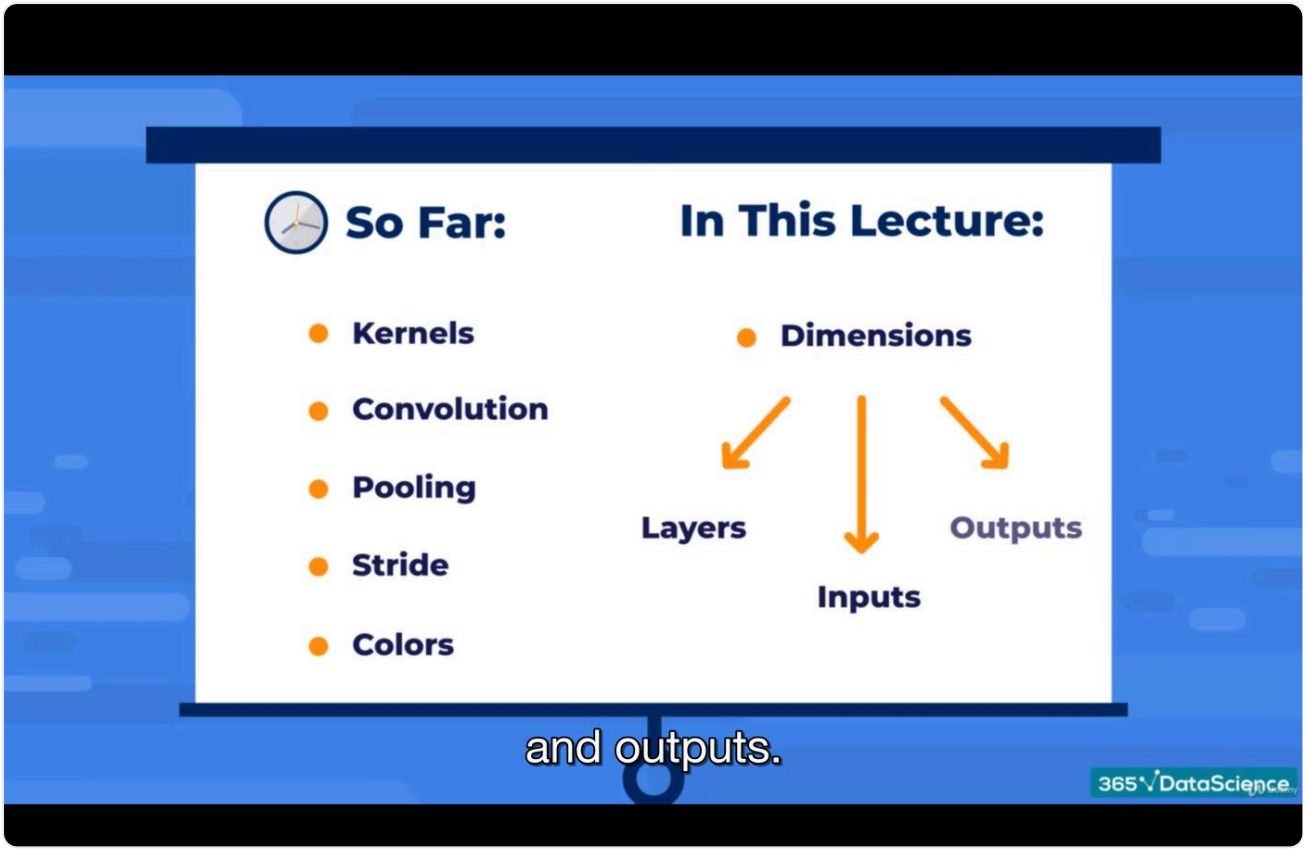

FIGURE 11.2

That is why in this lecture, we will take a quick look at the dimensions of the various layers, inputs and outputs.

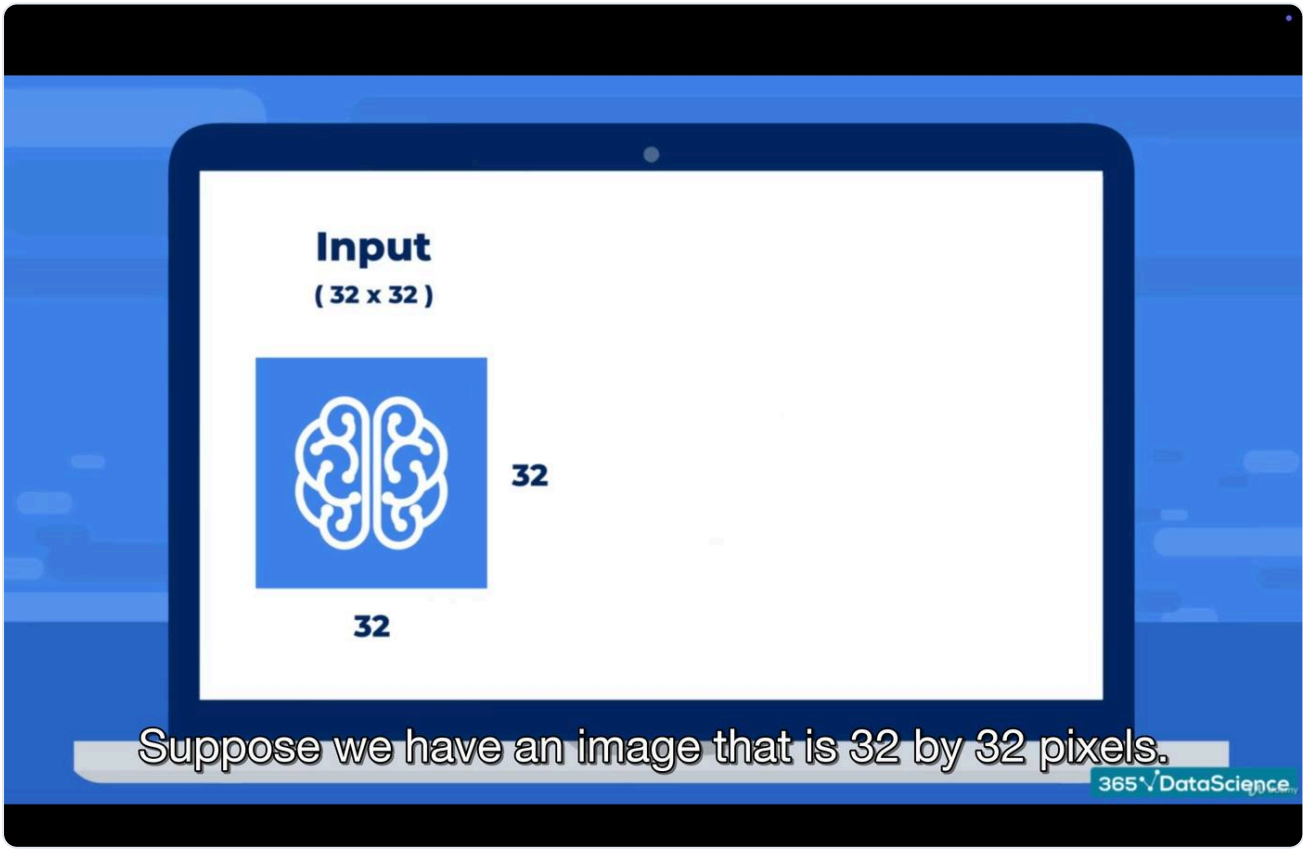

FIGURE 11.3

Suppose we have an image that is 32 by 32 pixels. If it is a colored image, its actual dimensions are 32 by 32 by three.

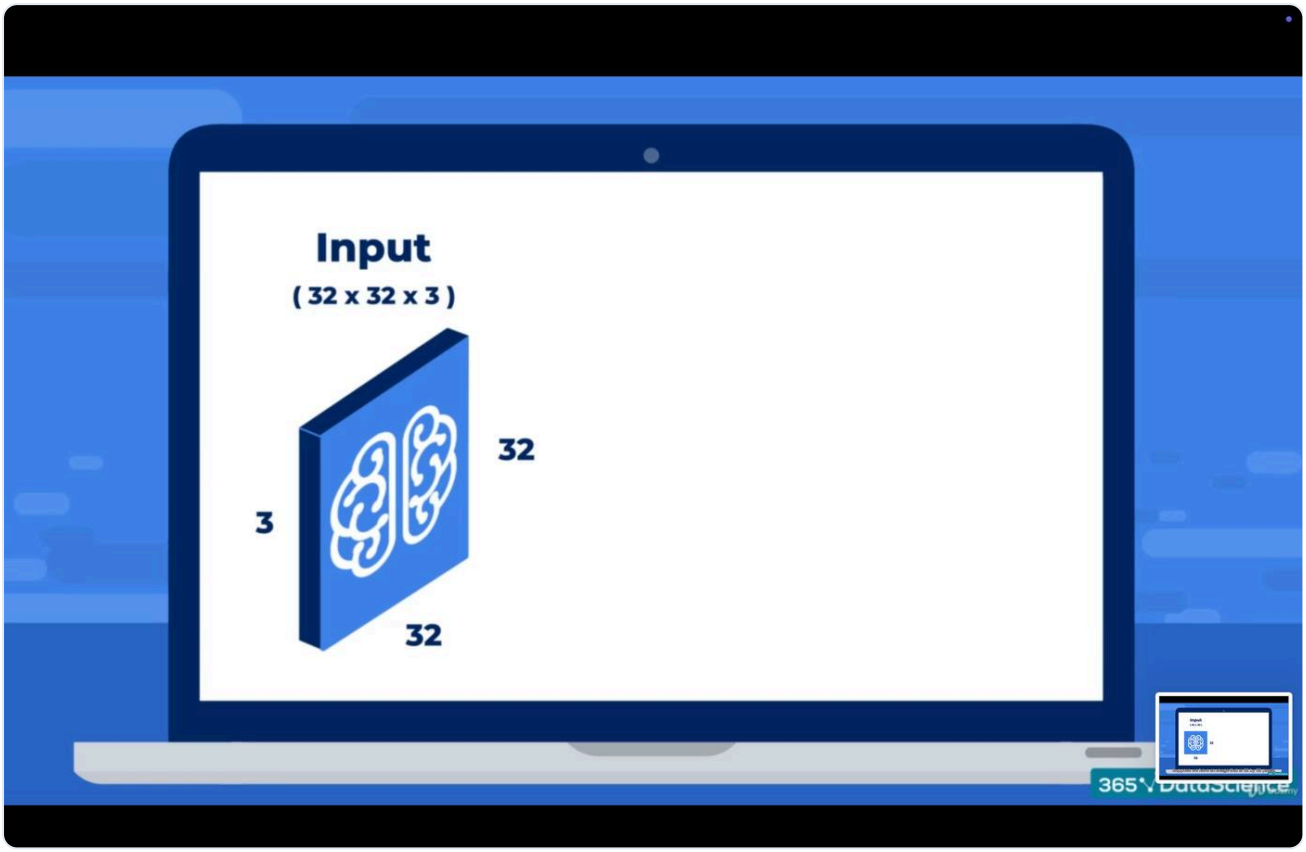

FIGURE 11.4

So we have this image as an input to a convolutional layer.

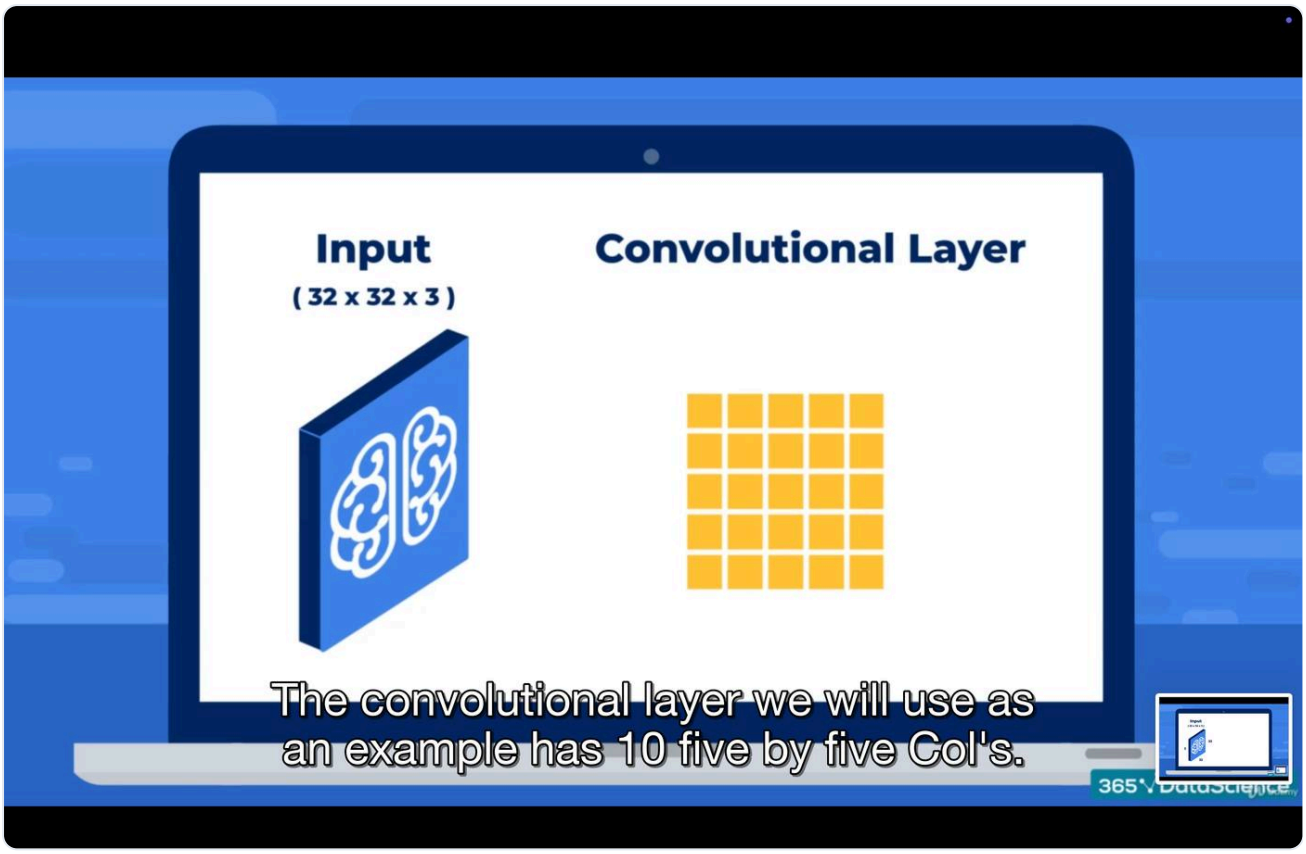

FIGURE 11.5 | Input Convolutional Layer

The convolutional layer we will use as an example has 10 five by five kernels.

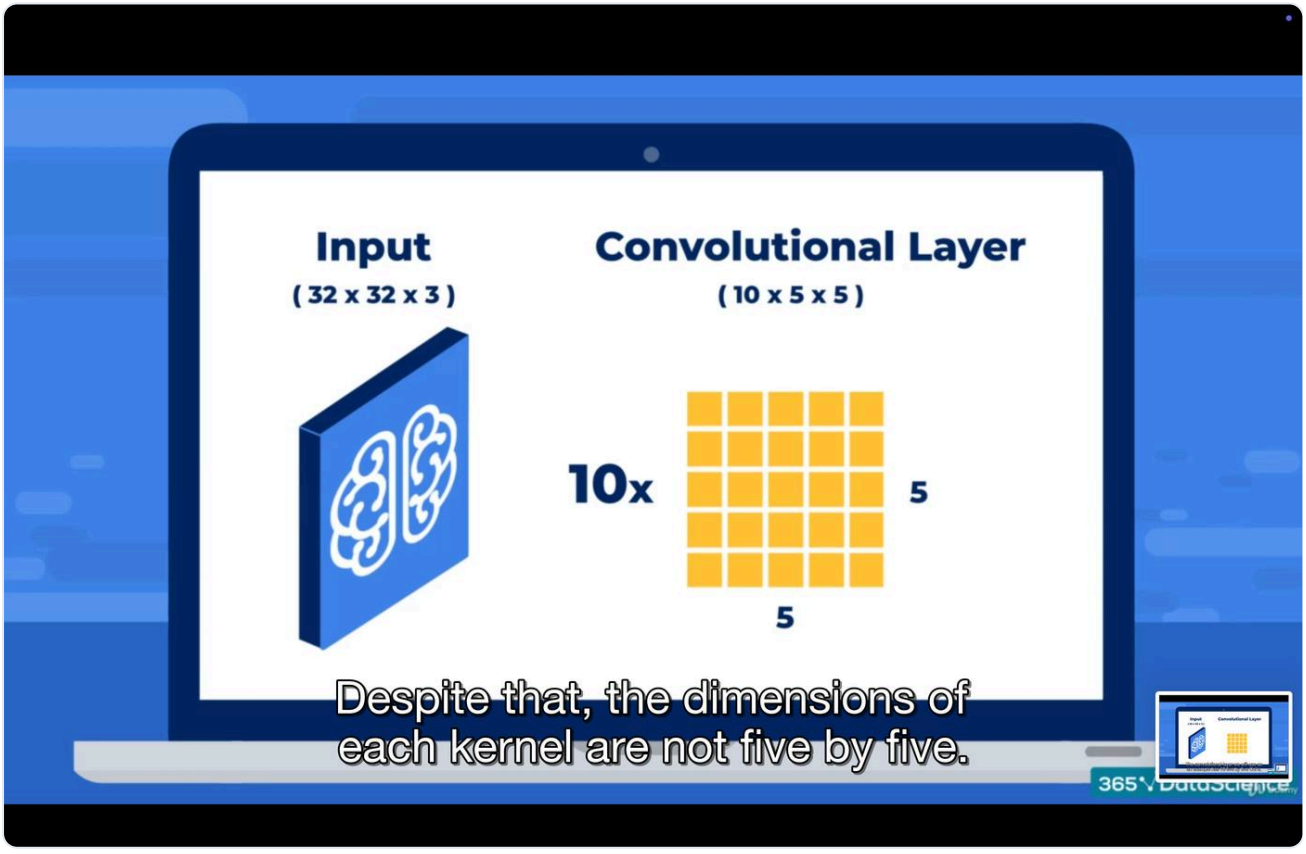

FIGURE 11.6 | Input Convolutional Layer

Despite that, the dimensions of each kernel are not five by five. They are actually five by five by three because of the color of the image.

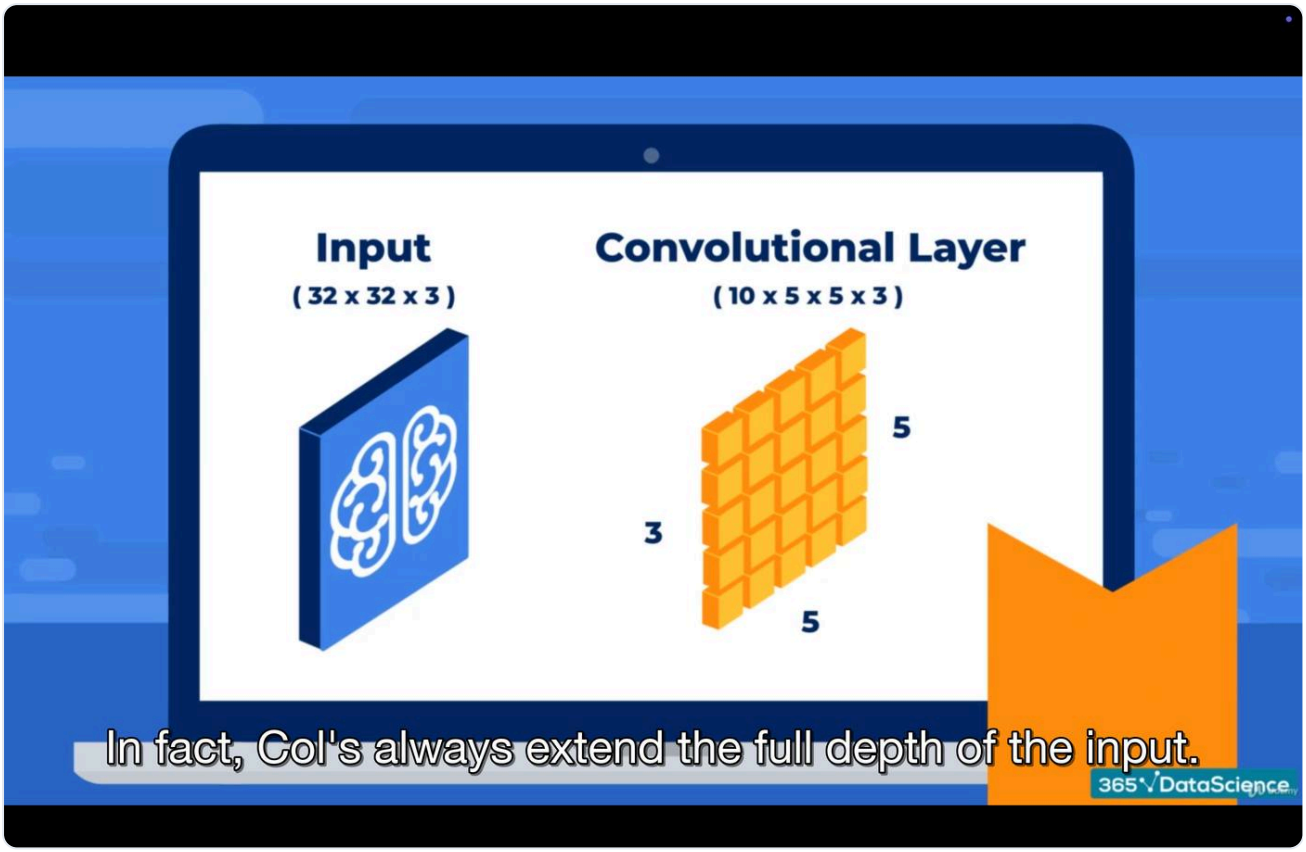

FIGURE 11.7 | Input Convolutional Layer

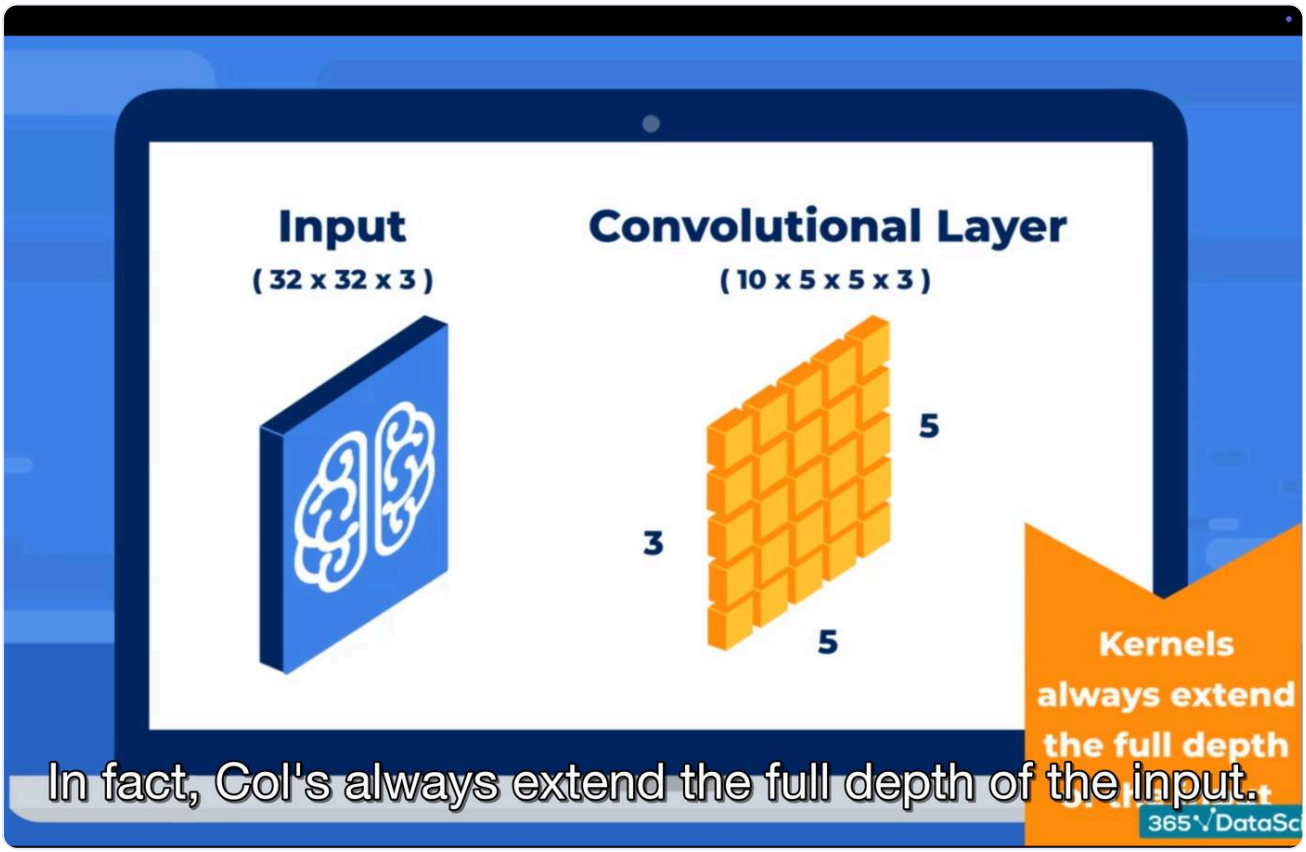

FIGURE 11.8 | Input Convolutional Layer

In fact, kernels always extend the full depth of the input.

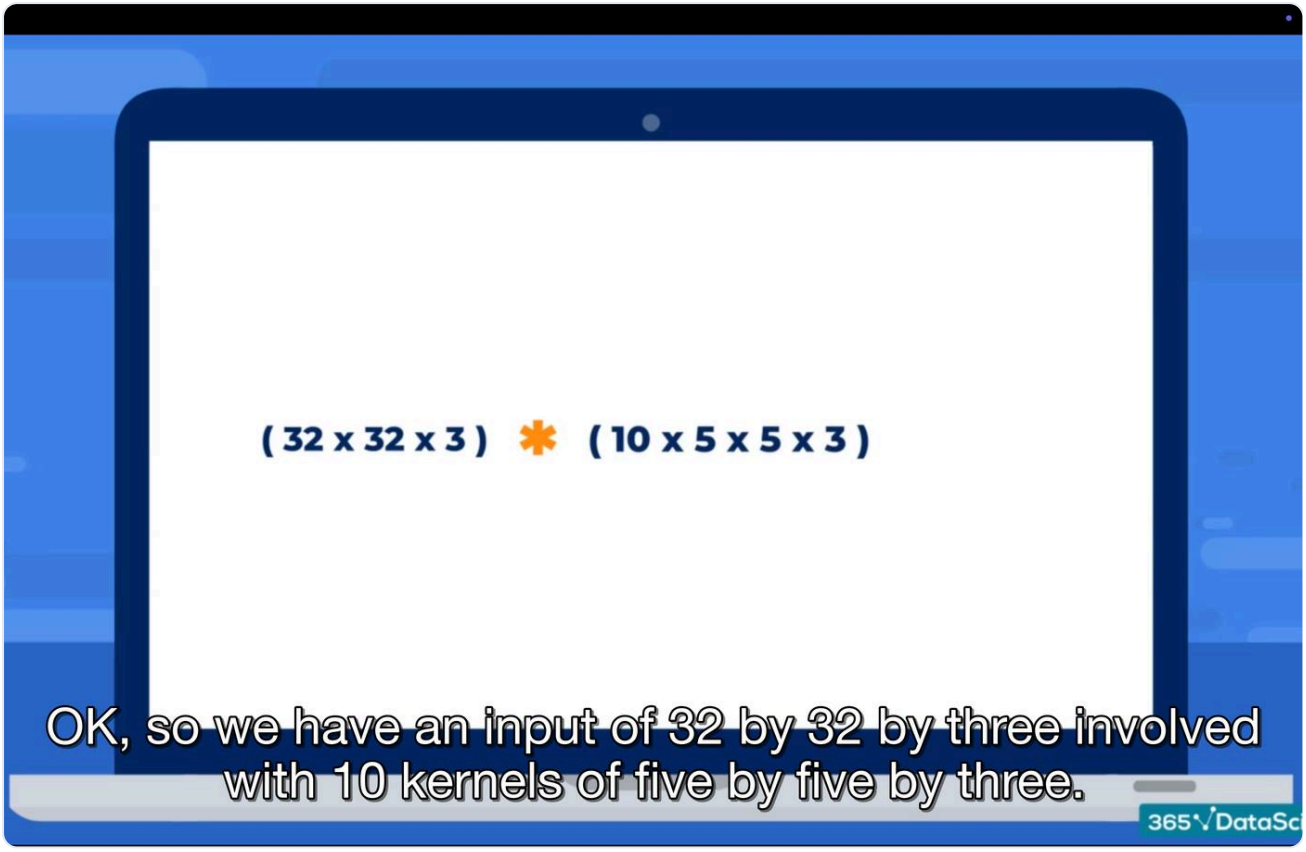

FIGURE 11.9

OK, so we have an input of 32 by 32 by three involved with 10 kernels of five by five by three. What is the result of this?

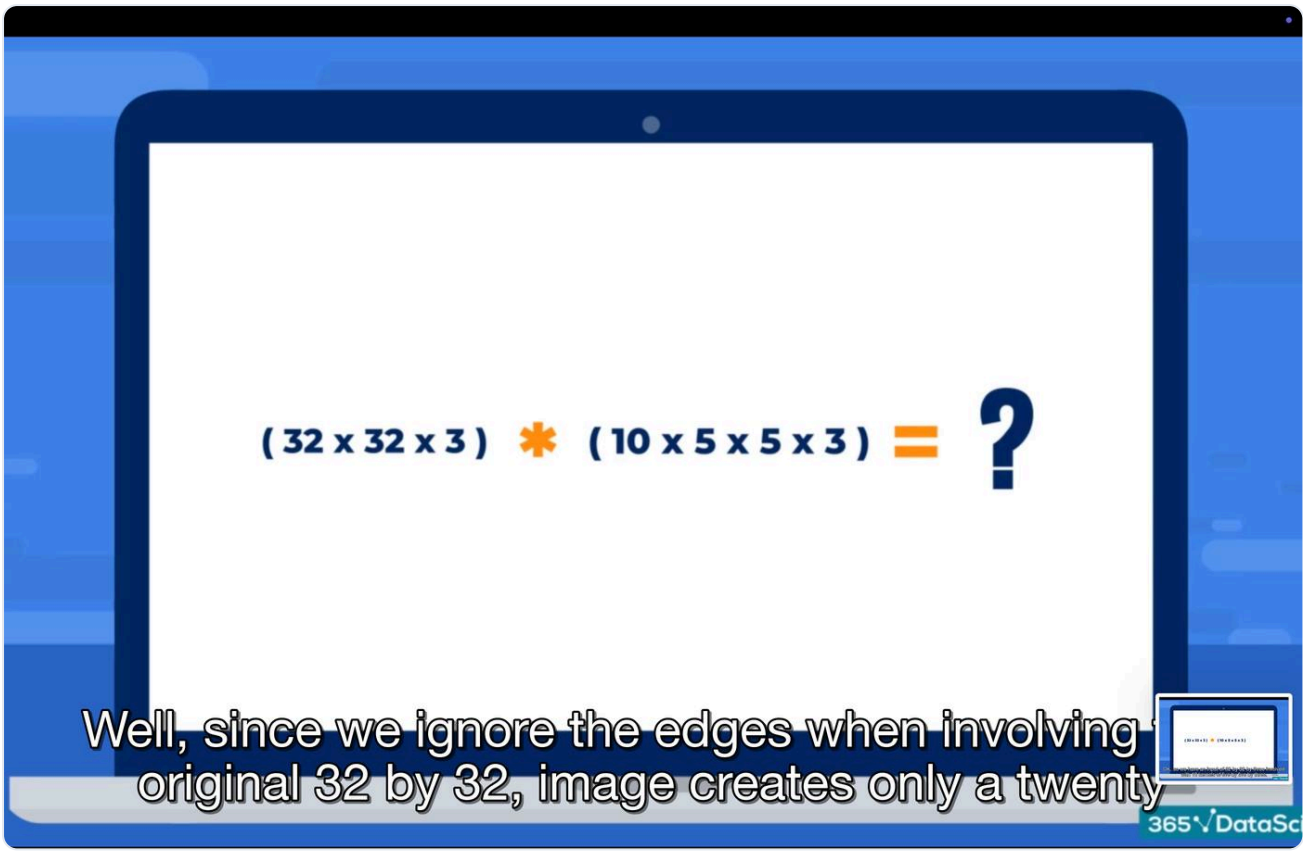

FIGURE 11.10

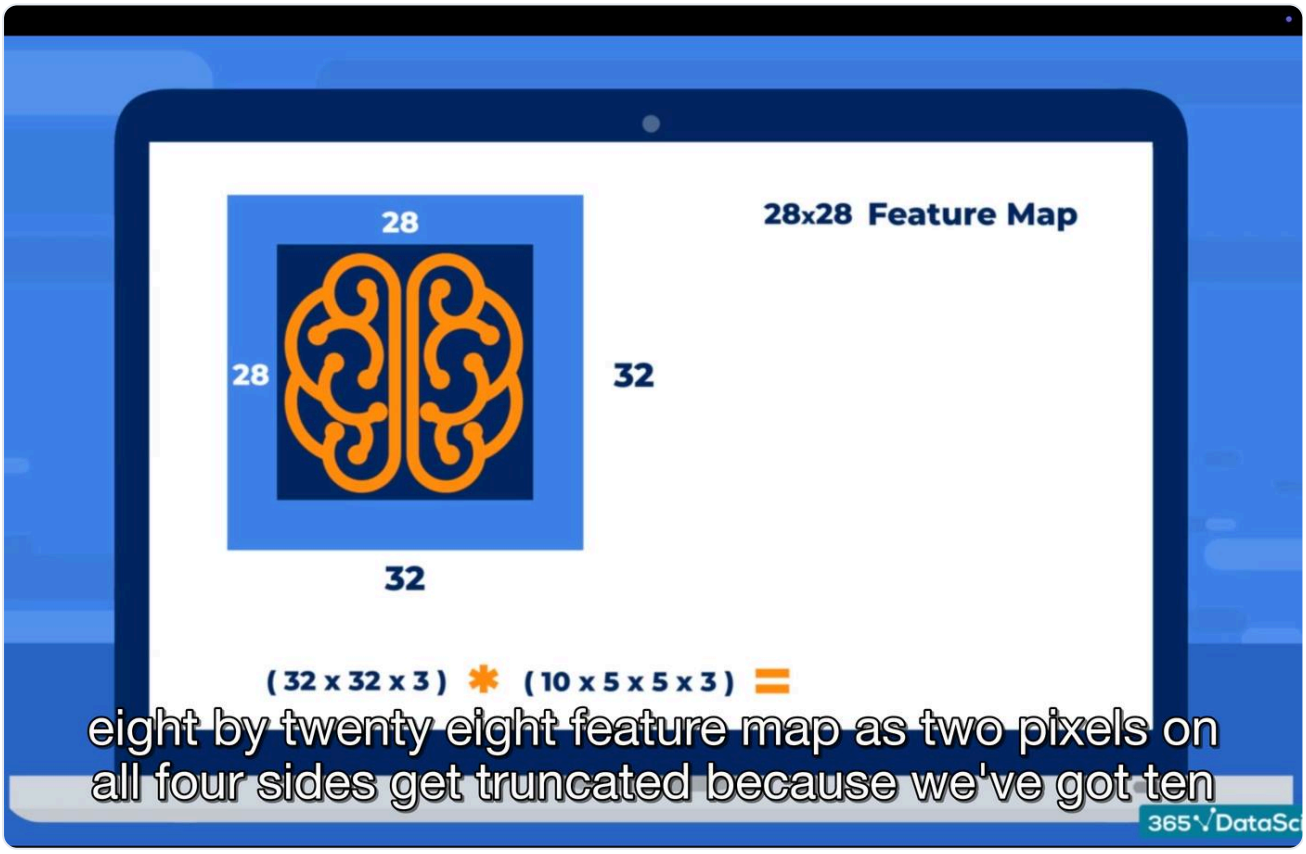

FIGURE 11.11

Well, since we ignore the edges when involving the original 32 by 32, image creates only a twenty eight by twenty eight feature map as two pixels on all four sides get truncated because we've got ten kernels. There are ten different convolutions in total. Therefore we've got ten feature maps.

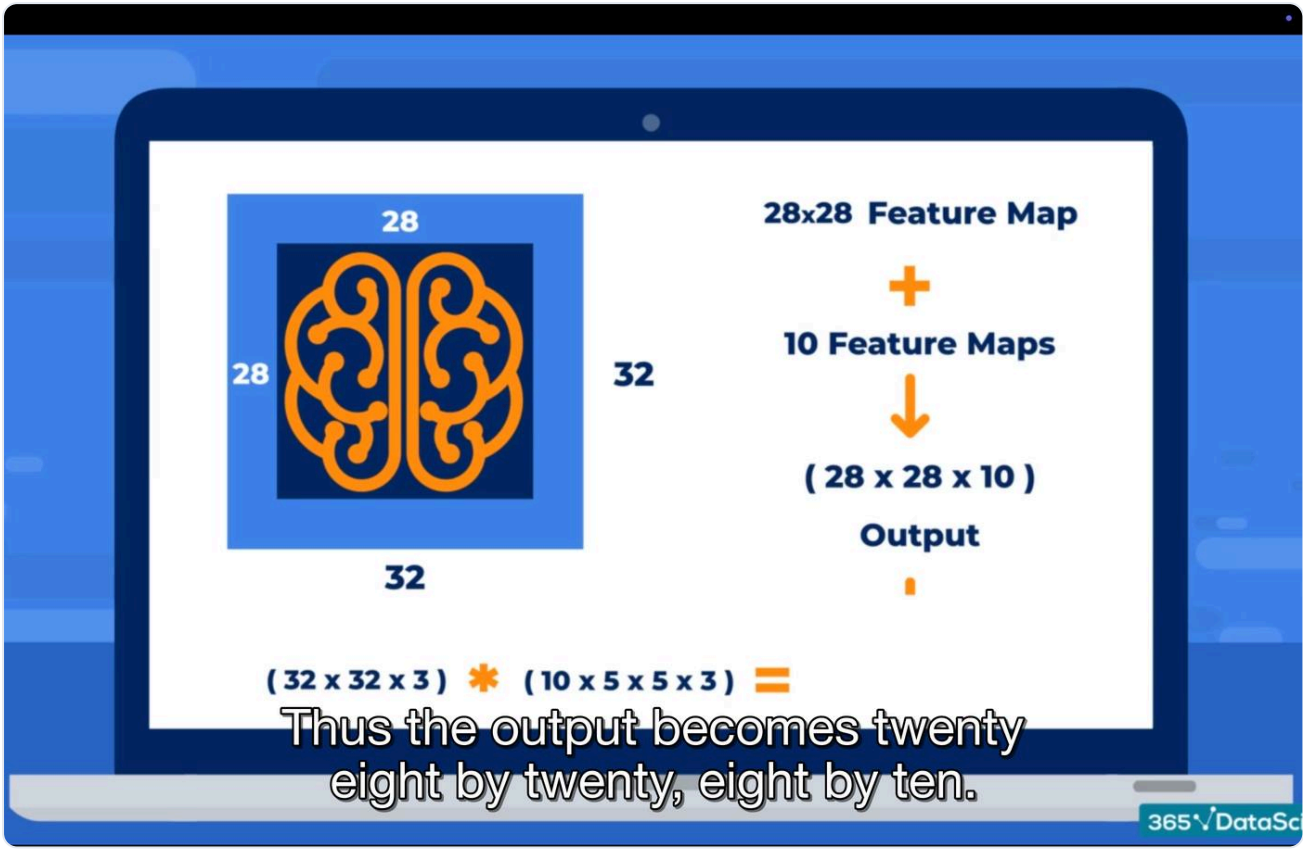

FIGURE 11.12 | 10 Feature Maps

Thus the output becomes twenty eight by twenty, eight by ten.

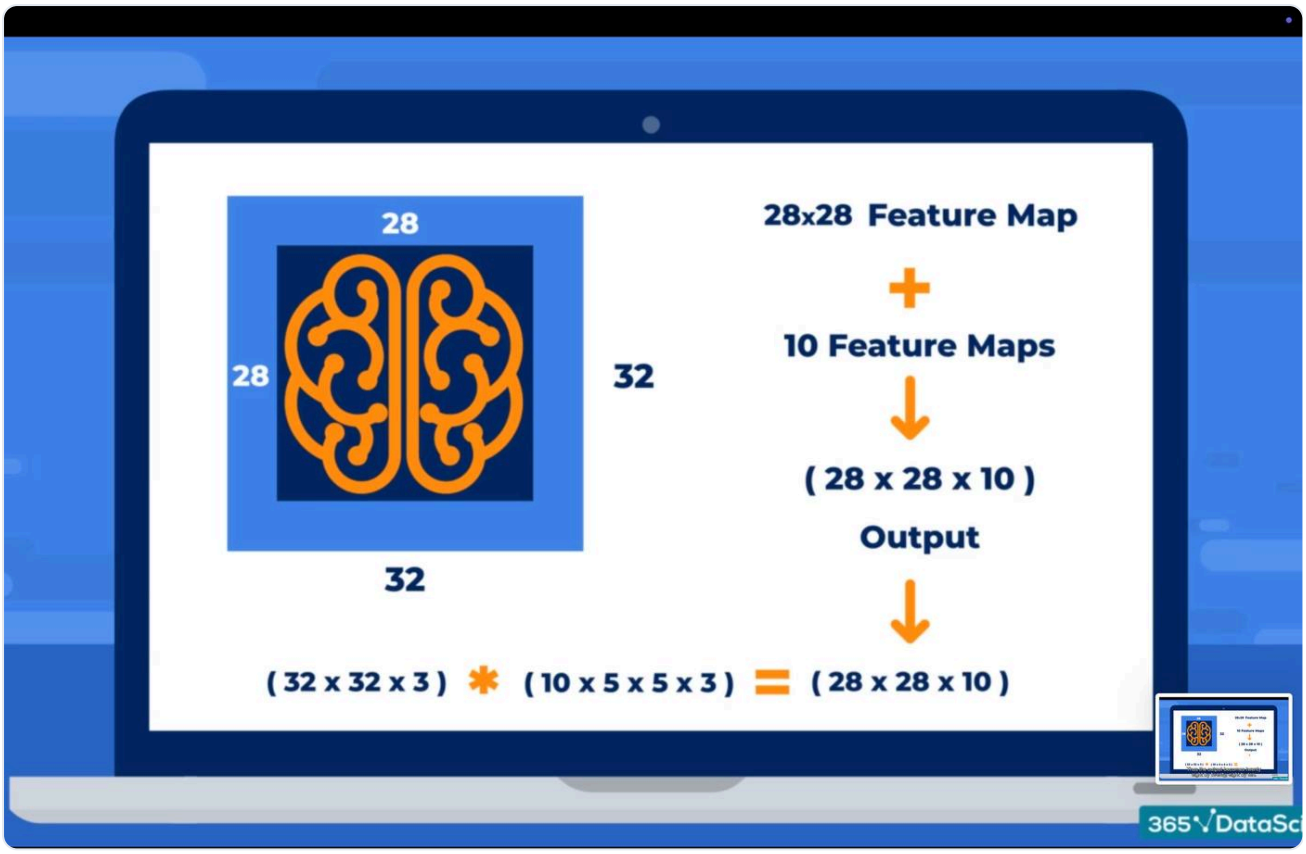

FIGURE 11.13 | 10 Feature Maps

All right, now, let's say we have a max pool player after the first convolutional player in that case, just divide each spatial dimension by two.

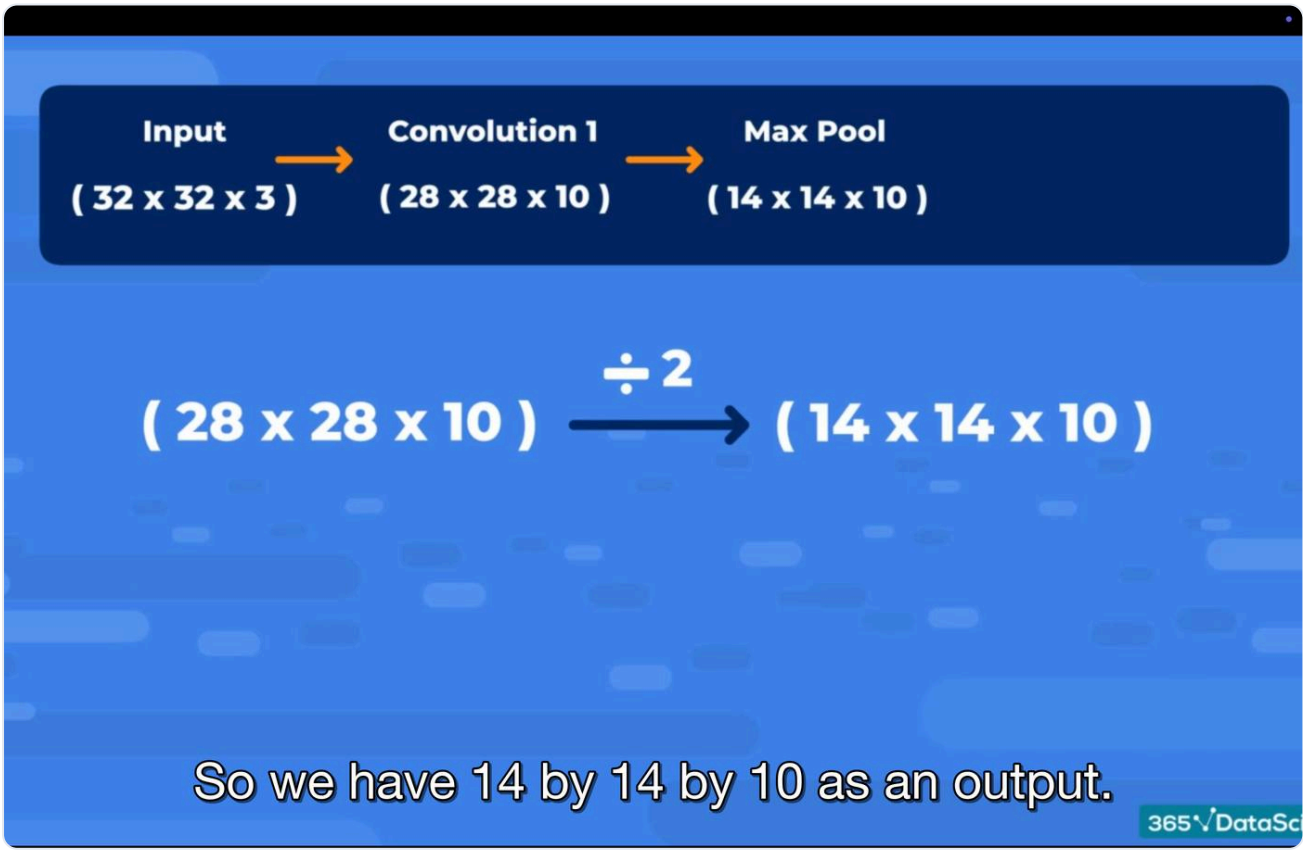

FIGURE 11.14 | Input Convolution 1 Max Pool

So we have 14 by 14 by 10 as an output.

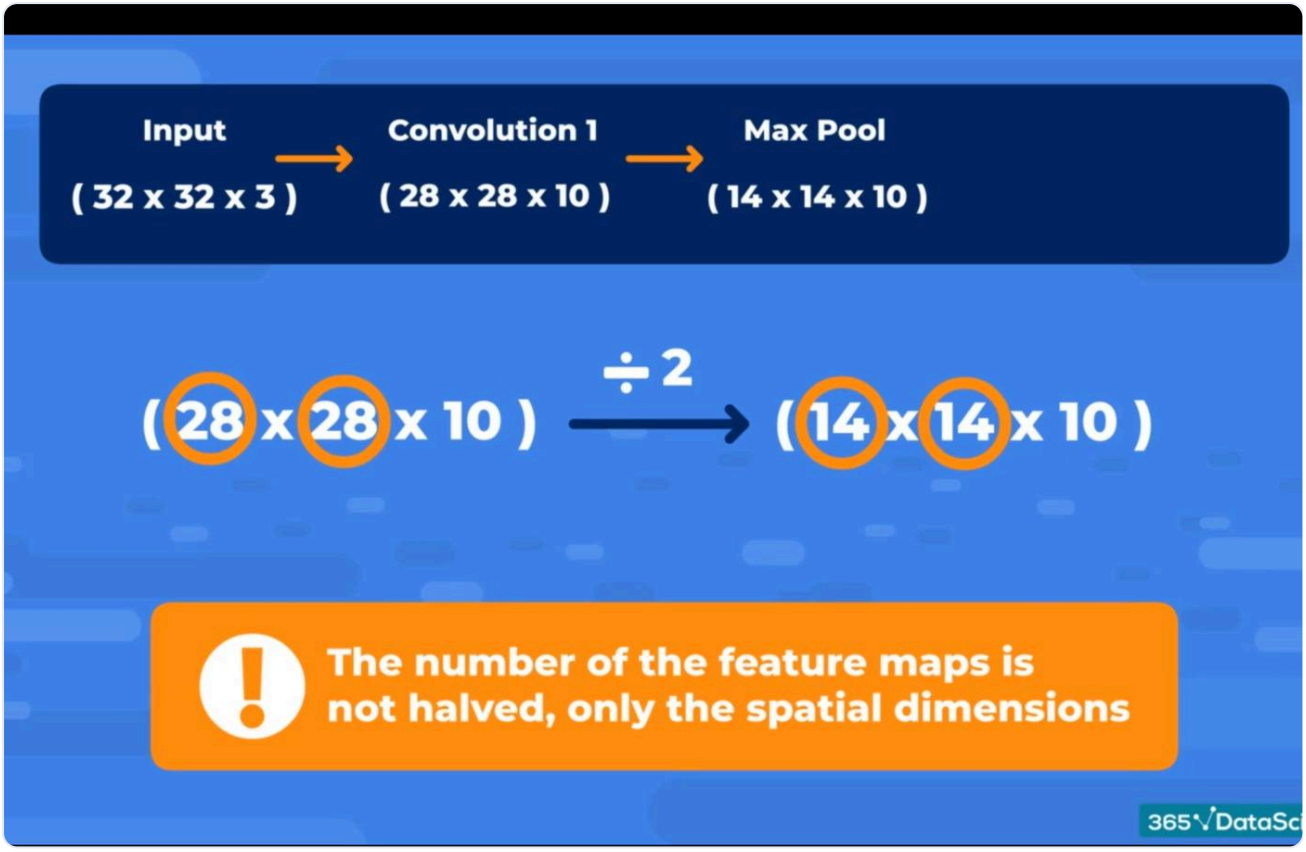

FIGURE 11.15 | Input Convolution 1 Max Pool

Notice have a number of the feature maps is not having only the spatial dimensions. Great.

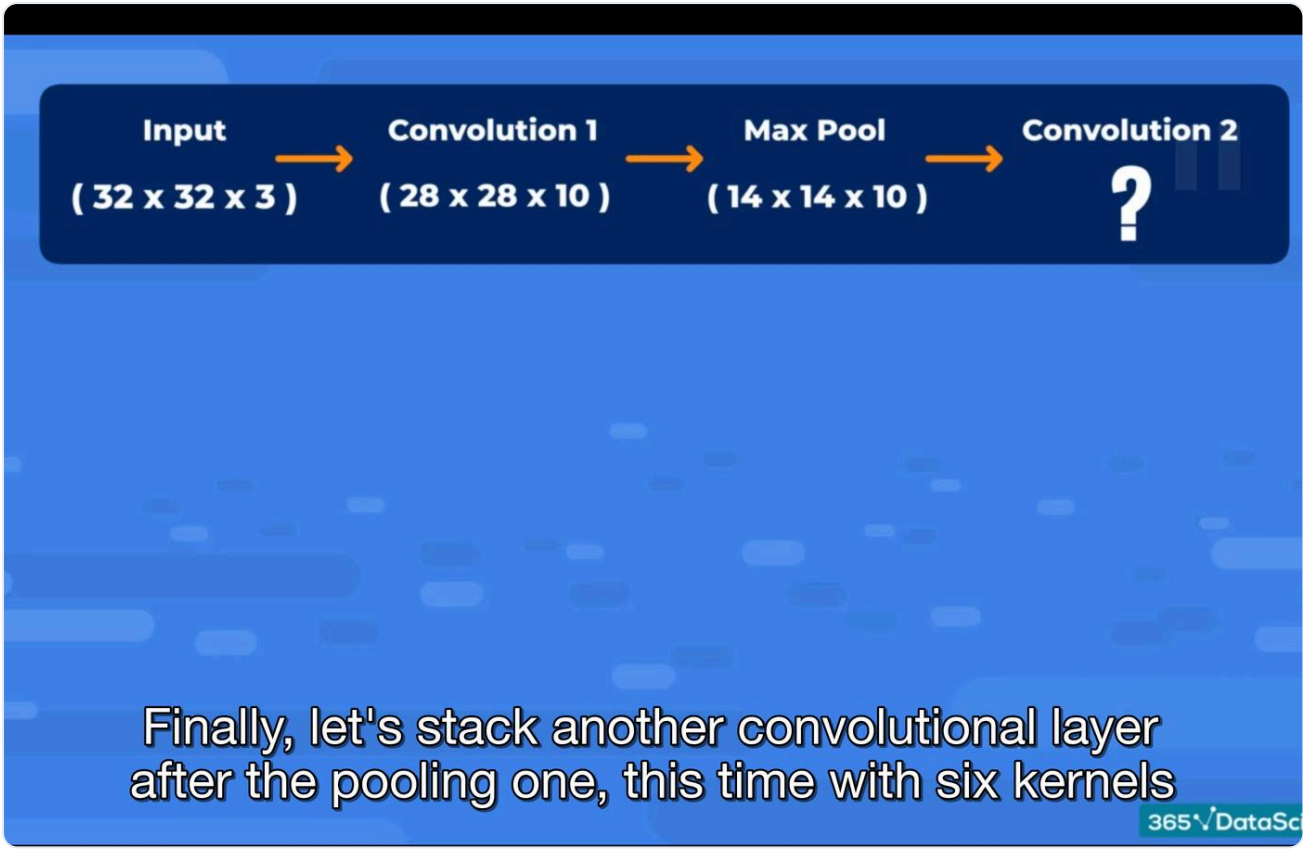

FIGURE 11.16 | Input Convolution 1 Max Pool Convolution 2

Finally, let's stack another convolutional layer after the pooling one, this time with six kernels of size, three by three.

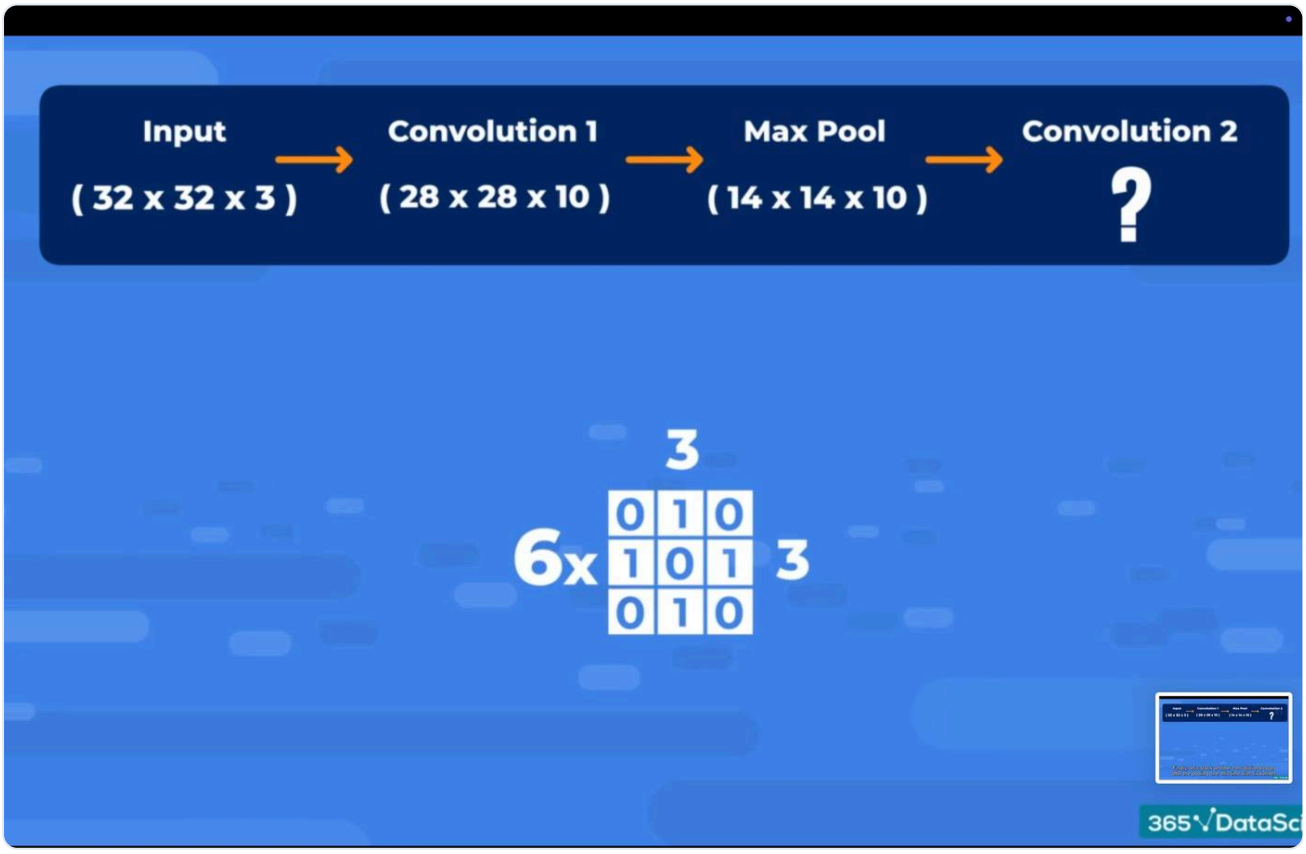

FIGURE 11.17 | Input Convolution 1 Max Pool Convolution 2

So we have 14 by 14 by 10 as an input.

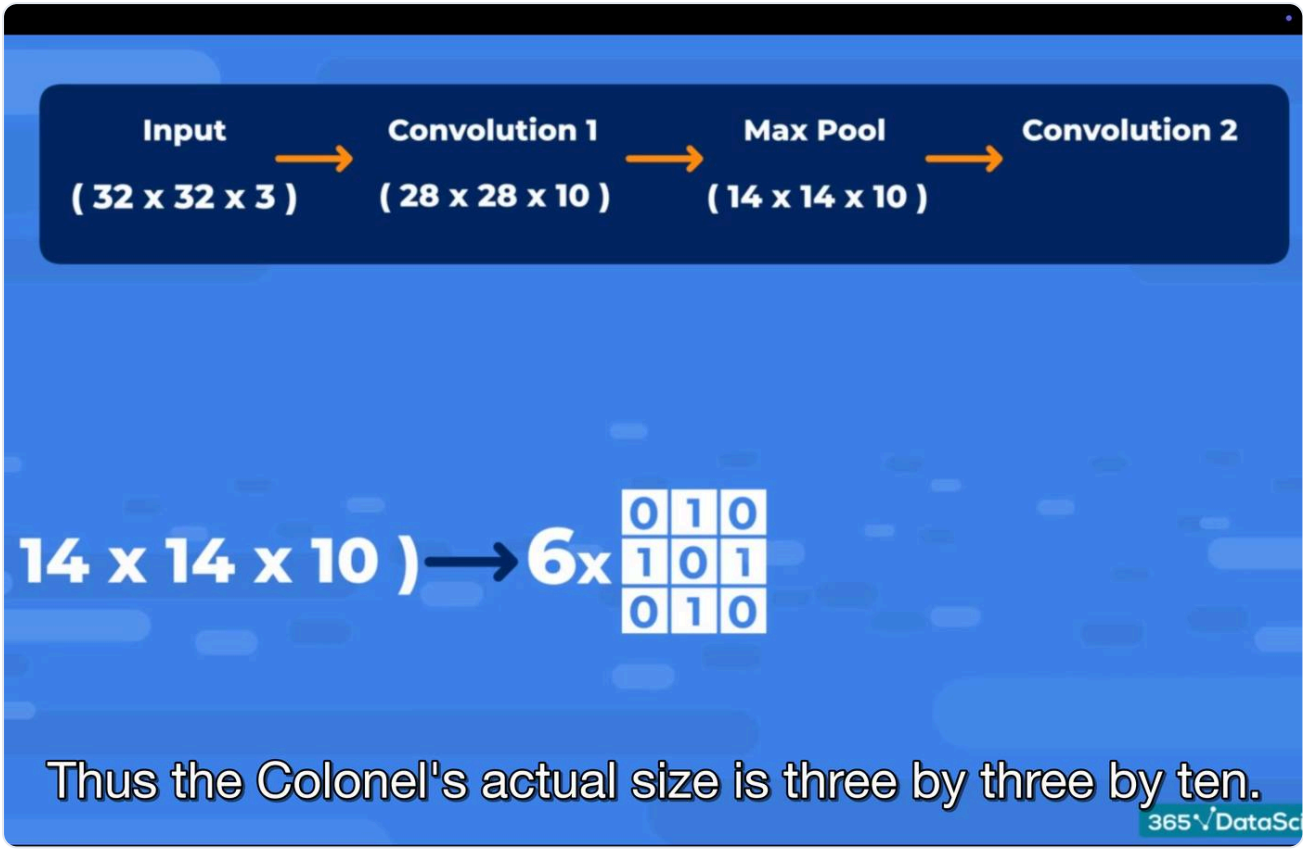

FIGURE 11.18 | Input Convolution 1 Max Pool Convolution 2

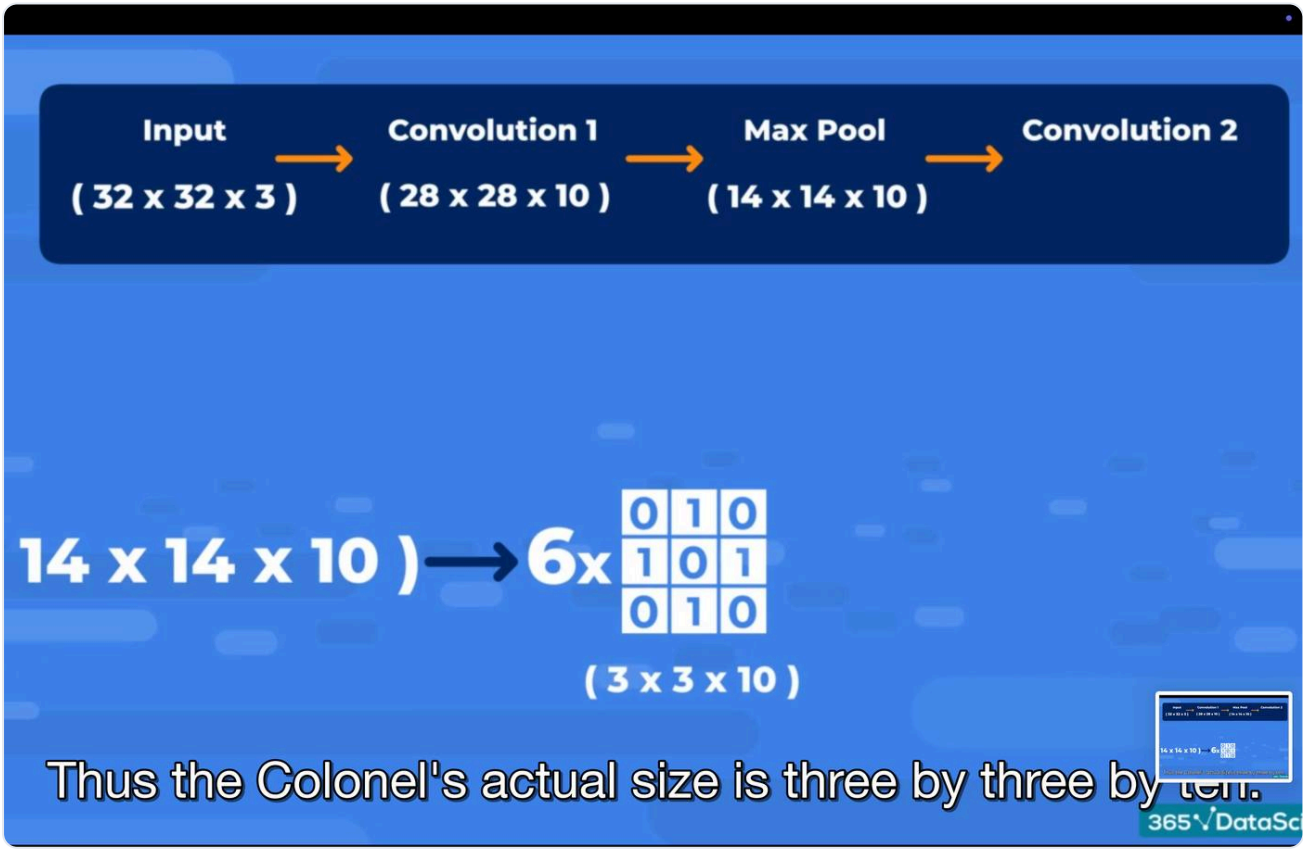

FIGURE 11.19 | Input Convolution 1 Max Pool Convolution 2

Thus the kernel's actual size is three by three by ten.

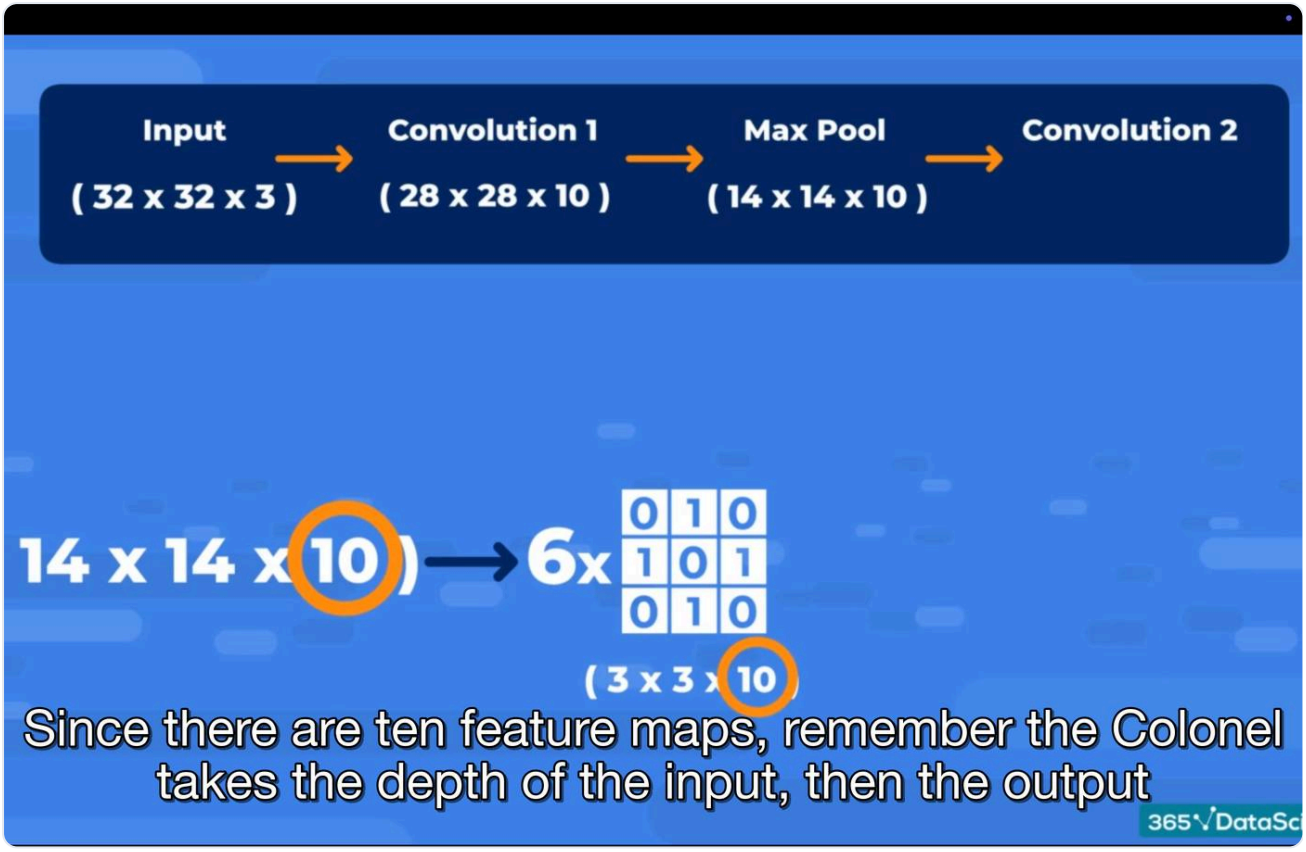

FIGURE 11.20 | Input Convolution 1 Max Pool Convolution 2

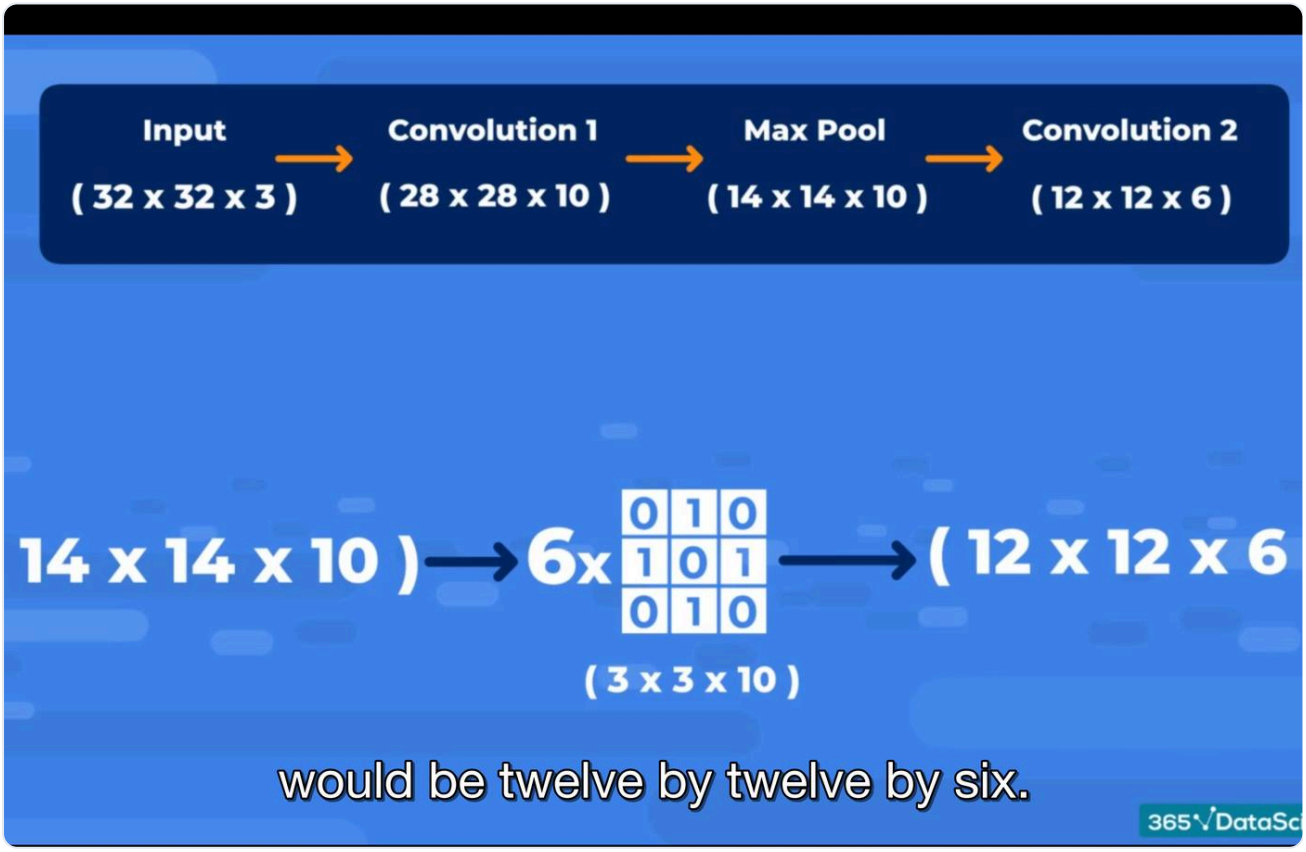

FIGURE 11.21 | Input Convolution 1 Max Pool Convolution 2

Since there are ten feature maps, remember the kernel takes the depth of the input, then the output would be twelve by twelve by six.

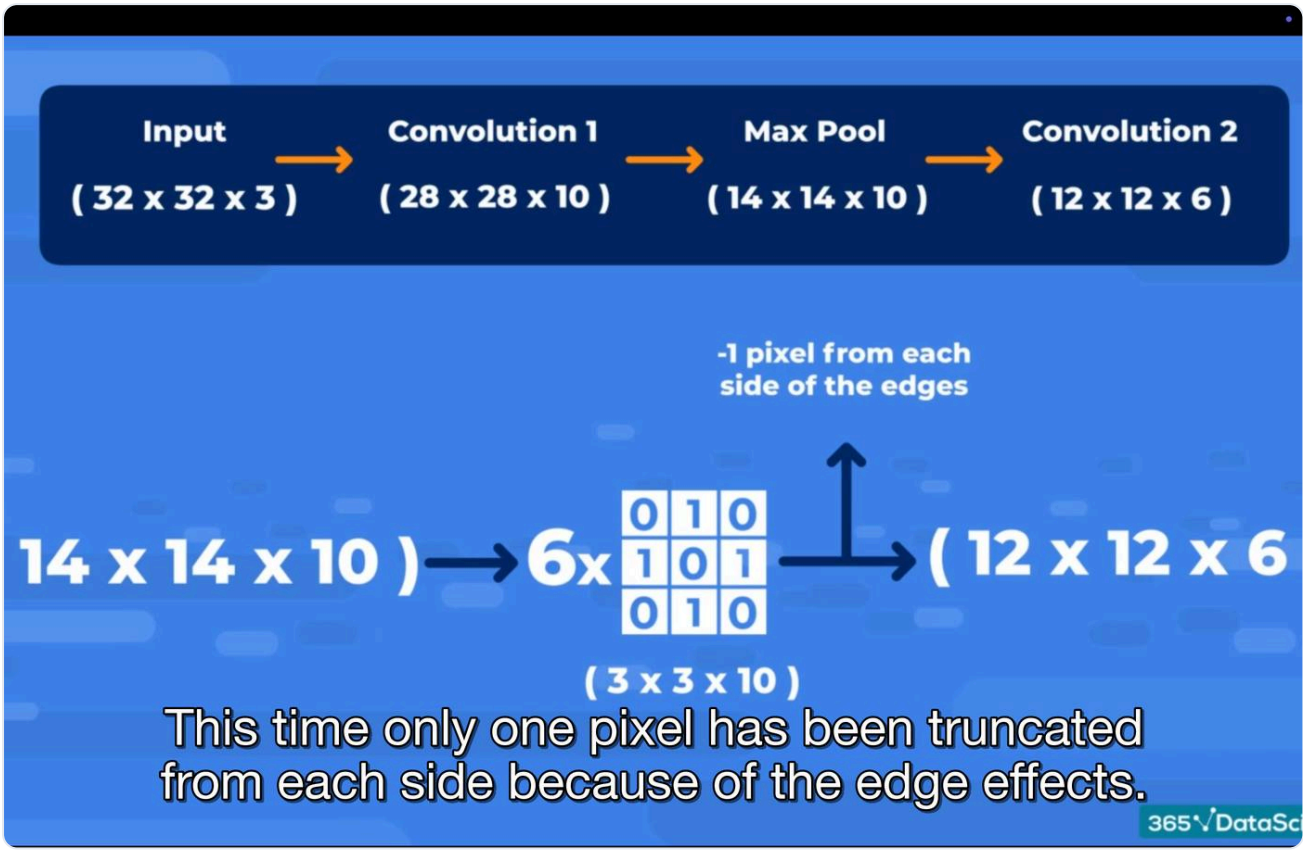

FIGURE 11.22 | Input Convolution 1 Max Pool Convolution 2

This time only one pixel has been truncated from each side because of the edge effects.

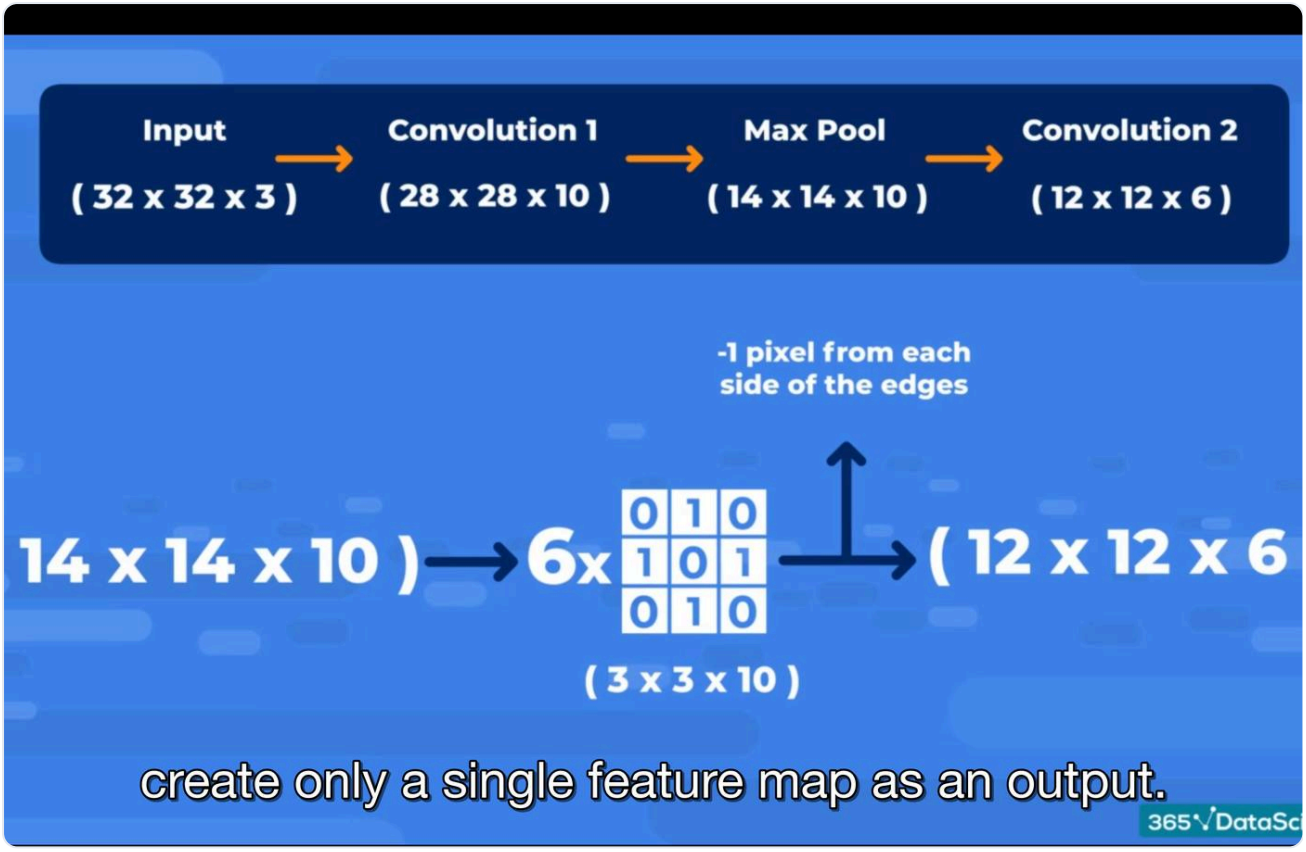

FIGURE 11.23 | Input Convolution 1 Max Pool Convolution 2

Also, we only get six feature maps as a single kernel, takes all previous feature maps at once to create only a single feature map as an output. All right, when working with high level libraries, it may seem unimportant to know all the dimensions, but it is a good skill to have. So make sure you understand why all inputs and outputs are the way they are in this lecture. OK, that was all the theory about convolutional neural networks. In the next section, we will create our first CNN in practice and we'll train it on the MNIST data set. See you there. And thanks for watching.

LECTURE 12

Name Formula Derivative Graph Range

Since we are going to build our first convolutional neural network in the next section, I figured a quick revision session wouldn't hurt if you were feeling confident in your neural networks knowledge. Feel free to skip this section. If, on the contrary, you are not all too familiar with the terms used here, it might be a good idea to check out our materials on feed forward neural networks before jumping in to CNN. All right. Now, let's start with the revision. So the main concept of deep neural networks is to achieve complex, nonlinear behavior by stacking relatively simple layers on top of each other. A single isolated layer is well understood. However, when combined with others, much more complex behavior can emerge.

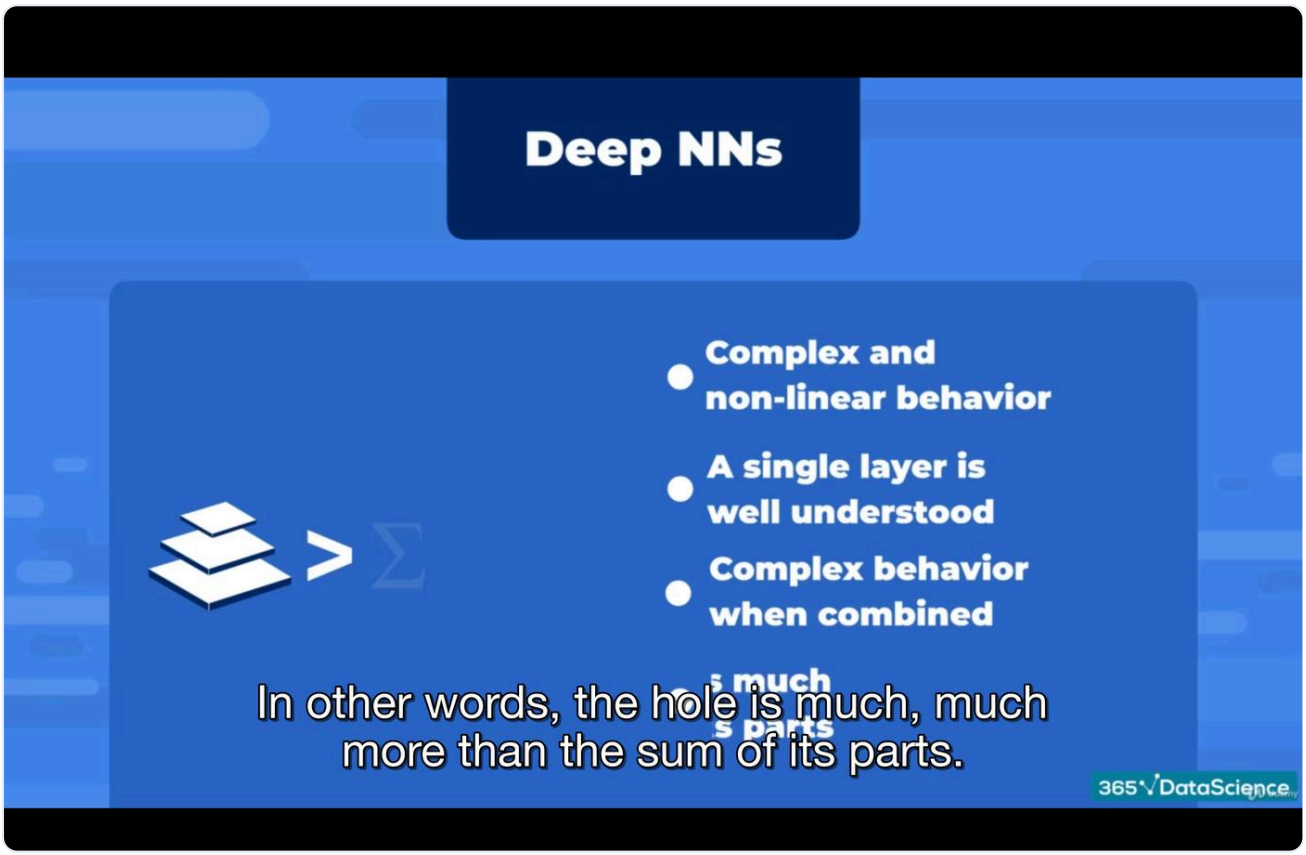

FIGURE 12.1

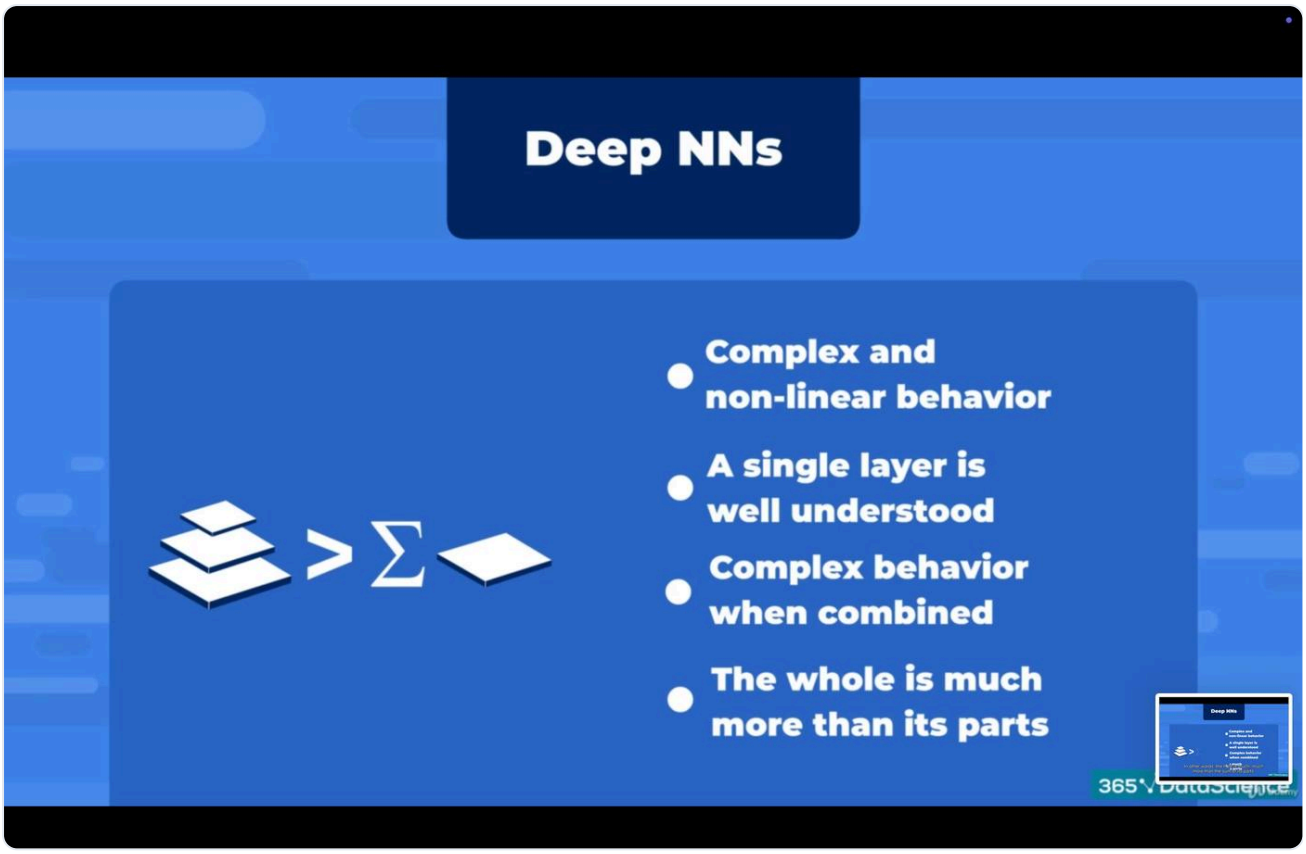

FIGURE 12.2

In other words, the hole is much, much more than the sum of its parts.

FIGURE 12.3

Now, apart from linear regressions and other linear models in a neural network, we always add a non-linearity to the linear combination of each layer. Reason is that if we only stack linear layers, the result would still behave linearly in machine learning context.

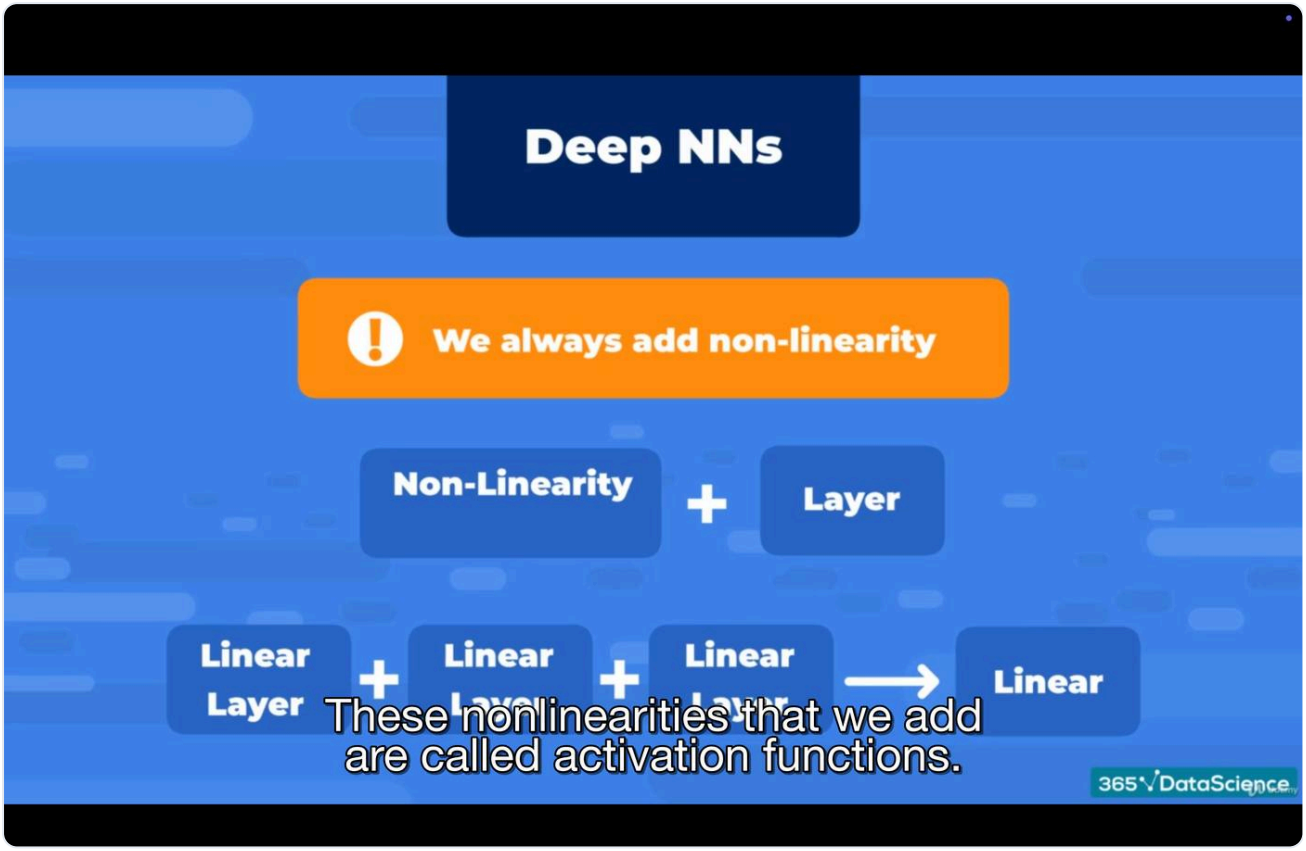

FIGURE 12.4

These nonlinearities that we add are called activation functions. OK, here is a table with four of the most commonly used activation functions. Let's go through one row to see how the table is organized. First, we have the sigmoid, also known as the logistic function. Here is its formula. Next to it we have its derivative. As you may recall, the derivative is an essential part of the gradient descent.

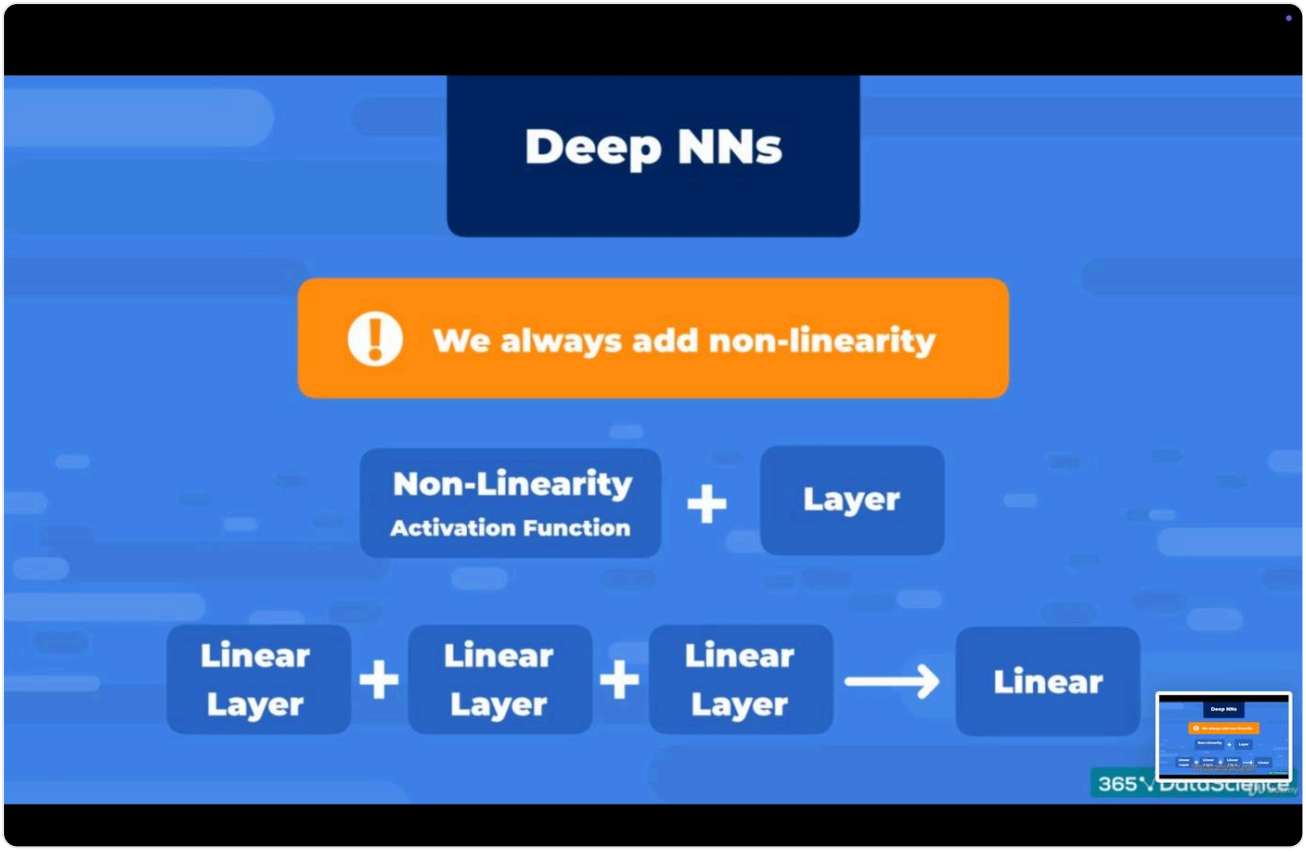

FIGURE 12.5

Naturally, when we work with TensorFlow, we won't need to calculate the derivative as TensorFlow. Does that automatically anyhow? The purpose of this lesson is to remind you of these functions, their graphs and ranges. So here is the functions graph and finally we have its range. Once we have applied the sigmoid as an activator, all the outputs will be contained in the range from zero to one. So the output is somewhat standardized. All right, here are the other three common activators.

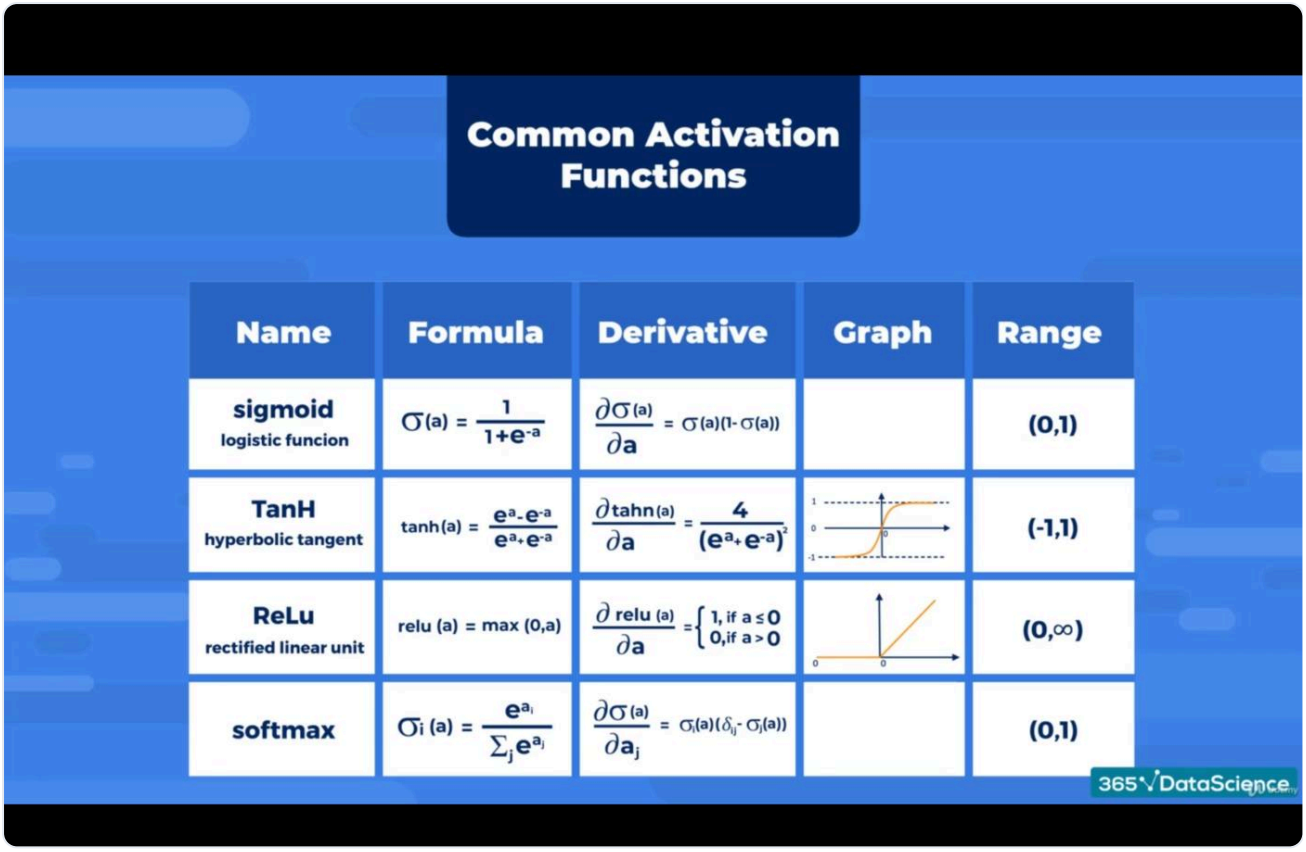

FIGURE 12.6

The technique also known as the hyperbolic tangent, the RELU, a.k.a. the rectified linear unit and softmax. You may notice that the graph of the softmax is missing, but don't worry about that. We will get to it in a minute. In practice, the most widely used activation function is the RELU and its many variations. In fact, we will use it extensively for our own networks. So what does it do? Well, as you can see, it is a relatively simple one, both from a computational and mathematical point of view. Its output is simply the input. If the input is greater than zero, otherwise it returns zero.

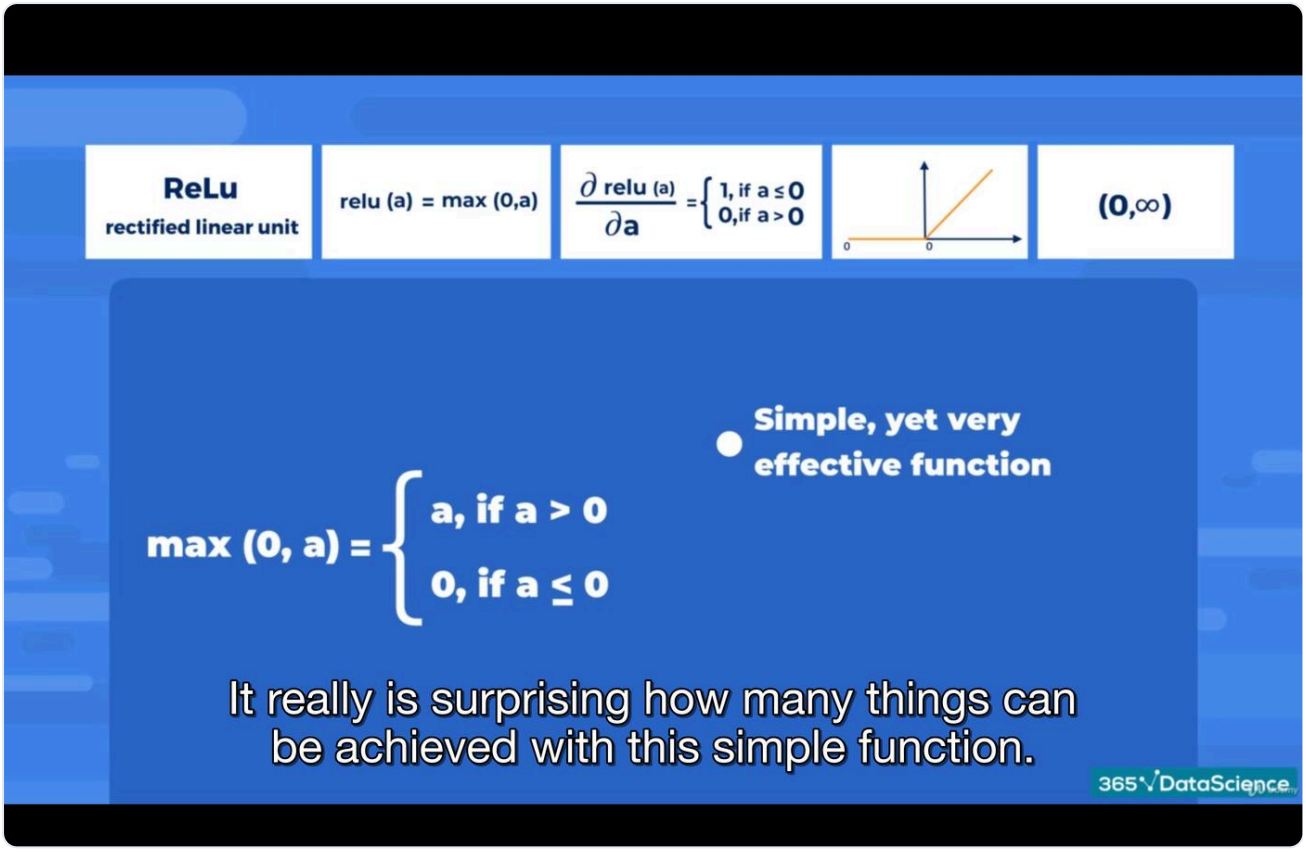

FIGURE 12.7

It really is surprising how many things can be achieved with this simple function. Of course, there are also situations in which Relu has some drawbacks.

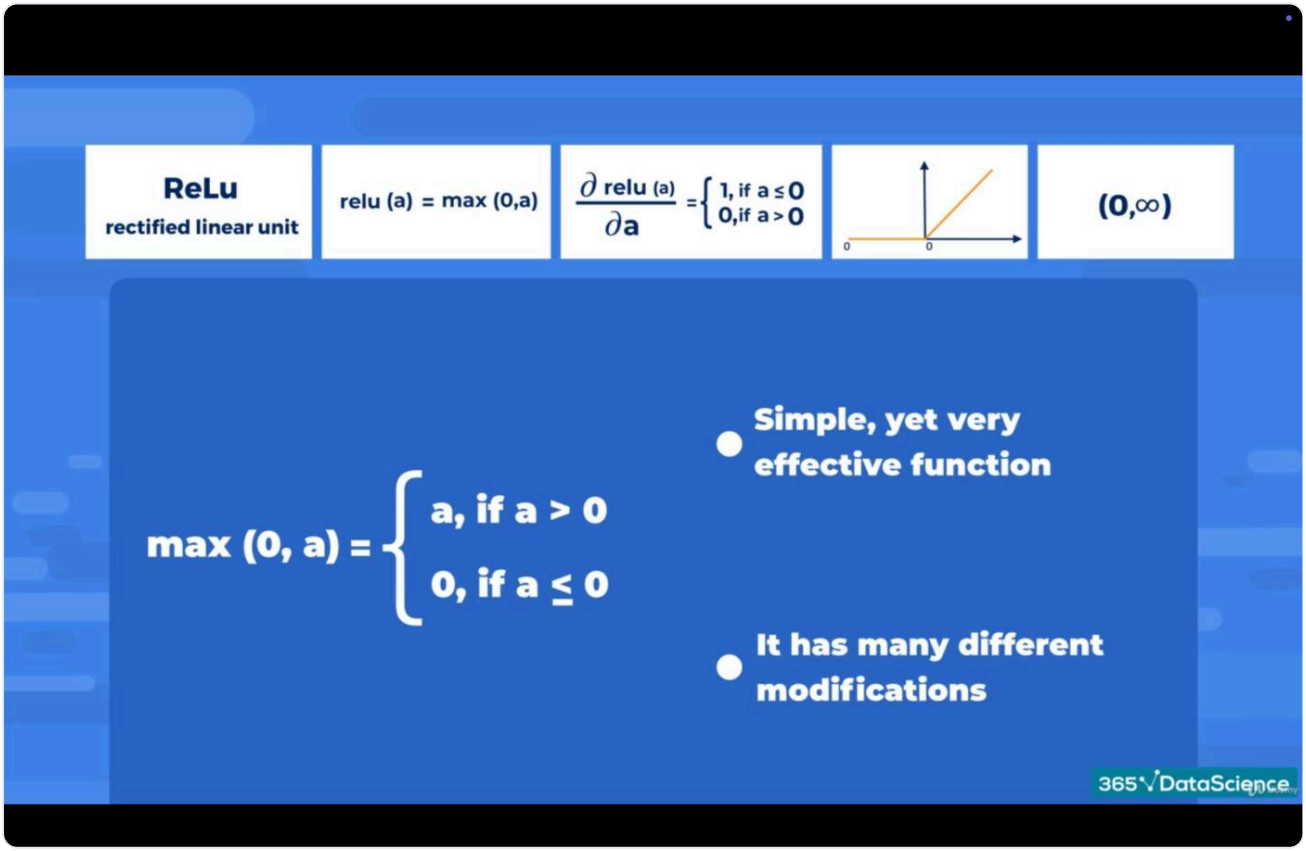

FIGURE 12.8

That's why there are many different modifications for specific use cases. Great. What about the softmax? Well, softmax is used specifically when dealing with classification.

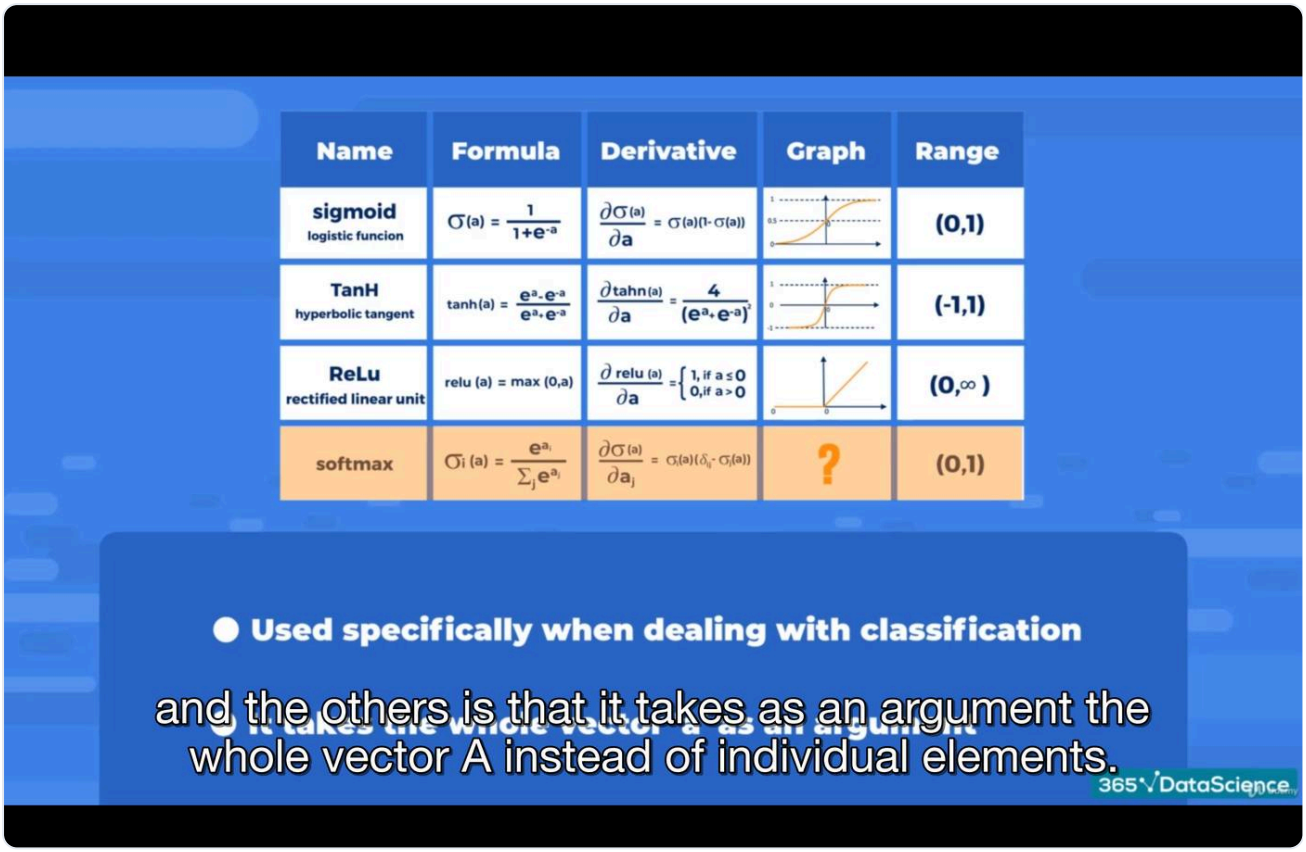

FIGURE 12.9 | Name Formula Derivative Graph Range

If we take a careful look at its formula, we would see that the key difference between this function and the others is that it takes as an argument the whole vector A instead of individual elements. So the softmax function is equal to the exponential of the element and position I divided by the sum of the exponential of all elements of the vector.

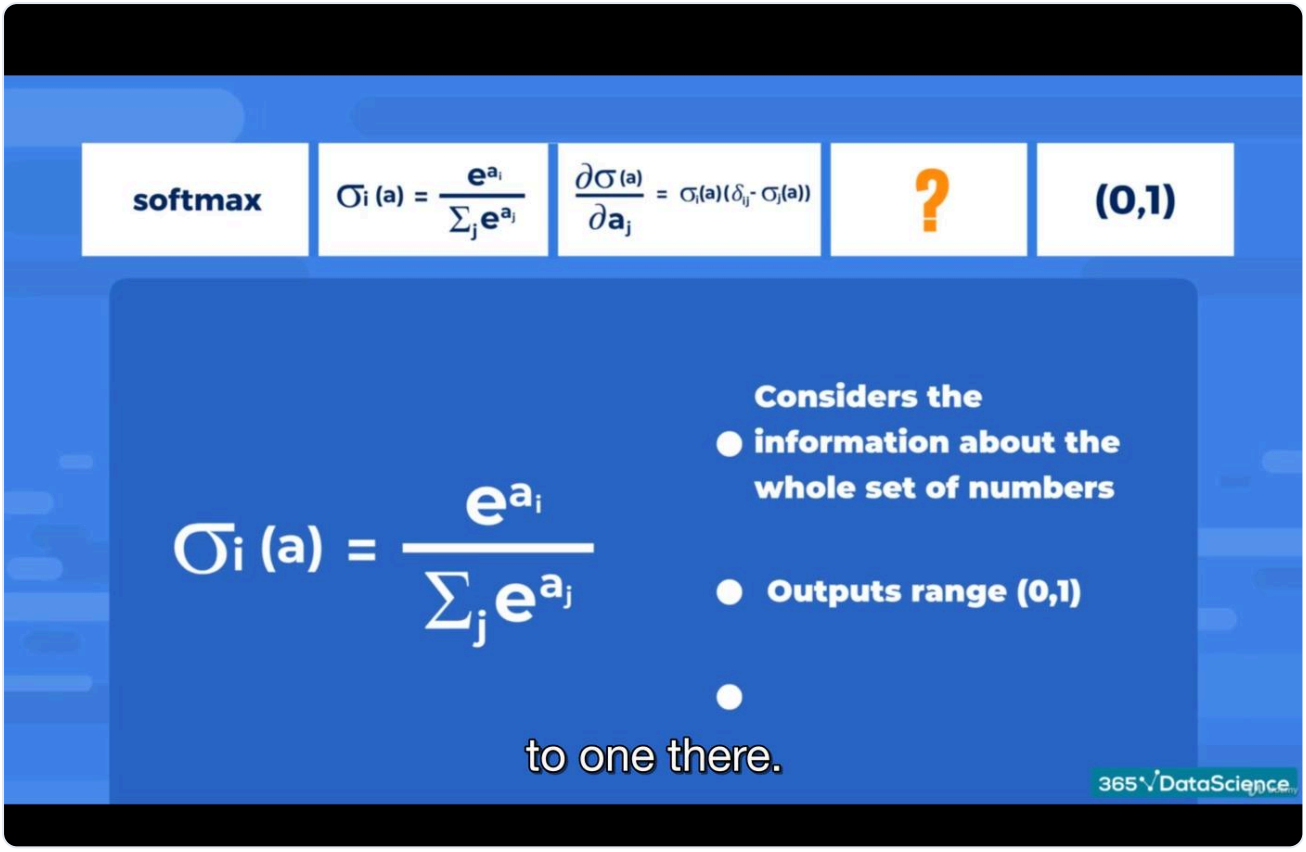

FIGURE 12.10 | Considers the

And while the other activation functions get an input value and transform it regardless of the other elements, the softmax considers the information about the whole set of numbers we have. A key aspect of the softmax transformation is that the values IT outputs are in the range from zero to one there.

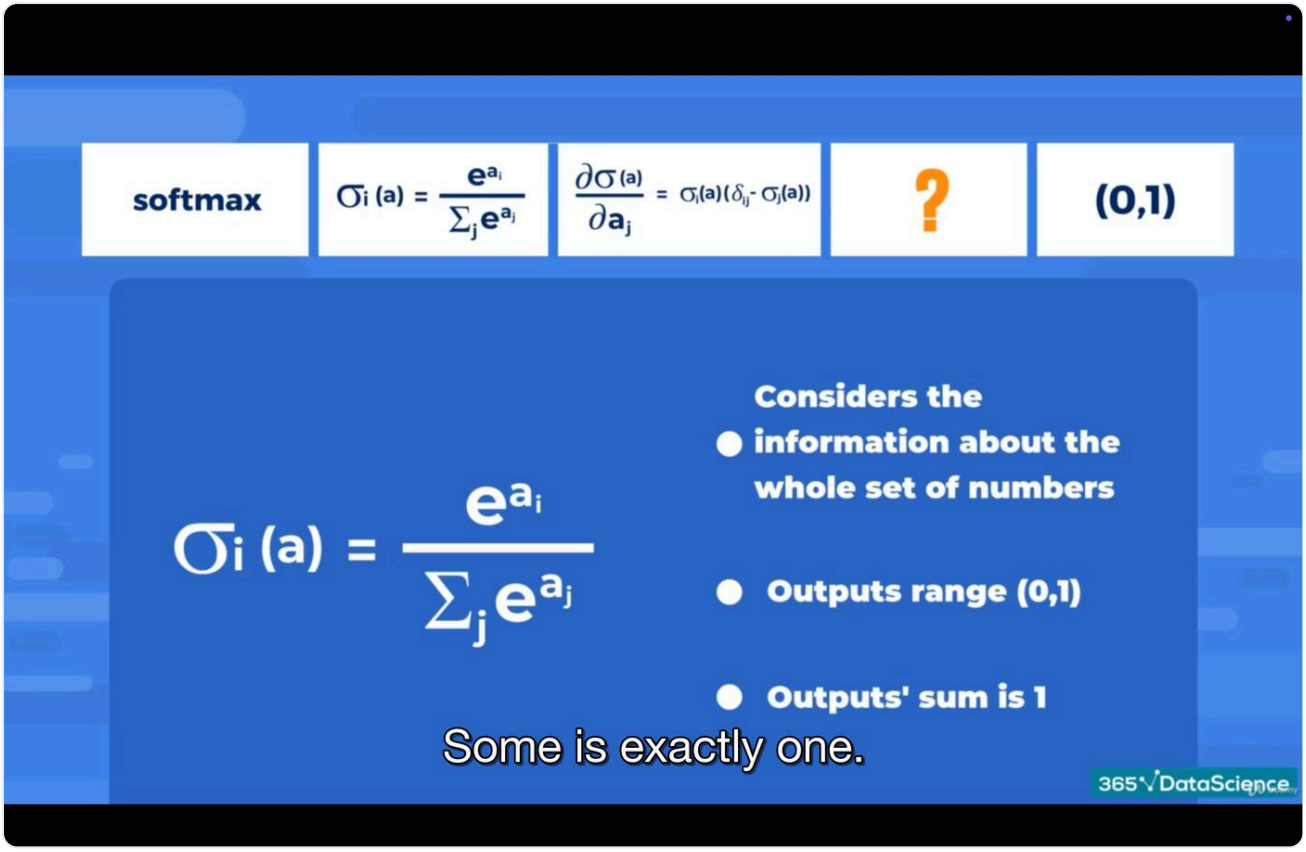

FIGURE 12.11 | Considers the

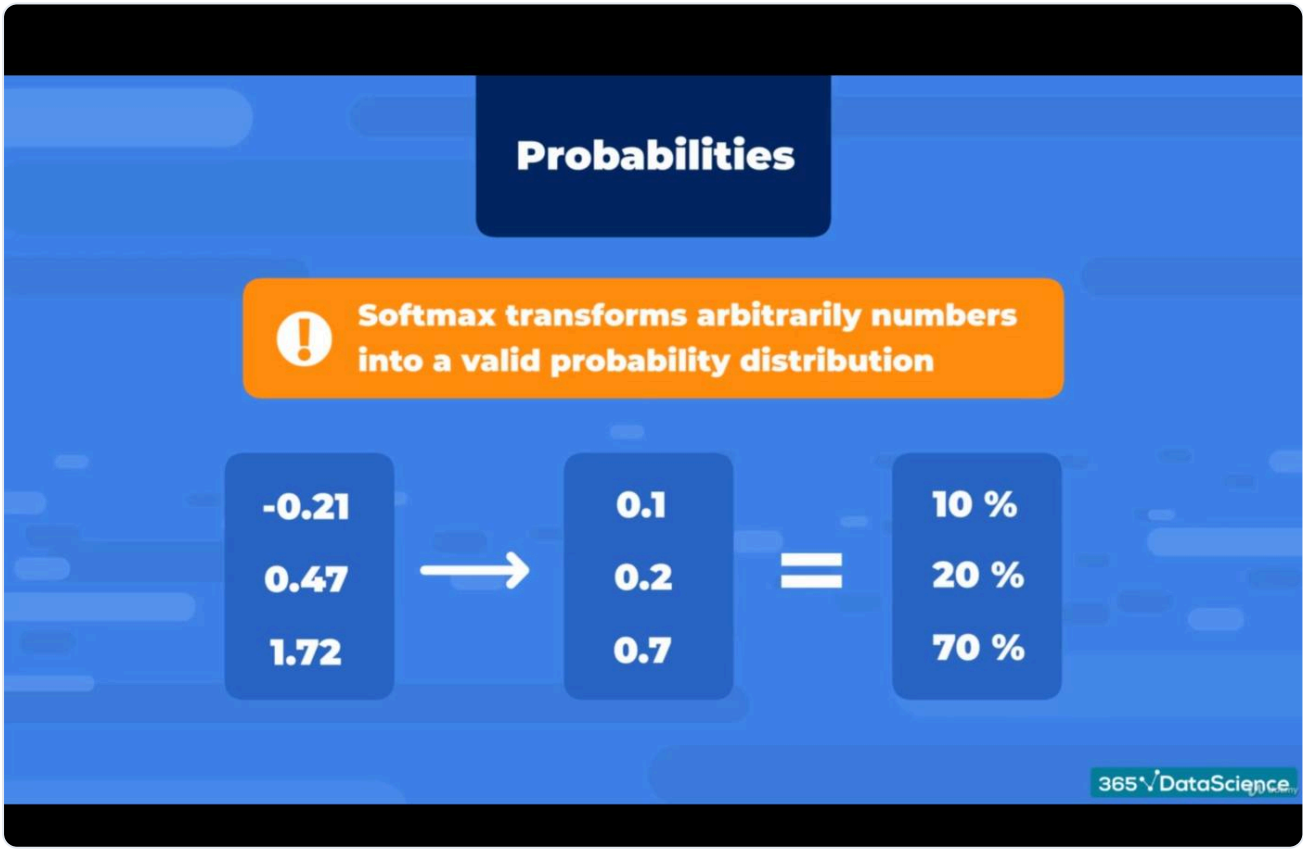

FIGURE 12.12

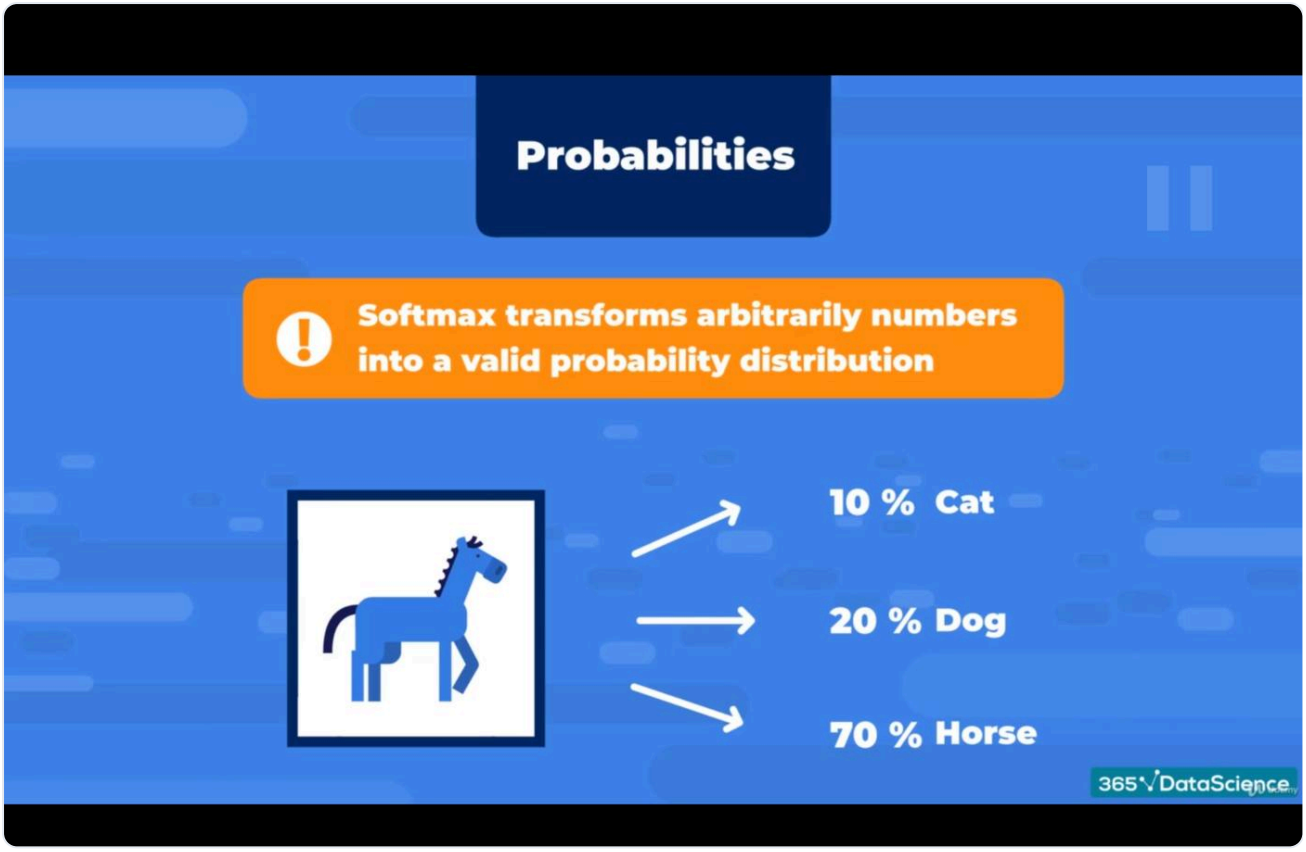

FIGURE 12.13

Some is exactly one. What else has such a property, hmm, probabilities, yes, probabilities indeed, the point of the softmax is to transform a bunch of arbitrarily large or small numbers that come out of previous layers and fit them into a valid probability distribution. This is extremely important and useful because when we deal with categorization problems, we want our output to be an easily interpretable number. That's why the softmax activation function is almost always used as the activation of the final output layer in classification problems. So no matter what happens before, the final output of the algorithm is a probability distribution. All right.

That was a quick recap of the activation functions. In the next video, we will talk about early stopping. Thanks for watching.

LECTURE 13

Overfitting and Validation Set

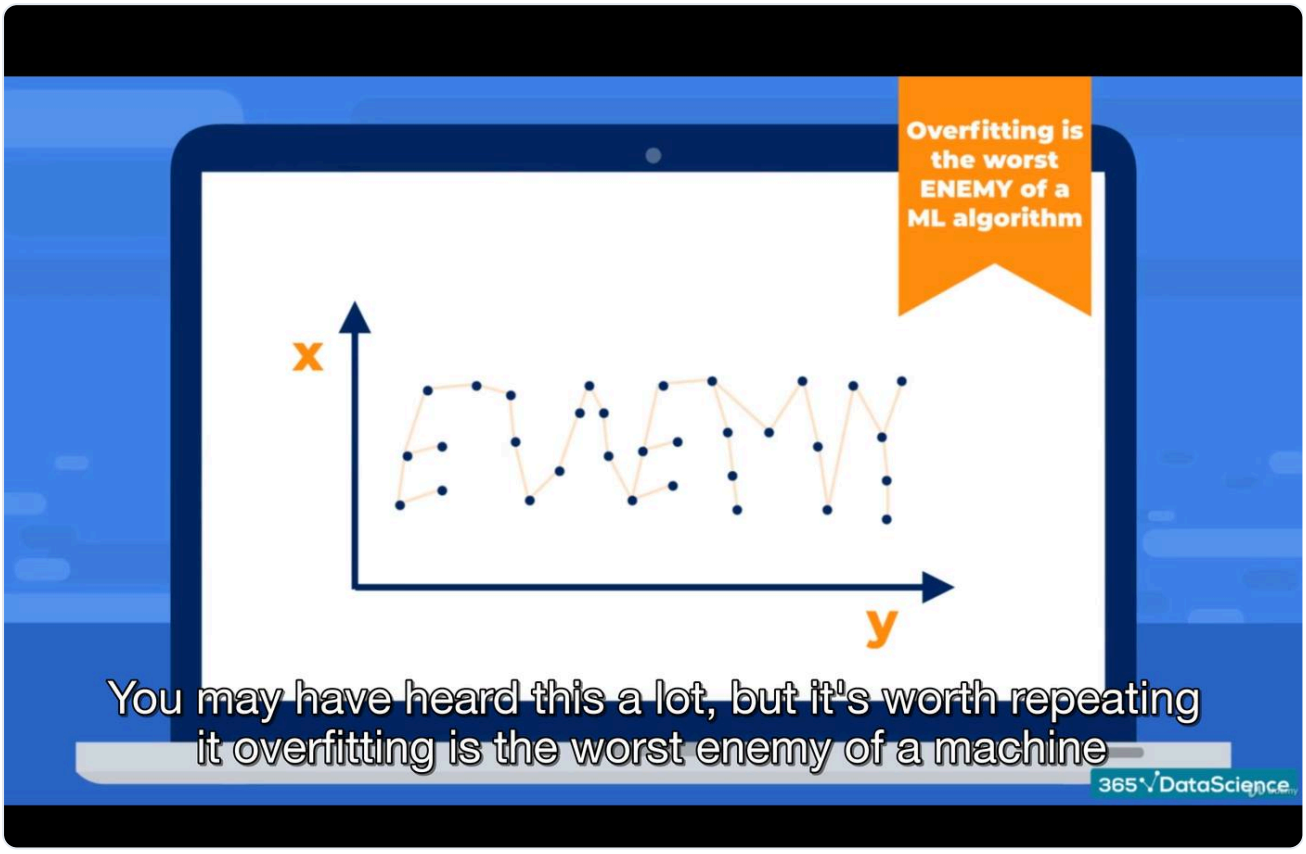

FIGURE 13.1

You may have heard this a lot, but it's worth repeating it overfitting is the worst enemy of a machine learning algorithm in this lecture. We will talk about the battle with Overfitting and the most common strategy to consider in that battle, or at least stopping, OK? Let's start with a quick recap on Overfitting in machine learning, what we fundamentally do is to construct a model with a certain number of parameters.

FIGURE 13.2

We then try to tune those parameters based on already existing data. This is the training process.

FIGURE 13.3

FIGURE 13.4

FIGURE 13.5

FIGURE 13.6

However, in doing so, we can accidentally tune our model to fit the training data perfectly.

FIGURE 13.7

FIGURE 13.8

That is, to also capture the noise of the data.

FIGURE 13.9

FIGURE 13.10

The problem with capturing the noise is that the model then misses the general pattern. That's what Overfitting is about. All right, so what can we do to prevent Overfitting from happening? Well, overfitting is not something that just switches on and off. It gradually builds up in the training process.

FIGURE 13.11 | HAUS One Techniguesto Counter It iS Fostopn

Thus, one technique to counter it is to stop the training when we detect overfitting. Cool. Now, the logical follow up question is how can we detect overfitting?

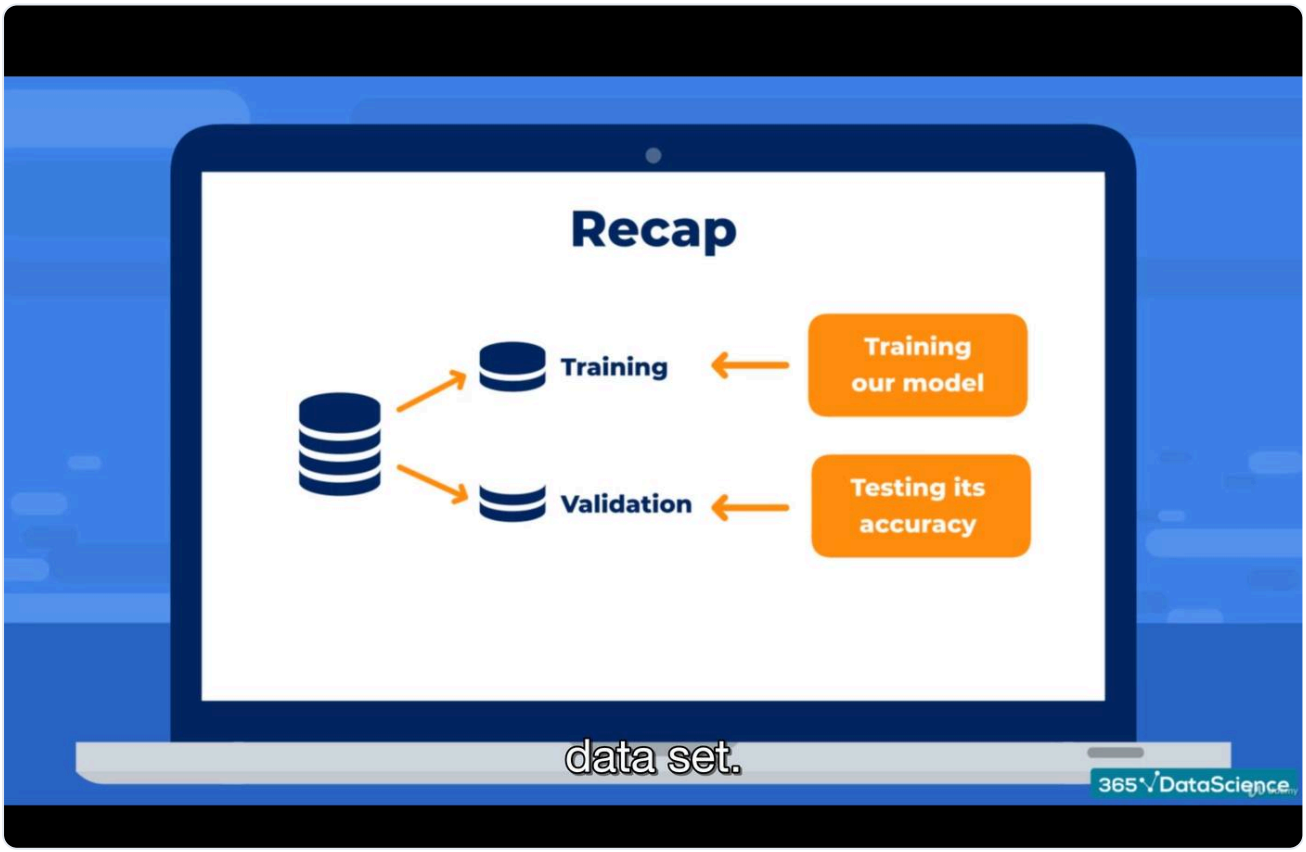

FIGURE 13.12

The answer is pretty simple. Split the data into a training set and a validation set. While we are training our model on the training set, we are testing its accuracy on the validation data set.

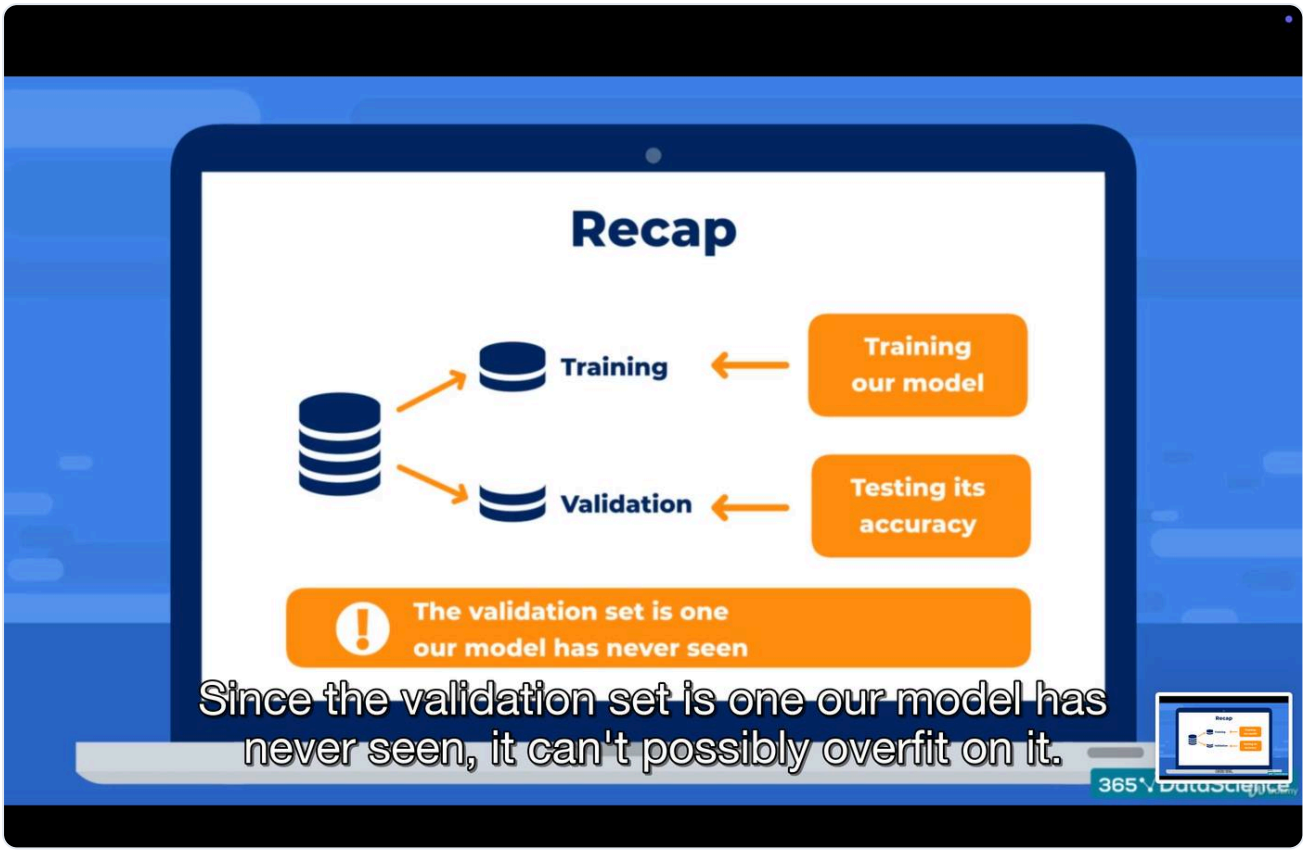

FIGURE 13.13

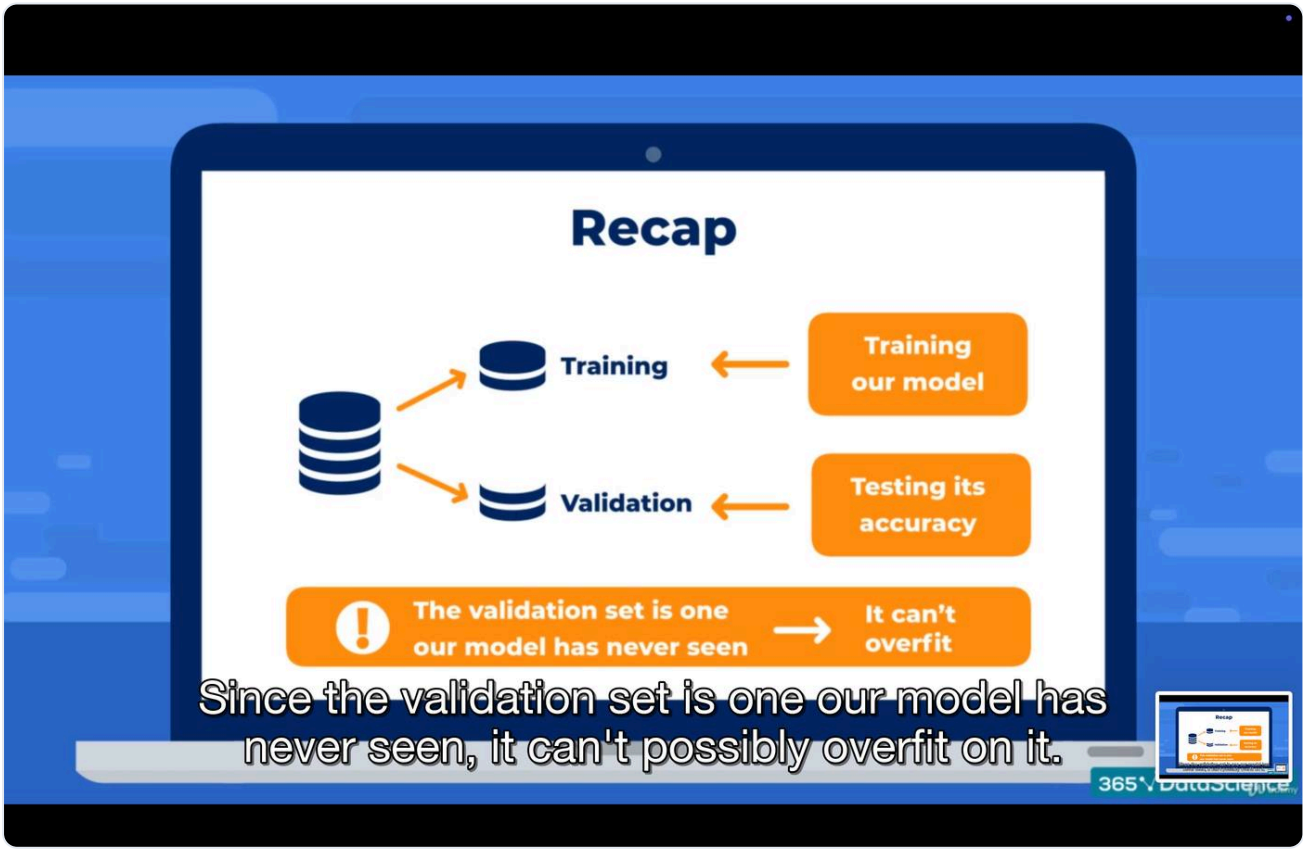

FIGURE 13.14

Since the validation set is one our model has never seen, it can't possibly overfit on it. Thus, to check for overfitting, we can track when the loss on the validation set increases or the accuracy decreases.

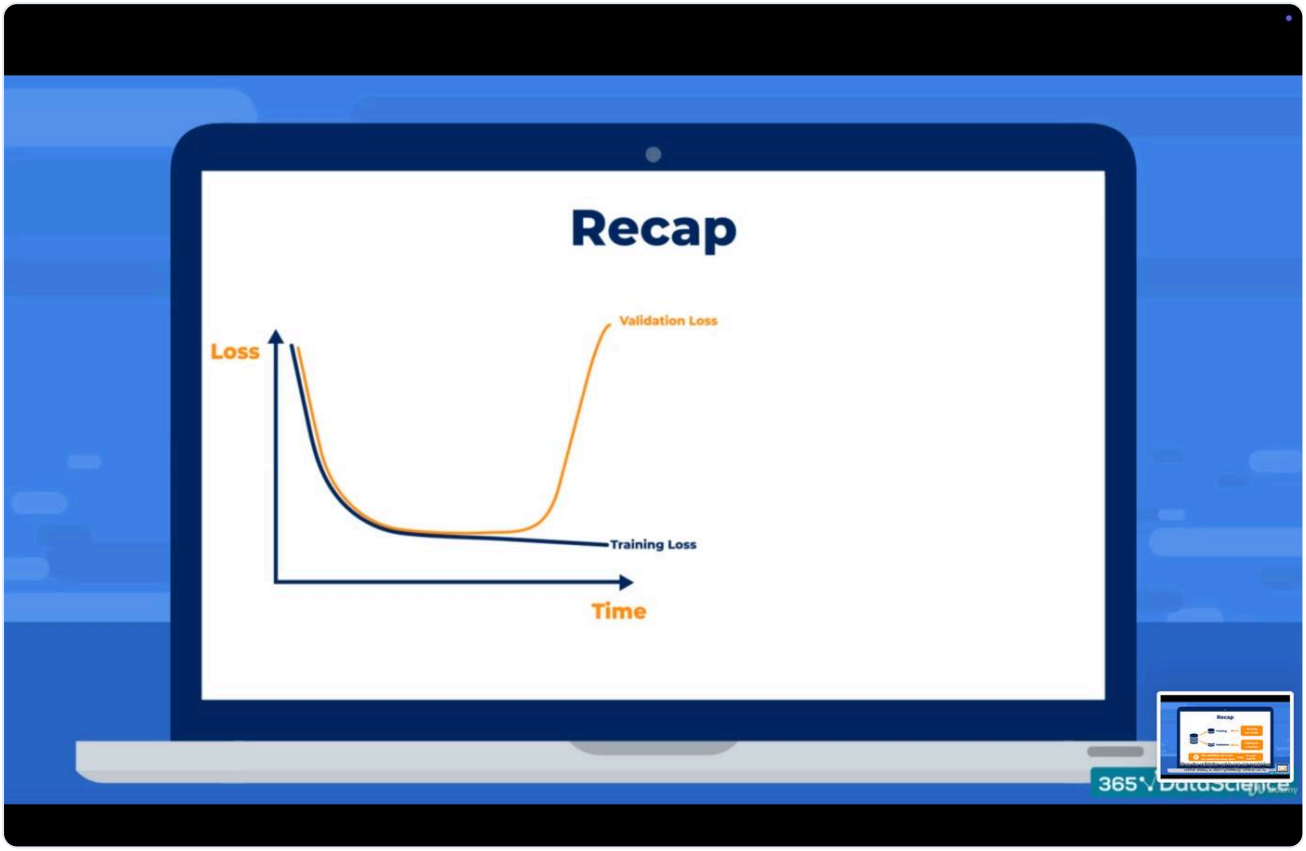

FIGURE 13.15

That is the point at which we have to take action, namely stop the training process.

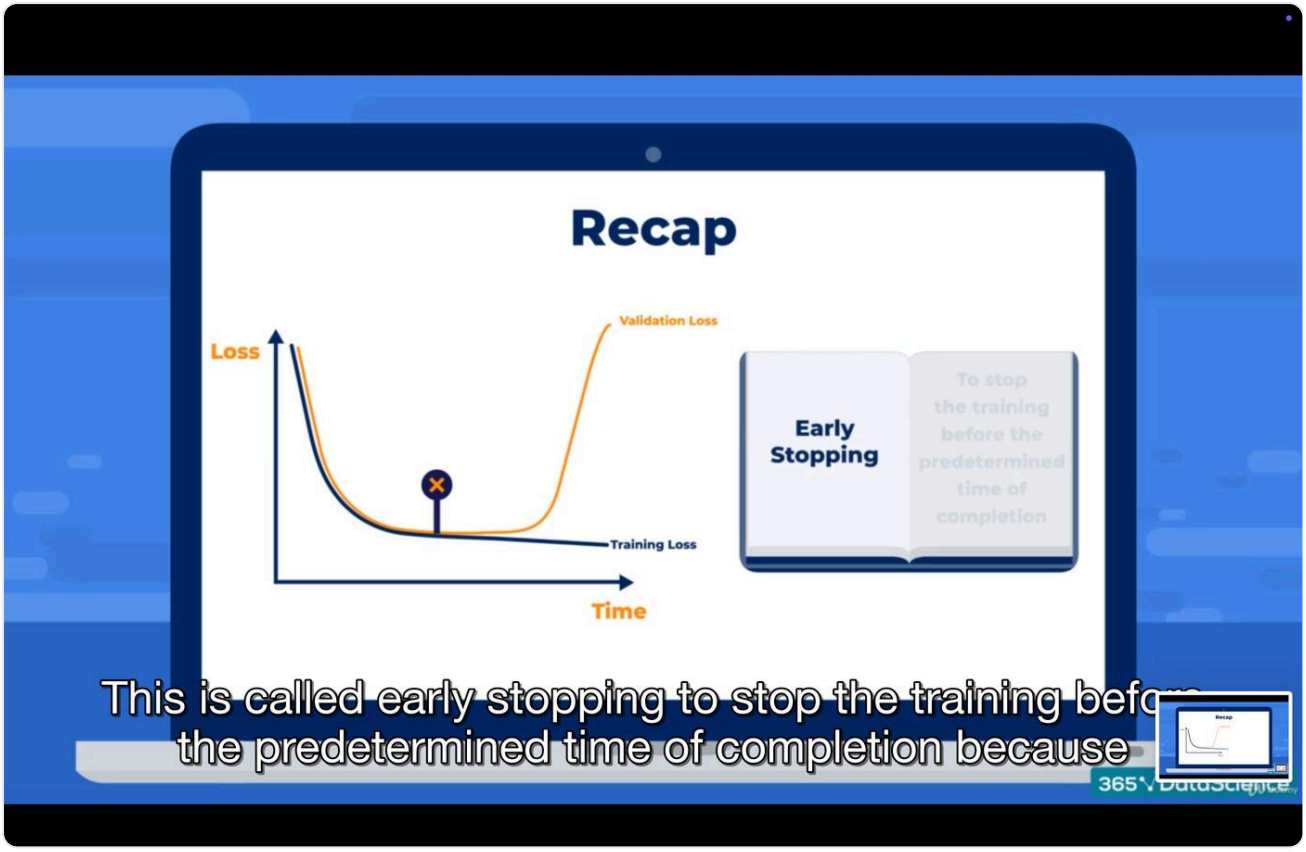

FIGURE 13.16

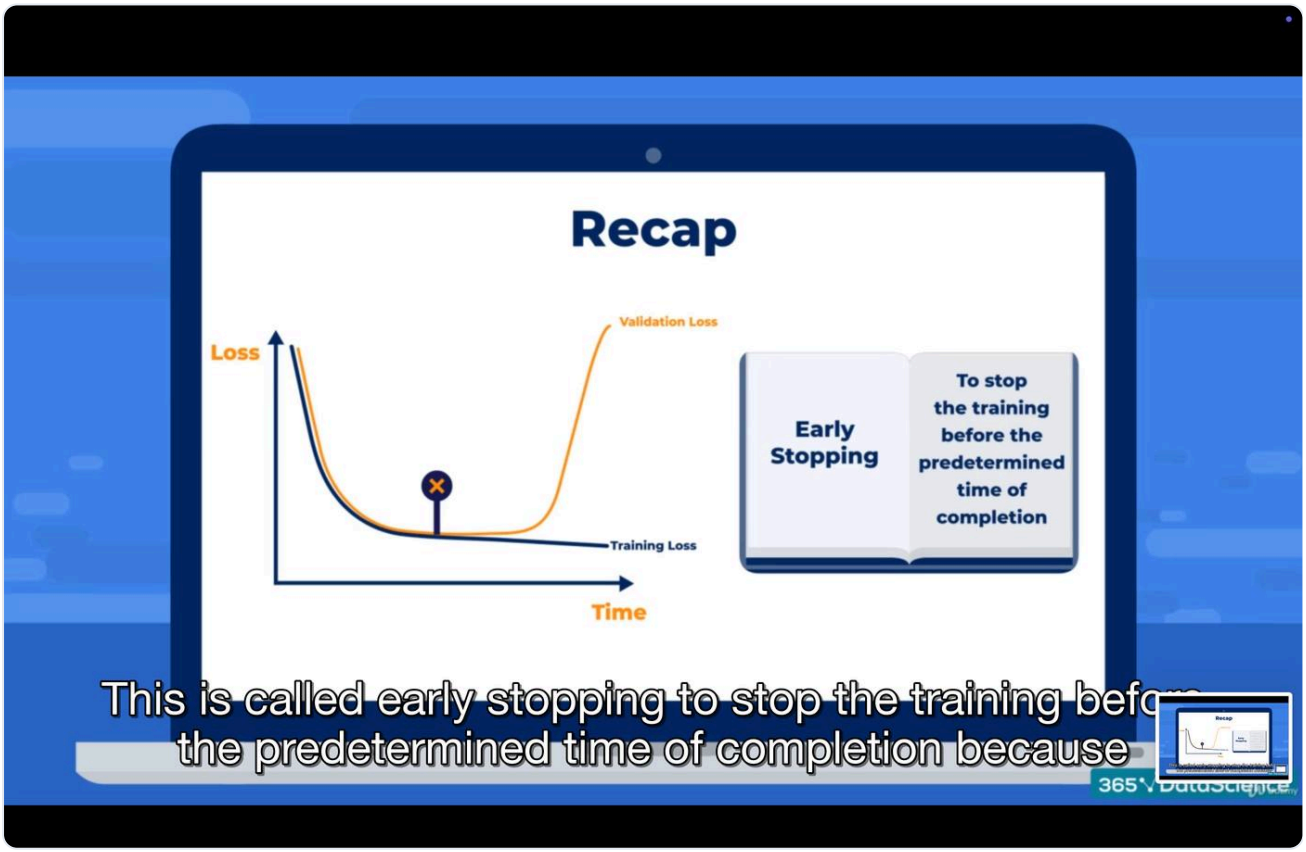

FIGURE 13.17

This is called early stopping to stop the training before the predetermined time of completion because we have detected overfitting great. We've covered activation functions, overfitting and early stopping. In the next video, we will quickly get over the optimizers. Thanks for watching.

LECTURE 14

Optimization Algorithms

In this video, we will talk a bit about the optimizers role in neural networks and their different variants as evident in the name machine learning is a process in which the computer gradually learns what parameters or weights fit the training data best. But how is this learning process actually implemented? Well, the answer is through optimizers.

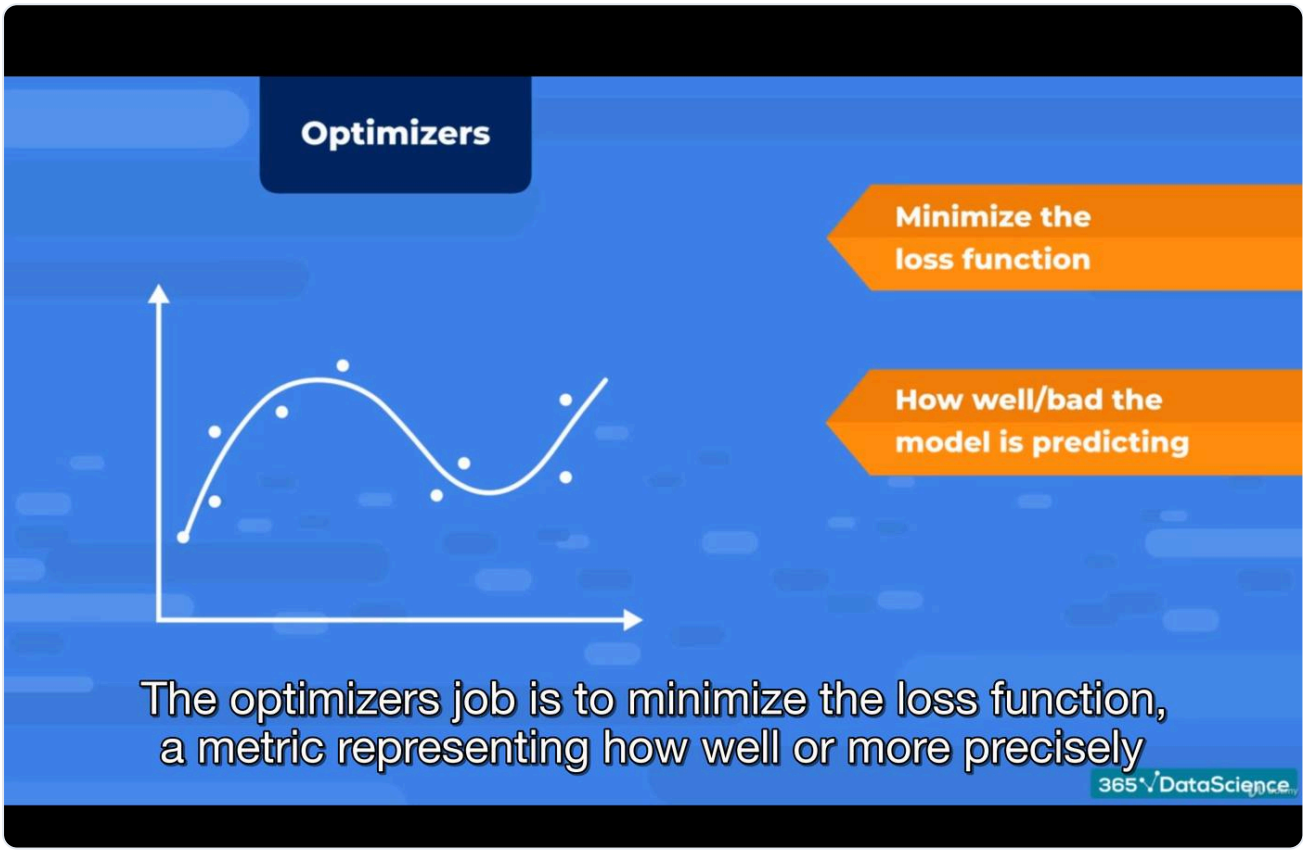

FIGURE 14.1

The optimizers job is to minimize the loss function, a metric representing how well or more precisely how bad the model is predicting.

FIGURE 14.2

The lower the loss, the better our algorithm is behaving. Thus, most optimizers center around lowering this loss at every epoch. The simplest yet highly effective optimization algorithm of the kind is called gradient descent. Its function is to always change the weights a little bit in the direction in which the losses optimize the most. This direction is obtained by calculating the gradient, hence the name of this approach.

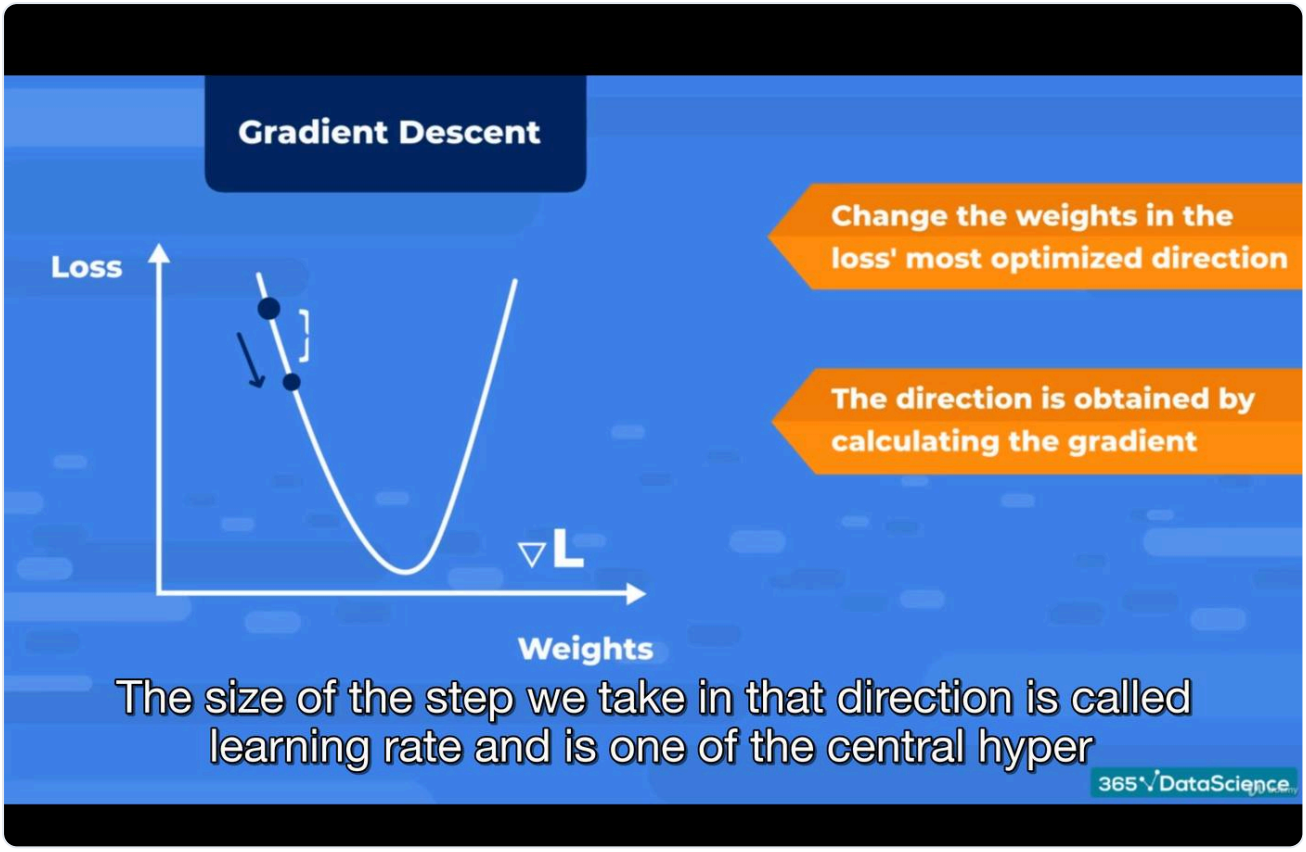

FIGURE 14.3 | Gradient Descent

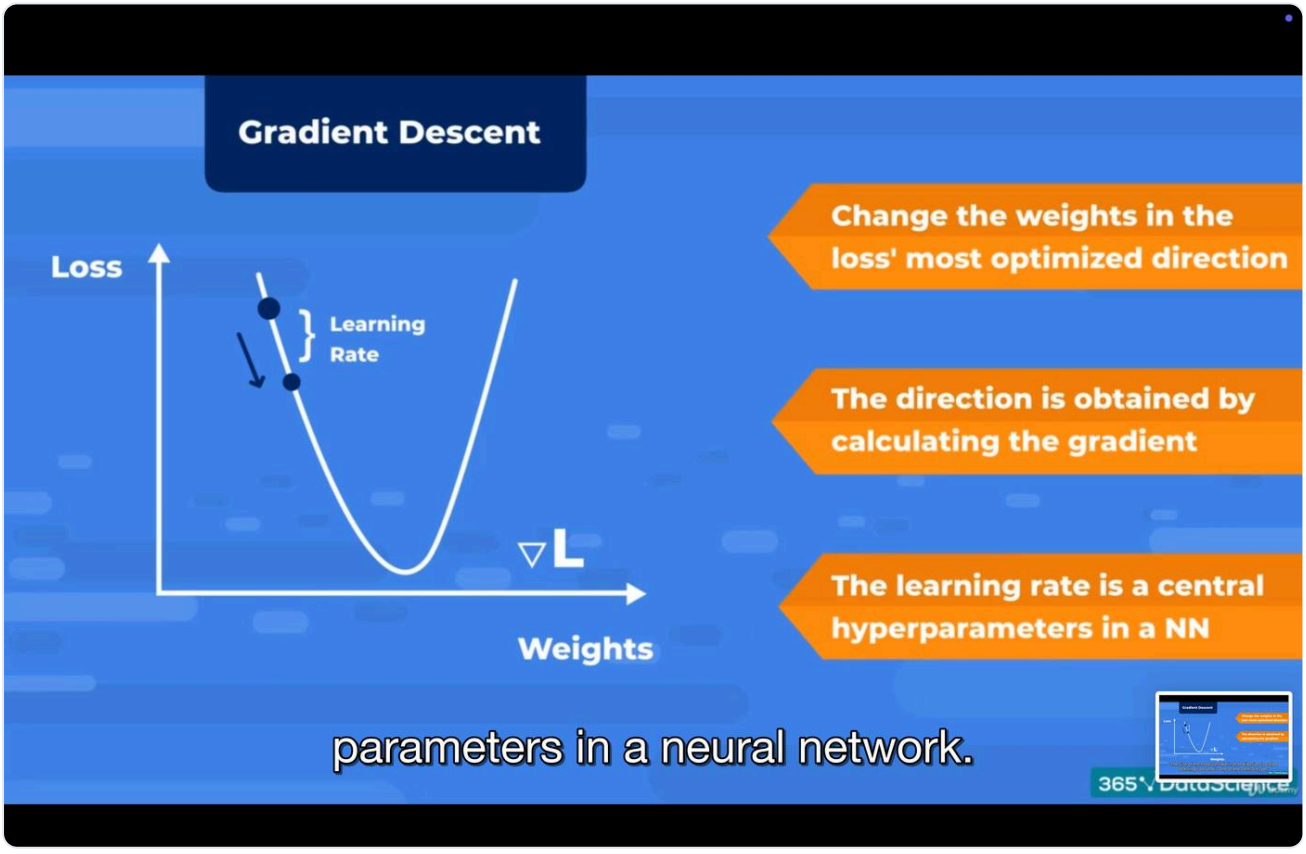

FIGURE 14.4 | Gradient Descent

The size of the step we take in that direction is called learning rate and is one of the central hyper parameters in a neural network.

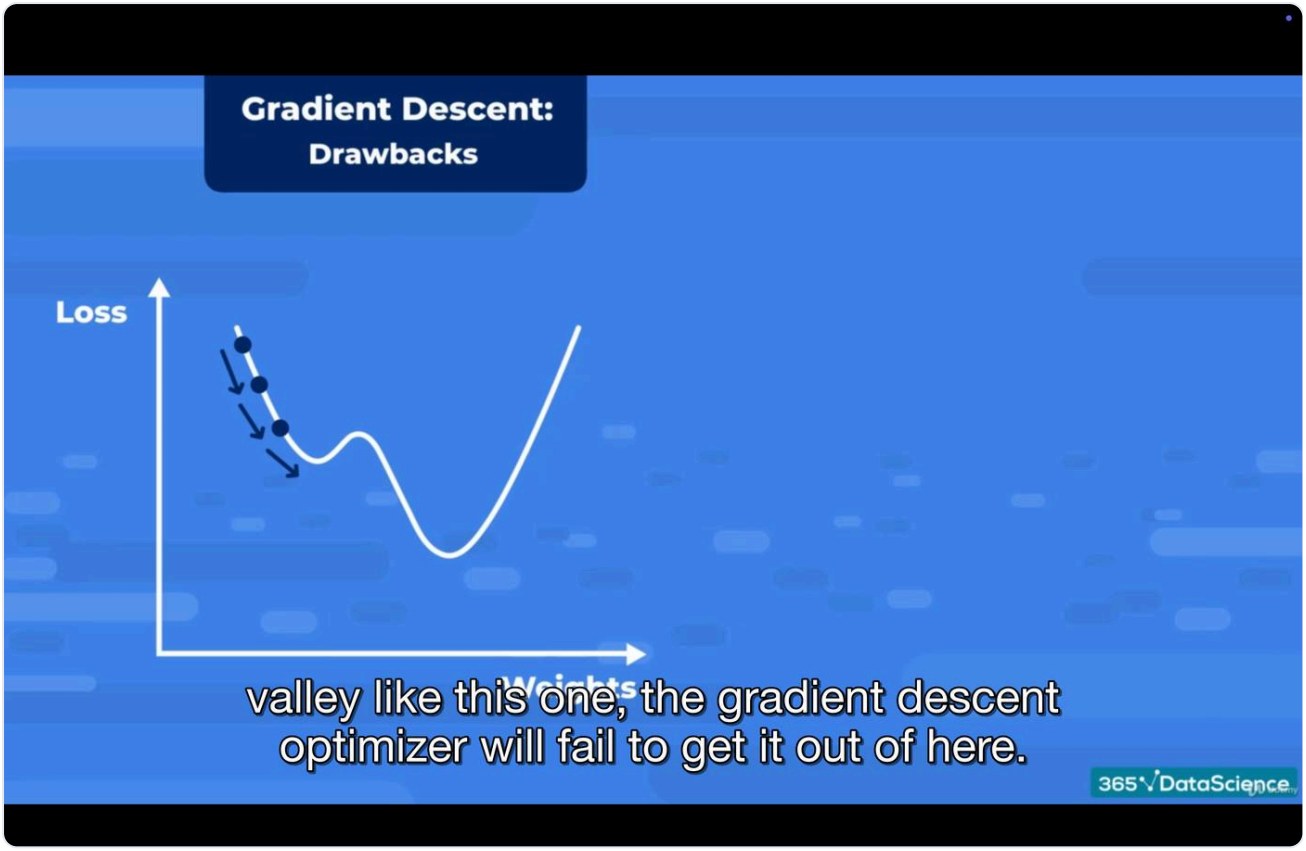

FIGURE 14.5

OK, this method, however, has obvious drawbacks, for example, if the network finds itself in a valley like this one, the gradient descent optimizer will fail to get it out of here.

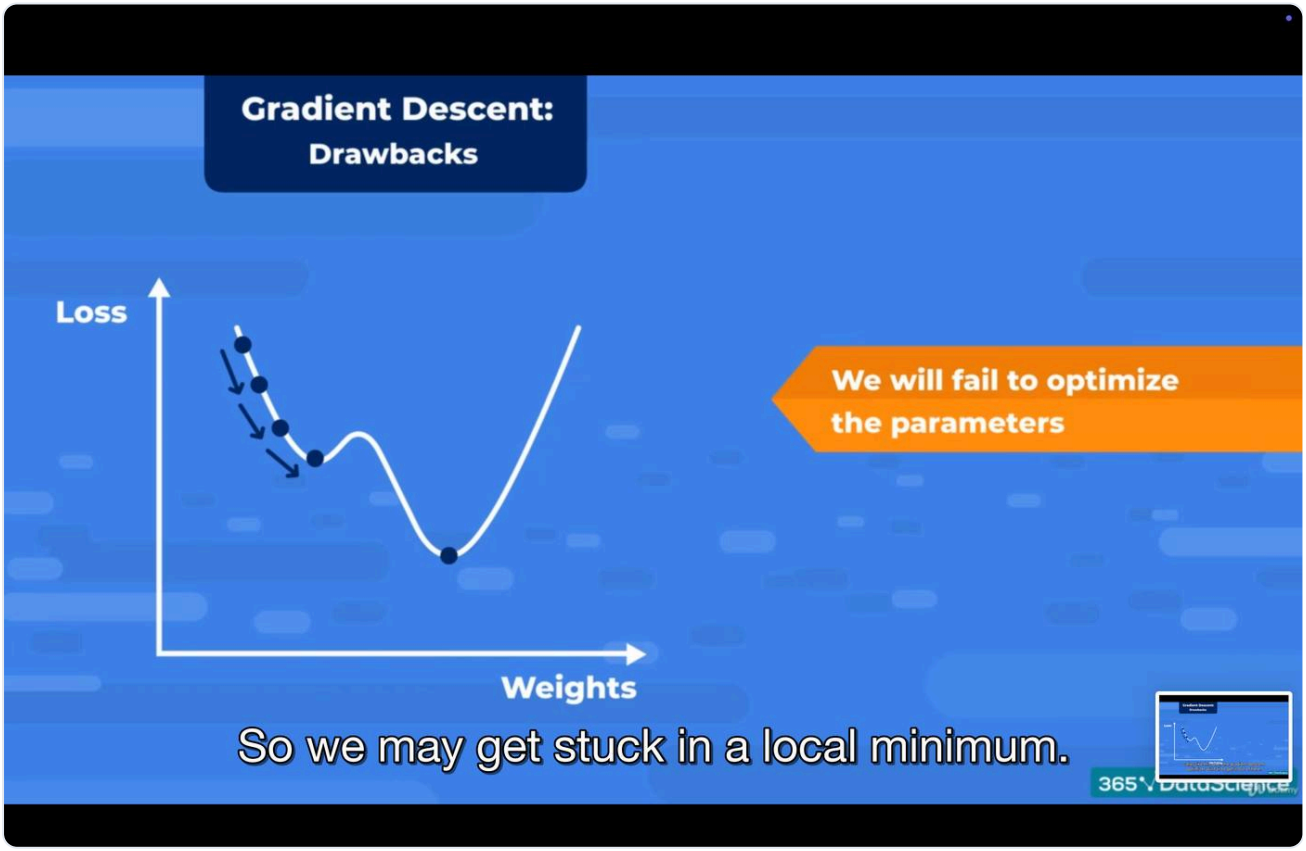

FIGURE 14.6 | Gradient Descent:

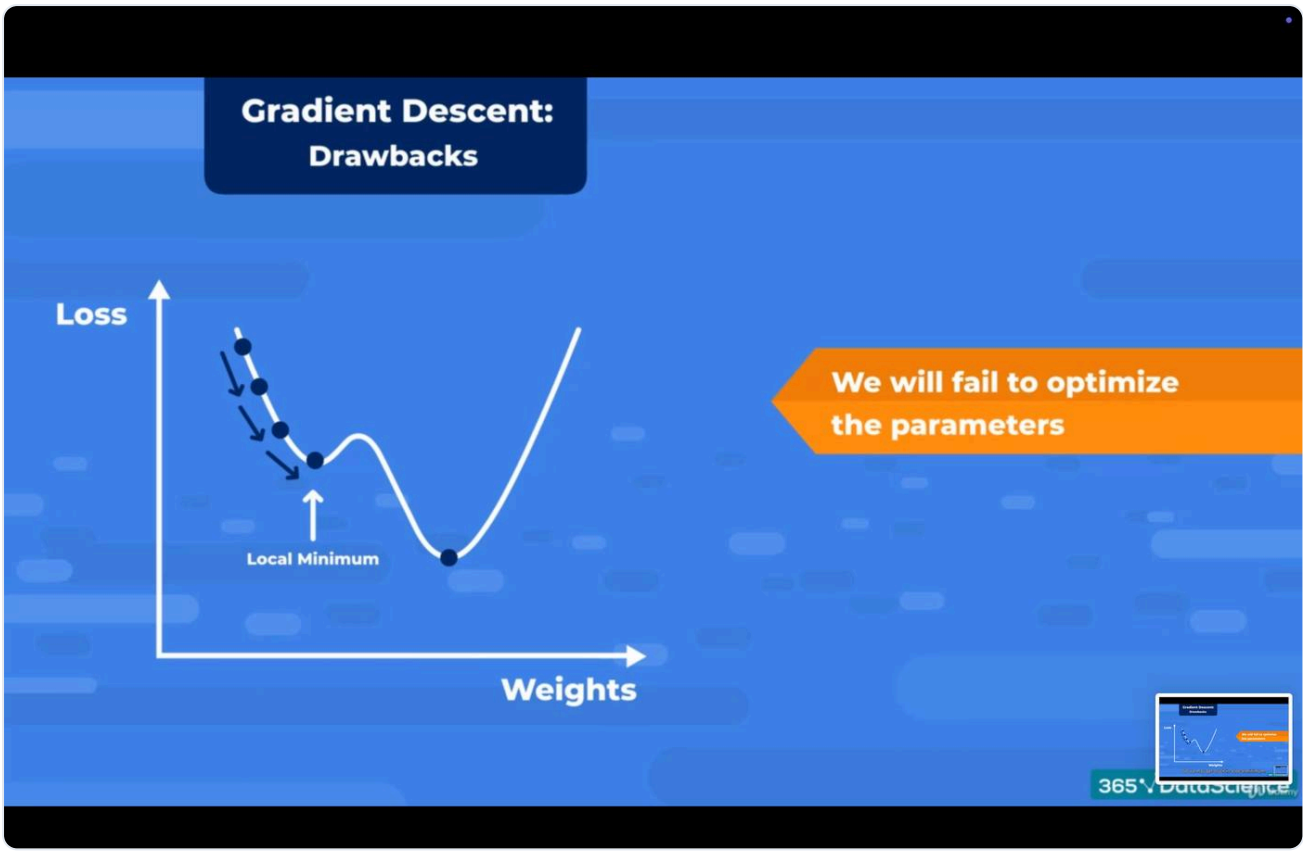

FIGURE 14.7 | Gradient Descent:

So we may get stuck in a local minimum.

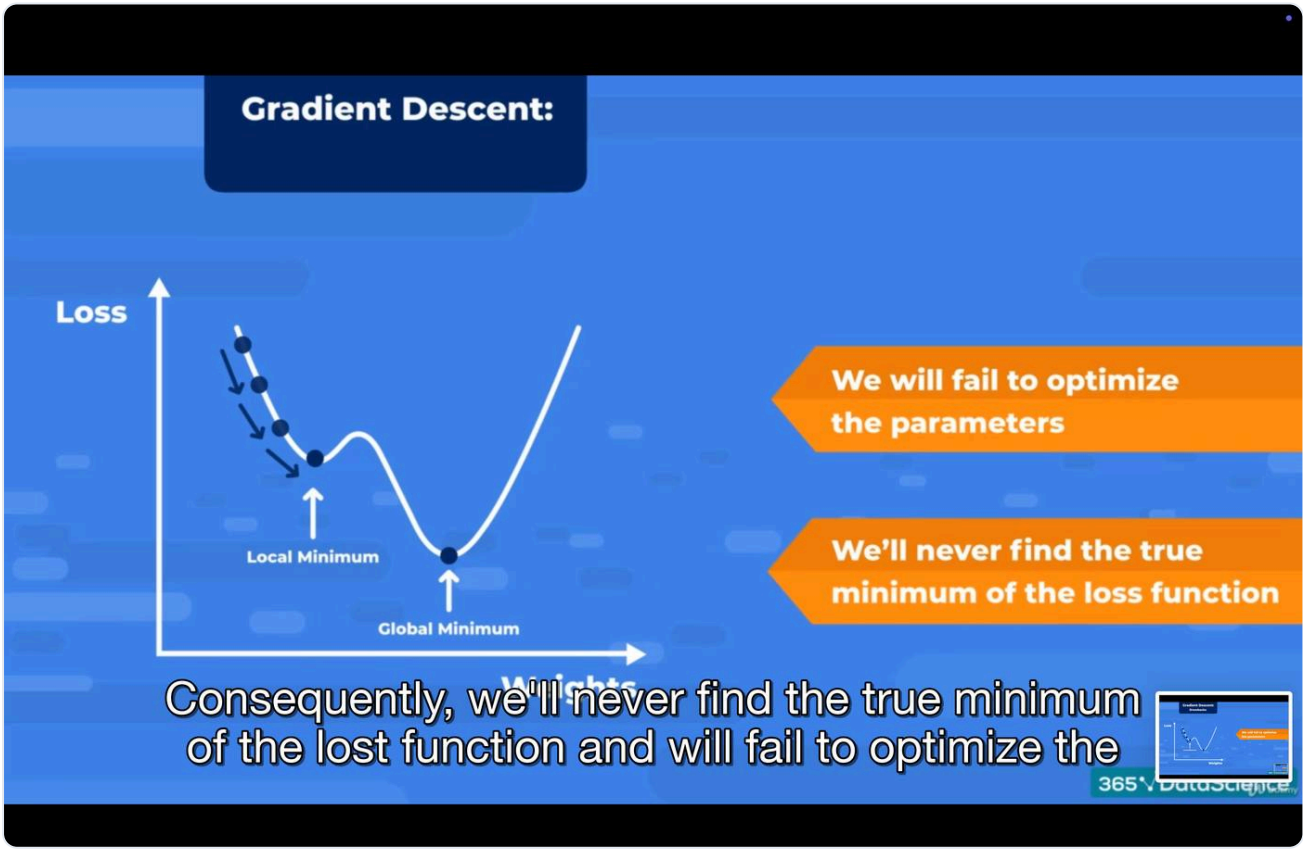

FIGURE 14.8 | Gradient Descent:

Consequently, we'll never find the true minimum of the lost function and will fail to optimize the parameters to correct. For this, we can use a modification of this optimizer called gradient descent with momentum.

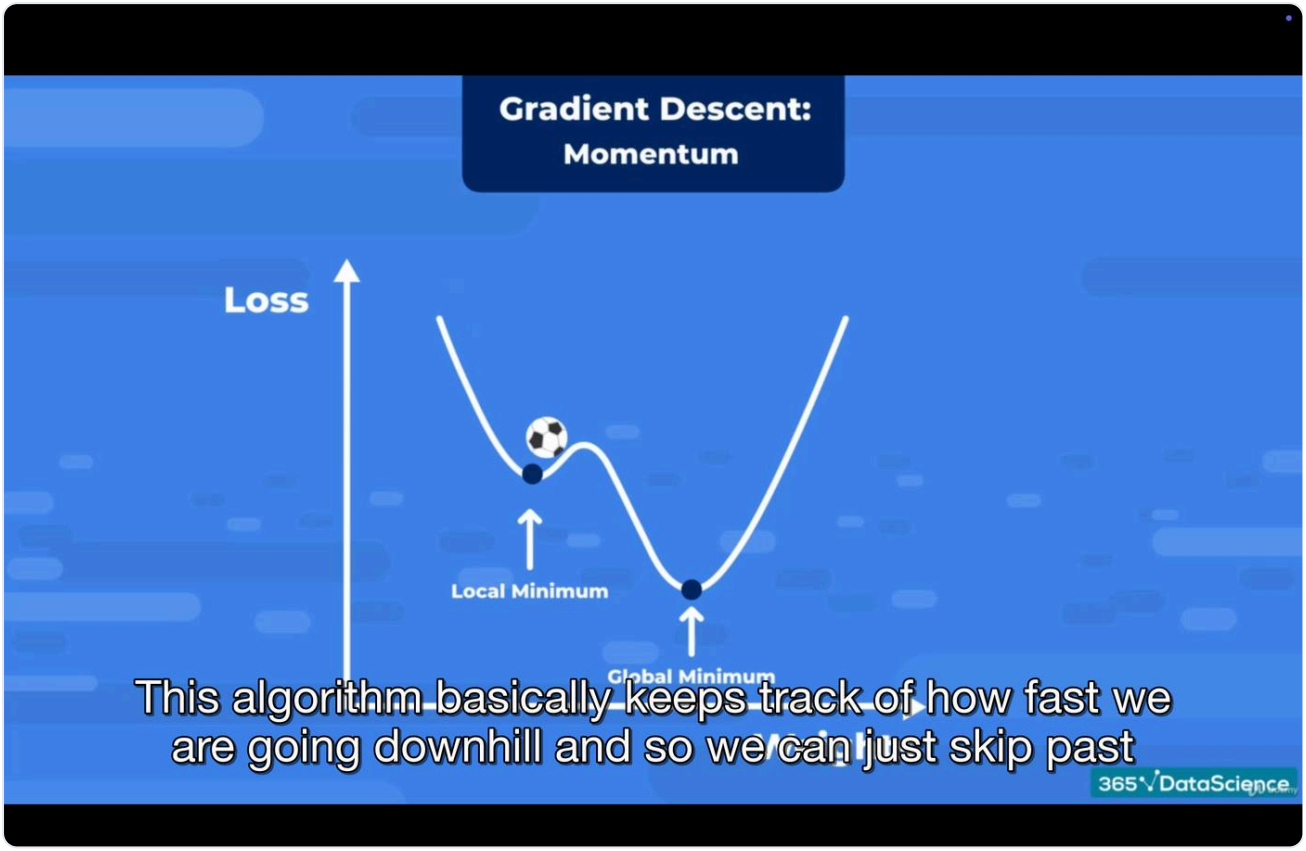

FIGURE 14.9 | Gradient Descent:

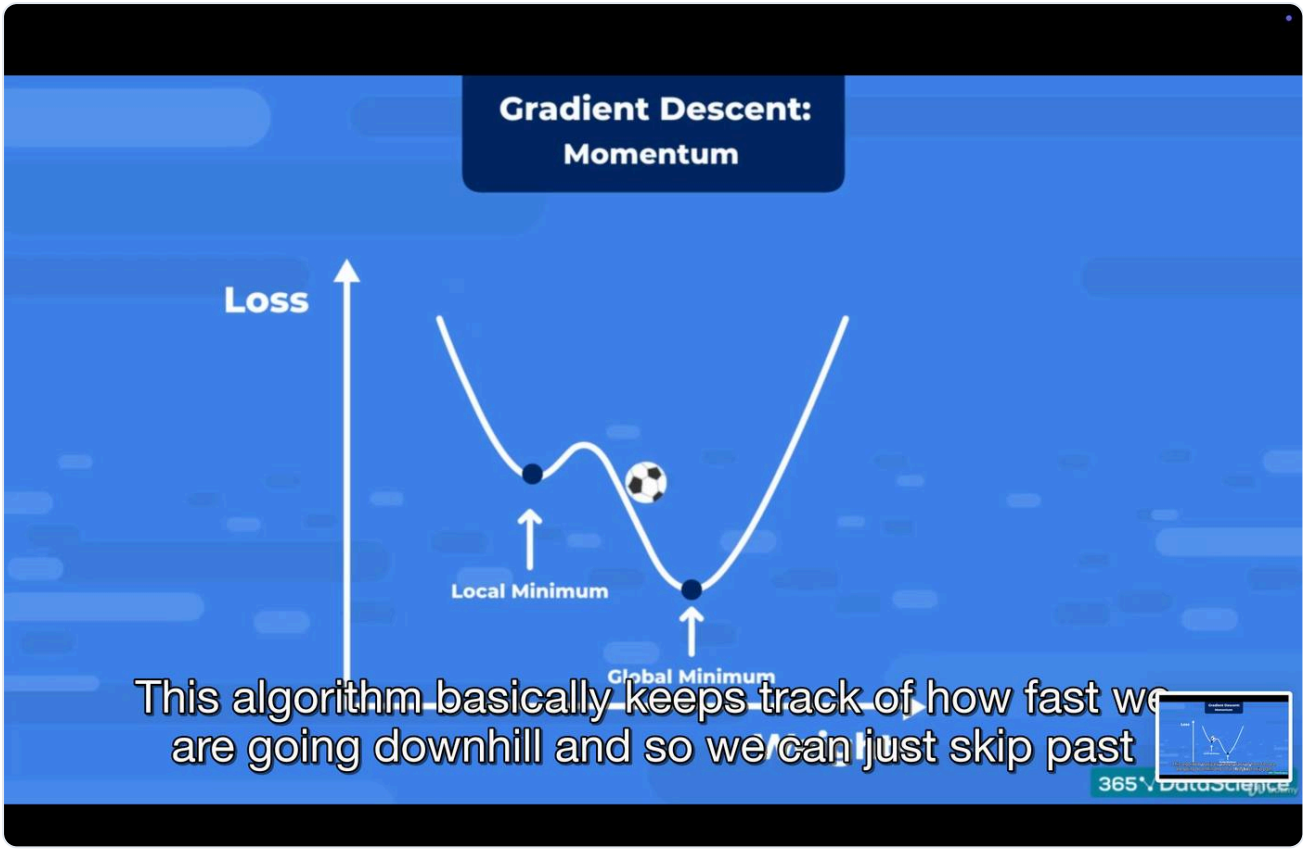

FIGURE 14.10 | Gradient Descent:

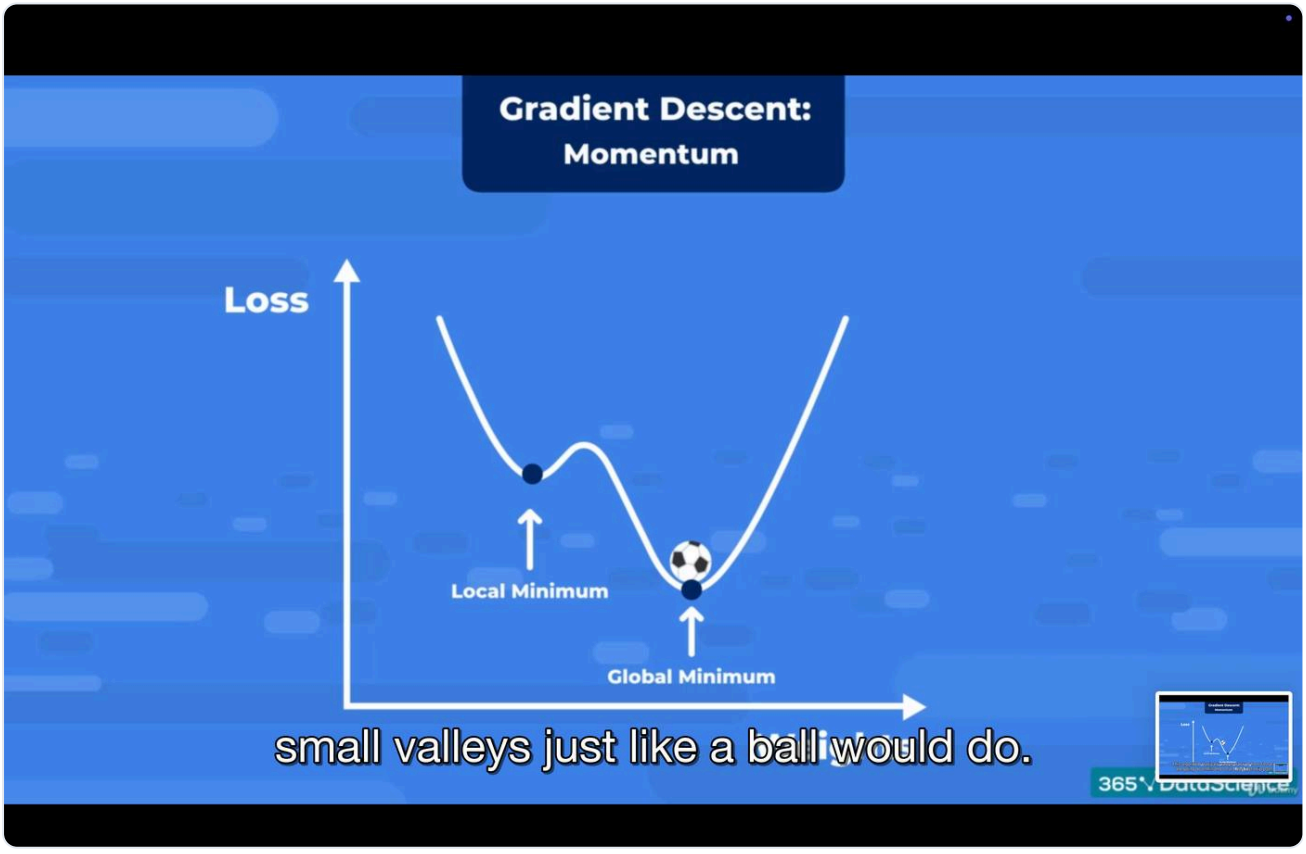

FIGURE 14.11 | Gradient Descent:

This algorithm basically keeps track of how fast we are going downhill and so we can just skip past small valleys just like a ball would do. Great, as you can imagine, this is not the only issue computer scientists have kept working to refine.

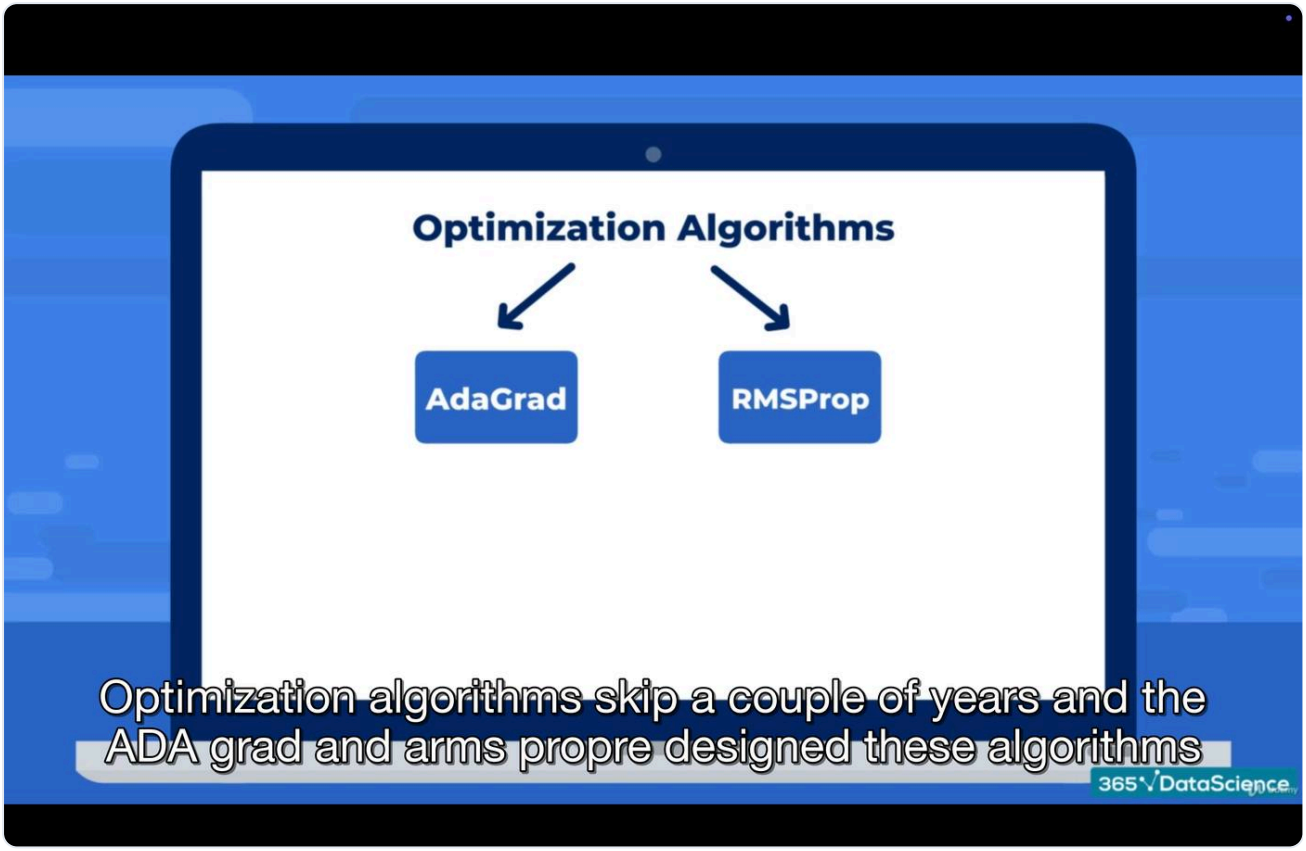

FIGURE 14.12 | Optimization Algorithms

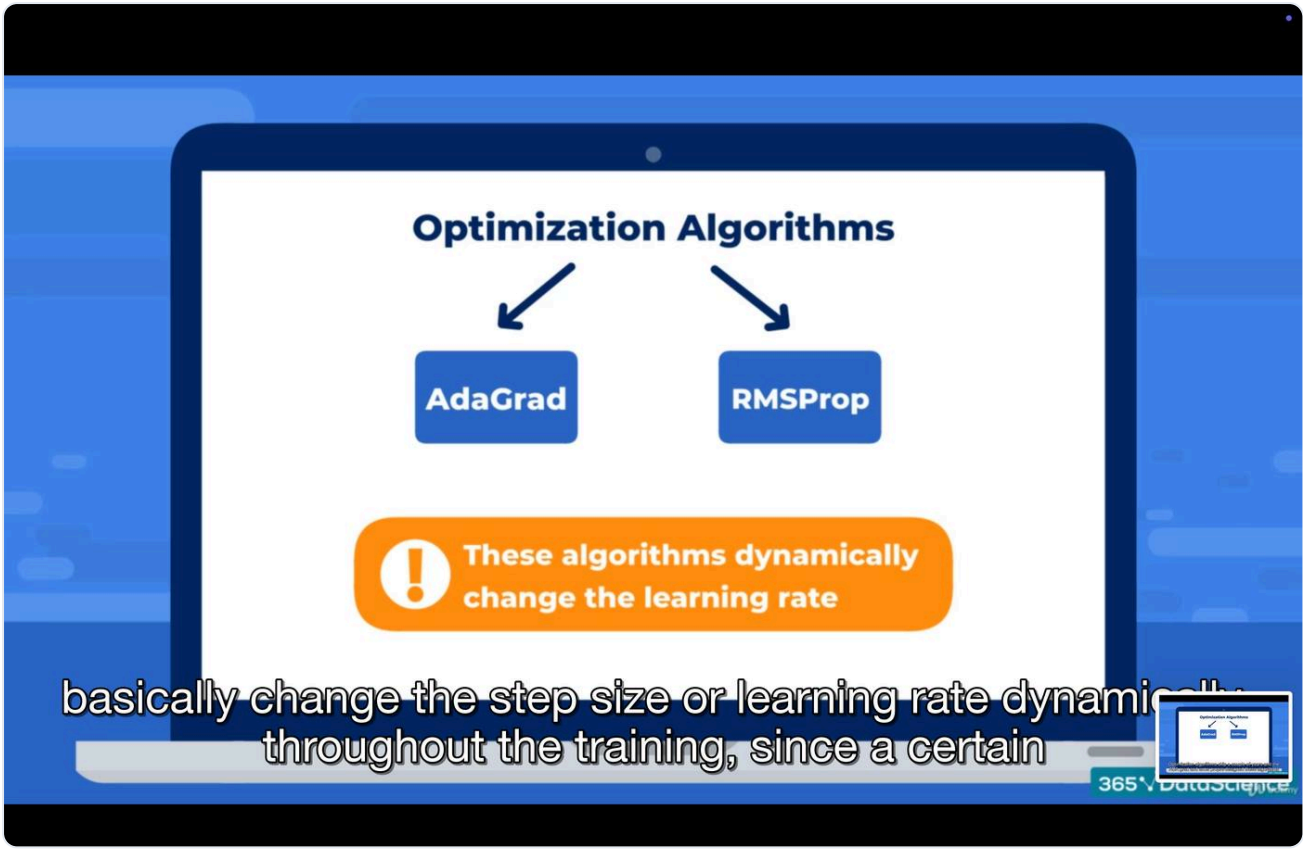

FIGURE 14.13 | Optimization Algorithms

Optimization algorithms skip a couple of years and the ADA grad and RMSprop designed these algorithms basically change the step size or learning rate dynamically throughout the training, since a certain learning rate might be good in the beginning, but not optimal at the end of the training process. Cool.

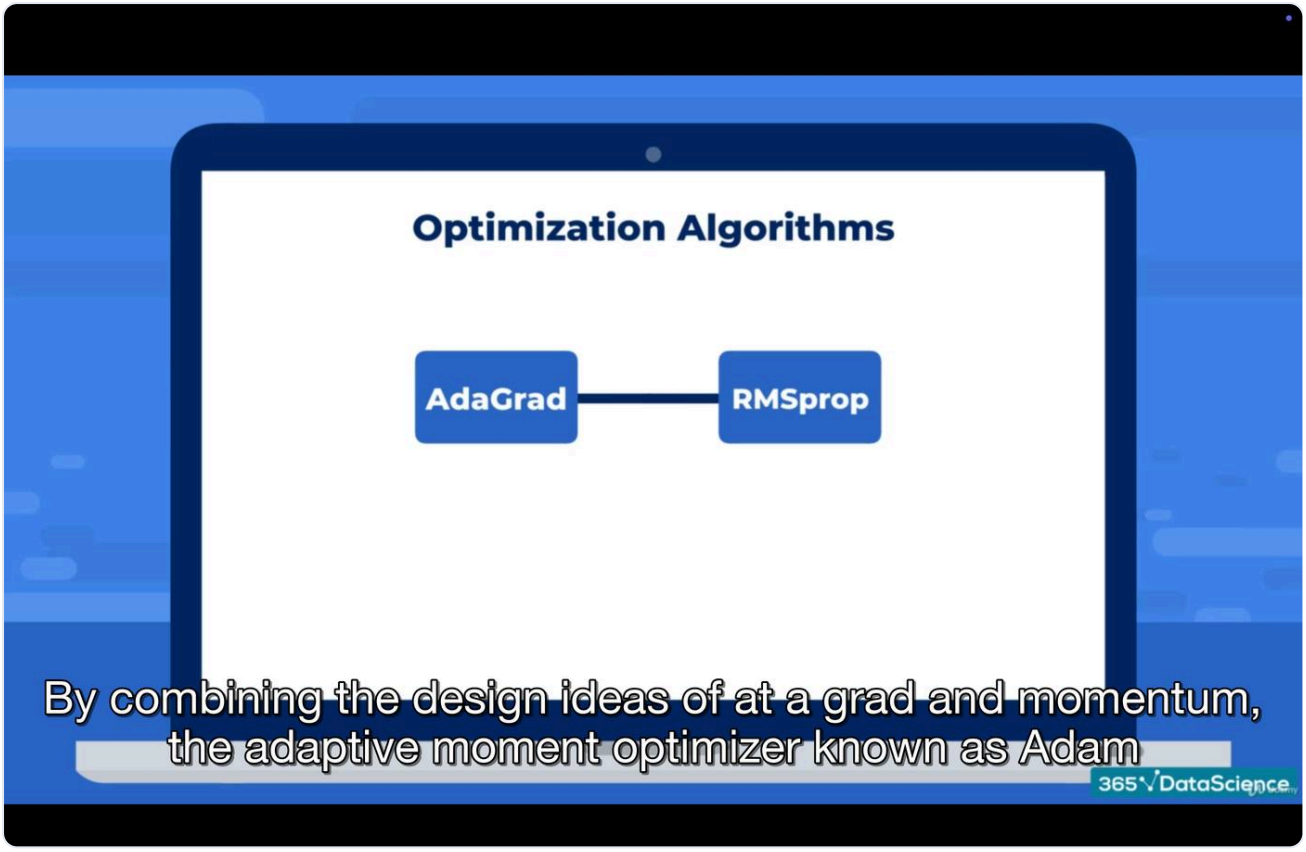

FIGURE 14.14 | Optimization Algorithms

By combining the design ideas of at a grad and momentum, the adaptive moment optimizer known as Adam was born. This optimizer is the state of the art method for estimation and is probably the only one you would need to use in your work.

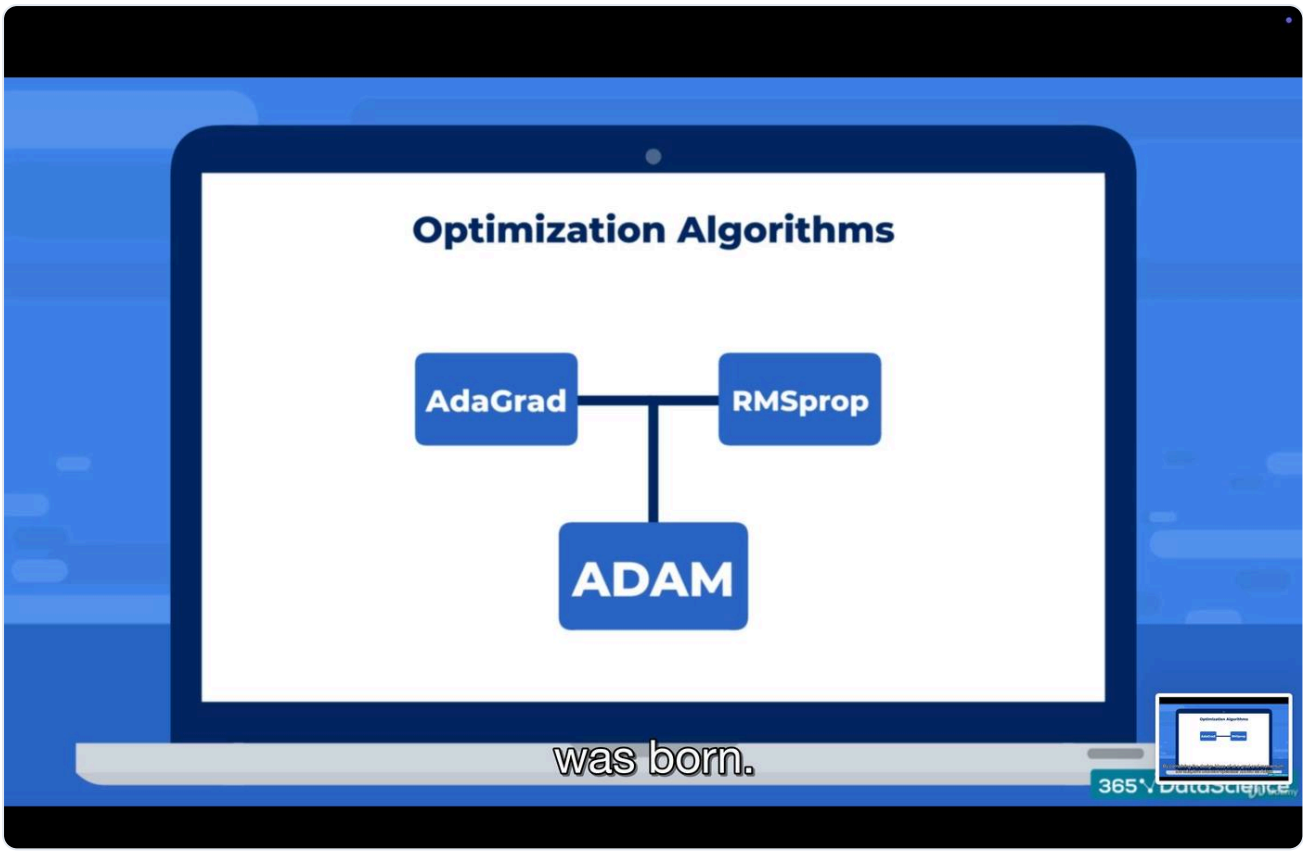

FIGURE 14.15 | Optimization Algorithms

It works well in almost all situations except for the most specific ones.

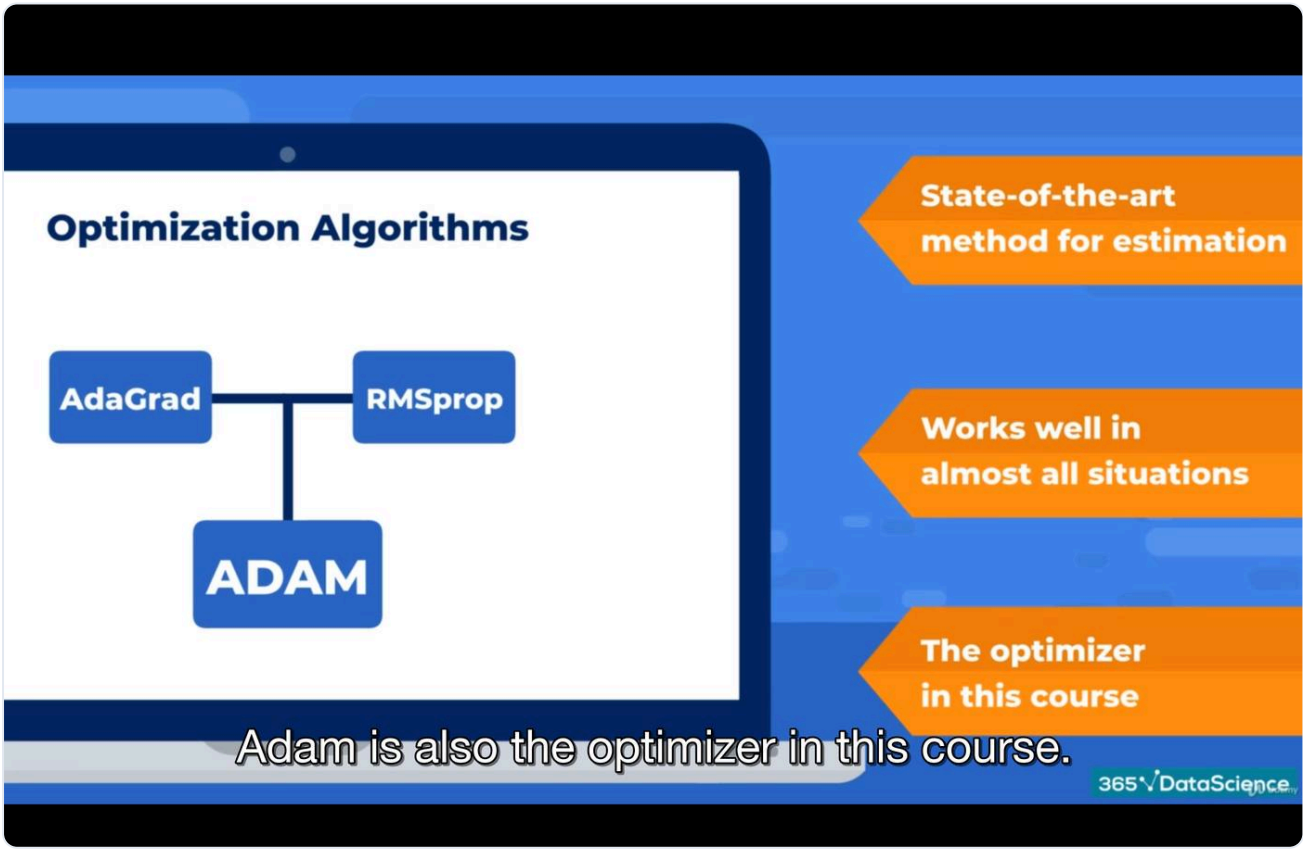

FIGURE 14.16 | Optimization Algorithms

Adam is also the optimizer in this course. All right, that's enough for the revision, it's time to get into the practical stuff in the next section, we will tackle the MNIST data set with the capabilities of convolutional neural networks. See you there. And thanks for watching.

LECTURE 15

Relevant Packages and Anaconda Python

Hey, this is a short but very important lecture, deep learning can be performed in many programming languages like C,R,Python, Java, Matlab etc. We have chosen Python for several reasons, all of which we will explore in the next lesson. Now, in order to continue with the course, you will need to install Anaconda, Python, three Jupyter Notebook and the relevant packages in this section. We will show you how to do that and we will help you to learn about the environment in which we will code. Even if you already have Anaconda, Python three and Jupyter installed. This is where we will show you how to install the relevant packages. So should you need just the last part?

Please refer to the last lecture of the section. If you have any technical problems during this section, don't hesitate to ask for our assistance in the Q&A section. Good luck and thanks for watching.

LECTURE 16

Julia File

Python and R are probably the two most widely used languages in data science. This course uses Python, and now we will see why. There are actually very few arguments against its usage. It is an open source general-purpose, high level language. I would like to quickly go over these points. Open source means that it's free to use and people can actually contribute to it. For example, the scikit-learn package that we will use extensively started as a Google summer of code project, but developed in one of the most popular free machine learning packages. If Python wasn't an open source framework, then that wouldn't have happened. Moreover, it is available for use in all major operating systems Windows, Mac and Linux.

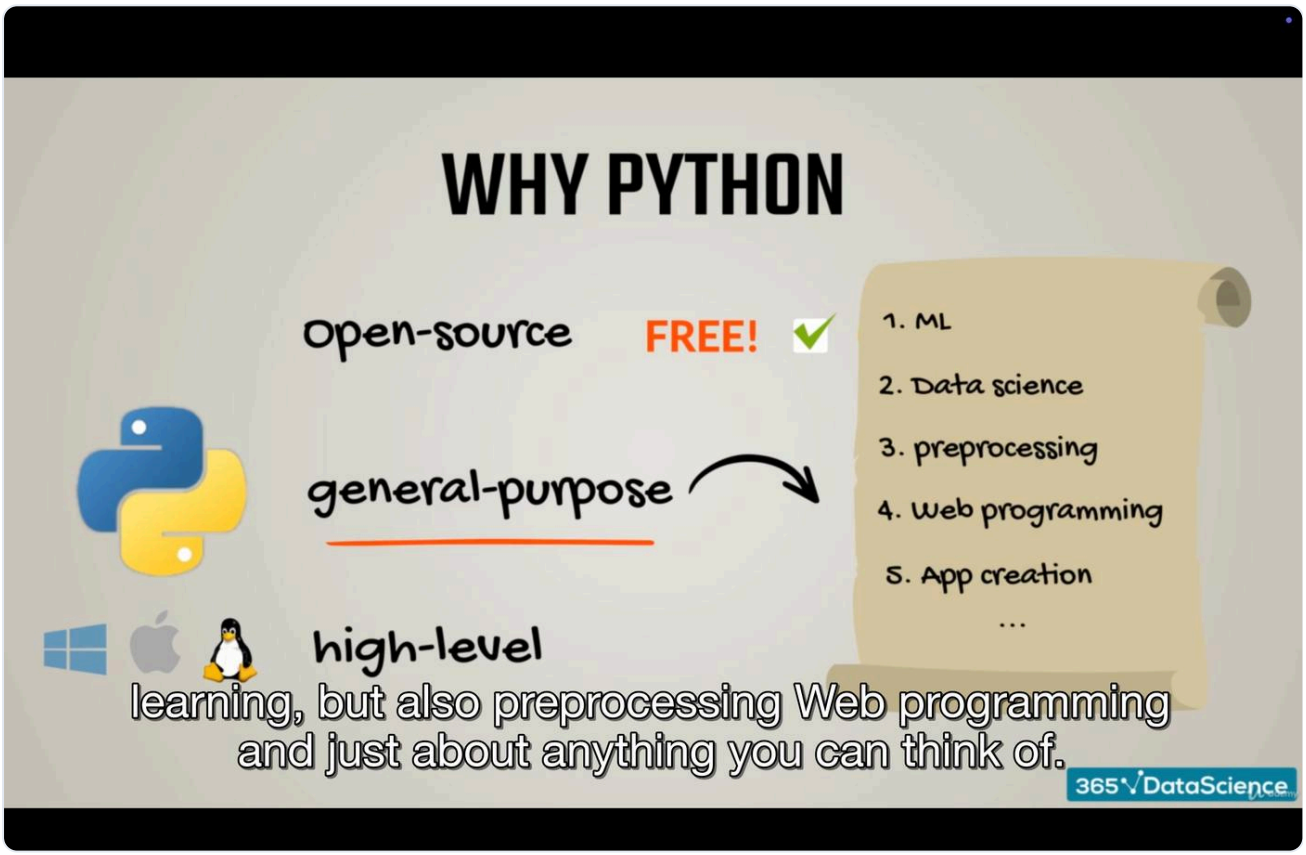

FIGURE 16.1 | 2. Data Science

General-Purpose shows that it is suitable for all types of needs, not only data science and machine learning, but also preprocessing Web programming and just about anything you can think of. We will focus on its analytical part.

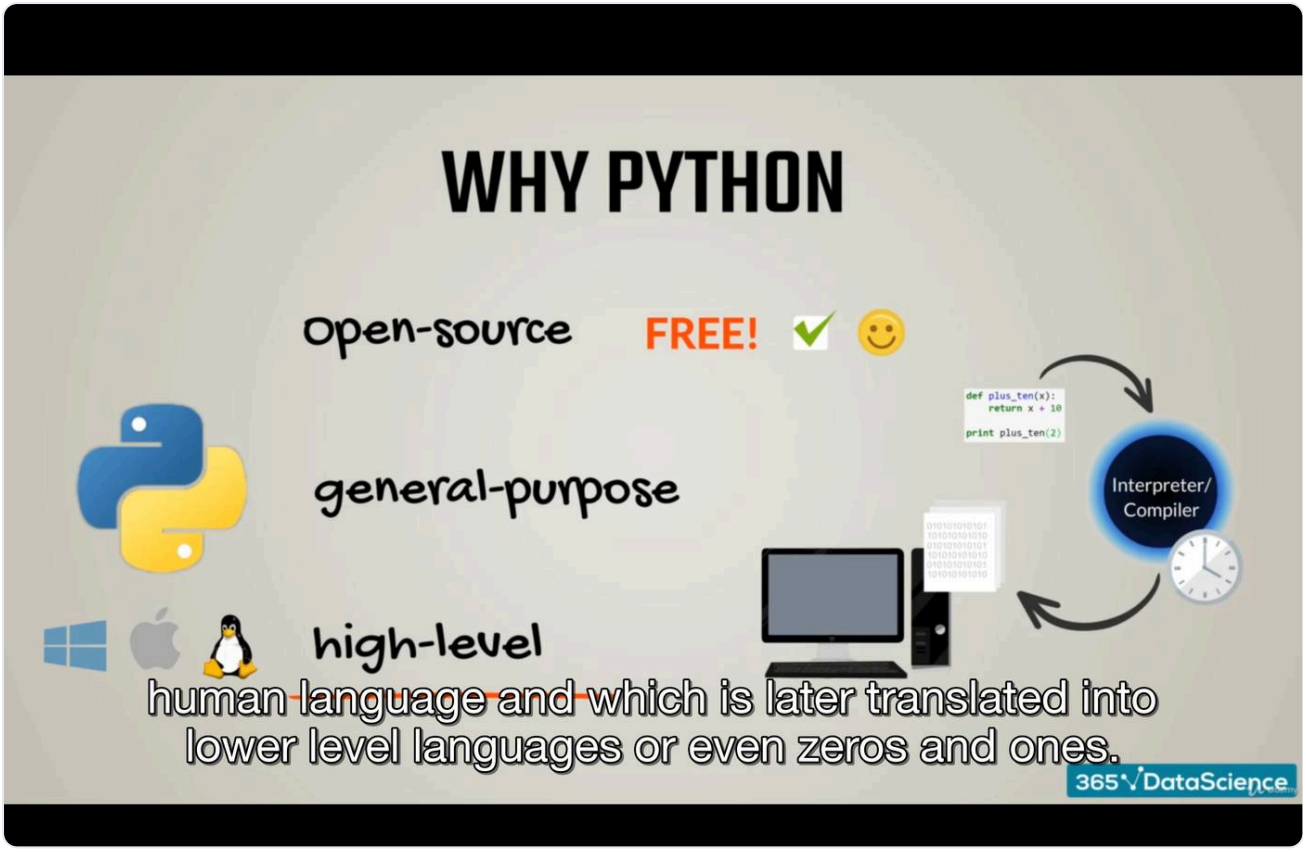

FIGURE 16.2

That's a bit more technical, but loosely explained it as an easy syntax, which is close to logical human language and which is later translated into lower level languages or even zeros and ones. For example, the NumPy package actually goes to the programming language, see, and that's why it is so fast.

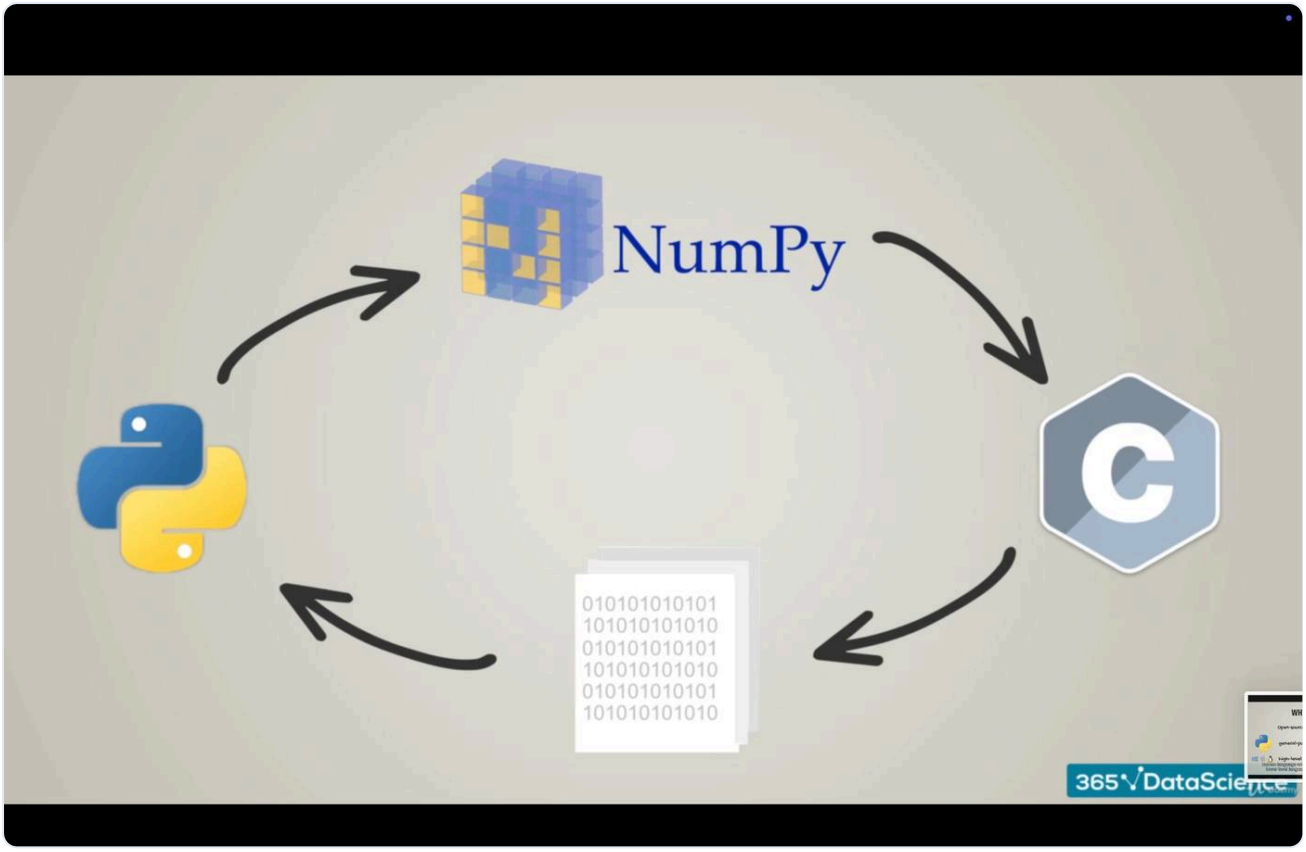

FIGURE 16.3

That said, we have to know that Python isn't among the fastest programming languages, however, it is very easy to learn and use and there are countless packages developed and maintained for it. OK, now we have to install Python in Jupyter, but why isn't there just one software application called Python that you can install on your computer that is automatically updated and that runs everything smoothly?

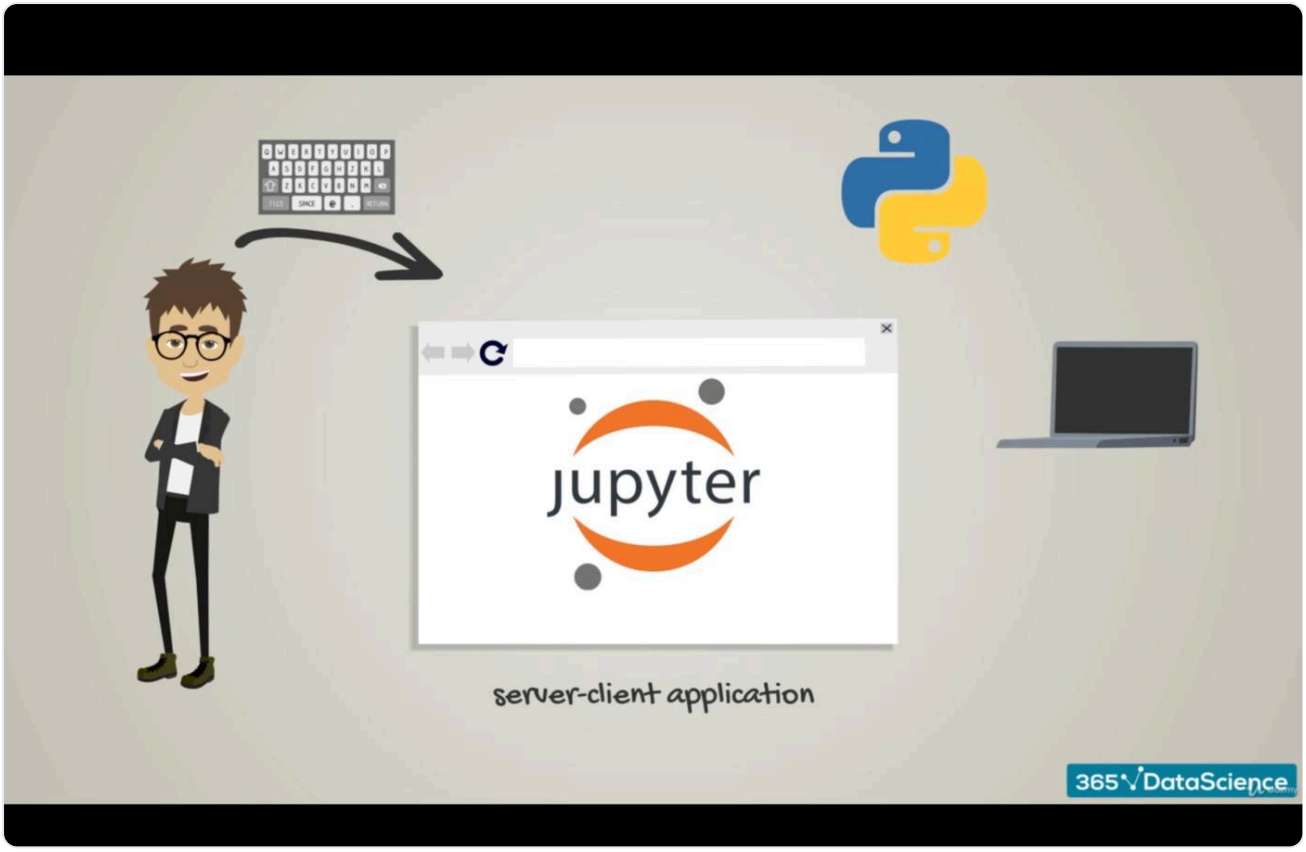

FIGURE 16.4

Well, Python is a programming language, it can allow you to communicate with the computer. To do that, you'll need the help of a specific software or an application, namely the Jupyter notebook app, which is more often called Jupyter, can help us do that.

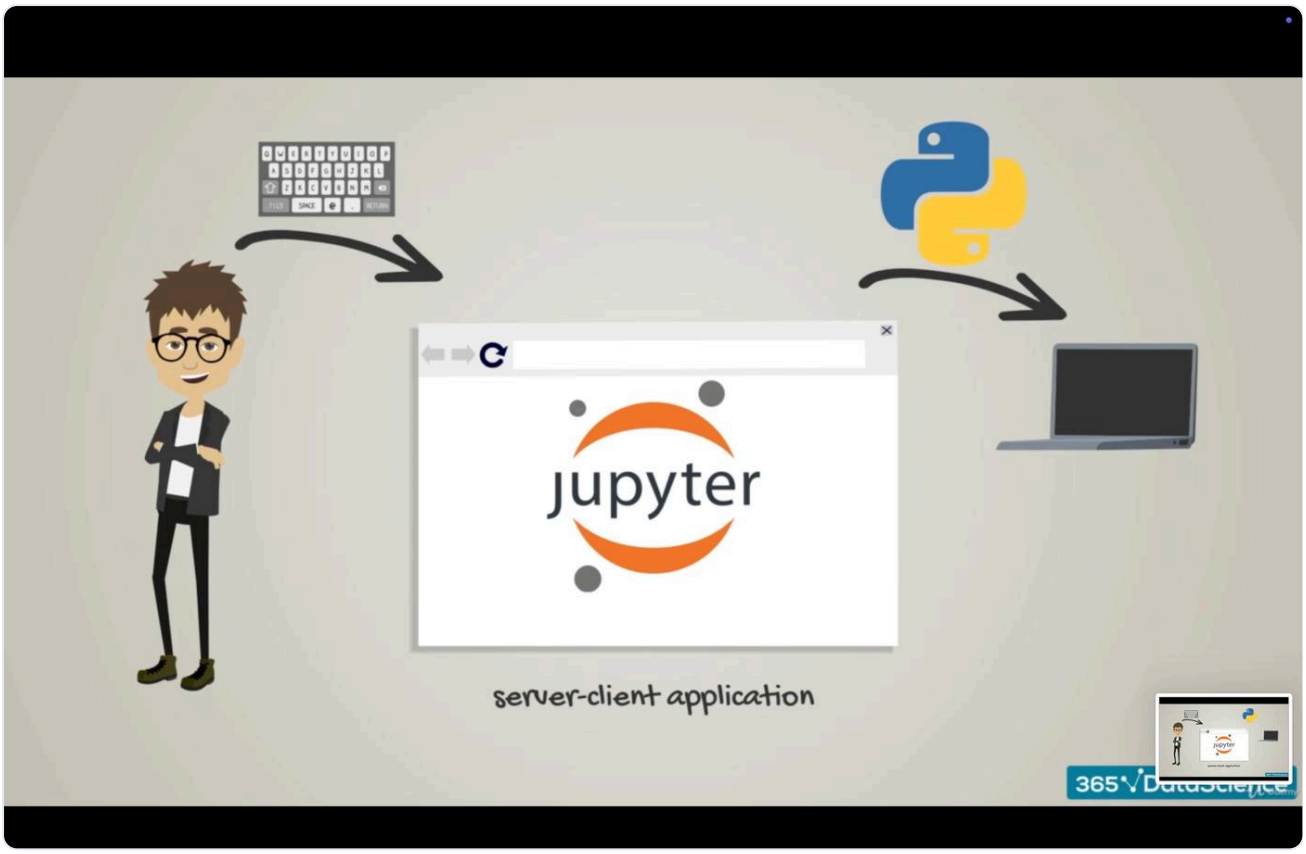

FIGURE 16.5

It is a server client application that allows you to edit your code through a Web browser. Consider the following graph.

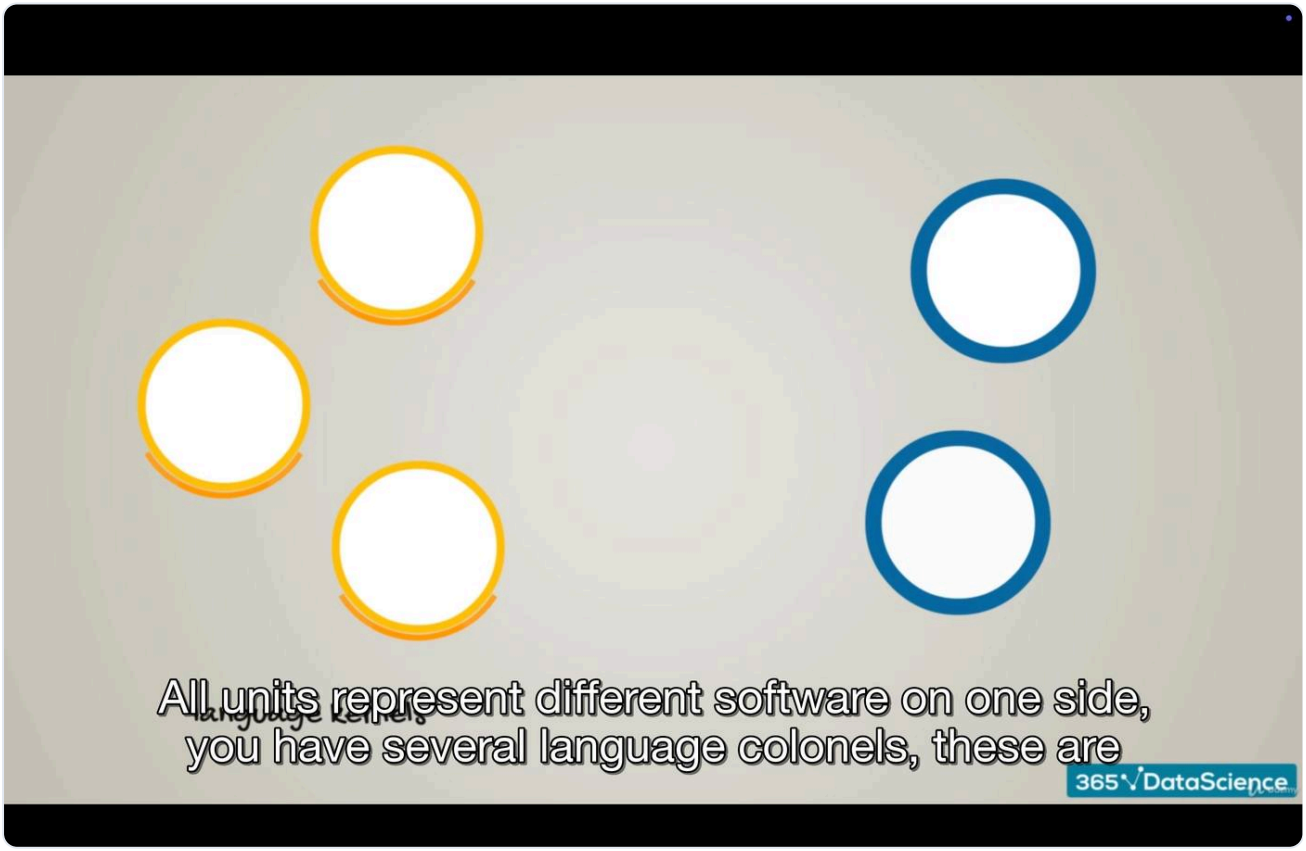

FIGURE 16.6

All units represent different software on one side, you have several language colonels, these are programs designed to read and execute code in a specific programming language like Python R or Julia. The Jupyter installation always comes with an installed python kernel and the other kernel is going to be installed additionally.

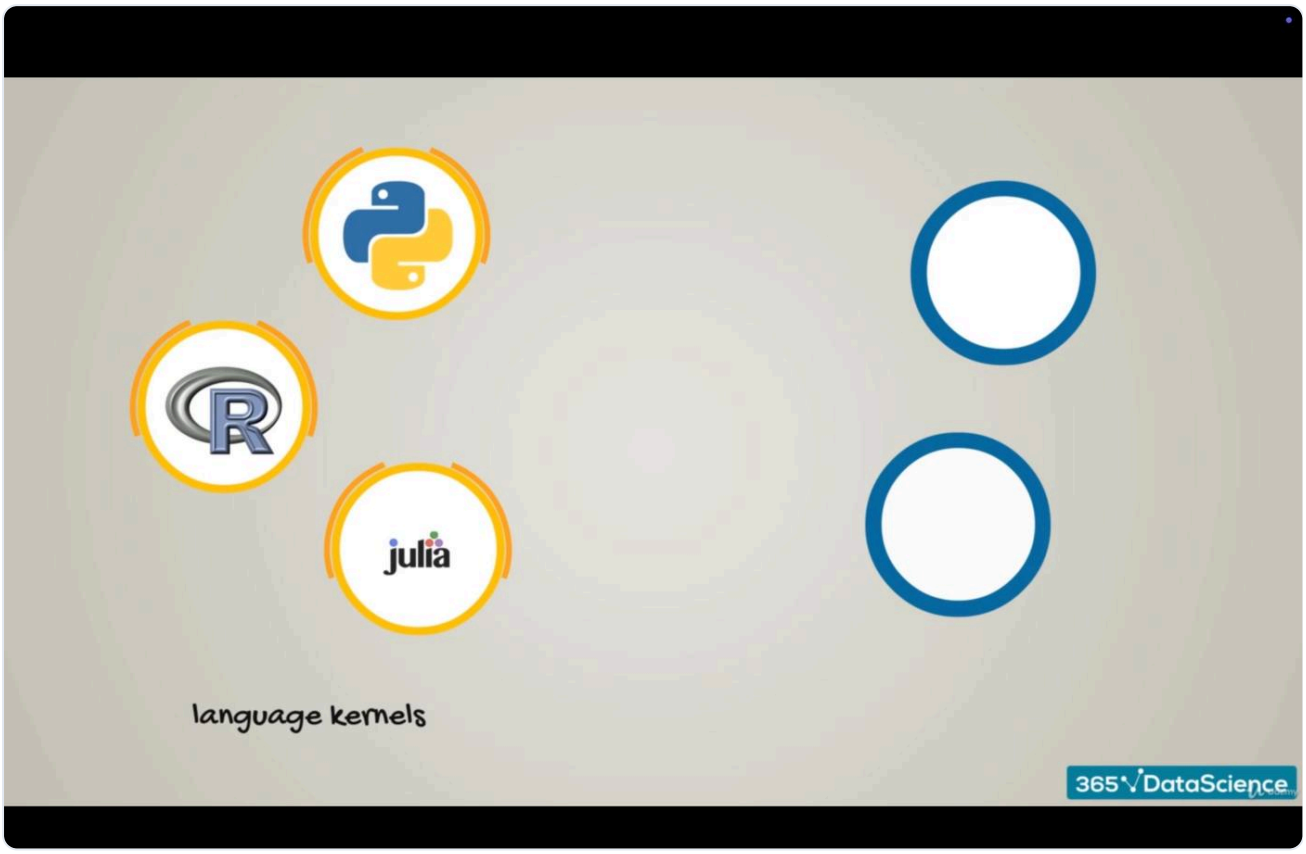

FIGURE 16.7

On the other side, you have various types of interfaces where you can write code.

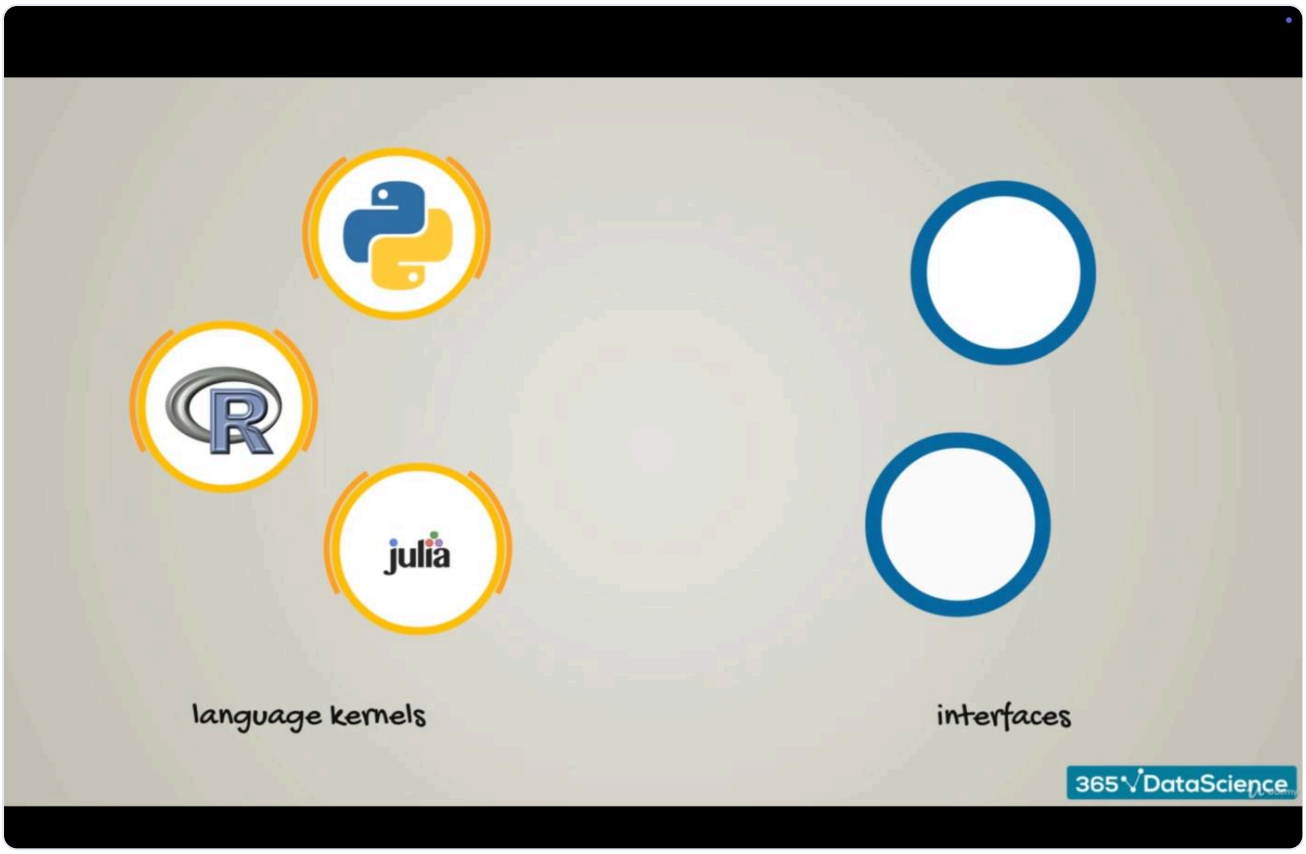

FIGURE 16.8

They represent the clients, an example of such a client is the Web browser. The Jupyter server provides the environment where a client is matched with a corresponding languages kernel.

FIGURE 16.9

In our case, we will focus on Python and a Web browser as a client or as an interactive shell.

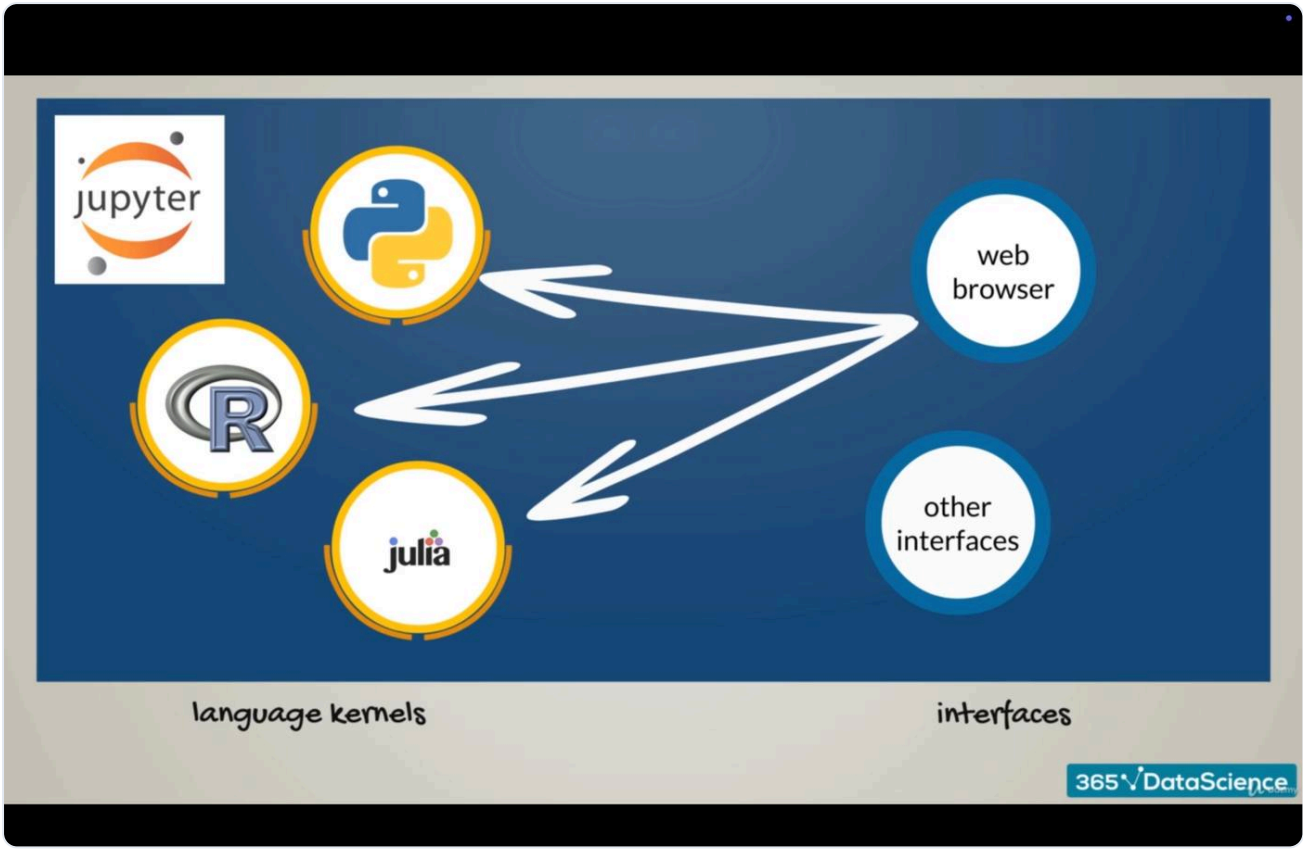

FIGURE 16.10

Your work will be stored on a notebook document and since will be strictly using the Python language. It will be called Eye Python notebook file with a file format dot IPY and be.

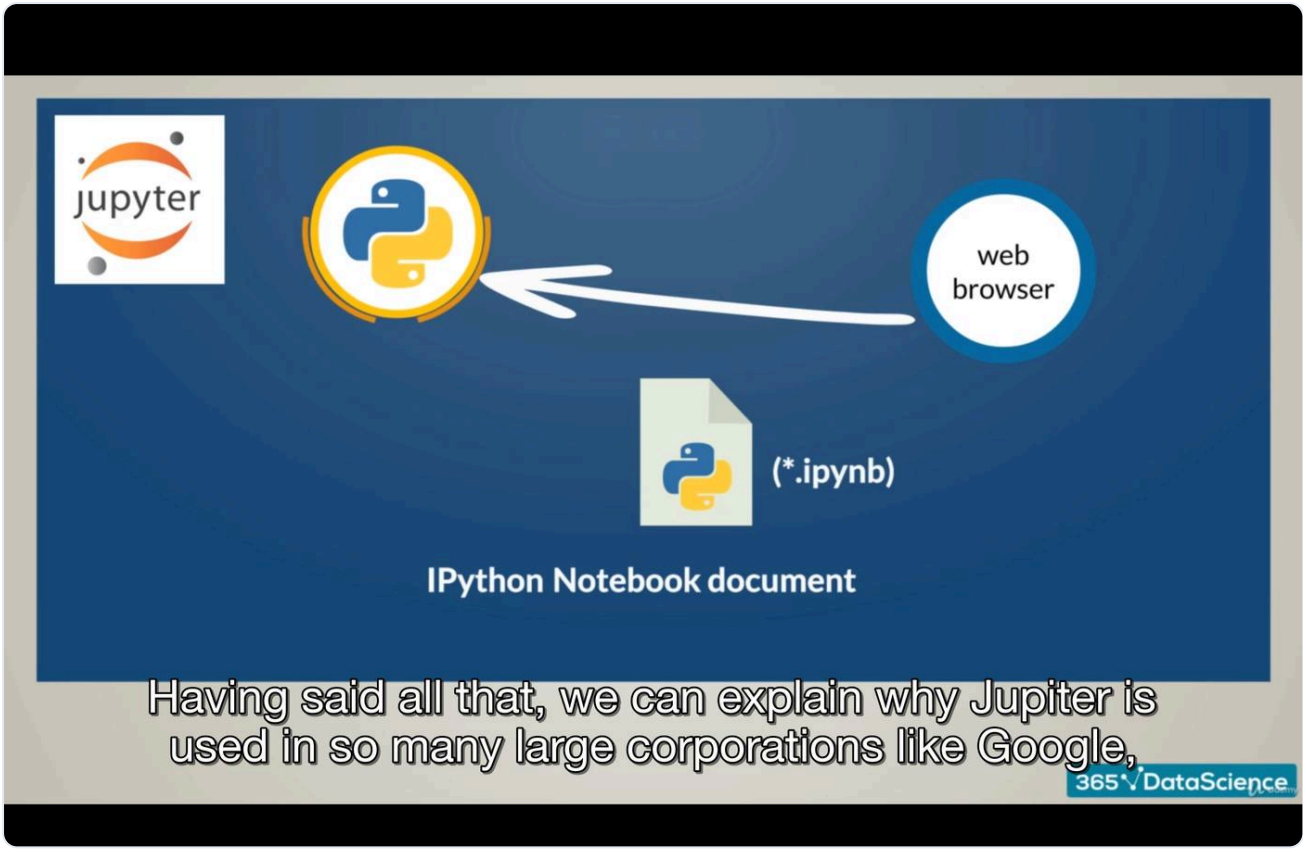

FIGURE 16.11

Having said all that, we can explain why Jupyter is used in so many large corporations like Google, Microsoft and IBM. For its design, it is well suited for demonstrations of programming concepts and training. First, in large corporations, solving a particular task could require coding in a few languages, say Python, Ah Julia or PHP.

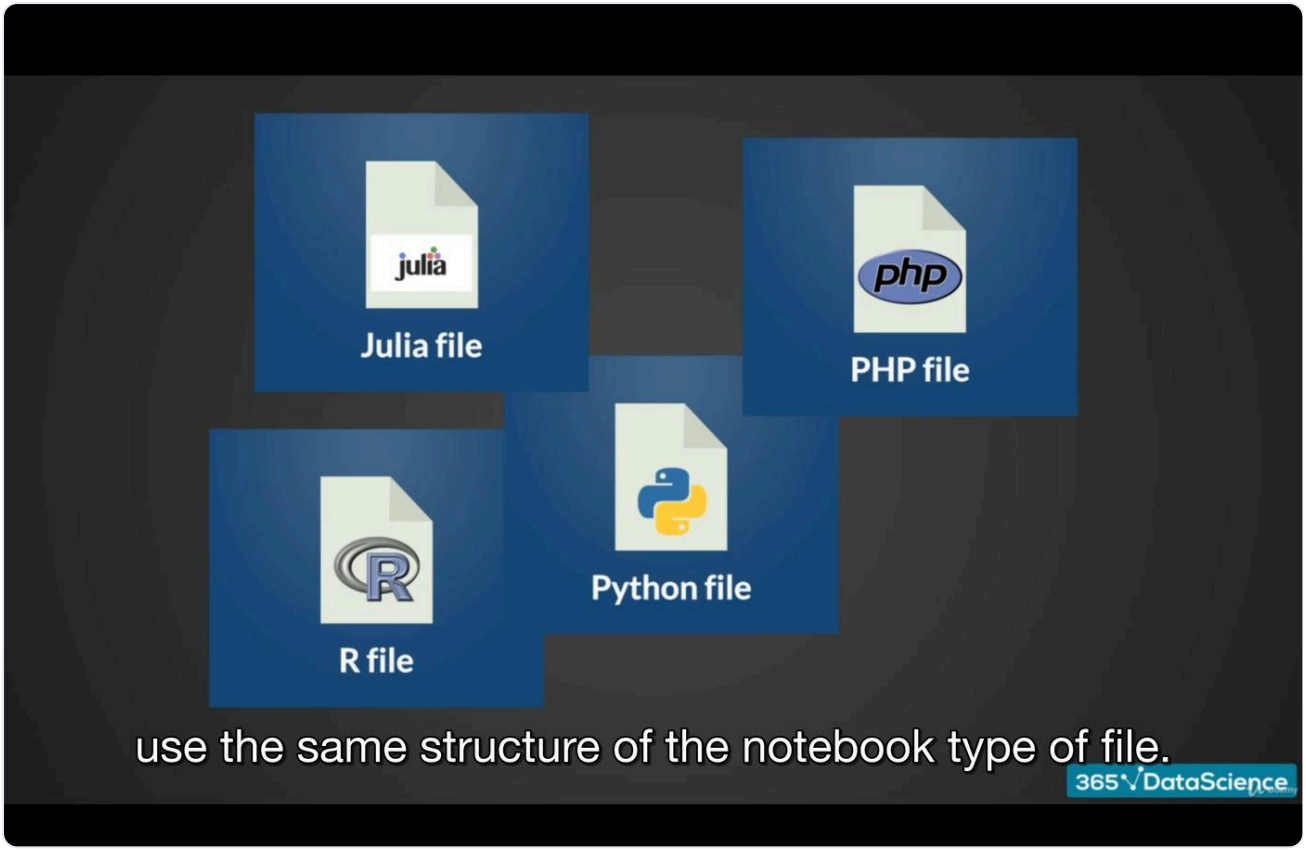

FIGURE 16.12 | Julia File

Instead of installing different interfaces for each language kernel you need, Jupyter allows you to use the same structure of the notebook type of file. Simply, each notebook you create will connect to the language kernel you request.

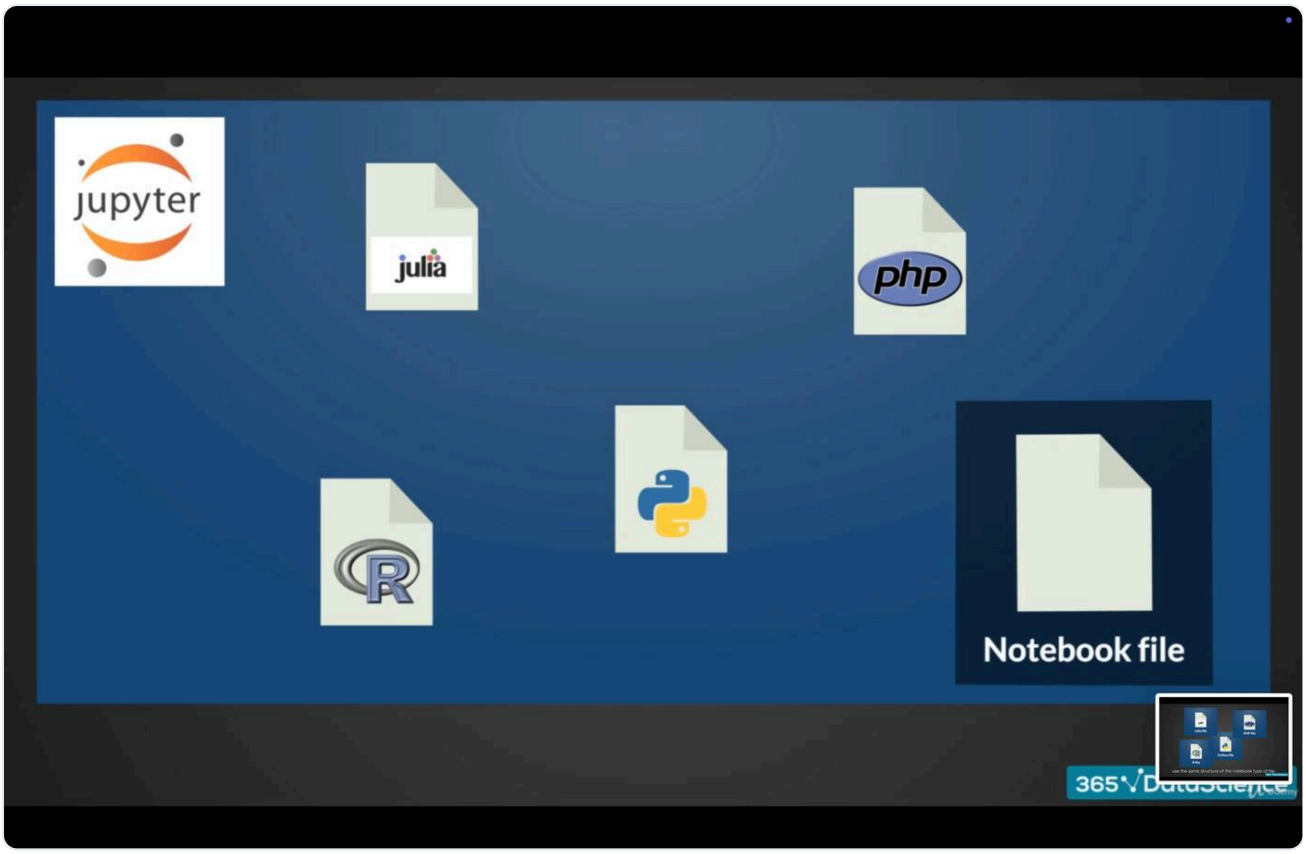

FIGURE 16.13

Consider also this file can be easily stored locally or on a remote server.

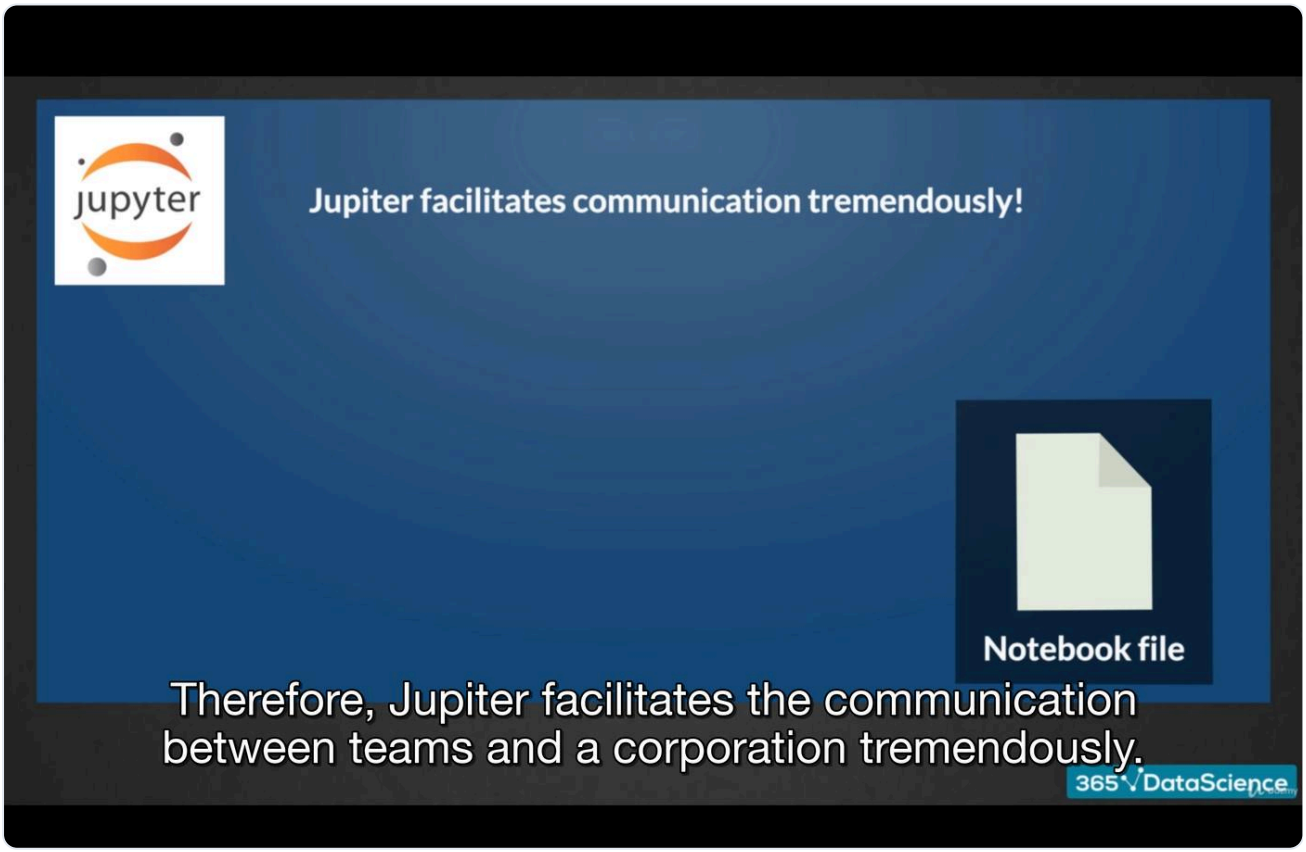

FIGURE 16.14

Therefore, Jupyter facilitates the communication between teams and a corporation tremendously.

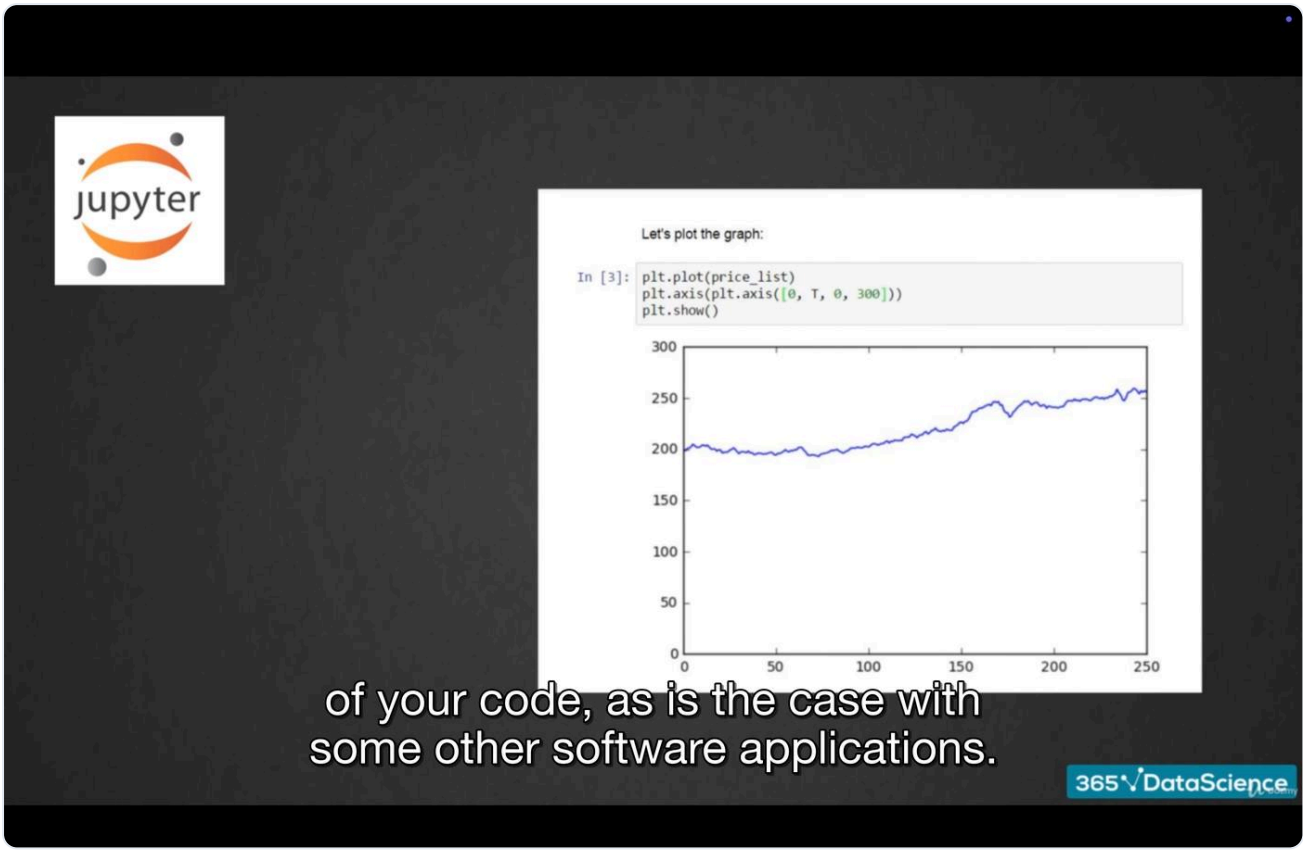

FIGURE 16.15

Second, Jupyter is not a text editor that opens a new window every time you execute a different part of your code, as is the case with some other software applications.

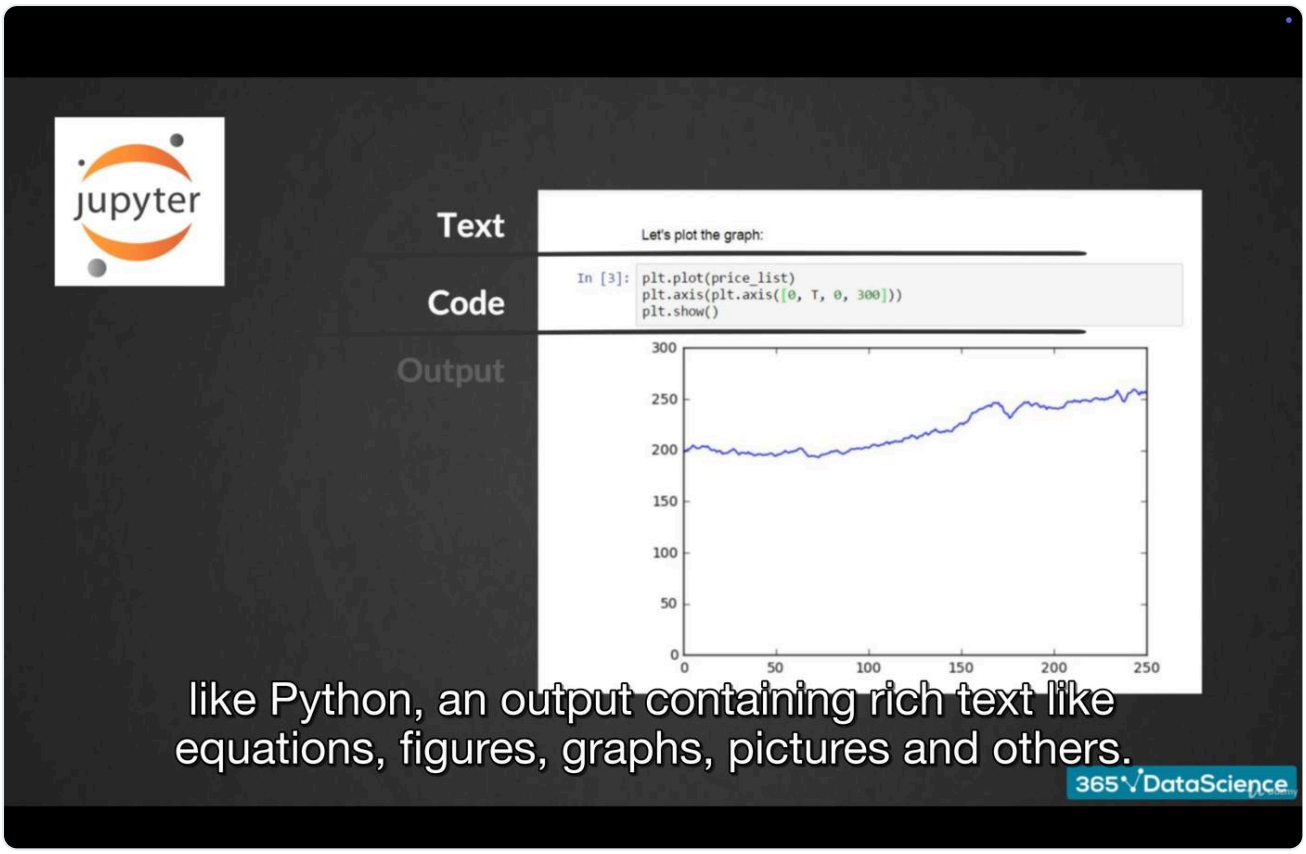

FIGURE 16.16

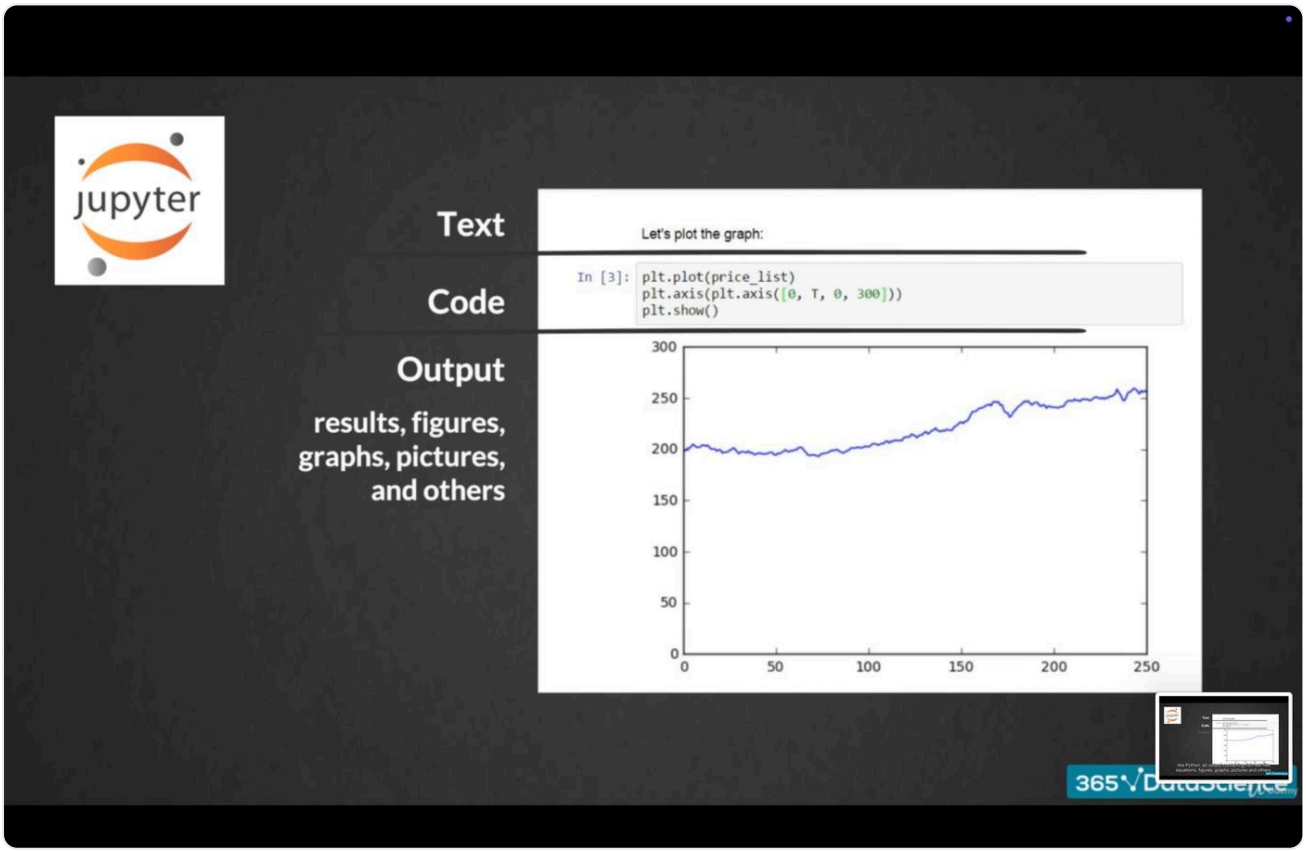

FIGURE 16.17

In the same file, you can have pure text that can communicate a message to the reader computer code like Python, an output containing rich text like equations, figures, graphs, pictures and others. This simplifies the process of the workflow immensely, and Jupyter notebook is increasingly preferred over other software packages. That's why we'll use it to.

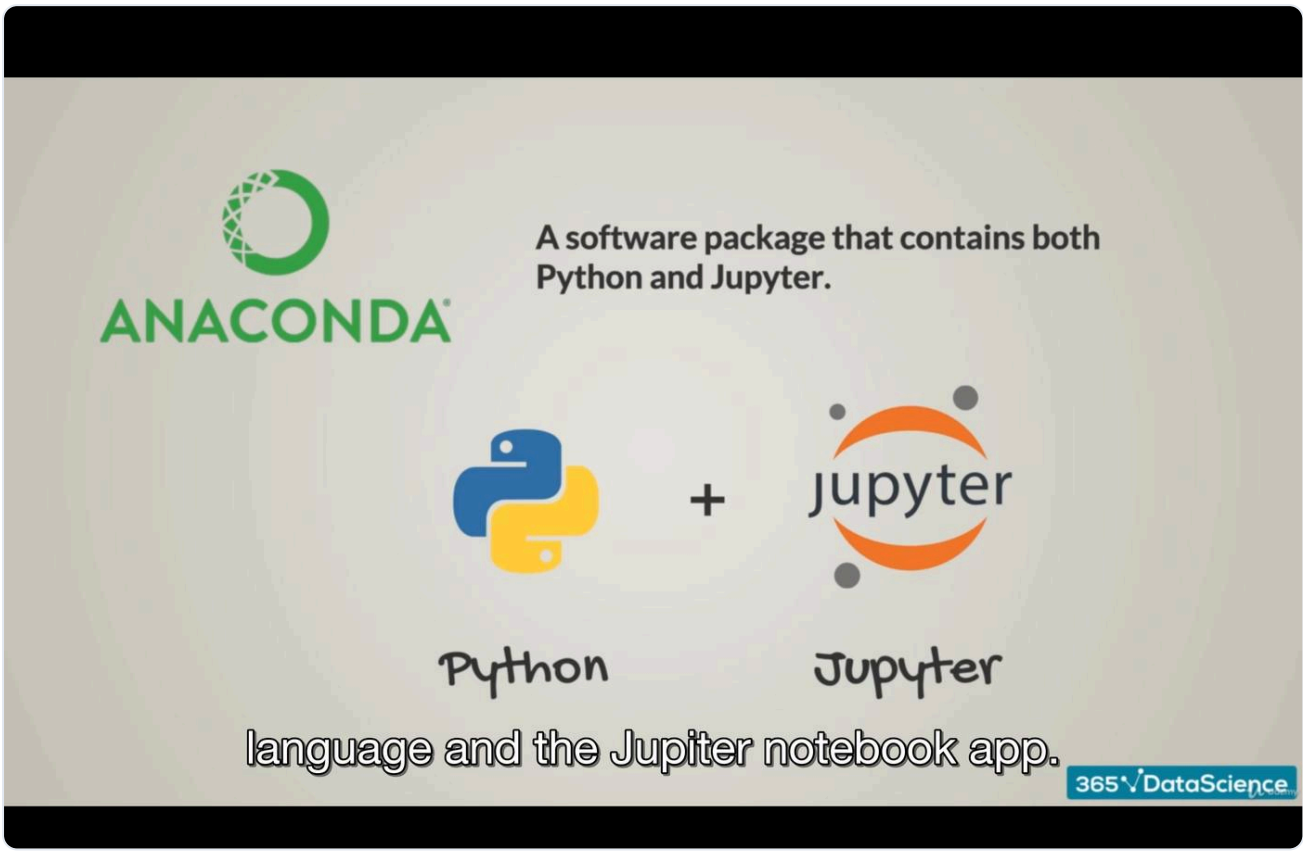

FIGURE 16.18 | A Software Package That Contains Both

The next step would be to install Anaconda, a software package that contains both the Python programming language and the Jupyter notebook app. Thank you for watching.

LECTURE 17

Python and Anaconda

There are various ways to install Python on your computer, but especially for new users, it is highly recommended to opt for Anaconda. It will install not only Python, but also the Jupyter notebook app and many scientific computing and data science packages. Let's open Anaconda Dotcom and click on the download button. In the homepage. You have to pick one of the three operating systems Windows, Mac or Linux. I will show you how to install Anaconda on Windows, but the procedure is identical. If you're going to use the Mac or Linux version now, you must choose the best among the four provided options. Do you need Python two or Python three? And then there is a 32 bed or a 64 bit version depending on the windows you have installed.

I know it sounds strange to maintain older version of the programming language as one of the main options. The reason for that is that a lot of companies had written Python to code and didn't have the resources to switch to Python three when it came out. Thus, Python two is there for legacy reasons. All new code, however, is written on Python three, including the code we will write in this course. In fact, TensorFlow is not available under Python two, so we will definitely need to go for Python three. When you choose between a 32 or 64 bit version, you can simply check your operating system. Newer computers are surely running with 64 bit processors, but if you would like to verify this before you begin, you could open your control panel menu from the start window, then select the system and security icon, then click on System Icon.

In my case, it is 64 bit. So when I go back to the Anaconda website, I will select the 64 bit Python three version while waiting to complete the download. You can decide whether to leave your email to Anaconda. This is not a necessary step, so you could also select no thanks and you are good to go. When the download is finished, please double click on the file to run the application. What follows is nothing different from the standard Windows installer. Agree and press next. Until you have the chance to specify a destination folder, you could do this after selecting the browse button when satisfied with the indicated directory. Click next one more time.

We suggest you only the second advanced option. The first option will automate more complicated processes, but causes some problems with previously installed software, while the second will register Anaconda as a default python on your computer. If you have not installed some other package, finally click, install and proceed until you see the window where you can finalize the installation. We will not use Anaconda Cloud just yet and we can learn about the Anaconda support later. So Allen tick both boxes. What you just install is the whole Anaconda distribution, the python language, a text editor, many applications and packages. You also have the Jupyter notebook app.

You can open the START menu and select the respective icon. From there, a new window will pop up. It will take a few seconds for the app to load. Once this is done, your Web browser will open a new tab with a Jupyter dashboard. In our next video will study the dashboard in more detail and will learn how to use Jupyter. See you there. Thanks for watching.

LECTURE 18

Python Notebook

OK, so in this lesson, we'll do a quick tour of the Jupyter Dashboard as soon as you load the notebook, the Jupyter Dashboard opens each file and directory has a checkbox next to it. By taking an undertaking an item, you could manipulate the respective object. That means you can duplicate or shut down a running file. In addition, you can rename move and delete folders. The selection menu allows you to select all the files in the console of the same type by expanding this button, for example, you can mark all the folders or all the running files. In addition, you can check this little box that will directly select all the items on the page. The logic regarding the directories management is the same as the one of an operation system.

Files can be grouped into folders and folders can contain other folders. From the upload button in the top right corner, you can upload a notebook into the directory you were in, the standard Explorer box opens and when you select a file, you can click on Open and it will immediately appear in your directory. Finally, you could expand the new button from the list that falls. You will most likely need to create a new text file, a new folder or a new notebook file. A notebook vial can contain code in any of the languages in your notebook section, when you create a new Python notebook file, it will be recorded in the eye python notebook format DOT IPY and be for brief, don't fear this other new name ie Python.

Think of it as the predecessor of Jupyter. Consider the eye python notebook format as a legacy file format. All right, let's carry out a few operations, I will rename this folder to exercises. OK, done now I will create a new Python notebook file by expanding the new button and selecting one of the two Python formats I have here. Let's use the default Python format, the browser immediately opens a new tab for me. This is the interactive shell we mentioned earlier here. You will write your code and see its output. More on that in our next lecture. Thank you for watching.

LECTURE 19

Cell

OK, great, now that we know more about the dashboard, we are ready to examine the shell and see how we can code in Jupyter. The field you see here is called a cell. You can access a cell by pressing enter. Once you've done that, you'll be able to see the cursor so you can start typing code. The gray box is called input field. I will put a short code that says Acts is a less composed of four numbers one, two, three and four. After that, I can ask the computer to print this list for me by typing X. I can execute these commands in two ways, the first one is to hold control and then press enter by doing this. The machine will execute the code in the cell and I will stay there, meaning I will not have created or selected another cell.

Observed that an output field with the same number as the input field appeared, input and output, fields of the same number are grouped together. The output represents the machine's response to your commands provided in the corresponding inputs. And the respective field cannot be modified, the second option allows for a more fluid code writing to execute the same code, hold shift and then press enter. The previous two commands are being executed and then a new cell where you can write code is created, if you use shift and enter, you can continue typing code easily. It is amusing that you can cut copy and paste cells, you can use the buttons on the main menu.

Since we are keen supporters of ergonomics, we suggest you get used to the keyboard shortcuts when coding, you will mainly have to use the keyboard. So it's worth memorizing different keyboard combinations that would allow you to work faster. Let's see how we can apply shortcuts in practice.

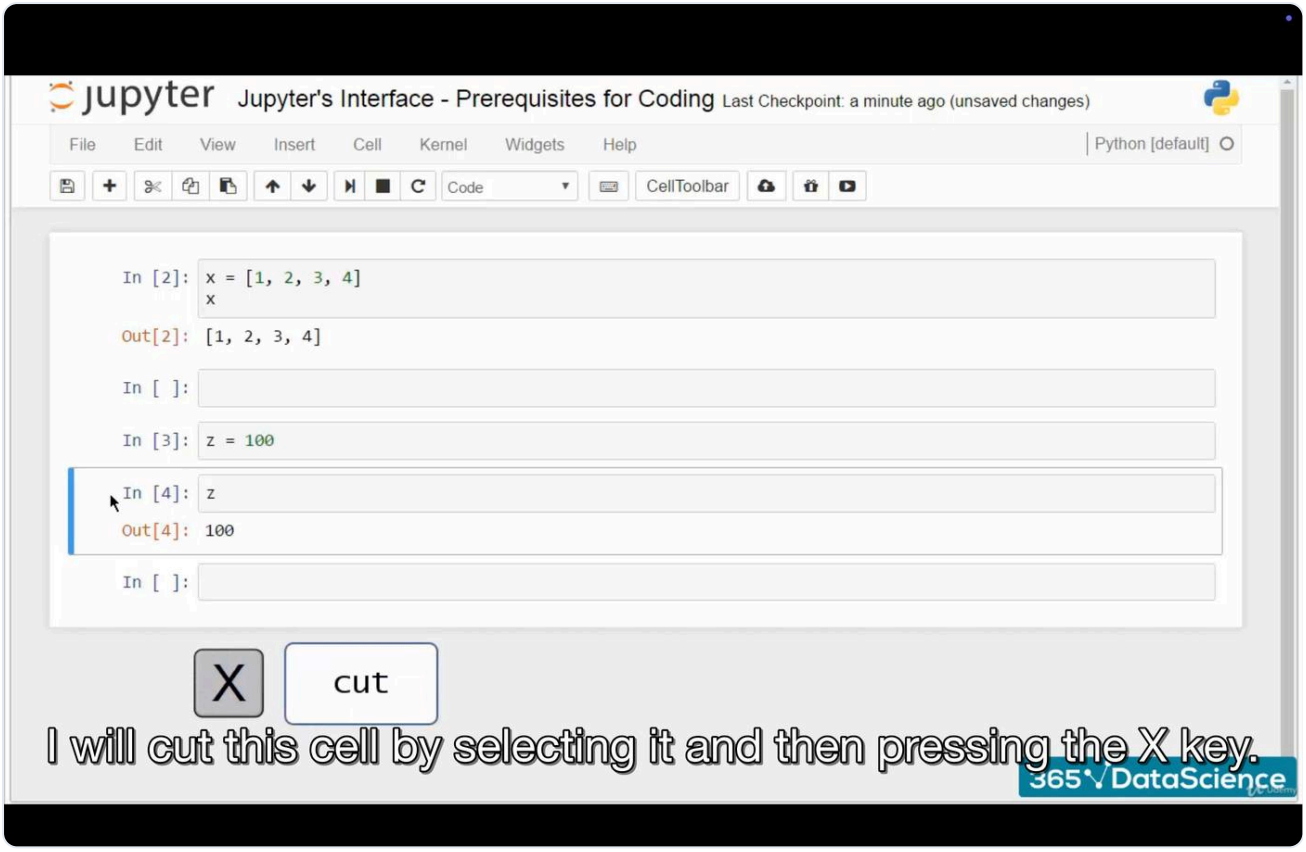

FIGURE 19.1

I will cut this cell by selecting it and then pressing the X key.

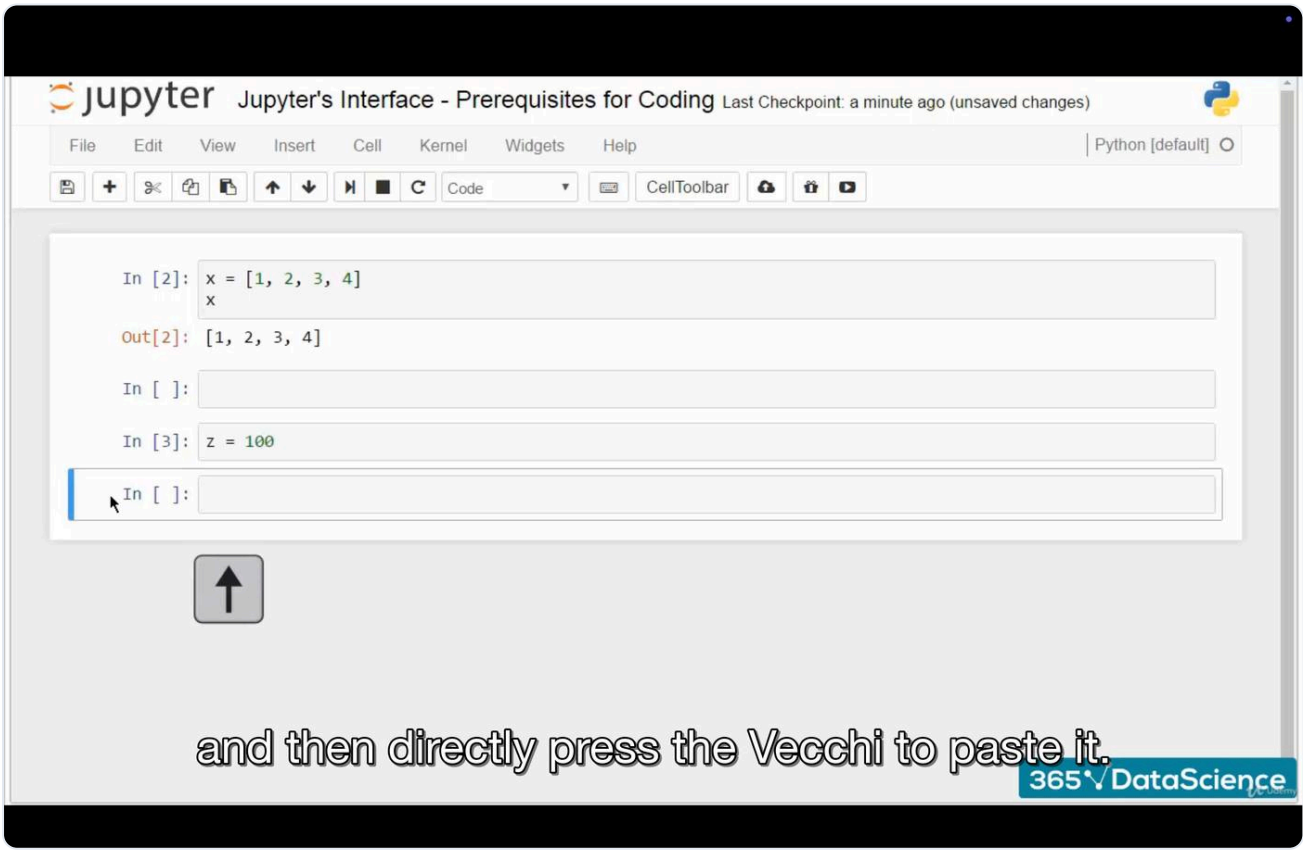

FIGURE 19.2

Remember, you can always use the Arrow keys to navigate along your notebook file, so I will move up and then directly press the Vecchi to paste it.

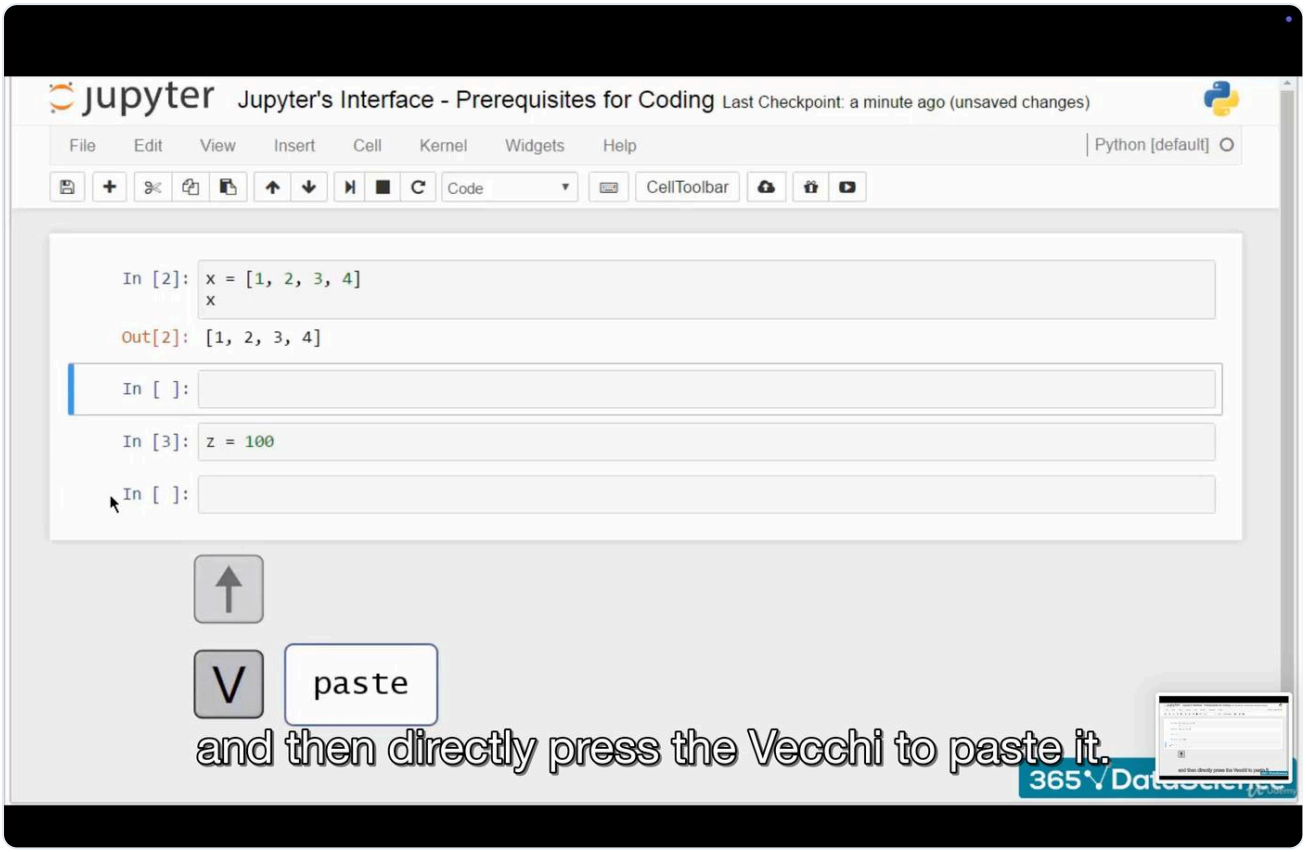

FIGURE 19.3

Well, my cell was moved up here below the cell I had selected before I had pressed the PVC.

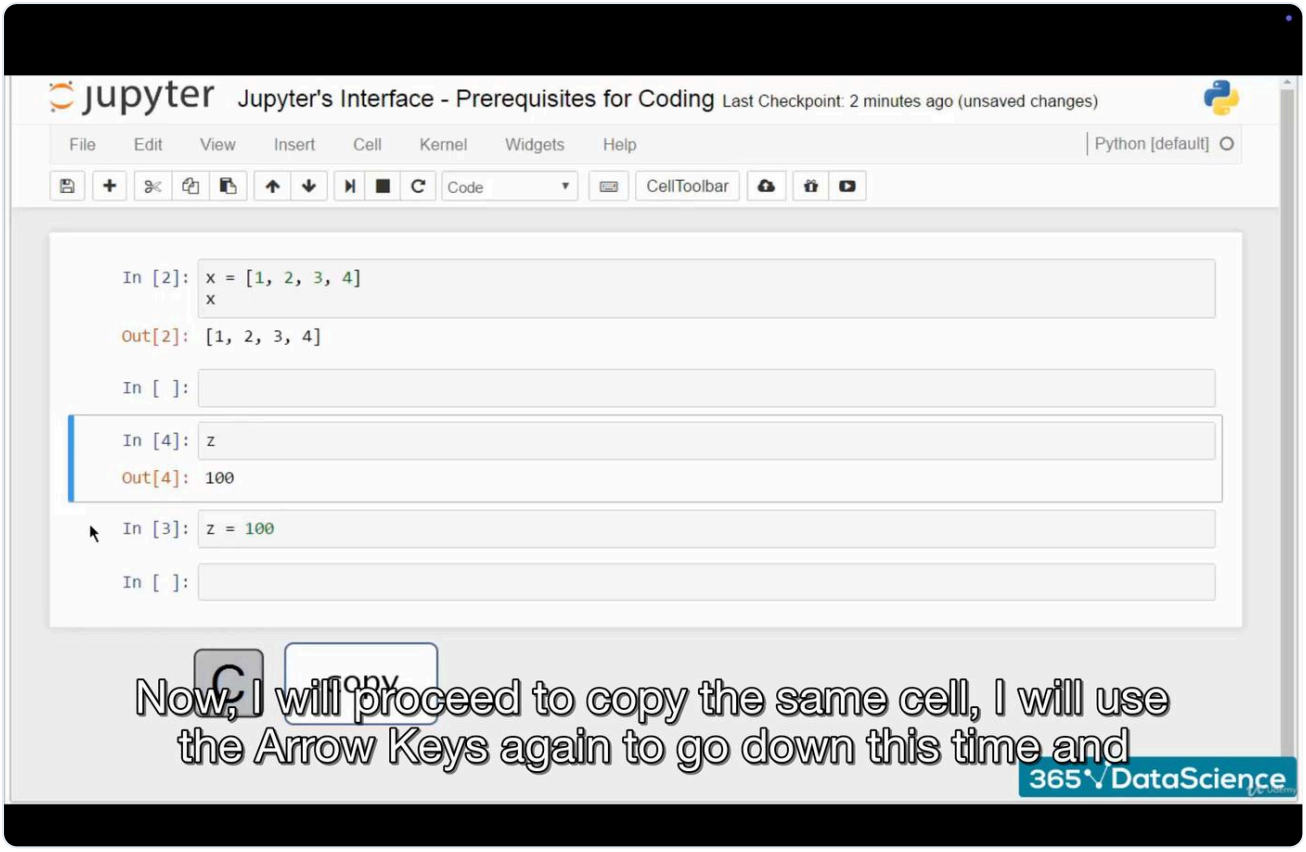

FIGURE 19.4

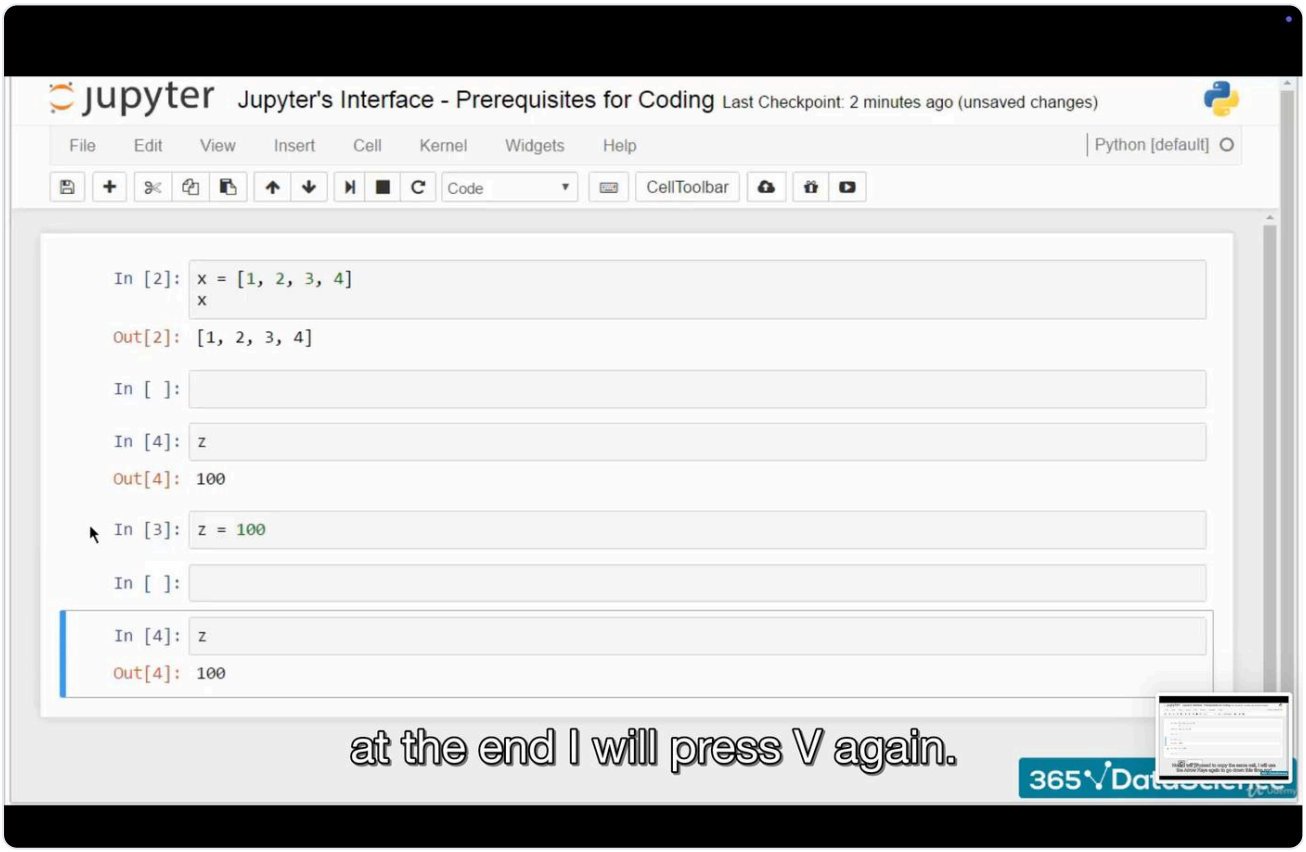

FIGURE 19.5

Now, I will proceed to copy the same cell, I will use the Arrow Keys again to go down this time and at the end I will press V again.

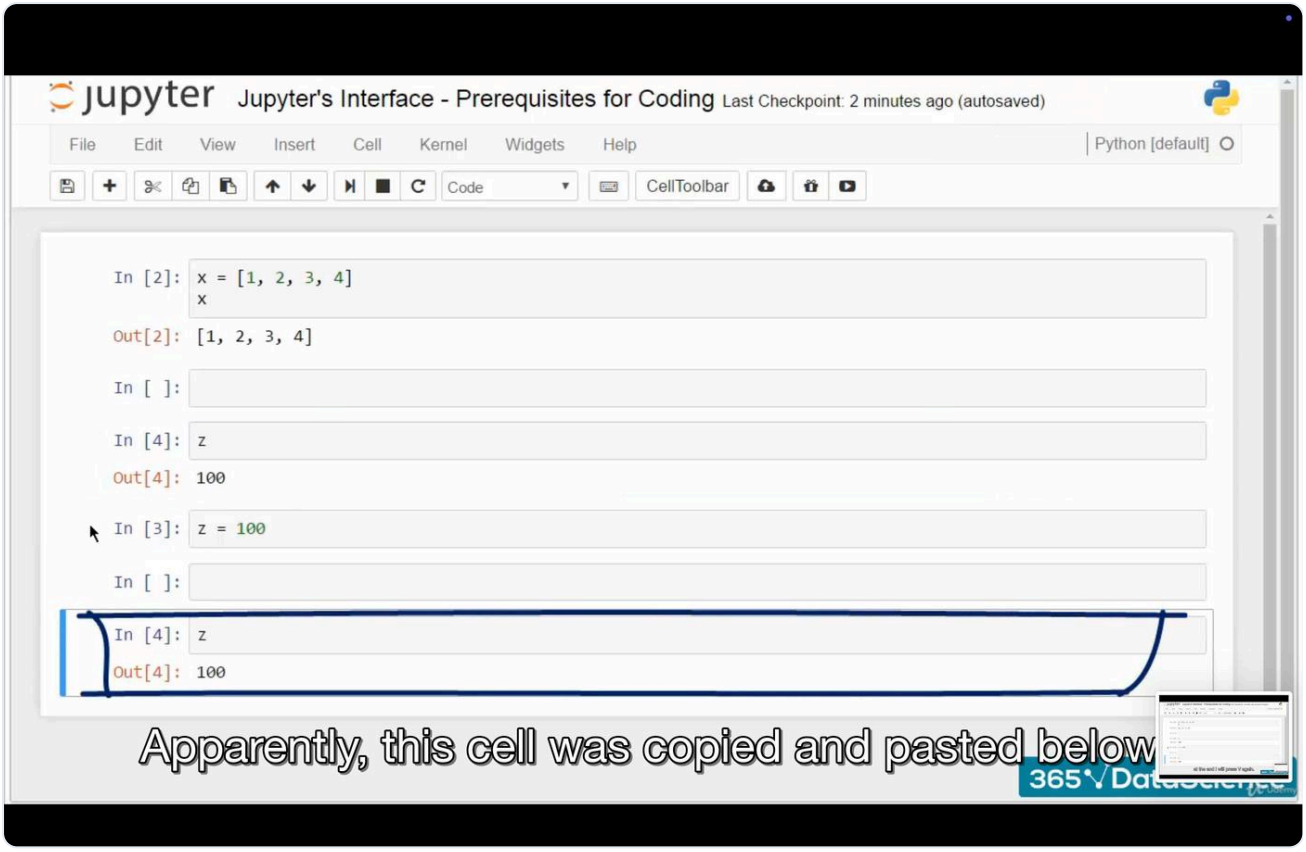

FIGURE 19.6

Apparently, this cell was copied and pasted below.

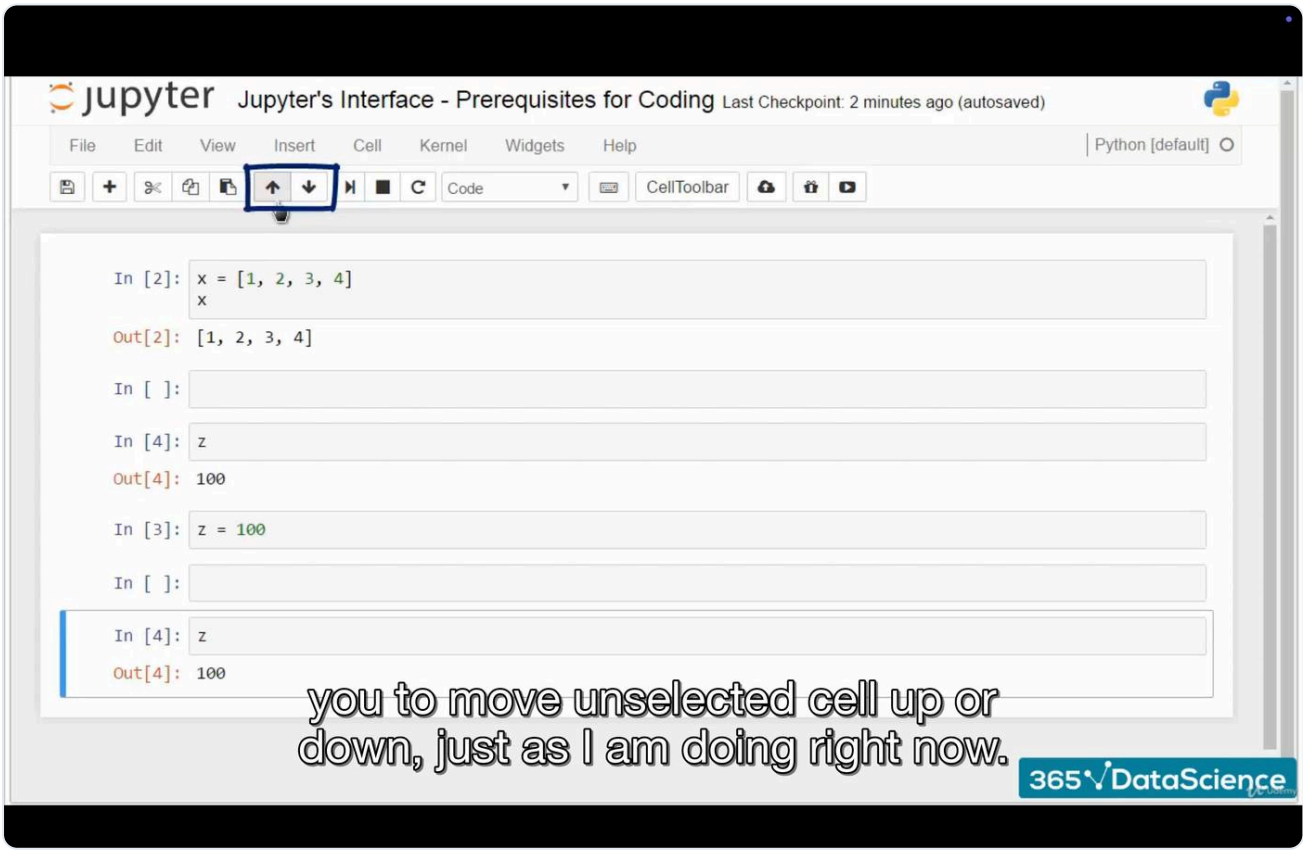

FIGURE 19.7

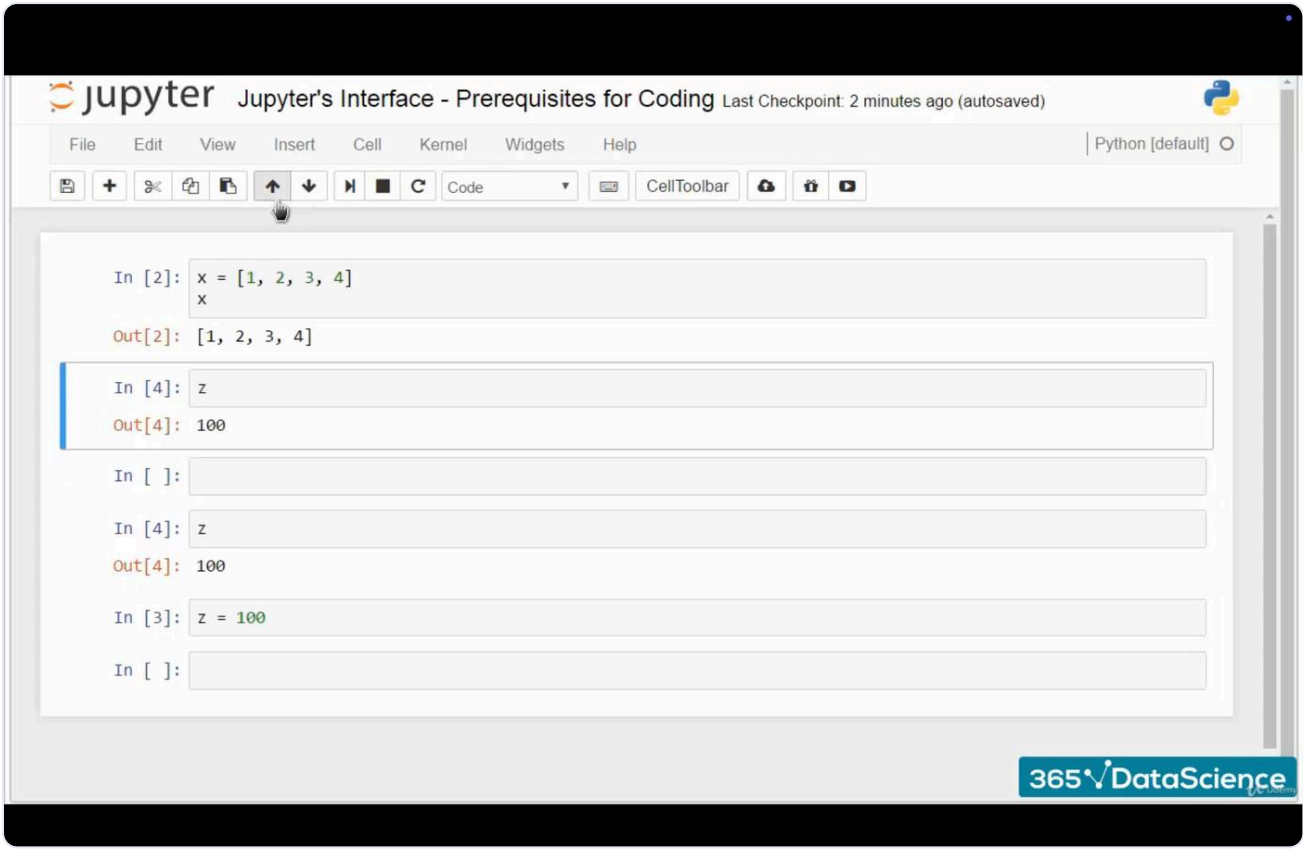

FIGURE 19.8

The other two buttons in the menu cannot be substituted by keyboard shortcut combinations, they allow you to move unselected cell up or down, just as I am doing right now.

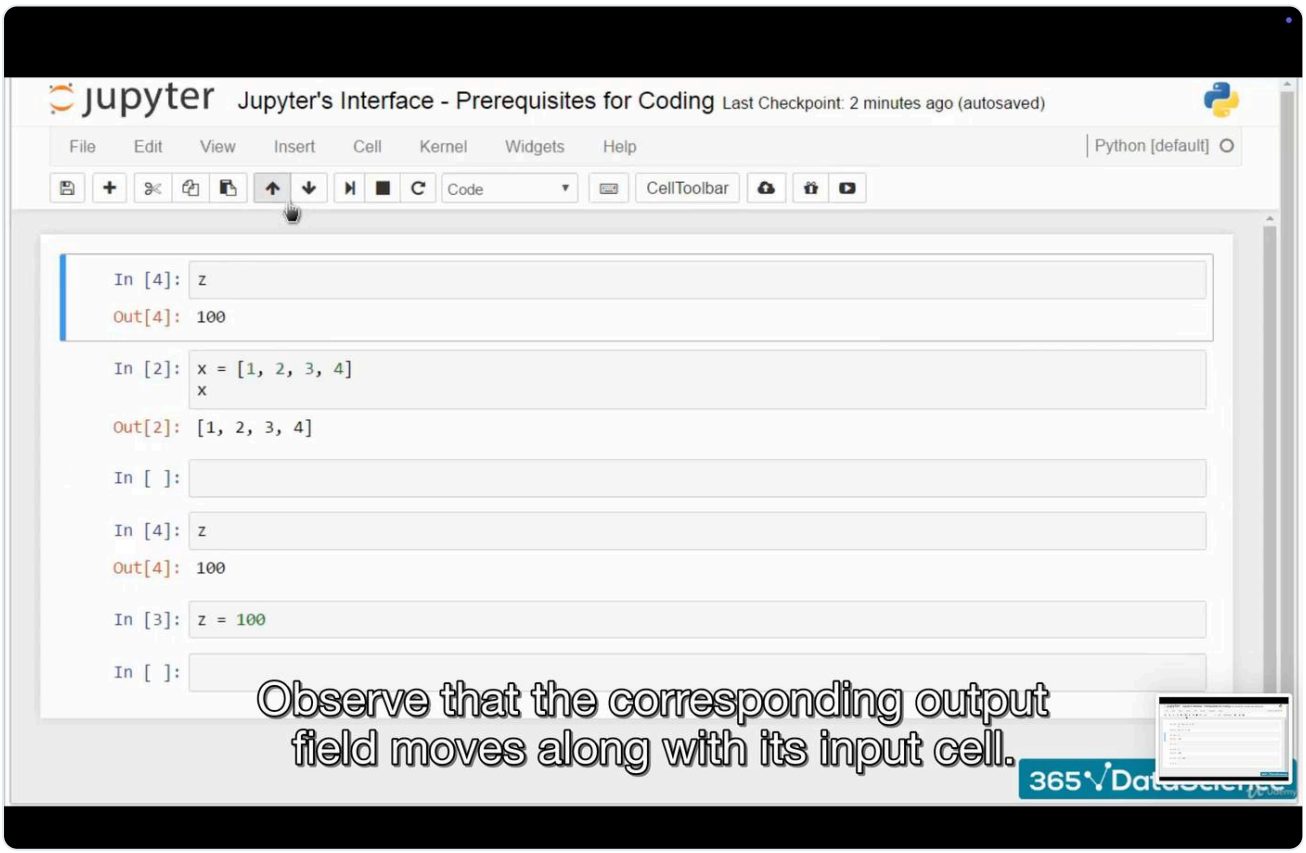

FIGURE 19.9

Observe that the corresponding output field moves along with its input cell. Lovely. Instead of carrying out a command with shift and enter, you might prefer clicking on this button with the mouse, the result will be the same. What is interesting is that when you work with word complex code that requires tougher calculations from the computer while running the code, a little star will appear over here before the square brackets. Sometimes this process might take too long to complete. So to stop it or break it, this classical stop symbol can do that for you. Three other shortcuts could accelerate your coding a lot.

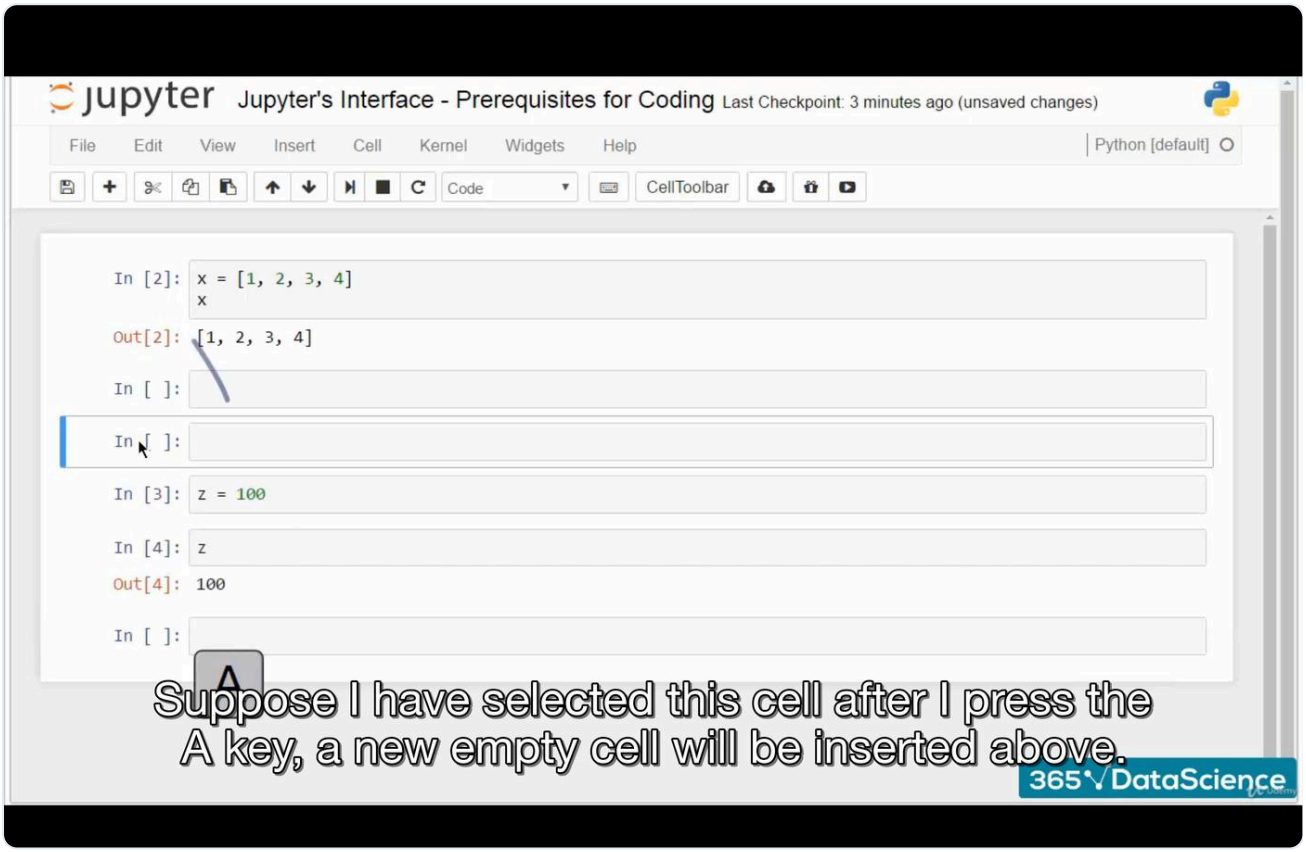

FIGURE 19.10

Suppose I have selected this cell after I press the A key, a new empty cell will be inserted above.

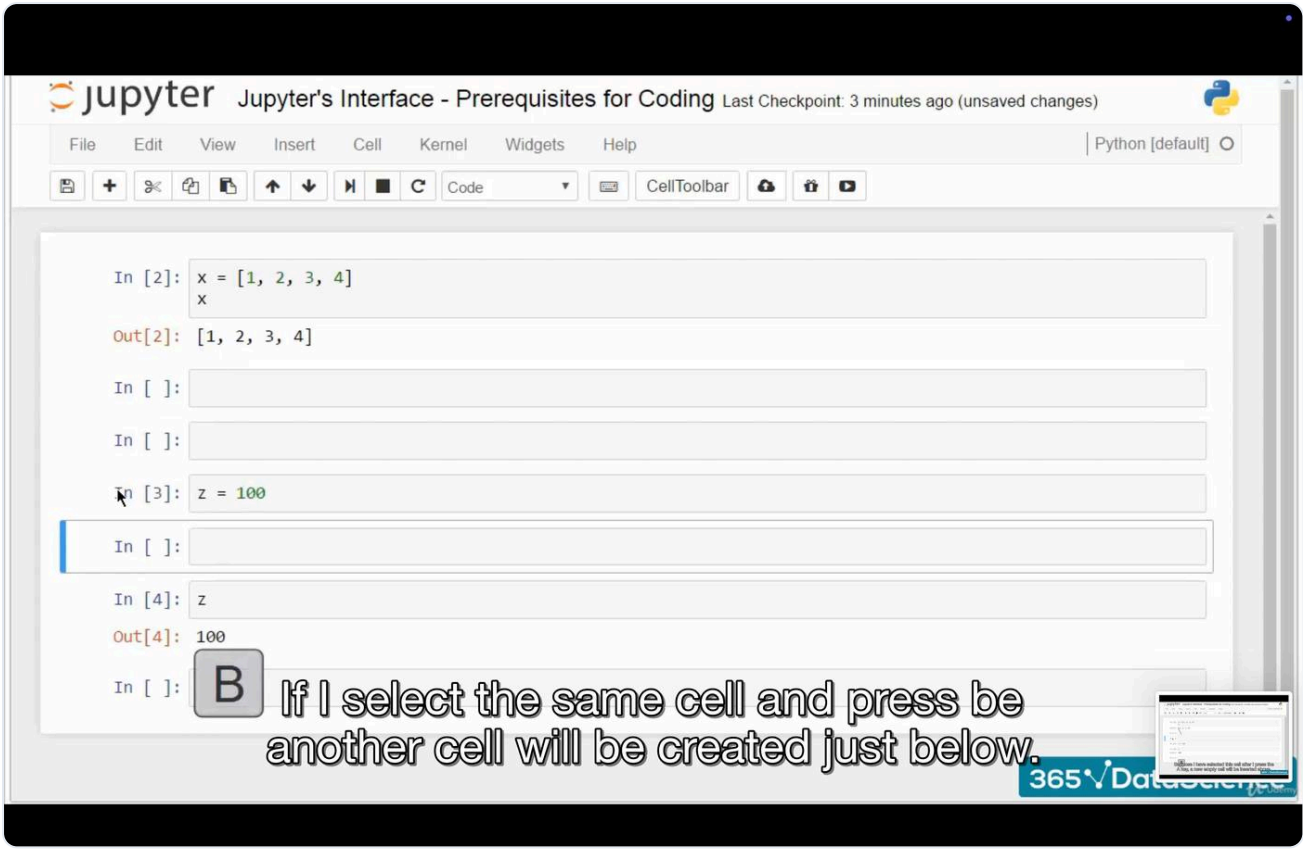

FIGURE 19.11

If I select the same cell and press be another cell will be created just below.

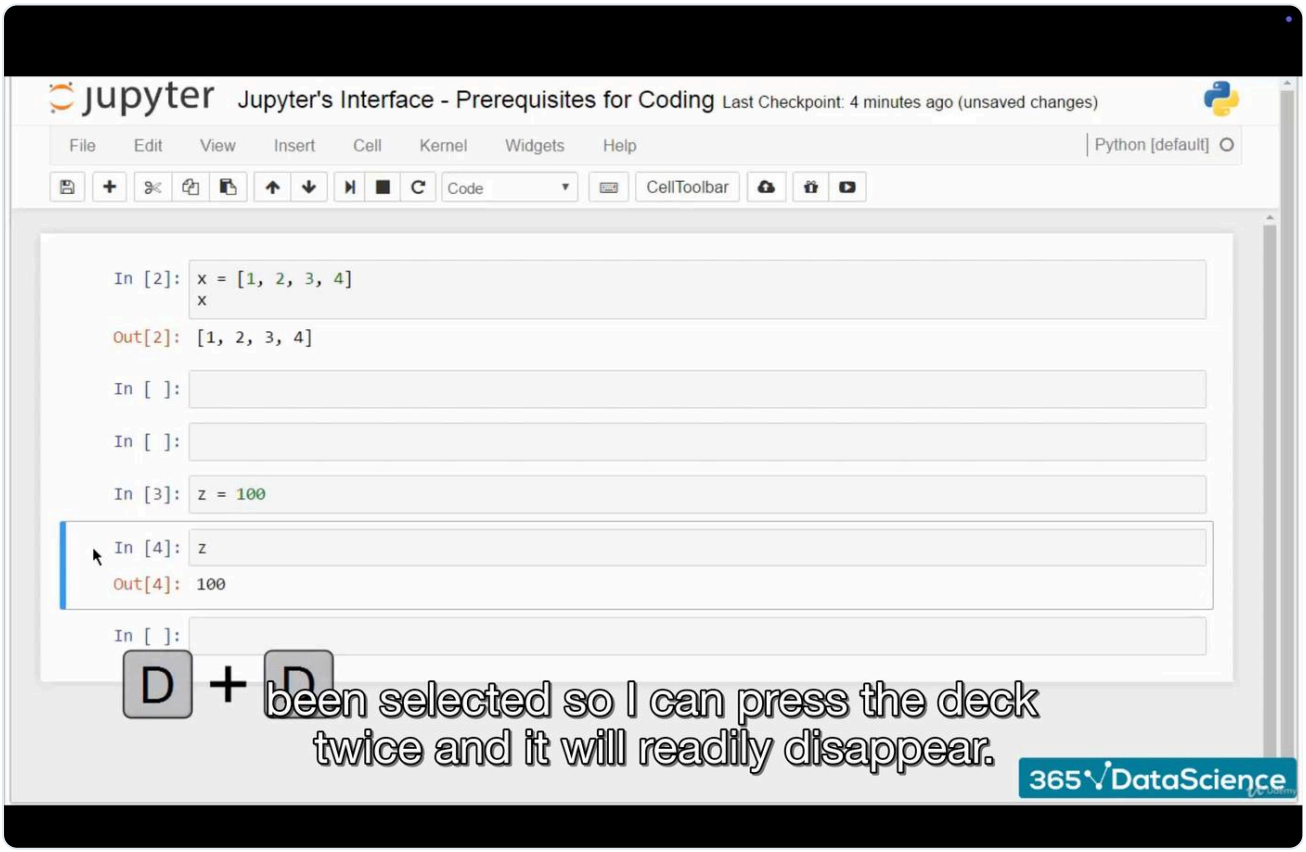

FIGURE 19.12

Imagine the last step was a mistake and I need to delete that empty cell in this case, it has already been selected so I can press the deck twice and it will readily disappear. Wonderful. All the cells we saw so far were code cells. Let's see what is a markdown cell.

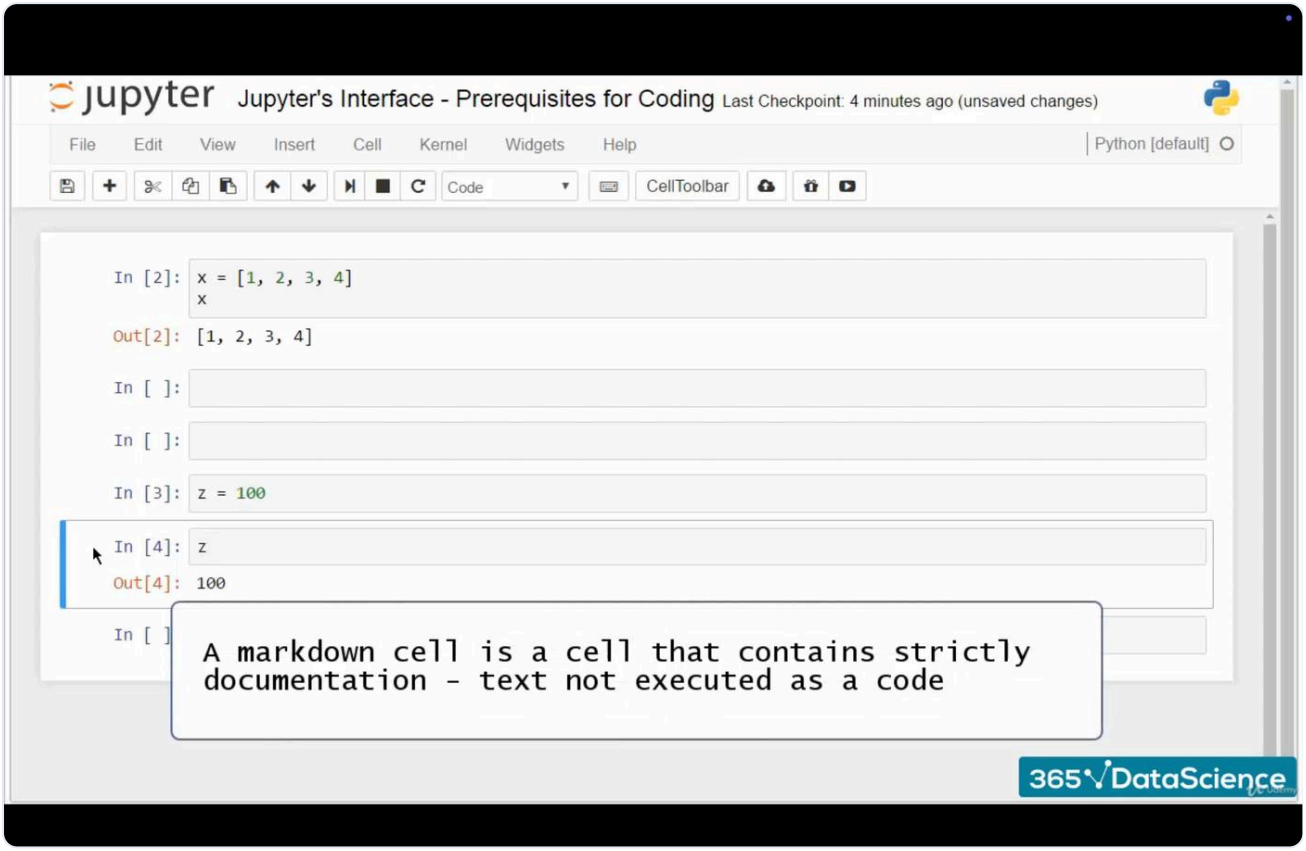

FIGURE 19.13

It is a cell that contains strictly documentation text not executed as a code. It will contain some message you would like to leave to the reader of the file to convert a selected cell into a markdown cell.

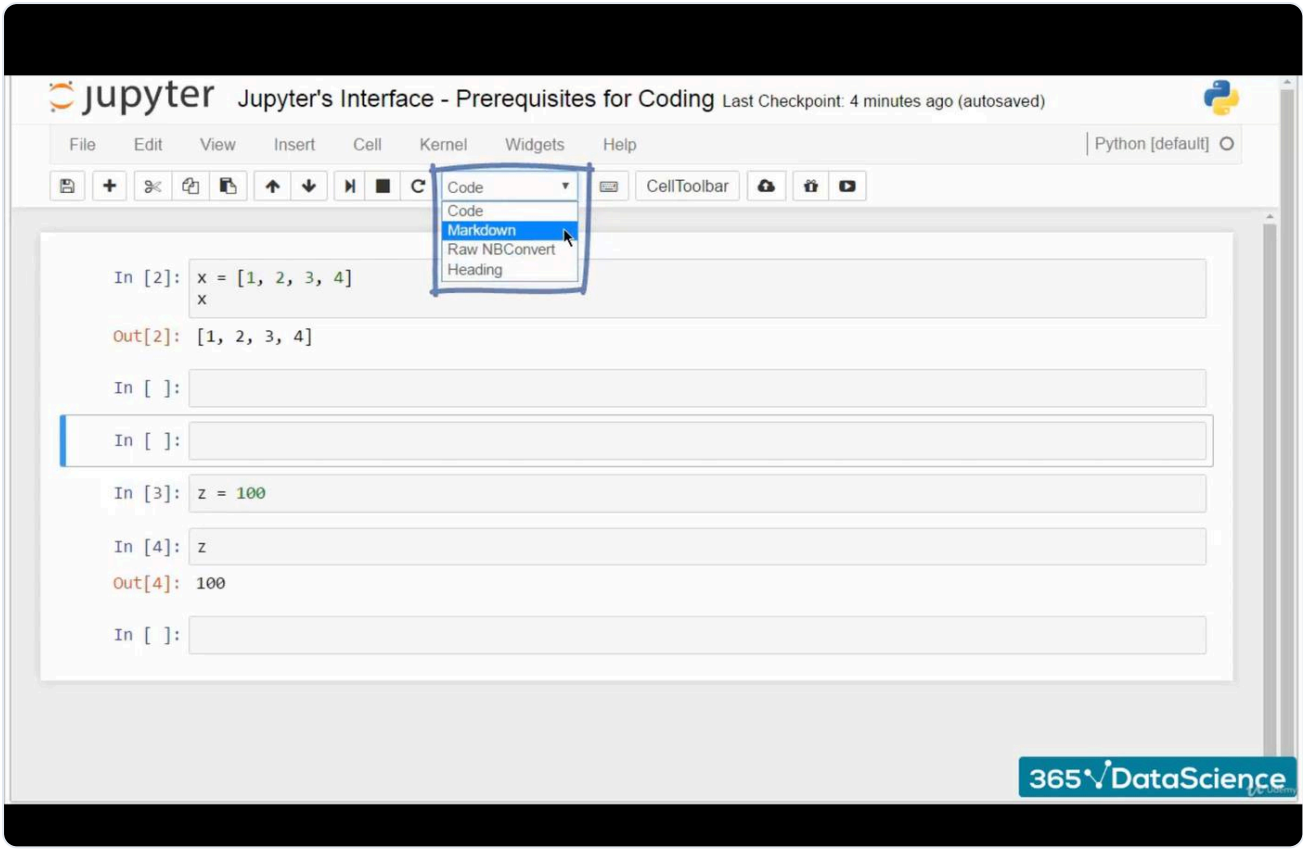

FIGURE 19.14

You should either expand this list and opt for markdown.

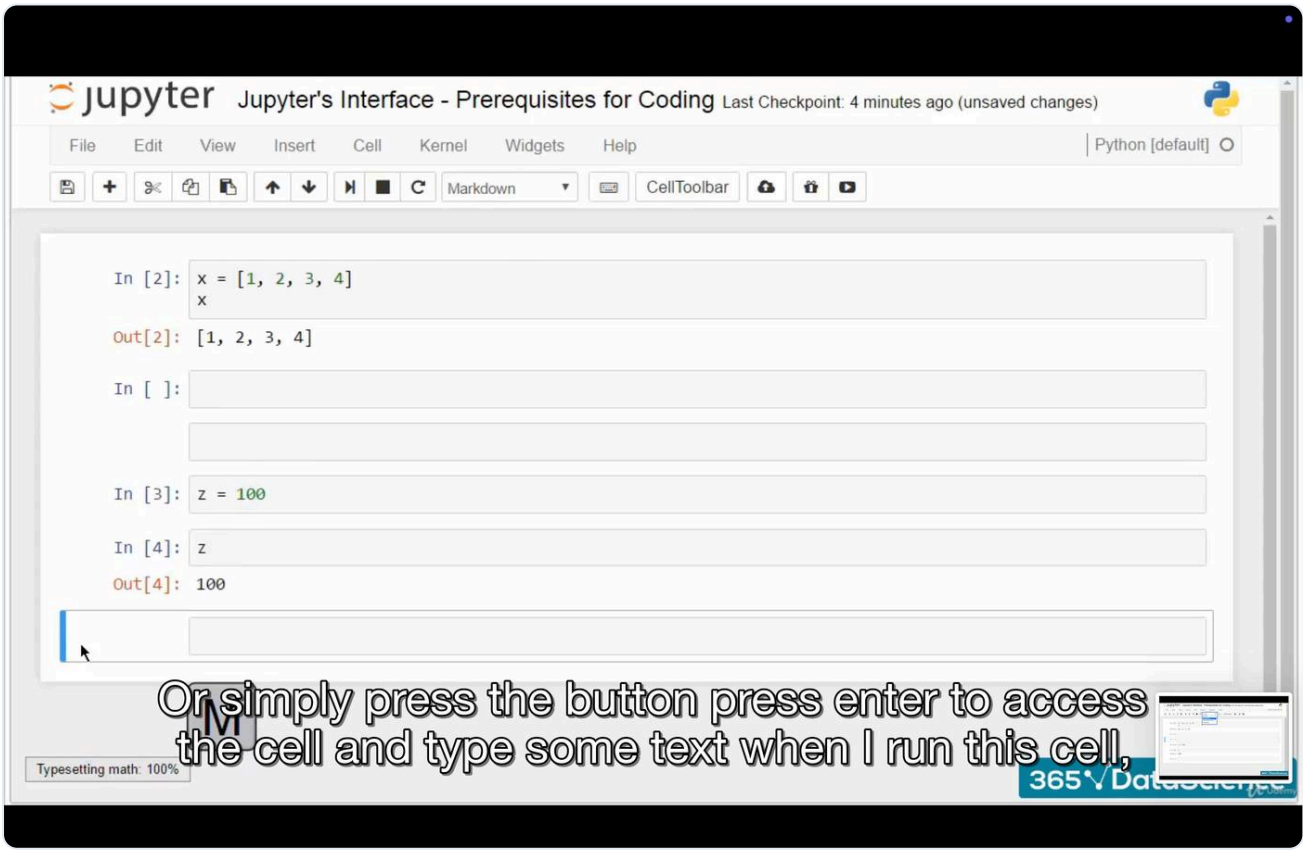

FIGURE 19.15

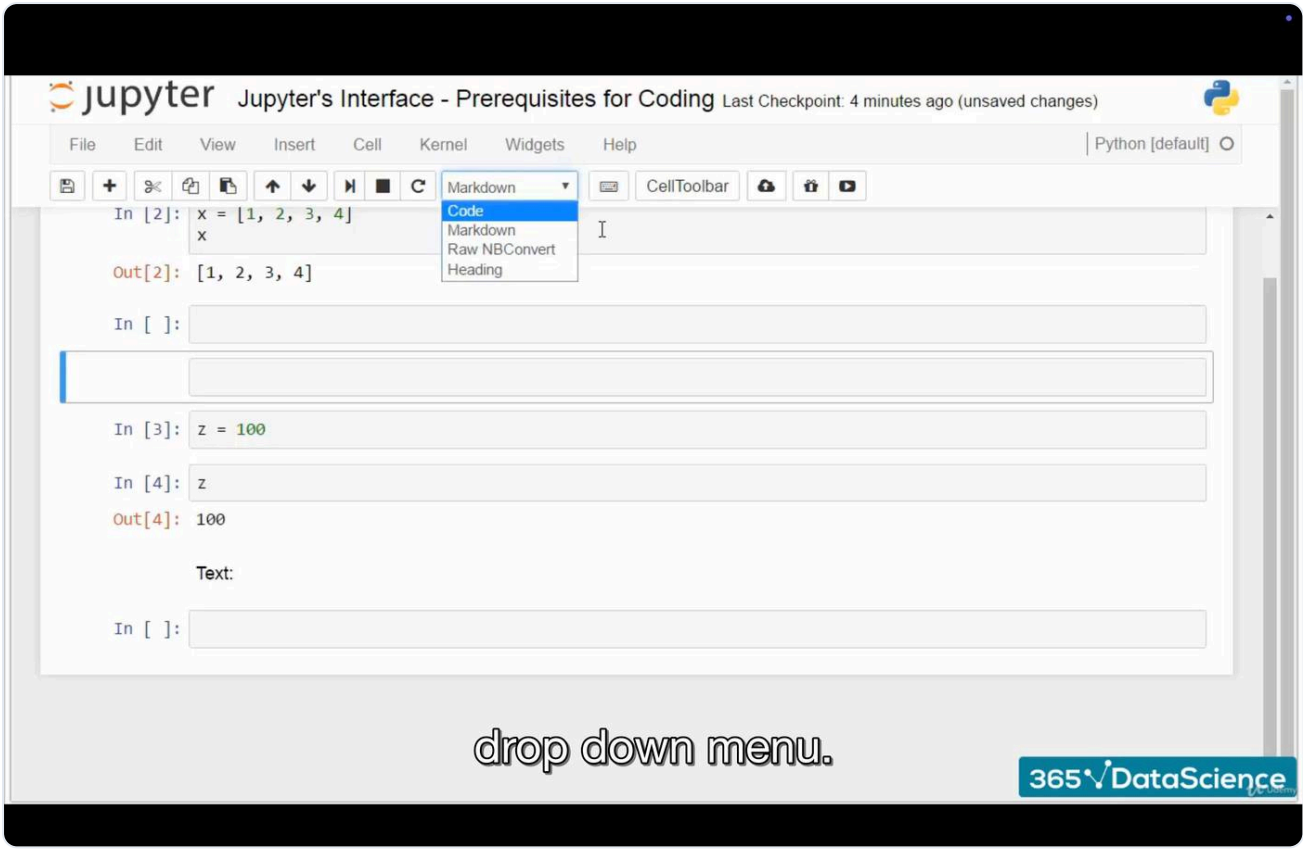

FIGURE 19.16

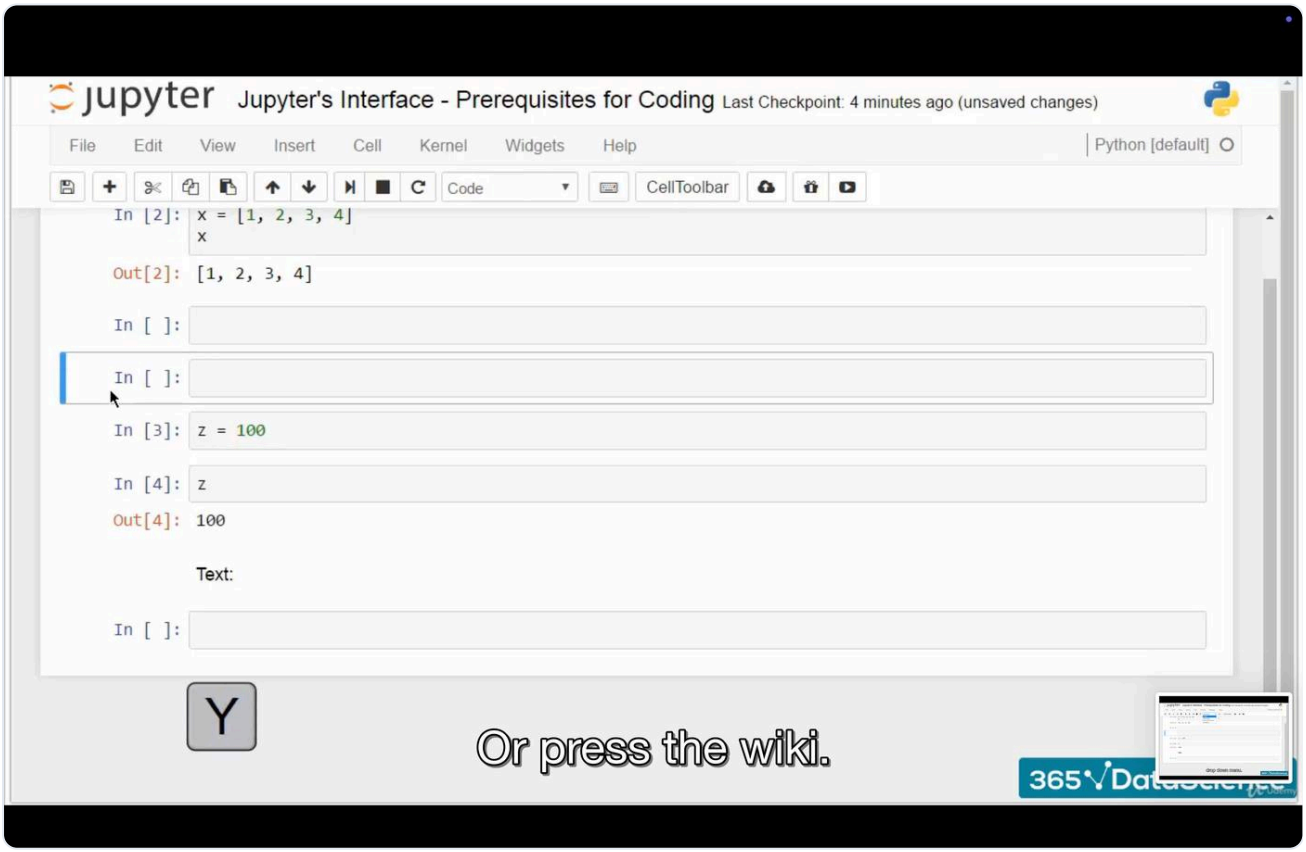

FIGURE 19.17

Or simply press the button press enter to access the cell and type some text when I run this cell, the output will be a simple statement. Now, if you want to turn a markdown cell back to a code cell, select it and opt for code from this drop down menu. Or press the wiki. To conclude, there are two advantages of using Jupyter first, when your code becomes longer marked down, cells turn out to be useful as they allow you to leave comments and explain the solution you've created. This is why practitioners love to use them. The other benefit of using this app is you can select and execute whichever cell you want to. You need not run all the previous cells to run a particular cell.

This allows for solving a problem in pieces and saves a lot of computation time. This was a long but indispensable lesson, thank you for watching.

LECTURE 20

Install TensorFlow and PIP Install

Finally, let's look at the libraries we will use in this course, the good thing about Anaconda is that many libraries among which Dumpy and pandas come automatically with it, that's a strong plus of Anaconda. You don't have to install the main packages separately as you might have to do if you were to use some other software for programming in Python. Moreover, many packages related to machine learning also come with Anaconda.

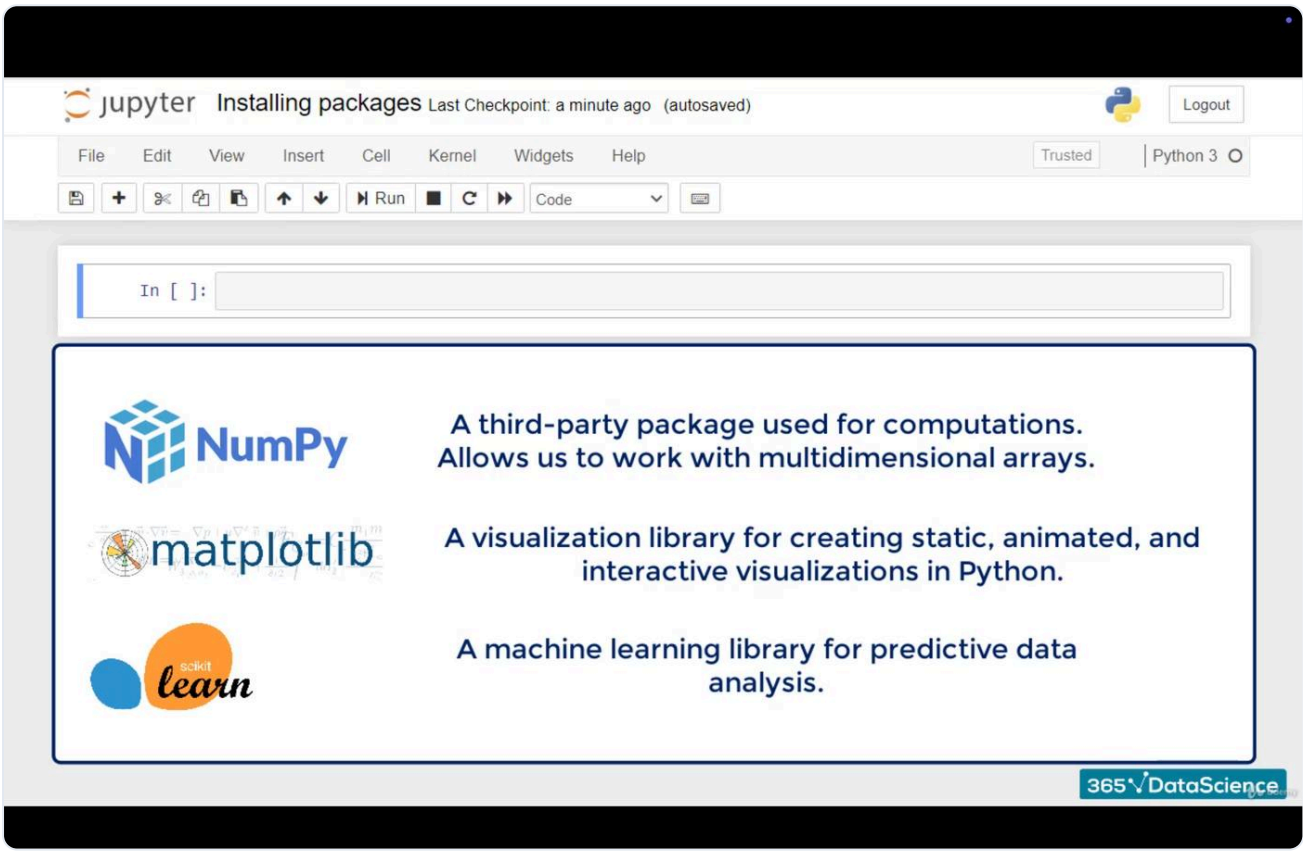

FIGURE 20.1

Matplotlib. Itertools and scikit-learn.

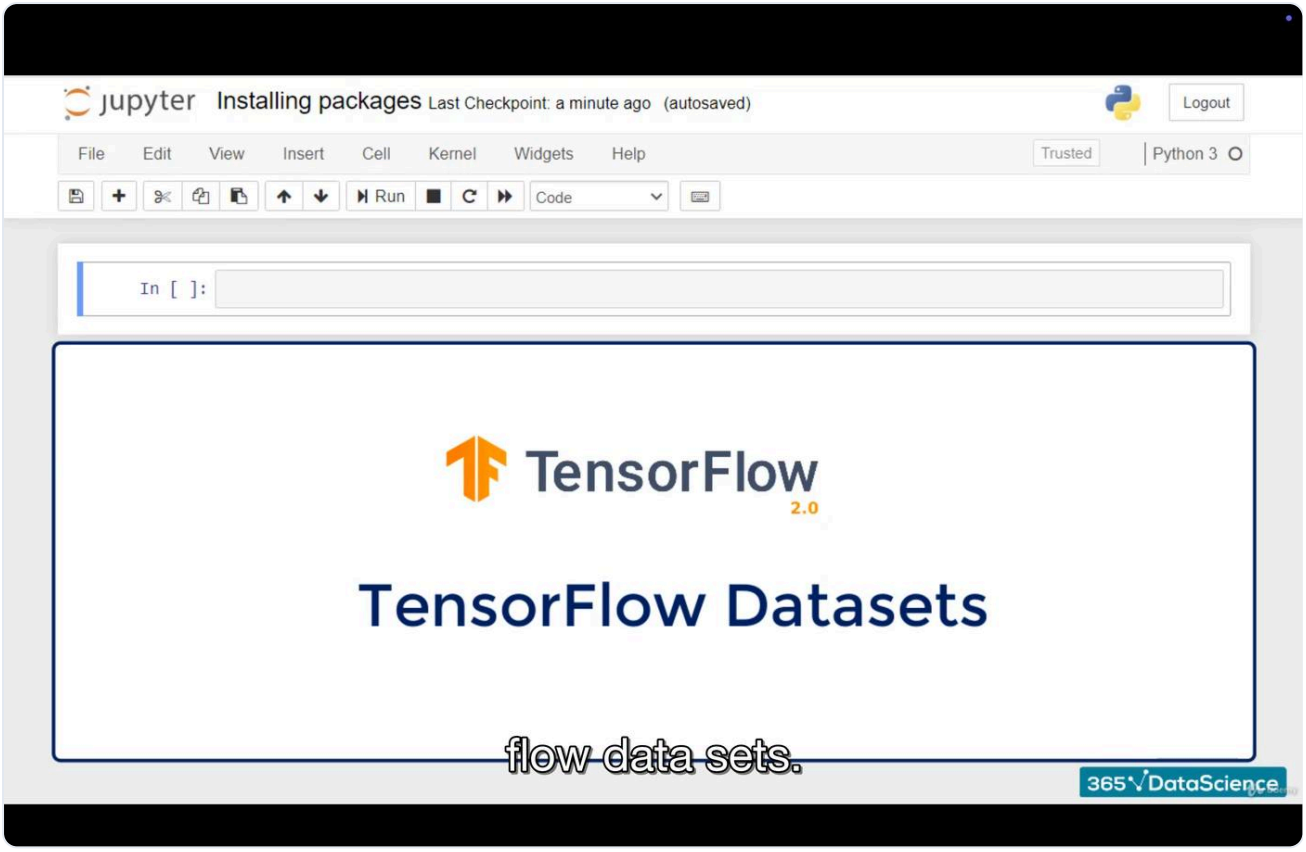

FIGURE 20.2

Apart from the automatically included packages, we will also take advantage of TensorFlow and TensorFlow Datasets. The quickest way to install it is by opening your start menu and searching for the Anaconda prompt. To install TensorFlow, please type PIP, install TensorFlow in the following way.

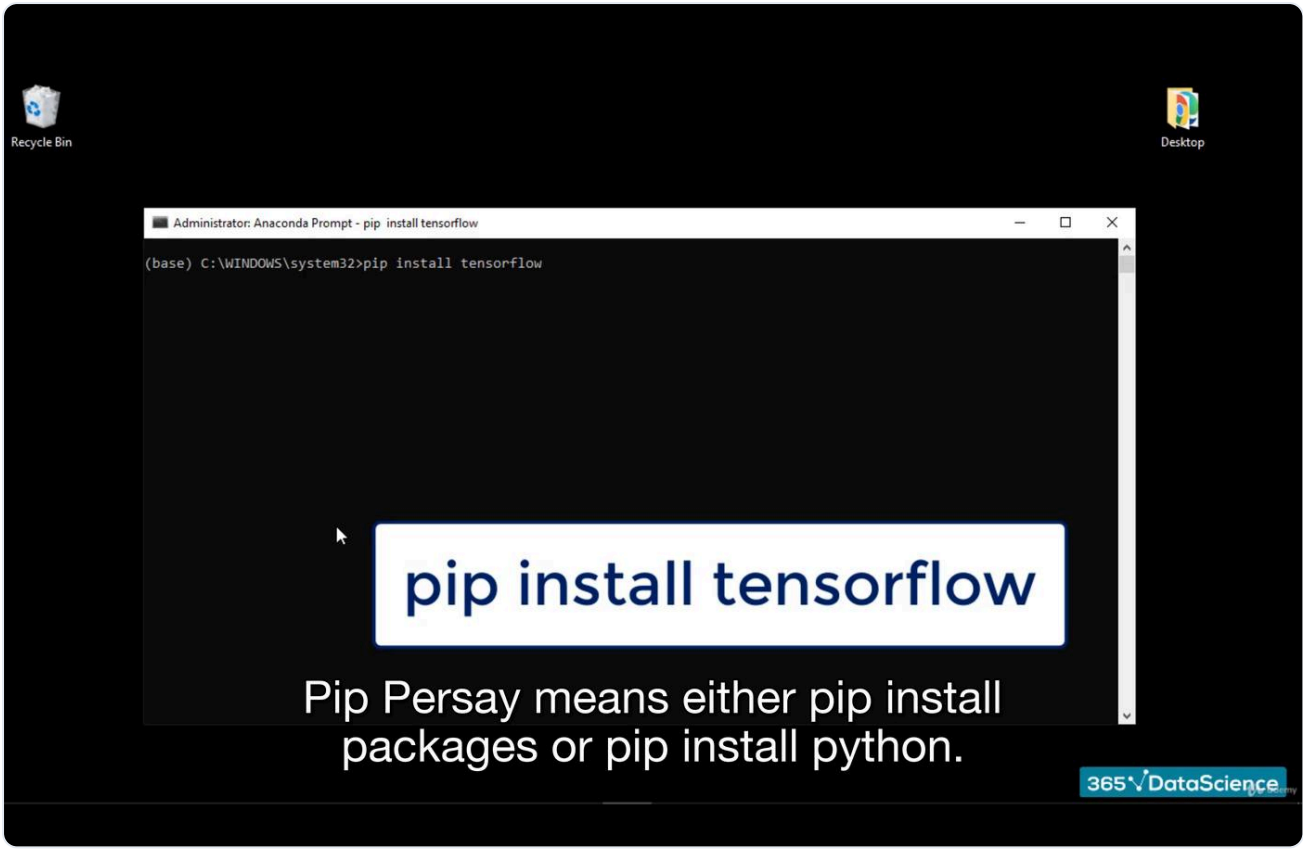

FIGURE 20.3

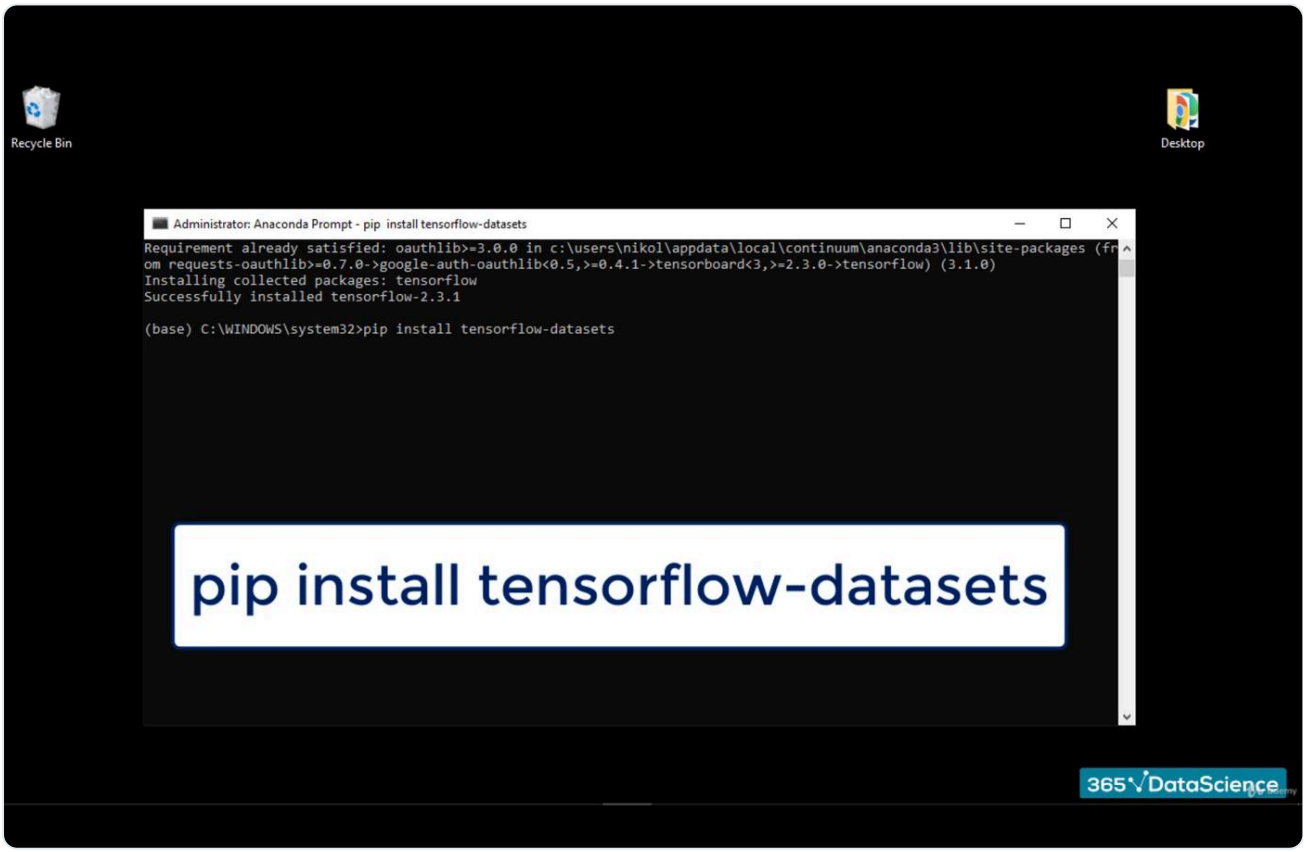

FIGURE 20.4 | MM Administrator: Anaconda Prompt - Pip Install Tensorflow-datasets

Pip Persay means either pip install packages or pip install python. Strange, but true. Now press enter. This should install TensorFlow to with all the accompanying packages. Now, to install TensorFlow data sets, please type PIP, install TensorFlow dash data sets. When the operation is done, you will be set up for the whole course. This is one of the quickest and easiest ways to install modules and packages on your computer in general.

LECTURE 21

Problem to Solve Methodology

We've been rambling about convolutional neural networks for quite some time now, and you may be getting bored already, although I do believe the theory part was necessary. I think it is now time to get more hands on. So in this section, we're going to do just that, create our own CNN and train it on the MLS data set. Exciting, right? If this is your first neural network, then you must be wondering what is the most data set if this is not your first, and then you must be wondering why the missed again? Well, in a nutshell, when talking about neural networks, there is a single data set that everybody has or should have heard of the M.S. data set. It can be considered as the starting point or the trivial problem in AML.

It consists of 70000 images of handwritten digits from zero to nine. Each image has a size of twenty eight by twenty eight pixels and they are in greyscale. Each one is accompanied by a label denoting the digit illustrated. So this dataset is used for classification problems. All the images have been heavily pre-process to contain little to no noise and in general are quite sterile. Thus, a high classification accuracy can be easily achieved and even better due to the small inputs. It doesn't require a lot of computational power.

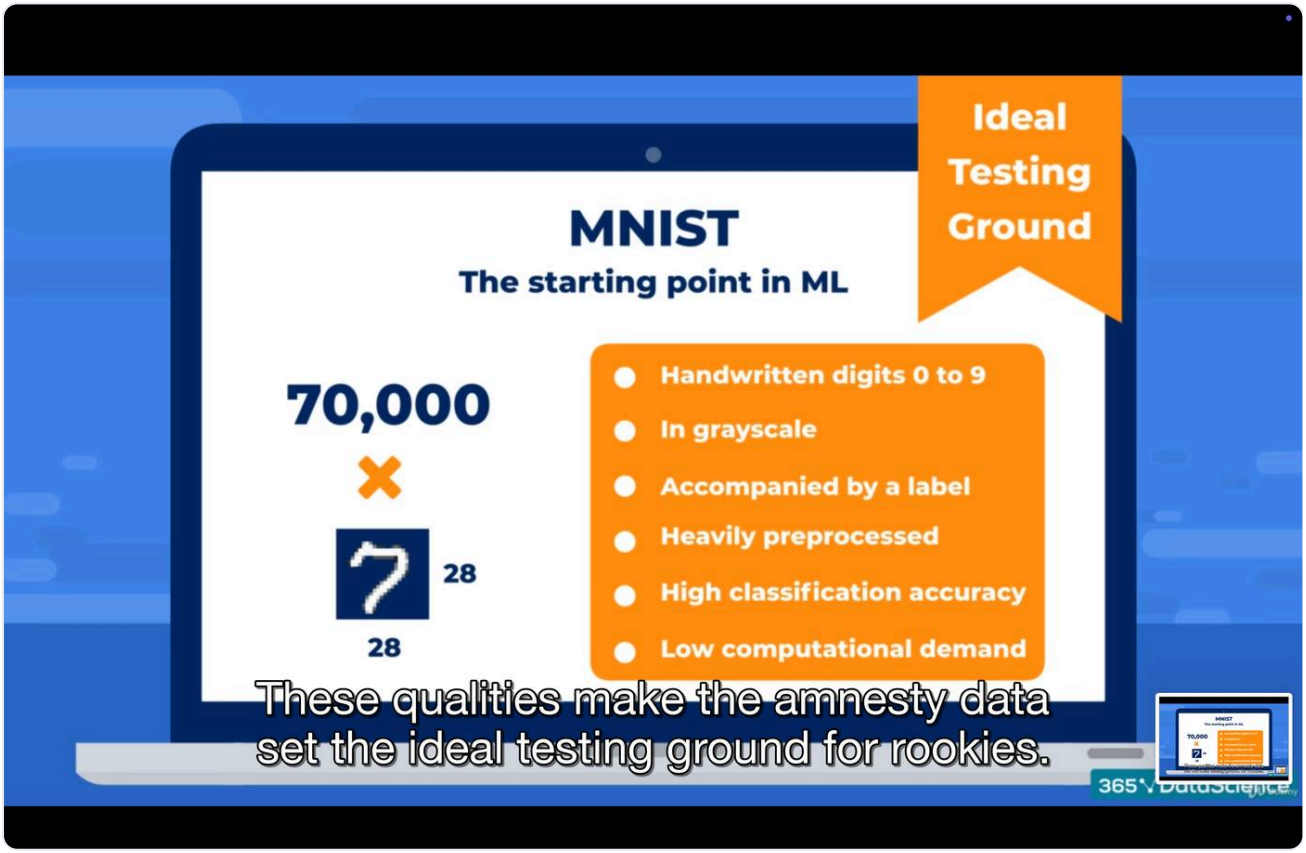

FIGURE 21.1 | The Starting Point in ML

These qualities make the MNIST data set the ideal testing ground for rookies. In addition, if you have followed our courses so far, you know we have already created a simple feed forward neural network on the MNIST before. So that is only natural to do that here as well. Great. The missed

images have been split into two sets, 60000 images form the training sets and 10000 are part of the testing. One noticed that there is no validation set since we are very well prepared. Theoretically, we know that this is unacceptable.

FIGURE 21.2

We will need to create a validation data set to set an early stopping mechanism and prevent overfitting. But let's leave this for later on. For the time being, we should focus on creating a plan to tackle this problem first. What is the problem that we will be solving? Well, it is a simple classification problem. That is to say, our model will take a single image as an input and output the number that is written on that image. How about the methodology? Of course, we are going to tackle this task using a CNN. We will use a Jupyter notebook to write in Python three for the neural networks part itself.

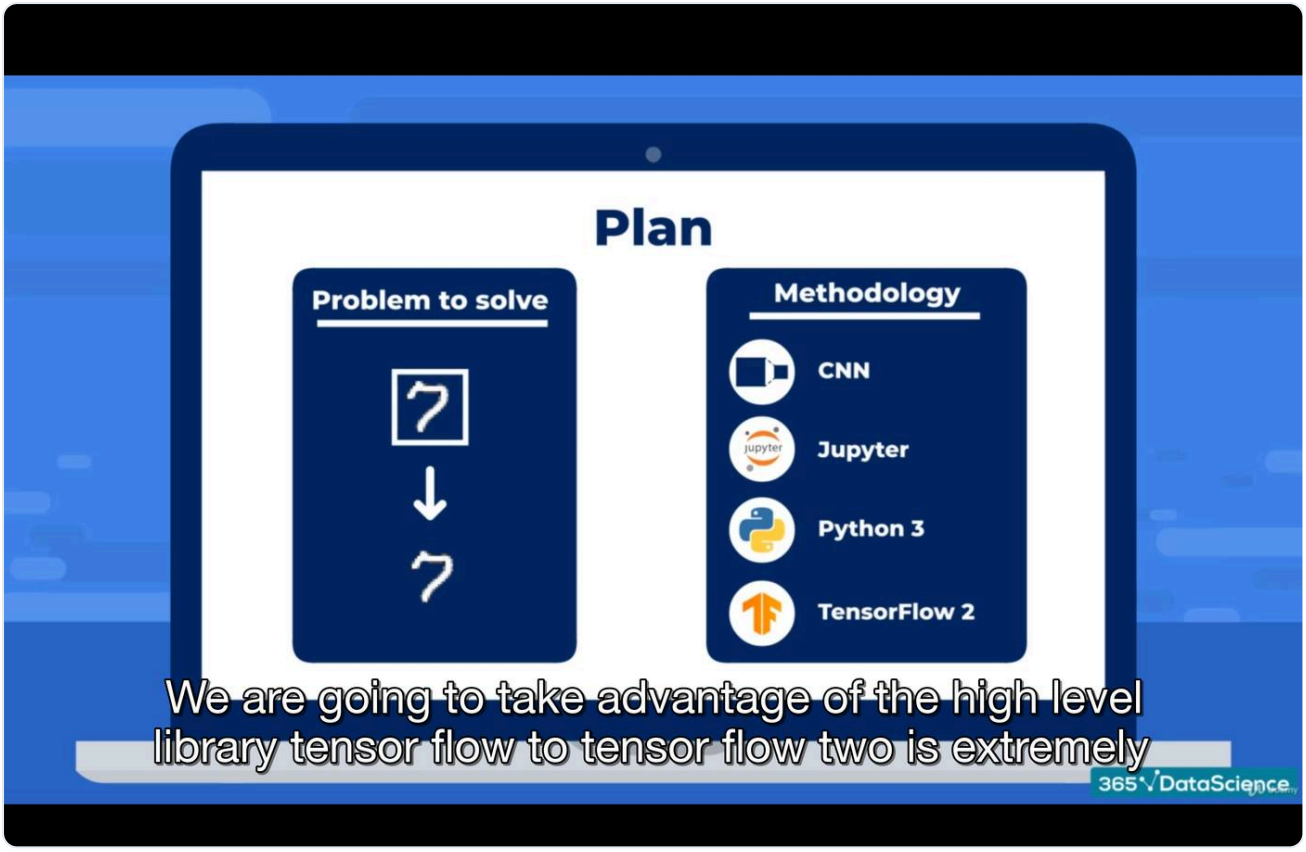

FIGURE 21.3 | Problem to Solve Methodology

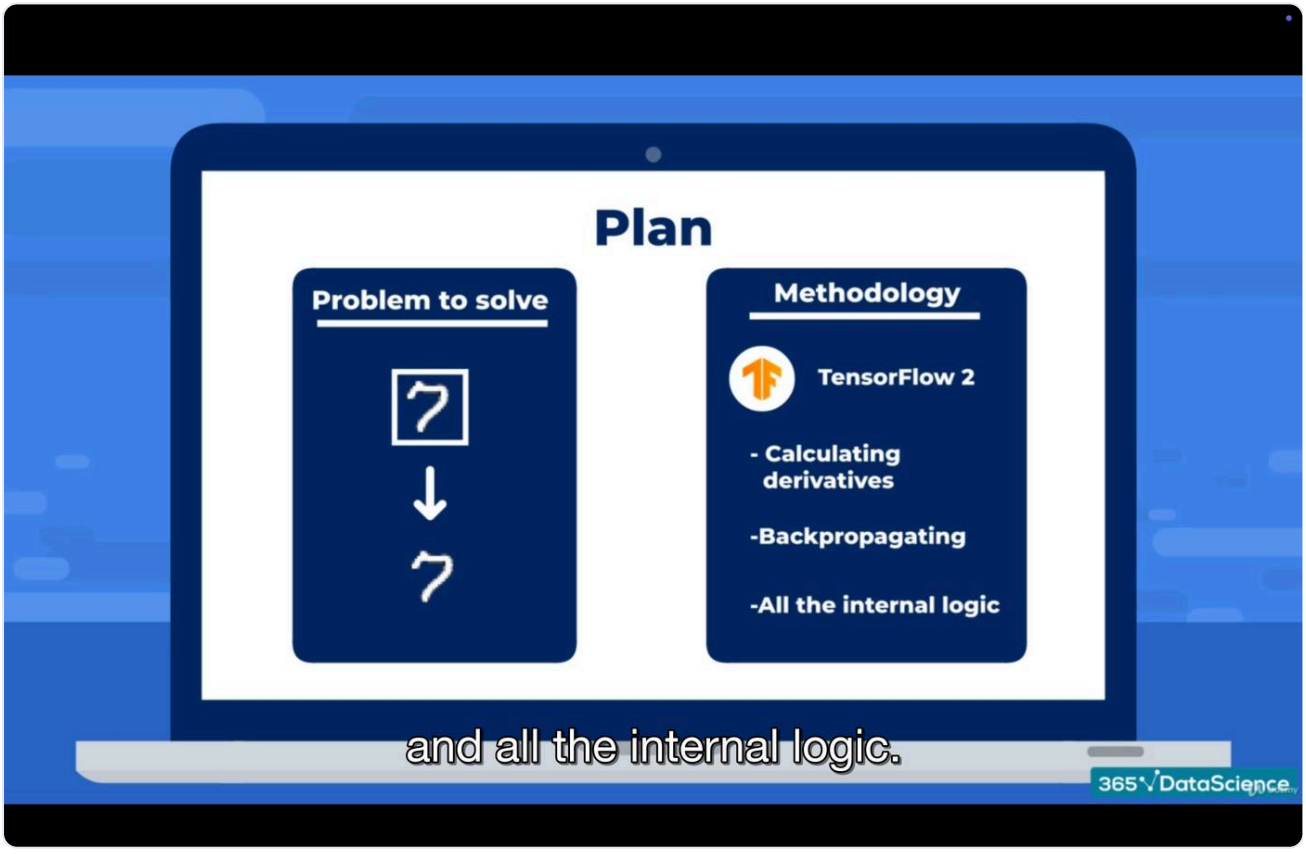

FIGURE 21.4 | Problem to Solve Methodology

We are going to take advantage of the high level library TensorFlow to TensorFlow two is extremely useful as it takes care of all the little details, such as calculating derivatives back propagating and all the internal logic. All we have to do is indicate what layers we want to have in the model and any extra behavior we would like to have. This makes the ideal choice for training neural networks. All right. With that out the way, here's the workflow we are going to follow in this section. In the next lecture, we will reflect on our model and what layers to include in the network. Then we will pre-process the data we have so that it's ready to be trained on afterwards.

We can create the neural network itself, train it and finally test it. So let's get going it.

LECTURE 22

The CNN Architecture

Hello again. In this lecture, we will discuss the architecture we will use for our network architecture in this context means to outline what layers we are going to use, how we are going to connect them, and what some of the hyper parameters would be. So why is it important to construct this architecture now and not just start directly with the code? Well, as with any other project, it is a good practice to think about the big picture first. Sometimes the data set may dictate what our neural network would look like. Other times it may be the opposite. Given our model, we may pre-process the data in a different way to make it more suitable. In any case, it is encouraged to first think about the problem and have some sort of a plan to follow.

All right.

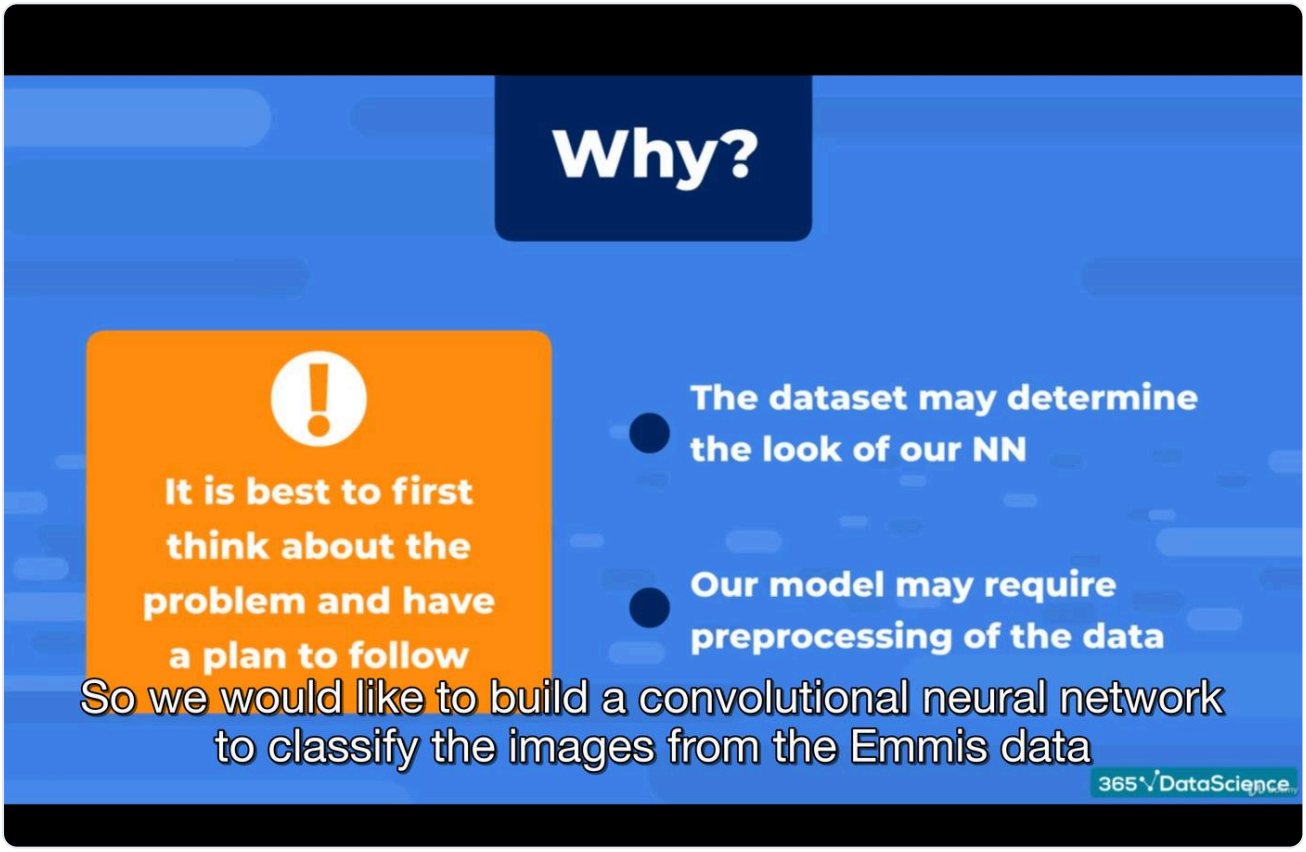

FIGURE 22.1

So we would like to build a convolutional neural network to classify the images from the Mnist data set. What should the first layer be in this case? Given that our input is the image itself, it is only logical to have a convolutional layer.

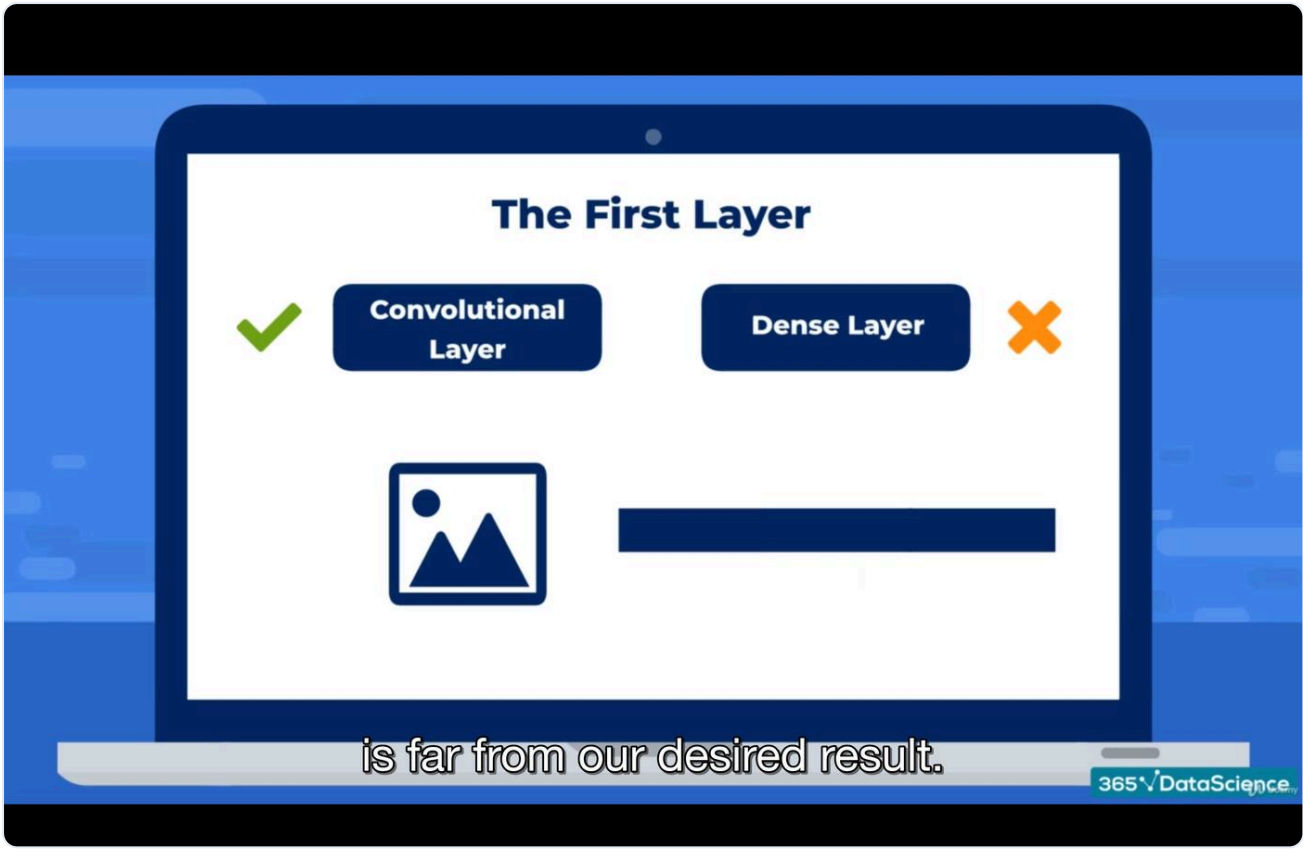

FIGURE 22.2 | The First Layer

First, we can't really have a dense layer as that would unpack the 2D image to a one D vector, which is far from our desired result. Now, in order to keep track of our network, we will visualize the layers in a diagram. There are many ways to do that. For this course we will have a rectangle represent each layer with arrows denoting the connections between the layers.

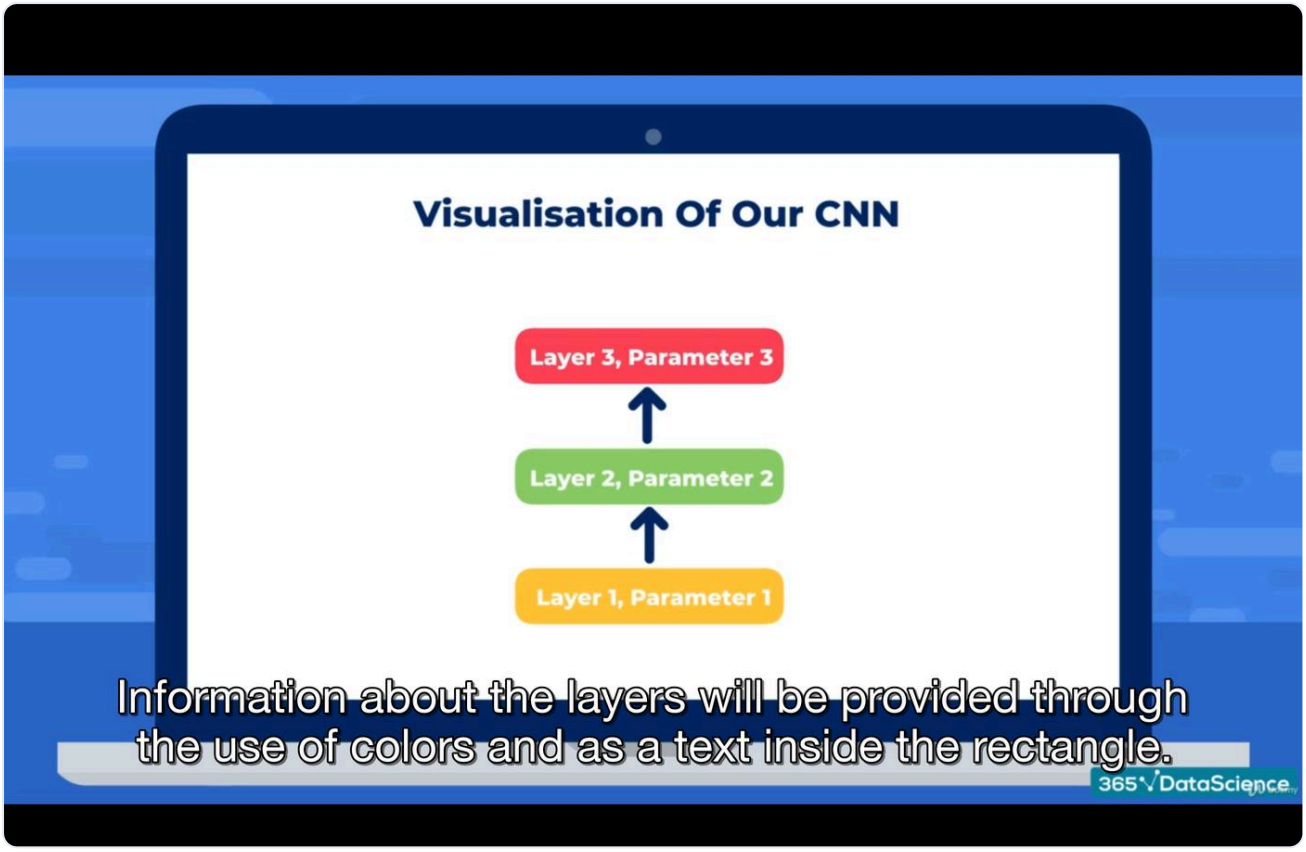

FIGURE 22.3 | Visualisation of Our CNN

Information about the layers will be provided through the use of colors and as a text inside the rectangle. OK, so our first layer is a convolutional one, here is its rectangle inside we have conv, which is short for convolutional. However, we are not done with this layer just yet. There are some hyper parameters that we need to agree upon.

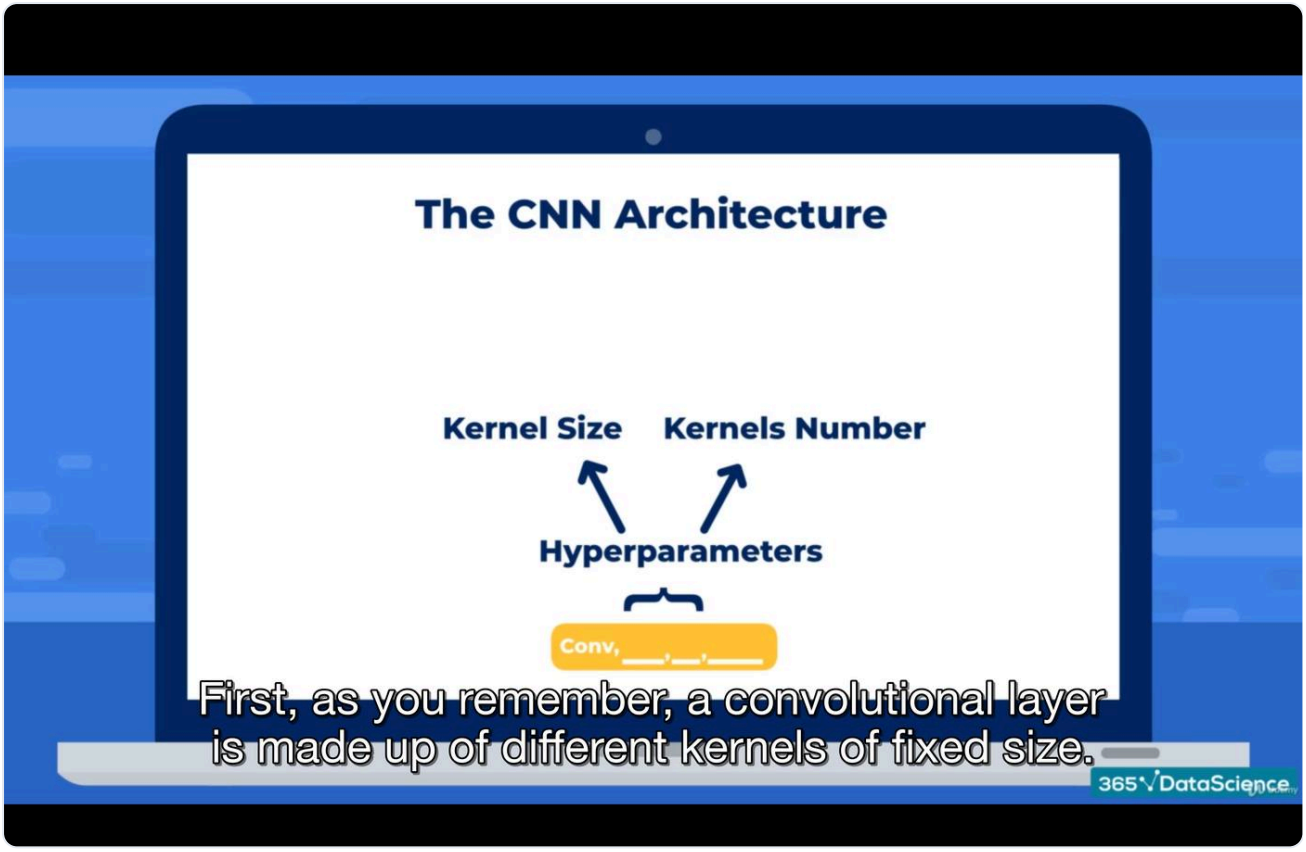

FIGURE 22.4 | The CNN Architecture

First, as you remember, a convolutional layer is made up of different kernels of fixed size. So let's decide what the size of our filters should be. Now is worth mentioning the main mantra of machine learning. Everything depends upon your specific network and data set. Indeed, generalisations about which hyper parameters work or which don't are very hard to make. Usually the only way to know is to just test and experiment with a lot of different values.

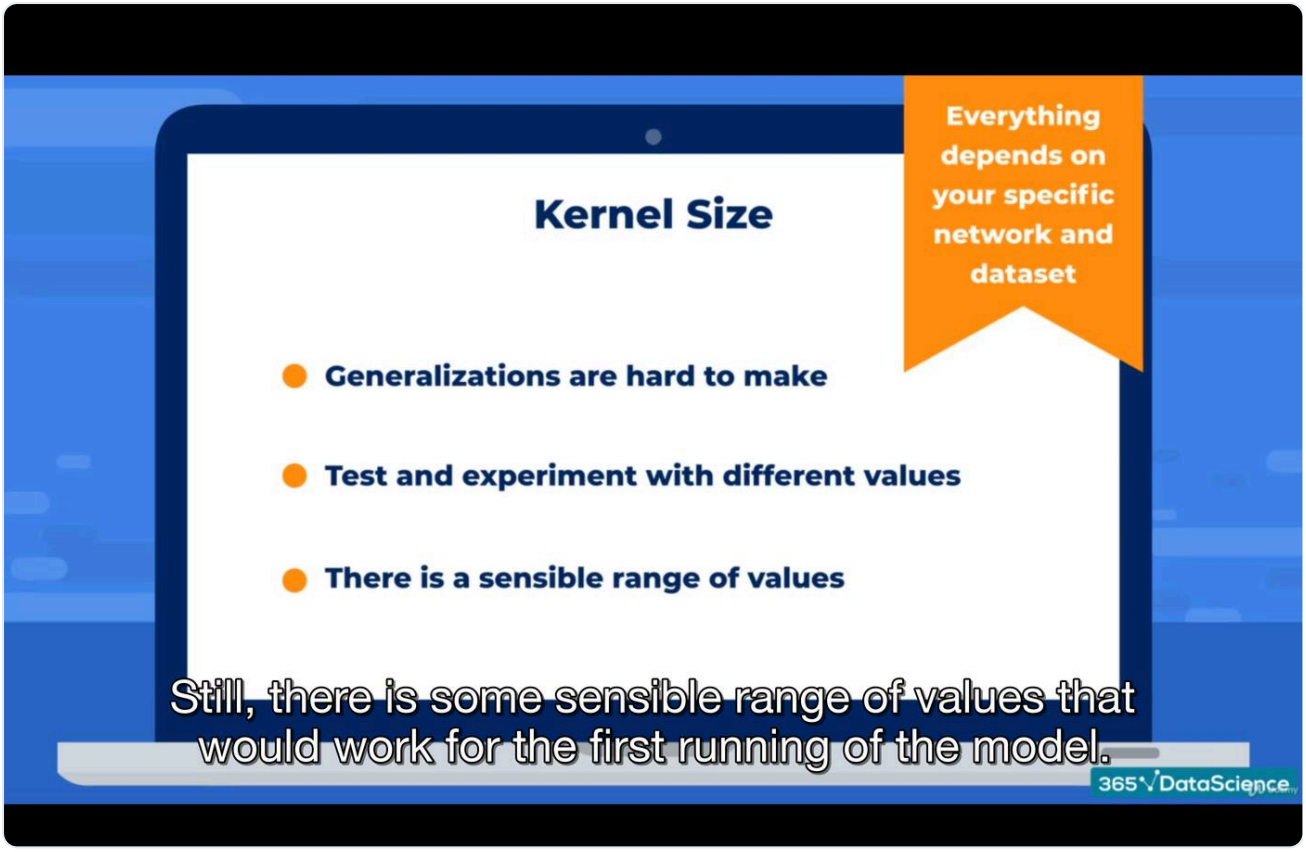

FIGURE 22.5 | Kernel Size

Still, there is some sensible range of values that would work for the first running of the model. Let's think rationally about those. So far we have always discussed and considered square kernels.

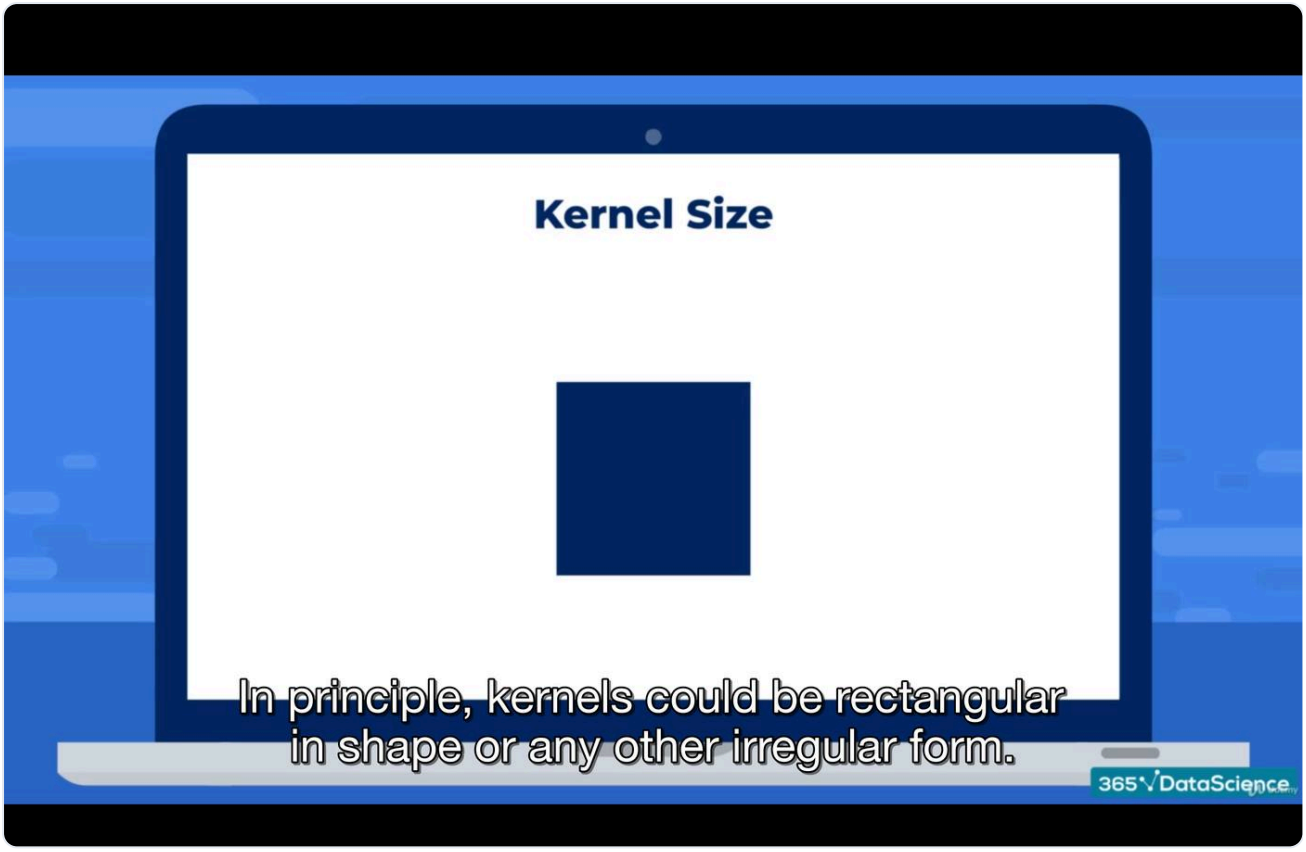

FIGURE 22.6

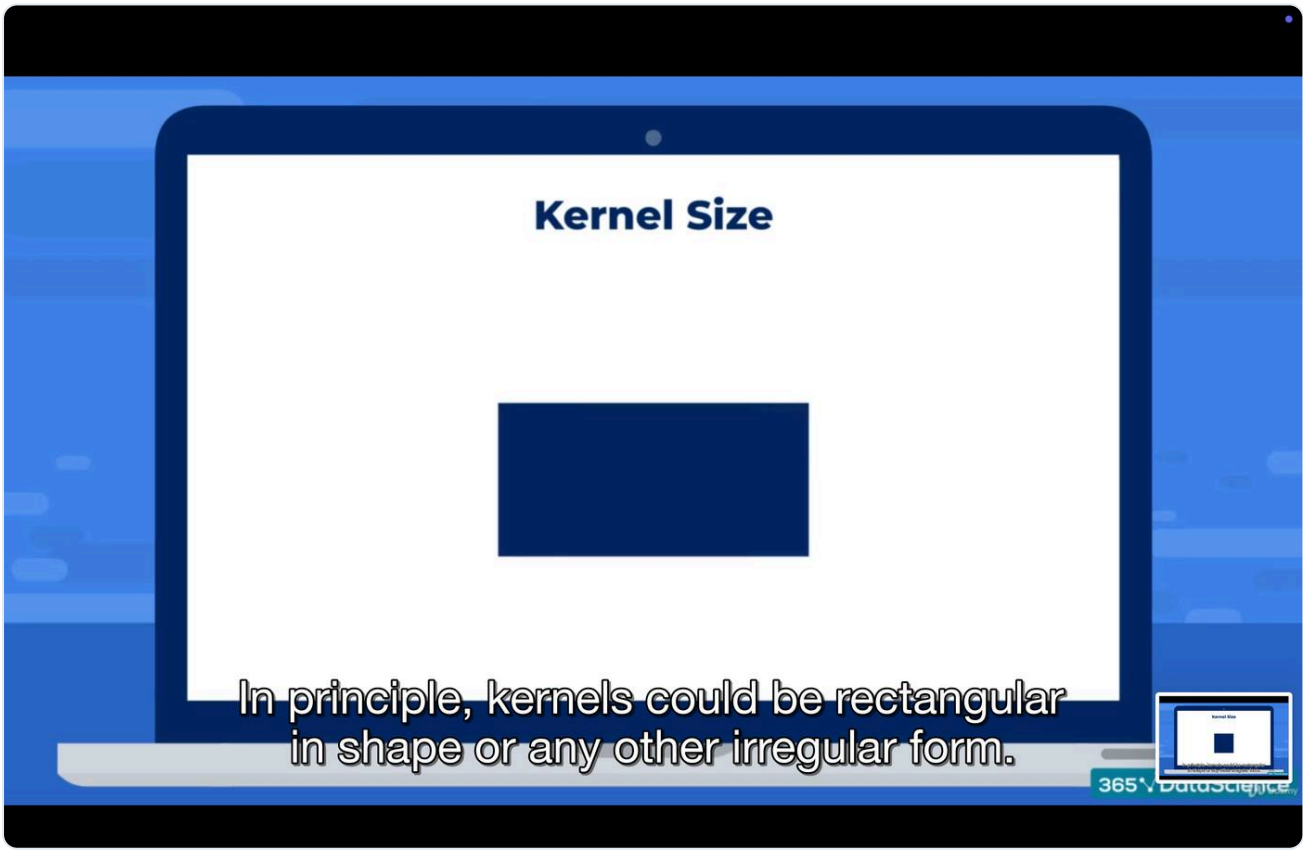

FIGURE 22.7

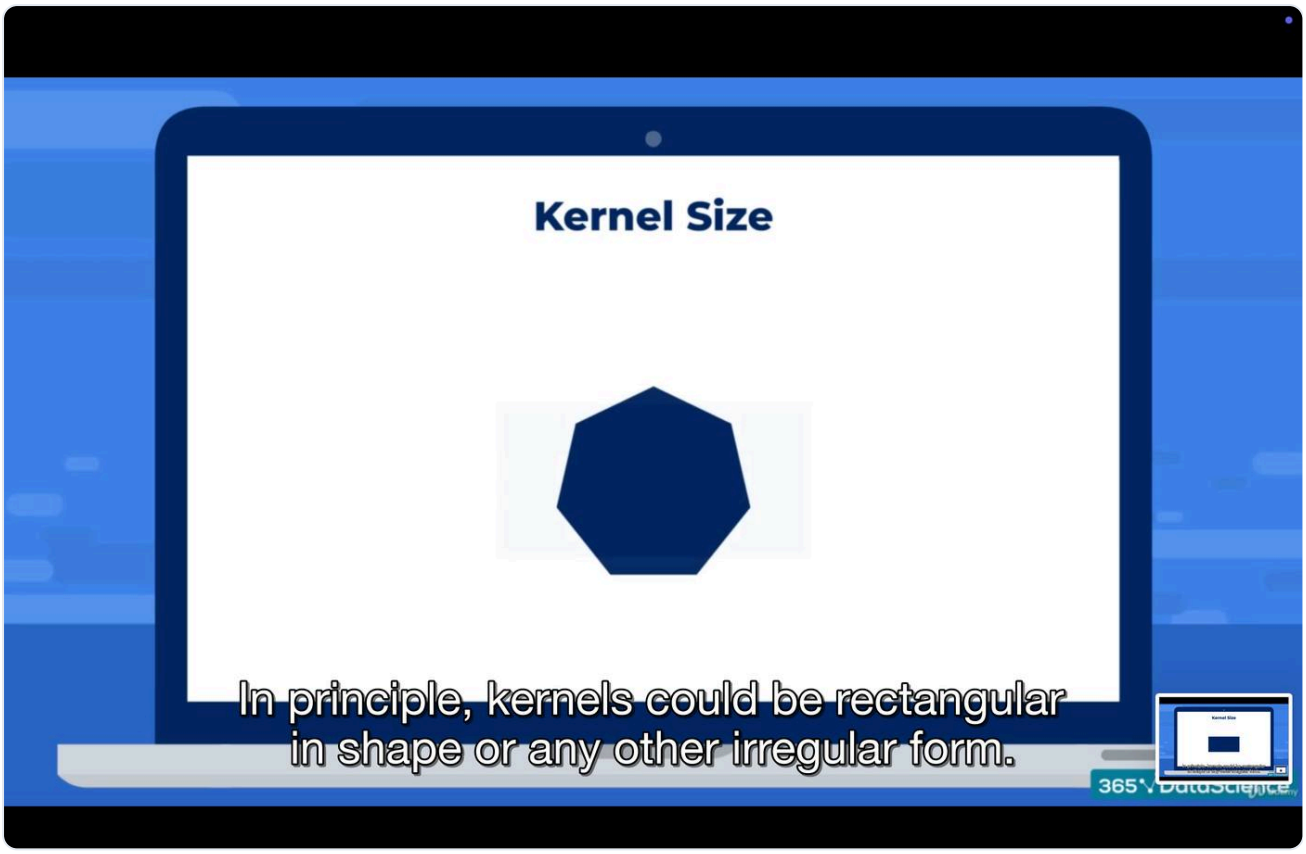

FIGURE 22.8

In principle, kernels could be rectangular in shape or any other irregular form.

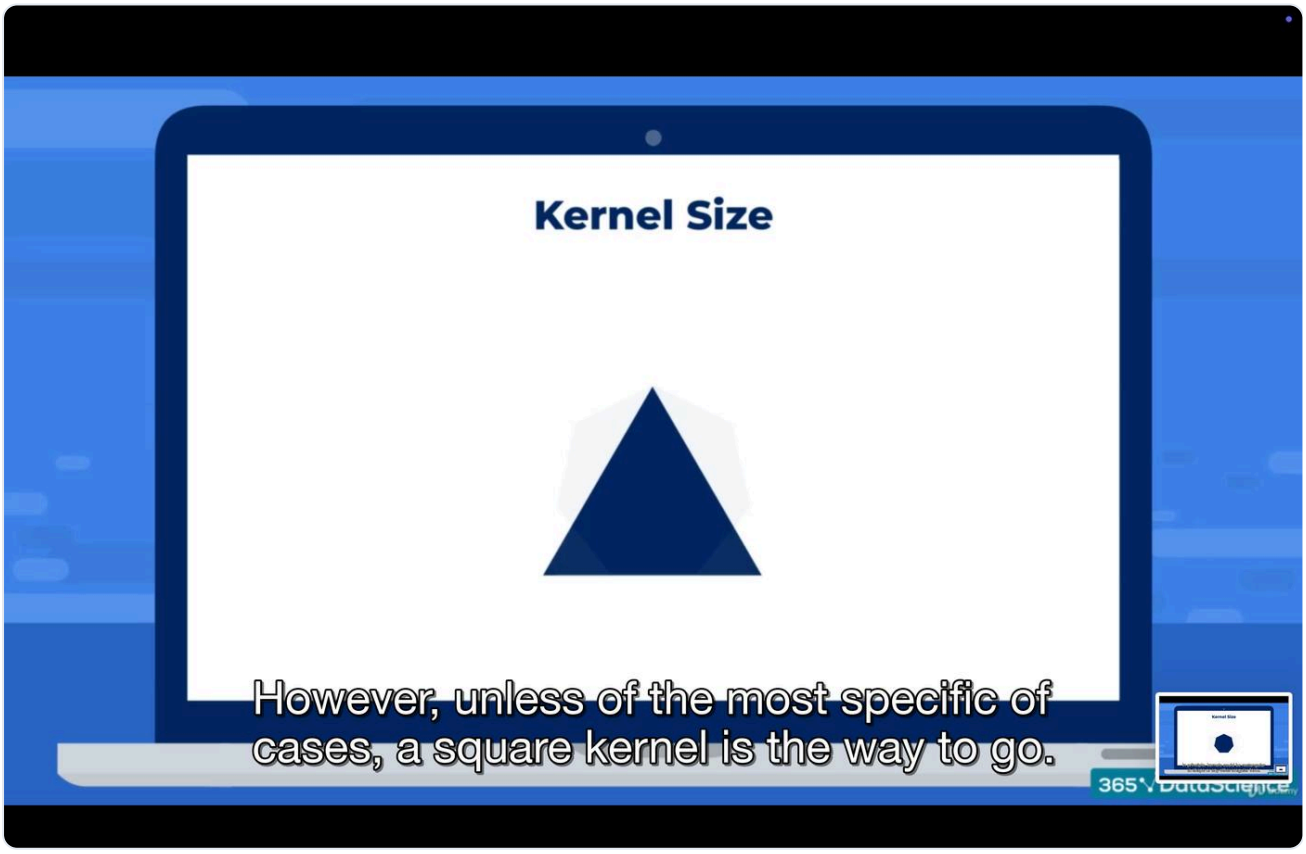

FIGURE 22.9

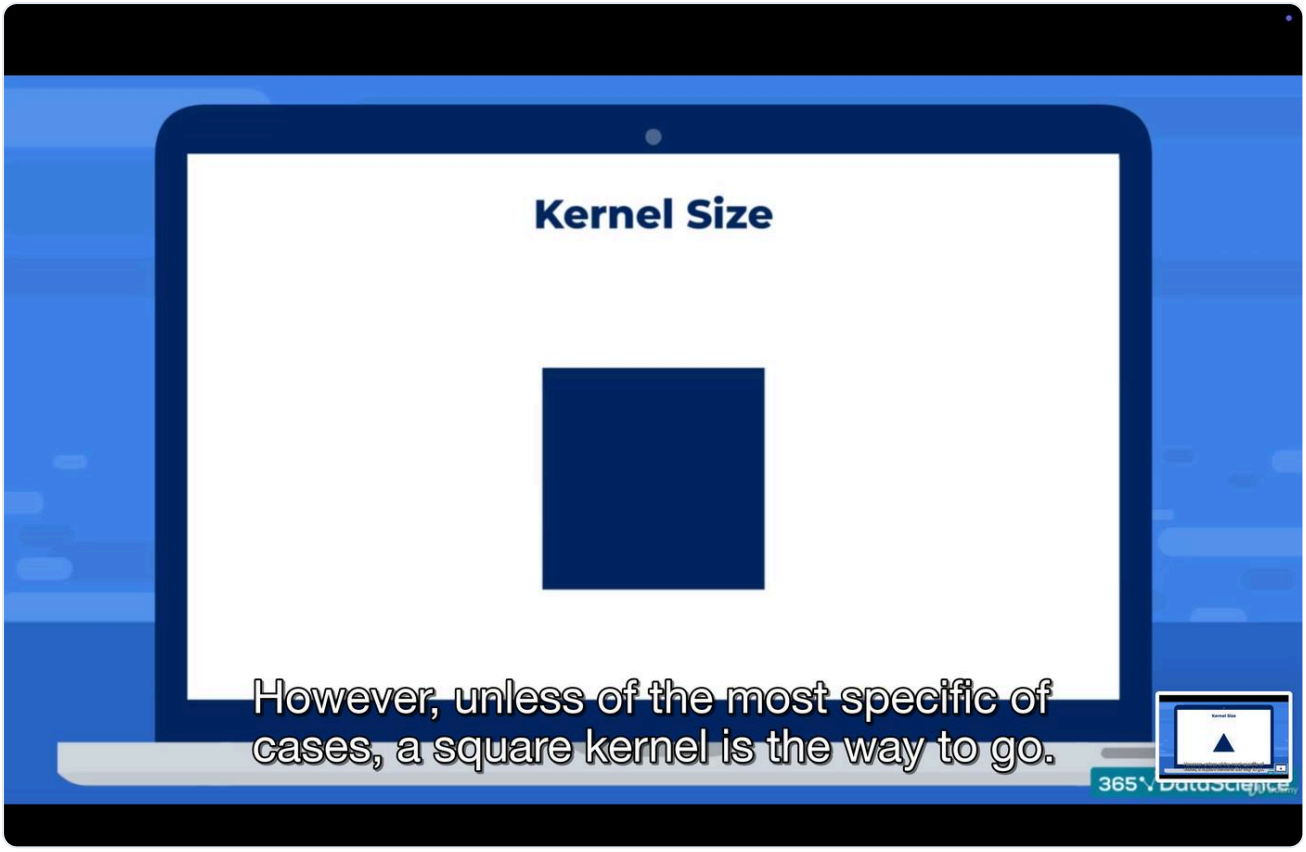

FIGURE 22.10

However, unless of the most specific of cases, a square kernel is the way to go. So we will stick with it. The shape is not the only restriction, though. Our square kernel should have signs with a size. That is an odd number to see why that is the case.

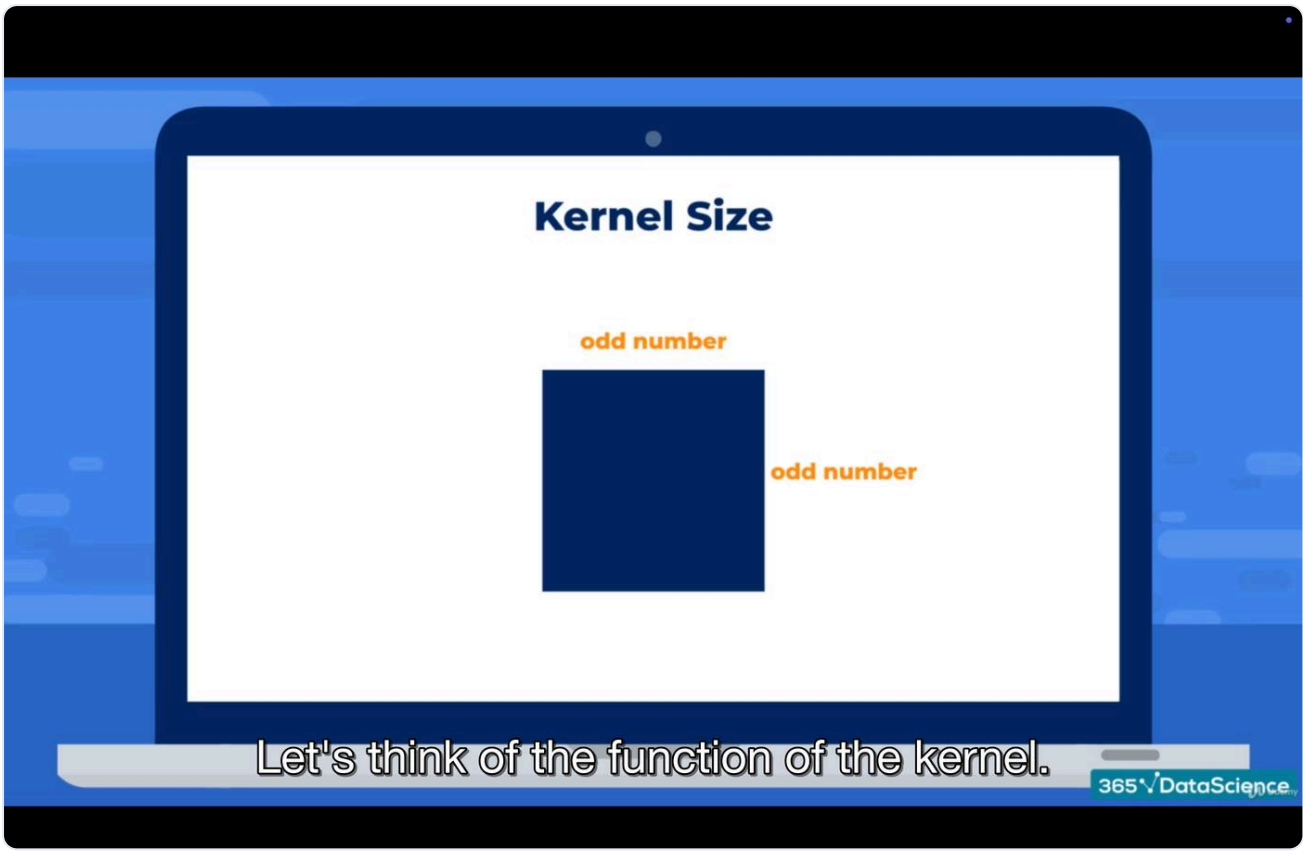

FIGURE 22.11

Let's think of the function of the kernel. It should take one pixel on its surroundings in the original image and manipulate those values to obtain a new value for the pixel.

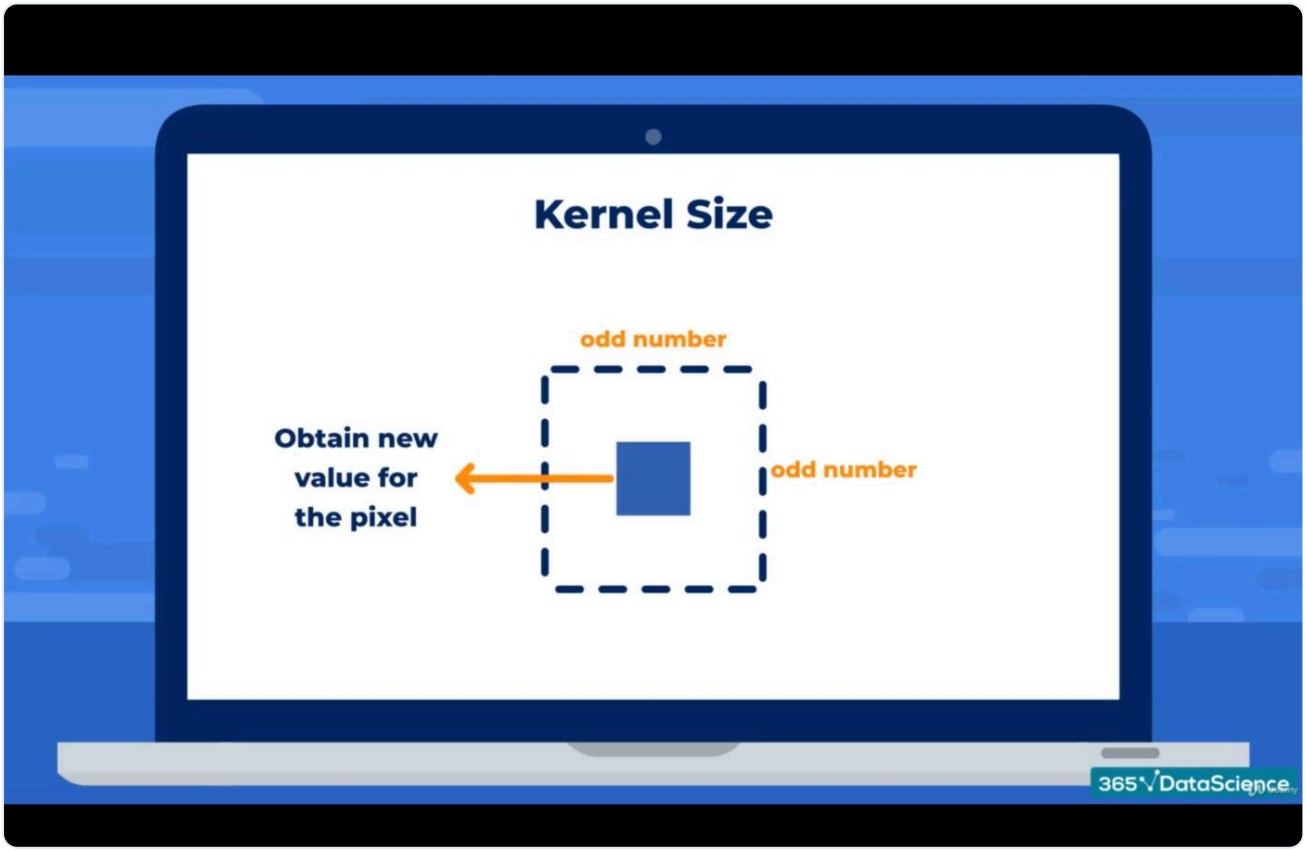

FIGURE 22.12 | Kernel Size

Therefore, an essential quality of a kernel is that it can be associated with a single main pixel.

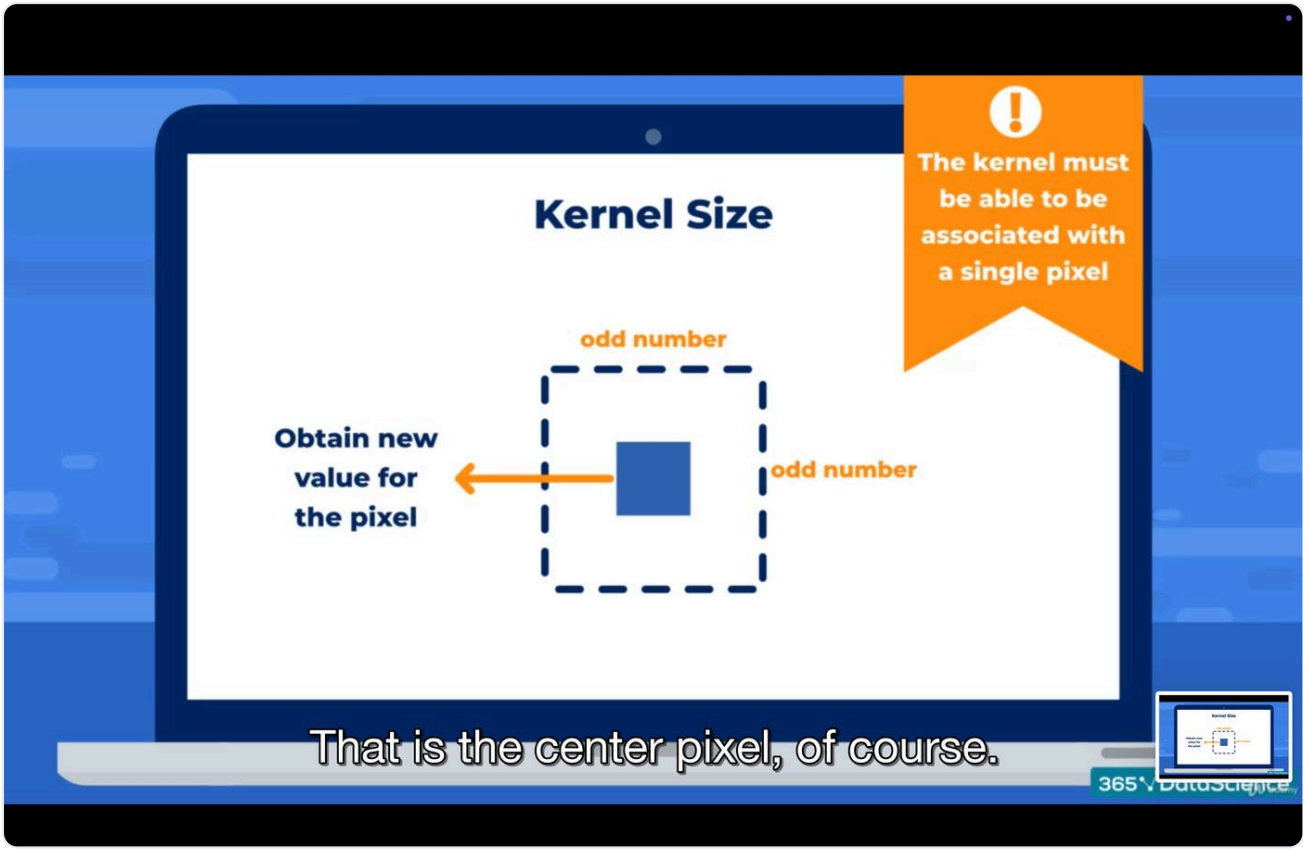

FIGURE 22.13 | Kernel Size

That is the center pixel, of course. But what happens when we have an even size? For instance, four by four.

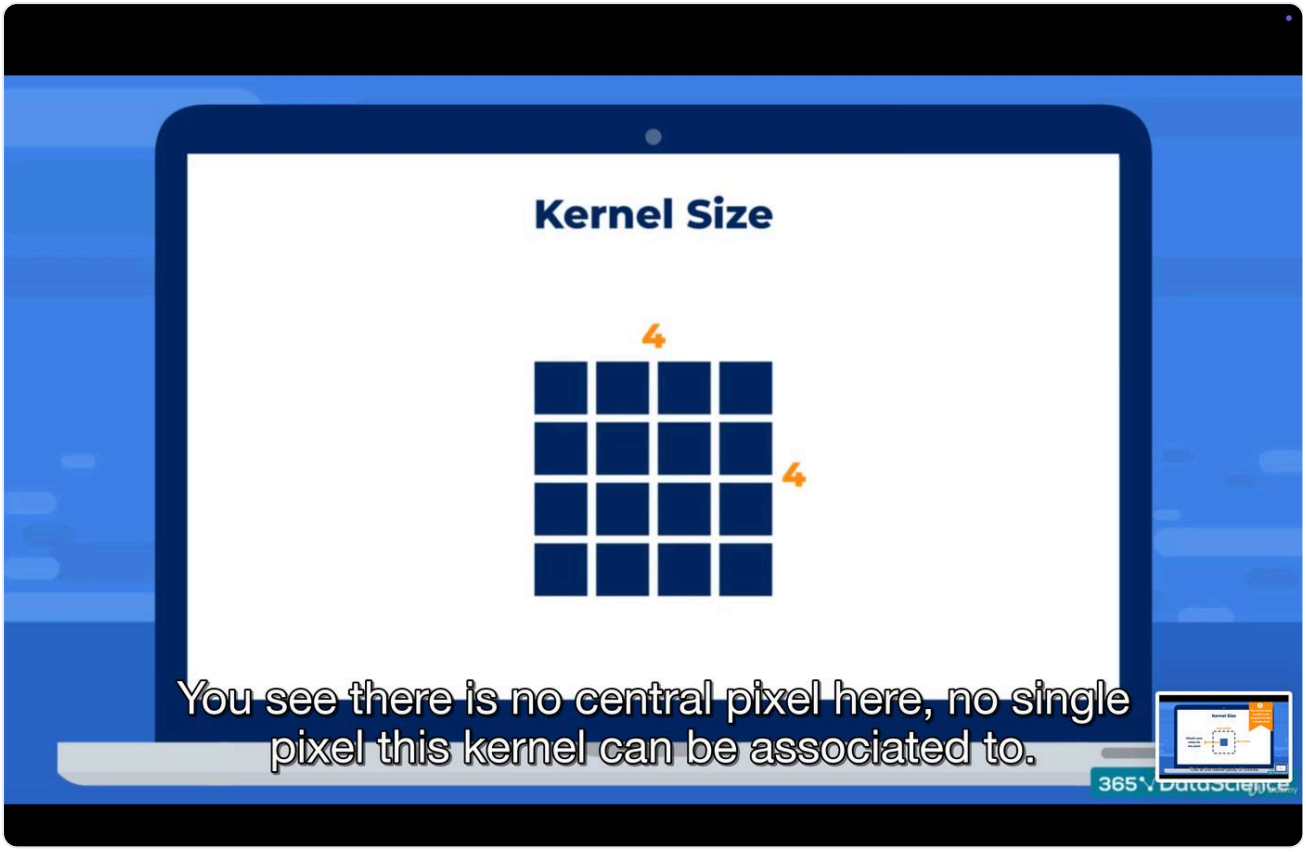

FIGURE 22.14 | Kernel Size

You see there is no central pixel here, no single pixel this kernel can be associated to.

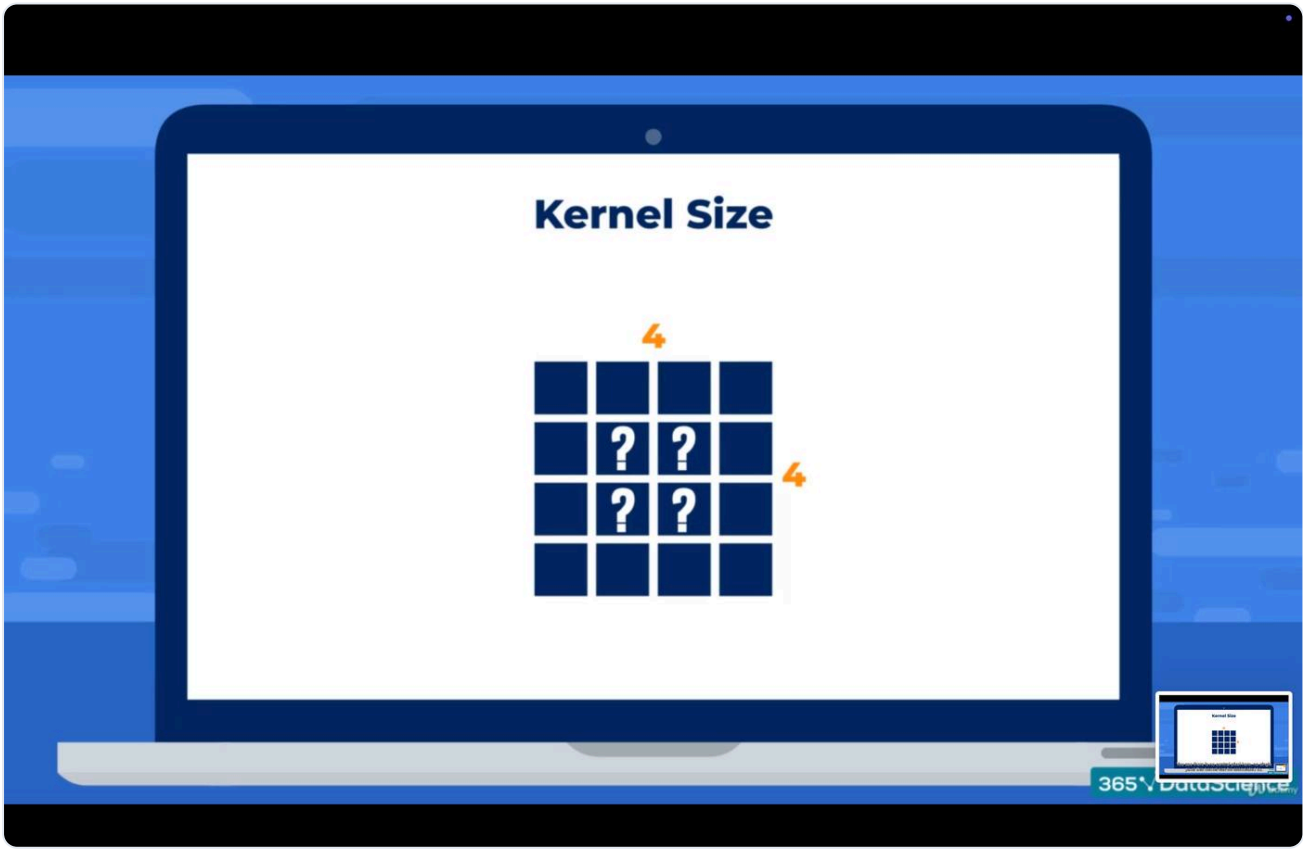

FIGURE 22.15

That's why if we want to focus on a certain pixel, the allowed kernel sizes are always odd.

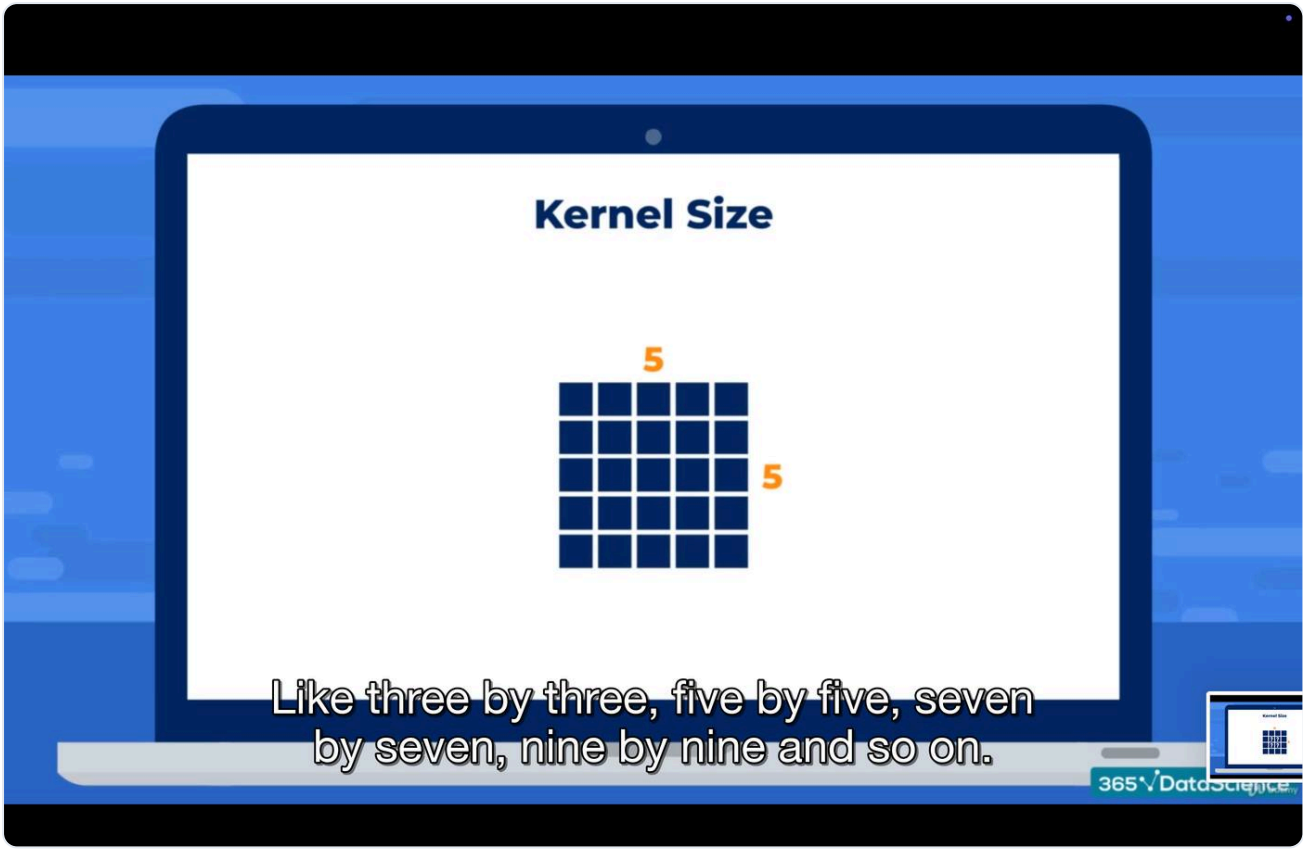

FIGURE 22.16 | Kernel Size

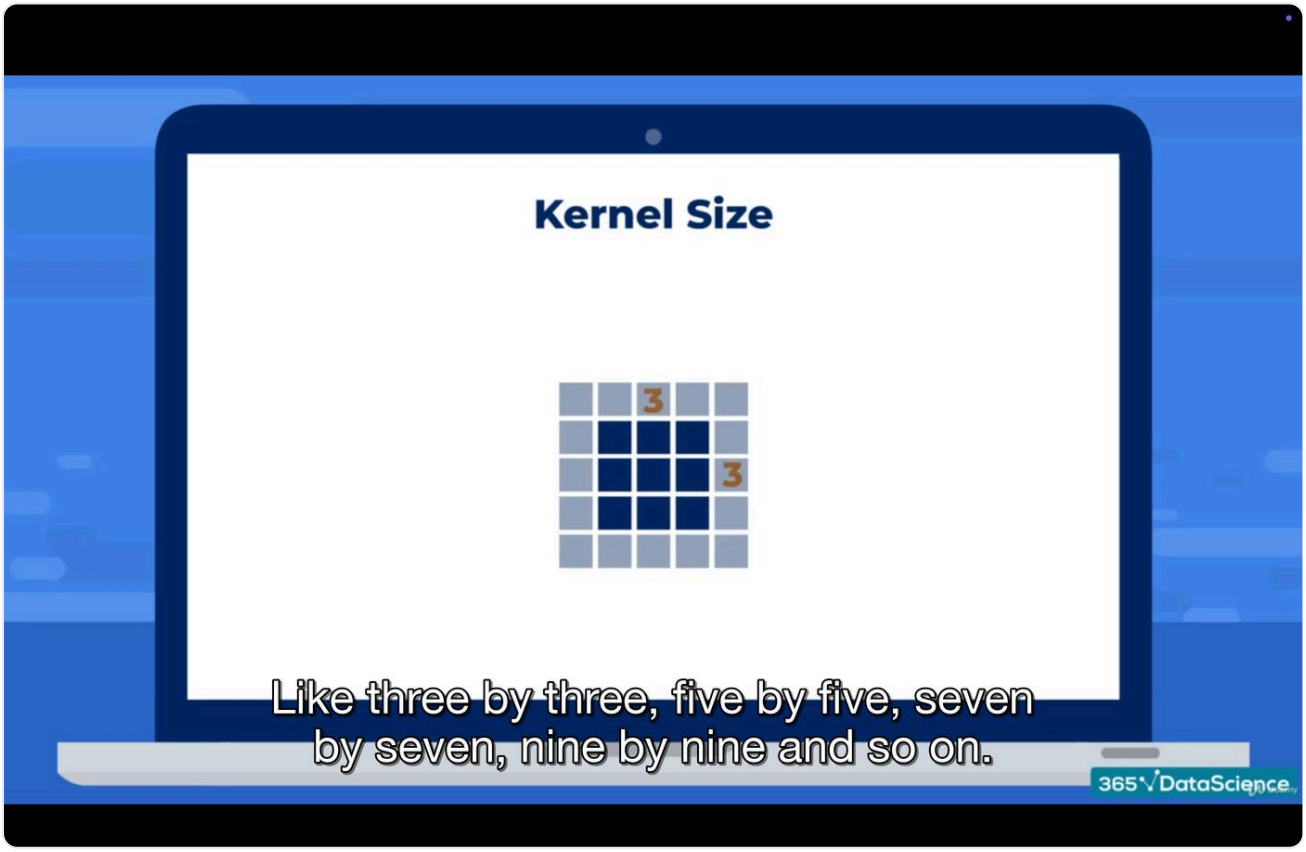

FIGURE 22.17 | Kernel Size

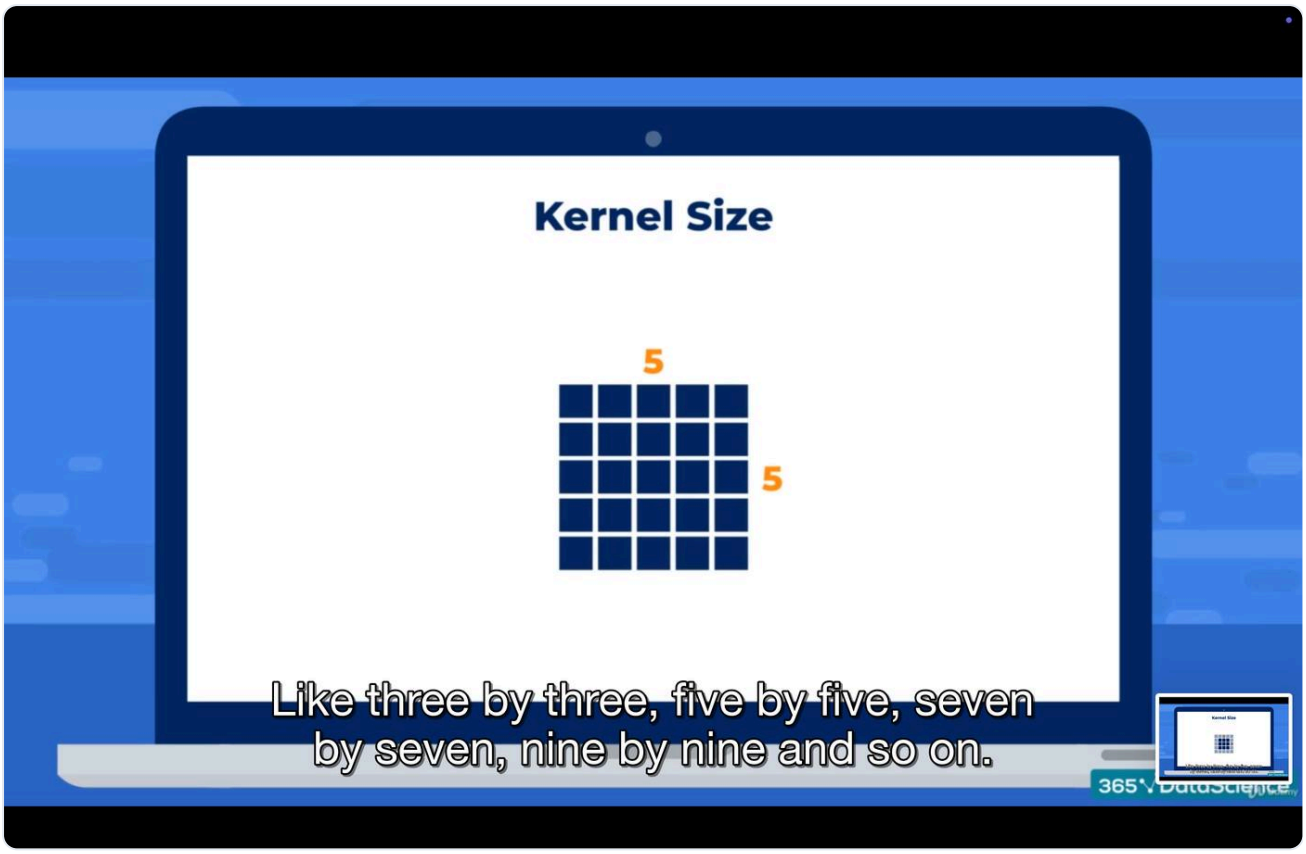

FIGURE 22.18 | Kernel Size

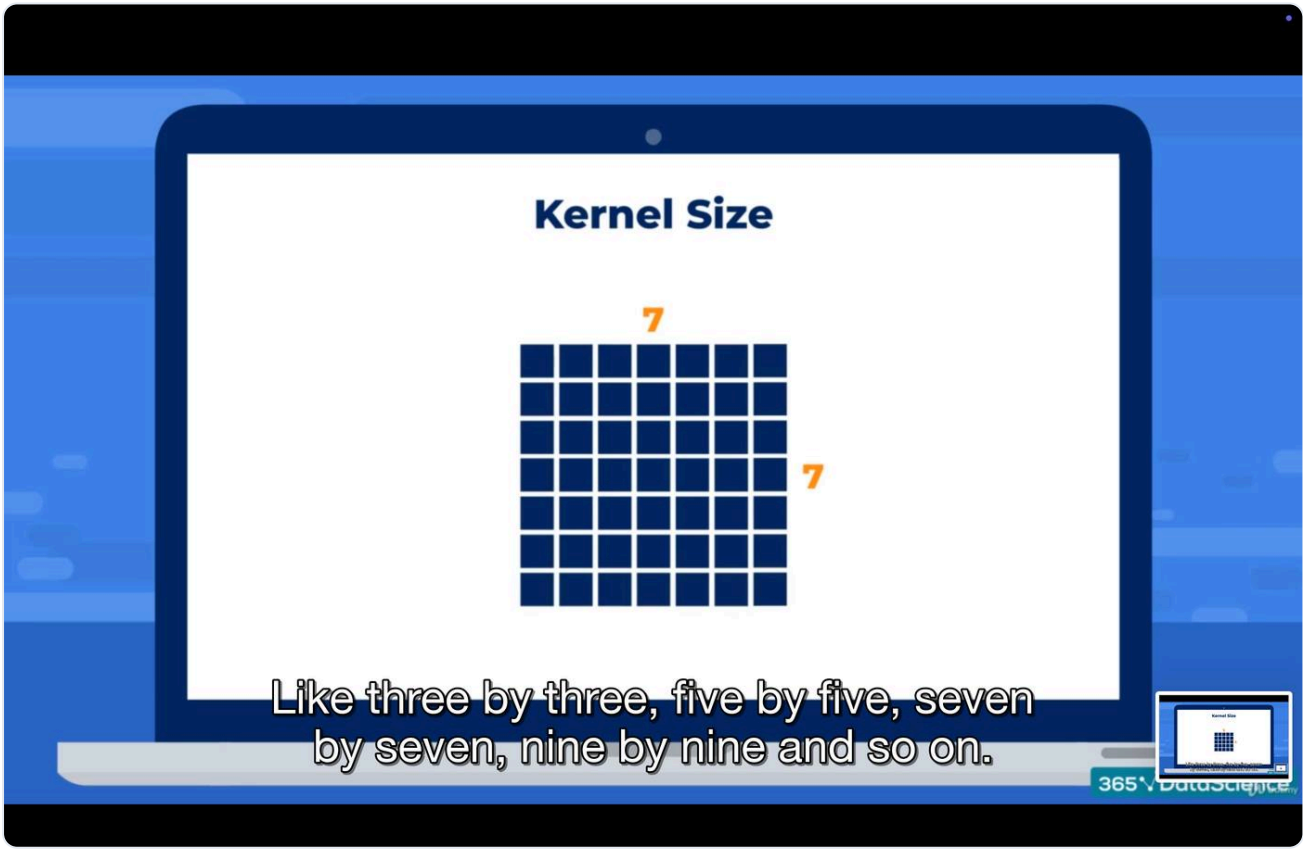

FIGURE 22.19 | Kernel Size

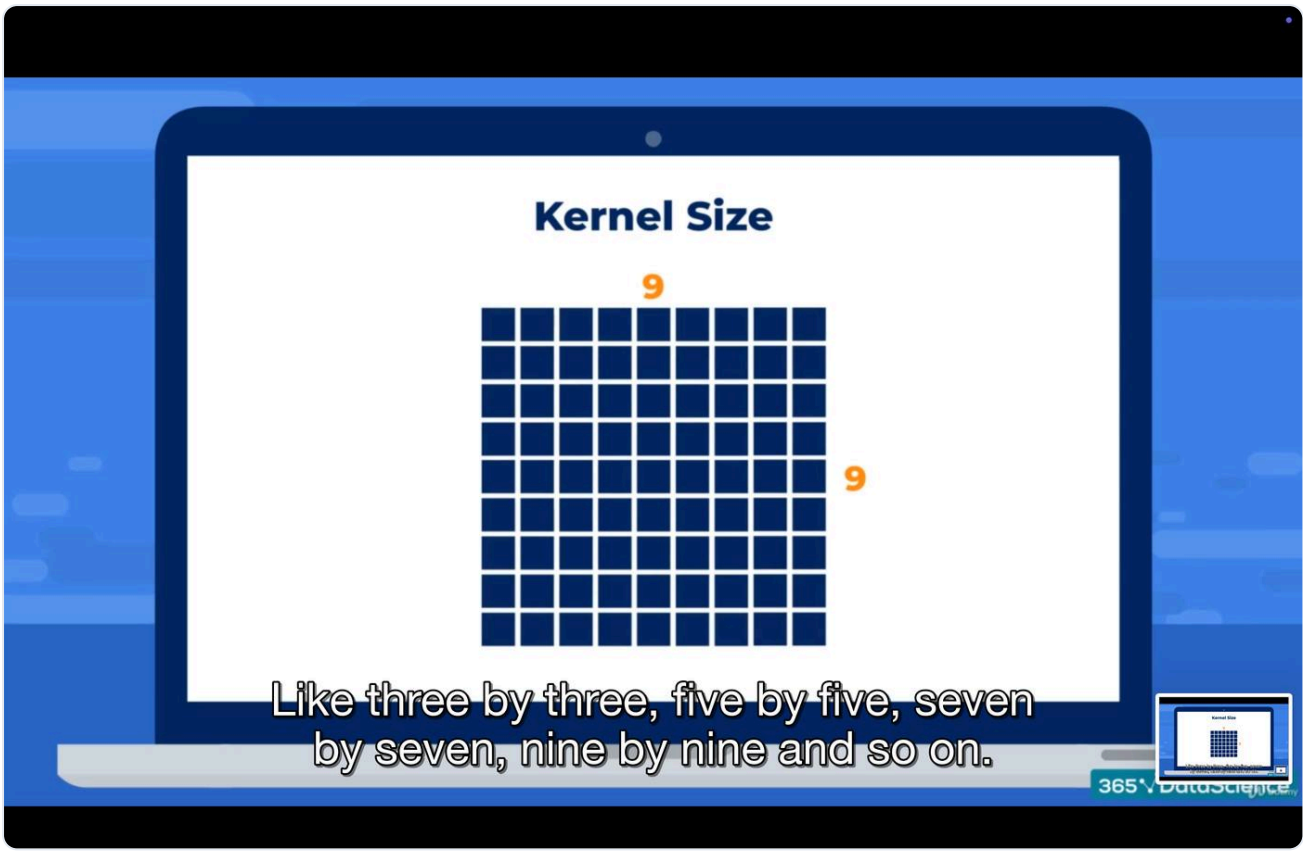

FIGURE 22.20 | Kernel Size

Like three by three, five by five, seven by seven, nine by nine and so on.

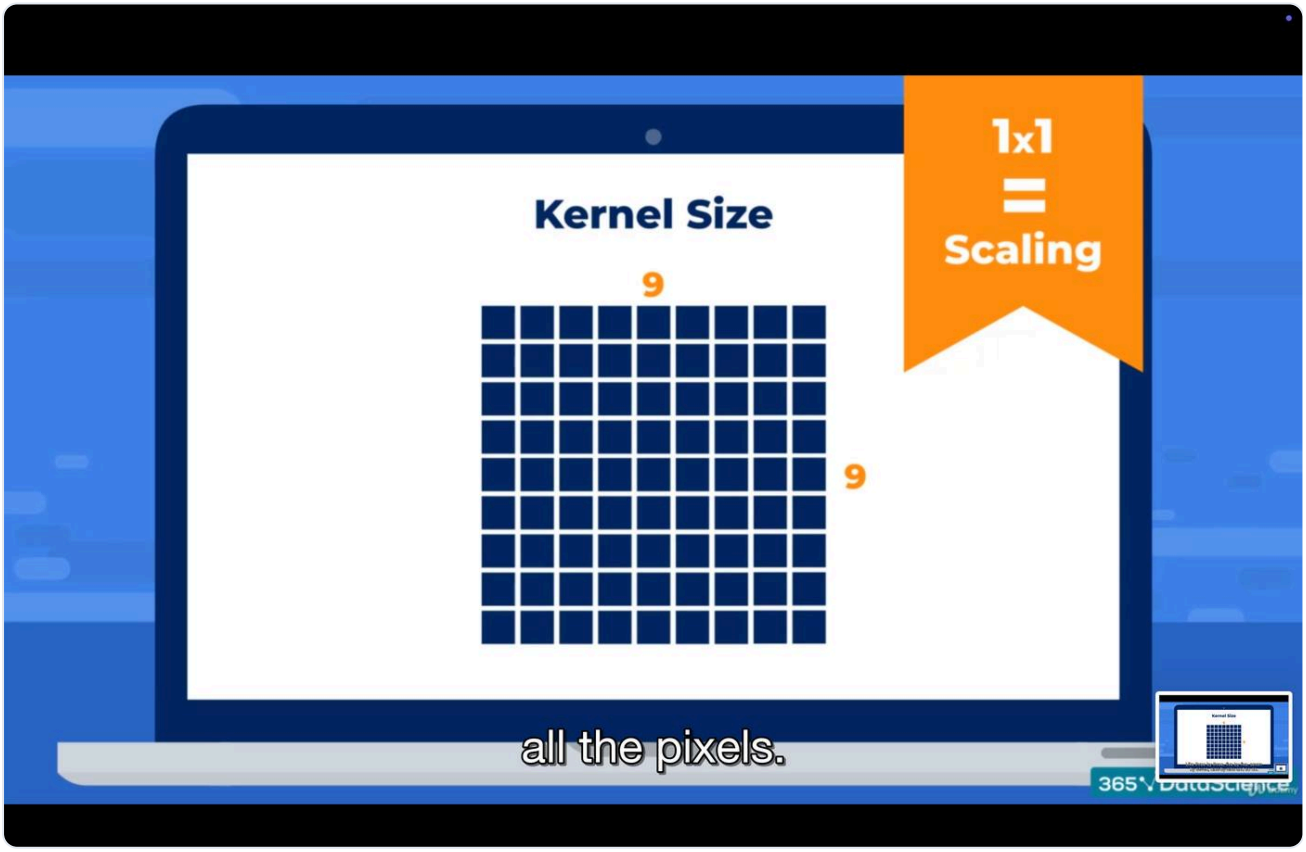

FIGURE 22.21

We skipped one by one as it only considers the value of one pixel and thus is equivalent to just scaling all the pixels. All right. But what are the suitable values for our particular situation?

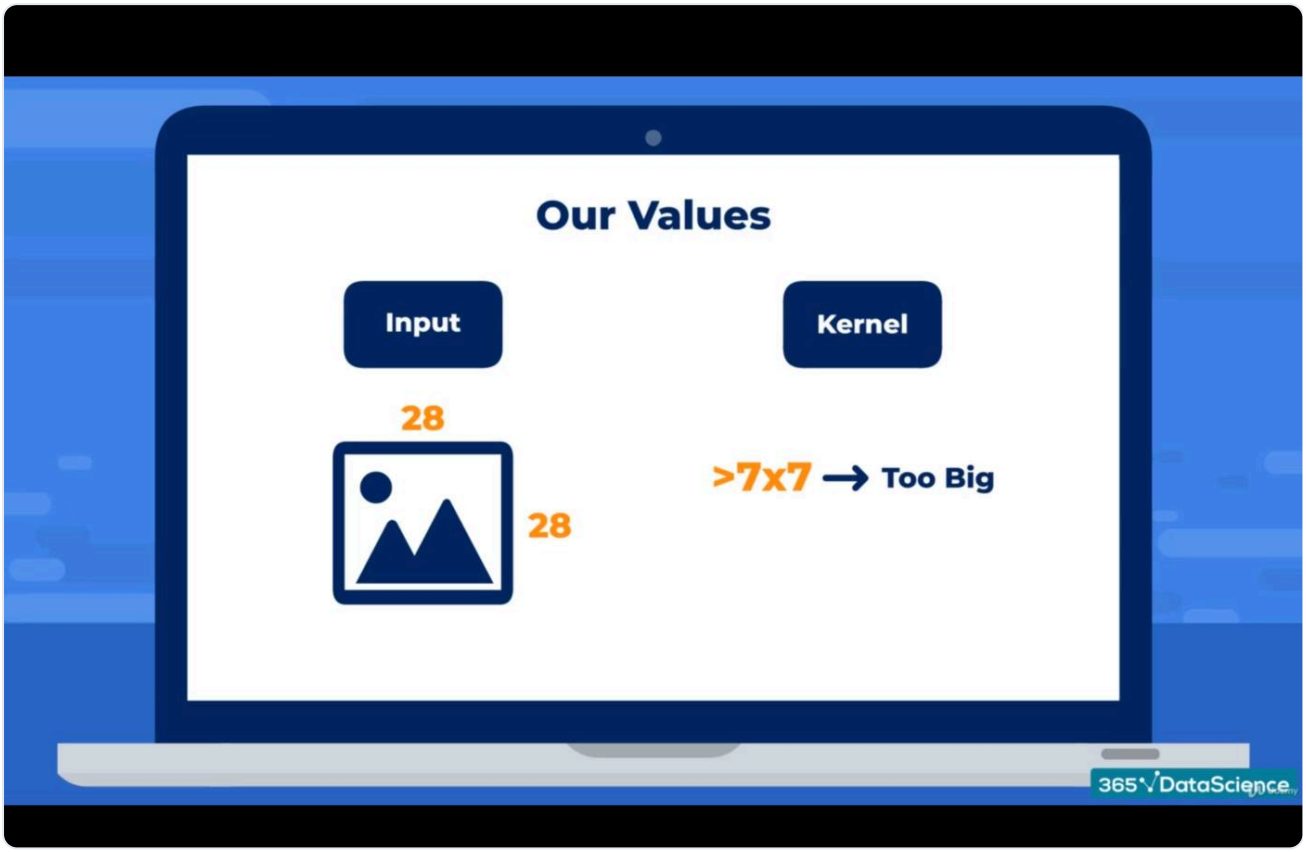

FIGURE 22.22 | Our Values

Well, our input is an image of size twenty eight by twenty eight. So we may not want our filter to be bigger than seven by seven because that might be too big compared to the image. On the other hand, in general it is believed that there is no minimum size for the kernel.

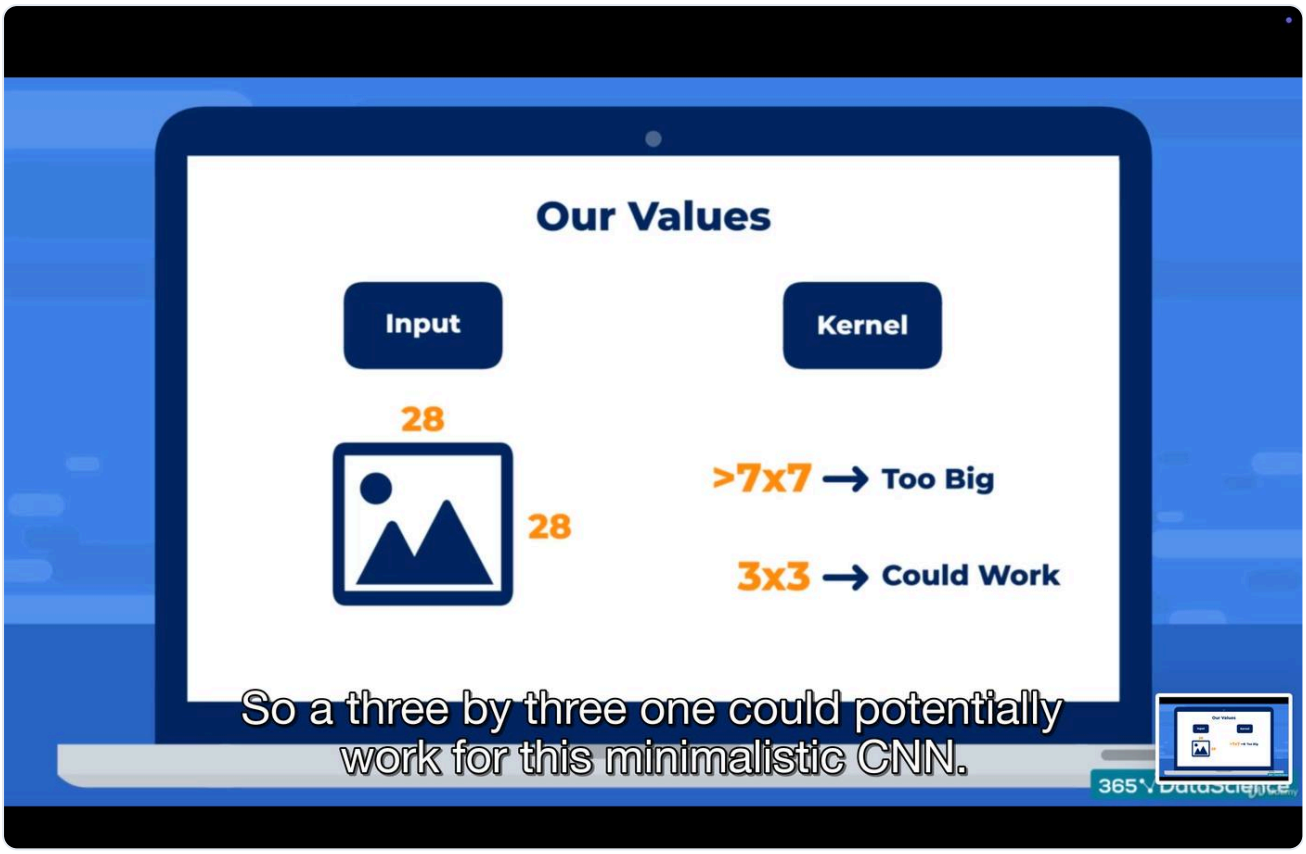

FIGURE 22.23 | Our Values

So a three by three one could potentially work for this minimalistic CNN. Let's settle for the middle ground kernel size of five by five.

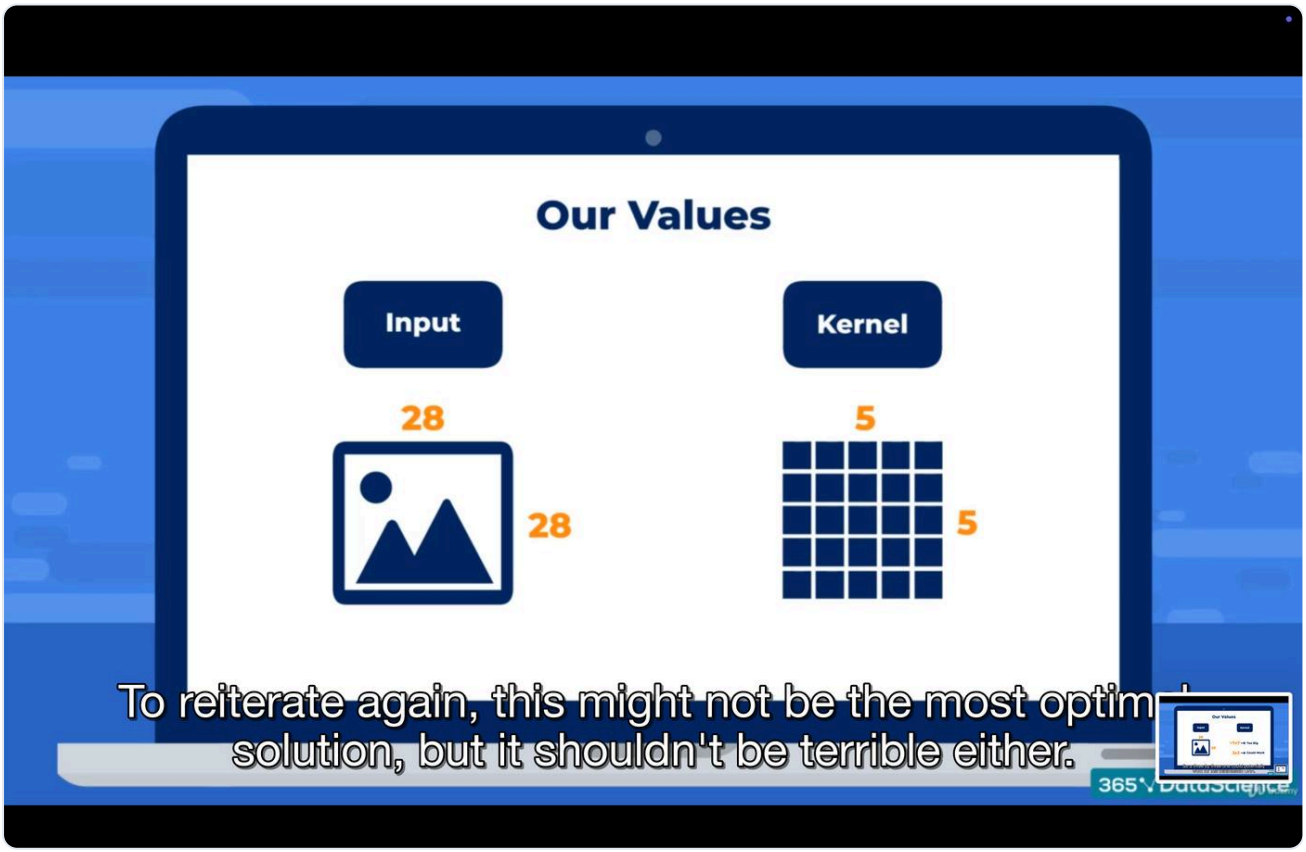

FIGURE 22.24 | Our Values

To reiterate again, this might not be the most optimal solution, but it shouldn't be terrible either. Now we need to determine how many kernels of this kind we want to have.

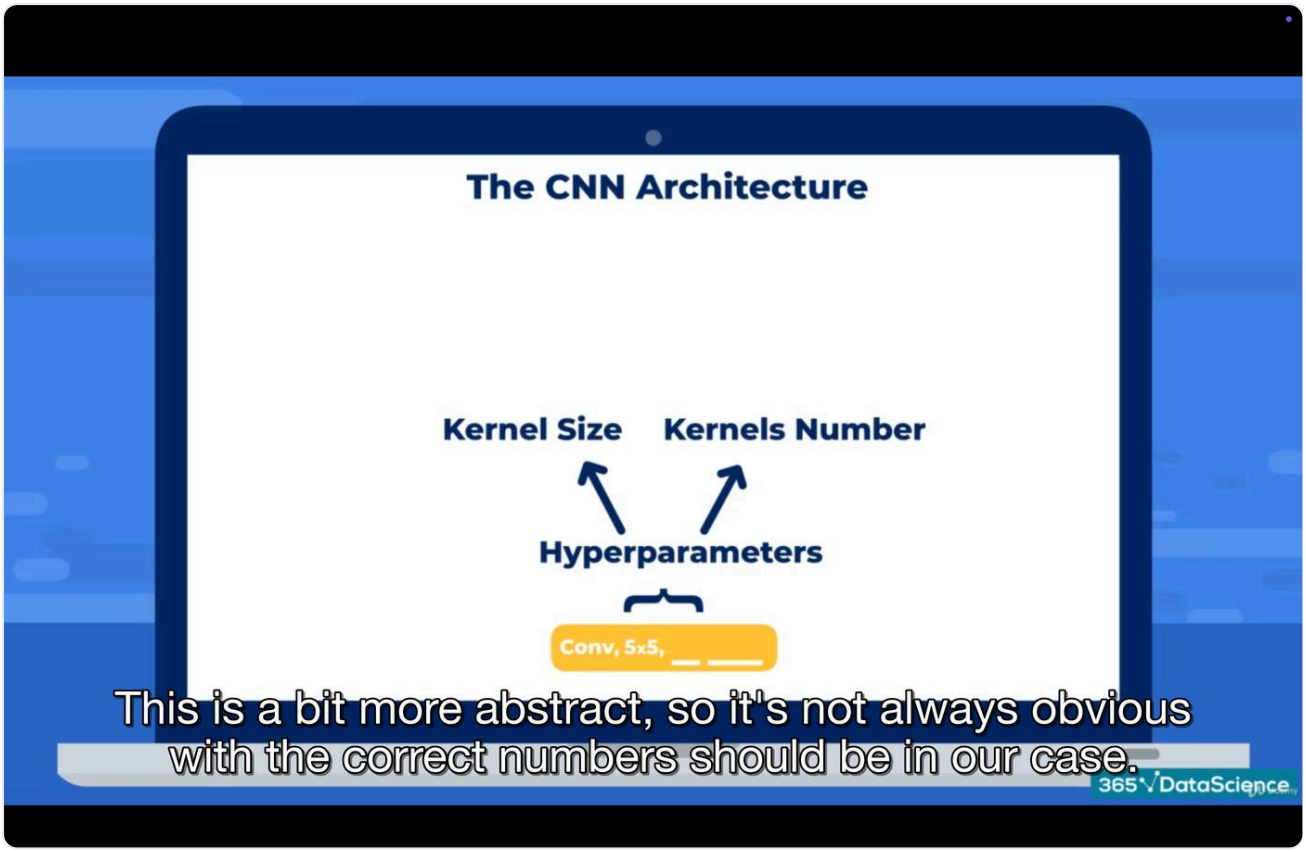

FIGURE 22.25 | The CNN Architecture

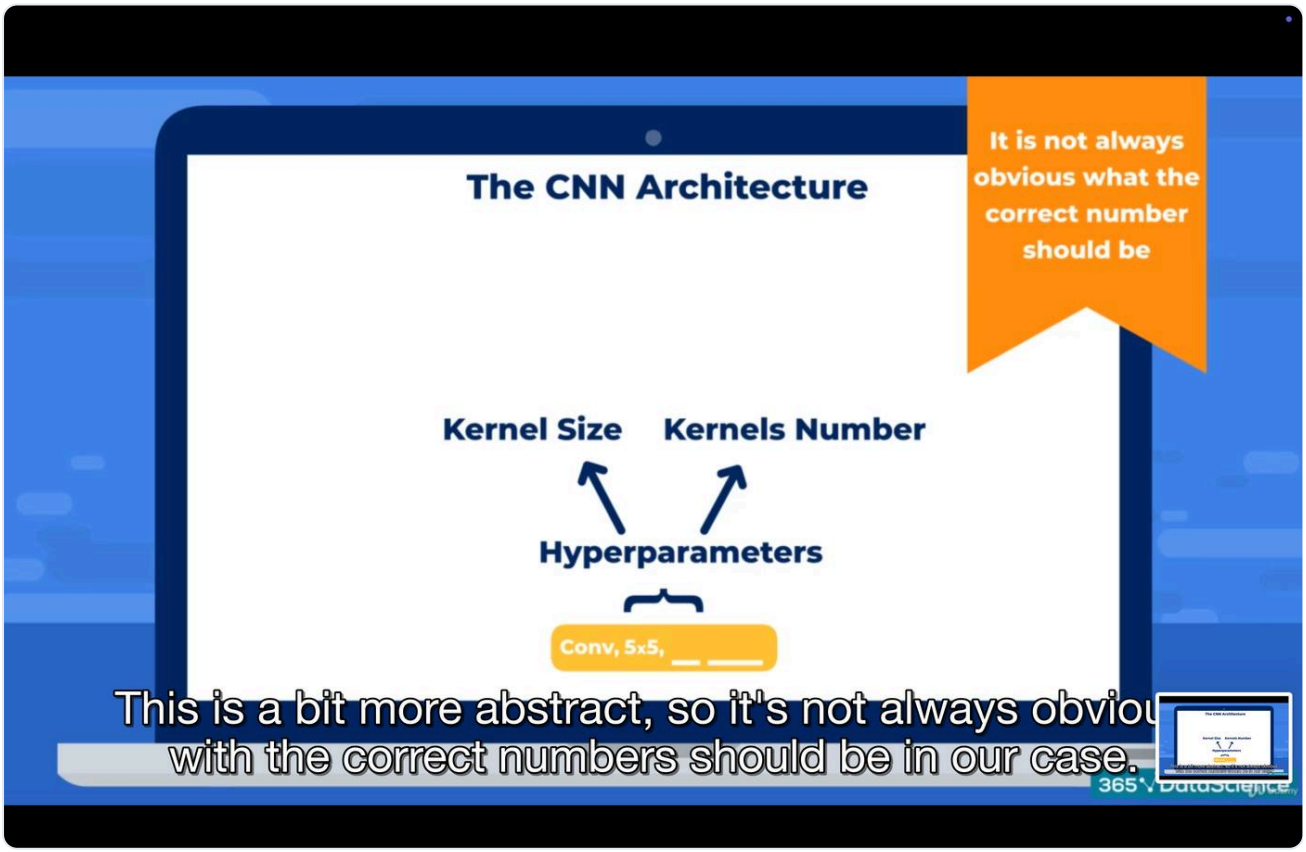

FIGURE 22.26 | The CNN Architecture

This is a bit more abstract, so it's not always obvious with the correct numbers should be in our case. Let's go for 50 kernels.

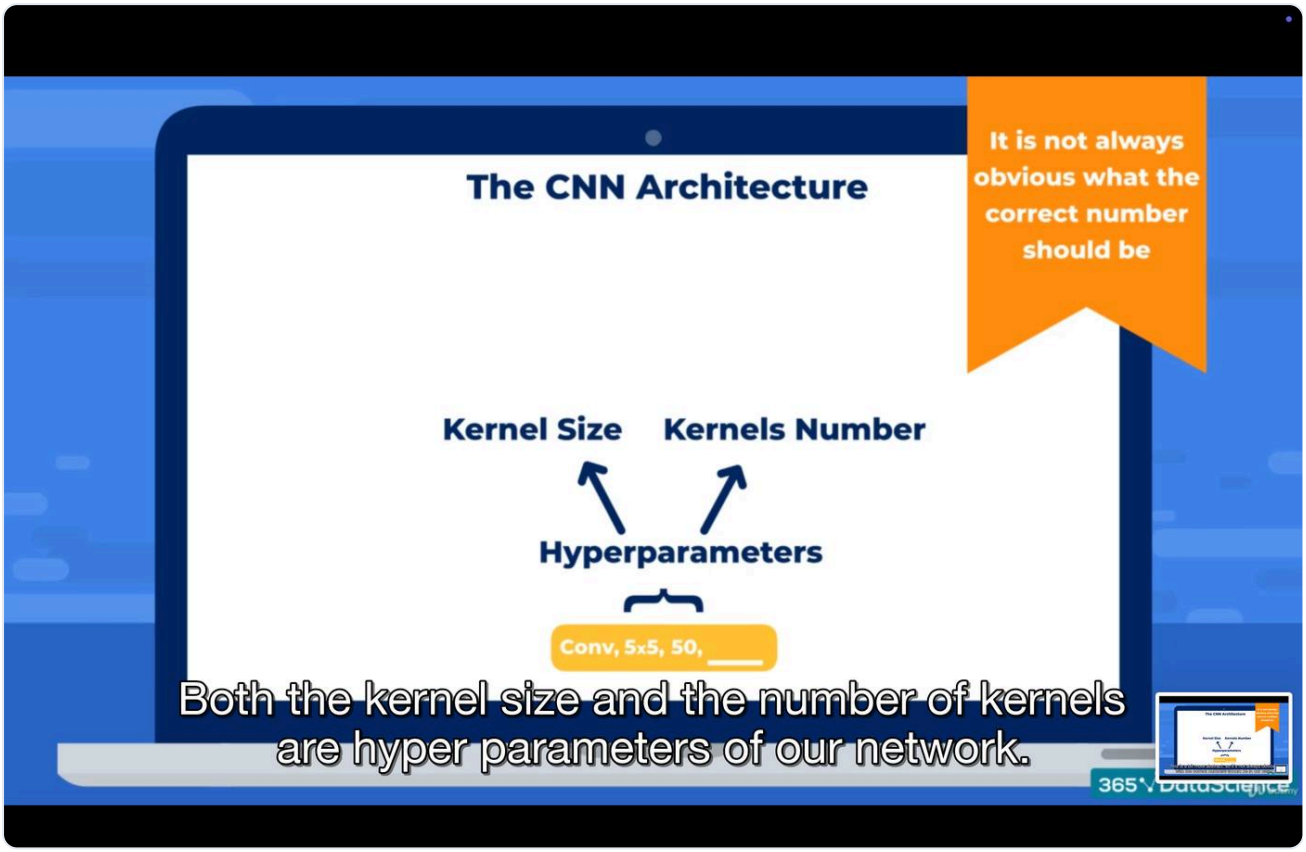

FIGURE 22.27 | The CNN Architecture

Both the kernel size and the number of kernels are hyper parameters of our network.

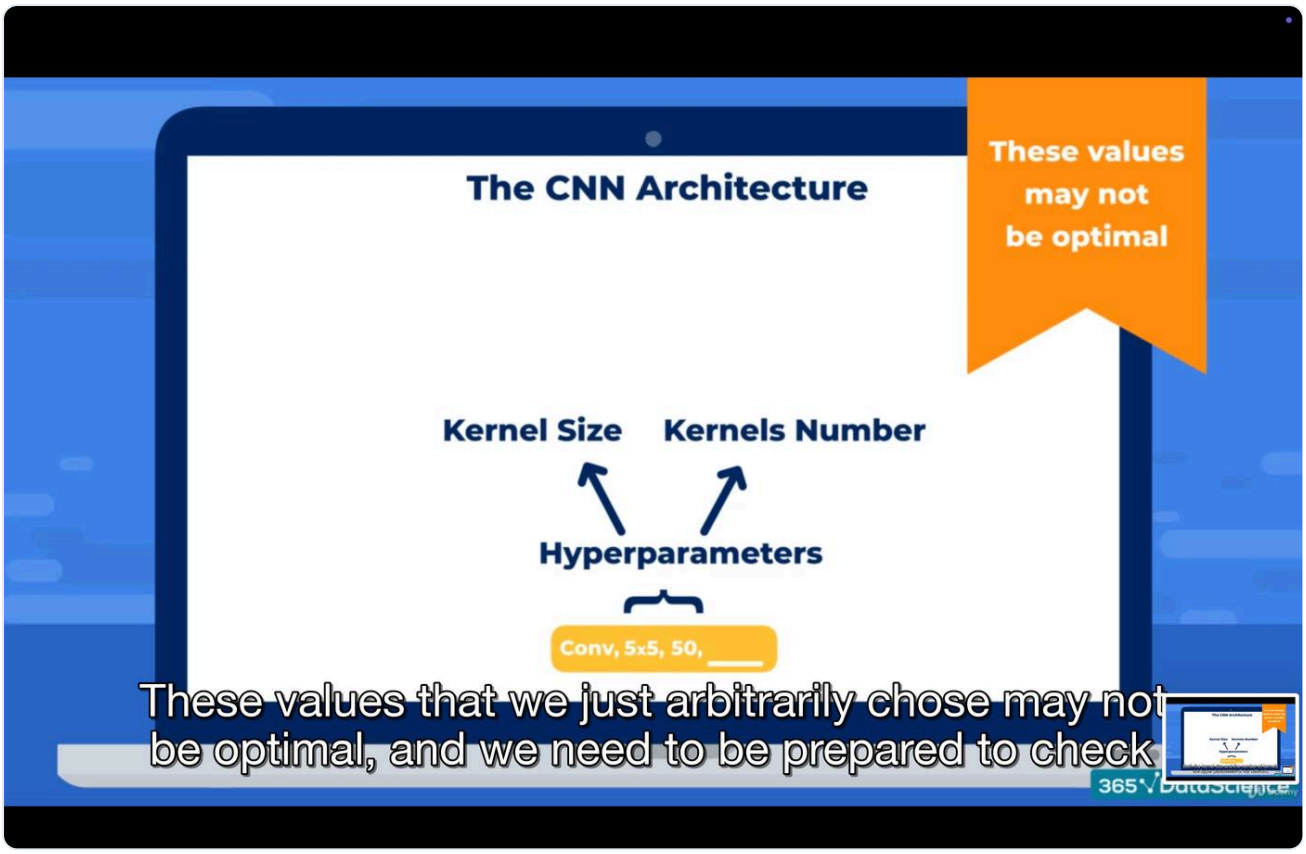

FIGURE 22.28 | The CNN Architecture

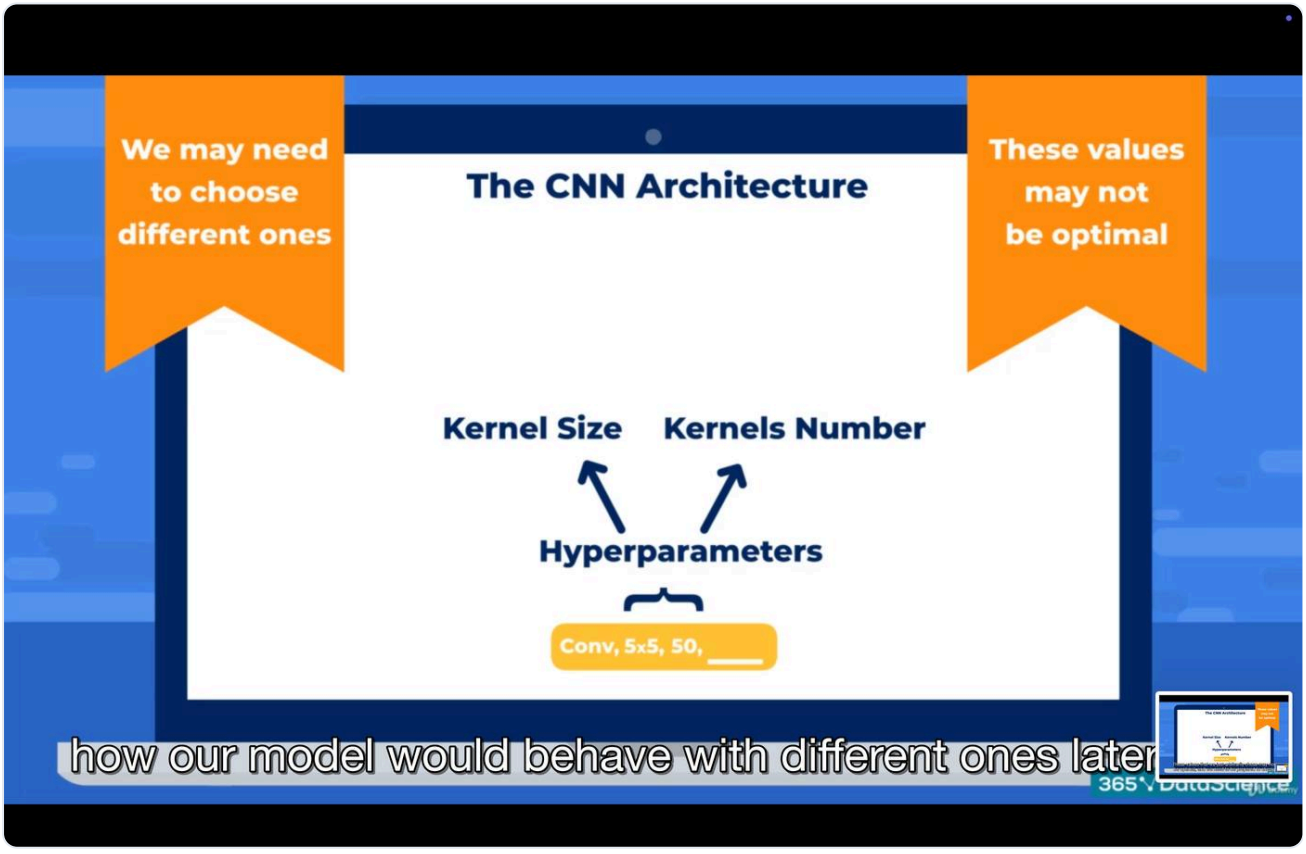

FIGURE 22.29 | The CNN Architecture

These values that we just arbitrarily chose may not be optimal, and we need to be prepared to check how our model would behave with different ones later on. For now, though, it is good to have some sensible starting point. The final decision to make is what activation function to use.

FIGURE 22.30 | The CNN Architecture

We will use the regular RELU as it behaves well in most situations. Great. Now we have completed the first layer.

FIGURE 22.31 | The CNN Architecture

So what's the next one? As we've already discussed, typically a convolutional layer is followed by a max pool one and we don't want to break this tradition now. So our second layer will be a max pool.

FIGURE 22.32 | The CNN Architecture

We will also follow the conventions and have a two by two kernel with stride to so far so good in a true deep network fashion, we can now stack another convolutional layer on top of those two. This should be simple because we can just copy the previous two layers, right?

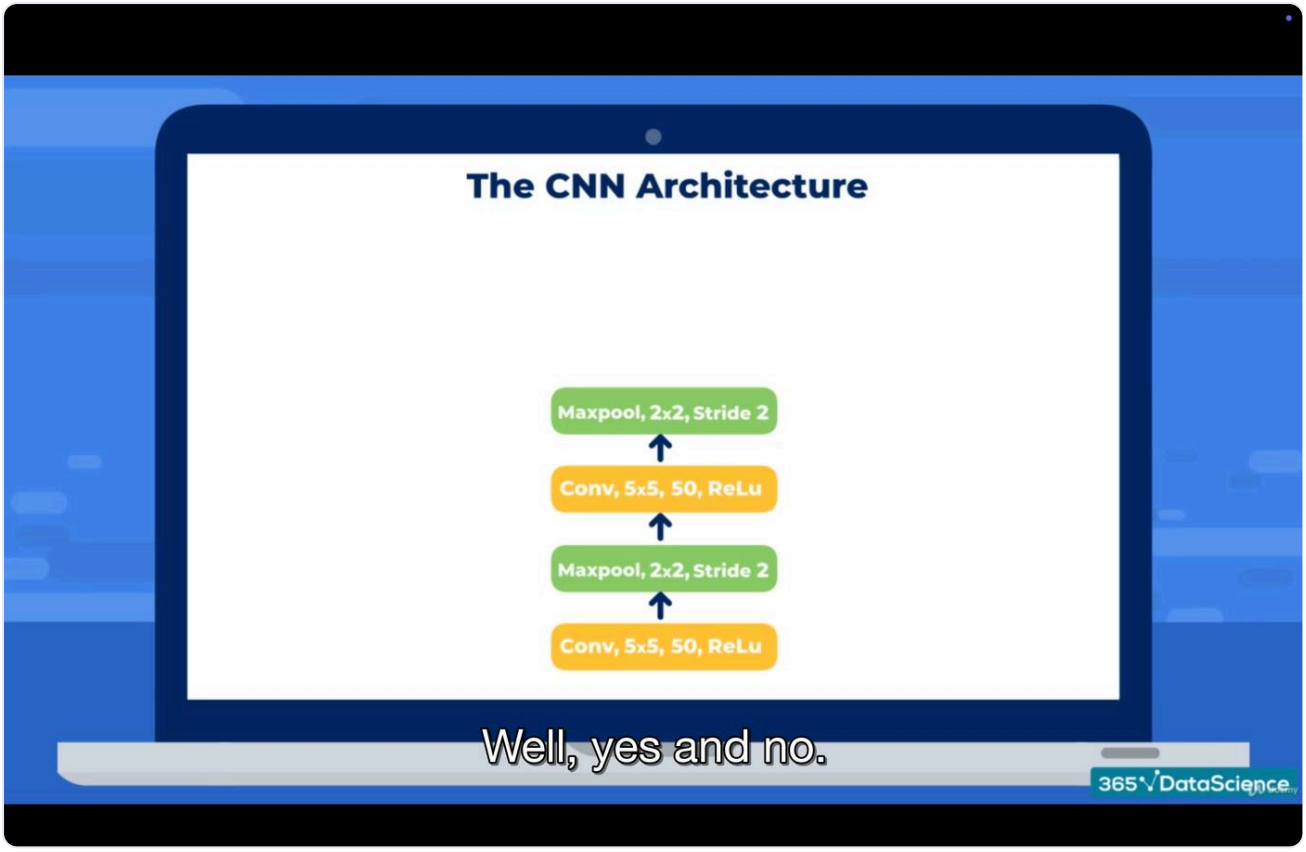

FIGURE 22.33 | The CNN Architecture

Well, yes and no. Indeed, we can have the third layer be identical to the first one and the fourth layer be identical to the second one. However, let's think about the dimensions of it. So our input is twenty eight by twenty eight by one.

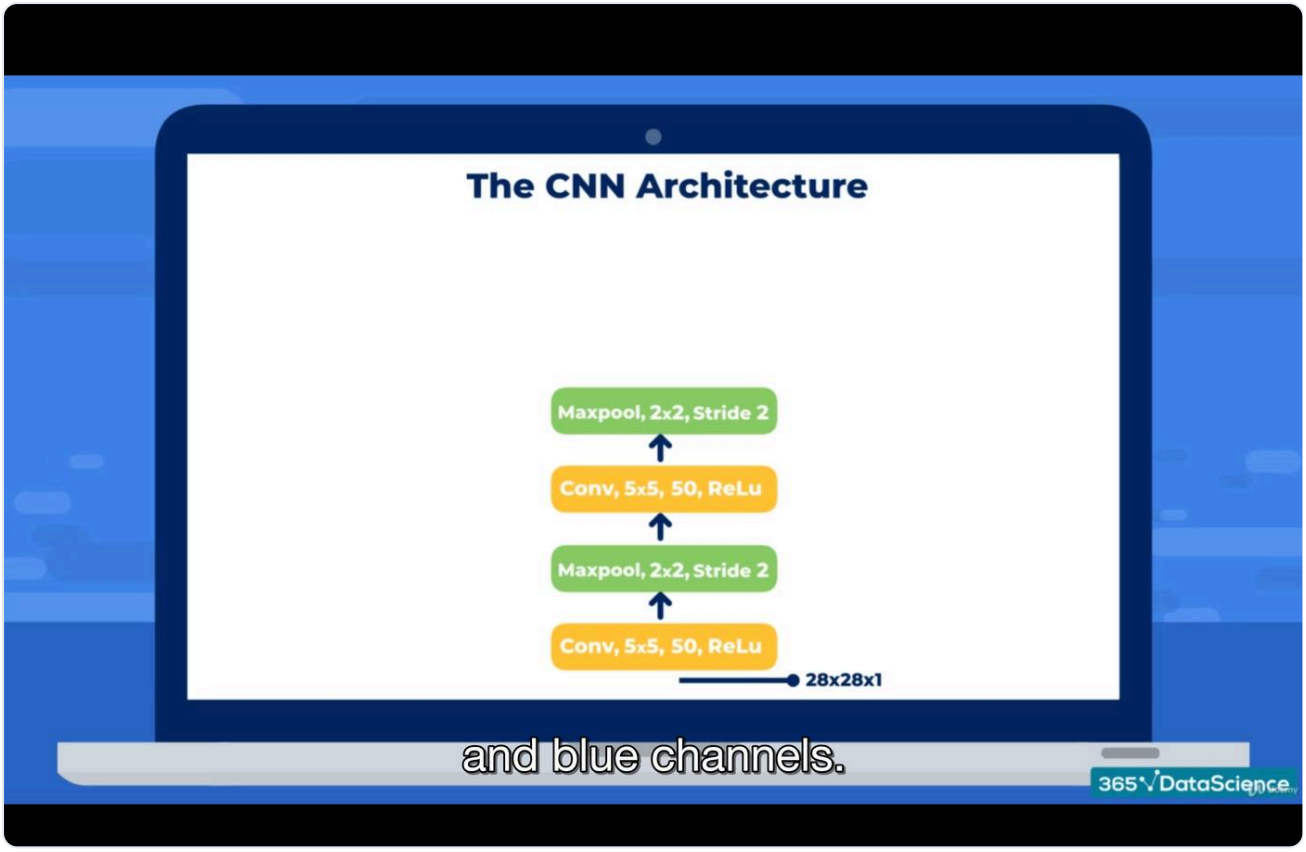

FIGURE 22.34 | The CNN Architecture

The last dimension would have been three if we worked with color images representing the red, green and blue channels. But since we are in grayscale, it's only one. OK, this input is transformed by the first convolutional layer. We are using a five by five kernel and we are ignoring the edges. Therefore, the image will get scale from twenty eight by twenty eight to twenty four by twenty four.

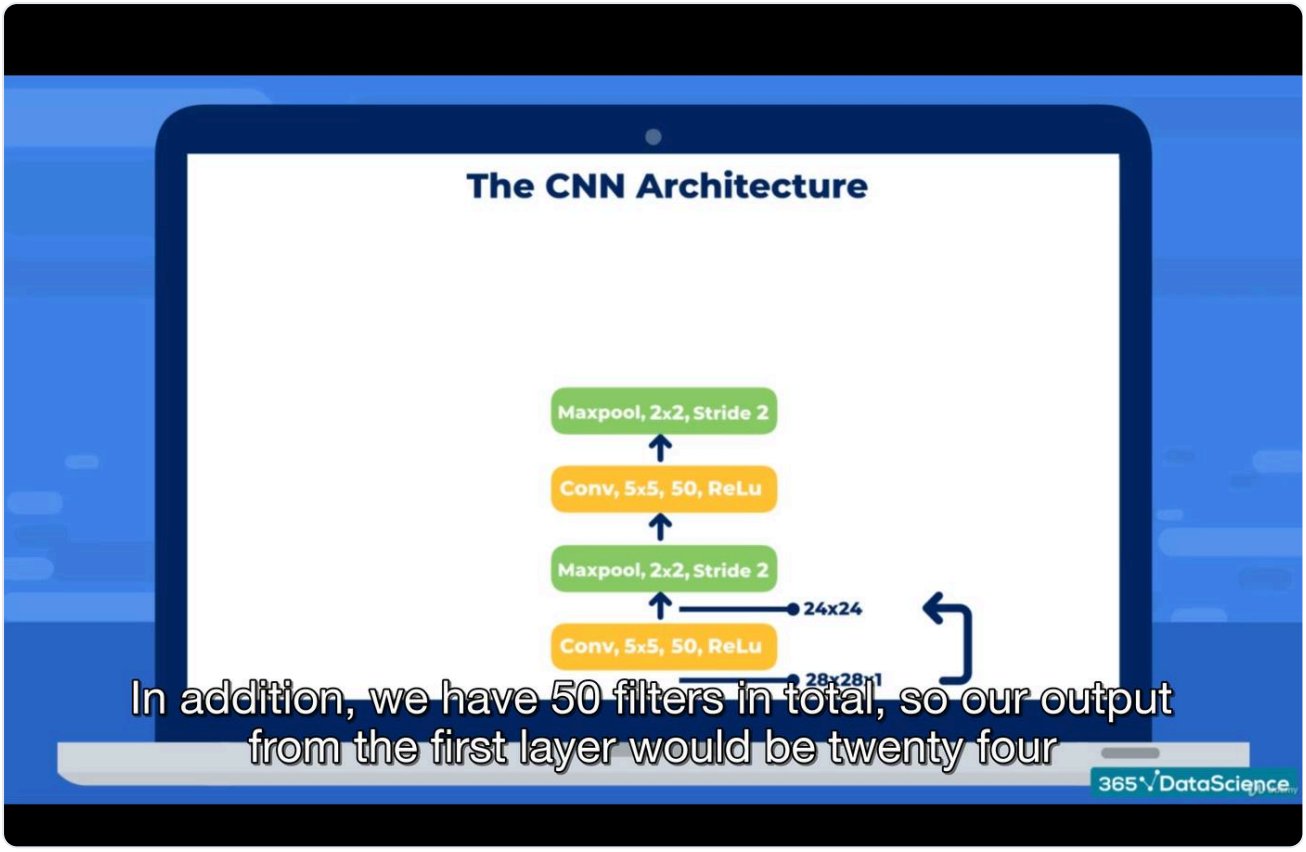

FIGURE 22.35 | The CNN Architecture

In addition, we have 50 filters in total, so our output from the first layer would be twenty four by twenty four by 50. Then we have a max pool layer as we know it only cuts the spatial dimensions in half. Thus when its input is twenty four by twenty four by 50, it will output twelve by twelve by 50.

FIGURE 22.36 | The CNN Architecture

And here comes our second pair of layers, when we decided that a five by five kernel is an appropriate size, we were considering a twenty eight by twenty eight input's. Now, this convolutional layer is receiving a 12 by 12 spacial input. So having a smaller kernel, let's say three by three, might work better for this one.

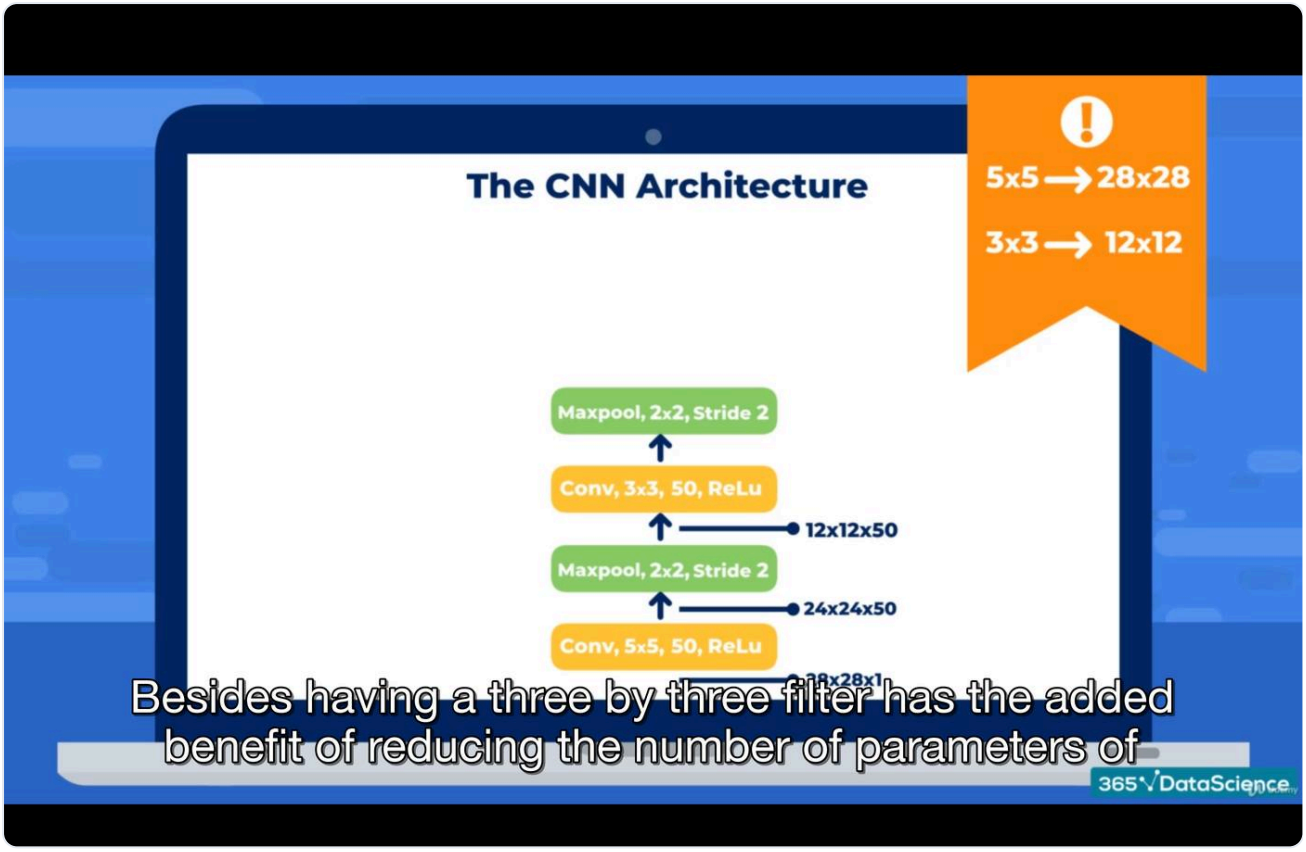

FIGURE 22.37 | The CNN Architecture

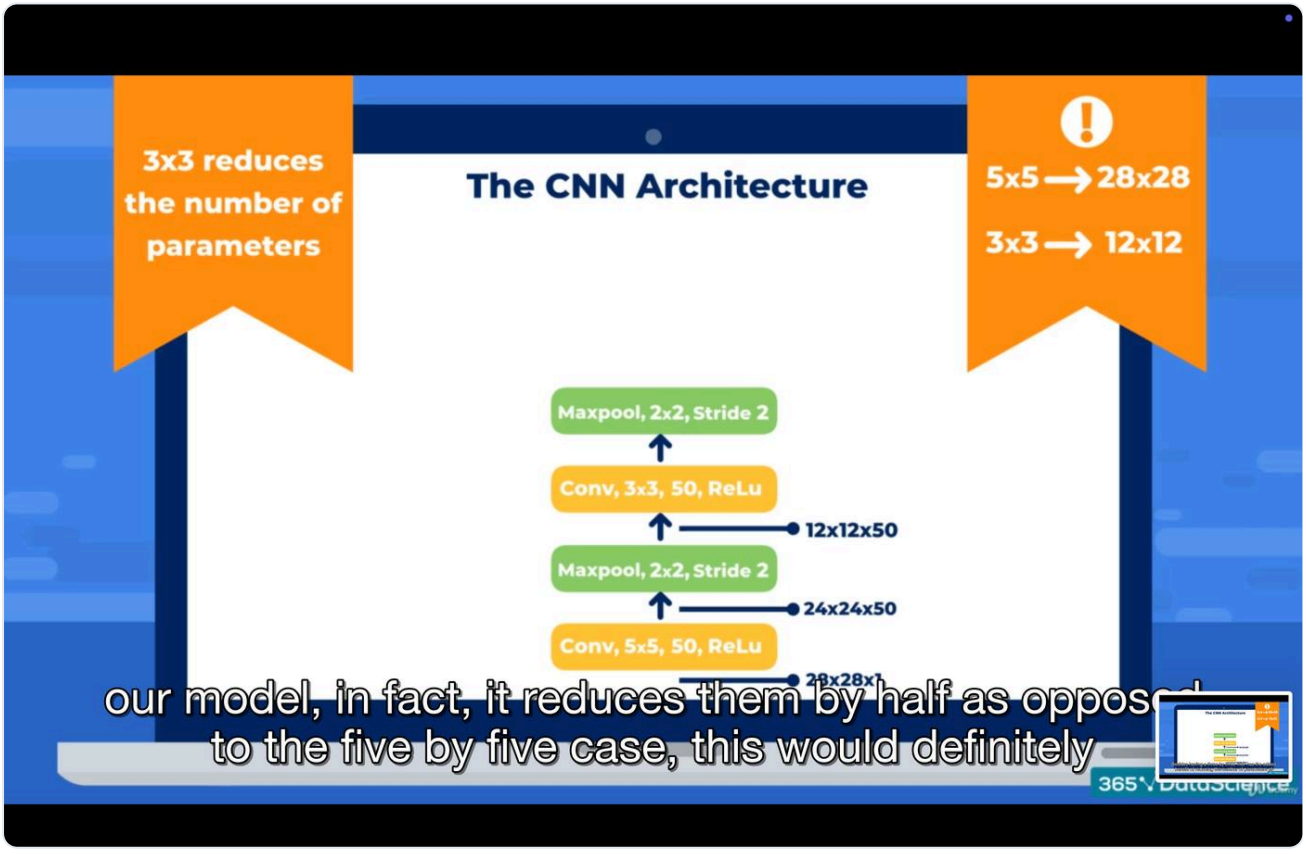

FIGURE 22.38 | The CNN Architecture

Besides having a three by three filter has the added benefit of reducing the number of parameters of our model, in fact, it reduces them by half as opposed to the five by five case, this would definitely speed up the training. All right. Not to be a killjoy, but I would like to stress this one more time. This reasoning may not be applicable in our situation at this point. The input to this layer is actually a set of feature maps with feature maps.

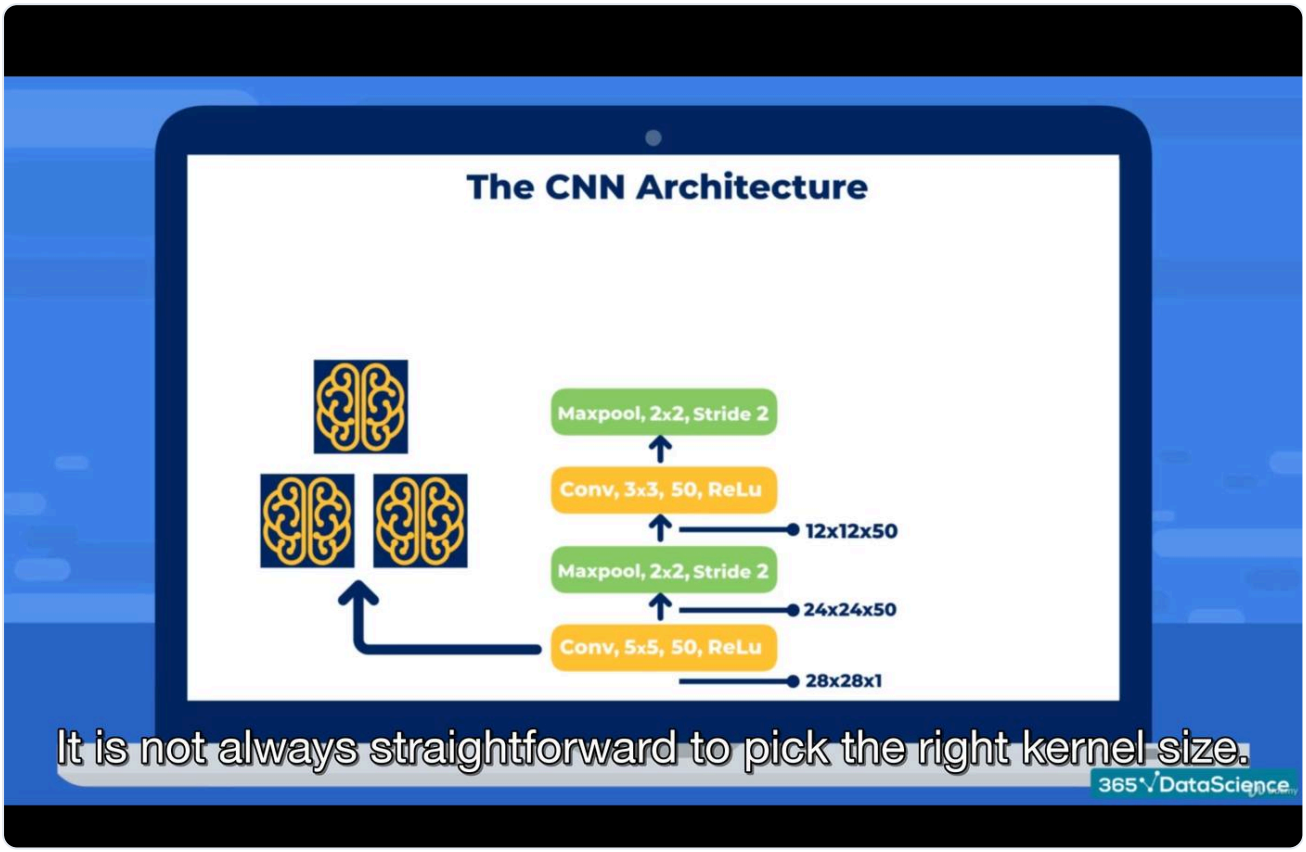

FIGURE 22.39 | The CNN Architecture

It is not always straightforward to pick the right kernel size. Therefore, our choice may not have been ideal, but we have to start somewhere, so we will roll with it. We will leave the number of feature maps and the activation function the same as in the first layer. The feature maps are 50 with the RELU activation. Great. We now have the first four layers of our network worked out. Conceptually, however, we are not done yet at this point in the pipeline.

FIGURE 22.40 | The CNN Architecture

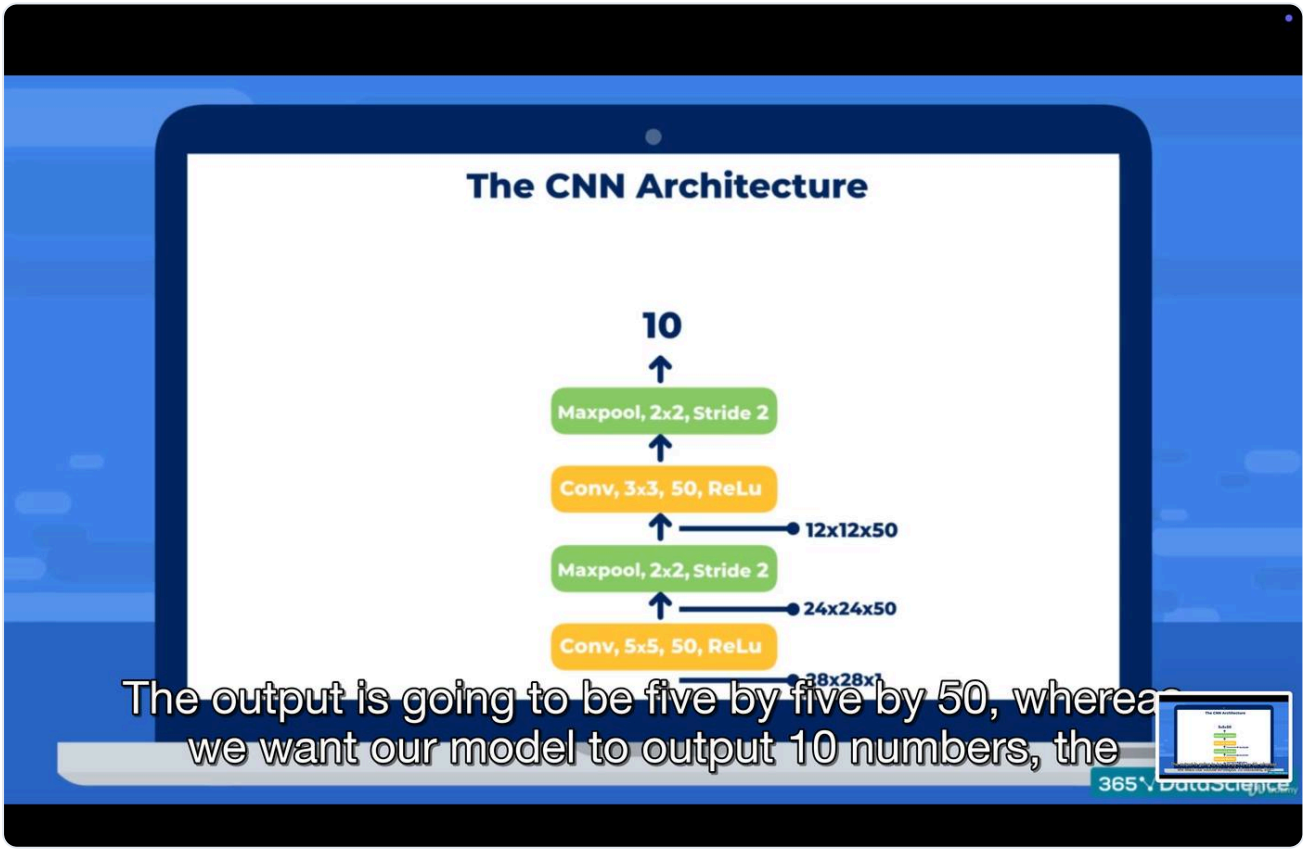

FIGURE 22.41 | The CNN Architecture

The output is going to be five by five by 50, whereas we want our model to output 10 numbers, the probabilities of each class to be present on the image. So how can we achieve this? Let's use the stuff we already know. Our output from the fourth layer is multidimensional and we want to make it one day.

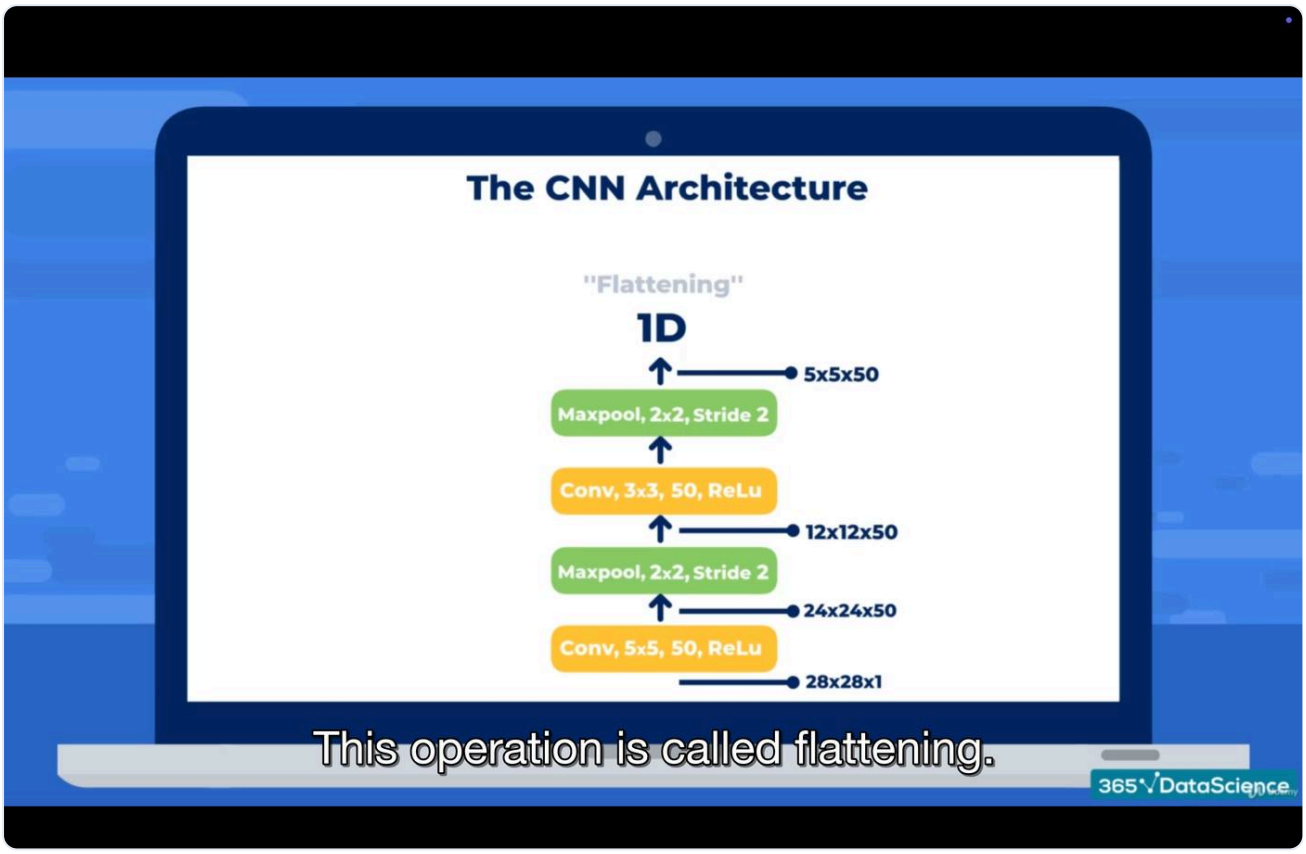

FIGURE 22.42 | The CNN Architecture

This operation is called flattening.

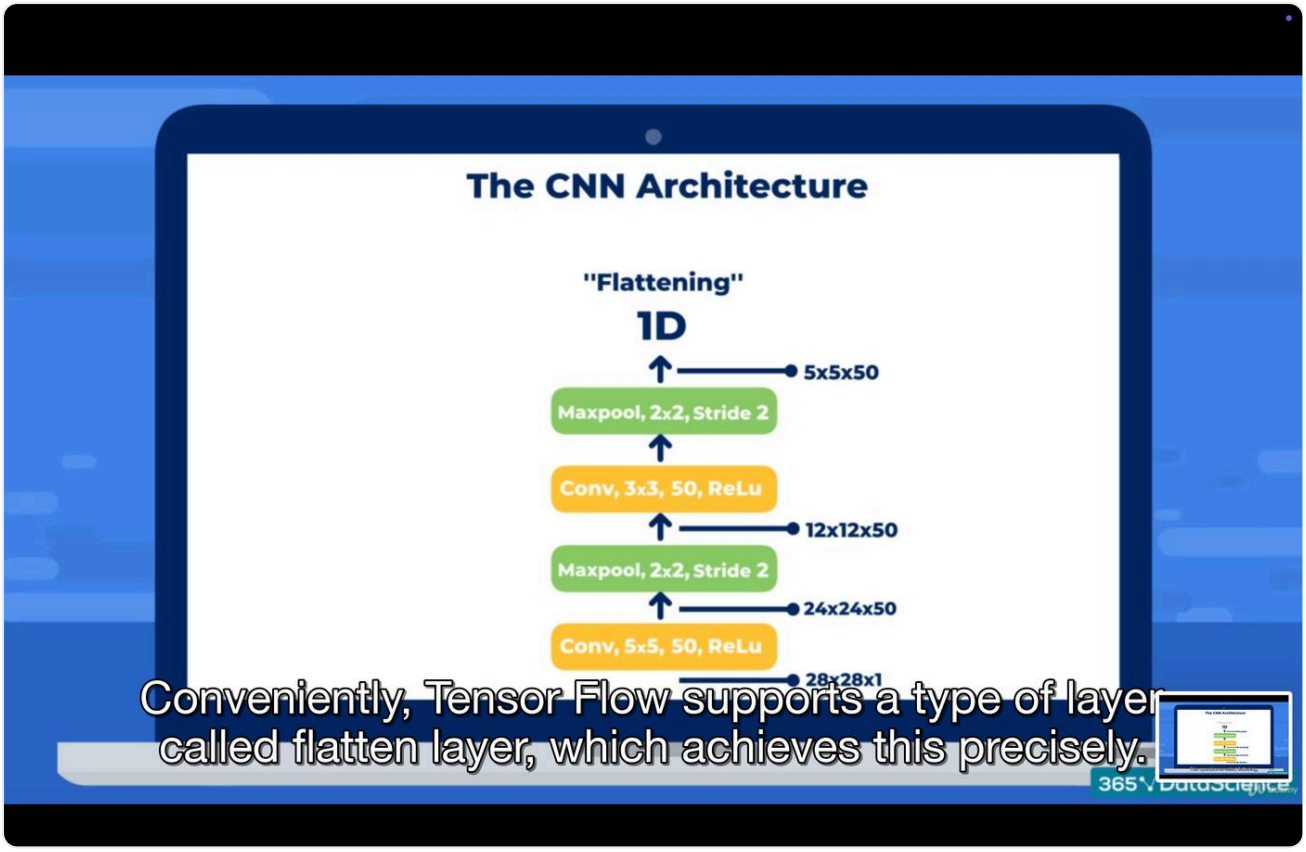

FIGURE 22.43 | The CNN Architecture

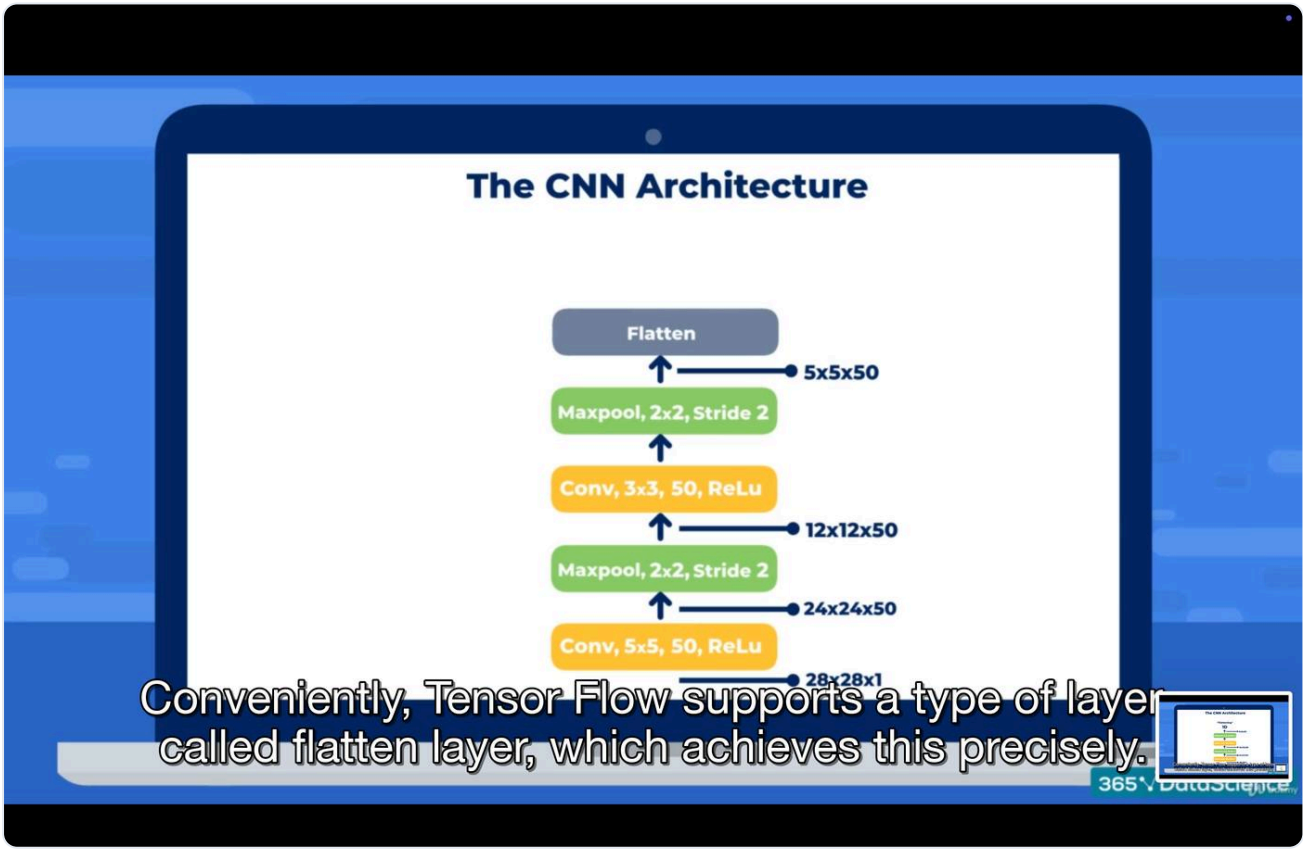

FIGURE 22.44 | The CNN Architecture

Conveniently, TensorFlow supports a type of layer called flatten layer, which achieves this precisely. It unpacks the input to a long vector of numbers.

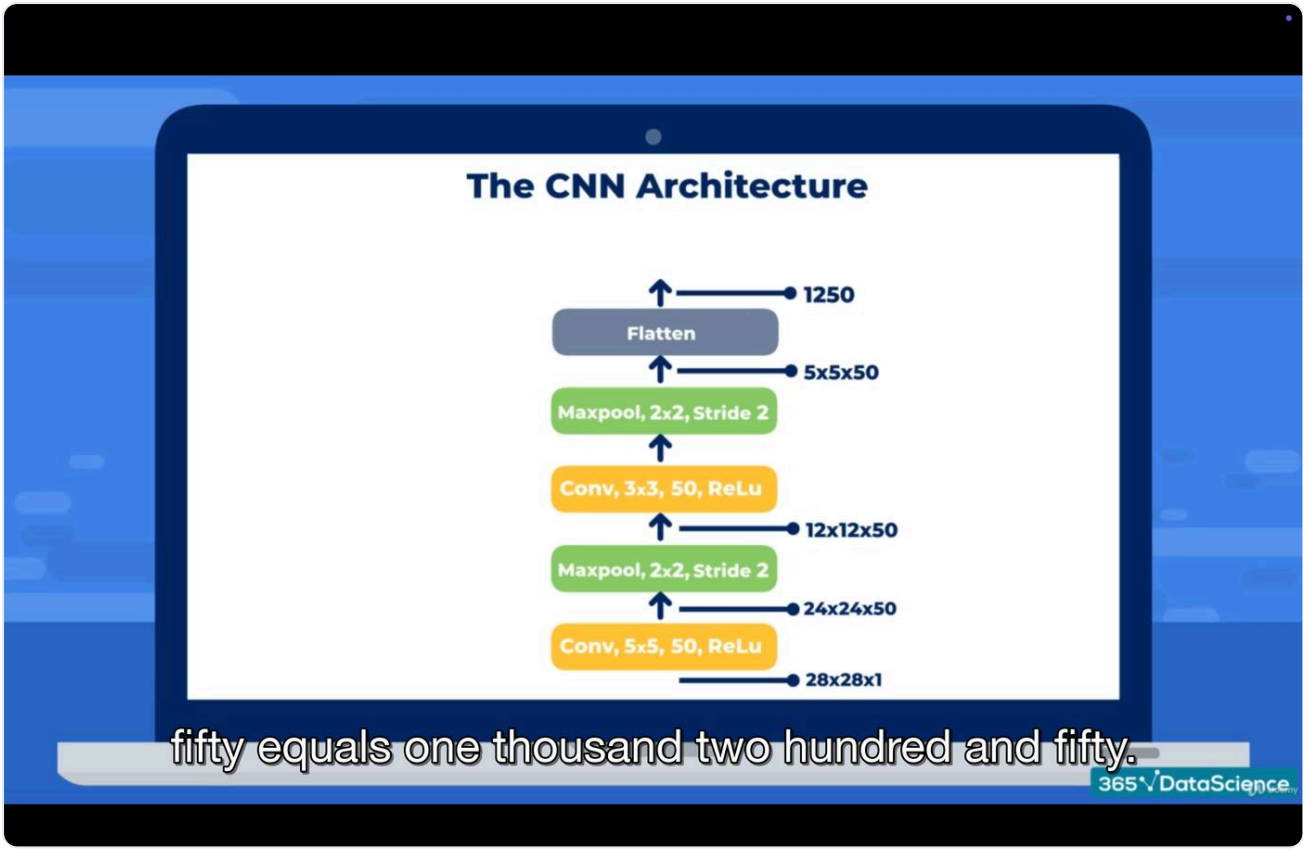

FIGURE 22.45 | The CNN Architecture

In our case, that would be a vector of size one thousand two hundred and fifty as five times five times fifty equals one thousand two hundred and fifty. Now that we have a single vector, we know what to do.

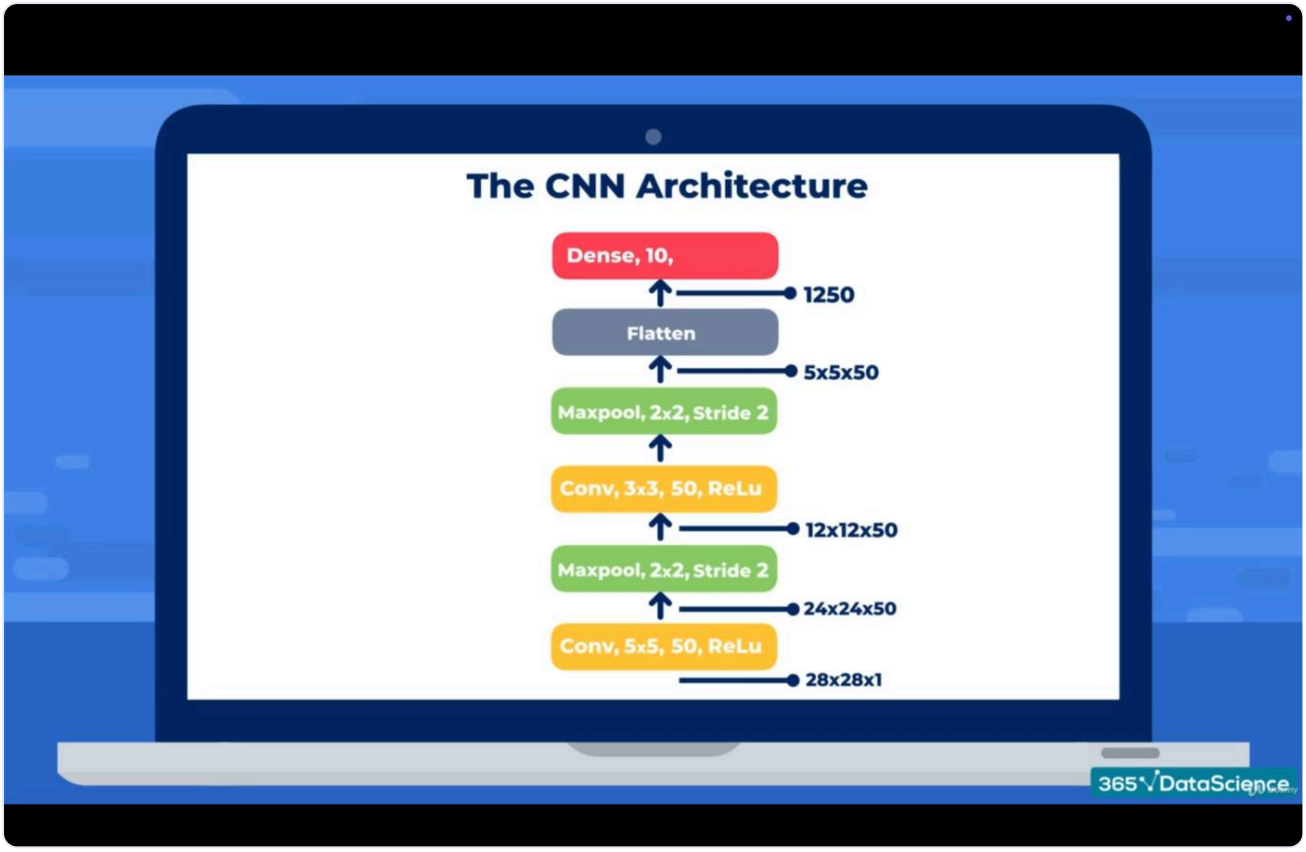

FIGURE 22.46 | The CNN Architecture

Displace a dense layer that outputs ten numbers. Those correspond to the ten different digits we want to classify. For those of you who need a refresher, let's have a quick recap on the dense layer.

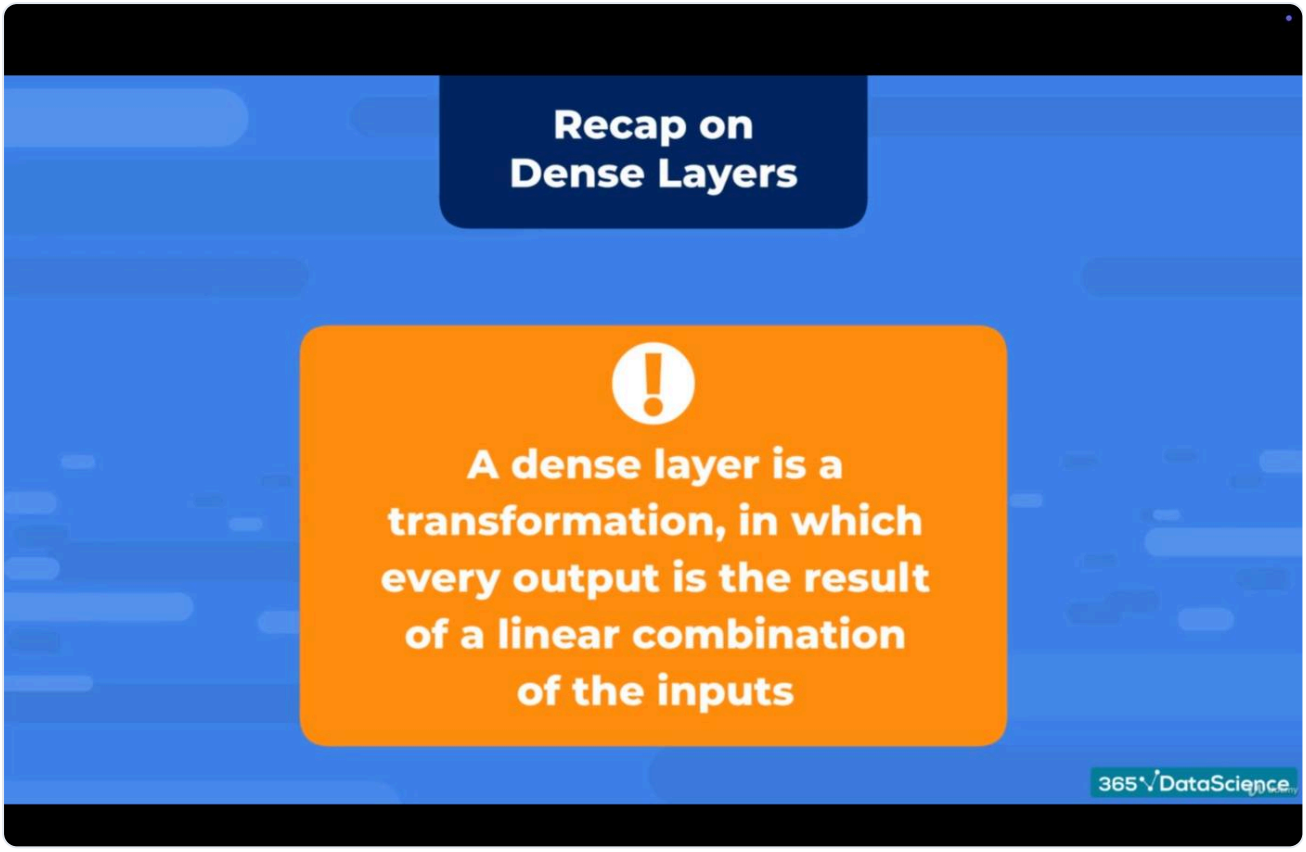

FIGURE 22.47 | Recap on

A dense layer is a transformation in which every output is the result of a linear combination of the inputs. So if we have 10 inputs labeled X1 to X 10, a single output is represented by this sum. An important characteristic of the dense layer is that it can have an arbitrary amount of those outputs, each being a different linear combination.

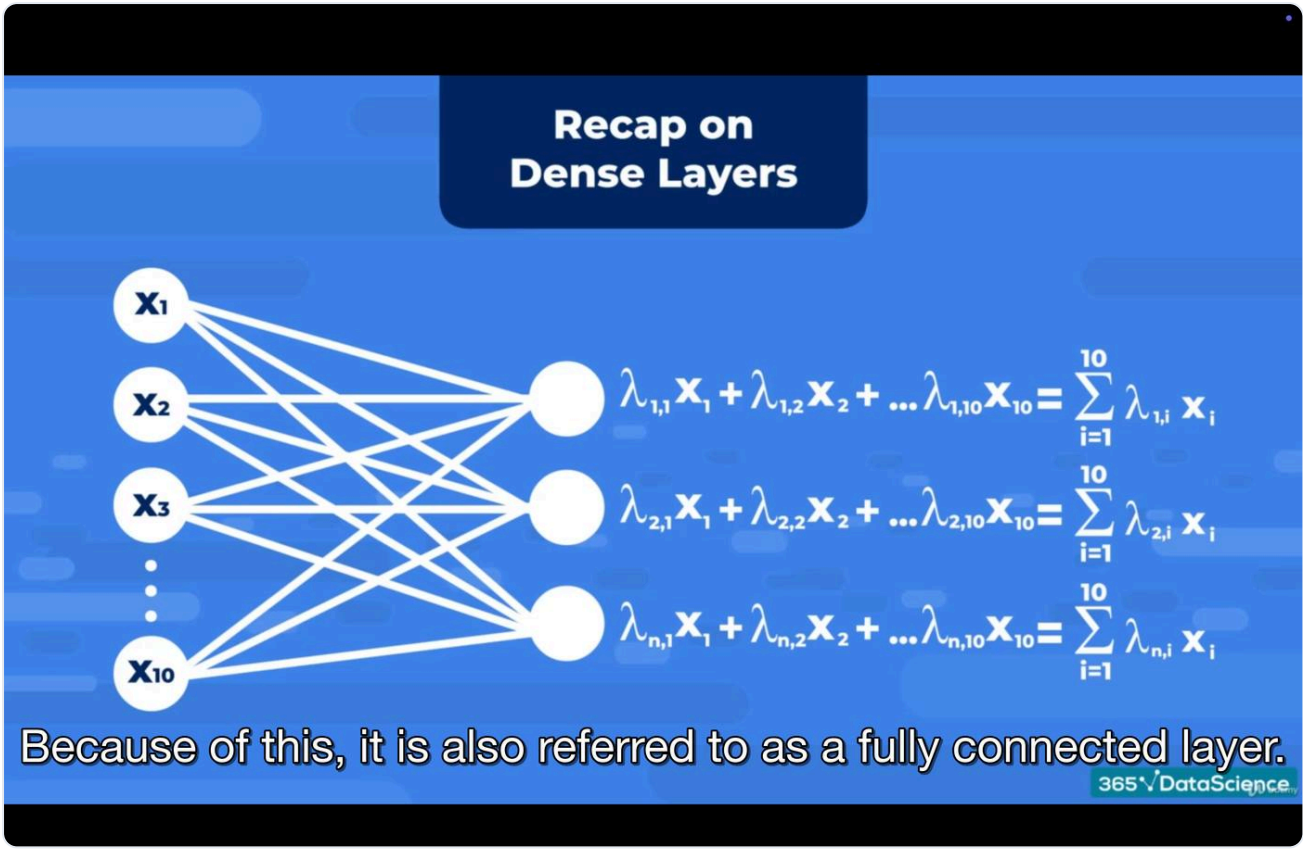

FIGURE 22.48 | Recap on

Because of this, it is also referred to as a fully connected layer. What about the activation function of this dense layer, you may ask?

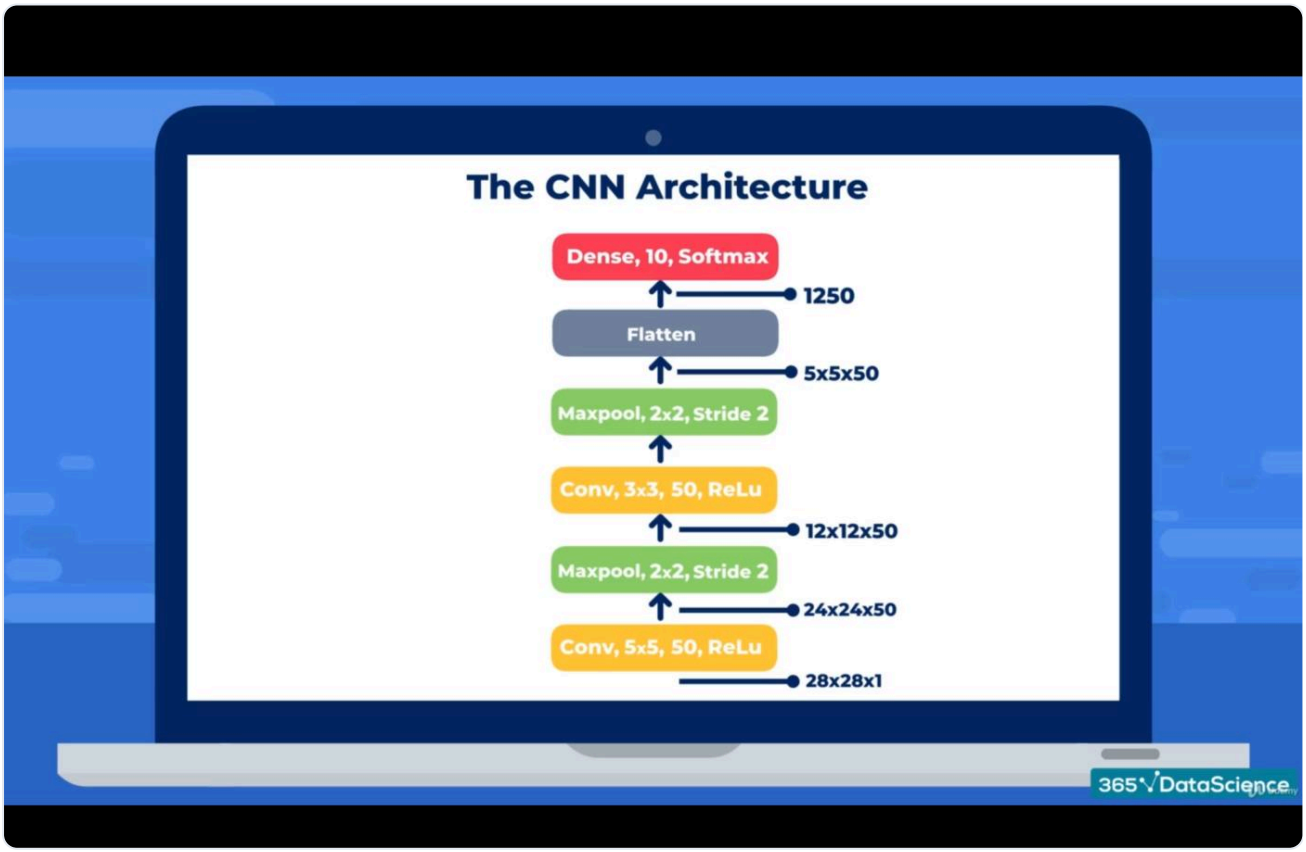

FIGURE 22.49 | The CNN Architecture

Well, if you remember, when it comes down to classification and probabilities, there is one function that stands out from the crowd and that is softmax. It transforms the output into a valid probability distribution. In other words, the output from our dense layer corresponds to the probability of a digit being illustrated on the image.

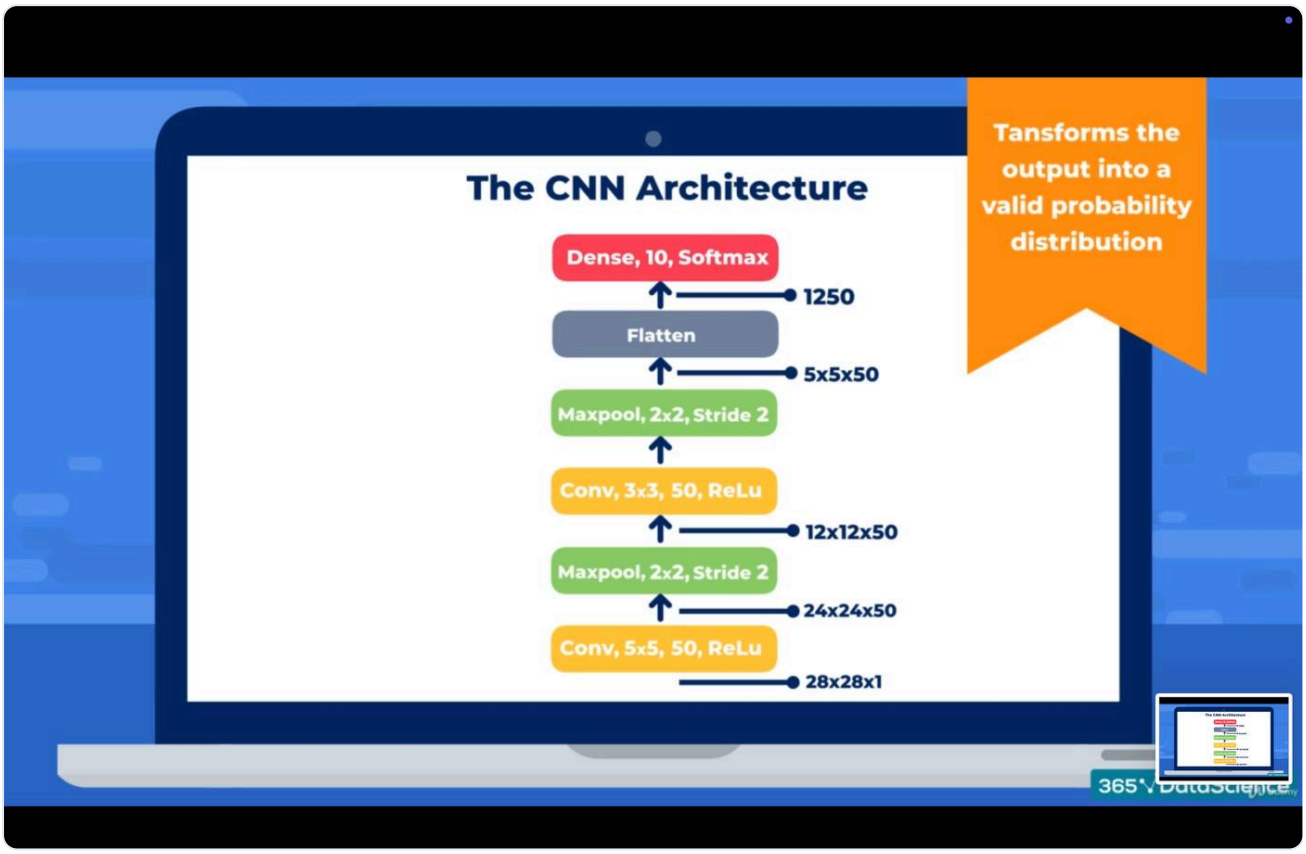

FIGURE 22.50 | The CNN Architecture

Marvelous. This is our final CNN architecture. Let's recap first, we have a convolutional layer followed by a max pool, a construction we talked about in the CNN introduction, then we have the same construction again, a convolutional layer with a max pool layer.

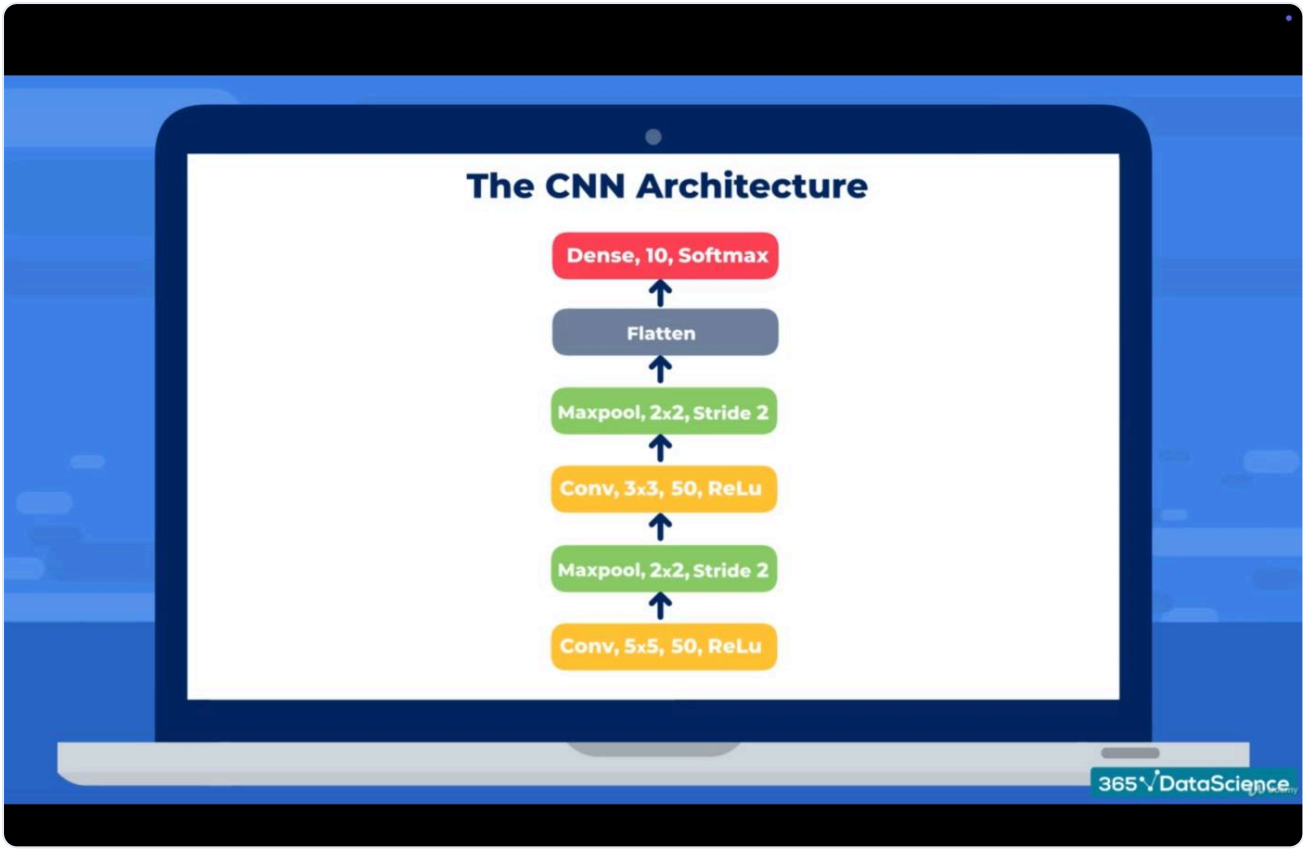

FIGURE 22.51 | The CNN Architecture

Those first four layers are intended to find multiple feature maps to help us identify the digit. Then in order to actually classify the image, we have a flattened layer followed by a dense one.

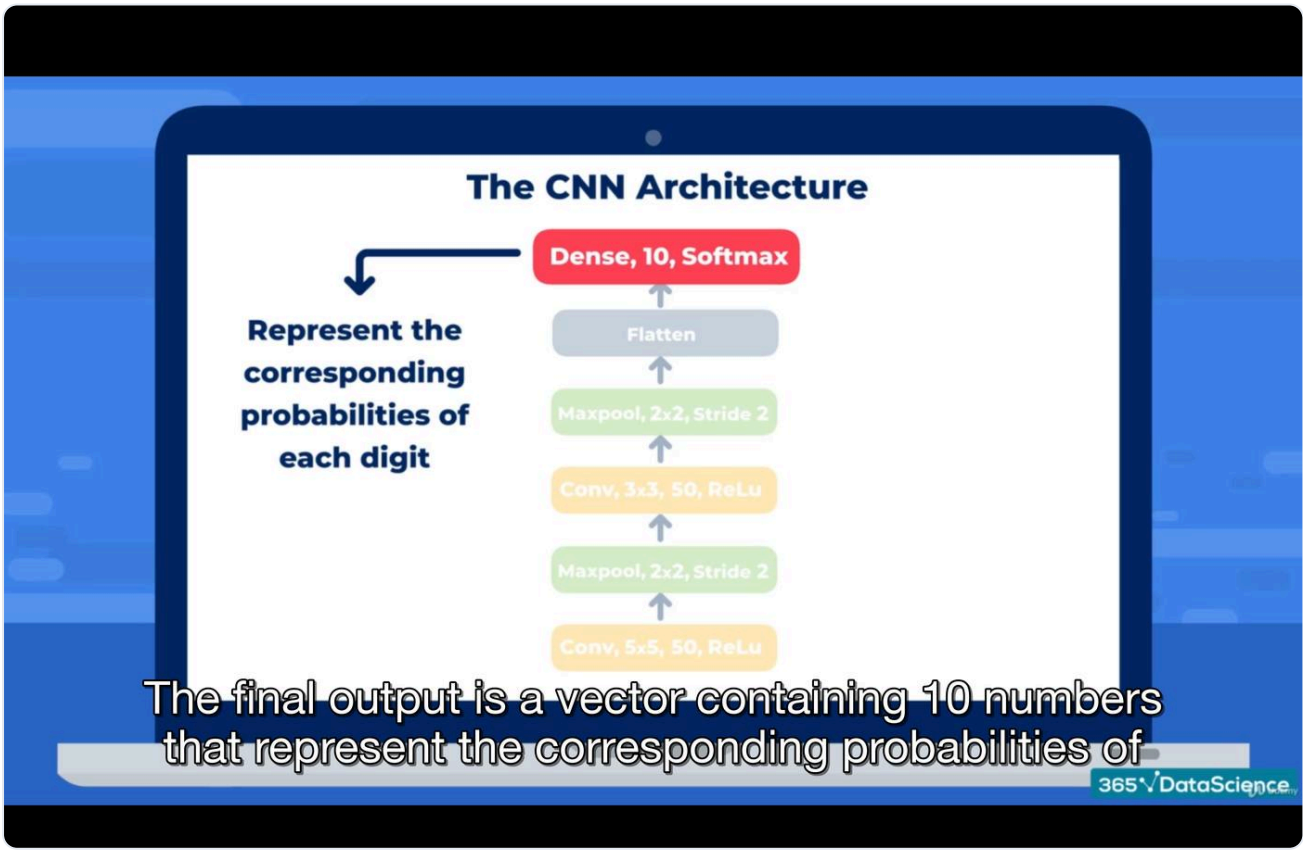

FIGURE 22.52 | The CNN Architecture

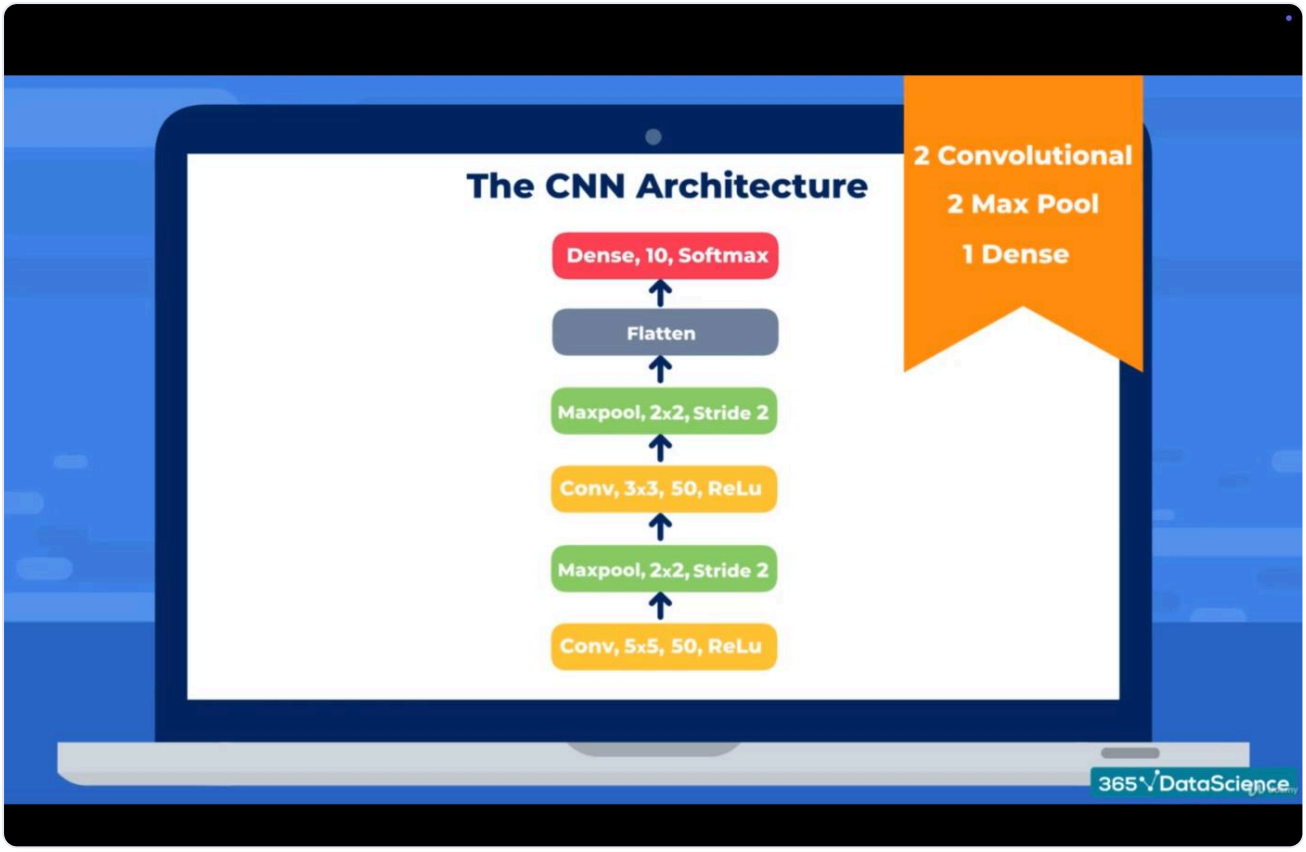

FIGURE 22.53 | The CNN Architecture

The final output is a vector containing 10 numbers that represent the corresponding probabilities of each digit. When we look at the big picture, our network seems pretty simple to convolutional layers, to max pool layers and one dense layer. However, as you saw, there are numerous details to consider. OK. Now we have the road plan for our quote, What is left is to actually write that code. So in the next lecture we will download and pre-process the MNIST data set to get it ready for training. See you there. And thanks for watching.

LECTURE 23

Data Set

see the notebook `1.ipynb` for the code.

Welcome back. We are now finally going to write some code we will download and pre-process the data so that it is ready for the neural network, the preprocessing would include scaling the data, splitting it into train and validation sets and batching it.

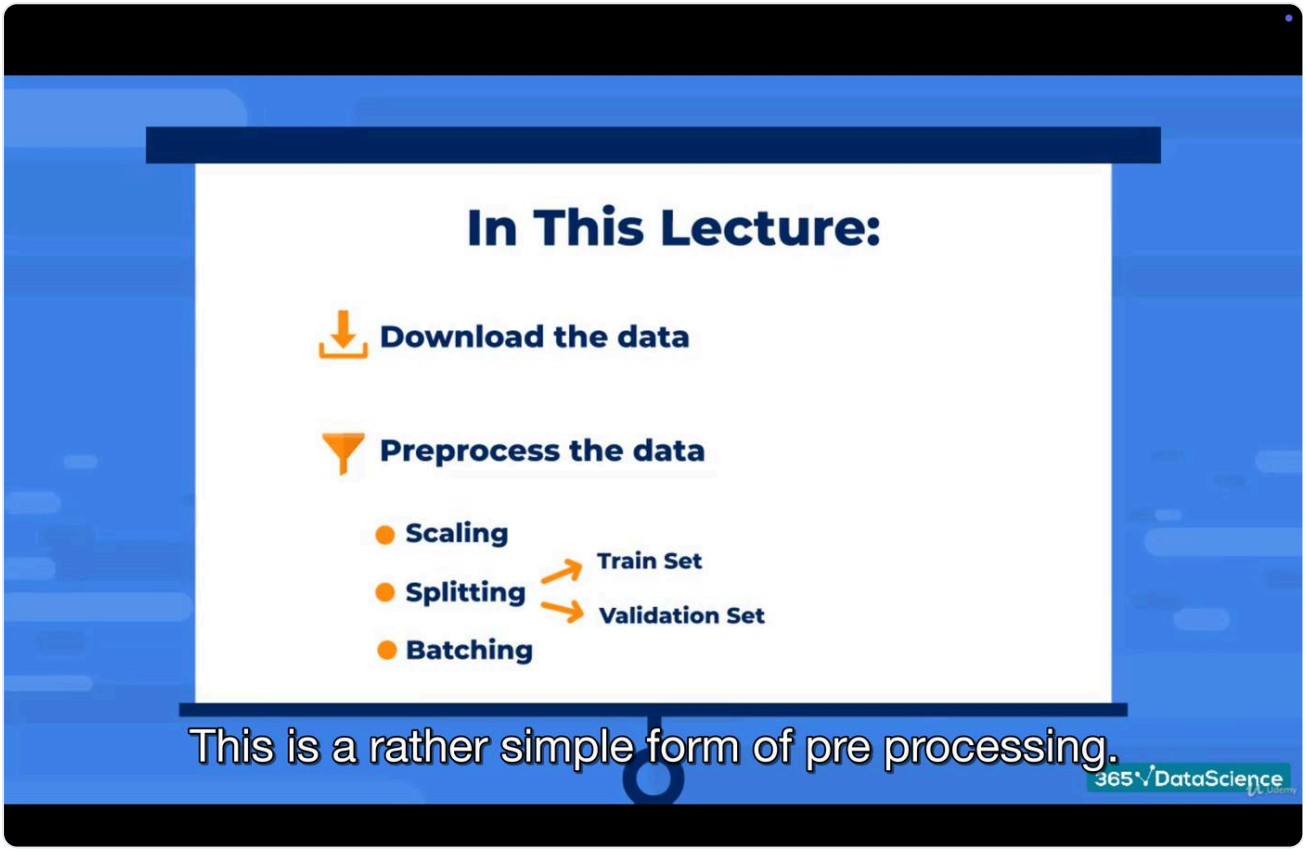

FIGURE 23.1 | In This Lecture:

This is a rather simple form of pre processing. We can even call it preparing the data instead.

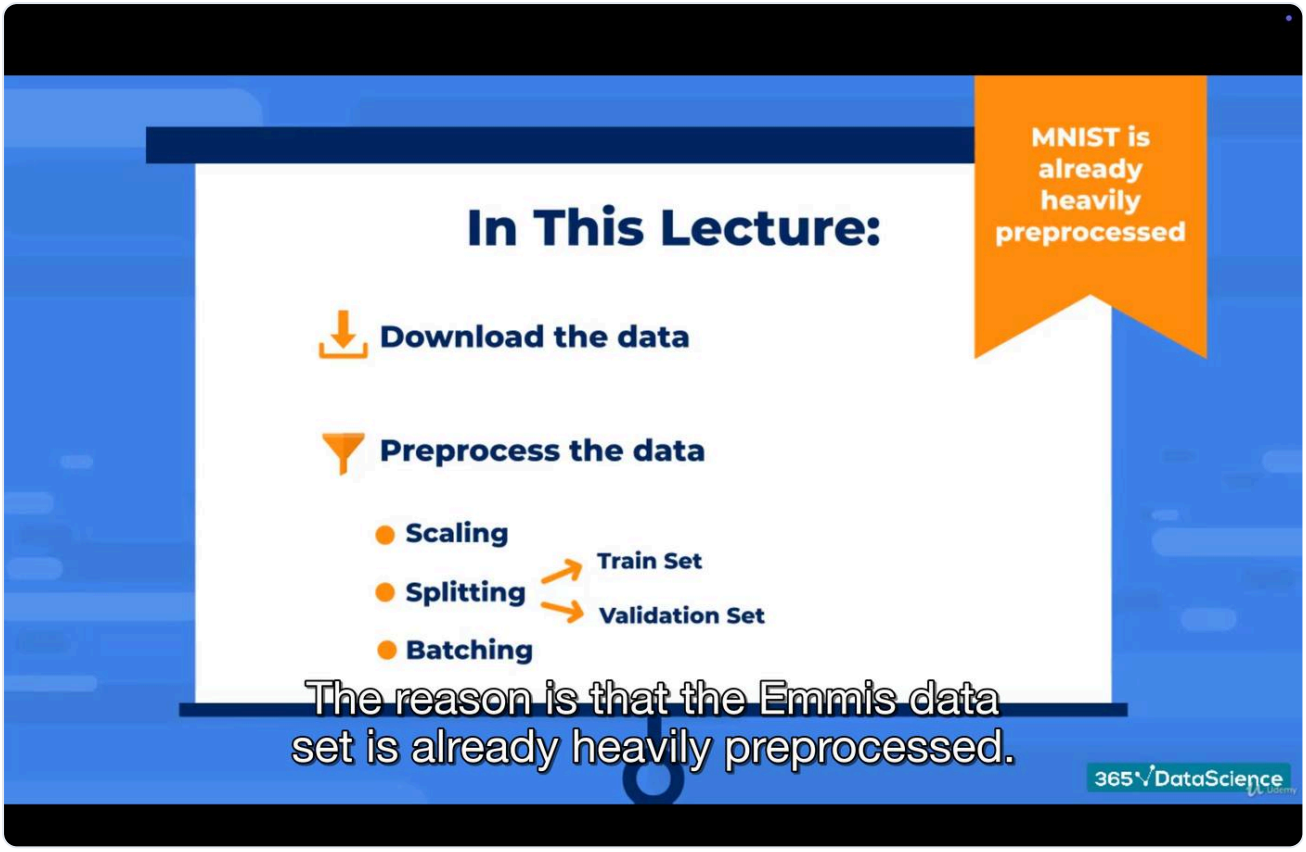

FIGURE 23.2 | In This Lecture:

The reason is that the MNIST data set is already heavily preprocessed. That is also the main reason why it is so popular. Let's jump straight to the code. All right, so here we are in this Jupyter notebook, the first thing we need to do is import the libraries we are going to use in this exercise for now. Those are only TensorFlow two and TensorFlow data sets. So we import TensorFlow as T.F. and TensorFlow data sets as TF D, we will use the former for the creation and training of the neural network and the latter to obtain the MNIST data set. Now, let's continue with the main program. First, we need to define some constants that we will use later on.

You don't have to worry about those yet. We will explain them later as we encounter them. Why define them now if we are not going to use them right away, you may ask? Well, it is good practice to have your parameters defined in a visible and accessible place, usually in big projects. That would be a dedicated source file. But since we are working in Jupyter, we will be defining them at the top of our program. Usually one would figure out what parameters their program requires at a later point. Then they would go back to the top and create the constants there. We do it now so that we don't break the flow later on. Good. Now for the minus data set. Feel free to explore it in the documentation here on this page.

Continuing with the code, let's download the MNIST data set. We can do that through the load function from the TensorFlow Data Sets library. In our case, the function would return the whole data sets as well as some meta information about it in a tuple. Since we want both pieces of data, we will create two variables to store them, minus data sets. And amnesty info. OK, to achieve this, we have two PhDs that load name equals amnesty with info equals true and as supervised

equals true, this command would download the most data set and store it on our computer. Thus, the next time you run it, it would fetch the data set from the hard drive instead of downloading it.

Let's run the cell. Note that this operation may take a bit of time because you're basically downloading hundreds of megabytes of images. OK, now the variable MNIST data set contains the whole data set. That means it holds the training as well as the test set consisting of all the images and their labels. However, in this form, the data set is not that useful. We want to have the training and test sets separated that can be easily achieved as the MNIST data set variable is a dictionary with the two sets labeled simply as train and test. Here is the code MNIST Train and MNIST Test equals MNIST data set. Brackets train MNIST data set brackets test. That's it, we have now separated the different sets.

OK, so we know that the images are stored in greyscale, this means that the value of every pixel is given by a single number that represents how white or black that pixel is. The number varies from zero to two hundred and fifty five in machine learning. However, we like numbers that are standardised in some way. So a common technique when dealing with images is to scale the pixel value to be between zero and one that can be achieved by simply dividing all pixels in the data set by two hundred and fifty five. Instead of going through all values in the data set, though, it is a better idea to create a function that can help us achieve that. We can then use this function in conjunction with the MAP method to apply it to every element in the data set.

All right, I will call this function scale. It takes an image and its label as arguments. We will first cast the image as a TensorFlow flow through to using the TensorFlow cast. We do this to avoid some problems with data types. Then we simply divide the image by two hundred and fifty five. Finally, the function returns, the transformed image and the label. Here we go. We didn't do anything with the label as the image still represents the same digit. However, we had to include it in the function because it's required in order to use the map method. The method takes a function and applies it to all elements in the data set. It does not modify the data set in place, but returns a new one instead.

That's why we need a new variable to store it. So here I will name the variable train and validation data. It is equal to minus train dot map and the name of our function scale, we can do the same for our test set and store it in the variable test data. When we run the cell, it will scale every image and save it to the corresponding data set variable. Great. So we are done, right? Well, not quite. As we mentioned in the first lecture of this section, the M.A. data set does not have a validation set. The validation set, though, is crucial to prevent overfitting. So we will need to split the training set manually to create a validation set. The first thing we need to decide is how big it should be.

A reasonable size in machine learning is considered around 10 percent of the training set. Since we know the number of training images to be sixty thousand, we could calculate and hard code the size of the validation set to six thousand hard coding such parameters is not advisable, though, so we assign the variable number of validation samples to be equal to 10 percent or point one of the training samples. How to get training samples, though? Well, that information is con-

tained in the info variable. That's the reason we kept that variable around. Therefore, we have point one times amnesty info dot split's. Train so far, we have extracted the info for the training set to get its size.

We just add dot now example's. We will also cast this variable to a TensorFlow integer. Since we are here, we might as well get the size of the test samples in a similar way. Amnesty info, dot splits test dot now examples. Now that we have the size of the validation sets, we need to take that many samples from the training set and extract them into a different set. There is one catch, though. The data set is often ordered or sorted in some way. For example, it could be the case that the first 6000 examples are of zeros, the next 6000 are ones and so on. However, we need the validation set to contain data with the same distribution as the training data.

To do this, we will shuffle the original training data to make it a random uniform distribution and then take the first 10 percent as the validation set. Shuffling the set is a straightforward process. With TensorFlow data sets, we can simply use the shuffle method so we have train and validation. Data equals train and validation data. Dot shuffle, buffer size. You may have noticed we opted for one of the parameters we define at the beginning of the code, the buffer size, we use it when we are dealing with enormous data sets. In such cases, we can't shuffle the whole data set in one go as we can't fit it all in the memory of the computer. So we need to shuffle it in batches.

The buffer size variable just states how big these shuffling batches should be. Know that the shuffle batches have nothing to do with the training batch size. In our case, amnesty is not that big of a data set, so we can shuffle it all at once. That's why the buffer size has been said to be as big as the whole training set. Sixty thousand. Running this cell has shuffled our data set, now we can proceed with splitting it into a training set and a validation set. We do this in two lines using the skip and take methods to get the training set. We write train data equals train and validation data. Dot skip, num validation samples, this line will return the data set except for the first and examples where NT in this case is no validation samples next to get the validation set, we analogously right train and validation data dot take num validation samples.

This is the opposite of skip. It takes the first and example's there we go. By running this cell, we have split our data set into train and validation. Fantastic. However, we are not done yet. We still need to bash the data set for optimal performance of the network. Let me show you how this is done. Train data equals train data, Dot batch. As with the previous operations, this one is straightforward to implement, it also includes the second parameter define in the beginning the batch size. We have set that one to one hundred and twenty eight. The batch size is another of parameter of the training process that can potentially have a big impact on the training time and the final accuracy.

As with all hyper parameters, to find the optimal value, you need to experiment. As a general rule of thumb, though, a bigger batch size but not extremely big helps speed up the training process, especially if the training is done on a GPU. On the other hand, a lower batch size gives better test accuracy. It is desirable to set the batch size to a value that is a power of two, for instance, 30 to 60 for one, twenty eight to fifty six and so on. For this run we have settled on one

twenty eight. The validation and test sets don't necessarily need to be batched as we don't backward propagate on them. However, the model expects these to be batched as well to get the proper dimensions.

That's why we will batch them with a batch size equal to their number of samples. So we do that for the validation data with a batch size of NUM validation samples. And the test data with no test samples. Fantastic, that's it, we are finally done with pre processing the data in the next lecture, we will build our network, train it and test its accuracy. Thanks for watching.

LECTURE 24

Early Stopping and Layer

Welcome to the most exciting part in this lecture. We will create our CNN and train it all with the help of TensorFlow, too. Now, let's outline the workflow when dealing with TensorFlow to there are three major steps. First, we define the configuration of the network. We already did this conceptually in the CNN architecture part. Next, we compile the model we outlined in the first step. Finally, we proceed to train the model on our data using the model dot fit method. OK, let's get started. Our first task is to tell TensorFlow what our network architecture is. Here is the recap of what we agreed upon. We have a convolutional layer, then a max pool again, a convolution plus a max pool.

Finally, we flatten our output from the previous layer into a one dimensional vector and get an output for each of the ten classes through a dense layer. All right, so how do we code this? Well, it turns out it is pretty easy to do with denser flow as it has all layers already defined. All we need to do is call them by name. So we will name our variable that stores the network architecture model pretty descriptive. It is equal to T.F. Dot Keras. Don't sequential. Sequential here refers to the fact that the layers come one after the other, they are in sequence. The argument for this is a list of all the layers, so here we will list all of the layers of our network along with their hyper parameters.

TensorFlow also provides an abstraction for the different layers through the T.F. Dot Keras dot layers class. This makes the process very straightforward. Great. Our first layer's should be a convolutional one, we can enter that by writing T.F. Dot Keras. Dot layer's. Dot com, 2d. The name of the convolutional layer in TensorFlow is conv, the 2D part reflects our input, which is a 2D image, see pretty simple. Now we need to specify the parameters for this layer. First is the number of kernels or filters. As we've discussed previously, this is 50. Then we have the kernel size, which we agree to be five. Now we will state the activation function to be Relu. Finally, we need to specify the shape of our input data in our case, that would be a greyscale 28 by 28 image.

So the input size would be 28, 28 one. That is the first layer with all its parameters. There are more settings you can tweak, such as the stride. However, their default values are sufficient for our purposes. If you wish to explore them, feel free to visit the official documentation. Now for the second layer, it is a max pool, it's TensorFlow name is Max pooling to. So we have tfw dot Keras. Dot layer's dot max pooling to. As a parameter, we have pool size equals two by two. This is the default value for this parameter, so we could have left it out for completeness, though, we will leave it. All right. Then in our model, we repeat those two layers, so let's just copy those lines and paste them here like that.

There are a couple of parameters we need to change, though. First, we decided to use a three by three kernel for the second convolutional layer. So we need to change this five to a three. Second, the input shape parameter is needed only when the layer is the first one in the network, therefore we will remove it. The last two layers should be one, unpacking the output and one fully con-

nected layer with 10 outputs, the TensorFlow name of the first one is flatten, while the second one is dense. So we write T.F. dot Keras dot layers dot flatten. We don't need to pass any parameters to this layer, that means we can continue with T.F. dot Keras, dot layers, dot dense.

As a parameter for the dense layer, we need to specify the number of outputs, in our case 10. That's it, we have outlined our model fantastic to check if the model has been initialized properly, we can take a look at its quick summary by writing model DOT summary. I will also add a parameter that defines the width of the printed summary to be 75 symbols. This is purely cosmetic with the purpose to fit the data in the line. OK, so running it produces this brief table view of all the layers, the dimensions of their output and their parameters, these last two columns are convenient as they save us the trouble of computing them manually, we can clearly see how the dimension of the input evolves throughout the network, something which we calculated by hand in a previous lecture.

If you are wondering why the first dimension is none in all layers. That's because normally all data is batched. Thus we don't pass a single image of size twenty eight by twenty eight by one, but actually pass a batch of, let's say, a thousand images all at once. That is supposed to be the first dimension. However, the model does not know our batch size just yet as we haven't trained it. So it leaves this dimension as none. Another interesting thing to note is the number of parameters in our architecture. We see that this model has thirty six thousand parameters in total, all of which are trainable. But what are trainable and non trainable parameters?

Well, trainable parameters are the weights of our network, the parameters that the model is trying to learn in our model. These correspond to the different numbers and the kernels and the final dense layer. All the parameters we've talked about so far are trainable. Sometimes, however, you may need to multiply your input by a constant number, say two point six, and you would want that to happen every time. In other words, this parameter should not be changed during the learning process. These kinds of parameters are non trainable. All right, last thing about parameters, we can see that the majority of this model's parameters come from the second convolutional layer.

Why is that? Why hasn't the max pool before it reduced the dimensions of the inputs and thus reduce the parameters? Also, the first conv layer has a kernel size of five, while the second has a size of three. So why doesn't the first layer contain the most parameters? Well, the key here is to not overly concentrate on the spatial dimension and think a bit about the third dimension of the input tensor. At first we have a tensor with dimensions. Twenty eight by twenty eight by one. So each of the 50 kernels in the first layer is five by five by one. Remember, the last dimension of the kernel is the same as the input. However, the input to the second convolutional layer is 12 by 12 by 50.

That means that the 50 kernels in this layer have to match the last dimension of 50, thus obtaining the shape of three by three by 50. So a five by five by one kernel has twenty five parameters, while a three by three by 50 has four hundred and fifty parameters. That's why the second layer contributes the most to the total parameters. The increase in depth of the input far more outweighs the decrease in the spatial dimensions. Great. Now that this is clear, let's continue for-

ward. Some of you may have noticed that I did not specify any activation function for the last dense layer. Weren't we supposed to have a softmax activation there in order to convert the output to a probability?

That is true. There is one caveat in this case, though. The thing is that including the softmax directly into the layer is discouraged as it's impossible to provide an exact and numerically stable lost calculation for all models when using a softmax output. In other words, it might be OK for most of your work with CNN, but sometimes the calculations may get messed up because of this. So what do we do in this case? Well, we can actually incorporate the softmax into the lost function itself. All right. Speaking of the lost function, in TensorFlow, there are already existing classes for the different loss functions that we can use to define an object variable.

Recall that for classification problems, we need the sparse categorical cross entropy loss function. This is as simple as writing loss. FN equals T.F. Dot Keras dot losses. Dot sparse categorical cross entropy. This would be the lost function if we had a softmax activation. However, since we didn't add it, we need to set the parameter from Logits to true. This tells TensorFlow to incorporate the softmax into the lost function. OK, now we need to compile the model. This will combine the model with the loss, function and optimizer, and it will prepare our network for training. Again, we do this in one line model dot compile brackets optimizer equals atom loss equals loss.

Field and matrix equals the list containing accuracy. By running the cell, we have prepared our network for training so far so good before proceeding with training, we have to think about Overfitting. We know very well that our main tool against Overfitting will be early stopping or stopping the training process when the validation loss increases, in TensorFlow early stopping is a callback. Callbacks are functions that are called at the end of each episode the appropriate code to define an early stopping mechanism. In TensorFlow is early stopping. Equals T.F. Dot Keras dot callbacks. Dot early stopping brackets in the brackets we have the parameters monitor equals validation laws, which indicates what property to monitor while training.

Then we have min delta equals zero patients equals to verbose equals zero and mode equals auto. These parameters define that the program should stop the training process when the validation loss starts to increase for two subsequent epics. Of course, you are encouraged to play around with those parameters to better understand their behavior. Great. Now for the last thing we need to do, train our newly created network. To do this, we will use the fit method of our variable model. The parameters of this method are the important bit here, the first one is our training data sets, which we have in the trained data variable. Then we need to specify the number of epochs here, we will use the third variable we defined at the start num epochs, which we said to 20.

Now, to implement early stopping, we need to pass it to the callbacks parameter. There are many other callbacks other than early stopping, and we may wish to implement more than one. That's why we specify the callbacks and a list. We just passed the callback variable like that continuing. We add the validation data. Finally, we will set verbose to to this specifies what to print out during the training process to means that it will print info only at the end of each epic verbose equals one, on the other hand, will display progress bars for every batch. That's it. This is all we need to right now. When we run this cell, the training process will begin.

Bear in mind that depending on the machine you are running, this can take more than five minutes. Right? Notice how coding the actual network part is easier and quicker to write than the preprocessing if you know what you were doing. That is often the case, as you may have a clear idea on how to build your network. But the data set may give you a headache because of missing values on standardized variables and many other issues. OK, it looks like our network has finished training, however, it finished on epic nd way earlier than the twenty epics we set for the training. That is because of the early stopping we implemented. It identified that the validation loss started increasing here and after two epochs terminated the learning process.

Since we set the restore best weights parameter to true in the callback, our model is now using the weights from this epic here with the minimum validation loss great from the printout, we can see that the accuracy of the model is around 99 percent. But is that the true accuracy? Obviously not. This is just the training or validation accuracy. However, that is not the final accuracy of our model. To get the actual accuracy, we have to evaluate the network on the test data set. We will do precisely that in the next lecture. Thanks for watching.

LECTURE 25

Validation Accuracy and Data Set

Welcome back. Last time we train a network to recognize the mnist digits, it is time to test its capabilities. Now, we can do this with the evaluate method, which returns the loss and accuracy on the provided data set as we are aiming to test the network. It is only natural to use the test data set. We will also save the output to variables for future use. OK. Running the sell has forward propagated the test set, let's print those numbers in a pleasant format with enough decimal places like this. All right. Now, this here is the proper accuracy of our model. Notice it is a bit lower than the validation accuracy as it is supposed to be. Bear in mind, though, that the test accuracy is evaluated only at the end of all work related to the model just before exporting it.

If we wanted to tune the different hyper parameters, we need to use the validation accuracy. That is because in the process of finding the best type of parameters, we run the model. Each time we make a change, then we check the validation. Accuracy has improved. After 10 to 20 different combinations, we may reach a set up of hyper parameters with outstanding validation accuracy. But these are not the best type of parameters. These are just the hyper parameters that fit the validation data set best. Essentially, we are overfitting the hyper parameters. That's why we need the test. It is a data set the model has truly never seen before. Fantastic. We have created our very own convolutional neural network, exciting stuff.

We've managed to get 99 percent accuracy on the MNIST data set with it, and that was with a very simple model for the CNN with only two convolutional layers. This accuracy, though, is much higher than what we achieved with a simple feed for deep network. In the deep learning course, we managed to obtain around 97 percent, and those deep networks tend to perform better on the small scale. Sanitized images of the MNIST data set those twenty eight by twenty eight images are too small to extract the full potential of the CNN net. Having done all of the training and testing, you must be eager to see the network's capabilities in action. Well then let's feed one image to the model and see what the responses.

This is the code that we will be using to achieve that, we will not go over it, but we have prepared detailed comments on it and you can inspect it on your own to see how it works. OK, now let's check with the first image. Looks like so we enter one for the index variable eye and run the cell. Well, that's the first image in the test set. It contains a handwritten too. We can see the corresponding label below it. The question is now, what does the model output when presented with this image? Well, let's run the next cell to find out. And this bar chart pops on the screen. It plots the probability of each digit to be present in the image as calculated by our model.

As we can see for this particular image, the network correctly predicted the digit is a two with almost 100 percent confidence. Let's check some more images, take the 10th image, for example. And again, the model predicts a one with 99 plus percent confidence. Now for the 26th image, a three running the next cell, and surely the network predicts correctly. Continuing with 34, this time, there is a seven on the image slightly tilted to the left, let's obtain the model's predictions.

Well, well, well, it seems like the network was confused, it predicted that there's 40 percent chance that this image represents a three. Thirty five percent for a two, around 17 percent chance for a one and only six percent for the true label of seven.

So the network was fooled by the fact that the digit was skewed a bit to one side. OK, let's look at one more image, say, the five hundred and eighty ninth one. All right. So what we have illustrated here is a nine. But wait, the label below says eight. Apparently this image has been labeled as an eight in the data set, but it definitely looks like a nine to me. Well, that managed to fool even me, so I don't expect the model to be able to handle it. Let's see. Yeah. As we can see from the bar chart, the model is 96 percent sure that this was a nine. Interestingly, the second highest probability is eight with two percent. All right, we can continue looking at the results from each of the 10000 thousand images we have in the test set, you can explore that on your own.

The code is included in the resources with this lecture. Great. Let's recap what we've done in this section. First, we got familiar with the data set, and based on that, we figured a simple CNN architecture to use. That was before starting to write the code. Then we spent some time pre processing the data sets so we could make the most use of it. Finally, given we knew what our model is, we quickly coated it with TensorFlow to trained it and tested it. We have done a lot in this section, but there is still a lot to learn. For example, how to visualize parts of the learning process. To better understand our network, tune in the next section where we will introduce TensorBoard and answer this question.

Thanks for watching.

LECTURE 26

Date Time and TensorBoard

See the notebook `2.ipynb` for the code.

Hello there and welcome back. In this section, we will discuss the basic concepts of TensorBoard and how to use it TensorBoard as an extension to TensorFlow that provides tools for logging useful data and visualization of numerous metrics connected with the neural network. This helps you understand how your networks behave and find the optimal hyper parameters to use.

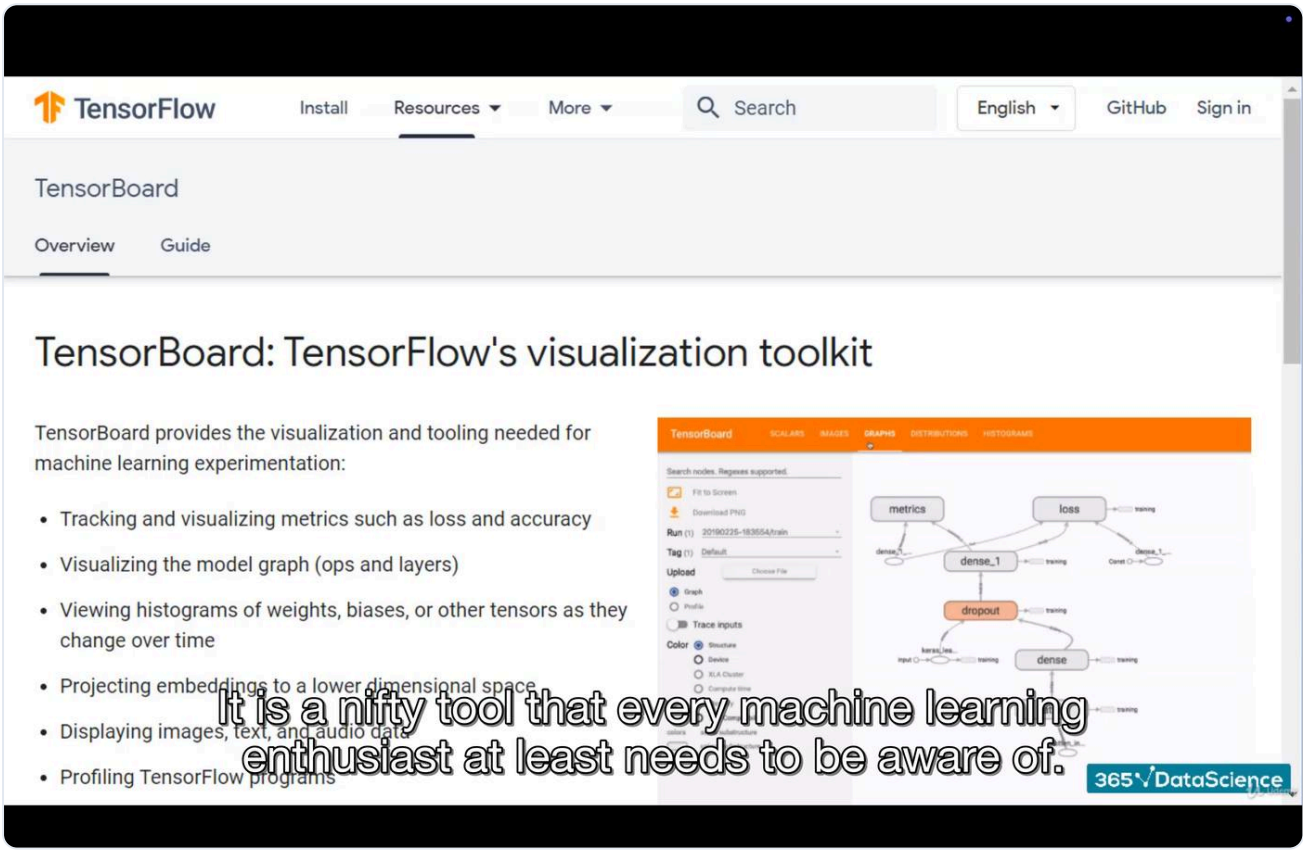

FIGURE 26.1

It is a nifty tool that every machine learning enthusiast at least needs to be aware of. However, TensorBoard has many applications, some of which are more specific. So in this section we will show you the basic stuff you can do with TensorBoard without going into too much detail. First, we will long the training process and see how to visualize general characteristics.

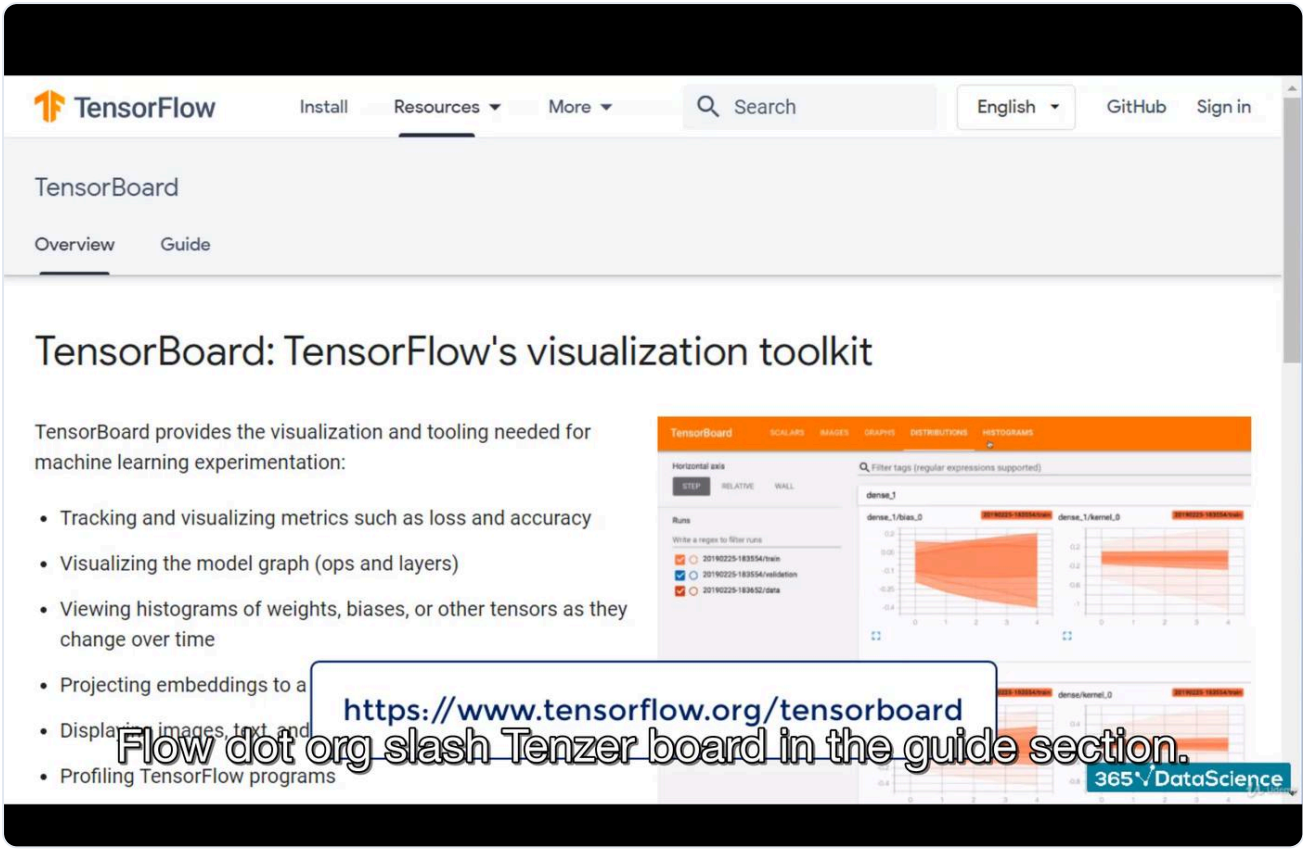

FIGURE 26.2

Then in the next lecture we will explain the concept of a confusion matrix and only after that we are going to perform hyper parameter tuning using TensorBoard. OK, let's get started. You can find the official resources concerning TensorBoard on its dedicated Web page here at TensorFlow dot org slash TensorBoard in the guide section.

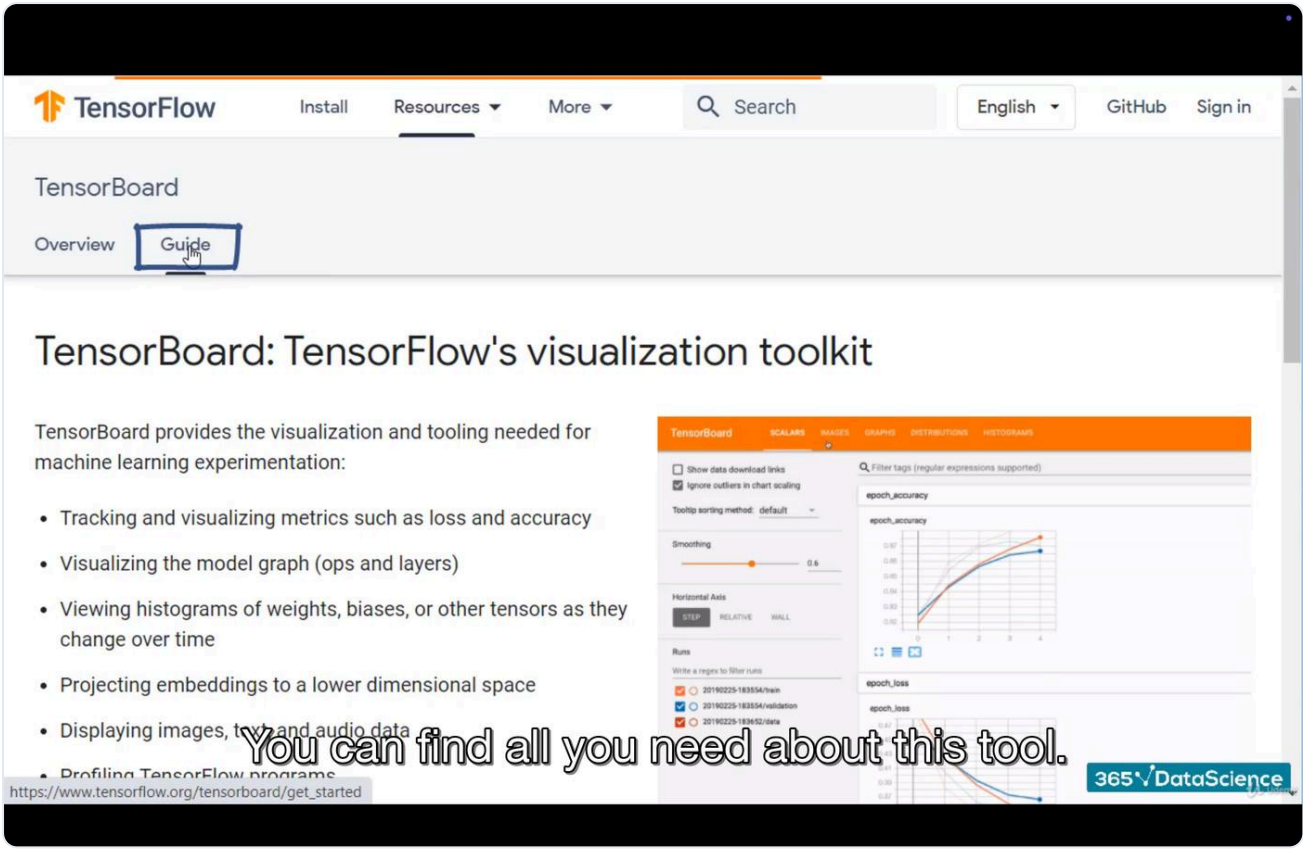

FIGURE 26.3

You can find all you need about this tool. It has many articles that outline different use cases. Since we will only show you the basics, feel free to explore any additional topics here. Now for this lecture, we will use our eminent example program from the last section. It is a good practice example as it is a relatively simple CNN and we already understand what it does and how it works. Luckily, TensorBoard is easy to implement into already existing code, we just need a couple of additional lines. In general, it works by exporting the quantities we are interested in to a logging file. The latter is updated throughout the learning process. All right, here is our code from the previous section.

Our task is to modify it so we can take advantage of the basic logging functionality of 10 support. So let's get down to business first. We need to import an additional module date time. This will help us with the logging we mentioned before. Let's start from the top and import the date time package to our project. Next up is the pre processing of the data. Well, this step has nothing to do with TensorBoard. So this whole section remains as is. We will just run the cells quickly here. Now we arrive at the CNN parts since we don't want to change the network itself, the model and lost function remain the same. Again, we compiled and defined the early stopping by running these cells.

Now we need to add a couple of lines of code. First, we will define the relevant directory where we'll log our data in this notebook. We will probably train our model more than once with different type of parameters. So creating different folders for the different runs would be a good idea to easily distinguish between them. We will incorporate the local date and time into the name. That's why we imported the date time module. This is a very neat way to have a unique folder for

each iteration and even have them in chronological order. OK, so the variable where we store the location will be called log's der, it is equal to log's double slash fit, double slash, then we will append the current date time.

So plus date time, dot, date time dot. Now, finally, let's also add some convention about the format we will display. I like year, month, day dash our minute seconds. So Dot strftime brackets and your preferred format make sure to include all this information no matter in what order to make sure your folders are truly unique. Now this line simply defines the path to store the long data and as a string, some folders are defined with a double slash to avoid confusion with the python escape symbol. You can change this directory to whatever you like, but adding the timestamp is strongly encouraged. So far, this has nothing to do with TensorBoard at all to be able to use it.

First, we need to create a callback for it. We already implemented one callback the early stopping. In case you need a hint, a callback is a function or an operation that is executed at the end of each episode. Now, similarly to the early stopping, we will add TensorBoard as a callback. We define the TensorBoard callback in just one line. TensorBoard callback equals T.F. Dot Keras dot callback. DOT TensorBoard. Brackets in the brackets we define log der equals log der and histogram frequency equals one. That's it, running this cell creates the TensorBoard carmac, but does not incorporate it into the training process. Recall that this is just a variable we need to pass to the callback parameter in the fit method to incorporate it into our model.

We add this callback as another item in the callback list like that. A very important thing to note here is that the early stopping callback should always be the last element of the list. Otherwise, it bugs out and does not execute. Great, let's outline the changes we've made so far, we added the module date time at the beginning, then we define the directory where we save the log files and created the TensorBoard callback here. Finally, in the fit method, we added this callback, that's all. And now when we run all the cells, our network will be trained just as before, but the training process will be recorded. Let's speed the training a bit. OK, now we have trained our network and recorded the training itself, so what next?

Well, if you have ever seen TensorBoard before, you probably imagine that we will start making some neat visualizations. We're almost there. So how do we access those visuals? Well, we need to start the actual TensorBoard application. We do this with the following two commands percent load TensorBoard. And present tense board. Dash, dash logged deer logs slash fit. The first command closed the extension itself and the second one starts it and states the directorate where we look for the locks, so we run this cell and here it is. That is the TensorFlow broad extension. A word of warning, though, on Windows 10. Support has trouble starting if the extension has been running.

So the first time you start it, it will run properly. But if you subsequently try to restart it or open a different directory, the extension will encounter an error. That happens because Windows cannot clean up the temporary data used by TensorBoard after you finish with it. Luckily, there is a quick workaround. All you need to do is write two commands in the show. First, open the command prompt. In there you need to paste the following two lines, one after another. Task kill. I

am TensorBoard, exi SGF and DTL sgu precent Tempy percent backward slash dot TensorBoard info backward slash star. These will end already active TensorBoard processes and clean the temporary data associated with TensorBoard so you can run it again if either of those gives an error.

That's OK. You can ignore it. Once you have entered these commands in the command prompt, you can rerun the cell. Usually the first attempt to run the cell will result in a timeout. Then if you run it again, it will start up correctly. For more details on this issue, you can check this thread on GitHub TensorBoard in Jupyter Localhost refused to connect issue on Windows. The Tuckerman's are here. I suggest you bookmark this page or copy the two statements on some local file because you will need them every time you want to start another 10 support instance. All right, back to the code, let's take a look at the interface of the extension down here to the left, you can see the different runs of training we did with our network.

We have trained it only once, but there are two runs displayed. That's because one is for the training sets and the other is for the validation set. When having multiple runs, you can choose which ones to display the data for by ticking or until the checkbox is next to them. OK. You can see up here that right now we have opened the scalars tab in this tab, different metrics such as the loss and accuracy are displayed. You can even log your custom scalars to be displayed here. Let's take a look at how the accuracy evolves with the epochs by default. These graphs are smoothed a bit as to look more continuous. This may be useful when we have train the model for a big number of epochs, but that is not the case here.

So we will remove the smoothing with this slider to the left. We will set it to zero. On the graph we can see two lines. One is the training set accuracy. The other is the validation. One, we can see how the accuracy cross both sets increases throughout the training process. Notice that by hovering with the mouse above the graph, it displays the exact numbers. Below is a similar graph showing the loss of our model instead of the accuracy for our simple network. These may not be that useful, but for real, more complicated models. You may gain insights into their performance from these graphs. Good. That was the scalars tab. Moving on, we have the graphs tab.

Let's click it and see what we get. Hmm. We can see all the layers of our model. That's because here is visualize the computational graph of our network. This is quite similar to the graphic we had when describing our model qualitatively. This graph shows all the layers and computations that are made in the network and how they are all connected. You can zoom in to see specific parts of the graph on the connecting arrows. You can also see the dimensions of the inputs and outputs, which can be very useful, especially with big models. When you click on a layer, a window with additional information pops on the right, you can also double click a layer to expand it.

When you expand it, it shows all the internal operations carried out, such as the addition of the bias, the activation function and others. Ten support also provides functionality to name the different layers with custom names. So this graph view is very flexible. All right, this sounds great. There are other tabs, such as the distribution's one and the histograms in those you can see and track the evolution of the various tensors as the network is learning. As you see, TensorBoard can be useful in many ways besides the things we've shown so far. You can also plot images or other data, including audio. You can track many complex metrics during the training, such as the confusion matrix, which is exactly the focus of our next video.

See you there. And thanks for watching.

LECTURE 27

Confusion Matrix

see the notebook `3.ipynb` for the code.

So far, when evaluating a model, we have only considered the accuracy and loss metrics today. We are going to introduce another useful metric related to classification problems called confusion matrix and see how we can implement it. With the help of 10 support. The confusion matrix, sometimes called error matrix, is a table that shows which categories our network excels at predicting and which ones it struggles to deal with. While not too complex as an idea, a confusion matrix completely takes the place of the accuracy metric.

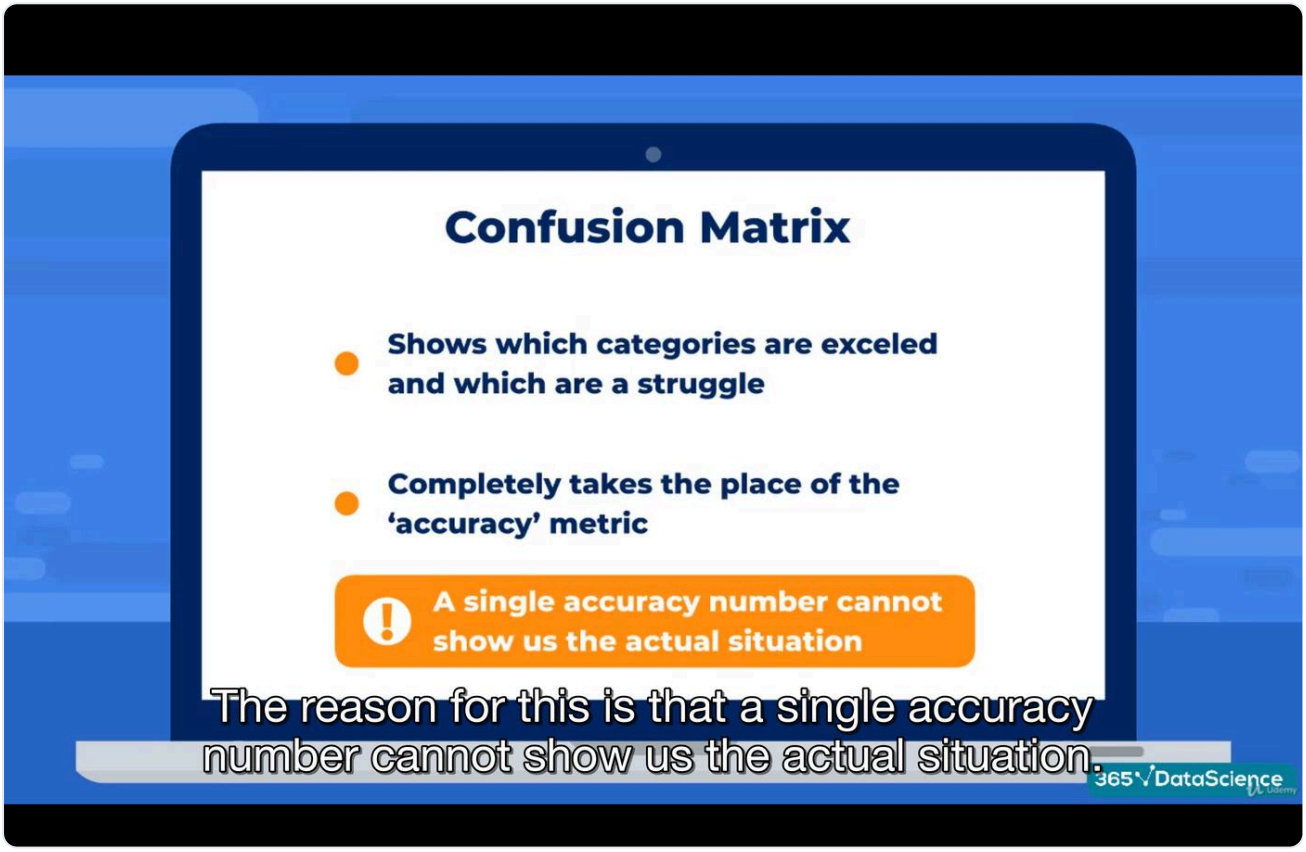

FIGURE 27.1 | Confusion Matrix

The reason for this is that a single accuracy number cannot show us the actual situation.

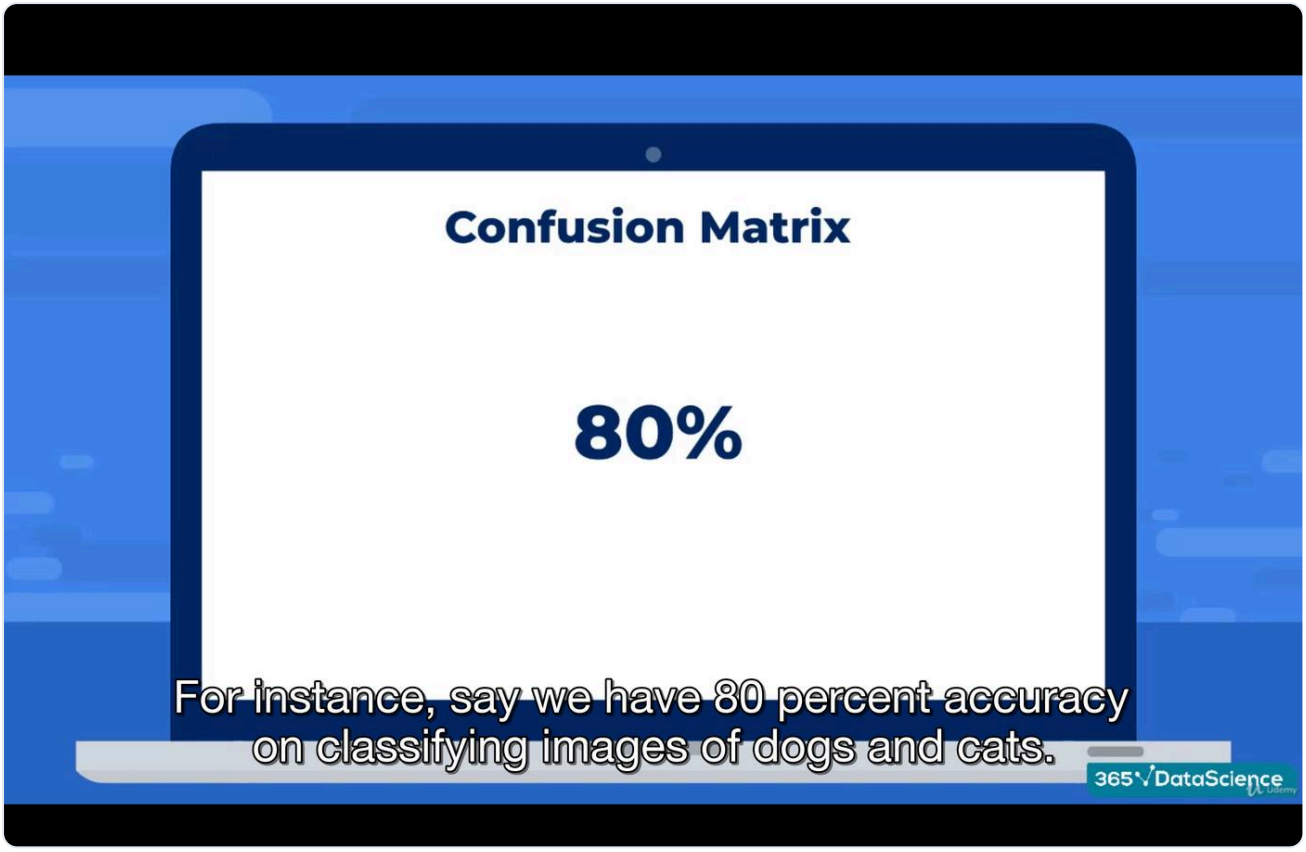

FIGURE 27.2 | Confusion Matrix

For instance, say we have 80 percent accuracy on classifying images of dogs and cats.

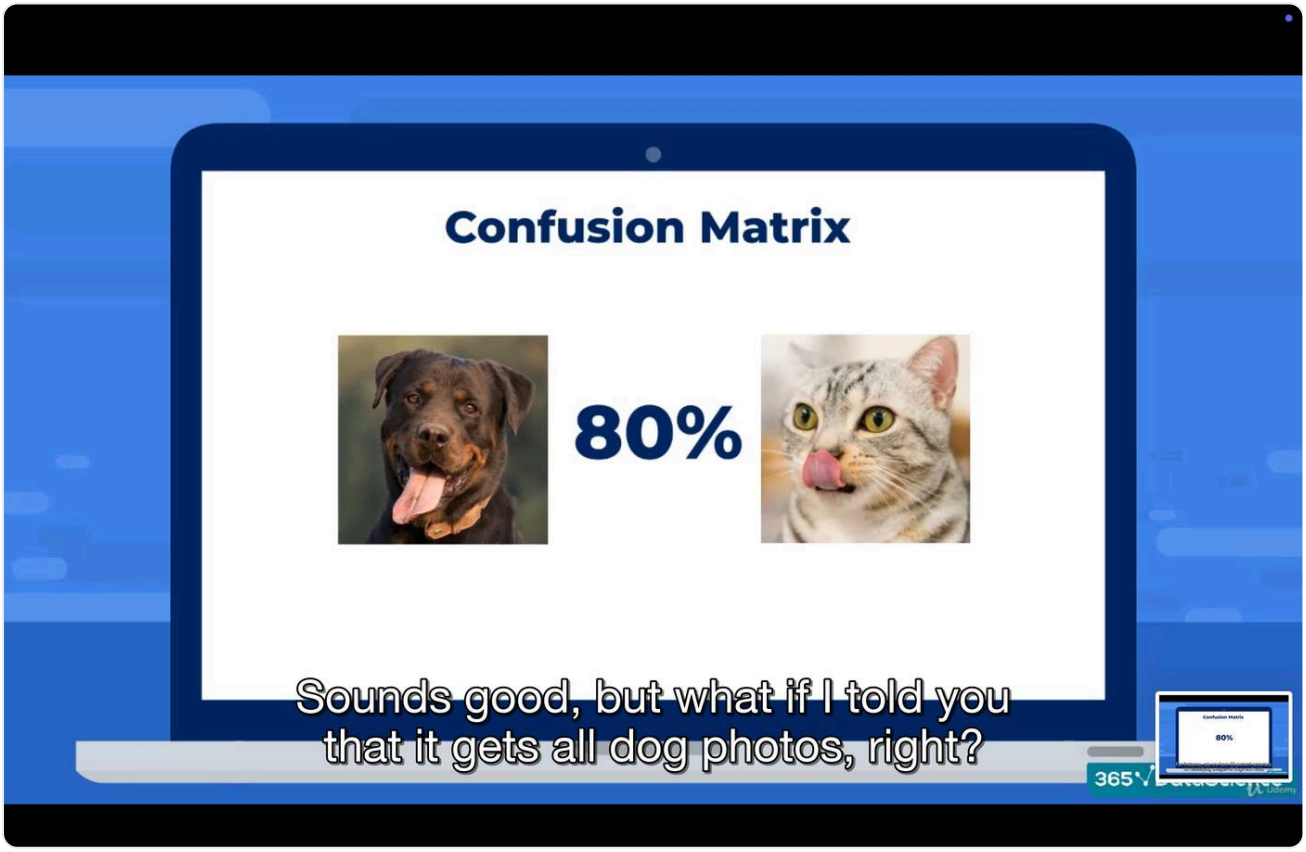

FIGURE 27.3 | Confusion Matrix

Sounds good, but what if I told you that it gets all dog photos, right?

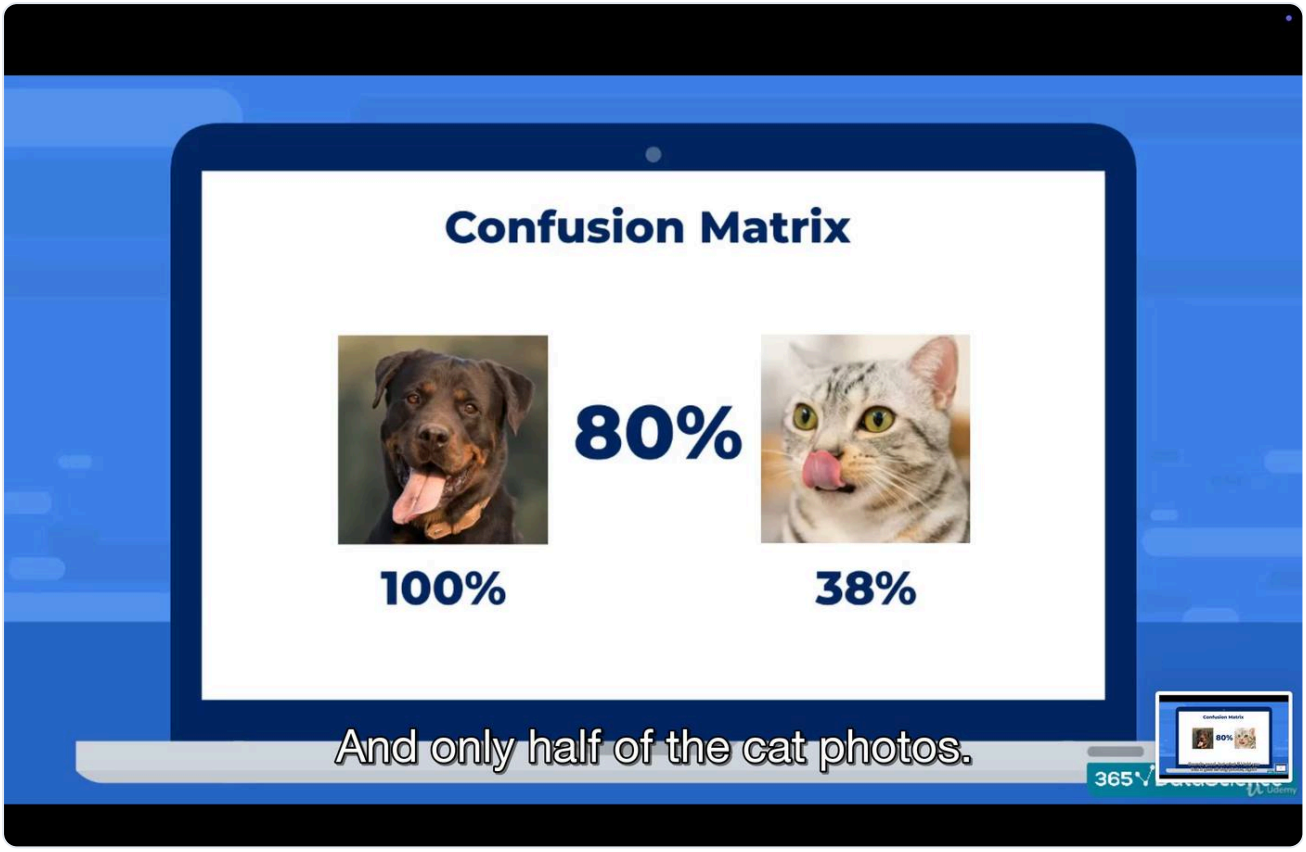

FIGURE 27.4 | Confusion Matrix

And only half of the cat photos. Right. Although the accuracy itself is not bad. This sounds like a lousy model. Now, the general question that we want to ask at the end of the training is where does the network struggle the most?

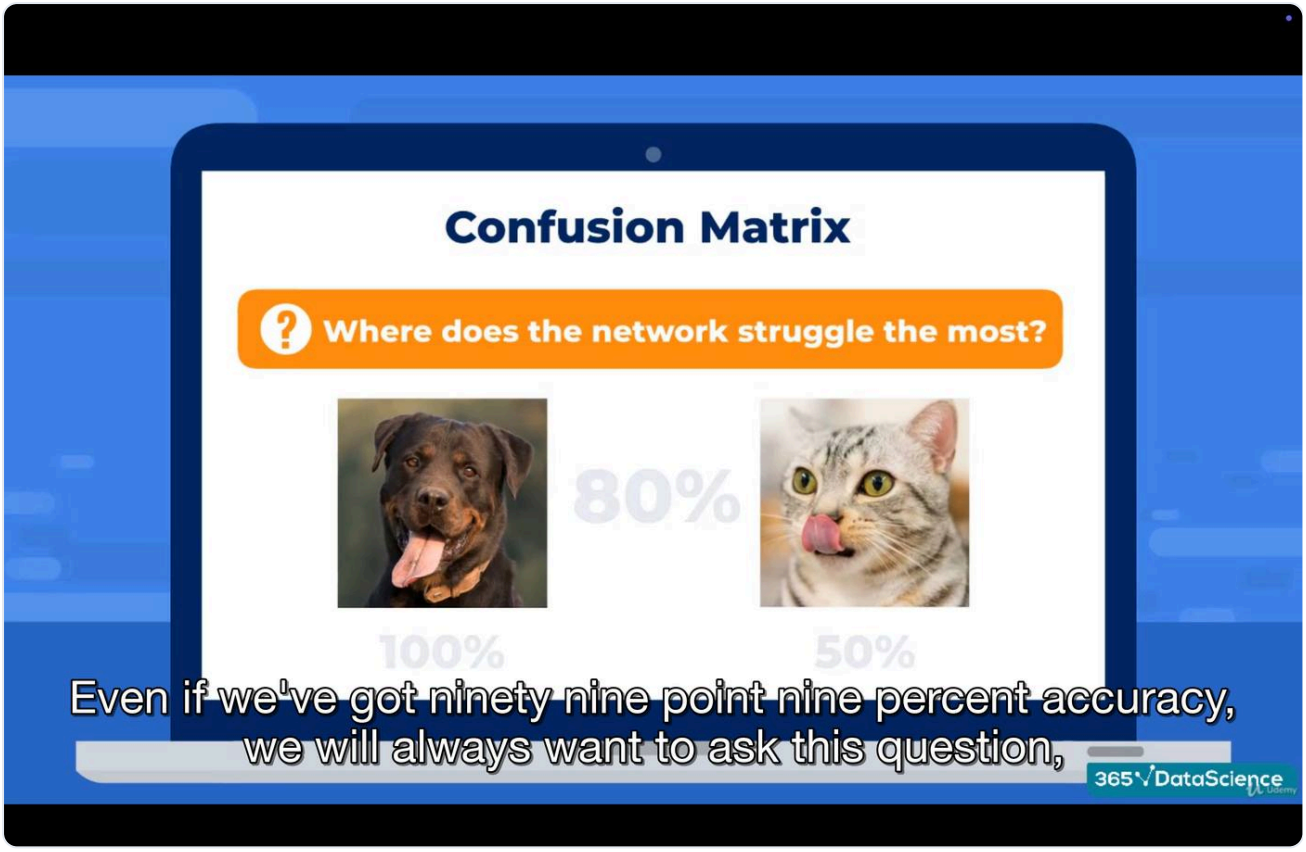

FIGURE 27.5 | Confusion Matrix

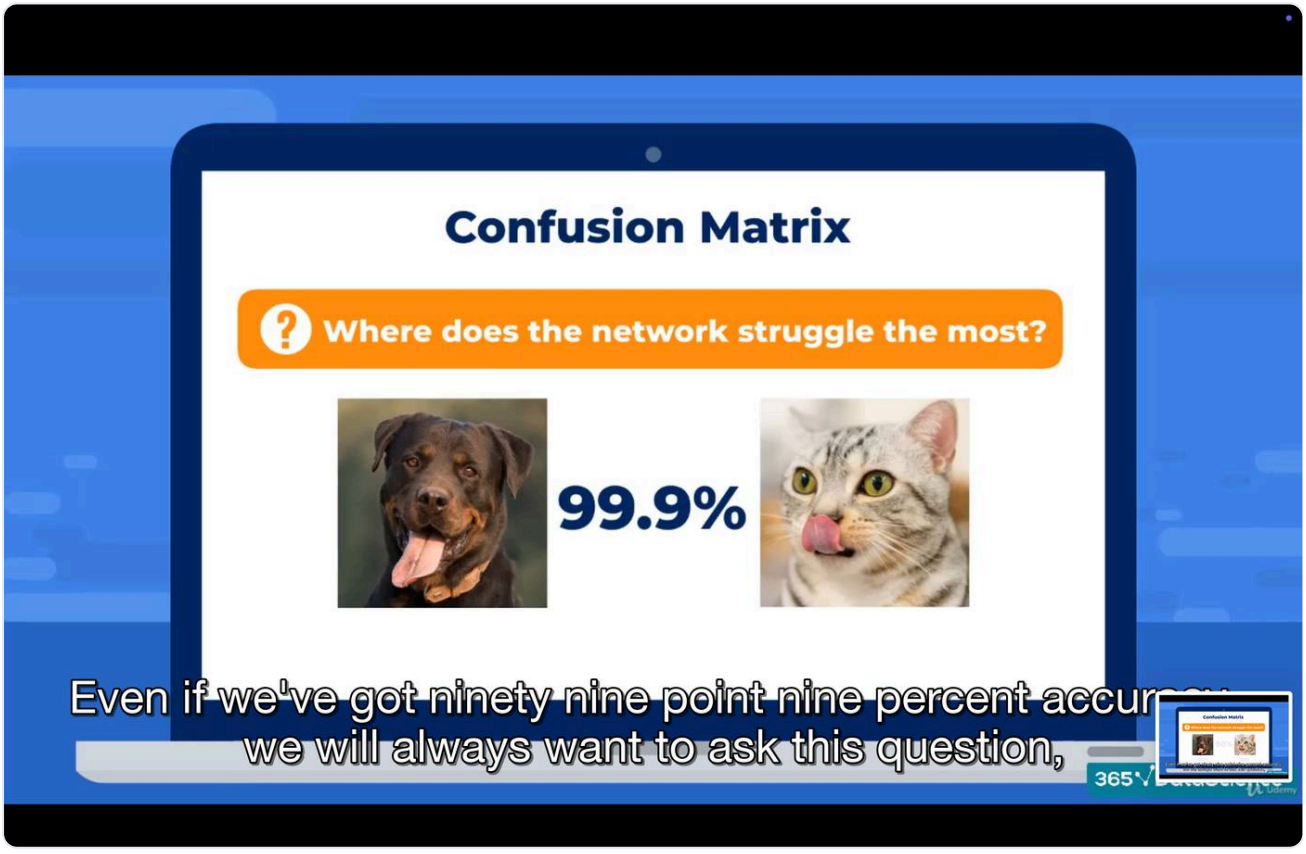

FIGURE 27.6 | Confusion Matrix

Even if we've got ninety nine point nine percent accuracy, we will always want to ask this question, are the cats images problematic or the dog ones? Maybe they are equally hard for the model to distinguish. And this is why the confusion matrix is so important.

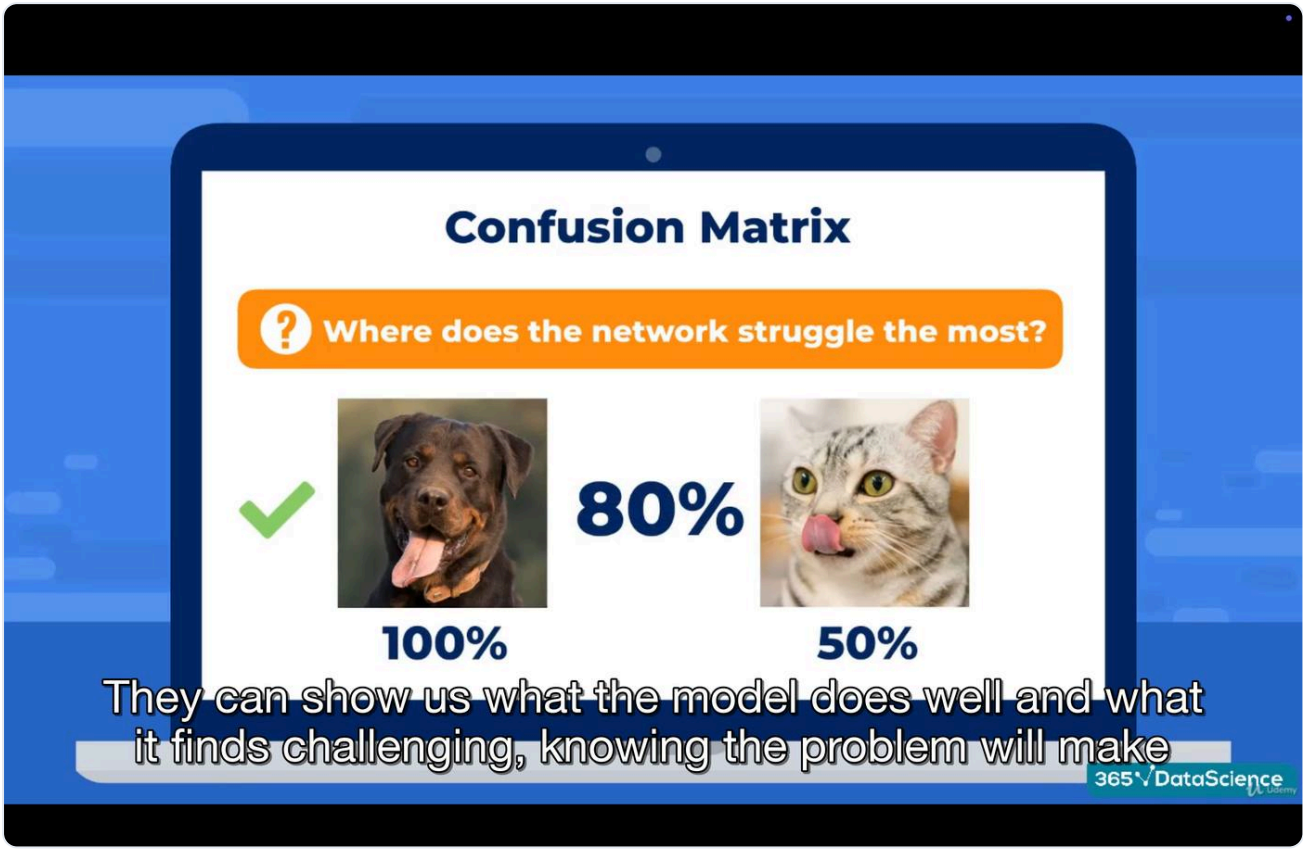

FIGURE 27.7 | Confusion Matrix

They can show us what the model does well and what it finds challenging, knowing the problem will make improving the model much easier. All right. Now that you're convinced we should use a confusing matrix, let's see what it looks like. A confusion matrix is a table where on the Y axis we have the true labels of our data, for instance, cat, dog and Wolf on the x axis are the predicted labels.

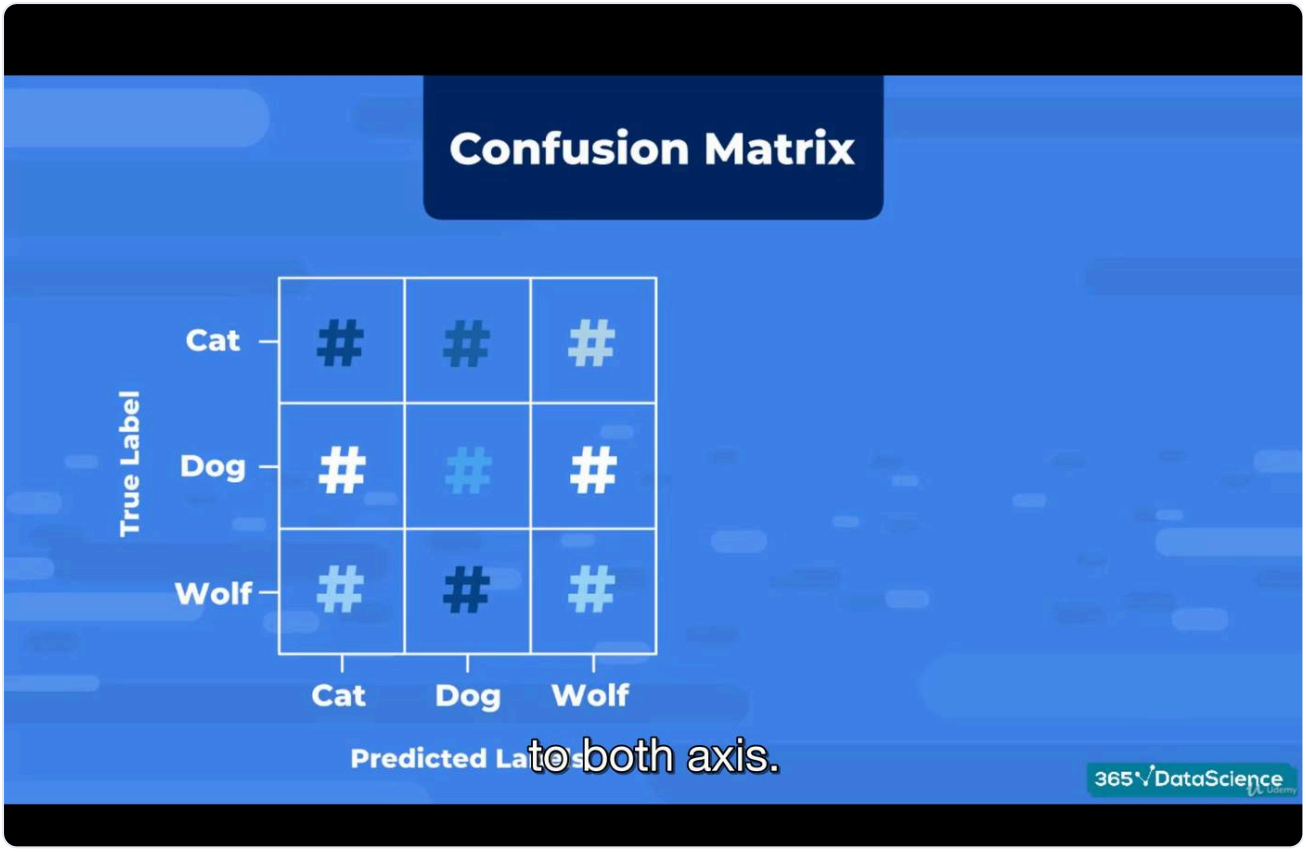

FIGURE 27.8 | Confusion Matrix

That is the ones our network assigned to the images, the cell show, the number of cases that conform to both axis. They are usually color coded, often with blue. For example, suppose we test a network with 20 images of cats, 20 of dogs and 20 of wolves.

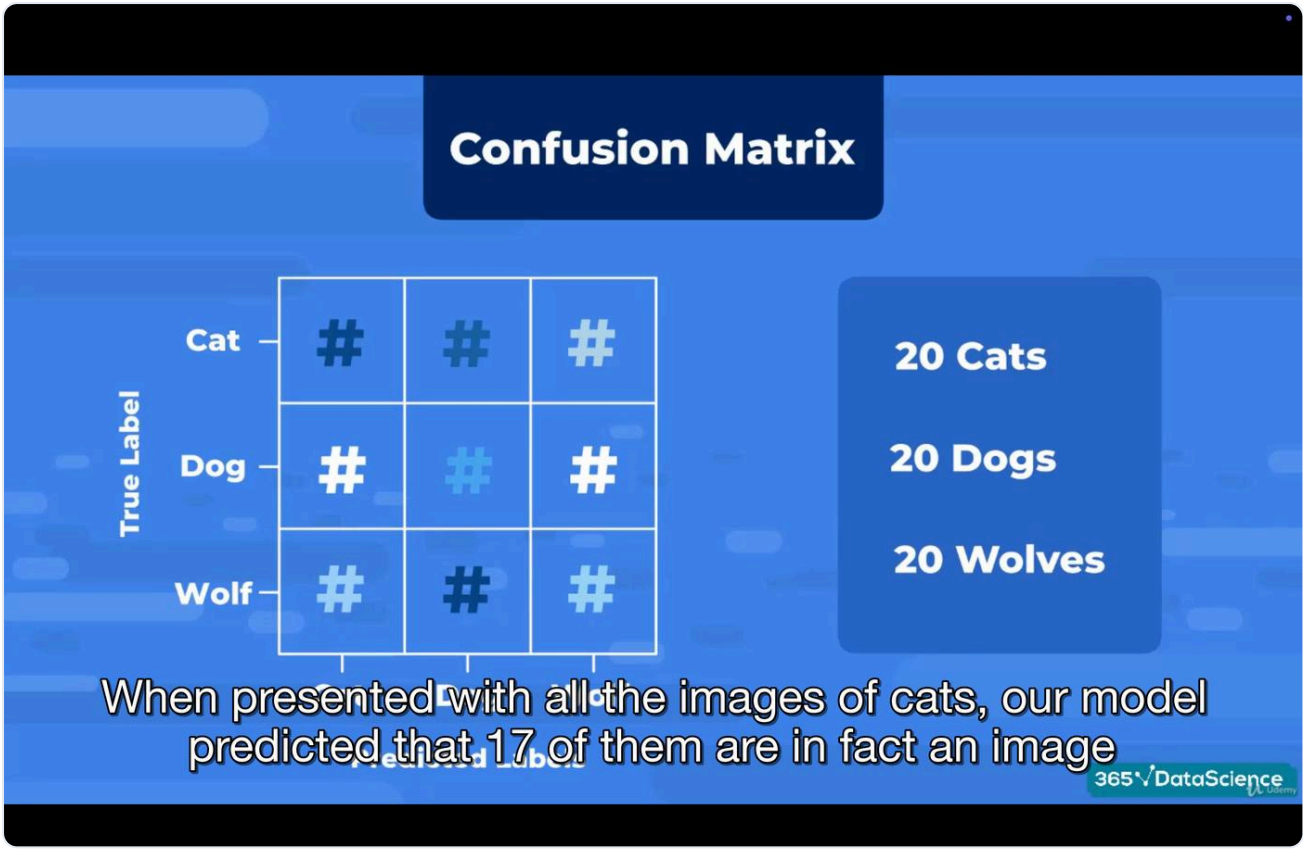

FIGURE 27.9 | Confusion Matrix

When presented with all the images of cats, our model predicted that 17 of them are in fact an image of a cat. Two were a dog and one was misclassified as a wolf for the dogs. The network assumed two were cats, 13 were dogs and five were wolves. And finally, for the wolves, it correctly asserted that zero our cats. However, it misidentified two to be dogs and the rest 18 were correctly classified as wolves. Notice how the cells are filled with different intensities of blue.

FIGURE 27.10 | Confusion Matrix

The higher the intensity, the higher the number in the cells. This makes it easy to read the Matrix at a glance, even when it becomes more cluttered with a higher number of classes. Another often applied modification to this table is its normalization. Instead of showcasing the exact numbers of classified images, it is often easier to work with percentages. So instead of having this five here, we ask How much is that out of the twenty dog photos?

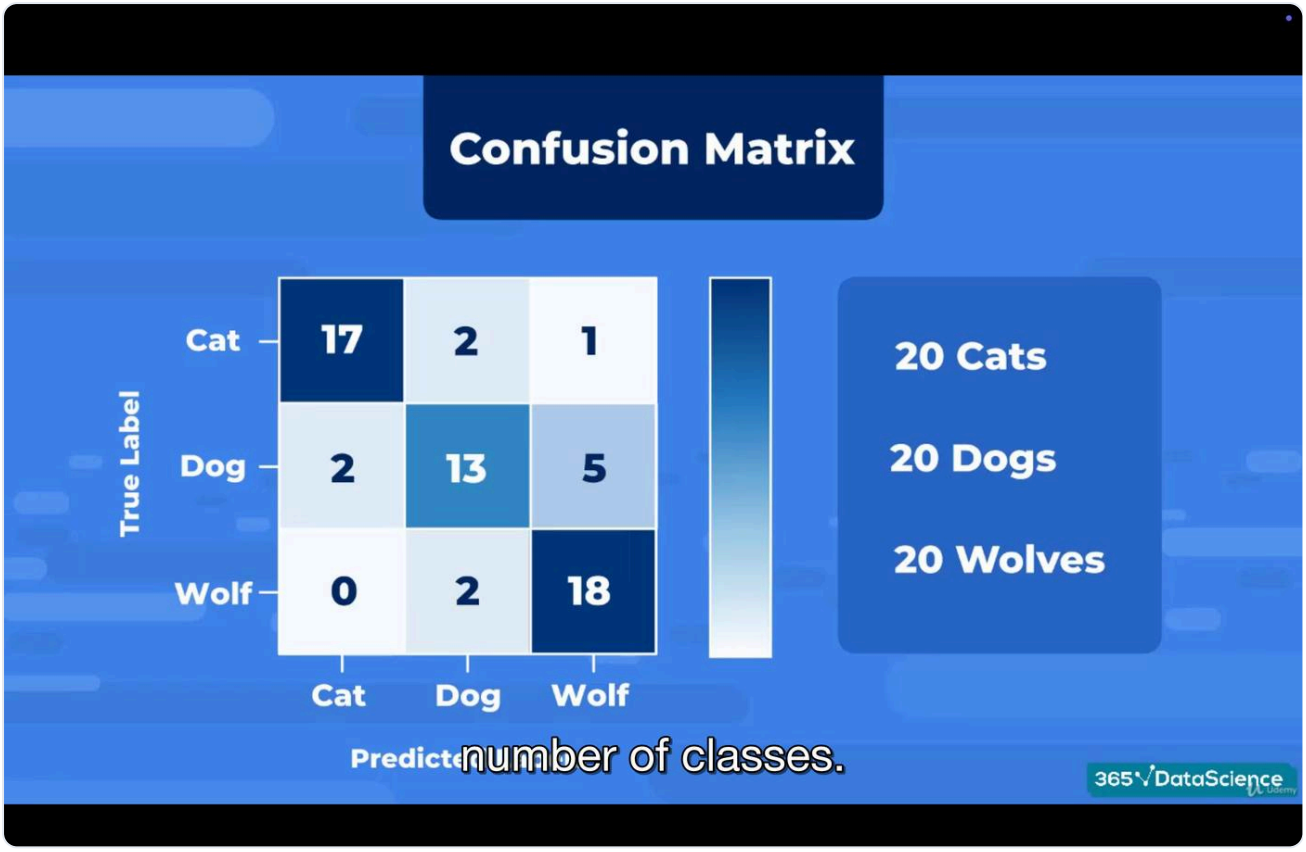

FIGURE 27.11 | Confusion Matrix

The answer is twenty five percent. So the cell shows zero point twenty five. That is done for the whole confusion matrix. In fact, most of the confusion matrices you will encounter will be normalized. However, it is important to remember that the normalization is done for rows, not columns. Thus, the sum of every row will always add up to 100 percent.

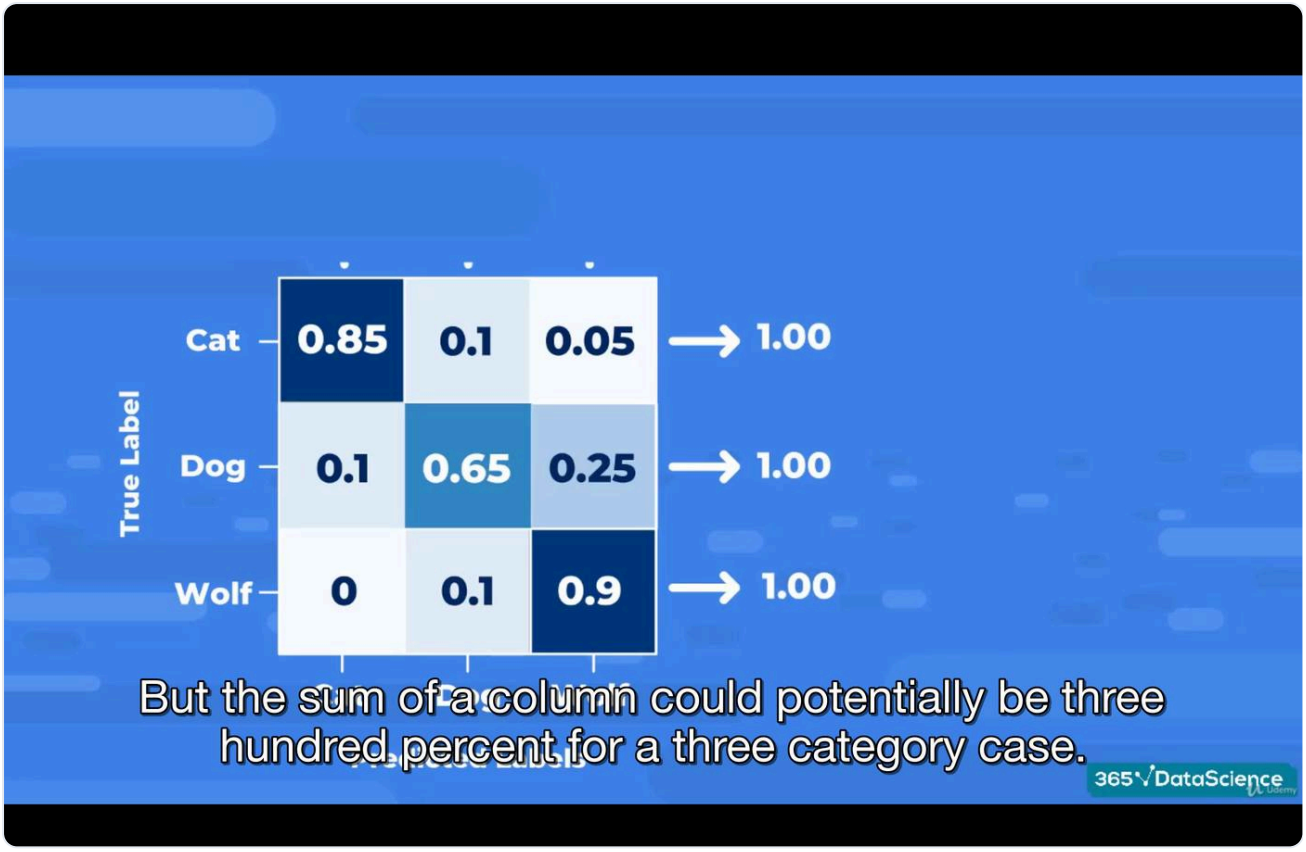

FIGURE 27.12

But the sum of a column could potentially be three hundred percent for a three category case.

FIGURE 27.13

Great, now reading this table is not that difficult in general, the assumption is that the majority of cases are classified correctly, otherwise the accuracy would be too low for the confusion matrix to be useful.

FIGURE 27.14

These correct labels lie all on the diagonal, so only the deviations from this diagonal are important.

FIGURE 27.15

The color here helps a lot, as most other cells should be around zero and therefore white. Those cells that have significant MIS classifications will stick out due to the coloring. For the example confusion matrix. We can agree that the network does not struggle much with images of cats and wolves. However, it often thinks that the image of a dog is a wolf. Given this information, we may begin to create assumptions about the data set. Maybe the wolves are very similar to each other. They might all have the same fur color and pattern. Maybe the data set has them all in the same pose due to the prevalence of that iconography in pop culture. Thus, the network has a much easier time learning what a picture of a wolf is.

In contrast, the dog class may consist of wildly different breeds, some small, others larger.

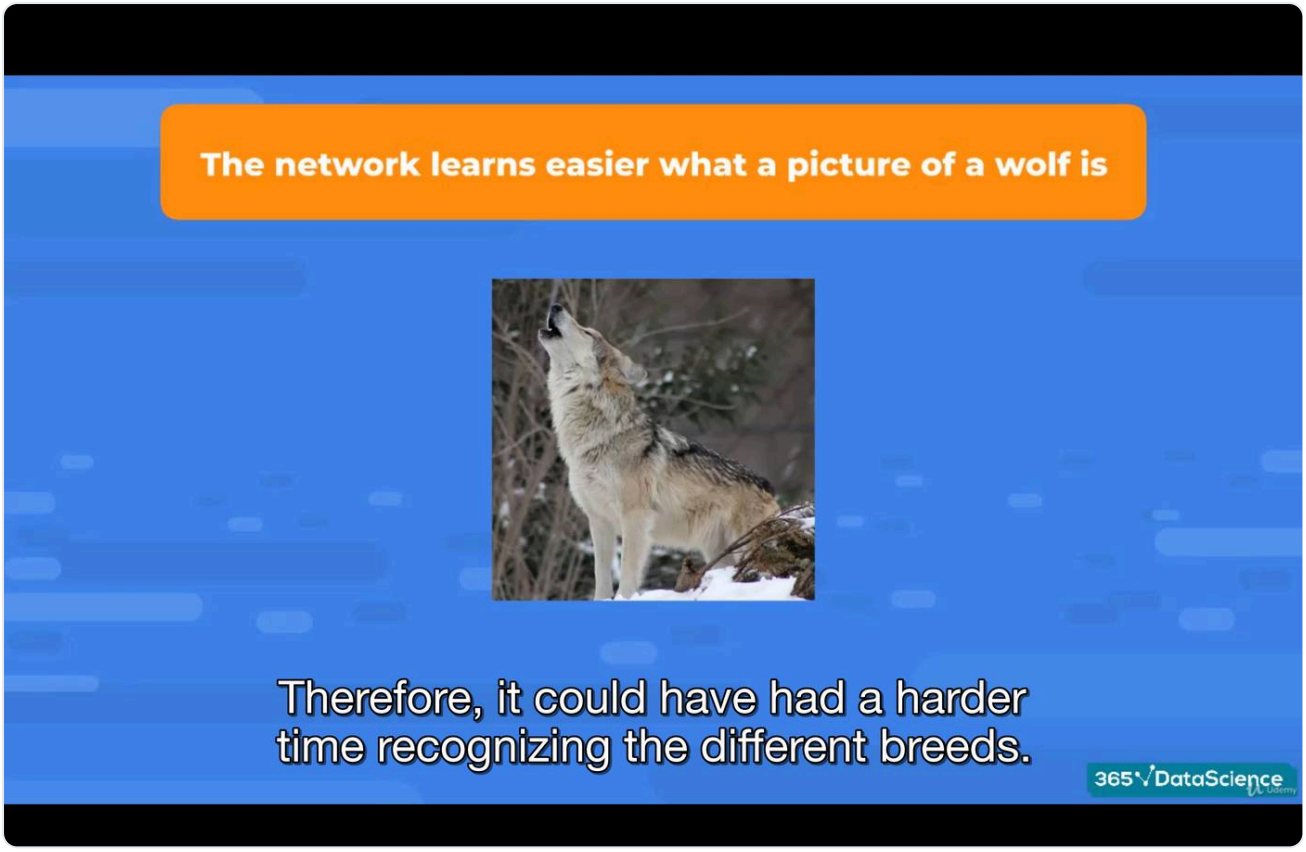

FIGURE 27.16

Therefore, it could have had a harder time recognizing the different breeds. So it learned that Chihuahua is a dog.

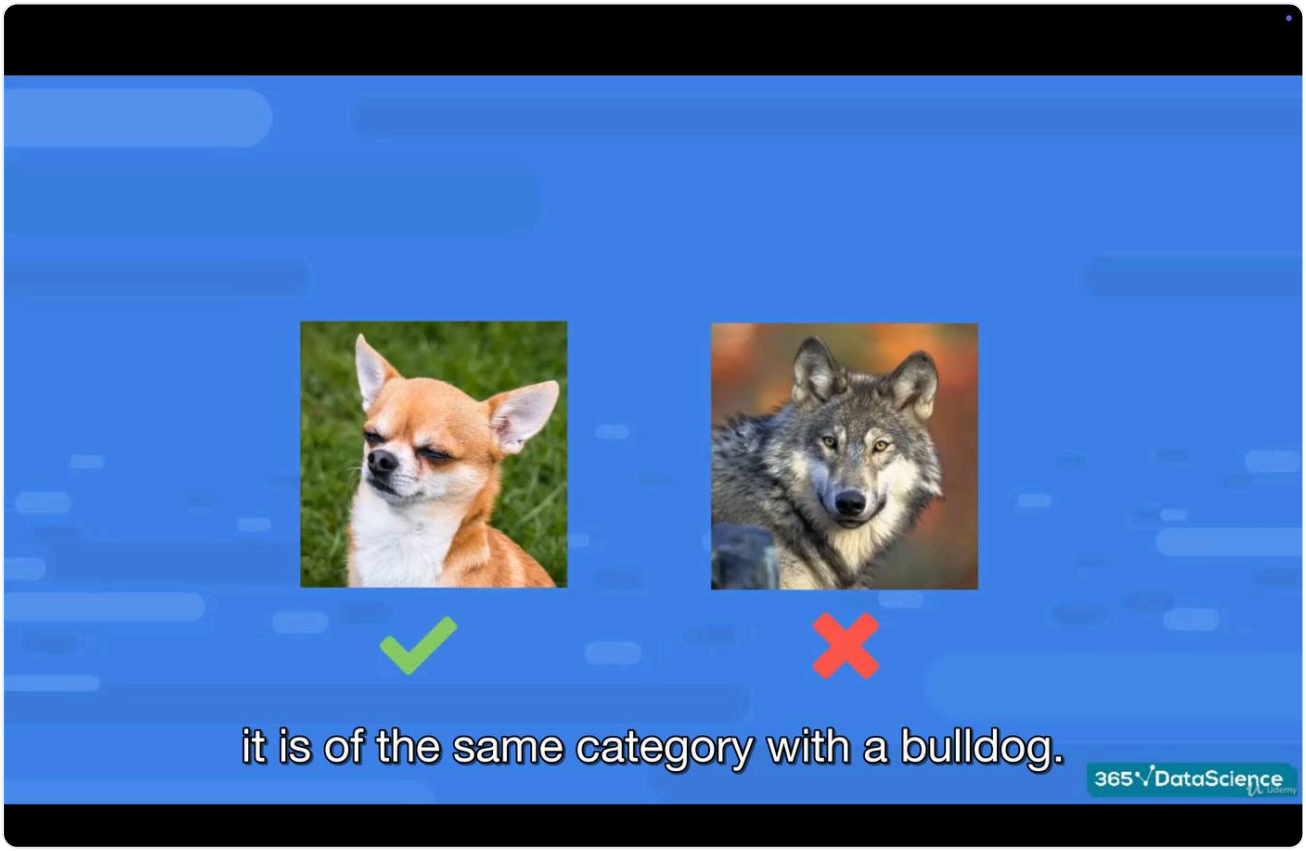

FIGURE 27.17

But the Siberian husky is so close in appearance to the wolves that it can't cope with the fact that it is of the same category with a bulldog.

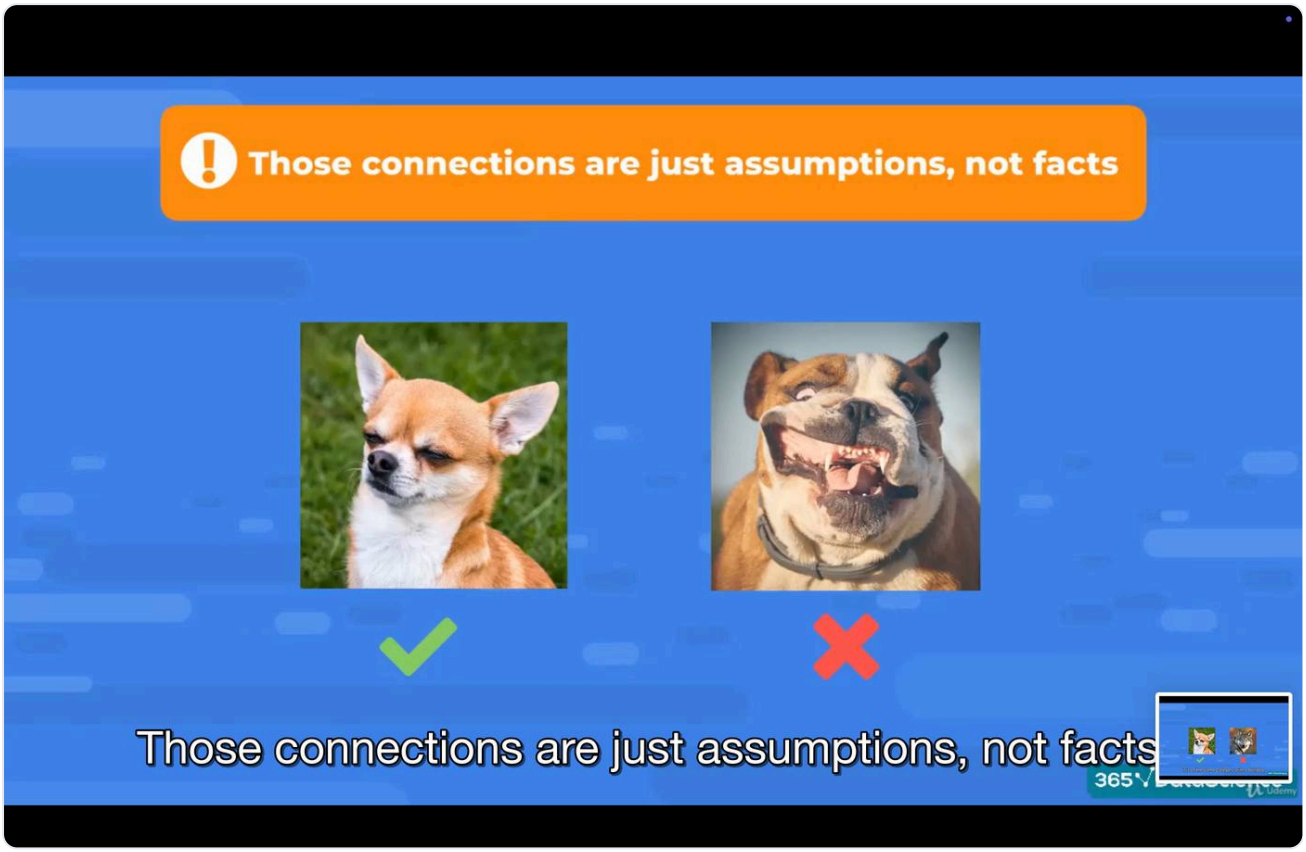

FIGURE 27.18

Those connections are just assumptions, not facts. However, with the help of the confusion matrix, we may construct a testable hypothesis such as this one that can later, with more research, be verified or rejected. Good. Now that we've learned what a confusion matrix is, let's see how to implement it in the code. We will not delve into every single line of code, but just the general idea. This implementation has been taken from the TensorBoard guide on image data as it is clean and structured code and there is no need in reinventing the wheel. There are three main points and logging and displaying the confusion matrix that we will define in functions, one of them is to plot the Matrix visually in Matplotlib, given the already computed values.

Another is to convert the plot to an actual image. And the last one is about computing the numbers and logging the info to TensorBoard. All right, specifically for the confusion matrix, but we need these additional libraries and packages io, itertools, scikit-learn, dot matrix and matplotlib dot pyplot. Now let's inspect the code for the three functions. First is the plot confusion matrix, which has a very descriptive name. It creates the Matplotlib plot. Given the numerical data, it takes the numerical values system and the class names as input. The first line creates an empty figure of the given size and inches. Here you can see the color map that is chosen is blue.

Here is the title and the next three lines arranged the X and Y labels given the class names. This line of code here normalizes the matrix, that is, it transforms the numbers into percentages, the next block ensures that the next color would be visible regardless of the background shade of blue. For white background, it uses black text. And for a very blue background, it goes with white

text. Finally, it states the axis labels and returns the figure. Next up, we have the plot to image which converts the return figure to PNG. Nothing much to talk about here. So let's continue with the next function. Finally, we have the log confusion matrix function. It calculates all the numbers and logs, the result from the previous two functions to TensorBoard.

I have to point out, though, that we need to define this file writer variable outside the function. The parameter inside the brackets is the directory where we want to log the results. I will denote it with s.m standing for confusion matrix. So to calculate the Matrix, we first need to get all the predicted labels from the model which is done to the first two lines, then the Matrix itself is calculated by the scikit-learn package. After that, it creates the confusion matrix image using the previous two functions and finally logs the results for every episode. Let me just point out that these variables here are global. Those can be substituted with the names you gave them.

Another point is that it is here where you define the class names. Be careful with that as the name should be in the correct order and it's easy to forget to change them when you copy your code. Great. At the very least, we need to define the callback that would go into the fit method. It basically says that at the end of each epic, python needs to run this function. That's it. Fantastic. Now let's see it implemented in the amnesty code. Here is the unmodified code from the previous section. The first thing to change is the packages to be imported. As we've discussed for the confusion matrix, we need io, itertools, num pi, scikit-learn, dot matrix and Matplotlib.

OK, next, we have the preprocessing, but it remains largely the same, except for a single addition at the end of all that stuff, we need to add this for loop. Its purpose is to extract the images and their labels from the validation data as an umpire.

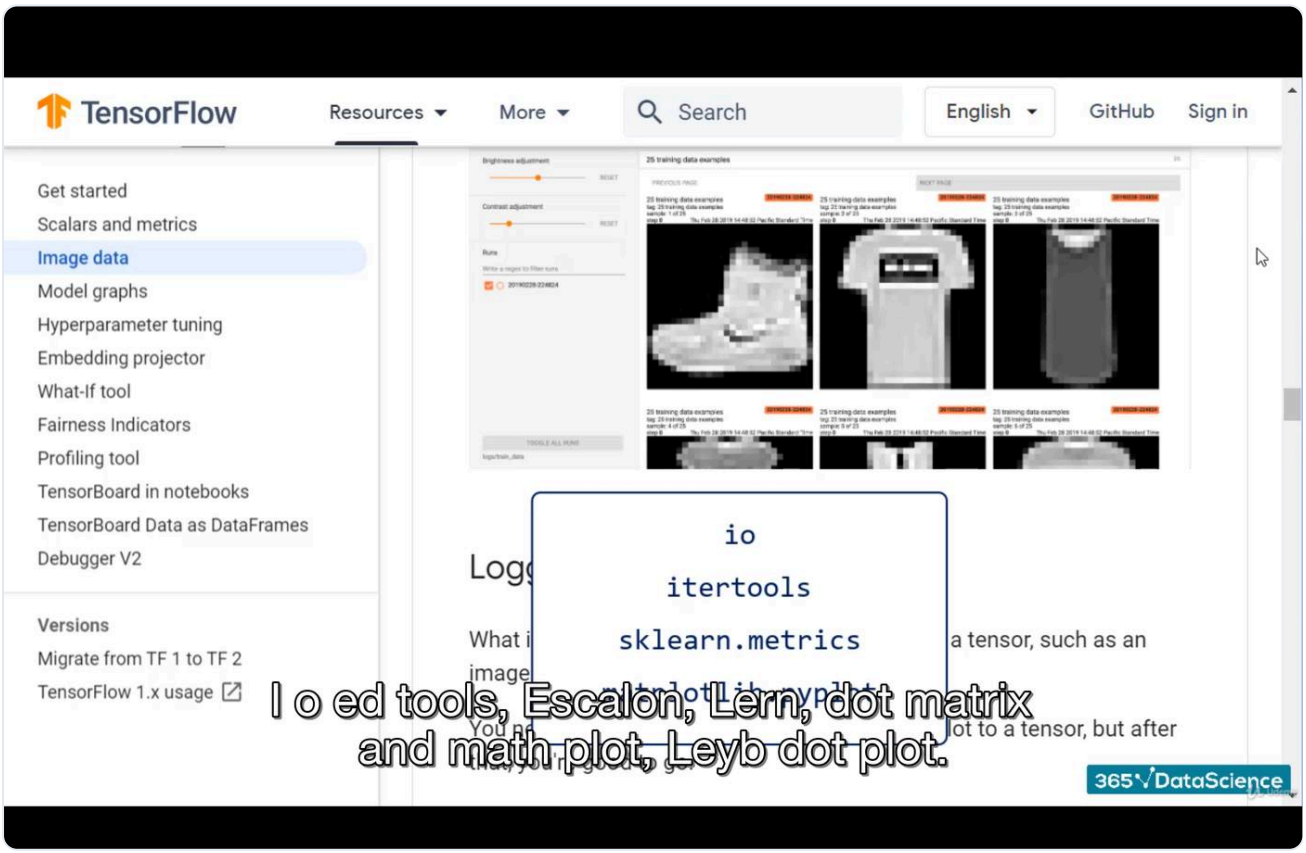

FIGURE 27.19 | Image Data

We need to do this since the variable validation data is that the custom data set type, if for a different project you are working with data that is in the form of arrays already. You can skip that part. All right. Then we have the part of the code dealing with a network. This code is actually going to remain the same. We just need to add the additional functions. A logical place to insert them is after the model dot compile line. First, we define the logging directory, it will again be logs fit. However, in contrast to the previous lecture, we will not use the current date and time. We will just enumerate the different runs. So this time it will be run one.

Then we can copy and paste the functions we talked about. We have plot confusion, matrix, followed by the plot to image function and finally the log confusion matrix function. In this last function, we may need to change some variable names first. This model here refers to the variable of the model we are currently training. Fortunately, we have named our variable model as well. So that's fine. These two variables here refer to the images and labels that should serve as a basis for our confusion matrix. In our case, we are going to determine the matrix based on the validation set, so we must change them to images Val and labels Val respectively. That's why we added that cell at the preprocessing part.

Finally, we need to specify the class names since we are classifying digits. These are simply the digits zero to nine. We have the three functions done, all we have left to do is define the callbacks. As we've discussed, we have this s.m callback to deal with the confusion matrix. We will also define the standard TensorBoard login callback, OK, adding those callbacks to the fit method. And we are done now. We have a functioning program that regularly logs the confusion matrix. Let's train the network and check the results. And we are back after the model has been trained. Let's get straight to the most exciting part, visualizing the confusion matrix. Now we start TensorBoard with a standard commands.

However, we get an error. That is the problem of TensorBoard on Windows we mentioned earlier. So let's follow the fix we provided here. I am opening the command prompt in it. I will paste the two commands outline in the workaround. Here is the first and now the second. That's it. Now, going back to Jupyter, we can run the TensorBoard, sell it now starts to process stuff and after a while it is going to time out just like that. If I run it a second time, though, it starts just fine. Here it is the familiar screen with loss and accuracy graphs. If we take a look to the left, we see three logs run one train, run one validation and run one system. What we are looking for is the s.m one.

It stands for confusion matrix. But why doesn't it show up then? We have selected the Cameroun, haven't we? Well, the confusion matrix is an image.

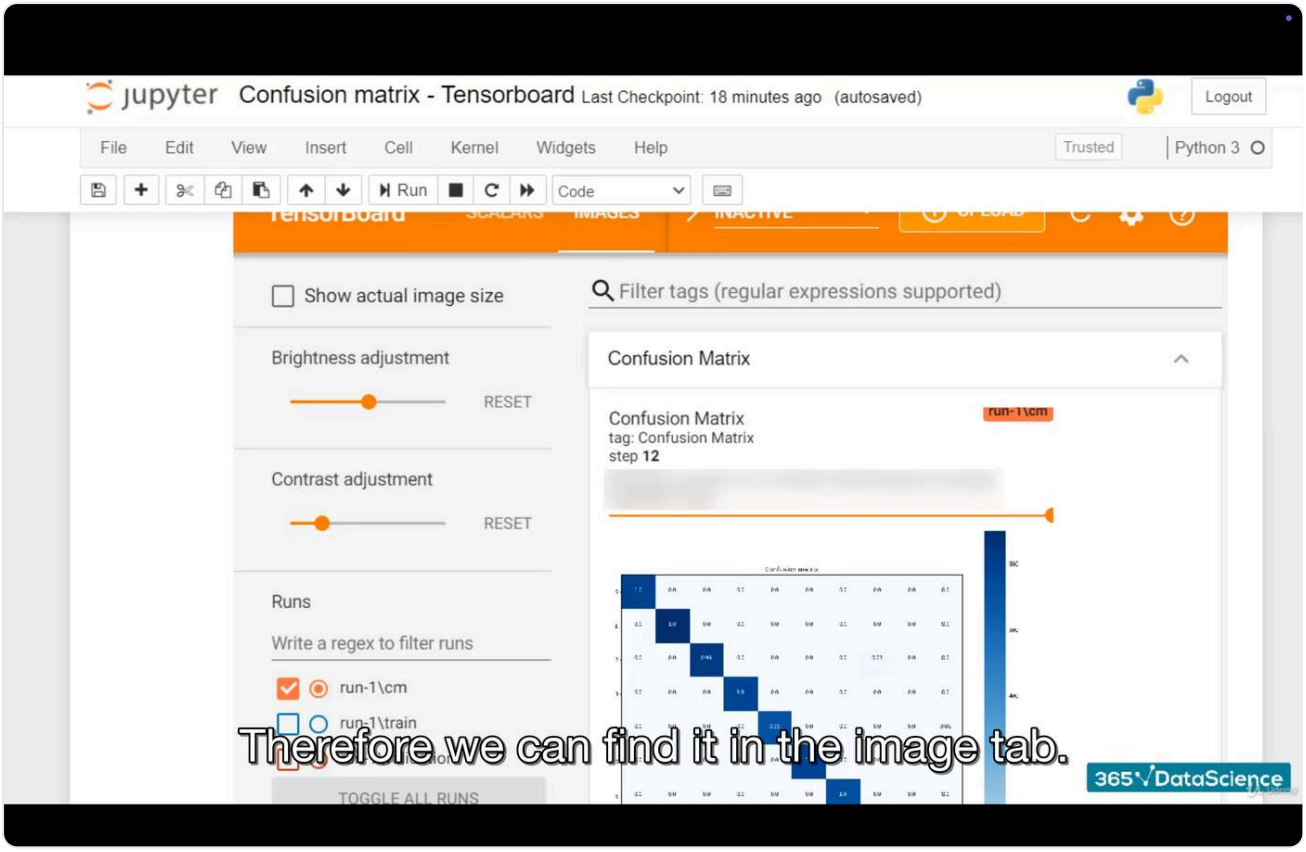

FIGURE 27.20

Therefore we can find it in the image tab. There it is. However, it is rather small. We can't actually see the numbers. In fact, the confusion matrix always shows up, scaled down like this. We can scale it up by just left clicking it like this. Alternatively, we can open it in a new tab to see it in its original size, great. What info can we extract from it now? Well, in the amnesty example, we can't really see anything useful here since our model has huge accuracy. The confusion matrix reduces to just the diagonal later in the course when we work with our own data. The confusion matrix will come in quite handy. That doesn't mean you shouldn't experiment with it.

In fact, for homework, I suggest you tune down the Amnesty Network on purpose so that it has lower accuracy and then check the confusion matrix. You can choose inadequate hyper parameters or just change the network architecture. The idea is to then examine the confusion, matrix and study which digits our models struggles with. All right, that's enough on this topic. Next time we will see how to tune the parameters of a network with the help of TensorBoard. Thanks for watching.

LECTURE 28

Filter Size and Hyper Parameters

Check the notebook `4.ipynb` for the code.

In our last lecture, we introduced the confusion matrix and implemented it with 10 support, this time we will do something a bit more advanced. Namely, we will try to tune some of the hyper parameters of our network using TensorBoard. Good, the integration of TensorBoard in this case is a bit trickier, however, we will not focus on the code too much because our purpose is to show you what's possible and give you a general idea of how you can achieve it. You can always dive deeper into the code on your own if you need to. OK, so again, we start with our MNIST code. This time we won't be logging our runs with the timestamp, so we don't need to import date time.

Nevertheless, we need to import a hyper parameter plugin from TensorBoard dot plugins. Dot hparams import API. Here, HP stands for hyper parameters, all right, again, the preprocessing section does not need to be changed, so we leave it as is. Now we need to define which type of parameters we will be testing. For this example, we will tune to hyper parameters. The first one is the kernel size in our convolutional layers, while the second one is the optimizer we train the network with. So starting with the filter size, we will define a variable called HP filter size, it is equal to HP dot h param. As parameters, we need to specify its name, which is filter size and the range in which we will test it, in our case, we will test it for a filter with size three by three, five by five and seven by seven.

So we write HP dot discrete three five seven. Next we have the optimizer. HP dot h param. Brackets, Optimizer, HP, dot discrete. Adam and SGD. In this test, we will try out these two optimizers, recall that SGD stands for stochastic gradient descent and it is one of the simplest optimizers available, whereas Adam is a much more sophisticated one. So we expect that Adam would be way better than SGD in practice. This may not be a very useful test to make. However, for educational purposes we will compare the two. So far, we have defined what type of parameters and values to test. Now we need to set the metric that will we be interested in when comparing them.

This would be the test accuracy. Thus we define metric accuracy equal to accuracy. Now we need to log some setup info, here is the code for that briefly, this relates to a file located in the HParam tuning subfolder of the logs folder. In this file, we state the hyper parameter variables and the accuracy metric. So far, so good. Moving forward, we now need to train the network multiple times with every different type of parameter value and test its accuracy. For this purpose, we will create a function that trains and tests our model. This is pretty much copy pasta of our existing code, just wrapping it into a function. However, there is a caveat. We will not hard code the hyper parameters, but instead we will pass them to the function.

So let's get going. I will name the function in question train test model. The parameter passed, the function will be called hparams, it will be a dictionary containing the different values. Let's paste the stuff we will need from our previous code inside the function. First is the model of our

network copy and paste. Then we need the loss function and the model dots compile line. Continuing, we have the early stopping. Finally, we have the training of the model with model dot fit and the evaluation of its accuracy. Now, let's make the necessary changes first, we will replace the kernel size inside the convolutional layers with each Prem's square brackets, HP filter size noticed that I do that for both convolutional layers.

Thus, in this test, both will have the same kernel size three, five or seven. The other parameter we need to change is the optimizer, which I will set to hparams square brackets HP Optimizer. Finally, we need to return the test accuracy of this model. Great hope everything so far was clear. We will now pick up the pace of it and explain the rest of the code briefly. Here we have a function that would initiate the training process and log all the data. It takes two parameters the directory where we will log the results and the hyper parameter values here. It opens a file to write in this line writes the current values of the hyper parameters being tested.

Then it invokes the training function with the appropriate hyper parameters, which returns the accuracy for this combination. Finally, we log this accuracy as well. All right, the final piece of the puzzle is to have a double for loop to loop over every combination of hyper parameters and invoke the function run for each one. So here is that piece of code. Here are the two for loops iterating over the respective hyper parameters. Then we define a dictionary with the current values and a name for the logging directory. These two print statements print out useful info to the user. Finally, we call the function run to train the network and log the results.

There you go. Now our code is complete. But before we run it, there is one last thing to mention time when having multiple training runs of our model, the time it takes to complete may increase exponentially. For example, in our case we have three different values for the filter size and two values for the optimizer. This means that to exhaust all possible combinations, we will need to train our models six times. If, let's say, a single model is trained for five minutes, that means this program will need to run for at least 30 minutes. You need to take this into account every time you said something like this up. OK, we will now run this cell, which will train our network with all six combinations of hyper parameters and jump straight to when the training is over.

Great. Now that all the computations are over, it's time to see the results. So let's start TensorBoard. We do that with the usual two commands, percent load extension, TensorBoard and percent TensorBoard. Dash dash log, Dyax here. You need to be careful to type the correct directory for the logs first. Don't forget, we have to enter the commands in the command, prompt the task, kill and delete one. Now we can safely start TensorBoard. So here is the visual interface. This time, the scalars tab is not of much use to us. What we want is the H programs tab standing for hyper parameters to the left. You can confirm which values of the hyper parameters to show.

Now this table view shows how each combination has performed. You can sort it with this section here to the left. Let's sorted by accuracy in descending order that is from best to worst. It is immediately apparent that there are two clusters, one with the optimizer and one with Adam, where the runs with Adam perform much better, just as expected. Let's now switch from table

view to parallel coordinates view, we can also expand the graph a bit, so it is clearly visible. This may seem a bit confusing at first, but it is actually pretty simple. On the left, we have the filter size, while on the right there are the optimizers. In the middle is the accuracy.

Each line goes through exactly one filter size and exactly one optimizer, so it represents one run of the network with the particular parameters. The place where it intersects the middle line is the accuracy of this configuration. The graph demonstrates that all lines from the optimizer go to the bottom of the accuracy while the ones from the atom cluster at the top. So it is obvious that Adam performs much better. On the other hand, the different kernel sizes coupled with Adam performs somewhat similarly. Thus our size argument in the beginning is relevant. This goes to show that with complex things, as CNN's often it is important to test out many different configurations.

Finally, there is the scatterplot matrix view, it consists of a series of graphs that plot accuracy against the filter size or against the optimizer with every run represented as a point. All right, so this was a somewhat brief introduction to tech support in this section, we covered the general idea of this extension and some of its more common uses. Remember, though, this is just the tip of the iceberg. Feel free to take a look at the documentation and official guides and to experiment with it on your own networks. So far, we have shown you how to use CNN to solve one of the most basic problems in computer vision, the finest example. Now I feel we are closing on a more serious, practical project.

That's why in the next section we will introduce a couple of techniques for improving the performance of neural networks in general. See you there. And thanks for watching.

LECTURE 29

Data Set and MNIST Data

So far, we have considered what convolutional layers are, how they work and how they are incorporated into the structure of a neural network to obtain a convolutional neural network. Up to that point, it was all theoretical. Then in order to show it in practice, we implemented a somewhat simple CNN on the minus data set. Given that the MNIST data set is a very clean and simple one, it seemed like a good starting point to show how the theoretical stuff are coded in practice. However, the advantages that make the remnants a good foundation turn into disadvantages when it comes to learning how to construct models in practice and what to expect from them.

You see, the MNIST data set is way too clean. First, the digits are always drawn in the center of the frame. Second, all images are black and white.

FIGURE 29.1

All of these make it extremely easy for most networks to deal with the amnesty. Even a simple linear classifier can achieve accuracy of more than ninety two point four percent.

FIGURE 29.2

Thus, this was not a challenge worthy of the usage of complex neural nets. Nonetheless, we still achieved our goal to learn the appropriate syntax for implementing science. But now there is another task on our list.

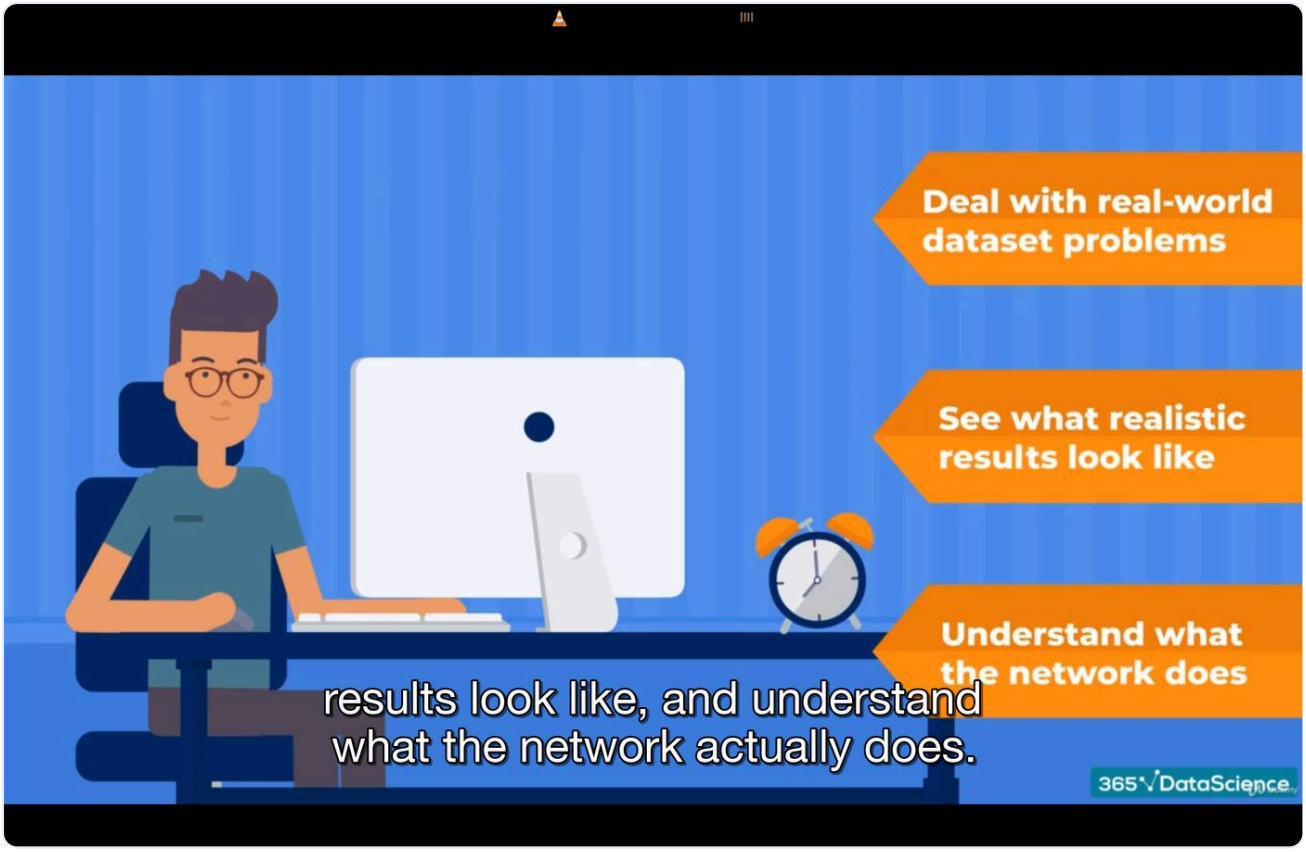

FIGURE 29.3

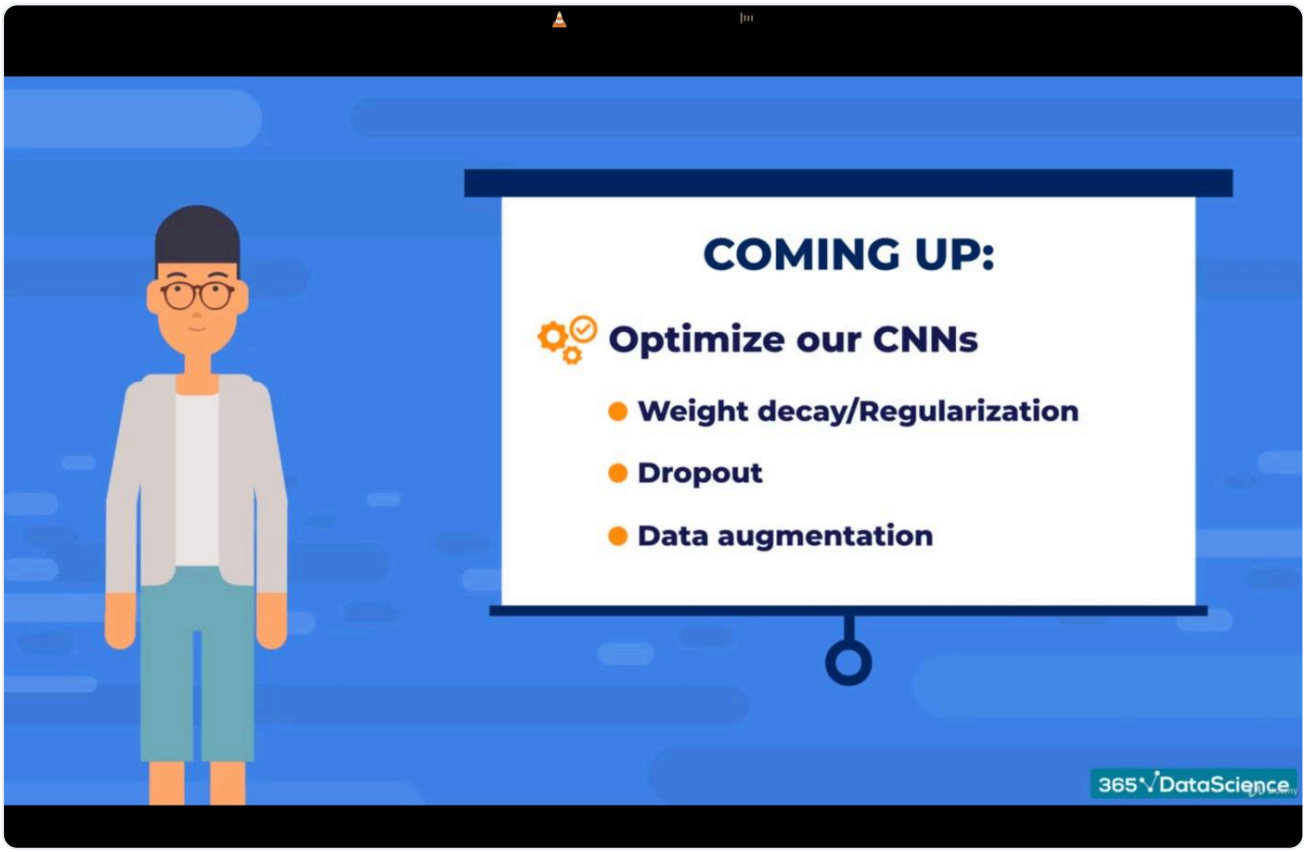

FIGURE 29.4

It's time to learn how to deal with problems that arise from real world datasets, see what realistic results look like, and understand what the network actually does. To tackle this, we have prepared a very real data set with unique goals and challenges, but we'll have to wait until next section to work on it. Why in the next section? Why not start with it immediately? Well, the data said we will consider is rather complicated. So first we will try to optimize our seasons to better suit the problem. That said, there are a number of techniques that can help us optimize a model. And in this section we will explore three of the most common ones that improve the performance and predictability of neural networks, no matter if CNN or not.

These are weight decay or Regularisation Dropout and Data Augmentation. If you are already familiar with these and know the specifics, just skip this section and jump straight to the practical example. All right. With this out of the way, let's start off with the first technique, regularization. Thanks for watching.

LECTURE 30

Inhibit the Capacity

In this lecture, we will talk about regularisation, we will discuss its purpose and how it works. OK.

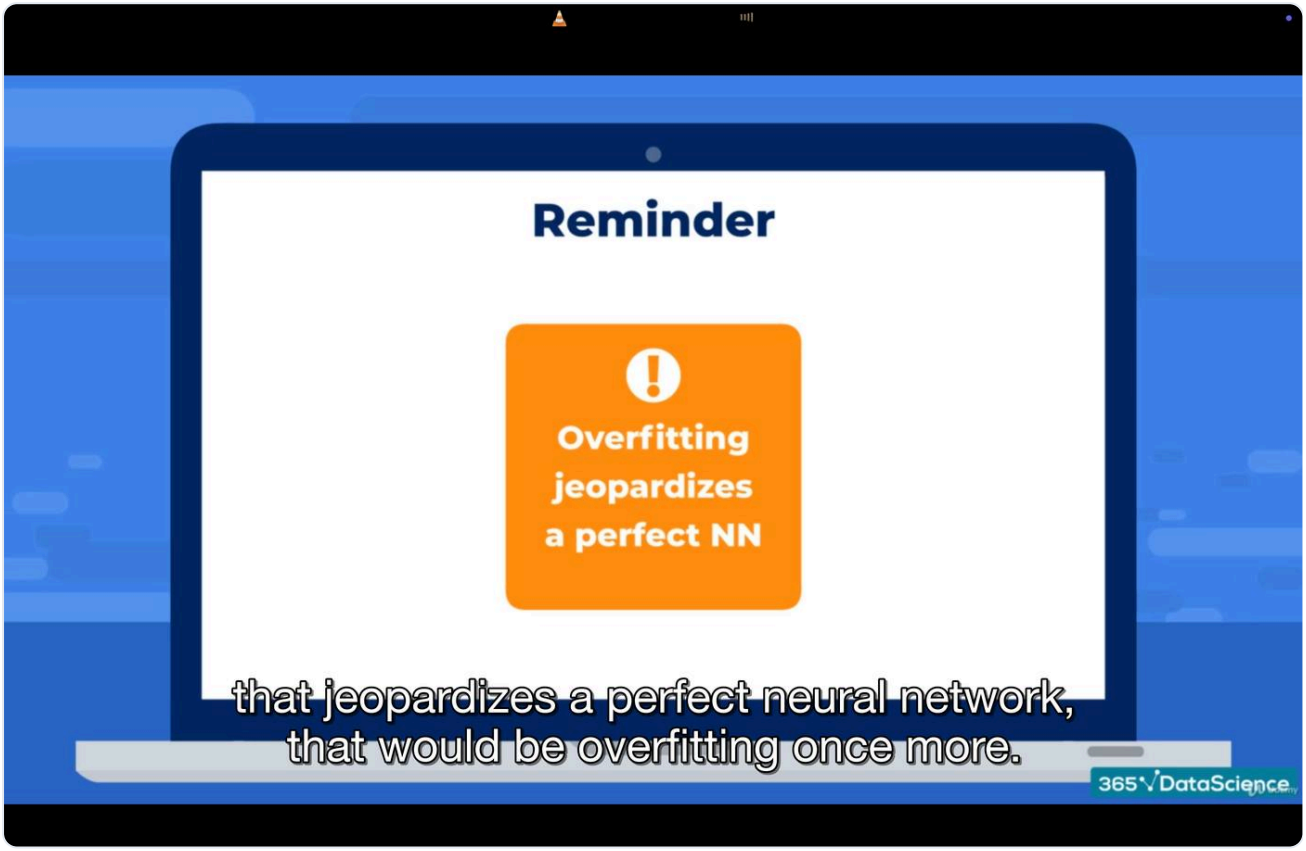

FIGURE 30.1

At this point, you probably know it by heart, but we still need to remind you, if there was one thing that jeopardizes a perfect neural network, that would be overfitting once more.

FIGURE 30.2

Overfitting refers to situations where the model has fit the training data so well that the model captures the noise and random fluctuations.

FIGURE 30.3 | Early Stopping

Now, we have already discussed how to use the validation, said an early stopping in order to prevent OVERFITTING from happening.

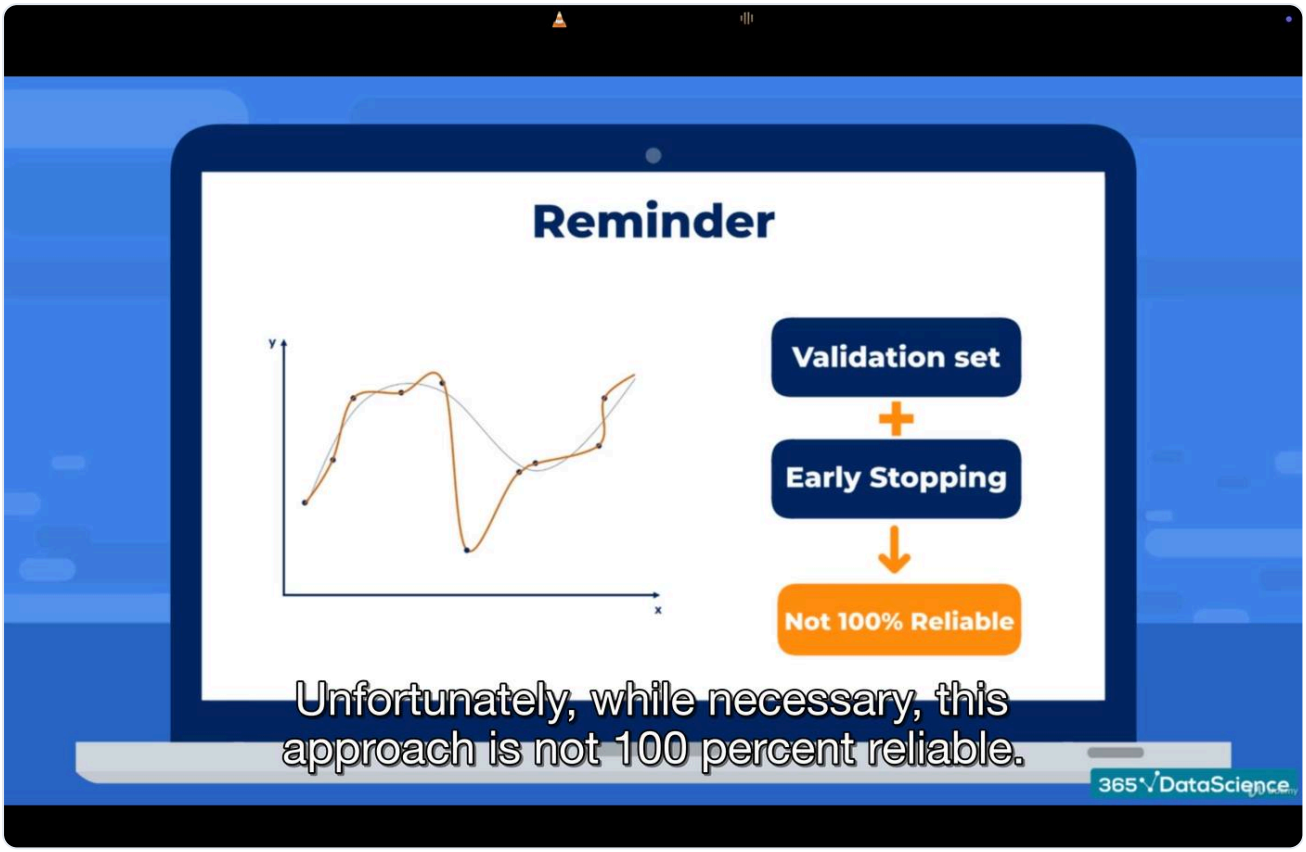

FIGURE 30.4 | Validation Set

Unfortunately, while necessary, this approach is not 100 percent reliable. There may be certain situations where the validation loss stays the same or increases marginally, but the model is still in the process of obtaining valuable insights from the training set.

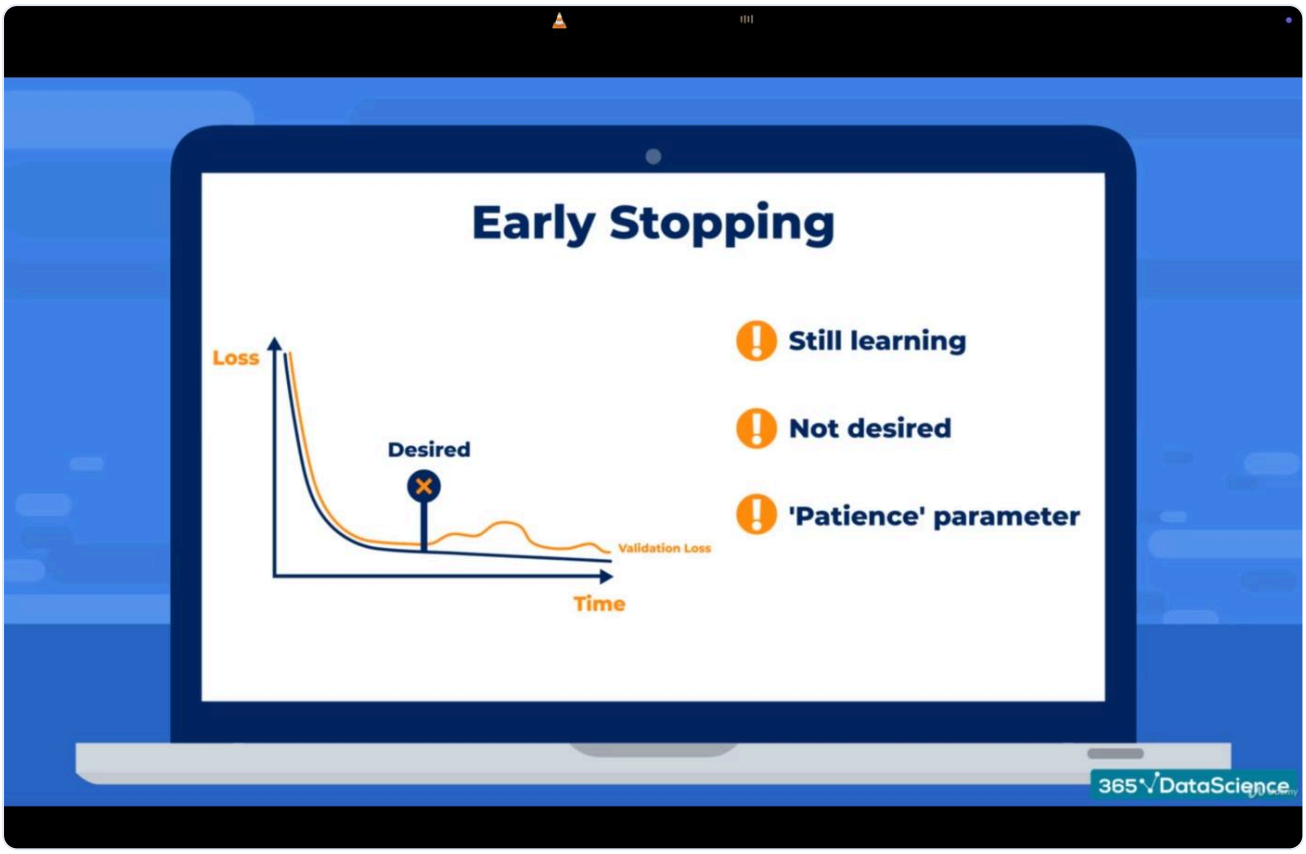

FIGURE 30.5 | Early Stopping

Therefore, we don't want the early stopping to kick in at this moment.

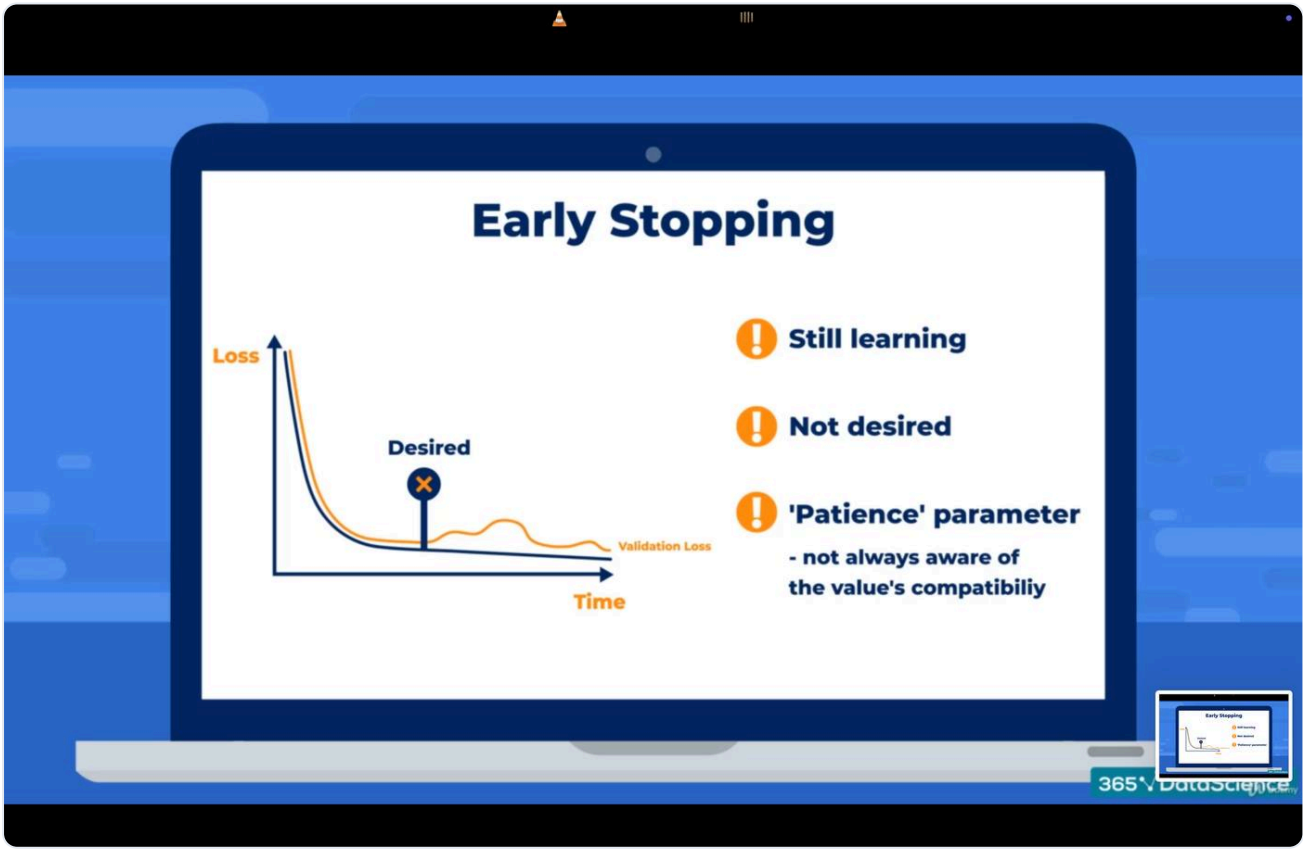

FIGURE 30.6 | Early Stopping

In theory, that can be countered by fiddling with the patient's parameter of the early stopping callback. Unfortunately, we are not always aware what the value should be or whether a particular value is any good.

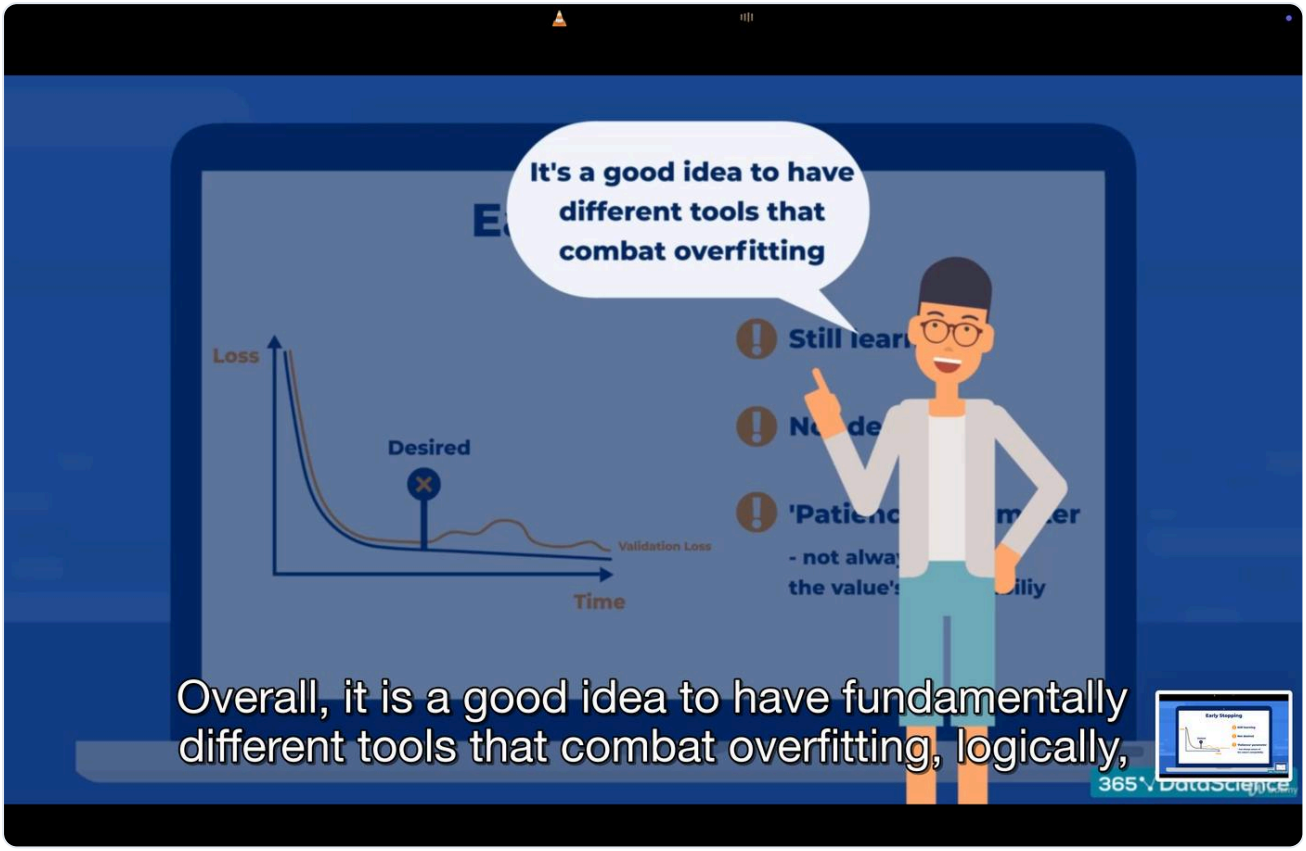

FIGURE 30.7

Overall, it is a good idea to have fundamentally different tools that combat overfitting, logically, different tools are useful in different situations and nothing stops us from employing them all. Good now. In what other way can we prevent Overfitting? Actually, there's a very clever answer to this question. Since Overfitting happens, when the model learns too much, we can combat this by interfering with the model's ability to learn. In other words, we decrease the model's capacity. At first, this seems like a counterproductive measure. After all, the whole purpose of machine learning is to make the machine learn.

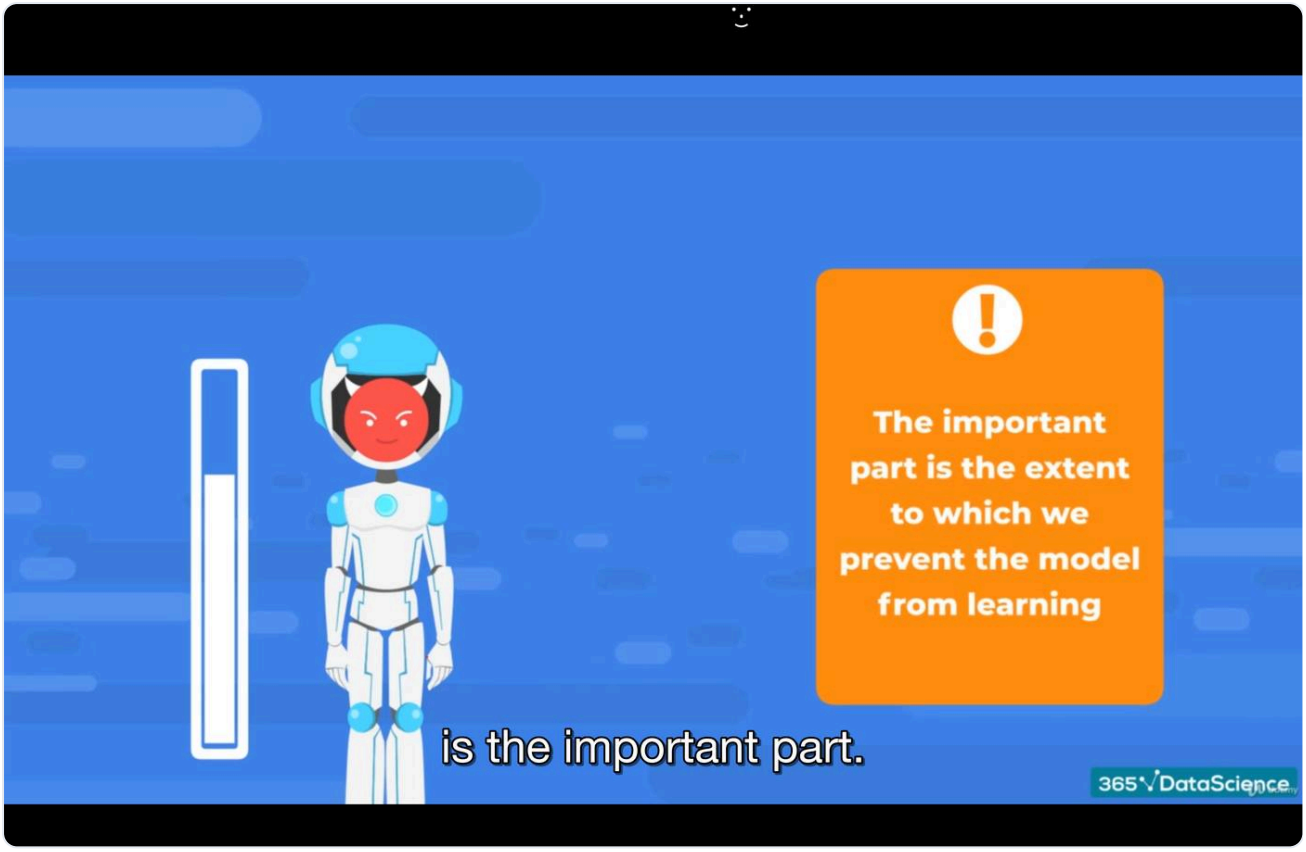

FIGURE 30.8

But of course, the devil is in the detail, and the extent to which we prevent the model from learning is the important part. All right, so how do we decrease the capacity of the model to learn there are various ways to achieve? That's probably the simplest one, is to make the model itself simpler.

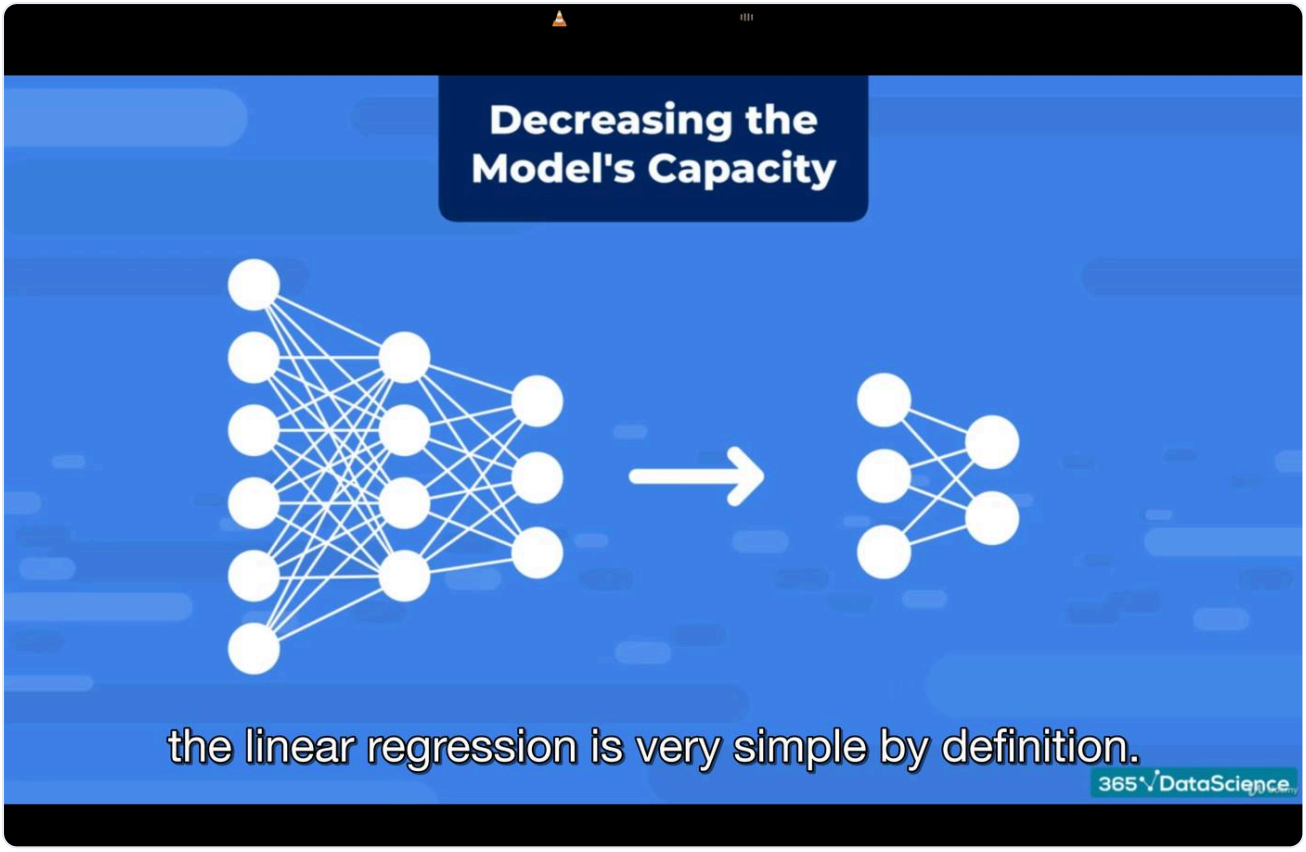

FIGURE 30.9

In fact, if you are employing a linear regression, no one will ever mention overfitting to you as the linear regression is very simple by definition. Now, downscaling the complexity to a linear regression is a bit drastic, isn't it? There must be some middle ground, right? Yes, and that's what regularization is all about.

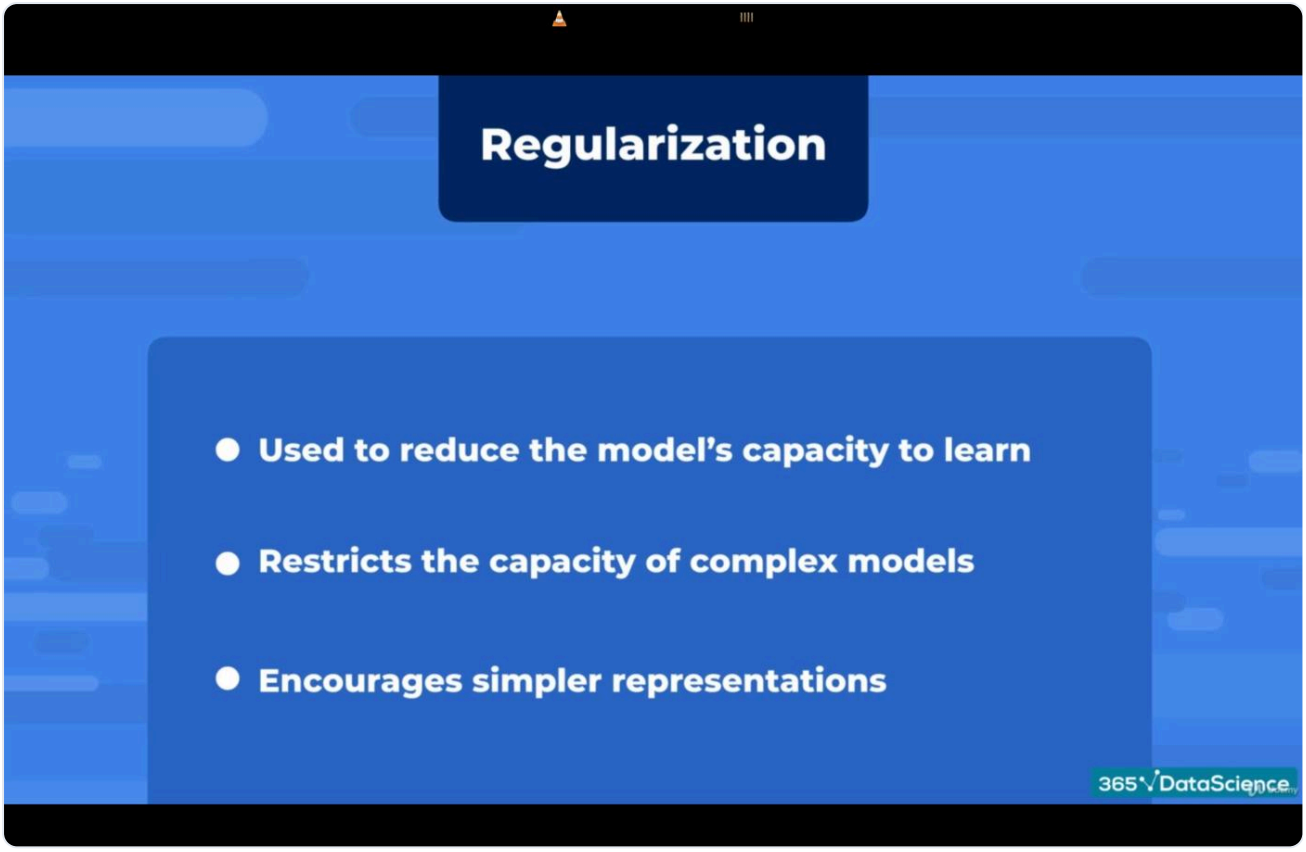

FIGURE 30.10

In a nutshell, regularization is a method used to reduce the model's capacity to learn more precisely, it restricts the capacity of complex models and encourages simpler representations of the solution. So far, so good. Let's get into the nitty gritty details.

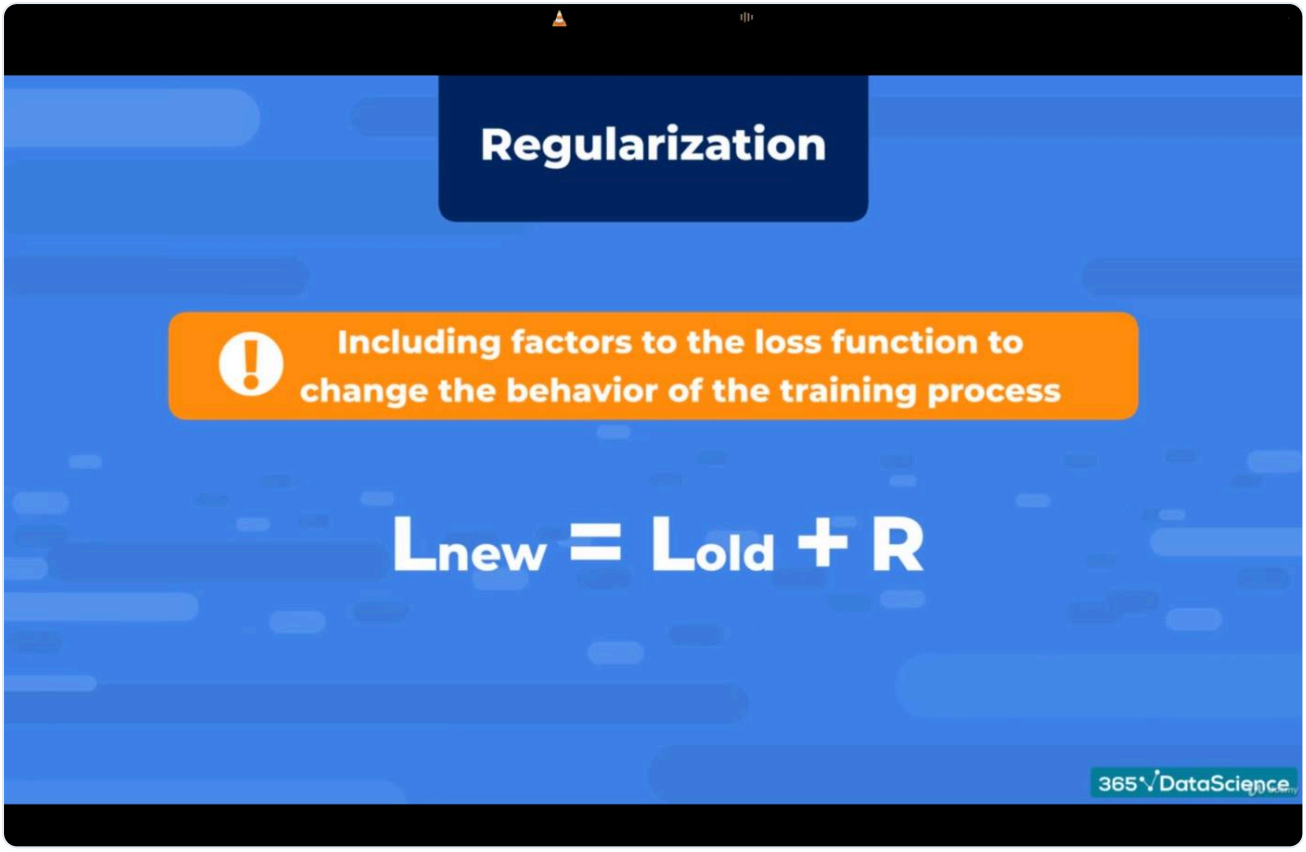

FIGURE 30.11

Technically speaking, regularization consists of including additional factors to the loss function to change the behavior of the training process. So the new loss is equal to the old loss, plus the regularization factor. And since the back propagation is solely based on the loss, changing the loss affects the optimization of weights.

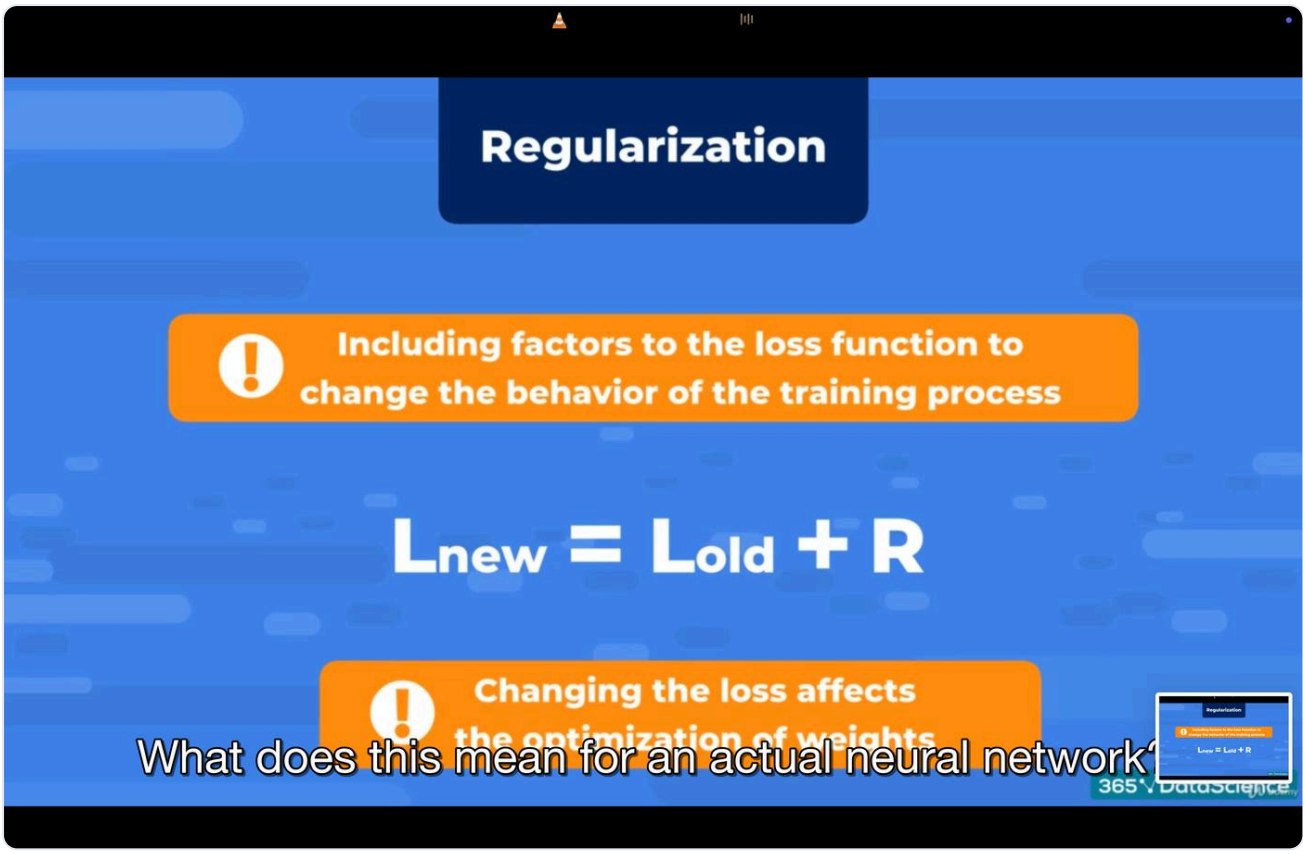

FIGURE 30.12

What does this mean for an actual neural network? Well, we can incentivize or penalize behavior. For instance, we can increase the loss when some unfavorable event occurs.

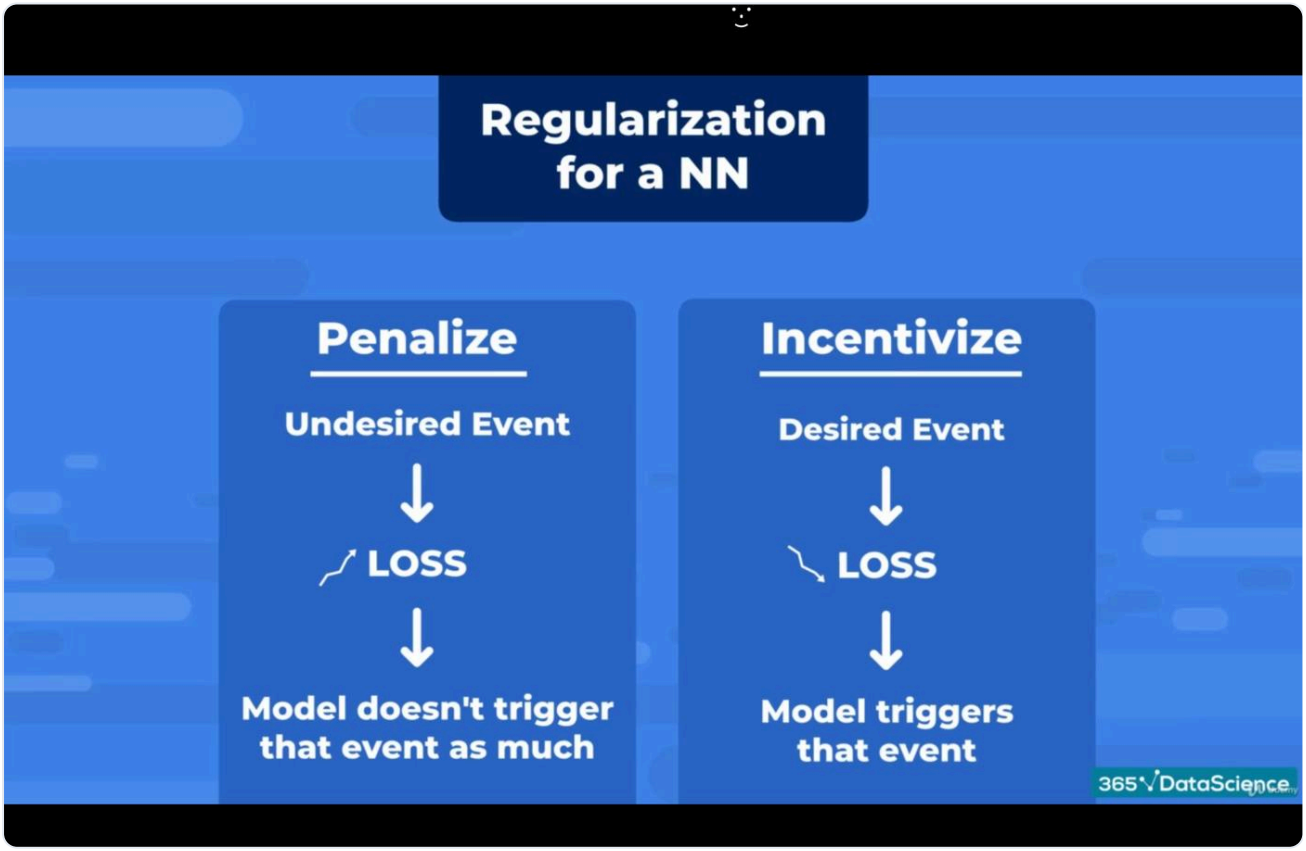

FIGURE 30.13

Subsequently, the model will adjust itself to not trigger that event as much. Inversely, we can decrease the loss in a situation where we want to encourage the model to follow the desired pattern. Great, going back to our goal.

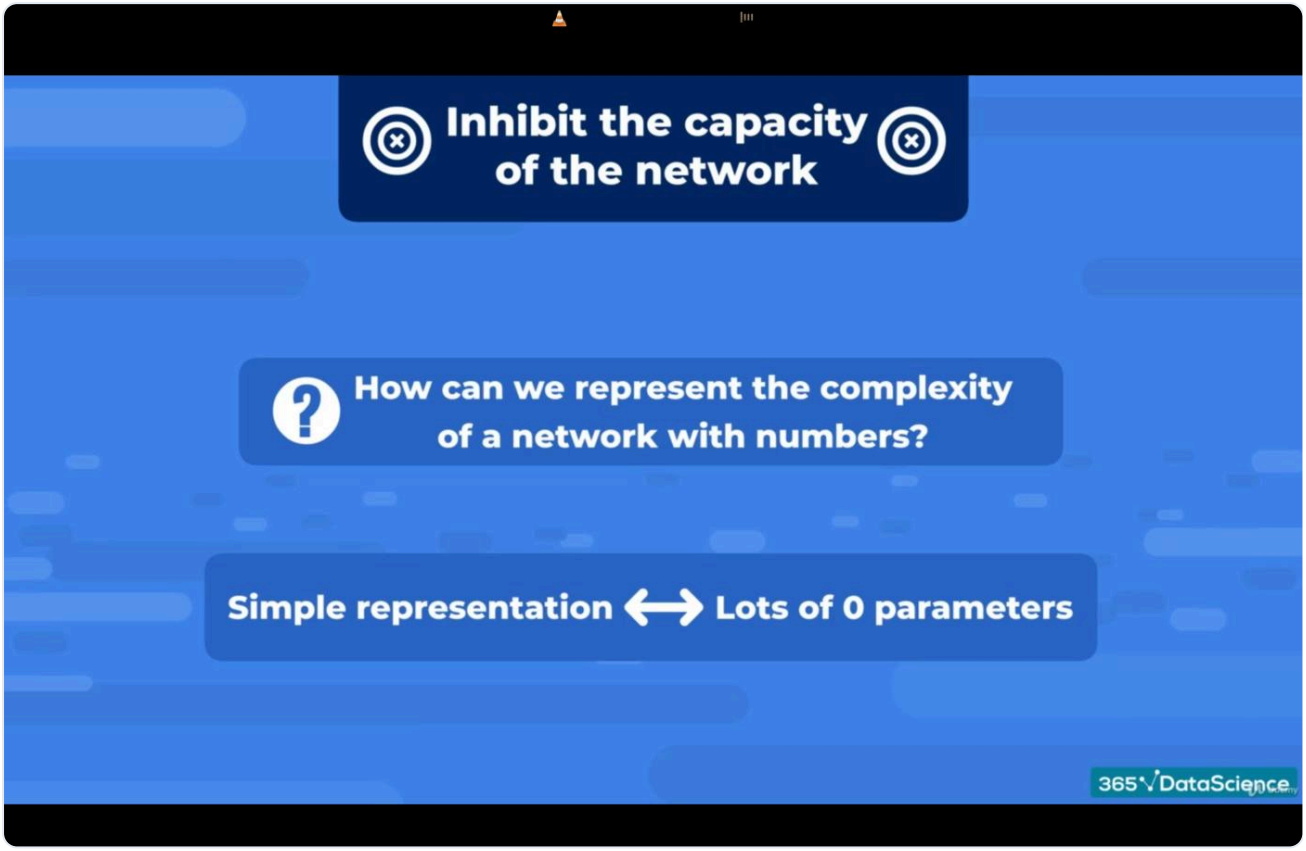

FIGURE 30.14 | Inhibit the Capacity

We want to inhibit the capacity of the network, so how can we represent the complexity of a network with numbers that can be added to the loss function? Well, intuitively, if a model has learned a very simple representation of the solution, we would expect a significant portion of the parameters to be zero or close to zero. We can convince ourselves of the validity of this statement by looking at what actually happens if a weight is set to zero.

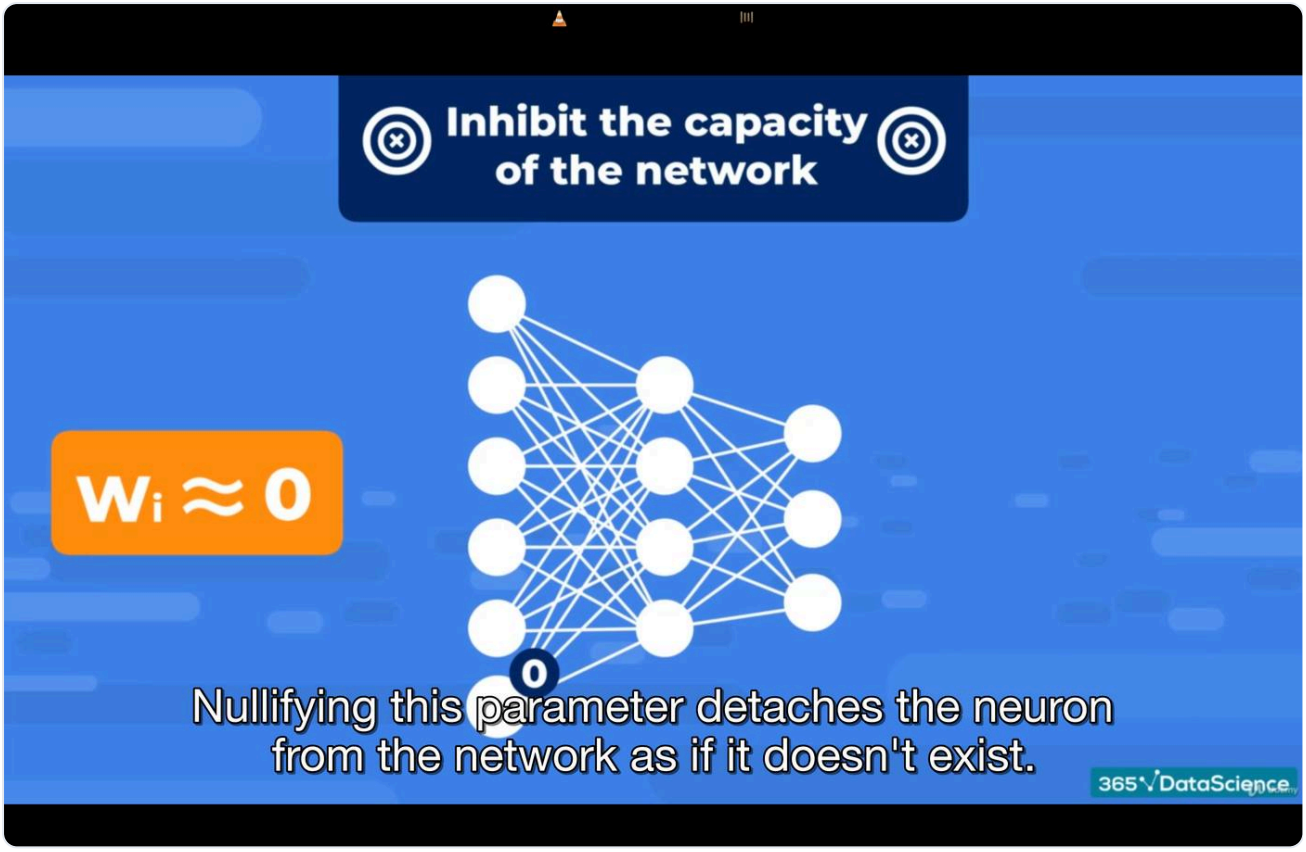

FIGURE 30.15 | Inhibit the Capacity

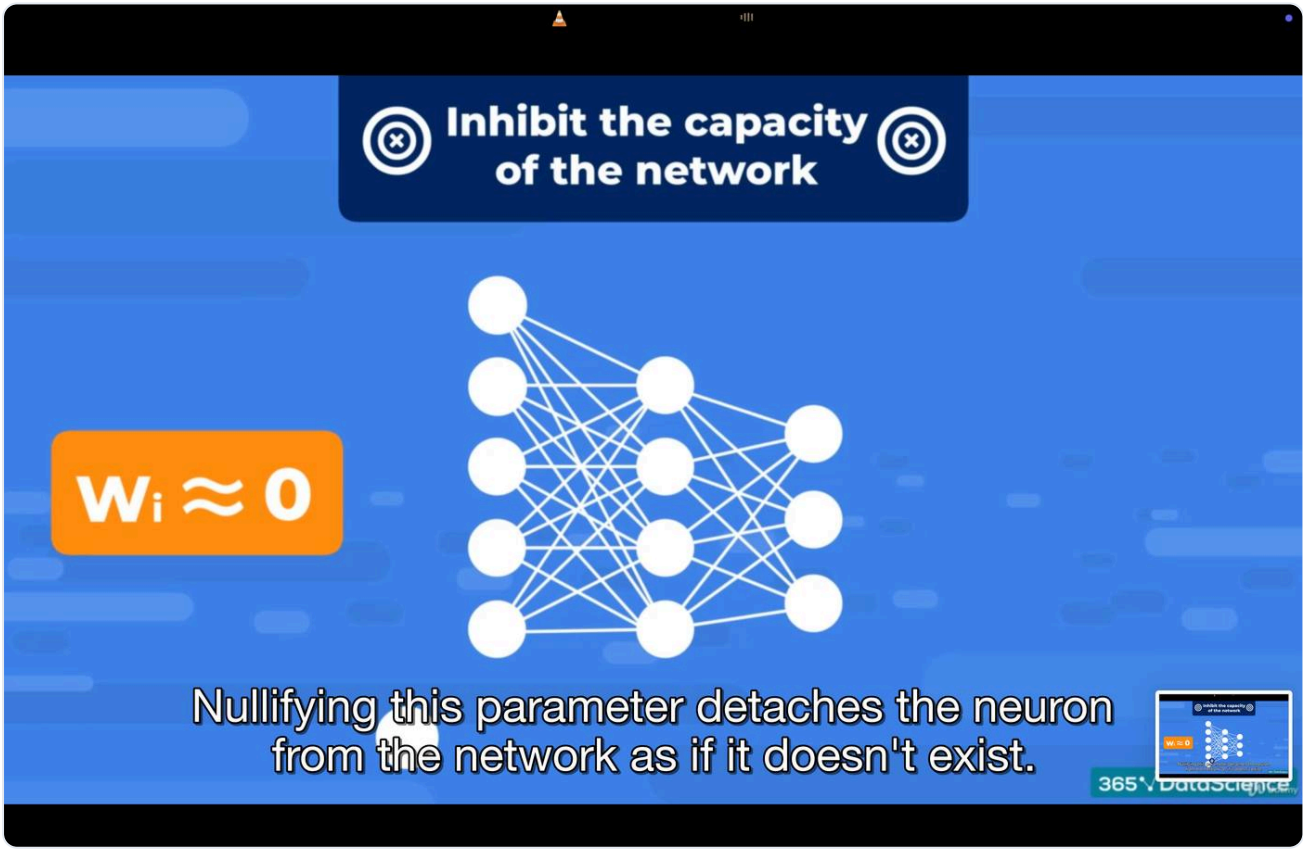

FIGURE 30.16 | Inhibit the Capacity

Nullifying this parameter detaches the neuron from the network as if it doesn't exist.

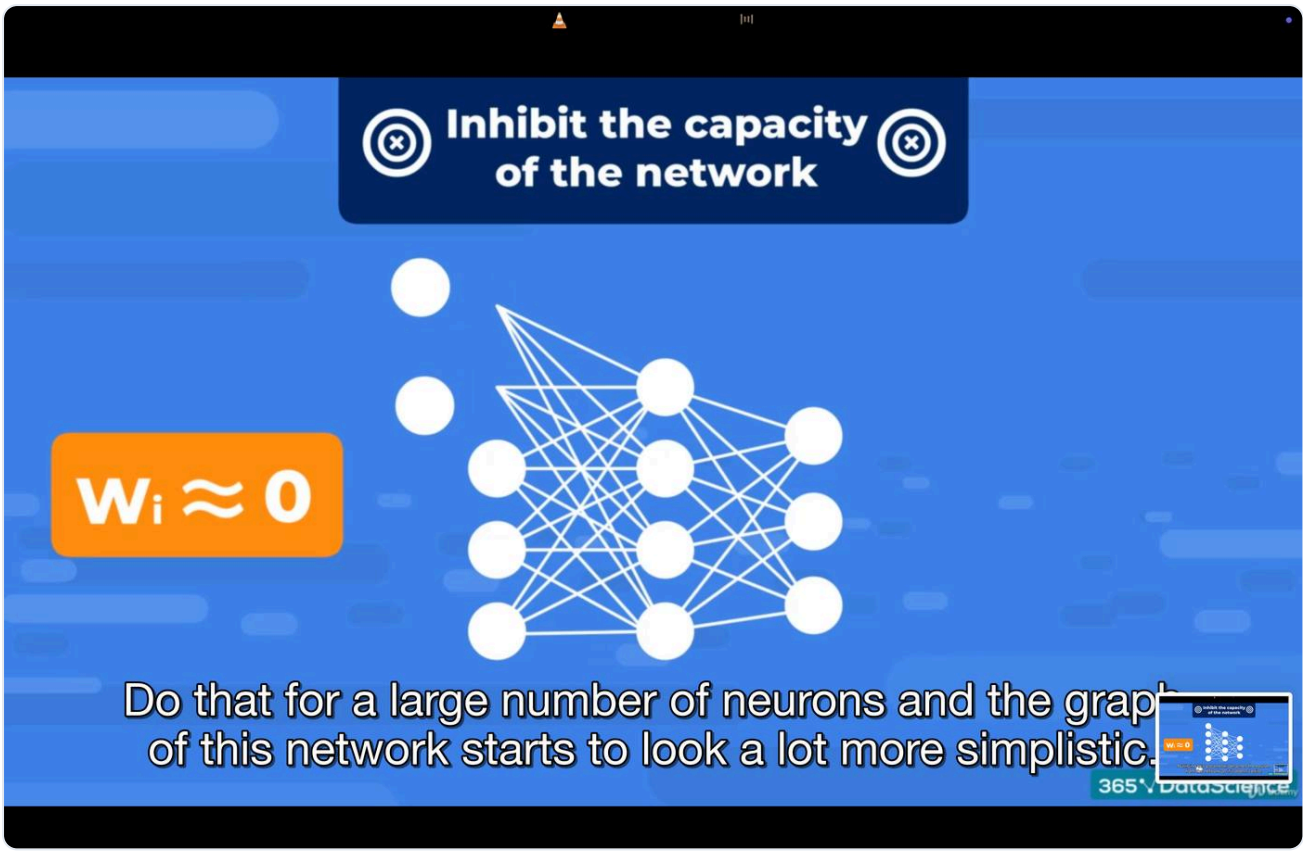

FIGURE 30.17 | Inhibit the Capacity

Do that for a large number of neurons and the graph of this network starts to look a lot more simplistic.

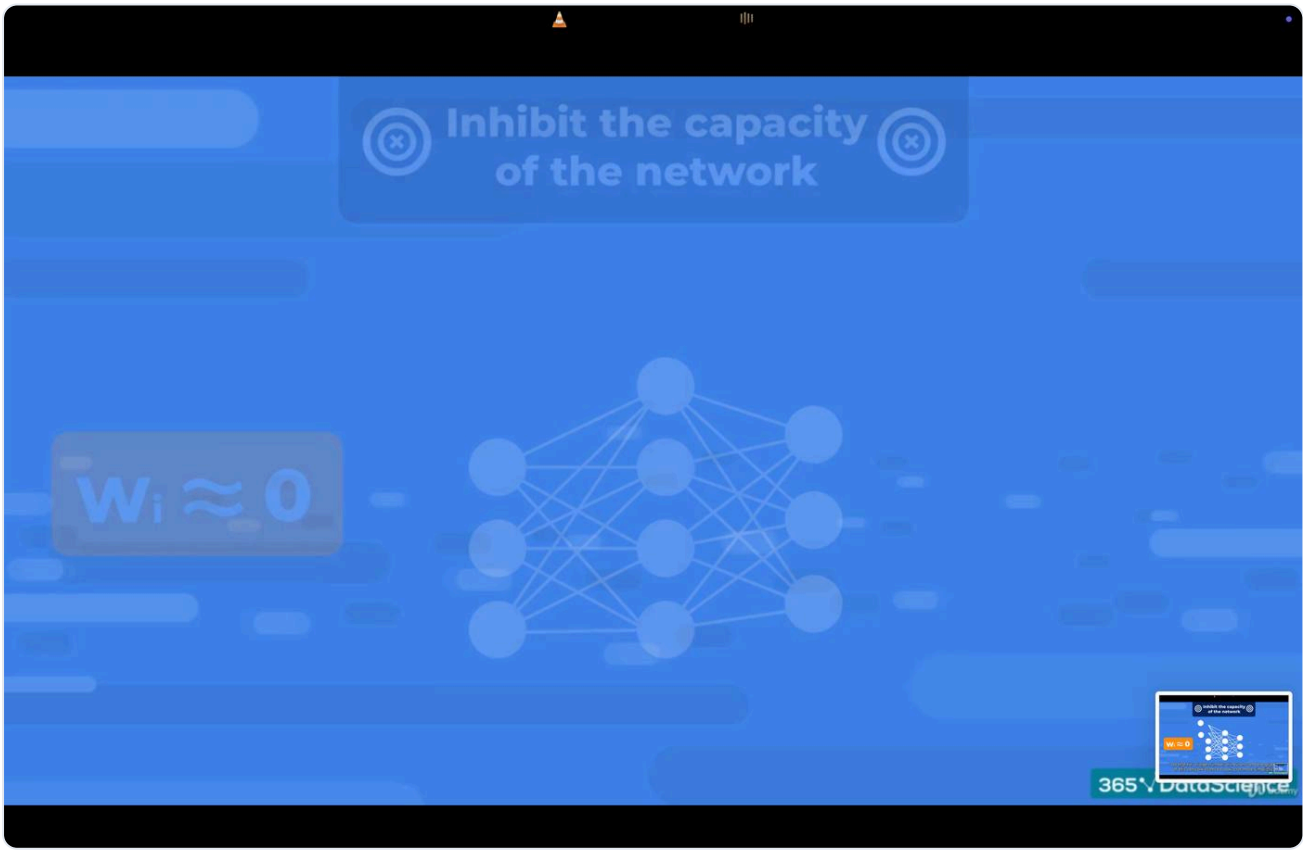

FIGURE 30.18

Conversely, if the model is complex, most, if not all of the weights would be non-zero numbers.

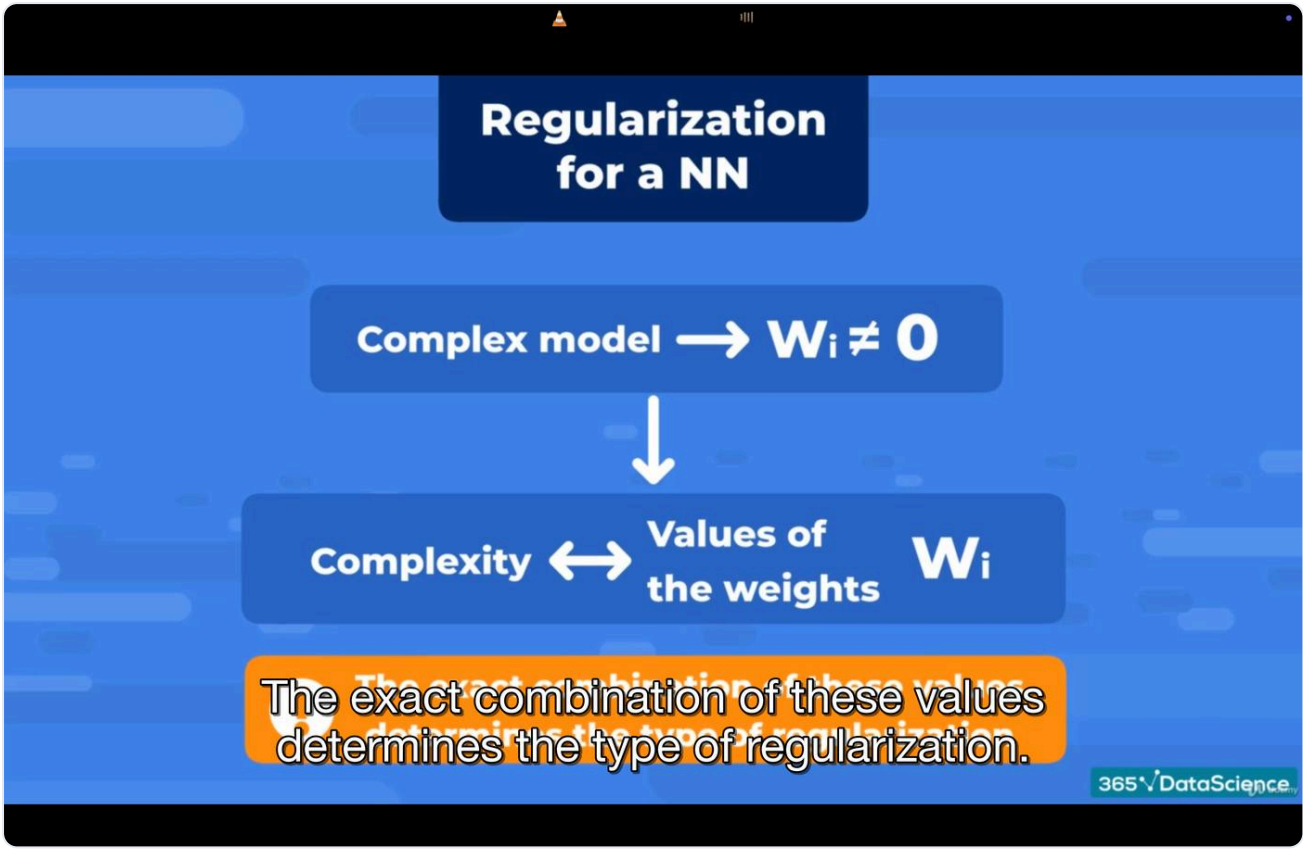

FIGURE 30.19

Therefore, we can conclude that the complexity or capacity of our model can be described by the values of the weights.

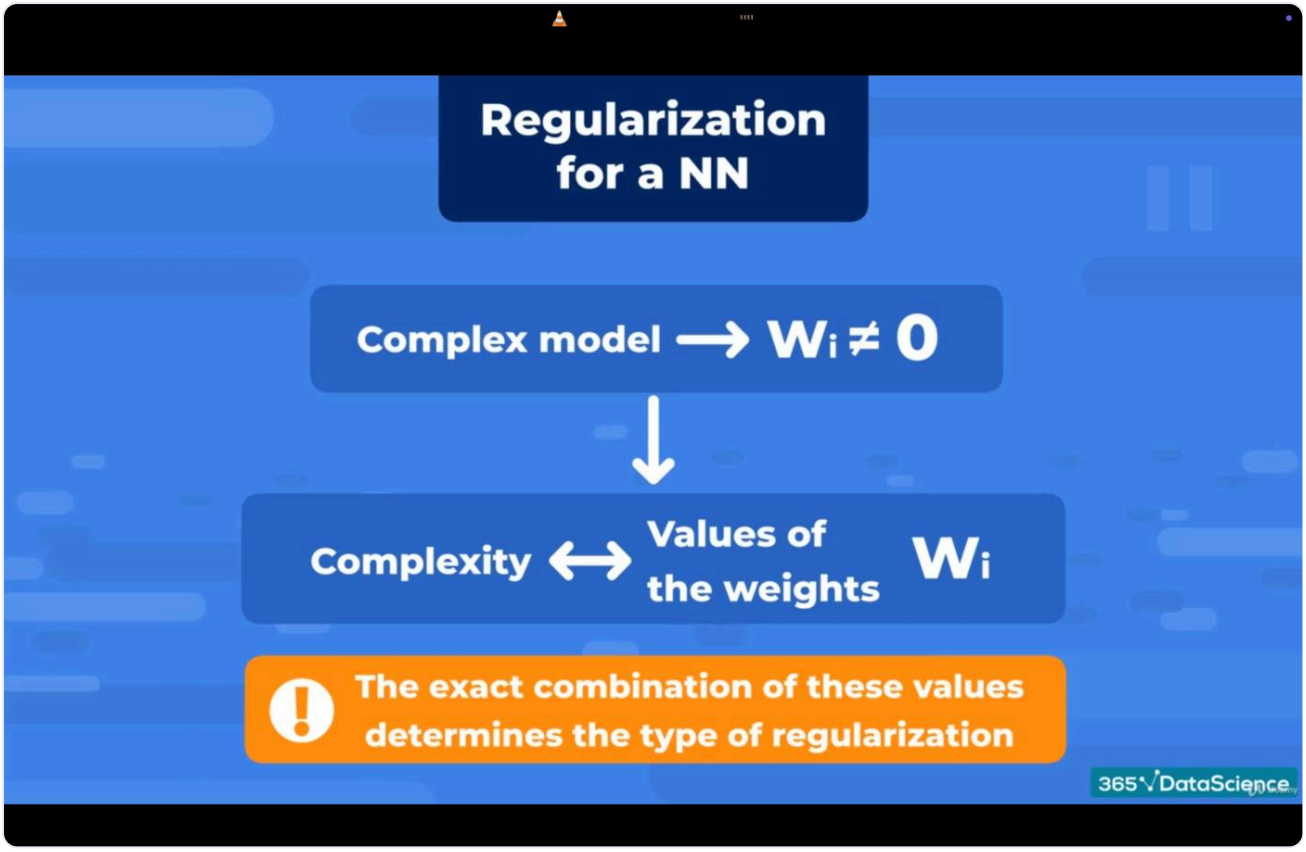

FIGURE 30.20

The exact combination of these values determines the type of regularization. OK, now that we know what regularization is all about, let's get more concrete. In the next lecture, we will discuss L2 regularisation and its similarities and differences with weight decay. Thanks for watching.

LECTURE 31

L2 Regularization Weight Decay

Continuing from where we left off in this lecture, we will talk about the two to Regularisation technique, discuss how it connects to weight decay.

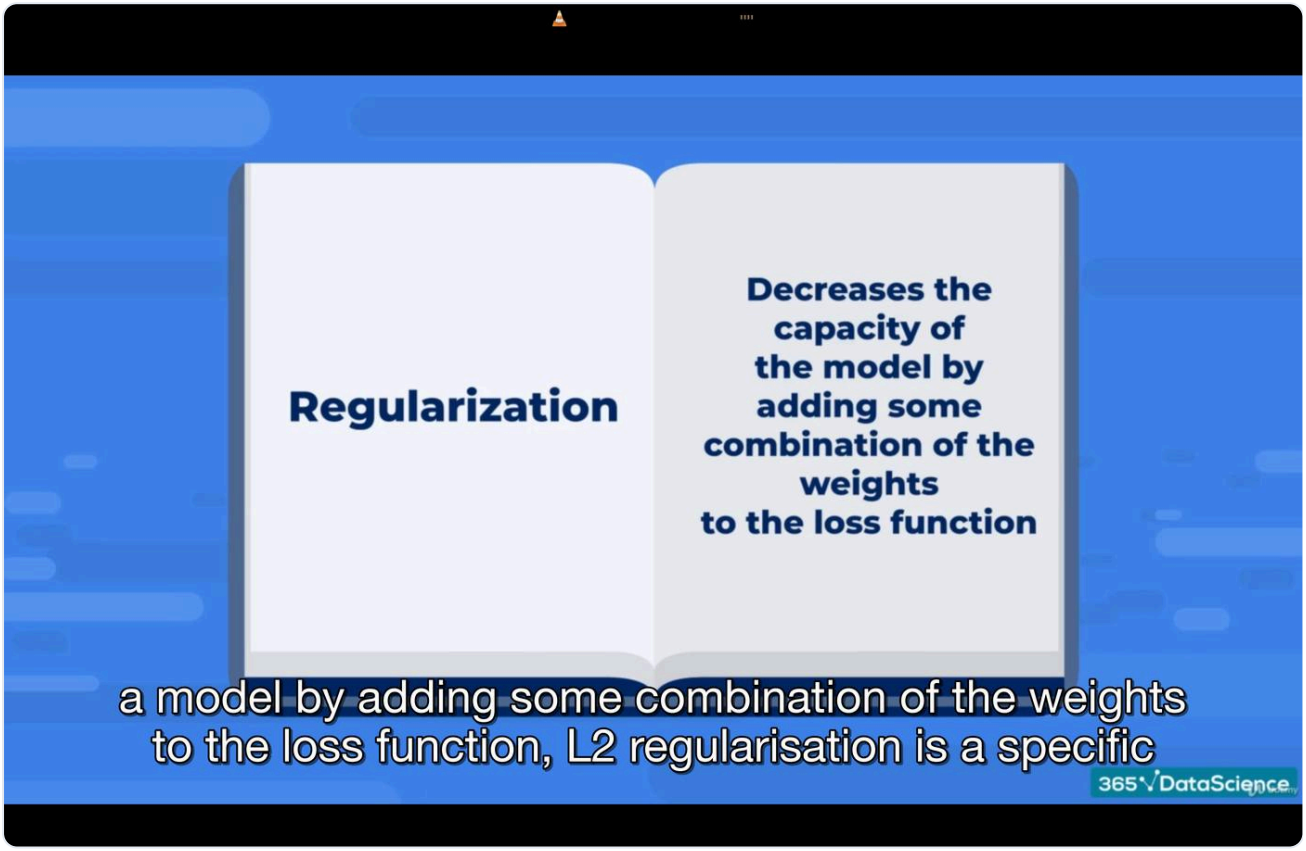

FIGURE 31.1

OK, last time we came to the conclusion that a regularisation technique can decrease the capacity of a model by adding some combination of the weights to the loss function, L2 regularisation is a specific type of regularization where the specific function we add is the L2 mathematical norm. L2 norm is the sum of all weights squared modulated by a parameter lambda.

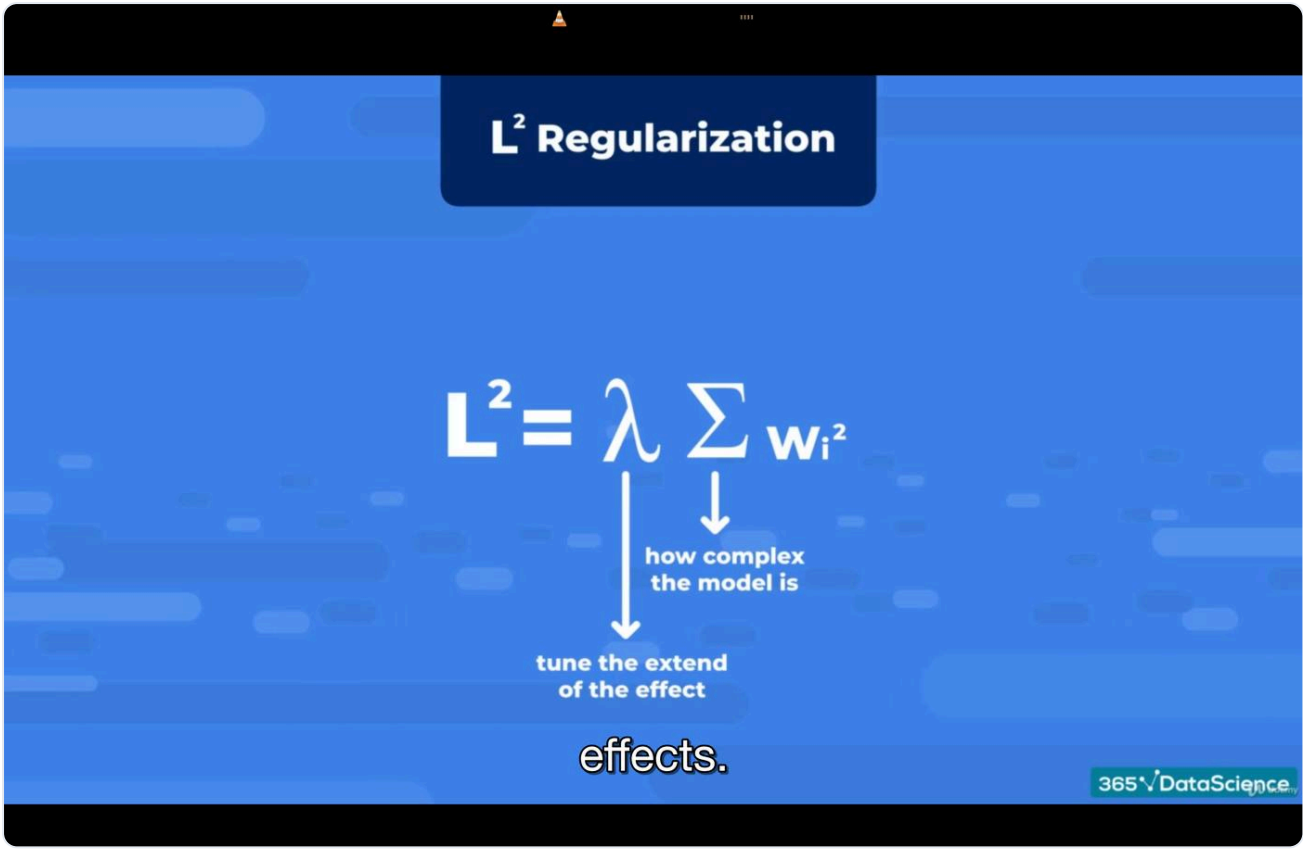

FIGURE 31.2

The sum represents how complex the model is, while the factor in front lambda is there to tune the effects.

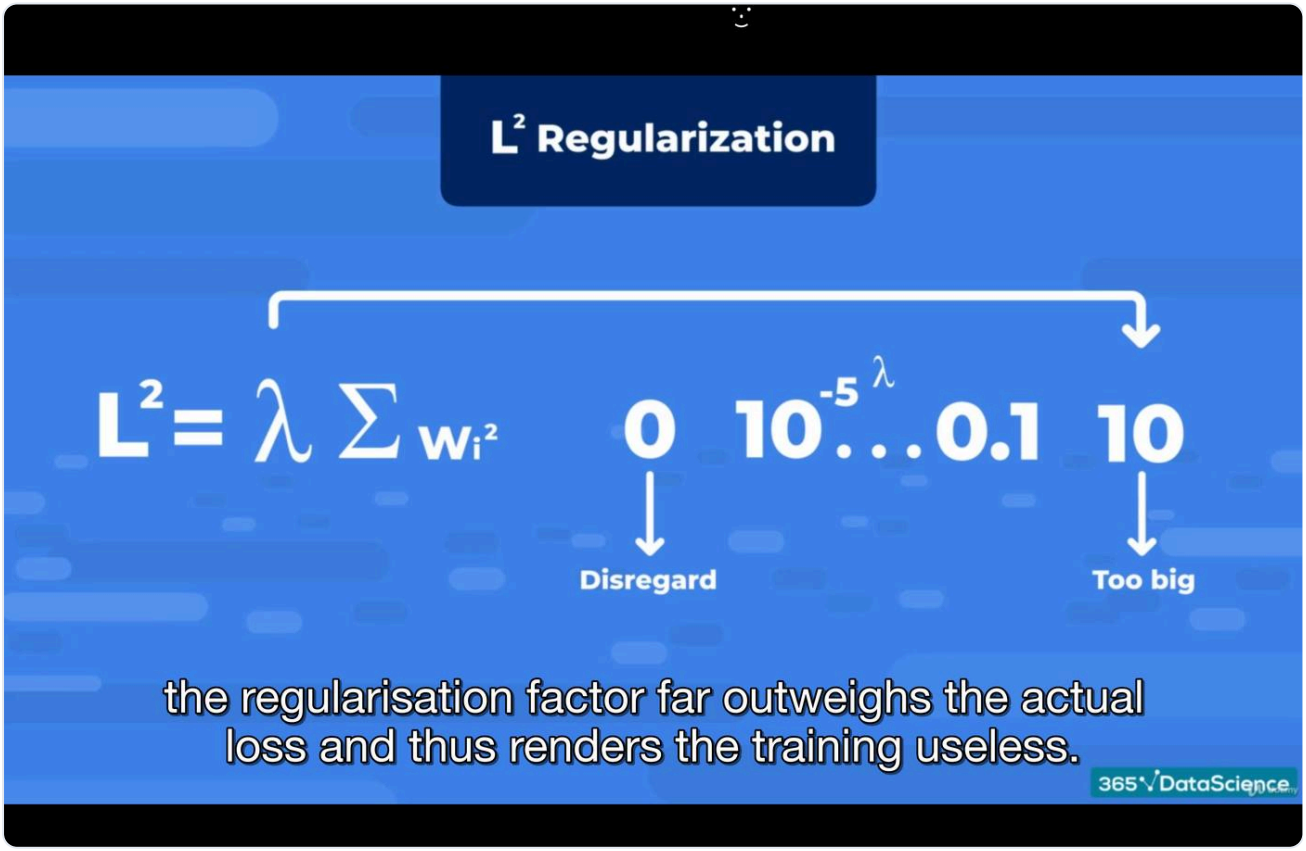

FIGURE 31.3

Lambda of zero implies the complete disregard the L to regularisation, while a big value may mean that the regularisation factor far outweighs the actual loss and thus renders the training useless. This makes Lambda another hyper parameter to consider.

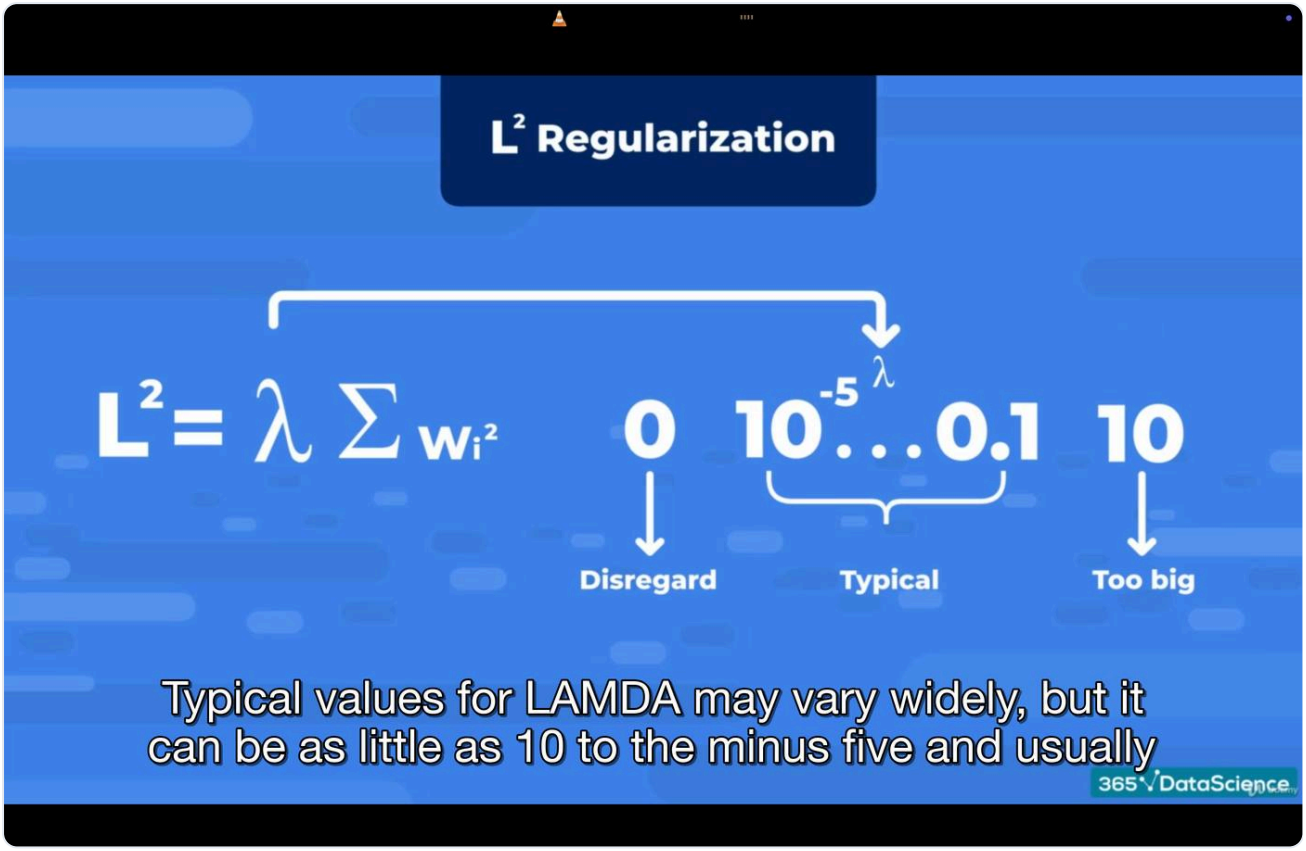

FIGURE 31.4 | TWolaally S for LAMBDA May Vary Widely, Outiit

Typical values for Lambda may vary widely, but it can be as little as 10 to the minus five and usually doesn't go above zero point one. Great, the interaction between the normal on regularise lost and the additional factor has interesting effects on the optimization of the weights. What actually happens is that the weights that are not absolutely necessary for the model are scaled down as close to zero as possible. On the other hand, the ones that are essential for the performance of the model are barely changed. Thus, the purpose of L2 regularisation is to simplify the model by scaling down all non-essential weights.

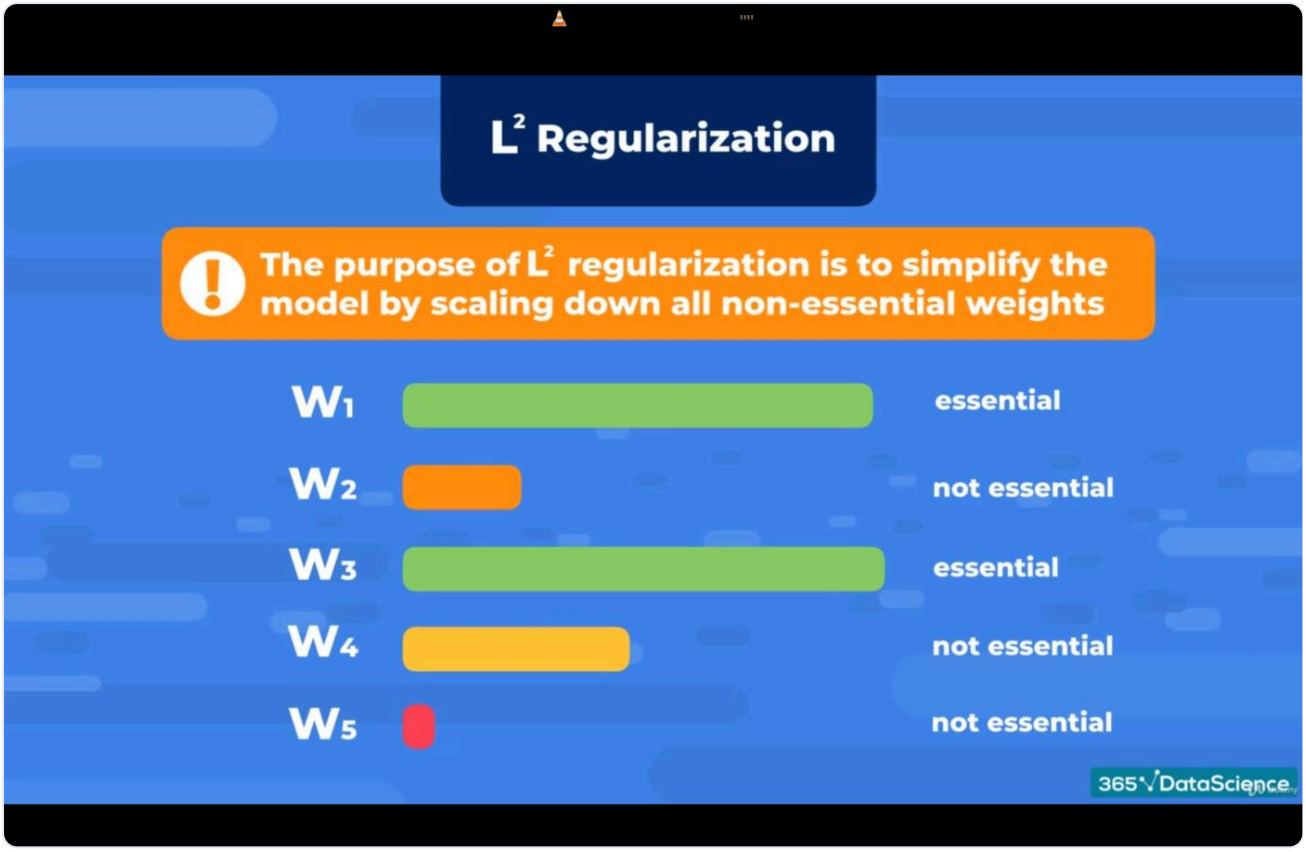

FIGURE 31.5

All right, so how does the update rule for the weights change? Because of the additional factor? Here's the original update rule when no factors are present in the loss. This signifies how the new weight should be obtained from the old one and the derivative. With respect to the loss here, BTA is the learning rate. What about the L2 regularisation case?

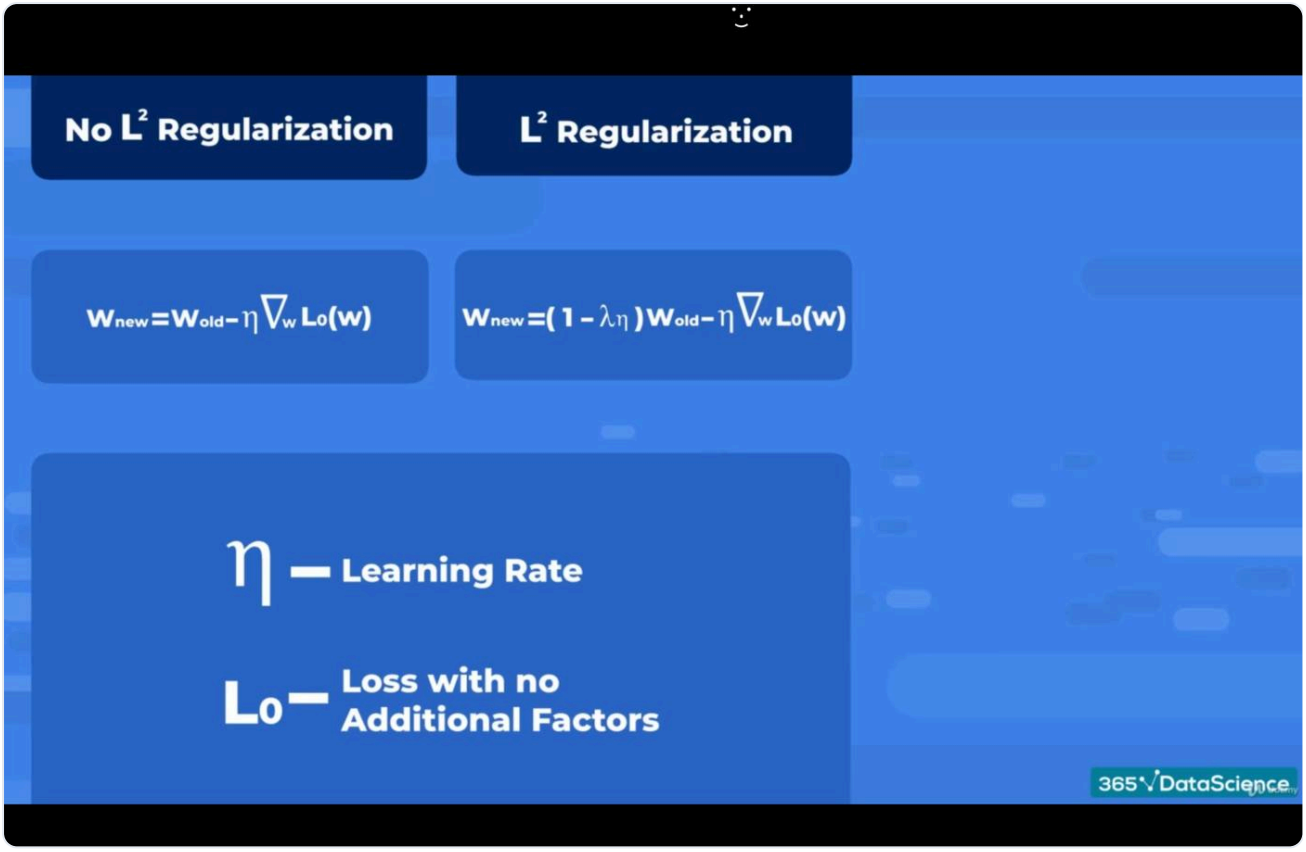

FIGURE 31.6 | No L2 Regularization L2 Regularization

Well, if you do the math, the update rule evolves into this one. We should know that the loss element here refers to the UN regularised loss. As you can see, the only difference is this extra factor of Lambda times theta. So effectively, L2 regularization is decreasing the contribution from the old weights. That is why L2 regularization is sometimes called weight decay.

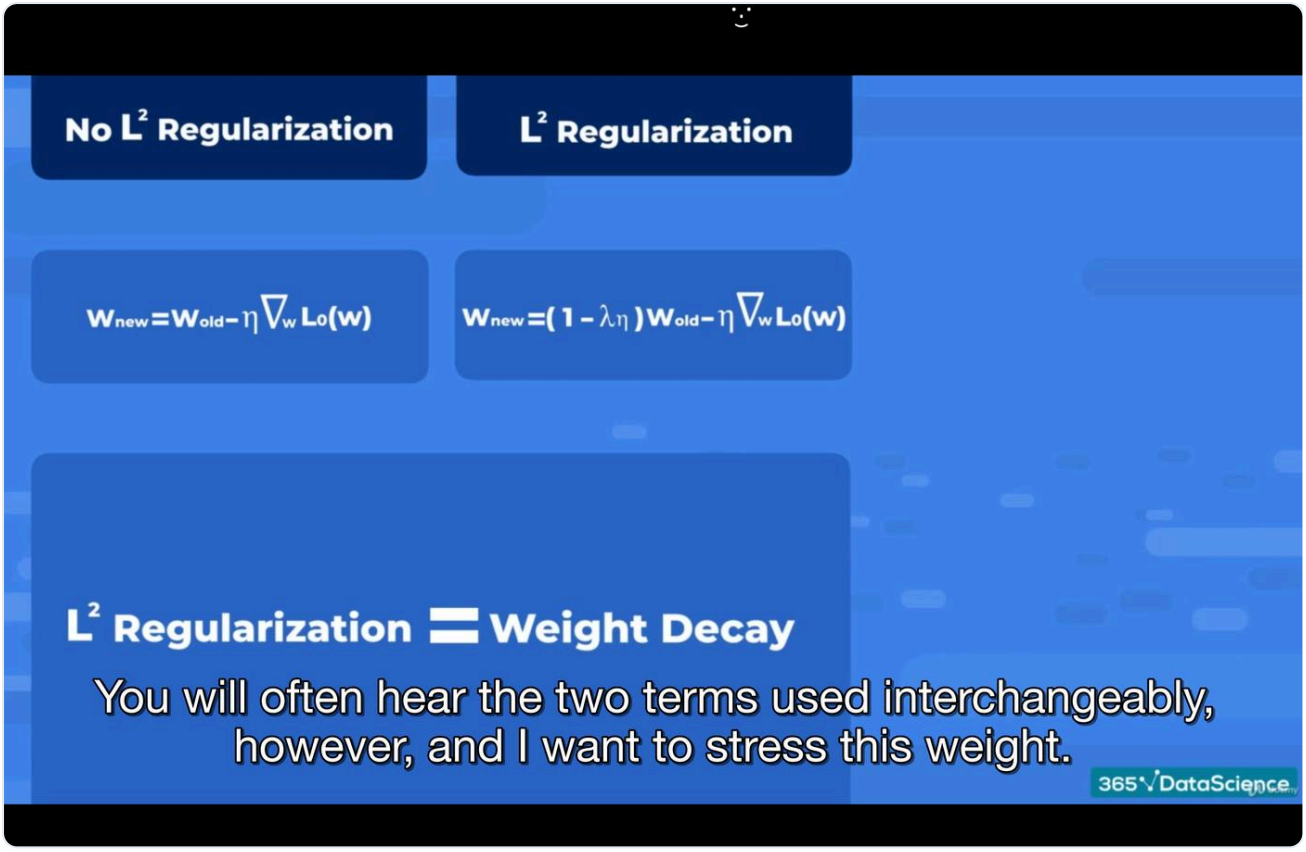

FIGURE 31.7 | No L2 Regularization L2 Regularization

You will often hear the two terms used interchangeably, however, and I want to stress this weight. Decay is actually a different thing.

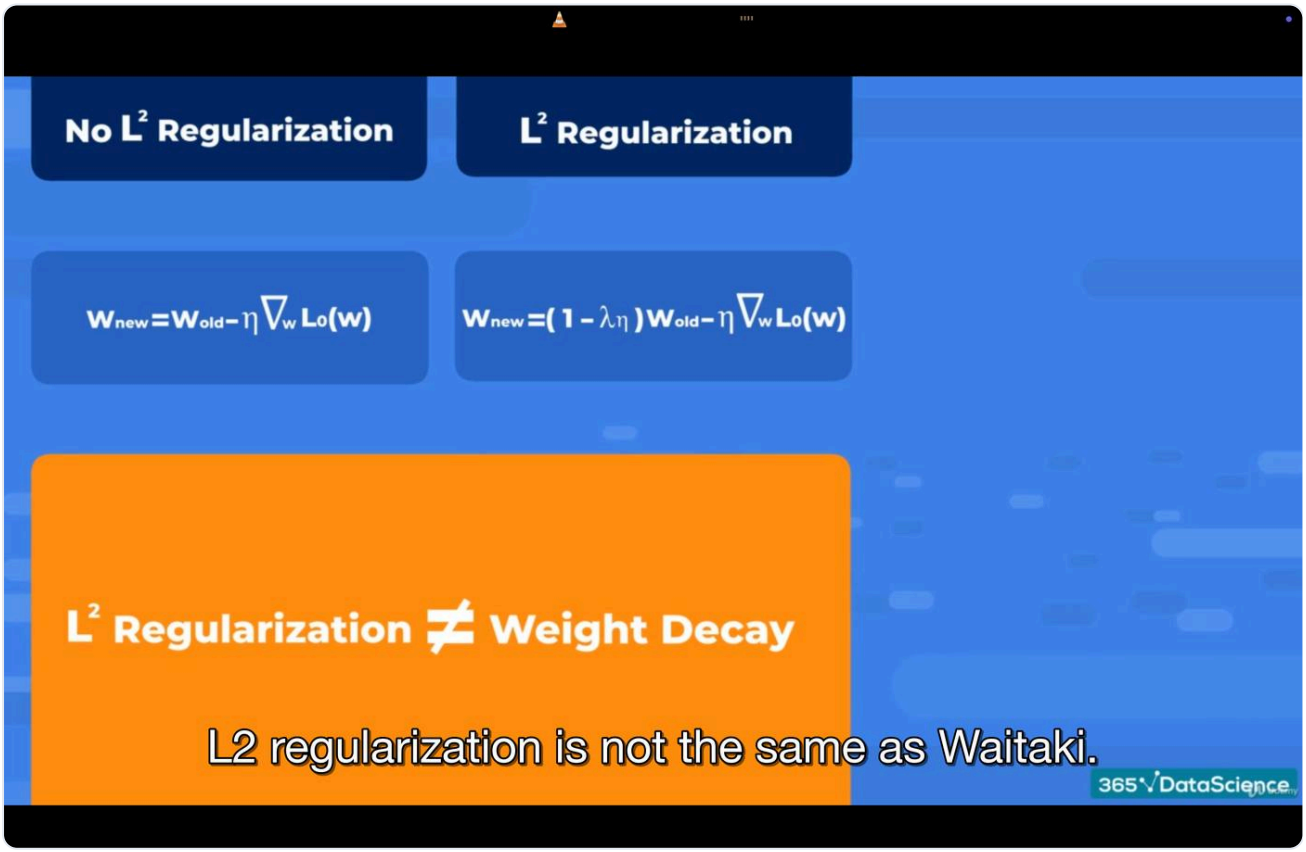

FIGURE 31.8 | No L2 Regularization L2 Regularization

L2 regularization is not the same as weight decay. To see this, here is the update rule for the actual weight k.

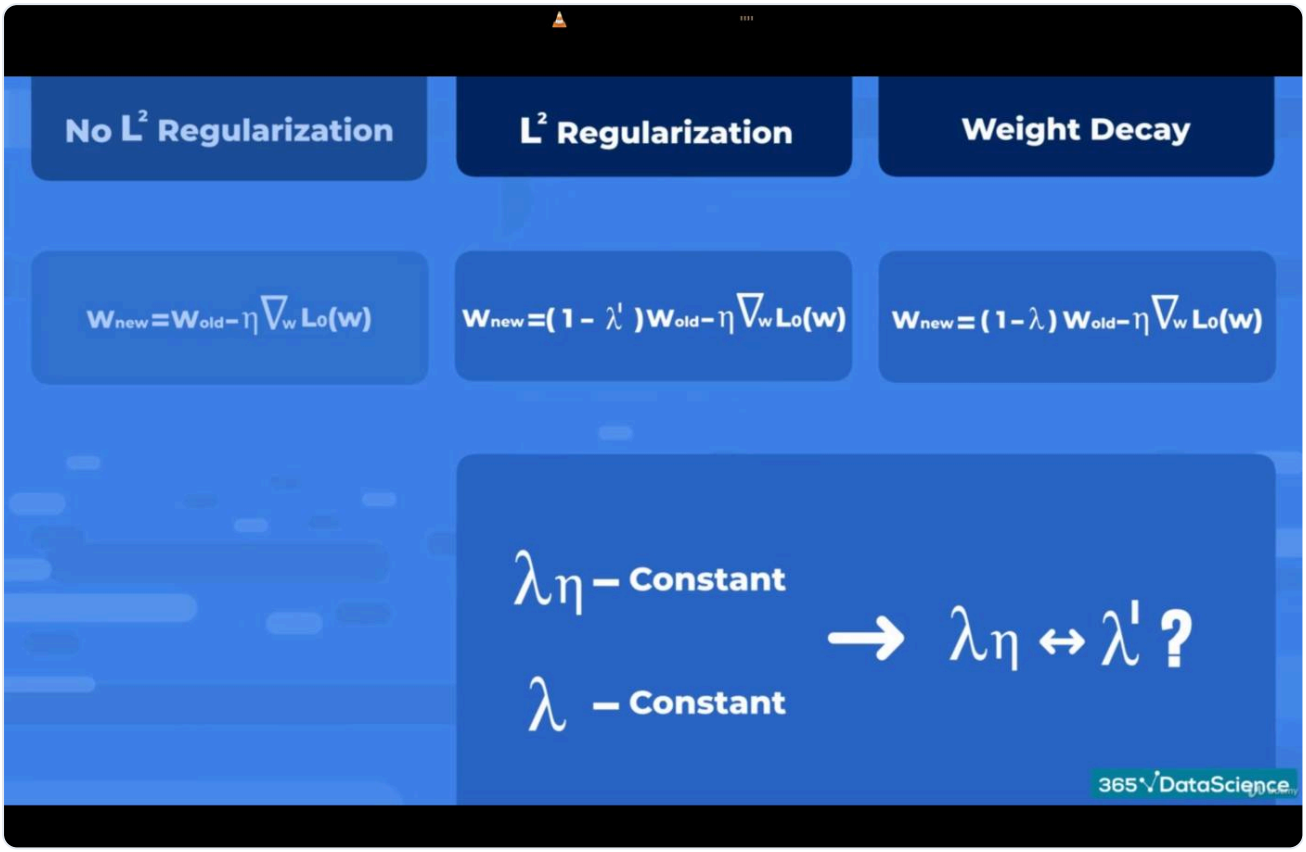

FIGURE 31.9 | L2 Regularization Weight Decay

When comparing the two equations, we can see that the weight to K one is missing in here, but what's the big deal? Lambda Lambda Times, etc.. It's all just some constant isn't it.

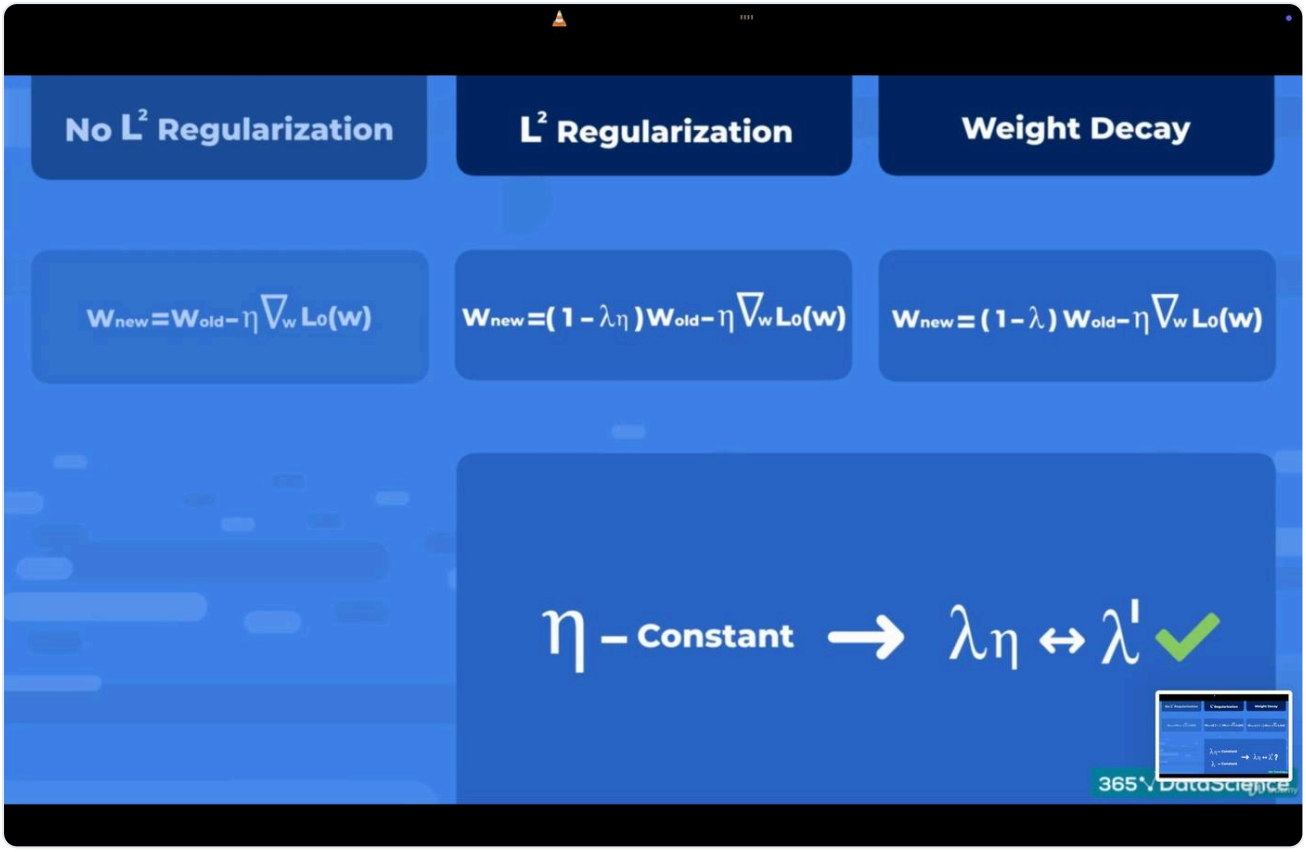

FIGURE 31.10 | L2 Regularization Weight Decay

We can just substitute Lambda times theta with a new constant lambda prime and we've got the same equation right. Well if the learning rate is constant then yes.

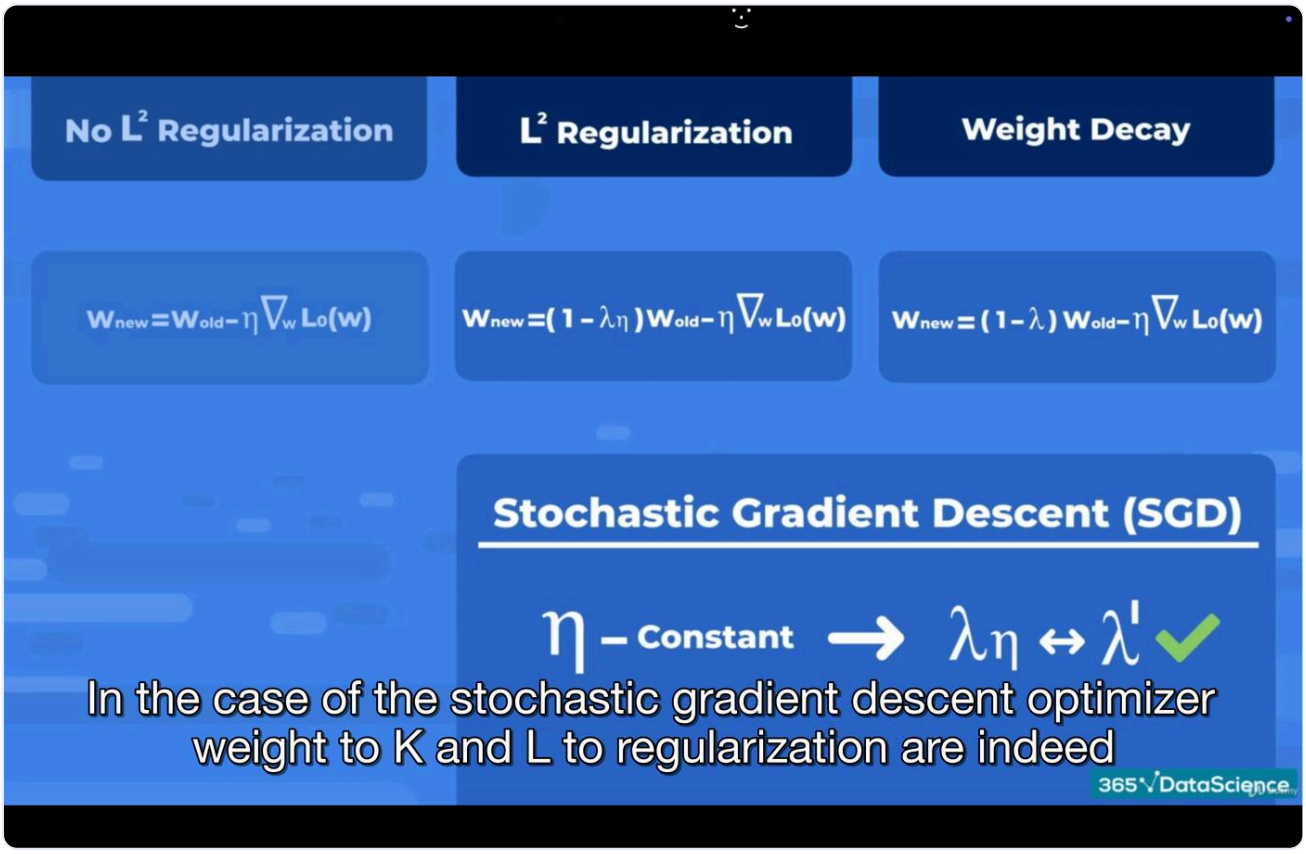

FIGURE 31.11 | L2 Regularization Weight Decay

In the case of the stochastic gradient descent optimizer weight to K and L to regularization are indeed equivalent.

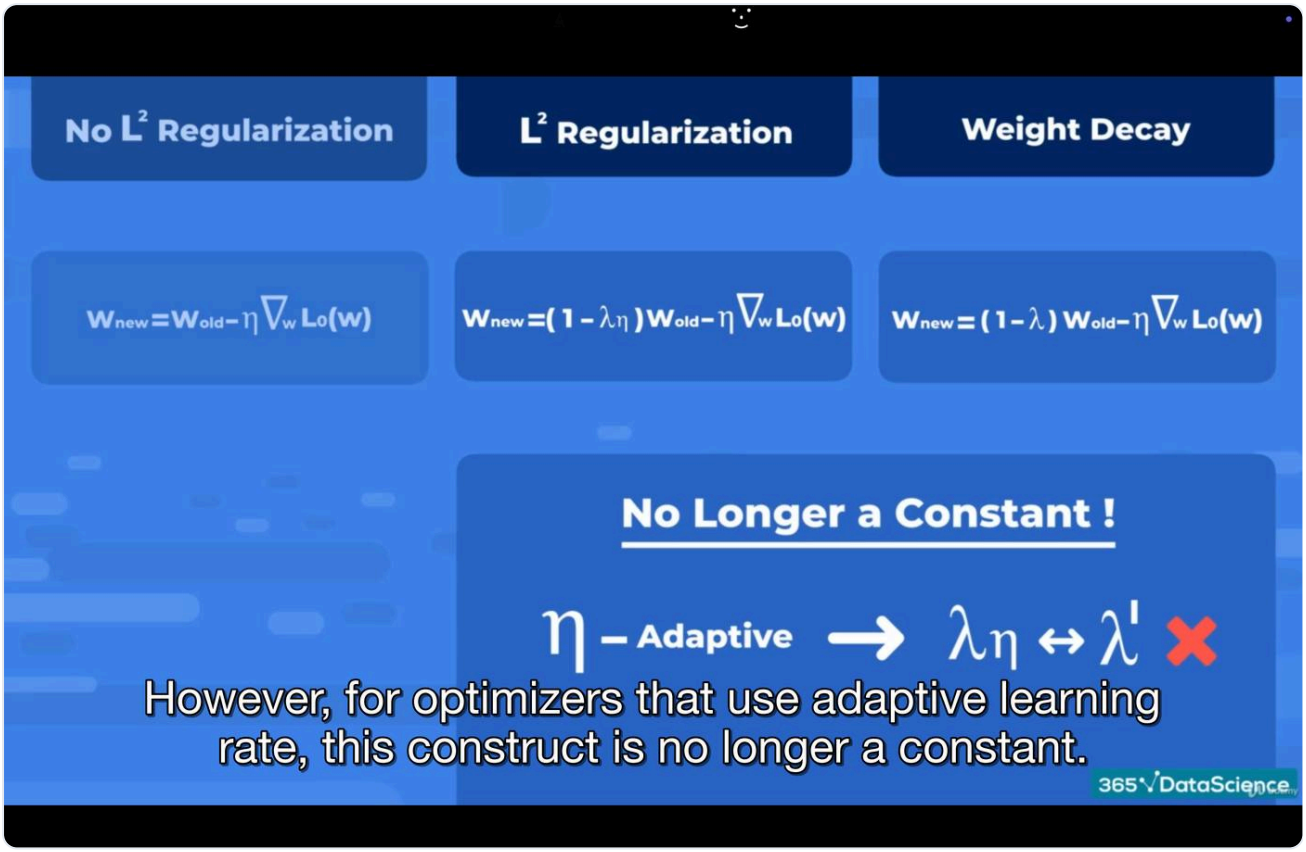

FIGURE 31.12 | L2 Regularization Weight Decay

However, for optimizers that use adaptive learning rate, this construct is no longer a constant. This is equivalent to having a lambda that changes values during the training.

FIGURE 31.13 | L2 Regularization Weight Decay

That means that the regularization effect would be different in the beginning and at the end of the training.

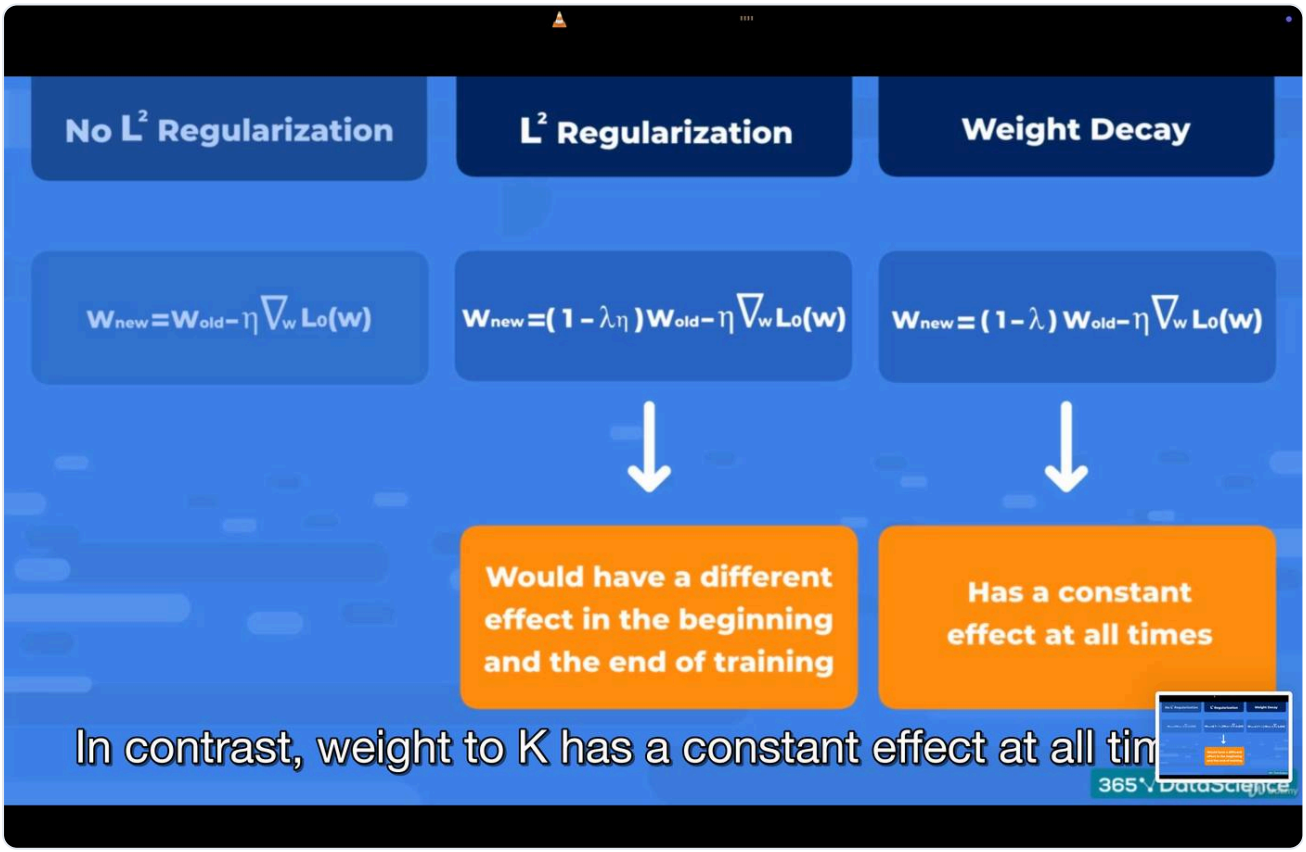

FIGURE 31.14 | No L2 Regularization L2 Regularization Weight Decay

In contrast, weight to K has a constant effect at all times. While both techniques are equally effective, it is important to be aware of their differences. All right, let's summarize what we've learned so far. Overfitting comes in different shapes and forms. Therefore we need different techniques to combat it.

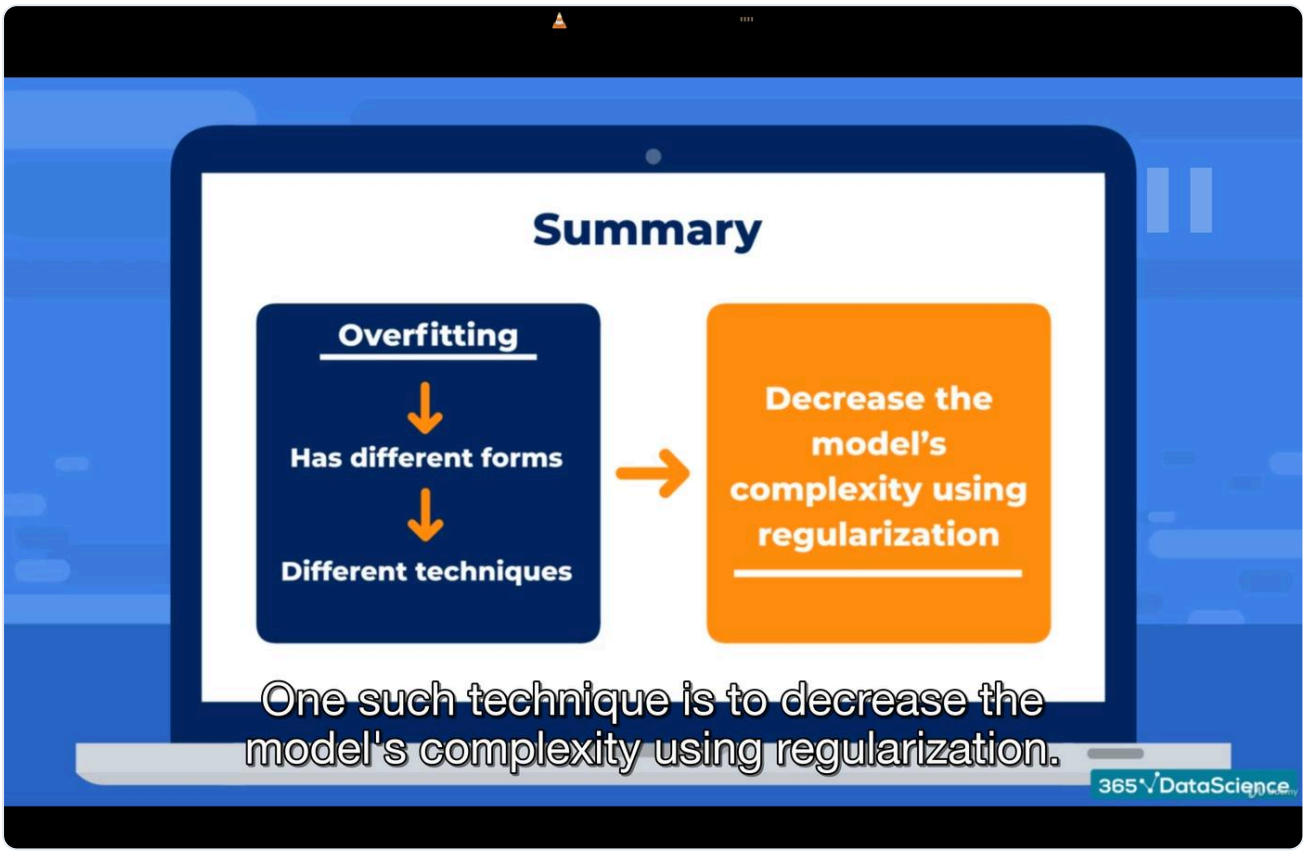

FIGURE 31.15

One such technique is to decrease the model's complexity using regularization. L2 regularization is one of the most popular regularization does. It consists of adding the sum of squared model weights to the loss function due to the form of the equation for the new weight. L2 regularization is also known as weight decay. However, weight decay is a separate concept that has slightly different update rule.

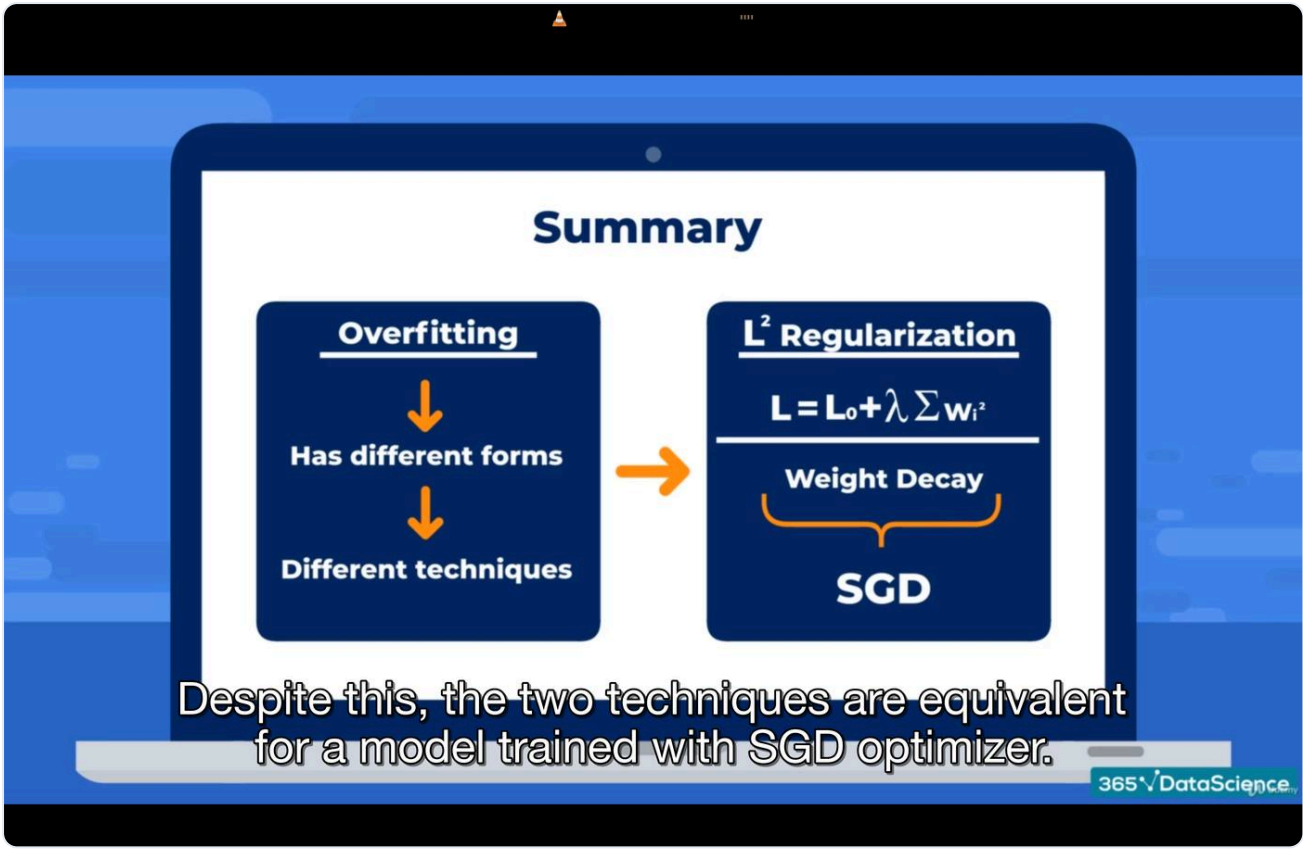

FIGURE 31.16 | Overfitting L2 Regularization

Despite this, the two techniques are equivalent for a model trained with SGD optimizer. We have seen the intricacies of regularisation, but in the beginning, we promised to talk about dropout and data augmentation as well, and we are going to keep this promise as the next lecture is all about dropout. See you there. And thanks for watching.

LECTURE 32

Neurons

Last time we introduced two different techniques to combat Overfitting, L2, Regularisation and Wait decay, in this video, we will consider a fundamentally different method to achieve the same result. The dropout technique. OK.

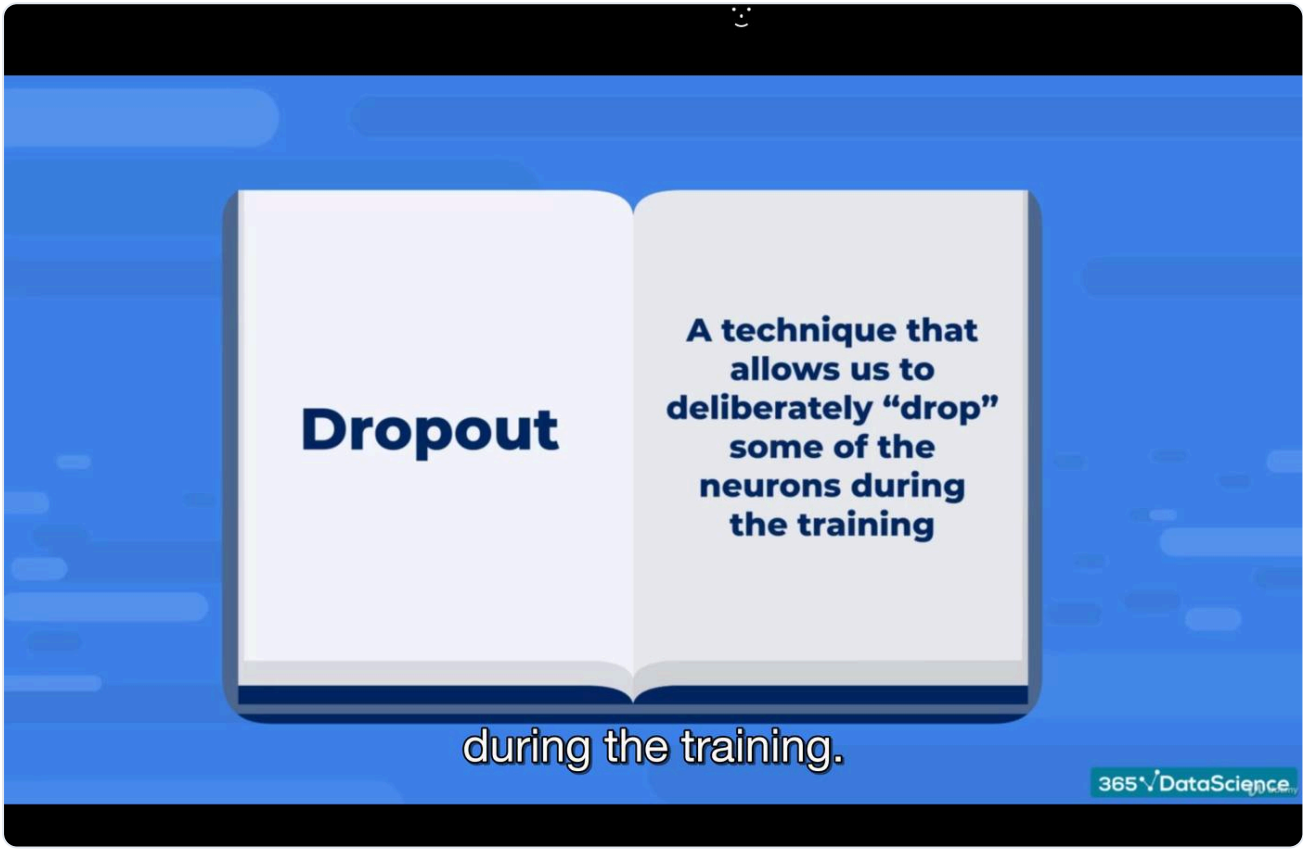

FIGURE 32.1 | A Technique That

As the name suggests, Drop-Out is a technique that allows us to deliberately drop some of the neurons during the training. How do we do that?

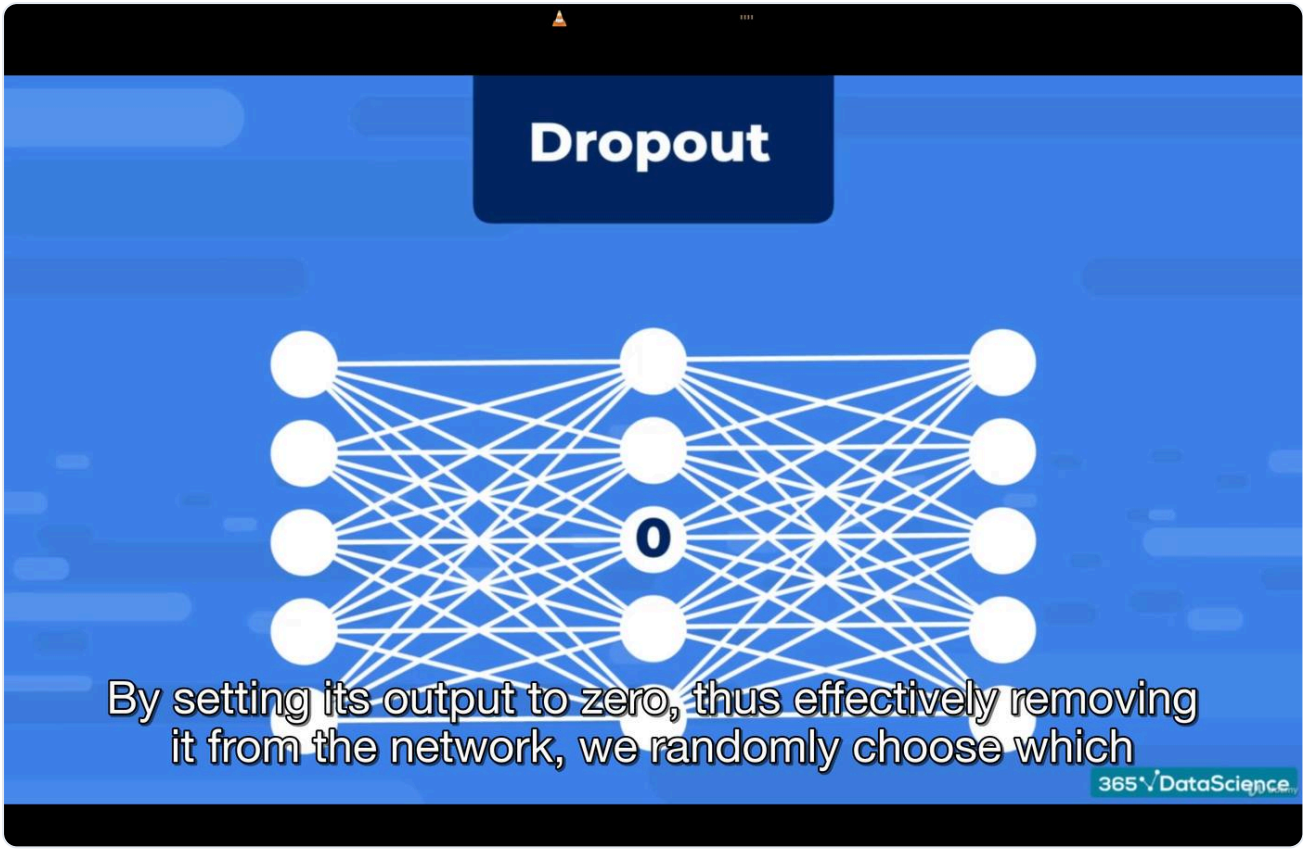

FIGURE 32.2

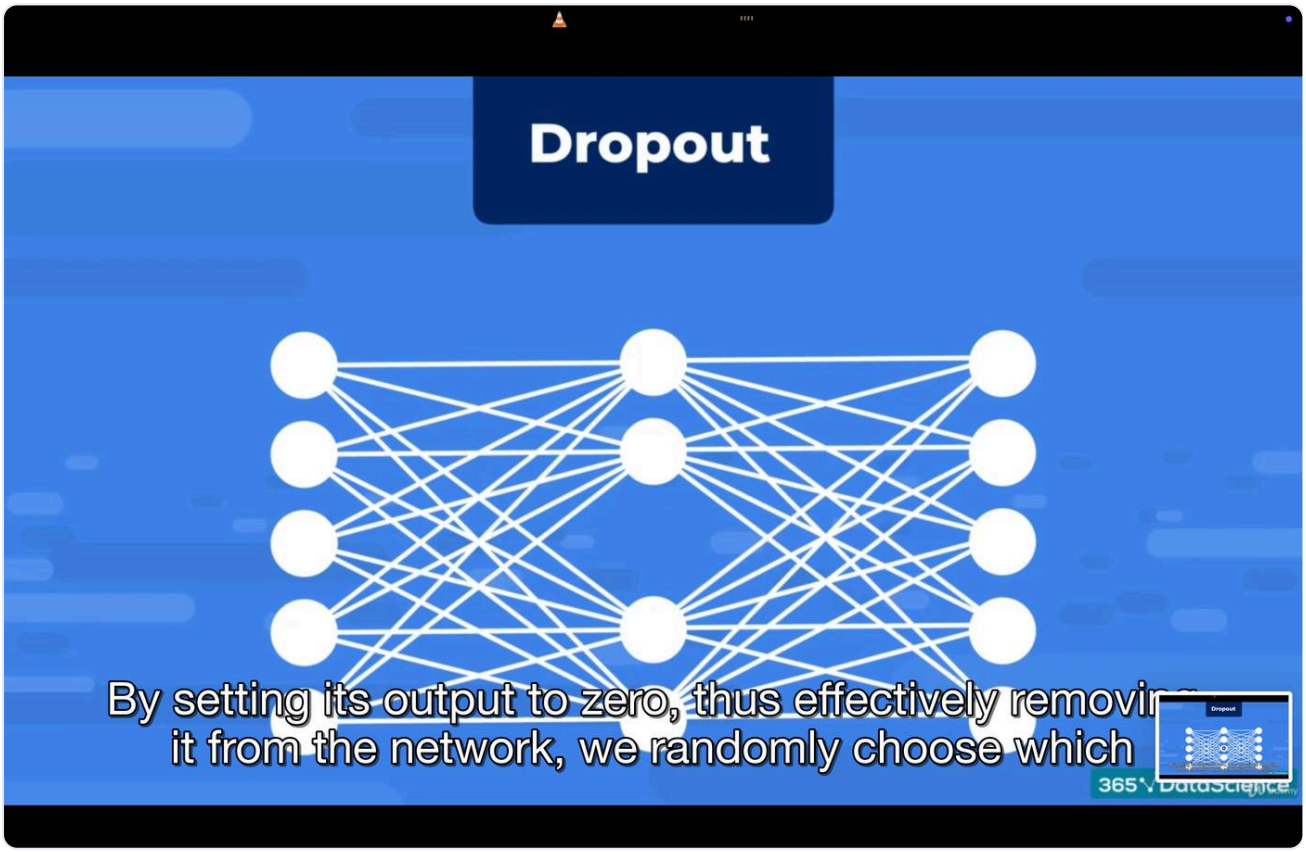

FIGURE 32.3

By setting its output to zero, thus effectively removing it from the network, we randomly choose which neurons to drop from all neurons in a given layer at the beginning of each episode.

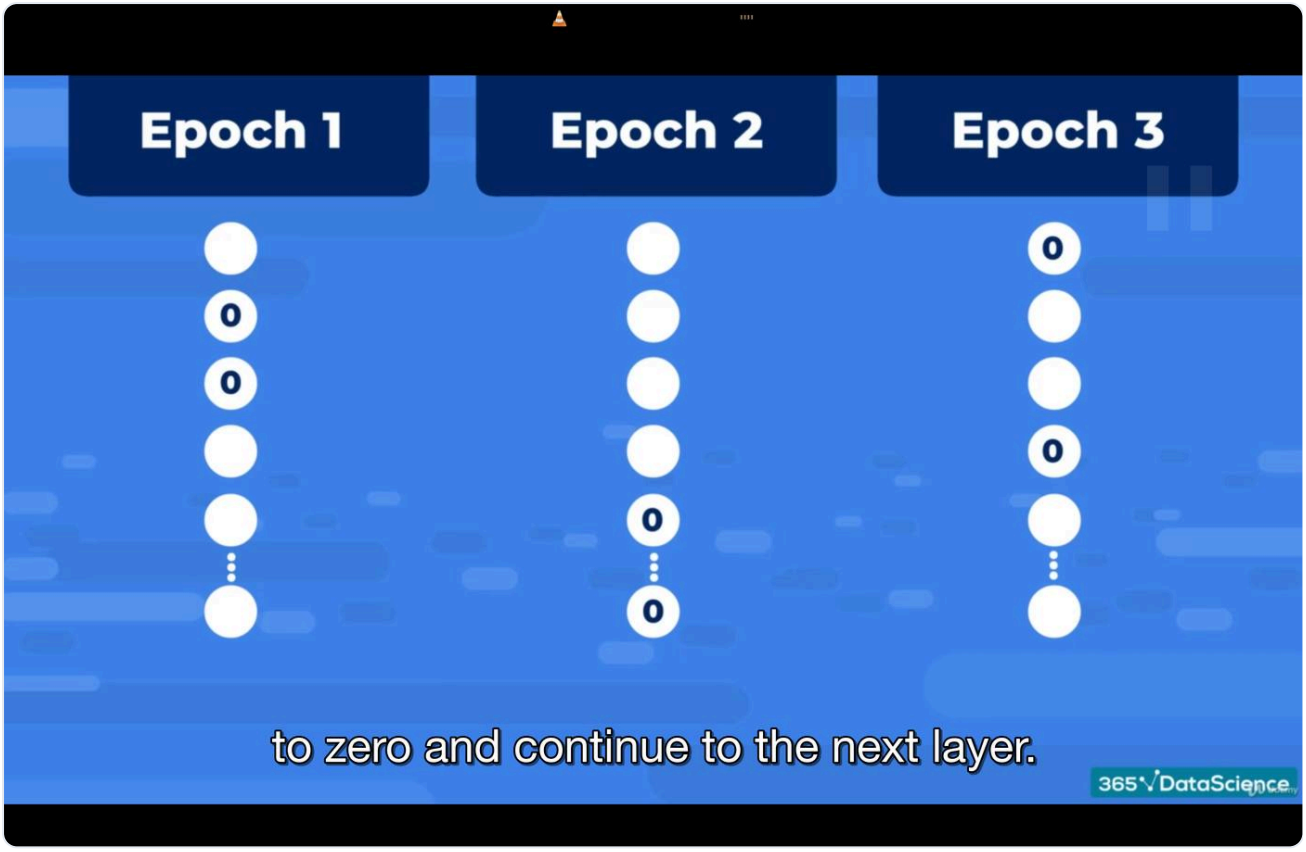

FIGURE 32.4

So, for example, if we have a dense layer with one hundred outputs, we may randomly set 10 of those to zero and continue to the next layer.

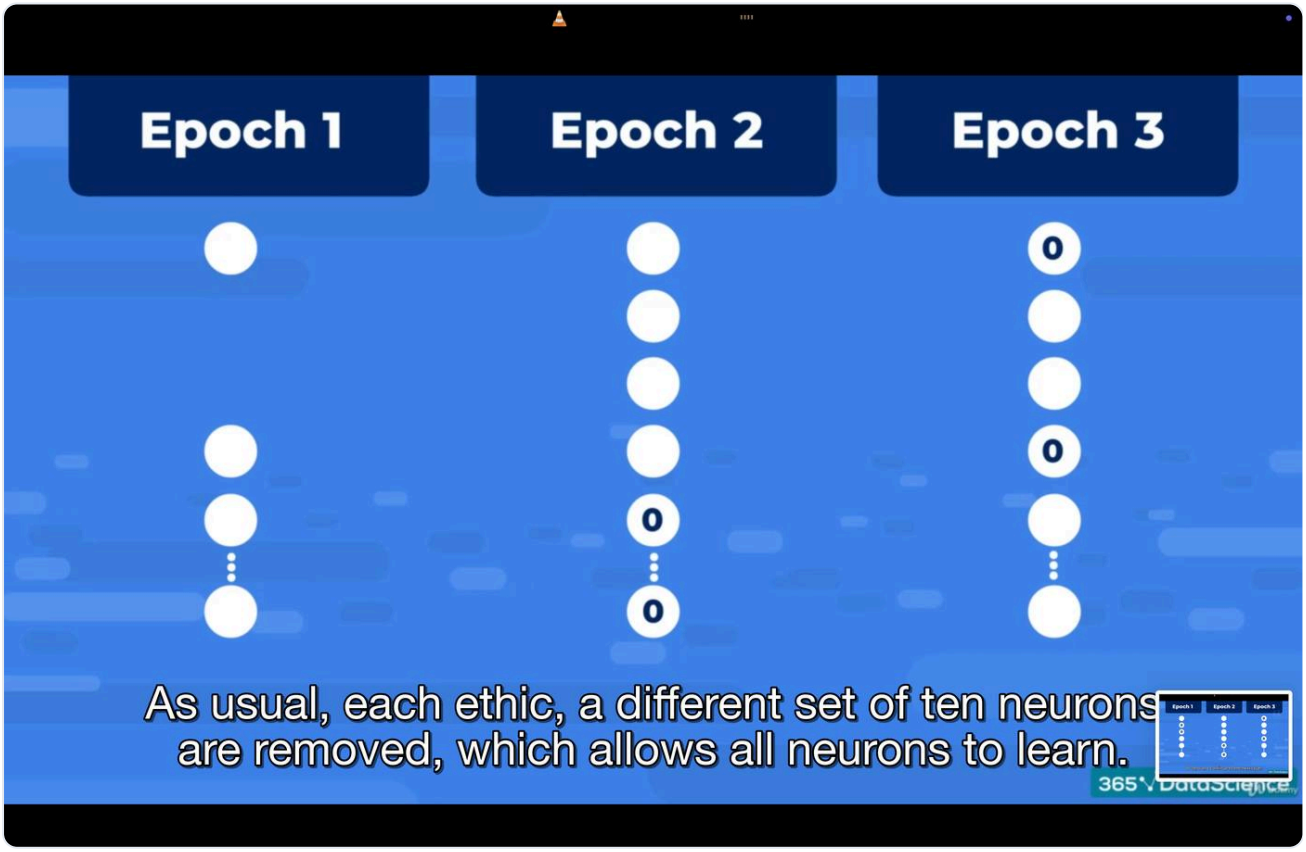

FIGURE 32.5

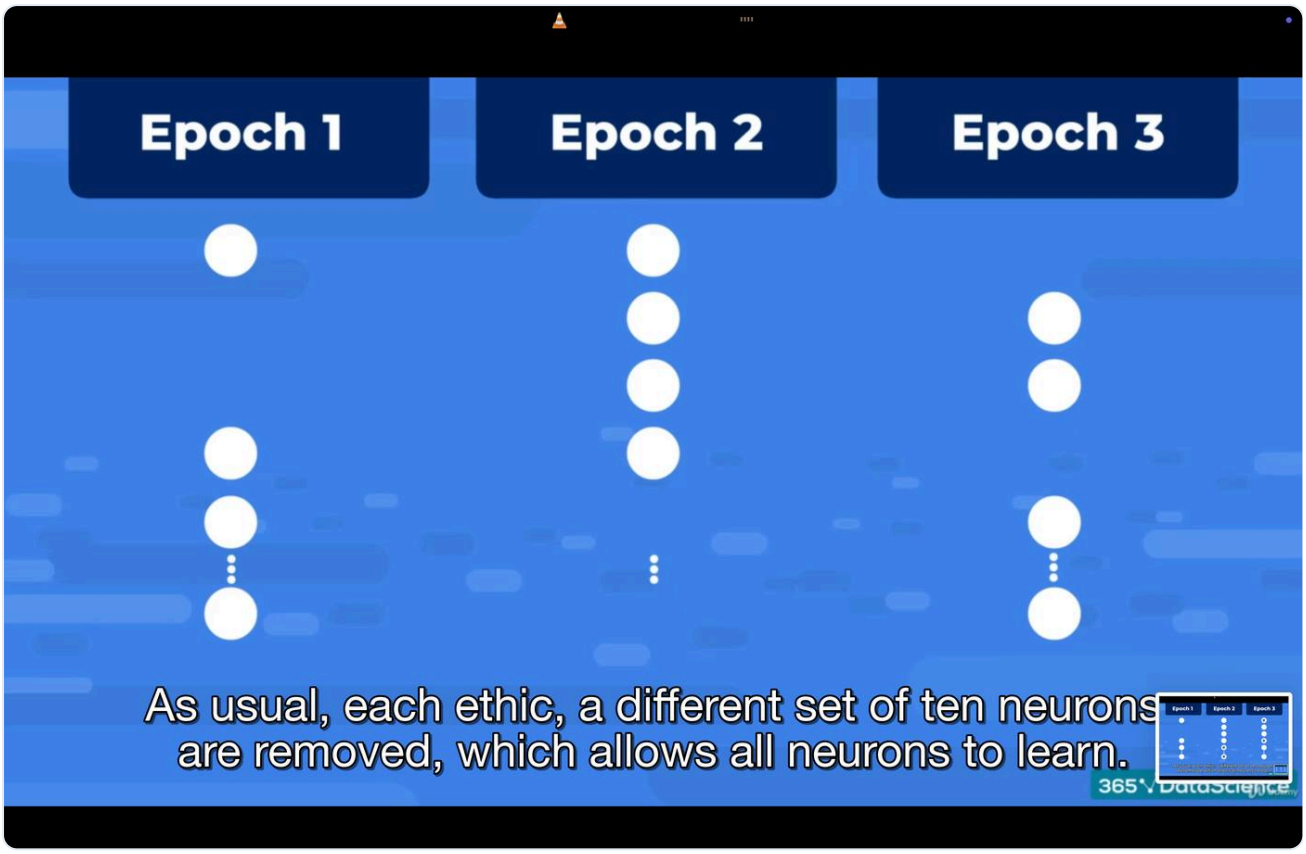

FIGURE 32.6

As usual, each ethic, a different set of ten neurons are removed, which allows all neurons to learn.

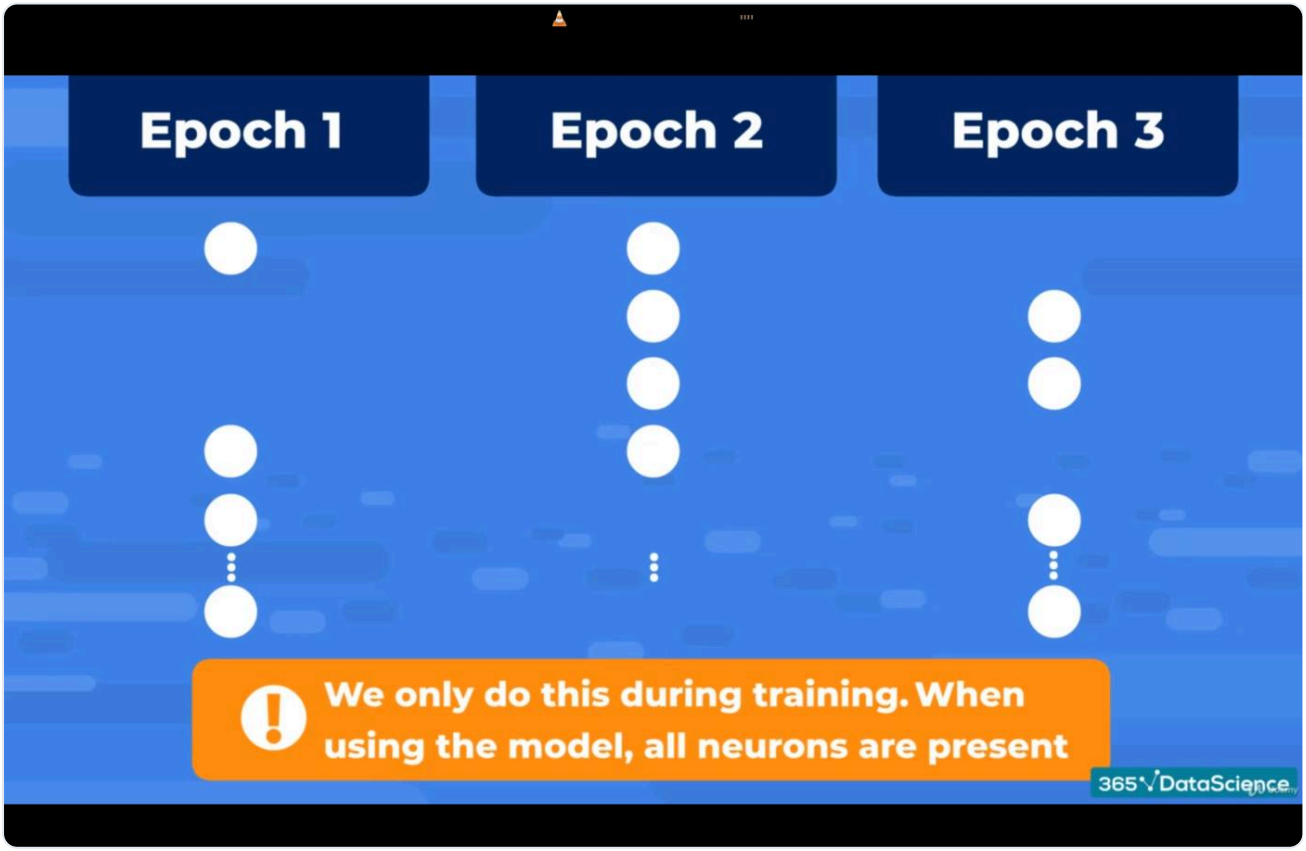

FIGURE 32.7

Remember that we only do this during training, when evaluating and actually using the model. All neurons are present. O kay, that is not too difficult to imagine, but it seems a bit jarring. We are a moving parts of our network when training it on purpose, even worse. This is not done consistently, but in a random fashion.

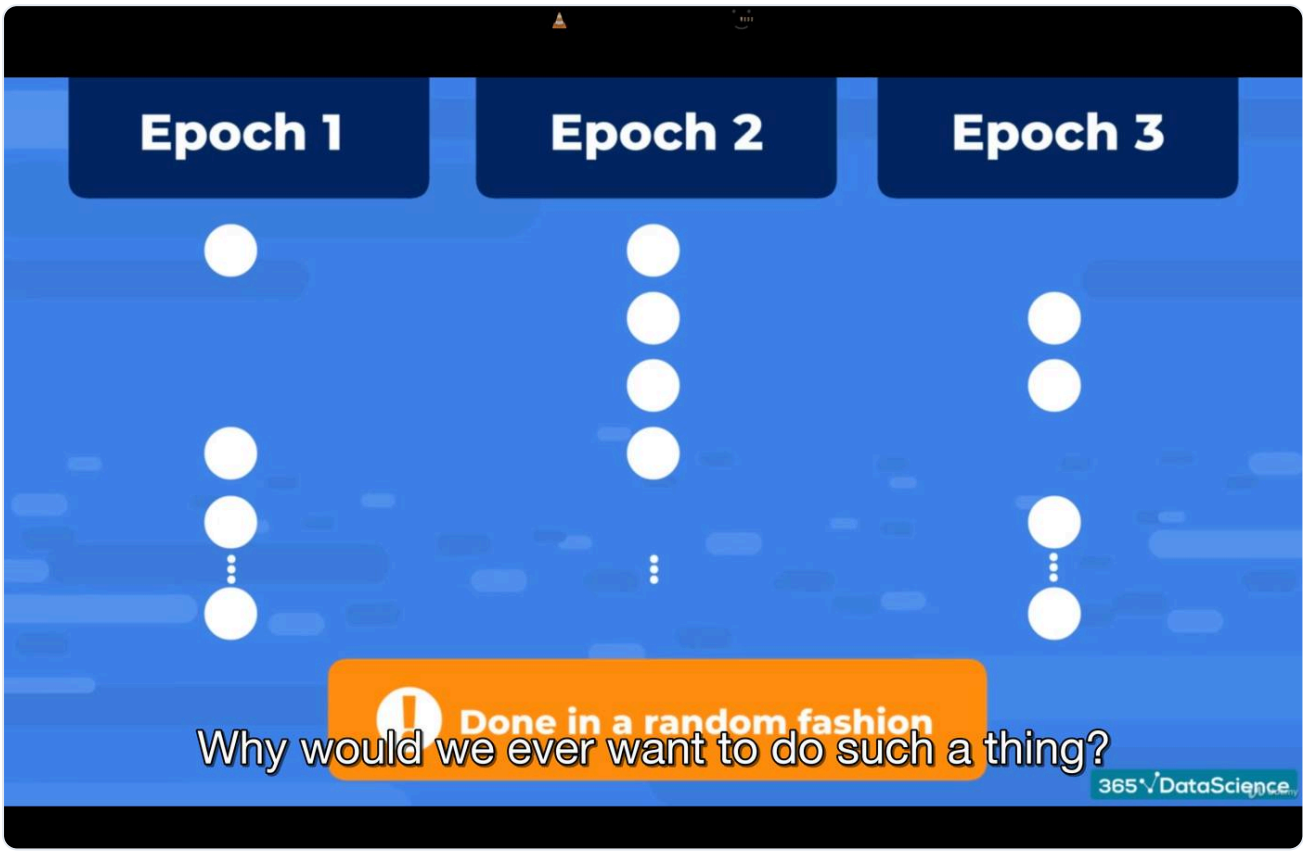

FIGURE 32.8

Why would we ever want to do such a thing? Well, the idea behind this intriguing concept is to make the neurons a bit more independent from each other.

FIGURE 32.9

By removing some of their colleagues.

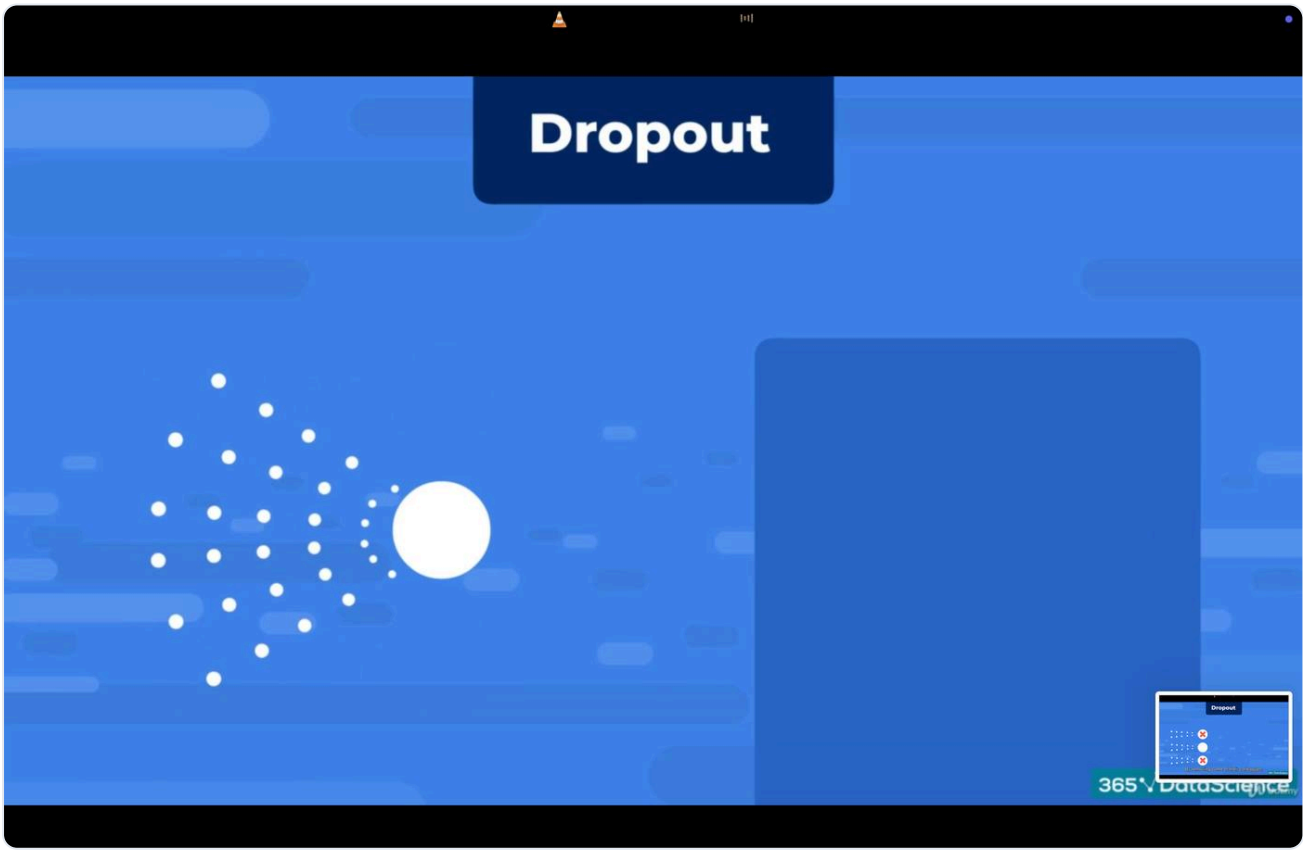

FIGURE 32.10

The remaining neurons need to cover for them.

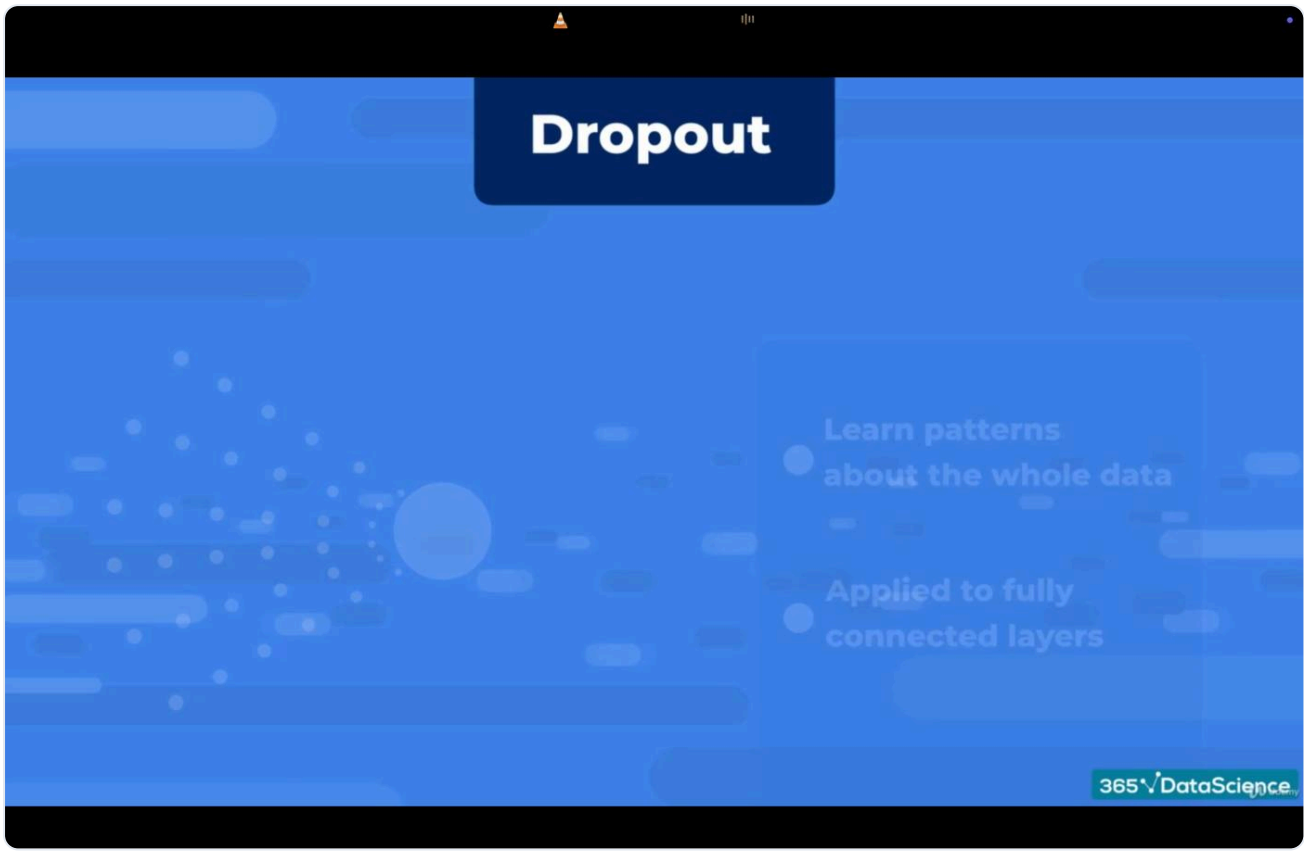

FIGURE 32.11

They are forced to learn patterns about the whole data, not just some specific features they specialize in.

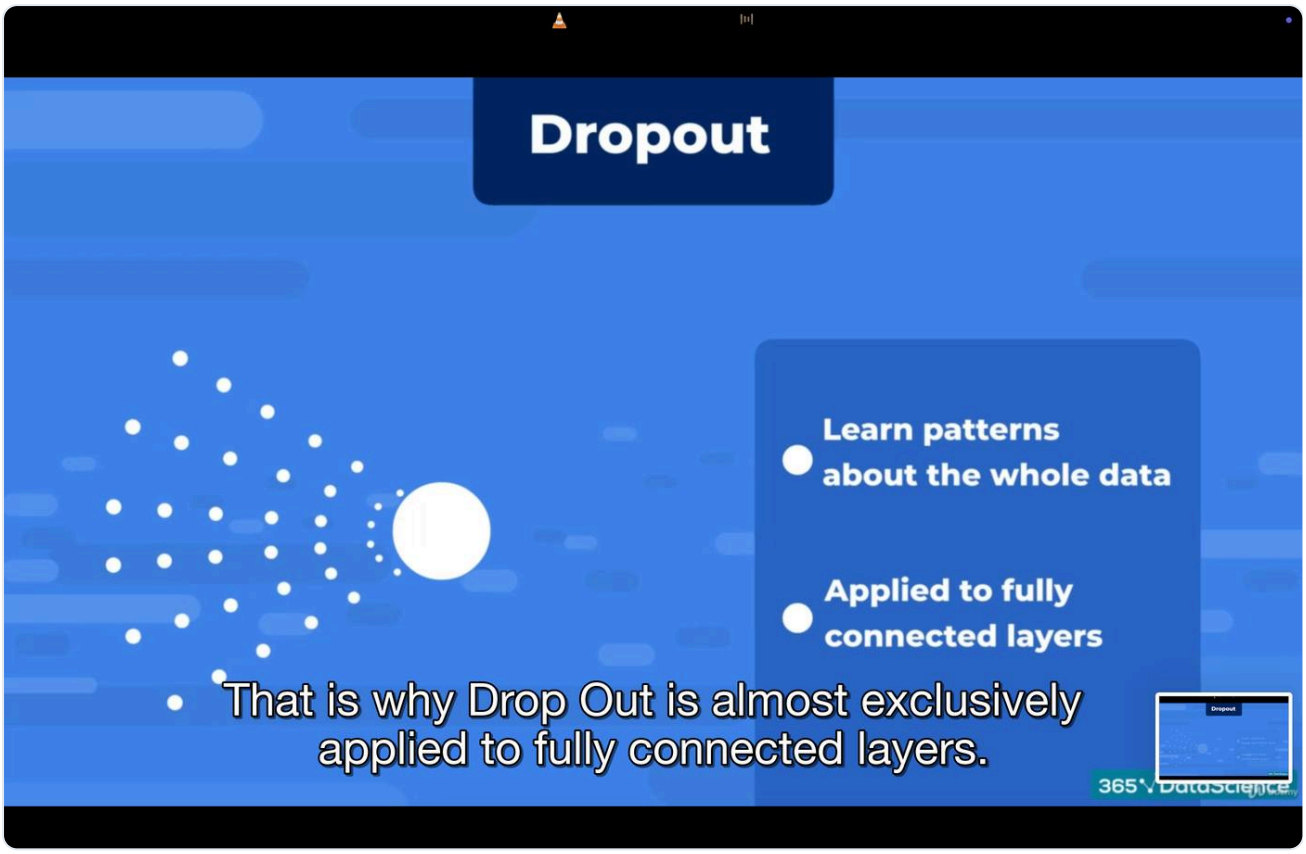

FIGURE 32.12

That is why Drop Out is almost exclusively applied to fully connected layers. Dropout usually has one hyper parameter associated with it the percentage of neurons to be randomly dropped.

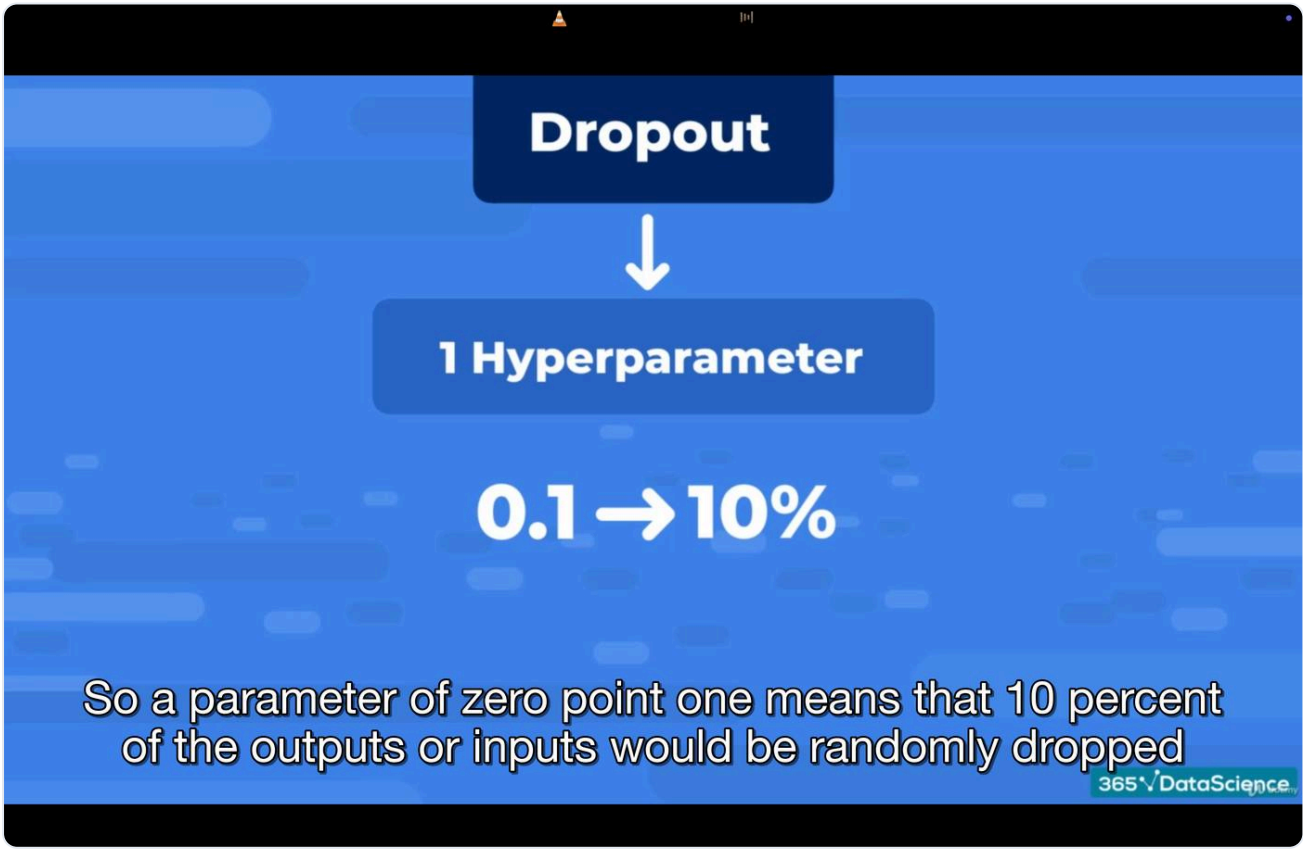

FIGURE 32.13

So a parameter of zero point one means that 10 percent of the outputs or inputs would be randomly dropped to zero. Typical values for this parameter may vary between zero point one and zero point five.

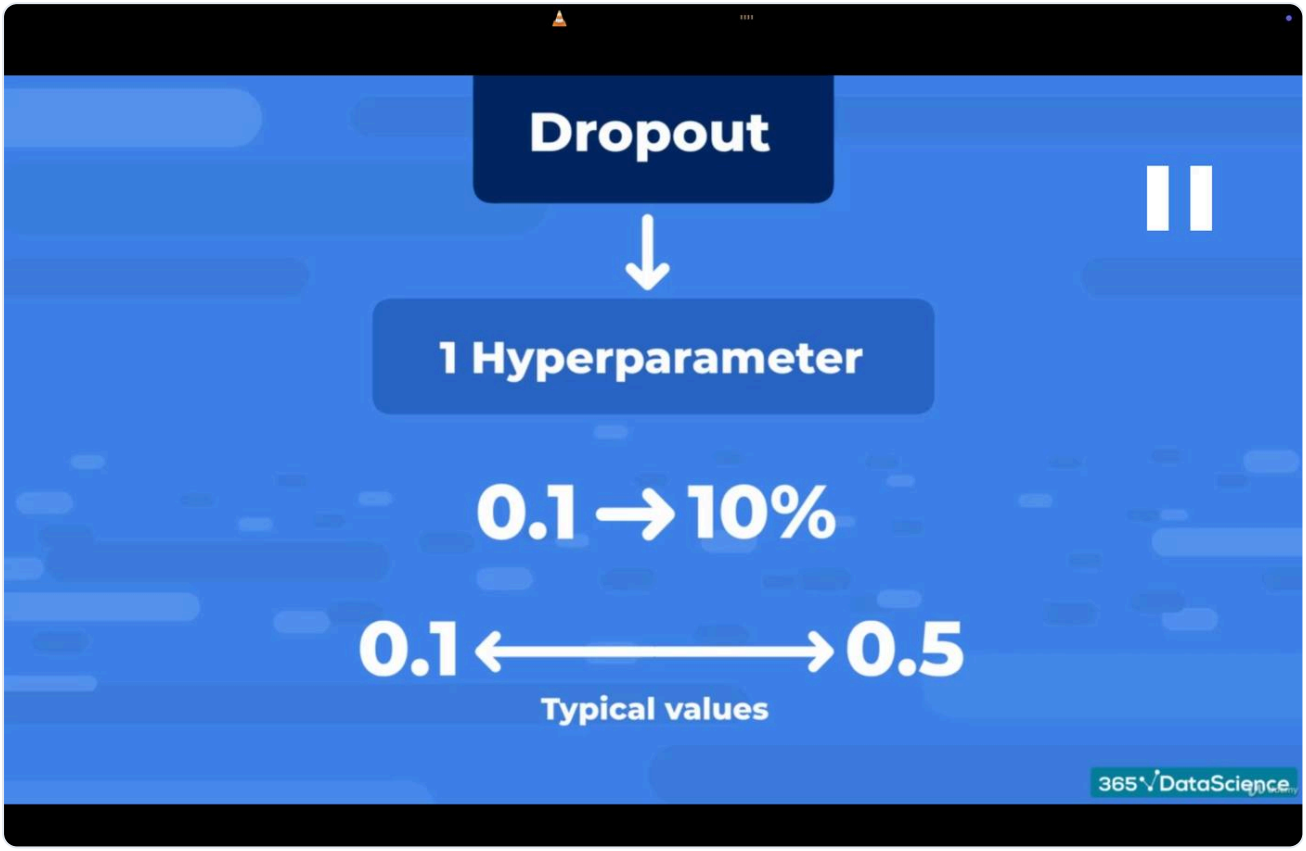

FIGURE 32.14

Another advantage of drop out is that it's very easy to implement an existing code and architecture, as you will see later in the practical example.

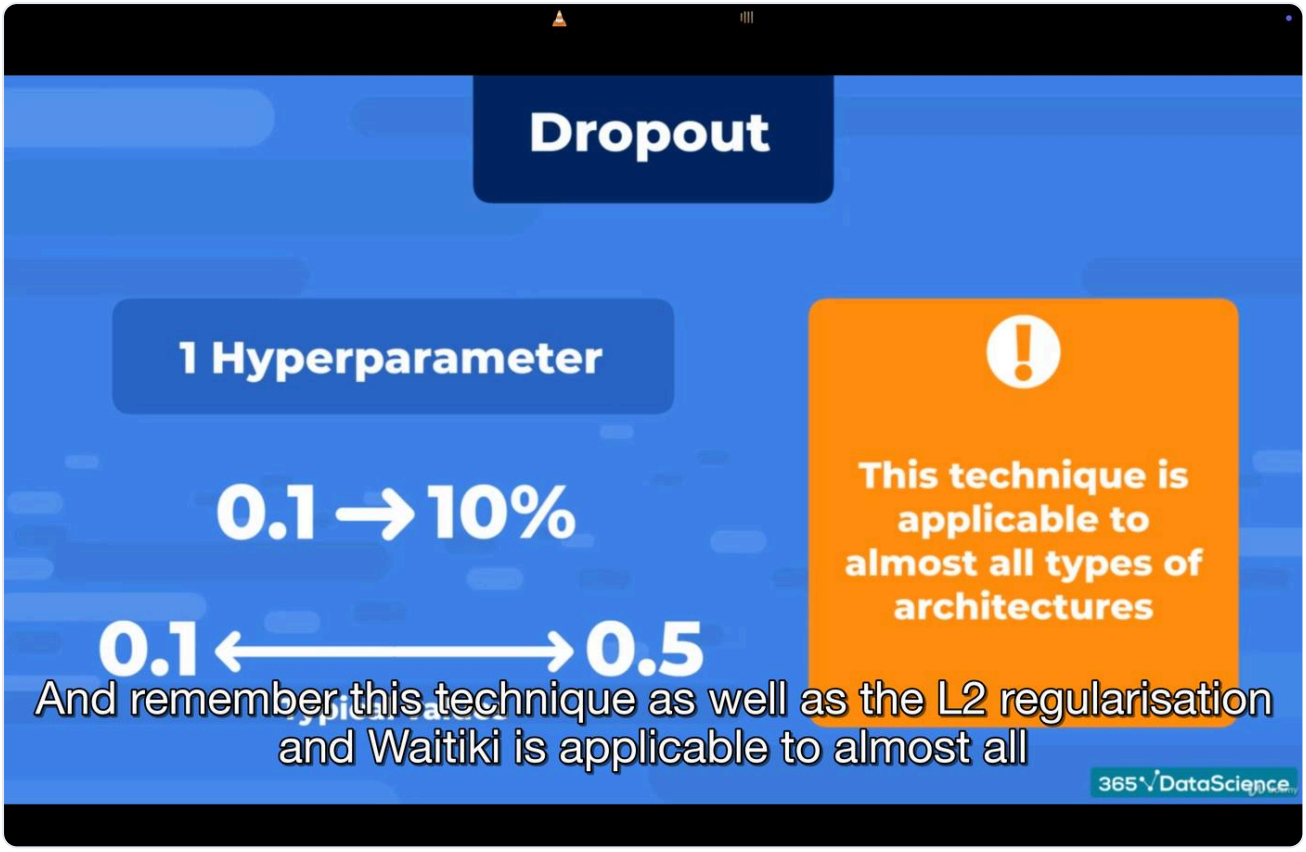

FIGURE 32.15

And remember this technique as well as the L2 regularisation and Waitiki is applicable to almost all types of architectures, not just for convolutional neural networks. Good. Now some technical details about drop out.

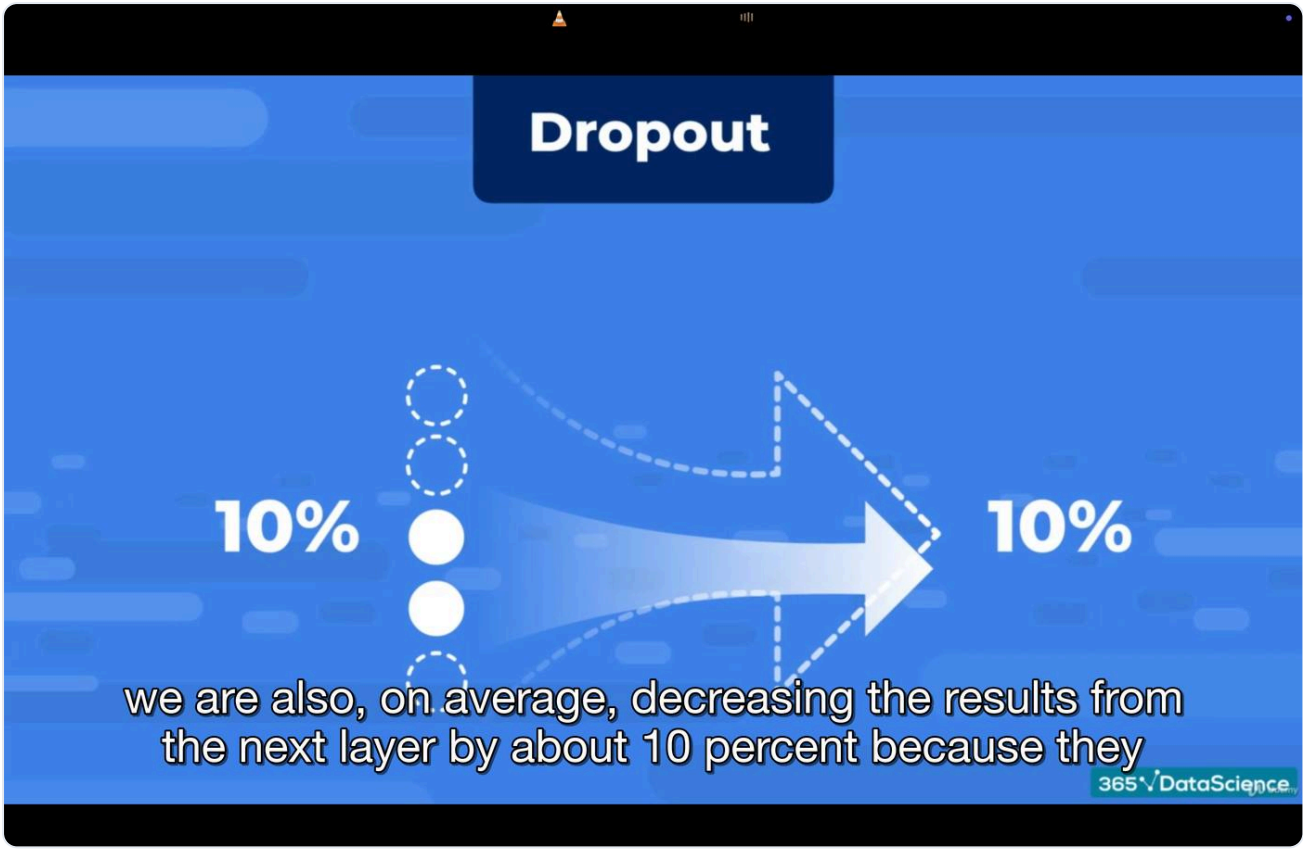

FIGURE 32.16

When we set 10 percent of the neurons to zero, we are not only forcing the other neurons to learn more, we are also, on average, decreasing the results from the next layer by about 10 percent because they are linear combinations of the previous outputs. This is how we introduce a change in the whole chain of computations by decreasing the size of the outputs pretty much uniformly across the next layers. So to work properly, the dropout needs to scale the remaining outputs up a bit easier computations.

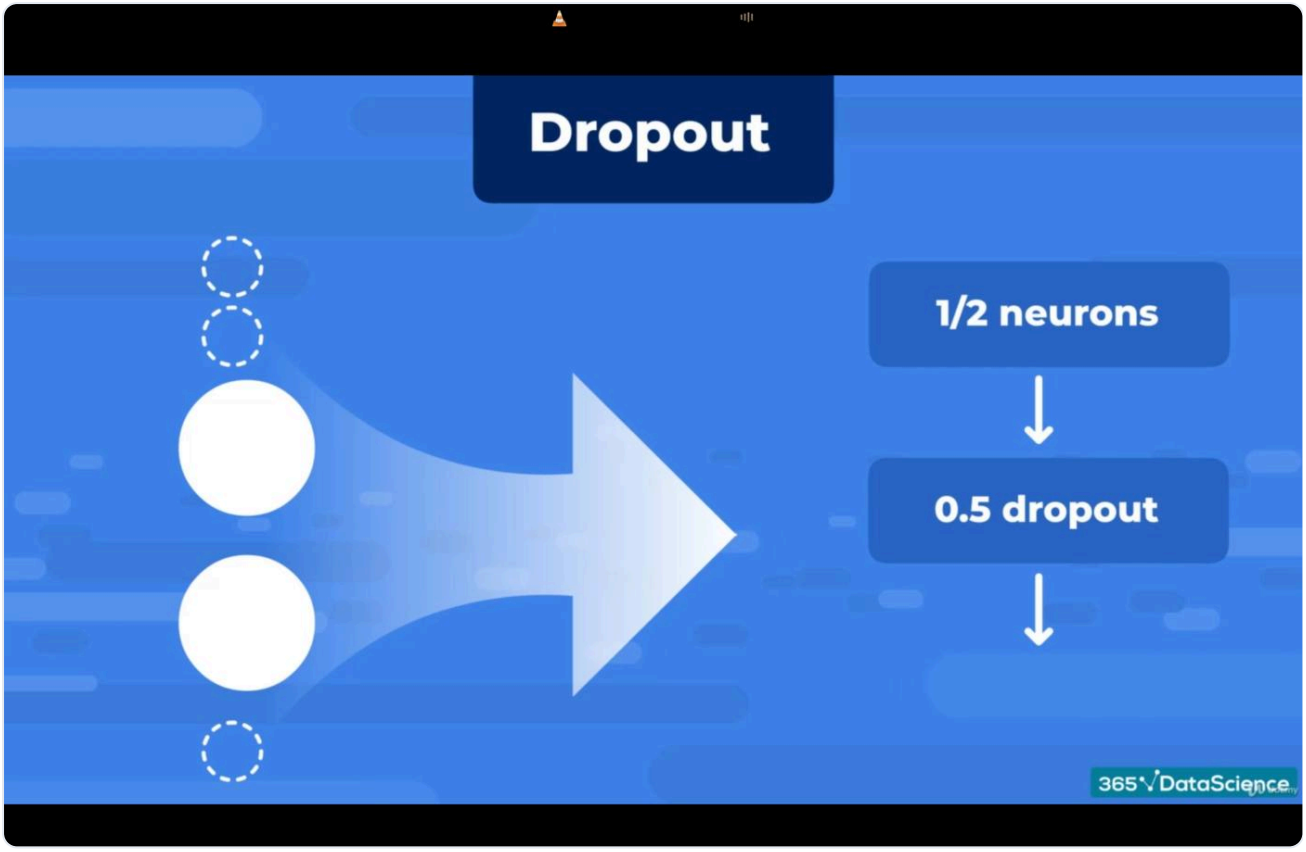

FIGURE 32.17

Let's take a look at a situation where we remove half of the neurons. That is a drop out of zero point five. Then if we want the next layer to work properly, we need to double the remaining half outputs.

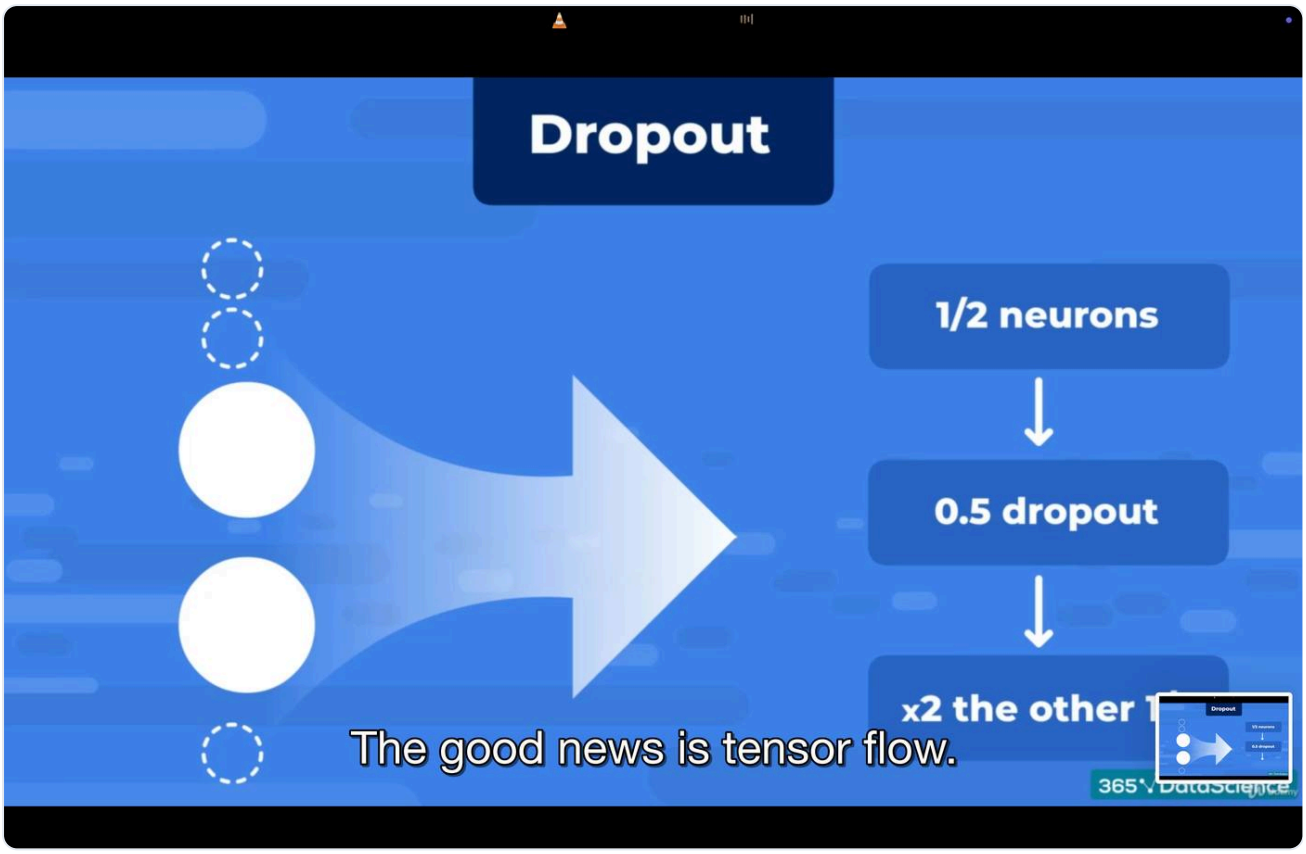

FIGURE 32.18

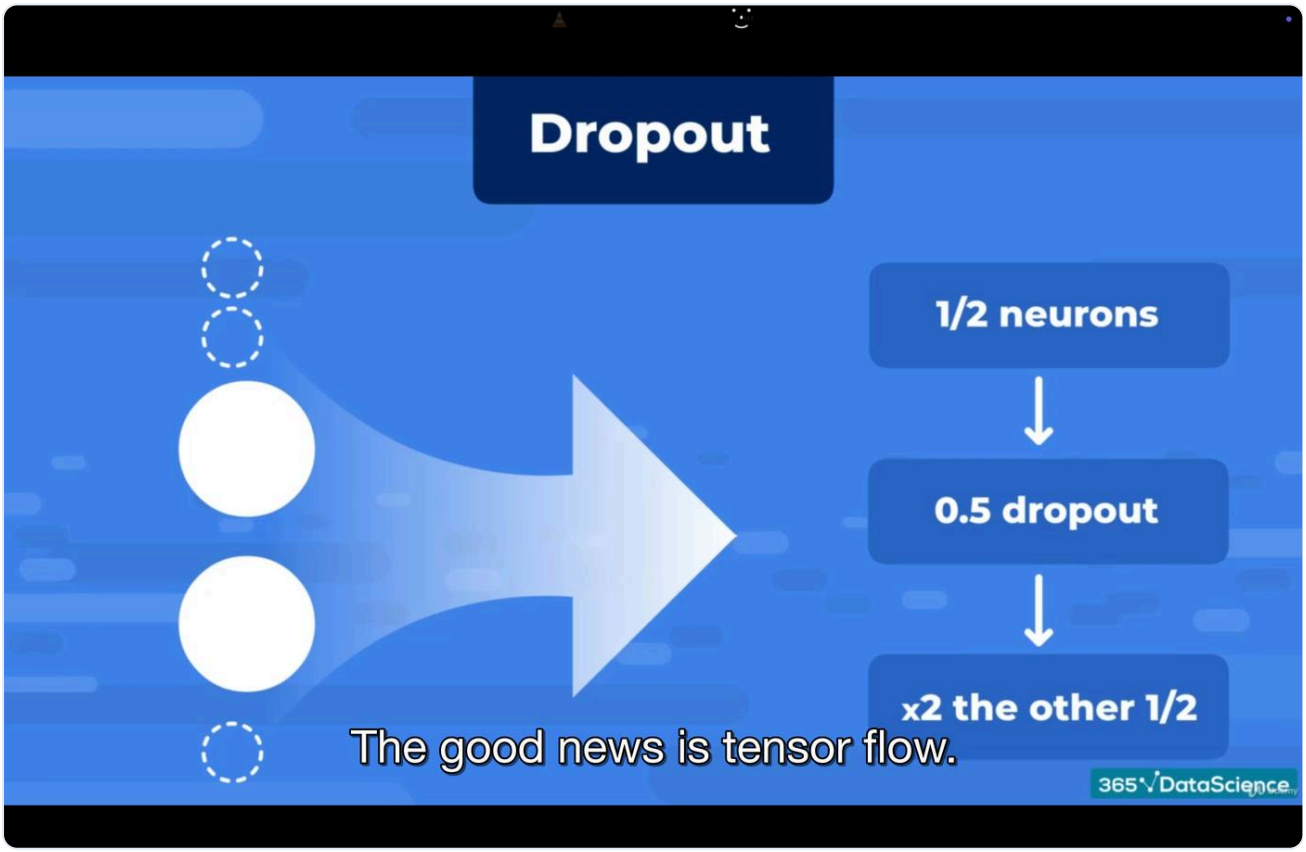

FIGURE 32.19

The good news is TensorFlow. Does this scaling automatically for us and we don't have to worry about it. All right. That is enough about dropouts. The rest will be practice. But before that, join us in the next video to learn about data augmentation. Thanks for watching.

LECTURE 33

Data Augmentation

So far, we introduced techniques focus on helping the network perform better by dealing with Overfitting. However, in this lecture, we will do something different for a change, discuss a technique that actually focuses on the dataset rather than the network. When we introduce convolutional networks, theoretically, we pointed out how a normal network with only dense layers may struggle to identify an already learned image if it is slightly moved to the side. Back then, we said that convolutional networks do not have this problem. While this is not wrong, it is only partially true.

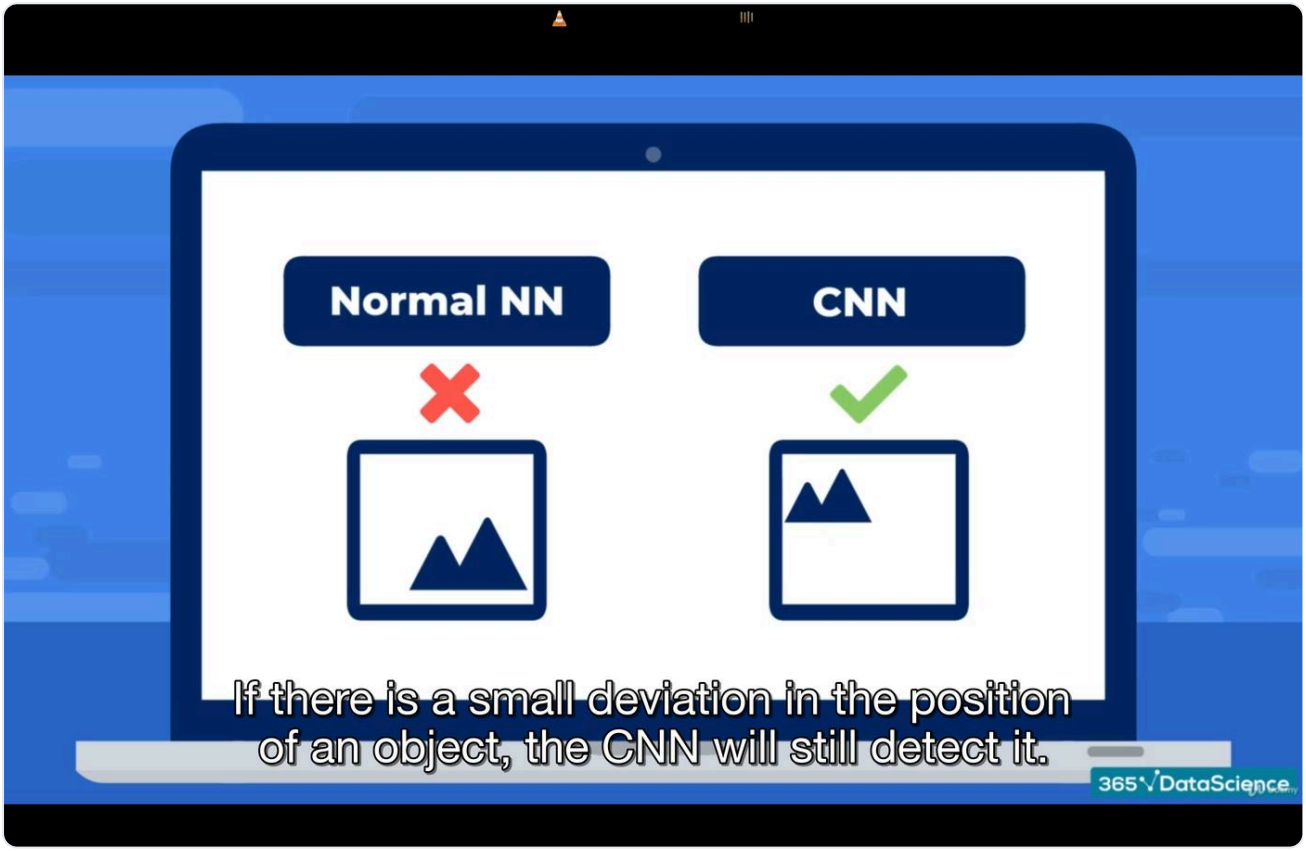

FIGURE 33.1

If there is a small deviation in the position of an object, the CNN will still detect it. Problem comes when the change is larger. To restate a CNN can only learn the patterns present in the training data. If we trained it on recognizing genes, it won't be able to recognize shoes.

FIGURE 33.2

This might seem obvious, but our perception on what the training data is often differs from what it is in reality. For example, let's say we train a cat classifier with numerous images of cats in different poses. We might think that's enough for the network to learn to recognize cats.

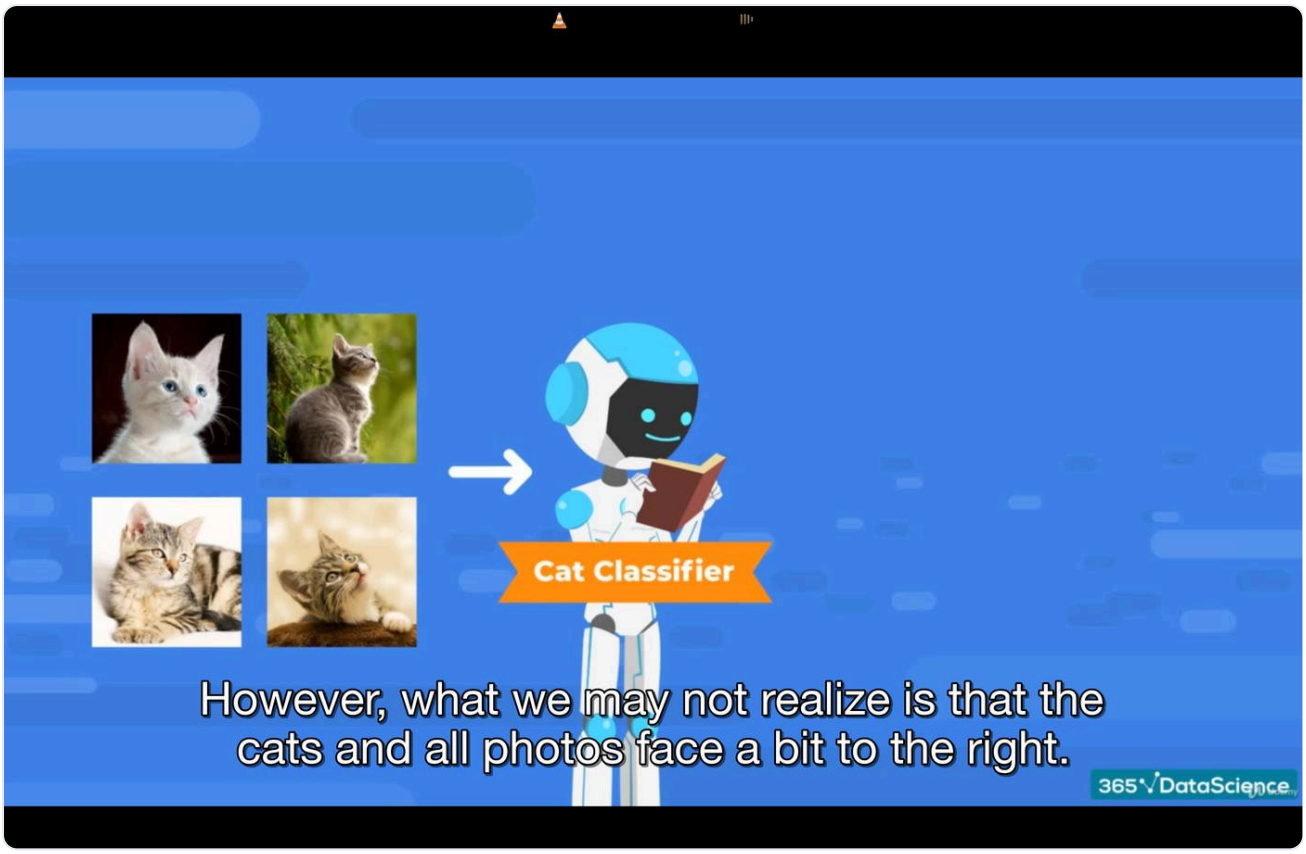

FIGURE 33.3

However, what we may not realize is that the cats and all photos face a bit to the right.

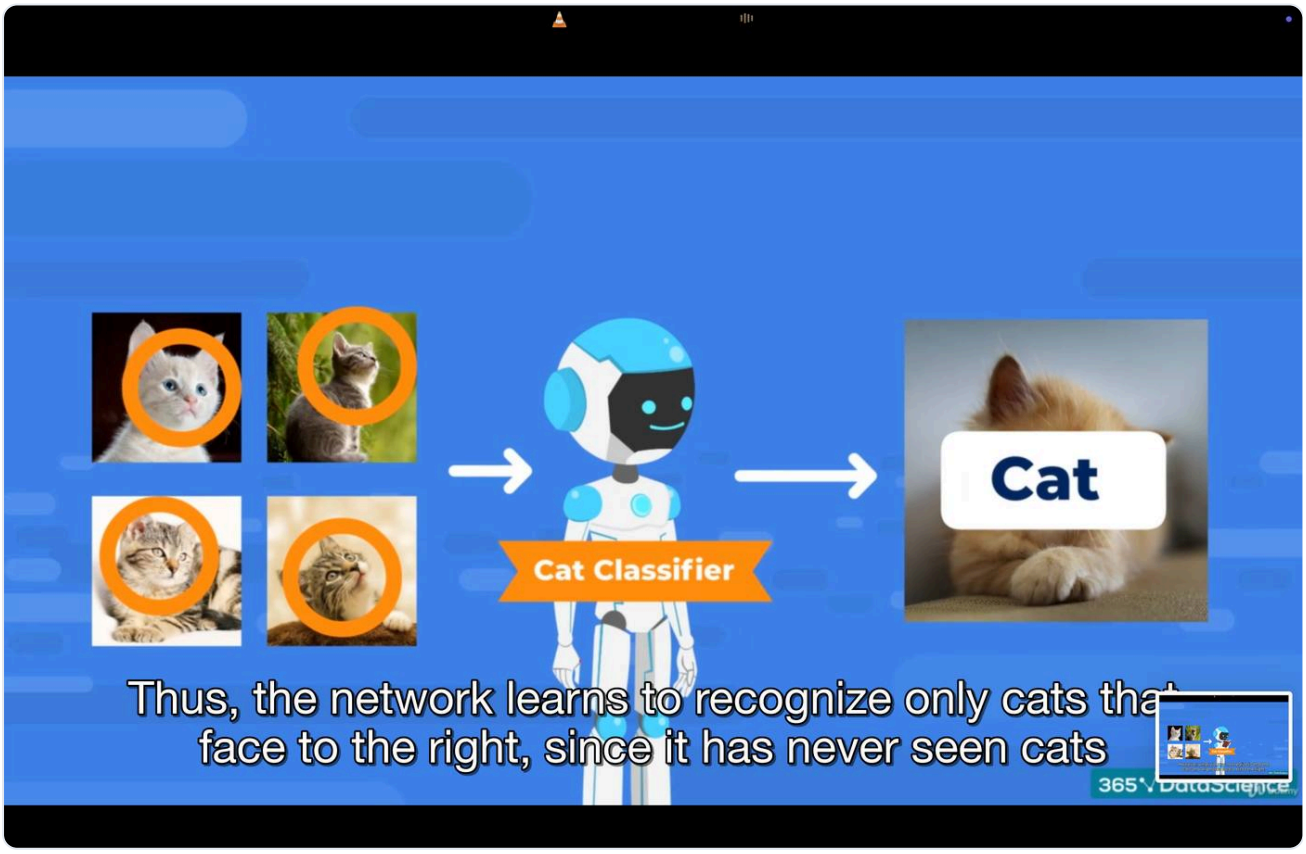

FIGURE 33.4

Thus, the network learns to recognize only cats that face to the right, since it has never seen cats looking to the left.

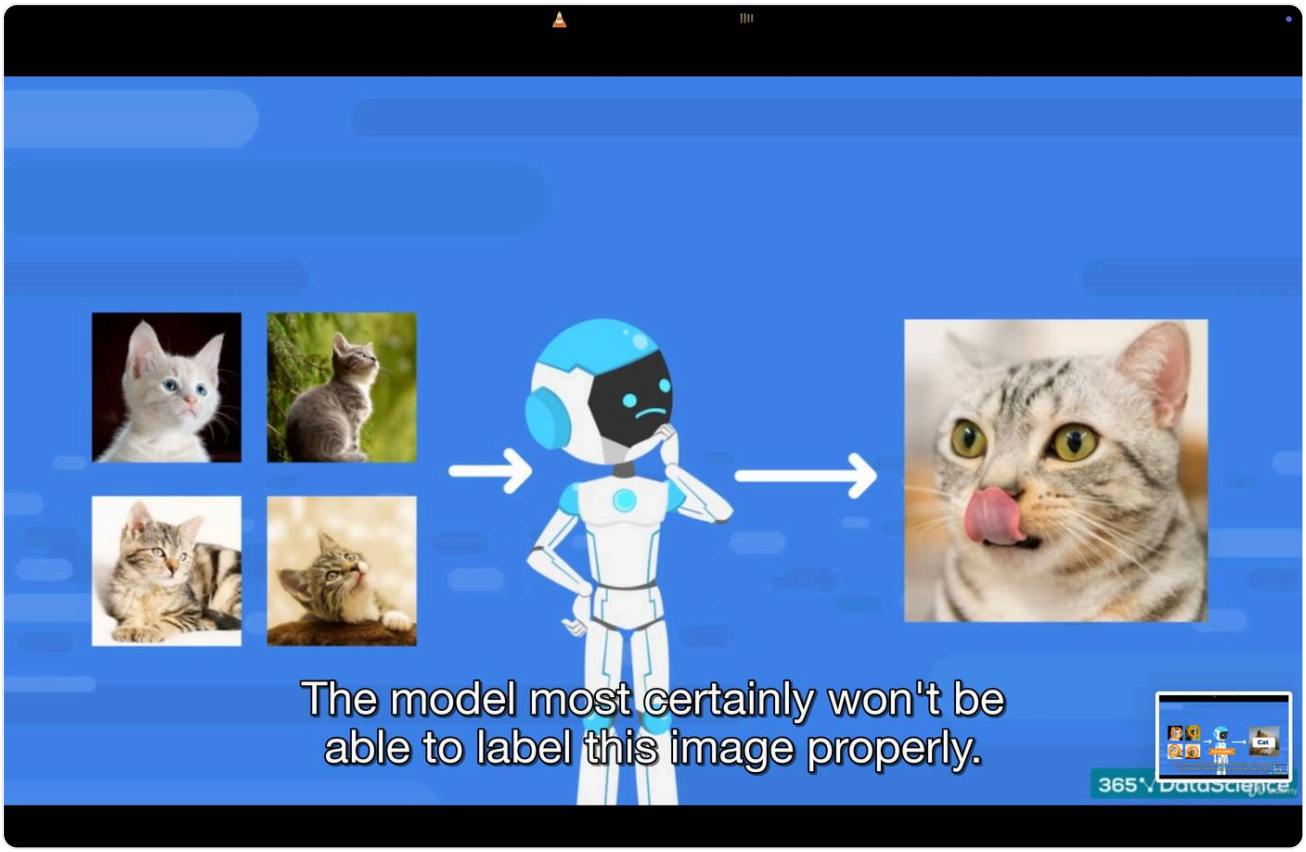

FIGURE 33.5

The model most certainly won't be able to label this image properly.

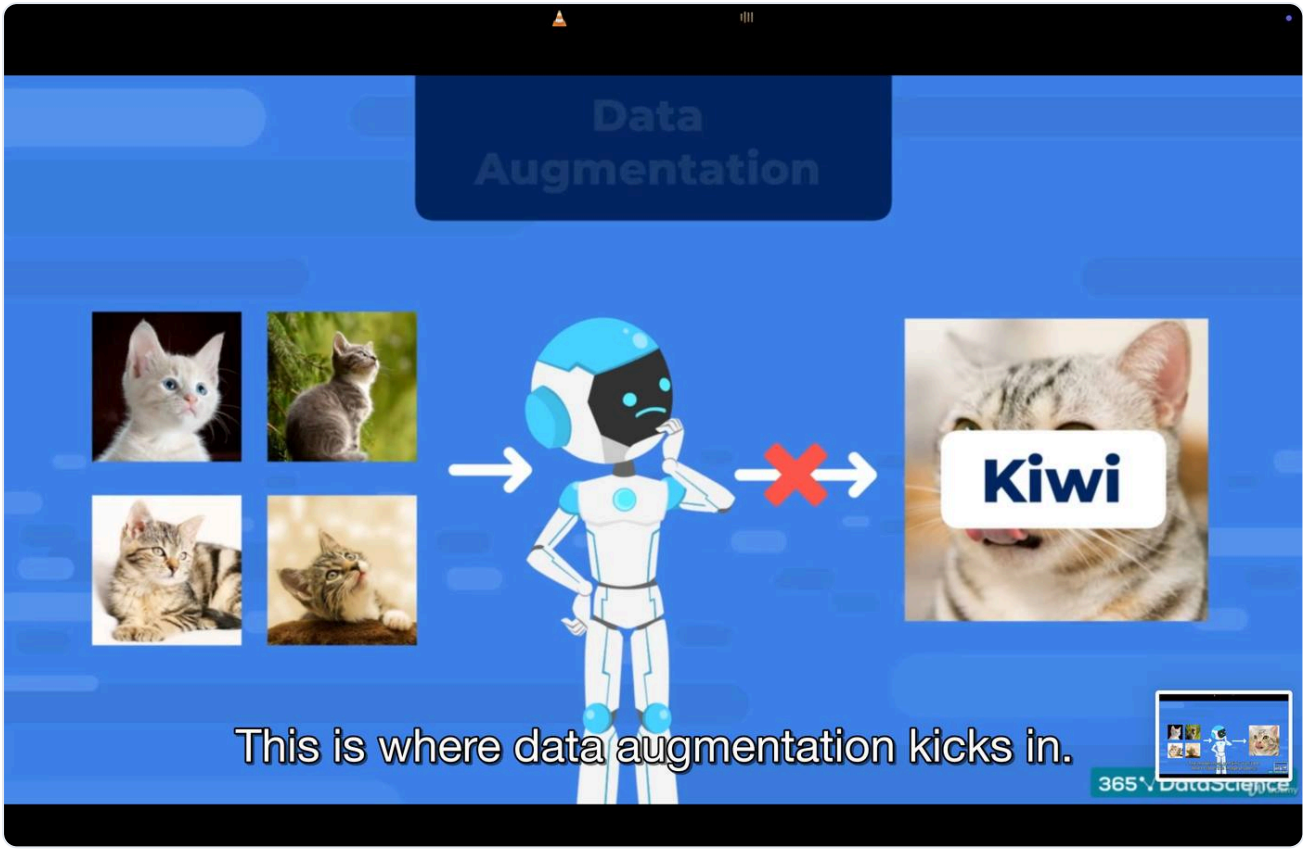

FIGURE 33.6

This is where data augmentation kicks in. The idea behind data augmentation is to change or transform some of the training data points so that there is a greater diversity in the areas we're interested in.

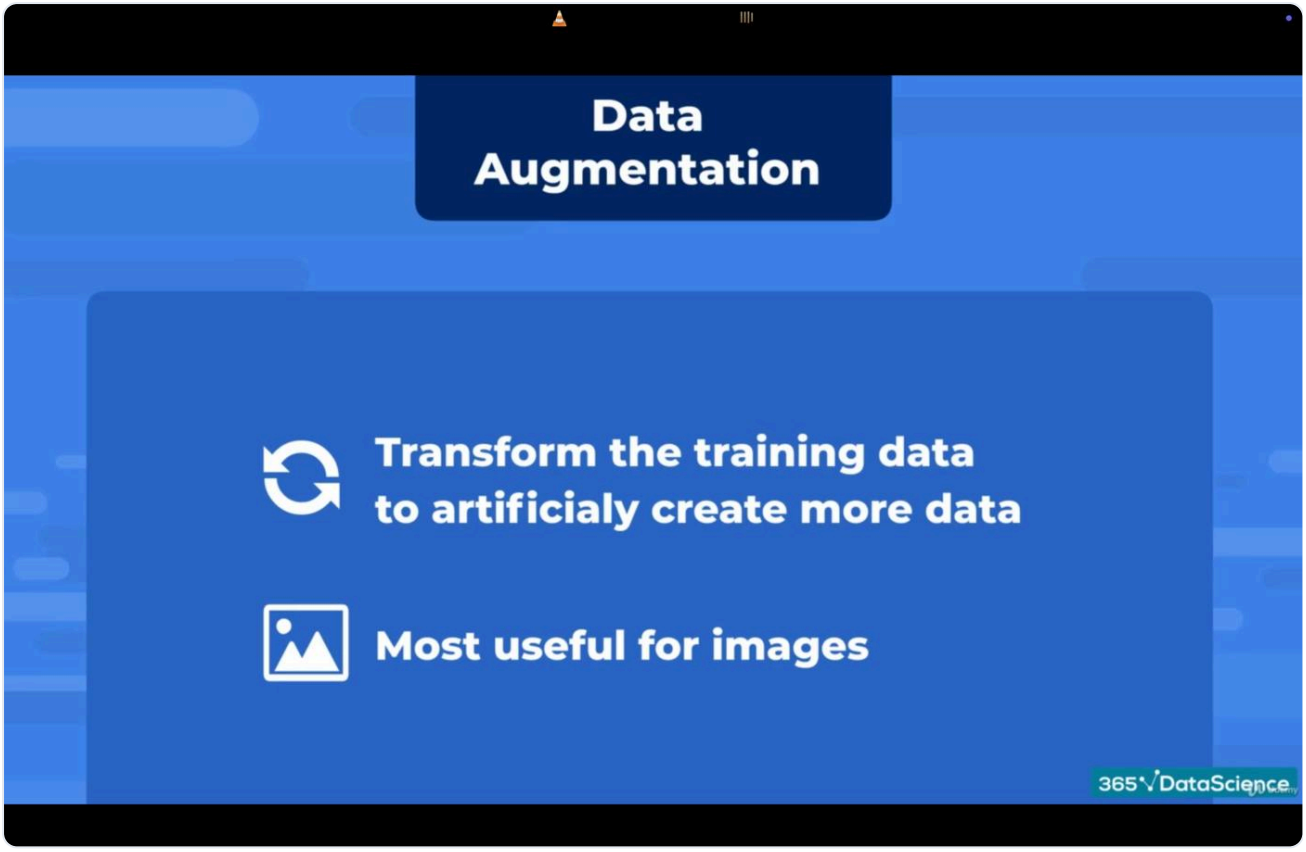

FIGURE 33.7

This technique is most useful for, but not limited to image data. Since that's more or less the topic of our course, we must added to our anend arsenal. OK, what are some data augmentation ideas?

FIGURE 33.8

Well, a common transformation, maybe mirroring the image that would be the technique we can take advantage of in view of our previous cat example, if we take our initial data set and just mirror it, we get additional images of cats that face to the left. Thus the model will be less biased and less likely to be fooled.

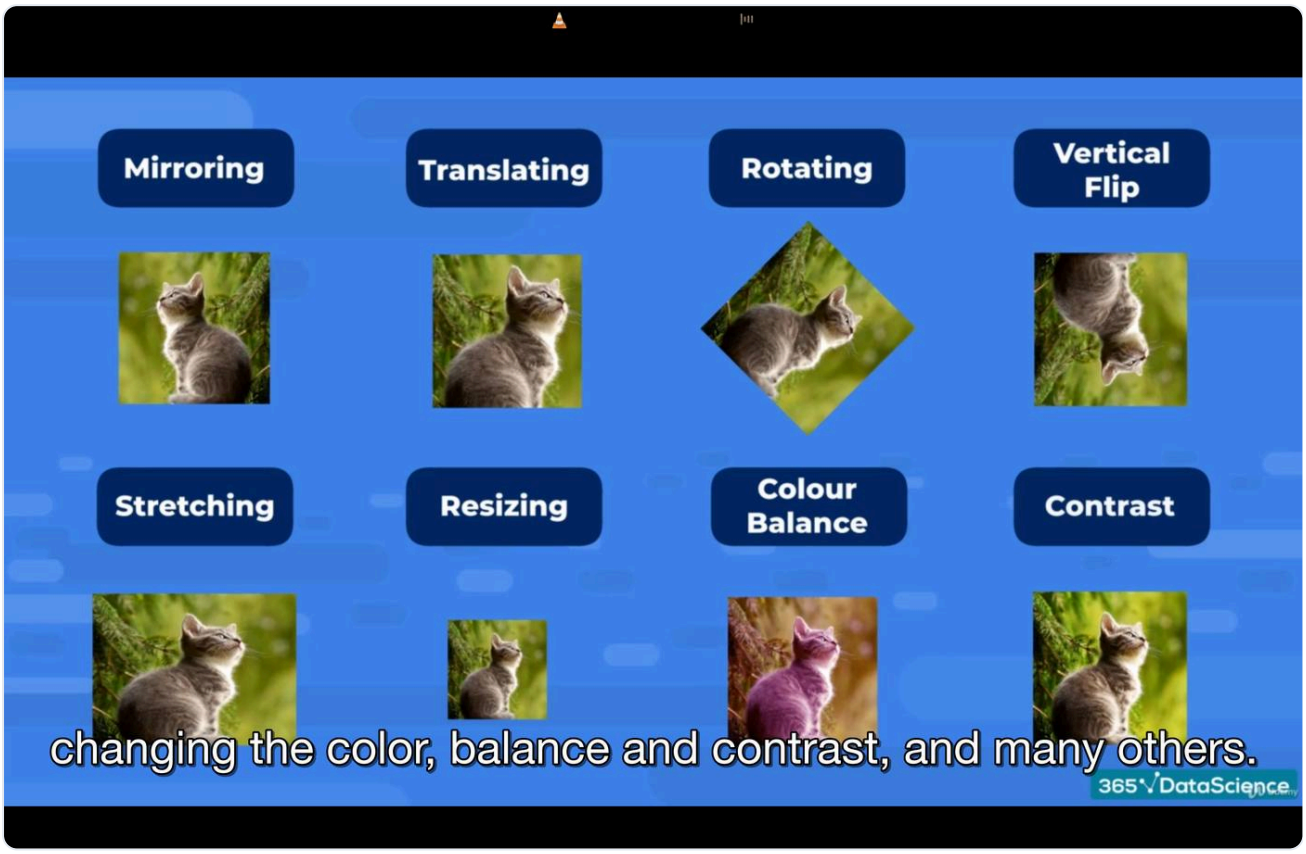

FIGURE 33.9

However, a number of other transformations are also viable, including translating the object of interest in different parts of the frame, rotating it, flipping it vertically, stretching and resizing it, changing the color, balance and contrast, and many others. The moto of machine learning is that everything is problem specific and this applies in full force here.

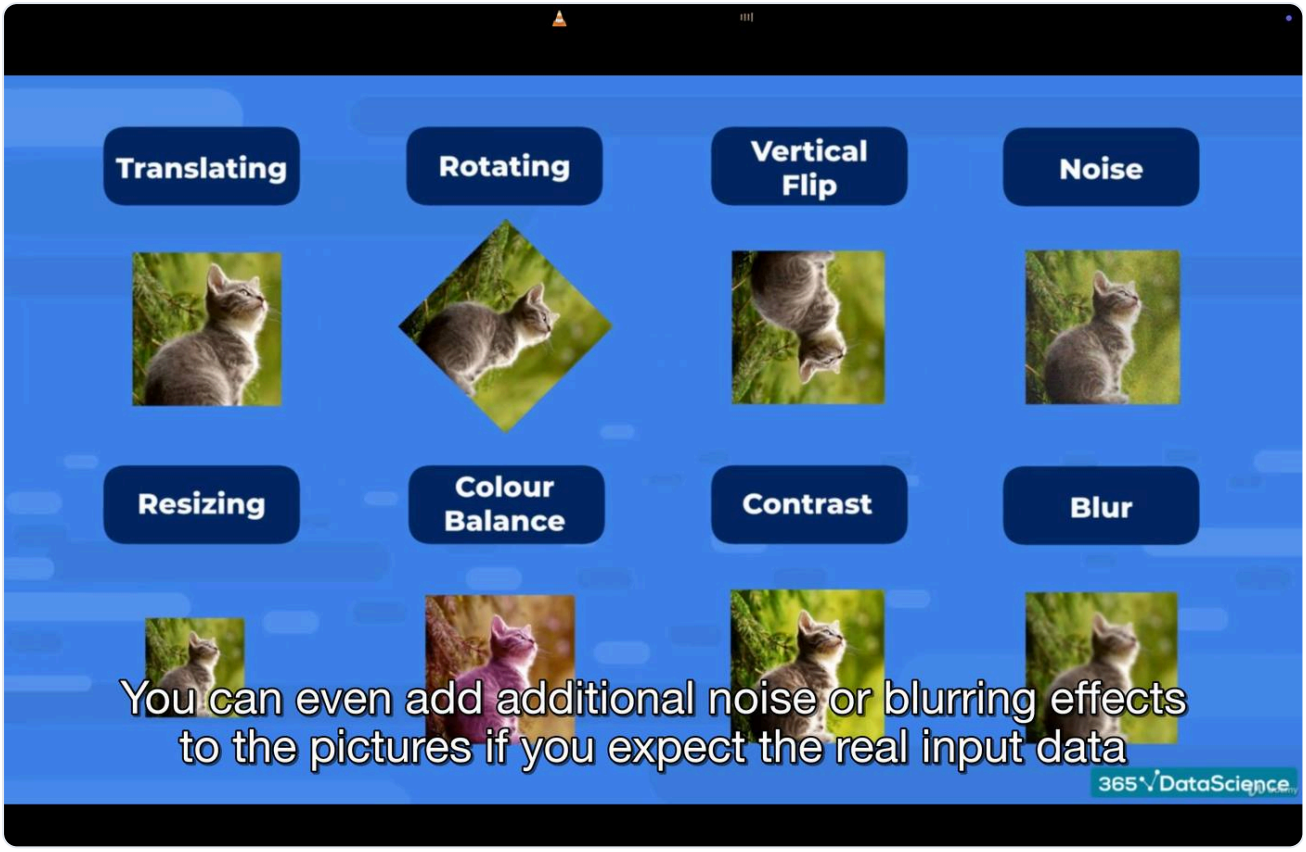

FIGURE 33.10

You can even add additional noise or blurring effects to the pictures if you expect the real input data to be of poor quality. All these techniques depend on what you expect your model to be able to cope with and whether it is present in the training set. Great. It is important to realize that the purpose of data augmentation is not to increase the network's performance.

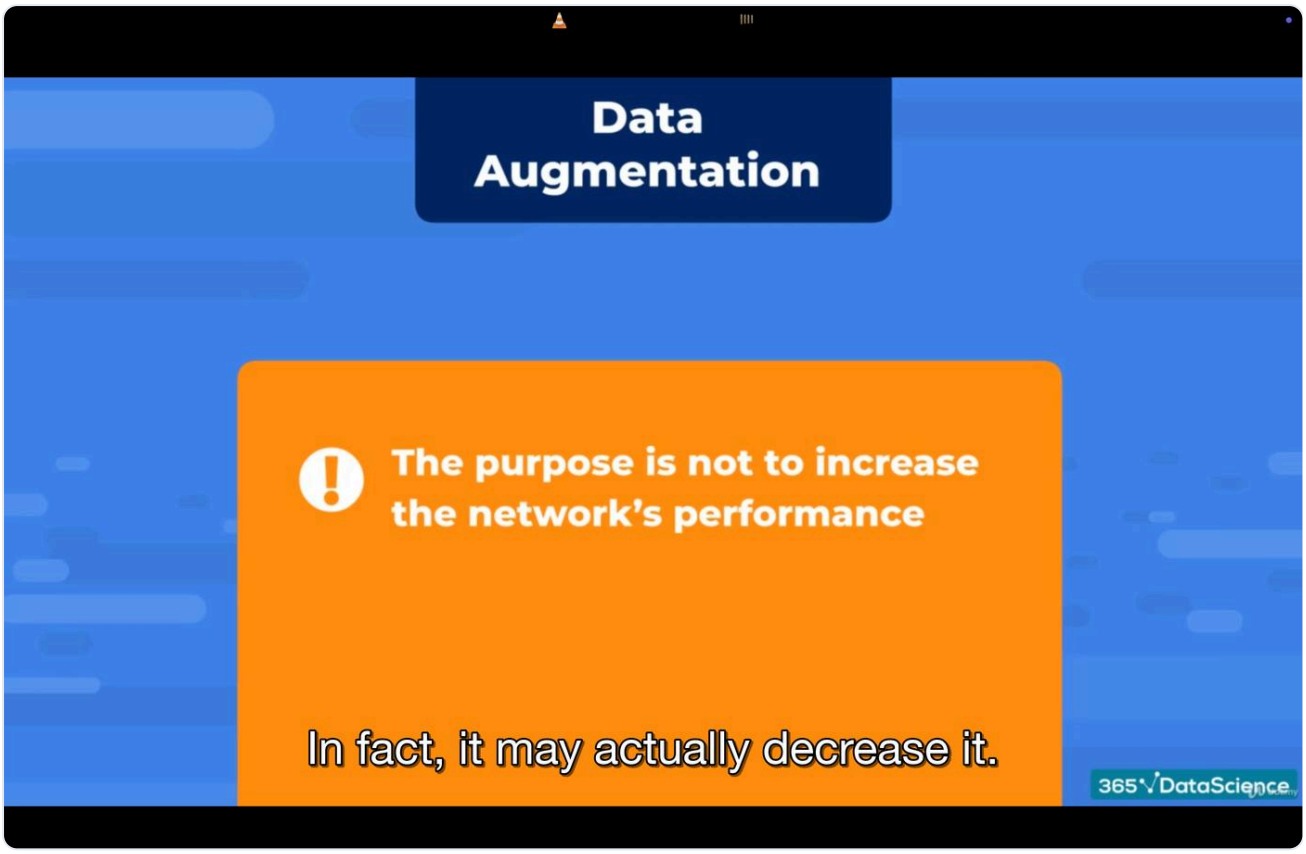

FIGURE 33.11

In fact, it may actually decrease it. Since it generates a new class of examples the model has to recognize.

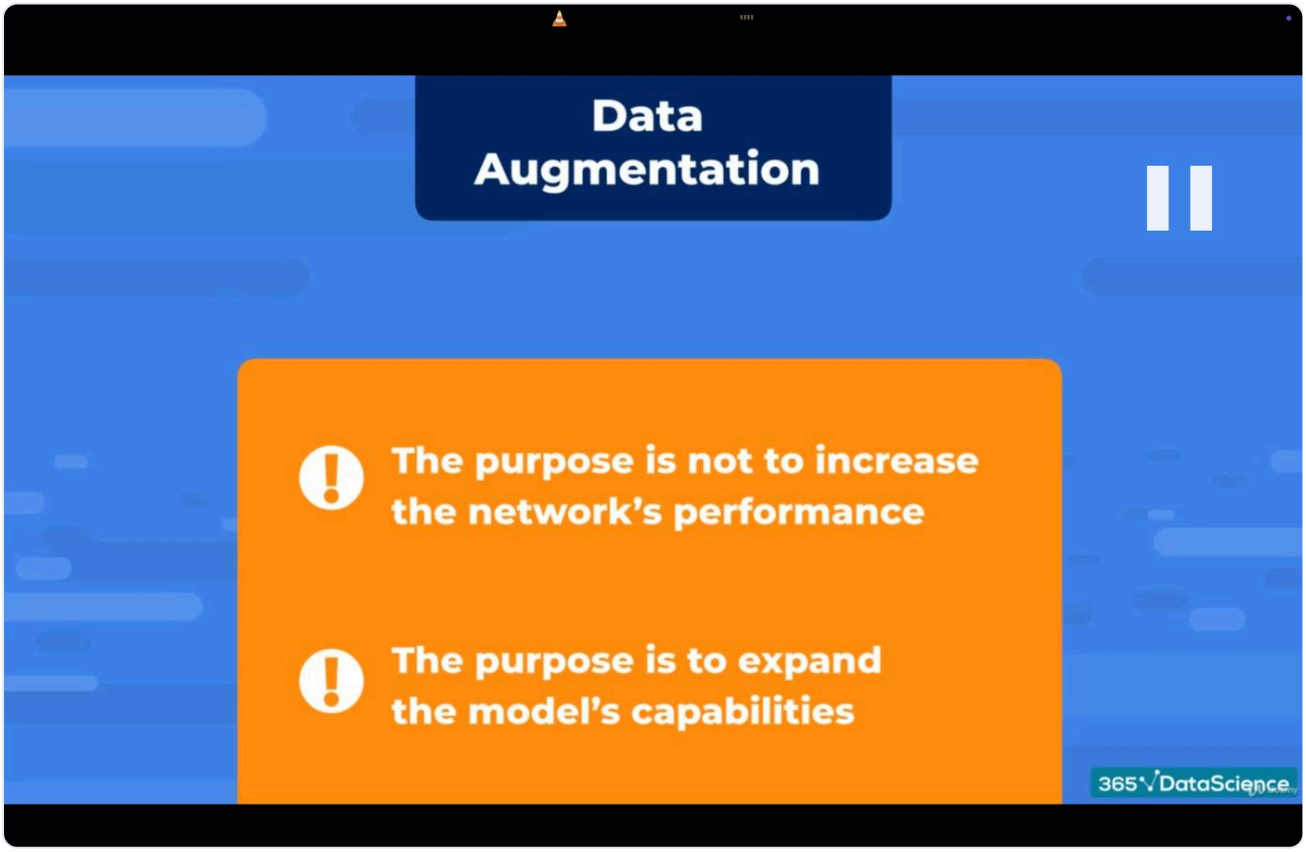

FIGURE 33.12

Its purpose is to expand the models capabilities beyond with a default training set provides and data augmentation is often the easiest way to achieve that, as obtaining a new, richer data set is usually hard. All right, that concludes the section on some of the techniques for improvement of neural networks performance. Bear in mind that the field of machine learning is vast and there are many more ways to do that. Some more specialized than others, however, will examine these three common techniques in a practical setting in the next section. In the next section will also finally tackle a real world practical example, a business case that will give you a better idea of how things actually turn out.

So without further ado, let's jump straight into the next section. Thanks for watching.

LECTURE 34

Products and Online Platform

So far, we have implemented convolutional neural networks only in, let's say, laboratory environment, however, real world projects and situations are not as straightforward as the amnesty example. There are lots of obstacles along the way and the results are not always satisfactory or what we want them to be.

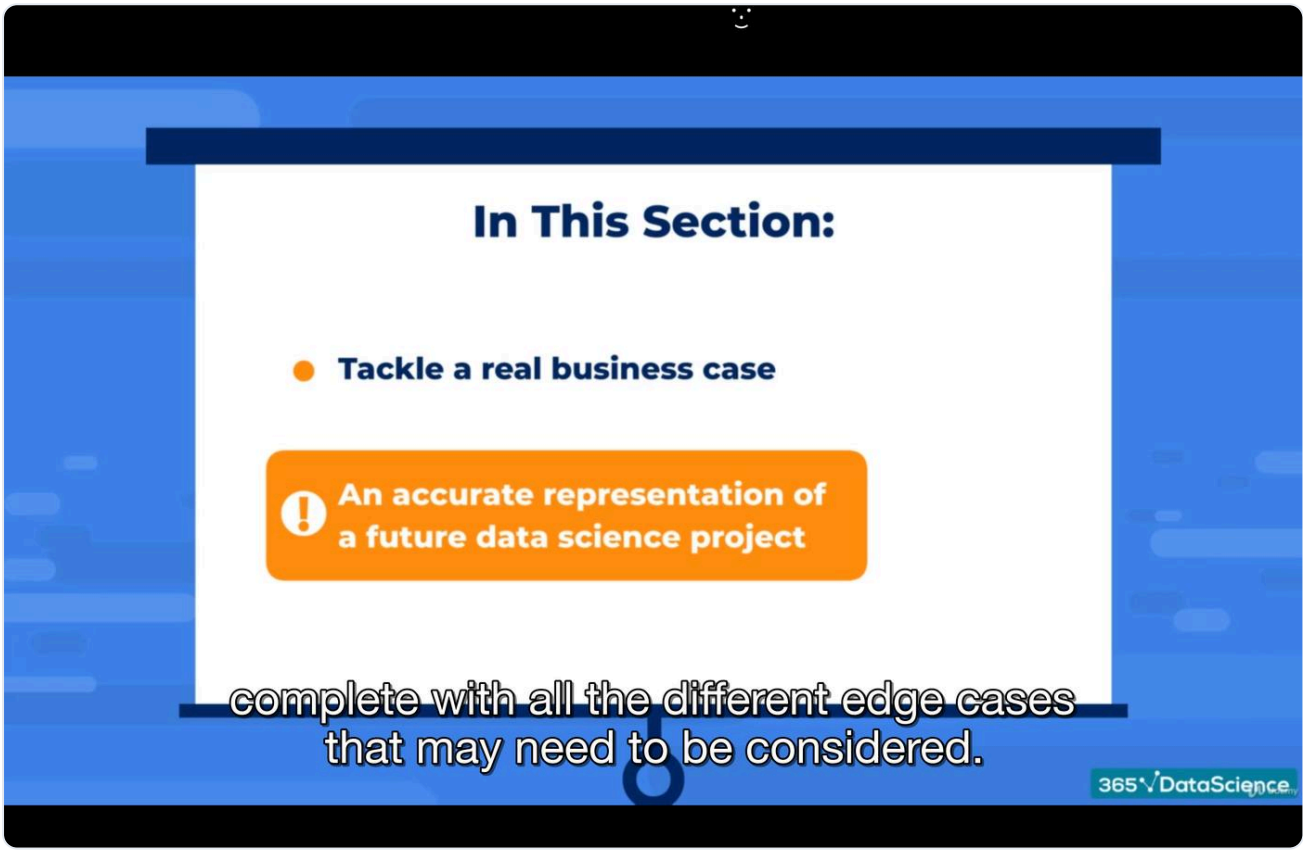

FIGURE 34.1 | In This Section:

That is why in this section we are going to tackle a real business case, although it isn't 100 percent complete with all the different edge cases that may need to be considered. It's still perfectly represents a project you'd have to tackle at some point in your career as a data scientist. OK, so here is the situation. Imagine you are a fashion designer. Your brand has so far created thousands of models of different shirts, shoes, trousers, accessories, et cetera. And now you've decided it's time to open an online version of your shop. So you need to take photos of all the products, upload them to your online platform and label all the goods. Here lies the problem.

Let's say you have the names of your products and maybe even the photos ready and available. But do you have all these products labeled with the specifics your online platform demands? Probably not. Moreover, going forward, you definitely intend to add more and more items to your shop and to go through thousands of photos and manually label them would take too much time and effort.

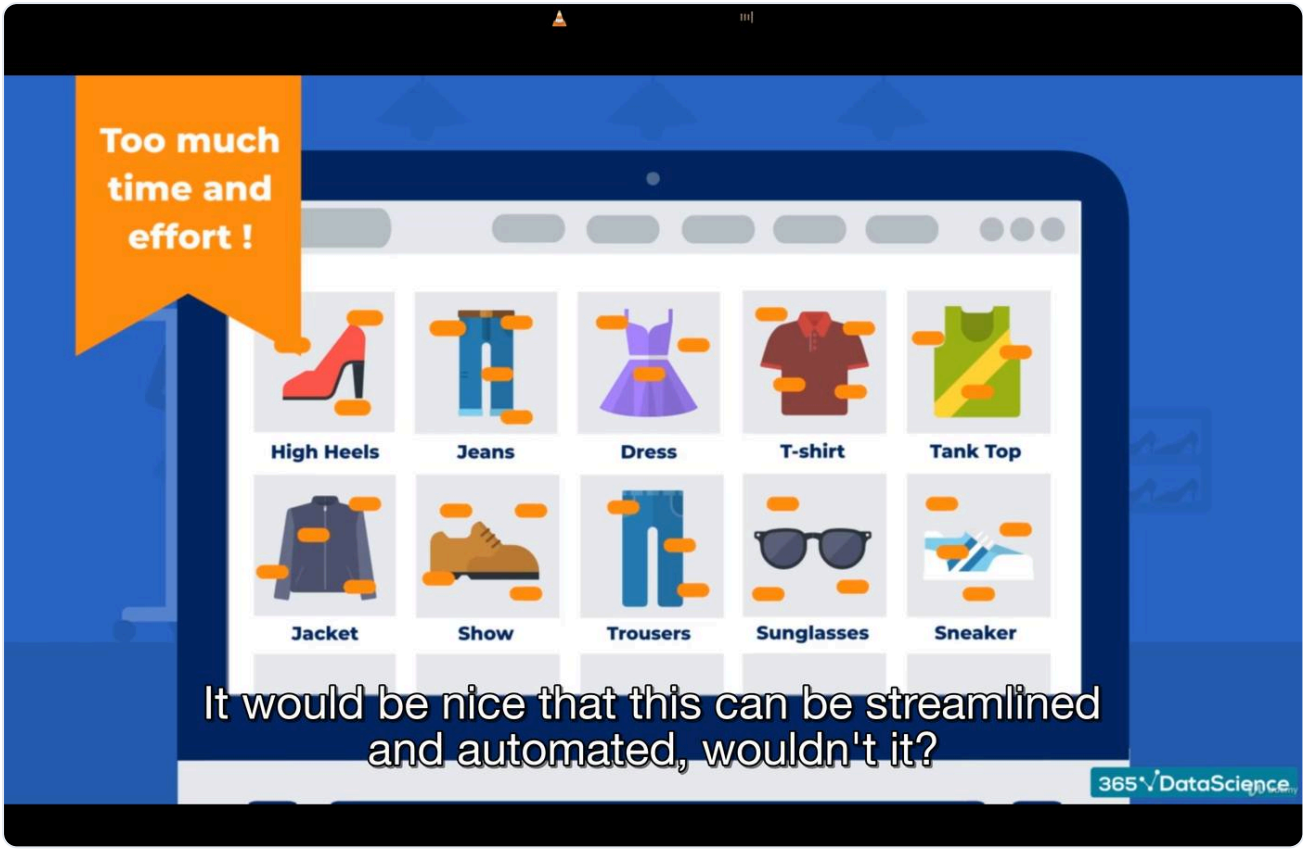

FIGURE 34.2

It would be nice that this can be streamlined and automated, wouldn't it? I think you know where I'm going with this, but that's not all. Here is another situation. Picture this. You have a website where people can upload and sell their own clothes. Thus it is not your own products that are up on the platform.

FIGURE 34.3

But any person can join and upload photos of what they want to sell to benefit the people who are searching to buy.

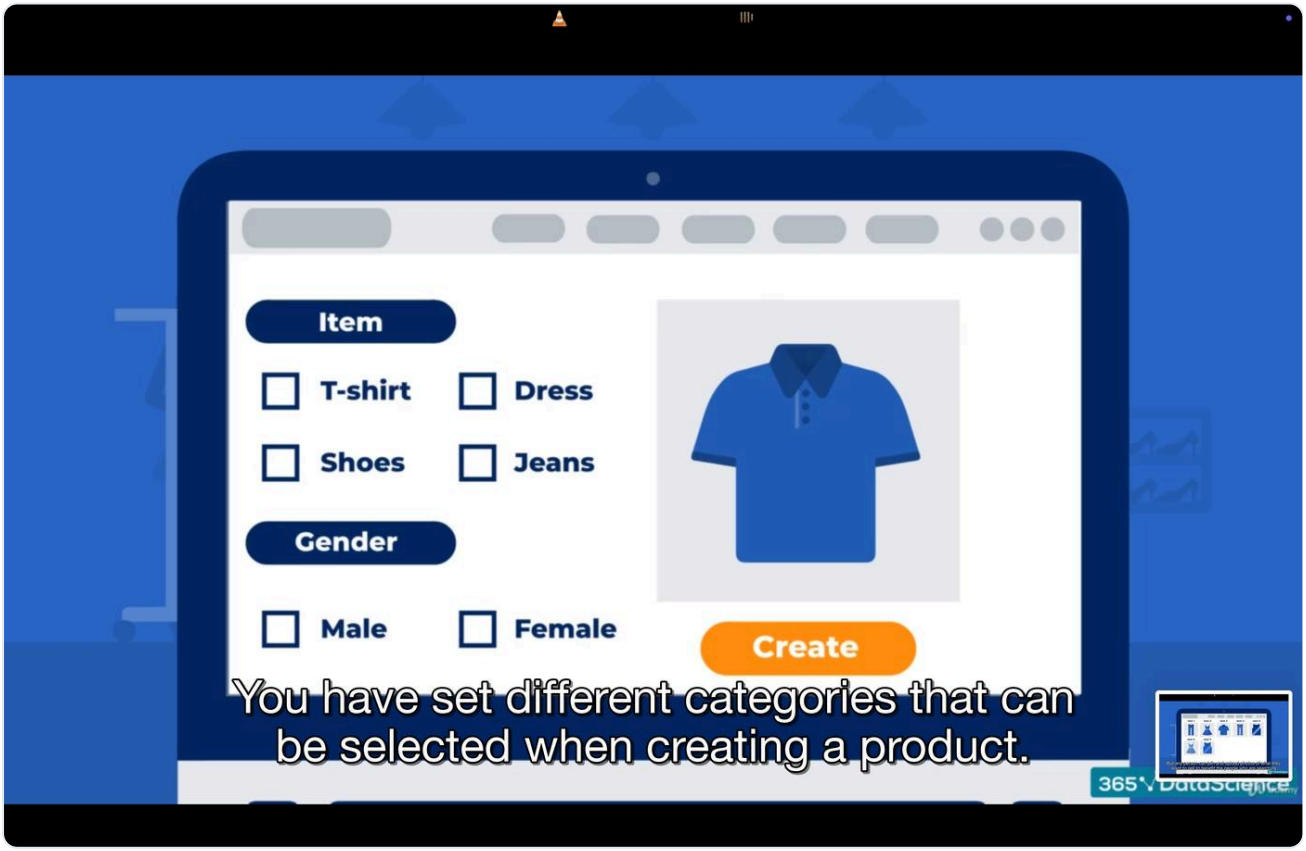

FIGURE 34.4

You have set different categories that can be selected when creating a product. Is it a shoe or a T-shirt, male or female and so on. At first it all seems fine. All the products are labeled and are easy to search. However, you soon begin to notice that some people don't take the time to categorize their products. You shrug it off. Well, that's not my problem after all. It's their a product that won't be found and sold.

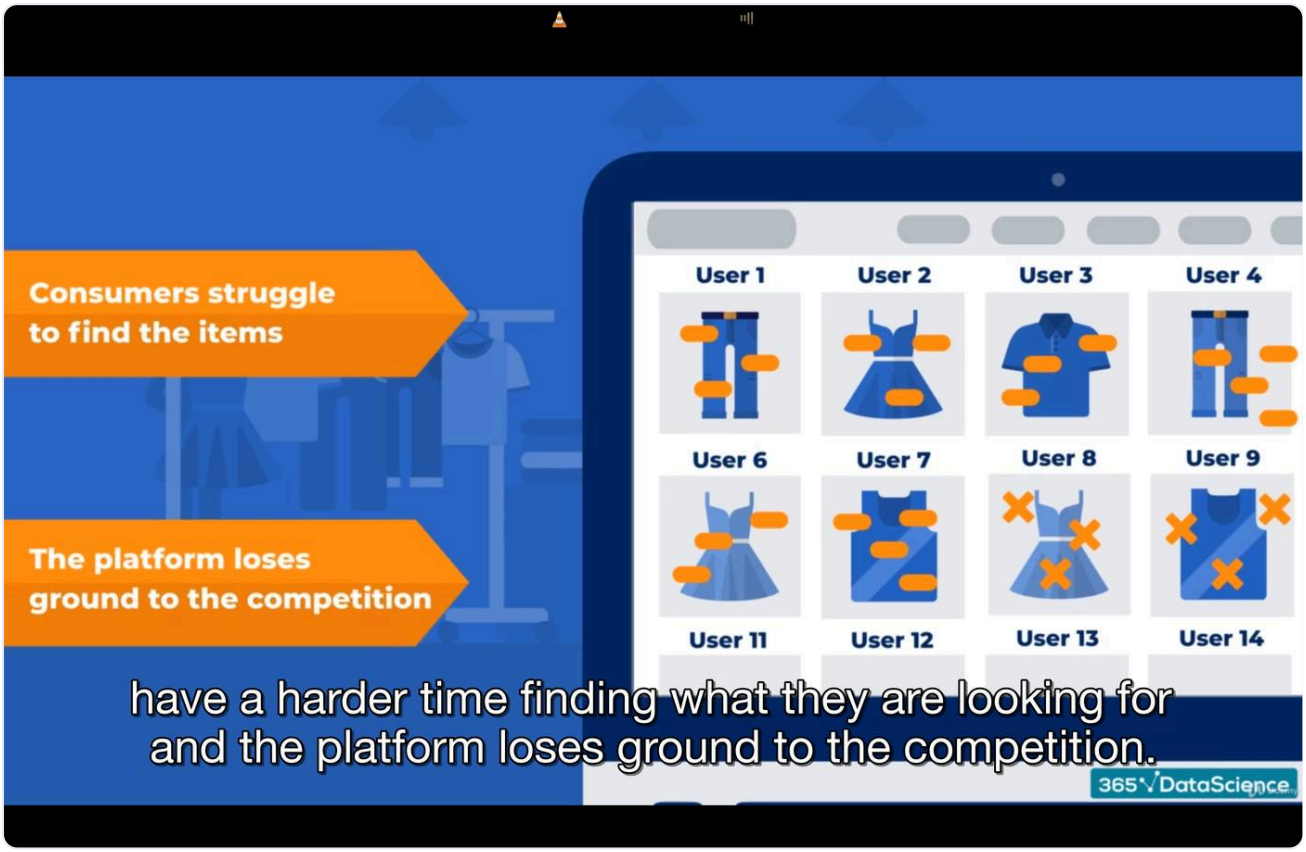

FIGURE 34.5

With a little bit more thought, though, you realize that this actually hurts you a lot as the consumers have a harder time finding what they are looking for and the platform loses ground to the competition. So you decide to create an automatic system to assign labels whenever they are missing or wrong.

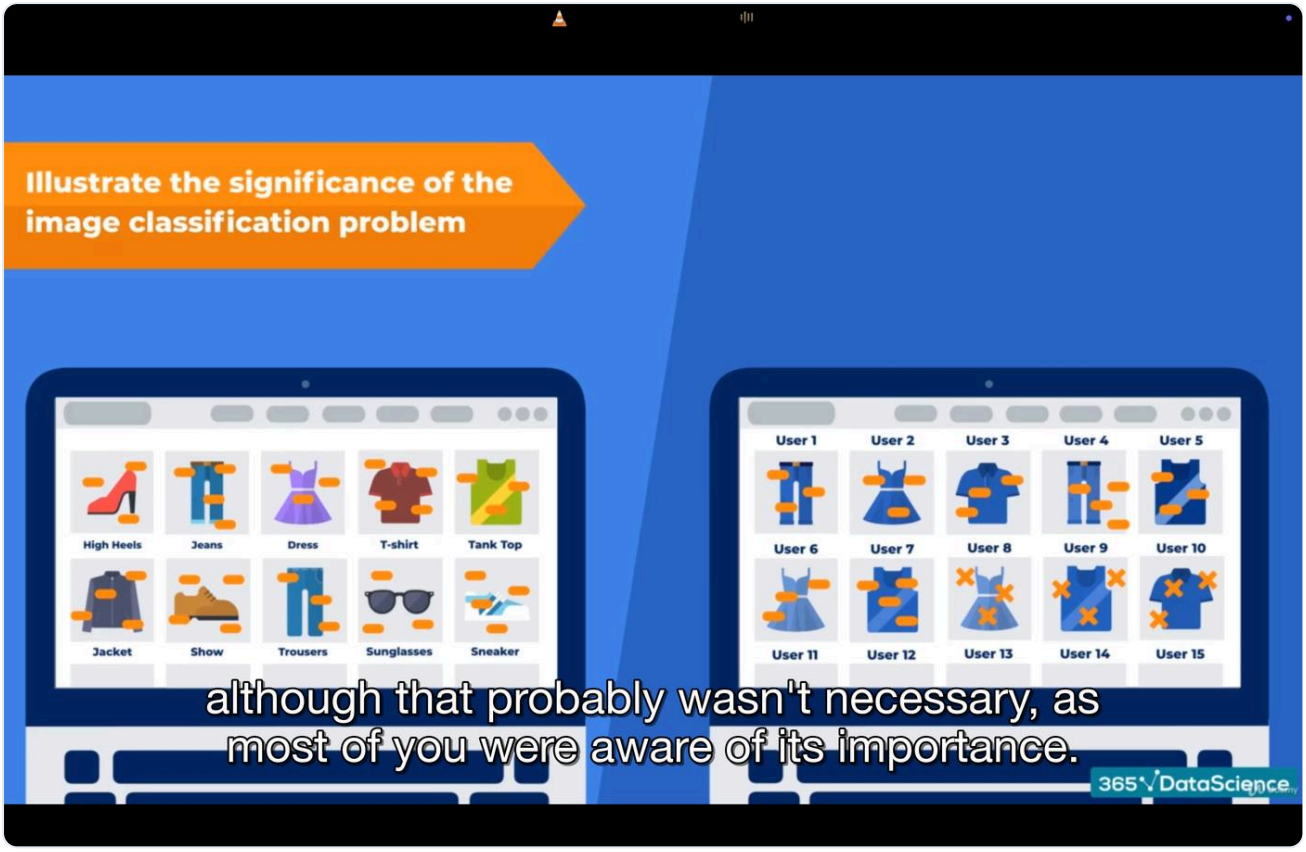

FIGURE 34.6

Those two hypothetical situations illustrate the significance of the image classification problem, although that probably wasn't necessary, as most of you were aware of its importance.

FIGURE 34.7

However, they also serve another purpose to provide real world use cases. For our practical example. That's right. In this section, our job would be to create scenes that classify and label images of clothes and other fashion items.

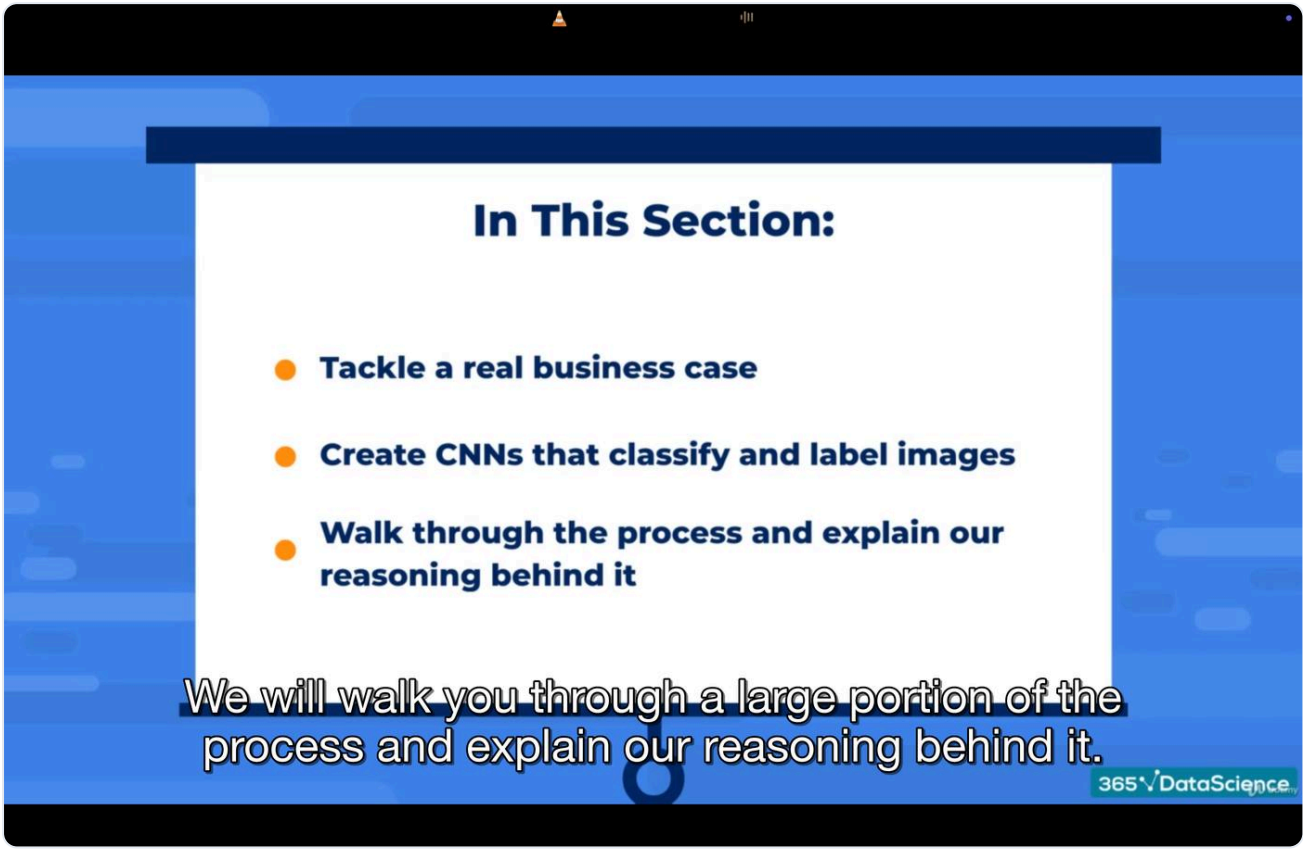

FIGURE 34.8 | In This Section:

We will walk you through a large portion of the process and explain our reasoning behind it. Exciting, right. In the next lecture, we are going to discuss the data sets and the objective in more detail. Stay tuned. And thanks for watching.

LECTURE 35

Data Set and Shoes

In this video, we're going to discuss the data set that we will be using and our objectives for this practical example. All right. So as we've mentioned in the last lecture, we are going to train a CNN to recognize and label different fashion items in order to train a network, though we need a data set. And the data set is a crucial point here. It dictates how the networks are going to be structured and what they will learn when considering clothing items. There are generally two types of images, images that show a fashion model wearing the particular item and images that show only the clothing item or accessory.

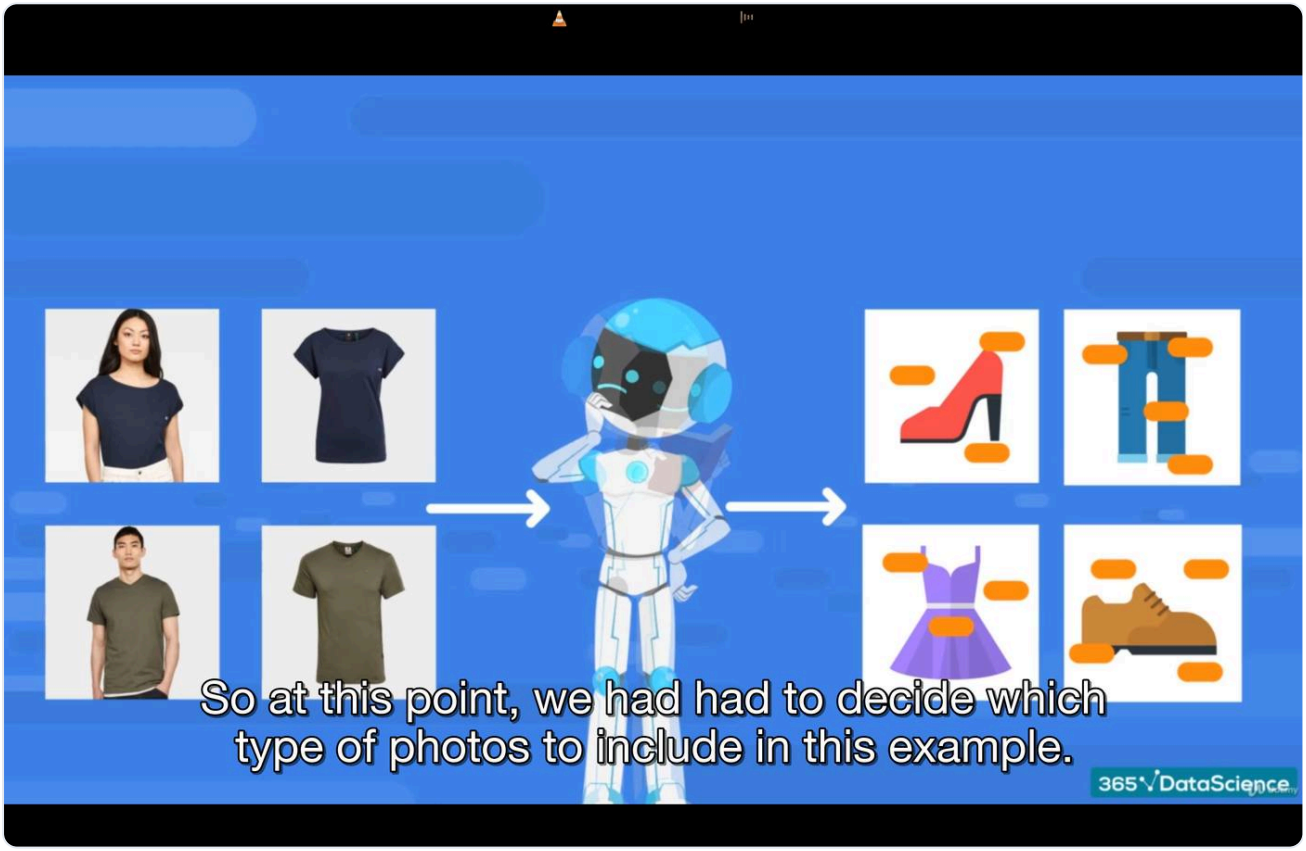

FIGURE 35.1

So at this point, we had had to decide which type of photos to include in this example.

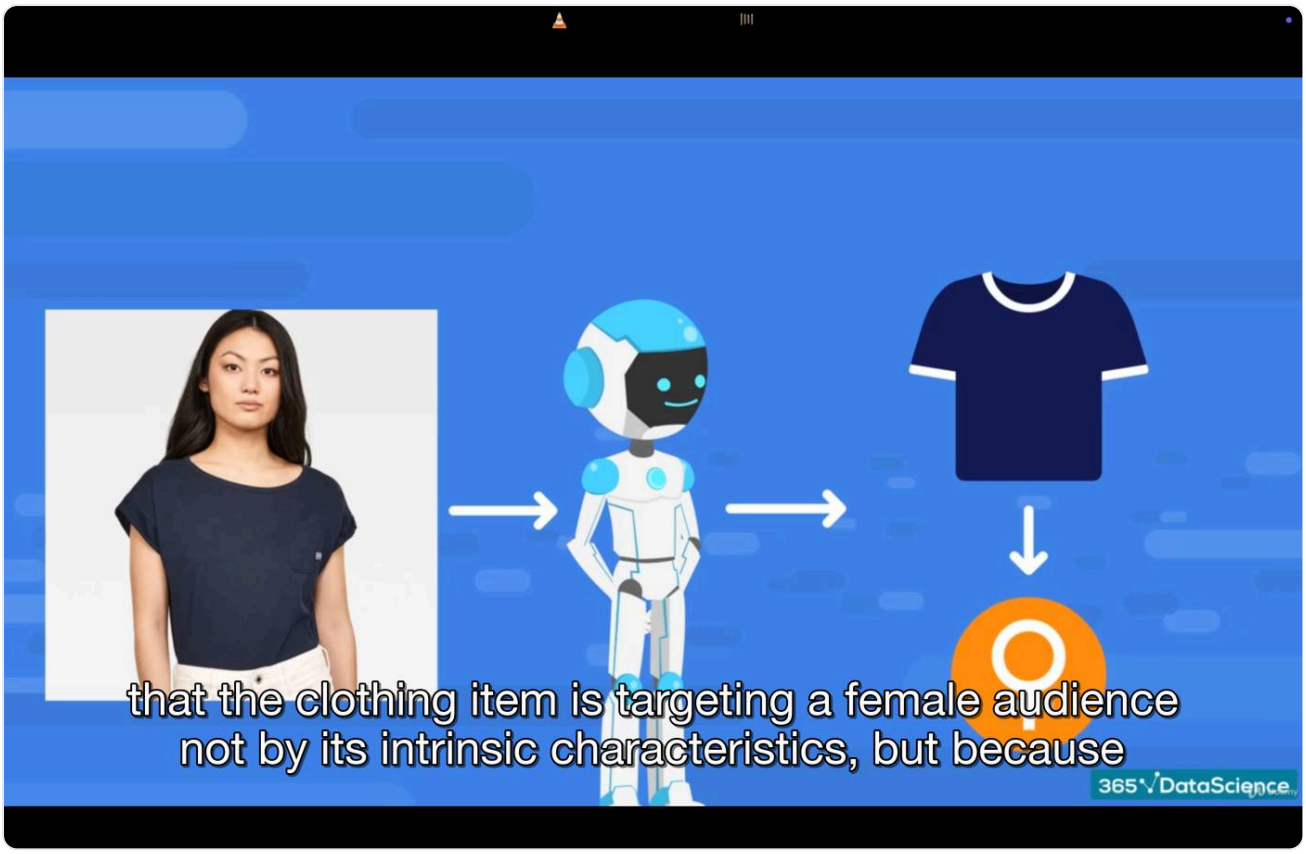

FIGURE 35.2

When considering the images with a fashion model, we imagine that the networks may learn to understand that the clothing item is targeting a female audience not by its intrinsic characteristics, but because the model wearing it is a woman. In the same way that CNN may learn that when only the top part of a male model is visible, the image shows a t shirt.

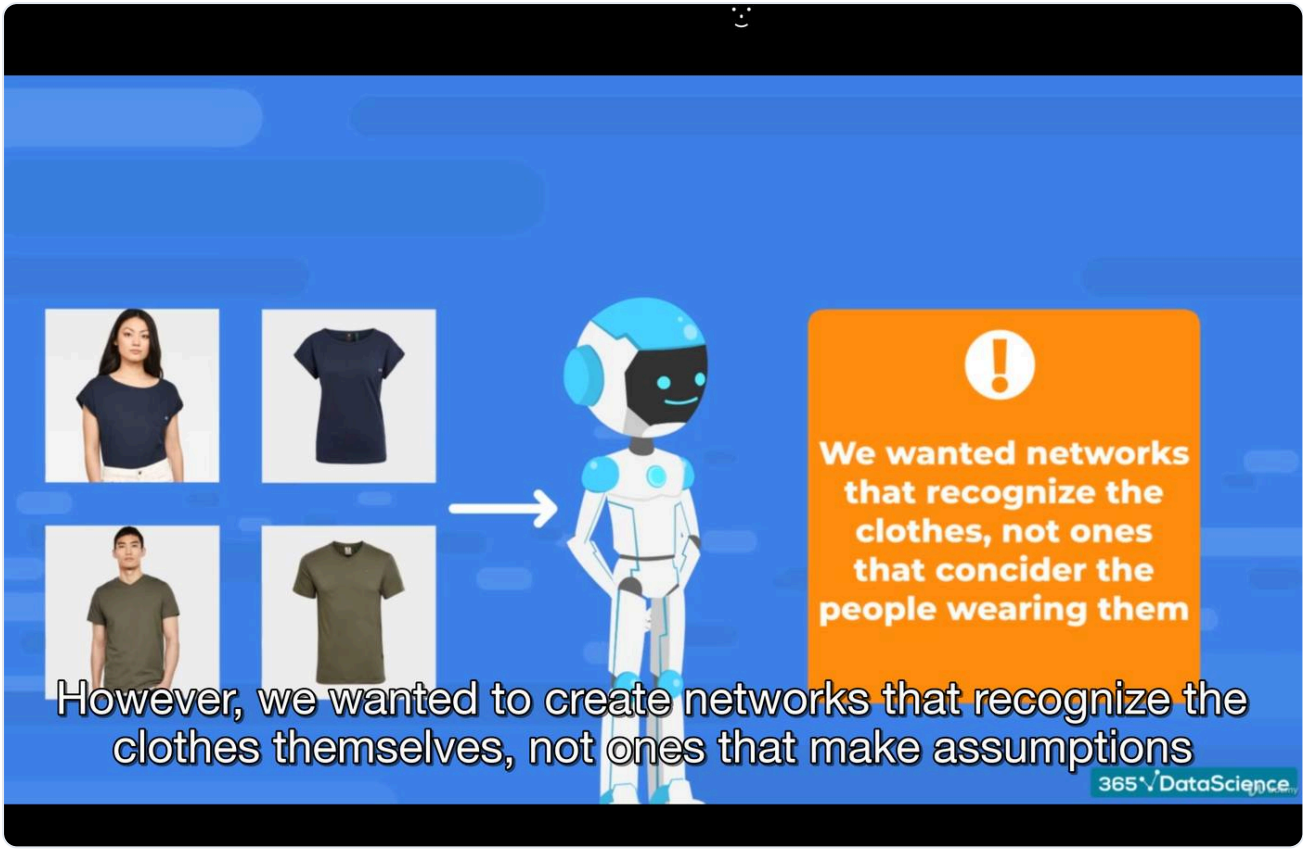

FIGURE 35.3

However, we wanted to create networks that recognize the clothes themselves, not ones that make assumptions based on the people wearing them. So we opted for the second type of images, those that only show the item framed against a consistent background.

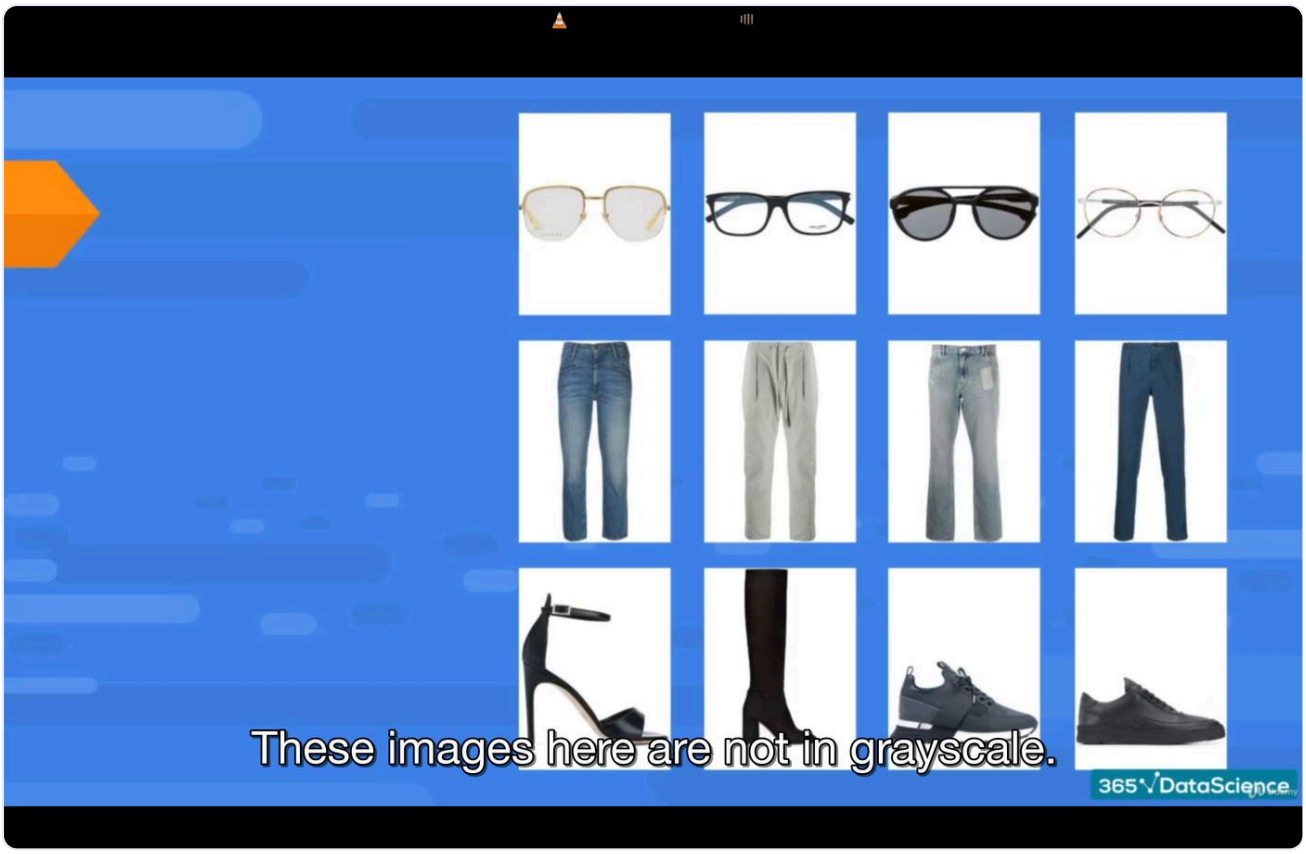

FIGURE 35.4

OK, we have created the data set ourselves by collecting such photos and labeling them. And here are the images we extracted.

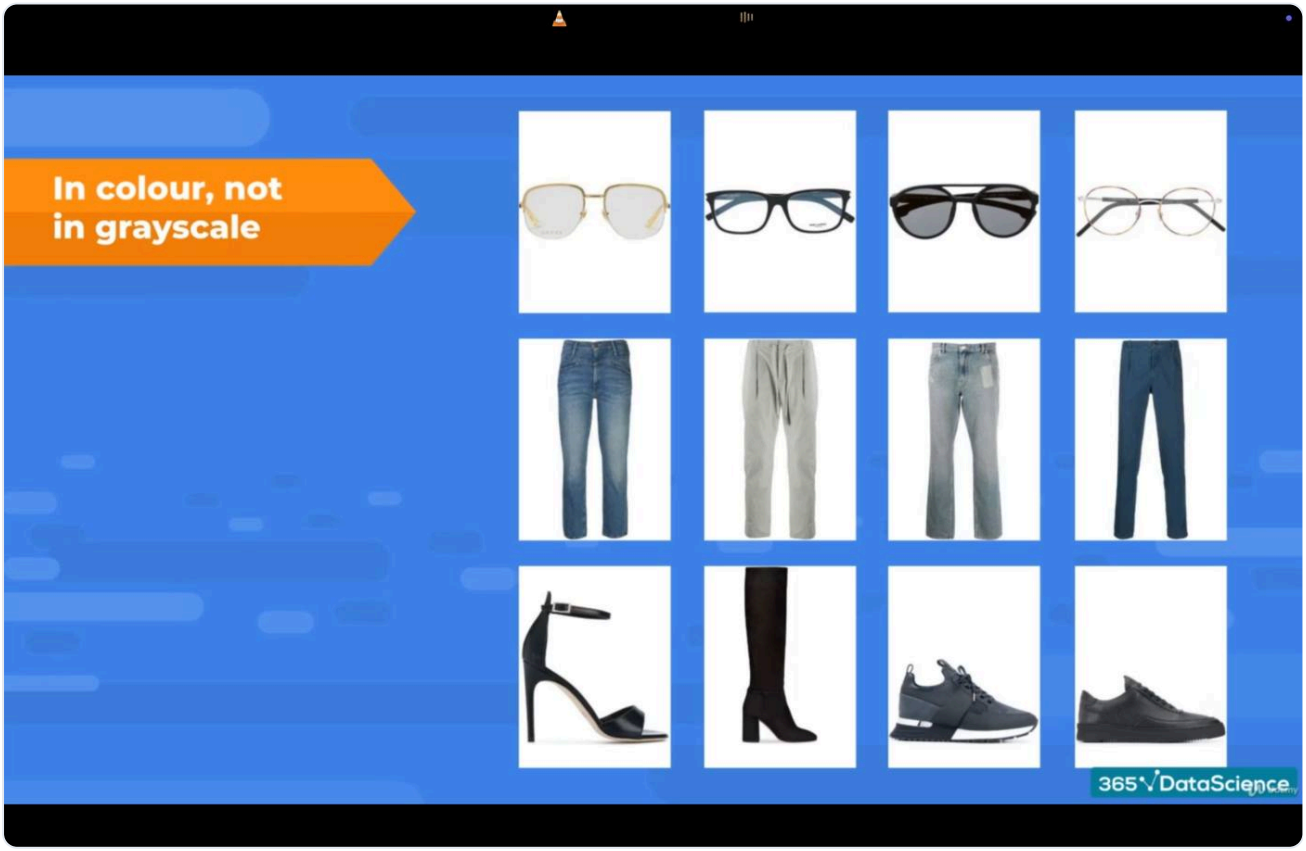

FIGURE 35.5

In contrast to the amnesty example.

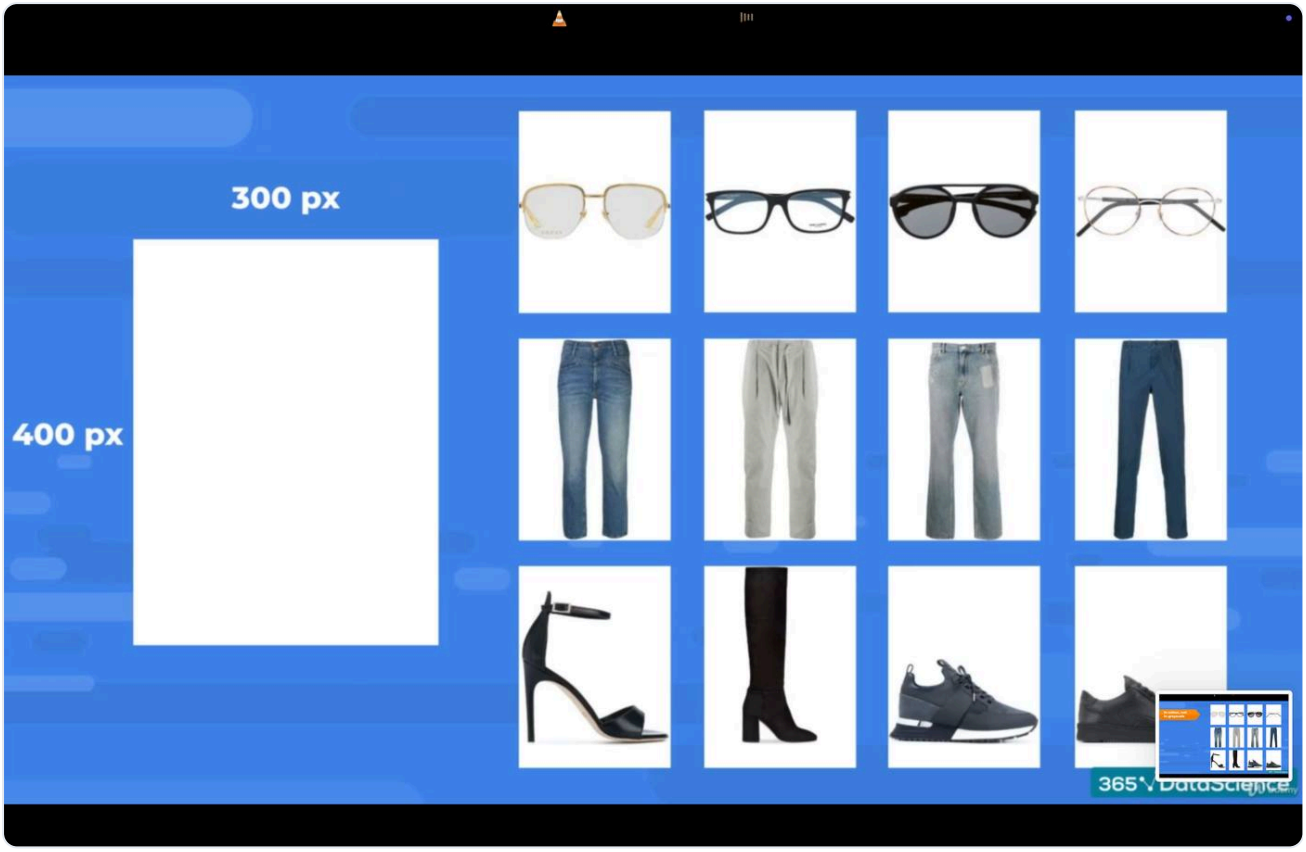

FIGURE 35.6

These images here are not in grayscale. They are in color.

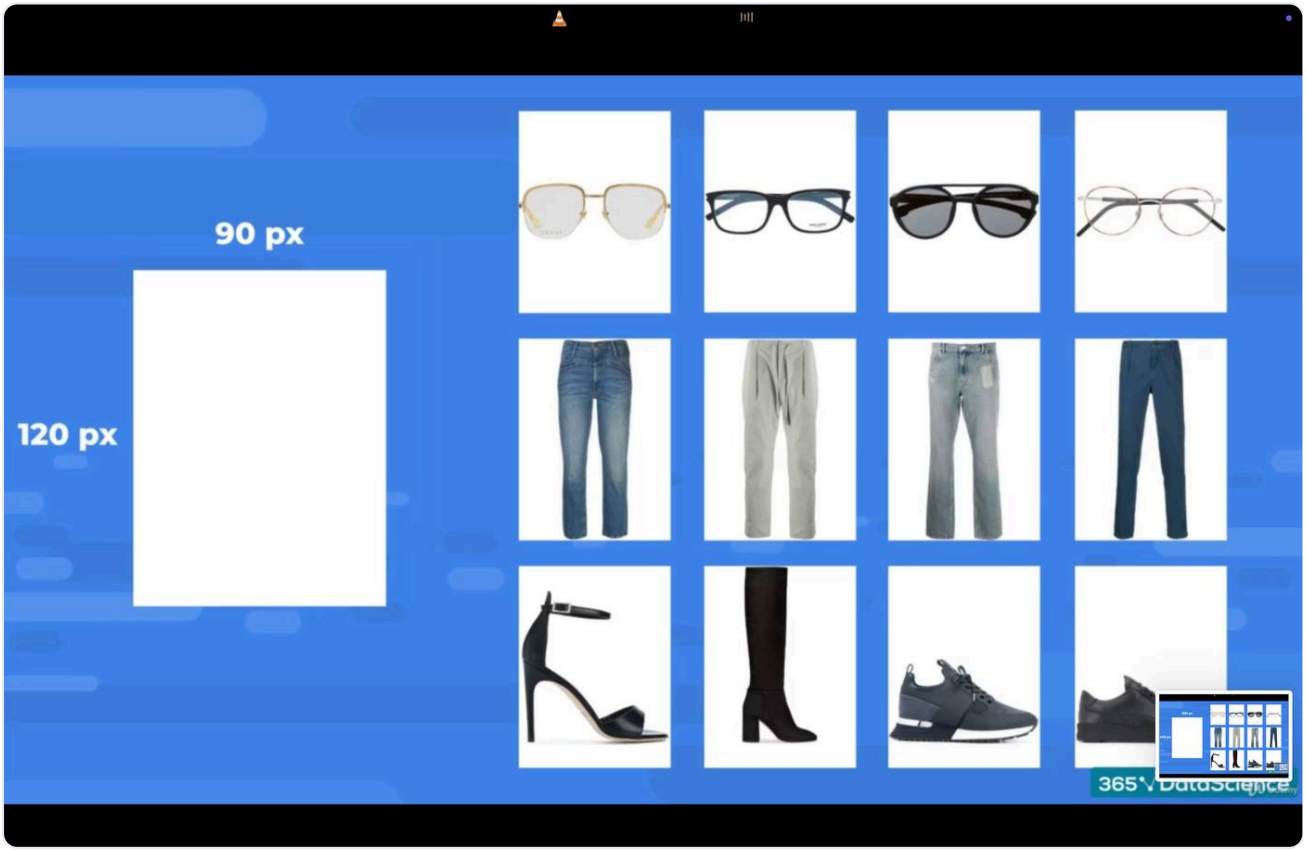

FIGURE 35.7

Each of them has dimensions of 300 by 400 pixels. However, for this practical example, we have scale them down to 90 by 120.

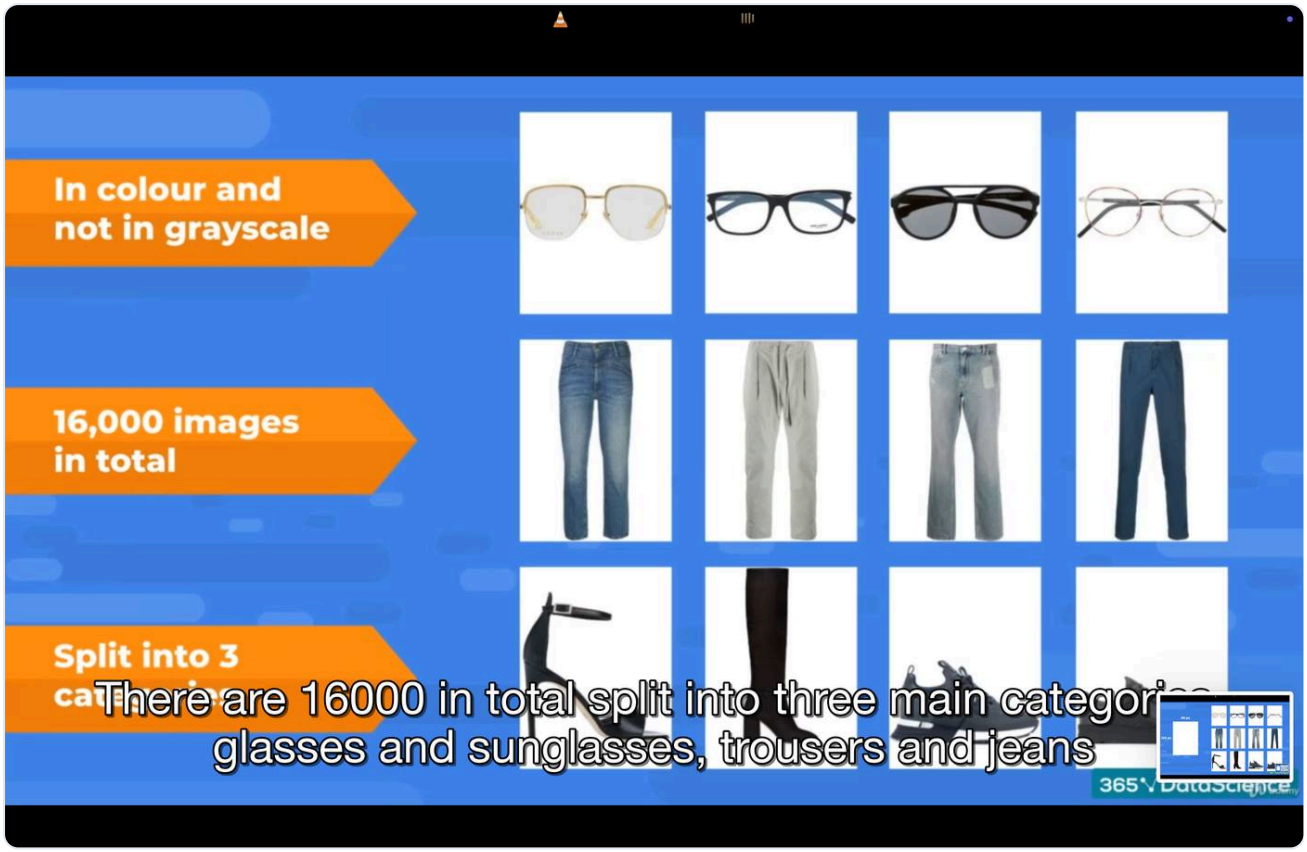

FIGURE 35.8

There are 16000 in total split into three main categories glasses and sunglasses, trousers and jeans and shoes. Since our goal is to learn to label the different items in a useful manner.

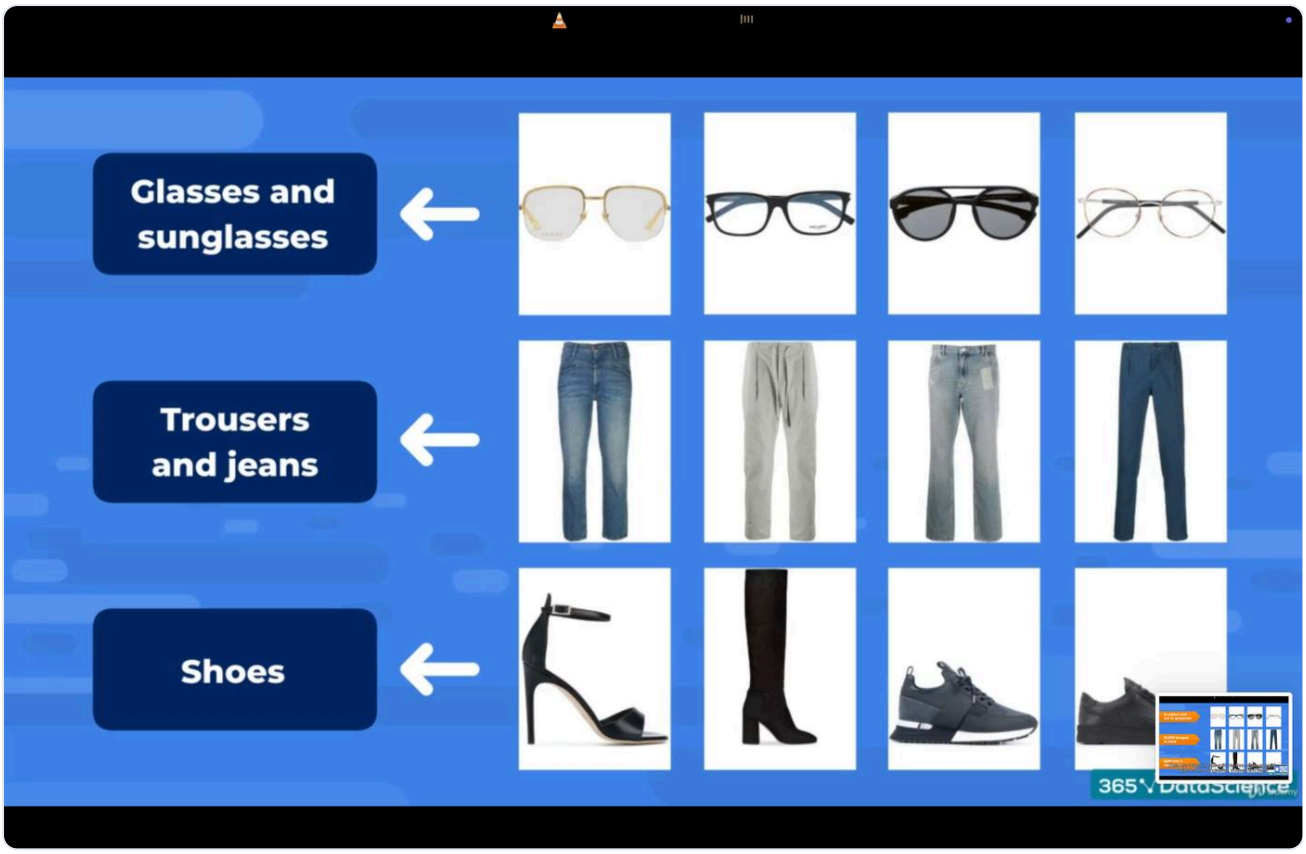

FIGURE 35.9 | Glasses and

We need more than just those three labels. That's why every class has additional subclasses. The glasses and sunglasses only have one subclass.

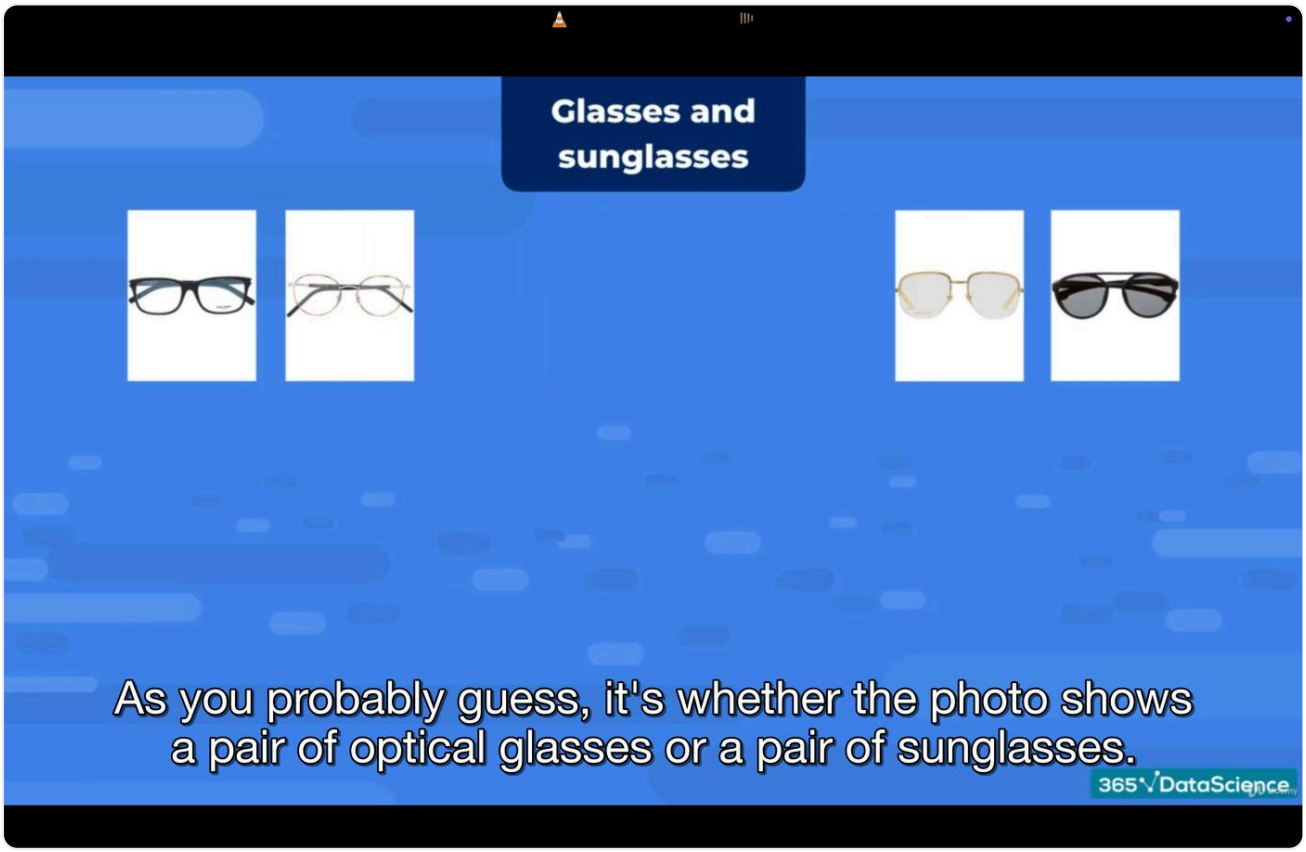

FIGURE 35.10 | Glasses and

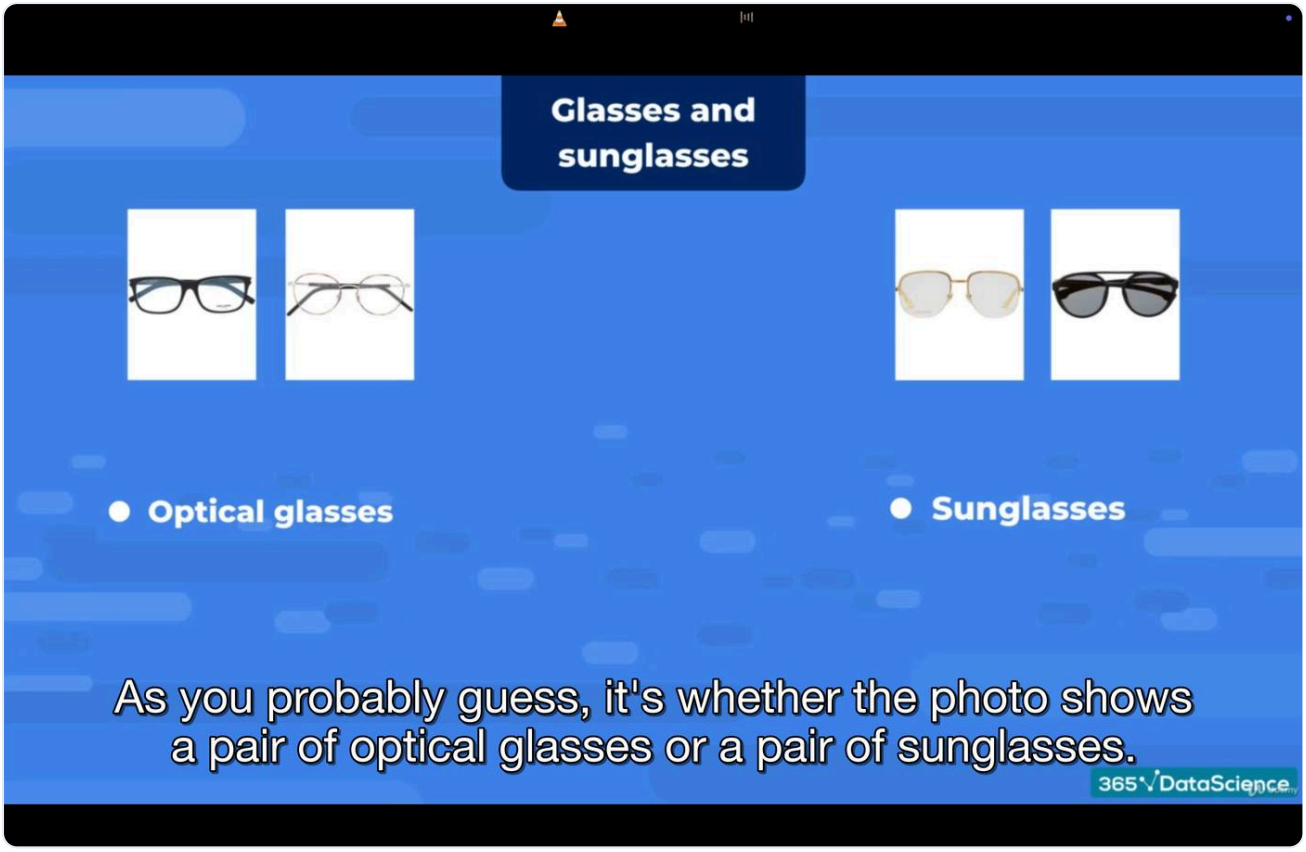

FIGURE 35.11 | Glasses and

As you probably guess, it's whether the photo shows a pair of optical glasses or a pair of sunglasses. Trousers and jeans are more complicated, apart from trousers and jeans. We can also differentiate between male and female.

FIGURE 35.12

FIGURE 35.13

Thus there are four variations in total male trousers, female trousers, male jeans, female jeans. As for the shoes, it's the most complex category of all, with 11 subclasses in total. There are, of course, male and female shoes. The men choose subclause consist of boots, trainers or sneakers, sandals, flip flops and slippers as one subcategory, then formal shoes and finally others.

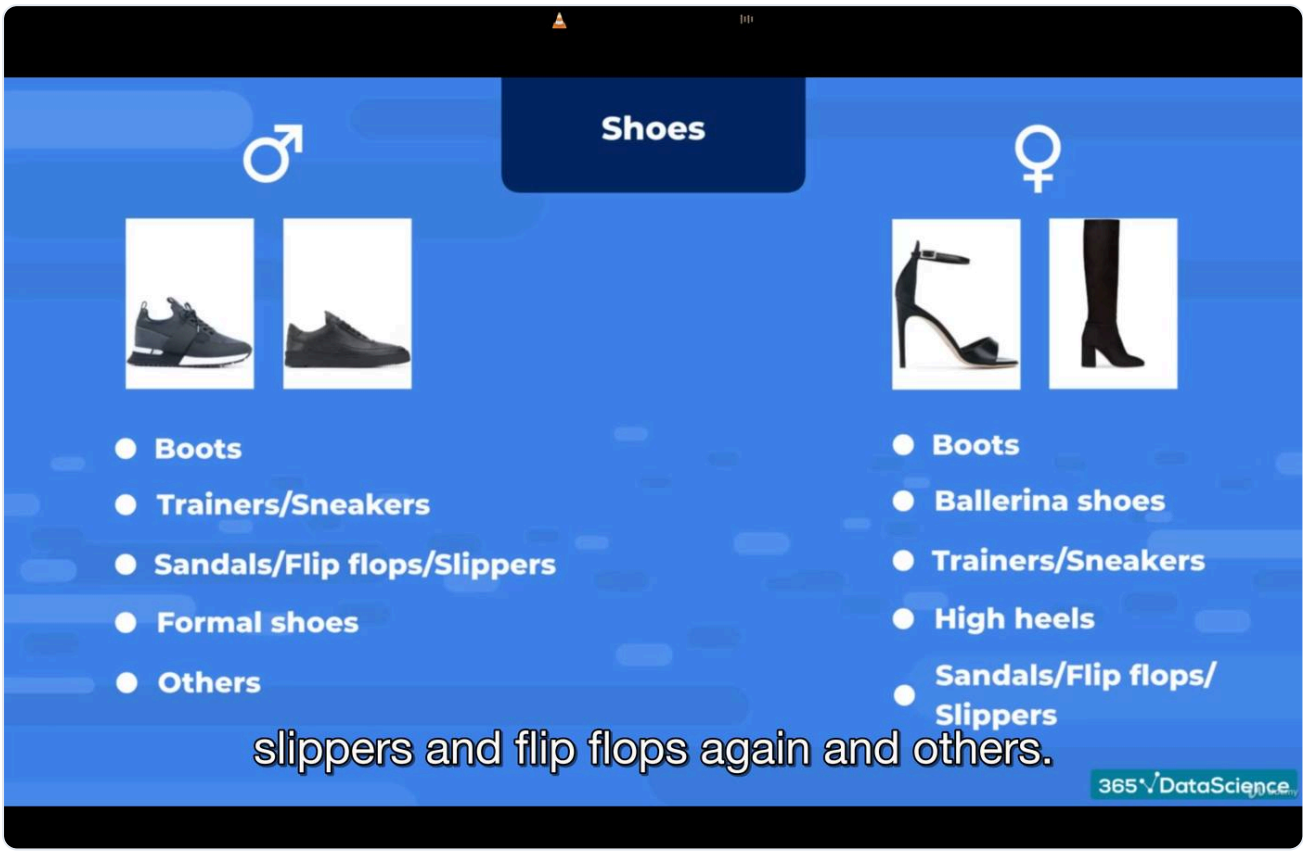

FIGURE 35.14

The women's side includes boots, ballerina shoes, trainers or sneakers, high heeled shoes, sandals, slippers and flip flops again and others. All right.

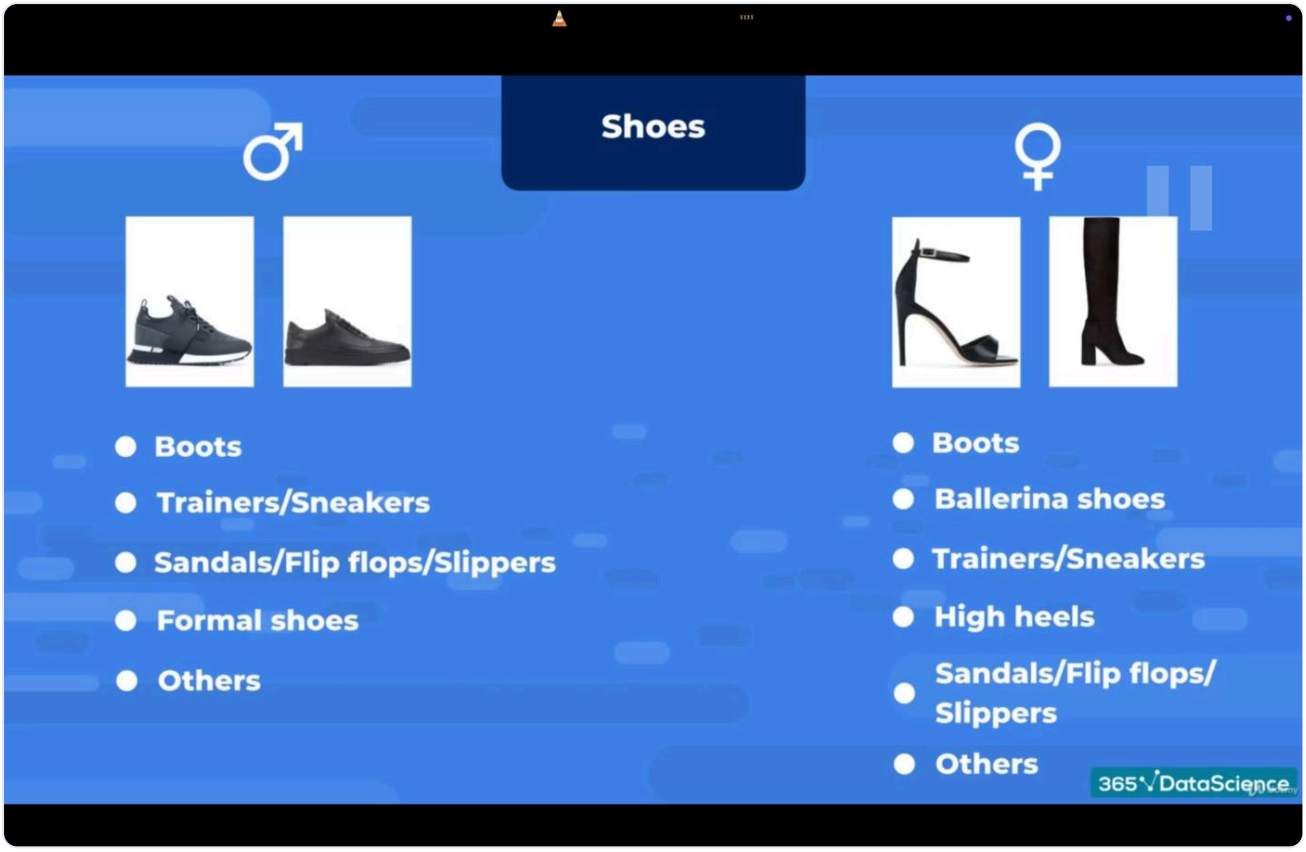

FIGURE 35.15

Our objective in this section will be to return all labels associated with a given image.

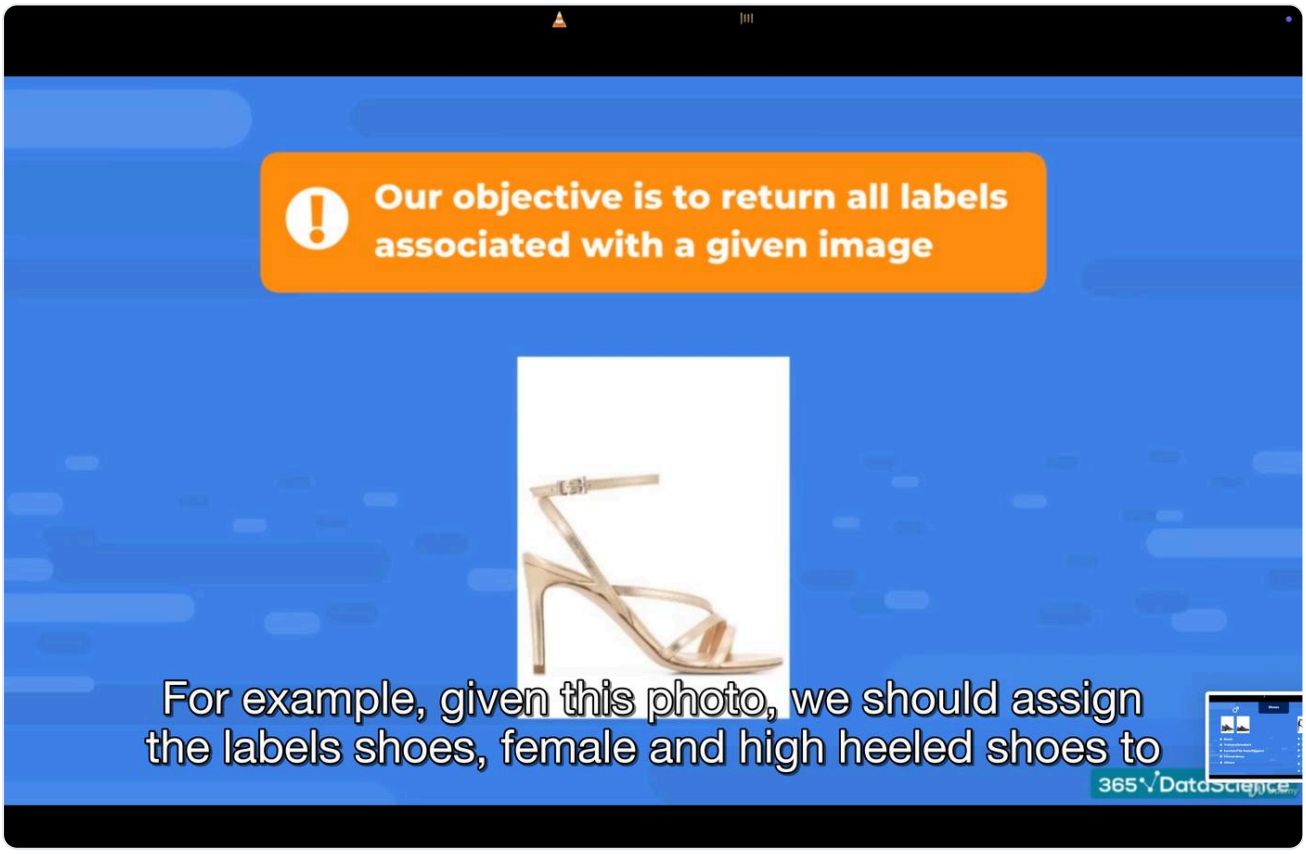

FIGURE 35.16

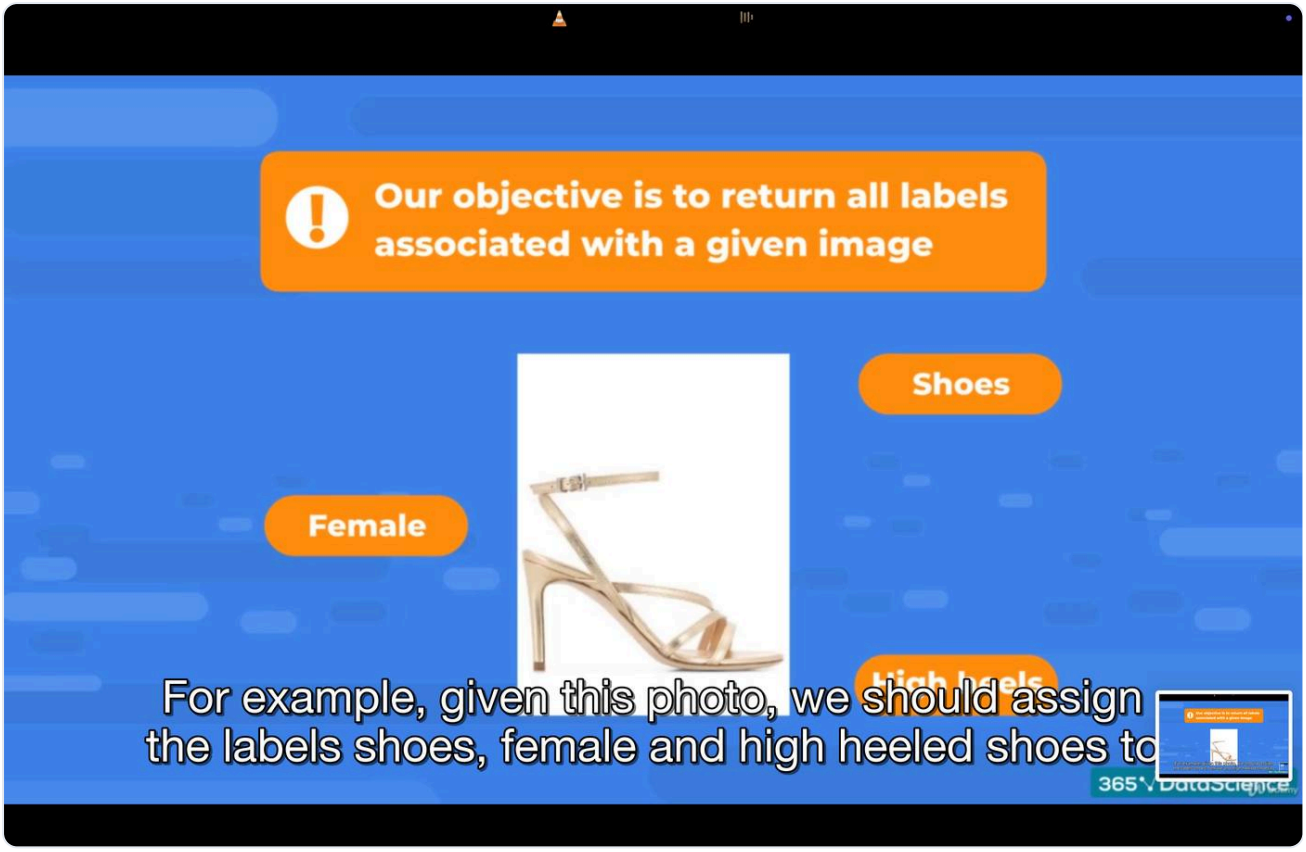

FIGURE 35.17

For example, given this photo, we should assign the labels shoes, female and high heeled shoes to it. Good. As we've seen, this data set is not a simple linear one with only one type of labels. Considering all the different labels and sub labels, it seems inefficient to try and create only one network that needs to deal with all of it.

FIGURE 35.18 | Not a Simple nor

In fact, it is much better to create different networks for the different tasks that are of the same level. Let me illustrate what I mean. We can say the three main classes are on the same level. Therefore we can create one network that will perform the primary classification of the image. Is it an image of glasses, trousers or a shoe?

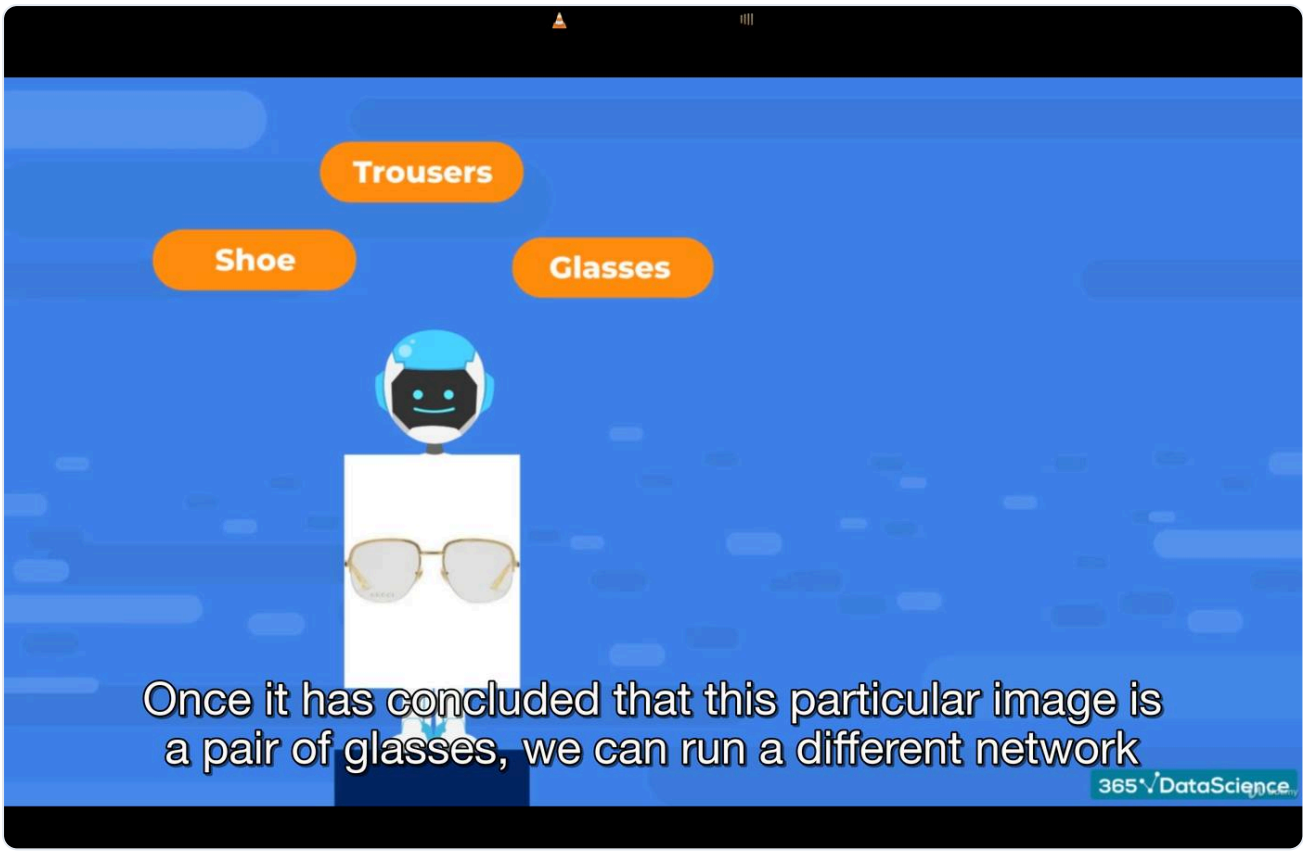

FIGURE 35.19

Once it has concluded that this particular image is a pair of glasses, we can run a different network to label it either as optical glasses or sunglasses.

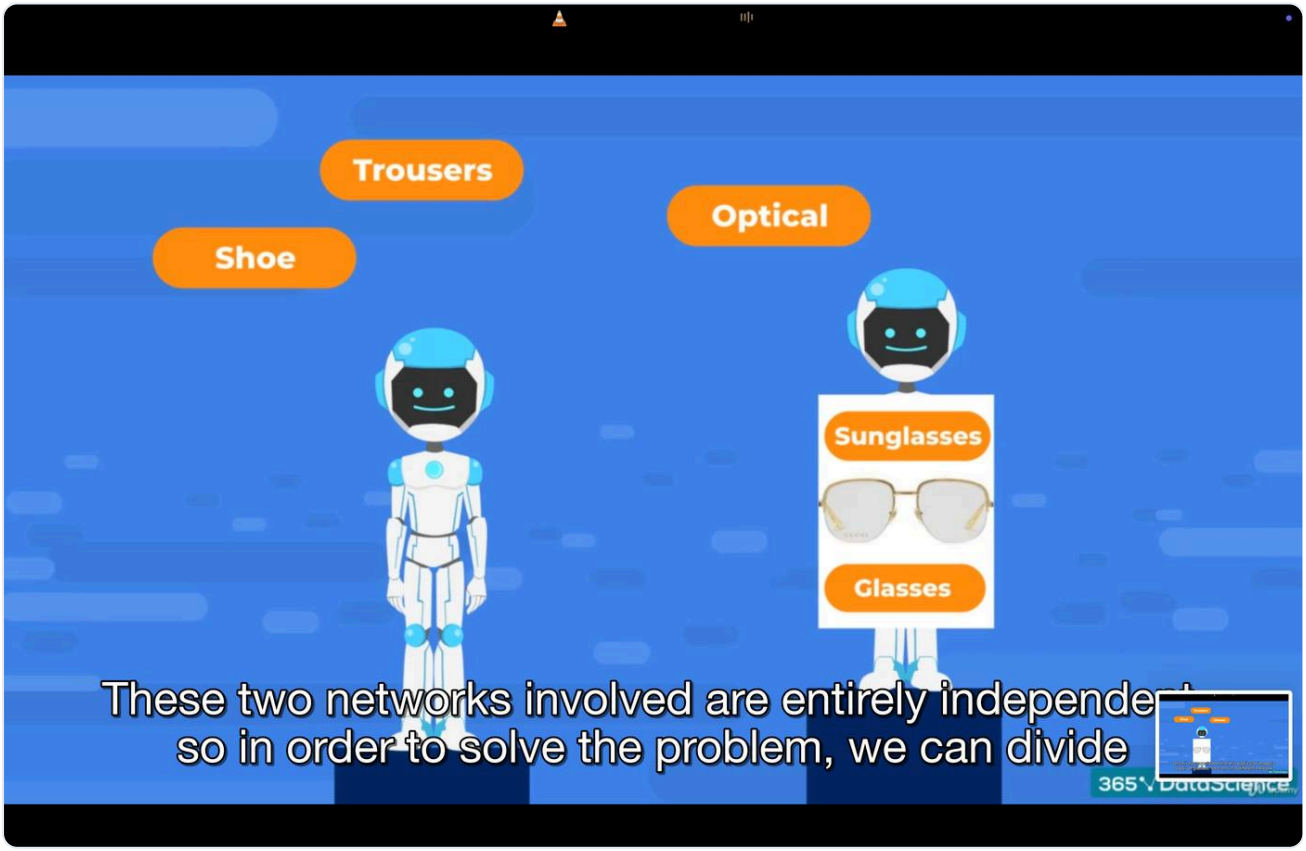

FIGURE 35.20

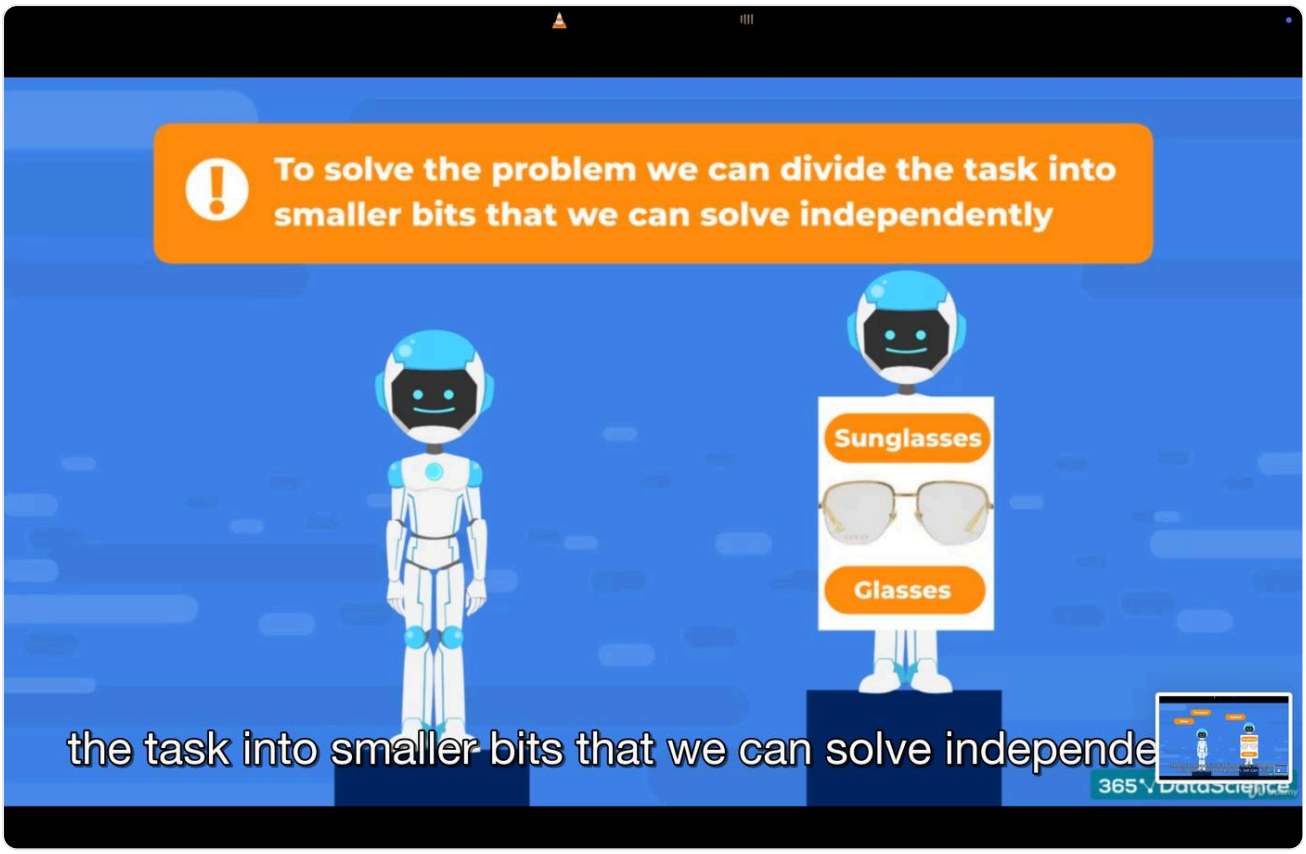

FIGURE 35.21

These two networks involved are entirely independent, so in order to solve the problem, we can divide the task into smaller bits that we can solve independently. Great, we will do just that in the next lectures, starting with the primary classification following this video. There is a short info about the data set with all the numbers you need to know. Thanks for watching.

LECTURE 36

Image Module

Welcome back. In this lecture, we will quickly go over the conversion of an image file to an numpy array. We will not get into details, rather, we'll just show you the main idea. All right. So last lecture, we saw the images in our data set. However, neural networks don't work with image files. They work with tensors. Therefore, we need to convert all those images into NumPy arrays that TensorFlow can work with. Not only that, but the same computational power and memory.

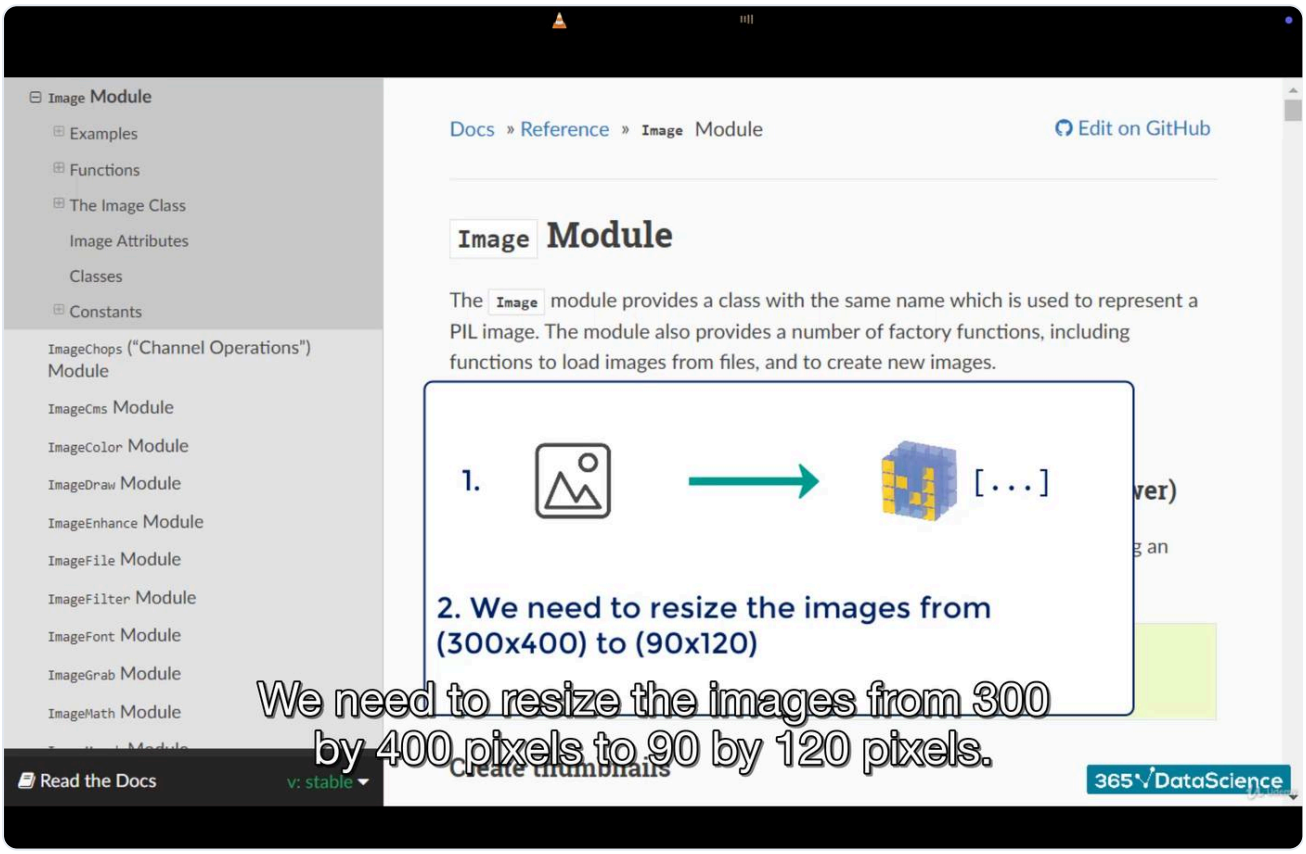

FIGURE 36.1 | Image Module

We need to resize the images from 300 by 400 pixels to 90 by 120 pixels. This is a standard thing you may want to do when dealing with computer vision. So how do you do it? Well, one of the easiest ways in Python is to use the image module from the pillow library. It provides a lot of nifty methods for working with images. We will only use a couple of those, but you can explore them all in the documentation page here. Let's take this image here as an example. It is in the same directory as the notebook file we will use.

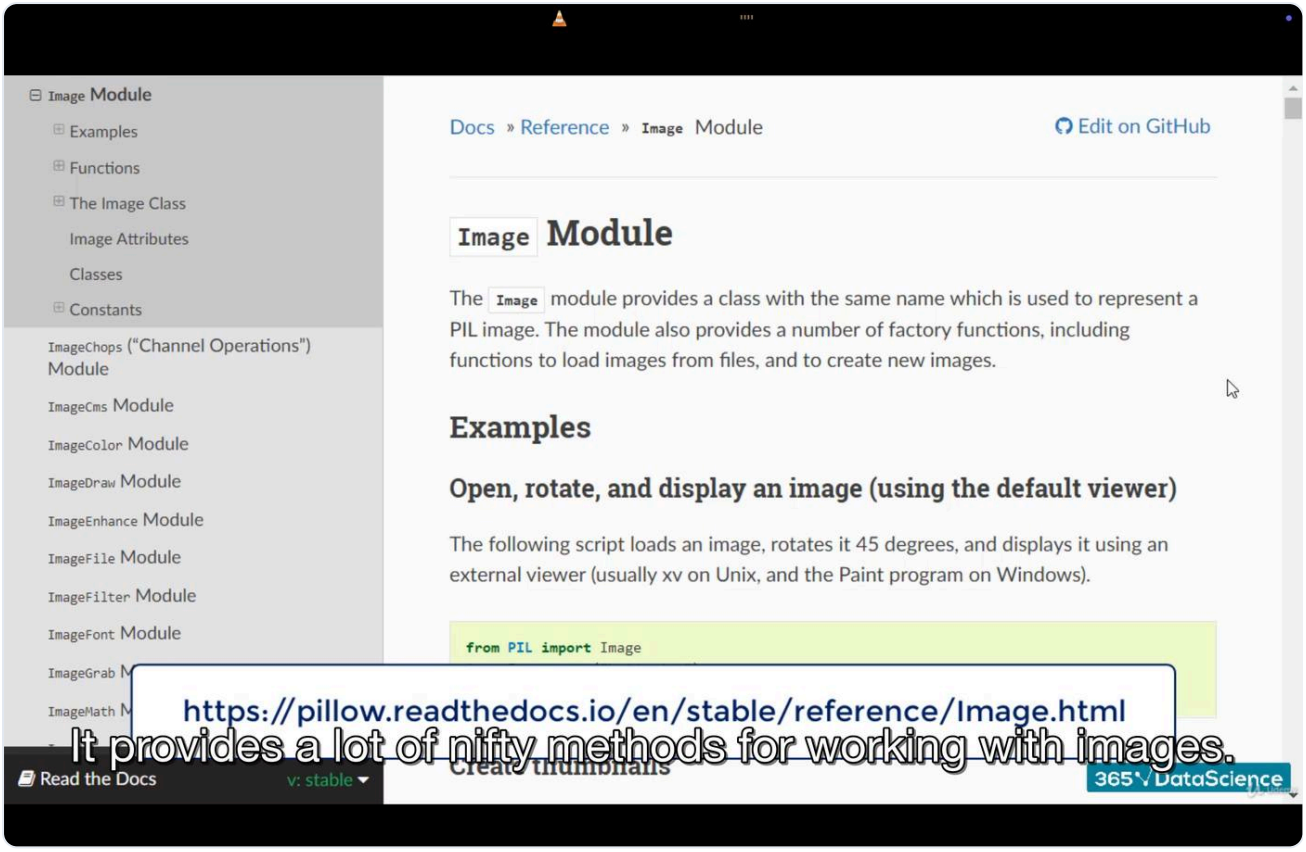

FIGURE 36.2 | Image Module

So here we import NumPy and the image module. Let's also save the file name of our image. Now, loading the image into a variable is as easy as writing image equals image, dot open file name. After executing this line, we can manipulate the image using the variable great, if you're curious to learn more, here is a short list of functions and methods that may come in handy when you deal with images. OK, now that we have our image in the variable, we can visualize it with the show method, it opens it with a default photo viewing program as we have not done anything yet. It is still the same photo. Next on our list is the task to scale it down. We can do this with the thumbnail method.

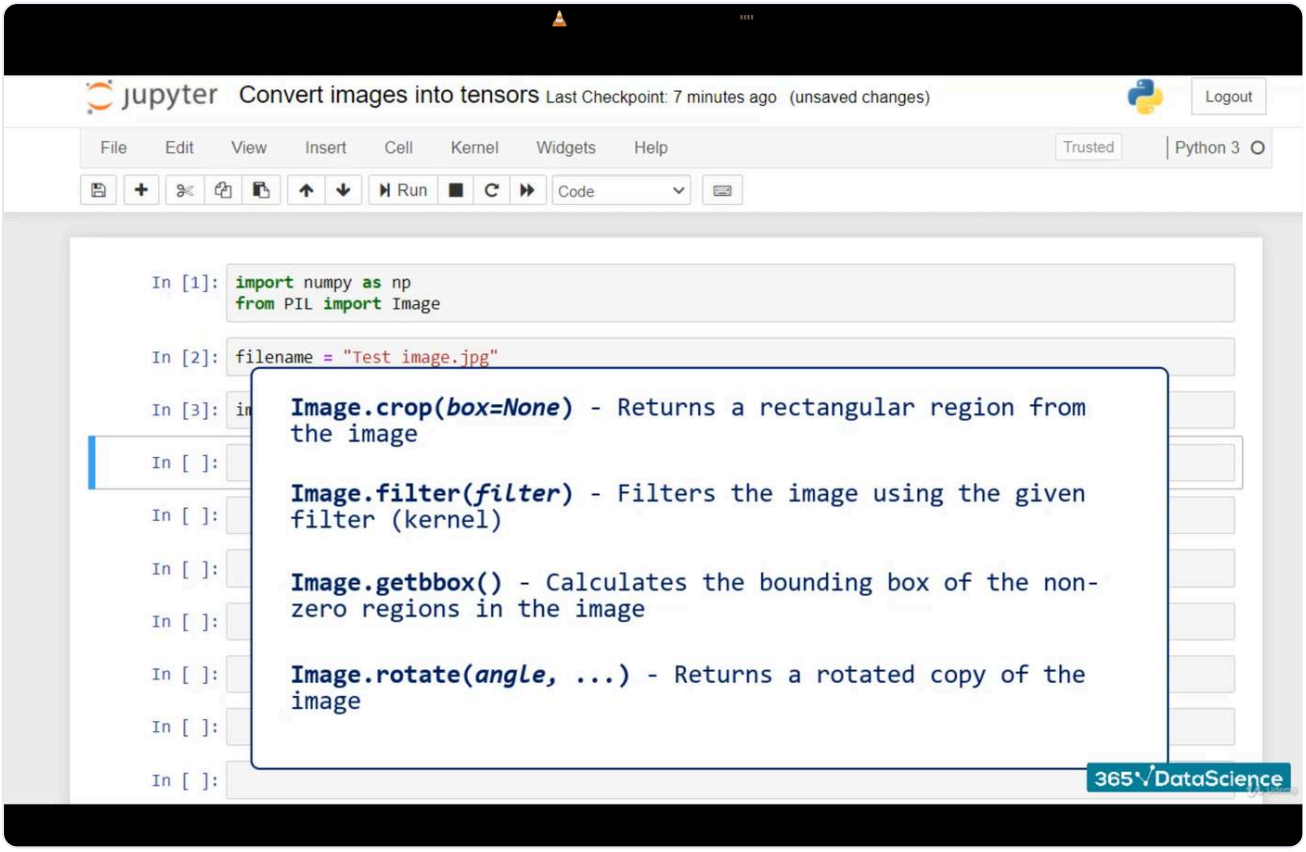

FIGURE 36.3

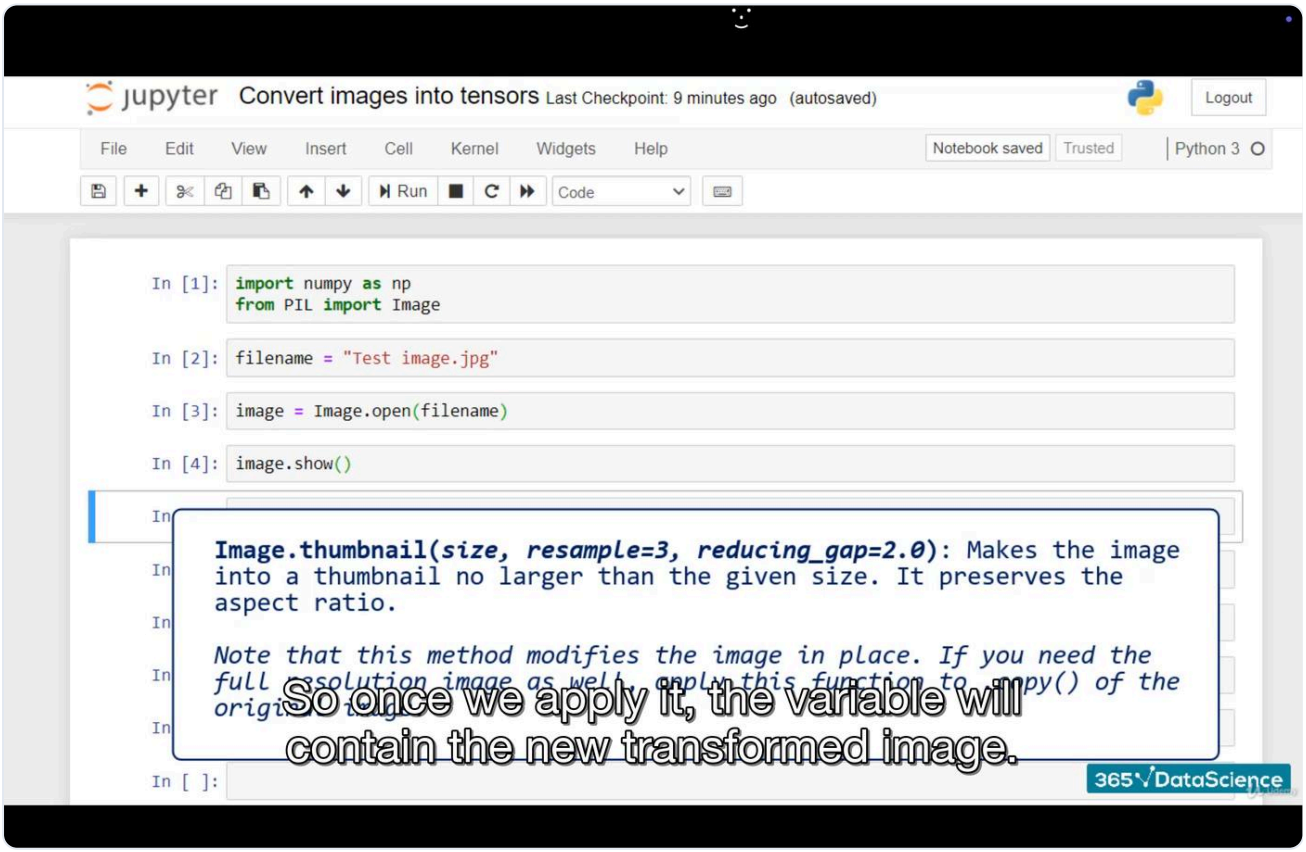

FIGURE 36.4

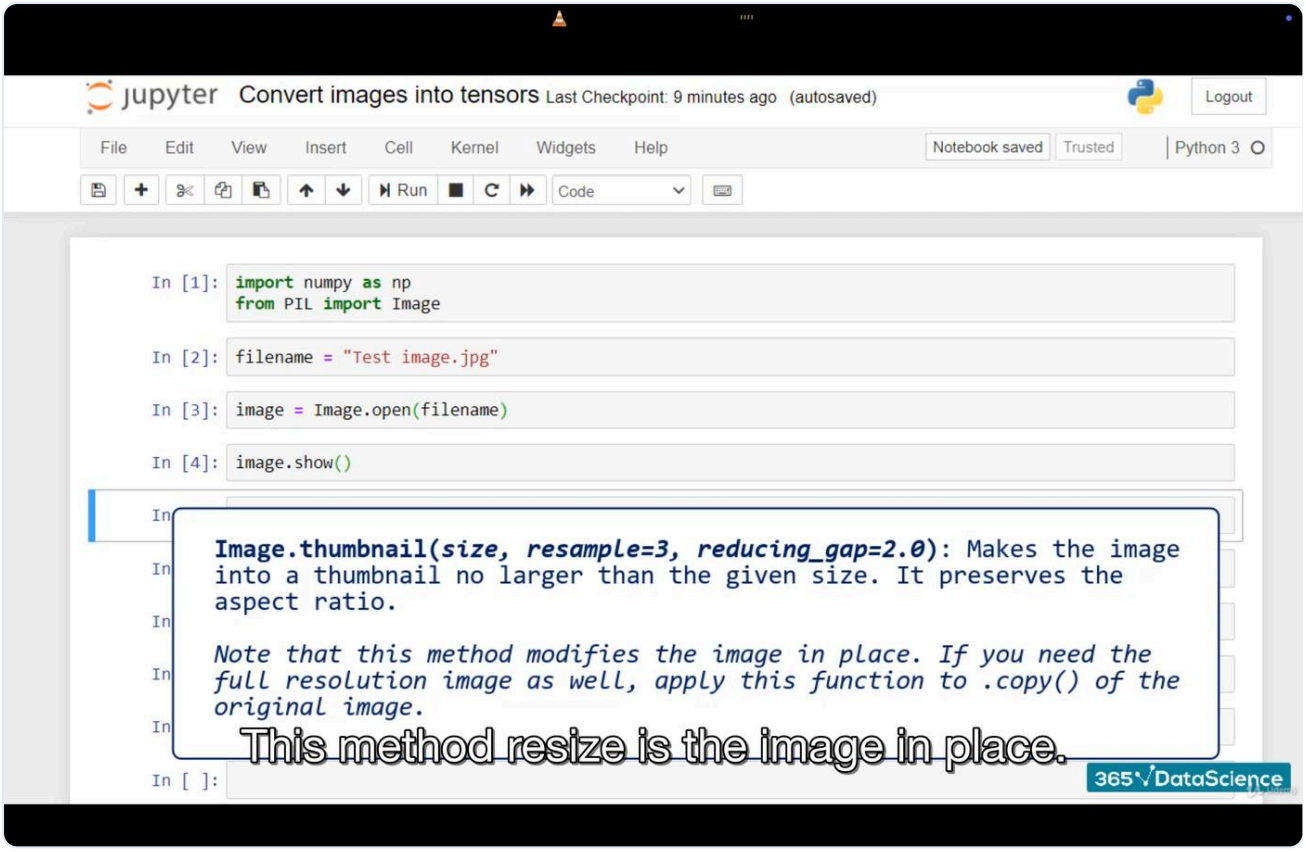

FIGURE 36.5

This method resize is the image in place. So once we apply it, the variable will contain the new transformed image. It also preserves the aspect ratio and that is exactly what we want. So we write image dot thumbnail. In the brackets, we must specify the new size, which will be 90 by 120, so far, so good. Now let's visualize the transformed image image dot show. We are greeted with a smaller version of the same image. OK, we have resize the image. We now need to convert it to a NumPy array. And that is really simple. Just past the variable to the asarray function. Like this image or a. Equals NP dot asarray image. There you go. We now have an array representation of the image stored in the variable image, very smooth and easy this way.

We converted all the images into numbers for our data set as a final note. Let's check the dimension of the array. NP dot shape imagery. 120 by 90 by 3:00 a.m.. Wasn't it supposed to be just 90 by 120? Well, the three is there, since this is a color image, that means each pixel has three values, a red, green and blue value. As for the spatial dimensions. This is just the convention in the field. You can picture it as if this is a list of all the rows and there are 120 rows. All right. In the next lecture, we will lay the groundwork for some of the code we will use in this section. Thanks for watching.

LECTURE 37

Confusion Matrix

In this video, we are going to discuss some of the important code concepts so that later on when we write the full code, it will be less confusing. Of course, we won't linger too much on a line by line basis. You can inspect the code thoroughly on your own. Cool, let's think a little bit about how this business case is going to work.

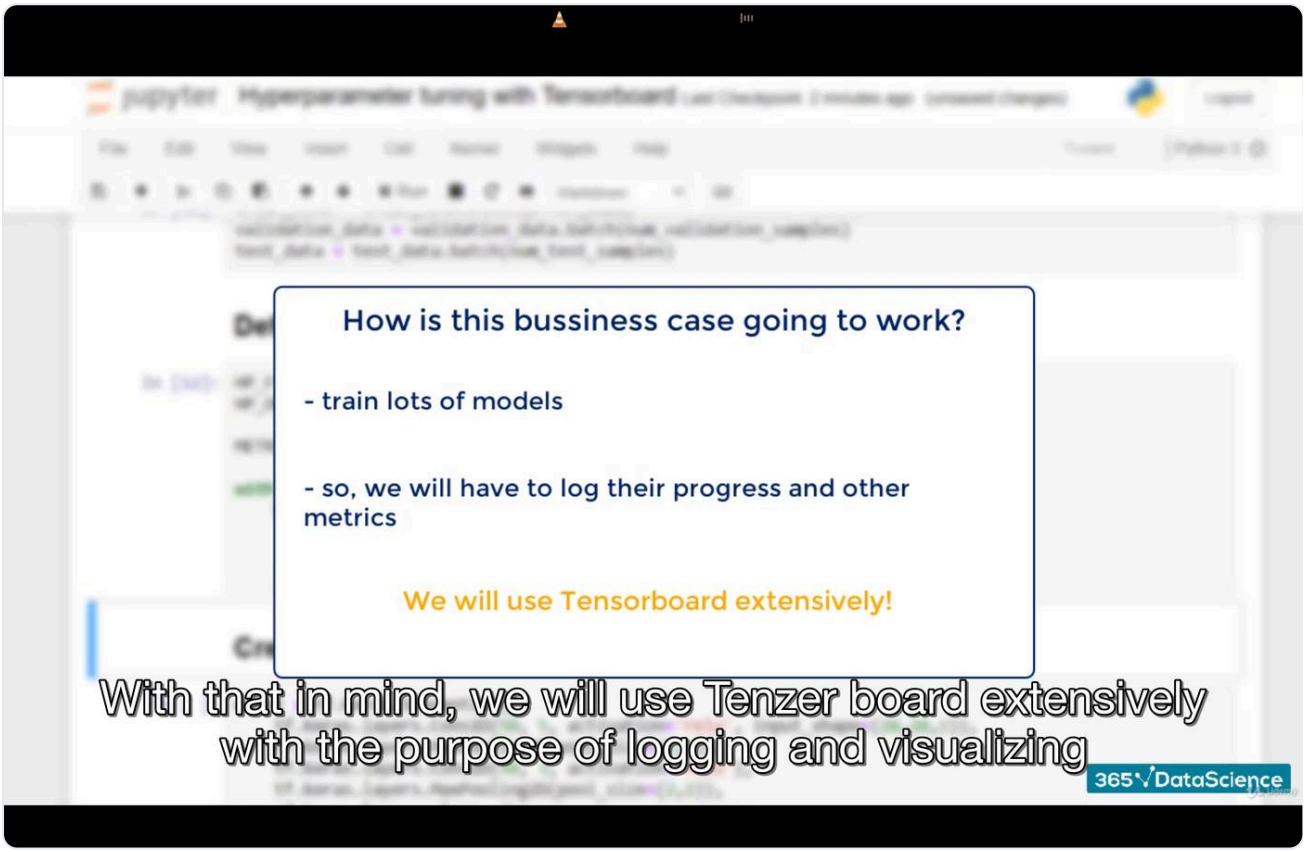

FIGURE 37.1 | How Is This Bussiness Case Going to Work?

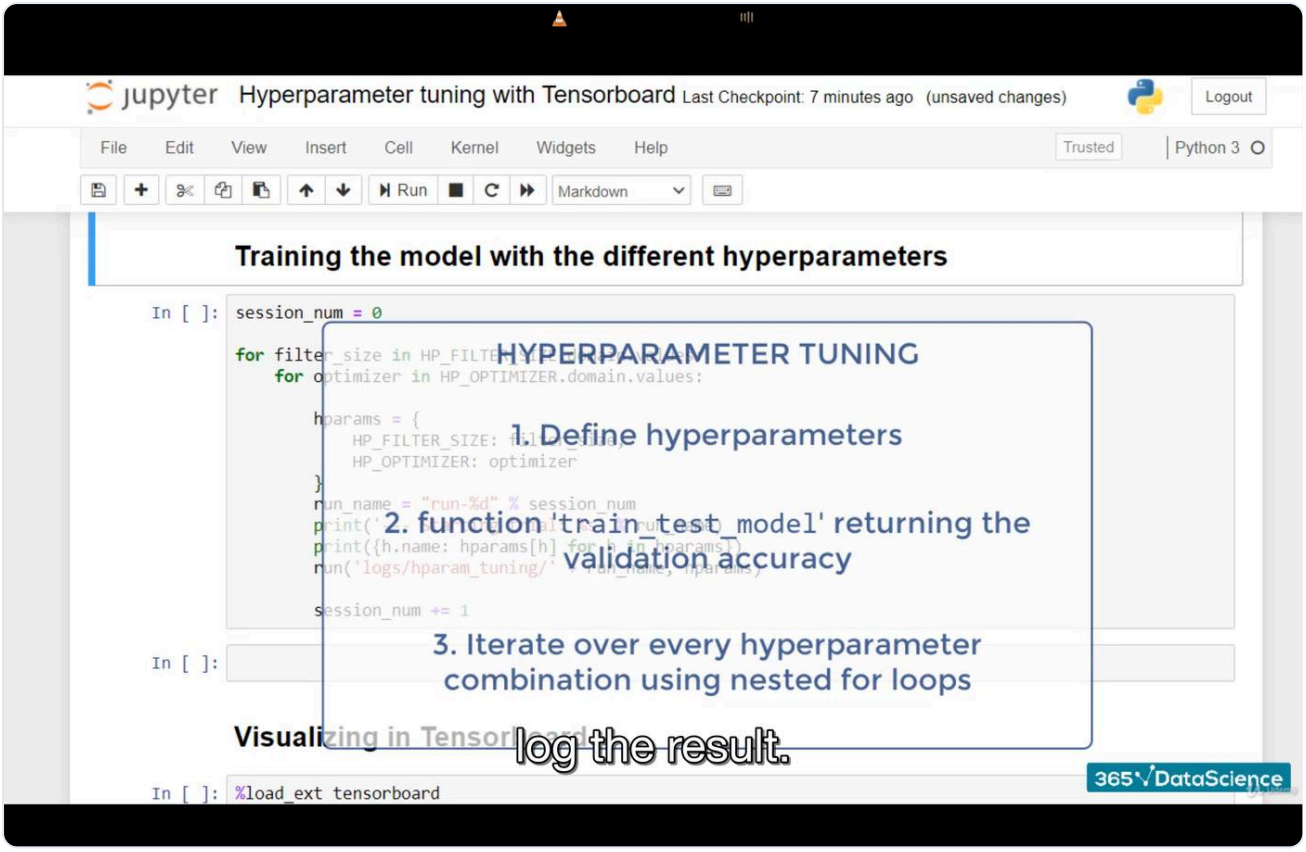

FIGURE 37.2

We will obviously need to create and train models, lots of models, actually, so we will have to log their progress and other metrics to compare them and decide which one does better. With that in mind, we will use TensorBoard extensively with the purpose of logging and visualizing these metrics. So let's quickly recap the structure of TensorFlow boards. Hyper parameter tuning. First, we define the type of parameters and their values at the start and write this information in the logs. Then we define a function to train a model with a given set of hyper parameter values. This function returns the test accuracy of this model. After that, we define another function with a purpose to start the first function and log the result.

Finally, we have a nested for loop that iterates over every set of hyper parameters and calls the run function. So all in all, for every different combination of hyper parameters, we train and test to model and log the result. For more information, you can check our video and the topic in the TensorFlow Broad section or look at the official TensorBoard guides. All right, so we have our hyper parameter tuning set up, but what metric should we track in order to compare them? The obvious one is the accuracy of our model when dealing with classification. However, there is another useful metric that we discussed in the TensorBoard section confusion matrix.

Here's a quick recap. A confusion matrix, sometimes called an error matrix, is a table that shows the categories our network struggles to deal with on the y axis of the table. We have the true labels of our data on the x axis or the predicted labels. That is the ones our network assigned to the pictures. The cells show the number of cases that conform to both axes. They are usually color coded, often with blue good. Now let's recall how the matrix is implemented. There are three

main points in logging and displaying the confusion matrix that are defined in functions. First, we have the plot confusion matrix, which plots the Matrix visually in Matplotlib, given the already computed numbers.

Then there is plot to image, which you guessed it converts the plot to an actual image. And the last one named Log Confusion Matrix was about computing the actual numbers using learn and logging the info to TensorBoard. Finally, we should define the callback to start the whole process. That is how we will implement the confusion matrix and the upcoming code in the next lecture. Speaking of in the next video, we will solve our first task, the Primary Classification Network. See you there. And thanks for watching.

LECTURE 38

Primary Classification

Welcome back, let's quickly recap our objectives for this practical example, we have a data set of images of different clothes and accessories. Our goal is to create networks that will label all these images accordingly. We explore the labels and sub label structure of our data, and given its complexity, we decided to break it into several smaller objectives that are solvable by a single model. The first of these is the primary classification.

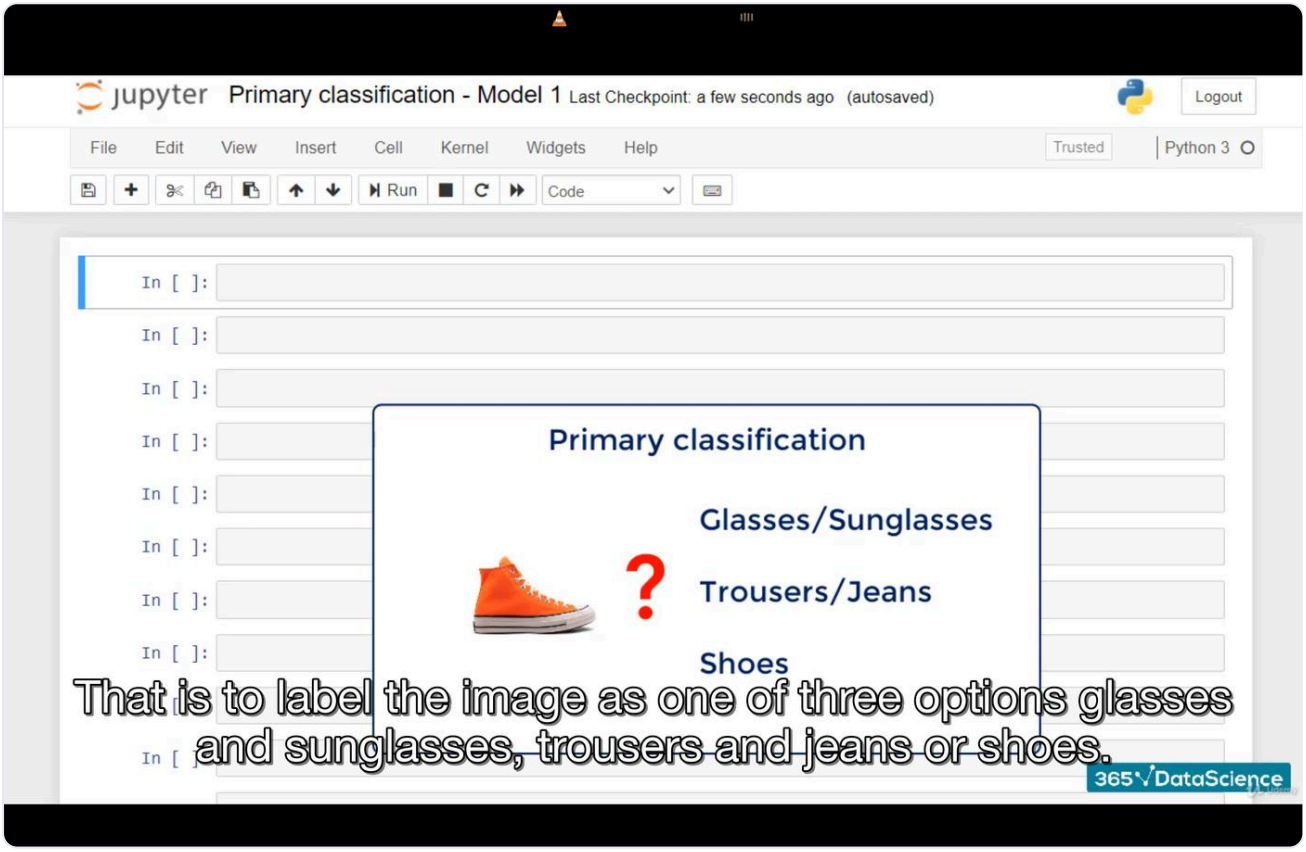

FIGURE 38.1 | Primary Classification

That is to label the image as one of three options glasses and sunglasses, trousers and jeans or shoes. And that is exactly the topic of this video. The Primary Classification Network. All right, let's get started. First, we need to think about what our approach would be. We could, for instance, agree on certain complex network architecture and try to find the optimal hyper parameters for every many task.

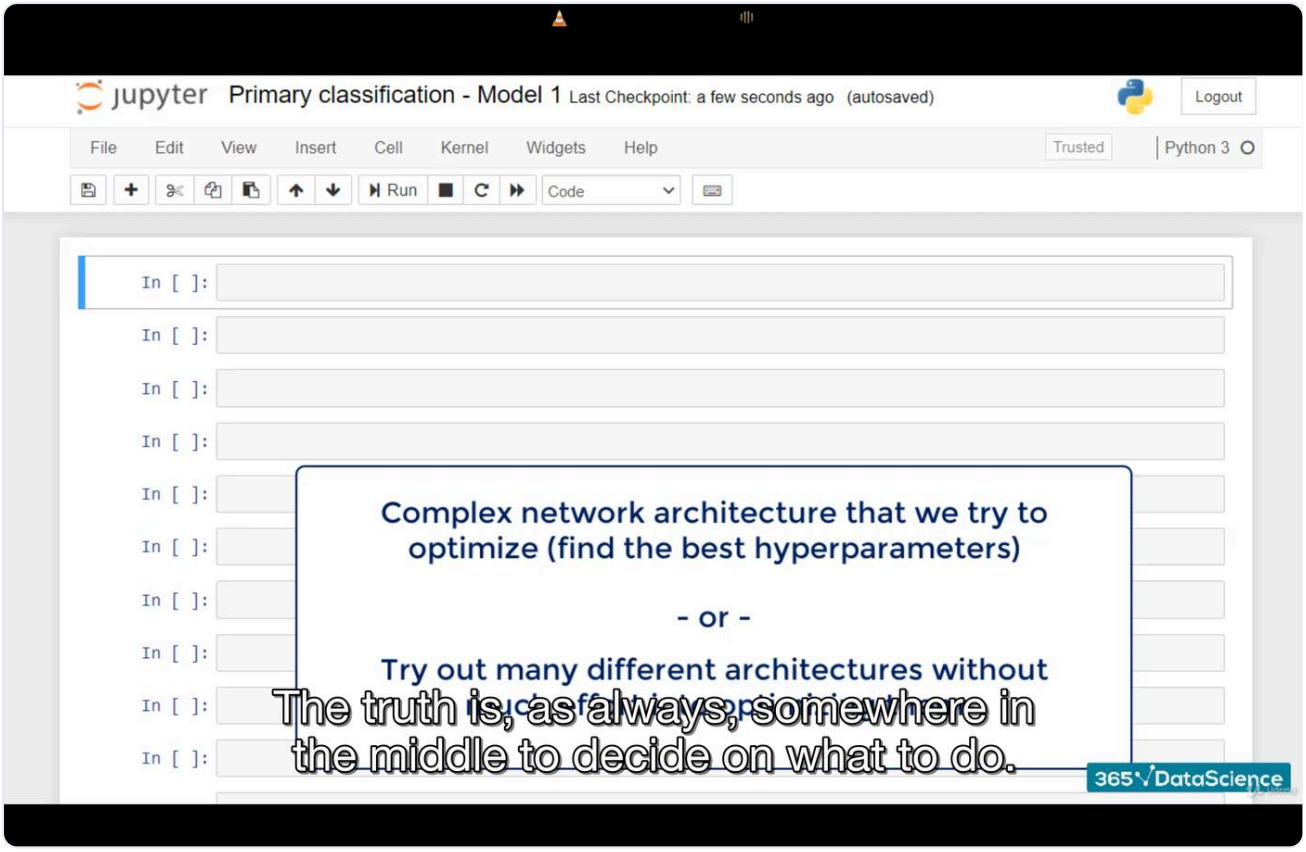

FIGURE 38.2 | Complex Network Architecture That We Try to

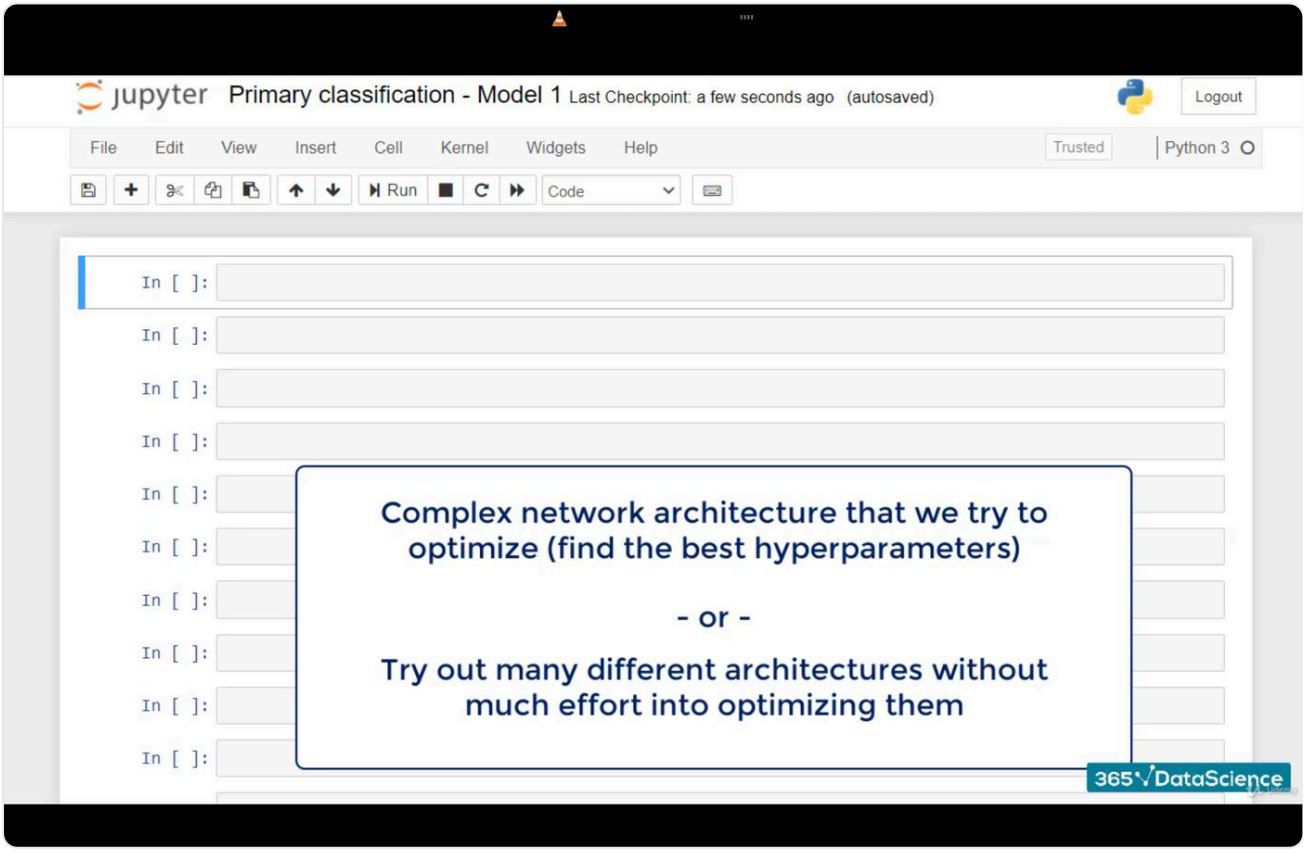

FIGURE 38.3

Or we might try out a thousand different architectures without too much tuning of the hyper parameters. The truth is, as always, somewhere in the middle to decide on what to do. However, we need to be aware of the limitations. A rule of thumb I heard once was that a possible model could be made in two weeks, a good one in three months and a great model in more than a year. Diminishing returns definitely play a big part in machine learning, and we certainly don't have a year to spend on just this practical. Even more so as those timelines are probably for computers with powerful GPS or clusters of computers with more than 20 times the speed of the ordinary laptop I use to train the networks.

Furthermore, we have numerous, many models to create. In light of all of this, we would proceed as follows.

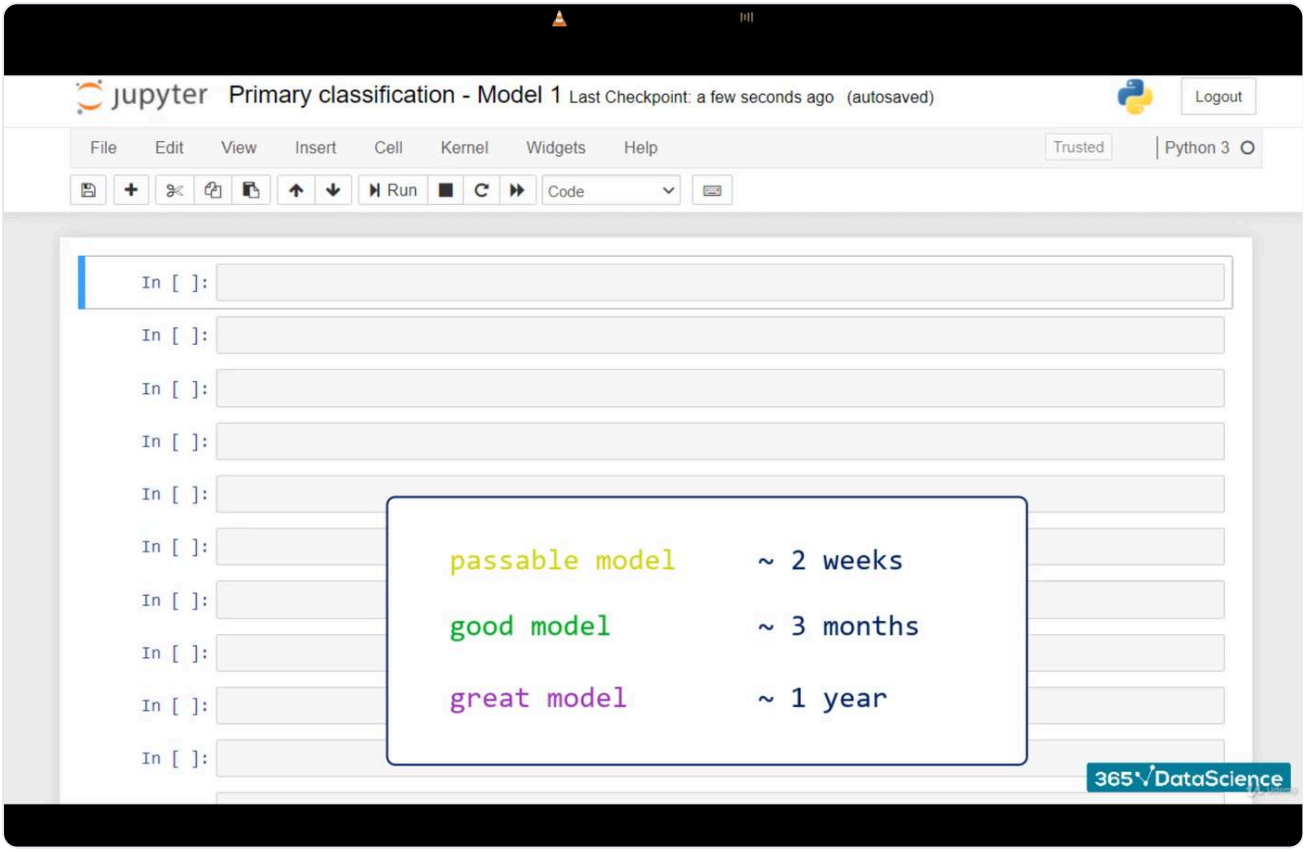

FIGURE 38.4

We are going to do several of the tasks in this practical enough to teach you how to proceed and solve them. We will explain all of the methods and techniques employed leading you to apply them on the rest of the tasks to sharpen your understanding. In fact, this practical is open ended, meaning it allows you to set yourself a different goal and accomplish it. We will simply provide you with a general framework to solve those goals. All right. Now let's see what the plan is. First, we will concentrate on the main labeling. Then we will continue with finding the best approach for the subclasses of trousers and jeans.

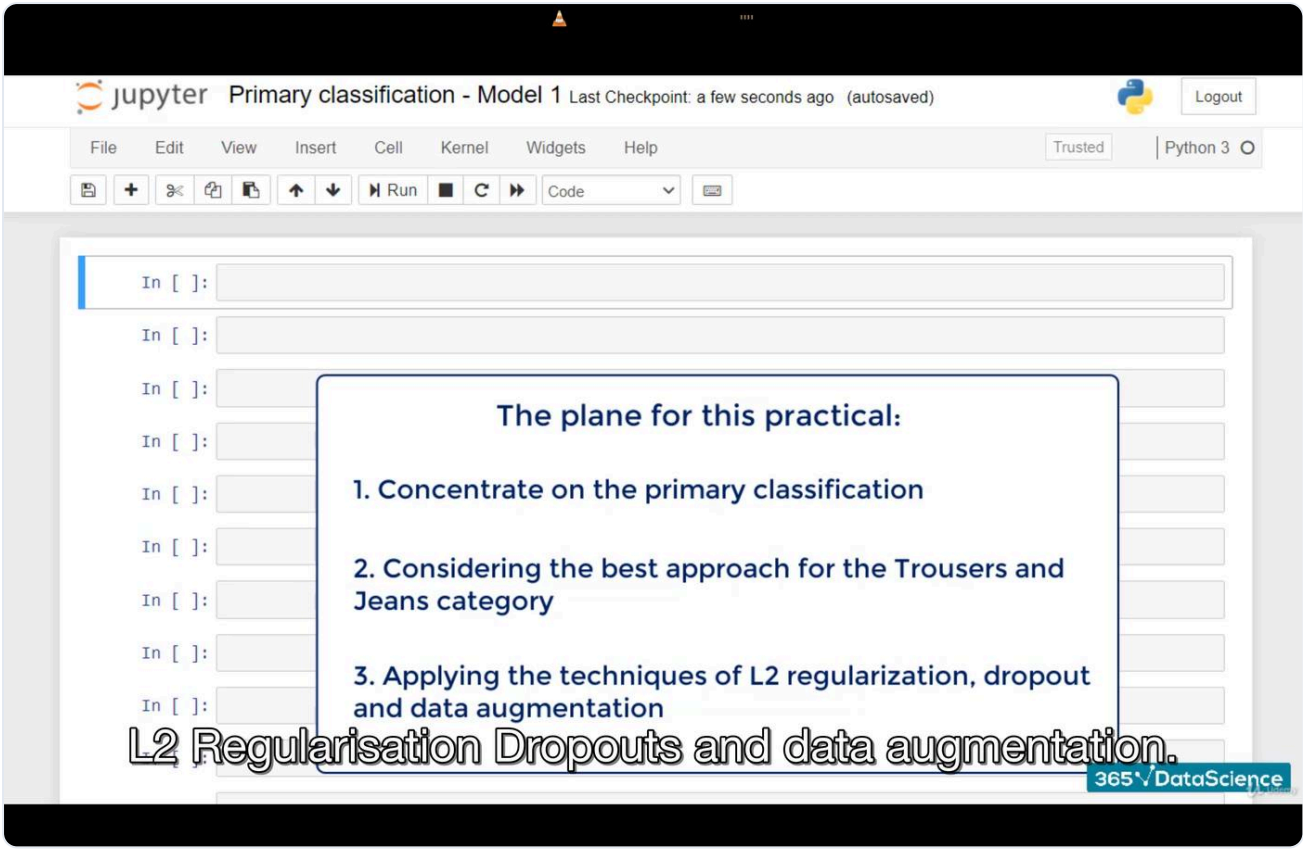

FIGURE 38.5

And finally, we will keep it off by applying the three techniques we mentioned in the previous section, L2 Regularisation Dropouts and data augmentation. That being said, we will not focus much on the primary classification as it is the most similar case to the next example we already did. Good with that out of the way, let's get down to it. First, the packages for our purposes, we are going to need all these packages, IO, itertools, scikit-learn and Matplotlib for the confusion, matrix and NumPy for the data sets. Of course, TensorFlow and TensorBoard are there for well, you wouldn't be here if you didn't know their purpose.

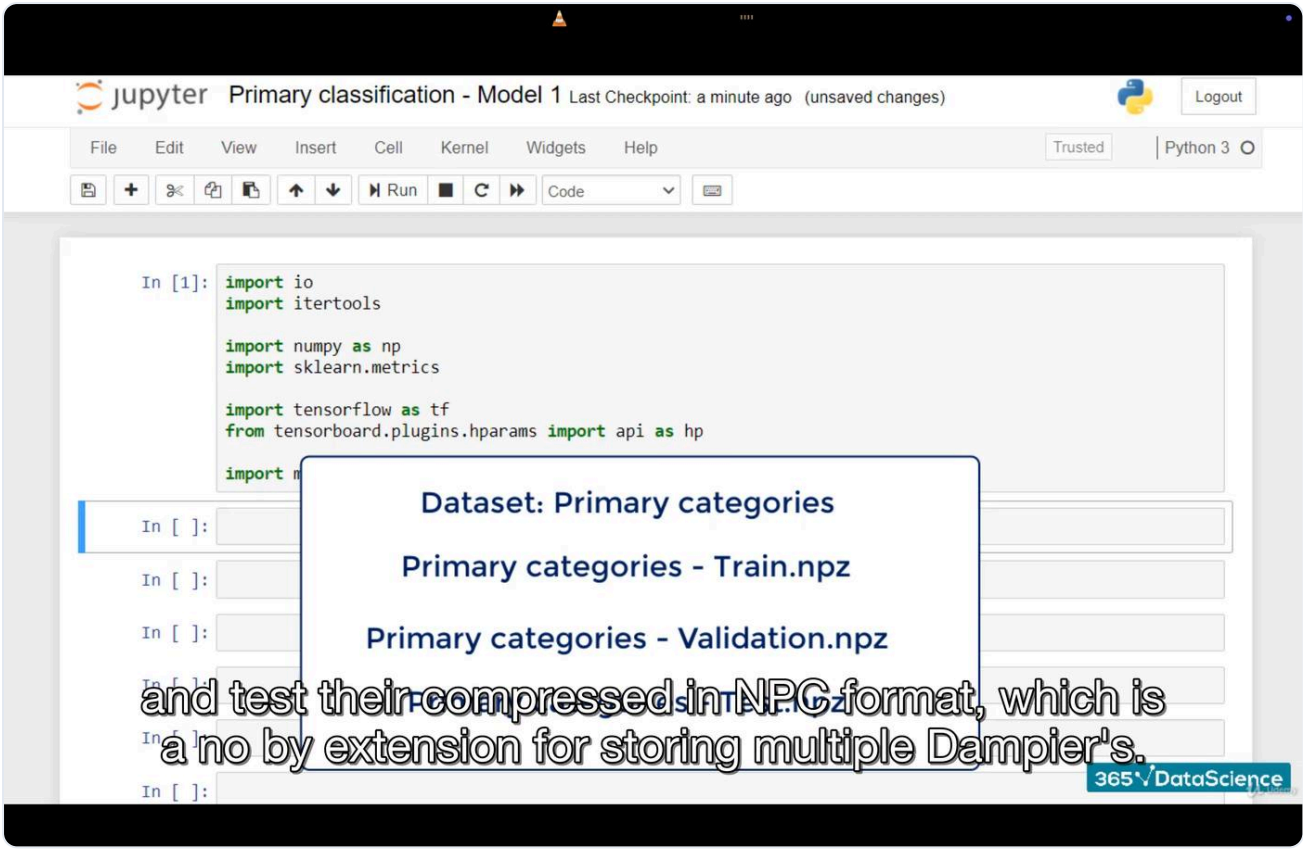

FIGURE 38.6

Moving on swiftly. We can't do machine learning without a data set. If you recall, we highlighted the structure and contents of our data sets earlier in this section for this problem. The data sets we will need are called primary categories with all three variations, train validation and test their compressed in NPZ format, which is a NumPy extension for storing multiple NumPy arrays. Each category has the same structure.

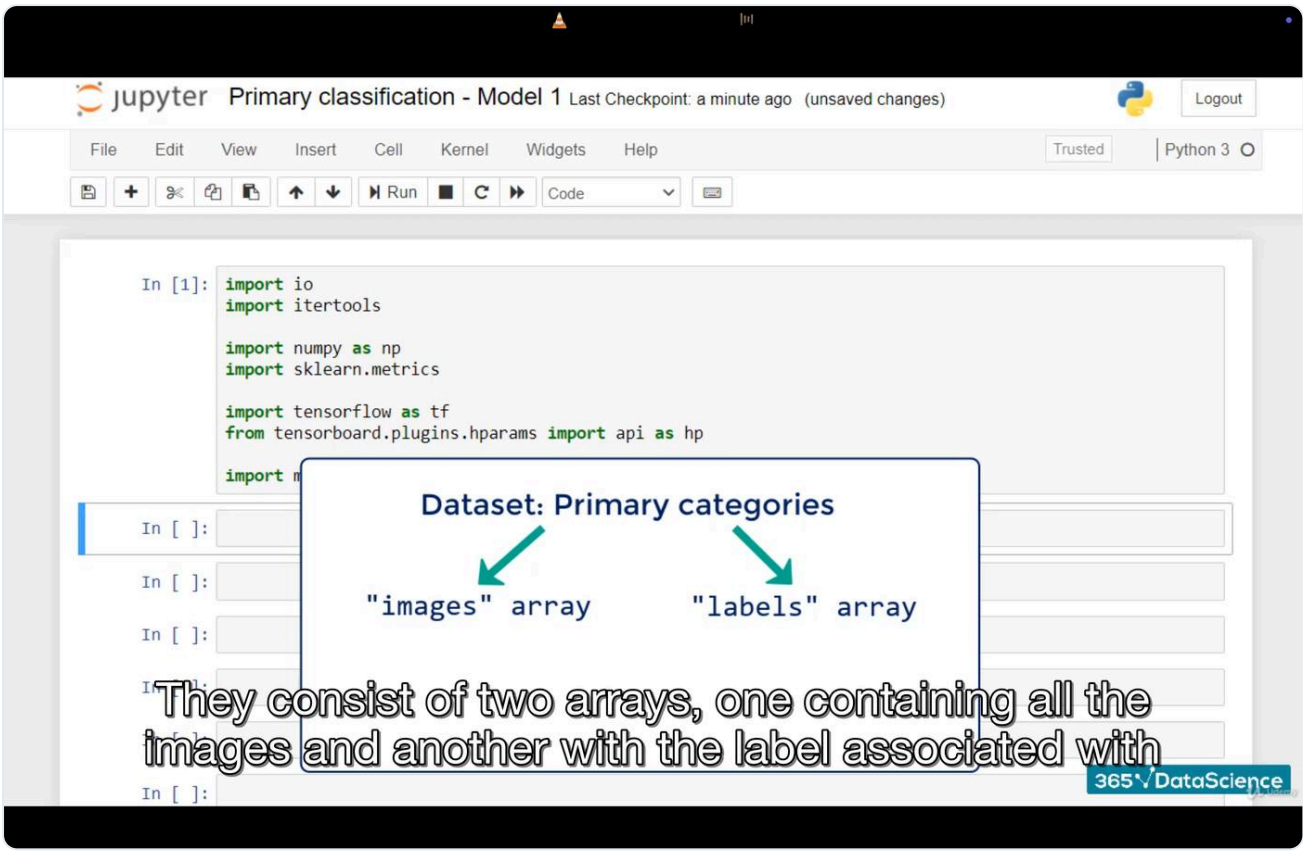

FIGURE 38.7

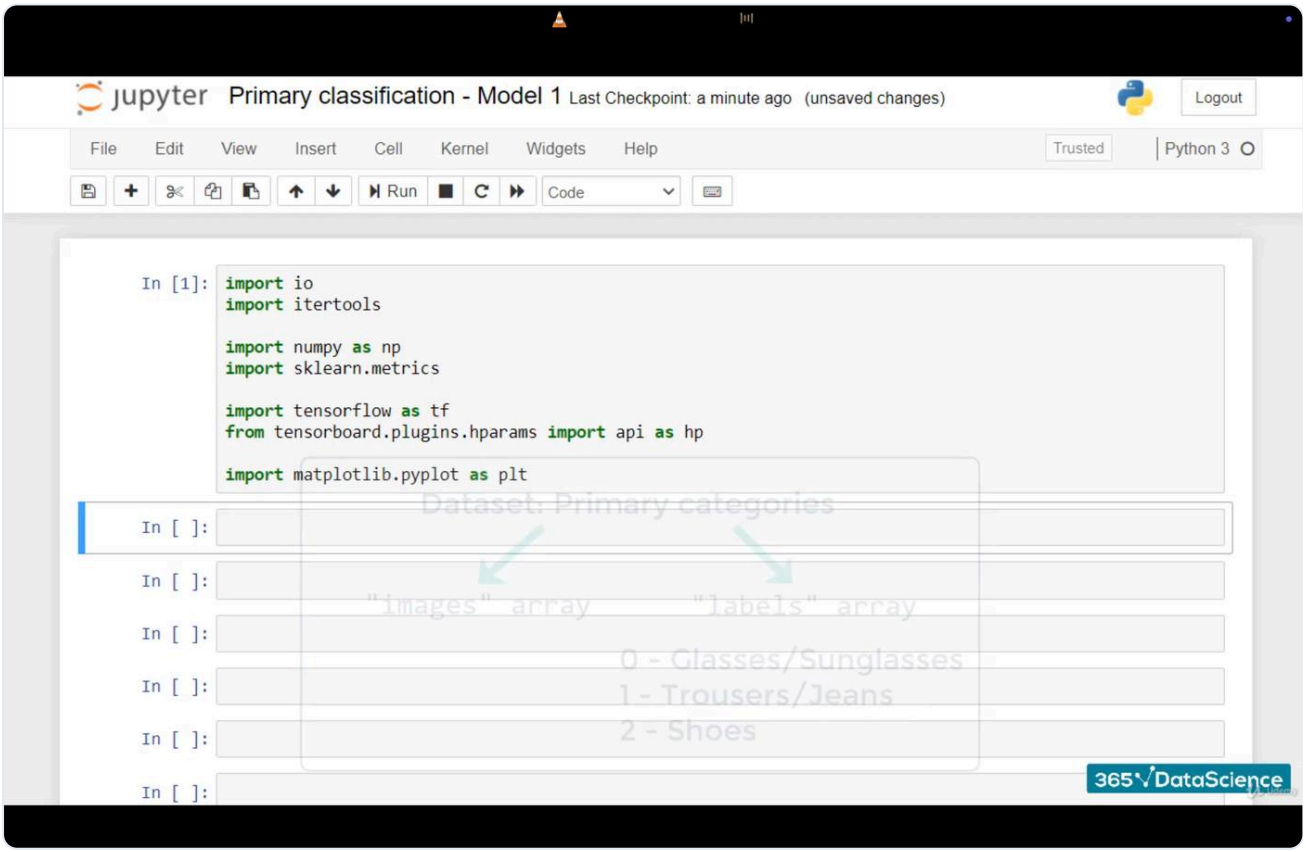

FIGURE 38.8

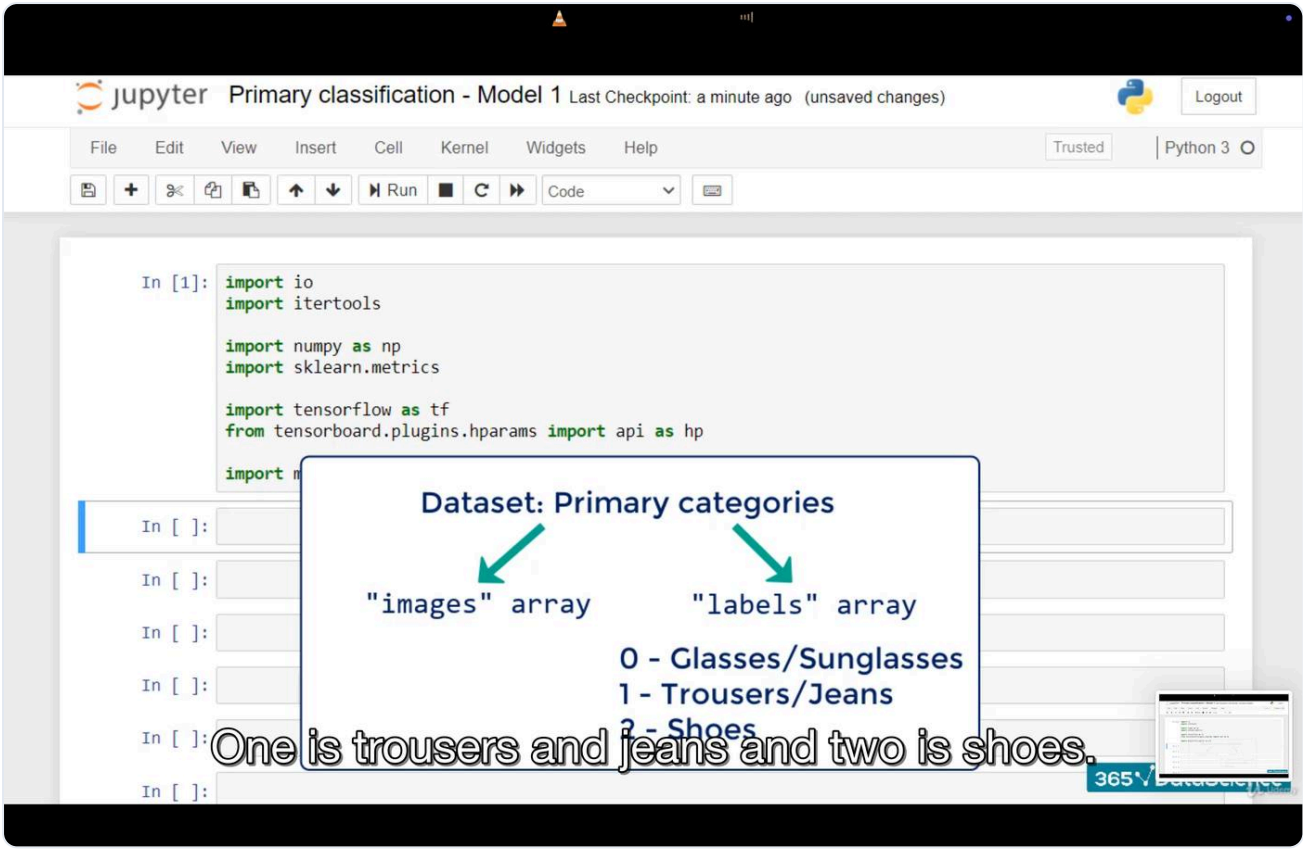

FIGURE 38.9

They consist of two arrays, one containing all the images and another with the label associated with each image. The encoding is like this. Zero represents glasses and sunglasses. One is trousers and jeans and two is shoes. OK, to retrieve the data sets, we can use the NP Dott load function and state the file path to the NPC file. I have chosen to store them in a folder called Data Sets now, in order to import the data sets, we write those three lines. Now we have to separate the inputs and the targets in our case, those are the images from the labels. Let's look at the training set, which is stored in the variable data train. The process of extracting an array from it is as simple as treating it as a dictionary.

So to get the array with all the images we write, images, train equals data, train brackets, images. Same for the labels, only this time we change images to, well, labels. And that's our two arrays for the training set, the names for the different arrays are generated when creating the NPC file. So if you create your own data set, there's no need to follow the same name convention as me. OK, we can repeat the same code with a change of variable names for the other two sets. So far, so good to continue this streak of data preparation, let's pre-process the numbers a bit since we are using no higher res for the data as opposed to the TensorFlow data sets object.

We use that the amnesty example, the preprocessing part is rather simple. We can easily scale the pixel values between the standard zero and one and one line images. Train equals images train divided by two hundred and fifty five. Of course, we do the same for the validation and test images as well. That's neat, but it is not the only benefit of using the umpires when working with this data type TensorFlow can automatically shuffle and bash the data set for us. We do it with the fit method, so our work with the preprocessing part is over. We will continue with the architecture and training of the model in the next video. Thanks for watching.